

RHEL 9 RHCSA 讲义

RHEL 9 实操考试备考讲义

目录

[完成一场红帽实操考试的方法](#)

[Shell 解析、引用、展开与命令组合](#)

[标准输入输出、重定向与管道](#)

[本地帮助、命令发现与软件能力查询](#)

[文件系统层次、路径、文件类型与链接](#)

[文件查找、文本筛选与批量处理](#)

[复制、归档、压缩与远程传输](#)

[用户、组与账号生命周期](#)

[传统权限、umask 与特殊权限位](#)

[ACL 与协作目录](#)

[sudo 与最小特权授权](#)

[进程、作业、信号与调度优先级](#)

[systemd Unit、服务与依赖关系](#)

[系统日志、journal、rsyslog 与日志轮转](#)

[系统时间、时区、RTC 与 chrony](#)

[一次性任务、周期任务与 systemd Timer](#)

[RPM 包、文件归属与软件事务](#)

[DNF 仓库、模块流与包组](#)

[NetworkManager、IPv4 地址与路由](#)

[主机名、NSS 与 DNS 名称解析](#)

[SSH 客户端、服务端与密钥认证](#)

[firewalld 与完整服务访问链](#)

[块设备、分区表与设备签名](#)

[文件系统、标签、UUID 与容量管理](#)

[挂载、fstab、Swap 与启动持久性](#)

[LVM 逻辑存储](#)

[NFS 客户端与远程文件系统](#)

[autofs 自动挂载](#)

[SELinux 模型、模式、上下文与 AVC 证据](#)

[SELinux 文件规则、端口类型与 Boolean](#)

[启动链、GRUB、Target 与系统恢复](#)

[Podman 镜像、容器与 Rootless 运行](#)

[容器持久化、用户 systemd 与 Quadlet](#)

[RHCSA 综合任务与证据矩阵](#)

通用章 完成一场红帽实操考试的方法

红帽实操考试要求把系统从给定初态配置到可验证的目标终态。真正困难的不是记住某一条命令，而是在有限时间内持续回答三个问题：题目要求改变哪个对象，当前证据支持哪一步，怎样证明修改既生效又能够保持。本章建立 RHCSA 与 RHCE 共用的读题、操作和验收方法，并分别给出一页检查清单。

概念 目标终态（desired end state）

定义：目标终态（desired end state）是题目真正评分的系统状态；当前状态是操作前或某一步之后系统实际呈现的状态；持久配置决定状态能否在重启、服务重载或下一次自动化执行后继续成立。

理解：三者不能由同一条查询自动证明。

概念 对象、约束与证据构成实操题的最小模型。对象可能

定义：对象、约束与证据构成实操题的最小模型。

理解：对象可能是用户、文件、服务、块设备、端口、Inventory 主机或受管节点；约束说明必须保留什么、不能改变什么；证据用于决定下一步并证明终态。

操作语义

调查命令读取对象而不改变目标状态；配置命令或 Playbook 改变对象；验证命令从配置、当前状态、功能、外部访问、持久性和幂等性中证明一个明确层次。

操作语义

RHCSA 的主线是“手工修改单机终态”，RHCE 的主线是“用 Inventory、变量、模块和 Playbook 表达多机终态”。RHCE 的 `ok` 或 `changed` 只描述任务执行结果，不能替代受管节点上的功能验收。

[知识专题] 把题目改写成对象、状态、约束与证据

① [知识点] 先找评分对象，不从命令名开始

一道题可能写着“配置服务开机可用”。真正的评分对象至少包含软件包、配置文件、服务当前状态、启动配置、监听端口以及必要的访问链路。若只看到“服务”就立即执行 `systemctl start`，得到的只是一个局部当前状态。

读题时把每一项要求压缩成四列：对象是什么，当前状态未知在哪里，目标终态是什么，限制条件是什么。例如“保留现有数据扩展卷”中的“保留现有数据”是硬约束，它会排除重新格式化这条看似快速的路径。

② [知识点] 把模糊动词改写成可判定状态

“配置”“启用”“部署”“保证可访问”都不是单一动作。`enabled` 描述开机启动配置，`active` 描述当前运行；防火墙已放行不代表服务正在监听；目录存在不代表文件已经挂载；Playbook 没有 `failed` 不代表模板内容正确。

每个目标都应有能够判定对错的状态表达，例如：用户 UID 为指定值且属于给定附加组；挂载源为目标 UUID、当前已挂载且 `fstab` 可验证；服务处于 `active` 与 `enabled`，端口正在监听且本机或远端访问成功。

③ [边界] 已知条件不是永远可信的当前证据

题目提供的初始状态用于缩小调查范围，但操作过程中对象会互相影响。修改网络可能使远程连接中断；重新加载防火墙可能暴露漏洞；一个错误的 `fstab` 条目会影响启动。完成相关操作前仍应通过最小查询确认关键前提，尤其是设备身份、已有数据、目标主机范围和变量来源。

[Cheatsheet] 读题四问：对象是什么？当前缺什么证据？目标如何判定？哪些状态必须保留？把“配置成功”拆成配置、当前状态、功能与持久性。

[操作专题] 用最小调查建立可恢复的操作基线

① [操作点] 只查询会改变决策的字段

调查不是把所有命令运行一遍，而是为下一步选择证据。存储题先建立设备、文件系统、挂载和 LVM 映射；服务题先看配置语法、状态、日志与监听；RHCE 先确认项目目录、配置来源、Inventory 匹配、SSH 与提权。

可把调查结果保留在考试草稿中：记录题号、对象、当前证据、待验证项。不要在系统中创建会干扰评分的复杂跟踪平台，也不要把口令或 Vault 明文复制到草稿。

② [操作点] 优先选择最小且可重复的修改

最小修改只改变题目要求的对象。编辑配置前保留系统原有结构；已有记录时先查询再选择新增或修改；能够使用专用模块表达目标状态时，不用无条件执行的 `shell`。破坏性命令只能在设备身份、数据要求和题意都明确时使用。

一次完成一个可验证层次。服务配置可按“软件包 → 配置语法 → 服务当前与启动状态 → 监听 → 防火墙 → 功能”推进；存储可按“设备 → 容量对象 → 文件系统 → 当前挂载 → 持久配置 → 数据”推进。

③ [验证点] 每次修改立即留下局部证据

局部验证的价值是缩小故障范围。配置文件修改后先做语法或结构检查，再启动服务；写入 `fstab` 后在重启前运行静态验证和实际挂载；Playbook 先做语法检查、列出主机范围并限制到小组执行。

局部证据必须注明边界：`systemctl is-active` 证明当前运行，不证明开机启动；`findmnt` 证明当前挂载，不证明 `fstab`；`ansible-playbook --syntax-check` 只证明控制端能够解析结构，不证明远端任务可执行。

[Cheatsheet] 查询只看影响决策的字段；修改只推进一个目标层；每步立即验证并写清证明边界；破坏性操作前重新确认对象与保留要求。

[诊断专题] 卡题时保留现场，让下一条证据推进判断

① [诊断点] 先给失败定位层次

失败可以先粗分为输入或语法、对象不存在、权限或连接、容量或依赖、当前状态、持久配置、最终功能。这个分类不等于根因，却能决定下一条查询。命令不存在时先查帮助和软件包；服务失败看状态与日志；远端 `unreachable` 先查 Inventory、SSH 和提权，不应先改业务任务。

② [诊断点] 不用重复执行代替调查

重复运行相同命令只在幂等验证或确认瞬态问题时有意义。`lvextend` 报空间不足，应查询 VG 余量；端口新增提示已存在，应查询现有类型；模板任务失败，应查看变量、模板路径和渲染语法。错误信息是证据入口，不是要求“换几个参数试试”。

③ [诊断点] 暂时跳过时保存缺失证据

若一道题耗时过长，记录已经成立的层次、最后错误、尚未确认的对象和可能影响的其他题。尽量保留现场，不使用删除、重建或全局放宽权限来换取短暂成功。完成其他题后再回来，新的系统状态和剩余时间可能让判断更清晰。

[Cheatsheet] 现象 → 所在层 → 当前证据 → 一个假设 → 下一条查询；不要重复失败命令；跳题时记录“已证实、未证实、最后错误、影响范围”。

[操作专题] RHCSA：把单台主机验收到可重启终态

① [操作点] 按风险和依赖安排顺序

先确认身份、主机名、网络和仓库等基础条件，再处理用户、软件、服务、存储、SELinux、容器与恢复任务。高风险的磁盘、启动和网络修改应在证据充分时执行，并在修改后立即验证。题目之间共享对象时，记录后一次修改是否会覆盖前一题。

② [验证点] 一页 RHCSA 检查清单

- 网络地址、路由、DNS、主机名与名称解析符合题意；
- 仓库可用，指定软件包存在，服务配置语法正确；
- 用户 UID、主组、附加组、密码策略、权限、ACL 与 `sudo` 分别核对；
- 服务同时检查 `active`、`enabled`、监听、日志、`firewalld` 与最终访问；
- 分区、PV/VG/LV、文件系统、Swap、当前挂载与 `fstab` 分层检查；
- SELinux 保持 `Enforcing`，文件规则、当前标签、端口类型与 `Boolean` 使用最小入口；
- `cron/at`、时间同步、日志持久性和容器用户服务按题意验证；
- `findmnt --verify`、`mount -a` 等重启前检查没有未处理错误；
- 不依赖临时 `chmod`、临时标签、`runtime-only` 防火墙或手工启动状态；
- 最后从题目清单逐项复核，不以“命令都执行过”代替验收。

③ [边界] 重启是高价值但有前提的验证

重启能够暴露启动配置、挂载和服务依赖问题，但错误的网络或 `fstab` 也可能让系统难以恢复。应先完成静态验证、当前功能检查并保留恢复路径，再根据考试环境和题意决定是否重启。不能把未经检查的重启当作排错第一步。

[Cheatsheet] RHCSA：调查 → 手工修改 → 当前状态 → 功能 → 持久配置 → 交叉影响 → 最终清单；重启前先验证网络、`fstab` 与关键服务。

[操作专题] RHCE：从控制端语法推进到多节点幂等终态

① [操作点] 先证明执行边界

确认当前工作目录中的 `ansible.cfg` 是否生效，Inventory 是否解析到预期主机与组，连接用户、Python 与 `become` 是否可用。执行前使用 `--syntax-check`、`--list-hosts`、`--list-tasks`，必要时以 `--limit` 缩小第一轮范围。

② [验证点] 一页 RHCE 检查清单

- 项目目录、`ansible.cfg`、Inventory 与 `requirements` 文件位于题目要求路径；
- 主机组、嵌套关系、变量目录和 Vault 密码来源能够被当前项目读取；
- YAML 语法、模块 FQCN、字段类型、模板与变量名经过静态检查；
- 第一轮限制范围执行，区分 `failed`、`unreachable`、`changed`、`ok` 与 `skipped`；
- Handler 只由真实变更通知，错误控制没有掩盖必须失败的任务；
- 文件内容、所有者、模式、服务、端口、防火墙、SELinux、存储和挂载在受管节点验证；
- 不把控制节点文件存在误认为远端文件已经正确；
- 扩展到全部目标主机后再次核对 `recap` 与每组终态；
- 第二次执行不应出现无法解释的 `changed`，并再次检查远端功能；
- Vault 内容保持加密，输出和提交文件中没有明文秘密。

③ [验证点] 第二次执行只证明幂等性的一个维度

第二次执行没有 `changed`，说明模块认为目标状态已满足；若模板本身写错、Inventory 漏掉主机或验证对象选错，错误终态同样可能稳定不变。因此幂等检查必须和受管节点上的真实文件、服务、端口、挂载与业务访问组合使用。

[Cheatsheet] RHCE：配置/Inventory → 语法与主机范围 → 限制执行 → 远端终态 → 扩展执行 → 第二次执行 → 再验收；`recap` 不是最终功能证明。

[经典任务] 在混合任务清单中建立可验证的执行顺序

你接手一组尚未开始的考试任务：其中包含静态网络、软件仓库、共享目录、持久挂载、非标准 Web 服务，以及一个面向多组主机的自动化部署。部分任务共享服务、路径和端口；存储设备已有状态未知；自动化项目已提供若干变量文件。

目标终态：在不删除未知数据、不降低 SELinux 保护、不泄露敏感变量的前提下，为这些任务建立一条可执行顺序。每个任务都要说明操作前需要的证据、完成后最少验证层次、可能影响的其他任务，以及最终综合验收方法。

限制条件：不能用重建对象代替排错；不能以服务 active 代替外部访问；不能以 Playbook recap 成功代替远端终态；暂时跳过的任务必须能够恢复上下文。

验收要求：执行计划能够区分 RHCSA 与 RHCE 路径，包含重启前检查、幂等性检查、交叉影响检查和两份最终清单。

请先独立完成。参考分析与推荐路径从下一页开始。

[参考解答] 用证据门把高风险任务和依赖任务串起来

① [操作点] 读题拆解与依赖排序

先建立对象表：网络与名称解析影响仓库和远程连接；仓库影响软件包；存储设备身份决定能否写入；Web 访问依赖服务配置、监听、防火墙与 SELinux；自动化依赖控制端配置、Inventory、SSH、提权和变量。共享对象旁标记题号，避免后续操作覆盖已有终态。

推荐顺序是先做低风险基础调查，再完成网络、名称解析和仓库；随后处理身份、文件与服务；设备证据充分后处理存储；最后完成 SELinux 与外部功能闭环。RHCE 项目先做控制端静态检查和小范围执行，再扩展到所有受管节点。

② [验证点] 为每类任务设置证据门

网络任务至少验证连接配置、活动地址、路由、DNS 与必要连通；共享目录验证身份、组继承、权限与新文件行为；存储验证设备归属、容量对象、文件系统、当前挂载、持久条目与数据；Web 服务验证配置语法、active/enabled、监听、firewalld、SELinux 和最终请求。

自动化在执行前证明配置来源、主机范围和变量可解析；执行后在远端检查文件、服务与功能；第二次执行检查无法解释的变更。任何证据门失败时停留在该层调查，不删除已经正确的下层对象。

③ [诊断点] 卡题与最终收束

卡题记录“目标对象、已成立状态、最后错误、缺失证据、共享影响”。转去其他题时不留下临时 Permissive、全局权限放宽或半写入的关键配置。返回后从缺失证据继续，而不是重新执行全部命令。

最终按 RHCSA 与 RHCE 检查清单逐项验收。重启前先检查持久网络、fstab 和关键服务；是否重启取决于恢复条件。RHCE 在全量执行和第二次执行后仍要检查受管节点终态。

[Cheatsheet] 建对象表与共享影响 → 基础证据 → 低风险依赖 → 高风险存储/网络 → 服务完整链 → RHCE 小范围到全量 → 持久与幂等 → 最终清单。

[本章收束] 让每一步都能回答“改变了什么，证明了什么”

实操考试的稳定方法不是背一条固定流水线，而是持续维护对象、状态和证据之间的对应关系。调查只读取会影响决策的信息；修改只推进一个目标层；验证说明自己能证明什么、不能证明什么；卡题时保留现场并记录缺失证据。

RHCSA 最终把单机配置验收到当前、功能和持久状态；RHCE 还要证明主机范围、变量化、多节点终态与第二次执行。只要不把局部成功扩大成整体结论，就能把复杂任务拆成可恢复、可验证的小闭环。

Shell 解析、引用、展开与命令组合

从输入字符到最终 argv：把引用、变量、glob、环境与退出状态放进同一条推理链。

对象模型操作语义验证诊断经典任务

大字号阅读版

本章阅读导航

先抓住一条主线：Shell 不会把输入行原样交给程序。它先识别词和操作符，再按引用上下文完成展开、拆分和路径名匹配，最终形成命令名、参数列表、执行环境与退出状态。

- 01识别词与引用判断哪些字符仍是 Shell 语法
- 02参数与命令替换把变量值或命令输出嵌入当前词
- 03词拆分未引用展开结果可能变成多个参数
- 04路径名展开glob 模式可能变成多个真实路径
- 05最终 argv确认参数数量、顺序与精确文本
- 06执行与状态区分环境边界并保存退出状态

专题地图

- 1. 知识专题从输入文本到最终参数列表
- 2. 知识专题引用与转义：决定哪些字符还是语法
- 3. 知识专题展开流水线：一个词为什么改变内容或数量
- 4. 操作专题Shell 变量、导出属性与子进程环境
- 5. 操作专题从目标参数反向构造安全命令
- 6. 知识专题退出状态与命令组合
- 7. 知识专题当前 Shell 与子 Shell
- 8. 诊断专题从异常结果反推解析阶段

阅读时持续回答

- 1. 哪些字符仍属于 Shell 语法？
- 2. 哪些内容会发生展开？
- 3. 展开结果会不会继续被拆分？
- 4. glob 会生成几个真实路径？
- 5. 程序最终获得几个参数？
- 6. 变量是否进入子进程环境？
- 7. 哪一条命令的退出状态被保存？
- 8. 状态修改发生在当前 Shell 还是子 Shell？

章节边界

重定向、文件描述符与管道留给第 02 章；本地帮助体系完整展开留给第 03 章；本章不扩展为完整 Bash 脚本编程课程。

第 01 章 · 正文

输入行不是程序参数：先重建 Bash 的处理链

你在终端里看到的是一行字符，程序真正接收到的却是一组参数。Bash 会先识别词和控制操作符，再根据引用上下文执行参数展开、命令替换、词拆分和路径名展开，最后才选择命令并执行。很多看似“命令不会用”的故障，本质上是程序从一开始就收到了错误数量或错误内容的参数。

```
name='report final'
printf '<%s>\n' $name
printf '<%s>\n' "$name"
printf '<%s>\n' '$name'
```

这三条命令分别可能产生两个参数、一个展开后的参数和一个字面的 `$name`。本章不把“变量外面最好加引号”当作口号，而是要求你始终能回答：输入中的哪些字符仍是语法、哪些内容会被替换、替换结果是否会再拆分、模式会匹配多少路径、程序最终获得多少参数，以及命令结束后哪一个状态被保留。

源文本 → Shell 词 → 引用上下文 → 参数展开 / 命令替换 → 词拆分 → glob → 最终 argv → 执行环境 → 退出状态

核心概念：先把对象和边界分清

概念

Shell 词与最终 argv Shell 词是 Bash 解析命令行时识别的语法单位，它可能最终成为命令名或一个参数；最终 argv 则是展开、拆分、路径名匹配和引用移除之后真正交给程序的字符串数组。源文本里看见三个“片段”不代表程序一定收到三个参数，反过来也一样。最可靠的证据不是目测命令行，而是用参数观察器确认参数数量和每个参数的精确文本。

概念

引用上下文 单引号、双引号与反斜杠不是执行前最后添加的保护壳，而是从解析阶段开始改变字符是否仍具有语法含义。单引号使内部字符保持字面值；双引号允许参数展开和命令替换，但通常保持展开结果为一个词；反斜杠只保护紧随其后的一个字符。引用只能保证 Shell 层的参数边界，不能证明路径存在、权限正确或业务目标完成。

概念

参数展开 参数展开用 `$name` 或 `${name}` 读取当前 Shell 中的变量值。花括号最重要的基础作用是明确变量名边界，例如 `${name}01` 表示变量 `name` 后接字面 `01`。未设置、已设置为空、未引用空展开和显式空参数并不是同一状态，必须通过最终参数数量来区分。

概念

命令替换 `$(command)` 会在子 Shell 环境中执行命令，并把其标准输出嵌入当前词；末尾连续换行会被移除。命令替换捕获的是文本，不是“成功或失败”这一状态。若替换结果应当保持一个参数，应把整个替换放在双引号中；若希望修改当前 Shell 的目录或变量，则不能依赖命令替换内部的状态变化。

概念

词拆分 词拆分主要作用于未引用的参数展开和命令替换结果，并依据 IFS 将文本分成字段。因此变量值中已有空格时，`$value` 可能生成多个参数，而 `"$value"` 通常保持一个参数。源文本中的空格属于较早的词法边界，变量值中的空格是否再次拆分则由引用上下文决定。

概念

路径名展开 (glob) glob 由 Shell 在目标程序启动前完成，`*`、`?` 和字符集合模式可能被替换成零个、一个或多个路径。模式被双引号包住后不会展开；默认情况下 `*` 通常不匹配名称开头的点，无匹配模式通常会保留为字面文本。Shell 选项和 locale 会改变部分边界，因此异常时要先调查当前 Shell 状态。

概念

Shell 变量与导出属性 Shell 变量属于当前 Shell 的状态；`export` 添加的是“随后启动的子进程可以继承”这一属性。`set`、`export -p` 和 `env` 观察的对象不同：当前 Shell 状态、带导出属性的变量，以及当前进程环境。子进程获得的是启动时的环境副本，不能反向修改父 Shell。

概念

退出状态 命令或命令列表完成后会向 Shell 返回一个整数状态，`0` 通常表示按该命令自身约定成功，非零表示未成功。 `$?` 只保存最近一条命令或命令列表的状态，会被下一条命令立即覆盖；`&&` 与 `||` 正是依据这个状态短路执行。命令状态为零只能证明局部接口成功，不能自动证明完整业务终态正确。

概念

当前 Shell 与子 Shell { list; } 在当前 Shell 中执行，变量和工作目录修改可能保留；(list) 与 \$(...) 建立子 Shell 边界，内部状态通常不会回写父 Shell。选择分组形式的核心不是括号外观，而是任务是否要求保留状态，以及是否只需要捕获文本或隔离影响。

操作语义速查：先记住关键入口的接口形状

这里不是完整 man page，而是本章最关键的查询、构造和验证入口。进入正文后，每个专题再按对象、证据和边界展开。

type / type -a

SYNOPSIS

```
<pre><code>type [-afptP] name [name ...]</code></pre>
<p>确认当前 Bash 如何解析一个名称，区分 alias、函数、builtin 和 <code>PATH</code> 中的外部命令。</p>
<dl><dt><code>type name</code></dt><dd>显示当前优先解析结果。</dd><dt><code>-a</code></dt><dd>尽可能列出所有可用入口。</dd><dt><code>-P</code></dt><dd>只按 <code>PATH</code> 查找外部可执行文件；它会忽略 builtin 和函数。</dd></dl>
```

printf 与参数观察器

SYNOPSIS

```
<pre><code>printf format [arguments ...]
```

bash -c 'command_string' name [argument ...]

printf 用显式格式显示字符串边界；bash -c 启动新的 Bash，把 name 作为 \$0，其余参数依次成为 \$1、\$2……。

```
printf 'arg=<%s>\n' "$@"
```

逐个显示参数，并给每个参数增加可见边界。

```
"$#"
```

读取位置参数数量。

```
"$@"
```

在双引号内保留每个位置参数的独立边界。

set / export -p / env

SYNOPSIS

```
<pre><code>set
```

export [-fn] [name[=value] ...] env [name=value ...] [command [argument ...]]

三个入口分别观察当前 Shell 状态、带导出属性的变量和进程环境，不能互相替代。

set

显示当前 Shell 的变量和函数，范围最宽。

export -p

显示具有导出属性的变量。

env

显示当前进程环境，或给单条命令构造临时环境。

变量赋值、临时环境与清理

SYNOPSIS

```
<pre><code>name=value
```

name=value command unset name export -n name

赋值改变当前 Shell 变量；命令前赋值只为该次命令提供环境；删除变量和取消导出属性是两种不同操作。

name=value

等号两侧不能随意加入空格。

name=value command

为该命令提供临时环境，不等于永久持久配置。

unset name

删除变量。

```
export -n name
```

保留当前 Shell 变量值，但取消后续子进程继承属性。

```
; / && / ||
```

SYNOPSIS

```
<pre><code>command1 ; command2
```

command1 && command2 command1 || command2

使用前一命令的退出状态决定后续命令是否执行；`;` 不判断状态，`&&` 成功继续，`||` 失败继续。

```
status=$?
```

在执行任何其他命令前立即保存最近状态。

左结合

`a && b || c` 在 `a` 失败或 `b` 失败时都可能执行 `c`，不能无条件当成严格 if/else。

短路

被跳过的命令没有运行，也不会产生新的退出状态。

```
( ... ) / { ...; } / $(...)
```

SYNOPSIS

```
<pre><code>( list )
```

```
{ list; } result=$(command)
```

三种形式分别用于隔离执行环境，在当前 Shell 中分组，以及捕获子 Shell 命令输出。

```
( list )
```

在子 Shell 环境执行，变量和目录修改通常不保留。

```
{ list; }
```

在当前 Shell 执行；右花括号前必须有分号或换行，左右花括号需要词边界。

```
$(command)
```

捕获标准输出并嵌入当前词；若结果应保持一个参数，通常使用 `"$(command)"`。

[知识专题] 从输入文本到最终参数列表

Shell 排错最稳定的切入点不是先猜“这个命令的选项是不是写错了”，而是先重建最终参数列表。命令行中的空格、引号、美元符号和星号都可能影响参数数量或内容；只有知道程序实际收到什么，才能继续判断目标命令如何解释这些参数。

① [知识点] 命令名、选项和普通参数在 Shell 层都是词

下面的命令：

```
ls -l /etc/hosts
```

经过 Shell 处理后，可以近似理解为：

```
argv[0] = ls
argv[1] = -l
argv[2] = /etc/hosts
```

第一个词用于选择要执行的命令，后续词成为参数。`-l` 被称为选项，是因为 `ls` 按自身接口把它解释成选项；对 Bash 来说，它首先只是一个普通参数。Shell 不会自动理解每个程序的选项语义。

这一区分很重要：

- Bash 决定词边界、展开和命令组合；
- 目标程序决定参数是什么意思；
- 某个参数以 `-` 开头，不代表 Shell 会特殊处理它；
- 某个字符串看起来像路径，也不代表 Shell 会先检查它是否存在。

② [知识点] 未引用空白和控制操作符建立边界

通常情况下，空格和制表符分隔命令中的词：

```
printf '%s\n' alpha beta
```

这里 `printf` 收到格式参数和两个待格式化参数。若要让空格成为一个参数内部的数据，必须引用或转义：

```
printf '<%s>\n' 'alpha beta'
printf '<%s>\n' alpha\ beta
```

换行通常结束当前命令。反斜杠紧接换行时，二者一起被移除，用于把一条命令续写到下一行：

```
printf '%s\n' \
'a long argument'
```

`;`、`&&` 和 `||` 是控制操作符，它们分隔命令或命令列表，不会作为普通参数传给左侧命令。它们的条件语义将在退出状态专题中展开。

③ [知识点] 名称解析决定实际执行对象

同一个名称可能对应：

- `alias`；
- Shell 函数；
- Bash builtin；
- `PATH` 中找到的外部可执行文件。

使用：

```
type printf
type -a printf
type export
type bash
```

`type` 回答“当前 Bash 会把这个名称解析成什么”；`type -a` 尽可能显示所有可用入口。它比只查找文件路径更适合解释 Shell 语义，因为 builtin 和函数可能根本没有一个独立的外部文件路径。

命令被找到并不代表任务已经完成。`type` 只能证明解析入口，不能证明参数正确、权限足够或命令执行后的业务终态正确。

④ [操作] 使用参数观察器显示最终边界

`printf` 可以让不可见的参数边界变得可见：

```
printf 'arg=<%s>\n' one 'two words' ''
```

预期有三次格式应用，其中最后一个参数为空字符串。为了同时查看参数数量和每个位置，可使用一个只读的 Bash 参数观察器：

```
bash -c '
printf "argc=%s\n" "$#"
printf "arg=<%s>\n" "$@"
' _ one 'two words' ''
```

这里 `_` 占据新 Bash 的 `$0`，其后的词依次成为 `$1`、`$2`……。观察器不解释目标程序的选项，只负责回答：

```
最终有多少个参数？
每个参数的精确文本是什么？
是否存在空参数？
一个变量是否意外变成多个参数？
```

⑤ [知识点] 源文本、Shell 词和最终参数是三个不同层次

不要把下面三种状态混为一谈：

1. 源文本：你在终端输入的字符，包括引号和反斜杠；
2. Shell 词：解析时识别出的词，仍可能包含待展开表达式；
3. 最终参数：展开、拆分、路径名匹配和引用移除之后交给命令的字符串。

引号通常不会作为字符出现在最终参数中。它们主要用于控制解析和展开：

```
printf '<%s>\n' "hello world"
```

最终参数是 `hello world`，不是包含双引号字符的 `"hello world"`。若目标参数本身必须包含引号字符，就要把这些引号作为数据另行引用或转义。

Cheatsheet

先用 ``type`` 确认执行入口。
再用参数观察器确认 `argc` 和每个 `arg`。
Shell 决定参数边界；目标程序决定参数语义。

[知识专题] 引用与转义：决定哪些字符还是语法

引用不是最后给文本“加保护壳”，而是在 Shell 识别和展开命令时持续影响字符的含义。最顺的判断方法是先问目标：需要完全保持字面文本，还是允许变量和命令替换，但仍必须保持一个参数？

① [知识点] 反斜杠保护紧随其后的一个字符

未引用的反斜杠让下一个字符失去特殊含义：

```
printf '%s\n' Your\ cost
printf '%s\n' \$HOME
printf '%s\n' \*
```

分别产生：

- 一个包含空格的参数 `Your cost`；
- 字面 `$HOME`；
- 字面 `*`。

反斜杠只保护紧随其后的字符，不会自动保护后面的整段文本。反斜杠换行是特殊情况：它们组成续行标记，二者都不会进入最终参数。

② [知识点] 单引号保留内部所有字符的字面值

单引号中的 `$`、`*`、反斜杠和空格都作为普通字符：

```
printf '<%s>\n' '$HOME *.txt \n'
```

单引号内部不能直接嵌入单引号。需要字面单引号时，必须结束当前引用、提供一个被保护的单引号，再开始新的引用。例如：

```
printf '%s\n' 'don'\''t'
```

这仍然是一个 Shell 词，由三个相邻片段拼接而成：`'don'`、`\'` 和 `'t'`。

适合使用单引号的典型目标：

- 显示字面的 `$HOME`；
- 保存含大量 Shell 特殊字符的固定模板；
- 防止变量、命令替换和 glob 发生。

③ [知识点] 双引号允许有限展开并保持一个词

双引号中仍会发生：

- 参数展开，例如 `"$HOME"`；
- 命令替换，例如 `"$(date +%F)"`；
- 某些反斜杠处理。

双引号会抑制普通的词拆分和路径名展开，因此变量值即使包含空格或星号，通常也会作为一个参数：

```
value='one two *'
printf '<%s>\n' "$value"
```

双引号不是“所有特殊字符都失效”。它的核心用途是：允许必要的替换，同时保留结果的参数边界。

④ [知识点] 双引号中的反斜杠不是通用转义

在双引号内部，反斜杠只在有限字符前保留转义作用，常见的是 \$、反引号、双引号、反斜杠和换行。其他字符前的反斜杠可能被保留为数据。因此，不要把下面两种写法机械地理解成完全等价：

```
printf '%s\n' "\\$HOME"
printf '%s\n' "\\q"
```

第一条用于保护美元符号；第二条中的反斜杠通常仍会作为参数内容的一部分。实际写命令时，应让引用结构表达目标，不要堆叠无意义反斜杠。

⑤ [知识点] 相邻片段会拼接为同一个 Shell 词

Shell 允许一个词由多种引用上下文拼接：

```
prefix='daily'
printf '<%s>\n' "$prefix"-report\ final
```

最终只有一个参数：

```
daily-report final
```

这个能力用于精确控制一个参数中哪些部分展开、哪些部分保持字面。例如：

```
printf '%s\n' 'literal-$HOME-'"$USER"
```

前半段保持字面，后半段允许变量展开，但它们仍共同组成一个参数。

⑥ [操作] 按目标选择引用方式

目标	推荐形式	原因
完全保持字面文本	'text'	阻止所有普通 Shell 展开
允许变量或命令替换，并保持一个参数	"\$value"、"\$(command)"	抑制词拆分和 glob
只保护下一个字符	\\$、\	局部转义
变量目录含空格，但后面的模式仍需 glob	"\$dir"/*.txt	只引用变量部分
参数本身需要包含引号字符	用另一种引用或局部转义	语法引号不会自动进入参数

⑦ [边界] 引用解决的是 Shell 层问题，不自动验证数据语义

"\$path" 能保证路径值作为一个参数，却不能证明：

- 路径存在；
- 当前用户有权限；
- 目标程序把它解释为路径；
- 对该路径执行操作是安全的。

引用是参数完整性的必要工具，不是完整的业务验证。

Cheatsheet

完全字面化 → 单引号。
允许展开但保持一个参数 → 双引号。
只保护下一个字符 → 反斜杠。
语法引号通常不会进入最终参数。

[知识专题] 展开流水线：一个词为什么会改变内容或数量

Shell 展开最容易被错误地学成一串互不相关的符号。更稳定的模型是跟踪“一个词在每一步会发生什么”：它可能被替换成新的文本，未引用结果可能被拆成多个词，随后每个词中的 glob 模式又可能变成多个路径。

① [知识点] 引用上下文从解析阶段就影响展开

可以用下面的简化流水线理解本章范围：

```
识别词与引用上下文
→ 参数展开和命令替换
→ 对未引用展开结果进行词拆分
→ 路径名展开
→ 移除语法引用字符
→ 形成最终参数
```

完整 Bash 还包括大括号、波浪号和算术展开等阶段。本章只轻量说明前两者，不扩展为完整脚本语言课程。

不要误解为“先删掉所有引号，再做展开”。引号在前面的解析和展开阶段已经决定哪些字符具有特殊含义；最后的引用移除只是把作为语法使用的引号和反斜杠从最终参数中去掉。

② [知识点] 参数展开读取变量值

最基本的参数展开形式：

```
name='servera'
printf '%s\n' "$name"
printf '%s\n' "${name}01"
```

花括号用于明确变量名边界。若写成：

```
printf '%s\n' "$name01"
```

Bash 会尝试读取名为 `name01` 的变量，而不是 `name` 后接字面 `01`。

本章只要求掌握 `$name` 和 `${name}` 的基本形式。`${parameter:-word}`、字符串截取、数组和间接展开等高级家族留给 Bash 脚本专题。

③ [知识点] 未设置、已设置为空和显式空参数不是同一状态

考虑：

```
unset value
empty=''
```

下面几种结果需要区分：

```
bash -c 'printf "argc=%s\n" "$#" _ $value'
bash -c 'printf "argc=%s\n" "$#" _ "$value"'
bash -c 'printf "argc=%s\n" "$#" _ "$empty"'
bash -c 'printf "argc=%s\n" "$#" _ '''
```

未引用的空展开可能不产生参数；双引号中的空展开或显式 `''` 会保留一个空参数。这个边界在命令要求一个“存在但为空”的字段时非常重要。

④ [知识点] 命令替换把命令标准输出嵌入当前词

推荐形式：

```
today=$(date +%F)
printf '%s\n' "$today"
printf '%s\n' "report-$(date +%F)"
```

`$(...)` 比反引号更易读、易嵌套。命令替换会移除替换结果末尾连续的换行；中间换行仍可能保留。若命令替换未被双引号包围，其结果还可能进行词拆分和路径名展开。

命令替换捕获的是文本输出，不是退出状态。退出状态仍由命令或赋值命令按 Bash 规则提供，并可通过 `$?` 读取。

⑤ [知识点] 命令替换在子 Shell 环境中执行

下面的修改不会按直觉保留到父 Shell：

```
place=$(cd /tmp; pwd)
pwd
```

`cd /tmp` 发生在命令替换创建的子 Shell 环境中；父 Shell 当前目录不因此改变。`place` 获得的是命令输出。

这一区分解释了两种不同目标：

需要捕获文本 → `\$(...)``
需要修改当前 Shell 状态 → 在当前 Shell 直接执行或使用 `{ ...; }`

⑥ [知识点] 词拆分主要作用于未引用的展开结果

设：

```
value='one two'
```

比较：

```
bash -c 'printf "argc=%s\n" "$#"; printf "<%s>\n" "$@"' _ $value
bash -c 'printf "argc=%s\n" "$#"; printf "<%s>\n" "$@"' _ "$value"
```

第一条通常得到两个参数，第二条得到一个参数。源文本中的空白在词法分析阶段建立边界；变量展开结果中的空白，则可能在后续词拆分阶段依据 IFS 产生多个字段。

因此“变量含空格时要加双引号”只是表象，真正机制是：双引号抑制了展开结果的词拆分。

⑦ [边界] IFS 会改变未引用展开结果的拆分规则

IFS 是 Bash 用于字段拆分的变量。管理员环境中它通常保留默认空白字符集合。若它被异常修改，未引用变量可能出现难以理解的参数边界。

诊断时可以查看：

```
printf 'IFS=<%q>\n' "$IFS"
```

本章不教学如何用自定义 IFS 编写解析脚本；只要求知道它是参数数量异常的一项状态证据。不要为了“修复一个命令”在共享 Shell 中随意重设 IFS，应先确认来源和影响范围。

⑧ [知识点] 路径名展开把模式变为文件名列表

常见模式：

模式	含义
*	零个或多个字符
?	任意一个字符
[abc]	集合中的一个字符
[!abc]	不在集合中的一个字符
[[digit:]]	当前 locale 中的数字字符类

例如：

```
printf '<%s>\n' /etc/*.conf
```

Bash 在执行 printf 前把模式替换成匹配路径列表。printf 自身并不会读取目录并解释星号。

⑨ [知识点] 默认 glob 有三个容易误判的边界

- 通常不匹配名称开头的点：* 不会自动包含 .hidden；
- 无匹配时通常保留原模式：程序可能收到字面 *.txt；
- 匹配顺序受 locale 等状态影响：不要把显示顺序当成跨环境不变的业务事实。

Bash 选项如 nullglob、failglob 和 dotglob 可以改变这些行为。RHEL 9 实机上的默认选项应通过真实 Shell 查询，应在目标 Shell 中查询，本章不预设其状态。

⑩ [操作] 只让需要的部分参与 glob

假设目录变量可能包含空格：

```
dir='/tmp/monthly reports'
```

正确的混合形式：

```
printf '<%s>\n' "$dir"/*.txt
```

解释：

- `"$dir"` 保持目录值为一个片段；
- `/` 和 `*.txt` 与它拼成同一个 Shell 词；
- 星号未引用，因此整个路径模式仍会参与路径名展开。

两个典型错误：

```
printf '<%s>\n' "$dir/*.txt" # 星号被引用，成为字面字符
printf '<%s>\n' $dir/*.txt   # 目录变量先可能被词拆分
```

⑪ [知识点] 大括号展开不是文件匹配

```
printf '%s\n' file{1,2,3}.txt
printf '%s\n' node{01..03}
```

大括号展开按语法生成字符串，不检查这些文件或主机是否存在。它发生得较早，结果还可能继续经历后续展开。

本章只要求能区分：

```
`{a,b}` → 生成候选字符串
`[ab]` → 路径名模式中的一个字符集合
```

⑫ [知识点] 波浪号展开有自己的引用边界

未引用的 `~` 位于适当位置时可展开为主目录：

```
printf '%s\n' ~
printf '%s\n' ~/logs
```

引用后的波浪号通常保持字面：

```
printf '%s\n' "~/logs"
```

需要在变量或配置中稳定表示当前用户主目录时，`"$HOME/logs"` 往往比依赖引号外的 `~` 更清晰。

Cheatsheet

```
变量值要保持一个参数 → `"$value"`。
目录变量含空格但仍需 glob → `"$dir"/*.txt`。
无匹配模式可能作为字面参数传入，必须观察 argv。
`${a,b}` 生成字符串；`[ab]` 参与路径名匹配。
```

[操作专题] Shell 变量、导出属性与子进程环境

变量问题必须同时区分三个层次：当前 Shell 中是否有值、变量是否具有导出属性、真正启动的子进程是否收到了该环境项。只在父 Shell 中 `echo` 出一个值，不能证明外部程序也能看到它。

① [操作] 创建和修改当前 Shell 变量

基本形式：

```
name=value
count=40
path='/tmp/report data'
```

`=` 两侧不能出现用作分隔的空格：

```
name = value
```

这不会被解析成赋值，而会尝试执行为 `name` 的命令并传入参数。变量名通常由字母、数字和下划线组成，不能以数字开头。变量名区分大小写。

赋值只改变当前 Shell 状态。两个独立终端各自运行不同 Shell，它们不会自动共享后续和普通变量修改。

② [查询] 使用 `set` 查看当前 Shell 状态

```
set
```

`set` 输出当前 Shell 的变量和函数，内容通常很多。已知变量名时，应优先精确显示：

```
printf 'name=<%s>\n' "$name"
```

需要判断变量是否存在而不是只看值时，完整脚本会使用更精细的参数展开或 `declare -p`；这些属于脚本语言扩展，本章只建立“值为空不等于一定未设置”的边界。

③ [操作] 使用 `export` 添加导出属性

```
mode=production
export mode
```

或一次完成：

```
export mode=production
```

`export` 不是把变量“发布到整个操作系统”，而是给当前 Shell 中的名称添加导出属性。此后由该 Shell 启动的子进程会在自己的初始环境中收到对应名称和值。

查看导出声明：

```
export -p
```

④ [查询] `set`、`export -p` 和 `env` 回答不同问题

命令	主要观察对象
<code>set</code>	当前 Bash 的变量和函数
<code>export -p</code>	当前 Bash 中带导出属性的名称
<code>env</code>	执行 <code>env</code> 这个进程实际收到的环境

诊断环境继承时，这三项可以形成证据链：

```
当前 Shell 是否有值
→ 是否具有导出属性
→ 子进程环境是否实际包含
```

⑤ [验证] 用新 Bash 检查真实继承

```
scope=private
bash -c 'printf "scope=<%s>\n" "$scope"'

export scope
bash -c 'printf "scope=<%s>\n" "$scope"'
```

第一条新 Bash 不应依赖未导出的普通变量；第二条在 `export` 后应能够读取。该方法验证的是新 Bash 进程的环境，仍不代表所有远程会话或将来登录都自动拥有这个变量。

⑥ [知识点] 子进程不能反向修改父 Shell

```
value=parent
bash -c 'value=child; printf "%s\n" "$value"'
printf '%s\n' "$value"
```

子进程可以修改自己的变量和环境，但父 Shell 的 `value` 仍保持原值。环境继承是父到子的初始复制，不是双向共享存储。

⑦ [操作] 给单条命令提供临时环境

```
MODE=test bash -c 'printf "%s\n" "$MODE"'
env MODE=test bash -c 'printf "%s\n" "$MODE"'
```

前置赋值针对这条简单命令的执行环境。它适合临时改变一个程序的 locale、模式或其他环境输入，而不必先永久导出到后续所有子进程。

验证边界：命令内部看到 `MODE=test`，不代表当前父 Shell 在命令完成后保留同样变量状态。

⑧ [操作] 删除变量与取消导出属性

```
unset mode
export -n mode
```

- `unset mode` 删除变量；
- `export -n mode` 取消导出属性，但变量值仍可留在当前 Shell。

操作后分别用 `printf`、`export -p` 和新的 `bash -c` 验证，不要只运行其中一条命令就假定全部状态都符合预期。

⑨ [边界] 当前变量不等于持久配置

在交互 Shell 中执行：

```
export EDITOR=vim
```

只改变当前 Shell 及其后续子进程。新的登录或新的独立终端是否得到该值，取决于 Bash 启动模式和启动文件配置。

启动文件完整加载顺序属于后续 Shell 环境专题。本章只建立接口：

```
当前 Shell 状态
✗ 新会话初始状态
✗ 系统范围持久配置
```

⑩ [安全边界] 不要把秘密值直接暴露在不必要的环境中

环境变量适合许多普通配置，但并不是无条件安全的秘密存储。进程环境可能被同一用户的诊断工具、转储或日志间接暴露。考试任务若没有要求，不要把口令、令牌等敏感值随意导出到广泛的子进程树。

Cheatsheet

```
`set` 看当前 Shell。
`export -p` 看导出属性。
`env` 或 `bash -c` 看子进程实际环境。
当前 export 不等于新登录持久配置。
```

[操作专题] 从目标参数反向构造安全命令

复杂命令不应从“拼一整行字符串”开始，而应先写出目标参数表。每个参数是一个整体还是允许展开成多个路径，决定了变量、命令替换和 glob 应处于什么引用上下文。

① [操作] 先写目标 argv

假设目标程序应收到：

```
arg 1 = --label
arg 2 = nightly report
arg 3 = /tmp/data set/a.txt
arg 4 = /tmp/data set/b.txt
```

再设计 Shell 文本：

```
label='nightly report'
dir='/tmp/data set'
command --label "$label" "$dir"/*.txt
```

这里 `--label`、`"$label"` 和每个匹配路径分别形成独立参数。

② [操作] 默认引用变量展开

变量值来自路径、用户名、日期标签或其他动态数据时，默认使用：

```
"$value"
```

只有明确希望一个变量值进一步被拆成多个参数或被当作 glob 模式时，才考虑省略引号；这类设计应有清晰证据和边界，不应依赖“当前值碰巧没有空格”。

③ [操作] 命令替换期望一个参数时放入双引号

```
stamp="$(date +%F)"
printf '<%s>\n' "report-$stamp"
```

或直接：

```
printf '<%s>\n' "report-$(date +%F)"
```

未引用命令替换可能因输出中的空白或 glob 字符改变参数数量。即使当前命令输出看起来简单，也应让引用表达长期意图。

④ [操作] 只对模式部分保留 glob 能力

```
"$dir"/*.conf
```

不要把整个路径模式放在双引号中，也不要让可能含空格的目录变量保持未引用。

执行修改性命令前，先将目标命令替换成参数观察器：

```
bash -c '
printf "arg=%s\n" "$#"
printf "arg=<%s>\n" "$@"
' _ "$dir"/*.conf
```

确认参数数量和每个路径后，再进入真正操作。

⑤ [边界] 不使用 eval 修补引用问题

把整条命令放入字符串，再交给 eval 会触发第二次 Shell 解析。变量值中的空格、引号、分号或命令替换语法可能在第二次解析时获得新的控制能力。

本章的默认策略是：

```
使用正常参数边界
→ 直接调用目标命令
→ 不把命令和数据混成一条待再次解析的字符串
```

⑥ [验证] 命令执行前后分别验证

执行前：

- type 确认入口；
- 参数观察器确认 argv；
- 检查 glob 是否匹配到预期集合；
- 检查变量和环境来源。

执行后：

- 立即保存退出状态；
- 使用目标对象的查询接口验证终态；
- 不把“参数看起来正确”扩大为“业务一定完成”。

Cheatsheet

```
先写目标参数表，再写 Shell 命令。
变量默认双引号。
只让确实需要的 glob 保持未引用。
不要用 `eval` 代替正确的参数设计。
```

[知识专题] 退出状态与 ;、&&、||

Shell 不只传递字符串，也传递命令完成后的状态。命令组合的关键不是“上一条有没有输出”，而是上一条实际返回的退出状态。状态是易失的：一旦执行下一条命令，\$? 就会被新状态覆盖。

① [知识点] \$? 保存最近命令或命令列表的状态

```
some_command
status=$?
printf 'status=%s\n' "$status"
```

正确顺序是先保存，再执行用于显示或验证的其他命令。错误顺序：

```
some_command
printf 'finished\n'
printf 'status=%s\n' "$?"
```

最后打印的是第一个 printf 的状态，不再是 some_command 的状态。

② [知识点] 0 和非零只按命令自身约定解释

Unix 命令通常使用：

0	按命令定义成功
非零	未成功或出现特定结果

但“成功”的范围由命令决定。例如，命令成功读取配置文件，不代表远端服务可用；创建文件的命令状态为 0，也不代表文件内容、权限和持久性都符合完整任务要求。

某些查询命令会用不同非零值区分“未找到”“发生错误”等情况。需要精确解释时，应查该命令帮助或手册，而不是把所有非零值都压缩成同一个原因。

③ [操作] ; 表示无条件顺序执行

```
command1; command2
```

无论 command1 状态如何，只要 Shell 能继续解析，都会尝试执行 command2。它适合表达单纯顺序，不适合用来保护依赖前一步成功的操作。

命令列表的最终状态通常来自最后实际执行的命令，因此：

```
false; true
```

整个列表的最终状态是 true 的状态，而不是前面的失败状态。

④ [操作] && 只在左侧成功时执行右侧

```
command1 && command2
```

若 command1 返回 0，执行 command2；否则跳过右侧。它适合表达“只有前置条件成功才继续”。

示例：

```
test -d /etc && printf '%s\n' 'directory exists'
```

这里只说明条件组合机制，test 的完整表达式语法不在本章展开。

⑤ [操作] || 只在左侧失败时执行右侧

```
command1 || command2
```

左侧非零时执行右侧。它常用于提供失败提示或最小替代动作：

```
test -r /etc/hosts || printf '%s\n' 'not readable'
```

不要把所有失败都立即连接到破坏性修复。|| 只告诉你左侧非零，具体原因仍需证据。

⑥ [知识点] `&&` 与 `||` 左结合

```
a && b || c
```

按下面的结构理解：

```
(a && b) || c
```

因此 `c` 在两种情况下可能执行：

1. `a` 失败；
2. `a` 成功，但 `b` 失败。

这不是严格等价于“如果 `a` 成功就执行 `b`，否则执行 `c`”。若 `b` 自身可能失败且不希望触发 `c`，应使用更明确的控制结构；完整 `if` 语法留给 Bash 脚本章节。

⑦ [知识点] 短路执行会影响哪些命令真正运行

```
false && printf '%s\n' never  
true  || printf '%s\n' never
```

右侧没有执行，因此不能假定它产生了输出、修改了状态或生成了自己的退出结果。一个列表的最终状态应根据最后实际执行的命令判断。

⑧ [验证] 分别验证状态和目标对象

推荐链路：

```
执行命令  
→ 立即保存 ` $? `  
→ 解释该状态能证明什么  
→ 查询目标对象当前状态  
→ 检查持久状态或功能状态（若任务要求）
```

本章的参数观察器可以证明 `argv`；退出状态可以证明命令的自报结果；它们都不能单独替代目标对象的最终验收。

Cheatsheet

```
` $? ` 只能读取最近状态，必须立即保存。  
`;` 无条件继续。  
`&&` 成功继续。  
`||` 失败继续。  
`a && b || c` 中，b 失败也会触发 c。
```

[知识专题] 命令分组与子 Shell 的必要边界

命令分组解决两个问题：把多条命令作为一个列表处理，以及控制这些命令是在当前 Shell 还是隔离的子 Shell 环境中执行。最重要的判断不是符号长什么样，而是变量和工作目录修改是否需要保留。

① [知识点] (`list`) 在子 Shell 环境执行

```
value=outside  
(  
  value=inside  
  cd /tmp  
  printf 'inside value=%s pwd=%s\n' "$value" "$PWD"  
)  
printf 'outside value=%s pwd=%s\n' "$value" "$PWD"
```

圆括号内能看到父 Shell 的初始状态，但其中的后续修改不会保留到父 Shell。它适合：

- 临时改变目录；
- 临时修改变量或 Shell 选项；
- 需要隔离一组命令副作用的场景。

② [知识点] { list; } 在当前 Shell 执行

```
value=outside
{
    value=inside
    cd /tmp
}
printf 'value=%s pwd=%s\n' "$value" "$PWD"
```

变量和目录修改会保留。语法边界必须准确：

- { 和 } 需要作为独立的保留字；
- 结束 } 前必须有分号或换行终止列表。

错误形式：

```
{ cd /tmp }
```

这里缺少在 } 前终止 cd /tmp 的分号或换行。

③ [知识点] 两种分组都产生退出状态

分组的退出状态来自其中最后实际执行的命令：

```
( true; false )
printf '%s\n' "$?"

{ false; true; }
printf '%s\n' "$?"
```

如果分组内部使用 && 或 ||，还必须先判断哪些命令实际执行。

④ [知识点] 命令替换也是子 Shell 边界，但目的不同

```
result=$(cd /tmp; pwd)
```

这里：

- 子 Shell 中执行 cd 和 pwd；
- 标准输出被捕获到 result；
- 父 Shell 工作目录保持不变。

普通圆括号分组主要用于隔离执行；命令替换主要用于把输出嵌入当前词。不要把两者都简化成“开了一个括号”。

⑤ [边界] 本章不提前展开重定向和管道

命令分组常与以下形式结合：

```
把整个分组的输出写入一个目标
把分组放在管道一侧
```

这些涉及标准输入输出、文件描述符和管道进程模型，归第 02 章。本章只建立：分组是一个命令列表，有自己的执行环境和退出状态。

⑥ [操作决策] 根据状态保留需求选择分组

```
需要隔离 `cd`、变量或选项修改 → `( ... )`
需要让修改留在当前 Shell      → `{ ...; }`
只需要获取命令输出            → `${...}`
```

三者都要继续检查参数、输出和退出状态，不能仅凭符号完成验证。

Cheatsheet

圆括号：子 Shell，修改不保留。
花括号：当前 Shell，修改保留。
命令替换：子 Shell 中执行并捕获输出。

[诊断专题] 从异常结果反推解析阶段

Shell 错误诊断应沿着最有区分度的证据推进。不要反复给命令添加引号或反斜杠直到“看起来能用”；先确定实际命令入口、最终参数、环境继承和退出状态，再做最小修改。

① [诊断] 含空格变量被拆成多个参数

症状：目标命令报告参数过多，或路径被分成几段。

当前证据：

```
printf 'value=<%s>\n' "$value"
bash -c 'printf "argc=%s\n" "$#"; printf "arg=<%s>\n" "$@"' _ $value
```

假设：未引用参数展开经过词拆分。

下一条最有区分度的证据：对比 `argc`：

```
bash -c 'printf "argc=%s\n" "$#"; printf "arg=<%s>\n" "$@"' _ "$value"
```

最小修复：在需要一个参数的位置使用 `"$value"`。

再验证：参数数量、完整路径和目标命令终态分别检查。

② [诊断] 程序收到字面的 `*.txt`

症状：错误中直接出现星号，或参数观察器显示 `/path/*.txt`。

调查顺序：

```
模式是否被整体放入引号
→ 当前目录或目录前缀是否正确
→ 是否存在匹配项
→ `set -f` 是否关闭 glob
→ `nullglob` / `failglob` 是否改变行为
```

最小修复不一定是“去掉所有引号”。目录变量含空格时，应改为：

```
"$dir"/*.txt
```

③ [诊断] glob 匹配数量比预期多或少

检查：

- 模式中的 `*`、`?` 和字符类范围；
- 是否误用了大括号展开；
- 隐藏文件是否需要包含；
- 当前 locale 是否影响字符类和排序；
- 目录中真实对象集合。

用参数观察器先枚举实际匹配结果，不要直接把不确定模式交给删除、权限修改或批量移动命令。

④ [诊断] 父 Shell 有值，子命令看不到变量

证据链：

```
printf 'parent=<%s>\n' "$name"
export -p
bash -c 'printf "child=<%s>\n" "$name"'
```

假设：变量存在但未导出，或启动的并不是预期子进程。

最小修复：

```
export name
```

或只给目标命令临时环境：

```
name=value command
```

再验证：在真正目标进程或等价的新 Bash 中读取，而不是只在父 Shell 打印。

⑤ [诊断] 变量或工作目录在命令后恢复原值

症状：括号内 `cd` 或赋值显示成功，但后续 Shell 未保留。

调查：

```
是否使用 ` ( ... ) `
→ 是否发生在 ` $(...) ` 内
→ 是否由另一个 ` bash -c ` 执行
→ 目标是否确实要求修改父 Shell
```

若目标是保留当前 Shell 状态，可改用直接执行或 `{ ...; }`。若目标只是隔离临时状态，则恢复原值正是正确行为，不应修复。

⑥ [诊断] `$?` 与预期命令不对应

症状：状态总是 0，或显示的是打印命令的结果。

最常见原因是状态被覆盖：

```
command
printf 'command finished\n'
status=$?
```

最小修复：

```
command
status=$?
printf 'status=%s\n' "$status"
```

再检查该状态能证明什么，并继续验证目标对象。

⑦ [诊断] `&&` / `||` 执行了意外分支

把表达式按左结合重新加括号：

```
a && b || c
↓
(a && b) || c
```

逐步记录状态：

```
a
sa=$?
b
sb=$?
```

实际排错时不要为了记录状态而改变原列表的执行条件；可以先使用等价、安全的测试命令复现，或把命令分别运行并记录。重点确认：是 `a` 失败，还是 `b` 在 `a` 成功后失败。

⑧ [诊断] 看起来执行了错误的命令

症状：参数正确，但行为与预期工具不同。

先运行：

```
type -a name
```

检查 alias、函数、builtin 和外部文件。必要时在受控诊断中使用明确路径或 `command builtin` 绕过函数/alias，但不要把绝对路径硬编码成所有场景的默认答案。

⑨ [诊断] 一条安全的通用调查链

症状

- ``type -a`` 确认入口
- 参数观察器确认 `argc/argv`
- 检查引用位置和展开类型
- 比较 ``set`` / ``export -p`` / 子进程环境
- 立即保存退出状态
- 判断当前 Shell 与子 Shell 边界
- 最小修复
- 重做参数、状态和目标终态验证

Cheatsheet

参数错 → 看 `argc/argv`。
变量错 → 比较当前值、导出属性和子进程环境。
状态错 → 立即保存 ``$?``。
修改未保留 → 检查是否进入子 Shell。

[经典任务] 精确构造并验证一个复杂参数列表

任务环境

当前登录用户在 Bash 中工作。实验环境已经准备好以下目录和文件：

```
/tmp/rhcsa01 data/  
├─ report 01.txt  
├─ report 02.txt  
└─ notes.md
```

当前 Shell 已有变量：

```
work='/tmp/rhcsa01 data'  
label='nightly report'  
scope='private'
```

`scope` 当前只是普通 Shell 变量，尚未导出。

提供以下只读参数观察器：

```
bash -c '  
printf "argc=%s\n" "$#"   
printf "arg=<%s>\n" "$@"   
' _
```

任务目标

在观察器后补充参数，使新 Bash 精确收到四个参数：

1. 字面文本 `$HOME`；
2. 一个参数 `nightly report-YYYY-MM-DD`，日期由 `date +%F` 在执行时生成；
3. `/tmp/rhcsa01 data/report 01.txt`；
4. `/tmp/rhcsa01 data/report 02.txt`。

随后完成以下验证：

1. 证明 `scope` 未导出时，新 Bash 不能依赖它；
2. 导出 `scope` 后，证明新 Bash 能读取 `private`；
3. 分别运行一个明确成功和一个明确失败的命令，并在执行其他命令前保存状态；
4. 使用 `&&` 仅在成功后显示 `SUCCESS-BRANCH`；
5. 使用 `||` 仅在失败后显示 `FAILURE-BRANCH`；
6. 用变量赋值和 `pwd` 对比 `( ... )` 与 `{ ...; }` 的状态保留差异。

限制条件

- 不使用 `eval`；
- 不把整条命令保存为字符串再执行；

- 不使用重定向或管道；
- 不修改或删除提供的文件；
- 不假设 glob 一定匹配，必须检查参数观察结果；
- 不把退出状态 0 扩大解释为完整业务终态。

验收矩阵

验收项	证据
参数总数	观察器显示 <code>argc=4</code>
字面美元符号	第一个参数为 <code>\$HOME</code> ，不是真实主目录
空格未拆分	第二个标签和每个文件路径保持完整
命令替换发生	第二个参数含实际日期，不含字面 <code>\$()</code>
glob 发生	后两个参数分别是两个 <code>.txt</code> 路径
未混入非目标文件	<code>notes.md</code> 不在参数列表
未导出变量不可见	第一次子 Bash 不获得可依赖的 <code>scope</code>
导出后可见	第二次子 Bash 显示 <code>private</code>
状态保存及时	打印的是目标命令状态，而非后续 <code>printf</code> 状态
短路分支正确	成功只触发 <code>&&</code> ，失败只触发 <code> </code>
子 Shell 隔离	圆括号内变量和目录修改不保留
当前 Shell 修改	花括号内变量和目录修改按设计保留

[参考解答] 先重建参数，再验证环境和状态

① 调查当前对象和变量

先确认正在使用 Bash，并观察相关命令的解析入口：

```
type bash
type -a printf
type export
```

查看变量时使用明确边界：

```
printf 'work=<%s>\n' "$work"
printf 'label=<%s>\n' "$label"
printf 'scope=<%s>\n' "$scope"
```

这一步证明父 Shell 中变量当前有值，但还没有证明 `scope` 已导出，也没有证明 glob 能匹配目标文件。

② 构造四个目标参数

完整观察命令：

```
bash -c '
printf "argc=%s\n" "$#"
printf "arg=<%s>\n" "$@"
' _ \
'$HOME' \
"$label-$(date +%F)" \
"$work"/*.txt
```

逐项解释：

- `'$HOME'` 使用单引号，因此美元符号和变量名保持字面；
- `"$label-$(date +%F)"` 允许变量和命令替换，同时保持一个参数；
- `"$work"/*.txt` 只引用目录变量，保留 `*.txt` 的路径名展开；
- glob 应匹配两个 `.txt` 文件，因此最终总参数数为 4。

推荐验证结果形态：

```
argc=4
arg=<$HOME>
arg=<nightly report-YYYY-MM-DD>
arg=</tmp/rhcsa01 data/report 01.txt>
arg=</tmp/rhcsa01 data/report 02.txt>
```

这里的 `YYYY-MM-DD` 只表示执行时的实际日期格式，本章不预设一个固定日期。

③ 分析三个典型错误

错误一：

```
"$HOME"
```

这会得到真实主目录，而不是字面 `$HOME`。

错误二：

```
$label-$(date +%F)
```

标签含空格，未引用展开结果可能被拆成多个参数。

错误三：

```
"$work/*.txt"
```

星号在双引号中失去路径名展开能力，程序会收到字面模式。

还有一种危险修正：

```
$work/*.txt
```

目录变量自身未引用，会先因空格产生错误边界，不能作为正确方案。

④ 验证变量导出前后的差异

先确认导出声明中是否有 `scope`：

```
export -p
```

使用新 Bash 检查：

```
bash -c 'printf "child scope=<%s>\n" "$scope"'
```

然后导出并再次验证：

```
export scope
bash -c 'printf "child scope=<%s>\n" "$scope"'
```

也可以使用更明确的环境查询：

```
env
```

但 `env` 输出很长；针对具体变量时，新 Bash 的明确打印更容易审阅。验证结论应限制为：当前 Shell 后续启动的这个子 Bash 能否读取变量，不扩大为新的登录会话也会持久拥有该值。

⑤ 及时保存退出状态

使用 Bash builtin `true` 和 `false` 构造不产生破坏的明确状态：

```
true
ok_status=$?
printf 'true status=%s\n' "$ok_status"

false
fail_status=$?
printf 'false status=%s\n' "$fail_status"
```

关键是赋值紧跟目标命令。若先执行 `printf`，原状态会被覆盖。

⑥ 验证短路执行

```
true && printf '%s\n' 'SUCCESS-BRANCH'
false && printf '%s\n' 'SHOULD-NOT-RUN'

false || printf '%s\n' 'FAILURE-BRANCH'
true  || printf '%s\n' 'SHOULD-NOT-RUN'
```

这些证据分别说明：

- `&&` 只在左侧状态为 0 时执行；
- `||` 只在左侧非零时执行；
- 被短路跳过的命令没有实际运行。

⑦ 对比圆括号和花括号

为避免永久改变到未知目录，先保存当前工作目录：

```
start_dir=$PWD
state=outside
```

圆括号：

```
(
    state=inside-subshell
    cd /tmp
    printf 'subshell state=<%s> pwd=<%s>\n' "$state" "$PWD"
)
printf 'after subshell state=<%s> pwd=<%s>\n' "$state" "$PWD"
```

父 Shell 中应仍为 `state=outside`，目录仍为 `start_dir`。

花括号：

```
{
    state=inside-current-shell
    cd /tmp
}
printf 'after group state=<%s> pwd=<%s>\n' "$state" "$PWD"
```

此时修改保留在当前 Shell。完成演示后恢复目录：

```
cd "$start_dir"
```

恢复动作也应检查退出状态和 `pwd`，不能假定路径永远可用。

⑧ 分层验收

参数层： `argc=4`，每个 `arg` 内容正确。
展开层：字面美元、变量、命令替换和 `glob` 各按预期发生。
环境层： `export` 前后子进程可见性不同。
状态层：成功为 0、失败非零，且保存未被覆盖。
组合层： `&&` 和 `||` 只执行正确分支。
Shell 层：圆括号隔离，花括号保留。
边界层：没有使用重定向、管道、`eval` 或破坏性文件操作。

[本章收束] 把每条命令还原成参数、环境和状态

完成本章后，面对一条含变量、空格、星号和命令替换的命令，不应只凭视觉判断。稳定的工作方法是：

1. 用 ``type -a`` 确认实际执行入口。
 2. 写出目标 `argv`：每个参数应是什么。
 3. 用单引号、双引号或反斜杠控制语法与数据边界。
 4. 跟踪参数展开、命令替换、词拆分和 `glob`。
 5. 用参数观察器核对 `argc` 和每个 `arg`。
 6. 用 ``set``、``export -p`` 和子 `Bash` 区分变量与环境。
 7. 命令后立即保存 ``$?``。
 8. 用 ``;``、``&&``、``||`` 表达明确的顺序或条件。
 9. 根据状态是否要保留，选择当前 `Shell` 或子 `Shell`。
 10. 再用目标对象的查询接口完成最终验收。

主要判断表

需求	首选形式
完全保持字面文本	<code>'text'</code>
变量展开后保持一个参数	<code>"\$value"</code>
捕获命令输出并保持一个参数	<code>"\$(command)"</code>
目录变量含空格且需要 <code>glob</code>	<code>"\$dir"/*.txt</code>
让子进程继承变量	<code>export name</code>
只给一条命令临时环境	<code>NAME=value command</code>
保存最近退出状态	<code>status=\$?</code> ，紧跟目标命令
无条件继续	<code>a; b</code>
成功后继续	<code>a && b</code>
失败后继续	<code>a b</code>
隔离变量和目录修改	<code>(...)</code>
在当前 <code>Shell</code> 保留修改	<code>{ ...; }</code>

向下一章交接

本章已经回答：命令如何形成参数、如何选择执行对象、如何返回状态。第 02 章《标准输入输出、重定向与管道》将在此基础上继续回答：

- 命令从哪里读取数据？
`stdout` 和 `stderr` 分别去哪里？
重定向操作符何时生效？
管道如何连接多个进程？

本章不把这些问题的提前压缩成几条符号，以避免把“参数解析”和“文件描述符连接”混为同一个机制。

∴ {cover}

标准输入输出、重定向与管道

从 FD 路由到管道状态：把数据去向、证据边界与失败判断放进同一条推理链。

对象模型操作语义验证诊断经典任务

大字号阅读版

...

... {reading-nav}

本章阅读导航

先抓住一条主线：Shell 不是在命令结束后搬运屏幕文字，而是在命令启动前建立文件描述符连接。学习时始终把“数据路径”和“退出状态”分成两条证据链。

01识别数据源谁写 stdout、谁写 stderr、谁读取 stdin

02标出 FD为每个命令写出 0、1、2

03建立管道先连接左 stdout 与右 stdin

04应用重定向再从左到右改写各阶段端点

05保留证据文件、终端与 tee 各证明一层

06判断状态\$?、PIPESTATUS 与 pipefail

专题地图 - **知识专题** 从终端表象回到文件描述符模型 - **操作专题** 构造可判定的 stdout、stderr 和退出状态 - **操作专题** 覆盖、追加与分别保存两个输出流 - **知识专题** `2>&1` 的复制语义与顺序差异 - **知识专题** 管道连接对象与 stderr 旁路 - **操作专题** 用 `tee` 在流经位置保留证据 - **操作专题** 用 `PIPESTATUS` 和 `pipefail` 判断失败 - **操作专题** here-document 的必要范围 - **诊断专题** 从表象回溯 FD 路由和阶段状态 - **经典任务** 分流证据：定位中间阶段失败

阅读时持续回答 1. 当前命令的 FD 0、1、2 分别指向哪里？ 2. 哪一步打开、截断、追加或复制了端点？ 3. 普通管道传递的是哪个流？ 4. stderr 是否仍沿原端点旁路？ 5. `tee` 文件证明数据到达了哪个位置？ 6. 哪个命令的退出状态被 `\$?` 保存？ 7. `PIPESTATUS` 是否已被后续命令覆盖？ 8. 当前证据不能证明什么？ ...

{nav-note} **前后章边界：第 01 章负责引用和展开；本章只轻量引用 here-document delimiter 的引用效果。`grep`、`sed`、`awk` 的处理语义留给第 05 章，Journal 与 rsyslog 留给第 13 章。 ...

...

第 02 章 · 正文

在终端里看到“文字出现在哪里”，很容易让人误以为 stdin 固定来自键盘、stdout 和 stderr 固定显示在屏幕上。实际上，命令只面对一组已经打开的文件描述符。Shell 先解析命令行；对于管道，它先建立相邻阶段的默认连接，再按每个命令从左到右应用显式重定向，最后启动各阶段。因此真正需要追踪的是“FD 编号当前指向哪个端点”，而不是屏幕上最后出现了什么。

本章围绕一条统一主线推进：

数据由谁产生
→ 通过哪个 FD 离开或进入命令
→ 管道先建立了哪些默认连接
→ 显式重定向如何从左到右改写端点
→ 哪些数据被保存，哪些仍然可见
→ 每个阶段以什么状态结束
→ 现有证据能证明什么，又不能证明什么

最常见的误判都可以从这条主线解释：命令主体没有成功，但 `>` 已经把旧文件清空；`2>&1` 顺序写反后，错误仍出现在终端；管道末端成功退出，却掩盖中间阶段失败；`tee` 文件有数据，却不能证明上游和下游都成功。第 01 章已经建立 Shell 解析和引用前提，本章只在 here-document 处轻量引用；文本筛选和日志系统分别留给后续章节。

核心概念

... {concept-block}

概念

文件描述符（file descriptor）是进程访问已打开输入输出对象的编号。0、1、2 是常用入口，不是键盘、屏幕或某个文件本身；终端、普通文件、管道和 `/dev/null` 才是它们可能指向的端点。判断一条命令时，应写出“FD 编号 → 当前端点”，而不是把编号直接翻译成设备。 ...

... {concept-block}

概念

标准输入、标准输出与标准错误 是程序约定使用的三个数据通道，默认对应 FD 0、1、2。stdout 通常承载可继续处理的数据，stderr 通常承载诊断信息，但这只是程序接口约定；文字是否“像错误”、终端是否着色，都不能替代对通道和退出状态的独立观察。 ...

... {concept-block}

概念

重定向目标 是 Shell 在命令启动前为某个 FD 建立的新端点。`>`、`>>` 和 `2>` 会打开文件，`2>&1` 会复制另一个描述符在该时刻的端点。重定向按从左到右应用，所以顺序不是排版差异，而是状态变化顺序；目标无法打开时，命令主体可能根本没有开始。...

...

概念

管道 (pipeline) 是由 Shell 建立的一组相邻进程连接。普通 `A | B` 把 A 的 stdout 连接到 B 的 stdin，stderr 默认仍沿各自原端点输出。管道传递的是数据流，不是把左侧结果变成右侧命令参数；每个阶段仍是独立执行对象，并拥有独立退出状态。...

...

概念

PIPESTATUS 是 Bash 保存最近一个前台管道各阶段退出状态的数组，元素顺序与管道从左到右一致。它回答“哪一段失败”，但会被后续简单命令更新，因此必须立即复制；数据文件存在、终端出现结果或单一 `$?` 都不能替代逐阶段状态。...

...

概念

pipefail 是当前 Bash 的管道汇总策略。默认情况下，管道状态来自最后一个阶段；启用 `pipefail` 后，汇总状态取最右侧非零阶段，全部为零时才为零。它让中间失败更容易暴露，却不会列出所有失败阶段，也不会自动保存数据或替代 **PIPESTATUS**。...

操作语义速查

...

...

`printf` / `cat`

SYNOPSIS

```
printf FORMAT [ARGUMENT ...]
cat [FILE ...]
```

用 `printf` 产生可预测的 stdout，用 `cat` 把文件或 stdin 原样送到 stdout，建立不受业务状态干扰的最小观察环境。

重要参数 / 形式

```
printf '%s\n' 'DATA:alpha'
```

明确产生一行 stdout，优先于依赖实现差异的 `echo`。

```
printf '%s\n' 'WARN:beta' >&2
```

让本次 `printf` 的 FD 1 复制 FD 2 当前端点，构造确定的 stderr。

```
cat file / cat < file
```

前者由 `cat` 打开参数文件；后者由 Shell 先把 FD 0 指向文件。

...

...

`<` / `>` / `>>`

SYNOPSIS

```
command < input
command > output
command >> output
```

分别为 stdin 选择文件、覆盖 stdout 目标、追加 stdout 目标。文件打开和截断发生在命令主体运行前。

重要参数 / 形式

```
< input
```

让 FD 0 从文件读取。

```
> output / 1> output
```

以覆盖方式打开 stdout 目标：既有内容可能先被截断。

```
>> output / 1>> output
```

在文件末尾追加，不清空旧内容。

...

...

`2>` / `2>&1`

SYNOPSIS

```
command 2> error.log
command > all.log 2>&1
```

2> 打开文件并把 FD 2 指向它；2>&1 不打开文件，而是复制 FD 1 在该时刻的端点。

重要参数 / 形式

```
2> error.log / 2>> error.log
    覆盖或追加 stderr。
> all.log 2>&1
    先改变 FD 1，再让 FD 2 复制它，两个流进入同一文件。
2>&1 > out.log
    FD 2 先复制原 stdout，随后仅 FD 1 改向文件；两者不等价。
```

...

... {,quickref-entry}

管道 |

SYNOPSIS

```
producer | consumer
producer 2>&1 | consumer
```

普通管道把左侧 stdout 连接到右侧 stdin。stderr 默认旁路；需要合流时显式复制 FD 2。

重要参数 / 形式

```
A | B
    A 的 FD 1 → 管道 → B 的 FD 0。
A 2>&1 | B
    A 的 stdout 与 stderr 一起进入 B。
A | & B
    Bash 简写；正文优先显式形式以保留 FD 推理。
```

...

... {,quickref-entry}

tee

SYNOPSIS

```
tee [OPTION]... [FILE]...
```

从 stdin 读取数据，同时写入指定文件和自身 stdout，使数据可以留存后继续进入下游。

重要参数 / 形式

```
tee evidence.log
    覆盖写入证据文件并继续输出。
-a
    追加到文件，不覆盖既有内容。
A | tee before.log | B
    文件只证明数据到达 B 之前的 tee 位置。
```

...

... {,quickref-entry}

set -o pipefail

SYNOPSIS

```
set -o pipefail
set +o pipefail
```

改变当前 Bash 对整条管道的单一汇总状态计算方式。

重要参数 / 形式

```
set -o pipefail
    启用；汇总状态取最右侧非零阶段。
set +o pipefail
    关闭并恢复默认“只看最后阶段”的规则。
```

作用范围

属于当前 Shell 会话状态；临时调查结束后应按原状态恢复。

...

... {,quickref-entry}

\$? / PIPESTATUS

SYNOPSIS

```
pipeline_rc=$?
stage_rc=("${PIPESTATUS[@]}")
```

\$? 保存最近命令或管道的单一汇总状态，**PIPESTATUS** 保存最近前台管道的逐阶段状态。

重要参数 / 形式

```
pipeline_rc=$? stage_rc=("${PIPESTATUS[@]}")
    在同一个赋值命令中立即复制上一条管道的两类状态。
${stage_rc[*]}
    后续展示已保存数组；不要直接展示原 PIPESTATUS 后再声称它仍代表目标管道。
```

...

...

[知识专题] 从终端表象回到文件描述符模型

前面的概念块已经给出对象轮廓，本专题继续把它展开成可操作的路由模型。最常见的误区是把 stdout 等同于屏幕、把 stdin 等同于键盘；更准确的判断是：Shell 在命令启动前建立描述符映射，命令只通过编号读写，终端、普通文件、管道和 /dev/null 只是可能的端点。

① [知识点] 0、1、2 是默认文件描述符

一个普通命令启动时通常至少继承三个描述符：

描述符	通道	典型用途	交互式 Shell 中的通常端点
0	stdin	程序读取的输入	当前终端
1	stdout	正常结果、可继续处理的数据	当前终端
2	stderr	诊断、警告和错误信息	当前终端

“通常端点”不是永久绑定。以下三条命令都让 **cat** 输出相同内容，但输入来源不同：

```
cat input.txt
cat < input.txt
printf '%s\n' 'payload' | cat
```

第一条由 **cat** 自己打开文件；第二条由 Shell 先把 FD 0 指向文件；第三条由 Shell 把上游 stdout 和 **cat** 的 stdin 连接起来。

② [知识点] 文件描述符编号与端点必须分开理解

可以把描述符理解为进程内部的一张小型路由表：

```
FD 0  -> 当前输入端点
FD 1  -> 当前正常输出端点
FD 2  -> 当前诊断输出端点
```

端点可以是：

- 控制终端；

- 普通文件；
- 管道的读端或写端；
- `/dev/null`；
- 本章不展开的其他打开对象。

因此，1 不是“屏幕”的代号，2 也不是“红色错误文字”的代号。颜色由终端或程序决定，Shell 只处理数据通道。

③ [知识点] stdout、stderr 与退出状态不是同一类证据

以下命令故意同时产生正常输出、错误输出和非零退出状态：

```
bash -c '
    printf "%s\n" "DATA:alpha"
    printf "%s\n" "WARN:beta" >&2
    exit 7
'
```

这三个事实应分别判断：

```
stdout 中有什么
stderr 中有什么
命令退出状态是什么
```

程序完全可以向 stderr 写一条警告后返回 0；也可以没有输出任何错误文字却返回非零状态。排错和考试验收都不能只看文字是否“像错误”。

④ [知识点] Shell 在命令运行前准备描述符

重定向不是命令执行完成后的文本搬运。Shell 解析命令行后，先打开目标、复制描述符或建立管道，再启动命令。由此得到两个重要结论：

1. > 可能在目标命令真正运行前就截断文件；
2. 目标文件无法打开时，命令主体通常不会开始执行。

这也是为什么“命令失败”与“文件没有变化”不能画等号。

[Cheatsheet] 0/1/2 是描述符编号；终端、文件和管道是端点；数据通道与退出状态必须分别取证；重定向由 Shell 在命令启动前建立。

[操作专题] 用 printf、cat 和 bash 构造可判定的输入输出

真实命令常同时受权限、语言环境、目录内容和服务状态影响。为了把 FD 关系单独看清，本专题先用可控的生产者与消费者，让每条数据的通道和退出状态都可判定；这里不追求复杂业务，只建立后续所有验证的最小实验。

① [操作] 使用 printf 产生确定的 stdout

作用对象：当前命令的 FD 1。基本语义：按格式写出数据；默认写入 stdout。基本形式：

```
printf '%s\n' 'DATA:alpha'
```

建议始终显式写格式字符串。与不同行为实现的 echo 相比，printf 更适合构造包含反斜杠、连字符或空字符串的确定测试数据。

② [操作] 使用 >&2 构造确定的 stderr

```
printf '%s\n' 'WARN:beta' >&2
```

这里 >&2 让该次 printf 的 stdout 复制 FD 2 当前指向的端点。结果是文字沿 stderr 路径离开命令。该语法也再次说明：重定向作用的是描述符，而不是字符串的“类型”。

③ [操作] 使用 cat 观察 stdin 到 stdout 的桥接

作用对象：文件参数或 FD 0 中的数据。基本语义：按顺序读取输入并写向 stdout。典型形式：

```
cat evidence.txt
cat < evidence.txt
printf '%s\n' 'payload' | cat
```

本章只使用 cat 作为透明读取与验证入口。行号、不可见字符显示等能力留给后续文本处理内容。

④ [操作] 使用 `bash -c` 同时控制两个流和退出状态

```
bash -c '  
  printf "%s\n" "DATA:alpha"  
  printf "%s\n" "WARN:beta" >&2  
  exit 7  
'
```

这个受控命令适合验证：

- `stdout` 是否进入预期文件；
- `stderr` 是否仍出现在终端；
- 合并顺序是否正确；
- 保存的退出状态是否仍为 `7`。

验证边界：这些命令用于建立可判定的关系；在实际系统中仍应运行命令并保存真实输出、文件内容和退出状态。

[Cheatsheet] 用 `printf` 生产明确数据，用 `>&2` 选择 `stderr`，用 `cat` 验证文件或 `stdin`，用 `bash -c '...; exit N'` 构造可控状态。

[操作专题] 覆盖、追加与分别保存 `stdout/stderr`

重定向文件时，真正需要判断的不是符号长什么样，而是“哪个描述符以什么方式打开哪个目标”。覆盖和追加的差异发生在打开文件时；`stdout` 和 `stderr` 是否分开，则由描述符编号决定。

① [操作] 使用 `>` 覆盖 `stdout` 目标

```
printf '%s\n' 'new data' > output.log
```

`>` 等价于显式写法 `1>`。若目标不存在，Shell 尝试创建；若目标存在，通常先截断为零长度，再把 `FD 1` 指向它。

操作后至少验证两层：

```
cat output.log  
printf 'rc=%s\n' "$?"
```

注意：第二条得到的是 `cat` 的状态，不再是最初 `printf` 的状态。需要保留原命令状态时，必须紧跟在原命令后赋值：

```
printf '%s\n' 'new data' > output.log  
command_rc=$?  
cat output.log  
printf 'command_rc=%s\n' "$command_rc"
```

② [操作] 使用 `>>` 追加 `stdout`

```
printf '%s\n' 'next data' >> output.log
```

`>>` 让写入发生在文件末尾，不清空已有内容。追加不会自动去重，也不会补救上一行缺少换行的情况。验证时应检查旧证据是否仍存在，而不只看新增内容。

③ [操作] 使用 `2>` 与 `2>>` 单独保存 `stderr`

```
bash -c 'printf "DATA\n"; printf "WARN\n" >&2; exit 7' \  
> stdout.log 2> stderr.log
```

此时：

```
FD 1 → stdout.log (覆盖)  
FD 2 → stderr.log (覆盖)
```

追加错误证据时使用：

```
command 2>> stderr.log
```

不要把 `2>` 读成“第二次重定向”。数字 `2` 明确选择 `stderr`。

④ [操作] 同时选择输入和输出目标

```
cat < input.txt > output.txt
```

FD 0 和 FD 1 分别指向两个文件。只要路径不同，这种“从文件读取、写入另一文件”的模型很清楚。

高风险反例：

```
cat data.txt > data.txt
```

Shell 可能先截断 `data.txt`，`cat` 随后读到的已经是空文件。不能把普通覆盖重定向当作安全的原地编辑工具。

⑤ [边界] `/dev/null` 会丢弃证据

```
command 2> /dev/null
```

这可以在已经明确错误无关时减少噪声，但不应成为未知故障的默认答案。考试或调查阶段若仍需证明错误类型，应先保存或观察 `stderr`，而不是直接丢弃。

⑥ [边界] `noclobber` 可能改变覆盖行为

某些 Shell 会话可能启用 `noclobber`，使普通 `>` 拒绝覆盖既有文件。遇到“路径可写但覆盖失败”时，应先查询当前 Shell 选项，不要直接删除文件或改权限。本章不把 `noclobber` 展开为独立配置专题。

[Cheatsheet] `> / 1>` 覆盖 `stdout`；`>> / 1>>` 追加 `stdout`；`2>` 覆盖 `stderr`；`2>>` 追加 `stderr`；保存原命令 `$?` 必须发生在下一条验证命令之前。

[知识专题] `2>&1` 的本质是复制当前端点，顺序决定结果

`2>&1` 是本章最容易被错误记忆的语法。它不是“把 2 写进 1”，也不是“将错误写到名为 1 的文件”。它表示：让 FD 2 复制此刻 FD 1 的端点。因为重定向按从左到右应用，FD 1 在该位置指向哪里，会直接决定结果。

① [知识点] `2>` 与 `2>&1` 完成不同动作

```
2> error.log : 打开 error.log, 让 FD 2 指向该文件
2>&1          : 复制 FD 1 当前端点, 让 FD 2 指向同一端点
```

`>` 后面通常是路径；`>&` 明确表示描述符复制。两者不能混为“都是错误重定向”。

② [知识点] `> all.log 2>&1` 将两个流都送入文件

按顺序展开：

```
初始: FD 1 → 终端, FD 2 → 终端
> all.log: FD 1 → all.log, FD 2 → 终端
2>&1: FD 2 复制 FD 1 当前端点
最终: FD 1 → all.log, FD 2 → all.log
```

典型形式：

```
command > all.log 2>&1
```

③ [知识点] `2>&1 > out.log` 只把 `stdout` 送入文件

```
初始: FD 1 → 终端, FD 2 → 终端
2>&1: FD 2 复制 FD 1 当前端点, 即终端
> out.log: FD 1 → out.log, FD 2 仍指向原终端
最终: FD 1 → out.log, FD 2 → 终端
```

因此：

```
command 2>&1 > out.log
```

与上一种写法不等价。重定向不存在“Shell 自动帮你理解最终意图”的步骤。

④ [操作] 用受控命令验证顺序

```
bash -c '  
    printf "%s\n" "DATA:alpha"  
    printf "%s\n" "WARN:beta" >&2  
    exit 7  
' > combined-a.log 2>&1  
rc_a=$?
```

另一种顺序：

```
bash -c '  
    printf "%s\n" "DATA:alpha"  
    printf "%s\n" "WARN:beta" >&2  
    exit 7  
' 2>&1 > combined-b.log  
rc_b=$?
```

验收应同时检查：文件内容、终端仍可见的流，以及保存的 `rc_a`、`rc_b`。

⑤ [边界] Bash 合并简写不是基础模型的替代品

Bash 支持：

```
command &> all.log  
command &>> all.log
```

它们分别表示合并覆盖和合并追加。为了清晰展示 FD 关系，并兼顾其他 Bourne 风格 Shell，本章参考答案优先使用：

```
command > all.log 2>&1  
command >> all.log 2>&1
```

[Cheatsheet] `2>&1` 复制的是“此刻的 FD 1”；从左到右追踪端点；`>file 2>&1` 与 `2>&1 >file` 不等价。

[知识专题] 管道连接的是 stdout 与 stdin，不是命令参数

管道把相邻进程连接成数据流。左侧命令不需要知道右侧是谁，右侧也不需要把左侧输出解析成命令行参数。Shell 建立一个管道对象，让左侧 FD 1 指向写端、右侧 FD 0 指向读端。

① [知识点] A | B 的默认连接关系

```
A 的 FD 1 → 管道写端  
管道读端 → B 的 FD 0
```

因此：

```
printf '%s\n' 'payload' | cat
```

`payload` 是 `cat` 从 `stdin` 读取的数据，不是 `cat` 的文件名参数。

② [知识点] stderr 默认绕过普通管道

```
bash -c 'printf "DATA\n"; printf "WARN\n" >&2' | cat
```

默认只有 `DATA` 沿管道进入 `cat`；`WARN` 继续沿左侧命令原来的 FD 2 端点输出。普通 `|` 不会自动“收集所有屏幕文字”。

③ [操作] 显式让 stderr 进入管道

```
producer 2>&1 | consumer
```

Bash 也提供简写：

```
producer |& consumer
```

为了让重定向顺序清晰，本章主要使用显式形式 `2>&1 |`。只有在确认运行环境是 Bash 且可读性更好时，才使用 `|&`。

④ [知识点] Shell 先建立管道，再应用每个命令自身的重定向

```
producer > saved.txt | consumer
```

管道原本计划连接 `producer` 的 `stdout`，但 `> saved.txt` 随后把该命令的 `FD 1` 改向文件。结果是数据进入 `saved.txt`，`consumer` 通常收不到该数据。

这不是管道“失效”，而是后应用的命令级重定向覆盖了端点。

⑤ [知识点] 每个管道阶段都是独立的执行对象

以下管道可能把数据正确写入 `evidence.log`，同时中间阶段返回非零：

```
printf '%s\n' 'payload' |  
  bash -c 'cat; exit 9' |  
  tee evidence.log
```

数据路径和状态路径必须分别判断。只看文件或终端内容，无法证明中间阶段成功。

⑥ [边界] 管道是流，不保证完整记录边界

管道传递字节流。程序是否按行读取、何时缓冲、是否提前关闭读端由各程序决定。RHCSA 级操作通常使用行文本工具，但不能把“每行一定立即到达”当成管道本身的保证。

[Cheatsheet] | 默认连接左 `stdout` 到右 `stdin`；`stderr` 旁路；`2>&1 |` 显式合流；命令自身重定向可以覆盖管道端点；管道数据可见不等于各阶段成功。

[操作专题] 用 `tee` 在流经位置保留证据

普通 `>` 会把 `stdout` 从后续管道改向文件。`tee` 则作为一个真实的管道阶段读取 `stdin`，把相同数据同时写入一个或多个文件和自己的 `stdout`，因此可以在不截断后续处理链的情况下留下证据。

① [操作] 覆盖写入证据文件并继续输出

```
producer | tee evidence.log | consumer
```

作用对象：`tee` 收到的 `stdin`。基本语义：将输入复制到文件，并原样写到 `stdout`。验证：

```
cat evidence.log
```

该文件证明数据到达了 `tee` 所在位置，但不证明上游和下游全部成功。

② [操作] 使用 `-a` 追加证据

```
producer | tee -a evidence.log | consumer
```

不带 `-a` 时，指定文件会按覆盖方式打开；带 `-a` 时追加。长期调查前应明确是否需要保留历史，避免无意覆盖旧证据或把多次实验混在一起。

③ [知识点] `tee` 的位置定义证据含义

```
A | tee before-B.log | B | tee after-B.log | C
```

- `before-B.log`：B 处理前的数据；
- `after-B.log`：B 处理后的 `stdout`；
- 两个文件不同，可能说明 B 修改、筛选或未输出数据；
- 两个文件相同，也不能单独证明 B 的退出状态为零。

④ [操作] 同时观察终端和保存文件

当 `tee` 位于管道末端：

```
producer | tee evidence.log
```

stdout 继续连接终端，因此终端可见和文件留存可以同时获得。若只想保存、不想终端显示，仍可在 `tee` 后重定向其 stdout；但这样会改变证据可见性，应写明目的。

⑤ [诊断] 文件没有写入时先检查 `tee` 自身

症状：上游似乎有输出，终端也可能显示内容，但证据文件缺失或不完整。

调查链：

```
确认数据是否真的到达 tee
→ 检查 tee 对应的管道阶段状态
→ 检查路径、父目录、权限和可用空间
→ 用 cat 验证实际文件
→ 修复后重新执行并再次检查状态
```

不要因为 `tee` 是“辅助命令”就忽略它的失败。

[Cheatsheet] `tee file` 覆盖，`tee -a file` 追加；文件证明数据到达 `tee` 的位置；`tee` 是管道阶段，也有独立退出状态。

[操作专题] 用 `$?`、`PIPESTATUS` 和 `pipefail` 判断管道失败

管道的各阶段可以独立成功或失败，但 Shell 还必须给整条管道提供一个汇总状态。Bash 默认使用最后阶段的状态，这使“上游失败、下游仍正常退出”的管道看起来成功。正确调查需要同时保存汇总状态和逐阶段状态。

① [知识点] 默认管道状态来自最后阶段

```
printf '%s\n' 'payload' |
bash -c 'cat; exit 9' |
tee evidence.log
```

在默认 Bash 规则下，管道 `$?` 反映最后一个 `tee` 阶段，而不是中间 `bash` 阶段。若 `tee` 成功，汇总状态可能是零。

② [操作] 使用 `${PIPESTATUS[@]}` 查看每个阶段

Bash 的 `PIPESTATUS` 数组按从左到右顺序保存最近一个前台管道各阶段的状态：

```
PIPESTATUS[0] → printf
PIPESTATUS[1] → bash -c
PIPESTATUS[2] → tee
```

典型查看形式：

```
printf '%s\n' "${PIPESTATUS[@]}"
```

但直接运行这条查看命令本身又会形成新的简单命令，随后数组会被更新。因此更稳妥的做法是立即复制。

③ [操作] 在一个赋值命令中同时保存汇总和逐阶段状态

```
pipeline_rc=$? stage_rc=("${PIPESTATUS[@]}")
```

Bash 在展开赋值右侧时仍能读取上一条管道留下的 `$?` 和 `PIPESTATUS`，随后把它们保存到普通变量。之后再执行展示命令：

```
printf 'pipeline_rc=%s\n' "$pipeline_rc"
printf 'stage_rc=%s\n' "${stage_rc[*]}"
```

不要在保存前先运行 `cat`、`echo`、`printf`、`test` 或其他命令。

④ [操作] 启用 `pipefail` 改变汇总规则

```
set -o pipefail
```

启用后：

- 所有阶段都返回零，管道状态为零；
- 有阶段非零，管道状态取最右侧非零阶段的状态。

关闭：

```
set +o pipefail
```

`pipefail` 是当前 Shell 的行为状态。临时调查应在受控范围开启并在结束后恢复，避免无意改变后续脚本或交互命令的判断。

⑤ [知识点] `pipefail` 与 `PIPESTATUS` 解决不同问题

工具	回答的问题
<code>\$?</code>	Shell 对最近命令或管道的单一汇总判断是什么
<code>PIPESTATUS</code>	最近管道的每个阶段分别是什么状态
<code>pipefail</code>	管道汇总状态是否应该暴露中间或上游的非零结果

`pipefail` 不能告诉你所有失败阶段，也不会保存输出数据；`PIPESTATUS` 也不会自动让脚本停止。两者不能互相替代。

⑥ [诊断] 管道看起来成功时的证据链

症状：结果文件有数据，`$?` 也是 0
→ 假设：最后阶段成功掩盖了上游失败
→ 下一条证据：立即保存 `PIPESTATUS`
→ 最小修复：在需要可靠汇总的范围启用 `pipefail`
→ 再验证：比较汇总状态、逐阶段状态和数据文件

⑦ [边界] 不把 `set -e` 混入本章结论

`set -e` 与管道、条件命令和函数存在更复杂的交互。本章只训练 `pipefail` 的状态汇总语义，不把“开启 `set -euo pipefail`”当作所有脚本的无条件默认答案。完整脚本错误处理留给 Shell 脚本章节。

[Cheatsheet] 默认 `$?` 看最后阶段；`PIPESTATUS` 看所有阶段；状态必须立即复制；`pipefail` 取最右侧非零；数据证据和状态证据仍要分别验证。

[操作专题] 用 here-document 提供必要的多行标准输入

当命令需要多行 `stdin` 时，here-document 可以让 Shell 从当前脚本或命令块中收集文本，并把结果连接到命令的 `FD 0`。它属于输入重定向，而不是临时文件语法或注释语法。

① [操作] 基本 `<<DELIMITER` 形式

```
cat <<EOF
line one
line two
EOF
```

Shell 持续读取，直到遇到只包含结束词的行。开始和结束词应保持一致，结束词不要附带多余空格。

② [知识点] 未引用结束词时正文可能发生展开

```
name='student'
cat <<EOF
user=$name
home=$HOME
EOF
```

未引用 `delimiter` 时，变量展开、命令替换和算术展开等可能发生。各类引用与展开的完整规则属于前一章《Shell 解析、引用、展开与命令组合》；本章只说明它们会影响最终送入 `stdin` 的文本。

③ [操作] 引用结束词以保留正文原样

```
cat <<'EOF'
user=$name
host=$(hostname)
EOF
```

引用开始处的 `delimiter` 会抑制 here-document 正文中的相关展开。引号只写在开始 `delimiter` 上，结束行仍写裸文本 `EOF`。

④ [边界] <<- 只剥离前导 TAB

```
cat <<-EOF
    line indented with a TAB
EOF
```

<<- 便于在脚本中用 TAB 缩进正文和结束词，但不会剥离普通空格。编辑器把 TAB 自动替换为空格时，结束词可能无法按预期匹配。

⑤ [验证] 把 here-document 当作 stdin 检查

```
cat > evidence.txt <<'EOF'
$HOME
$(hostname)
EOF
cat evidence.txt
```

验收重点是文件中是否保留字面量，而不是“命令是否打印过内容”。

⑥ [边界] 不在本章扩展其他输入结构

here-string <<<、进程替换 <(...)、命名管道和协进程不属于本章必要范围。遇到这些语法时只需识别其不属于普通 here-document，不在此展开。

[Cheatsheet] <<EOF 提供多行 stdin；未引用 delimiter 可能展开；<<'EOF' 保留字面量；<<- 只移除前导 TAB。

[诊断专题] 从表象回溯 FD 路由、数据证据和阶段状态

输入输出故障通常不是“某个符号记错”这么简单。有效诊断应先确定症状属于数据路由、文件打开、管道拓扑还是退出状态，再选择最有区分度的下一条证据。

① [诊断] 终端没有输出

```
症状：运行命令后终端为空
→ 当前证据：只知道终端没有显示
→ 假设 A：stdout 被重定向到文件
→ 假设 B：stdout 进入管道，下游没有再输出
→ 假设 C：命令没有产生 stdout，只产生 stderr 或无数据
→ 下一证据：逐项标注 FD 1/2 端点，检查目标文件和下游状态
→ 最小修复：改正端点或在需要的位置加入 tee
→ 再验证：终端、文件、退出状态分别检查
```

“终端为空”不能直接推出命令没有执行。

② [诊断] 错误仍显示在终端

常见原因：

- 只重定向了 stdout；
- 2>&1 出现时 FD 1 仍指向终端；
- 错误由管道中另一个阶段产生；
- 目标重定向本身打开失败，Shell 把诊断写到当前 stderr。

下一条最有区分度的证据不是继续加更多 2>，而是从左到右画出每个命令自己的 FD 映射。

③ [诊断] 命令失败后目标文件被清空

```
症状：命令没有得到预期结果，但旧文件内容消失
→ 当前证据：目标使用了 >
→ 假设：Shell 在命令主体运行前已截断目标
→ 下一证据：检查目标是否同时作为输入、命令主体是否真正启动
→ 最小修复：使用不同输出路径，验证后再替换；需要历史时使用明确追加
→ 再验证：先检查输出文件，再检查原始输入是否仍完整
```

不应通过无调查地修改权限、删除原文件或使用强制选项解决。

④ [诊断] 管道 \$? 为零但业务结果不可信

症状：最后文件存在，\$? 为 0，但怀疑前面阶段失败
→ 当前证据：默认汇总只代表最后阶段
→ 下一证据：重新执行并立即保存 PIPESTATUS
→ 最小修复：在该调查范围启用 pipefail
→ 再验证：数据文件 + pipeline_rc + stage_rc

若原始执行已经过去且未保存状态，不能从现有文件反推出每个阶段的真实退出值，只能在可重现条件下重新取证。

⑤ [诊断] tee 文件为空或缺失

优先区分：

上游没有数据
tee 没有运行
tee 无法打开目标
tee 写入过程中失败
下游状态与 tee 文件无关

最小证据组合：

```
pipeline_rc=$? stage_rc="{${PIPESTATUS[@]}}"
cat evidence.log
```

再结合目标目录权限和空间进行下一层调查。

⑥ [诊断] here-document 没有结束或内容被意外替换

- 一直等待输入：检查结束词拼写、缩进和尾随空格；
- \$VAR 被替换：检查开始 delimiter 是否未引用；
- 使用 <<- 仍无法匹配：检查缩进是否为普通空格而非 TAB；
- 文本内含结束词：选择更独特的 delimiter。

[Cheatsheet] 先问“哪个 FD 指向哪里”，再查文件和阶段状态；不要用终端表象代替数据证据，不要用最后阶段状态代替整条管道。

[知识专题] 工作迁移：把重定向变成可审计的调查方法

在真实服务器中，重定向常用于收集证据、生成报告输入、保存批处理结果或为后续自动化提供稳定文件。高质量操作需要在执行前明确证据目的，在执行后区分内容、状态和持久文件三个层次。

① [工作方法] 先决定证据是否需要保留历史

- 临时替换：使用新的、独立路径；
- 连续调查：使用 tee -a 或带时间边界的文件名；
- 仅减少已确认噪声：才考虑 /dev/null；
- 不确定错误：优先单独保存 stderr。

② [工作方法] 给不同数据流明确命名

```
command > stdout.log 2> stderr.log
```

把把所有内容无条件合并更利于判断“业务数据缺失”还是“诊断信息增加”。需要完整顺序记录时可以合并，但应在任务说明中写明目的。

③ [工作方法] 把状态与证据文件一起交付

一个调查包至少应说明：

运行的命令
执行时的工作目录和用户
stdout/stderr 的保存路径
管道汇总状态
各阶段状态（适用时）
文件内容能够证明的范围
未验证的外部终态

④ [工作方法] 为后续自动化保留确定性接口

自动化工具通常分别接收 stdout、stderr 和返回码。手工阶段先建立正确模型，后续才不会把“任务无失败”扩大为“输出内容正确”，也不会因为把错误合并进数据文件而破坏解析。

[Cheatsheet] 变更前确定证据目的；输出和错误尽量分流；复杂管道保存逐阶段状态；交付时说明证据边界。

[经典任务] 分别保存输出和错误，并证明重定向顺序

任务环境

你在 Bash 中工作，当前目录为：

/tmp/rhcsa02-lab

目录中可能已经存在以下文件，且可能含有旧内容：

stdout.log
stderr.log
combined-a.log
combined-b.log

使用以下受控命令作为调查对象：

```
bash -c '  
  printf "%s\n" "DATA:alpha"  
  printf "%s\n" "DATA:omega"  
  printf "%s\n" "WARN:beta" >&2  
  printf "%s\n" "WARN:gamma" >&2  
  exit 7  
'
```

目标终态

1. 第一次执行时，用覆盖方式把 stdout 保存到 `stdout.log`，把 stderr 保存到 `stderr.log`。
2. 在运行任何其他验证命令前，把受控命令的退出状态保存到变量 `command_rc`。
3. 第二次执行时，把 stdout 追加到 `stdout.log`，仍把 stderr 覆盖写入 `stderr.log`。
4. 分别执行以下两种组合，并保存各自退出状态：

```
> combined-a.log 2>&1  
2>&1 > combined-b.log
```

5. 使用 `printf` 加标签，再用 `cat` 显示四个文件内容。
6. 用文件描述符状态说明两种组合为什么不同。
7. 不得把 stderr 丢入 `/dev/null`，不得使用输入文件同时覆盖自身，不把 `&>` 作为主要答案。

验收矩阵

验收层	需要证明的终态
stdout 分流	<code>stdout.log</code> 只保存 DATA 行，并能证明第二次为追加
stderr 分流	<code>stderr.log</code> 只保存最近一次执行的 WARN 行
合并 A	<code>combined-a.log</code> 同时接收 DATA 和 WARN
组合 B	<code>combined-b.log</code> 只接收 stdout，stderr 仍沿原终端端点
状态	保存的受控命令状态为其定义的非零值，而非后续 <code>cat</code> 的状态
解释	能从左到右说明 <code>2>&1</code> 复制时 FD 1 的端点
安全	没有因原地覆盖而破坏输入证据

典型错误

- 先运行 `cat`，再读取 `$?`；
- 把 `2>&1` 当成一个文件路径；
- 没有区分覆盖和追加；

- 只看终端，未检查文件；
- 根据“文件有数据”判断命令成功。

[经典任务] 构造可定位任一阶段失败的调查管道

任务环境

在 Bash 中使用以下管道：

```
printf '%s\n' 'payload' |
bash -c 'cat; exit 9' |
tee evidence.log
```

三个阶段分别承担不同职责：阶段 1 产生 `payload`；阶段 2 转发输入但故意以状态 9 结束；阶段 3 用 `tee` 写入 `evidence.log` 并继续输出。

目标终态

- 在默认 Shell 行为下执行管道。
- 不运行其他简单命令，立即在一次赋值中保存 `pipeline_rc=?` 与 `stage_rc=("${PIPESTATUS[@]}")`。
- 显示汇总状态和各阶段状态，解释默认汇总为何可能没有暴露阶段 2 的失败。
- 记录当前 `pipefail` 状态：开启后重新执行、立即保存两类状态，再恢复原状态。
- 使用 `cat evidence.log` 验证数据到达 `tee`。
- 明确写出：文件有 `payload` 能证明什么，不能证明什么。

验收矩阵

验收层	需要证明的终态
数据	<code>payload</code> 到达 <code>tee</code> 所在位置并写入文件
默认汇总	汇总规则只取最后阶段状态
逐阶段	数组位置与三段管道从左到右对应，并暴露中间非零状态
<code>pipefail</code>	启用后汇总状态反映最右侧非零阶段
会话边界	调查完成后没有无意保留改变的 Shell 选项
结论边界	数据落盘不等于整条管道全部成功

典型错误

- 执行 `echo $?` 后才查看 `PIPESTATUS`；
- 把 `PIPESTATUS` 当成单一数字；
- 认为 `pipefail` 会列出所有失败阶段；
- 开启 `pipefail` 后不恢复会话状态；
- 仅用 `cat evidence.log` 完成验收。

[参考解答] 经典任务一：分流、追加和顺序验证

以下命令是一组可重复的推荐验证序列。执行时应保留文件内容与退出状态作为实际证据。

① 建立目录并进入任务位置

```
mkdir -p /tmp/rhcsa02-lab
cd /tmp/rhcsa02-lab
```

② 覆盖方式分别保存两个流

```
bash -c '
printf "%s\n" "DATA:alpha"
printf "%s\n" "DATA:omega"
printf "%s\n" "WARN:beta" >&2
printf "%s\n" "WARN:gamma" >&2
exit 7
' > stdout.log 2> stderr.log
command_rc=$?
```

此时的目标关系：

```
FD 1 → stdout.log
FD 2 → stderr.log
command_rc → 受控命令的退出状态
```

③ 验证覆盖结果与保存状态

```
printf '%s\n' '--- stdout.log ---'
cat stdout.log
printf '%s\n' '--- stderr.log ---'
cat stderr.log
printf 'command_rc=%s\n' "$command_rc"
```

预期关系：stdout 文件只含两条 DATA，stderr 文件只含两条 WARN，保存状态对应 exit 7。验收时以实际运行保存的文件内容和退出状态为准。

④ 第二次执行，stdout 追加、stderr 覆盖

```
bash -c '
  printf "%s\n" "DATA:alpha"
  printf "%s\n" "DATA:omega"
  printf "%s\n" "WARN:beta" >&2
  printf "%s\n" "WARN:gamma" >&2
  exit 7
' >> stdout.log 2> stderr.log
append_rc=$?
```

验收重点：stdout.log 保留第一次两条 DATA 并新增两条；stderr.log 因使用 2> 仍只有最近一次两条 WARN。

⑤ 验证合并顺序 A

```
bash -c '
  printf "%s\n" "DATA:alpha"
  printf "%s\n" "DATA:omega"
  printf "%s\n" "WARN:beta" >&2
  printf "%s\n" "WARN:gamma" >&2
  exit 7
' > combined-a.log 2>&1
rc_a=$?
```

展开：

```
> combined-a.log: FD 1 先指向文件
2>&1: FD 2 再复制 FD 1 当前端点
最终两个流都进入 combined-a.log
```

⑥ 验证组合顺序 B

```
bash -c '
  printf "%s\n" "DATA:alpha"
  printf "%s\n" "DATA:omega"
  printf "%s\n" "WARN:beta" >&2
  printf "%s\n" "WARN:gamma" >&2
  exit 7
' 2>&1 > combined-b.log
rc_b=$?
```

展开：

```
2>&1: FD 2 复制当时 FD 1 的终端端点
> combined-b.log: 随后只把 FD 1 改向文件
最终 stdout 进入文件，stderr 仍指向原终端
```

⑦ 分层显示最终证据

```
printf '%s\n' '=== stdout.log ==='
cat stdout.log
printf '%s\n' '=== stderr.log ==='
cat stderr.log
printf '%s\n' '=== combined-a.log ==='
cat combined-a.log
printf '%s\n' '=== combined-b.log ==='
cat combined-b.log
printf 'append_rc=%s rc_a=%s rc_b=%s\n' "$append_rc" "$rc_a" "$rc_b"
```

⑧ 结论边界

- `cat` 成功只能证明文件当前可读，不能替代之前命令状态；
- `combined-a.log` 中存在 WARN 证明 `stderr` 被合并到文件，不证明命令成功；
- `combined-b.log` 不含 WARN 是顺序语义的结果，不代表命令没有产生 `stderr`。

[参考解答] 经典任务二：保存逐阶段状态并使用 `pipefail`

① 查询并记录初始 `pipefail` 状态

```
if shopt -qo pipefail; then
    pipefail_was_on=true
else
    pipefail_was_on=false
fi
```

上述查询写法较长。考试现场若只需临时开启并在任务结束明确关闭，也可记录自己的操作边界。这里强调的是不要无意污染后续会话。

② 默认行为下执行并立即保存状态

```
set +o pipefail

printf '%s\n' 'payload' |
    bash -c 'cat; exit 9' |
    tee evidence.log
pipeline_rc=$? stage_rc=("${PIPESTATUS[@]}")
```

随后才展示：

```
printf 'pipeline_rc=%s\n' "$pipeline_rc"
printf 'stage_rc=%s\n' "${stage_rc[*]}"
```

按语义，数组从左到右对应三个阶段。默认汇总只取最后阶段，因此即使阶段 2 非零，`pipeline_rc` 仍可能为零。

③ 开启 `pipefail` 后重新执行

```
set -o pipefail

printf '%s\n' 'payload' |
    bash -c 'cat; exit 9' |
    tee evidence.log
pipefail_rc=$? pipefail_stage_rc=("${PIPESTATUS[@]}")
```

展示：

```
printf 'pipefail_rc=%s\n' "$pipefail_rc"
printf 'pipefail_stage_rc=%s\n' "${pipefail_stage_rc[*]}"
```

按 Bash 语义，汇总状态应反映最右侧非零阶段；逐阶段数组仍用于确认具体位置。

④ 恢复初始状态

```
if [ "$pipefail_was_on" = true ]; then
    set -o pipefail
else
    set +o pipefail
fi
```

⑤ 验证数据文件

```
printf '%s\n' '=== evidence.log ==='
cat evidence.log
```

⑥ 分层结论

evidence.log 中存在 payload
→ 证明 payload 到达 tee 且 tee 成功写入该文件

阶段 2 状态非零
→ 证明中间处理阶段按设计失败

默认 pipeline_rc 可能为零
→ 只反映最后阶段成功，不能代表全管道成功

pipefail_rc 为非零
→ 汇总规则暴露了最右侧非零阶段

⑦ 更简洁的考试型写法

若题目只要求在当前受控 Bash 中可靠判断管道，并允许任务后明确关闭：

```
set -o pipefail
pipeline | tee evidence.log
pipeline_rc=$? stage_rc("${PIPESTATUS[@]}")
set +o pipefail
```

仍需注意：关闭 pipefail 的命令必须在状态已经保存之后执行。

[本章收束] 用同一模型解释重定向、管道和证据

完成本章后，面对任何输入输出命令，不应先猜“该加哪个符号”，而应按固定顺序还原路由和证据：

1. 标出命令的 FD 0、1、2；
2. 从左到右应用重定向，记录每一步端点变化；
3. 对管道逐段确认“左 stdout → 右 stdin”，并单独追踪 stderr；
4. 判断数据在哪个位置仍可观察、保存或被丢弃；
5. 在任何新命令之前保存退出状态；
6. 对复杂管道同时保留汇总状态和 PIPESTATUS；
7. 写清楚每份证据能证明什么、不能证明什么。

⋮ {method-box} 工作方法：先画路由，再读结果。只要能把每个阶段写成“FD 编号 → 当前端点”，>、2>&1、管道和 tee 就不再是孤立符号；只要把数据证据和退出状态分开，管道“看起来成功”的误判也会明显减少。⋮

主要判断表

需求或现象	首选形式或下一条证据	证明边界
从文件提供 stdin	command < input	只证明 FD 0 来源
覆盖或追加 stdout	> / >>	覆盖可能先截断；追加不自动去重
单独保存或合并 stderr	2> error / > all 2>&1	仍需单独保存退出状态
只把 stdout 送入下游	A B	stderr 默认旁路
把两个流送入下游	A 2>&1 B	合流后下游无法区分原始通道
保存并继续传递	A tee [-a] file B	文件只证明数据到达 tee 位置
保存汇总与逐阶段状态	rc=\$? + PIPESTATUS	必须在任何后续简单命令前复制

需求或现象	首选形式或下一条证据	证明边界
暴露中间失败	<code>set -o pipefail</code>	仍不能指出全部失败阶段
多行 stdin	<code><<EOF</code> / <code><<'EOF'</code>	delimiter 引用决定正文是否展开

向下一章交接

本章已经回答数据从哪里进入命令、stdout/stderr 去哪里、重定向何时生效、管道如何连接进程，以及怎样判断中间阶段失败。下一章《本地帮助、命令发现与软件能力查询》将说明怎样定位 `printf`、`cat`、`tee` 和 `bash` 的真实入口、synopsis、手册与软件包来源。第 05 章再展开 `grep`、`sed`、`awk` 等管道消费者，第 13 章才完整处理 Journal、rsyslog 和日志轮转。

本地帮助、命令发现与软件能力查询

从一个陌生名称开始，沿着当前 Shell、帮助索引、RPM 数据库与 DNF 元数据建立可复核的本机证据链。

对象模型操作语义证据边界诊断经典任务

大字号阅读版

本章阅读导航 先抓住一条主线：****先确认当前系统把名称解释成什么，再决定去哪里查语法；已有路径查询本地所有权，缺失路径查询仓库提供能力。****

01确认问题层名称、对象、位置、所有权还是提供能力

02确认当前对象alias、function、builtin 或外部命令

03选择帮助源help、--help、man 或 info

04发现未知能力whatis、apropos 与 man 索引

05定位软件来源rpm -qf 与 dnf provides

06限制证据结论说明能证明什么、不能证明什么

专题地图 - ****知识专题**** 名称、对象、位置和软件来源为何必须分层 - ****操作专题**** 让当前 Bash 说明一个名称会怎样解析 - ****操作专题**** 按对象选择 `help`、`--help`、`man` 与 `info` - ****操作专题**** 从 man section 和标准标题提取操作依据 - ****操作专题**** 不知道命名时按关键字发现本地能力 - ****操作专题**** 用 `rpm -qf` 证明已安装文件归属 - ****操作专题**** 用 `dnf provides` 查询缺失路径的候选提供者 - ****诊断专题**** 让互相矛盾的查询结果回到各自证据层 - ****经典任务**** 无互联网、禁止安装时完成一份本地调查记录

阅读时持续回答 1. 我现在调查的是名称、文件还是软件包能力？ 2. 当前 Bash 会优先执行哪个对象？ 3. 这个对象的第一帮助入口是什么？ 4. 打开的 man page 属于哪个 section？ 5. 查询结果来自当前会话、本机数据库还是仓库元数据？ 6. 空结果意味着对象不存在，还是帮助/索引/仓库不可见？ 7. 这条证据能证明到哪一层？ 8. 下一条最有区分度的证据是什么？

章节边界：RPM 安装、升级、删除和验证事务留给第 16 章；DNF 仓库、模块流和包组留给第 17 章；互联网搜索不作为第一证据。

第 03 章 · 正文

面对陌生命令，最常见的误判不是“少记了一个参数”，而是还没有确认调查对象，就直接复制一个看似相关的答案。相同名称在当前 Bash 中可能是 alias、function、builtin，也可能是磁盘中的外部程序：一个路径可以由本机已安装 RPM 包拥有，也可以只是管理员手工创建；仓库中某个软件包能够提供一个文件，也不代表它已经安装，更不代表当前 Shell 已经能够调用它。

本章以“名称解析 → 帮助路由 → 能力发现 → 软件来源 → 证据边界”为主线。它不把十几个查询工具堆成命令清单，而是训练你从当前证据决定下一条查询：先用 Bash 自己确认名称如何解析，再为该对象选择合适的本地帮助；不知道准确名称时，从 man 索引按功能查找；已经存在的路径交给 RPM 数据库；缺失命令或路径交给 DNF 可见的软件包元数据。每一步都必须说明结论的观察范围。

后续第 04 章会完整解释路径、文件类型和链接；本章只把“路径”当作查询键。第 16、17 章会处理软件事务和仓库配置；本章只做只读调查，不通过安装或改仓库来制造答案。

概念

Shell 名称解析 是当前 Bash 把命令位置上的名称映射为可执行对象的过程。解析结果受 alias、function、builtin、`PATH`、命令哈希缓存和当前会话状态影响，因此“系统里有这个文件”与“当前 Shell 会执行这个文件”不是同一个结论。观察入口是 `type` 与 `command -V/-v`。

概念

命令类型 描述名称最终指向的对象类别：alias 是解析阶段的文本替换，function 是当前 Shell 中保存的命令体，builtin 由 Bash 进程内部实现，外部命令则对应文件系统中的程序或脚本。同名对象可以并存，优先级和帮助入口也不同。

概念

当前环境证据 是“在这个用户、这个 Bash 会话、这个 `PATH` 与这些启动配置下”的观察结果。更换用户、非交互 Shell、登录方式或环境变量后，`type`、`command -v` 甚至可用帮助文档都可能变化。结论必须连同作用范围记录，不能写成永远不变的系统事实。

概念

帮助入口层次 反映文档由谁维护以及信息深度。Bash builtin 优先查 `help`；外部程序常用 `--help` 快速确认形式；完整本地参考进入 `man`；部分 GNU 工具有节点式 `info` 文档。入口之间是分工关系，不是“哪个命令更高级”。

概念

man section 与关键字索引 解决两个不同问题。section 用来区分同名但对象不同的页面，例如命令 `passwd(1)` 与文件格式 `passwd(5)`；`whatis` 和 `apropos` 使用手册索引，在已知名称或只知道功能时发现页面。索引空结果不自动等于系统没有该能力。

概念

已安装文件归属 是本地 RPM 数据库记录的路径与已安装包之间的关系。`rpm -qf /path` 证明的是“这台主机的 RPM 数据库认为谁拥有该路径”，不是仓库中唯一的提供者，也不自动证明文件内容没有被修改。

概念

仓库能力查询 是从 DNF 当前可见的软件包集合中查找能够提供某个路径或 capability 的候选。`dnf provides` 的结果受启用仓库、架构和元数据状态影响；它只能建立候选提供关系，不能替代安装状态、Shell 解析和最终功能验证。

操作语义速查

操作语义先建立关键入口地图。下面每组只给出最常用形式；完整判断、输出和诊断在后续专题展开。

`type` / `command` **SYNOPSIS**

```
type [--aftP] name [name ...]
command [-pVv] command [argument ...]
```

从当前 Bash 的解析环境确认一个名称是 alias、function、builtin 还是外部命令。重要参数 / 形式：`type name`：显示当前优先解析结果。`type -a name`：尽可能列出当前环境可见的所有同名入口。`command -V name`：输出适合人工阅读的详细解析说明。`command -v name`：输出简洁表示并提供可用于脚本判断的退出状态；结果不一定是路径。

`help` / `COMMAND --help` **SYNOPSIS**

```
help [-dms] [pattern ...]
COMMAND --help
```

`help` 查询当前 Bash 版本的 builtin 和语法主题；`--help` 是外部程序常见的快速帮助约定。 **重要参数 / 形式** `help builtin`：显示 builtin 的完整 Bash 帮助。
`help -s builtin`：只看简短使用形式。 `COMMAND --help`：快速确认 Usage、常用选项和参数形式；并非所有程序都实现完全相同的接口。

`man` / `whatis` / `apropos` **SYNOPSIS**

```
man [section] page
whatis name ...
apropos keyword ...
```

`man` 阅读本地手册页；`whatis` 查已知名称的一行描述；`apropos` 按名称与描述中的关键字发现页面。 **重要参数 / 形式** `man 5 passwd`：显式选择 section 5，读取文件格式而不是命令页。 `man -f name`：与 `whatis name` 对应。 `man -k keyword`：与 `apropos keyword` 对应。

`info` **SYNOPSIS**

```
info [option ...] [menu-item ...]
```

阅读以节点和菜单组织的 GNU 长文档，适合跨主题说明和教程式材料。 **重要参数 / 形式** `info topic`：打开指定主题或菜单项。 `n` / `p` / `u`：前往下一节点、上一节点或上一级。 `q`：退出。最小安装不保证每个工具都有 Info 文档。

`whereis` **SYNOPSIS**

```
whereis [options] name ...
```

在已知的标准位置中查找二进制、源代码和手册线索；它不模拟当前 Bash 的完整名称解析。 **重要参数 / 形式** `whereis name`：同时显示可见的二进制、源码和手册位置线索。 `-b`：只查二进制。 `-m`：只查手册。

`rpm -qf` **SYNOPSIS**

```
rpm -qf FILE [FILE ...]
```

查询本机 RPM 数据库，确定已有路由哪个已安装软件包拥有。 **重要参数 / 形式** `-q`：进入查询模式。 `-f FILE`：以文件路径选择拥有它的已安装包。 `rpm -ql PACKAGE`：反向列出某个已安装包记录的文件清单；软件验证事务留给第 16 章。

`dnf provides` **SYNOPSIS**

```
dnf provides PROVIDE-SPEC [PROVIDE-SPEC ...]
```

从 DNF 当前可见的软件包数据中查询能够提供某个路径或 capability 的候选。 **重要参数 / 形式** `dnf provides /usr/sbin/name`：已知准确路径时进行精确查询。
`dnf provides \*/name`：只知道 basename 时查询路径尾部；引号防止当前 Shell 提前展开 `\*`。 结果来源：记录候选包、架构、仓库/安装来源与匹配路径；命中不等于已经安装。

知识专题 名称、对象、位置和软件来源为何必须分层

一个命令背后至少存在四个问题：当前 Shell 会执行什么、磁盘上有哪些候选位置、本机哪个包拥有目标路径、仓库中哪个包能够提供目标。四个问题分别由不同数据源回答。真正稳定的调查不是追求所有命令输出一致，而是让每条结果回到自己的证据层。

① 知识点 当前 Shell 可调用对象不只有外部文件

对象	本质	常见作用范围	身份入口	第一帮助入口
alias	解析阶段的名称替换	当前 Shell，常来自启动文件	type	先看展开，再查真实命令
function	Shell 中保存的命令体	当前 Shell 或导出的函数环境	type、declare -f	读函数定义，再查内部对象
builtin	Bash 进程内部实现	当前 Bash	type、command -V	help
外部命令	文件系统中的程序或脚本	PATH、权限和文件状态共同决定	type、command -v	--help、man、info

同名对象可以并存。test 可能既是 builtin，也有 /usr/bin/test；alias 可能给外部命令预置参数；function 可能完全覆盖同名文件。只知道文件路径，仍不能直接推断当前命令位置会调用它。

② 知识点 四层证据可以不同，而且这种不同通常是正常的

```
当前 Bash 会执行什么
→ type / command -V / command -v

标准位置中有哪些相关二进制或手册线索
→ whereis; which 只作外部路径线索

本机某个路径由哪个已安装包拥有
→ rpm -qf /absolute/path

DNF 当前可见的软件包中谁能提供该路径
→ dnf provides /path 或 '*/basename'
```

例如，`type` 可以显示 `alias`，`whereis` 同时显示 `/usr/bin/name` 与手册路径，`rpm -qf` 证明 `/usr/bin/name` 属于已安装包，`dnf provides` 又列出多个不同架构或仓库版本。它们回答的问题不同，不能选一条覆盖全部事实。

③ 知识点 证据有会话、主机和元数据边界

`type` 与 `command` 是当前 Bash 会话证据。切换用户、改变 `PATH`、进入非交互 Shell 或载入不同启动文件后，解析结果可能变化。Bash 还可能缓存外部命令路径；文件移动或软件更新后，必要时用 `hash -r` 清理当前会话缓存再复查。

`rpm -qf` 依赖本机 RPM 数据库；`dnf provides` 依赖 DNF 当前能看到的软件包集合。仓库启停、订阅、架构和元数据更新时间都会影响候选。调查记录应保留“在哪台主机、哪个用户、哪个会话、哪个仓库范围”这些上下文。

④ 边界 帮助文本是操作依据，不是目标状态证明

在 `man page` 中找到参数，只能证明写法有本地依据；`command -v` 成功只能证明当前 Shell 能解析名称；`dnf provides` 命中只能证明存在候选提供关系。它们都不能替代后续业务对象上的当前状态、持久状态和功能验收。

判断口诀：名称先问 Bash；位置只作线索；已有路径问 RPM；缺失路径问 DNF；每条结论都写明观察范围。

操作专题 让当前 Bash 说明一个名称会怎样解析

当现象是“这个名称执行的不是我以为的程序”或“脚本中找不到命令”，最有区分度的第一条证据是当前 Shell 自己的解析结果。`type` 适合人工调查，`command -V` 给详细说明，`command -v` 给简洁结果和退出状态；`which` 与 `whereis` 只能补充位置线索。

① 查询 用 `type` 建立当前优先对象与同名候选

```
type name
type -a name
type -t name
```

- `type name`：报告当前优先解析结果；
- `type -a name`：尽可能列出 `alias`、`function`、`builtin` 和外部路径等所有可见候选；
- `type -t name`：只输出稳定类别词，如 `alias`、`function`、`builtin`、`file`，适合机器判断。

输出文案会受 Bash 版本与 `locale` 影响。复习时应识别对象类别，不肯完整英文句子。若 `type -a` 同时显示 `builtin` 与外部路径，当前优先项由普通 `type name` 证明。

② 查询 用 `command -V/-v` 区分人工说明与简洁判断

```
command -V name
command -v name
```

`command -V` 输出较详细、适合人工阅读的说明。`command -v` 返回简洁表示：外部命令常返回路径，`builtin` 常只返回名称，`alias` 可能返回定义文本。因此不能把所有成功结果当成文件路径。

脚本只需要判断当前环境能否解析名称时，可使用：

```
if command -v name >/dev/null 2>&1; then
    printf '%s\n' 'name is resolvable in this shell'
fi
```

这只证明“可解析”，不证明程序具备权限、依赖、配置或业务功能。

③ 边界 `which` 只能降级为外部路径线索

`which name` 通常按当前 `PATH` 寻找外部可执行文件，但实现、`alias` 包装和初始化环境可能不同。若 `type name` 显示 `alias`，而 `which name` 显示 `/usr/bin/name`，应得出：

```
当前 Bash 优先使用 alias
/usr/bin/name 是同名外部候选
```

不要为了让两条输出一致而删除 `alias`。先确认题目或工单要调查的是当前行为，还是外部文件本身。

④ 查询用 `whereis` 补充二进制、源码和手册位置

```
whereis name
whereis -b name
whereis -m name
```

`whereis` 根据其已知的常用搜索目录以及显式指定的范围查找；`-b` 只看二进制，`-m` 只看手册。它不读取当前 Shell 的 `alias`、`function` 和 `builtin` 优先级，也不证明路径由哪个包拥有。

⑤ 诊断 更新后仍指向旧路径时先区分缓存与对象遮蔽

先重新取证：

```
type -a name
command -V name
```

若结果证明只是 Bash 的外部命令哈希缓存，再执行：

```
hash -r
```

随后用相同查询复查。`hash -r` 不会修改 `PATH`、`alias`、`function` 或软件安装状态；若文件确实不存在，应转到软件包层继续调查。

验证：``type`/'command`` 能证明当前 Bash 怎样解析。不能证明：文件归属、仓库提供者、程序运行成功或业务终态。

操作专题 按对象选择 `help`、`--help`、`man` 与 `info`

帮助入口的选择由对象类型决定。最常见的低效路径是：对 `builtin` 机械尝试 `--help`，对外部程序只读一屏短帮助，或在命令名未知时逐个猜测。稳定方法是先用 `type` 确认对象，再进入由对象维护者提供的本地文档。

① 操作 Bash builtin 优先使用 `help`

```
type cd
help cd
help -s cd
help -d cd
```

`help cd` 读取当前 Bash 版本的完整 builtin 说明；`-s` 只看简短语法，`-d` 只看短描述。`cd`、`read`、`export`、`type`、`command` 等应优先从这里开始。

同名 builtin 与外部命令并存时，帮助源也必须分开：

```
help test
man 1 test
```

前者解释 Bash builtin，后者解释外部 `test(1)`；功能相似不代表是同一个对象。

② 操作 `COMMAND --help` 用于快速确认常用形式

```
command-name --help
```

短帮助通常适合在几十秒内确认 Usage、常用选项、参数是否带值和子命令入口。它是广泛使用的约定，不是每个程序都完全一致的强制接口；有些程序使用 `subcommand --help`，有些把详细文件、环境变量和退出状态留在 `man` 或 `Info` 中。

快速确认常用写法

→ `--help`

需要完整语义、文件、退出状态和关联主题

→ `man`

需要节点式长文档或教程式说明

→ `info`

③ 操作 `man` 是完整本地参考的主入口

`man page`

`man section page`

手册页随相关软件包或文档包安装。`man page` 查不到时，先检查对象是否为 builtin、主题名称和 section 是否正确，再考虑手册是否未安装；不能立即推出系统没有该命令或能力。

分页器中的高频导航：

键	用途
<code>/pattern</code>	向后搜索
<code>n / N</code>	下一个 / 上一个匹配
<code>g / G</code>	页首 / 页尾
<code>q</code>	退出

④ 操作 `info` 是可选的节点式补充

`info topic`

Info 以节点、菜单和交叉引用组织内容。`n`、`p`、`u` 分别进入下一节点、上一节点和上一级，`Enter` 跟随菜单或链接。最小化系统不保证所有 Info 文档存在；缺少节点时回到 `--help` 和 `man`，而不是把它解释为命令不存在。

⑤ 流程 建立可重复的本地帮助路由

`type / command -V`

|

└ `alias` → 看展开内容，再查真实命令

└ `function` → 查看函数定义，再查内部调用对象

└ `builtin` → `help name`

└ `external` → `name --help` → `man [section] name` → `info name`

只知道功能，不知道名称

└ `apropos keyword / man -k keyword`

互联网文档可能更新更快，但考试环境可能无外网，且本地文档与当前安装版本直接对应，所以本地帮助应是第一证据。

验证：找到与当前对象匹配的帮助入口，并能从中定位语法。不能证明：命令已安装到所有环境、配置正确或操作终态成立。

操作专题 从 `man section` 和标准标题提取操作依据

打开 `man page` 只是起点。真正要训练的是快速回答四个问题：页面描述的对象是谁、命令骨架怎样读、关键参数改变什么、相关文件和下一张手册在哪里。
`section` 是对象分类，不是难度等级。

① 知识点 用 `section` 区分同名对象

Section	主要对象	RHCSA 常见用途
1	用户命令与 Shell 命令	普通命令调用形式
2	系统调用	深度排错或编程接口
3	库函数	程序库接口
4	特殊文件与设备	设备和内核接口文件
5	文件格式与配置文件	字段、配置语法、文件结构

Section	主要对象	RHCSA 常见用途
6	游戏	本考试很少使用
7	约定、协议与杂项	信号、协议、正则等整体主题
8	系统管理命令	管理员常用工具
9	内核例程	内核开发接口

引用手册时常写作 `passwd(1)`、`passwd(5)`。括号中的数字是 section，不是版本号。

② 操作 显式 section 消除歧义

```
man passwd
man 1 passwd
man 5 passwd
```

默认 `man passwd` 按系统配置的 section 顺序选择匹配页，通常先进入命令页。任务要求读取 `/etc/passwd` 字段时，必须显式进入 `man 5 passwd`。同理，文件格式通常看 section 5，管理命令常见于 section 8，但最终仍以页面对象为准。

③ 知识点 正确读取 SYNOPSIS 的形式符号

SYNOPSIS 是形式语法，不是必须原样复制的一条固定命令：

- 固定命令名和选项按原样输入；
- FILE、NAME 等占位词替换成实际值；
- [...] 表示可选部分；
- ... 表示前一项可以重复；
- A | B 表示替代形式；
- 多行 Synopsis 往往表示多种调用模式，不应拼成一行执行。

读完后应能写出目标命令骨架，并解释哪些部分是固定语法、哪些需要替换。

④ 查询 用标准标题定位所需证据

标题	主要问题
NAME	页面对象及一句话用途是什么
SYNOPSIS	命令或接口的形式怎样写
DESCRIPTION	对象机制和默认行为是什么
OPTIONS	参数如何改变语义、是否带值
EXIT STATUS	返回码怎样解释
ENVIRONMENT	哪些环境变量影响行为
FILES	相关配置、数据或状态路径在哪里
EXAMPLES	维护者给出的典型组合是什么
SEE ALSO	下一张关联手册是什么

页面不一定包含全部标题。查参数先搜 OPTIONS 或具体选项，查配置路径看 FILES，查同名对象与关联页看 SEE ALSO。

⑤ 验证 把帮助内容转成可核对的命令计划

在执行真实变更前、至少写清：

```
作用对象
→ 命令骨架
→ 关键参数及取值
→ 相关文件或前置状态
→ 操作后验证入口
→ 帮助页没有证明的部分
```

例如 `man page` 说明 `-f FILE` 选择文件，只能证明参数语义；目标文件存在、权限允许、结果内容正确仍需要后续证据。

man 快读：先看 NAME 确认对象，再看 SYNOPSIS 还原骨架；OPTIONS 找参数，FILES 找路径，SEE ALSO 找下一跳；同名页面显式指定 section。

操作专题 不知道命令名时按关键字发现本地能力

有时题目只描述“修改密码过期时间”或“查找压缩归档工具”，并没有给出命令名。此时不应随机尝试名字，而应把任务语言压缩成稳定名词，从 `man` 索引发现候选主题，再打开具体页面确认。

① 查询 `whatis` 适合已知准确名称

```
whatis passwd
man -f passwd
```

`whatis` 从索引中返回与名称匹配的一行描述，通常包含主题、section 和短说明。它适合确认同名页面有哪些，不适合用模糊的自然语言问题搜索全部内容。

② 查询 `apropos` 与 `man -k` 按关键字发现主题

```
apropos password
man -k password
```

输出通常由三部分组成：

```
主题名称 (section) - 一行描述
```

筛选时先看 section，再看描述是否对应任务对象。例如“password aging”可尝试 `password`、`aging`、`expiry` 等稳定名词；过窄的整句搜索反而容易漏掉页面。

③ 操作 从候选列表回到具体 man page

```
apropos password
man 1 chage
man 5 login.defs
```

发现只是候选阶段。必须打开页面确认 `NAME`、`SYNOPSIS`、相关文件和参数边界，不能只凭一行描述执行命令。

④ 诊断 空结果先区分关键词、文档与索引

```
症状: apropos 没有结果
|
|— 关键词过窄或语言不匹配
|— 本机没有对应 man page
|— whatis 索引缺失或陈旧
```

下一条证据应是：

```
whatis known-command
man known-command
```

若已知页面能打开但索引查询异常，再由有权限的管理员检查：

```
mandb
```

然后重复原查询。只有重建索引后同一关键字恢复，才足以把问题归到索引层。`mandb` 维护手册索引，不安装目标业务软件。

⑤ 边界 索引搜索不是全文互联网搜索

`apropos` 主要匹配手册名称和短描述，结果受本地语言、已安装手册和索引影响。未命中不代表整个 Linux 生态没有该工具；命中也不代表命令已在当前 `PATH` 可调用。后续仍需 `type`、`man` 和软件包证据。

验证：从关键词得到候选主题，并通过具体 man page 复核。不能证明：候选命令已安装、当前 Shell 可调用或任务已经完成。

操作专题 用 `rpm -qf` 证明已安装文件归属

当问题是“本机 RPM 数据库把这个路径记录给哪个已安装包”，首选 `rpm -qf`，而不是仓库搜索。查询键应使用准确路径；该查询读取包数据库，即使文件后来缺失，数据库中的所有权记录仍可能存在，因此还要把“数据库所有权”与“文件当前存在”分开验证。

① 操作 以绝对路径查询本机所有者

```
rpm -qf /usr/bin/chage
```

合格结论应写成：

本机 RPM 数据库记录 `/usr/bin/chage` 由查询结果中的已安装包拥有。

不要只写“`chage` 来自某包”，因为同名文件可能位于不同目录，且其他主机的安装状态可能不同。

② 输出 区分三种状态

现象	允许判断	下一条证据
返回包名/NEVRA	本机 RPM 数据库存在该路径的所有权记录	用 <code>rpm -ql package</code> 反向核对；另查路径是否实际存在
提示文件不属于任何包	该输入路径没有匹配的本地 RPM 所有权记录	核对路径拼写；若文件存在，判断是否为手工或运行时内容
路径当前不存在，但查询仍返回包名	数据库仍记录该包应拥有此路径	第 16 章再做包完整性与恢复验证

“无所有者”不自动等于恶意文件，也不自动等于应删除。管理员脚本、编译安装内容和运行时生成文件都可能不属于 RPM 包；反过来，数据库有所有权记录也不证明文件此刻仍存在。

③ 查询用 `rpm -ql` 做反向核对

```
rpm -ql package-name
```

如果 `rpm -qf` 返回包名，可以从包到文件清单反向确认路径。此处仍是只读查询；包签名、完整性验证、安装、升级和删除留给第 16 章。

④ 边界 所有权不等于内容完整和命令可用

`rpm -qf` 不证明：

- 文件当前实际存在；
- 文件内容与软件包原始内容一致；
- 当前用户有执行权限；
- 路径位于当前 `PATH`；
- 程序依赖与配置正常；
- 仓库中只有这一个提供者。

需要确认当前 Shell 是否能调用时，回到 `type`；需要查询缺失路径的候选包时，使用 `dnf provides`。

本地归属：准确路径 → ``rpm -qf`` → 记录数据库所有权 → ``rpm -ql`` 反向核对；文件是否存在、是否完整和是否可用仍要另行验证。

操作专题 用 `dnf provides` 查询缺失路径的候选提供者

命令不存在时，先用 `type` 或 `command -v` 证明当前 Shell 无法解析；随后需要从可靠文档或路径约定得到尽可能准确的目标路径，再让 DNF 查询谁能够提供它。`dnf provides` 是能力发现，不是安装动作。

① 操作 已知准确路径时优先精确查询

```
dnf provides /usr/sbin/semanage
```

精确路径能减少误匹配。输出中至少记录：候选包名、版本/发行号、架构、来源仓库或安装状态，以及实际匹配的路径或 capability。

② 操作 只知道 `basename` 时使用受保护的路径模式

```
dnf provides '*/semanage'
```

引号非常重要：它把 `*` 原样交给 DNF，而不是让当前 Shell 先按工作目录文件名展开。模式越宽，候选越多；拿到结果后应检查真实匹配路径，而不是只看第一个包名。

③ 输出 多个候选时按目标环境筛选

多个结果可能来自不同架构、版本、仓库或已安装/可用状态。筛选顺序：


```
匹配路径是否正确
→ 架构是否符合主机
→ 来源仓库是否为当前允许范围
→ 版本与模块/系统约束是否匹配
→ 是否已经安装（若输出明确显示）
```

本章不执行安装。真正选择、安装和事务验收转第 16、17 章。

④ 诊断 无结果时不要立刻猜包名

```
症状: dnf provides 无候选
|
├─ 文件名或路径拼写错误
├─ 查询模式过窄或过宽
├─ 目标文件属于未启用仓库
├─ 元数据不可用或陈旧
└─ 当前架构/版本确实没有提供者
```

下一条最有区分度的证据依次是：确认准确路径、调整受保护模式、查看 DNF 是否能读取当前仓库元数据。仓库创建、启停和修复属于第 17 章，本章只记录阻断位置。

⑤ 验证 把“可提供”与“已可调用”分开

即使以后完成安装，仍需分层复查：

```
dnf/rpm 安装状态
→ rpm -qf 实际路径的本地所有权
→ type / command -v 当前 Shell 解析
→ command --help / man 语法依据
→ 真实任务的功能验证
```

`dnf provides` 只完成第一步之前的候选发现。

验证：当前 DNF 可见数据中存在与目标路径匹配的候选提供者。不能证明：候选已安装、命令在 PATH 中、配置正确或业务功能成立。

诊断专题 让互相矛盾的查询结果回到各自证据层

本章的高频故障并不是某个命令“坏了”，而是把不同层的输出当作同一种事实。诊断时先写清症状和当前证据，再选择一条能够排除一类假设的查询；最小修复后必须回到原层复查。

① 诊断 `command not found`

```
症状
→ 当前 Shell 无法解析名称

当前证据
→ type name
→ command -v name

假设
├─ 拼写错误
├─ alias/function 只在另一个会话存在
├─ 外部文件不在 PATH
├─ Bash 仍缓存旧路径
└─ 目标软件尚未安装

下一条证据
→ type -a / command -V
→ 确认预期路径
→ 必要时 hash -r
→ dnf provides 准确路径或受保护模式
```

最小处理由证据决定。不要在未确认包和仓库范围前直接安装相似名称的软件。

② 诊断 `type` 与 `which` 不一致

```
若 type 显示 alias/function/builtin
而 which 显示 /usr/bin/name
```

下一条证据是 `type -a name`。它能证明当前优先对象及同名外部候选。结论应写“当前 Bash 先使用前者的，外部文件仍存在”，而不是挑一条输出宣布另一条错误。

③ 诊断 命令可执行但 `man name` 查不到

当前证据

→ type name

假设

└─ 对象是 builtin, 应使用 help

└─ 手册主题名不同

└─ 需要显式 section

└─ 本地手册未安装

下一条证据

→ help name

→ name --help

→ whatis / apropos

→ man section topic

先确认对象和主题，再决定是否在文档缺口。

④ 诊断 `apropos` 空结果

先用已知页面验证两层：

man ls

whatis ls

- `man ls` 也失败：优先调查手册是否安装；
- `man ls` 成功但 `whatis ls` 异常：索引层更可疑；
- 已知查询正常：调整关键字、语言和 section 预期。

需要重建索引时执行 `mandb`，再重复原查询。不能只因空结果就宣布系统没有相关能力。

⑤ 诊断 `rpm -qf` 无所有者，但 `dnf provides` 有候选

这两条结果完全可以同时成立：本机路径可能未安装、路径不存在、由管理员手工创建，或查询路径与候选匹配路径不同；仓库仍可声明某包能够提供它。

下一条证据：

确认输入路径是否真实存在

→ 核对 rpm 查询使用的绝对路径

→ 记录 dnf 实际匹配路径

→ 不把仓库候选倒推成本机所有权

⑥ 安全边界 找到强制选项不等于应当使用

本地帮助可能显示 `--force`、`--nodeps`、`--allowerase` 等绕过机制。找到它们只能证明工具支持这些形式，不能证明题目允许破坏依赖、覆盖文件或扩大变更范围。本章默认只读调查；软件事务和高风险修改必须回到相应章节的对象、约束与验收链。

⑦ 工作迁移 工单中保留可重建判断的最小证据

建议记录：

原始症状、名称或路径

当前用户、Shell 和 PATH 范围

关键 type / command 结果

实际帮助主题与 section

rpm 所有权或 dnf 候选提供者

未证明的层次与后续功能验证入口

这使同事或自动化 Agent 可以复核判断，而不是只看到一句“安装某包即可”。

诊断链：症状 → 当前证据 → 分层假设 → 一条高区分度查询 → 最小修复 → 回到原层复查。

经典任务 在无互联网、禁止安装软件的前提下完成本地能力调查

环境

你以普通用户登录一台 RHEL 9 主机，当前交互 Shell 为 Bash。本机能够读取已安装 man page、RPM 数据库和 DNF 当前可见的软件包元数据。任务期间不允许访问互联网，不允许安装、删除或升级软件包，也不允许修改仓库配置。

当前状态

管理员准备进行账号维护，但遇到以下未知项：

1. 不确定 `test` 在当前 Shell 中是 builtin、外部命令，还是同时存在多个候选；
2. 不知道 `cd` 的准确语法应从哪里查询；
3. 需要阅读 `/etc/passwd` 的文件格式，而不是执行修改密码的 `passwd` 命令；
4. 只知道要调查“password aging”，不知道准确命令名；
5. 需要证明 `/usr/bin/chage` 由哪个已安装软件包拥有；
6. 当前无法调用 `semanage`，需要确定 DNF 当前可见数据中哪个包能够提供该命令文件。

目标终态

提交一份调查记录。每个对象必须写明：

- 使用的查询命令；
- 为什么选择该入口；
- 从输出中提取的字段或结论；
- 该证据不能证明什么；
- 查询无结果时，下一条最有区分度的证据。

限制条件

- 不把 `which` 作为最终命令类型证据；
- 不凭记忆直接写 `semanage` 的包名；
- 不写死具体包版本、架构或仓库名称；
- `apropos` 无结果时必须调查关键词、man page 与索引；
- `dnf provides` 命中后不得声称软件已安装；
- 不使用互联网搜索作为第一证据；
- 不通过改 `PATH`、删除 `alias` 或安装软件来让输出“看起来一致”。

验收矩阵

评分对象	必须出现的证据	合格判断	明确边界
<code>test</code> 对象类型	<code>type / type -a</code> ，辅以 <code>command -V/-v</code>	能区分当前优先对象与同名候选	不证明外部文件归属
<code>cd</code> 帮助	<code>type cd</code> 、 <code>help cd</code>	明确 <code>cd</code> 是 Bash builtin	不把结果当外部程序路径
<code>/etc/passwd</code> 格式	<code>man 5 passwd</code>	明确 section 5 为文件格式	不证明当前文件内容正确
password aging 发现	<code>apropos</code> 或 <code>man -k</code>	从名称、section 与描述识别候选	仍需打开具体页面
<code>/usr/bin/chage</code> 归属	<code>rpm -qf /usr/bin/chage</code>	只声明本机已安装包所有权	不证明文件完整或仓库唯一
<code>semanage</code> 提供者	<code>dnf provides '*/semanage'</code> 或准确路径查询	记录候选包、架构、来源与匹配路径	不声明已安装或可调用
失败诊断	对空结果给出下一条证据	不把空结果直接解释为能力不存在	不越权修改仓库或安装软件

参考解答 先确认对象，再沿证据层推进

以下是调查骨架。本章未连接受控 RHEL 9 虚拟机，因此不编造固定版本、locale、仓库或完整输出；实际环境应记录主机返回的真实内容。

① 确认 `test` 的当前解析对象与所有候选

```
type test
type -a test
command -V test
command -v test
```

参数选择：

- 普通 `type` 证明当前优先对象；
- `-a` 展开同名候选，便于识别 builtin 与外部文件并存；
- `command -V` 提供人工可读说明；
- `command -v` 提供简洁结果和退出状态，但 builtin 可能只返回名称。

分层验收：只声明当前 Bash 如何解析。若要证明外部候选的包归属，把 `type -a` 返回的真实路径交给 `rpm -qf`。

② 为 builtin `cd` 选择帮助入口

```
type cd
help cd
help -s cd
```

`type` 先确认对象，`help cd` 再读取当前 Bash 的完整说明；只需快速看形式时使用 `help -s cd`。不以 `cd --help` 代替 builtin 的标准帮助入口。

③ 显式选择 `/etc/passwd` 的文件格式页

```
whatis passwd
man 5 passwd
```

`whatis` 用来观察同名主题和 section，`man 5 passwd` 明确进入文件格式页。阅读时至少定位：

```
NAME          页面对象
DESCRIPTION   文件用途与字段
FILES         相关路径
SEE ALSO      passwd(1)、shadow(5) 等下一跳
```

若只执行 `man passwd`，必须额外证明实际打开的是 section 5，否则不满足题目要求。

④ 按功能描述发现 password aging 相关主题

```
apropos password
man -k password
```

两条是同一类索引查询入口。根据输出的主题名称、section 与描述筛选候选，再打开具体页面，例如：

```
man 1 chage
```

若结果过多，换用 `aging`、`expiry` 等稳定名词；若完全无结果，先检查已知页面与索引：

```
man passwd
whatis passwd
```

确认手册可读而索引异常后，才由有限用户考虑：

```
sudo mandb
apropos password
```

修复后必须重复原查询，才能证明索引层已恢复。

⑤ 查询 `/usr/bin/chage` 的本地 RPM 所有权

```
rpm -qf /usr/bin/chage
```

记录真实返回包。结论必须限定为：

本机 RPM 数据库记录该已安装包拥有 `/usr/bin/chage`。

需要反向核对时：

```
rpm -ql package-name
```

这仍不证明文件当前存在、内容未修改、当前用户可执行或仓库中只有该提供者。

⑥ 查询 `semanage` 的候选提供者

先证明当前 Shell 无法解析：

```
type semanage
command ~v semanage
```

只知道 `basename` 时：

```
dnf provides '*/semanage'
```

若已经从可靠文档得到准确路径，则优先：

```
dnf provides /usr/sbin/semanage
```

从实际输出记录候选包、版本/发行号、架构、来源与匹配路径。不要凭记忆写死包名；不要把候选关系写成已安装状态。

无结果时按顺序调查：拼写与路径、模式是否被引号保护、DNF 元数据是否可读、目标仓库是否在当前允许范围。仓库修复和安装留给第 16、17 章。

⑦ 汇总证据矩阵

对象	证据层	允许声明	不能扩大为
<code>test</code>	当前 Bash 解析	当前优先对象与同名候选	软件包所有权或功能健康
<code>cd</code>	Bash builtin 帮助	当前 Bash 的语法	外部程序安装状态
<code>passwd(5)</code>	本地文件格式手册	字段语义与关联路径	<code>/etc/passwd</code> 当前内容无误
<code>apropos password</code>	man 索引	本地相关候选主题	命令已安装或已执行
<code>rpm -qf</code>	本机 RPM 数据库	路径的已安装包所有权记录	文件当前存在、内容完整或仓库唯一提供者
<code>dnf provides</code>	DNF 当前可见软件包数据	候选提供关系	已安装、可调用或功能完成

⑧ 典型错误与修正

错误	为什么不合格	最小修正
只执行 <code>which test</code>	忽略 builtin、function 和 alias	用 <code>type / type -a</code>
直接 <code>man passwd</code>	没有证明页面属于 section 5	使用 <code>man 5 passwd</code>
<code>apropos</code> 空结果后宣布没有命令	未区分关键词、文档与索引	检查已知页面和 <code>whatis</code> ，必要时 <code>mandb</code>
把 <code>command -v cd</code> 的 <code>cd</code> 当路径	builtin 的简洁结果可以只是名称	结合 <code>type cd</code> 与 <code>help cd</code>
把 <code>rpm -qf</code> 与 <code>dnf provides</code> 都翻译成“来自哪个包”	混淆本地所有权与候选提供关系	分别记录本机数据库与 DNF 数据范围
<code>dnf provides</code> 命中后写“已安装”	候选关系不等于安装状态	后续用 RPM/DNF 安装查询验证
未引用 <code>*/semanage</code>	通配符可能先被当前 Shell 展开	使用单引号保护模式

本章收束 把“不知道”转换成一条可复核的本机调查路径

本章的工作方法不是记住所有工具，而是稳定选择下一条证据：

```
未知名称或异常行为
→ type / command 确认当前 Shell 对象
→ help / --help / man / info 选择匹配的帮助源
→ SYNOPSIS / OPTIONS / FILES / SEE ALSO 提取操作依据
→ whatis / apropos 在名称未知时发现主题
→ rpm -qf 证明本机已有路径的包所有权
→ dnf provides 证明 DNF 可见数据中的候选提供关系
→ 将每条结论限制在它实际证明的层次
```

主要判断表

问题	第一入口	结论范围	下一层
当前 Shell 会执行什么	<code>type / command -V</code>	当前会话的解析对象	帮助或实际路径
是否有同名候选	<code>type -a</code>	当前环境可见的候选集合	选择目标对象
builtin 怎样使用	<code>help</code>	当前 Bash builtin 语义	操作后状态验证
外部程序常用形式	<code>--help</code>	快速 Usage 与常用选项	<code>man / info</code>
同名 man page 怎样消歧	<code>man section page</code>	特定对象的本地手册	读取标准标题
只知道功能怎样发现命令	<code>apropos / man -k</code>	本地索引候选	打开具体 man page
已有路径属于哪个已安装包	<code>rpm -qf</code>	本机 RPM 所有权	安装/验证章节
缺失路径由谁提供	<code>dnf provides</code>	DNF 可见候选提供者	仓库与事务章节
查询结果互相矛盾	先标注各自证据层	不强求输出一致	选择高分度下一证据

工作方法

1. 先写问题层，再敲命令；
2. 先用当前环境证据确认对象，再进入帮助；
3. 读手册时从对象和形式开始，不从示例盲抄；
4. 空结果先调查可见性与索引，不直接宣布对象不存在；
5. 软件来源必须区分本机所有权与仓库候选；
6. 任何“找到”都不是最终功能证明。

向下一章交接

本章把路径当作查询键，但尚未解释路径如何解析、文件类型怎样区分、硬链接与符号链接如何影响对象身份。第 04 章《文件系统层次、路径、文件类型与链接》将在这里继续：当你已经找到一个路径，下一步要判断它在文件系统中究竟指向什么。

本章未连接受控 RHEL 9 虚拟机，也未执行命令级 live test。环境相关的真实输出应在后续实验中按本章证据链回填。

RHEL 9 · RHCSA 实操讲义

文件系统层次、路径、 文件类型与链接

从名称到对象：沿路径分量看清目录项、inode、链接与删除后的生命周期。

对象模型操作语义验证诊断经典任务

大字号阅读版

本章阅读导航

先抓住一条主线：路径只是逐级查找名称的指令；目录项把名称映射到 inode；真正的对象身份、链接行为和删除结果必须从 inode 与剩余引用中判断。

- 01 目录树与起点先判断路径从 `/` 还是当前目录开始
- 02 逐级解析把路径拆成分量，找第一处无法继续的位置
- 03 名称与 inode 把目录项、inode 和对象数据分开
- 04 文件类型区分名称后缀、inode 类型和内容格式
- 05 硬链与软链判断同 inode 还是独立路径对象
- 06 验证与诊断按名称、链接原文、路径链和对象身份取证

专题地图

- 知识专题单根目录树与 FHS 职责
- 知识专题路径解析与逻辑/物理路径
- 知识专题目录项、inode 与对象生命周期
- 知识专题文件类型与证据边界
- 操作专题从路径表面深入真实对象
- 知识专题硬链接与符号链接
- 操作专题创建、验证、替换和删除链接
- 诊断专题逐级定位路径和链接故障
- 经典任务修复相对链接：观察删除后的分化

阅读时持续回答

1. 这条路径从哪里开始解析？
2. 第一个失败的路径分量是哪一个？
3. 我看到的是名称、链接文本还是最终对象？
4. 两个名称是否处于同一文件系统并共享 inode？
5. 当前命令会观察链接自身还是跟随目标？
6. 删除的是哪个目录项，还有哪些引用存在？
7. 这条证据能证明什么，又不能证明什么？

第 04 章 · 正文

在命令行里看到 `/srv/app/current/config.yml` 时，人很容易把它当作一个完整、固定的“文件”。Linux 实际处理的是一串路径分量：从根目录或当前工作目录出发，在每一级目录中查找名称；名称通过目录项映射到 inode；inode 再描述对象类型、元数据、链接计数以及数据位置。路径中还可能插入符号链接保存的目标文本，所以表面名称与最终对象可能完全不同。

如果把名称、对象和内容混在一起，就会产生高频误判：内容相同就认为是同一个文件；删除“原文件”就认为对象已经消失；`readlink` 有输出就认为目标可访问；硬链接失败后改用复制却仍声称“同 inode”；路径报错时只检查最终文件，而没有寻找第一个失败的目录分量。

本章以“路径文本如何解析到对象”为主线，先建立目录树、目录项、inode 和文件类型模型，再用 `pwd`、`stat`、`readlink`、`realpath` 与 `namei` 分层取证，最后解释硬链接、符号链接和删除名称后的生命周期。传统权限留给第 08 章，完整的 `find` 留给第 05 章，挂载遮蔽与持久配置留给第 24 章。

概念

目录项（directory entry）是目录中“名称到 inode”的映射。路径解析每前进一级，都是在当前目录中查找下一个目录项。名称属于目录项而不是普通文件 inode，因此重命名、硬链接和删除名称首先改变的是目录中的映射关系。

概念

inode 是文件系统内部描述对象身份与元数据的记录，包含对象类型、模式位、所有者、大小、时间、链接计数以及数据引用。inode 编号只在所属文件系统内唯一，所以比较对象身份至少要同时核对文件系统标识和 inode。

概念

路径解析（path resolution）是从某个起点逐级解释路径分量的过程。中间分量必须能够继续作为目录使用；遇到符号链接时，系统把链接保存的目标文本插入剩余路径继续解析。诊断的关键不是重复访问最终文件，而是找到第一处无法继续的分量。

概念

绝对路径与相对路径的差别首先是解析起点：绝对路径从 `/` 开始，相对路径从进程当前工作目录开始。`.` 与 `..` 是路径分量；波浪号 `~` 则属于 Shell 展开，不是内核路径解析规则。

概念

文件类型 是 inode 的对象属性，普通文件、目录、符号链接、块设备、字符设备、FIFO 和 socket 的操作语义不同。扩展名只是名称的一部分，`file` 判断的是内容或格式；名称后缀、inode 类型和内容格式必须分开取证。

概念

硬链接（hard link）是另一个指向同一 inode 的目录项。多个硬链接是平等名称，不存在永久的“原件”和“副本”；它们必须位于同一文件系统，普通工作流也不为目录建立硬链接。删除一个名称只会减少链接计数。

概念

符号链接（symbolic link）是拥有自己 inode 的独立对象，其数据是一段目标路径文本。它可以跨文件系统、可以指向目录、也可以在目标不存在时创建；代价是目标移动或删除后可能悬空，并在同一路径后来出现新对象时重新绑定。

操作语义

以下入口构成本章最小接口地图。先明确命令观察或改变的对象，再选择参数；正文专题会继续解释证据边界，不在每个小节点重复整套手册页。

`pwd / realpath`

SYNOPSIS

```
pwd [-LP]
realpath [OPTION]... FILE...
```

确认相对路径的解析起点，并在需要时展开符号链接得到规范路径。

重要参数 / 形式

`pwd`

显示 Shell 记录的逻辑工作目录。

`pwd -P`

显示展开符号链接后的物理工作目录。

`realpath -e PATH`

要求所有分量存在，用于证明当前路径完整可解析。

`realpath -m PATH`

允许分量缺失，适合规范化计划创建或当前失效的路径文本。

`ls / stat / file`

SYNOPSIS

```
ls [OPTION]... [FILE]...
stat [OPTION]... FILE...
file [OPTION]... FILE...
```

分别观察目录项摘要、inode 元数据和内容格式；三者回答的是不同层次的问题。

重要参数 / 形式

`ls -ld -- PATH`

查看名称自身；`-d` 避免把目录展开为内容列表。

`stat -- PATH`

默认不解引用最终符号链接，因此可观察链接自身的 inode。

`stat -L -- LINK`

跟随最终符号链接，观察目标对象。

`%d / %i / %h / %F / %N`

设备标识、inode、硬链接计数、类型和名称/链接表示。

`file -L -- LINK`

跟随链接判断目标内容格式；它不证明两个名称共享 inode。

`ln`

SYNOPSIS

```
ln [OPTION]... TARGET LINK_NAME
```

默认创建硬链接：在目标目录中增加一个指向现有 inode 的新名称。

重要参数 / 形式

`ln TARGET HARD_LINK`

要求目标存在，并且新名称与目标位于同一文件系统。

--

结束选项解析，避免以 - 开头的名称被当作选项。

跨文件系统

会失败并出现类似 Invalid cross-device link 的错误；不能用复制后仍声称“同一 inode”。

目录硬链接

普通管理员工作流不使用；题目要求链接目录时选择符号链接。

ln -s

SYNOPSIS

```
ln -s [OPTION]... TARGET_TEXT LINK_NAME
```

创建独立的符号链接对象，并把 TARGET_TEXT 原样保存为链接数据。

重要参数 / 形式

-s

创建符号链接而不是硬链接。

-r

与 -s 配合，按链接位置生成相对目标；使用前仍要验证最终文本。

相对目标

从符号链接所在目录解释，而不是从创建命令时的当前目录解释。

-f

会删除已有目标名称，风险较高；本章不把无调查的 ln -sf 当作默认修复。

readlink

SYNOPSIS

```
readlink [OPTION]... FILE...
```

无参数时读取符号链接保存的原始目标；带规范化参数时可跟随路径链。

重要参数 / 形式

readlink -- LINK

只显示链接保存的原始文本，不保证目标存在。

readlink -f PATH

递归跟随符号链接并规范化；除最后一个分量外必须存在。

readlink -e PATH

要求所有路径分量都存在。

readlink -m PATH

允许分量缺失；结果不是“当前对象存在”的证明。

namei

SYNOPSIS

```
namei [OPTION]... PATHNAME...
```

沿路径逐级显示分量，遇到符号链接时展开目标，是定位第一处路径故障的核心入口。

重要参数 / 形式

namei -l -- PATH

长格式显示类型、模式摘要、所有者与组，并跟随符号链接。

namei -x -- PATH

标记挂载点或文件系统边界；具体字符表现以目标 util-linux 版本为准。

namei -n -- PATH

不跟随符号链接，用于只观察表面路径分量。

证据边界

它能定位路径层次，但传统权限的完整计算留给第 08 章。

[知识专题] 单根目录树与 FHS：先知道路径大致属于哪里

Linux 把普通文件、设备节点、运行时接口和挂载进来的其他文件系统组织到同一棵目录树中。FHS 目录用途能帮助管理员快速判断“应去哪里找”，但它不替代查询：发行版可能使用兼容符号链接，某个目录也可能是单独挂载点或虚拟文件系统。最稳妥的切入方式是先理解目录职责，再用 `ls -ld`、`readlink`、`findmnt` 等证据确认当前系统的真实布局。

① [知识点] `/` 是路径解析的根，不等于某一块磁盘

绝对路径以 `/` 开头，并从当前进程看到的根目录开始解析。所有普通路径都接入这棵树，但树中的不同子目录可以来自不同文件系统。因而“同一棵目录树”不等于“所有名称都在同一文件系统”，这也是硬链接可能在两个看似相邻的目录之间失败的原因。

本章只建立这一接口：路径所在文件系统可用 `stat`、`df` 或 `findmnt` 查询。分区、挂载配置、挂载遮蔽和 `/etc/fstab` 留到“文件系统挂载与持久性”章节。

② [知识点] 常用目录按职责理解，而不是孤立背诵

配置与身份

`/etc` 保存主机特定的系统与服务配置；`/home` 是普通用户主目录的常见上级；`/root` 是 root 自己的主目录。用户真实主目录仍以账户记录为准。

软件与共享内容

`/usr` 主要承载发行版提供的程序、库和共享数据；`/usr/local` 用于把本地管理员安装内容与发行版管理内容分开；`/boot` 保存启动相关内容。

可变与临时状态

`/var` 保存日志、队列、缓存和数据库等可变持久数据；`/run` 保存当前启动周期的运行状态；`/tmp` 可能被自动清理，不能当作可靠持久存储。

内核与设备接口

`/dev` 提供设备节点；`/proc` 提供进程与内核运行信息；`/sys` 暴露设备、驱动和内核对象。它们都在目录树中，但多数内容不是普通持久磁盘数据。

③ [知识点] RHEL 9 的兼容目录可能本身就是符号链接

现代 RHEL 系统通常把 `/bin`、`/sbin`、`/lib`、`/lib64` 统一到 `/usr` 对应位置，并保留兼容符号链接。学习时不要只背“`/bin` 放普通命令”，而应看到对象关系：表面路径可能是符号链接，最终程序位于 `/usr/bin`。

推荐在目标系统查询：

```
ls -ld /bin /sbin /lib /lib64
readlink /bin
realpath -e /bin
```

这组命令分别显示名称自身、链接保存的文本和规范化后的最终路径。具体布局以目标 RHEL 9 系统的证据为准。

[Cheatsheet] `/` 是统一命名空间的根；`/etc` 放配置，`/var` 放可变持久数据，`/run` 放当前启动周期状态；`/proc`、`/sys`、`/dev` 是运行接口；常见兼容目录先用 `ls -ld` 和 `readlink` 查询。

[知识专题] 路径解析：路径字符串怎样一步步到达对象

路径不是对象身份，而是查找对象的指令。诊断路径问题时，第一步不是立即查看最终文件，而是确定解析起点、拆分路径分量，并找出第一个无法继续的分量。只要把路径解析看成逐层查表，`No such file or directory`、`Not a directory` 和符号链接循环就不再是模糊报错。

① [知识点] 绝对路径和相对路径只是在起点上不同

绝对路径以 `/` 开头，从根目录解析：

```
/etc/ssh/sshd_config
```

相对路径不以 `/` 开头，从当前工作目录解析：

```
configs/app.yml
../archive/report.txt
```

相对路径的含义依赖当前工作目录。执行脚本或计划任务时，不能假定其当前目录与交互式终端一致；稳定任务应明确工作目录或使用绝对路径。

`pwd` 显示 Shell 当前工作目录：

```
pwd
```

波浪号 `~` 是 Shell 在执行命令前进行的展开，不是内核路径解析中的特殊路径分量。其完整语义属于“Shell 解析、引用与展开”章节。

② [知识点] `.`、`..`、尾随斜线和路径分量都有操作语义

- `.` 表示当前目录；
- `..` 表示父目录；在根目录中继续使用 `..` 仍停留在根；
- `/` 分隔路径分量，不能作为普通文件名的一部分；
- NUL 字符不能出现在路径名中；
- Linux 文件名区分大小写；
- 路径末尾的 `/` 要求解析结果具有目录语义。

例如，对一个普通文件使用 `report.txt/`，可能得到“不是目录”的错误。这个尾随斜线不是无意义装饰，它改变了最终分量的要求。

③ [知识点] 符号链接会把目标文本插入剩余路径继续解析

假设：

```
/srv/app/current -> releases/v2
```

访问：

```
/srv/app/current/bin/start
```

解析到 `current` 时，内核读取链接中的 `releases/v2`，从链接所在目录 `/srv/app` 继续解释相对目标，然后再拼接剩余的 `bin/start`。所以相对符号链接的目标不是相对于“创建链接时的当前目录”，而是相对于“符号链接所在目录”。

④ [知识点] 逻辑路径和物理路径可以同时成立

Shell 可能记录用户通过符号链接进入目录的逻辑路径，而物理路径是展开符号链接后的真实目录链：

```
pwd          # 通常显示逻辑路径
pwd -P       # 展开符号链接，显示物理路径
realpath -e .
```

逻辑路径适合保持用户导航语境，物理路径适合调查对象落点。两者回答不同问题，不应简单判定其中一个“错误”。

[Cheatsheet] 绝对路径从 `/` 开始，相对路径从 `pwd` 开始；`.` 是当前目录，`..` 是父目录；尾随 `/` 要求目录语义；相对符号链接从链接所在目录解释；物理路径用 `pwd -P` 或 `realpath -e`。

[知识专题] 目录项与 inode：名称不是对象本身

人通常通过名称识别文件，内核则需要把名称解析到 inode。这个区别是理解链接和删除行为的核心。看到两个路径内容相同，不代表它们是同一个对象；看到 inode 数字相同，也必须先确认它们属于同一文件系统。

01路径文本/srv/app/current/config.yml

→

02逐级目录项名称 → inode

→

03inode类型、元数据、链接计数

→

04对象数据内容或特殊对象接口

判断边界：路径是查找指令，不是对象身份；名称相同、内容相同或 inode 数字相同，都不足以脱离文件系统上下文单独证明“同一对象”。

① [知识点] 目录项保存“名称到 inode”的映射

目录不是“装着文件内容的盒子”；从命名角度看，它维护一组目录项。每个目录项至少把一个名称映射到所属文件系统上的 inode。路径解析本质上是在每一级目录中查找下一个名称。

因此，重命名或在同一文件系统中移动文件，通常主要改变目录项：旧目录删除一条名称映射，新目录增加一条名称映射，对象 inode 可以保持不变。跨文件系统移动无法直接沿用原 inode，工具通常需要复制数据并删除旧名称。

② [知识点] inode 保存对象元数据和数据位置

inode 通常包含：

- 对象类型与模式位；
- 所有者 UID 和组 GID；

- 文件大小和占用块信息；
- 访问、修改、状态变更等时间；
- 硬链接计数；
- 指向文件数据或文件系统数据结构的引用。

文件名不保存在普通文件的 inode 中。一个 inode 可以被多个目录项引用，这正是硬链接的基础。权限的计算规则属于第 08 章，本章只把模式位视为 inode 元数据之一。

③ [知识点] 对象身份至少要比较“文件系统 + inode”

inode 编号只需在所属文件系统内唯一。两个路径的 inode 数字碰巧相同，如果设备或文件系统不同，仍然是不同对象。静态调查可使用：

```
stat -c 'device=%d inode=%i links=%h type=%F name=%N' PATH
```

比较两个路径是否为同一对象时，至少核对 %d 与 %i。sha256sum 相同只能证明读取到的字节内容相同，不能证明 inode、所有者、权限、时间或链接关系相同。

④ [知识点] 链接计数与打开引用共同决定对象生命周期

每增加一个普通文件硬链接，inode 的链接计数增加。删除一个名称只是移除一个目录项并减少链接计数。最后一个硬链接被删除后：

- 如果没有进程打开该对象，文件系统可以回收 inode 和数据空间；
- 如果仍有进程持有打开文件描述符，对象会继续存在，直到最后一个引用关闭。

所以“目录中已经看不到文件名”不等于“数据已经立刻从系统中消失”。这也是日志文件被删除后空间可能暂时没有释放的底层原因。

[Cheatsheet] 目录项是“名称 -> inode”；inode 不保存普通文件名；对象身份比较 %d:%i；内容哈希不能证明链接关系；删除名称后还要看剩余硬链接和打开引用。

[知识专题] 文件类型：名称、inode 类型和内容格式是三个维度

Linux 把目录、符号链接、设备节点、FIFO 和 socket 都放在文件系统命名空间中，但它们的操作语义不同。扩展名只是名称的一部分，不决定 inode 类型：file 输出的“文本”“ELF”或“压缩数据”又是内容格式判断。稳定调查要明确自己在问哪一种“类型”。

① [知识点] ls -l 的首字符表示对象类型

首字符	类型	常见示例
-	普通文件	配置、文本、程序、归档
d	目录	/etc、/var/log
l	符号链接	/bin 等兼容路径
b	块设备	磁盘或逻辑块设备节点
c	字符设备	终端、随机数等字符设备节点
p	FIFO/命名管道	进程间数据流接口
s	socket	Unix 域 socket 路径

类型字符之后的权限位属于第 08 章。本章只训练识别类型并选择合适证据。

② [知识点] ls、stat 和 file 回答不同问题

```
ls -ld -- PATH
stat -- PATH
file -- PATH
```

- ls -ld 查看名称自身及摘要，-d 防止把目录展开成其内容列表；
- stat 查看 inode 元数据，默认不解引用最终符号链接；
- file 使用 magic、内容特征等判断文件格式，默认通常把符号链接报告为链接对象。

需要检查链接目标时，可显式使用：

```
stat -L -- LINK
file -L -- LINK
```

③ [知识点] 扩展名和内容不能替代对象类型证据

名为 `report.txt` 的对象可能是目录、符号链接或二进制文件；没有扩展名的程序也可以是 ELF 可执行文件或脚本。考试和工作中都应先用对象证据，再用内容证据：

```
对象类型: ls -ld / stat
内容格式: file
字节一致: sha256sum
```

[Cheatsheet] `ls -l` 首字符看 inode 类型；`stat` 看元数据；`file` 看内容格式；扩展名不是可靠类型：查看目录自身用 `ls -ld`，跟随最终链接用显式 `-L`。

[操作专题] 从路径表面深入到真实对象：选择正确的查询接口

调查路径时，最常见的错误是只执行一条 `ls -l` 就下结论。稳定流程应从命名上下文开始，分别检查名称自身、链接原文、解析链、最终对象和对象身份。每条命令都只证明一个层次。

名称层 `ls -ld` 证明目录项表面状态，不证明整条路径可达。

对象层 `stat` 证明 inode 元数据，不证明内容符合业务目标。

链接原文 `readlink` 证明保存的文本，不证明目标存在。

路径链 `namei -l` 定位逐级解析，不替代权限规则分析。

① [操作] 使用 `pwd` 与 `ls -ld` 固定命名上下文

作用对象：当前 Shell 工作目录和指定目录项。

基本形式：

```
pwd
ls -ld -- PATH
```

关键语义：`--` 结束选项解析，避免以 `-` 开头的名称被误当作参数。`-d` 让目录和指向目录的符号链接以名称自身为观察对象，而不是列出目录内容。

验证边界：`ls -ld` 可显示类型字符和链接箭头，但不能单独证明目标链中的每一级都存在，也不能证明两个路径共享 inode。

② [操作] 使用 `stat` 比较链接自身、目标和对象身份

作用对象：最终路径分量的 inode，或在 `-L` 下解引用后的目标 inode。

```
stat -- PATH
stat -L -- PATH
stat -c 'device=%d inode=%i links=%h type=%F size=%s name=%N' -- PATH
```

常用格式字段：

字段	含义
%d	设备标识的十进制值
%i	inode 编号
%h	硬链接计数
%F	文件类型文本
%s	字节大小
%n	输入名称
%N	带引用和链接目标的名称表示

验证边界：`stat` 证明对象元数据，不证明内容符合业务要求。远程或特殊文件系统还可能存在缓存和实现差异。

③ [操作] 使用 `readlink` 读取符号链接保存的原始目标

作用对象：符号链接自身的数据内容。

```
readlink -- LINK
```

输出可能是绝对路径，也可能是相对文本。它只回答“链接保存了什么”，不保证目标存在，也不保证这段文本从链接父目录解释后能到达预期对象。

不要用 `cat LINK` 读取链接原文：普通文件读取通常会跟随链接并读取目标内容。

④ [操作] 使用 `realpath` 获取规范路径

```
realpath -e -- PATH
realpath -m -- PATH
```

- `-e` 要求所有路径分量存在，适合证明当前路径确实可解析；
- `-m` 允许分量不存在，适合规范化计划创建或当前失效的路径文本。

`realpath -e` 失败而 `readlink` 成功，通常说明链接对象存在，但目标解析链中有缺失或错误分量。

⑤ [操作] 使用 `namei` 查看逐层路径链

```
namei -l -- PATH
namei -x -- PATH
```

`namei` 逐层显示路径分量，遇到符号链接时继续展开，并通过缩进表示上下文。`-l` 以长格式显示类型和模式摘要；`-x` 可标记挂载点或文件系统边界，具体输出以目标系统版本为准。

帮助入口：

```
man stat
man readlink
man realpath
man namei
```

[Cheatsheet] 当前目录 `pwd`；名称自身 `ls -ld`；inode 和链接数 `stat`；链接原文 `readlink`；存在的规范路径 `realpath -e`；逐层链路 `namei -l`。

[知识专题] 硬链接：多个平等名称共享同一个 inode

硬链接不是“指向原文件的快捷方式”，而是同一 inode 的另一个目录项。创建后，两个名称在对象层面地位平等，无法从 inode 判断哪个是“原件”。这种语义决定了硬链接的验证方式、文件系统边界和删除行为。

① [知识点] 创建硬链接只增加名称和链接计数

```
ln -- TARGET NEW_NAME
```

成功后，`TARGET` 与 `NEW_NAME` 应在同一文件系统中具有相同 inode，链接计数相应增加：

```
stat -c '%d:%i links=%h %N' -- TARGET NEW_NAME
```

通过任一名称修改文件内容，另一个名称读取到同一对象的数据。所有者、权限和普通文件时间属于共享 inode，不存在为每个硬链接单独保存一套普通文件元数据。

② [知识点] 硬链接不能跨文件系统

目录项引用的是所属文件系统内部的 inode。新名称若位于另一个文件系统，就不能直接引用源文件系统的 inode，典型失败表现为 `Invalid cross-device link`。

调查时先比较目标和新名称父目录：

```
stat -c 'device=%d mount=%m name=%N' -- TARGET NEW_PARENT
```

需要跨文件系统保持引用关系时，通常选择符号链接；需要独立数据时选择复制。二者终态不同，不能互换声称完成。

③ [知识点] 普通工作流不为目录创建硬链接

目录硬链接会让目录图出现额外父路径和潜在循环，破坏树状遍历和生命周期管理。Linux 文件系统保留 `.`、`..` 等受控目录关系，但普通管理员不应尝试用强制参数为目录创建硬链接。题目要求链接目录时，使用符号链接，并明确验证解析结果。

④ [知识点] 删除任意一个硬链接只删除一个名称

```
rm -- ONE_NAME
```

只要仍有其他目录项引用 inode，数据就能通过剩余名称访问。最后一个硬链接删除后，还要看是否存在打开文件描述符。硬链接的稳态判断应基于链接计数和可访问名称，不要把某个名称称为永久“源文件”。

[Cheatsheet] `ln TARGET_NAME` 创建同 inode 的新名称；用 `%d:%i` 验证；链接计数增加；不能跨文件系统；不为目录创建硬链接；删除一个名称不会影响剩余硬链接。

[知识专题] 符号链接：独立对象保存一段目标路径

符号链接与硬链接最根本的区别不是命令多了 `-s`，而是对象模型不同。符号链接拥有自己的 inode，文件内容是一段路径文本；访问者在允许跟随链接的场景中，再解析这段文本到目标对象。因此它可以跨文件系统、可以指向目录、可以先于目标创建，也会发生悬空和重新绑定。

① [知识点] 链接自身、目标文本和最终对象必须分开观察

```
ln -s -- TARGET_TEXT LINK_NAME
```

创建后有三层事实：

1. `LINK_NAME` 是一个符号链接对象，有自己的 inode；
2. `readlink LINK_NAME` 返回保存的 `TARGET_TEXT`；
3. 跟随该文本后可能到达一个目标对象，也可能失败。

对应证据：

```
stat -- LINK_NAME
readlink -- LINK_NAME
stat -L -- LINK_NAME
```

② [知识点] 绝对目标和相对目标具有不同迁移特性

绝对目标以 `/` 开头，从当前进程根目录解析：

```
ln -s /srv/app/releases/v2 /srv/app/current
```

相对目标从链接所在目录解析：

```
ln -s releases/v2 /srv/app/current
```

当整个 `/srv/app` 目录树一起迁移时，相对链接通常更便携；只移动链接本身而不保持相对布局，则可能失效。选择哪一种取决于目标终态，不是“相对永远更好”或“绝对永远更清楚”。

③ [知识点] 符号链接可以跨文件系统并指向目录

符号链接只保存文本，不直接引用目标 inode，所以链接对象和目标可以位于不同文件系统，也可以把目录作为目标。跨文件系统只是允许创建，不代表目标一定存在或可访问。

④ [知识点] 悬空链接仍然是存在的对象

目标删除或移动后，符号链接本身仍存在：

```
ls -ld -- LINK_NAME
readlink -- LINK_NAME
```

但跟随解析会失败：

```
realpath -e -- LINK_NAME
stat -L -- LINK_NAME
```

如果以后在同一路径创建一个新对象，符号链接会解析到新对象，而不是继续绑定旧 inode。这种“按名称重新绑定”是符号链接和硬链接生命周期差异的重要考点。

⑤ [边界] 删除符号链接时不要给目录链接追加 `/`

要删除指向目录的符号链接本身，应对链接名称执行：

```
rm -- LINK_NAME
```

不要使用 `rm -r LINK_NAME/` 作为默认操作。尾随 `/` 会要求目录语义并可能让工具进入目标目录上下文，扩大误操作风险。删除前先用 `ls -ld` 和 `test -L` 确认对象。

[Cheatsheet] 符号链接有自己的 inode；原始文本用 `readlink`；目标用 `stat -L`；相对目标从链接所在目录解释；可跨文件系统、可指向目录、可悬空；删除链接本身不要追加 `/`。

[操作专题] 创建、验证、替换和删除链接的完整闭环

链接操作看似只有一条 `ln`，真正容易失分的是目标顺序、已有名称、目录目标语义和验证不足。稳定做法是操作前建立基线，创建后分别验证名称、链接原文、对象身份和最终解析结果；替换时只删除已经确认的链接对象。

① [操作] 创建并验证硬链接

作用对象：已存在普通文件的 inode 与新目录项。

```
ln -- /srv/linklab/source/report.txt \
    /srv/linklab/archive/report.hard
```

验证：

```
ls -li -- /srv/linklab/source/report.txt \
    /srv/linklab/archive/report.hard
stat -c 'device=%d inode=%i links=%h type=%F name=%N' -- \
    /srv/linklab/source/report.txt \
    /srv/linklab/archive/report.hard
```

验收重点是相同的设备标识和 inode，不是名称相似或内容哈希相同。

② [操作] 创建并验证符号链接

```
ln -s -- releases/v2 /srv/reporting/current
```

分层验证：

```
ls -ld -- /srv/reporting/current
readlink -- /srv/reporting/current
namei -l -- /srv/reporting/current/bin/report
realpath -e -- /srv/reporting/current/bin/report
stat -- /srv/reporting/current
stat -L -- /srv/reporting/current
```

如果目的名称已存在为目录，`ln` 可能把新链接创建到该目录内。需要把目的名称严格当作一个路径时，可在确认目标系统支持后使用 `-T/--no-target-directory`，或先调查并清理明确的旧名称。

③ [操作] 安全替换现有符号链接

不要直接把 `ln -sf` 当作默认答案。先调查：

```
ls -ld -- /srv/reporting/current
if test -L /srv/reporting/current; then
    readlink -- /srv/reporting/current
fi
```

确认它确实是允许替换的符号链接后，执行最小修改：

```
rm -- /srv/reporting/current
ln -s -- releases/v2 /srv/reporting/current
```

随后重新执行原始验证链。真实业务环境还应评估并发访问和原子切换需求；RHCSA 静态任务至少不能在未调查时删除未知目录或普通文件。

④ [操作] 删除名称并验证生命周期

删除一个硬链接名称：

```
rm -- /srv/linklab/source/report.txt
stat -c '%d:%i links=%h %N' -- /srv/linklab/archive/report.hard
```

删除符号链接名称：

```
rm -- /mnt/labfs/report.soft
```

`unlink NAME` 也能删除单个非目录名称，但考试和日常工作中 `rm -- NAME` 更常见。无论使用哪个入口，都要理解它改变的是目录项。

[Cheatsheet] 操作前 `ls -ld`；硬链接 `ln TARGET NAME`，验证 `%d:%i`；符号链接 `ln -s TARGET_TEXT NAME`，验证 `readlink + namei + realpath`；替换前确认 `test -L`；删除链接本身不加 `/`。

[诊断专题] 路径与链接故障：从症状推进到第一处失败

路径故障经常在最终命令处只显示一句错误。高质量诊断不是反复重建链接，而是先保留当前状态，再找出第一处无法解析的分量。统一链路如下：

```
症状
-> 当前目录与输入路径
-> 名称自身
-> 链接原文
-> 逐层路径链
-> 最终对象或文件系统身份
-> 最小修复
-> 原链路再验证
```

① [诊断点] `readlink` 有输出，但 `realpath -e` 失败

症状：链接看起来存在，访问目标却报告不存在。

当前证据：

```
ls -ld -- LINK
readlink -- LINK
```

假设：链接对象存在，但目标文本从链接父目录解析后包含缺失分量、错误层级或已经移动的名称。

下一条最有区分度的证据：

```
namei -l -- LINK
realpath -e -- LINK
```

最小修复：只修改错误的链接名称或目标文本，不修改无关目录和权限。

② [诊断点] `Not a directory` 表示中间分量类型错误

例如 `/srv/app/config/main.yml` 中，`config` 实际是普通文件或指向普通文件的链接。最终 `main.yml` 是否存在已经不是首要问题。

```
namei -l -- /srv/app/config/main.yml
```

找到第一个不是目录却有剩余分量的对象。若问题是权限导致无法穿越，记录证据并转到“本地权限、umask 与特殊权限”章节，不在本章用 `chmod 777` 绕过。

③ [诊断点] `Too many levels of symbolic links` 优先调查链接环

常见结构：

```
A -> B
B -> A
```

或更长的间接循环。保留两个链接对象，使用：

```
ls -ld -- A B
readlink -- A
readlink -- B
namei -l -- A
```

最小修复是纠正其中一个目标文本，使路径链收敛到实际对象；不要无差别删除整组目录。

④ [诊断点] `Invalid cross-device link` 说明硬链接跨越文件系统边界

```
stat -c 'device=%d mount=%m name=%N' -- SOURCE DEST_PARENT
```

如果设备标识或挂载点不同，硬链接的对象模型不成立。根据任务选择：

- 需要按路径引用同一目标：符号链接；
- 需要独立副本：复制；
- 题目必须硬链接：把新名称放到源文件所在文件系统。

不能用符号链接替代硬链接后仍声称满足“同 inode”验收。

⑤ [诊断点] 名称已删除但空间或进程访问仍存在

假设链：

```
仍有其他硬链接
-> 链接计数尚未归零
-> 或某进程仍持有打开文件描述符
```

先检查已知剩余名称的 `stat`。若链接计数已经归零但已识别进程仍在使用的对象，可在进程章节的帮助下检查 `/proc/<PID>/fd/`，或在系统安装 `lsuf` 时使用相应查询。不要为了释放空间直接无调查终止进程。

⑥ [边界] 最终符号链接是否跟随取决于具体命令

路径中间分量的符号链接通常需要展开才能继续解析；最终分量是否跟随则可能受命令和参数影响。例如：

- `stat LINK` 检查链接自身；
- `stat -L LINK` 检查目标；
- `readlink LINK` 读取链接文本；
- `rm LINK` 删除链接名称。

诊断时必须说明使用的是哪一种语义，不能把所有命令都概括为“默认跟随链接”。

[Cheatsheet] 悬空链接：`readlink -> namei -l -> realpath -e`；非目录分量：找第一处类型错误；链接环：保留并展开链接；跨文件系统：比较设备与挂载点；删除后仍占用：查其他硬链接和打开引用。

[经典任务] 调查并修复失效的相对符号链接

环境与当前状态

应用目录如下：

```
/srv/reporting/releases/v2/bin/report
/srv/reporting/current
```

`/srv/reporting/releases/v2/bin/report` 是题目指定的正确对象。应用固定通过以下路径访问：

```
/srv/reporting/current/bin/report
```

当前 `current` 是一个符号链接，但该访问路径失败。你不知道链接保存的文本是否正确。

目标终态

- `/srv/reporting/current` 必须是符号链接；
- 链接保存的目标必须是相对文本 `releases/v2`；
- `/srv/reporting/current/bin/report` 必须解析到题目给定对象；
- 不修改 `releases/v2` 中任何文件内容；
- 不修改权限、所有者、SELinux 或挂载配置；
- 不使用无调查的 `ln -sf`；
- 保留调查和验收命令。

验收证据

```
名称层: ls -ld
原始链接层: readlink
路径链接层: namei -l
规范路径层: realpath -e
对象层: stat / stat -L
内容类型层: file
```

请先独立完成。参考解答从新页开始。

[经典任务] 建立硬链接与符号链接，并观察删除名称后的分化

环境与当前状态

题目已经准备：

```
/srv/linklab/source/report.txt
/srv/linklab/archive/
/mnt/labfs/
```

题目同时保证 `/mnt/labfs` 与 `/srv/linklab` 属于不同文件系统。本章不负责创建或挂载文件系统。

目标终态与操作过程

1. 在 `/srv/linklab/archive/` 创建 `report.hard`，它必须是源文件的硬链接；
2. 在 `/mnt/labfs/` 创建 `report.soft`，其目标文本必须是绝对路径 `/srv/linklab/source/report.txt`；
3. 证明硬链接与源名称共享对象；
4. 证明符号链接拥有独立 inode，但跟随后到达源对象；
5. 删除原名称 `/srv/linklab/source/report.txt`；
6. 证明硬链接仍访问旧对象，而符号链接已经悬空；
7. 在原路径创建一个新的普通文件；
8. 证明符号链接现在解析到新对象，而硬链接仍引用旧对象。

限制条件

- 不得把复制品冒充硬链接；
- 不得在 `/mnt/labfs` 尝试创建跨文件系统硬链接后忽略失败；
- 不得删除 `archive/report.hard`；
- 所有判断必须用设备标识、inode、链接计数和链接原文证明；
- 不依赖伪造的固定 inode 数字或命令输出。

[参考解答] 任务一：按路径链调查并最小修复

① 建立名称与链接原文基线

```
ls -ld -- /srv/reporting/current
readlink -- /srv/reporting/current
```

先确认 `current` 确实是符号链接，并记录它当前保存的文本。若它是普通文件或目录，不能直接删除，应停止并重新确认题目环境。

② 展开当前失败链

```
namei -l -- /srv/reporting/current/bin/report
realpath -e -- /srv/reporting/current/bin/report
```

`namei -l` 用于定位第一处错误分量。`realpath -e` 失败只说明当前完整路径不能全部解析；不能据此直接修改权限或创建未知目录。

③ 核对题目指定目标

```
ls -ld -- /srv/reporting/releases/v2 \
/srv/reporting/releases/v2/bin/report
stat -c 'device=%d inode=%i links=%h type=%F name=%N' -- \
/srv/reporting/releases/v2/bin/report
file -- /srv/reporting/releases/v2/bin/report
```

这一步证明目标名称存在及其对象类型，不声称已经执行或通过业务功能测试。

④ 只替换已经确认的符号链接

```
if test -L /srv/reporting/current; then
    rm -- /srv/reporting/current
    ln -s -- releases/v2 /srv/reporting/current
else
    printf '%s\n' 'current 不是符号链接, 停止修改' >&2
fi
```

相对文本 `releases/v2` 从 `/srv/reporting` 解释, 得到 `/srv/reporting/releases/v2`。

⑤ 分层再验证

```
ls -ld -- /srv/reporting/current
readlink -- /srv/reporting/current
namei -l -- /srv/reporting/current/bin/report
realpath -e -- /srv/reporting/current/bin/report
stat -- /srv/reporting/current
stat -L -- /srv/reporting/current/bin/report
file -- /srv/reporting/current/bin/report
```

验收时应明确:

- `ls -ld` 证明 `current` 是链接;
- `readlink` 证明原始文本为 `releases/v2`;
- `realpath -e` 证明完整路径当前可解析到 `v2`;
- `stat` 与 `stat -L` 分别观察链接自身和目标对象;
- `file` 只判断内容类型, 不代替业务执行验证。

典型错误

- 在 `/srv/reporting` 中运行 `ln -s ../releases/v2 current`: 该文本会从链接父目录解释为 `/srv/releases/v2`;
- 使用 `ln -sf` 覆盖未知对象: 可能隐藏原有目录或错误名称状态;
- 只看 `readlink` 输出就声称修复: 目标文本存在不等于完整链可解析;
- 修改权限解决 `ENOENT`: 错误层级尚未确认。

[参考解答] 任务二: 用对象身份观察硬链接与符号链接分化

① 确认文件系统边界与当前对象

```
stat -c 'device=%d mount=%m type=%F name=%N' -- \
/srv/linklab/source/report.txt \
/srv/linklab/archive \
/mnt/labfs
```

确认源文件与 `archive` 在同一文件系统, 而 `/mnt/labfs` 不同。题目已经给出该条件, 命令用于保留验收证据。

② 创建硬链接和符号链接

```
ln -- /srv/linklab/source/report.txt \
/srv/linklab/archive/report.hard

ln -s -- /srv/linklab/source/report.txt \
/mnt/labfs/report.soft
```

③ 删除前建立三对象基线

```
stat -c 'device=%d inode=%i links=%h type=%F name=%N' -- \
/srv/linklab/source/report.txt \
/srv/linklab/archive/report.hard \
/mnt/labfs/report.soft

readlink -- /mnt/labfs/report.soft
stat -L -c 'device=%d inode=%i links=%h type=%F name=%N' -- \
/mnt/labfs/report.soft
```

应从关系上判断:

- 源名称与 `report.hard` 的设备标识和 inode 相同；
- `report.soft` 自身 inode 不同；
- `stat -L report.soft` 到达的设备与 inode 与源对象相同；
- `readlink` 显示绝对目标文本。

④ 删除原名称并验证分化

```
rm -- /srv/linklab/source/report.txt

stat -c 'device=%d inode=%i links=%h type=%F name=%N' -- \
    /srv/linklab/archive/report.hard

ls -ld -- /mnt/labfs/report.soft
readlink -- /mnt/labfs/report.soft
realpath -e -- /mnt/labfs/report.soft
```

最后一条命令此时应无法证明存在的最终路径。不要伪造具体错误文本；不同语言环境的提示可能不同。

⑤ 在原路径创建新对象并再次比较

```
printf '%s\n' 'new generation' > /srv/linklab/source/report.txt

stat -c 'device=%d inode=%i links=%h type=%F name=%N' -- \
    /srv/linklab/source/report.txt \
    /srv/linklab/archive/report.hard

stat -L -c 'device=%d inode=%i links=%h type=%F name=%N' -- \
    /mnt/labfs/report.soft
realpath -e -- /mnt/labfs/report.soft
```

最终关系应为：

```
archive/report.hard -> 仍引用删除前的旧 inode
source/report.txt   -> 新创建的 inode
report.soft         -> 按目标路径解析到新 inode
```

⑥ 内容层辅助验证

在对象身份已经证明后，可以用内容作为辅助证据：

```
cat -- /srv/linklab/archive/report.hard
cat -- /srv/linklab/source/report.txt
cat -- /mnt/labfs/report.soft
```

即使内容偶然相同，也不能替代 `%d:%i` 的身份判断。

典型错误

- 只比较 inode 数字，不比较文件系统；
- 在 `/mnt/labfs` 创建硬链接失败后改用复制，却仍声称“同一对象”；
- 删除原名称后把悬空符号链接当作已经删除；
- 原路径重建后认为硬链接也自动指向新对象；
- 只用 `sha256sum` 判断链接类型和 inode 关系。

[本章收束] 从“看见一个路径”升级为“识别一条对象解析链”

本章最重要的迁移不是多记几条命令，而是改变判断顺序：

```
路径文本
-> 解析起点
-> 每一级目录项
-> inode 和文件类型
-> 是否经过符号链接
-> 最终对象身份
-> 名称删除后的剩余引用
```

遇到文件问题时，先问自己正在证明哪一层：

- `ls -ld` 证明名称自身；
- `stat` 证明 inode 元数据；
- `file` 证明内容格式；
- `readlink` 证明链接原文；
- `realpath` 证明规范路径；
- `namei` 证明逐层解析链；
- `%d:%i` 证明对象身份；
- 链接计数和打开引用解释生命周期。

下一章将在这个对象模型上学习文件查找、文本筛选与批量处理。届时 `find` 选择的不是抽象“文件名”，而是目录树中满足类型、元数据和路径条件的一组对象。

主要判断表

你要回答的问题	首选证据	它不能单独证明
当前 Shell 从哪里解释相对路径？	<code>pwd / pwd -P</code>	目标是否存在
名称自身是什么类型？	<code>ls -ld -- PATH</code>	符号链接目标链是否完整
两个名称是否共享对象？	<code>stat -c '%d:%i %h'</code>	内容是否满足业务要求
链接保存了什么文本？	<code>readlink -- LINK</code>	文本能否解析成功
路径最终规范化到哪里？	<code>realpath -e -- PATH</code>	原始链接文本是什么
哪一级目录开始失败？	<code>namei -l -- PATH</code>	失败是否由第 08 章权限规则造成
删除名称后对象是否仍存活？	链接计数 + 打开引用	哪个业务进程应被终止

向下一章交接

下一章“文件查找、文本筛选与批量处理”会把本章对象模型变成选择条件：`find` 遍历的是目录项和对象状态，类型、路径、时间和所有者都只是筛选维度。进入下一章前应保持一个习惯：先确认自己正在选择名称、路径还是对象状态，再决定是否批量操作。

最终 Cheatsheet

```
pwd
ls -ld -- PATH
stat -c 'device=%d inode=%i links=%h type=%F name=%N' -- PATH
stat -L -- LINK
file -- PATH
readlink -- LINK
realpath -e -- PATH
namei -l -- PATH
ln -- TARGET HARD_LINK
ln -s -- TARGET_TEXT SYMLINK
rm -- NAME
```

硬链接：同文件系统、同 inode、平等名称、删除一个名称仍可存活

符号链接：独立 inode、保存路径文本、可跨文件系统、可悬空和重新绑定

删除：先删除目录项；最后硬链接和最后打开引用都消失后，对象才可最终回收

本章阅读导航

先抓住一条主线：先从目录树中选择正确的文件集合，再把文件内容视为记录与字段，最后把每个路径安全地交给批量命令，并用独立证据证明没有漏改和误伤。

专题地图

- 知识专题 从目录树对象到记录流
- 知识专题 `find` 的搜索空间与可见性
- 操作专题 把任务要求翻译成 `find` 谓词

- 操作专题 表达式、逻辑与动作
- 知识专题 `locate` 与索引时间边界
- 知识专题 正则与 `grep` 记录选择
- 操作专题 字段、排序、去重与计数
- 操作专题 `sed`、`awk` 与结构化转换
- 操作专题 `NUL`、`xargs` 与批量 `argv`
- 诊断专题 空集合、过宽集合与部分失败

阅读时持续回答

1. 当前选择的是文件对象、路径名称，还是文件内容中的记录？
2. 搜索起点、深度和文件系统边界是否准确？
3. 模式属于 `glob`、固定字符串、BRE 还是 ERE？
4. 一条记录在哪里结束，字段又怎样拆分？
5. 特殊文件名会不会被空白或换行拆坏？
6. 预览集合与执行集合是否来自同一筛选表达式？
7. 当前证据能证明什么，又不能证明什么？
8. 下一条最有区分度的证据是什么？

01限定搜索空间
02建立对象谓词
03预览候选集合
04选择记录与字段
05构造安全 `argv`
06执行分层验收

第五章 文件查找、文本筛选与批量处理

系统管理员面对的通常不是一个已经知道路径的文件，而是一组随目录树变化的对象：先按路径、类型、所有者、大小或时间找到候选文件，再从文件内容中选择记录、拆分字段、排序聚合，最后才可能执行修改。最常见的误判，是把这几个对象层混成一层：用文件名模式代替内容匹配、把按行输出当作任意文件名协议、在未确认候选集合时直接使用 `sed -i`，或者看到子命令退出为零就认为所有目标都已经达到终态。

本章采用“文件集合 → 文本记录 → 字段和键 → 参数向量 → 批量副作用 → 分层验证”的主线。前一章已经建立路径、文件类型和链接的基础；本章只引用这些对象，不重新展开路径解析。Shell 展开与引用留在第 01 章，复制、归档、压缩和远程传输留给第 06 章；`sed` 与 `awk` 只讲完成本章任务所需的选择、替换、字段和小型聚合，不扩展成完整编程教材。

本章的核心判断链 先证明搜索范围 → 再证明候选集合 → 再证明记录与字段合同 → 最后执行副作用并重新查询终态

概念

文件集合 是在某一搜索空间和某组条件下成立的对象集合。它不是一份永久不变的路径清单：目录树可能变化、权限可能阻止遍历、链接策略可能改变观察对象。判断集合时必须同时保留搜索起点、表达式、标准错误和生成时间；仅看到若干输出路径，不能证明没有遗漏。

概念

`find` 表达式 是对每个遍历对象求值的条件与动作组合，由测试、逻辑运算符和动作共同构成。相邻测试默认使用 AND，AND 的优先级高于 OR，动作本身也有真值并参与短路求值。因此，复杂命令必须先用括号明确逻辑，再把统一的预览或执行动作放到完整条件之后。

概念

正则表达式 描述文本记录中的字符模式，不等于 Shell glob。`*.conf` 是路径名称模式，`*\*.conf$` 才是正则形式；`grep -F` 又把模式视为固定字符串。选择错误的模式语言，会让命令看似合理却得到过宽、过窄或完全不同的集合。

概念

记录与字段 是文本处理的两层合同。记录边界先回答“一条输入在哪里结束”，字段边界再回答“记录内部怎样拆列”。`grep`、`sed` 默认按换行处理记录，awk` 默认一行一条记录并根据 `FS` 拆字段。若记录边界已经破坏对象，后续字段工具无法把它恢复。`

概念

NUL 分隔 利用 Linux 文件名不能包含 NUL 字节这一事实，把任意路径名称作为完整记录传递。`find -print0`、`grep -Z` 等生产端必须与 xargs -0` 等消费端 成对出现。NUL 解决的是名称分隔问题，不会自动解决目录树在“检查后、使用前”发生变化的竞态。`

概念

批量执行边界 是从“我找到了什么”切换到“我要改变什么”的分界线。安全流程把预览、执行和验收拆开：预览证明集合正确，执行把每个路径作为独立 `argv` 传给子命令，验收重新查询新状态、旧状态、反例、备份和错误流。命令成功只证明命令按自身约定结束，不能替代终态验证。

操作语义速查

这里先建立关键命令的接口地图。正文专题会继续解释为什么这样选择、怎样验证以及在哪些边界下不能直接执行。

`find`

SYNOPSIS

```
find [OPTIONS] [START...] [EXPRESSION]
```

实时遍历一个或多个起点，对每个对象求值表达式，并通过动作输出或传递匹配对象。

重要参数 / 形式

START...

明确搜索起点；关键变更优先使用绝对路径。

`-type` / `-name` / `-user` / `-size` / `-mtime`

把任务条件翻译为可解释的对象测试。

`\( A -o B \) / ! TEST`

显式表达 OR、分组和否定，避免优先级误判。

`-print` / `-printf`

用于可读预览和审计输出。

`-print0`

以 NUL 终止每个路径，供支持 NUL 的下游消费。

`-exec command -- {} +`

由 `find` 直接把多个路径构造为独立参数。

`locate` / `updatedb`

SYNOPSIS

```
locate [OPTIONS] PATTERN...
updatedb [OPTIONS]
```

`locate` 查询预先建立的名称索引；`updatedb` 重新扫描允许的目录并更新索引。结果速度快，但时效性低于实时遍历。

重要参数 / 形式

`locate PATTERN`

快速生成路径候选，不直接证明对象当前存在。

`-i`

名称匹配时忽略大小写。

`-n N`

只显示前 N 个匹配，适合快速抽样。

`updatedb`

更新索引；排除规则和权限仍决定哪些路径可被收录。

`stat -- PATH`

把索引候选带回实时证据。

`grep`

SYNOPSIS

```
grep [OPTIONS] PATTERN [FILE...]
```

从文本记录中选择与模式匹配的记录；模式引擎和输出形式会直接改变结果语义。

重要参数 / 形式

`-F`

把模式当作固定字符串，避免不需要的正则解释。

`-E`

使用扩展正则表达式。

`-r` / `-R`

递归读取目录；符号链接处理边界不同，使用前需确认。

`-n` / `-i` / `-v`

显示行号、忽略大小写、反选。

`-l` / `-L` / `-c` / `-q`

输出匹配文件名、不匹配文件名、计数或仅使用退出状态。

`cut` / `sort` / `uniq` / `tr` / `wc`

SYNOPSIS

```
cut OPTION... [FILE...]
sort [OPTION]... [FILE]...
uniq [OPTION]... [INPUT [OUTPUT]]
tr [OPTION]... STRING1 [STRING2]
wc [OPTION]... [FILE]...
```

这些工具适合对合同明确的记录流执行字段提取、排序、相邻去重、字符转换和计数。

重要参数 / 形式

`cut -d DELIM -f LIST`

按单字符分隔符提取固定字段。

`sort -t DELIM -k KEY / -n / -r`

明确字段分隔、排序键、数字比较和方向。

`sort ... | uniq -c`

先让相同记录相邻，再计数；排序会改变原始顺序。

`tr -d / -s`

删除字符集合或压缩重复字符，不做字符串子串替换。

`wc -l / -c / -w`

统计换行、字节或单词；数字必须解释成明确对象。

sed

SYNOPSIS

`sed [OPTIONS] SCRIPT [INPUTFILE...]`

按地址选择记录并执行替换、打印或删除等编辑动作。默认先把结果写到标准输出，再决定是否修改文件。

重要参数 / 形式

`-n 'ADDRESS p'`

关闭默认输出，只打印明确选择的记录。

`-E`

在脚本中使用扩展正则表达式。

`s/OLD/NEW/g`

替换一条记录中的全部匹配；未加 `g` 时只替换第一个。

`-e SCRIPT / -f FILE`

组合多个脚本或从脚本文件加载规则。

`-i.SUFFIX`

原地替换并保留备份；执行前仍应先预览。

awk

SYNOPSIS

`awk [OPTIONS] 'PATTERN { ACTION }' [FILE...]`

把输入解释为记录与字段，根据条件执行打印、计算和小型聚合。

重要参数 / 形式

`-F DELIM`

设置输入字段分隔符。

`-v NAME=VALUE`

在处理输入前安全传入外部值。

`$0 / $1... / NF`

整条记录、字段和当前字段数。

`NR / FNR`

全部输入累计记录号与当前文件记录号。

`BEGIN / END / OFS`

初始化、最终汇总和稳定输出字段合同。

xargs

SYNOPSIS

`xargs [OPTIONS] [COMMAND [INITIAL-ARGS]]`

从输入项目构造命令参数并分批执行。它处理的是 `argv`，不应被用来把路径重新拼成 Shell 代码。

重要参数 / 形式

`-0`

按 NUL 读取输入，必须与 NUL 生产端配对。

`-r`

GNU 扩展：输入为空时不运行命令。

`-n N`

限制每次调用使用的输入项目数。

`-I REPL`

占位替换模式，适合参数位置不能简单追加的场景，但通常降低批量效率。

`--`

在被调用命令支持时结束其选项解析，保护以连字符开头的路径。

[知识专题] 从目录树对象到记录流：先确定自己正在处理什么

同一条流水线里可能同时出现文件对象、路径文本、文本行、字段和命令参数。它们看起来都能在终端上显示为字符串，但安全边界完全不同。理解这些层次，是后续所有工具选择的前提。

① [知识点] 文件对象、路径名和内容是三个筛选维度

`find` 主要依据目录树和元数据选择文件对象，例如类型、所有者、权限、大小和时间；`grep` 依据文件内容选择记录；文件名模式则只筛选路径名称。扩展名并不决定 Linux 文件类型，名称中含有 `.conf` 也不保证内容是配置文本。

稳定的判断顺序通常是：

```
限定目录树和文件类型
→ 按所有者、名称、大小、时间等缩小对象集合
→ 再读取候选文件内容
→ 按文本模式继续筛选
```

这样既能减少无关 I/O，也能避免把目录、设备或符号链接目标误当成普通文本文件处理。

② [知识点] 记录边界先于字段边界

一条记录在哪里结束，决定了后续工具能否保持对象完整。文本日志通常使用换行分隔记录；冒号、逗号或制表符可能用于记录内部字段。文件名集合则不能默认使用换行，因为文件名本身可以包含换行。

```
记录边界：一条输入何时结束
字段边界：一条记录内部怎样拆列
```

`cut -d: -f1` 只有在每条记录确实使用冒号分隔时才可靠。`awk -F:` 也依赖同一结构假设。面对面向人的、按空格对齐的输出，不应仅凭“看起来像第十列”建立长期脚本。

③ [知识点] 路径列表最终要变成独立 `argv`，而不是重新拼成 Shell 代码

外部命令接收的是参数向量。路径中的空格不应该被再次拆分，引号字符也不应该被解释为新的 Shell 语法。因此批量处理的目标是把每个路径作为一个独立参数传给子命令，而不是把所有名称拼成一行，再交给 `sh -c` 猜测。

安全接口包括：

```
find START TESTS -exec command -- {} +
find START TESTS -print0 | xargs -0 -r command --
```

第一种由 `find` 直接构造参数；第二种要求生产端和消费端都使用 NUL 协议。

④ [知识点] 本章的状态模型是“集合状态 + 转换状态 + 副作用状态”

需要分别验证：

- 搜索范围是否正确；
- 候选集合是否完整且不过宽；
- 文本转换是否保持了记录结构；
- 批量命令是否收到正确 `argv`；
- 每个目标是否达到终态；
- 应被排除的对象是否保持不变。

[Cheatsheet] `find` 选对象，`grep` 选记录，字段工具拆列，`xargs` 构造 `argv`；先确定记录分隔符，再确定字段分隔符；任意文件名集合优先使用 NUL。

[知识专题] `find` 的搜索空间：起点、遍历和可见性

`find` 的条件写得再精确，如果搜索起点和遍历边界不对，结果仍然会为空、过宽或跨入不应处理的文件系统。调查 `find` 命令时，首先拆开阅读“从哪里开始”和“怎样遍历”，然后再看测试和动作。

① [知识点] 起点决定可见对象集合

基本形式是：

```
find START... EXPRESSION
```

可以提供一个或多个起点。省略起点时，GNU `find` 从当前目录 `.` 开始。相对起点依赖当前工作目录；绝对起点不依赖当前目录。考试和生产变更中，建议把关键起点写清楚，避免因切换目录而改变搜索范围。

```
find /etc /usr/local/etc -type f -name '*.conf'
find . -maxdepth 1 -type f
```

传给 `-name` 的通配模式应由引号保护：

```
find /etc -name '*.conf'      # 模式由 find 解释
find /etc -name *.conf       # Shell 可能提前展开，语义依赖当前目录
```

② [知识点] 深度选项限制“进入多深”，不替代其他条件

常见边界：

- `-maxdepth 0`：只检查起点本身；
- `-maxdepth 1`：检查起点和直接成员；
- `-mindepth 1`：不让起点本身参与匹配；
- `-xdev` 或 `-mount`：不下降到其他文件系统。

```
find /srv/data -mindepth 1 -maxdepth 1 -type f
find / -xdev -type f -size +1G
```

`-xdev` 限制的是目录遍历跨文件系统，不代表结果只属于某种设备类型。它在从 `/` 开始调查大文件时特别重要，可避免进入 `/proc`、网络挂载和其他独立文件系统。

③ [知识点] 符号链接跟随策略改变被观察的对象

GNU `find` 默认采用 `-P`：遍历时不跟随符号链接，测试看到的是链接本身。`-L` 会跟随符号链接，可能进入链接指向的目录树，并改变 `-type` 等测试的观察对象；`-H` 只对命令行起点中的符号链接作特殊处理。

本章默认不使用 `-L` 执行批量修改。需要跟随链接时，必须先明确：

```
链接本身是否是目标
→ 目标对象是否在允许范围内
→ 是否可能形成循环或跨越边界
→ 修改后怎样证明没有触及额外对象
```

④ [知识点] 权限错误和空集合不是同一种证据

遍历目录通常需要执行权限，读取目录成员名称还需要读取权限。`find` 可能同时输出部分结果并在标准错误中报告 `Permission denied`。如果只看标准输出为空，无法区分：

- 真正没有匹配；
- 起点不存在；
- 无权进入子目录；
- 条件写错；
- 错误被重定向后忽略。

调查时保留 `stderr`，必要时分开记录：

```
find /srv -type f -name '*.conf' \
> /tmp/find.out 2> /tmp/find.err
```

[Cheatsheet] 先看起点，再看 `-maxdepth/-mindepth/-xdev`；默认 `-P` 不跟随遍历中的符号链接；标准输出为空不能证明没有对象，必须同时检查 `stderr` 和退出状态。

[操作专题] 用 `find` 测试把自然语言要求翻译为对象谓词

稳定的 `find` 命令应让每个要求对应一个可解释测试。先逐项验证测试，再组合逻辑，最后才添加动作。不要一开始就把长表达式和破坏性命令写在一起。

① [操作] 名称、路径和文件类型

作用对象：遍历到的当前路径。

基本形式：

```
find START -type f -name '*.conf'
find START -path '*/conf.d/*.conf'
```

关键边界：

- `-name` 只匹配 basename，不含前导目录；
- `-path` 匹配从起点产生的完整路径文本；
- `-iname` 执行不区分大小写的名称匹配；
- `-type f/d/l/b/c/p/s` 分别筛选常规文件、目录、符号链接、块设备、字符设备、FIFO 和套接字。

```
find /etc -type f -name '*.conf'
find /srv -type f -path '*/conf.d/*.conf'
find /dev -type b
```

名称模式由 `find` 解释，不是正则表达式。`*.conf` 中的 `*` 匹配任意字符序列；它不会自动表达“文件内容中的任意文本”。

② [操作] 所有者、组和权限

典型形式：

```
find /var -user root -group mail
find /srv -nouser -o -nogroup
find /home -type f -perm -002
```

权限测试必须区分：

- `-perm MODE`：模式精确匹配；
- `-perm -MODE`：指定的所有权限位都必须存在；
- `-perm /MODE`：指定权限位中至少一个存在。

```
find /srv -type f -perm 0640      # 精确为 0640
find /srv -type f -perm -0600    # 至少具备用户读写
find /srv -type f -perm /002     # 任意目标中 other 写位存在
```

`-perm` 是筛选接口。完整的传统权限、umask 和特殊权限位属于《传统权限、umask 与特殊权限位》一章，本章只解释完成筛选所需语义。

③ [操作] 大小条件中的单位和正负号

基本形式：

```
find /usr/share -type f -size +52428800c -size -104857600c
```

常用单位包括 `c` 字节、`k` KiB、`M` MiB、`G` GiB。`+n` 表示大于 `n` 个单位，`-n` 表示小于 `n` 个单位，无符号形式按单位块取整匹配。由于取整存在，不能把 `-size 1M` 简化为“精确等于 1 MiB”。需要精确边界时用 `stat -c %s` 抽查字节数。

```
find /srv -type f -size +10M
find /srv -type f -size -1048576c
stat -c '%s %n' /srv/example
```

④ [操作] 修改时间和参照文件

`-mmin` 按分钟计数，`-mtime` 按完整 24 小时周期计数：

```
find /home/student -type f -mmin -120
find /var/log -type f -mtime +7
```

通用数值语义：

- `-n`：少于 `n` 个计数单位；
- `+n`：多于 `n` 个计数单位；
- `n`：取整后正好为 `n`。

`-newer REFERENCE` 通过参照文件比较修改时间，适合表达“某个标记文件之后”：

```
touch -d '2026-07-12 09:00:00' /tmp/cutoff
find /srv -type f -newer /tmp/cutoff
```

静态任务中应清楚记录参照文件的来源，不能编造考试环境时间戳。

⑤ [验证] 对边界对象进行 `stat` 抽查

`find` 结果只说明测试在遍历时成立。对时间和大小的边界值，抽取最接近上下界的对象并使用 `stat` 验证：

```
find /srv -type f -size +50M -printf '%s\t%TY-%Tm-%Td %TH:%TM\t%p\n' |
sort -n | head
```

`-printf` 是 GNU `find` 的格式化输出，适合审计；它不自动添加换行，格式中需要明确写 `\n`。

[Cheatsheet] `-name` 看 `basename`，`-path` 看路径；`-perm MODE/-MODE//MODE` 分别是精确、全部位、任一；大小和时间数值有取整；关键边界用 `stat` 抽查。

[操作专题] `find` 表达式、逻辑和动作：让命令可以被逐项证明

`find` 表达式由选项、测试、动作和运算符组成。它对每个遍历对象从左向右求值，并使用短路逻辑。复杂命令的排错关键是把表达式拆成可验证的片段。

① [操作] 隐式 AND、OR、NOT 和括号

相邻测试之间默认是 AND：

```
find /var -type f -user root -group mail
```

等价于：

```
find /var -type f -a -user root -a -group mail
```

AND 的优先级高于 OR。下面命令并不表示“普通文件且名称为 `.conf` 或 `.ini`”：

```
find /etc -type f -name '*.conf' -o -name '*.ini'
```

它会让右侧 `.ini` 分支绕过 `-type f`。正确写法显式分组：

```
find /etc -type f \( -name '*.conf' -o -name '*.ini' \)
```

NOT 可写为 `!` 或 `-not`，Shell 中的 `!` 应避免被历史展开干扰，常用反斜杠或单引号保护上下文。

② [知识点] 短路求值决定动作是否执行

对于 `A -a B`，如果 A 为假，则 B 不再求值；对于 `A -o B`，如果 A 为真，则 B 不再求值。动作也有返回真值，因此把 `-print` 放在 OR 表达式某一侧会改变哪些对象被显示。

稳定做法是：

```
先用完整测试形成候选
→ 再在表达式末尾放统一动作
```

```
find /etc -type f \( -name '*.conf' -o -name '*.ini' \) -print
```

③ [操作] 使用 `-print`、`-printf` 和 `-ls` 预览

没有显式动作时，GNU `find` 通常隐式使用 `-print`。为了审查集合，建议显式写出动作，并根据需求选择格式：

```
find /srv -type f -name '*.conf' -print
find /srv -type f -name '*.conf' \
  -printf '%m\t%u:%g\t%s\t%TY-%Tm-%Td %TH:%TM\t%p\n'
find /srv -type f -name '*.conf' -ls
```

面向人的预览可以使用换行；准备交给机器处理任意文件名时应使用 `-print0`。

④ [操作] `-exec ... {} \;` 与 `-exec ... {} +`

```
find /srv -type f -name '*.conf' -exec stat -- {} \;  
find /srv -type f -name '*.conf' -exec stat -- {} +
```

- `{}` `\;`: 通常每个匹配对象执行一次命令;
- `{}` `+`: 尽可能批量把多个路径作为参数, 减少进程启动;
- `--`: 在支持该约定的子命令中结束选项, 防止路径被当作选项;
- `{}`: 由 `find` 替换为完整参数, 不需要通过文本分词。

`-execdir` 在匹配对象所在子目录执行命令, 可减少部分路径竞态和选项注入风险, 但它改变工作目录, 并要求认真处理 `PATH`。本章把它作为安全边界和选择题, 不把它描述为万能保护。

⑤ [操作] 交互确认不是批量变更的完整验收

`-ok` 或 `-okdir` 会逐项询问确认, 适合少量临时操作, 但不能替代候选清单、审计记录和执行后验证。大量对象逐项输入 `y` 容易产生疲劳误判, 也不利于复现。

[Cheatsheet] AND 高于 OR, 复杂 OR 必须括号; 动作也参与逻辑; 预览优先 `-print/-printf`; 批量参数优先 `-exec ... {} +`, 逐个执行才使用 `\;`。

[知识专题] `locate` 与 `updatedb`: 快速结果为什么不等于实时事实

`locate` 通过名称数据库快速返回候选路径, 适合“我大概知道文件叫什么, 但不知道在哪”的发现阶段。它不遍历当前目录树, 因此速度快, 但证据强度低于实时查询。

① [知识点] `locate` 查询的是上次索引时的名称状态

```
locate logrotate.conf  
locate -i networkmanager.conf  
locate -n 5 passwd
```

常用选项:

- `-i`: 忽略大小写;
- `-n N`: 最多显示 `N` 个结果;
- 不同实现还可能提供只显示当前仍存在对象的选项, 使用前应查看本机 `locate --help`。

刚创建的文件可能尚未进入数据库, 已删除的文件也可能暂时仍在结果中。`locate` 未找到不能证明文件不存在。

② [知识点] `updatedb` 的排除规则决定哪些路径永远不被索引

管理员可运行:

```
sudo updatedb
```

这只说明发起了一次数据库更新, 不代表所有挂载点、文件系统和目录都被纳入。实现可能根据文件系统类型、挂载选项或配置中的排除路径跳过目录。数据库具体路径和定时更新方式具有实现和版本差异, 应通过本机软件包、手册页和配置确认, 不把旧版 `mlocate` 路径当成 RHEL 9 永久常量。

③ [诊断点] 候选路径必须回到实时证据

```
locate app.conf |  
head  
  
stat -- /candidate/path  
find /expected/root -type f -name 'app.conf'
```

诊断顺序:

```
locate 无结果或出现陈旧路径  
→ 确认命令实现和数据库状态  
→ 检查 updatedb 排除范围  
→ 必要时更新数据库  
→ 用 stat/find 确认当前对象
```

[Cheatsheet] `locate` 是名称索引, 不是实时遍历; `updatedb` 更新索引, 但受排除策略影响; 最终对象必须用 `stat` 或限定范围的 `find` 确认。

[知识专题] 正则表达式与 `grep`：选择记录而不是猜字符串

正则表达式描述文本模式，Shell glob 描述路径名称模式。两者共享少数字符，但语义不能互换。写 `grep` 前先确定要匹配固定文本，基本正则还是扩展正则，并用引用阻止 Shell 解释模式。

① [知识点] glob、固定字符串、BRE 和 ERE 是不同语言

场景	入口	示例
路径名称通配	Shell 或 <code>find -name glob</code>	<code>*.conf</code>
固定文本	<code>grep -F</code>	<code>server[1]</code>
基本正则 BRE	<code>grep</code>	<code>^root:</code>
扩展正则 ERE	<code>grep -E</code>	<code>^(sshd sudo):</code>

`grep -F` 不把 `.`、`[`、`*` 等视为正则元字符，搜索配置字面量时更安全。模式通常放在单引号中：

```
grep -F 'server[1]' file
grep -E '^(sshd|sudo):' file
```

② [知识点] 锚点、字符类和重复表达边界

常见 ERE：

- `^`、`$`：记录开头和结尾；
- `.`：任意单个字符；
- `*`、`+`、`?`：前一项重复 0 次以上、1 次以上、0 或 1 次；
- `{m,n}`：重复次数范围；
- `[...]`：字符集合；
- `[^...]`：否定字符集合；
- `(A|B)`：分组和选择；
- `[:space:]`、`[:digit:]`、`[:alpha:]`：POSIX 字符类。

匹配完整配置行时显式锚定：

```
grep -E '^[[:space:]]*Listen[[:space:]]+8080[[:space:]]*$' file
```

它不会误选 `# Listen 8080`、`Listen 80800` 或包含额外字段的行。

③ [操作] 控制输出形式，而不是默认打印所有匹配行

```
grep -nE 'PATTERN' file      # 匹配行和行号
grep -vE 'PATTERN' file      # 反选
grep -cE 'PATTERN' file      # 每个输入的匹配行数
grep -lE 'PATTERN' files...   # 只显示有匹配的文件名
grep -LE 'PATTERN' files...   # 只显示没有匹配的文件名
grep -qE 'PATTERN' file       # 静默判断
grep -oE 'PATTERN' file       # 只输出匹配片段
```

递归搜索：

```
grep -rE 'PATTERN' /etc/app
```

`-r` 和 `-R` 的符号链接处理边界不同，必须通过本机手册确认后再用于会跨链接的目录树。若已经用 `find` 精确限定文件类型和范围，通常不需要让 `grep` 再次自行遍历。

④ [知识点] 退出状态是三态证据

正常情况下：

- `0`：至少选择到一条记录；
- `1`：没有选择到记录；
- `2`：发生错误。

因此“无匹配”不是命令执行错误。脚本或流水线必须把 `1` 和 `2` 分开，尤其不能把错误文件、权限失败和真正无匹配混为一谈。`-q` 在发现匹配后可能提前退出，诊断输入错误时应谨慎使用。

⑤ [边界] locale 和二进制判断会改变结果

字符范围、大小写折叠和排序可能受 locale 影响。优先使用 POSIX 字符类；只有任务明确要求字节顺序时，才对局部命令设置 LC_ALL=C。grep 认为输入是二进制时可能只报告“binary file matches”；需要文本处理前先确认文件类型和内容，不把 -a 当成无条件默认。

[Cheatsheet] 字面量用 -F，ERE 用 -E；完整行用 ^...\$；输出行、文件名、数量或退出状态是不同接口；退出 1 是无匹配，2 才是错误。

[操作专题] cut、sort、uniq、tr 与 wc：小工具也要有明确输入合同

这些工具适合构造短而确定的记录流水线，但前提是输入结构明确。不要为了命令短而忽略记录分隔、字段分隔、排序规则和计数对象。

① [操作] 用 cut 提取稳定分隔字段

```
cut -d: -f1,3 /etc/passwd
cut -d: -f1-3 file
cut -c1-20 file
```

- -d CHAR：指定单字符字段分隔符；
- -f LIST：选择字段；
- -c LIST：按字符位置选择；
- -s：不输出没有分隔符的记录。

cut 不理解 CSV 引号、转义和嵌套分隔符；复杂结构应使用对应解析器。面向人的对齐输出中连续空格数量不稳定，也不适合作为 cut -d ' ' 的长期输入接口。

② [操作] 用 sort 明确排序键和比较方式

```
sort file
sort -n file
sort -t: -k3,3n -k1,1 /etc/passwd
LC_ALL=C sort -k2,2nr -k1,1 report.tsv
```

常用选项：

- -n：数字比较；
- -h：理解常见可读大小后缀；
- -r：逆序；
- -t CHAR：字段分隔符；
- -k START,END：排序键范围；
- -u：排序后只保留唯一记录。

sort 默认受 locale 排序规则影响。任务要求完全确定的机器顺序时，局部设置 LC_ALL=C，不要改变整个登录环境。

③ [操作] uniq 只折叠相邻重复记录

```
sort userlist.txt | uniq
sort userlist.txt | uniq -c
uniq -d sorted.txt
uniq -u sorted.txt
```

uniq 不会在整个未排序文件中任意寻找重复，它只比较相邻记录。先 sort 会改变原顺序；如果业务要求保持首次出现顺序，应使用 awk '!seen[\$0]++' 等显式状态方法，并说明内存和输入规模边界。

④ [操作] tr 处理字符集合，不处理字符串子串

```
tr '[:lower:]' '[:upper:]' < file
tr -d '\r' < windows.txt
tr -s '[:space:]' ' ' < file
```

tr -d 'root' 删除的是字符集合中的 r、o、t，不是删除单词 root。子串替换使用 sed 或 awk。tr 从标准输入读取数据，常位于管道中。

⑤ [操作] `wc` 的数字必须解释为明确对象

```
wc -l file      # 换行数量
wc -w file      # 按当前规则识别的单词数
wc -c file      # 字节数
wc -m file      # 字符数
```

最后一条记录没有换行时，`wc -l` 不会把它计为一个换行。因此“文件有多少业务记录”只有在记录合同为“一条记录一个换行且末尾完整”时才等于 `wc -l`。

[Cheatsheet] `cut` 适合稳定单字符分隔；`sort` 明确键、数字和 locale；`uniq` 只看相邻；`tr` 操作字符集合；`wc -l` 统计换行，不自动等于业务对象数。

[操作专题] `sed` 的选择和替换：先把结果写到标准输出

`sed` 逐条读取输入记录，在模式空间中执行脚本，然后默认输出处理后的记录。它适合地址选择、短替换和简单流编辑；原地修改是额外模式，不应成为首次试验的默认动作。

① [操作] 用地址选择记录

```
sed -n '10,20p' file
sed -n '/^BEGIN$/,/^END$/p' file
sed -n '/ERROR/p' file
```

- 单个行号选择一行；
- `ADDR1,ADDR2` 选择范围；
- `/REGEXP/` 选择匹配记录；
- `$` 表示最后一行；
- `-n` 关闭默认输出，`p` 只打印显式选中记录。

如果忘记 `-n` 又使用 `p`，匹配行通常会出现两次：一次来自默认输出，一次来自 `p` 命令。

② [操作] 使用 `s///` 替换并控制作用范围

```
sed -E 's/old/new/' file
sed -E 's/old/new/g' file
sed -E 's#^/opt/old#&/archive#' file
sed -E 's/^([:]+):([:]+)$/\2:\1/' file
```

- 默认只替换每条记录的第一次匹配；
- `g` 替换该记录内所有匹配；
- `&` 代表完整匹配文本；
- `\1`、`\2` 引用捕获组；
- 分隔符不必是 `/`，处理路径时可改用 `#` 或 `|`。

配置修改应使用锚点和字符类限制完整行，不要只写宽泛的 `s/8080/8081/`。

③ [操作] 多个脚本使用 `-e` 或脚本文件

```
sed -e '/^[[[:space:]]*#/d' -e '/^[[[:space:]]*$/d' file
sed -f cleanup.sed file
```

脚本开始变长、包含多条相互依赖规则时，应保存为可审查文件，而不是继续堆叠在一行中。复杂解析或跨记录状态通常更适合 `awk`。

④ [边界] `-i` 是文件替换操作，需要备份和元数据检查

```
sed -i.rhcsa05.bak -E 's/^Listen 8080$/Listen 8081/' file
```

带后缀会在同目录创建备份；不带后缀不生成备份。`sed -n -i` 如果脚本没有显式输出，可能把文件写空。原地编辑通常通过临时文件替换原路径，还可能影响硬链接关系、ACL、扩展属性或 SELinux 上下文，真实环境应按目标文件类型验证。

安全流程：

```
stdout 预览
→ diff 或样本核对
→ 建立候选清单
→ 使用明确备份后继续执行
→ 验证内容、备份和元数据
```

[Cheatsheet] `-n` 配合 `p` 做选择；`s///g` 控制同一记录内多次替换；`&` 是完整匹配，`\1` 是捕获组；首次尝试不使用 `-i`，批量原地修改必须保留备份。

[操作专题] `awk` 的记录、字段、条件和小型聚合

`awk` 最适合“每条记录都有稳定字段，我需要按条件选择、计算并格式化输出”的任务。本章只建立 RHCSA 和日常运维所需的记录模型、条件动作和小型聚合，不扩展为完整编程教材。

① [知识点] 默认输入模型是记录和字段

```
awk -F: '{print $1, $3}' /etc/passwd
```

常用内建状态：

- `$0`：当前完整记录；
- `$1`、`$2`…：当前字段；
- `NF`：当前记录字段数；
- `NR`：所有输入累计记录号；
- `FNR`：当前文件内记录号；
- `FS`：输入字段分隔符；
- `OFS`：输出字段分隔符；
- `RS`、`ORS`：输入和输出记录分隔符。

```
awk -F: -v OFS='\t' '{print NR, $1, $3}' /etc/passwd
```

② [操作] 使用 `pattern { action }` 选择记录

```
awk -F: '$3 >= 1000 {print $1, $3}' /etc/passwd
awk -F: '$2 == "sshd" && $4 == "FAIL" {print $3}' events.log
awk '$0 ~ /ERROR/ {print NR, $0}' app.log
```

模式为真时执行动作；省略动作时默认打印整条记录；省略模式时对每条记录执行动作。正则匹配使用 `~`，否定匹配使用 `!~`。

字段数也是重要验证条件：

```
awk -F: 'NF != 5 {print FNR, $0 > "/dev/stderr"}' events.log
```

③ [操作] 使用变量和 `-v` 传递外部参数

```
awk -F: -v user="$target_user" '$1 == user {print $0}' /etc/passwd
```

`-v` 在 `awk` 开始处理前赋值，避免把 Shell 内容直接拼入 `awk` 源码。Shell 变量仍需正确引用。对来自不可信输入的动态正则，需要额外评估其是否应作为固定字符串处理。

④ [操作] 计数、求和和关联数组

```
awk -F: '
$2 == "sshd" && $4 == "FAIL" {
    count[$3]++
    bytes[$3] += $5
}
END {
    for (user in count)
        print user, count[user], bytes[user]
}
' events.log
```

`awk` 关联数组按键聚合，但遍历数组的顺序未必符合业务要求。需要确定性顺序时，把结果交给 `sort`，不要假设数组天然按用户名或插入顺序输出。

⑤ [操作] 用 `printf` 固定输出合同

```
awk -F: 'BEGIN {OFS="\t"}
{printf "%s\t%d\n", $1, $3}
' /etc/passwd
```

`print` 自动使用 `OFS/ORS`，`printf` 由格式字符串控制。后续还要排序或读取的结果，应明确使用制表符或其他稳定分隔符，并避免混入说明文字。

[Cheatsheet] `$0` 是记录，`$1` 是字段，`NF` 是字段数，`NR/FNR` 是记录号；条件写在动作前；外部变量用 `-v`；聚合数组输出顺序不确定，排序交给 `sort`。

[操作专题] 特殊文件名、NUL 和 `xargs`：保持每个路径都是一个参数

换行分隔的路径列表只在你能够证明文件名不含换行时才可靠。系统级批量修改默认不能做这个假设。NUL 协议解决记录边界问题，`xargs` 或 `find -exec` 解决参数构造问题。

① [知识点] 空格不是最难的文件名，换行才会破坏“每行一个路径”

合法文件名可以包含：

- 空格和制表符；
- 单引号、双引号和反斜杠；
- 通配符字符；
- 换行；
- 前导连字符。

不能包含的是 `/`（目录分隔符）和 NUL。 `for f in $(find ...)`、`find | while read` 的默认形式、普通 `xargs` 都可能因空白分词或反斜杠处理破坏名称。

② [操作] 生产端和消费端必须使用同一种记录协议

```
find /srv -type f -print0 |
xargs -0 -r stat --
```

- `-print0`：每个路径后输出 NUL；
- `xargs -0`：只把 NUL 视为项目分隔符；
- `-r`：输入为空时不运行命令；
- `--`：结束子命令选项。

内容筛选后继续保持 NUL：

```
find /srv -type f -name '*.conf' -print0 |
xargs -0 -r grep -lZ -E '^Listen[[:space:]]+8080$' -- |
xargs -0 -r stat --
```

这里 `grep -Z` 让文件名输出以 NUL 结束。管道的每一段都必须检查是否保持了同一边界。

③ [操作] `xargs` 的批量大小和替换模式

```
xargs -0 -r -n 20 command --
xargs -0 -r -I '{}' command -- '{}'
```

- `-n N`：每次最多传 `N` 个项目；
- `-s SIZE`：限制构造命令行大小；
- `-I`：按输入记录替换占位符，通常意味着逐条或小批执行；
- `-P`：并行执行，可能打乱顺序并放大副作用，本章不把它作为修改任务的默认答案。

不要同时把 `-I` 当成高效批量接口。需要把所有路径附加到命令末尾时，普通 `xargs -0` 或 `-exec ... {} +` 更直接。

④ [比较] `-exec ... {} +` 与 `xargs -0` 的选择

场景	推荐入口
候选直接来自同一条 <code>find</code> ，子命令可接收多个路径	<code>-exec command -- {} +</code>
候选要经过 <code>grep -Z</code> 等多个 NUL 处理阶段	<code>xargs -0 -r</code>
需要在每个对象所在目录执行	评估 <code>-execdir</code>

场景	推荐入口
需要逐项确认	少量对象可用 <code>-ok/-okdir</code>
需要并行修改	默认不做，先证明幂等性和顺序无关

⑤ [边界] NUL 解决名称分隔，但不消除目录树竞态

从预览到执行之间，文件可能被删除、替换、改名或更改所有者。对高风险目录和不可信用户可写目录，必须评估：

- 候选清单生成时间
- 执行时对象身份是否仍相同
- 子命令是否会跟随符号链接
- 是否需要更在更受控的目录或维护窗口操作

[Cheatsheet] 任意文件名集合使用 NUL；`-print0`、`grep -Z`、`xargs -0` 必须成对；空输入加 `-r`；直接来自 `find` 时优先 `-exec ... {} +`；NUL 不解决 TOCTOU 竞态。

[诊断专题] 结果为空、过多、错序或批量动作异常时怎样推进

文本流水线故障最容易诱发“继续加一个管道试试”。稳定诊断应把集合选择、记录转换和副作用分层，找到第一处证据偏离预期的位置。

① [诊断] find 结果为空或过少

- 症状：没有候选或明显少于预期
- 当前证据：起点、stderr、退出状态、最简单 `find` 输出
- 假设：起点错误、无遍历权限、模式被 Shell 展开、时间/大小边界取整
- 区分证据：移除动作和后半条件，逐个添加测试
- 最小修复：修正起点、引用、权限或边界
- 再验证：数量、边界对象、应纳入样本

```
find /srv -maxdepth 1 -print
find /srv -type f -print
find /srv -type f -name '*.conf' -print
```

② [诊断] 结果过多或包含错误类型

优先检查：

- OR 表达式是否缺少括号；
- `-name` 与 `-path` 是否混淆；
- 是否忘记 `-type f`；
- 是否因 `-L` 跟随链接进入额外目录；
- `grep` 是否没有锚定完整行；
- 反选 `-v` 是否作用在错误阶段。

验证不能只看正例，还要主动检查一个应被排除的反例。

③ [诊断] 排序、去重或计数不符合直觉

- 重复仍存在
- 检查是否只是非相邻重复
- 检查大小写、尾随空白和不可见字符
- 用 `sed -n l` 或 `od` 观察记录
- 决定是否先标准化、排序，再 `uniq`

数字错序通常来自文本比较；字段键过宽会把后续字段也纳入比较；locale 可能改变字符顺序。`wc -l` 少一条时检查最后一条记录是否缺少换行。

④ [诊断] 批量动作只处理部分文件

保留以下证据：

- 原始 NUL 候选清单；
- 可读转义预览；
- 子命令 `stderr`；
- 批次数和退出状态；
- 修改后重新生成的命中清单。

可能原因包括命令行被拆批、某批子命令失败、文件在执行前变化、权限不同、文件内容不再匹配。不要仅重复执行整个命令；先定位未完成对象的共同特征。

⑤ [验证] 采用“集合、内容、反例、备份、错误”验收矩阵

集合：执行对象与预览对象一致
内容：每个目标达到目标文本状态
反例：范围外对象保持原状
备份：需要回退的原始内容存在
错误：`stderr` 和退出状态没有被管道吞掉

[Cheatsheet] 诊断时移除副作用，从最简单查询逐步加条件；每一步既查正例也查反例；批量任务保留原始候选清单，不能只靠最终输出回忆处理了谁。

[经典任务] 安全筛选并批量修改配置文件

环境与当前状态

系统中存在目录：

`/srv/app/conf.d/`

目录树中包含普通 `.conf` 文件、子目录、符号链接以及名称中含空格、换行或前导连字符的文件。部分普通文件由 `appsvc` 用户拥有。文件内容可能包含：

```
Listen 8080
  Listen 8080
# Listen 8080
Listen 80800
```

目标终态

只处理同时满足以下条件的对象：

1. 位于 `/srv/app/conf.d/` 目录树；
2. 不跨入其他文件系统；
3. 是普通文件，不跟随并修改符号链接目标；
4. 所有者为 `appsvc`；
5. `basename` 以 `.conf` 结尾；
6. 小于 1 MiB；
7. 最近 7 个完整 24 小时计数范围内修改；
8. 包含完整有效配置行 `Listen 8080`，允许行首尾空白和字段间多个空白。

将完整有效行统一替换为：

```
Listen 8081
```

限制条件

- 不使用 `for f in $(find ...)`；
- 任意文件名必须保持完整；
- 批量修改前必须保存机器可读 NUL 清单和可读预览；
- 空集合时不得执行 `sed`；
- 每个修改文件保留 `.rhcsa05.bak` 备份；
- 不修改注释行、`Listen 80800` 或范围外文件；
- 不声称完成 live test，答案只给出推荐命令和静态验收方法。

验收证据

评分对象	证据
静态候选集合	NUL 清单、转义预览、数量
内容候选集合	只包含完整有效旧行的文件
特殊文件名	每个路径保持一个 NUL 记录
新内容	每个候选至少存在目标新行

评分对象	证据
旧内容	候选中不再存在完整有效旧行
备份	每个修改文件旁存在指定后缀备份
反例	注释、相似行和范围外文件未变
错误	stderr、子命令失败和空输入均可区分

[参考解答] 安全筛选并批量修改配置文件

以下命令是静态设计，未在本会话控制的 RHEL 9 虚拟机中执行。真实环境应先在副本或维护窗口验证。

① 调查：建立基线并确认目录边界

```
find /srv/app/conf.d -xdev -maxdepth 2 \
-printf '%y\t%u:%g\t%s\t%p\n' |
head -n 50
```

目的不是立刻得到最终集合，而是确认起点、层次、对象类型和所有者分布。高风险目录中还应检查是否存在其他用户可写目录和符号链接。

② 只按元数据生成静态候选 NUL 清单

```
find /srv/app/conf.d \
-xdev \
-type f \
-user appsvc \
-name '*.conf' \
-size -1048576c \
-mtime -7 \
-print0 \
> /root/rhcsa05-static-candidates.nul
```

参数含义：

- `-xdev`：不下降到其他文件系统；
- `-type f`：只处理普通文件；默认 `-P` 不跟随遍历中的符号链接；
- `-size -1048576c`：严格小于 1,048,576 字节；使用 `c` 避免 MiB 单位向上取整造成边界误判；
- `-mtime -7`：最近 7 个完整 24 小时计数范围；
- `-print0`：保存任意文件名。

空集合数量可用不会破坏名称的方式计算：

```
xargs -0 -r -n 1 printf '.\n' \
< /root/rhcsa05-static-candidates.nul |
wc -l
```

可读预览使用转义表示，不把它重新作为机器输入：

```
xargs -0 -r -n 1 printf '%q\n' \
< /root/rhcsa05-static-candidates.nul \
> /root/rhcsa05-static-preview.txt

less /root/rhcsa05-static-preview.txt
```

③ 按内容继续筛选并保持 NUL 协议

```
xargs -0 -r \
grep -LZ -E \
'^[[:space:]]*Listen[[:space:]]+0000[[:space:]]*$' -- \
< /root/rhcsa05-static-candidates.nul \
> /root/rhcsa05-content-candidates.nul
```

这里绝不能使用只搜索 `0000` 的宽泛模式。`grep -LZ` 每个匹配文件只输出一次，并以 NUL 终止名称。

再次生成可读预览和数量：


```
xargs -0 -r -n 1 printf '%q\n' \
< /root/rhcsa05-content-candidates.nul \
> /root/rhcsa05-content-preview.txt

xargs -0 -r -n 1 printf '\.\\n' \
< /root/rhcsa05-content-candidates.nul \
wc -l
```

④ 在不修改文件的情况下预览替换结果

抽取一到数个候选。先运行不带 `-i` 的 `sed`：

```
xargs -0 -r -n 1 \
sed -n -E \
'/^[[[:space:]]*Listen[[[:space:]]+8080[[[:space:]]*$/p' \
-- \
< /root/rhcsa05-content-candidates.nul
```

随后对代表性文件比较转换前后：

```
sed -E \
's/^[[[:space:]]*Listen[[[:space:]]+8080[[[:space:]]*$/Listen 8081/' \
-- /path/to/representative.conf
```

确认不会改变注释、其他端口或额外字段后再执行。

⑤ 批量修改并保留备份

```
xargs -0 -r \
sed -i.rhcsa05.bak -E \
's/^[[[:space:]]*Listen[[[:space:]]+8080[[[:space:]]*$/Listen 8081/' \
-- \
< /root/rhcsa05-content-candidates.nul
```

`-r` 保证候选为空时不运行 `sed`。NUL 清单是执行对象的审计真源，不要在执行前改成普通换行列表。

⑥ 分层验证

验证每个候选包含新行：

```
xargs -0 -r \
grep -LZ -E \
'^[[[:space:]]*Listen[[[:space:]]+8081[[[:space:]]*$/ -- \
< /root/rhcsa05-content-candidates.nul \
> /root/rhcsa05-verified-new.nul

cmp /root/rhcsa05-content-candidates.nul \
    /root/rhcsa05-verified-new.nul
```

串行 `xargs` 下，若每个输入文件都输出一次且顺序未改变，两份 NUL 清单应一致。真实环境还应检查 `grep/xargs` 的 `stderr` 和退出状态。

检查旧行残留：

```
xargs -0 -r \
awk '
/^[[[:space:]]*Listen[[[:space:]]+8080[[[:space:]]*$/ {
    printf "%s%c", FILENAME, 0
    nextfile
}
' \
-- \
< /root/rhcsa05-content-candidates.nul \
> /root/rhcsa05-old-remains.nul

test ! -s /root/rhcsa05-old-remains.nul
```

检查备份：

```
xargs -0 -r -n 1 \
  sh -c 'test -f "$1.rhcsa05.bak" || { printf "missing backup: %q\n" "$1" >&2; exit 1; }' \
  sh \
  < /root/rhcsa05-content-candidates.nul
```

检查范围外反例：对预先记录的注释行、Listen 80800 和非 appsvc 文件重新计算内容校验或精确搜索，确认未变化。

⑦ 典型错误

- `find ... -name *.conf`：模式可能被当前 Shell 提前展开；
- `find ... | xargs sed -i`：空白和换行会拆坏文件名；
- `grep -l '8080'`：会选中注释和相似值；
- 预览一套条件、执行时重写另一套条件：集合可能漂移；
- `sed -i` 不带备份：失去最小回退材料；
- 只验证新行存在：不能证明旧行已消失，也不能证明范围外对象未变。

[经典任务] 从结构化事件日志生成可审计聚合报告

环境与输入合同

输入文件：

```
/srv/audit/events.log
```

有效记录使用冒号分隔五个字段：

```
timestamp:service:user:result:bytes
```

文件还可能包含注释行、空行、字段数错误、非数字字节字段和其他服务记录。

目标终态

1. 注释和空行不进入统计；
2. 非 5 字段或字节字段非数字的记录写入异常清单，并带原始行号；
3. 只统计服务为 `sshd` 或 `sudo` 且结果为 `FAIL` 的有效记录；
4. 按用户输出失败次数和字节总量；
5. 先按失败次数数字降序，再按用户名确定性排序；
6. 报告使用制表符分隔；
7. 单独输出总失败记录数和总字节数；
8. 原始文件不得修改。

验收关系

```
所有非注释、非空输入
= 结构有效记录 + 异常记录
```

```
总失败记录数
= 各用户失败次数之和
```

```
总失败字节数
= 各用户字节总量之和
```

[参考解答] 从结构化事件日志生成可审计聚合报告

① 建立输出文件并避免残留旧结果

```
report=/srv/audit/failures.tsv
invalid=/srv/audit/events.invalid
summary=/srv/audit/events.summary

: > "$report"
: > "$invalid"
: > "$summary"
```

显式清空可避免“本次没有异常记录，但旧异常文件仍然存在”的误判。

② 使用 `awk` 验证结构、筛选和聚合

```
awk -F: \
-v OFS='\t' \
-v invalid="$invalid" \
-v summary="$summary" '
/^[[[:space:]]*#/ || /^[[[:space:]]*$/ { next }

NF != 5 || $5 !~ /^[[[:digit:]]+$/ {
    print FNR, $0 > invalid
    next
}

($2 == "sshd" || $2 == "sudo") && $4 == "FAIL" {
    count[$3]++
    bytes[$3] += $5
    total_count++
    total_bytes += $5
}

END {
    for (user in count)
        print user, count[user], bytes[user]

    print "TOTAL", total_count + 0, total_bytes + 0 > summary
}
' /srv/audit/events.log |
LC_ALL=C sort -k2,2nr -k1,1 > "$report"
```

关键语义：

- `FS=`：明确输入字段；
- `NF != 5` 和数字正则保护聚合；
- `OFS='\t'` 给下游稳定字段合同；
- 数组遍历顺序不可信，因此交给 `sort`；
- `LC_ALL=C` 只作用于本次排序；
- `+ 0` 让没有命中时仍输出数字 0。

③ 验证报告结构和排序

```
awk -F '\t' 'NF != 3 {print FNR, $0 > "/dev/stderr"; bad=1} END {exit bad}' \
"$report"

head "$report"
tail "$report"
cat "$summary"
cat "$invalid"
```

④ 验证汇总恒等式

```
awk -F '\t' '
{count += $2; bytes += $3}
END {print "TOTAL", count + 0, bytes + 0}
' "$report"
```

将此输出与 `events.summary` 比较。异常清单中的行号应能回到原始文件抽查。

⑤ 典型错误

- 使用 `cut -d: -f...` 后忽略字段数异常；
- 把 `bytes` 作为文本排序或拼接；
- 假设 `for (key in array)` 已按键排序；
- 直接把说明标题写入 TSV，破坏下游字段合同；
- 没有清空异常文件，导致旧错误混入本次结果；
- 只检查报告非空，没有验证总数恒等式和反例。

[本章收束] 把命令链升级为可证明的数据处理流程

本章建立的不是一组孤立命令，而是一条可以反复使用的工作方法：先把任务改写为对象条件，再固定并审查候选集合；只有记录和字段合同明确后，才执行转换或批量动作；最后用独立查询证明目标达到、旧状态消失、反例未变且错误没有被吞掉。

① [操作决策] 一条可执行的工作方法

限定搜索空间

→ 把自然语言要求翻译为对象谓词

→ 只输出候选并检查 `stderr`

→ 检查数量、样本、边界对象和反例

→ 明确记录与字段分隔符

→ 选择固定字符串、正则或字段条件

→ 使用 `-exec ... {} +` 或完整 NUL 协议构造 `argv`

→ 执行最小副作用

→ 重新查询新状态、旧状态、反例、备份和错误流

② [验证] 章末检查清单

- ☐ 起点、深度、文件系统和链接策略已经明确；
- ☐ 文件对象条件与内容条件没有混用；
- ☐ `glob`、固定字符串、BRE 和 ERE 选择正确；
- ☐ 记录边界先于字段边界确定；
- ☐ 任意文件名没有通过空白分词或命令替换传递；
- ☐ 预览集合和执行集合来自同一筛选真源；
- ☐ 空集合不会意外执行批量命令；
- ☐ 新状态、旧状态、范围外反例和错误流均已检查；
- ☐ 没有把命令退出为零扩大为业务终态正确；
- ☐ 需要实机确认的版本或元数据行为已记录，而不是伪造输出。

③ [知识点] 主要判断表

需求或症状	首选入口	关键边界	下一层证据
实时选择目录树对象	<code>find</code>	起点、权限、深度、链接、表达式	<code>-printf</code> 、 <code>stderr</code> 、 <code>stat</code> 抽查
快速按名称找候选	<code>locate</code>	索引可能过期或排除路径	<code>stat</code> 或限定范围的 <code>find</code>
精确查找字面文本	<code>grep -F</code>	不需要正则时避免误解	行号、文件名、计数与反例
按正则选择记录	<code>grep -E</code>	引用、锚点、字符类、locale	代表性匹配和退出状态
固定分隔字段提取	<code>cut</code>	只能依赖稳定单字符分隔符	字段数抽查
条件、计算或聚合	<code>awk</code>	<code>FS</code> 、 <code>NF</code> 、数字字段、数组无序	汇总恒等式与异常清单
排序后统计重复	<code>sort uniq -c</code>	<code>uniq</code> 只处理相邻重复，排序改变顺序	首尾样本与总数
选择后直接批量调用	<code>find ... -exec ... {} +</code>	候选来自 <code>find</code> ，参数通常追加在末尾	子命令状态与终态查询
任意上游传递路径	<code>... -print0 xargs -0 -r ...</code>	两端都必须使用 NUL	转义预览、空输入与批次检查
修改文本文件	<code>sed / awk</code>	先 <code>stdout</code> 预览，再备份修改	新旧模式、备份和反例

④ [边界] 本章能证明什么，不能证明什么

本章的查询和验证可以静态说明推荐命令、参数语义、数据边界与验收路径，但当前会话没有可控 RHEL 9 虚拟机，因此没有声称真实执行过这些任务。尤其是 `locate` 的具体实现与数据库路径、`sed -i` 对 ACL/SELinux 标签/硬链接关系的实际影响，以及大批量 `xargs` 的拆批和失败传播，需要在目标 RHEL 9 环境中再次观察。

⑤ [知识点] 向下一章交接

本章交付给第 06 章的是一个已经限定、预览并验证过的文件集合。下一章会讨论怎样复制、归档、压缩或远程传输这些对象，并继续验证目录结构、元数据、压缩格式和远端终态。本章不提前展开 `cp`、`tar`、`scp` 或 `rsync`；但下一章应复用这里建立的安全入口：不要在未经审查的路径集合上直接执行复制或归档。

本章总 Cheatsheet

```
# 实时查找与审计预览
find START TESTS -print
find START TESTS -printf 'FORMAT'

# 复杂逻辑
find START TESTS -type f \( -name '*.conf' -o -name '*.ini' \) -print

# 直接安全批量传参
find START TESTS -exec command -- {} +

# NUL 流水线
find START TESTS -print0 | xargs -0 -r command --

# 固定字符串与正则选择
grep -F 'literal text' file
grep -E '^[[:space:]]*KEY[[:space:]]+VALUE[[:space:]]*$', file

# 字段与聚合
awk -F: -v OFS='\t' 'CONDITION {print $1, $3}' file |
LC_ALL=C sort -k2,2n

# sed 先预览, 后备份修改
sed -E 's/OLD/NEW/' file
sed -i.bak -E 's/OLD/NEW/' file
```

::: {cover}

RHEL 9 · RHCSA 实操讲义

06

复制、归档、压缩与远程传输

从源集合到目标集合：把层级、成员、属性、远端路径和完整性证据放进同一条操作链。

对象模型操作语义验证诊断经典任务

大字号阅读版

...

... {reading-nav}

本章阅读导航

先抓住一条主线：文件操作不是“把一个名字搬过去”，而是把源对象或成员集合，按照明确的层级、属性和删除边界，变换为可验收的目标集合。

- 01确认源集合对象、成员、隐藏项、链接
- 02定义目标层级目录本身还是目录内容
- 03选择变换方式复制、部署、归档、传输、同步
- 04控制风险边界覆盖、属性、删除、远端路径
- 05执行最小操作先基线或 dry-run，再改变状态
- 06分层验收成员、路径、字节、元数据

专题地图

- 知识专题 数据集合、路径层级与元数据
- 操作专题 ``cp` / `mv`` 与覆盖边界
- 操作专题 ``install`` 部署文件
- 知识专题 ``tar`` 成员、路径与元数据
- 操作专题 压缩、查看和安全提取
- 操作专题 ``scp` / `sftp`` 远端路径
- 操作专题 ``rsync`` 增量同步
- 操作专题 ``sha256sum`` 与分层验收
- 诊断专题 从症状推进到下一条证据
- 经典任务 归档试提取与远端同步

阅读时持续回答

1. 源是一个对象，还是一个成员集合？
2. 目标存在时，层级会怎样改变？
3. 归档、压缩、传输、同步分别改变哪一层？
4. 远端相对路径由谁解释？
5. ``rsync`` 源末尾斜杠改变了什么？
6. 哪些属性必须保留，执行身份是否允许？
7. 命令成功后，还缺哪一层证据？

章节边界：SSH 密钥与 ``sshd`` 留给第 20 章；NFS 留给第 26 章；本章不把 ``rsync`` 扩展为完整备份体系。

...

第 06 章 • 正文

系统管理员面对的并不是孤立的“文件”，而是一个由路径名称、目录层级、普通文件字节、链接和元数据共同组成的数据集合。把这个集合复制到同一台主机的另一个位置、封装为归档、压缩后交付，或者同步到远端，看似都叫“搬文件”，实际改变的对象、风险和验收方式并不相同。

最常见的误判并不是命令拼错，而是对象边界理解错了：复制了目录本身而不是目录内容；归档里多出或少了一层目录；远端相对路径被解释到意外位置；`rsync` 因一个末尾斜杠改变了同步根；摘要一致却误以为权限、所有者和时间戳也一定一致；命令退出为零，却没有检查目标端实际状态。

本章以“源集合 → 变换或传输 → 目标集合 → 分层证据”为主线。前一章已经建立查找和筛选源对象的方法，本章从已经确定的源集合开始；下一章将进入用户、组和账号生命周期，本章只把现有执行身份作为属性和远端路径的一项约束。

⋮ {concept-box} 概念 源对象与目标对象 是一次文件操作的两端。源对象回答“读取什么”，目标对象回答“最终应出现在哪里以及以什么层级出现”。目标是否已存在、目标是文件还是目录，会改变同一条命令的结果，所以在覆盖、移动或同步前，应先用 `stat`、`test` 或目录树证据确认两端，而不是只凭命令外形推测。⋮

⋮ {concept-box} 概念 归档成员 (archive member) 是 `tar` 归档内部可列出、可提取的名称和对象。创建归档时，本地路径会被转换为成员名称；提取时，成员名称决定恢复出的顶层和目录结构。归档文件能够打开，只证明容器可读，不证明成员前缀、隐藏项和恢复位置符合任务要求。⋮

⋮ {concept-box} 概念 压缩流 (compressed stream) 是对一个文件或字节流进行编码后的表示。`gzip`、`bzip2` 和 `xz` 可以压缩 `tar` 产生的归档流，但它们本身不负责把多个目录项组织成成员集合。因而“归档”和“压缩”必须分开理解：先决定成员和层级，再决定是否以及如何压缩。⋮

⋮ {concept-box} 概念 远端路径 (remote path) 由远端登录身份和远端文件系统解释。`host:relative/path` 不是本地当前目录的延伸；它通常从远端用户的起始目录解释。可靠做法是显式写出用户和绝对目标路径，或在 `sftp` 中先用 `pwd` 与 `lpwd` 分清两端。⋮

⋮ {concept-box} 概念 增量同步 (incremental synchronization) 是比较源集合与目标集合后，只传输或调整被判定为不同的对象。`rsync` 的源末尾斜杠决定同步根，`--delete` 决定是否清理目标额外成员；它们改变的不是显示格式，而是目标树本身，因此必须在执行前通过 `dry-run` 审查。⋮

⋮ {concept-box} 概念 完整性校验 (integrity verification) 用摘要判断被比较字节是否一致。`sha256sum` 相同可以证明指定文件的字节相同，却不能证明路径、类型、模式、所有者、时间戳、ACL 或安全上下文相同。一个完整验收必须把成员、层级、字节和元数据证据组合起来。⋮

⋮ {semantic-map}

操作语义速查

先建立关键接口地图，再进入正文细节。以下命令组覆盖本章最常用的查询、修改和验证入口；参数采用纵向定义，避免把语义压成一长行。

cp / mv

SYNOPSIS

```
cp [OPTION]... SOURCE DEST
cp [OPTION]... SOURCE... DIRECTORY
mv [OPTION]... SOURCE DEST
mv [OPTION]... SOURCE... DIRECTORY
```

cp 创建副本并保留源；mv 改变名称或位置。两者都受目标是否存在、目标类型和覆盖策略影响。

重要参数 / 形式

cp -r / cp -R

递归复制目录树。

cp -p

请求保留模式、所有权和时间戳；实际结果仍受权限和文件系统限制。

cp -a

归档式递归复制，适合保留链接和常用属性；不等于已证明所有属性落地。

SOURCE/.

明确复制已有源目录的全部成员，包括隐藏成员，进入既定目标目录。

install

SYNOPSIS

```
install [OPTION]... SOURCE DEST
install -d [OPTION]... DIRECTORY...
```

把准备好的文件部署到目标位置，并在同一操作中表达目标目录、模式、所有者和组。

重要参数 / 形式

-D

创建目标文件缺失的父目录。

-d

创建目录对象，而不是复制普通文件。

-m MODE

显式设置目标模式。

-o OWNER / -g GROUP

设置目标所有者或组，需要相应权限。

tar

SYNOPSIS

```
tar -cf ARCHIVE MEMBERS...
tar -tf ARCHIVE
tar -xf ARCHIVE [-C DIRECTORY]
```

把多个成员及其元数据组织为归档，或非破坏性查看成员，再提取到受控目录。

重要参数 / 形式

-c

创建归档。

-t

列出成员，不提取。

-x

提取成员。

`-f` ARCHIVE

指定归档文件；归档名紧随 `-f`。

`-C` DIRECTORY

改变后续路径参数的解释位置，具有顺序语义。

`-z` / `-j` / `-J`

分别选择 `gzip`、`bzip2`、`xz` 压缩。

`gzip` / `bzip2` / `xz`

SYNOPSIS

```
gzip [OPTION]... FILE...
bzip2 [OPTION]... FILE...
xz [OPTION]... FILE...
```

压缩单个文件或字节流；目录树需要先通过 `tar` 形成成员集合。

重要参数 / 形式

`-d`

解压。

`-k`

保留输入文件，避免默认替换原文件。

`file` ARCHIVE

检查实际格式；后缀只是命名约定。

`scp` / `sftp`

SYNOPSIS

```
scp [OPTION]... SOURCE... TARGET
sftp [user@]host
```

通过既有 SSH 通道复制或交互传输。本章只处理客户端路径语义，不展开密钥和 `sshd` 配置。

重要参数 / 形式

`[user@]host:path`

远端操作数：相对路径由远端登录语境解释。

`scp -r`

递归复制目录。

`scp -p`

请求保留修改时间、访问时间和模式。

`pwd` / `lpwd`

在 `sftp` 中分别查看远端和本地工作目录。

`rsync`

SYNOPSIS

```
rsync [OPTION]... SOURCE... DEST
rsync [OPTION]... SOURCE... [user@]host:DEST
```

比较源集合与目标集合并同步差异。源末尾斜杠和删除选项直接改变目标树。

重要参数 / 形式

`-a`

归档模式，常用组合为递归并保留链接、模式、时间、所有者、组和设备信息；不包含 `-HAX`。

`-v`

输出较详细的处理信息。

`-n` / `--dry-run`

只预览，不改变目标。

`-i`

逐项显示变化，适合审查计划。

`--delete`

删除接收端的额外成员，只用于题目明确要求镜像时。

`SOURCE/` 与 `SOURCE`

前者同步目录内容；后者通常把源目录本身放入目标。

`sha256sum`

SYNOPSIS

```
sha256sum [OPTION]... FILE...
sha256sum -c CHECKSUM_FILE
```

为普通文件字节建立摘要证据，或按清单检查；不能代替路径、成员和元数据验收。

重要参数 / 形式

`sha256sum FILE`

计算文件摘要。

`sha256sum -c SHA256SUMS`

按清单检查文件字节。

`stat PATH`

与摘要配合，检查类型、模式、所有者和时间戳。

...

[知识专题] 数据集、路径层级与元数据：复制的对象到底是什么

复制或传输前，最顺的切入点不是立刻选择 `cp` 还是 `rsync`，而是把目标改写成可观察的状态：源集合包含什么，目标应该出现什么，是否允许保留额外成员，需要保留哪些元数据，以及哪类证据才足以验收。只有先确定这些问题，目录末尾斜杠、归档成员前缀和远端路径才有明确含义。

① [知识点] 字节、名称与元数据是三个独立状态维度

普通文件最直观的是字节内容，但系统管理任务通常还关心：

- 路径名称和目录层级；
- 文件类型，是普通文件、目录还是符号链接；
- 访问模式、所有者和组；
- 修改时间等时间戳；
- ACL、扩展属性和 SELinux 上下文；
- 硬链接关系。

`sha256sum` 相同不代表这些维度相同。反过来，两个文件的大小和时间相同，也不能独立证明字节相同。验收时应根据题目明确要求选择证据，而不是把一个局部指标扩大成“完全一致”。

② [知识点] 目录本身与目录内容不是同一个复制目标

假设源为 `/srv/source`，其中有 `a.conf` 和隐藏成员 `.env`。常见意图有两种：

```
复制目录对象：目标中形成 source/a.conf 和 source/.env
复制目录内容：a.conf 和 .env 直接进入既定目标目录
```

`cp`、`tar` 和 `rsync` 对源操作数的解释方式不同，但共同原则是：不要用模糊通配符代替明确的集合边界。`/srv/source/*` 通常漏掉点开头的成员；复制全部内容时，`/srv/source/.` 比 `*` 更能直接表达意图。

③ [知识点] 目标是否存在会改变结果层级

执行前至少回答：

```
目标不存在？
目标是普通文件？
目标是已有目录？
目标目录中是否已有同名成员？
```

例如把一个文件复制到不存在的路径会创建该路径；复制到已有目录会在目录中使用源基名。复制目录时，目标目录是否已经存在可能决定是创建目标目录，还是在目标目录内再形成一层源目录。稳定做法是先用 `stat` 或 `test` 建立目标基线，再根据预期树形结果选择命令。

④ [知识点] 命令成功只是执行状态，不是交付终态

退出状态为零通常说明命令按自身规则完成，但不能证明：

- 隐藏成员已包含；
- 归档顶层正确；
- 远端文件落在预期目录；
- 所有者和时间戳得到保留；
- 目标没有意外额外成员；
- `--delete` 没有超出授权范围。

因此每个操作必须预先写出验收证据，而不是执行后再凭感觉挑几条命令。

⑤ [知识点] “保留属性”表示操作意图，最终仍受身份和文件系统约束

`cp -a`、`scp -p`、`rsync -a` 或 `tar` 的保留选项都表达一种保留策略，但实际能否写入所有者、组、ACL、扩展属性或安全上下文，还取决于：

- 目标端执行身份；
- 目标文件系统是否支持该属性；
- 中间协议和工具是否携带该属性；
- 提取时的用户和 `umask`；
- 是否显式启用了 ACL、`xattr` 或 SELinux 元数据支持。

因此参数说明应写成“请求保留哪些属性”，验证说明应写成“目标端实际是什么”。

[Cheatsheet] 先拆分字节、层级和元数据；复制目录前确定是目录本身还是成员集合；目标存在状态会改变结果；退出状态只证明局部；保留参数不替代目标端 `stat`、成员检查和摘要检查。

[操作专题] 使用 `cp` 与 `mv` 安全复制、改名和移动

`cp` 和 `mv` 都接受源与目标，但它们改变的对象不同。`cp` 新建一个副本，源仍然存在；`mv` 改变名称或位置，成功后通常不再保留原路径。操作前应先确认源类型、目标类型、覆盖风险和需要保留的属性，操作后则分别检查两端。

① [操作] 使用 `cp` 复制单个文件

作用对象：一个普通文件及其目标路径。

基本形式：

```
cp SOURCE DESTINATION
cp SOURCE... DIRECTORY
```

单个源和不存在的目标路径会创建一个新文件；多个源必须以目录作为目标。复制前可用：

```
stat -- SOURCE DESTINATION 2>/dev/null
```

建立源和目标基线。若目标已有同名文件，默认行为通常是覆盖目标内容，因此高风险位置应先备份或使用交互确认，而不是把强制覆盖作为默认操作。

验证：

```
stat -- DESTINATION
sha256sum -- SOURCE DESTINATION
```

第一条确认对象和元数据，第二条只确认字节。

② [操作] 递归复制目录并选择属性策略

普通递归复制可使用 `cp -R` 或 `cp -r`。需要尽可能保留链接、模式、所有权、时间戳和扩展属性时，常用：

```
cp -a SOURCE DESTINATION
```

`-a` 是归档复制模式，适合已有目录树迁移，但并不意味着无需验证。普通用户无法把任意所有者写入目标；目标文件系统也可能不支持源端的全部属性。

③ [操作] 复制目录内容时使用 `SOURCE/.`

要把源目录的所有成员，包括隐藏成员，直接放入已有目标目录：

```
cp -a /srv/source/. /srv/destination/
```

这里的 `.` 是源目录自身的目录项语义，避免了 `/srv/source/*` 漏掉隐藏成员。执行后应检查：

```
find /srv/destination -mindepth 1 -maxdepth 2 -printf '%y %P -> %l\n' | sort
```

此命令只用于观察结果树；复杂筛选和批量处理的完整机制属于前一章。

④ [操作] 控制覆盖而不是事后补救

常见选项包括：

- `-i`：覆盖前询问；
- `-n`：不覆盖已有文件，但版本和脚本兼容性需要谨慎；
- `-u`：仅在源更新时复制，不能替代内容校验；
- `--backup`：覆盖前保留目标备份。

交互选项适合人工操作，不适合需要无人值守和确定终态的脚本。稳定流程是先列出将被覆盖的目标，再选择备份、跳过或明确替换策略。

⑤ [操作] 使用 `mv` 改名或移动

基本形式：

```
mv SOURCE DESTINATION
mv SOURCE... DIRECTORY
```

在同一文件系统内，移动通常主要改变目录项关系，速度很快；跨文件系统移动不能直接重命名，工具通常需要复制目标后删除源。跨文件系统时故障边界更复杂：目标可能已部分出现，而源仍然存在。因此重要移动后要同时验证：

```
test ! -e SOURCE
stat -- DESTINATION
```

并根据任务需要检查内容和元数据。不要仅因目标出现就认为源已安全删除。

⑥ [操作] 处理符号链接时明确“链接对象”还是“链接指向内容”

`cp` 的递归和解引用选项会影响符号链接。归档模式通常保留链接本身；显式解引用会复制链接指向的内容。操作前应通过：

```
stat -c '%F %N' -- PATH
```

确认对象类型。若任务要求保留目录树结构，默认应避免无意展开符号链接，否则可能复制超出预期集合的内容。

⑦ [验证] 复制和移动后的分层验证

推荐最小链：

```
源与目标存在状态
→ 目标层级和类型
→ 普通文件字节
→ 模式、所有者、组、时间戳
→ 链接关系
```

示例：

```
find /srv/destination -printf '%y %P -> %l\n' | sort
stat -c '%A %a %U %G %y %n' /srv/destination/important.conf
sha256sum /srv/source/important.conf /srv/destination/important.conf
```

[Cheatsheet] `cp` 保留源，`mv` 改变源路径；复制目录内容用 `SOURCE/.`；`cp -a` 请求归档式保留但仍要验证；跨文件系统 `mv` 要检查两端；符号链接先确认复制链接还是目标内容。

[操作专题] 使用 `install` 把文件部署到明确终态

`install` 不是软件包管理器。它面向“把一个已准备好的文件部署到目标位置，同时明确创建父目录和设置目标属性”的任务。与先 `cp` 再 `chmod` 相比，它把内容复制和目标模式表达在同一操作中，更适合安装脚本、考试任务和可重复部署。

① [操作] 部署文件并指定模式

```
install -m 0640 app.conf /etc/myapp/app.conf
```

`-m` 设置目标模式，不是从源文件继承模式。若目标父目录已经存在，这条命令会复制内容并把目标模式设为 `0640`。

② [操作] 使用 `-D` 创建缺失的父目录

```
install -D -m 0640 app.conf /etc/myapp/app.conf
```

`-D` 为目标文件创建缺失的父目录。它适合“单个文件 + 完整目标路径”的部署。若需要独立创建目录并指定目录模式，应使用：

```
install -d -m 0750 /etc/myapp
```

③ [操作] 指定目标所有者和组

具有足够权限时可使用：

```
install -D -o root -g appops -m 0640 app.conf /etc/myapp/app.conf
```

目标用户和组必须存在；账号和组的创建属于下一章。普通用户无法任意设置所有者，因此参数存在不代表操作一定能完成。

④ [操作] 区分内容更新与目标属性语义

`install` 默认面向部署终态，不是通用的“原样克隆”工具。题目应先明确目标文件需要的模式、所有者和组，再通过参数显式设置；不要把源文件的偶然元数据直接当作部署终态。

⑤ [验证] 部署后同时验证内容与属性

```
sha256sum app.conf /etc/myapp/app.conf
stat -c '%U %G %a %y %n' /etc/myapp/app.conf
```

摘要相同证明字节一致；`stat` 才回答所有者、组、模式和时间。若部署目录是服务配置目录，服务是否真正读取配置属于对应服务章节，不由文件存在自动证明。

[Cheatsheet] `install -m` 设置目标模式；`-D` 创建文件父目录；`-d` 创建目录；`-o/-g` 需要相应权限；`install` 面向部署终态，不等于完整保留源元数据。

[知识专题] `tar` 的成员、路径和元数据模型

`tar` 的核心对象不是压缩后缀，而是归档成员。创建归档时，源路径被转换为成员名称；查看归档时，成员名称决定未来提取出的层级；提取时，当前目录、目标目录、执行身份和保留选项共同决定恢复结果。最可靠的学习路径是先不压缩，理解创建、查看和提取，再叠加 `gzip`、`bzip2` 或 `xz`。

① [知识点] 归档是成员序列，压缩是归档流的表示方式

归档可以包含多个文件、目录、链接和元数据。压缩只把这个归档流变小。因而：

```
多个成员 → tar 归档 → 可选压缩 → 单个归档文件
```

单独执行 `gzip file` 会生成一个压缩文件，但不能把多个目录成员组织成可逐项列出和恢复的归档。

② [知识点] `c`、`t`、`x` 是互斥的主要操作

- `-c` / `--create`：创建；
- `-t` / `--list`：列出成员；
- `-x` / `--extract`：提取；
- `-f` `ARCHIVE`：指定归档文件。

命令的核心骨架是：

```
tar -cf archive.tar MEMBERS...
tar -tf archive.tar
tar -xf archive.tar
```

`-f` 后紧跟归档文件名。把归档名误当成员，或把成员误当归档名，是高频语法错误。

③ [知识点] 成员名称决定提取后的目录层级

若在 `/srv` 下执行：

```
tar -cf app.tar app
```

归档成员通常以 `app/` 开头。若进入 `/srv/app` 后归档 `.`，成员将以 `./` 开头，提取后不会自动形成名为 `app` 的顶层目录。任务要求固定顶层时，应从其父目录创建归档。

④ [知识点] `-C` 改变后续路径的解释位置

```
tar -C /srv -cf app.tar app
```

`-C /srv` 先改变 `tar` 处理后续成员时的工作目录，归档中保存 `app/...`，而不是本机绝对路径。`-C` 具有位置语义：若命令中多次出现，它只影响之后的路径参数，不能把它当成无顺序的装饰选项。

⑤ [知识点] 绝对路径成员具有覆盖风险

课程示例指出，`tar` 默认会从绝对源路径移除前导 `/`，以相对成员名称存储。这能降低提取时直接覆盖系统绝对路径的风险。即便工具有保护，未知归档仍应先列出成员并提取到空目录；不要把安全完全寄托在默认保护上。

⑥ [知识点] 归档元数据有默认范围和扩展范围

普通 `tar` 归档会保存常用模式、时间、所有者信息和链接类型，但 ACL、扩展属性和 SELinux 上下文通常需要显式选项，例如：

```
--acls
--xattrs
--selinux
```

保存了元数据也不代表任何用户都能原样恢复。`root` 与普通用户提取时的所有者结果不同；`umask` 和保留权限选项也会影响模式。

⑦ [知识点] 查看成员是创建和提取之间的安全闸门

```
tar -tf archive.tar.xz
tar -tvf archive.tar.xz
```

现代 `tar` 通常可从归档头识别压缩类型，因此只查看成员时常不必重复压缩选项。`-t` 不修改目标目录，适合先确认顶层、绝对路径、`...`、链接和成员集合。

[Cheatsheet] `c/t/x` 先选主要操作，`f` 后接归档名；成员名称决定提取层级；固定顶层从父目录归档或使用 `-C`；未知归档先 `tar -tf`；ACL、`xattr` 和 SELinux 元数据不是默认“全部保留”。

[操作专题] 创建、压缩、查看并安全提取归档

一个可靠的归档流程不是“执行一条 `tar` 命令”，而是先建立源集合基线，创建归档，非破坏性列出成员，在受控空目录试提取，再比较字节、层级和关键元数据。压缩算法的选择位于这个流程中间，不应遮蔽成员语义。

① [操作] 创建未压缩归档

```
tar -C /srv -cf /root/app.tar app
```

这条命令把 `/srv/app` 作为 `app/...` 成员写入 `/root/app.tar`。创建前确认目标归档文件是否已存在，因为同名归档可能被替换。

② [操作] 使用 `gzip`、`bzip2` 或 `xz` 压缩

对应短选项：

```
-z gzip    常见后缀 .tar.gz
-j bzip2   常见后缀 .tar.bz2
-J xz      常见后缀 .tar.xz
```

示例：

```
tar -C /srv -czf /root/app.tar.gz app
tar -C /srv -cjf /root/app.tar.bz2 app
tar -C /srv -cJf /root/app.tar.xz app
```

选项大小写必须精确，尤其是 `-j` 与 `-J`。

③ [操作] 独立压缩和解压单个文件

```
gzip report.txt
bzip2 report.txt
xz report.txt
```

通常会生成对应压缩文件并移除原始未压缩文件；需要保留原文件时应查看相应工具的保留选项，不要假设源一定仍在。解压可用：

```
gunzip report.txt.gz
bunzip2 report.txt.bz2
unxz report.txt.xz
```

`gzip -l` 和 `xz -l` 可在解压前查看压缩和未压缩大小，辅助评估空间，但不能证明内部 `tar` 成员正确。

④ [验证] 创建后列出成员和验证格式

```
file /root/app.tar.xz
tar -tf /root/app.tar.xz
tar -tvf /root/app.tar.xz
```

`file` 识别外层格式；`tar -tf` 验证成员名称；`tar -tvf` 增加类型、模式、所有者和时间等信息。三者证明的层次不同。

⑤ [操作] 提取到新建空目录

```
verify_dir=$(mktemp -d)
tar -C "$verify_dir" -xf /root/app.tar.xz
```

对压缩归档，`tar` 通常能自动识别算法。未知归档先在空目录提取，避免覆盖已有业务路径。提取后检查：

```
find "$verify_dir" -printf '%y %P -> %l\n' | sort
```

⑥ [操作] 控制权限和所有者恢复

普通用户提取时通常成为提取文件的所有者；`root` 可以恢复归档记录的所有者。当前 `umask` 可能收紧提取模式，保留权限选项可以改变该行为。任务要求精确模式时，必须在目标端用 `stat` 验证，不能只看 `tar` 没报错。

⑦ [验证] 比较源和试提取结果

先固定比较根，再生成普通文件摘要清单：

```
(
  cd /srv/app
  find . -type f -print0 | sort -z | xargs -0 -r sha256sum
) > /tmp/source.sha256

(
  cd "$verify_dir/app"
  find . -type f -print0 | sort -z | xargs -0 -r sha256sum
) > /tmp/extracted.sha256

cmp -s /tmp/source.sha256 /tmp/extracted.sha256
```

这只比较普通文件字节和相对名称。目录模式、链接目标、所有者和时间仍需独立检查。

[Cheatsheet] `-z/-j/-J` 分别对应 `gzip/bzip2/xz`；先 `file` 看外层，再 `tar -tf` 看成员；未知归档提取到空目录；`root` 与普通用户的所有者恢复不同；摘要清单不替代链接和元数据检查。

[操作专题] 使用 scp 与 sftp 理解远端路径

本章只讨论已具备 SSH 连通性和认证条件时的文件传输。SSH 服务端配置、密钥认证和主机密钥处理属于第 20 章。这里最重要的是分清本地端、远端端、传输方向和远端路径解释位置。

① [操作] 识别远端操作数

常见远端语法为：

```
[user@]host:path
```

上传：

```
scp local.tar.xz student@serverb:/home/student/incoming/
```

下载：

```
scp student@serverb:/home/student/incoming/result.txt ./
```

冒号左侧标识远端主机和可选用户，右侧是远端路径。远端路径为空或相对时可能落入远端用户主目录语境，不应凭本地路径推测。

② [操作] 使用绝对路径消除远端落点歧义

考试和变更任务中，优先使用明确绝对路径：

```
scp archive.tar.xz student@serverb:/home/student/incoming/archive.tar.xz
```

执行前可先通过 SSH 验证目标：

```
ssh student@serverb 'pwd; stat -c "%F %a %U %G %n" /home/student/incoming'
```

此处只把 SSH 当作远端证据入口，不展开 SSH 配置。

③ [操作] 递归传输目录和保留时间戳

`scp -r` 递归复制目录；`scp -p` 请求保留修改时间、访问时间和模式。不同权限和后端下的属性保留仍需远端 `stat` 验证。对复杂目录树、增量同步和删除策略，应优先考虑 `rsync`，而不是反复完整复制。

④ [操作] 使用交互式 sftp

```
sftp student@serverb
```

会话中本地和远端各有独立工作目录：

<code>pwd / cd</code>	远端工作目录
<code>lpwd / lcd</code>	本地工作目录
<code>put</code>	上传
<code>get</code>	下载
<code>mkdir</code>	创建远端目录
<code>ls / lls</code>	查看远端 / 本地

开始传输前同时执行 `pwd` 和 `lpwd`，能显著减少方向和落点错误。

⑤ [操作] 递归上传或下载目录

交互式示例：

```
sftp> lcd /srv/export
sftp> cd /home/student/incoming
sftp> put -r app
```

或下载：

```
sftp> lcd /var/tmp/review
sftp> get -r /home/student/result
```

传输进度为 100% 只能说明客户端完成了传输流程，仍需在目标端检查层级、字节和必要属性。

⑥ [知识点] RHEL 9 中 scp 命令与底层协议要分开

RHEL 9 的 OpenSSH scp 默认使用 SFTP 协议实现传输；旧 SCP 协议可通过兼容选项强制使用。课程扫描材料中的安全警告反映了旧 SCP 协议的历史问题，但不能简单推出“RHEL 9 的 scp 命令一定使用旧协议”。本章保留 scp 命令的考试与运维语义，同时把底层协议差异标记为版本敏感点。

⑦ [验证] 远端传输后的最小验收

```
ssh student@serverb '
  stat -c "%F %a %U %G %s %y %n" /home/student/incoming/archive.tar.xz
  sha256sum /home/student/incoming/archive.tar.xz
'
```

本地计算同一文件摘要后比较。若目标是目录，还要检查是否多包一层，以及隐藏成员是否存在。

[Cheatsheet] 远端语法是 [user@]host:path；优先使用绝对路径；sftp 有本地和远端两套工作目录；100% 进度不等于终态正确；RHEL 9 的 scp 默认后端是版本敏感事实，和命令界面分开理解。

[操作专题] 使用 rsync 同步差异并控制删除边界

rsync 的核心不是“更快的 scp”，而是比较源集合与目标集合，根据选择规则和属性策略决定哪些对象需要创建、更新、跳过或删除。首次同步可能接近完整复制，后续同步才体现差异传输优势。任何涉及目标覆盖或删除的命令，都应先 dry-run。

① [知识点] 源目录末尾斜杠定义同步根

```
rsync -a /srv/source /srv/target/
```

结果通常包含：

```
/srv/target/source/...
```

而：

```
rsync -a /srv/source/ /srv/target/
```

表示把 source 的内容直接同步到目标：

```
/srv/target/...
```

这不是格式偏好，而是结果层级语义。执行前应先写出预期目录树。

② [操作] 使用归档模式同步常用属性

```
rsync -a SOURCE/ DESTINATION/
```

-a 等价于一组常用选项，通常包含递归、保留符号链接、模式、时间、组、所有者和设备等意图。但它不包含所有扩展属性：

- -H：保留硬链接关系；
- -A：保留 ACL；
- -X：保留扩展属性，SELinux 上下文通常也依赖 xattr；

这些选项会增加成本和权限要求，只在任务需要时使用。

③ [查询] 先用 dry-run 和逐项输出审查计划

```
rsync -ani /srv/source/ student@serverb:/home/student/stage/
```

- -n / --dry-run：不执行更改；
- -i / --itemize-changes：逐项说明将发生的变化；
- -v：增加过程信息。

检查重点：目标主机和路径、源末尾斜杠、将创建的顶层、将覆盖的对象以及是否出现删除项。

④ [操作] 执行本地或远端同步

本地：

```
rsync -a /srv/source/ /srv/target/
```

上传到远端：

```
rsync -a /srv/source/ student@serverb:/home/student/stage/
```

从远端下载：

```
rsync -a student@serverb:/home/student/result/ /var/tmp/result/
```

远端两端不能都同时是远端操作数；至少一端是本地执行环境。

⑤ [边界] `--delete` 是镜像策略，不是默认同步选项

`--delete` 允许删除目标端在源集合中不存在的成员。它只适用于目标明确要求“精确镜像源”的场景。风险组合包括：

- 源路径写错或为空；
- 目标路径写错；
- 源末尾斜杠改变同步根；
- 排除规则改变可见源集合；
- 在未 `dry-run` 时直接执行。

安全流程：

```
rsync -ani --delete SOURCE/ DESTINATION/  
# 审查所有 *deleting 项后，再决定是否执行
```

若任务要求保留远端额外文件，必须明确排除 `--delete`。

⑥ [验证] 第二次 `dry-run` 无变化只证明 `rsync` 的比较规则已收敛

执行后再次运行相同 `dry-run`：

```
rsync -ani SOURCE/ DESTINATION/
```

无输出通常说明 `rsync` 根据当前比较策略认为无需更新。默认快速检查主要依赖大小和修改时间等元数据；它不是独立的全量字节证明。高价值文件仍可使用 `sha256sum` 抽查或建立清单。

⑦ [知识点] 所有者保留需要目标端权限

`-a` 包含所有者和组保留意图，但普通远端用户通常不能创建任意 UID/GID 所有者。若题目只要求内容、模式和时间，不应把源 `root` 所有者当作必须终态；若确实要求所有者，需确保目标端以足够权限运行，并在目标端验证。

⑧ [诊断] 反复同步同一文件时先比较大小、时间和两端时钟

若每次 `dry-run` 都显示同一文件变化：

1. 用 `stat` 比较大小和纳秒级修改时间；
2. 检查文件是否被应用持续改写；
3. 检查两端时钟和文件系统时间精度；
4. 再决定是否需要内容校验或调整比较策略。

不要一开始就用强制校验所有文件掩盖持续写入或时间异常。

[Cheatsheet] 源尾随斜杠决定同步根；`-a` 不包含硬链接、ACL 和全部 `xattr`；先 `-n -i` 再执行；`--delete` 只用于明确镜像目标；第二次 `dry-run` 是差异收敛证据，不等于全部字节和属性已经独立证明。

[操作专题] 使用 `sha256sum` 建立字节证据并完成分层验收

摘要是非常有价值的内容证据，但它的边界必须清楚。摘要函数把一个字节序列映射为固定长度值；比较同一算法的摘要，可以高置信度判断字节是否相同，却不会自动描述路径、权限或来源真实性。

① [操作] 计算单个文件摘要

```
sha256sum archive.tar.xz
```

输出包括摘要和文件名。传输前后应对同一个逻辑文件分别计算，不要把不同压缩算法生成的归档摘要直接要求相同。

② [操作] 创建和检查摘要清单

```
sha256sum file1 file2 > SHA256SUMS
sha256sum -c SHA256SUMS
```

检查清单时，文件名按清单记录的路径解释。移动清单或改变当前目录后，应确认路径仍能解析到目标文件。

③ [边界] 摘要清单本身必须可信

`sha256sum -c` 只说明当前文件与给定清单一致。若清单和文件由同一不可信来源同时被修改，检查仍可能通过。数字签名、可信发布通道和供应链真实性不在本章完整展开，但不能把普通摘要称为“身份认证”。

④ [操作] 对目录建立稳定的普通文件摘要视图

目录不是一个单一字节流。可先固定比较根和成员顺序：

```
(
  cd /srv/source
  find . -type f -print0 | sort -z | xargs -0 -r sha256sum
) > /tmp/source.sha256
```

在目标根执行同样流程，再使用 `cmp` 或 `diff` 比较清单。该方法把相对名称纳入文本清单，但仍不检查目录、符号链接和元数据。

⑤ [验证] 组合成员、字节和元数据证据

完整验收示例：

```
find TARGET -printf '%y %P -> %l\n' | sort
sha256sum SOURCE_FILE TARGET_FILE
stat -c '%F %a %U %G %y %n' TARGET_FILE
```

- `find`：层级、类型和链接目标；
- `sha256sum`：普通文件字节；
- `stat`：模式、所有者、组和时间。

⑥ [知识点] 归档摘要与归档成员摘要回答不同问题

归档文件摘要相同，证明整个归档字节相同；两个归档成员内容相同，却可能因为成员顺序、时间戳或压缩参数不同而得到不同的归档摘要。任务若要求“传输文件未改变”，比较归档摘要足够；若要求“恢复后的数据集合正确”，还需提取和成员级验证。

[Cheatsheet] `sha256sum` 只看字节；`-c` 相对清单检查；清单本身不提供来源真实性；目录先固定成员集合和相对根；归档摘要和恢复后成员验证回答不同问题。

[诊断专题] 从“文件不对”推进到最有区分度的证据

“复制失败”“归档不对”或“同步异常”都过于宽泛。诊断应先把症状归入路径层级、成员集合、字节、元数据、远端落点或删除范围，再选择下一条最有区分度的证据。不要反复重跑命令，因为重跑可能继续覆盖或删除目标证据。

① [诊断] 目标多出一层目录

症状：预期 `/target/a.conf`，实际为 `/target/source/a.conf`。

当前证据：

```
find /target --mindepth 1 --maxdepth 2 -printf '%y %P\n' | sort
```

假设：`rsync` 源缺少尾随斜杠，或复制目录到已有目标目录时把目录本身作为成员。

最小修复：先重新写出期望树，调整源根；不要直接把所有文件移动一层，除非已确认目标中没有其他合法成员。

② [诊断] 隐藏成员缺失

症状：普通文件存在，但 `.env` 或 `.config` 不在目标。

当前证据:

```
find SOURCE -mindepth 1 -maxdepth 1 -printf '%P\n' | sort
find TARGET -mindepth 1 -maxdepth 1 -printf '%P\n' | sort
```

假设: 使用了 `SOURCE/*`, Shell 展开漏掉点号成员。

最小修复: 使用 `SOURCE/.` 或直接把目录交给能够递归处理的工具。

③ [诊断] 归档可读取但成员顶层错误

症状: `tar -tf` 成功, 但成员以 `./` 开头或缺少要求的顶层目录。

下一条证据: 只查看成员前若干行和创建命令所处目录。

```
tar -tf archive.tar.xz | sed -n '1,30p'
pwd
```

假设: 在源目录内部归档 `.`, 或 `-C` 和成员参数组合错误。

最小修复: 从源目录父级重建归档, 例如 `tar -C /srv -cJf ... app`。

④ [诊断] 摘要相同但权限或时间错误

症状: 内容检查通过, 应用仍拒绝读取或任务要求的时间未保留。

下一条证据:

```
stat -c '%F %a %U %G %y %n' SOURCE TARGET
```

假设: 未请求保留、执行身份不足、目标文件系统不支持, 或提取时受到 `umask` 影响。

最小修复: 只修正题目要求的属性, 并再次验证; 权限模型的完整讲解归后续权限章节。

⑤ [诊断] 传输显示成功但远端找不到文件

症状: 客户端显示 100%, 预期绝对路径不存在。

下一条证据:

```
ssh USER@HOST 'id; pwd; find "$HOME" -maxdepth 3 -name "EXPECTED" -print'
```

假设: 使用了远端相对路径、登录用户与预期不同, 或在 SFTP 会话中远端工作目录已改变。

最小修复: 使用明确绝对路径, 或在 `sftp` 中先 `pwd` 后 `put/get`。

⑥ [诊断] `rsync --delete` 计划删除大量对象

症状: `dry-run` 出现大量 `*deleting`。

下一条证据:

```
printf 'SOURCE=%q\nDEST=%q\n' "$src" "$dst"
find "$src" -mindepth 1 -maxdepth 2 -printf '%P\n' | sort | sed -n '1,50p'
```

同时确认远端路径和源末尾斜杠。

最小修复: 停止实际执行, 修正同步根或目标; 不得用无调查的强制选项继续。

⑦ [诊断] 第二次 `rsync` 仍反复显示变化

当前证据: `rsync -ani` 的逐项变化类型。

下一条证据: 比较两端大小、修改时间、文件是否持续写入以及时钟状态。

最小修复: 先消除持续写入或时间差, 再决定是否使用内容校验。不要用更激进的同步掩盖上游状态变化。

⑧ [诊断] `tar` 报压缩格式不匹配

症状: 使用 `-z` 提取 `.tar.xz` 时出现“not in gzip format”。

下一条证据:

```
file archive
tar -tf archive
```

假设：手工指定的算法与实际文件不匹配，或扩展名错误。

最小修复：让 tar 自动识别，或使用与实际格式对应的参数；不要仅根据文件名后缀下结论。

⑨ [诊断] `install` 内容正确但服务仍不工作

当前证据：摘要和 `stat` 已满足文件终态。

判断边界：本章到此只能证明部署文件正确。应用是否读取该路径、配置语法是否有效、服务是否重载属于对应服务和应用章节。下一条证据应转向应用自己的验证命令或日志，而不是继续重复复制。

[Cheatsheet] 先把症状归类为层级、成员、字节、元数据、远端落点或删除范围；下一条证据应能区分假设；重跑可能破坏证据；最小修复后按原验收层重新验证。

[经典任务] 创建保留关键属性的 `xz` 压缩归档并完成试提取验收

环境与当前状态

服务器上已有发布目录：

```
/srv/app-release/
├── .release-env
├── bin/start.sh
├── conf/app.conf
├── data/banner.txt
└── current -> data
```

现在需要把它交付给另一名管理员。`/root/submission/` 已存在，里面可能有旧文件，但不存在本次日期命名的目标归档。以下流程先明确预期成员和验收证据，再给出推荐操作；只有在目标主机实际执行并收集证据后，才能声明任务完成。

目标终态

1. 在 `/root/submission/` 创建 `app-release-YYYY-MM-DD.tar.xz`；
2. 归档使用 `xz` 压缩；
3. 所有成员必须位于单一顶层 `app-release/` 下；
4. 必须包含隐藏文件 `.release-env`；
5. 不能保存以 `/srv/` 开头的绝对成员名；
6. 不得修改源目录；
7. 创建后先列出成员，再提取到新建空目录；
8. 验证普通文件字节、符号链接目标和 `bin/start.sh` 模式；
9. 不能把“归档文件存在”作为唯一验收。

限制条件

- 不直接提取到 `/srv/`、`/` 或已有业务目录；
- 不使用 `*` 作为成员集合；
- 不关闭 SELinux、不使用 `chmod 777`；
- 不把示例命令或单条成功信息当作已经完成的目标端验收。

验收证据

层次	推荐证据
归档对象	<code>stat</code> 、 <code>file</code>
成员与顶层	<code>tar -tf</code>
试提取层级	<code>find</code>
普通文件字节	稳定相对摘要清单
链接	<code>find -type l -printf</code> 或 <code>readlink</code>
关键模式	<code>stat -c '%a %n'</code>

[参考解答] 经典任务一

① 调查并建立源基线

```
src=/srv/app-release
out=/root/submission
stamp=$(date +%F)
archive="$out/app-release-$stamp.tar.xz"

stat -- "$src" "$out"
test ! -e "$archive"
find "$src" -printf '%y %P -> %l\n' | sort
stat -c '%F %a %U %G %y %n' "$src/bin/start.sh"
```

`test ! -e` 防止无意覆盖同名交付物。`find` 同时显示隐藏成员和符号链接，避免用 `ls` 或 `*` 形成不完整基线。

② 从父目录创建具有固定顶层的归档

```
tar -C /srv -cJf "$archive" app-release
```

参数解释：

- `-C /srv`：把后续成员从 `/srv` 解释；
- `-c`：创建归档；
- `-J`：使用 `xz`；
- `-f "$archive"`：指定归档文件；
- `app-release`：归档成员以该顶层开始。

③ 非破坏性验证外层和成员

```
stat -- "$archive"
file -- "$archive"
tar -tf "$archive"
```

应确认：首层为 `app-release/`，列表中有 `app-release/.release-env`，不存在以 `/srv/` 或 `/` 开头的成员。

可做静态成员检查：

```
members=$(mktemp)
tar -tf "$archive" > "$members"
grep -Fx 'app-release/.release-env' "$members"
! grep -E '^(^|/)\.\.(/|$)' "$members"
```

④ 提取到新建空目录

```
verify_dir=$(mktemp -d)
tar -C "$verify_dir" -xf "$archive"
find "$verify_dir" -printf '%y %P -> %l\n' | sort
```

此时预期根为 `$verify_dir/app-release`。

⑤ 比较普通文件字节

```
(
  cd /srv/app-release
  find . -type f -print0 | sort -z | xargs -0 -r sha256sum
) > /tmp/app-source.sha256

(
  cd "$verify_dir/app-release"
  find . -type f -print0 | sort -z | xargs -0 -r sha256sum
) > /tmp/app-extracted.sha256

diff -u /tmp/app-source.sha256 /tmp/app-extracted.sha256
```

没有差异只证明相对名称对应的普通文件字节一致。

⑥ 验证链接和关键模式

```
readlink /srv/app-release/current
readlink "$verify_dir/app-release/current"
stat -c '%a %n' /srv/app-release/bin/start.sh \
"$verify_dir/app-release/bin/start.sh"
```

若任务还要求 ACL、扩展属性或 SELinux 上下文，需要在创建和提取时加入对应 tar 选项，并使用 `getfacl`、`getfattr` 或 `ls -Z` 单独验收。本任务只要求关键普通模式和链接目标，因此不扩大范围。

⑦ 结果声明边界

只有在实际执行并取得成员、路径、字节和元数据证据后，才能声明任务完成。示例流程本身不构成运行证据；在没有目标主机输出时，应把结论限定为“已给出推荐操作与验收方法”。

[经典任务] 安全同步目录到远端并证明源目录末尾斜杠语义正确

环境与当前状态

本地源目录：

```
/srv/site/
├── .well-known/status
├── index.html
└── assets/logo.txt
```

远端目标目录：

```
student@serverb: /home/student/stage/site/
```

目标目录已存在，并有一个远端管理员留下的 `remote-note.txt`。本次任务要求更新源中已有内容，但不得删除该远端额外文件。

目标终态

1. `/srv/site/` 的成员直接进入远端 `site/`；
2. 不能形成 `site/site/`；
3. 隐藏目录 `.well-known` 必须同步；
4. 保留普通模式和修改时间；
5. `remote-note.txt` 必须保留；
6. 实际执行前必须 dry-run；
7. 执行后再次 dry-run，不应出现预期外差异；
8. 抽查 `index.html` 的 SHA-256；
9. 不要求把本地 root 所有者映射为远端 root。

限制条件

- 不使用 `--delete`；
- 不修改 SSH 服务端配置；
- 不把一次 `rsync` 成功扩大成远端业务可用；
- 不编造服务器地址、输出或实时验证结果。

[参考解答] 经典任务二

① 调查两端和写出预期层级

本地：

```
find /srv/site -printf '%y %P -> %l\n' | sort
```

远端：


```
ssh student@serverb '
id
stat -c "%F %a %U %G %n" /home/student/stage/site
find /home/student/stage/site -mindepth 1 -maxdepth 2 -printf "%y %P -> %l\n" | sort
'
```

预期是 `index.html`、`assets/` 和 `.well-known/` 直接位于远端 `site/`，因此源必须写成 `/srv/site/`，带末尾斜杠。

② 先 dry-run 并逐项审查

```
rsync -ani /srv/site/ \
student@serverb:/home/student/stage/site/
```

检查清单中不应出现：

- 目标形成新的 `site/` 顶层；
- `*deleting remote-note.txt`；
- 意外远端路径。

本任务禁止 `--delete`，因此远端额外文件应保留。

③ 执行同步

```
rsync -a /srv/site/ \
student@serverb:/home/student/stage/site/
```

`-a` 请求递归并保留常用模式、时间和链接等属性。普通远端用户不能任意恢复本地 `root` 所有者，因此本任务不把所有者映射列为验收目标。

④ 再次 dry-run 验证差异收敛

```
rsync -ani /srv/site/ \
student@serverb:/home/student/stage/site/
```

无预期外输出说明 `rsync` 的当前比较规则认为两端不需再次更新；这不替代字节抽查。

⑤ 验证远端层级和额外文件

```
ssh student@serverb '
find /home/student/stage/site -mindepth 1 -maxdepth 2 -printf "%y %P -> %l\n" | sort
test -f /home/student/stage/site/remote-note.txt
test ! -e /home/student/stage/site/site
'
```

⑥ 抽查字节、模式和时间

本地：

```
sha256sum /srv/site/index.html
stat -c '%a %y %n' /srv/site/index.html
```

远端：

```
ssh student@serverb '
sha256sum /home/student/stage/site/index.html
stat -c "%a %y %n" /home/student/stage/site/index.html
'
```

比较摘要、模式和修改时间。若摘要相同而时间不同，应回到 `rsync` 输出和目标文件系统能力调查，而不是直接宣布任务失败或强制覆盖。

⑦ 镜像变式的边界

只有题目明确要求“远端必须与源完全一致，不得保留任何额外成员”时，才考虑：

```
rsync -ani --delete /srv/site/ student@serverb:/home/student/stage/site/
```

必须先人工审查所有删除项，再执行实际同步。本题存在必须保留的 `remote-note.txt`，所以 `--delete` 是错误答案。

[本章收束] 从命令记忆转向数据集的证据闭环

本章的共同对象不是某个命令，而是从源集合到目标集合的一次可验证变换。无论使用 `cp`、`tar`、`scp` 还是 `rsync`，都应先明确源边界和目标层级，再决定是否保留元数据、是否允许覆盖或删除，最后用互不替代的证据关闭任务。

确认源集合和目标边界

→ 选择复制、部署、归档、传输或同步模型

→ 控制层级、覆盖、属性和删除策略

→ 先预览或建立基线

→ 执行最小操作

→ 验证成员、路径、字节和元数据

工作方法：把模糊的“搬过去”改写成四个可判定问题

1. 搬什么？

是单个对象、目录本身、目录内容，还是归档成员集合？
2. 搬到哪里？

目标路径由本地还是远端解释，目标是否已存在，顶层应长什么样？
3. 保留什么？

只要求字节，还是还要求模式、属主、时间戳、链接、ACL 或扩展属性？
4. 怎样证明？

用成员表证明集合，用路径树证明层级，用摘要证明字节，用 `stat` 证明元数据。

主要判断表

目标或现象	首选入口	关键判断	最少再验证
本地创建副本	<code>cp</code>	目录本身还是目录内容	层级、摘要、 <code>stat</code>
本地改名或移动	<code>mv</code>	是否跨文件系统	两端存在状态、目标内容
部署文件并指定属性	<code>install</code>	目标目录和模式是否明确	摘要、模式、属主
把多个成员封装为单文件	<code>tar</code>	成员前缀和顶层是否正确	<code>tar -tf</code> 、试提取
压缩一个文件或归档流	<code>gzip</code> / <code>bzip2</code> / <code>xz</code>	压缩不是归档	<code>file</code> 、可解压性
一次性远端复制	<code>scp</code>	远端路径由谁解释	远端 <code>stat</code> 、层级
交互式远端浏览和传输	<code>sftp</code>	本地/远端工作目录不同	<code>lpwd</code> 、 <code>pwd</code> 、目标检查
目录差异同步	<code>rsync</code>	源末尾斜杠和删除范围	<code>dry-run</code> 、第二次比较、摘要
证明普通文件字节	<code>sha256sum</code>	摘要不证明元数据	<code>stat</code> 、成员和路径检查

向下一章交接

本章已经能够把数据集送到正确位置并证明内容、层级和必要属性。下一章《用户、组与账号生命周期》将回答另一个问题：这些文件由谁拥有、哪些账号应存在、用户和组的生命周期如何改变访问边界。本章只在部署和传输时引用现有用户身份，不提前展开账号创建、密码策略和组成员关系。

RHEL 9 · RHCSA 实操讲义
07

用户、组与账号生命周期

从名称映射到会话凭据：把创建、修改、停用、期限与数据边界放进同一条生命周期链。

对象模型操作语义验证诊断经典任务

大字号阅读版

本章阅读导航

先抓住一条主线：本地账号管理不是“执行一条创建或锁定命令”，而是让名称解析、UID/GID、组关系、账号数据库、口令期限、家目录和新会话凭据共同达到可验证终态。

- 01识别身份对象区分名称、UID/GID、主组与附加组
- 02读取数据库状态定位 passwd、shadow、group 与 gshadow
- 03建立变更基线用 getent、id、passwd -S 与 chage -l 取证
- 04推进生命周期创建、修改、锁定、过期和删除
- 05刷新会话身份比较数据库状态与现有进程凭据
- 06验证数据边界检查家目录、数字所有权与删除遗留

专题地图

知识专题从名称到数字身份：用户、组与会话
知识专题四个本地账号数据库
操作专题先查询再变更：建立可信基线
操作专题默认值、skeleton 与精确创建
操作专题修改用户、组 and 家目录
知识专题停用账号的多状态模型
操作专题su、newgrp 与会话身份
诊断专题从登录失败或“改了不生效”推进证据
经典任务精确创建：安全停用并评估删除

阅读时持续回答

- 1. 当前看到的是名称映射，还是进程实际持有的凭据？
- 2. 主组和附加组分别记录在哪里？
- 3. 操作改变的是口令、期限、Shell，还是账号记录？
- 4. 组变更是否已经进入一个新会话？
- 5. 家目录字段变化是否伴随数据迁移？
- 6. 密码锁定、到期、nologin 与删除分别阻止什么？
- 7. 删除前哪些文件仍由旧 UID/GID 拥有？
- 8. 当前证据能证明什么，又不能证明什么？

阅读提示：每次操作都先判断它改变的是名称映射、数字身份、口令期限、家目录数据还是会话凭据；随后选择能够证明该层终态的证据。

第 07 章 · 正文

服务器上的“用户”并不只是一个可以输入的用户名。系统真正依赖的是名称到 UID/GID 的映射、一个主组与若干附加组、四个本地账号数据库、口令和期限字段、家目录与登录 Shell，以及某个已经运行的进程实际持有的凭据。只观察其中一层，就会得到看似合理却错误的结论：账号已经加入组，旧 Shell 仍没有新组；口令已经锁定，已有会话或非口令认证仍可能存在；用户已经删除，旧 UID 拥有的数据仍留在文件系统中。

本章以“身份对象 → 数据库状态 → 生命周期操作 → 新会话验证 → 数据边界”为主线。先理解名称、数字身份和会话凭据为什么必须分开，再进入创建默认值、用户和组的创建与修改、口令期限、停用与删除；最后用证据链处理“用户不能登录”“组已修改但仍不生效”“账号已删除却留下数字所有者”等问题。

本章只负责本地用户、组和账号生命周期。身份参与文件访问判定的规则留给第 08 章《传统权限、umask 与特殊权限位》；特权授权留给第 10 章《sudo 与最小特权授权》；SSH 密钥认证留给第 20 章《SSH 客户端、服务端与密钥认证》。

... {concept-card} 概念 UID / GID 是系统使用的数字身份。用户名和组名是名称服务提供的人类可读映射，文件元数据和进程凭据最终都能落到 UID 与 GID。改名通常不改变原 UID，删除后重用 UID 却可能让新账号继承对旧文件的名称解释，因此数字身份的调查贯穿创建、修改和删除。...

... {concept-card} 概念 主组与附加组 描述用户所属组的两个不同层次。每个用户恰好有一个主组，其 GID 位于用户条目中；附加组用于补充成员关系，通常在组数据库的成员字段中表达。/etc/group 某组成员列表为空，不代表没有用户把它作为主组。...

... {concept-card} 概念 账号数据库 由 /etc/passwd、/etc/shadow、/etc/group 和 /etc/gshadow 共同组成：公开属性与受保护字段被分离保存。理解字段是为了读证据和诊断关联状态，而不是把直接编辑文本文件当作日常管理方法；正常生命周期应由专用命令同时维护关联数据库和锁。...

... {concept-card} 概念 认证状态 不是单一开关。口令字段可用或锁定、账号是否到期、登录 Shell 是否允许交互、认证服务是否接受其他凭据，以及是否已有会话，分别属于不同层次。passwd -l 只锁定口令字段，不能被扩大解释为“这个身份在所有入口和所有现有进程中立即消失”。...

... {concept-card} 概念 密码期限 把口令最后修改日、最短和最长期限、警告期、过期后的非活动期与账号到期日分开记录。密码过期要求更新凭据，账号到期限制账号继续使用：二者应分别通过 passwd -S 与 chage -l 观察，不能用一个日期替代全部状态。...

... {concept-card} 概念 当前会话身份 是进程在会话建立时取得的 UID、主 GID 和附加组集合。管理员修改数据库后，已经运行的 Shell 不会自动刷新全部组凭据。id USER 读取名称服务计算出的身份，而目标用户新登录或 su - USER 后在会话内运行 id，才能证明新凭据已经进入实际进程。...

::: {ops-quick} 操作语义 以下七组入口覆盖本章最关键的查询、创建、修改、停用、删除和会话验证。先识别命令改变或观察的对象，再记参数。

getent / id

SYNOPSIS

```
getent database [key ...]
id [OPTION]... [USER]
```

通过 NSS 查询账号或组的解析结果，并汇总指定用户或当前进程的数字身份与组凭据。

重要参数 / 形式

getent passwd USER

查询用户条目；结果可能来自本地文件或其他 NSS 身份源。

getent group GROUP

查询组条目和显式成员；不能单独列出所有把它作为主组的用户。

id USER

显示名称服务计算出的用户身份。

id

显示当前进程实际持有的凭据，适合验证新会话。

useradd

SYNOPSIS

```
useradd [options] LOGIN
useradd -D [options]
```

创建本地用户条目，并按显式参数和默认配置决定 UID、组、家目录、Shell、期限与 skeleton 初始化。

重要参数 / 形式

-u UID

指定用户数字身份；创建前检查名称服务和现有文件是否冲突。

-g GROUP

指定主组；目标组通常应先存在。

-G G1,G2

指定初始附加组列表。

-d HOME

设置家目录字段，本身不保证目录被创建。

-m

创建家目录并复制 skeleton 内容。

-s SHELL

指定登录 Shell。

usermod

SYNOPSIS

```
usermod [options] LOGIN
```

修改已有本地账号的名称、数字身份、组关系、家目录、Shell、口令字段或账号到期状态。

重要参数 / 形式

-aG GROUP USER

把用户追加到附加组；**-a** 只有与 **-G** 组合才表达追加。

-G G1,G2 USER

用给定列表替换附加组集合；遗漏已有组会将其移除。

-L / -U

锁定或解锁口令字段；不等于终止已有会话。

-d HOME -m USER

同时修改家目录字段并请求移动旧家目录内容。

-l NEW OLD

修改登录名；不会自动改完所有外部引用。

userdel

SYNOPSIS

```
userdel [options] LOGIN
```

删除本地账号记录；是否删除默认家目录和邮件池由参数决定，账号在其他路径中的数据不由命令自动发现。

重要参数 / 形式

userdel USER

删除账号记录，默认保留家目录和其他数据。

userdel -r USER

同时删除记录中的家目录和邮件池；不代表清理任意挂载点或业务目录。

删除前调查

保存旧 UID/GID，并按数字身份查找外部文件。

groupadd / groupmod / groupdel

SYNOPSIS

```
groupadd [options] GROUP
groupmod [options] GROUP
groupdel [options] GROUP
```

管理本地组的名称和 GID；删除前必须确认没有用户继续把该 GID 作为主组，并调查旧 GID 拥有的数据。

重要参数 / 形式

groupadd -g GID GROUP

使用指定 GID 创建组。

groupmod -n NEW OLD

修改组名，数字身份保持不变。

groupmod -g GID GROUP

修改 GID；文件系统中的旧数字所有权需要单独处理。

groupdel GROUP

删除组；被用户作为主组时通常会被阻止。

passwd / chage

SYNOPSIS

```
passwd [options] [LOGIN]
chage [options] LOGIN
```

passwd 管理口令字段和快速状态，**chage** 管理密码期限与账号到期日。

重要参数 / 形式

passwd -S USER

查看口令状态摘要；结合现场输出判断，不背诵伪造状态文本。

passwd -l USER / passwd -u USER

锁定或解锁口令字段。

chage -m MIN -M MAX -W WARN USER

设置最短、最长和警告期限。

chage -I DAYS USER

设置密码过期后的非活动期。

chage -E DATE USER

设置账号到期日。

chage -d 0 USER

要求用户下次使用口令登录时修改密码。

su / newgrp

SYNOPSIS

```
su [options] [-] [USER [ARGUMENT...]]
newgrp [-] [GROUP]
```

建立新的用户或组身份 Shell，用于验证数据库变更是否已经进入实际进程凭据。

重要参数 / 形式

```
su USER
    切换用户，但通常保留较多当前环境。
su - USER
    请求登录式环境，更接近一次新的登录会话。
newgrp GROUP
    启动一个以目标组作为当前组的新 Shell；exit 返回上一层。
会话内 id
    验证实际 UID、主 GID 与附加组，而不是只看数据库记录。
```

...

[知识专题] 从名称到数字身份：用户、组和会话是什么关系

账号管理最容易出现误判，是把“用户名存在”当成完整身份。实际上，一个用户条目把名称映射为 UID、主 GID、家目录和登录 Shell；组条目把组名映射为 GID 和显式成员；进程在创建会话时取得一组真实凭据。先分清这三层，后续创建、改名、改组和删除才不会破坏数据归属。

① [知识点] 用户名和 UID 分别解决可读性与系统身份

用户名用于登录、命令输入和展示；UID 是本地系统识别用户的核心数字。id、ps 和文件元数据最终都能落到数字身份。UID 0 是 root；系统账号与普通账号的默认范围由本机 /etc/login.defs 等配置决定，不能只凭习惯猜测现场边界。

账号改名通常不改变 UID，因此原有文件仍由同一个数字身份拥有。相反，删除账号后再把相同 UID 分配给新账号，旧文件会被解释为新账号所有，这正是删除前必须调查数字身份的原因。

② [知识点] 每个用户只有一个主组，但可有多个附加组

主组 GID 位于用户数据库中。用户新建文件时，默认组所有者通常取自主组，但具体继承与权限规则留给第 08 章《传统权限、umask 与特殊权限位》。附加组用于把用户加入额外的协作或角色集合。

/etc/group 的成员列表为空，并不能证明无人属于该组：用户可能通过 /etc/passwd 中的主 GID 把它作为主组。因此查询一个用户应优先使用 id USER，而不是只查看某一条组记录。

③ [知识点] 用户私有组是一种默认组织方式，不是数字必须相等的规则

RHEL 普通用户通常会获得一个与用户名同名的用户私有组，并把它作为主组。这种约定便于协作权限设计，但 UID 与 GID 在语义上是两个独立命名空间，不求数值永远相同。创建共享组时，应先检查组名和 GID 是否冲突，而不是为了“整齐”强行复用数字。

④ [知识点] 账号数据库与运行进程持有的凭据不是同一个时刻

id alice 由当前名称服务查询账号关系；alice 已经打开的 Shell 中运行 id，显示的是该进程实际持有的 UID、主 GID 和附加组。管理员执行 usermod -aG developers alice 后，数据库立即变化，但旧 Shell 不会自动刷新。验证最终使用身份时必须建立新会话。

⑤ [知识点] 本地账号只是 NSS 可能返回的一类来源

getent passwd alice 和 getent group developers 通过系统名称服务查询，结果可能来自本地文件，也可能来自其他配置的身份源。本章的 useradd、usermod 和 userdel 只用于本地账号；看到 getent 有结果，不代表应直接用本地工具修改远程目录账号。

... {cheatsheet} Cheatsheet 名称便于人读，UID/GID 才是系统身份；主组在用户条目中，附加组在组成员关系中；id USER 看数据库解析，用户现有 Shell 中的 id 看实际会话；删除或重用数字身份前先调查文件。...

[知识专题] 四个本地账号数据库：字段如何共同描述一个身份

四个数据库不是四份重复名单，而是把公开属性与受保护信息分开保存。读懂字段的目的是不是鼓励手工编辑，而是能够解释 getent、passwd -S、chage -l 的结果，并在账号异常时知道下一条证据在哪里。

① [知识点] /etc/passwd 的七个字段定义账号基本属性

典型结构：

```
login:password-placeholder:UID:GID:GECOS:home:shell
```

字段	说明	常见判断
login	登录名	名称必须唯一
x	口令占位	实际受保护字段在 shadow 中
UID	用户数字身份	删除和重用风险围绕它展开
GID	主组数字身份	不等于 /etc/group 显式成员列表
GECOS	注释或说明	可用 usermod -c 修改
home	家目录路径	字段变化不等于数据已移动
shell	登录 Shell	nologin 只限制正常交互 Shell

② [知识点] /etc/shadow 把口令状态和期限拆成九个字段

典型结构：

```
login:password:lastchg:min:max:warn:inactive:expire:reserved
```

- password：口令哈希或特殊锁定标记；不要将其前缀固定解释为某一种算法，现场算法受版本和策略影响。
- lastchg：上次修改口令距 Unix epoch 的天数。
- min、max、warn：最短期限、最长期限、提前警告天数。
- inactive：口令过期后允许延迟多少天，之后账号因口令失效而不可用。
- expire：账号到期日，与“口令到期日”不是同一个字段。

passwd -S USER 适合快速查看口令状态；chage -l USER 更适合查看期限。二者互补，不能只看一条输出。

③ [知识点] /etc/group 的四个字段定义组和显式成员

典型结构：

```
group:password-placeholder:GID:member1,member2
```

最后一个字段主要列出附加成员。某用户把该组作为主组时，通常不会因此自动出现在成员列表中。组口令机制在现代系统中很少作为推荐授权方式，本章只识别字段，不把共享组口令作为常规方案。

④ [知识点] /etc/gshadow 保存受保护的组管理信息

典型结构：

```
group:encrypted-password:administrators:members
```

它可以表达组管理员和成员，并与 /etc/group 配合。普通 RHCSA 用户和组任务主要通过 useradd、usermod、groupadd 等工具完成；直接维护组管理员或组口令不是本章经典任务的主线。

⑤ [知识点] 专用工具同时维护关联文件和锁，不应默认手工编辑

直接编辑可能造成 passwd 与 shadow、group 与 gshadow 不一致，也可能绕过文件锁和字段校验。普通生命周期操作使用专用命令；只有在系统恢复等受控场景下才考虑 vipw、vigr 等安全编辑入口，并且那属于更高级的恢复边界。

::: {cheatsheet} Cheatsheet passwd 管公开属性，shadow 管口令与期限；group 管 GID 和显式成员，gshadow 管受保护的组管理字段；主组不能只靠 group 成员列表判断；正常管理用专用命令，不直接改文本数据库。:::

[操作专题] 先查询再变更：建立账号与组的可信基线

创建或修改前必须先回答：名称是否已被解析、目标 UID/GID 是否已占用、账号当前属于哪些组、口令和期限处于什么状态、当前会话是否陈旧。查询顺序应从无副作用的名称服务证据开始，再进入受保护字段和数据路径。

① [查询] 用 getent 查询名称服务中的用户和组

作用对象：系统 NSS 配置能够解析的数据库。

基本形式：

```
getent passwd alice
getent group developers
getent passwd 2401
getent group 4200
```

按名称和按数字都无结果，才说明在当前名称服务视图中没有冲突。`grep '^alice:' /etc/passwd` 只能证明本地文件中是否有行，不能替代 NSS 查询。

② [查询] 用 `id` 汇总用户身份，并区分指定用户与当前会话

```
id alice          # 查询名称服务计算出的 alice 身份
id                # 查看当前进程实际持有的身份
id -u alice
id -g alice
id -G alice
id -nG alice
```

修改组后，管理员侧 `id alice` 已显示新组，而用户旧 Shell 中 `id` 未显示，这是会话尚未刷新，不是 `usermod` 必然失败。

③ [查询] 用 `passwd -S` 和 `chage -l` 分别查看口令与期限

```
passwd -S alice
chage -l alice
```

`passwd -S` 的状态字样可能随实现和本地化不同，重点判断“可用、锁定、无口令”以及日期字段，不在静态讲义中伪造固定输出。`chage -l` 用于核对上次修改、最短/最大期限、警告、失效和账号到期。

④ [调查] 记录家目录和数字所有权基线

```
getent passwd alice
stat -c 'path=%n uid=%u gid=%g owner=%U group=%G' /srv/home/alice
find /srv -xdev -uid 2401 -print
```

修改 UID、移动家目录或删除账号前，至少记录旧 UID/GID、家目录路径和账号在目标业务路径中的文件。`find /` 可能成本很高，真实工作中应按挂载点和业务范围分批调查。

⑤ [查询] 使用帮助入口确认现场工具语义

```
man useradd
man usermod
man userdel
man groupadd
man groupmod
man groupdel
man passwd
man chage
man getent
```

课程给出核心路径，man page 用于核对现场版本的参数、默认行为和限制。

⋮ {cheatsheet} Cheatsheet `getent` 查解析，`id USER` 查数据库身份，当前 Shell 的 `id` 查会话凭据；`passwd -S` 查口令状态，`chage -l` 查期限；变更 UID/GID 或删除前记录数字身份和文件清单。⋮

[操作专题] `useradd` 默认值与 `skeleton`：新账号从哪里继承初始状态

`useradd` 不是只把一行追加到 `/etc/passwd`。它会合并显式参数、系统默认值和 `skeleton` 内容，并更新多个数据库。考试题需要确定终态时，应把关键属性写成显式参数；真实工作修改全局默认值时，要清楚它只影响后续创建。

① [操作] 用 `useradd -D` 查看可管理的默认项

```
useradd -D
```

常见项目包括默认组、家目录基路径、非活动期、到期日、Shell、`skeleton` 目录和是否创建邮件池。输出以现场版本为准。`useradd -D -s /bin/bash` 等形式可以修改部分默认值，但不代表所有创建策略都由该命令管理。

② [知识点] `/etc/default/useradd` 与 `/etc/login.defs` 职责不同

- `/etc/default/useradd`：`useradd` 自身的默认属性，例如家目录基路径、默认 Shell、`skeleton` 目录等。

- `/etc/login.defs`：UID/GID 范围、系统账号范围、默认口令期限、家目录创建策略等多项系统级默认。

命令行参数优先表达当前账号目标。修改这些配置不会回溯更改现有账号；现有用户要用 `usermod`、`passwd` 或 `chage` 单独处理。

③ [操作] `/etc/skel` 只在创建家目录时复制初始内容

```
ls -la /etc/skel
useradd -m -k /etc/skel alice
```

隐藏文件同样属于 skeleton 内容。`-k` 需要与创建家目录的操作配合。修改 `/etc/skel` 不会自动同步既有用户家目录。

④ [边界] 系统账号与普通账号的默认行为不同

`useradd -r service1` 从系统账号范围选择 UID，并通常不应假设会自动创建家目录。服务账号是否需要家目录、Shell 或数据目录必须由服务设计决定；不要为了“能登录测试”给服务账号默认分配交互式 Shell。

⑤ [操作] 用显式参数避免依赖未知默认值

考试题指定家目录、Shell、UID、主组和期限时，建议显式写出：

```
useradd -u 2401 -g ops -G developers,audit \
-d /srv/home/lina -m -s /bin/bash \
-e 2030-12-31 lina
```

命令只是请求；之后仍要分别验证账号字段、组关系、家目录和期限。

⋮ {cheatsheet} Cheatsheet 参数覆盖默认；`useradd -D` 只展示部分默认；`/etc/login.defs` 影响后续创建，不回溯；`/etc/skel` 只有创建家目录时才复制；目标明确时用 `-m` 等显式参数。⋮

[操作专题] 创建精确的用户与组终态

稳定的创建流程不是先运行 `useradd` 再看缺什么，而是先确定组、数字身份、家目录、Shell、附加组和期限，再按依赖顺序创建。组必须先存在，附加组列表必须完整，口令状态和首次登录策略要单独设置。

① [操作] 创建普通组或指定 GID 的组

```
groupadd ops
groupadd -g 4200 ops
groupadd -r appsvc
```

`-g` 指定 GID；`-r` 创建系统组。创建前用 `getent group NAME` 和 `getent group GID` 检查名称与数字冲突。

② [操作] 组合 `useradd` 的身份参数

参数	作用
<code>-u UID</code>	指定 UID
<code>-g GROUP</code>	指定主组，组必须已存在
<code>-G G1,G2</code>	指定附加组列表
<code>-c COMMENT</code>	账号说明
<code>-s SHELL</code>	登录 Shell

```
useradd -u 2401 -g ops -G developers,audit \
-c 'Operations trainee' -s /bin/bash lina
```

③ [操作] 明确家目录是否创建及其位置

参数	作用
<code>-d PATH</code>	设置家目录字段
<code>-m</code>	创建家目录并复制 skeleton
<code>-M</code>	明确不创建家目录
<code>-k DIR</code>	指定 skeleton 目录，配合 <code>-m</code>

仅有 `-d` 不应被理解为“一定搬入数据”；创建任务中使用 `-d PATH -m` 才能同时表达字段和目录。

④ [操作] 设置账号到期和口令失效期

```
useradd -e 2030-12-31 -f 5 lina
```

`-e` 设置账号到期日；`-f` 设置口令过期后多少天转为失效。它们不是同一个时间点。更完整的期限策略通常在创建后用 `chage` 设置并验证。

⑤ [操作] 设置初始口令并强制下次修改

```
passwd lina
chage -d 0 lina
```

交互式 `passwd` 避免把明文口令写入命令历史。模拟题中可能出现 RHEL 扩展 `passwd --stdin`，但它不应成为真实运维默认示范。

⑥ [验证] 按对象分层验收

```
getent passwd lina
getent group ops
getent group developers
getent group audit
id lina
passwd -S lina
chage -l lina
stat /srv/home/lina
su - lina -c 'id'
```

最后一条建立新会话，验证实际取得的 UID、主组和附加组。若目标账号被要求下次修改口令，非交互式 `su -c` 可能受期限策略影响，应根据任务选择验证时机。

∴ {cheatsheet} Cheatsheet 先组后用户；`-g` 主组、`-G` 附加组；`-d` 设置字段、`-m` 创建目录；`-e` 账号到期、`-f` 口令过期后的失效期；口令和期限要单独设置并验证。∴

[操作专题] 修改账号：名称、UID、组、家目录和 Shell 不能混成一次“改用户”

账号修改往往影响不同对象。改名主要改变名称映射；改 UID 会改变数字身份；改家目录字段不一定移动数据；改组关系需要新会话生效。每次变更都应写清作用对象和数据边界，避免把一个参数的成功扩大为完整迁移。

① [操作] 修改登录名，但不要假设其他名称自动同步

```
usermod -l lina2 lina
```

这会修改登录名。它不会自动保证家目录路径、邮件别名、应用配置、计划任务说明或同名私有组都按新名称迁移。若目标还要求调整家目录和组名，应作为独立步骤执行并分别验证。

② [操作] 修改 UID 前先调查旧数字拥有的数据

```
old_uid=$(id -u lina2)
find /srv -xdev -uid "$old_uid" -print
usermod -u 2501 lina2
```

工具可能处理家目录中的一部分所有权，但账号在其他文件系统、共享目录、备份或应用数据库中的数字引用必须另行调查。修改后按新旧 UID 再查一次，不能只看 `id`。

③ [操作] 正确区分附加组替换和追加

```
usermod -G developers,audit lina2      # 设为这一完整列表
usermod -aG oncall lina2               # 在现有列表上追加
```

漏掉 `-a` 会把未列出的附加组移除。需要移除某个组时，应先得到完整目标列表，再用 `-G` 设置终态，而不是随意拼接命令。

④ [操作] 主组与附加组使用不同参数

```
usermod -g ops lina2
usermod -aG audit lina2
```

`-g` 改主组；`-G` 管附加组。修改后用 `id lina2` 查看数据库结果，并让用户在新会话中运行 `id` 验证实际凭据。

⑤ [操作] 修改家目录字段与移动数据是两个语义

```
usermod -d /srv/home/lina2 lina2      # 只设置新路径
usermod -d /srv/home/lina2 -m lina2    # 设置路径并请求移动内容
```

移动前检查目标路径是否存在、容量、挂载点和业务占用。跨文件系统、SELinux 上下文和正在运行的用户进程可能使真实迁移更复杂；本章记录这些为后续 live test 项，不以静态命令保证所有现场成功。

⑥ [操作] 修改登录 Shell 并识别边界

```
usermod -s /bin/bash lina2
usermod -s /sbin/nologin service1
```

`nologin` 阻止正常交互式登录 Shell，但不删除账号，不终止已有进程，也不能替代对其他认证和服务访问路径的调查。

∴ {cheatsheet} Cheatsheet `-l` 改登录名，不等于完成全局改名；`-u` 改数字身份，外部数据要查；`-G` 替换、`-aG` 追加；`-d` 改字段、`-d -m` 请求搬家；组和 Shell 变化要在新会话验证。∴

[操作专题] 组账号生命周期：创建、改名、改 GID 与删除

组的名称、GID、显式成员和“被哪些用户作为主组”是四个不同问题。`getent group` 能看到组记录和显式成员，却不能直接列出所有以它为主组的用户；删除组前必须补上这一层调查。

① [操作] 创建组并验证名称与数字

```
getent group ops
groupadd -g 4200 ops
getent group ops
```

不要用命令成功替代验证。若指定 GID，继续用 `getent group 4200` 确认数字映射唯一。

② [操作] 修改组名

```
groupmod -n operations ops
```

改名主要改变名称映射，不改变 GID。文件仍保存原 GID，因此名称展示会随映射变化。引用旧组名的 `sudoers`、应用配置或脚本不会自动全部更新；`sudo` 规则本身归第 10 章，本章只指出依赖调查。

③ [操作] 修改 GID 前调查数字所有权

```
old_gid=$(getent group operations | cut -d: -f3)
find /srv -xdev -gid "$old_gid" -print
groupmod -g 4300 operations
```

`groupmod -g` 改的是组数据库的数字映射。旧 GID 拥有的外部文件不应假设全部自动变更，修改后要按旧 GID 和新 GID 重新调查。

④ [诊断] 找出哪些用户把组作为主组

```
gid=$(getent group operations | cut -d: -f3)
getent passwd | awk -F: -v gid="$gid" '$4 == gid {print $1}'
```

`groupdel` 在组仍是某个用户主组时会拒绝。先为这些用户指定新的主组，再尝试删除。

⑤ [操作] 删除组并确认遗留数据

```
groupdel operations
getent group operations
find /srv -xdev -gid 4300 -print
```

组记录消失不代表旧 GID 的文件已处理。要根据业务要求迁移、重新归组或保留审计记录。

∴ {cheatsheet} Cheatsheet 组名与 GID 分开；`groupmod -n` 改名，`groupmod -g` 改数字；`getent group` 不会完整展示主组用户；`groupdel` 前查主组引用和旧 GID 文件。∴

[操作专题] 口令状态与账号期限：passwd 和 chage 各回答什么

“用户不能登录”可能来自口令锁定、没有有效口令、口令已过期并超过失效期、账号已到期，或登录 Shell 被禁止。把这些状态分开，才能选择最小修复，而不是反复重置口令。

① [操作] 设置口令并查看状态

```
passwd alice
passwd -S alice
```

普通用户通常只能修改自己的口令，root 可以管理任意本地账号。输出中的状态标记应结合现场帮助解释，不在静态教材中依赖一份固定本地化文本。

② [操作] 锁定和解锁口令

```
passwd -l alice
passwd -u alice
# 或使用 usermod -L / usermod -U 管理口令字段
```

锁定通常通过在口令哈希前加入不可匹配标记来阻止口令认证。它不保证禁用 SSH 公钥、其他认证机制、已有会话或服务进程。解锁前先确认账号确实存在可用口令，避免把无口令账号变成意外状态。

③ [操作] 设置口令期限

```
chage -m 2 -M 45 -W 7 -I 5 alice
```

参数	含义
-m	两次修改之间的最短天数
-M	口令最大有效天数
-W	到期前警告天数
-I	口令到期后再过多少天使账号因口令失效

④ [操作] 设置账号到期日

```
chage -E 2030-12-31 alice
chage -l alice
```

账号到期日与口令最大期限独立。一个账号可以有尚未到期的口令，但账号整体已到期；也可以账号没有固定到期日，但口令需要周期更换。

⑤ [操作] 强制下次登录修改口令

```
chage -d 0 alice
```

这会把上次修改日设置为需要立即更新的状态。启用后应在合适的交互登录路径验证；非交互命令可能因必须改口令而失败，这不是服务故障。

⑥ [知识点] /etc/login.defs 的默认期限只影响后续创建

PASS_MIN_DAYS、PASS_MAX_DAYS 和 PASS_WARN_AGE 等默认值不会自动重写现有 shadow 条目。修改全局策略后，应分别处理既有账号并用 chage -l 验证。

⋮ {cheatsheet} Cheatsheet passwd -S 看口令状态，chage -l 看期限；锁口令不等于完全停用账号；-M/-m/-W/-I 管密码生命周期，-E 管账号到期，-d 0 强制下次修改。⋮

[知识专题] 停用账号不是一个开关：密码锁定、账号到期、Shell 与已有会话

“停用用户”必须先翻译成具体终态。是只禁止口令登录，还是禁止所有新的登录？是否允许服务继续以该 UID 运行？已有 Shell 和进程是否需要另行处置？数据要保留还是删除？不同机制作用于不同层，不能用一个命令代替全部答案。

① [比较] 密码锁定只针对口令字段

passwd -l 或 usermod -L 锁定口令认证。它适合在不删除账号和数据的情况下阻止密码使用，但不应被描述为“用户从此不能通过任何方式访问系统”。SSH 密钥属于第 20 章，当前只记住这一边界。

② [比较] 账号到期限制新的账号使用

`chage -E DATE` 或 `usermod -e DATE` 设置账号到期。需要立即使账号过期时，应依据现场 man page 选择明确的过去日期或工具推荐值，并用 `chage -l` 验证。账号到期不会自动结束已经运行的进程。

③ [比较] `nologin` 控制正常交互式 Shell

把 Shell 设置为 `/sbin/nologin` 常用于不需要交互登录的服务账号。它不是数据删除机制，也不阻止 `systemd` 以该用户启动服务进程。是否影响特定协议取决于该协议是否真正调用登录 Shell。

④ [边界] 已有会话和进程需要独立调查

锁定、过期或改 Shell 后，使用 `who`、`w`、`logins` 和进程查询确认是否仍有会话或进程。如何安全终止进程归第 11 章；本章只要求管理员不要声称账号状态变化已经清除了运行实例。

⑤ [决策] 停用优先保留可审计身份，再决定删除

离职或临时停用通常先记录 UID/GID、组、家目录和数据清单，再锁定口令、设置到期，并保留账号记录以避免数字身份失去名称解释。删除应在数据移交和审批后进行，而不是停用的默认第一步。

⋮ {cheatsheet} Cheatsheet 锁口令、过期账号、`nologin` 和删除作用层不同；任何一种都不会自动终止已有进程；停用先保留证据和数字身份，删除要等数据边界明确。⋮

[操作专题] 当前会话身份变化：`su`、`su -` 与 `newgrp`

管理员经常在数据库正确时仍看到功能失败，原因是测试发生在旧会话。会话工具的价值不是“看起来换了提示符”，而是建立新的进程凭据和环境，再用 `id`、`pwd`、`env` 证明实际状态。

① [操作] `su USER` 建立目标用户 Shell，但保留较多当前环境

```
su alice
id
pwd
```

不带 `-` 的 `su` 主要改变用户身份，环境和工作目录可能保留较多原会话内容。它适合观察身份差异，但不能假设完全模拟正常登录。

② [操作] `su - USER` 请求登录式环境

```
su - alice
id
pwd
printf '%s\n' "$HOME" "$SHELL" "$PATH"
```

`su -` 通常切换到目标家目录并建立登录式环境。具体变量由 `util-linux`、PAM 和系统配置共同影响，因此最终以会话内证据为准。

③ [验证] 组变更后用新会话确认实际附加组

```
id alice
su - alice -c 'id'
```

第一条确认数据库解析，第二条确认新会话取得的凭据。若二者仍不同，再调查 NSS 缓存、PAM 或远程身份源，而不是重复执行 `usermod`。

④ [操作] `newgrp GROUP` 在新 Shell 中切换当前主组

```
id
newgrp developers
id
exit
```

用户必须有资格使用目标组。`newgrp` 启动新的 Shell，并把该 Shell 的主组切换为目标组；退出后返回上层会话。它不会永久改写 `/etc/passwd` 中的默认主组。

⑤ [边界] Shell 嵌套和测试完成后的恢复

连续使用 `su` 和 `newgrp` 会产生嵌套 Shell。测试后使用 `exit` 一层层返回，并再次运行 `id` 确认当前身份。不要在不清楚所处 Shell 层级时执行破坏性管理命令。

⋮ {cheatsheet} Cheatsheet `su` 改身份但可能保留环境，`su -` 更接近登录会话；数据库 `id USER` 与会话内 `id` 分别验证；`newgrp` 临时切换新 Shell 的主组，`exit` 返回。⋮

[操作专题] 删除用户和组：账号记录、家目录与其他数据的边界

删除是账号生命周期中风险最高的操作，因为名称记录消失后，文件仍保留数字 UID/GID。`userdel -r` 的作用范围有限，不能代替全系统数据调查；组删除也不会自动重写旧 GID 的所有文件。

① [操作] `userdel USER` 只删除账号记录，不默认删除家目录

```
userdel alice
```

执行前记录 UID、主组、附加组、家目录和数据清单。账号删除后，原文件可能显示数字所有者或无人拥有状态。

② [操作] `userdel -r USER` 删除默认家目录和邮件池，但不是全盘清理

```
userdel -r alice
```

它主要处理账号数据库、用户家目录和邮件池。用户在 `/srv`、其他挂载点、共享存储和应用目录中的文件需要单独处理。不得把 `-r` 描述成“删除该用户所有数据”。

③ [调查] 删除前按数字 UID/GID 查找数据

```
uid=$(id -u alice)
gid=$(id -g alice)
find /home /srv -xdev \( -uid "$uid" -o -gid "$gid" \) -print
```

真实系统应按挂载点和业务目录扩展范围，记录结果并决定归档、移交、重新所有或保留。

④ [验证] 删除后查找无名称解释的数据

```
getent passwd alice
find /home /srv -xdev \( -nouser -o -nogroup \) -print
```

`-nouser` 与 `-nogroup` 是事后审计入口，不是自动删除清单。不要在没有审批的情况下把匹配结果直接交给 `rm`。

⑤ [边界] 避免 UID/GID 过早重用

新账号若取得旧 UID，会立即被解释为旧文件的所有者。停用阶段保留账号记录、删除阶段保留数字清单、创建阶段检查数字冲突，是防止信息泄漏的三道边界。

⑥ [操作] 用户私有组是否删除必须重新查询

`userdel` 对同名用户私有组的处理受组状态和配置影响。执行后使用 `getent group USER` 核对；若组仍存在，再确认是否有其他成员、是否作为其他用户主组，以及旧 GID 文件是否已处理，最后才决定 `groupdel`。

⋮ {cheatsheet} Cheatsheet `userdel` 删账号不删家目录；`-r` 也不覆盖任意外部数据；删除前按 UID/GID 查，删除后用 `-nouser/-nogroup` 审计；不要过早重用数字身份；同名私有组要重新查询。⋮

[诊断专题] 从“用户不能登录”或“改了仍不生效”推进到最小修复

账号故障不能从重置口令开始。先确定账号是否被解析，再分别检查口令、期限、Shell、组和会话。每一步都应选择能排除一类假设的证据，避免把多种状态揉成一句“账号有问题”。

① [诊断] 账号存在但无法登录

```
症状：用户报告认证失败
→ getent passwd USER：账号是否解析，Shell 与家目录是什么
→ passwd -S USER：口令是否锁定或未设置
→ chage -l USER：口令和账号是否到期
→ 检查目标访问方式是否使用登录 Shell
→ 最小修复：只修改实际异常的字段
→ 建立新会话再验证
```

若口令可用、期限正常、Shell 正常，再进入具体认证服务日志；SSH 的密钥和服务端配置归第 20 章。

② [诊断] 已把用户加入组，但访问仍失败

当前证据: `id USER` 已显示新组
假设 A: 用户仍在旧会话
下一条证据: 让用户在当前 Shell 运行 `id`
最小修复: 重新登录或建立新 `su` 会话
再验证: 新会话 `id` + 目标功能测试

若新会话也没有该组，检查 `getent group`、NSS 来源和 `usermod -aG` 是否作用于本地账号。

③ [诊断] 改了家目录，但登录后仍到旧位置或没有数据

```
getent passwd USER      # 字段是否已改
stat OLD NEW            # 目录是否存在，数据是否移动
su - USER -c 'pwd; id'  # 新登录式会话看到什么
```

字段正确但新目录无数据，通常说明只使用了 `-d` 而没有搬迁，或移动过程失败。先补齐数据，再检查权限和 SELinux；权限细节归后续章节。

④ [诊断] `groupdel` 报组是某用户主组

```
gid=$(getent group GROUP | cut -d: -f3)
getent passwd | awk -F: -v gid="$gid" ' $4 == gid {print $1}'
```

为列出的用户选择正确新主组，逐个 `usermod -g NEWGROUP USER`，验证后再删除。不要用强制参数绕过身份关系。

⑤ [诊断] 删除账号后目录显示数字所有者

记录数字 UID/GID，查找相关文件，确认是否应归档、移交或重新分配。若尚未重用 UID，可以在受控情况下临时重建相同数字的账号帮助恢复名称解释，但这属于恢复决策，不能在未调查时直接执行。

⑥ [诊断] 锁定账号后用户仍有活动

先区分活动类型：已有 Shell、已有进程、SSH 密钥新登录、服务账号进程或其他协议。锁口令只能排除密码认证。根据目标分别处理账号到期、访问服务配置和运行进程；不要把一个层的修复扩大为所有层都已关闭。

∴ {cheatsheet} Cheatsheet 登录失败按“解析→口令→期限→Shell→具体认证服务”调查；组变更要比较数据库与旧会话；家目录字段和数据分开；组删除先查主组引用；锁口令后仍活动先识别访问类型。∴

[经典任务] 创建指定 UID、组、家目录和期限的项目账号

环境与当前状态

- `developers` 与 `audit` 组已经存在。
- `/srv/home` 已存在，`/etc/skel/.company-profile` 已准备。
- `ops` 组和 `lina` 用户尚不存在。
- 系统中可能已有自定义 UID/GID，不能假设 `2401` 和 `4200` 一定空闲。
- 当前无可由本会话控制的 RHEL 9 VM，以下为推荐静态操作与验证路径。

目标终态

1. 创建组 `ops`，GID 为 `4200`。
2. 创建用户 `lina`，UID 为 `2401`。
3. `lina` 的主组为 `ops`，附加组为 `developers,audit`。
4. 家目录为 `/srv/home/lina`，实际创建并复制 `skeleton` 内容。
5. 登录 Shell 为 `/bin/bash`。
6. 账号到期日为 `2030-12-31`。
7. 口令最短期限 2 天、最长期限 45 天、提前 7 天警告、过期后 5 天失效。
8. 设置题目提供的临时口令，并要求下次交互登录时修改。

限制条件

- 不直接编辑四个账号数据库。
- 不使用非唯一 UID/GID 绕过冲突。
- 不用 `-G` 破坏题目未要求移除的既有组；本任务为新账号，可一次给出完整附加组列表。
- 不把命令成功或单条 `id` 输出扩大为全部终态正确。

- 不在命令行明文写出临时口令。

验收证据

层次	证据
组名称与 GID	<code>getent group ops</code>
用户字段	<code>getent passwd lina</code>
UID、主组、附加组	<code>id lina</code>
家目录与 skeleton	<code>stat /srv/home/lina</code> 、检查 <code>.company-profile</code>
口令状态	<code>passwd -S lina</code>
期限	<code>chage -l lina</code>
新会话身份	在允许的验证时机执行 <code>su - lina</code> 后 <code>id</code>

[参考解答] 创建精确项目账号

① 调查名称与数字冲突

```
getent group ops
getent group 4200
getent passwd lina
getent passwd 2401
getent group developers
getent group audit
```

预期：ops、4200、lina、2401 没有现存映射；两个附加组存在。任何冲突都应先停下并按题目调整，不使用非唯一参数强行创建。

② 创建组

```
groupadd -g 4200 ops
getent group ops
```

验证组名与 GID，而不是只看退出码。

③ 创建用户及家目录

```
useradd -u 2401 -g ops -G developers,audit \
-d /srv/home/lina -m -s /bin/bash \
-e 2030-12-31 lina
```

关键参数：

- `-u 2401`：指定 UID；
- `-g ops`：设置主组；
- `-G developers,audit`：设置完整附加组列表；
- `-d ... -m`：同时设置字段并创建目录；
- `-s /bin/bash`：设置登录 Shell；
- `-e`：设置账号到期日。

④ 设置口令与期限

```
passwd lina
chage -m 2 -M 45 -W 7 -I 5 lina
chage -d 0 lina
```

`passwd` 交互输入题目给定的临时口令。`chage -d 0` 会影响后续交互登录，因此应在完成无需登录的静态验证后再设置，或根据考场操作顺序安排。

⑤ 分层验证

```
getent passwd lina
getent group ops
getent group developers
getent group audit
id lina
passwd -S lina
chage -l lina
stat -c '%n %u:%g %U:%G' /srv/home/lina
ls -la /srv/home/lina
```

重点判断：

- passwd 条目中的 UID、主 GID、家目录 and Shell；
- id 中的主组与两个附加组；
- 家目录确实存在并包含 .company-profile；
- chage -l 中的最短、最大、警告、失效和账号到期符合目标；
- 口令状态不是锁定或无口令。

⑥ 新会话验证边界

需要确认实际登录身份时，可先暂时不设置 `chage -d 0`，完成：

```
su - lina -c 'id; pwd'
```

随后再执行 `chage -d 0 lina`。若已经强制下次改口令，非交互式 `su -c` 可能失败，这是期限策略的预期影响，不能误判为组配置错误。

典型错误

- 只写 `-d /srv/home/lina`，目录字段改变但目录未创建。
- 把 `-g developers` 当作附加组参数，错误改变主组。
- 用空格分隔 `-G` 的组名，而不是逗号列表。
- 设置了全局 `/etc/login.defs` 后，误以为既有用户期限会自动变化。
- `id lina` 正确便停止验收，没有检查家目录、口令和期限。

[经典任务] 安全停用外部人员账号并评估删除

环境与当前状态

- 本地账号 `vendor1` 存在，可能仍有登录会话和运行进程。
- 家目录中有资料，`/srv/vendor-data` 下可能有该 UID 拥有的数据。
- 业务要求立即阻止新的账号使用，但当天禁止删除任何数据。
- 一周后将依据数据移交结果决定是否删除账号。

第一阶段目标

1. 记录用户名、UID、主 GID、附加组、家目录、Shell、口令和期限。
2. 记录账号拥有的业务数据与当前会话线索。
3. 锁定口令并使账号立即到期，保留账号记录与全部数据。
4. 明确记录：已有会话和进程不会被这些账号操作自动结束。
5. 验证数据库中的停用状态。

第二阶段删除评估

1. 按保存的 UID/GID 搜索家目录外的数据。
2. 确认归档、移交或重新所有方案。
3. 决定使用 `userdel` 还是 `userdel -r`，并说明两者都不能自动处理任意外部路径。
4. 检查同名用户私有组和旧 GID 文件。
5. 不使用 `--force` 绕过调查。

[参考解答] 安全停用与删除评估

① 建立身份和期限基线

```
getent passwd vendor1
id vendor1
passwd -S vendor1
chage -l vendor1
uid=$(id -u vendor1)
gid=$(id -g vendor1)
printf 'uid=%s gid=%s\n' "$uid" "$gid"
```

把输出保存到变更记录。不要在删除后才尝试猜测旧数字。

② 调查数据和活动线索

```
find /home /srv -xdev \( -uid "$uid" -o -gid "$gid" \) -print
who
w
ps -u vendor1 -f
```

`who`、`w` 和 `ps` 只用于记录活动线索。终止会话和进程属于第 11 章，应经过业务确认后处理。

③ 锁定口令并使账号到期

一种明确的静态路径是：

```
passwd -l vendor1
chage -E 0 vendor1
```

这里用 `0` 把账号到期字段设置为 Unix epoch 起始日，使其在当前日期下已经到期；执行后仍必须用 `chage -l` 核对现场解释。不要只运行 `passwd -l` 就声称账号所有访问方式都已禁用。

④ 验证停用状态

```
passwd -S vendor1
chage -l vendor1
getent passwd vendor1
```

账号记录、UID/GID 和家目录字段仍应存在；口令显示锁定，账号到期日显示已过去。再记录已有会话或进程仍需独立处理的事实。

⑤ 一周后的删除决策

若要求保留家目录：

```
userdel vendor1
```

若已经完成家目录归档并明确要求删除默认家目录和邮件池：

```
userdel -r vendor1
```

两种方式执行前都要再次按保存的 UID/GID 调查 `/srv/vendor-data` 等外部路径。执行后验证：

```
getent passwd vendor1
getent group vendor1
find /home /srv -xdev \( -uid "$uid" -o -gid "$gid" -o -nouser -o -nogroup \) -print
```

⑥ 典型错误

- 先 `userdel -r`，再发现业务数据尚未移交。
- 锁定口令后声称 SSH 密钥、已有会话和服务进程都已停止。
- 只查家目录，不查 `/srv`、挂载卷和共享目录。
- 删除账号后立即创建新用户，导致旧 UID 被重用。
- 把 `find ... -nouser` 的结果直接批量删除，没有人工审批。

[本章收束] 用生命周期和证据管理身份

本章把账号管理从“记命令”还原成一条对象链：名称先解析为 UID/GID，主组与附加组共同构成数据库身份，四个账号数据库保存公开属性、口令与期限，创建和修改命令推进持久状态，而新登录、`su -` 或 `newgrp` 才让新的组凭据进入进程。停用和删除又分别作用于口令、期限、Shell、账号记录与数据，任何一步都不能被概括为“用户没了”。

工作方法

- 确定目标身份
 - 读取名称与数字基线
 - 区分数据库字段和会话凭据
 - 选择最小生命周期操作
 - 验证账号、组、期限和家目录
 - 建立新会话验证实际身份
 - 删除或重用 UID/GID 前调查数据

操作前尽量记录用户名、UID、主 GID、附加组、家目录、Shell、口令状态与期限；操作后按同样维度复查。这样既能服务考试评分，也能在真实系统中保留审计和回退线索。

主要判断表

目标或症状	首要证据	该证据能证明	仍不能证明
用户是否被系统解析	<code>getent passwd USER</code>	当前 NSS 能返回账号条目	当前旧会话是否已采用新组
用户属于哪些组	<code>id USER</code>	数据库解析出的主组和附加组	既有 Shell 的实际组集合
口令是否锁定	<code>passwd -S USER</code>	口令字段的状态摘要	SSH Key、已有会话或服务进程是否停止
密码或账号是否到期	<code>chage -l USER</code>	密码期限和账号到期字段	具体认证服务的完整准入结果
家目录是否完成迁移	<code>passwd</code> 字段 + <code>stat</code> + 内容核对	路径、数字所有权与数据是否到位	文件访问权限是否符合业务规则
删除是否留下数据	保存的 UID/GID + <code>find</code>	指定范围中的数字所有权遗留	未扫描挂载点中是否还有数据

本章总 Cheatsheet

```
# 查询
getent passwd USER
getent group GROUP
id USER
passwd -S USER
chage -l USER
useradd -D

# 创建
groupadd -g GID GROUP
useradd -u UID -g PRIMARY -G G1,G2 -d HOME -m -s SHELL USER
passwd USER
chage -m MIN -M MAX -W WARN -I INACTIVE -E DATE USER

# 修改
usermod -l NEW OLD
usermod -aG GROUP USER
usermod -G G1,G2 USER
usermod -g PRIMARY USER
usermod -d HOME -m USER
usermod -s SHELL USER
groupmod -n NEW OLD
groupmod -g GID GROUP

# 会话
su USER
su - USER
newgrp GROUP

# 停用与删除
passwd -l USER
chage -E DATE USER
userdel USER
userdel -r USER
groupdel GROUP
```

向下一章交接

本章已经确定“谁在访问”：用户的 UID/GID、组集合和会话凭据。第 08 章《传统权限、umask 与特殊权限位》将继续回答“这些身份如何被文件模式位解释”，包括所有者、所属组、其他用户、默认创建模式以及 SUID、SGID 和 Sticky 位。本章不提前展开权限计算，但会把可靠的身份与组证据交给下一章。

RHEL 9 · RHCSA 实操讲义

传统权限、umask 与特殊权限位

从访问主体到路径逐级求值：把 owner/group/other、文件与目录 rwX、创建屏蔽和特殊位放进同一条证据链。

对象模型操作语义验证诊断经典任务

大字号阅读版

本章阅读导航

先抓住一条主线：权限不是一串孤立数字。每一次访问都由一个具有实际凭据的进程发起，内核逐级解析路径、为每个对象选中 owner/group/other 中唯一一类，再依据具体动作检查文件或目录的权限；新对象还要经过 `umask` 与父目录特殊位的影响。

- 01 确认真实主体有效 UID、有效 GID 与附加组
- 02 逐级解析路径每一级父目录都可能因缺少 `x` 阻断
- 03 选择权限类 owner / group / other 只命中一类，不叠加、不回退
- 04 把动作映射到权限文件与目录的 `rwX` 含义不同
- 05 处理创建与特殊位 `umask`、`setuid`、`setgid`、`sticky`
- 06 用真实用户验收静态状态、正向功能与负向隔离分层证明

专题地图

知识专题	传统 DAC 如何选择 owner/group/other
知识专题	文件与目录的 <code>rwX</code> 及父目录控制
操作专题	使用 <code>ls/stat/id/namei</code> 建立证据
操作专题	<code>chmod</code> 、 <code>chown</code> 、 <code>chgrp</code> 的最小变更
知识/操作	<code>umask</code> 计算、当前状态与持久验证
知识/操作	<code>setuid/setgid/sticky</code> 与组协作目录
诊断专题	路径不可达与递归修改风险
经典任务	协作目录：文件可读但路径不可达

阅读时持续回答

- 当前真正发起操作的进程是谁？
- 路径中第一处可能阻断的位置在哪里？
- 当前对象会选中 owner、group 还是 other？
- 原始动作需要文件权限还是父目录权限？
- 新对象的请求模式是什么，`umask` 清除了哪些位？
- 特殊位作用于可执行文件还是目录？
- 当前证据能证明什么，不能证明什么？
- 下一条最有区分度的证据是什么？

第 08 章 · 正文

用户看到 `Permission denied` 时，最容易犯的错误是盯着最终文件反复执行 `chmod`。Linux 的传统访问控制不是“看一行 `ls -l` 就结束”：访问请求由一个具有有效 UID、有效 GID 和附加组的进程发起；内核先逐级解析路径，再为每个相关 inode 选择 owner、group 或 other 中唯一一类权限；创建、删除和重命名还主要受父目录控制；新对象的权限则取决于程序请求的模式、进程的 `umask` 以及父目录是否具有 `setgid` 等属性。

本章沿着“主体 → 路径 → 权限类 → 动作 → 创建规则 → 功能证据”推进。前一章已经建立用户、组与会话身份，本章不重复账号生命周期；ACL 留给第 09 章，`sudo` 授权留给第 10 章，SELinux 留给第 28/29 章。这里使用 `sudo -u` 只为建立指定普通用户的验证进程。

概念

DAC (Discretionary Access Control, 自主访问控制) 是由对象 owner、group 与传统 mode bits 共同表达的访问控制层。它回答的是“这个进程按当前身份，能否对这个 inode 完成某个具体动作”，而不是抽象地判断“某用户有没有权限”。观察 DAC 时，要把进程凭据、路径分量和 inode 元数据放在一起。

概念

owner / group / other 是三个互斥的权限类别。内核先比较有效 UID；未命中 owner 时再检查有效 GID 与附加组；都未命中才使用 other。命中一类后不会把另外两类相加，也不会因为该类权限不足而回退到更宽松的一类。

概念

普通文件的 `rwX` 分别控制读取内容、修改现有内容和直接执行。文件自身的 `w` 不决定能否删除文件名，因为名称属于父目录；文件有 `r` 也不代表路径一定可达。

概念

目录的 `rwX` 分别控制枚举名称、修改目录项和 search/穿越。访问深层文件时，路径中的每一级目录都需要目标主体所选权限类的 `x`；创建、删除和重命名通常要求父目录的 `w+x`。

概念

`umask` 是进程持有的创建屏蔽状态，不是已有对象的默认 mode，也不是普通减法。最终传统权限等于程序请求模式按位清除 mask 中的位；它只能删除请求中的权限，不能凭空增加权限。

概念

`setuid`、`setgid` 与 `sticky` 是位于普通 `rwX` 之外的特殊位。`setuid/setgid` 可执行文件影响执行后进程的有效身份；`setgid` 目录影响新对象的 group 继承；`sticky` 目录限制公共可写目录中的删除与重命名。必须结合对象类型和相应执行位一起解释。

操作语义

以下入口分别观察对象元数据、修改 mode、修改 owner/group、控制新对象权限、展开路径和切换验证主体。先理解命令作用对象，再记关键形式。

`ls / stat`

SYNOPSIS

```
ls [OPTION]... [FILE]...
stat [OPTION]... FILE...
```

快速读取对象类型、符号权限、八进制 mode、owner 与 group；它们证明静态元数据，不证明目标用户一定能完成原始动作。

重要参数 / 形式

`ls -ld DIR`

查看目录本身，而不是列出目录内容。

`ls -la DIR`

包含隐藏名称，适合盘点目录项。

`stat -c FORMAT PATH`

固定输出 `%A %a %U:%G %u:%g %n``，便于变更前后比较。

chmod

SYNOPSIS

```
chmod [OPTION]... MODE[,MODE]... FILE...
```

修改 mode bits。符号模式适合在现状上做最小增量，八进制适合写出精确终态。

重要参数 / 形式

`u/g/o/a + - = rwxXst`

显式选择权限类、操作符与权限位。

`0640 / 0750`

普通文件与目录的完整八进制终态。

`2770 / 1777`

首位表达 setgid 或 sticky 等特殊位。

`-R`

递归修改，执行前必须盘点对象类型、符号链接和文件系统边界。

`X`

只对目录或原本已有执行位的对象添加执行/search 位。

chown / chgrp

SYNOPSIS

```
chown [OPTION]... [OWNER][:[GROUP]] FILE...
chgrp [OPTION]... GROUP FILE...
```

修改 inode 的 owner UID 或 group GID；它们不自动修改普通 `rwX`，也可能触发 setuid/setgid 被清除。

重要参数 / 形式

`chown OWNER:GROUP PATH`

同时设置 owner 与 group。

`chown :GROUP PATH`

只修改 group。

`chgrp GROUP PATH`

用更直接的命令表达“只改 group”。

`-R`

递归改变所有权；先确认绝对路径、挂载点、链接和对象集合。

umask

SYNOPSIS

```
umask [-p] [-S] [MODE]
```

查看或改变当前 Shell 进程及其后代的创建屏蔽状态。它不修改已有对象，持久性必须通过新会话验证。

重要参数 / 形式

`umask`

显示当前数值 mask。

`umask -S`

以符号形式显示允许保留的权限。

`umask 0007`

当前 Shell 及其后代屏蔽 other 的全部普通权限。

`requested & ~umask`

正确的按位清除模型。

namei

SYNOPSIS

```
namei [OPTIONS] PATH...
```

把完整路径拆成每一级分量，适合寻找第一处缺少目录 search 权限的阻断点。

重要参数 / 形式

`namei -l PATH`

逐级显示对象类型、符号权限、owner 与 group。

从上到下判断

结合目标用户的 `id`，确认每一级选中 owner/group/other 哪一类。

第一阻断点

优先修复第一处有区分度的问题，不一次修改多级目录。

`sudo -u / id`

SYNOPSIS

```
sudo -u USER [--] COMMAND [ARG]...
id [USER]
```

在本章只用于建立目标普通用户的测试进程并观察其凭据；sudoers 规则本身属于第 10 章。

重要参数 / 形式

`id USER`

查看账号数据库解析出的 UID、GID 与组。

`sudo -u USER id`

观察新建测试进程实际携带的身份。

`sudo -iu USER bash -lc 'COMMAND'`

需要登录式 Bash 环境时，通过目标用户的登录 Shell 执行命令；最终仍应按题目提供的真实登录方式复核。

正向 + 负向

目标用户完成原始动作，无关用户保持失败。

知识专题 从主体到 inode：传统 DAC 如何选择权限类

权限题的第一步不是换算八进制，而是确定访问主体和最终会被选中的权限类。只有把“进程身份”“对象所有权”和“操作类型”分开，才不会出现 owner 位不足时错误地继续借用 group 或 other 权限。

① 知识点 访问主体是进程凭据，不是屏幕上的用户名

每个进程都携带有效用户 ID、有效组 ID 和附加组集合。文件访问通常依据这些有效身份判断。`id USER` 查询账号数据库中该用户应具有的身份；在目标用户会话中执行 `id`，才能观察该进程当前实际携带的身份。管理员刚把用户加入新组时，旧登录会话可能仍保留旧的附加组集合，因此“数据库中已经是组成员”和“当前进程已获得组身份”必须分开验证。

```
id alice          # 查询 alice 应有的 UID/GID 与组
sudo -u alice id  # 以目标身份建立测试进程并查看身份
```

本章假定用户和组已经由第 07 章《用户、组与账号生命周期》建立，不重复展开账号创建和组成员维护。

② 知识点 owner、group、other 三类只选择一类

对一个对象进行传统权限检查时，可以使用以下判断顺序：

```
访问进程的有效 UID 等于对象 owner UID?
├ 是：只使用 owner 位
└ 否：进程的有效 GID 或附加组命中对象 group GID?
  ├── 是：只使用 group 位
  └ 否：使用 other 位
```

“最具体的匹配优先”不等于“权限不足时继续回退”。例如某文件为：

```
-r--rw----  alice project report.txt
```

alice 同时属于 project 组时，仍只使用 owner 的 `r--`，不能把 group 的 `rw-` 加到 owner 上，也不会在 owner 缺少写权限后继续尝试 group。

③ 知识点 权限判断必须先明确操作

“能否访问文件”不是一个可判定问题。需要先改写成具体动作：

- 读取文件内容；
- 修改现有内容；

- 直接执行文件；
- 列出目录中的名称；
- 穿越目录并访问已知名称；
- 在目录中创建新条目；
- 删除或重命名目录项。

相同 mode 对不同动作可能给出不同结果。排错时应复现原始动作，而不是只执行 `ls` 或只执行 `test -r`。

④ 知识点 root 与 capabilities 是普通 DAC 模型的边界

特权进程可以绕过许多普通 DAC 检查，但这不代表 mode 无意义，也不能用 root 测试代替普通用户验收。对普通可执行文件，即使是特权进程，在没有任何执行位时也不能简单把它当作普通直接执行目标；挂载选项、解释器、SELinux 等也可能继续限制执行。

考试和日常变更中，默认应以题目指定的普通用户验证，不把“root 成功”扩大为业务终态正确。

[Cheatsheet] `id USER` 看账号应有身份；目标会话中的 `id` 看当前进程身份；owner/group/other 只选一类，不相加、不回退；先把“访问”改写成读取、写入、穿越、创建、删除或执行。

知识专题 文件与目录的 `rwX`：同一字母控制不同能力

普通文件保存内容，目录保存“名称到 inode”的映射，因此相同的 `rwX` 在两类对象上的含义不同。权限诊断必须同时检查最终对象和父目录；很多“文件权限正确但仍失败”的问题，实际阻断点在路径中间的目录。

① 知识点 普通文件的 `r`、`w`、`x`

r 读取内容

允许读取普通文件的数据；它不证明父路径可穿越。

w 修改内容

允许修改或截断现有内容；删除文件名主要看父目录。

x 直接执行

允许把文件作为程序启动；格式、解释器、挂载和其他安全层仍需成立。

脚本即使可以被解释器显式读取，例如 `bash script.sh`，也不等同于脚本本身具有直接执行权限。题目要求“可执行”时，应按题意验证直接执行路径，而不是用解释器绕过评分对象。

② 知识点 目录的 `r`、`w`、`x`

r 枚举名称

允许读取目录项名称；没有 `x` 时，很多条目元数据仍无法取得。

w 修改目录项

允许创建、删除和重命名名称；通常必须与 `x` 配合。

x search / 穿越

允许访问已知名称并继续路径解析；即使没有 `r`，知道准确名称时仍可能访问。

常见组合：

- `r-x`：可列出并访问，但不能创建或删除；
- `--x`：不能枚举名称，知道准确名称时可继续访问；
- `r--`：可看到名称，但不能可靠取得条目元数据或进入子路径；
- `-wx`：可以在已知目录中创建、删除和改名，但不能正常列出全部名称；这是高风险而少见的组合。

③ 知识点 路径中的每一级目录都需要 `x`

访问 `/srv/team/reports/q1.txt` 不只检查 `q1.txt`：

```
/           需要 search
/srv        需要 search
/srv/team   需要 search
/srv/team/reports 需要 search
q1.txt      根据动作需要 r、w 或 x
```

任一父目录缺少目标主体所选权限类的 `x`，路径解析都会在该处停止。最终文件即使是 `0644`，也可能无法读取。

④ 知识点 创建、删除和重命名首先由父目录控制

文件名属于父目录的目录项。创建新文件需要父目录的 `w+x`；删除或重命名现有文件也主要需要父目录的 `w+x`，而不是文件自身的 `w`。因此：

- 只读文件可能被拥有父目录 `w+x` 的用户删除；
- 可写文件若父目录不可写，用户可以改内容但不能删除名称；
- `sticky` 位可进一步限制公共可写目录中的删除和重命名。

删除目录还必须满足对象类型和空目录等额外条件；本章关注权限控制点，不把命令自身的结构条件混同为权限。

⑤ 知识点 目录权限可以屏蔽更深层对象

传统权限通常不自动从父目录复制到已有子对象，但父目录可以通过路径穿越能力屏蔽其内部所有内容。不能因为子文件是 `0644` 就推断所有用户可读，也不能因为目录是 `0755` 就推断里面所有文件可读。

[Cheatsheet] 文件: **r** 读内容、**w** 改内容、**x** 直接执行; 目录: **r** 枚举名称、**w** 改目录项、**x** 穿越; 创建/删除/改名看父目录 **w+x**; 路径每一级都要 **x**。

操作专题 使用 `ls`、`stat`、`id` 与 `namei` 建立权限证据

查询工具的职责不同。`ls -l` 适合快速阅读, `stat` 适合精确取值, `id` 确认主体, `namei -l` 则把路径逐级展开。权限排错应组合使用, 而不是把任何一条命令当作完整证明。

① 操作用 `ls -l` 和 `ls -ld` 区分对象与目录内容

作用对象：指定路径或目录内容。基本形式：

```
ls -l FILE
ls -ld DIR
ls -la DIR
```

`ls -l DIR` 默认列出目录内容；要看目录本身，应使用 `-d`。长格式的首字符表示类型，之后九位依次为 owner、group、other 的 `rwx`。例如：

```
drwxrws---
| | | |
| | | | other
| | | | group; 执行位位置为 s, 表示 setgid 与 x 同时存在
| | | | owner
| | | | 目录
```

权限串末尾可能出现 + 或 。它们提示还存在 ACL 或安全上下文等扩展信息，但不在本章展开；出现这类标记时，应把它作为转入第 09 章 ACL 或第 28/29 章 SELinux 的信号，而不是忽略。

② 操作 用 `stat` 同时读取符号与八进制状态

作用对象：单个文件系统对象。典型形式：

```
stat PATH
stat -c '%F | %A | %a | %U:%G | %u:%g | %n' PATH
```

关键字段:

- **%F**：对象类型；
- **%A**：符号权限；
- **%a**：八进制权限；
- **%U:%G**：名称形式 owner/group；
- **%u:%g**：数字 UID/GID；
- **%n**：路径名。

名称解析异常时，数字 UID/GID 比显示名称更接近 inode 真相。变更前后使用相同格式，便于形成可比较基线。

③ 操作 用 id 区分账号记录与当前进程身份

```
id alice
sudo -u alice id
```

第一条查询系统为 `alice` 解析出的身份；第二条建立实际测试进程。若题目刚修改过组成员，真实登录会话可能还需要重新建立，不能仅依据 `id alice` 推断旧会话已刷新。

④ 操作 用 `namei -l` 逐级展开路径

作用对象: 完整路径中的每个分量。 典型形式:

```
namei -l /srv/team/reports/q1.txt
```

调查时从上到下寻找第一处目标主体缺少 `x` 的目录。`namei` 显示的是静态 mode 与 owner/group；仍需结合 `id` 判断该用户在每一级将选中 owner、group 还是 other。

⑤ 输出判断 `s/S/t/T` 同时编码特殊位和执行位

`-rwsr-xr-x` owner 的执行位位置：setuid 与 owner `x` 同时存在。

`-rwSr-xr-x` owner 的执行位位置：setuid 存在，但 owner `x` 缺失。

`-rwxr-sr-x` group 的执行位位置：setgid 与 group `x` 同时存在。

`-rwxr-Sr-x` group 的执行位位置：setgid 存在，但 group `x` 缺失。

`drwxrwxr-t` other 的执行位位置：sticky 与 other `x` 同时存在。

`drwxrwxr-T` other 的执行位位置：sticky 存在，但 other `x` 缺失。

大写字符不是“更强”，反而通常提示特殊位存在但相应执行位缺失，需要核对是否符合目标。

⑥ 验证 查询只能证明元数据，功能必须实际测试

`stat` 显示 `0660` 不能证明 bob 能写：bob 可能没有当前组身份，路径中间可能缺少 `x`，也可能还有 ACL 或 SELinux 限制。标准验证顺序为：

```
stat/namei/id 建立证据
→ 以目标用户执行原始动作
→ 以非授权用户执行负向动作
```

[Cheatsheet] 看目录本身用 `ls -ld`；精确 mode/UID/GID 用 `stat -c`；主体用 `id`；完整路径用 `namei -l`；`s/S/t/T` 要同时看特殊位和执行位；元数据正确后仍要实际用户测试。

操作专题 使用 `chmod` 表达增量修改与精确终态

`chmod` 改变的是 mode bits。符号模式适合最小增量修改，八进制适合把对象设置为明确终态。选择哪一种，应由题目是“在现状上添加/删除”还是“最终必须等于某模式”决定。

① 操作 符号模式的结构

基本形式：

```
chmod [ugoa][+|=][rwxXst] PATH
```

- `u`：owner；`g`：group；`o`：other；`a`：全部；
- `+`：添加；`-`：删除；`=`：用给定集合替换该类权限；
- `rwX`：普通权限；`X`：条件执行/search；`s`：setuid/setgid；`t`：sticky。

典型操作：

```
chmod g+w report.txt      # 只添加 group write
chmod o-rwx secret.txt    # 删除 other 的全部普通权限
chmod u=rw,g=r,o= file.txt # 精确替换三类普通权限
chmod g+s /srv/project     # 给目录添加 setgid
chmod +t /srv/dropbox      # 给目录添加 sticky
```

在脚本和考试答案中建议显式写出 `u/g/o/a`。省略 who 时，当前 umask 可能影响哪些类别被修改，容易让相同命令在不同会话产生不同终态。

② 操作 八进制模式写出精确权限

每一类权限按 `r=4`、`w=2`、`x=1` 求和：

```
7 = rwx
6 = rw-
5 = r-x
4 = r--
0 = ---
```

```
chmod 0640 report.txt
chmod 0750 scripts
chmod 2770 /srv/project
chmod 1777 /srv/dropbox
```

当特殊位属于评分终态时，使用四位形式更清楚：首位 `4=setuid`、`2=setgid`、`1=sticky`。八进制命令表达“整个模式的目标值”，执行前要确认不会意外删除本应保留的权限。

③ 参数 大写 `x` 适合目录树，不等于无条件 `x`

`X` 仅在对象是目录，或者对象原本任一类别已有执行位时添加执行/`search` 权限。典型用途：

```
chmod -R g+rwX /srv/project
```

它会给目录添加 `group search`，并为目录树中的对象添加 `group read/write`；普通、原本完全不可执行的文件不会仅因递归而被变成可执行文件。但若某文件原本任一类别已有 `x`，`X` 可能继续给指定类别添加执行位，因此仍要先盘点对象。

④ 操作 复制权限类时使用 `u/g/o` 作为权限来源

符号模式的权限部分也可以引用另一类：

```
chmod g=u file      # 让 group 普通权限等于 owner
chmod o=g file      # 让 other 普通权限等于 group
```

这适合“让某类与另一类一致”的任务，但不自动复制 `owner/group` 身份，也不等同于 `ACL`。

⑤ 验证 修改后同时检查符号、八进制与功能

```
chmod 2770 /srv/project
stat -c '%A %a %U:%G %n' /srv/project
```

随后必须以目标身份创建或访问对象。对目录而言，仅验证 `2770` 只能证明目录 `mode`，不能证明新文件具有目标组和组写权限。

⑥ 安全边界 不把 `chmod 777` 当作诊断方法

`777` 同时向所有本地主体开放读、写、执行或目录修改能力，会掩盖 `owner/group`、`umask`、`路径`、`ACL`、`SELinux` 等真正问题。推荐链路：

```
先定位哪一级、哪一类、缺哪一位
→ 做最小修改
→ 重新执行原始操作
```

[Cheatsheet] 增量用符号模式，精确终态用八进制；特殊位写四位；递归目录树优先考虑 `X` 而非无条件 `x`；显式写 `who`；变更后 `stat` + 实际用户测试；不使用 `777` 碰运气。

操作专题 使用 `chown` 与 `chgrp` 修改 `owner` 和 `group`

传统 `mode` 只有在正确的 `owner/group` 关系下才会选中预期权限类。修改所有权和修改权限是两种不同操作：`chmod` 不改变 `UID/GID`，`chown/chgrp` 也不自动补齐 `rwX`。

① 操作 `chown` 的常用形式

```
chown alice FILE      # 只改 owner
chown :project FILE   # 只改 group
chown alice:project FILE # 同时改 owner 与 group
```

推荐使用冒号分隔 `owner` 与 `group`。点号可能是合法用户名的一部分，使用旧式 `owner.group` 容易产生歧义。

② 操作 `chgrp` 只改变 `group`

```
chgrp project FILE
chgrp -R project DIR
```

`chgrp project FILE` 与 `chown :project FILE` 的目标相同。选用哪条命令可依据可读性和任务上下文，但操作后都应使用 `stat` 核对 `GID`。

③ 权限边界 普通用户不能任意转让 owner

通常只有特权用户可以吧文件 owner 改成其他用户。文件 owner 可以把 group 改成自己所属的某个组；特权用户可以设置为任意有效组。考试任务若要求确定 owner/group，通常以管理员身份执行，再以普通用户验证。

④ 边界 所有权变化可能清除 setuid/setgid

为防止权限提升，内核或工具在改变 owner/group、写入可执行文件等操作后可能清除 setuid/setgid 位。任何涉及特殊位的对象在 chown、chgrp 或内容变更后，都应重新执行：

```
stat -c '%A %a %U:%G %n' PATH
```

不要假定之前设置的 4755、2755 必然保留。

⑤ 安全 递归所有权变更要明确链接与边界

chown -R 能在很短时间内改变整个目录树。执行前至少确认：

- 展开的绝对路径是否正确；
- 目录是否为挂载点或包含其他文件系统；
- 树中是否存在符号链接；
- 是否真的所有子对象都应改变；
- 变更后哪些服务或用户依赖旧 owner/group。

可先使用 find 或 stat 建立清单，再做限定范围的修改。不要对不确定路径执行宽泛递归。

[Cheatsheet] chown USER 改 owner；chown :GROUP 或 chgrp GROUP 改 group；chown USER:GROUP 同时改；普通用户不能任意转让 owner；所有权或内容变化后重查特殊位；递归前先确认路径、挂载和符号链接。

知识专题 umask：新对象权限的按位清除模型

umask 不是“默认权限值”，也不是对已有对象执行的 chmod。它是进程状态，用于清除创建请求中的权限位。掌握按位模型，可以避免把八进制当普通十进制做减法，也能解释为什么同一个 umask 下不同程序仍可能创建出不同权限。

① 知识点 正确公式是请求模式按位清除 mask

最终普通权限 = 程序请求模式 AND (NOT umask)

常见命令行工具通常请求：

- 普通文件：0666，默认不请求执行位；
- 目录：0777。

例如 umask 0027：

文件：0666 & ~0027 = 0640
目录：0777 & ~0027 = 0750

② 知识点 umask 只能删除权限，不能增加权限

如果程序主动请求 0600，即使 umask 为 0000，结果也不会变成 0666。同理，普通文件的常见请求不含执行位，因此 umask 0000 通常也不会让新普通文件自动可执行。

③ 知识点 不能把 umask 当普通减法

0666 - 0027 在某些例子上看似得到正确结果，但它不是权限算法，遇到重叠位时会误导。例如请求模式本来没有某位时，mask 不能从别的位置“借位”。学习和排错应逐位清除：

```
请求  rw-rw-rw-
mask  ----w-rwx
结果  rw-r-----
```

④ 知识点 umask 属于进程并由子进程继承

在当前 shell 执行 umask 0007 后，由该 shell 启动的命令继承相同 mask，除非程序主动修改。退出该 shell 后，父进程或新登录流程可能提供另一值。因此：

- 当前 shell 中看到的值是当前状态；
- 配置文件中写入的是策略来源；

- 新登录会话中看到的值才是持久性证据；
- 新建样本对象的 mode 才是功能证据。

⑤ 边界 default ACL 会改变创建权限路径

如果父目录存在 default ACL，新对象会先从 default ACL 继承访问 ACL，再受创建请求中的权限限制；此时不能只按简单 umask 表推断最终有效权限。发现 `ls -ld` 末尾有 `+` 或 `getfacl` 显示 default 条目时，应转入第 09 章《ACL 与协作目录》。本章只建立接口，不展开 ACL mask 计算。

⑥ 计算表 常见 umask 的典型结果

0022 文件 **0644** · 目录 **0755**

owner 可写，其他只读或穿越。

0027 文件 **0640** · 目录 **0750**

组可读/穿越，other 隔离。

0007 文件 **0660** · 目录 **0770**

owner 与 group 协作，other 隔离。

0077 文件 **0600** · 目录 **0700**

仅 owner。

表格是假定程序请求 **0666/0777** 的典型结果，不替代实际创建与 `stat`。

[Cheatsheet] umask 是进程状态；公式是 `requested & ~mask`；只能清除不能授予；文件常见起点 **0666**、目录 **0777**；当前值、新会话值和新对象结果分层验证；default ACL 存在时转下一章。

操作专题 设置并验证当前与持久 umask

持久 umask 不是“在任意配置文件末尾写一行”这么简单。必须先明确目标主体、Shell 类型和会话入口，再修改对应来源，并用新会话验证。考试题若只要求当前任务进程的创建结果，显式在同一命令环境中设置反而更可控。

① 操作 查看与临时设置

```
umask          # 数值形式，常见输出如 0022
umask -S       # 符号形式，显示允许的权限
umask 0007     # 只改变当前 shell 及其后代
```

注意：umask -S 表示最终允许保留的权限类，而不是直接打印 mask 数字，阅读时不要把两种输出混为一谈。

② 验证 创建文件和目录样本

```
umask 0007
rm -f /tmp/umask-file
rm -rf /tmp/umask-dir
touch /tmp/umask-file
mkdir /tmp/umask-dir
stat -c '%A %a %U:%G %n' /tmp/umask-file /tmp/umask-dir
```

在没有 default ACL 且工具使用常见请求模式时，期望文件为 **0660**、目录为 **0770**。验证后清理测试对象，避免旧样本干扰下一次测试。

③ 配置 选择与目标会话匹配的启动文件

以 Bash 为例，登录 shell 与非登录交互 shell 的读取路径不同。可按题目范围选择：

- 用户登录 shell 策略：在用户登录启动文件中设置；
- 用户交互 Bash 策略：在用户的 Bash 初始化文件中设置；
- 系统范围：使用明确的系统 profile/bashrc 或 PAM 策略，并评估对现有账号影响。

不要宣称 `/etc/login.defs`、`/etc/profile` 或 `~/.bashrc` 中任一处必然覆盖所有服务、计划任务和非交互程序。systemd service、容器和应用自身也可能显式设置创建模式。

④ 验证 新会话与创建结果缺一不可

建议验收矩阵：

```
配置文件存在且语法正确
→ 新建目标登录会话
→ 新会话执行 umask
→ 在该会话创建文件和目录
→ stat 检查最终 mode
```

只查看配置文件不能证明它被读取；只查看 `umask` 不能证明程序请求模式；只查看样本不能说明持久来源正确。

⑤ 边界 协作目录不应只依赖每个人手工执行 `umask`

多人协作若要求稳定组写权限，`setgid` 负责组继承，合适 `umask` 负责普通创建上限；但不同入口可能具有不同 `mask`。若业务必须对多种程序和入口统一继承权限，应在第 09 章评估 default ACL，而不是假定所有用户都会保持同一 shell `umask`。

[Cheatsheet] 临时：`umask NNNN`；查看：`umask / umask -S`；验证：新文件 + 新目录；持久性：必须新建会话；先明确登录/交互/服务入口，再选配置来源；协作要求跨入口稳定时转 ACL。

知识专题 `setuid`、`setgid` 与 `sticky`：特殊位改变哪一层语义

特殊位不等于“额外的 `rwX`”。`setuid` 和 `setgid` 主要影响执行后进程的有效身份；`setgid` 还对目录提供组继承；`sticky` 则改变公共可写目录中的删除和重命名规则。判断时必须结合对象类型。

① 知识点 八进制首位 4/2/1

```
4 = setuid
2 = setgid
1 = sticky
```

可以组合，例如首位 `6` 同时包含 `setuid` 和 `setgid`。常见完整模式：

```
4755  setuid 可执行文件
2755  setgid 可执行文件
2770  setgid 组协作目录
1777  sticky 公共可写目录
```

② 知识点 `setuid` 作用于可执行文件的有效 UID

具有 `setuid` 的可执行文件被允许执行时，进程的有效 UID 通常取文件 owner，而真实 UID 仍表示发起用户。它用于让受控程序完成普通用户本来无权直接完成的特定操作。

安全边界：

- `setuid` 不是给普通脚本随意提权的方案；Linux 通常忽略脚本的 `setuid` 语义；
- 可执行文件必须由可信主体控制，不能让非特权用户修改；
- 内容或所有权变化后应重新检查特殊位；
- 不在本章展开 capabilities 和应用安全审计。

③ 知识点 `setgid` 对可执行文件改变有效 GID

`setgid` 可执行文件运行后，进程的有效 GID 通常取文件 group。它与 `setuid` 类似，但改变的是组身份。该机制不是 `setgid` 目录组继承的同义词，必须根据对象是“可执行文件”还是“目录”分别解释。

④ 知识点 `setgid` 目录让新对象继承目录 group

在 Linux 上，`setgid` 目录中的新文件通常继承目录 group，而不是创建进程的主组；新建子目录通常还会继承 `setgid` 位，从而继续保持组归属链。

它只解决“新对象属于哪个组”，不自动保证：

- group 拥有写权限；
- 已有对象改成目标组；
- 用户当前已经获得该组身份；
- ACL 或 SELinux 允许操作。

因此 `chmod 2770 DIR` 只是协作目录的一层。

⑤ 知识点 `sticky` 限制公共可写目录中的删除和重命名

在可写目录上设置 `sticky` 后，即使用户有目录 `w+x`，通常也只能删除或重命名：

- 自己拥有的目录项；

- 目录 owner 拥有的管理范围；
- 自己是文件 owner 的对象；
- 特权主体允许的对象。

典型公共临时目录为 1777：所有用户可以创建，但不能随意删除其他用户的文件。sticky 不阻止读取文件内容；文件本身的 `rwX` 仍单独判断。

⑥ 输出判断 大写 S/T 是警报，不是增强

```
-rwsr-xr-x   setuid 已设置, 但 owner 没有 x
-rwxr-Sr-x   setgid 已设置, 但 group 没有 x
-drwxrwxr-T   sticky 已设置, 但 other 没有 x
```

对可执行文件，缺少对应执行位往往意味着特殊执行身份无法按预期生效；对目录，`T` 还提示 other 不能穿越。看到大写字符应回到题目终态，不要机械认为“特殊位已经有了就正确”。

⑦ 边界 特殊位的目录/文件语义并不对称

- setuid 在 Linux 目录上通常没有通用的 owner 继承意义；
- setgid 在目录上具有重要组继承语义；
- sticky 在目录上用于删除/重命名限制，普通文件上的历史语义不属于 RHCSA 日常操作。

[Cheatsheet] 首位 4/2/1 = setuid/setgid/sticky；setuid/setgid 可执行文件改有效 UID/GID；setgid 目录继承 group；sticky 目录限制互删；`s/t` 含执行位，`S/T` 缺执行位；特殊位不替代普通 `rwX`。

操作专题 构造不使用 ACL 的 setgid 组协作目录

传统权限可以实现“一个固定组共同工作、other 完全隔离”的基本协作目录。完整方案至少包含目录 group、setgid、组 `rwX`、创建进程的 umask，以及两个组成员之间的交叉验证。

① 调查 先确认目录、组和测试主体

假定组 and 用户已存在：

```
getent group techdocs
id alice
id bob
id carol
stat -c '%F %A %a %U:%G %n' /srv/techdocs 2>/dev/null || true
namei -l /srv/techdocs
```

目标：alice、bob 是 techdocs 成员，carol 不是；目录 owner 为 root，group 为 techdocs，other 无权限。

② 操作 设置目录归属和 mode

```
install -d -o root -g techdocs -m 2770 /srv/techdocs
```

已有目录应先调查再分别使用 `chown root:techdocs` 与 `chmod 2770`，避免 `install -d` 的简洁形式掩盖已有内容状态。操作后：

```
stat -c '%A %a %U:%G %n' /srv/techdocs
```

期望静态终态为 `drwxrws--- 2770 root:techdocs`。

③ 配置 为创建进程提供组可写的 umask

在不使用 default ACL 的前提下，创建者需要使用允许 group write、屏蔽 other 的 mask，例如 `0007`：

```
sudo -u alice sh -c 'umask 0007; touch /srv/techdocs/alice.txt; mkdir /srv/techdocs/alice.d'
```

若题目要求持久策略，应在明确的登录环境中配置，再用新会话验证；不能只在这一条测试命令中设置。

④ 验证 检查组继承和最终模式

```
stat -c '%A %a %U:%G %n' \
/srv/techdocs/alice.txt /srv/techdocs/alice.d
```

在典型创建请求下，期望：

```
alice.txt 0660 alice:techdocs
alice.d   2770 alice:techdocs
```

子目录保留 `setgid`，才能让更深层新对象继续继承组。

⑤ 验证 第二名组员实际修改

```
sudo -u bob sh -c 'printf "%s\n" reviewed >> /srv/techdocs/alice.txt'
sudo -u bob test -w /srv/techdocs/alice.txt
```

`test -w` 是快速检查，实际追加才是更强的功能证据。失败时依次检查 bob 当前组、父路径 `x`、文件 `group`、文件 `group write`、`ACL/SELinux`。

⑥ 负向验证 非成员不能穿越或创建

```
sudo -u carol test -x /srv/techdocs
sudo -u carol touch /srv/techdocs/should-fail
```

负向命令预期失败。测试文件名必须是专用样本，避免破坏真实数据。

⑦ 边界 setgid 不回溯修复已有对象

目录加 `setgid` 后，已有文件不会自动改组、补写权限或获得 `setgid`。已有树若也必须迁移，应先盘点 `owner/group/mode`，再使用受控的 `chgrp`、`chmod` 或限定 `find` 操作；这属于一次独立变更，不能被“新对象继承”替代。

[Cheatsheet] `root:GROUP + 2770` 建立目录层；`setgid` 管新对象 `group`；`umask 0007` 让典型新文件/目录组可写且隔离 `other`；验证用 A 创建、B 修改、非成员失败；已有对象单独迁移。

诊断专题 文件显示可读却仍 Permission denied

这类故障最适合训练证据推进。最终文件 `mode` 只是链路末端的一项；正确方法是从真实主体开始，逐级找到第一处具有区分度的阻断点，再做最小修复。

① 症状 固定原始动作和目标主体

例如：

```
sudo -u dana cat /srv/reports/q1/report.txt
```

不要先改权限。记录：谁执行、完整路径、动作是读内容而不是列目录、错误发生在当前系统还是远程程序中。

② 当前证据 确认主体身份

```
id dana
sudo -u dana id
```

若组成员关系刚改变，应建立新会话。不要用管理员自己的 `id` 代表 `dana`。

③ 下一条最有区分度的证据 展开路径

```
namei -l /srv/reports/q1/report.txt
```

逐行判断 `dana` 在每个目录上选中 `owner/group/other` 哪一类，找到第一处缺少 `x` 的分量。若路径均可穿越，再检查最终文件 `r`。

④ 假设 区分父目录、最终文件与删除语义

- `cat` 失败：父路径 `x` 或文件 `r`；
- `echo >> file` 失败：父路径 `x` 或文件 `w`；
- `touch NEW` 失败：父目录 `w+x`；
- `rm file` 失败：父目录 `w+x`，以及 `sticky` 等额外限制；
- `ls DIR` 失败：目录 `r/x` 组合。

选择与原始症状最相关的假设，不要一次改多层。

⑤ 最小修复 只修改阻断对象和目标权限类

如果 `/srv/reports` 的 group 为 `analysts`，`dana` 属于该组，但目录 mode 为 `0740`，缺的是 group search。可在确认业务终态后执行：

```
chmod g+x /srv/reports
```

而不是把最终文件改成 `777`。修改后立即用 `stat` 和 `namei` 重建证据。

⑥ 再验证 用相同主体重做原始动作

```
sudo -u dana cat /srv/reports/q1/report.txt
```

再使用一个无关用户做负向测试，确保最小修复没有扩大给 `other`。

⑦ 边界 DAC 链路无异常后再进入下一安全层

若 `id`、`namei`、`stat` 均支持传统权限允许，但操作仍被拒绝，下一步依次检查：

- 第 09 章：访问 ACL、default ACL 和 mask；
- 第 28/29 章：SELinux 上下文、类型规则和 AVC 证据；
- 应用自身的访问控制或只读挂载等其他层。

[Cheatsheet] 固定主体和原始动作 → `id` → `namei -l` → `stat` → 判断选中的 u/g/o → 最小 `chmod/chown` → 相同用户复验 → 无关用户负向测试 → 再转 ACL/SELinux。

诊断专题 递归权限修改：先控制爆炸半径

递归命令会把一个小判断错误扩散到整棵树。`chmod -R 777`、`chown -R` 或对错误变量展开执行，可能破坏可执行文件、安全边界和服务数据。递归不是禁用功能，而是必须先建立对象集合、链接策略和可验证终态。

① 调查 先盘点对象类型和现状

```
find /srv/project -xdev -printf '%y %m %u:%g %p\n' | less
find /srv/project -xdev -type l -print
```

`-xdev` 可在适用时避免跨入其他文件系统，但是否使用取决于任务边界。确认树中是否有脚本、二进制、套接字、命名管道、挂载点和符号链接。

② 假设 文件和目录通常需要不同权限

目录需要 `x` 才能穿越，普通数据文件通常不应自动获得执行位。若目标是“组成员可读写并穿越目录”，可考虑：

```
chmod -R g+rwX /srv/project
```

仍需确认原本可执行文件是否应保持执行，以及 `other` 权限是否需要收紧。

③ 边界 明确符号链接跟随策略

递归工具对命令行参数中的符号链接、树内符号链接以及 `-H/-L/-P` 的处理不同。默认不要假定链接会或不会跟随；执行前查看对应 man page，并尽量对真实根目录路径操作。链接指向树外时，跟随可能把变更扩散到完全不同的位置。

④ 最小修复 按类型和条件限定

精确迁移常使用分开的 `find`：

```
find /srv/project -xdev -type d -exec chmod 2770 {} +
find /srv/project -xdev -type f -exec chmod 0660 {} +
```

这只是任务示例：给每个子目录 `setgid` 是否符合业务、是否存在应执行脚本，必须在执行前确认。不能把示例当成任何目录树的通用答案。

⑤ 验证 全量静态检查与抽样功能测试结合

```
find /srv/project -xdev -printf '%y %m %u:%g %p\n'
```

随后选择不同层级、不同 owner、不同对象类型进行跨用户测试。只检查根目录或随机一个文件不能证明整棵树正确。

[Cheatsheet] 递归前先 `find` 盘点；文件和目录分开；需要目录 search 时优先评估 `X`；明确 `-H/-L/-P` 与跨文件系统边界；按类型限定；变更后全量静态核对 + 多身份功能抽样。

经典任务 建立传统权限组协作目录并完成跨用户验收

环境与当前状态

系统已经存在：

- 组 `techdocs`；
- 用户 `alice`、`bob`，均为 `techdocs` 成员；
- 用户 `carol`，不是该组成员；
- `/srv` 可由所有用户穿越；
- `/srv/techdocs` 当前不存在；
- `alice` 与 `bob` 使用 Bash 登录，二人的 `~/.bash_profile` 已存在并会加载各自的 `~/.bashrc`，当前没有显式 `umask` 行。

本任务不允许使用 ACL；`sudo` 仅作为测试身份切换工具，不要求修改 `sudoers`。

目标终态

1. 创建 `/srv/techdocs`，owner 为 `root`，group 为 `techdocs`；
2. 目录模式为 `2770`；
3. `alice` 和 `bob` 的新登录 Bash 会话应使用 `umask 0007`；
4. `alice` 新建的普通文件应为 `0660 alice:techdocs`；
5. `alice` 新建的子目录应为 `2770 alice:techdocs`；
6. `bob` 能向 `alice` 创建的文件追加内容；
7. `carol` 不能穿越目录，也不能创建文件；
8. 不修改系统中其他用户的 `umask`，不使用 `chmod 777`。

验收证据

- `stat` 显示目录及样本对象的 mode 和 owner/group；
- 新登录会话中的 `umask`；
- `alice` 创建、`bob` 修改的实际操作；
- `carol` 的负向测试；
- 必要时用 `namei -l` 证明父路径没有额外阻断。

参考解答 经典任务一

以下命令是静态推荐解答，必须在真实 RHEL 9 环境中再次验证用户启动文件、实际组身份和创建结果。

① 调查现有身份与路径

```
getent group techdocs
id alice
id bob
id carol
namei -l /srv
```

确认 `alice`、`bob` 命中 `techdocs`，`carol` 不命中；确认 `/srv` 的父路径允许目标用户穿越。

② 创建目录并设置精确终态

```
install -d -o root -g techdocs -m 2770 /srv/techdocs
stat -c '%A %a %U:%G %n' /srv/techdocs
```

若目录已经存在，应先备份基线并分别执行最小变更，而不是假定目录为空。

③ 配置目标用户的登录 umask

题目明确限定“新登录 Bash 会话”。应根据系统现有 Bash 启动结构，把 `umask 0007` 放入 `alice`、`bob` 的登录配置范围，同时避免重复和覆盖其他逻辑。示意：

```
for user in alice bob; do
  home=$(getent passwd "$user" | cut -d: -f6)
  profile="$home/.bash_profile"
  cp -a "$profile" "$profile.rhcsa08.bak"
  grep -qxF 'umask 0007' "$profile" ||
    printf '\numask 0007\n' >> "$profile"
done
```

本任务已明确这两个文件存在且属于目标用户，因此示例保留其所有权并先建立备份。真实系统仍应先查看启动链；若现有逻辑在其他文件统一设置，应修改实际生效来源，而不是机械追加多处。

④ 建立新登录会话验证当前状态

```
sudo -iu alice bash -lc 'umask'
sudo -iu bob bash -lc 'umask'
```

期望两者为 `0007`。`sudo -iu` 在此仅模拟登录式测试入口；最终以题目提供的真实登录方式复核更可靠。

⑤ alice 创建样本并检查继承

```
sudo -iu alice bash -lc 'touch /srv/techdocs/alice.txt; mkdir /srv/techdocs/alice.d'
stat -c '%A %a %U:%G %n' \
  /srv/techdocs/alice.txt /srv/techdocs/alice.d
```

期望：

```
alice.txt  0660 alice:techdocs
alice.d    2770 alice:techdocs
```

如果 group 正确但没有 group write，检查 alice 新会话的 umask；如果 group 不正确，检查父目录 setgid 和实际创建位置。

⑥ bob 进行交叉写入

```
sudo -iu bob bash -lc 'printf "%s\n" reviewed >> /srv/techdocs/alice.txt'
sudo -iu bob tail -n 1 /srv/techdocs/alice.txt
```

这比只看 `test -w` 更接近真实终态。

⑦ carol 做负向测试

```
sudo -u carol test -x /srv/techdocs
sudo -u carol touch /srv/techdocs/should-not-exist
```

两条命令预期失败。随后确认没有意外产生样本：

```
test ! -e /srv/techdocs/should-not-exist
```

⑧ 最终证据矩阵

维度	命令	证明
目录静态状态	<code>stat /srv/techdocs</code>	<code>2770 root:techdocs</code>
登录策略	<code>sudo -iu USER bash -lc 'umask'</code>	新登录会话使用 <code>0007</code>
组继承	<code>stat alice.txt</code>	新文件 group 为 <code>techdocs</code>
文件 mode	<code>stat alice.txt</code>	新文件为 <code>0660</code>
子目录延续	<code>stat alice.d</code>	子目录为 <code>2770</code> ，继续 setgid
组内功能	bob 追加	第二名成员能实际写入
隔离	carol 负向测试	other 没有穿越/创建能力

典型错误

- 只设置 `chmod 2770`，却不处理创建进程 umask；

- 只在当前管理员 shell 执行 `umask 0007`；
- 用 root 创建样本，误判普通用户终态；
- 只执行 `ls -ld`，没有跨用户创建和修改；
- 使用 `777` 或提前引入 ACL，偏离任务边界；
- 假定 setgid 会回溯修改已有文件。

经典任务 定位“文件可读但路径不可达”并做最小修复

环境与当前状态

系统存在用户 `dana`，她属于组 `analysts`。目标文件：

```
/srv/reports/q1/report.txt
```

管理员已确认最终文件静态状态为：

```
0640 root:analysts
```

但执行：

```
sudo -u dana cat /srv/reports/q1/report.txt
```

得到 `Permission denied`。不允许删除重建对象，不允许把文件或目录改成 `777`，不要求配置 ACL 或 SELinux。

目标终态

1. 找到路径中第一个传统权限阻断点；
2. 只增加 `dana` 读取该文件所需的最小权限；
3. `dana` 能读取 `report.txt`；
4. 不属于 `analysts` 的用户 `erin` 仍不能读取；
5. 提交调查、修改和再验证证据。

参考解答 经典任务二

① 固定主体和原始失败

```
id dana
sudo -u dana id
sudo -u dana cat /srv/reports/q1/report.txt
```

记录原始失败，不先改最终文件。

② 读取最终对象状态

```
stat -c '%F %A %a %U:%G %n' /srv/reports/q1/report.txt
```

`0640 root:analysts` 表明 `dana` 若当前组集合命中 `analysts`，最终文件的 group `r--` 足以读取内容；这仍不能证明路径可达。

③ 展开每一级路径

```
namei -l /srv/reports/q1/report.txt
```

对 `/`、`/srv`、`/srv/reports`、`/srv/reports/q1` 逐级判断 `dana` 选中的权限类是否包含 `x`。假定发现 `/srv/reports` 为：

```
drwxr----- root analysts /srv/reports
```

它的 group 有 `r` 但没有 `x`，因此 `dana` 可以命中 group，却无法穿越。

④ 做最小修改

```
chmod g+x /srv/reports
stat -c '%A %a %U:%G %n' /srv/reports
namei -l /srv/reports/q1/report.txt
```

修改只为 analysts 增加 search，不改变 other，不修改最终文件。

⑤ 以相同主体再验证

```
sudo -u dana cat /srv/reports/q1/report.txt
```

期望成功。

⑥ 非授权用户负向测试

```
id erin
sudo -u erin cat /srv/reports/q1/report.txt
```

期望失败。若 erin 仍可读取，应检查路径中 other 权限、她的组身份以及是否存在 ACL。

⑦ 典型错误

- 把 report.txt 从 0640 改成 0644，但父路径仍不可穿越；
- 把所有父目录加 o+x，扩大给无关用户；
- 看到目录有 r 就认为可穿越；
- 用 root 执行 cat 作为验收；
- 一次修改多个目录，无法判断真正阻断点；
- DAC 证据已经允许后仍继续乱改 mode，而没有转入 ACL/SELinux 证据链。

本章收束 用“主体—路径—权限类—动作—证据”解决权限题

传统权限的核心不是一组数字，而是一条求值链：

```
真实访问主体
→ 路径逐级 search
→ 每个对象选择 owner/group/other 唯一一类
→ 根据具体动作检查文件或目录 rwx
→ 应用 setuid/setgid/sticky 的对象类型语义
→ 对新对象应用 requested mode 与 umask
→ 实际用户正向与负向验证
```

最小证据矩阵

问题	首选证据
用户当前是谁、有哪些组	目标进程中的 id
对象当前 mode/owner/group	stat -c
路径哪一级不可达	namei -l
应增量还是精确设置	chmod 符号/八进制模式
owner/group 是否正确	stat + chown/chgrp
当前创建 mask	umask、umask -S
持久 mask 是否生效	新登录会话 + 创建样本
setgid 协作是否成功	A 创建、B 修改、非成员失败
Permission denied 下一层	ACL、再到 SELinux

工作迁移

在真实系统中，权限修改应视为变更而不是命令练习：先记录基线，明确业务主体和动作，控制递归范围，避免链接越界，保留回滚所需的 owner/group/mode 清单，并在操作后用真实身份验证。后续使用 Ansible 自动化时，也应把目标写成可比较的 mode、owner、group 和创建策略，而不是把一串无条件 shell 命令当作幂等状态。

本章完成了传统 DAC 层。额外用户/组授权、default ACL 与 mask 进入第 09 章《ACL 与协作目录》；sudo 授权进入第 10 章《sudo 与最小特权授权》；SELinux 进入第 28、29 章。

RHEL 9 · RHCSA 实操讲义

09

ACL 与协作目录

从 ACL 条目到有效权限：把 access、default、mask、继承与多身份验收放进同一条证据链。

对象模型 操作语义 验证 诊断 经典任务

大号阅读版

:::: {navigation-page}

本章阅读导航

先抓住一条主线：ACL 不是“给用户写一行 `rwX` 就结束”，而是从主体匹配开始，经 `mask` 计算得到 `effective permissions`，再落到当前对象、未来新对象和真实身份功能测试。

... {model-grid}

01识别对象与主体确认文件、目录、owner、owning group 与 named entry

02区分 `access / default`当前访问与未来创建不是同一状态

03计算 `effectivenamed user` 与 `group class` 还要经过 `mask`

04执行最小修改新增、修改、删除时控制条目和递归范围

05验证继承时点已有对象、新文件和新目录分别取证

06多身份验收同时证明允许动作、拒绝动作和恢复依据

...

... {navigation-columns}

... {nav-col}

专题地图

类型	专题
知识专题	ACL 对象、条目与作用时点
知识专题	访问匹配、 <code>mask</code> 与 <code>effective</code>
操作专题	<code>ls</code> 、 <code>stat</code> 与 <code>getfacl</code> 取证
操作专题	<code>setfacl</code> 新增、修改、删除与 <code>mask</code> 控制
操作专题	<code>default ACL</code> 与创建时继承
操作专题	协作目录的当前、历史与未来状态
操作专题	ACL 复制、备份与恢复
诊断专题	条目存在但仍然 <code>Permission denied</code>
经典任务	建设协作目录与修复 <code>mask</code>

...

... {nav-col}

阅读时持续回答

- 当前判断的是 `access ACL`，还是 `default ACL`？
- 进程会匹配 `owner`、`named user`、`group class` 还是 `other`？
- 该条目是否受 `mask` 限制？
- `#effective:` 与名义权限是否一致？
- 目标是已有对象还是未来新对象？
- `chmod` 是否改变了 `mask`？
- 查询证据能证明什么，不能证明什么？
- 哪个真实身份动作才是最终验收？

...

...

....

第 09 章 · 正文

ACL 与协作目录：声明的权限为何不一定真正生效 传统 `owner`、`group`、`other` 三类权限适合表达多数单一团队场景，但真实目录常常同时存在“项目组可协作、审计员只读、个别人员需要额外权限”等要求。ACL 在不改变主要 `owner` 与 `owning group` 的前提下增加 `named user`、`named group` 和 `mask`，使授权能够精确到额外主体。本章最重要的不是记住 `setfacl` 选项，而是避免三种误判：`**ACL` 条目存在不等于有效权限存在；`default ACL` 存在不等于已有对象已经改变；命令成功不等于目标身份能够完成业务动作。`**` 因此，本章沿着“对象与条目 → `access/default` → `mask/effective` → 查询与修改 → 继承 → 多身份验证 → 诊断与恢复”推进。

传统模式位、umask、setgid 和 sticky 位已在第 08 章《传统权限、umask 与特殊权限位》中建立；本章只解释它们与 ACL 的接口。sudo 属于第 10 章《sudo 与最小特权授权》；SELinux 属于后续安全章节，本章在证据闭环后只指出分流方向，不提前展开。

当前对象存储了哪些 ACL 条目？

目标身份会匹配哪一种主体类别？

mask 后真正剩下哪些 effective permissions？

失败对象是历史文件、当前目录，还是未来新对象？

结构证据之后，哪个真实动作才能完成最终验收？

概念Access ACL 是文件或目录当前参与自主访问判断的规则集合。它描述 owner、named user、owning group、named group、mask 与 other 在“现在访问这个对象”时如何工作。目录上出现 default ACL，并不会替代当前目录的 access ACL；判断当前能否进入或读写，仍要读取 access 部分。

概念Default ACL 只能附着在目录上，是创建新对象时使用的 ACL 模板。它不会回溯修改已经存在的文件和子目录，也不直接决定父目录本身能否被访问。要验证 default ACL，必须真正创建新文件和新目录，再读取它们的 access ACL。

概念ACL entry 由主体类型、可选 qualifier 与 `rwx` 权限组成。`user::`、`group::`、`other::` 是基础条目；`user:alice:` 与 `group:qa:` 是额外主体条目；`mask::` 则描述整个 group class 的有效权限上限。读输出时要先判断“这一行代表谁”，再谈权限。

概念ACL mask 不是一个新的用户或组，而是 named user、owning group 与所有 named group 的统一上限。提高 mask 可能同时放宽多个条目的 effective 权限，收窄 mask 也可能让多个主体一起失去权限，因此修改 mask 前必须审阅整个 group class。

概念Effective permissions 是条目权限经过匹配规则与 mask 约束后真正参与访问判断的结果。`user:analyst:rw-` 与 `mask::r--` 同时存在时，analyst 实际只有 `r--`；`getfacl` 的 `#effective:` 比单独看到 named entry 更接近内核的最终判断。

概念继承、已有对象与新对象 构成 ACL 的时间维度。父目录的 default ACL 只初始化未来对象；已经存在的对象保留原 access ACL。协作目录因此必须分别处理当前目录、历史树和未来模板，并通过新文件、新目录以及不同身份的真实动作完成验证。

⋮ {ops-quickref}

操作语义以下入口分别观察 ACL 结构、修改当前规则、建立未来模板、复制或恢复元数据，并验证 `chmod` 与 mask 的交互。先理解命令作用对象，再记关键形式。

getfacl

SYNOPSIS

```
getfacl [-a|-d] [-e] [-p] file ...
getfacl -R -P -p directory
```

读取文件或目录的 access/default ACL、mask 与 effective 注释。它证明“对象存储了什么”，不能单独证明当前会话身份和最终业务动作。

重要参数 / 形式

getfacl -p PATH

保留绝对路径的前导 /，适合基线与审阅。

-a / -d

分别只显示 access ACL 或 default ACL，避免把当前规则与未来模板混读。

-e

强制显示所有受 mask 约束条目的 effective permissions。

-R -P

物理递归目录树，不跟随目录符号链接超出预期范围。

setfacl -m / -x / -b

SYNOPSIS

```
setfacl [-n|--mask] -m acl_spec file ...
setfacl -x acl_spec file ...
setfacl -b file ...
```

新增或修改 access ACL、删除指定条目，或清除扩展 access ACL。-b 是策略级清理，不应作为“不知道原因时”的默认排错动作。

重要参数 / 形式

-m u:alice:rw-

新增或修改 named user 条目。

-m g:qa:r--

新增或修改 named group 条目。

-m m::rw-

设置 group class 的统一上限：必须同时审阅其他受影响条目。

`-x u:alice / -x g:qa`

删除指定主体条目，不在删除规格中重复权限字段。

`-b`

清除 named user、named group、mask 等扩展 access ACL，保留基础 owner/group/other 条目。

`-n / --mask`

分别抑制 mask 重算，或强制根据条目重算 mask；两者都需要在完整 group class 证据下使用。

`setfacl -d / -k`

SYNOPSIS

```
setfacl -d -m acl_spec directory ...
setfacl -k directory ...
```

管理目录的 default ACL。default 只负责未来创建，不能证明当前目录或历史文件已经获得相同权限。

重要参数 / 形式

`d:u:auditor:r-x`

为未来对象模板加入 named user；目录通常需要穿越位。

`d:g::rwx / d:m::rwx / d:o::---`

明确 default owning group、default mask 与 default other。

`-k`

删除目录的 default ACL，不清除当前 access ACL。

新文件 / 新目录

必须分别创建并读取 ACL；普通文件不会因为模板中写了 x 就自动得到执行位。

ACL 复制、备份与恢复

SYNOPSIS

```
getfacl --access SOURCE | setfacl --set-file=- TARGET
getfacl -R -P -p DIRECTORY > acl.backup
setfacl --test --restore=acl.backup
setfacl --restore=acl.backup
```

把参考对象 ACL 复制到目标，或对目录树建立可审计的 ACL 元数据备份。恢复前应先预演，并确认备份中的路径、owner、group 与特殊位注释。

重要参数 / 形式

`--set-file=-`

从标准输入读取完整 ACL 并替换目标 ACL；不是增量追加。

`-R -P -p`

递归、物理遍历并保留绝对路径，适合目录树基线。

`--test --restore=FILE`

只显示恢复将产生的 ACL，不修改对象。

`--restore=FILE`

按备份恢复 ACL，并可能处理 owner、group 和特殊位；真实系统应先在测试树核对。

chmod 与 ACL 交互验证

SYNOPSIS

```
getfacl -e -p file
chmod g=MODE file
getfacl -e -p file
```

带扩展 ACL 的对象上，传统 group mode bits 通常映射 mask::。因此 `chmod g=...` 可能同时改变 named user、owning group 和 named group 的 effective 上限。

重要参数 / 形式

变更前后对比

对带 + 的对象执行 `chmod` 前后都运行 `getfacl -e`，不要只看 `ls -l`。

`ls -l` 的 `group` 位

在扩展 ACL 中通常显示 `mask`，而不一定直接等于 `group::` 条目。

最小修复

若目标只是恢复一个主体，先检查其他 `group class` 条目，避免通过放宽 `mask` 意外扩大授权。

...

[知识专题] ACL 不是“第四组权限”：先认清对象、条目与作用时点

传统模式位把访问者压缩为 `owner`、`group`、`other` 三类。ACL 没有推翻这套模型，而是在其上增加 `named user`、`named group` 和一个控制 `group class` 的 `mask`。理解 ACL 的最顺切入点不是背命令，而是把每一行映射到“谁、针对哪个对象、在什么时点生效”。

① [知识点] access ACL 管当前对象，default ACL 管未来创建

每个文件或目录都可以有 `access ACL`。目录还可以额外具有 `default ACL`：

```
access ACL
→ 现在访问这个文件或目录时参与判断

default ACL
→ 在目录内创建新对象时，用来初始化新对象的 access ACL
```

普通文件不能拥有 `default ACL`。目录的 `default ACL` 不参与该目录自身的访问检查，因此“目录上已经有 `default:user:auditor:r-x`”并不表示 `auditor` 现在就能进入这个目录；通常还需要相应的 `access ACL`。

② [知识点] 基础条目与扩展条目分别代表谁

一个完整 `access ACL` 可能包含：

文本条目	主体	是否需要 qualifier	是否受 mask 限制
<code>user::rwx</code>	文件所有者	否	否
<code>user:alice:rw-</code>	named user <code>alice</code>	是	是
<code>group::r-x</code>	文件所属组	否	是
<code>group:qa:r--</code>	named group <code>qa</code>	是	是
<code>mask::r-x</code>	<code>group class</code> 上限	否	不适用
<code>other::---</code>	未匹配其他主体者	否	否

`user::` 中第二段为空，表示文件所有者；`user:alice:` 的 `qualifier` 是 `alice`。`group::` 表示文件记录的 `owning group`，而不是“当前用户的主组”。

③ [知识点] 有 named user 或 named group 时，mask 成为必要结构

只包含 `user::`、`group::`、`other::` 的 ACL 可以直接映射传统模式位。出现 `named user` 或 `named group` 时，ACL 必须具有 `mask`。`setfacl` 通常会补齐必要条目并计算 `mask`，但这并不意味着自动得到的上限一定符合最终最小授权目标，仍要用 `getfacl` 检查。

④ [知识点] 目录 rwx 仍保留传统目录语义

ACL 条目中的 `rwx` 没有创造新的权限含义：

- 对普通文件，`r` 读取内容，`w` 修改内容，`x` 执行；
- 对目录，`r` 列出名称，`w` 修改目录项，`x` 穿越并访问已知名称。

因此给审计用户目录 `r--` 往往不够：他可能看到目录名，却无法访问其中已知文件。典型只读遍历授权是目录 `r-x`、普通文件 `r--`。

⑤ [知识点] ACL 属于自主访问控制的一层，不覆盖其他安全层

ACL 判断通过，只能说明这一层没有拒绝。路径上任一目录缺少 `x`、只读挂载、应用自身限制或 SELinux 拒绝，都可能让功能测试失败。本章诊断会在 ACL 证据完成后指出分流方向，但不展开 SELinux 规则。

[Cheatsheet] 当前对象看 `access`；未来对象看 `default`；基础条目是 `owner/group/other`；`named user`、`owning group`、`named group` 都进入 `group class`；目录只读通常需要 `r-x`。

[知识专题] 从条目到实际权限：访问匹配、mask 与 effective

ACL 输出里最危险的阅读方式是只找到自己的名字，然后把该行右侧权限当作最终结果。内核先选择匹配的主体类别，再对 group class 条目应用 mask。某些匹配一旦发生，即使权限不足，也不会继续回退到看起来更宽的其他条目。

① [知识点] 文件所有者匹配后不会再使用组或 other

访问检查首先比较进程 effective UID 与文件 owner UID。若匹配，只检查 `user::`。如果 owner 条目不含所请求权限，访问被拒绝，不会把文件所有者再当作所属组成员或 other 重新计算。

这与传统权限“最具体类别优先”一致，也解释了为什么文件所有者权限较窄时，所属组较宽并不能自动补足。

② [知识点] named user 匹配后同时受自己的条目和 mask 约束

若进程不是文件所有者，但 effective UID 匹配 `user:<NAME>:`，则：

```
named-user effective
= named-user entry n mask
```

若 `user:analyst:rw-` 而 `mask::r--`，analyst 的有效权限是 `r--`。即便 analyst 也属于某个具有写权限的 named group，named user 已匹配，不能依靠组条目绕过 named user 的结果。

③ [知识点] 组阶段会考虑 owning group 和所有匹配 named groups

若没有 owner 或 named user 匹配，内核比较进程 effective GID 和所有 supplementary groups：

- 是否匹配文件 owning group；
- 是否匹配一个或多个 named group。

只要某个匹配的组条目与 mask 共同包含所请求权限，就可以通过该项访问。调查时不能只读第一条组记录，应结合用户实际组集合和所有匹配组条目。

④ [知识点] `other::` 只用于没有匹配任何用户或组的情况

`other::` 不是“最后再补一点权限”。只要前面的 owner、named user 或组阶段已匹配，就按该阶段决定允许或拒绝；只有完全没有匹配时才检查 `other::`。

⑤ [知识点] mask 是整个 group class 的统一上限

mask 约束以下条目：

```
user:<NAME>:
group::
group:<NAME>:
```

它不约束：

```
user::
other::
```

因此 `setfacl -m m::rwx` 不是“只修复 analyst”。它可能同时放宽所有 named user、owning group 和 named group 的 effective 上限。最小修复前必须查看整个 group class。

⑥ [知识点] 扩展 ACL 存在时，`ls -l` 的 group 位通常显示 mask

若 ACL 有 mask，传统 group mode bits 对应 `mask::`，而不是简单对应 `group::`。因此：

```
-rw-r-----+
```

中间三位 `r--` 可能表示 mask 是 `r--`，文件所属组条目本身也许写着 `rw-`，但 effective 被截断。`chmod g=...` 会修改 group mode bits，也就可能改变 mask，并同时影响多个 ACL 条目。

[Cheatsheet] owner → named user → 任一匹配组 → other；named user 和组类都受 mask；`#effective:` 才是被 mask 后的结果；扩展 ACL 下 `ls` 的 group 位通常是 mask。

[操作专题] 用 `ls`、`stat` 与 `getfacl` 建立可判定的 ACL 证据

查询阶段的目标不是收集更多输出，而是回答四个明确问题：目标对象是谁、当前 access ACL 是什么、目录是否有 default ACL、条目是否被 mask 截断。`ls` 适合快速筛选，最终判断应回到 `getfacl`。

① [操作] 用 `ls -ld` 发现 ACL 线索，但不据此推断授权细节

作用对象：文件或目录的模式摘要。基本形式：

```
ls -ld /srv/atlas /srv/atlas/plan.md
```

权限字符串末尾出现 `+`，说明对象具有扩展 access ACL 或目录具有 default ACL。它不能告诉你有哪些 named 条目、mask 是什么，也不能证明目标用户具有权限。

② [操作] 用 `getfacl -p` 查看完整对象视图

```
getfacl -p /srv/atlas
```

`-p` 保留绝对路径的前导 `/`，适合基线、备份和审阅。典型输出：

```
# file: /srv/atlas
# owner: root
# group: project
# flags: -s-
user::rwx
user:auditor:r-x
group::rwx
mask::rwx
other::---
default:user::rwx
default:user:auditor:r-x
default:group::rwx
default:mask::rwx
default:other::---
```

头部给出路径、owner、group；`# flags`：可能显示 setuid、setgid 或 sticky 位。本章只使用它确认第 08 章已经建立的目录特殊位状态。

③ [操作] 用 `-a` 与 `-d` 分离当前状态和未来模板

```
getfacl -a -p /srv/atlas # 只看 access ACL
getfacl -d -p /srv/atlas # 只看 default ACL
```

把两者分开能防止将 `default:user:auditor:r-x` 错误读成当前目录授权。普通文件执行 `getfacl -d` 不会得到可用的 default ACL，因为普通文件不能拥有它。

④ [操作] 用 `-e` 强制显示所有 effective 权限

默认情况下，只有条目权限与有效权限不同时才显示 `#effective:`。诊断时可强制显示：

```
getfacl -e -p /srv/atlas/plan.md
```

例如：

```
user:analyst:rw-          #effective:rw-
group::rw-                #effective:rw-
mask::r--
```

这条证据直接说明写权限被 mask 截断，而不是 named user 条目缺失。

⑤ [操作] 在名称解析和递归范围有疑问时切换查询方式

```
getfacl -n -p /srv/atlas/plan.md # 显示数值 UID/GID
getfacl -R -P -p /srv/atlas      # 物理递归，不跟随目录符号链接
getfacl -R -P -s -p /srv/atlas   # 跳过只有基础 ACL 的对象
```

`-R` 会产生大量输出，应先确认目标根目录。`-P` 避免沿目录符号链接进入树外；`-L` 会逻辑跟随，除非确有需求，不应把它作为默认递归选择。

验证边界：`getfacl` 证明存储的 ACL 结构和 effective 计算，不能证明当前登录会话身份、父路径访问或最终业务动作。

帮助入口：`man getfacl`、`getfacl --help`。

[Cheatsheet] `ls` 的 `+` 只作提示；完整看 `getfacl -p`；当前与未来用 `-a/-d` 分开；mask 排错用 `-e`；递归优先 `-R -P`。

[操作专题] 修改当前对象的 access ACL：新增、修改、删除与 mask 控制

`setfacl` 的操作对象是文件或目录上已经存储的 ACL。最安全的工作方式是：先保存基线，只修改目标 entry，再立即读取 ACL 和进行功能测试。清空全部扩展条目虽然看起来简单，却可能删除仍然有效的历史授权。

① [操作] 用 `-m` 新增或修改 named user 和 named group

```
setfacl -m u:alice:rw- /srv/atlas/plan.md
setfacl -m g:qa:r-- /srv/atlas/plan.md
setfacl -m u:auditor:r-x /srv/atlas
```

`-m` 对不存在的条目执行新增，对已存在的同主体条目执行修改。目录是否需要 `x` 取决于目标动作；只读遍历通常使用 `r-x`。

一条命令可包含多个逗号分隔条目：

```
setfacl -m u:auditor:r--,g:qa:rw- /srv/atlas/plan.md
```

② [操作] 明确修改基础条目和 mask 的语义

```
setfacl -m g::rw- /srv/atlas/plan.md
setfacl -m m::rw- /srv/atlas/plan.md
setfacl -m o::--- /srv/atlas/plan.md
```

- `g::rw-` 修改 owning group 条目；
- `m::rw-` 修改整个 group class 的上限；
- `o::---` 修改未匹配其他主体者。

不要把 `g::` 与 `g:<GROUP>`：混淆。前者使用文件记录的 owning group，后者是额外 named group。

③ [操作] 理解自动 mask、`-n` 与 `--mask`

默认情况下，`setfacl` 会在需要时重新计算 mask，使其包含 owning group、所有 named user 和 named group 权限的并集；若同一操作显式给出了 mask，则默认尊重显式值。

```
setfacl -m u:analyst:rw- file           # 通常自动调整 mask
setfacl -n -m u:analyst:rw- file         # 不重新计算现有 mask
setfacl --mask -m u:analyst:rw-,m::r-- file # 即使显式给 mask 也重新计算
```

`-n` 不是通用“更安全”开关。保留过窄 mask 会让新条目看似成功但 effective 不足；自动重算又可能放宽其他条目的上限。正确做法是先读取整个 group class，再选择是否控制 mask。

④ [操作] 用 `-x` 删除一个主体条目

```
setfacl -x u:alice /srv/atlas/plan.md
setfacl -x g:qa /srv/atlas/plan.md
```

删除语法只描述 entry 类型和 qualifier，不应把权限字段作为目标。若目标条目不存在，命令可能报告无可删除的条目或保持不变；无论返回信息如何，都应再次用 `getfacl` 确认最终结构。

⑤ [操作] 区分 `-b` 与 `-k`

```
setfacl -b /srv/atlas/plan.md # 删除扩展 access ACL，保留 owner/group/other 基础条目
setfacl -k /srv/atlas        # 删除目录的 default ACL
```

`-b` 不等于“删除所有权限”，但会清除 named user、named group 和 mask 等扩展 access 条目；`-k` 只删除 default ACL。二者都是策略级变更，不应作为不知道原因时的排错捷径。

⑥ [操作] 用 `x` 表达目录穿越而避免给所有普通文件加执行位

`setfacl` 的权限字段支持大写 `X`：对象是目录，或该文件已经对某类主体具有执行位时，才设置执行权限。

```
setfacl -R -P -m g::rwx,u:auditor:r-X /srv/atlas
```

这适合树形目录的访问策略，但递归变更仍可能影响大量历史对象。高风险环境优先用 `find` 分开目录和普通文件，明确每类目标权限。

验证：


```
getfacl -e -p /srv/atlas/plan.md
sudo -u alice test -w /srv/atlas/plan.md
```

这里的 `sudo -u` 只作为切换测试身份的工具；`sudo` 授权配置属于第 10 章。

帮助入口：`man setfacl`、`setfacl --help`。

[Cheatsheet] `-m` 新增/修改；`-x` 删除一条；`-b` 清扩展 access；`-k` 删 default；默认可能重算 mask，`-n` 保留现有 mask；递归先限定范围。

[操作专题] default ACL 与创建时继承：分别处理目录、新文件和新子目录

default ACL 的本质是创建模板。它解决“未来谁会得到什么初始 ACL”，而不是回溯修复历史。理解继承必须同时观察父目录模板、创建程序请求的 mode、新对象 access ACL，以及新子目录是否继续保存 default ACL。

① [知识点] 新对象首先继承模板，但不能超过创建程序请求的 mode

父目录存在 default ACL 时，新对象用它初始化 access ACL，然后移除创建程序所请求 mode 中没有的权限。常见程序创建普通文件时请求 `0666`，所以即使 default ACL 写有 `x`，普通新文件通常也不会凭空得到执行位；新目录常以 `0777` 为上限，因而可以继承穿越位。

父目录没有 default ACL 时，才使用传统 mode 与 umask 路径。default ACL 存在时，不应再用“简单地把 umask 从 0666/0777 中减掉”预测结果，最终必须创建样本验证。

② [操作] 为目录建立完整 default ACL

假设 `/srv/atlas` 的 owning group 已是 `project`，希望项目组协作、auditor 只读、其他人无权：

```
setfacl -m \
d:u::rwx,d:u:auditor:r-x,d:g::rwx,d:m::rwx,d:o:--- \
/srv/atlas
```

创建 named default user 后，default owner、group、other 与 default mask 必须构成有效 ACL。`setfacl` 可补齐部分必要条目，但显式写出目标模板更便于审阅。

③ [知识点] default ACL 不改变父目录自己的 access ACL

前一条命令只设置模板。如果 auditor 还不能进入 `/srv/atlas`，应单独配置当前目录 access ACL：

```
setfacl -m u:auditor:r-x,m::rwx /srv/atlas
```

建立协作目录时通常要成对考虑：

```
当前目录 access ACL
+
未来对象 default ACL
```

④ [验证点] 新文件与新目录必须分别创建并读取 ACL

```
sudo -u alice touch /srv/atlas/from-alice.txt
sudo -u alice mkdir /srv/atlas/from-alice.d

getfacl -p /srv/atlas/from-alice.txt
getfacl -p /srv/atlas/from-alice.d
```

预期判断：

- 新文件得到从父目录 default ACL 派生的 access ACL；
- 新文件通常不凭 default ACL 获得执行位；
- 新目录得到 access ACL，并继续具有可传递给更深层对象的 default ACL；
- 实际权限仍不得超过创建程序请求的 mode。

⑤ [边界] 已有对象不会因父目录新增 default ACL 而改变

```
getfacl -p /srv/atlas/old.txt
```

若 `old.txt` 在 default ACL 建立前已经存在，它不会自动获得 auditor 或 project 的新条目。必须根据题目范围单独设置其 access ACL，或对明确的已有树执行受控递归修改。

[Cheatsheet] default 是创建模板；父目录当前访问还要 access；新文件与新目录分别验证；普通文件不凭模板自动得到 `x`；旧对象必须另行处理。

[操作专题] 构造协作目录：把当前、历史、未来和真实身份连成闭环

协作目录不是一条命令，而是一组相互独立的状态：目录所属组和 setgid 由第 08 章负责，ACL 负责额外主体和继承模板，功能测试负责证明真实用户能够完成预期动作。最常见的失败来自只配置其中一层。

① [操作] 先建立基线和授权矩阵

变更前记录：

```
ls -ld /srv/atlas
stat -c '%A %a %U %G %n' /srv/atlas
getfacl -R -P -p /srv/atlas > /root/atlas-acl.before
```

再把要求写成矩阵：

身份	目录	普通文件	目标动作
project 成员	<code>rwX</code>	<code>rw-</code>	创建、读取、修改
auditor	<code>r-X</code>	<code>r--</code>	遍历、读取
other	<code>---</code>	<code>---</code>	拒绝

矩阵先于命令，可以防止把“只读”误写成目录 `r--`，也能明确拒绝测试。

② [操作] 分别修复当前目录和已有树

当前目录：

```
setfacl -m g::rwX,u:auditor:r-X,m::rwX,o::--- /srv/atlas
```

对已有树，先预览范围，再按类型设置：

```
find /srv/atlas -xdev -type d -print
find /srv/atlas -xdev -type f -print

find /srv/atlas -xdev -type d -print0 \
  | xargs -0 -r setfacl -m g::rwX,u:auditor:r-X,m::rwX,o::---

find /srv/atlas -xdev -type f -print0 \
  | xargs -0 -r setfacl -m g::rw-,u:auditor:r--,m::rw-,o::---
```

这里按目录和普通文件分开，避免误给普通文件执行位。真实环境还应评估是否包含需要保持可执行的脚本；若有，应按题意保留而不是机械覆盖。

③ [操作] 为未来对象建立 default ACL

```
setfacl -m \
  d:u::rwX,d:u:auditor:r-X,d:g::rwX,d:m::rwX,d:o::--- \
  /srv/atlas
```

如果已有子目录也必须继续继承相同模板，需要在明确范围后为这些目录设置 default ACL；父目录新加模板不会回溯给旧子目录。

④ [验证点] 使用两个协作身份交叉写入

```
sudo -u alice sh -c 'printf "%s\n" alpha > /srv/atlas/team.txt'
sudo -u bob sh -c 'printf "%s\n" beta >> /srv/atlas/team.txt'
getfacl -e -p /srv/atlas/team.txt
```

alice 创建成功只能证明 alice 具有创建能力；bob 能追加才证明新文件的 group class 具有实际写权限，并且 bob 当前会话具有正确组身份。

⑤ [验证点] 同时验证 auditor 的允许与拒绝动作

```
sudo -u auditor cat /srv/atlas/team.txt
sudo -u auditor test ! -w /srv/atlas/team.txt
sudo -u auditor sh -c 'echo denied >> /srv/atlas/team.txt' # 应失败
sudo -u auditor touch /srv/atlas/denied.txt # 应失败
```

拒绝测试是最小特权验收的一部分。只验证“能读”而不验证“不能写”，可能遗漏过宽 mask 或错误的目录写权限。

[Cheatsheet] 基线 → 授权矩阵 → 当前目录 → 已有树 → default 模板 → 两个协作身份交叉写 → 只读身份允许/拒绝测试。

[操作专题] 复制、备份与恢复 ACL：让批量变更可审计、可预演

ACL 是文件元数据。复制一个配置文件的内容不会自动表达“把这个对象的 ACL 策略复制到另一个对象”。批量变更前应保留 ACL 基线；恢复时还要理解备份中可能包含 owner、group 和特殊位注释，不能把恢复当成无影响的单条 ACL 修改。

① [操作] 从参考对象复制 access ACL

```
getfacl --access /srv/atlas/reference.txt \  
| setfacl --set-file=- /srv/atlas/target.txt
```

getfacl 输出可作为 setfacl 输入，注释行会被忽略。--set-file 是替换目标 ACL，不是增量添加；执行前应比较目标现有授权，避免静默覆盖仍需保留的 named 条目。

② [操作] 在确实相同的策略下，把 access ACL 转为目录 default ACL

```
getfacl --access /srv/atlas \  
| setfacl -d -M- /srv/atlas
```

-d 将输入的普通条目应用为 default 条目。只有当“当前目录 access 策略”确实适合作为“未来对象模板”时才使用；目录当前可写不代表普通新文件也应该可执行。

③ [操作] 递归备份完整目录树 ACL

```
getfacl -R -P -p /srv/atlas > /root/atlas-acl.backup
```

- -R 递归；
- -P 物理遍历，避免沿目录符号链接越界；
- -p 保留绝对路径。

备份文件应作为变更证据保存，并检查非空、路径正确、包含预期对象：

```
test -s /root/atlas-acl.backup  
grep -F '# file: /srv/atlas' /root/atlas-acl.backup
```

④ [操作] 用 --test 预演恢复结果

```
setfacl --test --restore=/root/atlas-acl.backup
```

测试模式只列出将得到的 ACL，不修改对象。预演输出仍需人工核对范围、路径、owner/group 注释和特殊位影响。

⑤ [操作] 执行恢复并分层验证

```
setfacl --restore=/root/atlas-acl.backup  
getfacl -R -P -p /srv/atlas > /root/atlas-acl.after-restore
```

--restore 可尝试恢复 owner、owning group，并依据 flags 注释设置 setuid、setgid、sticky 位；若输入没有 flags 注释，相关特殊位可能被清除。因此在生产系统上应优先在副本或测试树验证恢复文件，而不是把它当作仅修改 named ACL 的轻量命令。

⑥ [边界] 内容归档与 ACL 元数据不是同一个问题

cp、mv、tar、rsync 是否保留 ACL 取决于具体命令、选项和目标文件系统。这些工具的完整语义归第 06 章《复制、归档、压缩与远程传输》。本章只要求：迁移或恢复后必须重新 getfacl，不能根据“文件已经复制成功”推断 ACL 也已保留。

[Cheatsheet] 复制 ACL 用 getfacl 管道；替换前先比较；备份用 -R -P -p；恢复先 --test；--restore 可能连 owner/group/特殊位一起处理；内容成功不等于 ACL 保留。

[诊断专题] ACL 条目明明存在，为什么用户仍然 Permission denied

诊断不能从“再加一个 rwx”开始。应从症状进入最有区分度的证据，确定失败位于身份、路径、ACL 匹配、mask、继承时点还是其他安全层。每一步都只做能够解释当前证据的最小修复。

① [诊断点] 先确认实际进程身份，而不是只看账号数据库

```
id analyst
sudo -u analyst id
```

账号刚加入新组时，已存在的 Shell 可能仍使用旧的 supplementary groups。若测试进程没有预期组，继续修改 ACL 会掩盖身份问题。应创建新会话或重新登录后再验证。

② [诊断点] 条目存在但 #effective: 缺少所需权限

症状：

```
user:analyst:rw-          #effective:rw-
mask::r--
```

假设：近期 `chmod g=r` 或显式 mask 修改收窄了 group class。下一条证据：

```
getfacl -e -p /srv/atlas/report.txt
```

最小修复前先检查所有受 mask 影响的条目。若只需要恢复 analyst 写入，而其他条目本来就无 `w`，可以提高 mask 上限；若其他条目声明了 `w` 但业务不应生效，应先收窄那些条目，再调整 mask。

③ [诊断点] default ACL 正确，但失败对象是历史文件

症状：新文件可读，旧文件不可读。证据：

```
getfacl -d -p /srv/atlas
getfacl -p /srv/atlas/old.txt
getfacl -p /srv/atlas/new.txt
```

若模板只出现在目录和新文件，说明 default ACL 工作正常，历史对象没有回溯。最小修复是给明确的旧对象或已有树设置 access ACL，而不是反复修改 default ACL。

④ [诊断点] 目录 ACL 有限，但父路径不可穿越

```
namei -l /srv/atlas/report.txt
```

任何一级父目录缺少目标身份的 `x`，最终文件 ACL 再宽也不可达。`namei` 与传统目录权限完整语义属于第 08 章，本章只把它作为 ACL 诊断前置证据。

⑤ [诊断点] `chmod` 修复了一个现象，却改变了整个 group class

扩展 ACL 存在时，执行：

```
chmod g=r /srv/atlas/report.txt
```

可能把 mask 改为 `rw-`，导致多个 named user/group 同时失去写权限。变更后应立刻比较：

```
ls -l /srv/atlas/report.txt
getfacl -e -p /srv/atlas/report.txt
```

不要只看到 `ls` group 位符合预期就结束，因为它可能表示 mask，而不是 owning group 条目本身。

⑥ [诊断点] ACL 和路径均允许时，转入下一安全层

若：

- 测试进程身份正确；
- 每级父目录可穿越；
- access ACL 匹配且 effective 包含所需权限；
- 文件系统不是只读；

仍失败，应保留这些证据并转向 SELinux、应用策略、挂载或其他层。不要用 `chmod 777` 或清空 ACL 来“验证”SELinux，因为这会破坏原有授权且通常不能解决非 DAC 拒绝。

诊断链：

症状

- 当前 `id`
- 父路径 `namei`
- `getfacl` 选择条目与 `mask`
- 区分旧对象/新对象
- 检查 `chmod` 历史
- 最小修复
- 真实身份再验证
- 其他安全层分流

[Cheatsheet] 先身份，再路径：条目存在看 `effective`；模板正确看对象创建时点；`chmod` 可能收窄 `mask`；ACL 证据闭环后再转 SELinux。

[经典任务] 维护一个已有内容的项目协作目录

环境

系统中已存在：

组：project
成员：alice、bob
只读审计用户：auditor
目录：/srv/atlas
目录所有者：root
目录所属组：project
目录模式：2770
已有对象：
 /srv/atlas/plan.md
 /srv/atlas/archive/q1.txt

第 08 章前置状态已经成立：账号与组成员关系正确、`/srv/atlas` 具有 `setgid`、父路径可穿越。本任务不要求创建用户、修改 `sudoers` 或处理 SELinux。

当前状态

- `/srv/atlas` 没有 default ACL；
- alice 能创建文件，但 bob 不一定能修改 alice 创建的新文件；
- auditor 无法读取历史文件；
- 已有文件不得删除、重建或覆盖；
- 变更前尚未保存 ACL 基线。

目标终态

1. project 组成员能进入 `/srv/atlas`，创建、读取和修改项目文件；
2. alice 创建的新文件能被 bob 追加；
3. 新建子目录继续继承同一协作策略；
4. auditor 能遍历目录并读取普通文件，但不能修改、创建或删除；
5. 已有的 `plan.md` 与 `archive/q1.txt` 也满足 auditor 只读；
6. other 不获得访问权；
7. 保存可用于恢复的 ACL 备份。

限制条件

- 不得使用 `chmod 777`；
- 不得清空全部 ACL 后重新配置；
- 不得删除或重建已有对象；
- 递归修改前必须列出目标范围；
- 不得把命令成功或 `getfacl` 输出代替多身份功能测试；
- 不能声称已在本会话的 RHEL 9 VM 中实测。

验收证据

层次	必须提交的证据
基线	<code>ls -ld、stat</code> 、递归 ACL 备份
当前目录	access ACL 与 default ACL
已有对象	两个指定文件的 <code>getfacl -e</code>

层次	必须提交的证据
新文件	alice 创建、bob 追加、ACL 输出
新目录	access ACL 与继续存在的 default ACL
auditor 允许	读取已有文件和新文件成功
auditor 拒绝	修改文件、创建文件失败
范围	递归操作未越出 <code>/srv/atlas</code>

请先独立完成。参考解答从下一页开始。

[参考解答] 维护已有项目协作目录

以下命令是依据题目环境给出的参考操作路径。本章未执行 RHEL 9 命令级实机验证，所有输出均应在真实练习环境中取得，不应照抄虚构结果。

① [调查] 确认基线、身份和对象范围

```
id alice
id bob
id auditor

ls -ld /srv/atlas
stat -c '%A %a %U %G %n' /srv/atlas
find /srv/atlas -xdev -printf '%y %p\n'

getfacl -R -P -p /srv/atlas > /root/atlas-acl.before
test -s /root/atlas-acl.before
```

判断：

- `id` 证明账号数据库解析出的组身份；真实测试仍要用新进程；
- `2770 root:project` 属于第 08 章前置，不在本任务重建；
- `-xdev` 限制 `find` 不跨文件系统，`-P` 限制 ACL 递归不跟随目录符号链接；
- 备份在变更前生成，便于审计和恢复。

② [操作] 配置当前目录 access ACL

```
setfacl -m g::rwx,u:auditor:r-x,m::rwx,o:--- /srv/atlas
```

参数解释：

- `g::rwx`：owning group project 对当前目录完全协作；
- `u:auditor:r-x`：auditor 可列出和穿越，但无目录写权限；
- `m::rwx`：group class 上限允许项目组写入；auditor 自身条目没有 `w`，所以不会因此获得写权限；
- `o:---`：其他主体无权。

③ [操作] 修复已有目录和普通文件

先确认类型：

```
find /srv/atlas -xdev -type d -print
find /srv/atlas -xdev -type f -print
```

再分类型修改：

```
find /srv/atlas -xdev -type d -print0 \
| xargs -0 -r setfacl -m g::rwx,u:auditor:r-x,m::rwx,o:---

find /srv/atlas -xdev -type f -print0 \
| xargs -0 -r setfacl -m g::rw-,u:auditor:r--,m::rw-,o:---
```

这里没有删除或重建任何文件。目录与普通文件分开，避免给所有普通文件增加执行位。若树中存在原本必须可执行的脚本，应在真实任务中按清单单独保留其执行权限，而不是机械套用第二条命令。

④ [操作] 给当前和已有子目录设置 default ACL

题目要求新子目录继续继承策略。根目录先设置：

```
setfacl -m \
d:u::rwx,d:u:auditor:r-x,d:g::rwx,d:m::rwx,d:o:--- \
/srv/atlas
```

如果已有 `/srv/atlas/archive` 也会直接接收新对象，应给已有目录设置同样模板：

```
find /srv/atlas -xdev -type d -print0 \
| xargs -0 -r setfacl -m \
d:u::rwx,d:u:auditor:r-x,d:g::rwx,d:m::rwx,d:o:---
```

这一步只对目录设置 default ACL；普通文件不能拥有 default ACL。

⑤ [结构验证] 读取当前、历史和模板状态

```
getfacl -a -e -p /srv/atlas
getfacl -d -e -p /srv/atlas
getfacl -e -p /srv/atlas/plan.md
getfacl -e -p /srv/atlas/archive/q1.txt
getfacl -d -e -p /srv/atlas/archive
```

判断重点：

- project 对目录 effective 包含 `rwx`，对普通文件包含 `rw-`；
- auditor 对目录 effective 为 `r-x`，对普通文件为 `r--`；
- other 为 `---`；
- root 和已有子目录具有预期 default ACL；
- 没有 `#effective:` 暴露意外的 mask 截断。

⑥ [功能验证] 创建新文件、新目录并交叉写入

```
sudo -u alice sh -c 'printf "%s\n" alpha > /srv/atlas/from-alice.txt'
sudo -u bob sh -c 'printf "%s\n" beta >> /srv/atlas/from-alice.txt'

sudo -u alice mkdir /srv/atlas/from-alice.d
sudo -u alice touch /srv/atlas/from-alice.d/nested.txt

getfacl -e -p /srv/atlas/from-alice.txt
getfacl -e -p /srv/atlas/from-alice.d
getfacl -d -e -p /srv/atlas/from-alice.d
getfacl -e -p /srv/atlas/from-alice.d/nested.txt
```

分层含义：

- alice 创建证明目录允许项目成员创建；
- bob 追加证明新文件 group class 的 effective 写权限正确；
- 新子目录同时检查 access 和 default ACL，证明策略可以继续向下传递；
- `getfacl` 仍不能替代实际写入，因此两类证据都要保留。

⑦ [功能验证] auditor 的允许与拒绝

```
sudo -u auditor cat /srv/atlas/plan.md
sudo -u auditor cat /srv/atlas/from-alice.txt
sudo -u auditor cat /srv/atlas/from-alice.d/nested.txt

sudo -u auditor sh -c 'echo denied >> /srv/atlas/from-alice.txt' # 预期失败
sudo -u auditor touch /srv/atlas/denied.txt # 预期失败
```

不能只执行 `test ! -w` 就结束，因为实际应用动作能同时暴露父路径、ACL 和创建/打开语义。失败命令应在练习环境中确认返回非零，并记录错误对象。

⑧ [恢复准备] 生成变更后备份并预演恢复

```
getfacl -R -P -p /srv/atlas > /root/atlas-acl.after
setfacl --test --restore=/root/atlas-acl.before
```

`--test` 只预演。真正执行 `--restore` 前应确认它可能恢复 owner、group 和特殊位。若本任务只要求保留回滚依据，不应为了“证明恢复可用”而破坏已经完成的目标状态。

典型错误

1. 只设置 default ACL，导致当前目录和历史文件仍不可访问；
2. 只对根目录设置 access ACL，忽略历史子目录和普通文件；
3. 将 auditor 目录权限写成 `r--`，导致无法穿越；
4. 把 mask 无条件设为 `rwX`，却没有检查其他 named 条目是否因此被放宽；
5. 用一条递归 `rwX` 给所有普通文件添加执行位；
6. 只检查 `getfacl`，没有用 `alice`、`bob`、`auditor` 做允许和拒绝测试；
7. 没有在变更前备份 ACL。

[经典任务] 诊断 named user 有 `rw-` 但实际无法写入

环境与症状

analyst 需要修改 `/srv/atlas/report.txt`。当前：

```
-rw-r-----+ 1 root project ... /srv/atlas/report.txt

user::rw-
user:analyst:rw-          #effective:r--
group::rw-               #effective:r--
group:qa:r--
mask::r--
other::---
```

analyst 能读取但不能追加。管理员回忆近期执行过：

```
chmod g=r /srv/atlas/report.txt
```

目标终态

- analyst 恢复读写；
- project owning group 仍可读写；
- qa 仍然只读；
- other 仍无权；
- 说明 `ls -l` group 位变化的原因；
- 以 analyst 实际身份完成追加验证。

限制条件

- 不得删除全部 ACL；
- 不得无调查执行 `mask::rwX`；
- 不得改变 owner 或 owning group；
- 不得给 qa 写权限；
- 必须保留修复前后 `getfacl -e` 证据。

请先独立完成。参考解答从下一页开始。

[参考解答] 修复被 `chmod` 收窄的 mask

① [当前证据] 确认身份和实际失败

```
sudo -u analyst id
sudo -u analyst sh -c 'echo probe >> /srv/atlas/report.txt'
```

若测试进程身份不正确，应先修复会话身份；本题已知 analyst 能读且 named user 匹配，继续调查 ACL effective。

② [区分性证据] 强制显示所有 effective 权限

```
getfacl -e -p /srv/atlas/report.txt
```

关键推理：

```
analyst 条目声明 rw-  
n mask r--  
= effective r--
```

owning group 的 `rw-` 也被同一 mask 截断为 `r--`。qa 条目本身只有 `r--`，即使 mask 提高到 `rw-`，qa 仍不会获得 `w`。

③ [最小修复] 把 group class 上限恢复到 `rw-`

```
setfacl -m m::rw- /srv/atlas/report.txt
```

为什么不是 `rwX`：普通数据文件不需要执行，`rw-` 已足以满足 analyst 和 project 写入，也不会扩大执行权限。

④ [再验证] 比较结构、模式摘要和功能

```
getfacl -e -p /srv/atlas/report.txt  
ls -l /srv/atlas/report.txt  
  
sudo -u analyst sh -c 'echo fixed >> /srv/atlas/report.txt'  
sudo -u analyst tail -n 1 /srv/atlas/report.txt
```

预期逻辑：

- `user:analyst:rw-` 不再显示被截断为只读；
- `group::rw-` effective 恢复为 `rw-`；
- `group:qa:r--` 仍只有只读，因为其条目本身没有 `w`；
- `ls -l` 的 group 位从 `r--` 变为 `rw-`，表示扩展 ACL 中的 mask 发生变化，不等同于只修改了 `group::`；
- analyst 实际追加成功。

⑤ [根因说明] `chmod g=r` 为什么影响 named user

扩展 ACL 有 mask 时，传统 group mode bits 对应 mask。`chmod g=r` 把 group 位设置为只读，也就把 `mask::` 收窄为 `r--`。named user analyst、owning group project 和 named group qa 都属于 group class，因此它们的 effective 上限一起变化。

这不是 `chmod` 与 ACL 相互独立，而是两者存在双向映射。以后对带 `+` 的对象执行 `chmod` 后，应立即运行 `getfacl -e`，检查是否改变了 named 条目的有效权限。

[本章收束] 把 ACL 还原成声明、有效、创建与功能四层证据

ACL 的稳定工作方法不是“先加 `rwX`”，而是从对象与证据推进：

```
建立基线与对象范围  
→ 分离 access ACL 和 default ACL  
→ 识别主体匹配与 mask  
→ 对当前、历史、未来状态做最小修改  
→ 用结构证据验证存储结果  
→ 用真实身份验证允许动作与拒绝动作  
→ 保存恢复依据
```

章末检查清单

- ☐ 已明确目标是当前对象、已有树还是未来新对象；
- ☐ 已分别读取 access ACL 和 default ACL；
- ☐ 已确认目标身份会匹配 owner、named user、group class 或 other；
- ☐ 已检查 `mask::` 和 `#effective::`；
- ☐ 已评估 `chmod` 是否改变 mask；
- ☐ 递归操作前已列出范围，并避免跟随目录符号链接越界；
- ☐ 新文件和新目录已分别验证继承；
- ☐ 已用至少两个协作身份完成交叉写入；
- ☐ 已验证只读主体既“能读”又“不能写/创建”；

- [] 已保存 ACL 基线或恢复文件，并理解 `--restore` 的影响范围。

主要判断表

观察或症状	首先说明什么	下一条最有区分度的证据	不能直接证明什么
<code>ls -l</code> 末尾出现 <code>+</code>	存在扩展 access ACL 或 default ACL	<code>getfacl -p</code>	具体主体与实际权限
<code>user:analyst:rw-</code>	名义条目包含读写	<code>getfacl -e -p</code>	effective 一定包含写
<code>default:user:auditor:r-x</code>	未来对象模板含 auditor	新建对象后 <code>getfacl</code>	父目录和旧文件已授权
新文件可协作、旧文件失败	default 可能正常	对比旧/新对象 access ACL	应继续修改 default ACL
<code>chmod g=r</code> 后 named user 失写	mask 可能被收窄	变更前 <code>getfacl -e</code>	只有 owning group 被修改
ACL 结构正确仍拒绝	失败可能不在 ACL 层	<code>id</code> 、 <code>namei -l</code> 、挂载状态，再转其他安全层	不能据此认定放宽为 <code>chmod 777</code> 是正确修复
<code>setfacl</code> 返回成功	修改请求被接受	重新 <code>getfacl</code> + 真实身份动作	业务终态正确

向下一章交接

本章解决的是“对象本身允许谁访问”的自主访问控制。下一章《`sudo` 与最小特权授权》处理的是“某个已登录用户可以代表谁执行哪些管理命令”。ACL 不授予提权能力，`sudo` 也不替代目标文件和目录的 ACL；两者分别控制数据访问边界与管理命令边界。

静态可信声明：本章依据 RHEL 9 课程资料以及 `acl(5)`、`getfacl(1)`、`setfacl(1)` 进行静态核对；未执行命令级实机验证，文中命令均作为推荐操作与验证路径。

sudo 与最小特权授权

把“给 sudo 权限”改写成可审计的策略请求：谁、在哪台主机、以谁身份、执行哪个程序和哪组参数。

对象模型操作语义验证诊断经典任务

大字号阅读版

本章阅读导航

先抓住一条主线：sudo 不是“把用户变成 root”，而是读取策略后，对一次具体调用请求进行匹配、认证、身份切换和执行。

- 01明确主体谁在发起请求
- 02确认位置规则在哪台主机成立
- 03固定 Runas命令最终以谁运行
- 04收缩动作绝对路径与参数边界
- 05校验策略片段、全局与可见授权
- 06验证边界允许与拒绝矩阵

专题地图

- 知识专题 sudo 的多维策略判定
- 知识专题 从左到右读懂 sudoers
- 知识专题 路径、参数、标签与加载顺序
- 操作专题 推导最小命令集合
- 操作专题 维护 `/etc/sudoers.d`
- 操作专题 `'sudo -l'`、真实调用与 `'sudoedit'`
- 诊断专题 规则可见但请求不匹配
- 经典任务 Web 运维组最小服务授权

阅读时持续回答

1. 调用者与 Runas 身份分别是谁？
2. 主机字段和命令字段中的 `'ALL'` 各表示什么？
3. 规则是否固定了真实绝对路径？
4. 命令参数是否仍可选择其他动作？
5. 本次无密码提示来自哪里？
6. 什么证据只证明策略，什么证据才能证明功能？
7. 哪些拒绝路径必须主动测试？

边界：用户与组生命周期归第 07 章；传统权限归第 08 章；Ansible 自动化授权归 RHCE 身份章节。
第 10 章 · 正文

管理员很少真正需要把一个普通用户“变成 root”。更常见的目标是：让值班人员只查看某个服务，让数据库管理员只执行一条备份命令，让应用维护者只编辑一份受控配置。`sudo` 的价值不在于提供另一条进入 root Shell 的捷径，而在于把一次特权调用拆成可判定的条件：谁在哪台主机上，准备以哪个目标身份，运行哪个可执行文件和哪组参数。

如果规则只写成 `%ops ALL=(ALL) ALL`，系统确实容易“用起来”，但授权边界几乎消失。相反，若只盯着一行 sudoers 语法而不验证真实调用，也会出现另一类错误：`sudo -l` 看得到规则，实际命令却因 Runas、路径或参数不匹配而被拒绝；或者命令已经获准执行，但目标服务本身仍然失败。

本章以“把业务要求收缩为可匹配请求”为主线，先建立 sudoers 对象模型，再进入配置、查询、验证和诊断。用户与组的创建留给第 07 章，文件权限留给第 08 章，服务内部状态留给第 12 章，自动化授权留给 RHCE 身份章节。

每次读到一条规则，都要重新回答：

谁发起请求？规则在哪台主机成立？命令以谁的身份运行？允许的是哪个程序和哪组参数？是否要求认证？哪些相似调用必须被拒绝？

概念

sudoers 规则 是对特权调用请求的策略描述，而不是一个模糊的“管理员角色”。规则把调用者、主机、Runas 身份和命令规格关联起来；任何关键维度不匹配，请求都不会命中预期授权。理解这条关系后，`'sudo -l'` 才不只是“有没有权限”的列表，而是后续逐字段核对的证据入口。

概念

主体 是发起 sudo 请求的用户、Unix 组或 `'User_Alias'`。主体回答“谁可以提出请求”，不回答命令最终以谁运行。组的创建、附加组生效和账号生命周期归第 07 章；本章只使用已经成立的身份事实。

概念

Runas 是命令执行时使用的目标用户和目标组。普通用户 `'alice'` 可以发起请求，而命令以 `'root'`、`'postgres'` 或指定组运行；调用者与 Runas 混淆，是“列表有规则但默认调用失败”的高频原因。

概念

主机匹配 决定规则在哪些主机上成立。单机任务常写 `ALL`，但这个 `ALL` 位于主机字段，并不等于允许所有命令。集中分发同一策略时，主机字段可以继续收缩授权位置；本章不展开 LDAP 或 IdM 策略。

概念

命令规格 由可执行文件路径和可选参数条件组成。绝对路径固定入口程序，参数继续固定程序允许执行的子动作；如果只写路径而不限制参数，用户通常可以自行选择参数，因此“参数也是授权边界”是本章最重要的判断。

概念

认证状态 与授权范围是两个维度。`PASSWD`、`NOPASSWD` 和认证时间戳影响本次是否需要验证调用者密码，却不会自动改变允许的命令集合。一次没有提示密码，既可能来自免密规则，也可能只是已有时间戳尚未失效。

概念

最小特权 不只是“成功命令很少”，还要求相似但未授权的路径、参数、对象和 root Shell 被明确拒绝。验收必须同时证明允许矩阵和拒绝矩阵；只要用户仍能通过 Shell、解释器、通配符或可修改脚本选择任意后续动作，边界就没有真正收紧。

操作语义

先建立五个关键入口的接口地图。正文后续只在需要时解释具体操作，不机械重复整份手册页。

`sudo`

SYNOPSIS

```
sudo [-u user] [-g group] [-n] command [argument ...]
sudo -k
```

按当前 sudoers 策略发起一次特权调用，必要时验证调用者身份，并以匹配到的 Runas 凭据执行目标程序。

`-u user`指定目标 Runas 用户；省略时通常请求默认目标用户。`-g group`指定目标 Runas 组；必须与规则允许的组匹配。`-n`非交互模式；需要密码时直接失败，适合验证免密边界。`-k`使后续需要认证的调用重新验证，避免旧时间戳干扰测试。

`sudo -l`

SYNOPSIS

```
sudo -l
sudo -l -U user
```

列出当前策略中与调用者相关的 Defaults、Runas 和命令规格，是匹配诊断的起点，不是最终功能验收。

`sudo -l`当前用户查看自己的可见授权。`-U user`由 root 或具有相应能力的管理员查看指定用户；不能假设普通用户可任意使用。

`visudo`

SYNOPSIS

```
visudo [-c] [-f sudoers-file]
```

以安全方式编辑 sudoers：处理并发锁、保存前解析，并在检查模式下验证配置。

`-f file`编辑或检查指定 sudoers 文件，适合 `/etc/sudoers.d` 片段。`-c`检查配置；配合 `-f` 检查片段，单独使用时检查完整策略树。

`sudoedit` / `sudo -e`

SYNOPSIS

```
sudoedit file [...]
sudo -e file [...]
```

让编辑器以调用者身份处理临时副本，再由 sudo 受控写回目标文件，避免直接授予通用 root 编辑器。

`sudoedit path`在 sudoers 中作为内建命令书写，不需要前导可执行路径。精确路径只列出需要编辑的文件，并检查父目录不受调用者控制。

`/etc/sudoers.d` 管理

TYPICAL FLOW

```
visudo -f /etc/sudoers.d/70-webops-httpd
visudo -cf /etc/sudoers.d/70-webops-httpd
visudo -c
```

把本地规则拆成可审计片段；文件名、词法加载顺序、所有者和模式都会影响最终策略。

文件名避免 `` 和结尾 ``~`; 使用等宽零填充编号表达词法顺序。控制权由 root 管理，推荐模式 `0440`；片段通过后仍需全局检查。

[知识专题] sudo 的本质：一次多维策略判定

面对“给某用户 sudo 权限”的要求，最先要问的不是“在哪一行加 ALL”，而是这次工作需要授权哪一类动作。把业务目标改写为可匹配的调用请求后，才能判断哪些字段必须固定、哪些字段可以复用、哪些能力必须明确拒绝。

① [知识点] 调用请求由多个维度共同组成

可把一次 sudo 请求抽象为：

```
调用者身份
× 当前主机
× 目标 Runas 用户/组
× 可执行文件路径
× 命令行参数
× 命令标签与 Defaults
× 执行环境
-----
允许或拒绝
```

这解释了为什么“命令名字看起来一样”仍可能不匹配。下面三个调用并不是同一个请求：

```
sudo /usr/bin/systemctl status httpd
sudo /usr/bin/systemctl --no-pager status httpd
sudo -u apache /usr/bin/systemctl status httpd
```

它们的参数或 Runas 身份不同。sudoers 不是按自然语言理解“都是查看 httpd”，而是按规则语义匹配。

② [知识点] 授权、认证、执行和审计是四个阶段

```
规则加载
→ 请求匹配
→ 调用者认证
→ 切换为 Runas 凭据并构造环境
→ 执行目标程序
→ 记录允许、拒绝或错误
```

- 授权回答是否允许这组调用条件。
- 认证通常验证调用者，而不是要求 root 密码。
- 执行由目标程序自己完成；sudo 不能保证服务、文件或网络的业务终态正确。
- 审计记录谁请求了什么以及结果，但日志的完整查询和轮转归第 13 章。

③ [知识点] root Shell 会把有限授权重新扩大为任意授权

允许 `/usr/bin/systemctl restart httpd` 与允许 `/bin/bash` 的安全意义完全不同。后者让用户在 root Shell 中自行选择后续命令，原本针对路径和参数的限制失去作用。因此以下能力通常不应作为最小授权的默认答案：

```
sudo -i
sudo -s
sudo /bin/bash
sudo /bin/sh
sudo 任意通用解释器
```

如果任务确实要求完整系统管理权限，那是另一种授权目标；不能把它包装成“最小特权”。

[Cheatsheet] sudo 不是“变成 root”；先分清调用者与 Runas，再固定主机、路径和参数。授权成功只证明策略允许，不证明目标程序成功。

[知识专题] 从左到右读懂一条 sudoers 规则

一条常见规则可按四段读取：主体、主机、Runas 和命令规格。读取时不要跳过主机字段，也不要把括号中的身份误认为调用者。

```
%webops ALL=(root:root) /usr/bin/systemctl --no-pager status httpd
```

① [知识点] 主体可以是用户、Unix 组或 User_Alias

```
alice    ALL=(root) /usr/bin/id
%webops  ALL=(root) /usr/bin/systemctl --no-pager status httpd
```

- `alice` 只匹配该用户。
- `%webops` 匹配 Unix 组成员；组的创建和成员生命周期归第 07 章。
- `User_Alias` 可以把多个主体组合为一个可复用名称。

别名名称通常使用大写字母开头的全大写标识，但安全性并不来自“用了别名”，而来自别名中成员以及后续命令范围是否正确。

② [知识点] 主机字段回答规则在哪些主机成立

单机 RHCSA 任务中常见 `ALL`：

```
%webops ALL=(root) /usr/bin/systemctl --no-pager status httpd
```

这里的第一个 `ALL` 是主机匹配，不是命令 `ALL`。在集中分发同一策略的环境中，主机字段可以限制特定主机或 `Host_Alias`。本章只讲本地文件中的匹配模型，不扩展 LDAP 或集中 `sudo` 策略。

③ [知识点] Runas 规格决定目标用户和目标组

```
alice ALL=(root) /usr/bin/id
alice ALL=(postgres) /usr/bin/psql
alice ALL=(root:adm) /usr/bin/id
```

括号中的第一部分是 Runas 用户，冒号后是 Runas 组。省略 Runas 规格时，常见默认目标用户为 `root`，但在阅读现有系统时仍应查看实际 Defaults 和规则，而不是只凭习惯猜测。

调用时可使用：

```
sudo -u postgres /usr/bin/psql
sudo -g adm /usr/bin/id
```

如果规则只允许 `(postgres)`，用户请求默认 `root` 身份时不会命中该授权。

④ [知识点] 命令列表与标签有作用范围

```
%webops ALL=(root) NOPASSWD: /usr/bin/systemctl --no-pager status httpd, \
      PASSWD: /usr/bin/systemctl restart httpd
```

逗号分隔两个命令规格。`NOPASSWD`：从出现位置开始影响后续命令，直到同一列表中被 `PASSWD`：等标签改变。为降低误读，复杂规则应换行并在需要切换认证要求的位置显式写出 `PASSWD`：或 `NOPASSWD`：，不要依赖读者记住远处标签的继承范围。

[Cheatsheet] `%` 是 Unix 组；主机 `ALL`、Runas `ALL` 和命令 `ALL` 位于不同字段，含义不同；标签从出现位置向后生效。

[知识专题] 别名、多个匹配项与最终生效规则

`sudoers` 允许用别名降低重复，也允许同一用户命中多条规则。这里最危险的误区是认为“更具体的规则一定覆盖更宽泛的规则”。实际调查必须考虑加载顺序和最后匹配结果。

① [知识点] 四类别名分别复用四类对象

```
User_Alias  WEBADMINS = alice, bob
Host_Alias  WEBSERVERS = servera, serverb
Runas_Alias WEBUSERS = root, apache
Cmnd_Alias  HTTPD_READ = /usr/bin/systemctl --no-pager status httpd
```

```
WEBADMINS WEBSERVERS=(WEBUSERS) HTTPD_READ
```

- `User_Alias`：调用者集合；
- `Host_Alias`：主机集合；
- `Runas_Alias`：目标身份集合；
- `Cmnd_Alias`：命令规格集合。

别名适合复用稳定集合。只有一条规则时，直接写清楚往往比引入多层别名更容易审计。

② [知识点] 多个匹配项按读取顺序应用

当一个调用者同时匹配多条规则时，不能只找到第一条就停止。发生冲突时，最后匹配值可能决定最终结果，而且“最后”不等于“最具体”。

例如先写精确免密，后面又有宽泛需密码规则：

```
%webops ALL=(root) NOPASSWD: /usr/bin/systemctl --no-pager status httpd
%webops ALL=(root) PASSWD: ALL
```

调查密码行为时必须同时看到两条及其顺序，不能只引用第一条证明 NOPASSWD。

③ [知识点] 否定项不适合作为宽泛授权的主要防线

下面的思路看似“允许全部但排除危险命令”：

```
alice ALL=(root) ALL, !/bin/bash
```

这不是可靠的最小授权设计。用户可能通过其他 Shell、解释器、编辑器逃逸、程序子命令或被允许程序的插件机制获得等价能力。优先设计短小允许列表，而不是先给 ALL 再尝试列举所有危险例外。

④ [知识点] 规则可读性也是安全控制的一部分

建议每个 `/etc/sudoers.d` 片段围绕一个明确职责：

```
70-webops-httpd
71-dbops-backup
80-helpdesk-account-status
```

同一片段中把业务说明写为注释、把命令按用途分组，并避免把完全无关的团队与命令塞进同一 `Cmnd_Alias`。可读性越差，越容易在后续追加规则时意外放宽边界。

[Cheatsheet] 别名只负责复用；多个匹配要看完整顺序；最小授权优先允许列表，不以“ALL 加否定项”为主结构。

[知识专题] 命令路径和参数共同定义授权动作

sudoers 中的命令规格不是一个模糊的“功能名称”。可执行文件路径回答运行哪个程序，参数字符串回答这个程序能执行哪个子动作。最小特权往往主要发生在参数层。

① [知识点] 使用绝对路径减少 PATH 解析歧义

```
alice ALL=(root) /usr/bin/systemctl --no-pager status httpd
```

规则应以目标机上的真实路径为准。编写前先查询：

```
command -v systemctl
readlink -f "$(command -v systemctl)"
```

不要从另一台发行版或旧笔记复制路径。绝对路径只固定了入口程序；若该程序内部再次按 PATH 调用子命令，仍要评估执行环境和程序本身的安全边界。路径固定也不等于文件内容不可变，因此还要确认可执行文件、脚本及父目录不受调用者控制。

② [知识点] 未写参数通常表示允许任意参数

```
alice ALL=(root) /usr/bin/systemctl
```

这通常不是“只允许无参数运行 systemctl”，而是允许用户自行选择参数，可能覆盖启动、停止、编辑、设置属性等大量动作。若目标是只查看 httpd，应把参数写入命令规格：

```
alice ALL=(root) /usr/bin/systemctl --no-pager status httpd
```

若确实要表达“不允许任何参数”，命令规格可在路径后写空参数字符串 `""`。这是一项精确语义，但 RHCSA 常规任务更常见的是把题目要求的实际参数完整写出；无论采用哪种形式，都要在目标机执行允许与拒绝测试。

③ [知识点] 参数必须按规则语义匹配

以下调用在 `sudoers` 匹配上可能不同：

```
/usr/bin/systemctl --no-pager status httpd
/usr/bin/systemctl status --no-pager httpd
/usr/bin/systemctl --no-pager status httpd.service
```

即使目标程序最终可能接受多种等价写法，`sudoers` 仍按命令行参数进行匹配。验收时应使用题目要求的精确调用，并单独测试未授权变式是否被拒绝。

④ [知识点] 通配符会跨越参数边界

参数通配符匹配的是组合后的参数字符串，`*` 可能匹配空白。规则：

```
%logreaders ALL=(root) /usr/bin/cat /var/log/app*
```

可能不只允许读取一个以 `/var/log/app` 开头的文件，还可能匹配追加的第二个敏感路径。参数范围简单时列出精确命令；范围复杂时优先用受控专用工具或经过版本确认的更严格匹配方式，不要把宽泛通配符当作考试捷径。

[Cheatsheet] 绝对路径固定程序；参数固定动作；没有参数限制通常过宽；通配符可匹配空白并跨越参数边界。

[知识专题] Shell、包装脚本和可修改对象为何会扩大权限

有些规则表面上只允许一个文件，实际却把后续命令选择权交给用户。判断安全边界时，不能只看 `sudoers` 一行，还要看被执行程序能否启动其他命令，以及用户能否修改程序、脚本、配置或其父目录。

① [知识点] 通用 Shell 和解释器等于可编程执行入口

以下规则都很难称为有限授权：

```
alice ALL=(root) /bin/bash
alice ALL=(root) /usr/bin/python3
alice ALL=(root) /usr/bin/perl
```

解释器可以读取脚本、执行系统调用并启动其他程序。即使规则尝试固定一部分参数，也要评估参数注入、模块加载和用户可写文件。

② [知识点] root 执行用户可写脚本会把脚本内容变成授权面

```
%ops ALL=(root) /usr/local/bin/restart-web.sh
```

这条规则是否安全，取决于：

```
脚本自身是否由 root 控制
→ 父目录是否由普通用户可写
→ 脚本调用子命令是否使用安全路径
→ 是否读取用户可控配置或环境
→ 参数是否会进入 Shell 解释
```

若 `ops` 能修改脚本或替换父目录中的文件，`sudoers` 即使写得很精确，用户仍能把任意内容变成 `root` 命令。

③ [知识点] 编辑器、分页器和可逃逸程序需要专门评估

许多交互程序支持启动 Shell、读取任意文件、写文件、加载插件或执行外部命令。不能因为命令名称是“查看器”或“编辑器”就默认安全。若目标只是修改某个配置文件，优先考虑 `sudoedit`；若目标只是读取状态，优先使用非交互、固定参数的命令。

④ [知识点] 目录可写性和文件所有权是授权链的一部分

本章不重复第 08、09 章的权限与 ACL 机制，但在授权前必须引用其结论：

```
namei -l /usr/local/bin/restart-web.sh
stat /usr/local/bin/restart-web.sh
getfacl /usr/local/bin/restart-web.sh
```

这些证据用于确认普通用户不能替换执行对象。若发现可写性问题，应先修复对象控制权，而不是通过增加 `sudo` 规则绕过。

[Cheatsheet] 固定脚本路径不等于固定行为；同时审查解释器、可逃逸程序、脚本内容、父目录和环境输入。

[操作专题] 从业务要求推导最小命令集合

最小授权不是在写完规则后“尽量删一点”，而是在动手前把允许动作和拒绝动作分别写清楚。这个过程能阻止一个常见错误：把“管理 httpd”直接翻译成“允许任意 systemctl”。

① [操作] 把模糊动词改写为可验证动作

原要求：

```
允许 webops 管理 Web 服务。
```

需要继续澄清为：

```
允许：查看 httpd 状态、重启 httpd
拒绝：停止、禁用、mask、编辑 unit、操作其他服务、进入 root Shell
目标身份：root
认证：查看免密、重启需调用者密码
```

这些条件会直接映射到命令路径、参数和标签。

② [操作] 在目标系统确认真实命令和调用形式

```
command -v systemctl
/usr/bin/systemctl --no-pager status httpd
/usr/bin/systemctl restart httpd
```

先以管理员身份确认目标命令本身有效，再写 sudoers。不要用 sudo 权限问题掩盖命令拼写、unit 名称或服务故障。

③ [操作] 编写短小、显式的允许列表

```
%webops ALL=(root) NOPASSWD: /usr/bin/systemctl --no-pager status httpd, \
    PASSWD: /usr/bin/systemctl restart httpd
```

这条规则明确：

- 主体为 %webops；
- 适用于本规则匹配的主机；
- 目标用户为 root；
- 只允许两个参数组合；
- 两个动作的认证要求不同。

④ [操作] 写规则时同步建立拒绝矩阵

至少列出并在验收时执行：

```
sudo /usr/bin/systemctl stop httpd
sudo /usr/bin/systemctl restart sshd
sudo /usr/bin/systemctl --no-pager status sshd
sudo -i
sudo -s
sudo /bin/bash
```

预期结果是被策略拒绝。拒绝测试不是“额外安全检查”，而是证明规则没有超出目标边界的必要证据。

[Cheatsheet] 先写允许/拒绝矩阵，再写 sudoers；路径和参数来自目标机；测试不能只跑成功路径。

[操作专题] 安全维护 `/etc/sudoers.d` 片段

直接使用普通编辑器修改 `/etc/sudoers`，一旦产生语法错误，可能使后续特权访问受阻。`visudo` 的价值不仅是打开编辑器，还包括锁定和语法检查。新规则优先放在独立片段中，便于审计、回退和更新保留。

① [操作] 使用 `visudo -f` 创建或编辑片段

```
visudo -f /etc/sudoers.d/70-webops-httpd
```

保存退出时进行语法检查。不要把 `vim /etc/sudoers.d/...` 作为默认教学路径。

② [操作] 选择会被 include 机制读取的文件名

常见 `/etc/sudoers` 包含目录导入指令，例如：

```
@includedir /etc/sudoers.d
```

一些 RHEL 9 系统仍可见兼容写法 `#includedir /etc/sudoers.d`。这里的 `#` 不是普通注释标记，而是旧式 include 指令的一部分。维护时以目标机现有 `sudoers` 为准，不要凭笔记替换语法。目录中的文件名不得包含点号，也不得以 `~` 结尾；编辑器备份文件因此通常不会被加载。

推荐使用零填充编号：

```
20-base-operators
70-webops-httpd
90-local-overrides
```

编号帮助表达预期读取顺序，但不能代替检查完整策略树。

③ [操作] 核对所有者和模式

```
chown root:root /etc/sudoers.d/70-webops-httpd
chmod 0440 /etc/sudoers.d/70-webops-httpd
stat -c '%U:%G %a %n' /etc/sudoers.d/70-webops-httpd
```

片段应由可信管理员控制。模式不安全、所有者错误或父目录被篡改都可能使策略失效或形成安全风险。

④ [操作] 先校验片段，再校验完整策略树

```
visudo -cf /etc/sudoers.d/70-webops-httpd
visudo -c
```

`-cf` 适合快速定位当前片段语法；`-c` 才能把 `/etc/sudoers` 和 include 片段作为整体检查。一个片段单独通过，不代表别名在完整策略中没有冲突，也不代表后续规则没有改变认证标签。

[Cheatsheet] `visudo -f` 编辑；文件名无 `.`、不以 `~` 结尾；root 控制、推荐 `0440`；片段与全局必须分别校验。

[操作专题] 用 `sudo -l` 与真实调用建立分层证据

`sudo -l` 是定位匹配规则的入口，但它不是最终验收。列表输出可能包含 Defaults、Runas 和命令规格；真正调用还会受到路径、参数、认证状态和目标程序自身状态影响。

① [操作] 当前调用者查看自己的授权

```
sudo -l
```

重点读取：

- 哪些 Defaults 适用；
- 允许的 Runas 身份；
- 命令绝对路径；
- 是否显示 NOPASSWD；
- 参数是否与预期一致。

② [操作] 管理员检查指定用户的列表

```
sudo -l -U alice
```

`-U` 不是任何普通用户都能任意使用。默认情况下，应由 root 或具备相应授权的管理员执行。若要专门授权“列出他人权限”，`sudoers` 还存在内建 `list` 能力，但这不是 RHCSA 本章的默认配置任务。

③ [操作] 消除认证时间戳对测试的干扰

```
sudo -k
sudo -n /usr/bin/systemctl --no-pager status httpd
```

- `sudo -k` 使下一次需要密码的操作重新认证；
- `sudo -n` 禁止交互提示，若需要密码则直接失败。

因此 `sudo -k` 后的 `sudo -n` 很适合验证某个精确命令是否真正具备 `NOPASSWD`，但不能证明交互式密码输入流程本身正确。

④ [操作] 同时测试允许和拒绝路径

```
允许命令成功
+ 未授权参数被拒绝
+ 其他服务被拒绝
+ root Shell 被拒绝
```

建议使用目标用户的新登录会话，避免旧会话的组成员关系或环境状态干扰。组刷新机制归第 07 章，本章只要求在验证前确认调用者当前身份正确。

[Cheatsheet] `sudo -l` 可见授权；`sudo -k` 清认证缓存；`sudo -n` 测非交互；最终必须执行允许与拒绝调用。

[知识专题] `NOPASSWD`、认证时间戳与非交互执行

“这次没有提示密码”并不能直接证明规则使用了 `NOPASSWD`。`sudo` 通常会缓存近期成功认证的状态；测试若不先清除时间戳，很容易得到错误结论。

① [知识点] `NOPASSWD` 只改变认证要求

```
%webops ALL=(root) NOPASSWD: /usr/bin/systemctl --no-pager status httpd
```

它不允许 `restart`、不允许其他服务，也不自动允许 `Shell`。命令范围仍由路径和参数决定。

② [知识点] 默认验证的是调用者密码

普通配置下，`sudo` 要求的是当前调用者自己的密码，而不是 `root` 密码。这使管理员不必共享 `root` 凭据，同时仍能把特权调用关联到具体账号。

③ [知识点] 认证时间戳是临时状态

一次成功认证后，后续调用在一段时间内可能不再询问密码。实际期限由配置决定，不能把课件中的某个分钟数当作所有系统的永久默认值。调查时读取目标系统 Defaults，并用 `sudo -k` 建立干净测试条件。

④ [知识点] 非交互任务要显式区分“授权失败”和“需要密码”

自动化或无终端环境常使用：

```
sudo -n <command>
```

它不会等待密码输入。失败可能表示命令未授权，也可能表示命令已授权但需要认证。应结合错误信息、`sudo -l` 和规则标签继续判断，不能仅凭退出非零就扩大为“`sudoers` 没有加载”。

[Cheatsheet] 无密码提示可能来自时间戳；测试前 `sudo -k`；`NOPASSWD` 改认证，不改命令范围；`-n` 失败要区分未授权与需密码。

[知识专题] 环境清理与 `secure_path` 的必要范围

命令在普通 `Shell` 中成功、经 `sudo` 后失败，原因不一定是授权。`sudo` 会按策略构造执行环境，`PATH`、`HOME`、编辑器变量和应用变量可能与调用者原环境不同。本章只建立识别和最小调查能力，不穷举所有环境选项。

① [知识点] `env_reset` 降低不受控变量进入特权进程的机会

启用环境重置时，只保留允许的变量并为目标命令构造较受控的环境。这有助于阻止用户通过加载路径、解释器变量或工具配置影响 `root` 进程。

不要为了让一个程序“先跑起来”就默认使用 `sudo -E` 保留整个环境。应先找出目标程序真正需要的变量，并评估其值是否可信。

② [知识点] `secure_path` 可以替换 `sudo` 命令使用的 `PATH`

若配置了 `secure_path`，`sudo` 会使用该值替换调用者的 `PATH`。它有两个直接目的：

- 让管理命令目录在 `PATH` 中可用；

- 降低受限用户通过操纵 PATH，使特权脚本调用恶意同名程序的风险。

③ [知识点] sudoers 的绝对路径不能自动保护程序内部调用

规则固定 `/usr/local/sbin/backup-app`，但脚本内部若执行：

```
tar -czf ...
```

而不是 `/usr/bin/tar`，行为仍可能依赖 PATH。应在脚本中设置安全 PATH 或使用绝对路径，并确保脚本和父目录不可被调用者修改。

④ [知识点] 环境诊断要先比较证据再修改 Defaults

```
env | sort
sudo env | sort
sudo -V
sudo -l
```

`sudo -V` 展示的详细策略信息通常需要管理员权限才能完整查看。比较差异后，只对明确需要的变量做最小配置；不要全局放宽 `env_keep` 或授予 `SETENV` 来掩盖单个应用的配置问题。

[Cheatsheet] 经 `sudo` 后环境可能不同：`secure_path` 影响 PATH；绝对入口不保证脚本内部安全；先比较环境，再做最小修改。

[操作专题] 用 `sudoedit` 授权受控文件编辑

直接允许 `sudo vim` 往往比预期更强：编辑器可能打开其他文件、写入任意路径或启动 Shell。`sudoedit` 的模型是把目标文件复制到临时位置，让编辑器以调用者身份运行，编辑完成后再由 `sudo` 受控写回。

① [操作] 在 sudoers 中把 `sudoedit` 写成内建命令

```
%webops ALL=(root) sudoedit /etc/httpd/conf.d/site.conf
```

`sudoedit` 是 `sudo` 内建能力，在 `sudoers` 中不应写前导路径。调用者执行：

```
sudoedit /etc/httpd/conf.d/site.conf
# 或
sudo -e /etc/httpd/conf.d/site.conf
```

② [操作] 只列出确实需要编辑的路径

```
%webops ALL=(root) sudoedit /etc/httpd/conf.d/site.conf, \
    sudoedit /etc/httpd/conf.d/tls.conf
```

避免直接允许整个 `/etc` 或使用宽泛通配符。若文件集合经常变化，先确认是否应改为受控部署流程，而不是持续扩大人工编辑范围。

③ [操作] 检查目标文件父目录是否由调用者可写

```
namei -l /etc/httpd/conf.d/site.conf
stat /etc/httpd/conf.d/site.conf
getfacl /etc/httpd/conf.d/site.conf
```

不得把 `sudoedit` 授权给位于调用者可写目录中的文件。否则调用者可能通过替换文件或链接改变真正被编辑的对象。现代 `sudo` 有额外防护，但安全设计仍应先确保路径控制权正确。

④ [操作] 验证编辑能力而不扩大为通用 root 编辑器

验收应证明：

- 指定文件能编辑；
- 未授权文件被拒绝；
- `sudo vim /etc/shadow` 被拒绝；
- 编辑器进程以调用者身份运行；
- 写回后的目标文件仍保持预期所有者和权限。

[Cheatsheet] `sudoers` 中写 `sudoedit`，不写 `/usr/bin/sudoedit`；按文件精确授权；父目录不可由调用者写；拒绝通用 `root` 编辑器。

[诊断专题] `sudo -l` 看得到规则，实际命令却不匹配

这个症状说明“至少有一条规则与用户相关”，但不等于当前请求的每个维度都匹配。诊断时不要立即追加 `ALL`，而应把列表中的命令规格和真实调用逐字段比较。

① [诊断] 先记录列表和真实调用

```
sudo -l
printf '%q ' /usr/bin/systemctl status httpd.service; echo
```

保留：调用者、Runas、可执行路径、每个参数及顺序。不要只记录自然语言“运行了 systemctl”。

② [诊断] 核对 Runas 身份

```
sudo /usr/bin/id
sudo -u apache /usr/bin/id
```

若规则只允许 `(apache)`，默认 `root` 请求会失败；若规则只允许 `root`，以 `-u apache` 调用同样不会命中。

③ [诊断] 核对路径、参数和值

列表允许：

```
(root) /usr/bin/systemctl --no-pager status httpd
```

实际调用：

```
sudo /usr/bin/systemctl status httpd.service
```

差异包括缺少 `--no-pager`、`unit` 参数不同，可能还有参数顺序不同。下一条最有区分度的证据不是重新安装 `sudo`，而是按列表中的精确形式调用。

④ [诊断] 检查后续规则和标签

```
visudo -c
sudo -l
```

同时阅读 `/etc/sudoers` 和 `/etc/sudoers.d` 中与主体相关的所有规则。若命令获准但密码行为与预期不同，重点查看后续 `PASSWD` / `NOPASSWD` 匹配，而不是只看第一条规则。

⑤ [诊断] 最小修复后同时复测允许和拒绝路径

修复选择只有两类：

- 调用者改用已授权的精确形式；
- 业务确实需要另一种形式时，增加另一条精确命令规格。

不要把规则改成 `/usr/bin/systemctl *`。修复后重跑原允许命令、原失败变式以及其他服务的拒绝测试。

[Cheatsheet] 列表可见不等于请求匹配；逐项比 Runas、路径、参数和顺序；修复精确差异，不扩大为通配符。

[诊断专题] 片段存在，但没有加载或行为与预期不同

这类故障通常落在三层：文件未进入 `include`、完整策略存在冲突、或认证与环境被误判。按最有区分度的证据推进，避免反复重写同一行规则。

① [诊断] 检查文件名、路径和元数据

```
ls -ld /etc/sudoers.d
ls -l /etc/sudoers.d
stat /etc/sudoers.d/70-webops-httpd
```

重点排除：

- 文件名包含 `.`；
- 文件名以 `~` 结尾；
- 写到了未被 `include` 的目录；

- 所有者或模式异常；
- 文件实际是编辑器临时文件。

② [诊断] 片段通过后继续检查全局

```
visudo -cf /etc/sudoers.d/70-webops-httpd
visudo -c
```

片段语法错误会直接阻断；片段通过而全局失败，则继续处理别名、其他片段或主文件中的问题。

③ [诊断] 区分免密规则与认证时间戳

```
sudo -k
sudo -n <exact-command>
```

- 精确命令成功：可支持 `NOPASSWD` 判断；
- 提示需要密码：命令可能获准但要求认证；
- 显示不允许执行：继续查匹配维度。

④ [诊断] 区分授权失败与环境导致的程序失败

若 `sudo` 已经启动程序，但程序报告“command not found”、找不到配置或缺少变量，应比较环境和 `secure_path`。不要因为目标程序失败就把 `sudoers` 改为 `ALL`。

[Cheatsheet] 先文件名和元数据，再片段和全局语法，再认证状态，最后区分环境与目标程序故障。

[诊断专题] sudo 已允许，但目标操作仍然失败

看到密码验证成功或 `sudo` 没有拒绝，不代表业务任务完成。目标程序可能因自身配置、文件权限、SELinux、服务依赖或输入错误而退出。本章只负责把故障边界分清，再转入自然归属章节。

① [诊断] 先确认策略层已经通过

证据包括：

- `sudo -l` 中存在匹配命令；
- 调用未出现 `sudoers` 拒绝信息；
- 目标程序已经输出自己的错误；
- 日志中的执行命令和 `Runas` 身份符合预期。

② [诊断] 读取目标程序自己的退出状态和错误

```
sudo <exact-command>
printf 'rc=%s\n' "$?"
```

不要只截取“sudo 成功认证”部分。参数错误、对象不存在和服务失败都由目标程序返回。

③ [诊断] 按对象归属转入相邻章节

- `systemctl` 已执行但 `unit` 失败：第 12 章；
- 目标文件仍不可写：第 08、09 章；
- `AVC` 拒绝：第 28、29 章；
- 日志查询和持久性：第 13 章。

本章保留接口，不复制相邻章节的完整诊断。

④ [诊断] 不用扩大授权掩盖目标程序问题

目标服务启动失败时，把规则从精确命令改为 `ALL` 不会修复服务配置，只会增加新的风险。最小修复必须作用于真正失败的对象，之后再从 `sudo` 调用入口完整复测。

[Cheatsheet] `sudo` 通过只证明策略允许；读取目标程序退出状态；故障转入自然章节；不要用更宽授权替代修复。

[经典任务] 为 Web 运维组建立可验证的最小服务授权

环境与当前状态

主机 `servera` 上已经存在 Unix 组 `webops`，用户 `alice` 是该组成员。`httpd` 已安装，管理员已确认以下命令在 `root` 身份下本身有效：

```
/usr/bin/systemctl --no-pager status httpd
/usr/bin/systemctl restart httpd
```

系统中当前存在一条过宽规则：

```
%webops ALL=(root) NOPASSWD: ALL
```

目标终态

1. 删除或替换上述过宽规则。
2. `webops` 成员只允许：
 - 免密查看 `httpd` 状态；
 - 输入调用者自己的密码后重启 `httpd`。
3. 不允许停止、禁用或 `mask httpd`。
4. 不允许操作其他 `unit`。
5. 不允许 `sudo -i`、`sudo -s`、Shell、解释器或通用 `root` 编辑器。
6. 新规则存入 `/etc/sudoers.d/70-webops-httpd`。
7. 不修改无关 `sudoers` 内容。

限制条件

- 必须使用 `visudo` 维护规则；
- 不得加入 `wheel`；
- 不得使用命令 `ALL` 或参数通配符；
- 本任务没有 RHEL 9 live VM，下面命令是推荐验证流程，不声称已实测。

验收证据

```
片段语法
→ 完整策略树
→ alice 可见规则
→ status 免密允许
→ restart 非交互失败、交互认证后允许
→ 其他动作与 root Shell 被拒绝
→ httpd 结果由 systemd 证据另行确认
```

[参考解答] 为 Web 运维组建立可验证的最小服务授权

① 调查现有规则和调用者状态

```
id alice
grep -Rns -- '%webops' /etc/sudoers /etc/sudoers.d 2>/dev/null
sudo -l -U alice
```

确认 `alice` 当前确实属于 `webops`，并定位过宽规则的真实文件。不要在未知来源上再追加一条更具体规则后就假设它会覆盖 `ALL`：读取顺序和最后匹配会影响结果。

② 使用 `visudo` 创建目标片段

```
visudo -f /etc/sudoers.d/70-webops-httpd
```

写入：

```
# webops: read httpd state without a password; restart with caller authentication.
%webops ALL=(root) NOPASSWD: /usr/bin/systemctl --no-pager status httpd, \
    PASSWD: /usr/bin/systemctl restart httpd
```

随后删除或收缩旧的 `%webops ... NOPASSWD: ALL`。不能保留宽泛规则并指望精确规则自动形成上限。

③ 核对文件控制权并校验配置

```
chown root:root /etc/sudoers.d/70-webops-httpd
chmod 0440 /etc/sudoers.d/70-webops-httpd
stat -c '%U:%G %a %n' /etc/sudoers.d/70-webops-httpd
visudo -cf /etc/sudoers.d/70-webops-httpd
visudo -c
```

`visudo -cf` 证明片段可解析；`visudo -c` 证明当前完整策略树通过语法与结构检查。两层都需要。

④ 以 alice 的新登录会话查看授权

```
su - alice
id
sudo -l
```

预期列表只出现两个精确 `systemctl` 调用，不出现 `(root) ALL`、Shell 或其他服务。

⑤ 验证免密状态命令

```
sudo -k
sudo -n /usr/bin/systemctl --no-pager status httpd
printf 'rc=%s\n' "$?"
```

`sudo -k` 清除认证时间戳；`-n` 禁止提示密码。若精确 `status` 命令成功，才能支持“该调用具备 NOPASSWD”的判断。

⑥ 验证重启仍要求认证

先进行非交互测试：

```
sudo -k
sudo -n /usr/bin/systemctl restart httpd
printf 'rc=%s\n' "$?"
```

预期因需要密码而失败，而不是直接执行。随后交互执行：

```
sudo /usr/bin/systemctl restart httpd
```

输入的是 `alice` 自己的密码。`sudo` 调用成功后，再用 `systemd` 层的证据确认服务状态；这部分对象模型归第 12 章。

⑦ 验证拒绝矩阵

```
sudo /usr/bin/systemctl stop httpd
sudo /usr/bin/systemctl disable httpd
sudo /usr/bin/systemctl restart sshd
sudo /usr/bin/systemctl --no-pager status sshd
sudo /usr/bin/systemctl status httpd.service
sudo -i
sudo -s
sudo /bin/bash
```

所有调用都应被拒绝。特别注意：`status httpd.service` 即使对 `systemctl` 来说可能指向同一 unit，也不是规则中授权的精确参数字符串。

⑧ 典型错误

- 保留旧 `%webops ... ALL`，导致拒绝测试仍可通过其他规则执行；
- 只执行 `visudo -cf`，未执行完整 `visudo -c`；
- `status` 测试前未执行 `sudo -k`，把认证时间戳误判为 NOPASSWD；
- 只验证两个允许命令，没有测试其他服务和 root Shell；
- 把 `sudo` 成功扩大为 `httpd` 功能健康。

[经典任务] 诊断可见规则与实际命令不匹配

已知证据

用户 `alice` 执行 `sudo -l` 时看到：

```
(root) /usr/bin/systemctl --no-pager status httpd
```

但她执行：

```
sudo /usr/bin/systemctl status httpd.service
```

系统拒绝该调用。管理员准备把规则改为：

```
alice ALL=(root) /usr/bin/systemctl *
```

任务要求

1. 解释为什么列表可见但真实调用仍不匹配；
2. 给出下一条最有区分度的验证命令；
3. 提供不扩大为通配符的最小修复方案；
4. 设计允许与拒绝测试。

[参考解答] 诊断可见规则与实际命令不匹配

① 比较规则与请求

规则要求的参数字符串是：

```
--no-pager status httpd
```

实际请求是：

```
status httpd.service
```

至少存在两处差异：缺少 `--no-pager`，最后一个参数由 `httpd` 变为 `httpd.service`。`sudo -l` 证明 `alice` 拥有一条相关授权，不证明任意 `systemctl status` 形式都会命中。

② 执行最有区分度的精确调用

```
sudo /usr/bin/systemctl --no-pager status httpd
```

若这条成功，说明规则加载、主体、主机、Runas 和路径大体成立，故障集中在参数匹配。若仍失败，再检查 `sudo -l` 中 Runas、后续规则和完整策略。

③ 选择最小修复

若业务没有要求 `httpd.service` 形式，直接让调用者使用已经授权的精确命令，不修改规则。

若业务明确要求两种形式，可增加第二条精确命令：

```
alice ALL=(root) /usr/bin/systemctl --no-pager status httpd, \
    /usr/bin/systemctl status httpd.service
```

不应使用：

```
alice ALL=(root) /usr/bin/systemctl *
```

因为它可能允许 `stop`、`disable`、`edit`、`mask` 以及其他 `unit`。

④ 重新验证

允许路径：

```
sudo /usr/bin/systemctl --no-pager status httpd
sudo /usr/bin/systemctl status httpd.service # 仅在第二种形式确有业务需要并已精确加入时
```

拒绝路径：

```
sudo /usr/bin/systemctl stop httpd
sudo /usr/bin/systemctl restart sshd
sudo /usr/bin/systemctl edit httpd
sudo -i
```

最后执行 `visudo -c`，并检查 `sudo -l` 不出现意外宽泛规则。

[本章收束] 用证据证明最小特权，而不是只写出一行规则

本章的工作方法可以压缩为一条可重复执行的链路：

```
把业务动词改写为允许动作和拒绝动作
→ 确认调用者、主机、Runas、可执行路径与参数
→ 用 visudo 维护独立片段
→ 校验片段和完整策略树
→ 用 sudo -l 读取可见授权
→ 清除认证时间戳后测试认证边界
→ 同时执行允许矩阵与拒绝矩阵
→ 把目标程序故障转入自然归属章节
```

规则设计阶段关注的是“能不能命中”；验收阶段还要关注“有没有意外命中”。因此，最小特权的证据必须同时包含成功路径和失败边界。只要仍能进入 root Shell、操作其他对象、改变未授权参数，或者通过用户可写脚本改变实际行为，就不能把规则称为最小授权。

主要判断表

看到的现象	当前最多能证明什么	下一条有区分度的证据
<code>visudo -cf</code> 通过	当前片段可被解析	<code>visudo -c</code> 检查完整策略树
<code>visudo -c</code> 通过	当前完整策略语法和引用关系可接受	以目标调用者执行 <code>sudo -l</code>
<code>sudo -l</code> 出现命令	存在相关可见授权	按列表中的 Runas、路径和参数精确调用
调用未被 sudoers 拒绝	策略层允许启动目标程序	读取目标程序错误、退出状态和功能证据
本次没有提示密码	可能是 NOPASSWD，也可能有认证时间戳	<code>sudo -k</code> 后执行 <code>sudo -n <精确命令></code>
精确允许命令成功	成功路径成立	继续测试其他参数、其他对象和 root Shell 被拒绝
经 <code>sudo</code> 后找不到子命令	可能是环境或 <code>secure_path</code> 差异	比较 <code>env</code> 、 <code>sudo env</code> 和脚本内部路径
<code>systemctl</code> 已运行但服务失败	sudo 授权链已推进到目标程序	转入第 12 章检查 unit、日志和服务功能

章末 Cheatsheet

```
# 安全编辑片段
visudo -f /etc/sudoers.d/70-webops-httpd

# 片段与完整策略校验
visudo -cf /etc/sudoers.d/70-webops-httpd
visudo -c

# 查看授权
sudo -l
sudo -l -U alice

# 清除认证时间戳并做非交互测试
sudo -k
sudo -n <exact-command>

# 指定 Runas 身份
sudo -u postgres <command>
sudo -g adm <command>

# 受控编辑
sudoedit /etc/httpd/conf.d/site.conf
```

安全边界：避免命令 `ALL`、参数宽通配符、通用 Shell、解释器、可逃逸编辑器，以及由调用者可修改的脚本或父目录。环境问题先比较普通环境与 `sudo` 环境，不要默认全局保留变量。

向下一章交接

本章把“是否允许启动某个特权命令”解释清楚。下一章《进程、作业、信号与调度优先级》接手命令启动后的运行实例：如何识别 PID、判断进程状态、控制前台作业、发送信号并调整调度优先级。不要把 `sudo` 授权成功误当成进程生命周期已经得到正确管理。

进程、作业、信号与调度优先级

从“看到一个 PID”到可审计的运行控制：先确认实例，再观察状态，最后实施最小干预。

对象模型操作语义验证诊断经典任务

大号阅读版

本章阅读导航 先抓住一条主线：**进程控制不是先找一条“能杀掉它”的命令，而是先证明目标进程是谁、当前处于什么状态、由谁管理，再选择最小影响的操作并用同一组证据复核。**

01 确认运行实例PID、用户、启动时间、完整命令行

02 还原关系PPID、线程、PGID、SID、TTY

03 判断状态R、S、D、T、Z 与资源维度

04 持续采样ps 快照、top 趋势、/proc 下钻

05 实施最小干预作业控制、TERM、nice / renice

06 分层验收原 PID、原业务特征、非目标实例、管理者

专题地图 | 类型 | 专题 | |---| | 知识专题 | 从程序文件到运行实例：进程、线程与身份关系 | | 知识专题 | 进程状态、生命周期与资源维度 | | 操作专题 | 用 `ps`、`pidof` 与 `pstree` 建立可信快照 | | 操作专题 | 用 `pgrep` 精确缩小集合，再决定是否操作 | | 操作专题 | 用 `top` 和 `/proc/` 建立持续证据 | | 操作专题 | Bash 前台、后台与作业生命周期 | | 操作专题 | 信号控制：先请求协作，再决定是否强制 | | 操作专题 | `nice` 与 `renice` 调整普通调度倾向 | | 诊断专题 | 从状态和负载症状推进到下一条证据 | | 经典任务 | 安全停止旧 worker；调查高负载无单一热点 |

阅读时持续回答 1. 当前 PID 仍然属于刚才确认的业务实例吗？ 2. 这个对象是单个进程、线程、进程组，还是 Shell 作业？ 3. `STAT` 是瞬时状态，还是持续性异常证据？ 4. 负载来自可运行竞争，还是不可中断等待？ 5. 当前命令会影响一个 PID，还是一个匹配集合？ 6. 命令成功能证明请求已发送，还是终态已达到？ 7. 原 PID 消失后，原业务特征是否出现了新 PID？ 8. 当前对象是否受 systemd 或其他监督器管理？

阅读边界：本章建立进程实例和 Shell 作业的调查与控制模型；systemd unit 生命周期留给第 12 章，日志证据留给第 13 章，cgroup 深度治理不在 RHCSA 主线展开。

第 11 章 · 正文

服务器上出现“程序正在运行”“负载很高”“进程杀不掉”“SSH 断开后任务也没了”时，真正需要处理的不是某一条孤立命令，而是一组容易混淆的运行对象：程序文件、进程实例、线程、父子关系、进程组、会话、控制终端和 Shell 作业。把命令名当作唯一身份、把 load average 当作 CPU 百分比、把 `kill` 返回成功当作进程已经退出，都会让一次看似简单的处理变成误伤或反复试错。

本章以“身份组合 → 状态与资源 → 查询证据 → 最小操作 → 再验证 → 管理层交接”为主线。先用 PID、PPID、用户、启动时间和完整命令行建立可复核的实例身份，再区分 R/S/D/T/Z、RSS、VSZ、I/O 和负载；随后训练 `ps`、`pgrep`、`top`、`/proc`、Shell 作业控制、信号以及 `nice` 值。若进程被外部管理者重新拉起，本章只负责识别这一边界，完整 `systemd` 控制留到下一章。

遇到任何“停掉它”或“调低它”的请求，先写出四个问题：- 我要操作的是哪个实例，身份由哪些字段共同证明？- 作用范围是单个 PID、进程组、作业，还是匹配集合？- 当前证据支持终止、暂停、继续，还是只支持继续调查？- 操作后怎样证明目标达到终态、非目标未受影响、管理者没有重新创建它？

概念

进程实例（process instance）是程序一次实际运行形成的内核对象，拥有 PID、凭据、地址空间、打开文件和当前状态。同一个可执行文件可以同时形成多个实例，因此程序路径或短命令名只能说明“它可能是什么”，不能单独证明“它就是本次要操作的对象”。

概念

线程（thread）是进程内部参与调度的执行单元；同一进程的线程通常共享地址空间和多数资源。进程级视图适合定位业务实例，线程级视图适合继续定位同一实例内部的 CPU 热点；线程转储和应用栈分析则超出本章边界。

概念

PID 与 PPID 分别标识当前进程实例和它的直接父进程。PID 只在进程存活期间唯一，退出后可能被复用；因此高风险操作前必须重新核对用户、启动时间、完整命令行和 PPID，不能把几分钟前记录的裸 PID 当作长期业务身份。

概念

进程组（process group）用 PGID 把若干进程组织成一个控制对象。一条 Shell 管道通常包含多个 PID，却位于同一进程组；终端的前台属性和由键盘触发的信号通常面向前台进程组，而不是只面向管道中的某个成员。

概念

会话与控制终端（session / TTY）连接了登录环境、进程组和终端控制。SID 标识会话，TTY 表示控制终端；`TTY=?` 常见于守护进程、`systemd` 服务或已经脱离终端的任务，它只说明没有控制终端，并不自动表示异常。

概念

进程状态（process state）是采样瞬间任务处于运行、可中断等待、不可中断等待、停止或已退出待回收等状态的压缩表示。状态字符不是健康评分：`S` 往往正常，`D` 需要调查等待依赖，`Z` 表示父进程尚未回收，均不能用一条更强的 `kill` 统一处理。

概念

Shell 作业（job）是当前交互 Shell 对一条命令或管道的管理记录。`%1` 这样的 jobspec 只在创建它的 Shell 中有效；它不是 PID，也不是跨会话的持久身份。后台运行、脱离终端和长期服务是三个不同问题。

概念

信号（signal）是内核向进程或进程组传递的异步通知。`SIGTERM` 是可协作处理的终止请求，`SIGKILL` 是无清理机会的强制终止；`SIGSTOP` / `SIGCONT` 控制停止与继续。发送成功只证明请求被内核接受，不证明业务终态已经达到。

概念

`nice` 值（niceness）表示普通 CPU 调度中的相对礼让程度，常见范围为 `20` 到 `19`：数值越低越有利，越高越礼让。它不是 CPU 配额，也不能修复 I/O 等待、锁竞争、内存压力或错误的管理策略。

操作语义以下入口分别负责“建立身份、缩小集合、保存趋势、读取上下文、控制作业、发送信号和调整调度倾向”。先理解作用对象，再记关键形式。

`ps` / `pidof` / `pstree`

SYNOPSIS

```
ps [selection] -o FIELDS [--sort=KEY]
pidof [options] PROGRAM
pstree [options] [PID|USER]
```

读取进程快照、快速取得程序 PID，或观察父子结构。`ps` 的核心是把“选谁”和“显示什么”分开。

``-e` / `-p PID``

选择全部进程，或只选择明确 PID。

``--ppid PID``

选择直接子进程，不递归整个后代树。

``-o pid,ppid,user,lstart,stat,args``

建立可复核的身份、关系和状态视图。

``--sort=-%cpu` / `--sort=-rss``

按 CPU 或 RSS 降序排列；负号表示降序。

`pgrep` / `pkill`

SYNOPSIS

```
pgrep [options] PATTERN
pkill [options] PATTERN
```

按名称、用户、父进程、会话、终端或完整命令行缩小集合；`pkill` 对同一集合发送信号。

``-a``

显示 PID 和完整命令行，适合在操作前人工审阅。

``-f` / `-x``

匹配完整命令行，或要求短命令名精确匹配。

``-u USER` / `-P PPID``

按有效用户或直接父 PID 收缩范围。

同条件预览

任何 `pkill` 前先运行对应 `pgrep -a`，之后再用原条件验收。

`top`

SYNOPSIS

```
top [-b] [-d SECONDS] [-n ITERATIONS] [-H] [-p PID]
```

周期采样系统和任务状态；批处理模式可把多个采样窗口保存为证据。

``-b``

批处理输出，适合重定向到文件。

``-d 2 -n 5``

每 2 秒采样一次，共 5 轮；先固定窗口再比较。

``-H``

展开线程视图，继续定位进程内部热点。

``-p PID``

只观察指定进程；与 `-H` 组合可观察该进程的线程。

`/proc/<PID>`

SYNOPSIS

```
cat /proc/PID/status
tr '\0' ' ' < /proc/PID/cmdline
readlink /proc/PID/exe
ls -l /proc/PID/fd
```

读取活进程的身份、状态、命令行、路径、文件描述符、I/O 和等待点。

``status` / `stat``

查看人类可读状态，或紧凑的机器字段。

``cmdline` / `exe` / `cwd``

确认参数、真实可执行文件和工作目录。

``fd` / `io` / `wchan``

观察打开资源、I/O 计数和当前等待位置。

活对象竞态

进程可在读取途中退出；“文件不存在”不等于它从未存在。

``jobs` / `bg` / `fg``

SYNOPSIS

```
jobs [-l]
bg [JOBSPEC]
fg [JOBSPEC]
```

管理当前交互 Shell 的作业表，把停止的作业放到后台继续，或拉回前台。

``jobs -l``

同时显示作业号和 PID，连接 Shell 与内核视图。

``bg %N``

向指定作业发送继续信号并在后台运行。

``fg %N``

把指定作业放回终端前台并等待。

``%N``

jobspec 只属于当前 Shell，不可带到另一个登录会话使用。

``nohup` / `disown``

SYNOPSIS

```
nohup COMMAND [ARG]... &
disown [-h] [JOBSPEC]
```

分别处理挂断信号和 Shell 作业表关系；二者都不等于把命令配置成长期系统服务。

``nohup COMMAND &``

让命令忽略常见挂断影响，并明确放入后台。

``disown %N``

从当前 Shell 作业表移除作业。

``disown -h %N``

保留作业记录，但标记为不随 Shell 发送 HUP。

边界

重启持久性、依赖、自动恢复和统一日志应交给 systemd。

``kill``

SYNOPSIS

```
kill [-SIGNAL] PID|JOBSPEC ...
kill -0 PID
kill -SIGNAL -- -PGID
```

向明确 PID、jobspec 或进程组发送信号；默认信号通常是 ``SIGTERM``。

``-TERM` / `-KILL``

先请求有序退出；重新确认后才考虑无清理机会的强制终止。

``-STOP` / `-CONT``

无条件停止，或继续已停止的任务。

``-0``

检查 PID 存在和发送权限，不验证业务身份或健康。

``-- -PGID``

负目标表示进程组，可能同时影响整条管道或一个作业。

``nice` / `renice``

SYNOPSIS

```
nice -n ADJUSTMENT COMMAND [ARG]...
renice --priority VALUE --pid PID
renice --priority VALUE --pgrp PGID
renice --priority VALUE --user USER
```


在启动时或运行中调整普通 CPU 调度倾向；只有证据支持 CPU 竞争时才是合适的最小干预。

```
`nice -n N`
    在继承值基础上增加调整量，常见默认调整量为 10。
`--priority VALUE`
    为运行中对象设置绝对 nice 值，避免 `n` 兼容语义歧义。
`--pid` / `--pgrp` / `--user`
    明确后续标识符是 PID、进程组还是用户集合。

权限
    普通用户通常只能让自己的任务更礼让；提高调度有利程度需要适当特权。
```

[知识专题] 从程序文件到运行实例：进程、线程与身份关系

进程调查最容易犯的错误，是把命令名当成唯一身份。真实系统里，同名 worker、预派生服务、解释器进程和多线程应用都很常见。可靠的切入点不是“找到一个看起来像的 PID”，而是建立可复核的身份组合，并理解这个实例与父进程、线程、进程组和会话的关系。

① [知识点] 程序、进程与线程是三个层次

程序文件是可执行内容；进程是该内容的一次运行实例，拥有独立的地址空间、凭据、打开文件、环境和状态；线程是进程中的调度执行单元，同一进程的线程通常共享地址空间和多数资源。

进程级视图适合回答“哪个业务实例占资源、由谁启动、是否仍存在”；线程级视图适合继续回答“同一个进程内部，哪个执行单元消耗 CPU”。`ps -eL`、`ps -T -p PID` 或 `top -H -p PID` 可以展开线程，但应用栈、Java 线程转储或数据库内部执行计划不属于本章。

② [知识点] PID 是当前实例标识，不是长期业务身份

每个进程都有 PID 和 PPID。PID 在进程存活期间唯一，但退出后可被复用。单独把 PID 写入笔记、脚本或延迟执行的命令，都可能形成检查—操作竞争窗口。

对高风险操作，至少组合以下字段：

字段	回答的问题
PID	当前实例是谁
PPID	直接由谁创建
USER / UID	以谁的身份运行
LSTART / ETIMES	何时启动、运行多久
COMM	内核记录的短命令名
ARGS / CMD	包含参数的完整命令行
SID / PGID / TTY	属于哪个会话、进程组和终端

因此，终止前的可靠核对不是 `ps -p 2450` 一眼扫过，而是明确读取需要证明身份的字段：

```
ps -p 2450 -o pid,ppid,user,lstart,etimes,sid,pgid,TTY,stat,args
```

③ [知识点] 父子关系解释创建、退出与回收

进程通常由父进程创建。子进程退出后，父进程应读取其退出状态并回收剩余的进程表项。若子进程已经退出而父进程尚未回收，就会暂时显示为僵尸。

```
父进程创建子进程
→ 子进程运行
→ 子进程退出并保留退出状态
→ 父进程 wait 回收
→ 进程表项消失
```

如果父进程先退出，仍存活的子进程会被 PID 1 或某个子收割器接管。所谓“孤儿”只描述父进程关系改变，不等于进程失控，也不等于必须终止。

④ [知识点] 一条管道可包含多个进程，但只有一个 Shell 作业

Bash 执行一条管道时，通常会启动多个进程，并把它们放入同一进程组。Shell 把整个管道登记为一个作业。终端维护一个前台进程组；用户按下 `Ctrl+C` 或 `Ctrl+Z` 时，信号通常发送给整个前台进程组，而不是只发送给管道最左侧的一个 PID。

这解释了三个常见现象：

- `jobs` 只显示一个作业，但 `ps` 可看到多个 PID；
- 前台管道中的多个命令会同时收到终端信号；
- 对单个 PID 发送信号，可能只影响管道中的一个成员。

⑤ [知识点] SID、PGID 与 TTY 连接了会话和终端控制

- **SID** 是会话 ID；登录 Shell 通常是会话首进程。
- **PGID** 是进程组 ID；一条管道的成员通常位于同一进程组。
- **TTY** 是控制终端；`?` 常表示进程没有控制终端。
- 终端的“前台”属性属于进程组，不属于某个孤立 PID。

守护进程、由 `systemd` 启动的服务和某些脱离终端的后台任务常显示 `TTY=?`。这本身不是异常，只说明它们不受当前终端的前后台控制。

[Cheatsheet] 终止或调优前同时核对 PID、用户、启动时间、完整命令行和 PPID；一条管道可有多个 PID，但 Bash 可把它视为一个 Job；终端信号通常面向前台进程组。

[知识专题] 进程状态、生命周期与资源维度

状态字符不是健康评分，而是采样瞬间“任务当前处于哪一类运行或等待状态”。资源字段也不能相互替代：CPU、物理内存、虚拟地址空间、I/O 和负载平均值分别回答不同问题。先把这些维度拆开，才能避免“看到一个大数字就直接杀进程”。

① [知识点] R、S、D、T、Z 的核心含义

状态	含义	第一判断
R	正在 CPU 上运行或位于可运行队列	持续大量出现才支持 CPU 竞争假设
S	可中断睡眠，等待事件	服务大部分时间处于 S 往往正常
D	不可中断睡眠，常见于内核 I/O 等待	先查等待点、存储、NFS、驱动和内核证据
T	被作业控制信号停止，或处于跟踪状态	查终端、作业、调试器与信号来源
Z	子进程已退出，父进程尚未回收状态	查 PPID 和父进程，不要继续“杀僵尸”

STAT 后还可能出现修饰符：`s` 表示会话首进程，`l` 表示多线程，`+` 表示位于前台进程组，`<` 表示较高调度优先级，`N` 表示较低调度优先级。

② [知识点] 僵尸已经退出，剩下的是父进程的回收责任

僵尸已释放普通地址空间和大部分资源，只保留 PID、退出状态等少量进程表信息。对僵尸发送 `SIGKILL` 没有意义，因为可执行实体已经退出。

正确调查链是：

```
找到 Z 状态 PID
→ 读取 PPID
→ 检查父进程是否仍存在、是否健康
→ 判断父进程是否持续产生未回收子进程
→ 再决定修复父进程或其管理单元
```

单个短暂僵尸可能在下一次采样中消失；持续累积才说明父进程没有正常回收。

③ [知识点] D 状态不是“信号更强就会立即消失”

`D` 表示任务处于不可中断等待。信号可被标记为待处理，但通常要等当前内核等待返回后才产生效果。因此，不断升级到 `SIGKILL` 不能替代 I/O 调查。

可优先观察：

```
ps -eo pid,ppid,stat,wchan:28,etimes,args | awk '$3 ~ /^D/'
cat /proc/<PID>/wchan
cat /proc/<PID>/io
findmnt
```

`wchan` 只是当前等待位置的线索，不应单独当作根因结论。系统日志属于第 13 章；本章只说明它是进一步核对内核、块设备或网络文件系统异常的入口。

④ [知识点] RSS 与 VSZ 不能直接互换

- **RSS** 是当前驻留在物理内存中的页面量。
- **VSZ** 是进程虚拟地址空间总量，可能包含共享库、映射文件、尚未实际驻留的区域和保留地址空间。
- `%MEM` 通常以 RSS 相对物理内存的比例计算。

因此，“VSZ 很大”不能直接推出“实际占用了同等 RAM”。内存异常需要结合 RSS 趋势、共享映射、系统可用内存和交换活动继续判断。

⑤ [知识点] load average 不是 CPU 使用百分比

`uptime` 和 `top` 通常显示最近 1、5、15 分钟的负载平均值。Linux 负载同时考虑处于可运行状态的任务和某些不可中断等待任务。因此负载高可能来自：

- 多个 **R** 状态任务竞争 CPU；
- 多个 **D** 状态任务等待本地或远程 I/O；
- 短时间并发峰值；
- 线程或短命子进程数量异常。

没有单个高 CPU 进程，不代表系统不存在 CPU 竞争；同样，CPU idle 仍有余量，也不代表高负载一定是“误报”。必须结合任务状态和多轮采样区分假设。

⑥ [知识点] 单次快照只是一帧，趋势才支持持续性判断

一次 **ps** 输出只能证明采样瞬间。短命进程、高频线程和波动型 CPU 热点可能在下一次采样中完全不同。需要保存证据时，应固定采样间隔和轮数，而不是不断手工刷新后凭印象判断。

```
top -b -d 2 -n 5 -w 200 > /tmp/top-sample.txt
```

[Cheatsheet] **R** 看可运行竞争，**D** 看内核等待，**T** 看暂停来源，**Z** 查父进程；RSS 是驻留物理内存，VSZ 是虚拟地址空间；load average 不是 CPU 百分比。

[操作专题] 用 **ps**、**pidof** 与 **pstree** 建立可信快照

ps 的价值不在于背诵某一套默认输出，而在于把“选择哪些进程”和“显示哪些字段”分开。调查前应先明确问题：是找同名实例、查看父子树、确认启动时间，还是按 CPU、内存和状态排序。

① [操作] 使用 **-e**、**-p**、**--ppid**、**-u** 与 **-C** 选择对象

作用对象：内核当前维护的进程快照。

基本形式：**ps** [选择选项] **-o** <FIELDS> [**--sort**=<KEY>]

关键边界：**-C** 按短命名选择，不等于匹配完整命令行；**--ppid** 只选直接子进程。

```
ps -e -o pid,ppid,user,stat,ni,%cpu,%mem,rss,etimes,args
ps -p 2450 -o pid,ppid,user,lstart,etimes,sid,pgid,tty,stat,args
ps --ppid 2450 -o pid,ppid,stat,etimes,args
ps -u appsvc -o pid,ppid,stat,lstart,etimes,args
ps -C report-worker -o pid,ppid,user,stat,etimes,args
```

② [操作] 用 **-o** 和 **--sort** 让输出服务于当前问题

CPU 调查、内存调查和身份核对需要不同字段：

```
# CPU 与运行状态
ps -eo pid,ppid,user,stat,ni,psr,%cpu,etimes,args --sort=%cpu

# 内存
ps -eo pid,ppid,user,%mem,rss,vsz,stat,etimes,args --sort=-rss

# 会话和终端
ps -eo pid,ppid,sid,pgid,tpgid,tty,stat,args --sort=sid,pgid,pid
```

字段太多会降低可读性。现场可使用 **ps L** 或 **ps --help output** 查看可用字段，再按问题组合。

③ [操作] 用 **ps --forest** 与 **pstree** 观察父子结构

```
ps -e --forest -o pid,ppid,user,stat,etimes,args
pstree -ap
pstree -aps 2450
```

pstree -a 显示参数，**-p** 显示 PID，**-s** 显示目标进程的祖先路径。相同子树可能被压缩显示，因此需要逐个 PID 的精确字段时仍回到 **ps**。

父子树可以揭示：

- worker 是否由同一个主进程派生；
- 目标是否由 Shell、计划任务或服务管理器启动；
- 杀掉一个子进程后主进程是否可能重建；
- 某个高负载进程是否只是更大进程树的一部分。

④ [操作] `pidof` 适合快速入口，不承担完整身份确认

```
pidof chronyd
pidof -s chronyd
```

`pidof` 适合查询已知程序名对应的 PID 列表；`-s` 只返回一个 PID。但它不会自动证明“这个 PID 就是业务题目所指实例”。多实例、解释器脚本和参数差异仍需
用 `ps` 或 `pgrep -a -f` 继续核对。

⑤ [验证点] 操作后用同一字段集合重新取证

若操作前使用下面的身份视图：

```
ps -p "$pid" -o pid,ppid,user,lstart,etimes,stat,args
```

操作后也应复用同样的字段和原匹配条件。不要只看“原 PID 是否消失”，还要检查原业务特征是否出现新 PID。

[Cheatsheet] 选择范围与输出字段分开；`-o` 定制字段，`--sort` 排序；`pstree` 看结构，`pidof` 只是快速入口；变更前后使用同一身份字段和匹配条件。

[操作专题] 用 `pgrep` 精确缩小集合，再决定是否操作

`pgrep` 适合把系统中的全部进程收缩为一个可审阅集合。真正的安全边界不是“会不会写正则”，而是先预览将被操作的对象，并理解不同条件之间如何组合。
`kill` 与 `pgrep` 使用相近的选择语义，因此每一条批量信号命令都应有对应的预览命令。

① [操作] 短命令名、完整命令行与精确匹配

```
pgrep -l report-worker      # PID + 短命令名
pgrep -a report-worker      # PID + 完整命令行显示
pgrep -a -x report-worker   # 短命令名精确匹配
pgrep -a -f '/opt/report/bin/worker --queue=night'
```

`-f` 把匹配范围扩展到完整命令行，适合区分同一程序的参数实例，也更容易误匹配。模式应尽量包含稳定路径和关键参数，进入 `kill` 前必须先用完全相同的
条件预览。

② [操作] 按用户、父进程、进程组、会话和终端收缩范围

```
pgrep -a -u appsvc report-worker
pgrep -a -P 2450
pgrep -a -g 6200
pgrep -a -s 6000
pgrep -a -t pts/2
```

常用选择条件：

选项	选择对象
<code>-u</code>	有效用户 ID
<code>-U</code>	真实用户 ID
<code>-P</code>	直接父 PID
<code>-g</code>	进程组 ID
<code>-s</code>	会话 ID
<code>-t</code>	控制终端
<code>-r</code>	运行状态集合

同一选项中的逗号列表通常表达多个候选值；不同选择条件同时存在时，目标必须满足全部条件。不要把条件叠加误解为“任意一个命中即可”。

③ [操作] 使用最旧、最新、运行时长和线程选择器

```
pgrep -a -o report-worker      # 最旧实例
pgrep -a -n report-worker      # 最新实例
pgrep -a -O 3600 report-worker # 启动超过 3600 秒
pgrep -w report-worker         # 列出线程 ID
```

“最旧”或“最新”只是排序条件，不自动等于“旧业务实例”或“正确目标”。仍应核对用户、父进程、完整参数和启动时间。

④ [操作] 把预览结果交给 ps 进行第二层确认

```
mapfile -t pids < <(pgrep -u appsvc -f '/opt/report/bin/worker --queue=old')
((${#pids[@]})) || { echo 'no candidates'; exit 0; }

ps -p "${IFS=,; echo "${pids[*]}"}" \
-o pid,ppid,user,lstart,etimes,sid,pgid,TTY,stat,args
```

这里的目标不是要求考试中必须写复杂 Shell，而是形成思维顺序：匹配集合只是候选，身份字段才是操作依据。

⑤ [安全边界] pkill 前后都使用原条件验证

```
pgrep -a -u appsvc -f '/opt/report/bin/worker --queue=old'
pkill -TERM -u appsvc -f '/opt/report/bin/worker --queue=old'
sleep 2
pgrep -a -u appsvc -f '/opt/report/bin/worker --queue=old'
```

若最终查询出现新 PID，不应直接再次 pkill。应比较启动时间和 PPID，判断是否由主进程、systemd、容器或其他监督器重新创建。

[Cheatsheet] -x 精确短命令名，-f 完整命令行，-u/-P/-g/-s/-t 按属性收缩；不同条件通常是 AND；pkill 前必须用同条件 pgrep -a 预览，之后再用原条件验收。

[操作专题] 用 top 和 /proc/<PID> 建立持续、可下钻的证据

ps 是一次快照，top 是周期采样，/proc 是进程级上下文。三者并不是互相替代，而是从系统概览逐步收缩到具体对象：先观察负载和任务状态分布，再选择 PID 或线程，最后读取命令行、工作目录、打开文件和 I/O 等上下文。

① [操作] 用批处理模式保存多轮 top 证据

```
top -b -d 2 -n 5 -w 200 > /tmp/top-5x2s.txt
```

参数	作用
-b	非交互批处理模式
-d 2	两轮间隔 2 秒
-n 5	共输出 5 轮后退出
-w 200	扩大输出宽度，减少命令行截断
-o %CPU	指定排序字段

固定间隔和轮数可以让前后对比使用相同采样窗口。首次采样中的百分比可能受累计统计影响，不应只取第一屏下结论。

② [操作] 按 PID、用户或线程下钻

```
top -p 2450
top -H -p 2450
top -b -H -p 2450 -d 2 -n 5 -w 200
top -u appsvc
```

-H 展开线程。进程总 CPU 高时，线程视图可继续定位热点；进程总 CPU 不高但负载高时，线程视图也可能揭示多个中等热点。线程 ID 不是另一个独立业务实例，操作前仍应理解其所属进程。

③ [操作] /proc/<PID>/status 与 stat 提供结构化身份和状态

```
cat /proc/2450/status
cat /proc/2450/stat
```

status 适合人工读取，包含名称、状态、PID、PPID、线程数、UID/GID、内存等字段。stat 是单行机器接口，字段数量多，命令名还可能包含空格和括号；没有可靠解析器时，不建议用简单的空格切割脚本硬解析全部字段。

④ [操作] 命令行、可执行文件与工作目录确认真实上下文

```
tr '\0' ' ' < /proc/2450/cmdline; echo
readlink -f /proc/2450/exe
readlink -f /proc/2450/cwd
```

`cmdline` 使用 NUL 分隔参数，直接 `cat` 可能显示为粘连文本。内核线程或已退出进程可能没有预期内容。`exe` 和 `cwd` 受权限、命名空间和进程生命周期影响，读取失败要区分“对象已消失”和“当前用户无权访问”。

⑤ [操作] 文件描述符与 I/O 帮助解释“它在等什么”

```
ls -l /proc/2450/fd
cat /proc/2450/io
cat /proc/2450/wchan
```

`fd/` 可显示打开的文件、管道和套接字链接；`io` 提供读写计数；`wchan` 显示当前内核等待位置。它们都是线索：文件描述符存在不证明业务读写正常，I/O 计数增长也不证明数据已持久化到远端系统。

⑥ [边界] `/proc` 是活对象接口，读取天然存在竞态

进程可在读取多个文件之间退出或执行新程序，因此：

- 某个文件刚读取成功，下一条路径可能已经不存在；
- 对高风险操作，应在发送信号前再次读取关键身份；
- 不要把 `/proc/PID` 消失等同于业务终态正确；
- 不要默认打印 `environ`，其中可能包含令牌、密码或其他敏感信息。

[Cheatsheet] `top -b -d -n` 保存趋势，`-H` 展开线程；`status` 看人类可读状态，`cmdline` 用 NUL 分隔，`exe/cwd` 看真实路径，`fd/io/wchan` 看资源与等待线索；活对象读取存在竞态。

[操作专题] Bash 前台、后台与作业生命周期

作业控制只管理当前交互 Shell 启动的命令或管道。它解决的是“在同一个终端会话里怎样暂停、继续和切换前后台”，不提供开机启动、自动重启、依赖、统一日志或资源治理。需要长期运行的业务应迁移到下一章的 `systemd` 模型。

① [操作] 使用 `&` 启动后台作业

```
find /srv -type f -printf '%p\n' > /tmp/srv-files.txt 2>&1 &
jobs -l
```

`&` 让 Shell 不等待命令完成并立即返回提示符。后台不等于脱离终端：进程仍可能继承终端、收到 HUP，或因尝试读取终端输入而停止。

② [操作] 用 `Ctrl+Z`、`bg` 与 `fg` 切换状态

```
# 前台命令运行时按 Ctrl+Z
jobs -l
bg %1
fg %1
```

`Ctrl+Z` 通常向前台进程组发送 `SIGTSTP`；`bg` 发送继续信号并让作业留在后台；`fg` 把作业重新设置为前台。需要交互输入的程序在后台读取终端时可能再次停止。

③ [知识点] `jobspec` 只属于当前 Shell

引用	含义
<code>%1</code>	作业 1
<code>%+</code> 或 <code>%%</code>	当前作业
<code>%-</code>	上一个作业
<code>%name</code>	以名称前缀匹配的作业

另一个终端没有同一作业表，即使它可以通过 `ps` 看到相同 PID。脚本和非交互 Shell 的作业控制行为也不同，因此不要把 `%1` 当成系统级稳定标识。

④ [操作] `jobs -l` 与 `ps` 分别提供 Shell 和内核视角

```
jobs -l
ps -p <PID> -o pid,ppid,sid,pgid,tpgid,ttty,stat,args
```

`jobs` 显示 `Done` 说明当前 Shell 已获知作业结束；作业从列表消失不保证所有双重 `fork` 的后代都结束。需要确认进程树时继续使用 `pstree` 或按 PGID 查询。

⑤ [操作] nohup 与 disown 处理终端挂断的不同部分

```
nohup /usr/local/bin/import-data \  
> /var/tmp/import-data.log 2>&1 &  
  
jobs -l  
disown -h %1
```

- nohup 启动命令时让它忽略 SIGHUP，并在标准流仍指向终端时调整重定向。
- disown -h 让 Bash 在退出时不向该作业发送 HUP，但保留作业表条目。
- disown %1 将作业从作业表移除。

nohup 不会自动把命令放到后台，因此常与 & 和显式重定向组合。两者都不提供失败重启、依赖关系和持久服务定义。

⑥ [工作迁移] 一次性长任务与长期服务使用不同方案

偶发的数据导入、临时编译和维护脚本，可以在充分记录 PID、输出和退出状态的前提下使用 nohup。需要反复运行、系统启动后自动存在、失败后恢复或统一审计的程序，应定义为 systemd service；这不是“命令写得更长”的问题，而是生命周期管理对象已经改变。

[Cheatsheet] 后台 &；列作业 jobs -l；暂停 Ctrl+Z；后台继续 bg %N；拉回前台 fg %N；jobspec 只属于当前 Shell；nohup/disown 不等于 systemd 服务。

[操作专题] 信号控制：先请求协作，再决定是否强制

信号是异步通知，不是“删除进程”的同义词。多数信号可被程序捕获、忽略或改变处理方式；SIGKILL 和 SIGSTOP 不能被捕获、阻塞或忽略。安全控制必须先确认对象和范围，再发送最小影响的信号，最后重新查询。

① [知识点] 常用信号及边界

信号	常见用途	边界
SIGHUP	终端挂断；部分程序约定为重读配置	是否 reload 由程序定义
SIGINT	交互中断，常来自 Ctrl+C	可被程序处理
SIGTERM	请求有序终止	默认首选，可被处理或忽略
SIGKILL	内核强制终止	无清理机会，只作最后手段
SIGSTOP	无条件停止	不能被程序处理
SIGTSTP	终端停止，常来自 Ctrl+Z	可被处理
SIGCONT	继续停止的进程	用于恢复 T 状态
SIGCHLD	子进程状态变化	父进程据此回收子进程

信号编号在不同体系结构上并非全部一致，命令和文档中优先使用名称。kill -l 可查看当前系统的信号名称与编号。

② [操作] kill 对明确 PID 或 jobspec 发送信号

```
kill -TERM 2450  
kill -CONT 2450  
kill -STOP 2450  
kill -TERM %1
```

未指定信号时，kill 默认发送 SIGTERM。Bash 的 kill 是 builtin，因此可以直接接受 jobspec。命令返回成功只说明内核接受了发送请求，不证明目标已经退出。

③ [操作] kill -0 检查存在与权限，但不是身份验证

```
kill -0 2450
```

信号 0 不执行普通信号动作，可用于检查 PID 是否存在以及当前用户是否有发送权限。但它不能证明：

- PID 仍属于原业务实例；
- 进程处于健康状态；
- 进程已经完成清理；
- 业务功能达到目标。

因此强制操作前仍需重新读取用户、启动时间和命令行。

④ [操作] 向进程组发送信号时明确负 PID 语义

外部 `/usr/bin/kill` 和 Shell builtin 在参数解析上可能存在细节差异。对进程组操作，应使用清晰形式并先确认 PGID：

```
ps -p 2450 -o pid,pgid,sid,tt,stat,args
kill -TERM -- -6200
```

负的目标值表示进程组。它可能影响整条管道或同一作业中的多个进程，不能把 PGID 当成“另一个 PID”。考试或脚本中，优先使用信号名称和 `--` 消除选项歧义。

⑤ [操作] `pkill` 按集合发送信号，但必须先预览

```
pgrep -a -u appsvc -f '/opt/app/bin/worker --queue=old'
pkill -TERM -u appsvc -f '/opt/app/bin/worker --queue=old'
```

`pkill` 的风险来自选择集合，而不是信号本身。模式过宽、用户条件遗漏或完整命令行中的公共参数，都可能让非目标进程一起收到信号。

⑥ [流程] 安全终止的标准链路

```
读取现状
→ 缩小候选集合
→ 确认用户、启动时间、完整命令行与父进程
→ 发送 SIGTERM
→ 等待合理时间
→ 查询原 PID 与原业务特征
→ 若仍存在，再次确认身份和状态
→ 最后才考虑 SIGKILL
```

示例骨架：

```
ps -p "$pid" -o pid,ppid,user,lstart,etimes,stat,args
kill -TERM "$pid"
sleep 2
ps -p "$pid" -o pid,ppid,user,lstart,etimes,stat,args
pgrep -a -u appsvc -f '/opt/app/bin/worker --queue=old'
```

⑦ [边界] 服务进程被重新拉起时，应切换管理层

若原 PID 消失但相同业务特征立即出现新 PID，应比较 PPID、启动时间和进程树。可能的原因包括：

- 主进程自动重建 worker；
- systemd 的重启策略；
- 容器运行时或其他监督器；
- 计划任务或人工重复启动。

本章到此停止直接反复 `kill`。若确认属于 systemd unit，后续控制入口应转到第 12 章。

[Cheatsheet] 默认先 TERM，等待并再查询；KILL 无清理机会；STOP/CONT 控制暂停恢复；`kill -0` 只查存在与权限；负目标可表示进程组；`pkill` 前用同条件 `pgrep -a`。

[操作专题] `nice` 与 `renice`：调整普通 CPU 调度倾向

`nice` 值不是 CPU 配额，也不保证某个任务在指定时间完成。它只是普通调度中的相对倾向：数值越低，任务越有利；数值越高，任务越愿意让出 CPU。只有在证据支持 CPU 竞争时，调整 `nice` 才可能是合适的最小干预。

① [知识点] `nice` 值方向与常见范围

现代 Linux 上常见范围为 `-20` 到 `19`：

```
-20  更有利
0    常见默认值
19   更不利
```

“提高 `nice` 值”意味着让进程更礼让，通常会降低其调度有利程度。不要把“数值更大”和“优先级更高”混在一起。

② [操作] nice 在启动新命令时调整 niceness

```
nice -n 10 /usr/local/bin/build-index
```

GNU `nice -n N` 表示在继承的 `niceness` 基础上增加调整值；未指定 `-n` 时默认调整 10。操作后应查看实际 `NI`：

```
ps -C build-index -o pid,user,ni,pri,stat,%cpu,etimes,args
```

③ [操作] renice 修改运行中的 PID、进程组或用户集合

为避免不同实现和兼容模式中 `-n` 相对/绝对语义的歧义，本章优先使用长选项：

```
renice --priority 15 --pid 2450
renice --priority 10 --pgrp 6200
renice --priority 10 --user batchsvc
```

- `--priority` 指定目标绝对 `nice` 值；
- `--relative` 指定相对调整量；
- `--pid`、`--pgrp`、`--user` 明确解释后续标识符的类型。

④ [边界] 普通用户通常只能让自己的任务更不利

普通用户一般只能操作自己拥有的进程，并只能把 `niceness` 调整到更不利的方向。把 `nice` 值降低为更有利的值通常需要适当特权。权限错误不能用反复尝试或宽泛 `sudo` 规则绕过；特权授权归第 10 章。

⑤ [验证点] 同时验证对象身份和实际 NI

```
ps -p 2450 -o pid,ppid,user,lstart,ni,pri,stat,%cpu,args
```

若 `PID` 在 `renice` 前后发生复用，单看命令返回值可能把操作归因到错误实例。验证时仍要保留身份字段。

⑥ [诊断边界] nice 不能修复 I/O、锁、内存或资源上限问题

当高负载来自 `D` 状态、远程文件系统等待、应用锁竞争或内存回收时，调整 `nice` 可能几乎没有帮助。`cgroup CPUWeight`、配额、实时调度和 `I/O` 优先级属于更深层资源治理，不在本章完整展开。

[Cheatsheet] `nice` 值越低越有利；新命令用 `nice`，运行中对象用 `renice`；优先用 `--priority/--relative` 消除歧义；操作后查 `NI`；它不是 `CPU` 配额，也不修复 `I/O` 根因。

[诊断专题] 从状态和负载症状推进到下一条证据

诊断不是把所有命令一次执行完，而是在竞争假设之间选择最有区分度的证据。每一步都应回答：当前证据支持什么、排除什么、下一条证据为什么比其他命令更有效。

① [诊断] z 状态持续累积

症状：多个 `Z` 状态持续存在
→ 当前证据：`PID`、`PPID`、启动时间、父进程身份
→ 假设：父进程未 `wait`；父进程卡死；父进程持续异常派生
→ 下一条证据：按 `PPID` 查看全部子进程和父进程状态
→ 最小修复：修复或按管理层重启父进程，而非杀僵尸
→ 再验证：`Z` 数量不再增长，旧表项被回收

```
ps -eo pid,ppid,stat,lstart,etimes,args | awk '$3 ~ /^Z/'
ps --ppid <PPID> -o pid,ppid,stat,lstart,etimes,args
ps -p <PPID> -o pid,ppid,user,stat,lstart,etimes,args
```

② [诊断] D 状态进程“杀不掉”

症状：`TERM/KILL` 后 `PID` 仍在，`STAT` 为 `D`
→ 当前证据：`wchan`、`/proc/PID/io`、共同挂载或设备
→ 假设：本地块设备等待、`NFS`/网络存储等待、驱动/内核问题
→ 下一条证据：同类 `D` 进程是否共享等待点或挂载
→ 最小修复：恢复依赖或按存储/网络层处理
→ 再验证：进程离开 `D`，待处理信号才可能生效

不要为了“证明命令有效”而持续发送更强信号。

③ [诊断] T 状态是预期暂停还是异常遗留

```
ps -p <PID> -o pid,ppid,sid,pgid,tpgid,ttty,stat,args
jobs -l
```

若属于当前 Shell 的停止作业，可用 `bg` 或 `fg` 继续；若无作业记录，则调查是否收到 `SIGSTOP`、是否被调试器跟踪、是否由另一个管理员暂停。不要盲目对所有 `T` 状态执行 `SIGCONT`。

④ [诊断] 负载高但无单一高 CPU 进程

优先并行检查四个假设：

假设	最有区分度的证据
多个任务共同竞争 CPU	多轮 <code>top</code> 的 <code>%Cpu</code> 、 <code>R</code> 数量、多个中高 <code>%CPU</code>
大量不可中断等待	<code>D</code> 状态数量、 <code>wchan</code> 、共同 I/O 或挂载
热点在线程	<code>top -H</code> 或 <code>ps -eL</code>
大量短命子进程	多轮采样与 <code>pstree</code> 的父进程派生模式

`renice` 只有在 CPU 竞争得到支持、且业务允许牺牲该任务进度时，才是可考虑的最小干预。

⑤ [诊断] 原 PID 消失但同名进程再次出现

症状：kill 后旧 PID 消失，新 PID 很快出现
→ 当前证据：新旧启动时间、PPID、完整命令行、进程树
→ 假设：父进程重建；systemd 重启；容器监督；计划任务
→ 下一条证据：新 PID 的祖先和管理归属
→ 最小修复：转到实际管理者，而不是继续追杀 PID
→ 再验证：目标业务实例和管理状态同时达到终态

⑥ [安全边界] 诊断中不使用破坏性捷径替代证据

本章不把以下做法作为默认答案：

- 未确认对象就 `kill -9`；
- 未预览集合就宽泛 `pkill -f`；
- 把所有同名进程一次性结束；
- 发现负载高就立即 `renice` 或重启；
- 读取并展示整个 `/proc/PID/enviro`；
- 把“命令成功”扩大为业务终态正确。

[Cheatsheet] Z 查父进程，D 查等待依赖，T 查作业和信号来源；高负载先区分 R、D、线程和短命进程；新 PID 出现时转到实际管理者。

[经典任务] 精确定位并安全停止多个同名 worker 中的旧实例

环境与当前状态

系统中存在用户 `appsvc` 启动的多个 `report-worker`：

- 当前实例使用参数 `--generation=current`；
- 一个旧实例使用参数 `--generation=legacy`；
- 所有实例的短命令名相同；
- 题目不提供固定 PID，因为 PID 会随每次实验变化；
- 旧实例可能由父进程重新创建。

目标终态

1. 只请求旧实例有序退出；
2. 所有 `--generation=current` 实例保持运行；
3. 首次不得使用 `SIGKILL`；
4. 操作后证明旧参数匹配集合为空；
5. 若出现新的 legacy PID，停止继续发送信号并识别管理者。

限制条件

- 不得直接执行宽泛 `kill -f report-worker`;
- 不得使用 `killall report-worker`;
- 不得只凭短命令名或一个旧 PID 操作;
- 不得把“原 PID 消失”当成完整验收。

验收证据

层次	必须证明
候选集合	只匹配 <code>appsvc</code> 的 <code>legacy</code> 参数实例
身份	用户、启动时间、PPID、完整命令行符合目标
操作	首次发送 <code>SIGTERM</code>
目标终态	<code>legacy</code> 匹配集合为空
非目标保护	<code>current</code> 实例仍存在
管理边界	未出现替代 <code>legacy</code> PID; 若出现则已定位祖先

[参考解答] 先固定集合，再终止并检查是否被重新拉起

① 调查：用完整参数和用户建立候选集合

```
pgrep -a -u appsvc -f \
  '/usr/local/bin/report-worker .*--generation=legacy'
```

这一步只建立候选集合。若无输出，当前已经没有符合条件的旧实例；若有多个结果，不能随意挑一个 PID，应继续检查每个实例。

② 身份确认：查看启动时间、父进程和完整命令行

```
mapfile -t legacy_pids < <(\
  pgrep -u appsvc -f \
    '/usr/local/bin/report-worker .*--generation=legacy'
)

if (($#legacy_pids[@])); then
  csv=$(IFS=,; echo "${legacy_pids[*]}")
  ps -p "$csv" \
    -o pid,ppid,user,lstart,etimes,sid,pgid,tty,stat,args
fi
```

确认每一行都是 `appsvc`、完整路径和 `legacy` 参数。还应记录 PPID，便于判断后续是否由同一父进程重建。

③ 非目标基线：记录 `current` 实例

```
pgrep -a -u appsvc -f \
  '/usr/local/bin/report-worker .*--generation=current'
```

这条证据用于证明操作没有误伤当前实例。

④ 操作：逐个向已确认 PID 发送 `TERM`

```
for pid in "${legacy_pids[@]"; do
  ps -p "$pid" -o pid=,user=,lstart=,args=
  kill -TERM "$pid"
done
```

在 `kill` 前重新执行一次 `ps`，用于降低 PID 在调查和操作之间退出并复用的风险。

⑤ 等待与分层验证

```
sleep 2

# 原 PID 是否仍存在
for pid in "${legacy_pids[@]"; do
    ps -p "$pid" -o pid,ppid,user,lstart,stat,args
done

# 原业务特征是否仍存在, 包括新 PID
pgrep -a -u appsvc -f \
    '/usr/local/bin/report-worker .*--generation=legacy'

# 非目标 current 实例是否仍在
pgrep -a -u appsvc -f \
    '/usr/local/bin/report-worker .*--generation=current'
```

⑥ 结果分支

- legacy 集合为空、current 仍在：达到目标终态。
- 原 PID 仍在但状态为 **D**：停止升级信号，进入 I/O 等待调查。
- 原 PID 仍在且身份未变、程序拒绝 TERM：重新确认影响后才考虑 SIGKILL。
- 出现新的 legacy PID：比较新 PID 的 PPID、祖先和启动时间；转向父进程或 systemd 管理层，不继续追杀 PID。

典型错误包括：只检查原 PID、用 `pkill -f report-worker` 误伤 current、看到多个结果后直接 `kill -9`，以及忽略父进程重建。

[经典任务] 调查负载高但没有单一高 CPU 进程

环境与当前状态

系统在最近 15 分钟持续报告较高 load average。初次打开 `top` 时：

- 没有一个进程长期占据极高 CPU；
- CPU idle 仍有一定余量；
- 不允许先重启、终止或修改 nice；
- 需要可用保存的证据区分 CPU 竞争、I/O 等待、线程热点和短命进程。

目标终态

形成一份能说明“当前最支持哪个假设、下一步进入哪一层”的调查记录，而不是只罗列命令。若证据不足，应明确保留哪些假设，而非强行给出根因。

限制条件

- 不伪造固定输出值；
- 不把 load average 当作 CPU 百分比；
- 不把 `top` 第一屏当作完整趋势；
- 不把 `renice` 作为未知根因的默认修复。

[参考解答] 用固定采样窗口区分 R、D、线程与短命进程

① 建立系统级基线

```
uptime
nproc
```

记录 1、5、15 分钟负载和逻辑 CPU 数量。二者的比较只用于理解压力尺度，不存在适用于所有系统的单一“超过 CPU 数量就必然故障”阈值。

② 保存多轮 `top` 样本

```
top -b -d 2 -n 5 -w 200 > /tmp/top-5x2s.txt
```

检查每轮中的：

- CPU user/system/iowait/idle 变化；
- **R**、**S**、**D** 等任务数量；
- 是否多个进程共同获得中等 CPU；

- 热点是否在轮次之间快速更换。

③ 使用定制 ps 同时查看状态、等待点和资源

```
ps -eo pid,ppid,user,stat,wchan:28,ni,psr,%cpu,%mem,rss,vsz,etimes,args \
--sort=-%cpu
```

解释分支：

- 多个 R 且 CPU idle 低：CPU 竞争假设增强；
- 多个 D 且共享相似 wchan：I/O 或依赖等待假设增强；
- 无持续热点但 load 高：线程或短命进程假设仍需检查。

④ 展开线程视图

```
ps -eL -o pid,lwp,ppid,psr,stat,ni,%cpu,comm --sort=-%cpu
# 或对具体 PID
top -b -H -p <PID> -d 2 -n 5 -w 200
```

若同一 PID 下多个线程共同消耗 CPU，进程级 %CPU 可能掩盖内部热点分布。这里只定位到线程，不进入应用栈分析。

⑤ 观察父进程和短命子进程模式

```
pstree -ap
for i in 1 2 3 4 5; do
  date '+%F %T'
  ps -eo pid,ppid,stat,%cpu,etimes,args --sort=-%cpu | head -n 30
  sleep 2
done > /tmp/ps-samples.txt
```

若某个父进程持续产生运行时间很短的子进程，单次快照可能看不到稳定的高 CPU 进程，但大量创建和退出仍会提高系统压力。

⑥ 对可疑 PID 读取 /proc 上下文

```
cat /proc/<PID>/status
cat /proc/<PID>/io
cat /proc/<PID>/wchan
tr '\0' ' ' < /proc/<PID>/cmdline; echo
```

对 D 状态，比较多个进程是否共享等待点和文件系统；对 CPU 竞争，确认任务身份、用户和业务作用，再决定是否需要业务限流或调整 nice。

⑦ 结论格式

调查记录至少写明：

```
症状：15 分钟负载持续偏高
证据：固定 2 秒间隔、5 轮 top；定制 ps；线程视图
当前最支持：例如 D 状态任务共享同一 NFS 等待点
仍未排除：例如短命子进程
下一条证据：进入 NFS/存储和内核日志层
当前不执行：kill、restart、renice，因为尚未证明它们针对根因
再验证：修复依赖后使用相同采样窗口比较负载和 D 数量
```

如果证据支持 CPU 竞争且允许降低某个批处理任务的吞吐量，才可以在记录基线后使用 renice --priority，并用同样的多轮采样验证影响。

[本章收束] 从“看到一个 PID”迁移到可审计的运行控制

本章真正需要保留的不是一长串参数，而是一套可重复的操作顺序：

```
身份组合
→ 关系与状态
→ 精确选择
→ 固定窗口采样
→ 最小影响操作
→ 同字段再验证
→ 检查原业务特征与管理者
```

工作方法

- 1. 先建立基线。记录目标 PID、用户、启动时间、完整命令行、PPID、PGID、SID、TTY 和状态；高负载场景还要固定采样间隔和轮数。
- 2. 把选择集合与操作分开。pgrep、ps 先证明候选集合；pkill、kill、renice 后改变状态。
- 3. 优先最小干预。能用 SIGTERM 请求有序退出，就不直接 SIGKILL；根因未知时不先重启或 renice。
- 4. 复用同一证据。操作前后使用同一字段和同一业务匹配条件，才能比较原实例、替代实例和非目标实例。
- 5. 识别管理层。原 PID 消失却出现新 PID，说明“进程实例”可能不是最终控制对象；应比较 PPID、祖先和启动时间，转向实际管理者。

主要判断表

看到的证据	能支持的判断	不能直接推出	下一条高区分度证据
ps 中 PID 存在	当前采样时存在一个该 PID	仍是几分钟前确认的实例；业务健康	用户、lstart、PPID、完整 args
kill 返回成功	发送请求被内核接受	进程已经退出；数据已安全落盘	等待后再次 ps，并按原特征 pgrep
kill -0 成功	PID 存在且当前用户有发送权限	身份正确、状态健康、终态正确	完整身份字段和业务验证
Z 状态	子进程已退出，父进程尚未回收	继续 kill -9 可以清除	PPID、父进程健康和僵尸增长趋势
D 状态	正处于不可中断等待	更强信号会立即生效	wchan、/proc/PID/io、共同挂载或设备
load average 高	可运行或不可中断等待任务较多	CPU 使用率一定高	多轮 top、R/D 数量、线程和短命进程
VSZ 很大	虚拟地址空间较大	已占用同等物理内存	RSS、系统可用内存、趋势和映射
原 PID 消失	原进程表项已不存在	业务实例不会再次出现	原匹配条件、PPID/祖先、启动时间
nice 值改变	普通调度倾向已变化	获得固定 CPU 份额；根因被修复	同窗口 CPU/负载采样和业务影响

最终 Cheatsheet

```
# 身份与关系
ps -p PID -o pid,ppid,user,lstart,etimes,sid,pgid,TTY,stat,ni,%cpu,%mem,rss,vsz,args
pstree -aps PID

# 精确选择并预览
pgrep -a -u USER -f 'STABLE_PATH .*KEY_ARGUMENT'

# 趋势与线程
top -b -d 2 -n 5 -w 200
top -b -H -p PID -d 2 -n 5 -w 200

# /proc 下钻
cat /proc/PID/status
tr '\0' ' ' < /proc/PID/cmdline; echo
cat /proc/PID/io
cat /proc/PID/wchan

# 作业控制
jobs -l
bg %N
fg %N

# 安全终止
kill -TERM PID
sleep 2
ps -p PID -o pid,ppid,user,lstart,stat,args
# 重新确认身份和影响后，最后才考虑：kill -KILL PID

# 普通调度倾向
nice -n 10 COMMAND
renice --priority 15 --pid PID
```

向下一章交接

本章把一个运行实例识别为“可能受外部管理的进程树”，但不负责定义持久服务终态。进入下一章《systemd Unit、服务与依赖关系》后，需要把这里的证据继续映射到 unit：由哪个 unit 创建进程、主 PID 与工作进程是什么关系、为什么进程会被重新拉起，以及如何通过 unit 状态而不是追逐 PID 完成控制与验收。

进入下一章时继续回答：

- 1. 哪个 unit 是这棵进程树的实际管理者？
- 2. MainPID、工作进程与 Restart= 策略怎样对应本章看到的 PID 变化？
- 3. “进程存在”“service active”和“业务功能正常”分别需要什么证据？
- 4. 停止、启动、重启与开机启用为什么是不同的状态维度？

本章不会提前展开 unit 文件、依赖顺序、启用状态和 drop-in。这里保留的交接证据是：完整身份、进程树、启动时间、原业务匹配条件，以及“原 PID 消失后是否出现替代实例”的观察结果。

RHEL 9 · RHCSA 实操讲义

12

systemd Unit、服务与依赖关系

从声明对象到真实功能：把加载来源、当前状态、持久配置、依赖顺序与服务进程放进同一条证据链。

对象模型操作语义验证诊断经典任务

大字号阅读版

本章阅读导航

先抓住一条主线

systemd 不只是“启动服务”的命令集合。它先加载并合并 unit 定义，再根据依赖和顺序建立 job，最后形成当前状态、进程身份与持久启动关系。任何一层成立，都不能自动替代最终功能验收。

专题地图

1. 知识专题 Unit 类型、名称、模板与实例
2. 知识专题 Load、Active、Sub 与 UnitFile 四层状态
3. 操作专题 status、show、cat 与依赖查询
4. 操作专题 当前控制、持久启用与 mask
5. 知识专题 Requires/Wants 与 After/Before
6. 知识专题 socket、path、timer 与 Restart 激活
7. 操作专题 drop-in、自定义 service 与 daemon-reload
8. 诊断专题 从症状推进到下一条有区分度的证据
9. 经典任务 受限账号服务与重新激活诊断

阅读时持续回答

1. 现在判断的是加载、当前、持久还是功能状态？
2. 看到的是 unit、job 还是进程？
3. 依赖关系是否同时声明了顺序？
4. 服务被谁拉入事务、被谁再次激活？
5. 实际加载的是发行版主文件还是管理员 drop-in？
6. 修改后需要 daemon-reload、应用 reload 还是 restart？
7. 哪条证据只能证明局部，下一层验收是什么？

01 识别 Unit 类型、完整名称与实例

02 确认来源主文件、drop-in 与优先级

03 读取状态 Load / Active / Sub / UnitFile

04 展开关系依赖、顺序与激活器

05 实施变更当前、持久与定义加载

06 分层验收身份、功能与重启后状态

第 12 章 · 正文

Linux 上看到一个进程，并不等于已经理解了它的管理对象。进程可能由管理员在 Shell 中直接启动，也可能由 systemd 根据一个 `.service` 声明创建；服务可能由默认 target 在引导时拉起，也可能由 `.socket`、`.path` 或 `.timer` 在事件发生时激活；管理员看到的 unit 内容还可能是发行版主文件、运行时定义与多个 drop-in 合并后的结果。

本章最常见的误判是把 `active` 当成业务正确，把 `enabled` 当成当前运行，把 `After=` 当成依赖拉起，或者把 `daemon-reload` 当成应用配置 reload。为避免这些错误，本章始终沿着下面的证据链推进：

磁盘上的 unit 定义

→ manager 的加载与合并结果

→ 依赖和顺序形成的 job 事务

→ unit 当前状态与 service 进程身份

→ 持久启动配置

→ 对外功能是否真正成立

第 11 章已经建立进程、PID 与信号的前置模型；本章只在确认 service 进程身份时引用它。完整日志检索转交第 13 章，timer 的日历表达式与调度验收转交第 15 章。

概念

Unit 是 systemd 管理的声明对象，可以描述 service、socket、path、timer、target、mount 等对象。unit 文件不是“正在运行的进程”，而是 manager 用来决定如何加载、激活、停止和关联对象的配置；观察它时要同时查看完整名称、类型和实际加载来源。

概念

Job 是 manager 为 start、stop、reload 等请求建立的一次事务任务，完成后即消失。unit 是可持续存在的管理对象，job 是针对 unit 的一次动作，service 激活后才可能产生一个或多个进程；三者不能混为同一个状态。

概念

LoadState 回答 manager 是否获得了可用定义，例如 loaded、not-found、error 或 masked。它只证明定义层，不证明 unit 已启动；新建或修改文件后，磁盘内容与 manager 内存中的定义还可能因为尚未 daemon-reload 而不一致。

概念

ActiveState 与 SubState 分别描述跨类型通用生命周期和类型专用细节。active 可能对应 service 的 running，也可能对应 oneshot 的 exited；因此不能只看到 active 就推断存在长期进程，更不能据此推断端口或业务结果正确。

概念

UnitFileState 描述持久激活配置，而不是当前运行状态。enabled 表示已按 [Install] 建立链接，disabled 表示尚未启用，static 通常表示没有普通 enable 入口但仍可被依赖或触发，masked 则阻断普通激活。

概念

依赖关系与顺序关系 是两张独立的图。Wants=、Requires= 决定是否把另一个 unit 拉入事务；After=、Before= 只决定已进入同一事务的对象谁先谁后。只写 After 不会自动启动对方，只写 Requires 也不保证启动顺序。

概念

激活器 是能在事件发生时拉起其他 unit 的对象，例如 socket 收到连接、path 观察到文件变化、timer 到达触发时刻。它们解释了“service 停止后为什么又回来”；另一个独立机制是 service 自身的 Restart= 策略。

概念

Drop-in 是管理员对现有 unit 做最小覆盖的配置片段，通常位于 /etc/systemd/system/UNIT.d/*.conf。它在发行版主文件之后合并，便于保留软件包升级路径；但列表型指令可能追加而非替换，覆盖 ExecStart= 时常需要先用空赋值清除旧列表。

操作语义以下六组入口先建立本章的接口地图。先确认作用对象，再选择参数；详细验证与边界在后续专题展开。

systemctl status / show / cat

SYNOPSIS

```
systemctl status UNIT [--no-pager] [--full]
systemctl show UNIT [-p PROPERTY,...]
systemctl cat UNIT
```

分别用于人工综合调查、读取精确属性和还原主文件与 drop-in 来源。

重要参数 / 形式

`--full / -l`

避免长行被省略，适合检查完整命令和路径。

`-p PROPERTY,...`

只输出指定属性，适合稳定判断。

`cat UNIT`

确认最终配置来自哪些文件，但仍需结合 show 判断 manager 状态。

start / stop / restart / reload

SYNOPSIS

```
systemctl start|stop|restart|reload UNIT
```

改变 unit 的当前状态；不会自动修改下一次引导时的持久配置。

重要参数 / 形式

restart

停止后重新启动，通常会中断当前进程。

reload

请求应用重读自身配置，前提是 unit 支持该动作。

reload-or-restart

支持 reload 时优先 reload，否则 restart。

enable / disable / mask

SYNOPSIS

```
systemctl enable|disable|mask|unmask [--now] [--runtime] UNIT
```

管理持久或运行时激活关系；mask 比 disable 更强，会阻断普通启动。

重要参数 / 形式

--now

在修改持久状态的同时改变当前状态。

--runtime

只写入 /run，重启后失效；适合临时控制，不等于持久配置。

mask

只在目标明确要求阻断普通激活时使用，先调查现有依赖。

list-dependencies

SYNOPSIS

```
systemctl list-dependencies [--reverse] [--all] UNIT
```

沿依赖图观察“它需要谁”或“谁需要它”，用于解释引导拉起和重新激活。

重要参数 / 形式

--reverse

反向查看依赖者，是调查“谁把它拉起来”的关键入口。

--all

显示默认可能省略的 inactive 依赖。

show -p Wants,Requires,After,Before

需要精确区分关系类型时读取属性。

systemctl edit / daemon-reload

SYNOPSIS

```
systemctl edit [--runtime] UNIT
systemctl daemon-reload
```

edit 创建管理员 drop-in；daemon-reload 让 manager 重新扫描并合并 unit 定义。

重要参数 / 形式

edit UNIT

默认写入持久管理员目录，优先于直接修改 vendor 文件。

edit --runtime UNIT

创建仅本次引导有效的 drop-in。

daemon-reload

只刷新 systemd 定义，不等于应用自身的 reload，也不会自动重启服务。

自定义 service / target

SYNOPSIS

```
[Service]
Type=... ExecStart=... User=... Restart=...
[Install]
WantedBy=multi-user.target

systemctl get-default
systemctl set-default TARGET
```

service 字段定义进程生命周期和身份；target 组织依赖并决定默认引导目标。

重要参数 / 形式

Type=
决定 systemd 如何判断启动完成和主进程。

ExecStart=
定义启动命令，默认不经过 Shell 解析。

User=
指定服务进程身份，必须同时检查目录访问能力。

Restart=
定义退出后的重启策略，不等同于 socket/path/timer 激活。

WantedBy=
说明 enable 时把链接放入哪个 target 的 wants 目录。

版本与环境边界 本章示例以 RHEL 9 课程与对应手册语义为基线。实际执行时应在目标主机核对 systemd 版本、unit 来源与应用行为；静态定义检查不能代替启动、跨重启和功能验收。

[知识专题] Unit 是声明对象：先分清类型、名称和实例

进入 systemd 时，最容易出现的第一类混淆是把 unit、进程和一次操作任务当成同一个东西。unit 是可持久描述的对象；systemd 为启动或停止 unit 建立 job；service 激活后才可能产生一个或多个进程。先把三者分开，才能正确理解状态、依赖和错误。

① [知识点] unit、job 与进程分别回答不同问题

- unit: systemd 管理的声明对象，例如 `sshd.service`；
- job: manager 为 start、stop、reload 等操作建立的事务任务，完成后消失；
- 进程: service 激活后产生的运行实例，由 PID 和 cgroup 表示。

一个 service 可以没有长期进程，例如已经完成的 `Type=oneshot`；一个 service 也可以拥有主进程和多个工作进程。反过来，一个普通进程如果没有归属于某个 service，也不能仅凭命令名推断应该用哪个 unit 控制。

```
systemctl status sshd.service --no-pager --full
systemctl show sshd.service -p MainPID,ControlPID,ControlGroup
```

`MainPID=0` 不一定说明 unit 不存在；对一次性任务、已退出任务或没有可识别主进程的类型，应结合 `ActiveState`、`SubState`、`Type` 与 `Result` 判断。

② [知识点] unit 后缀表示管理对象，不只是文件扩展名

后缀	管理对象	本章深度
<code>.service</code>	长期进程或一次性动作	完整展开
<code>.socket</code>	网络或 IPC socket	作为服务激活入口展开
<code>.path</code>	文件或目录事件	作为服务激活入口展开
<code>.timer</code>	日历或相对时间	只讲激活关系，时间表式归第 15 章
<code>.target</code>	一组依赖与同步点	展开默认目标和依赖组织
<code>.mount</code> / <code>.automount</code>	挂载对象	只识别，主归属存储章节
<code>.device</code> / <code>.swap</code>	设备与交换空间	只识别
<code>.slice</code> / <code>.scope</code>	cgroup 层级和外部进程集合	只识别，资源治理不展开

unit 名称与对象名称之间常有规则。例如挂载点 `/var/lib/data` 对应的 mount unit 名通常需要转义；这类名称转换应使用 `systemd-escape` 辅助，而不是手工猜测。

③ [知识点] 完整 unit 名可避免对象歧义

在许多 `systemctl` 命令中省略 `.service` 时，systemd 会尝试补全 service 后缀：

```
systemctl status sshd
systemctl status sshd.service
```

两种形式通常指向同一对象，但在讲义、脚本和诊断记录中优先写完整名称。完整名称可以清楚区分 `foo.service`、`foo.socket` 和 `foo.timer`，也便于一次查询相关 unit：

```
systemctl status report-gateway.service report-gateway.socket
```

④ [知识点] 模板 unit 与实例共享结构，但拥有独立状态

模板名称包含 `@`：

```
worker@.service
```

实例把实例名放在 `@` 与后缀之间：

```
worker@blue.service
worker@green.service
```

模板中可使用 `%i` 获取 unit 名称中已经转义的实例标识符，使用 `%I` 获取反转义后的实例名。每个实例是独立 unit，有自己的状态、cgroup、主 PID 和失败结果。对模板进行 enable 时，还要确认 `DefaultInstance=` 或题目是否明确要求具体实例。

⑤ [边界] 别名、生成 unit 和 transient unit 仍属于加载模型

软件包可能通过 `Alias=` 或符号链接提供别名；`/etc/fstab`、设备事件或 generator 可以在运行时产生 unit；`systemd-run` 可以创建 transient unit。这些对象未必对应管理员手写的固定文件，因此在 `/usr/lib/systemd/system` 没找到同名文件”不能直接证明 unit 不存在。

最有区分度的证据是：

```
systemctl show UNIT -p LoadState,FragmentPath,SourcePath,UnitFileState
systemctl cat UNIT
```

[Cheatsheet] unit 是声明对象，job 是一次管理事务，进程是运行实例；写完整后缀避免歧义：模板 `name@.service` 与实例 `name@instance.service` 分开；找不到固定文件时继续查 `FragmentPath`、生成来源和别名。

[知识专题] 四层状态模型：Load、Active、Sub 与 UnitFile

`systemctl status` 的一行输出经常同时出现 `loaded`、`enabled` 和 `active (running)`。这些词不是同义词，而是从不同层描述同一个 unit。考试和工作排错中，最重要的习惯之一，就是先说清楚正在判断哪一层。

① [知识点] LoadState 回答“manager 是否获得了有效定义”

常见值包括：

- `loaded`：定义已成功加载；
- `not-found`：未找到 unit 定义；
- `error`：加载或解析发生错误；
- `masked`：名称被屏蔽，通常解析到 `/dev/null`。

`LoadState=loaded` 只说明定义可用，不说明 unit 已经启动。修改或新建 unit 文件后，如果 manager 尚未重新读取定义，磁盘内容与内存中的加载结果也可能不一致。

② [知识点] ActiveState 是跨类型的通用生命周期

常见通用状态：

ActiveState	含义
<code>active</code>	unit 已达到其活动定义
<code>inactive</code>	unit 当前不活动
<code>activating</code>	正在启动或等待就绪
<code>deactivating</code>	正在停止
<code>failed</code>	最近一次活动过程失败并被记账

ActiveState	含义
reloading	正在执行 reload

对 service 来说，active 仍不能自动证明端口可访问、文件持续更新或应用逻辑正确。

③ [知识点] SubState 只能在 unit 类型上下文中解释

SubState 细化类型专用状态：

- service 常见 running、exited、dead、failed；
- socket 常见 listening；
- timer 常见 waiting；
- mount 常见 mounted。

active (exited) 常见于成功完成并保持活动语义的 oneshot unit，或兼容脚本式 unit。它不能被解释为“存在一个正在运行的守护进程”。需要长期主进程时，应继续查 Type 和 MainPID。

④ [知识点] UnitFileState 回答持久激活配置，而不是当前状态

本章重点训练四个结果：

- enabled：按 [Install] 建立了持久依赖链接；
- disabled：可启用，但目前没有对应启用链接；
- static：通常没有可供普通 enable 使用的安装声明，仍可被依赖、触发或手工激活；
- masked：unit 被强制阻断激活。

环境中还可能看到 alias、indirect、generated、transient 等结果。不要把所有非 enabled 都解释成 disabled，更不能把 static 当作失败。

⑤ [知识点] failed 是 manager 的失败记账，不是根因本身

失败信息应继续拆分：

```
systemctl show UNIT \
-p ActiveState,SubState,Result,ExecMainCode,ExecMainStatus
```

- Result 表示 systemd 对 unit 运行结果的分类；
- ExecMainCode 与 ExecMainStatus 描述主进程退出方式和状态；
- status 中的近期日志只能作为入口，完整日志检索转到第 13 章。

修复定义或应用问题后，reset-failed 可以清除 failed 记账和相关启动速率计数，但它不修复根因，也不自动启动服务。

⑥ [验证点] 将状态拆成五层验收矩阵

验收层	推荐证据	能证明什么	不能证明什么
定义层	cat、show FragmentPath/DropInPaths、verify	实际加载来源和静态定义	运行时一定成功
当前层	is-active、show ActiveState/SubState/Result	systemd 当前视角	重启后自动启动、业务正确
持久层	is-enabled、目标链接	引导依赖配置	当前已经运行
身份层	show MainPID + ps	实际 UID、命令和进程归属	对外功能正确
功能层	端口、请求、文件或数据检查	用户真正需要的终态	其他持久层自动正确

[Cheatsheet] LoadState 看定义能否加载；ActiveState 看通用当前状态；SubState 看类型细节；UnitFileState 看持久配置；failed 要继续查 Result 和退出状态；最终验收必须分层。

[操作专题] 用 systemctl 建立可判定的 Unit 证据视图

稳定的 systemd 调查不是先执行 restart，而是依次回答：对象是否存在、当前处于什么状态、采用了哪份定义、谁依赖或触发它。不同子命令回答不同问题，混用会产生错误推断。

① [操作] 区分已加载对象与磁盘上的 unit file

作用对象：manager 已加载的 unit，或搜索路径中的 unit 文件。

基本语义：list-units 查看当前已加载对象；list-unit-files 查看安装文件及启用状态。

典型形式: `systemctl list-units --type=service --all`; `systemctl list-unit-files --type=service`。

验证边界: 默认列表未出现不等于未安装; unit 可能 inactive 且未加载。

```
systemctl list-units --type=service --all
systemctl list-unit-files --type=service
systemctl list-units --state=failed
```

② [操作] `status` 用于人工调查入口

作用对象: 一个或多个 unit 的综合状态。

基本语义: 汇总加载来源、启用状态、当前状态、主 PID、cgroup 和少量近期刊志。

典型形式: `systemctl status UNIT --no-pager -l`。

关键参数: `--no-pager` 直接输出; `-l` 避免长行截断。

验证边界: `status` 适合人读, 不适合依靠颜色或整段文本做稳定脚本判断。

```
systemctl status report-gateway.service --no-pager -l
systemctl status report-gateway.service report-gateway.socket --no-pager -l
```

③ [操作] `is-active`、`is-enabled` 和 `is-failed` 用于单一判断

```
systemctl is-active report-gateway.service
systemctl is-enabled report-gateway.service
systemctl is-failed report-gateway.service
```

这些命令通过退出状态和短文本回答单一问题, 适合验收脚本。但三者仍需分别执行: `active` 不能代替 `enabled`, `not failed` 也不能证明 `active` 或功能正确。

④ [操作] `show -p` 读取 `manager` 的精确属性

作用对象: `systemd` 合并后的 unit 属性。

基本语义: 以 `NAME=VALUE` 输出稳定字段, 适合定位和自动检查。

典型形式: `systemctl show UNIT -p PROPERTY,...`。

关键属性: `LoadState`、`ActiveState`、`SubState`、`UnitFileState`、`MainPID`、`Result`、`FragmentPath`、`DropInPaths`、`TriggeredBy`、`Restart`。

```
systemctl show report-gateway.service \
-p LoadState,ActiveState,SubState,UnitFileState,MainPID,Result

systemctl show report-gateway.service \
-p FragmentPath,DropInPaths,Wants,Requires,After,Before,TriggeredBy,Restart
```

⑤ [操作] `cat` 还原主文件与 `drop-in` 的可读来源

`systemctl cat UNIT` 按 `manager` 识别的来源显示主文件和 `drop-in`:

```
systemctl cat report-gateway.service
```

它适合回答“管理员覆盖是否真的存在”和“当前读取的是哪个文件”。但 `cat` 显示的是文件片段, 不一定直观表示所有合并后的属性; 对列表重置、隐式依赖和生成属性, 继续使用 `show`。

⑥ [操作] `list-dependencies` 从正向和反向查看关系

```
systemctl list-dependencies report-gateway.service
systemctl list-dependencies --reverse report-gateway.service
systemctl list-dependencies --after report-gateway.service
systemctl list-dependencies --before report-gateway.service
```

- 默认正向回答“它依赖哪些对象”;
- `--reverse` 回答“谁依赖或可能拉起它”;
- `--after/--before` 从排序角度查看关系。

依赖树可能很大, 必须围绕故障 unit 收缩。不要把一张完整系统树直接当作根因。

[Cheatsheet] `list-units` 看已加载对象; `list-unit-files` 看磁盘与启用状态; `status` 做人工入口; `is-*` 做单一判断; `show -p` 读精确属性; `cat` 看配置来源; `list-dependencies --reverse` 查谁会拉起它。

[操作专题] 当前控制、持久启用与强制阻断必须分开

服务管理命令可以改变当前活动状态，也可以改变下次引导时的依赖配置。两个维度互不替代。考试题目中的“现在启动并设置开机自动启动”实际上包含两个目标：“禁止服务再被依赖拉起”又比普通 `disable` 更强。

① [操作] `start`、`stop`、`restart` 与 `reload` 改变当前状态

```
systemctl start httpd.service
systemctl stop httpd.service
systemctl restart httpd.service
systemctl reload httpd.service
systemctl reload-or-restart httpd.service
```

- `start` 激活 unit；
- `stop` 请求停止 unit；
- `restart` 停止后重新启动，通常会替换进程并造成中断窗口；
- `reload` 只在 unit 或应用定义了重载能力时可用；
- `reload-or-restart` 在不能 reload 时回退到 `restart`。

修改应用配置前应优先使用应用自己的语法检查，例如 `httpd -t`。无调查地 `restart` 不能代替配置核对。

② [边界] `stop service` 不一定消除所有激活入口

停止 `service` 只改变该 unit 当前状态。如果 `socket`、`path`、`timer`、依赖 unit 或外部管理器仍然活动，服务可能稍后再次被激活。遇到“stop 后又起来”，先查：

```
systemctl show UNIT -p TriggeredBy,Restart,PartOf
systemctl list-dependencies --reverse UNIT
```

这与第 11 章的“进程被重新创建”不同：本章从 unit 的触发和重启策略解释管理者行为，不通过反复 `kill` 进程解决。

③ [操作] `enable` 与 `disable` 只改变持久安装链接

```
systemctl enable chronyd.service
systemctl disable chronyd.service
```

`enable` 根据 unit 的 `[Install]` 声明建立链接，例如把 unit 加入 `multi-user.target.wants/`；`disable` 移除由 `enable` 创建的链接。默认情况下，两者不启动或停止当前服务。

```
systemctl enable --now chronyd.service
systemctl disable --now chronyd.service
```

`--now` 把持久操作与当前 `start/stop` 组合。操作后仍分别验证：

```
systemctl is-enabled chronyd.service
systemctl is-active chronyd.service
```

④ [知识点] `static` 不能通过普通 `enable` 解决

`static` 通常表示 unit 没有 `[Install]` 安装声明，或不提供普通 `enable` 入口。它仍可能：

- 被某个 `target` 的依赖直接拉起；
- 由 `socket/path/timer` 激活；
- 被其他 `service` 的 `Wants/Requires` 拉起；
- 被管理员手工 `start`。

题目要求“开机自动运行 `static unit`”时，应先调查它设计上由谁拉起，而不是机械添加一个不理解的 `[Install]`。

⑤ [操作] `mask` 阻止普通激活，强度高于 `disable`

```
systemctl stop legacy-mail.service
systemctl mask legacy-mail.service
systemctl is-enabled legacy-mail.service
```

`mask` 通常使 unit 名称解析到 `/dev/null`，从而阻止手工、依赖和触发激活。边界：

- `mask` 不应被当作普通“不开机启动”；

- mask 不一定自动停止已经运行的 unit，应明确 stop 或使用支持的 `--now`；
- unmask 只恢复可激活能力，不会自动 enable 或 start；
- 发现 vendor unit 被 mask 时，先调查原因再解除。

⑥ [操作] reset-failed 清理记账，不修复故障

服务连续快速失败可能触发启动速率限制。完成最小修复后可执行：

```
systemctl reset-failed report-agent.service
systemctl start report-agent.service
```

`reset-failed` 清除 failed 状态和相关计数，但不会修正错误路径、权限、语法或应用配置。必须先建立证据并修复根因。

⑦ [验证点] 操作完成后按目标逐层验证

```
systemctl is-active UNIT
systemctl is-enabled UNIT
systemctl show UNIT -p MainPID,User,Result
```

然后根据服务功能验证端口、请求、文件或数据。`systemctl start` 返回 0 只说明 manager 接受并完成了该操作，不代表应用已经通过所有功能验收。

[Cheatsheet] 当前状态用 start/stop/restart/reload；持久配置用 enable/disable，双目标加 `--now`；static 查激活者；强制禁止才 mask；修复后可 reset-failed；任何命令成功都要回到当前、持久、身份和功能验证。

[知识专题] 依赖强度与启动顺序是两张独立的图

systemd 可以并行启动互不冲突的 unit。为了决定“哪些对象进入同一事务”和“它们按什么先后执行”，systemd 分别维护需求关系和顺序关系。把 `Requires=` 直接解释为“在它之后启动”，是本章必须纠正的高频误区。

① [知识点] Wants= 表示较弱的需求关系

当 A 写有：

```
[Unit]
Wants=B.service
```

启动 A 时，B 通常也会被加入启动事务；但 B 启动失败不会自动让 A 必然失败。它适合“希望一起提供，但不是 A 建立基本功能的硬前提”的对象。

② [知识点] Requires= 表示更强的生命周期需求

```
[Unit]
Requires=B.service
```

启动 A 时，B 会被加入事务。B 被显式停止或失活时，A 通常也会受到需求关系影响。若要让 B 启动失败阻止 A 继续进入活动状态，通常还需要与 `After=B.service` 配合，确保 A 等待 B 的启动结果。

因此，“Requires 比 Wants 强”是正确起点，但不能简化成“无论顺序如何，B 失败时 A 永远不可能启动”。

③ [知识点] After= 与 Before= 只定义顺序

```
[Unit]
After=network-online.target
```

这只表示两个 unit 都在同一事务中时，本 unit 的启动动作排在对方之后；它不会单独把 `network-online.target` 拉入事务。关闭时顺序反向：启动时 After 的 unit 通常先停止。

`Before=` 是同一顺序边的反向表达。不要同时在两个 unit 中重复写互相矛盾的顺序。

④ [知识点] 常见组合同时表达“拉入”和“等待”

```
[Unit]
Wants=network-online.target
After=network-online.target
```

这表示希望网络在线目标进入事务，并在其之后启动当前服务。是否应该使用 `network-online.target` 取决于程序是否真的要求“网络已经配置到可用状态”；普通本地服务不应无差别依赖它。

⑤ [知识点] [Unit] 与 [Install] 解决不同阶段的问题

- [Unit] 中的 Wants/Requires/After/Before 是运行时依赖和顺序；
- [Install] 中的 WantedBy=、RequiredBy=、Alias= 用于 enable 时建立链接；
- [Install] 本身不在每次启动时直接参与运行时解析，它是安装动作的说明。

例如：

```
[Install]
WantedBy=multi-user.target
```

执行 enable 后，通常形成由 multi-user.target.wants/ 指向该 service 的链接，于是引导进入目标时能够拉起服务。

⑥ [知识点] 隐式和默认依赖也会进入最终图

socket 与同名 service、timer 与被触发 service、mount 与父路径、target 的默认依赖等都可能自动建立关系。最终判断应以 manager 合并结果为准：

```
systemctl show UNIT -p Wants,Requires,After,Before
systemctl list-dependencies UNIT
systemctl list-dependencies --reverse UNIT
```

⑦ [诊断点] “服务太早启动”要同时检查两张图

症状：服务在网络、挂载或准备任务完成前启动并失败。

```
当前证据：status/show 中服务启动失败
→ 假设一：前置对象根本没有进入事务
→ 证据：Wants/Requires 与反向依赖
→ 假设二：对象进入事务但没有排序
→ 证据：After/Before
→ 最小修复：补充真正需要的需求和顺序，不无差别依赖大 target
→ 再验证：重新加载、重启相关 unit、核对事务和功能
```

[Cheatsheet] Wants/Requires 决定“是否一起做”；After/Before 决定“谁先谁后”；After 不会拉起对象；Requires 的失败传播要结合排序理解；[Install] 只指导 enable；最终图用 show 与依赖查询确认。

[知识专题] 服务为什么会回来：Socket、Path、Timer 与 Restart

看到 service 从 inactive 再次变成 active 时，不能只说“systemd 自动重启了”。重新激活至少有四种不同来源：连接触发、路径事件、时间触发、进程退出后的重启策略，以及其他 unit 的依赖操作。区分来源，才能选择最小修复。

① [知识点] socket 激活把“监听入口”和“工作进程”分开

.socket unit 可以先占有网络或 IPC socket，在连接到来时启动对应 service。优点是服务可以按需启动，并由 systemd 统一管理监听入口。

```
systemctl status report-gateway.socket report-gateway.service
systemctl show report-gateway.service -p TriggeredBy
```

只停止 service 而保持 socket listening，下一次连接就可能再次激活 service。

② [知识点] path 激活根据文件系统事件启动 service

.path unit 可以监视路径存在、变化或目录内容变化。典型场景是“某个队列目录出现新文件后执行处理服务”。服务停止后，只要 path unit 仍 active，下一次事件仍会拉起它。

③ [知识点] timer 激活 service，但时间表达式属于第 15 章

.timer unit 通常激活同名或明确指定的 .service。本章只训练：

- timer 与 service 是两个独立 unit；
- 停止 service 不会停止 timer；
- 查再激活原因时应检查 timer；
- 完整 OnCalendar=、Persistent=、随机延迟和验收归“一次性任务、周期任务与 systemd Timer”。

④ [知识点] Restart= 处理进程退出，不是事件激活

常用策略：

- no：不自动重启；

- `on-failure`：非零退出、信号或超时等失败条件下重启；
- `always`：无论正常或异常退出都重启，但正常的 `manager stop` 不应被理解为进程自行退出；
- `on-abnormal`、`on-abort` 等用于更细边界。

`RestartSec=` 控制尝试间隔。重启策略还受启动速率限制影响。人工执行 `systemctl stop` 的语义与进程崩溃不同，不能用 `kill MainPID` 模拟正常停服。

⑤ [操作] 用属性把激活来源分开

```
systemctl show report-gateway.service \
    -p ActiveState,SubState,TriggeredBy,Restart,RestartUSec,NRestarts

systemctl list-dependencies --reverse report-gateway.service
systemctl status report-gateway.socket report-gateway.path report-gateway.timer
```

`NRestarts` 可帮助判断 `manager` 是否根据 `Restart` 重启，但属性可用性和展示形式需要在目标 RHEL 9 环境确认；本章不编造具体输出。

⑥ [边界] 最小修复只关闭真正不需要的入口

若要求“停止按连接启动，但仍允许管理员手工启动 `service`”，应 `stop/disable socket`，而不是 `mask service`。若要求“无论依赖还是手工都不得启动”，才考虑 `mask`，并先评估依赖影响。

只停 <code>service</code>	→ 当前进程停止，触发入口可能仍在
停并禁用 <code>socket/path</code>	→ 关闭指定事件入口， <code>service</code> 仍可手工启动
修改 <code>Restart</code> 策略	→ 改变进程退出后的 <code>manager</code> 行为
<code>mask service</code>	→ 强制阻断普通激活，影响最大

[Cheatsheet] `stop` 后再 `active`：查 `TriggeredBy`、反向依赖和 `Restart`；`socket/path/timer` 是外部事件入口，`Restart` 是进程退出策略；关闭哪个入口取决于目标，不默认 `mask` 全部。

[知识专题] Unit 从哪里加载：搜索路径、优先级和合并结果

真实服务器中的 `unit` 经常不是一个文件。软件包提供主文件，管理员添加 `drop-in`，生成器根据其他配置产生运行时 `unit`，某些工具再创建 `transient` 定义。排错的核心不是背出所有目录，而是理解“高优先级来源覆盖低优先级来源，`drop-in` 在主文件之后合并”。

① [知识点] 三个常用系统目录构成管理员心智模型

<code>/etc/systemd/system</code>	管理员持久配置，优先级高
<code>/run/systemd/system</code>	当前启动周期的运行时配置
<code>/usr/lib/systemd/system</code>	软件包或发行版提供的 <code>vendor unit</code>

RHEL 系统的完整搜索路径还可能包含 `generator`、`transient` 和 `control` 目录。需要确认当前环境时使用：

```
systemd-analyze unit-paths
```

② [知识点] 同名完整 unit 按高优先级来源替换

如果 `/etc/systemd/system/foo.service` 存在，它会遮蔽较低优先级的同名 `vendor` 主文件。完整复制 `vendor unit` 到 `/etc` 虽然有效，但会让后续软件包新增指令不再自动进入管理员副本，因此只在 `drop-in` 无法表达完整结构替换时采用。

③ [知识点] drop-in 在主文件之后按名称合并

常见位置：

```
/etc/systemd/system/foo.service.d/*.conf
/run/systemd/system/foo.service.d/*.conf
/usr/lib/systemd/system/foo.service.d/*.conf
```

高优先级目录中的同名 `drop-in` 覆盖低优先级同名文件；多个不同名称的 `drop-in` 按词法顺序合并。管理员应使用清晰前缀，例如 `10-restart.conf`、`20-environment.conf`，但不要依赖难以审计的文件名技巧隐藏语义。

④ [知识点] FragmentPath 和 DropInPaths 是实际来源证据

```
systemctl show foo.service -p FragmentPath,DropInPaths,SourcePath
systemctl cat foo.service
```

- `FragmentPath` 指向主 `unit` 来源；

- `DropInPaths` 列出 manager 采用的 drop-in;
- `SourcePath` 在由其他配置生成时可能提供来源线索。

⑤ [知识点] unit 定义与应用配置是两套加载链

```
/etc/systemd/system/foo.service.d/override.conf → systemd manager 读取
/etc/foo/foo.conf                               → foo 进程读取
```

修改前者需要 manager 重新读取 unit 定义; 修改后者通常需要应用 reload/restart。不要因为应用配置变化就无条件 `daemon-reload`, 也不要 unit 变化后只重启应用却忽略 manager 的旧定义。

⑥ [诊断点] 磁盘内容与 manager 状态不一致时先确认 reload

症状: 已经修改 `ExecStart=` 或 `User=`, 但服务仍按旧方式运行。

```
当前证据: cat 文件看到新内容, show 属性或进程仍是旧内容
→ 假设: manager 尚未重新加载, 或修改了低优先级被覆盖的文件
→ 下一证据: systemctl cat; show FragmentPath/DropInPaths; unit-paths
→ 最小修复: 修改正确来源并 daemon-reload
→ 再验证: cat/show 后 restart 服务, 核对 PID、UID 和功能
```

[Cheatsheet] 管理员持久配置在 `/etc`, 运行时在 `/run`, vendor 在 `/usr/lib`; 完整 unit 替换风险高, 局部变更优先 drop-in; 实际来源看 `FragmentPath/DropInPaths`; unit 与应用配置有不同 reload 链。

[操作专题] 用 Drop-in 做最小覆盖, 并让 Manager 读取新定义

直接编辑 vendor unit 会把本地修改与软件包内容混在一起, 升级时难以审计。更稳定的方式是先读取最终定义, 再使用 drop-in 只覆盖必要字段, 完成静态检查、重新加载和分层验证。

① [操作] 变更前保存定义基线

```
systemctl cat report-agent.service
systemctl show report-agent.service \
    -p FragmentPath,DropInPaths,Type,User,ExecStart,Restart
```

基线回答: 当前使用哪个主文件、已有多少 drop-in、准备覆盖的属性是什么。不要在不知道原始 `ExecStart=` 的情况下直接写覆盖。

② [操作] systemctl edit 创建管理员 drop-in

```
systemctl edit report-agent.service
```

编辑器中只写需要覆盖的部分:

```
[Service]
Restart=on-failure
RestartSec=5s
```

默认目标路径通常是:

```
/etc/systemd/system/report-agent.service.d/override.conf
```

`systemctl edit --runtime` 会写入运行时目录, 重启后消失: 普通考试持久变更不应误用。 `--full` 创建完整 unit 副本, 只有 drop-in 无法表达需求时使用。

③ [知识点] 标量覆盖与列表型指令重置方式不同

`Restart=`、`User=` 等单值属性通常由后出现的值覆盖。 `ExecStart=`、`Environment=` 等可多次出现或具有列表语义的指令, 不能总靠再写一行替换。

覆盖已有 `ExecStart=` 的常见形式:

```
[Service]
ExecStart=
ExecStart=/usr/local/libexec/report-agent --config /etc/report-agent.conf
```

第一行空赋值清除旧列表, 第二行建立新值。省略清空可能导致多个 `ExecStart=` 冲突, 尤其对非 oneshot service。

④ [知识点] ExecStart= 默认不经过交互 Shell

下面写法不会自动获得 Bash 的管道、重定向、&&、~ 和普通环境变量展开语义：

```
ExecStart=/usr/bin/echo hello > /var/lib/app/output
```

更稳妥的选择：

- 直接调用能够完成目标的程序及参数；
- 使用 StandardOutput= 等 systemd 指令；
- 复杂逻辑放入可审计脚本，再以绝对路径执行；
- 只有明确需要 Shell 语义时才写 /bin/bash -c '...'，并处理引用与退出状态。

⑤ [操作] 用 systemd-analyze verify 做静态检查

```
systemd-analyze verify /etc/systemd/system/report-agent.service
```

对 drop-in 场景，也可以把相关 unit 交给 verify，随后仍以 manager 实际加载结果为准。静态检查可以发现未知指令、缺少可执行文件、依赖引用等问题，但不能证明用户权限、运行目录、端口和业务逻辑都正确。

⑥ [操作] unit 定义变化后执行 daemon-reload

```
systemctl daemon-reload
systemctl cat report-agent.service
systemctl show report-agent.service -p DropInPaths,Restart,RestartUSec
```

daemon-reload 让 systemd 重新读取 unit 和 generator 输出。它不会自动重启已运行服务，也不会让应用读取自己的配置文件。下一步是否 restart 取决于变更字段和业务允许的中断窗口。

⑦ [操作] 撤销管理员覆盖前先确认影响

```
systemctl revert report-agent.service
```

revert 可删除由 systemctl 管理的本地覆盖并恢复 vendor 状态。执行前应备份需要保留的自定义内容，并确认不会同时删除其他管理员仍依赖的 drop-in。完成后重新加载、重新启动并分层验证。

[Cheatsheet] 先 cat/show 建基线；局部变更用 edit；列表型 ExecStart= 先空赋值；ExecStart 默认没有 Shell；verify 只做静态检查；unit 变更后 daemon-reload；最后 cat/show + restart + 功能验证。

[操作专题] 创建自定义 Service：Type、ExecStart、User 与 Restart

自定义 service 的目标不是“写一个能被 systemctl 接受的文件”，而是让 manager 能清楚识别主进程、运行身份、工作目录、成功与失败边界、重启策略和持久激活方式。越少依赖隐式 Shell 环境，unit 越容易验证和自动化。

① [知识点] 三个 section 各自承担一层职责

```
[Unit]
Description=...
Wants=...
After=...

[Service]
Type=...
User=...
ExecStart=...
Restart=...

[Install]
WantedBy=multi-user.target
```

- [Unit]：描述对象和运行时关系；
- [Service]：定义进程生命周期与执行环境；
- [Install]：说明 enable 时建立哪些链接。

Description= 便于人工识别，但不决定运行行为。

② [知识点] 根据程序行为选择 `Type=`

Type	manager 如何判断启动完成	典型对象
<code>simple</code>	启动 <code>ExecStart</code> 后立即认为已启动	前台长期程序, RHCSA 常用
<code>exec</code>	成功执行目标程序后才认为启动完成	支持该版本时可更早发现 <code>exec</code> 失败
<code>oneshot</code>	等待命令退出, 成功后完成	初始化、检查、一次性变更
<code>forking</code>	期望程序 <code>fork</code> 并让父进程退出	传统自守护程序, 常需 <code>PIDFile</code>
<code>notify</code>	程序主动发送就绪通知	原生支持 <code>sd_notify</code> 的服务

不要仅凭“守护进程”三个字使用 `forking`。现代程序若可以前台运行, 通常更适合由 `systemd` 直接拥有主进程。

③ [操作] 为长期前台程序建立基本 `service`

```
# /etc/systemd/system/heartbeat-worker.service
[Unit]
Description=Heartbeat file worker
After=local-fs.target

[Service]
Type=simple
User=svcwatch
Group=svcwatch
WorkingDirectory=/var/lib/heartbeat-worker
ExecStart=/usr/local/libexec/heartbeat-worker
Restart=on-failure
RestartSec=5s

[Install]
WantedBy=multi-user.target
```

关键点:

- `ExecStart` 使用绝对路径;
- 目标用户必须存在;
- 工作目录和写入目录必须允许该用户访问;
- `Restart=on-failure` 只在失败条件下重启;
- `WantedBy` 让 `enable` 有明确目标。

④ [操作] 为一次性任务使用 `Type=oneshot`

```
[Service]
Type=oneshot
User=backup
ExecStart=/usr/local/sbin/prepare-backup
```

任务成功退出后通常回到 `inactive`。只有后续 `unit` 需要把“已经完成”作为活动状态时, 才考虑 `RemainAfterExit=yes`。不要为了让 `status` 显示 `active` 而无条件添加它; 真正验收应检查 `Result`、退出状态和任务产物。

⑤ [知识点] `User=` 改变执行身份, 但不会自动准备资源

系统 `service` 中的 `User=svcwatch` 表示由系统 `PID 1` 以该账号启动进程。它不会自动:

- 创建账号;
- 创建工作目录;
- 修改文件所有权;
- 绕过传统权限或 `SELinux`;
- 提供交互 `Shell` 环境。

```
id svcwatch
namei -om /var/lib/heartbeat-worker
```

`SELinux` 上下文和 `AVC` 证据的完整处理归第 28、29 章; 本章只在权限正确仍失败时指出安全层边界。

⑥ [知识点] Restart= 要与退出语义和启动速率一起设计

```
Restart=on-failure
RestartSec=5s
```

- 对预期长期运行的 worker，异常退出可自动恢复；
- 对一次性任务，盲目重启可能形成重复副作用；
- 对配置永久错误，快速重启只会制造启动风暴；
- systemd 的启动速率限制会在短时间多次失败后阻止继续启动。

诊断时结合 Result、退出状态、NRestarts 和失败记账，不把“服务正在反复重启”误解为高可用。

⑦ [操作] 自定义 unit 的推荐部署链

```
systemd-analyze verify /etc/systemd/system/heartbeat-worker.service
systemctl daemon-reload
systemctl enable --now heartbeat-worker.service
systemctl is-active heartbeat-worker.service
systemctl is-enabled heartbeat-worker.service
```

随后确认：

```
systemctl show heartbeat-worker.service \
-p FragmentPath,ActiveState,SubState,UnitFileState,MainPID,User,Result

pid=$(systemctl show heartbeat-worker.service -p MainPID --value)
ps -p "$pid" -o pid,user,group,etimes,cmd
```

最后通过心跳文件时间、端口或真实请求验证功能。

⑧ [验证点] 失败时从最有区分度的层继续

症状	下一条证据
Unit not found	名称、路径、daemon-reload、FragmentPath
status=203/EXEC 等执行失败	ExecStart 绝对路径、权限、解释器和文件格式
status=217/USER 等身份失败	用户/组是否存在，User/Group 拼写
permission denied	目录遍历、文件权限、SELinux 证据
active 但无产物	实际 MainPID、用户、WorkingDirectory、应用行为
反复重启	Result、退出状态、Restart、启动速率

具体数字状态以目标环境的 status 和 man page 为准，讲义不伪造真实输出。

[Cheatsheet] [Unit] 管关系、[Service] 管进程、[Install] 管 enable；长期前台程序常用 simple，短任务用 oneshot；ExecStart 用绝对路径；User 不会自动准备目录；Restart 要防启动风暴；部署链 verify → daemon-reload → enable --now → 五层验收。

[知识专题] Target 与 Default Target: 组织依赖，而不是运行一个守护进程

.target unit 通常没有自己的长期进程。它提供一个可命名的依赖集合和同步点，让系统在引导、救援、图形界面或多用户环境中激活一组对象。target 的核心仍是依赖关系，不应把它理解成传统 SysV“运行级别进程”。

① [知识点] target 把一组 unit 组织成可激活目标

常见对象：

- multi-user.target：非图形多用户系统的主要服务集合；
- graphical.target：在多用户基础上加入图形环境；
- rescue.target、emergency.target：恢复环境，完整操作归第 30 章；
- network-online.target：表示网络已达到在线语义、由相关等待服务实现。

② [操作] 查询和设置持久默认目标

```
systemctl get-default
systemctl set-default multi-user.target
```

`set-default` 通常改变 `default.target` 的持久链接。它不等于立即切换当前系统状态。操作后应再次 `get-default` 并检查链接来源。

③ [知识点] start target 与 isolate target 的影响不同

```
systemctl start multi-user.target
```

`start` 将目标及其依赖加入当前事务，但不主动停止所有与目标无关的 `unit`。

```
systemctl isolate multi-user.target
```

`isolate` 会启动目标并停止不属于新目标依赖图的其他 `unit`，可能断开图形会话、网络或当前管理入口。它是高影响操作，必须确认远程访问和回退方式，不能作为普通“查看目标”的命令。

④ [知识点] target 的 wants/requires 目录是 enable 的常见结果

```
/etc/systemd/system/multi-user.target.wants/foo.service
```

这个链接表达 `target` 对 `foo` 的 `Wants` 关系。不要把目录中链接的存在直接解释为当前 `active`；它只证明持久依赖配置。当前状态仍用 `is-active`。

⑤ [边界] 恢复目标和启动链深度留给第 30 章

本章只建立 `target`、默认目标和 `isolate` 的对象模型。内核参数、GRUB、`initramfs`、`rescue/emergency` 登录、根文件系统修复和启动链故障归《启动链、GRUB、Target 与系统恢复》。

[Cheatsheet] `target` 是依赖集合；`get/set-default` 管持久默认目标；`start` 不等于 `isolate`；`isolate` 会停止不属于新图的 `unit`，风险高；`wants` 目录证明持久关系，不证明当前 `active`。

[知识专题] 系统 Manager、用户 Manager 与 `loginctl`

RHEL 9 同时可以运行系统级 `systemd manager` 和每个用户自己的 `user manager`。两者管理范围、`unit` 搜索路径、权限和生命周期不同。一个服务以普通用户身份运行，不等于它属于 `user manager`。

① [知识点] 系统 service 的 `User=` 仍由系统 PID 1 管理

```
[Service]
User=svcmwatch
```

该进程以 `svcmwatch` 身份运行，但 `unit` 仍由系统 `manager` 管理：

```
systemctl status heartbeat-worker.service
```

它适合系统范围服务和开机阶段服务，不依赖该用户登录。

② [知识点] `systemctl --user` 操作独立的用户 manager

```
systemctl --user list-units
systemctl --user status my-worker.service
```

`user unit` 常见管理员和用户路径与系统 `unit` 不同。执行命令需要正确用户会话、运行时目录和用户 `bus`。不要用 `root` 的环境机械代替目标用户执行后就声称用户服务可用。

③ [操作] `loginctl` 查询会话、用户 manager 和 `linger`

```
loginctl list-users
loginctl user-status USER
loginctl show-user USER -p State,Linger,Sessions
```

`Linger=yes` 表示该用户的 `manager` 可以在没有活动登录会话时保留，并可在引导时启动。它常用于需要跨登出运行的用户服务。

④ [边界] `linger` 不是普通系统 service 的必需条件

系统 `manager` 中配置 `User=` 的 `service` 不依赖 `linger`。只有 `unit` 真正属于 `systemctl --user` 管理范围且要求退出登录后继续运行时，才调查 `linger`。

`rootless Podman` 的用户 `unit`、`Quadlet` 和容器持久化归第 31、32 章；本章只建立 `user manager` 接口。

⑤ [验证点] 用户服务需要跨身份和跨会话验证

```
目标用户下 systemctl --user is-active/is-enabled
→ logindctl show-user 检查 Linger 与 State
→ 退出登录或重启后的持久性测试（需 live VM）
→ 应用功能验证
```

跨登出或重启后的持久性必须在目标 RHEL 9 主机上实际验证；当前状态与 Linger 属性不能代替该验收。

[Cheatsheet] 系统 service 中 User= 仍归 PID 1；systemctl --user 操作用户 manager；logindctl show-user 查 State/Linger；只有用户 manager 要跨登出运行时才需要 linger；容器具体流程留给 Podman 章节。

[诊断专题] 从症状到下一条证据：Unit 故障诊断链

systemd 排错的目标不是尽快找到一个能让状态变绿的命令，而是从当前证据区分：定义未加载、依赖错误、执行身份错误、程序退出、激活入口仍在、启动速率限制或功能层失败。每次只选择最能区分当前假设的下一条证据。

① [诊断] Unit ... could not be found

```
症状: start/status 报 unit 不存在
→ 当前证据: systemctl show UNIT -p LoadState,FragmentPath
→ 假设: 名称写错、文件不在搜索路径、主文件后缀错误、尚未 daemon-reload
→ 下一证据: systemd-analyze unit-paths; find 正确路径; systemctl cat
→ 最小修复: 修正名称或放入管理员路径, daemon-reload
→ 再验证: show LoadState/FragmentPath, 然后 start
```

不要通过复制另一个不相关 unit 或随意创建空文件掩盖名称错误。

② [诊断] Loaded: masked

```
症状: unit 存在但拒绝启动
→ 当前证据: is-enabled 返回 masked; FragmentPath 可能指向 /dev/null
→ 假设: 管理员或软件包有意屏蔽, 或遗留链接
→ 下一证据: ls -l 相关 unit 路径; 变更记录和依赖影响
→ 最小修复: 只有确认应恢复时 unmask
→ 再验证: is-enabled、start、持久配置与功能
```

unmask 不会自动 enable/start。

③ [诊断] enabled 但当前 inactive

可能原因：

- 当前系统尚未进入拉起它的 target；
- enable 后未加 --now，当前未启动；
- 服务启动失败后回到 inactive/failed；
- unit 受 Condition/Assert 约束；
- 依赖没有满足；
- 它是按需激活而非长期运行对象。

```
systemctl show UNIT \
    -p ActiveState,SubState,UnitFileState,Result,Conditions,ConditionResult
systemctl list-dependencies --reverse UNIT
```

不要反复 enable；先确认当前层发生了什么。

④ [诊断] active 后很快 failed 或反复重启

```
当前证据: ActiveState/SubState/Result/ExecMainStatus/Restart/NRestarts
→ 假设: 程序立即退出、配置错误、权限错误、Restart 导致循环
→ 下一证据: 实际 ExecStart、目标用户、工作目录、应用语法检查、unit 日志
→ 最小修复: 修正根因；必要时 stop 并 reset-failed
→ 再验证: 稳定运行时间、NRestarts 不继续增长、功能成立
```

完整日志过滤归第 13 章，但本章必须知道 status 的日志片段不是全部证据。

⑤ [诊断] stop 后再次 active

当前证据: TriggeredBy、Restart、反向依赖、相关 socket/path/timer 状态
→ 假设一: 事件 unit 再激活
→ 假设二: 进程退出触发 Restart
→ 假设三: 其他 target/service 再次启动它
→ 最小修复: 关闭真正不需要的入口或调整策略
→ 再验证: 重复触发条件, 确认目标行为

不要直接 mask service, 除非目标确实要求阻止所有普通激活。

⑥ [诊断] 修改 unit 后仍按旧命令或旧用户运行

当前证据: systemctl cat 与进程实际命令/UID
→ 假设: 修改了被覆盖的文件、drop-in 未加载、服务没有 restart
→ 下一证据: FragmentPath/DropInPaths; show ExecStart/User/MainPID
→ 最小修复: 改正正确来源、daemon-reload、按影响 restart
→ 再验证: 新 MainPID 的 user/cmd 与功能

⑦ [诊断] 依赖已写, 但服务仍早于前置对象

当前证据: show Wants/Requires/After/Before
→ 假设: 只有需求没有顺序, 或只有顺序没有拉入事务
→ 下一证据: list-dependencies 与排序视图
→ 最小修复: 补充最小必要关系
→ 再验证: 重新加载后执行受控启动, 观察两个 unit 的状态和业务结果

⑧ [诊断] active 且 enabled, 但业务不可用

此时 systemd 层已经提供两个正向证据, 下一步应转到功能链, 而不是继续 enable/restart:

监听端口或 socket 是否存在
→ 本机请求是否成功
→ 目标文件/数据是否持续更新
→ 应用权限和安全策略是否允许
→ 上游依赖是否可用

网络、firewalld、SELinux 和日志的深度分别属于其自然章节。

[Cheatsheet] 先 status/show 建基线; not-found 查名称、路径和 reload; masked 先查原因; enabled≠active; 反复重启查 Result/Restart; stop 后回来查触发和反向依赖; active+enabled 后转入功能证据。

[经典任务] 创建受限用户运行的 Heartbeat Service

环境

系统已经提供可执行程序:

```
/usr/local/libexec/heartbeat-worker
```

程序以前台方式长期运行, 每隔数秒更新:

```
/var/lib/heartbeat-worker/last-update
```

当前系统中:

- 不存在 heartbeat-worker.service;
- 不存在账号 svcwatch;
- 不存在数据目录 /var/lib/heartbeat-worker;
- 程序不需要网络;
- 不允许修改 /usr/lib/systemd/system。

目标终态

1. 创建不可交互登录的系统账号 svcwatch;
2. 程序必须由 svcwatch 运行, 不得以 root 运行;
3. unit 名为 heartbeat-worker.service;

- 4. 使用 `Type=simple` ;
- 5. `ExecStart=/usr/local/libexec/heartbeat-worker` ;
- 6. 异常退出时使用 `Restart=on-failure` , 等待 5 秒再重启;
- 7. 当前立即运行;
- 8. 引导进入 `multi-user.target` 时自动启动;
- 9. 心跳文件必须能被服务账号持续更新;
- 10. 不把“active”当作唯一验收证据。

限制条件

- 不使用 `chmod 777` ;
- 不关闭 SELinux;
- 不在 unit 中使用交互 Shell 的重定向技巧;
- 不通过 `root crontab` 或后台 `nohup` 代替 service;
- 不编造命令输出。

验收矩阵

层	验收要求
定义	主文件位于管理员路径, verify 无阻断性问题, FragmentPath 正确
当前	ActiveState/SubState 符合长期运行服务
持久	UnitFileState 为 enabled
身份	MainPID 的实际用户为 svcwatch
策略	Restart=on-failure , RestartSec=5s
功能	两次间隔读取心跳文件, 更新时间发生推进

[经典任务] 诊断 Service 停止后被重新激活

环境

系统存在:

```
report-gateway.socket
report-gateway.service
```

已知 service 的进程长期运行, unit 中配置 `Restart=on-failure` 。管理员观察到:

- 1. 执行 `systemctl stop report-gateway.service` 后, service 变为 inactive;
- 2. 访问对应 socket 后, service 再次 active;
- 3. 直接向 MainPID 发送致命信号后, 即使没有新连接, service 也会再次出现。

目标终态

- 分别证明第一次再激活来自 socket, 第二次来自 Restart 策略;
- 停止并禁用 socket 激活;
- 保留管理员手工启动 service 的能力;
- 保留 `Restart=on-failure` ;
- 不 mask service;
- 验证新连接不再自动拉起, 但手工 start 后 service 可正常工作。

验收矩阵

目标	证据
激活来源	TriggeredBy、Restart、NRestarts、反向依赖
socket 入口关闭	socket inactive 且 disabled
service 可手工启动	未 masked, 手工 start 成功
重启策略保留	Restart=on-failure

目标	证据
功能	手工启动后按题目给定方式访问成功

[参考解答] 经典任务一：Heartbeat Service

以下命令为静态推荐流程。账号参数和程序行为应以题目真实附件为准。

① 调查程序和现状

```
ls -l /usr/local/libexec/heartbeat-worker
file /usr/local/libexec/heartbeat-worker
getent passwd svcwatch
systemctl status heartbeat-worker.service --no-pager -l
```

确认程序存在且可执行，unit 和账号确实不存在。若程序是脚本，还要确认首行解释器存在。

② 创建受限系统账号和数据目录

```
useradd --system --home-dir /var/lib/heartbeat-worker \
--shell /sbin/nologin svcwatch

install -d -o svcwatch -g svcwatch -m 0750 /var/lib/heartbeat-worker
```

`install -d` 一次明确目录、所有者和权限，避免先创建再遗漏 `chown`。不要使用 `777`。

③ 创建 unit

```
cat > /etc/systemd/system/heartbeat-worker.service <<'EOF'
[Unit]
Description=Heartbeat file worker
After=local-fs.target

[Service]
Type=simple
User=svcwatch
Group=svcwatch
WorkingDirectory=/var/lib/heartbeat-worker
ExecStart=/usr/local/libexec/heartbeat-worker
Restart=on-failure
RestartSec=5s

[Install]
WantedBy=multi-user.target
EOF
```

这里没有无意义的网络依赖：服务只依赖本地文件系统和已经准备好的目录。

④ 静态检查并让 manager 读取定义

```
systemd-analyze verify /etc/systemd/system/heartbeat-worker.service
systemctl daemon-reload
systemctl cat heartbeat-worker.service
systemctl show heartbeat-worker.service \
-p FragmentPath,DropInPaths,Type,User,ExecStart,Restart,RestartUsec
```

若 `verify` 报告阻断性问题，应先修复，不要通过不断 `restart` 猜测。

⑤ 当前启动并配置持久激活

```
systemctl enable --now heartbeat-worker.service
```

⑥ 分层验证

```
systemctl is-active heartbeat-worker.service
systemctl is-enabled heartbeat-worker.service

systemctl show heartbeat-worker.service \
    -p LoadState,ActiveState,SubState,UnitFileState,MainPID,Result,User,Restart,RestartUsec

pid=$(systemctl show heartbeat-worker.service -p MainPID --value)
ps -p "$pid" -o pid,user,group,etimes,cmd
```

功能验证应读取两次真实时间或文件内容：

```
stat /var/lib/heartbeat-worker/last-update
sleep 6
stat /var/lib/heartbeat-worker/last-update
```

验收重点不是输出长什么样，而是第二次修改时间晚于第一次，且文件所有者、服务进程身份和 unit 状态符合要求。

⑦ 典型错误与修复方向

错误	证据	最小修复
ExecStart 路径错	status/Result、文件检查	改绝对路径、daemon-reload
账号不存在	getent、status 身份错误	创建正确系统账号
目录不可写	namei、ls、应用错误	修正最小所有权和权限
unit 已改但仍旧行为	cat/show 与进程不一致	daemon-reload 后 restart
active 但文件不更新	MainPID、用户、工作目录、应用行为	转入功能和安全证据
重启风暴	Restart/NRestarts/Result	stop，修正根因，reset-failed

[参考解答] 经典任务二：区分 Socket 激活与 Restart

① 建立初始证据

```
systemctl status report-gateway.service report-gateway.socket --no-pager -l

systemctl show report-gateway.service \
    -p ActiveState,SubState,UnitFileState,MainPID,TriggeredBy,Restart,RestartUsec,NRestarts

systemctl list-dependencies --reverse report-gateway.service
systemctl cat report-gateway.service report-gateway.socket
```

若 TriggeredBy 指向 socket，并且 socket 处于 listening，则连接触发假设成立；Restart=on-failure 则为异常退出后的另一条机制。

② 证明 socket 激活

```
systemctl stop report-gateway.service
systemctl is-active report-gateway.service
```

按题目给定方式发起一次连接，然后再次检查 service。不能编造测试地址或端口，必须使用任务附件提供的 socket 参数。

③ 证明 Restart 策略

先记录重启计数与 MainPID：

```
systemctl show report-gateway.service -p MainPID,NRestarts,Restart
```

在隔离测试环境中按题目要求模拟异常退出，再次查询 MainPID 与 NRestarts。生产环境不应为了证明机制随意杀业务进程；本章任务是受控实验设计。

④ 关闭指定激活入口

```
systemctl disable --now report-gateway.socket
systemctl is-active report-gateway.socket
systemctl is-enabled report-gateway.socket
```

不 mask service，因此管理员仍可手工启动：

```
systemctl is-enabled report-gateway.service
systemctl start report-gateway.service
systemctl is-active report-gateway.service
```

⑤ 再验证目标行为

- 1. stop service;
- 2. 发起连接，确认 socket 不再自动拉起；
- 3. 手工 start service；
- 4. 确认功能访问成功；
- 5. 查询 Restart=on-failure 未被修改。

```
systemctl show report-gateway.service -p Restart,TriggeredBy
```

⑥ 为什么不使用 mask

mask 会阻止管理员手工 start，也会改变题目明确要求保留的能力。最小修复是只关闭 socket 入口。只有目标是“任何普通方式都不得启动 service”时，才评估 mask。

[本章收束] 把 systemd 操作还原为对象和证据

本章的核心不是记住更多子命令，而是把每个动作定位到正确层：

- unit 名称与类型
- 加载来源和合并结果
- Wants/Requires 与 After/Before
- ActiveState/SubState/Result
- enabled/disabled/static/masked
- service 的 MainPID、User、ExecStart 与 Restart
- 触发入口和 default target
- 对外功能验收

遇到故障时，先问“当前缺少哪一层证据”，再选择 status、show、cat、依赖查询、systemd-analyze 或 loginctl。只有 unit 定义发生变化时才需要 manager 重新加载；只有目标明确要求当前与持久同时改变时才组合 --now；只有需要阻断所有普通激活时才 mask。

为后续章节保留的接口：日志的完整检索进入《系统日志、Journal、rsyslog 与日志轮转》；timer 的调度表达进入《一次性任务、周期任务与 systemd Timer》；恢复目标与引导链进入《启动链、GRUB、Target 与系统恢复》；用户容器服务与 Quadlet 进入容器篇。

主要判断表

看到的证据	可以得出的结论	仍不能证明	下一条证据
LoadState=loaded	manager 已获得可用定义	当前已运行、定义一定符合业务目标	查 ActiveState、FragmentPath 与 drop-in
ActiveState=active	unit 达到自身活动语义	开机自动启动、端口或服务可用	查 UnitFileState 与功能结果
UnitFileState=enabled	已建立持久激活关系	当前已经运行、依赖目标当前成立	查 is-active 与目标依赖
SubState=exited	类型专用状态为已退出	一定异常或一定存在守护进程	查 Type、RemainAfterExit、Result
Restart=on-failure	异常退出可能触发重启	socket/path/timer 不会再次激活	查 TriggeredBy 与反向依赖
systemd-analyze verify 无阻断错误	静态语法和部分引用可接受	运行时权限、环境与业务功能正确	实际启动并做身份、功能验收

工作方法与向下一章交接

处理 systemd 问题时，先固定 unit 的完整名称，再按“来源 - 状态 - 关系 - 进程 - 功能”收集证据。修改时优先做最小覆盖，修改后只在定义层发生变化时执行 daemon-reload，再根据目标选择 start、restart 或应用自身的 reload。不要在一次 restart 掩盖对象、依赖或权限问题。

本章到此只使用 systemctl status 中的少量近期信息作为诊断入口。下一章《系统日志、Journal、rsyslog 与日志轮转》将接手完整日志检索、启动轮次、时间范围、优先级与持久化问题；timer 的日历表达式与执行验收则留给第 15 章。

终章 Cheatsheet

```
# 对象、状态、来源
systemctl status UNIT --no-pager -l
systemctl show UNIT -p LoadState,ActiveState,SubState,UnitFileState,MainPID,Result
systemctl cat UNIT

# 当前与持久
systemctl start|stop|restart|reload UNIT
systemctl enable|disable UNIT
systemctl enable --now UNIT
systemctl mask|unmask UNIT

# 关系与激活
systemctl list-dependencies UNIT
systemctl list-dependencies --reverse UNIT
systemctl show UNIT -p Wants,Requires,After,Before,TriggeredBy,Restart

# 定义与静态检查
systemd-analyze unit-paths
systemd-analyze verify /etc/systemd/system/UNIT
systemctl daemon-reload

# 目标与用户 manager
systemctl get-default
systemctl set-default multi-user.target
loginctl show-user USER -p State,Linger,Sessions
```

最终判断:

```
定义正确
+ 当前状态正确
+ 持久状态正确
+ 进程身份正确
+ 功能结果正确
= 本章任务终态成立
```

∴ {cover}

RHEL 9 · RHCSA 实操讲义

13

系统日志、Journal、

rsyslog 与日志轮转

从“事件发生”到“证据可查”：把范围、字段、路由、持久化与轮转放进同一条证据链。
对象模型操作语义验证诊断经典任务
大字号阅读版

...

... {navigation}

本章阅读导航

先抓住一条主线：日志不是某一个文件，而是事件从产生、收集、筛选、路由、保存到轮转的完整证据链。每次查询或变更，都要先回答“我正在证明哪一层”。

∴ {model-grid} ∴ {model-step} 01

识别事件来源

内核、unit、syslog 客户端，还是应用自有文件？ ∴ ∴ {model-step} 02

限定 journal 范围

先定 boot 与时间，再定 unit、tag、PID 或字段。 ∴ ∴ {model-step} 03

解释结构化字段

区分消息正文、客户端字段和 journal 可信字段。 ∴ ∴ {model-step} 04

验证路由规则

用唯一事件证明 selector 命中并完成 action。 ∴ ∴ {model-step} 05

判断持久与保留

当前可读、跨 boot 存在、尚未被清理是不同状态。 ∴ ∴ {model-step} 06

验证文本轮转

先 dry-run，再在受控对象上轮转并验证新写入。 ∴ ∴

专题地图 - **知识专题** 从事件产生到可审计证据 - **操作专题** 按范围和字段收缩 journal - **操作专题** 解释结构化字段与输出格式 - **操作专题** 建立持久 journal 与保留预算 - **操作专题** 用 `logger` 构造验证事件 - **操作专题** 配置 rsyslog 本地路由 - **操作专题** 按层验证远端转发 - **操作专题** 管理 logrotate 状态与策略 - **诊断专题** 日志查不到、写不到、转不到和轮不动 - **经典任务** facility 路由与轮转；跨 boot 持久性

阅读时持续回答 1. 目标日志可能存在于哪个范围？ 2. 当前查询限定了哪个 boot 和时间窗口？ 3. 哪个字段能够唯一缩小对象？ 4. 这条证据来自客户端，还是 journal 补充？ 5. selector 命中后执行了什么 action？ 6. 当前可读是否等于重启后仍可读？ 7. logrotate 为什么认为“本次不轮转”？ 8. 操作结果能证明什么，又不能证明什么？ ∴ {note-box} 本章使用 `systemctl` 观察和应用日志服务变更，但不重新展开 unit 状态模型；时间戳异常只建立排查接口，时间同步完整机制留给第 14 章。 ∴

∴

∴ {chapter-opening}

第 13 章 · 正文

系统日志不是一个文件，而是一条证据链

一台 RHEL 9 主机上的日志可能同时出现在结构化 journal、传统 `/var/log` 文本文件、远端日志主机以及应用自己的目录中。`systemd-journald`、`rsyslog` 和 `logrotate` 都与“日志”有关，却分别管理不同对象：journald 收集并索引事件，rsyslog 根据规则执行写文件或转发，logrotate 管理普通文本文件的生命周期。

最常见的误判来自对象混淆。`rsyslog.service` 处于 active，只能证明进程当前运行，不能证明某条 selector 命中过；`journalctl -b -1` 没有输出，不等于上一次启动没有故障；出现 `.1` 或 `.gz`，也不等于写入进程已经切换到新的活动文件。

本章按“对象与范围 → 查询与字段 → 配置与路由 → 持久和保留 → 轮转与诊断”推进。第 12 章已经建立 systemd unit 的状态与控制入口；第 14 章将完整讲解时间、时区和 chrony；应用框架自己的日志库不在本章展开。

```
事件产生者
→ journald 收集、附加字段并存储
→ journalctl 按 boot、时间、对象和字段检索
→ rsyslog selector 选择消息并执行 action
→ 本地文件或远端接收端留下结果
→ logrotate 管理文本文件的轮转和保留
→ 用新事件、跨 boot 与轮转后写入完成验收
```

...

... {concepts}

核心概念

... {concept-card} 概念 Journal 是由 `systemd-journald` 维护的结构化事件集合，而不是一份普通文本文件。每条 entry 除了 MESSAGE，还可以包含 boot、进程、用户、unit、可执行文件和优先级等字段，因此能够按系统对象精确查询。journal 当前可查并不自动意味着它会跨重启保留；存储位置与保留策略必须另行确认。

...

... {concept-card} 概念 结构化字段 把“看到一行文字”变成“识别一条事件”。SYSLOG_IDENTIFIER、PRIORITY 等字段可能由客户端提供；以 _ 开头的 _PID、_UID、_SYSTEMD_UNIT、_BOOT_ID 等通常由 journal 在接收时补充，证据强度更高。字段不存在不等于事件无效，只说明不能用该维度筛选。...

... {concept-card} 概念 Boot 范围 是 journal 查询的第一层时间边界。-b 选择当前启动，-b -1 选择上一次仍可查询的启动；它们依赖 journal 中实际保留的 boot。只有当前 boot 可见时，应继续区分“从未持久保存”“历史已被清理”“权限不足”，而不是把空结果解释成没有事件。...

... {concept-card} 概念 Facility 与 severity 分别描述 syslog 消息的类别和严重程度。传统 rsyslog selector 以 facility.priority 选择消息；local0.notice 表示 notice 以及更严重级别，而 local0.=notice 只精确匹配 notice。severity 数字越小越严重，这与“数字越大越严重”的直觉相反。...

... {concept-card} 概念 日志路由 是“选择条件 + 动作”的组合。selector 命中后，action 可以写入本地文件、转发到远端或进入其他处理链。配置语法通过只证明文件可解析；只有带唯一标识的新测试事件真正出现在目标位置，才能证明规则已加载、已命中且 action 已完成。...

... {concept-card} 概念 持久与保留 是两个不同问题。持久性回答“重启后能否继续读取”，保留回答“在容量、时间和清理操作作用下能保存多久”。Storage=persistent 建立持久存储方向，但 SystemMaxUse=、MaxRetentionSec= 和 vacuum 仍会决定历史证据何时被回收。...

... {concept-card} 概念 logrotate 状态 记录普通文本日志上一次轮转的时间，并与策略中的 daily、weekly、size 等条件共同决定本次是否轮转。文件很大不代表一定到期，配置正确也不代表调度入口已经执行。-d 用于解释决策且不改文件，-f 会真实强制轮转，二者风险完全不同。...::

... {quickref}

操作语义速查

先建立接口地图，再进入正文。参数按“范围、对象、优先级、输出”纵向组织，避免把多个选项挤成一行。

... {quick-command}

journalctl

SYNOPSIS

```
journalctl [OPTIONS...] [MATCHES...]
```

检索 journal entry。稳定顺序是先确定 boot 与时间范围，再按 unit、tag、PID 或字段选择对象，最后决定优先级、输出格式和条数。

重要参数 / 形式

--list-boots

列出当前仍可查询的启动轮次、boot ID 和时间范围。

-b [ID|OFFSET]

选择一次启动；-b 为当前 boot，-b -1 为上一次仍被保留的 boot。

```
-u UNIT / -t TAG
    分别按 systemd unit 和 SYSLOG_IDENTIFIER 选择来源。
--since TIME / --until TIME
    限定时间窗口；查询为空时先放宽最窄条件并核对当前时间与时区。
-p LEVEL[..LEVEL]
    按 severity 过滤；单个级别包含该级别及更严重记录。
FIELD=value
    使用结构化字段匹配；不同字段通常为 AND，同一字段多个值通常为 OR。
-o FORMAT
    选择输出格式；verbose 适合发现字段，json-pretty 适合机器处理，cat 只显示正文。
```

...

::: {quick-command}

logger

SYNOPSIS

```
logger [OPTIONS...] [MESSAGE]
```

向 syslog/journal 链生成可控测试事件。验证配置时应使用唯一 tag 和唯一正文，避免把旧消息误当作新规则的结果。

重要参数 / 形式

```
-p facility.level
    设置 facility 与 severity，例如 local0.notice；它不直接指定目标文件。
-t TAG
    设置 SYSLOG_IDENTIFIER，便于 journalctl -t TAG 查询。
-i
    在消息中记录 logger 进程 ID；用于演示 PID 字段，不应把短命 PID 当长期身份。
-f FILE
    将文件内容逐行写入日志链；大文件和敏感内容使用前必须确认影响。
```

...

::: {quick-command}

journald 配置

SYNOPSIS

```
# /etc/systemd/journal.conf.d/60-local.conf
[Journal]
Storage=persistent
SystemMaxUse=512M
MaxRetentionSec=30day
```

通过主配置或 drop-in 改变 journal 的存储与保留策略。管理员配置优先放入 /etc/systemd/journal.conf.d/*.conf，避免直接修改 vendor 文件。

重要参数 / 形式

```
Storage=persistent
    使用 /var/log/journal 保存可跨重启读取的 journal。
Storage=volatile
    使用 /run/log/journal，重启后内容消失。
SystemMaxUse= / RuntimeMaxUse=
    分别限制持久和运行时 journal 的最大占用。
MaxRetentionSec=
    限制 journal 文件的最长保留时间；仍受磁盘与容量策略影响。
journalctl --flush
    在持久存储可用时，把运行时 journal 刷入持久区域；它不能恢复已经丢失的历史。
```

...

::: {quick-command}

rsyslog selector / action

SYNOPSIS

```
facility.priority    action
```

传统规则用 selector 选择消息，再把它交给文件或远端 action。本章以传统语法建立 RHCSA 主线，复杂 RainerScript 只用于说明工作边界。

重要参数 / 形式

```
local0.notice /var/log/exam-audit.log
    匹配 local0 的 notice 及更严重级别，并写入本地文件。
local0.=notice /var/log/exam-notice.log
    只精确匹配 notice。
*.info;mail.none /var/log/messages
    组合 selector，并使用 none 排除某个 facility。
*.* @loghost:514
    使用 UDP 转发。
*.* @@loghost:514
    使用 TCP 转发；连接建立不等于接收端已经落盘。
```

...

... {quick-command}

rsyslogd -N1

SYNOPSIS

```
rsyslogd -N1
```

在应用配置前执行静态解析检查。它适合发现语法错误、未知模块或配置读取问题，但不能证明 selector 会命中，也不能证明目标路径、网络或远端 action 可用。

重要参数 / 形式

```
-N1
    检查配置语法和基本有效性后退出，不启动第二个常驻 rsyslogd。
systemctl restart rsyslog.service
    在静态检查通过后让服务重新读取配置；unit 状态模型归第 12 章。
journalctl -u rsyslog.service -b
    查看应用配置后的服务日志；仍需以目标层新事件完成最终验证。
```

...

... {quick-command}

logrotate

SYNOPSIS

```
logrotate [OPTIONS...] <config-file>
```

读取轮转策略和状态文件，判断普通文本日志是否到期，并执行重命名、压缩、创建新文件和脚本钩子。先解释决策，再在明确测试对象上执行真实轮转。

重要参数 / 形式

```
-d
    debug/dry-run；解释读取到的策略和轮转判断，不修改文件或状态。
-f
    强制真实轮转，即使状态文件认为尚未到期；不得对未知生产对象直接使用。
-s STATEFILE
    使用独立状态文件，便于隔离受控测试。
rotate N / compress
    保留 N 份归档，并压缩旧文件。
size / daily / weekly
    分别按大小或时间周期建立触发条件。
create MODE OWNER GROUP
    轮转后创建新的活动文件并设置权限与属主。
postrotate ... endscrip
    轮转后通知写入进程重新打开文件；具体信号或命令必须匹配实际程序。
```

... ...

[知识专题] 从事件产生到可审计证据：先认清对象与状态

日志排错的第一步不是打开 `/var/log/messages`，而是判断事件可能经过哪条路径。相同事件可以同时存在于 journal 和文本文件；应用也可能完全绕开 syslog，直接写自己的日志。只有先画出路径，后续查询才有明确边界。

① [知识点] journald 把多种输入统一为 entry

`systemd-journald` 可以接收内核消息、启动阶段输出、systemd 服务的标准输出与标准错误，以及经 syslog 接口提交的消息。它为每条 entry 保存正文和元数据，使管理员能够按 boot、unit、进程、用户、主机和优先级查询。

journal 是查询对象，不等于“所有应用日志的全集”。若应用只写 `/opt/app/log/app.log`，journal 中可能没有对应正文。反过来，unit 的标准错误可能只存在于 journal，而没有被任何 rsyslog 规则写入文本文件。

② [知识点] rsyslog 处理的是规则链，不是“第二份 journal”

rsyslog 的基础模型是：

```
input
→ selector / filter
→ action
```

在典型 RHEL 9 主机中，它接收 syslog 消息，并根据 facility、severity 或属性执行 action。action 可以是本地文件、远端转发或其他处理。它不会替代 journal 的字段索引，也不会自动管理目标文件的长期轮转。

③ [知识点] 文本日志和 journal 拥有独立生命周期

journal 文件由 journald 自己维护，容量通过 journald 配置和 `journalctl --vacuum=*` 控制。普通文本文件由写入进程持续追加，再由 logrotate 按策略轮转。两套机制不能互相替代：

```
journalctl --vacuum-size=500M
```

不会轮转 `/var/log/exam-audit.log`；而：

```
logrotate /etc/logrotate.conf
```

也不会删除二进制 journal entry。

④ [知识点] “当前可读”“跨 boot 存在”“仍被保留”必须分别判断

- 当前可读：当前用户能在现有 journal 文件中查询到 entry。
- 跨 boot 存在：重启后仍保留前一 boot 的 journal。
- 仍被保留：容量、时间、磁盘压力或 vacuum 尚未删除它。
- 目标可读：当前用户具备读取该 journal 或文本文件的权限。

因此，`journalctl -b -1` 没有结果时，至少有四种假设，而不是一个结论。

⑤ [边界] 日志时间是坐标，不在本章重建时间系统

时间范围查询依赖系统时间与时区。若已知事件没有落在预期窗口，先放宽时间条件并查看当前时间；若多主机事件顺序异常，再把时间同步列为独立假设。RTC、chronyd、source、offset 和 step/slew 归第 14 章完整展开。

::: {cheatsheet} Cheatsheet 先问事件来源，再问 journal 范围；journald 负责收集与索引，rsyslog 负责选择与 action，logrotate 负责普通文本文件生命周期；“当前可读、持久、保留、权限”四个状态不能互相替代。::: :::

[操作专题] 先限定范围，再按对象收缩 journal

`journalctl` 参数很多，但可以稳定分成四组：范围、对象、优先级和输出。推荐顺序是“boot/时间 → unit/tag/字段 → priority → 条数与格式”。一开始就叠加所有条件，最容易制造一个无法解释的空结果。

① [查询] 用 `--list-boots` 证明有哪些启动轮次可查

```
journalctl --list-boots
journalctl -b --no-pager
journalctl -b -1 --no-pager
journalctl --boot=<BOOT_ID> --no-pager
```

`--list-boots` 是 `-b -1` 的前置证据。只有列表中存在上一 boot，`-b -1` 才有明确对象。列表只显示当前 boot，证明的是“当前可用 journal 里只有这一轮”，不能证明历史从未产生。

② [查询] 用 unit、tag 和字段表达来源

```
journalctl -u sshd.service -b --no-pager
journalctl -t exam-audit --since today --no-pager
journalctl _PID=2450 -b --no-pager
journalctl _SYSTEMD_UNIT=sshd.service _UID=0 \
--since '-30 min' --no-pager
```

`-u` 是按 systemd unit 的便捷匹配；`-t` 匹配 `SYSLOG_IDENTIFIER`。PID 可能在不同时间被复用，因此较长窗口必须同时限定 boot、时间或 unit。字段不存在时，查询不会自动改用其他字段。

可先枚举已有值：

```
journalctl -F _SYSTEMD_UNIT
journalctl -F SYSLOG_IDENTIFIER
```

③ [查询] 用时间窗口保留故障上下文

```
journalctl --since today --no-pager
journalctl --since '-15 min' --no-pager
journalctl --since '2026-07-12 13:00:00' \
--until '2026-07-12 13:20:00' --no-pager
```

绝对时间适合题目给出明确事件点，相对时间适合刚完成的验证。若结果为空，先去掉 `--until` 或扩大 `--since`，再核对 boot 与时区。不要把过窄窗口产生的空结果当作“事件不存在”。

④ [查询] 认识 priority 的方向

severity 顺序是：

名称	数字	典型含义
emerg	0	系统不可用
alert	1	必须立即处理
crit	2	严重状态
err	3	错误
warning	4	警告
notice	5	正常但重要
info	6	信息
debug	7	调试

```
journalctl -p warning -b --no-pager
journalctl -p err..alert -b --no-pager
```

单个 `warning` 包含 `warning` 及更严重记录。初次调查只看 `err` 可能遗漏错误前的 `warning` 或 `notice`，应先保留上下文，再缩小优先级。

⑤ [查询] 用条数、方向和 follow 控制阅读成本

```
journalctl -u sshd.service -b -n 80 --no-pager
journalctl -u sshd.service -b -r -n 30 --no-pager
journalctl -u sshd.service -f
```

`-n` 限制条数，`-r` 倒序显示，`-f` 跟随新消息。`-f` 适合观察新测试事件，却不能证明旧历史完整；终端中断 `follow` 也不会停止被观察服务。

⑥ [验证] 为每个查询写明证据边界

```
journalctl -u rsyslog.service -b -n 20 --no-pager
```

能证明：当前 journal 中存在与 rsyslog unit 关联的最近记录。不能证明：某条自定义 selector 已命中、目标文件可写、远端已接收。

∴ {.cheatsheet} Cheatsheet --list-boots 先确定 boot；-b/-u/-t/FIELD=value 再缩小对象；时间为空先放宽；-p 数字越小越严重；每条查询都要注明“能证明什么、不能证明什么”。∴ ∴

∴ {#RHCSA-13-O02 .topic .operation data-kind="operation-topic"}

[操作专题] 用字段解释记录，而不是只读消息正文

当一条日志看似“来自某服务”时，真正需要确认的是：谁发送、属于哪个 boot、由哪个 unit 管理、以什么身份运行、优先级是什么。-o verbose 是字段发现入口，后再选择最有区分度的字段进行匹配。

① [查询] 用 -o verbose 展开完整 entry

```
journalctl -u sshd.service -b -n 1 -o verbose
```

重点观察：

字段	用途	边界
MESSAGE	消息正文	可由客户端控制
PRIORITY	severity 数字	0 最严重，7 为 debug
SYSLOG_IDENTIFIER	客户端标识	可由客户端设置
_BOOT_ID	启动身份	跨 boot 的稳定范围字段
_PID / _UID / _GID	发送进程和身份	由 journal 接收侧补充
_COMM / _EXE	进程名与可执行文件	路径可能因升级变化
_SYSTEMD_UNIT	所属 systemd unit	并非所有 entry 都有
_HOSTNAME	记录中的主机名	主机名变更要结合时间

② [知识点] 下划线字段通常比客户端字段更可信

普通字段可以由客户端提交；以下划线开头的 trusted fields 通常由 journal 根据接收时环境生成，客户端不能任意伪造。因此调查来源时，优先组合 _BOOT_ID、_PID、_UID、_EXE 和 _SYSTEMD_UNIT，而不是只相信正文里写着“来自 sshd”。

可信不等于永远存在。内核消息、早期启动消息或脱离 unit 的进程可能缺少 _SYSTEMD_UNIT；此时应换用 _TRANSPORT、_COMM、_EXE 或 boot 范围继续调查。

③ [查询] 理解字段表达式的 AND 与 OR

```
journalctl _SYSTEMD_UNIT=sshd.service _UID=0 -b
```

不同字段之间是 AND：同时满足 unit 与 UID。

```
journalctl _SYSTEMD_UNIT=sshd.service \
_SYSTEMD_UNIT=chronyd.service -b
```

同一字段多个值通常为 OR：匹配任一 unit。

显式 + 可以连接两组完整表达式：

```
journalctl _SYSTEMD_UNIT=sshd.service + \
SYSLOG_IDENTIFIER=exam-audit
```

复杂表达式应先分别运行每个子条件，确认都有结果，再合并。否则一个拼写错误就会把整组查询变成空集。

④ [查询] 按阅读和机器处理目的选择输出格式

```
journalctl -u sshd.service -b -n 20 -o short-iso-precise
journalctl -t exam-audit -n 5 -o cat
journalctl -t exam-audit -n 2 -o json-pretty
```

- `short-iso-precise`：便于人工比较精确时间。
- `cat`：只显示正文，适合脚本读取单一消息，但会丢失字段上下文。
- `json-pretty`：保留结构供机器处理。
- `verbose`：发现字段，不适合作为日常大量输出。

改变 `-o` 只改变呈现，不会改变 entry 或其保存状态。

⑤ [验证] 输出中出现字段，不等于业务身份已经成立

`_SYSTEMD_UNIT=web.service` 证明 journal 把该进程归属于该 unit；它不证明 Web 端口正在监听、请求成功或内容正确。unit 功能验收仍要回到第 12 章建立的服务证据链。

∴ {cheatsheet} Cheatsheet 先用 `-o verbose` 发现字段；来源判断优先 trusted fields；不同字段 AND、同字段多值 OR；`-o cat` 会丢字段；字段归属不能替代业务功能验证。∴ ∴

∴ {#RHCSA-13-O03.topic.operation data-kind="operation-topic"}

[操作专题] 建立持久 journal，并控制占用与保留

持久化配置的目标不是“创建一个目录”，而是让有效配置、存储位置、当前 entry、跨 boot entry 和保留预算形成闭环。若当前维护窗口不允许安全重启，只能完成配置核对和重启前证据，不能声称跨重启已经成功。

① [查询] 先调查有效配置与当前存储

```
journalctl --list-boots
journalctl --disk-usage
ls -ld /run/log/journal /var/log/journal 2>/dev/null
systemd-analyze cat-config systemd/journald.conf
```

若目标版本不支持 `systemd-analyze cat-config` 的该用法，则直接检查：

```
grep -R -nE '^[[:space:]]*(Storage|SystemMaxUse|RuntimeMaxUse|MaxRetentionSec)= ' \
/etc/systemd/journald.conf \
/etc/systemd/journald.conf.d \
/usr/lib/systemd/journald.conf.d 2>/dev/null
```

调查要回答：有效 `Storage=` 是什么、是否存在管理员 drop-in、当前有几个 boot、journal 占用多大。

② [操作] 用管理员 drop-in 表达目标策略

```
# /etc/systemd/journald.conf.d/60-persistent.conf
[Journal]
Storage=persistent
SystemMaxUse=512M
MaxRetentionSec=30day
```

- `Storage=persistent`：把长期 journal 放入 `/var/log/journal`。
- `SystemMaxUse=512M`：限制持久 journal 使用量。
- `MaxRetentionSec=30day`：限制最长保留时间。

这些值是任务参数，不是推荐所有主机统一采用的默认值。生产环境应依据磁盘预算、故障回溯和审计要求确定。

③ [操作] 让运行中的 journald 读取配置

```
systemctl restart systemd-journald.service
journalctl --flush
journalctl --disk-usage
```

restart 和 flush 的对象不同：restart 让服务重新读取配置；flush 在持久存储可用时把运行时数据移入持久区域。两条命令成功仍不能证明“下一次重启后可查”，只能为跨 boot 验证建立条件。

④ [验证] 用唯一事件建立跨 boot 证据

```
TEST_ID="rhcsa13-persist-$(date +%s)"
logger -p local0.notice -t rhcsa13-persist "$TEST_ID"

journalctl -t rhcsa13-persist --since '-2 min' --no-pager
journalctl --list-boots
```

当前能证明：新事件已进入当前 journal，tag 和正文可查询。下一次安全重启后必须再执行：

```
journalctl --list-boots
journalctl -b -1 -t rhcsa13-persist --no-pager
```

只有在上一 boot 找到重启前 TEST_ID，才能证明跨重启持久。

⑤ [查询] 区分占用观察和清理动作

```
journalctl --disk-usage
journalctl --rotate
journalctl --vacuum-time=30d
journalctl --vacuum-size=500M
journalctl --vacuum-files=20
```

`--disk-usage` 只观察。vacuum 会删除归档 journal，是不可逆的证据清理动作。若确需清理，先确认审计与故障调查要求；必要时先 rotate，使活动文件归档后再按目标条件 vacuum。

⑥ [安全边界] 磁盘紧张时不要把 vacuum 当作第一反应

正确顺序是：

```
确认 journal 占用
→ 判断是否存在异常高频消息
→ 确认业务和审计保留要求
→ 调整长期预算或修复消息源
→ 只删除允许删除的归档证据
→ 再次检查占用和日志速率
```

直接清空日志会破坏事故现场，也可能让根因继续高速产生日志。

⋮ {cheatsheet} Cheatsheet 有效配置 + 存储目录 + 当前 TEST_ID + 跨 boot TEST_ID 才是持久闭环；`--disk-usage` 是观察，vacuum 是删除；新配置不能恢复已经丢失的历史。⋮ ⋮

⋮ {#RHCSA-13-O04 .topic .operation data-kind="operation-topic"}

[操作专题] 用 logger 构造可重复、可判定的验证事件

验证消息必须能与历史消息区分。固定正文如“test”可能早已存在，既不能证明新规则已经加载，也不能确定它来自本次操作。最小模式是“唯一 TEST_ID + 明确 facility/severity + 明确 tag”。

① [操作] 生成具有唯一身份的测试事件

```
TEST_ID="rhcsa13-$(date +%s)"
logger -p local0.notice -t exam-audit "$TEST_ID"
```

这里：

- local0 是 facility；
- notice 是 severity；
- exam-audit 成为 SYSLOG_IDENTIFIER；
- \$TEST_ID 是本次测试唯一正文。

② [验证] 先证明事件进入 journal

```
journalctl -t exam-audit --since '-2 min' --no-pager
journalctl SYSLOG_IDENTIFIER=exam-audit MESSAGE="$TEST_ID" \
  --since '-2 min' --no-pager
```

第一条适合人工查看，第二条把 tag 和正文组合起来。若 journal 没有事件，先调查 logger 调用、当前时间、权限和 journald；不要直接跳到 rsyslog 文件层。

③ [验证] 再证明目标 action 完成

```
grep -F "$TEST_ID" /var/log/exam-audit.log
```

journal 有而文件无，说明事件产生和 journald 收集基本成立，故障更可能位于 rsyslog 配置加载、selector、目标路径、权限或 SELinux 层。此时重新发送新的 TEST_ID，而不是复用旧消息。

④ [边界] logger -p 不指定目标文件

-p local0.notice 只描述消息类别和级别。消息写入哪个文件、是否远端转发，由 rsyslog 规则决定。把 logger -p 理解成“写文件选项”会混淆 producer 与 router。

⑤ [安全边界] 不把敏感内容当测试正文

日志可能被写入多个本地文件并转发到集中平台。口令、私钥、token、Vault 明文和个人数据不应作为验证消息。生产环境还应考虑日志采样、脱敏和保留政策，但应用日志框架不在本章展开。

⋮ {cheatsheet} Cheatsheet 每次变更后创建新 TEST_ID；先在 journal 找，再到目标 action 找；logger -p 设置消息属性，不指定目标文件；测试正文不能含秘密。⋮ ⋮

⋮ {#RHCSA-13-O05 .topic.operation data-kind="operation-topic"}

[操作专题] 配置 rsyslog selector 与本地文件 action

本地路由任务的核心不是写出一行配置，而是证明四件事：配置可解析、服务读取了新规则、测试消息满足 selector、文件 action 成功写入。每一层都使用不同证据。

① [知识点] facility.priority 默认是阈值匹配

```
local0.notice /var/log/exam-audit.log
```

它匹配 local0.notice、local0.warning、local0.err 等 notice 及更严重消息，不匹配 info 和 debug。

```
local0.=notice /var/log/exam-notice.log
```

只精确匹配 notice。若题目要求“notice 及以上”，使用 .=notice 会漏掉更严重事件。

② [操作] 在独立片段中写入最小规则

```
# /etc/rsyslog.d/60-exam-audit.conf
local0.notice /var/log/exam-audit.log
```

变更前先查冲突：

```
grep -R -n 'local0\|exam-audit' \
/etc/rsyslog.conf /etc/rsyslog.d 2>/dev/null
```

同一消息可以命中多条规则。发现已有规则时，应判断是否允许多目标写入，而不是直接删除系统配置。

③ [验证] 静态检查只能通过第一道门

```
rsyslogd -N1
```

能证明：配置文件和引用内容可被 rsyslogd 解析。不能证明：服务已重新读取、selector 会命中、目录可写、SELinux 允许写入、文件内容正确。

静态检查通过后应用：

```
systemctl restart rsyslog.service
systemctl is-active rsyslog.service
journalctl -u rsyslog.service -b -n 30 --no-pager
```

④ [验证] 使用新事件完成目标层验收

```
TEST_ID="rhcsa13-local-$(date +%s)"
logger -p local0.notice -t exam-audit "$TEST_ID"

journalctl -t exam-audit --since '-2 min' --no-pager
grep -F "$TEST_ID" /var/log/exam-audit.log
stat -c '%U %G %a %n' /var/log/exam-audit.log
```

目标文件中的 TEST_ID 证明 selector 与 file action 对这条消息成立；stat 再检查文件身份与模式。它仍不证明 logrotate 策略正确。

⑤ [诊断] journal 有消息、文件没有时选择下一条证据

```
journal 中 TEST_ID 存在
→ rsyslogd -N1
→ 检查 include 与规则顺序
→ 查看 rsyslog unit 日志
→ 检查目标目录和文件权限
→ 检查 AVC/SELinux 证据
→ 最小修复
→ 生成新的 TEST_ID 再验证
```

不要用 chmod 777 或关闭 SELinux 绕过调查。SELinux 完整规则归第 28、29 章，本章只把 AVC 作为日志写入失败的一条证据。

```
:: {cheatsheet} Cheatsheet facility.priority 是阈值, facility.=priority 是精确; -N1 只证明可解析; 最终必须以新 TEST_ID 进入目标文件验收。:: ::
:: {#RHCSA-13-O06 .topic .operation data-kind="operation-topic"}
```

[操作专题] 配置远端转发，并按层证明消息真正到达

远端日志链至少跨越发送规则、名称解析、网络、协议监听、接收规则和落盘六层。客户端建立连接只是中间证据，不能替代接收端的最终查询。

① [操作] 使用传统转发 action 表达协议

```
# UDP
*. * @loghost.example.com:514

# TCP
*. * @@loghost.example.com:514
```

单个 @ 表示 UDP，双 @@ 表示 TCP。TCP 提供连接和重传基础，但并不自动等于 TLS、可靠磁盘队列或端到端交付保证。

生产系统更常使用显式 action：

```
action(
    type="omfwd"
    target="loghost.example.com"
    port="514"
    protocol="tcp"
    queue.type="LinkedList"
)
```

本章不展开 TLS 证书、复杂队列和集中平台架构，只强调它们不是基础 TCP 配置自动获得的能力。

② [查询] 在客户端分层确认发送前提

```
rsyslogd -N1
getent hosts loghost.example.com
ss -tnp | grep ':514'
journalctl -u rsyslog.service -b -n 50 --no-pager
```

- 名称解析失败，应先修复目标解析，不要改 selector。
- TCP 没有连接，继续查路由、防火墙和监听。
- TCP 已连接，只证明传输层建立，不证明服务端规则命中或落盘。

③ [操作] 用唯一事件进行端到端测试

客户端：

```
TEST_ID="rhcsa13-remote-${date +%s}"
logger -p local0.notice -t rhcsa13-remote "$TEST_ID"
```

接收端最终应按真实配置查询：

```
journalctl -t rhcsa13-remote --since '-5 min' --no-pager
grep -R -F "$TEST_ID" /var/log 2>/dev/null
```

只有接收端出现同一 TEST_ID，才能证明消息跨主机到达。若题目要求指定文件，还必须在该文件中找到它。

④ [诊断] 按“发送 → 网络 → 接收 → action”推进

```
客户端 journal 有 TEST_ID
→ 客户端 rsyslog 配置可解析
→ 目标名称和地址正确
→ 网络与端口可达
→ 服务端监听正确协议
→ 服务端接收规则加载
→ 服务端目标文件或 journal 中出现 TEST_ID
```

每一步只验证一层。不要因为 telnet 或 TCP connect 成功，就跳过服务端日志与文件验收。

⑤ [安全边界] 不把无加密 TCP 描述成安全集中日志

远端日志可能包含账号、路径、主机结构和故障细节。跨不可信网络时，需要 TLS、证书验证、队列和访问控制。由于这些内容超出 RHCSA 本章范围，本章只标记边界，不提供“看似完整但未经验证”的生产模板。

```
:: {cheatsheet} Cheatsheet @ UDP, @@ TCP; 连接成功不是落盘；最终证据必须在接收端找到同一 TEST_ID；TLS 与可靠队列不是基础 TCP 自动拥有。:: ::
```

```
:: {#RHCSA-13-O07.topic.operation data-kind="operation-topic"}
```

[操作专题] 用 logrotate 管理文本日志的状态、轮转与保留

logrotate 的决策来自“策略 + 状态 + 当前文件”。排错时先让 -d 解释为什么轮转或不轮转，再决定是否受控的真实测试。直接 -f /etc/logrotate.conf 可能同时改变大量系统日志，是典型高风险捷径。

① [知识点] 配置来源与状态文件共同参与判断

常见入口：

```
/etc/logrotate.conf
/etc/logrotate.d/*
/var/lib/logrotate/logrotate.status
```

主配置通常 include 片段。状态文件记录上次轮转时间。即使文件已经很大，daily 规则若刚执行过，仍可能不轮转；size 规则则按当前大小判断。

② [知识点] 区分时间条件、大小条件和保留动作

指令	作用
daily / weekly / monthly	按时间周期判断
size 100M	达到大小即轮转
minsize 10M	时间到且至少达到该大小
maxsize 500M	即使时间未到，超过该大小也可轮转
rotate 7	保留 7 份归档
compress	压缩旧归档
delaycompress	下一轮再压缩最近归档
missingok	文件缺失时不报致命错误
notifempty	空文件不轮转
create 0640 root root	创建新活动文件并设置身份

触发条件回答“何时轮转”，保留动作回答“轮转后怎样处理”。两类指令不能混为一谈。

③ [操作] 编写自包含策略

```
# /etc/logrotate.d/exam-audit
/var/log/exam-audit.log {
    daily
    rotate 7
    compress
    missingok
    notifempty
    create 0640 root root
    postrotate
        /usr/bin/systemctl kill -s HUP --kill-whom=main \
            rsyslog.service >/dev/null 2>&1 || true
    endscript
}
```

`postrotate` 请求 `rsyslog` 重新打开文件。真实主机应与系统自带 `rsyslog` 轮转片段核对信号和命令；不同应用的 `reopen` 方式不同，不能机械复制。

④ [查询] 先 `dry-run` 解释决策

```
logrotate -d /etc/logrotate.conf
```

重点观察：目标片段是否被 `include`、文件是否匹配、读取到哪个状态、为何到期或未到期。`debug` 不修改文件和状态，适合首次审查。

⑤ [操作] 只在受控对象上强制真实轮转

```
logrotate -s /tmp/rhcsa13-exam.status \
    -f /etc/logrotate.d/exam-audit
```

这条命令会真实改动 `/var/log/exam-audit.log`。独立状态文件隔离测试状态，却不会隔离文件本身。执行前必须确认片段只包含目标测试文件。

⑥ [验证] 轮转结果至少检查四层

```
ls -l /var/log/exam-audit.log*
stat -c '%U %G %a %n' /var/log/exam-audit.log
```

再生成新事件：

```
POST_ID="rhcsa13-postrotate-$(date +%s)"
logger -p local0.notice -t exam-audit "$POST_ID"
grep -F "$POST_ID" /var/log/exam-audit.log
```

验收矩阵：

1. 归档文件出现；
2. 新活动文件存在且身份正确；
3. 保留和压缩符合策略；
4. 新 `POST_ID` 进入新活动文件，而非旧归档。

⑦ [知识点] `rename/create` 后写入者可能仍持有旧 `inode`

`logrotate` 重命名文件并创建新文件，不会自动改变进程已经打开的文件描述符。若写入进程不 `reopen`，它会继续写入被重命名的归档。`postrotate` 的意义是通知进程重新打开路径。

`copytruncate` 在原路径复制后截断，可兼容无法 `reopen` 的应用，但复制和截断之间存在日志丢失窗口，因此不是默认首选。

⑧ [诊断] 配置正确却从未自动执行

应检查 `logrotate` 的调度入口和运行记录，例如 `logrotate.timer`、`logrotate.service` 及其 `journal`。`timer` 的日历和持久触发机制归第 15 章，本章只把它作为“策略从未被调用”的下一条证据。

⋮ {cheatsheet} Cheatsheet 策略 + state + 文件决定轮转；`-d` 解释、不修改，`-f` 真操作；强制测试使用自包含片段和独立 `state`；最后用新 `POST_ID` 证明写入者切换。⋮ ⋮

⋮ {#RHCSA-13-D01 .topic .diagnosis data-kind="diagnosis-topic"}

[诊断专题] 日志查不到、写不到、转不到和轮不动

日志故障最容易陷入“多跑几次同一命令”。更有效的方法是把现象放回证据链，选择下一条最能区分假设的查询。每次修复后必须生成新事件，防止旧证据污染结论。

① [诊断] 已知事件在 `journalctl` 中查不到

症状：应用刚报错，但严格查询为空。假设：boot、时间、unit/tag、priority 或字段条件过窄；应用可能直接写文件。下一条证据：从宽到窄逐步移除条件。

```
journalctl --since '-30 min' --no-pager
journalctl -b --since '-30 min' --no-pager
journalctl -t <TAG> --since '-30 min' --no-pager
journalctl -u <UNIT> -b --since '-30 min' --no-pager
```

若宽查询仍无记录，再查应用自己的日志路径与输出方式。最小修复不是更换随机参数，而是修正错误范围或事件接入路径。

② [诊断] 只能看到当前 boot

```
--list-boots 只有 0
→ 检查有效 Storage=
→ 检查 /var/log/journal
→ 检查容量和 MaxRetentionSec
→ 检查是否执行过 vacuum
→ 检查读取权限
```

修改为持久存储只能保证后续 boot，无法恢复已经消失的上一 boot。再验证必须安排一次安全重启，并在 `-b -1` 中查找重启前 TEST_ID。

③ [诊断] journal 有 TEST_ID，目标文件没有

```
事件产生与 journald 收集基本成立
→ rsyslogd -N1
→ include 和 selector
→ rsyslog unit 日志
→ 目录/文件权限
→ AVC/SELinux
→ 最小修复
→ 新 TEST_ID
```

不要通过关闭 SELinux 或全局放宽权限跳过证据。若目标路径不是标准日志路径，需在 SELinux 章节进一步处理持久标签。

④ [诊断] 客户端已连接，接收端没有日志

TCP 连接只排除部分网络问题。继续检查：协议是否一致、接收端是否监听正确地址、input 是否加载、接收规则是否命中、action 是否有写入权限，最终在接收端寻找同一 TEST_ID。

⑤ [诊断] logrotate dry-run 表示“不需要轮转”

优先检查：

```
目标片段是否被 include
→ 文件是否匹配
→ 状态文件中的上次轮转时间
→ daily/weekly 与 size/minsize/maxsize 条件
→ 文件是否为空或缺失
```

若只是测试任务，使用独立 state 和自包含片段强制；若是生产对象，应尊重策略判断，不因“想看到 .1”而强制改变现场。

⑥ [诊断] 已经轮转，但新文件不增长

当前证据：归档存在、新活动文件存在、写入仍进入归档。假设：写入进程仍持有旧 inode。下一条证据：检查程序的 reopen 机制和打开文件，再核对 postrotate。最小修复：使用应用支持的 reload/HUP/reopen。再验证：新 POST_ID 只进入活动文件。

⑦ [安全边界] 磁盘压力与证据保全同时存在

日志占满磁盘时既要恢复可用性，也要保留调查证据。先识别增长最快的源和允许删除的范围；在无明确授权时，不删除全部 journal、不清空业务日志、不把长期预算问题掩盖为一次 vacuum。

∴ {diagnosis-chain} 通用诊断链

- 症状
- 当前证据属于哪一层
 - 列出 1~2 个可区分假设
 - 选择下一条最有区分度的证据
 - 做最小修复
 - 生成新的唯一事件
 - 在目标层再次验证

...

... {#RHCSA-13-C01 .topic .classic .task-section data-kind="classic-task"}

[经典任务] 为指定 facility 建立本地日志路由和轮转

环境与当前状态

一台 RHEL 9 主机运行审计脚本，脚本通过以下形式写消息：

```
logger -p local0.notice -t exam-audit "<MESSAGE>"
```

当前系统存在其他 rsyslog 规则，不允许删除或覆盖。`/var/log/exam-audit.log` 尚未稳定接收新消息，也没有专用 logrotate 片段。

目标终态

1. `local0.notice` 及更严重消息写入 `/var/log/exam-audit.log`；
2. 文件模式为 `0640`，所有者和组均为 `root`；
3. 日志按 `daily` 轮转，保留 7 份，旧归档压缩；
4. 空文件不轮转，文件缺失不导致任务失败；
5. 轮转后 rsyslog 继续向新的活动文件写入；
6. 每层验收使用本次生成的唯一事件。

限制条件

- 不删除现有 rsyslog 规则；
- 不使用 `chmod 777`；
- 不关闭 SELinux；
- 不对整个 `/etc/logrotate.conf` 无调查执行 `-f`；
- 不把 `rsyslog.service` active 当作最终结果；
- 未在目标主机完成真实轮转前，不得把配置核对写成“已验证成功”。

验收矩阵

层次	最少证据	不能扩大解释为
配置语法	<code>rsyslogd -N1</code>	selector 已命中
事件产生	journal 中出现 TEST_ID	文件已写入
本地路由	目标文件出现同一 TEST_ID	轮转策略正确
文件身份	<code>stat</code> 显示 <code>root root 640</code>	写入者已 reopen
轮转决策	<code>logrotate -d</code> 解释目标策略	已真实轮转
真实轮转	受控 <code>-s ... -f</code> 后出现归档	后续仍写新文件
写入者切换	新 POST_ID 进入活动文件	所有生产场景均验证
...		

... {#RHCSA-13-C02 .topic .classic .task-section data-kind="classic-task"}

[经典任务] 调查只能看到当前 boot、看不到上一次启动日志

环境与当前状态

服务器刚经历一次非计划重启。管理员执行：

```
journalctl --list-boots
journalctl -b -1
```

只能看到当前 boot。业务窗口暂不允许再次重启。

目标终态

- 1. 区分“未启用持久 journal”“旧记录已被策略清理”“当前用户权限不足”；
- 2. 使用 `/etc/systemd/journald.conf.d/60-persistent.conf` 明确持久存储；
- 3. 将持久 journal 最大使用量限制为 512M，最长保留 30day；
- 4. 生成唯一验证事件并完成当前可验证层次；
- 5. 写出下一次安全重启后的跨 boot 验收命令；
- 6. 明确此前已经丢失的历史不能被新配置恢复。

限制条件

- 不删除 `/var/log/journal`；
- 调查期间不执行 vacuum；
- 不以当前 boot 的测试事件声称跨 boot 已成功；
- 不抢先展开 chrony 调优；
- 当前窗口不允许重启时，必须将跨 boot 验收明确标为待执行。

验收矩阵

层次	当前可完成	需要重启后完成
有效配置	检查 <code>Storage=</code> 和 <code>drop-in</code>	确认配置持续有效
存储对象	检查 <code>/var/log/journal</code> 与占用	确认上一个 boot 仍在列表
当前事件	当前 boot 中找到 <code>TEST_ID</code>	<code>-b -1</code> 找到同一 <code>TEST_ID</code>
保留预算	静态核对 512M/30day	长期观察是否符合预算
历史恢复	明确不可恢复边界	不适用
...		

::: {#RHCSA-13-A01 .topic .answer .answer-section data-kind="answer-topic"}

[参考解答] 任务一：本地路由、轮转与写入者重新打开

① 调查当前对象和冲突

```
grep -R -n 'local0\|exam-audit' \
/etc/rsyslog.conf /etc/rsyslog.d 2>/dev/null
rsyslogd -N1
ls -l /var/log/exam-audit.log* 2>/dev/null
```

先确认是否已有 local0 规则和同名文件。若已有规则允许多目标写入，则保留；若规则冲突，先解释影响再做最小修改。

② 写入最小 rsyslog 规则

```
# /etc/rsyslog.d/60-exam-audit.conf
local0.notice    /var/log/exam-audit.log
```

选择 `local0.notice`，因为任务要求 notice 以及更严重消息。`local0.=notice` 会漏掉 warning、err 等更严重级别。

```
rsyslogd -N1
systemctl restart rsyslog.service
systemctl is-active rsyslog.service
journalctl -u rsyslog.service -b -n 30 --no-pager
```

`-N1` 是配置语法证据；`restart` 和 `active` 是加载与运行层证据，仍不是路由结果。

③ 生成唯一事件并完成本地路由验收

```
TEST_ID="rhcsa13-local-$(date +%s)"
logger -p local0.notice -t exam-audit "$TEST_ID"

journalctl -t exam-audit --since '-2 min' --no-pager
grep -F "$TEST_ID" /var/log/exam-audit.log
stat -c '%U %G %a %n' /var/log/exam-audit.log
```

若 journal 有而文件无，按“配置解析 → include/selector → unit 日志 → 路径权限 → AVC”推进。修复后创建新的 TEST_ID，不能用旧消息验收新配置。

④ 写入 logrotate 策略

```
# /etc/logrotate.d/exam-audit
/var/log/exam-audit.log {
    daily
    rotate 7
    compress
    missingok
    notifempty
    create 0640 root root
    postrotate
        /usr/bin/systemctl kill -s HUP --kill-whom=main \
        rsyslog.service >/dev/null 2>&1 || true
    endscript
}
```

参数选择：

- **daily**：每天进入一次时间判断；
- **rotate 7**：保留 7 份；
- **compress**：压缩旧归档；
- **missingok**：文件缺失时不报致命错误；
- **notifempty**：空文件不轮转；
- **create 0640 root root**：创建新活动文件；
- **postrotate**：请求 rsyslog 重新打开路径。

真实主机应核对系统自带 rsyslog 片段的 reopen 方式；不同版本或应用不能机械复制同一 postrotate。

⑤ 先解释轮转决策

```
logrotate -d /etc/logrotate.conf
```

确认目标片段被 include、路径被匹配、状态文件被读取，并记录本次是否到期及原因。debug 不改变文件。

⑥ 在受控状态下执行真实轮转

确认片段只包含测试目标后：

```
logrotate -s /tmp/rhcsa13-exam-audit.status \
-f /etc/logrotate.d/exam-audit

ls -l /var/log/exam-audit.log*
stat -c '%U %G %a %n' /var/log/exam-audit.log
```

-s 隔离状态文件，**-f** 仍会真实改变目标日志，因此只能在明确任务对象上执行。

⑦ 用新 POST_ID 证明写入者切换

```
POST_ID="rhcsa13-postrotate-$(date +%s)"
logger -p local0.notice -t exam-audit "$POST_ID"

grep -F "$POST_ID" /var/log/exam-audit.log
grep -R -F "$POST_ID" /var/log/exam-audit.log.* 2>/dev/null
```

期望：POST_ID 出现在新的活动文件，不出现在旧归档。只看到 **.1** 或 **.gz** 不能证明写入者已 reopen。

典型错误与分支

- **local0.=notice**：范围过窄；改为阈值 selector。

- `rsyslogd -N1` 通过但文件无消息：检查加载、selector、路径和 AVC。
- `logrotate -d` 表示未到期：若是考试受控测试，使用独立 state 强制；生产场景尊重策略。
- 轮转后仍写归档：修复 `reopen`，而不是立刻改用 `copytruncate`。
- `copytruncate`：只在应用无法 `reopen` 时评估，并接受复制/截断窗口的潜在丢失风险。

验证边界

`rsyslogd -N1`、配置文本和 `dry-run` 只能证明静态层次。真实轮转、权限保持以及写入者重新打开必须在目标 RHEL 9 主机上完成；`postrotate` 的信号或命令还应与系统自带 `rsyslog` 片段核对。:::

::: {#RHCSA-13-A02 .topic .answer .answer-section data-kind="answer-topic"}

[参考解答] 任务二：持久 journal 与跨 boot 验证计划

① 调查当前 boot、占用与配置来源

```
journalctl --list-boots
journalctl --disk-usage
ls -ld /run/log/journal /var/log/journal 2>/dev/null
systemd-analyze cat-config systemd/journald.conf
```

当前只有一个 boot 时，保留三类假设：从未持久、旧记录已清理、权限不足。若 `cat-config` 不可用，再逐个读取主配置和 `drop-in`，不能只看一个文件就认定有效值。

② 建立管理员配置目录并保留已有文件

```
mkdir -p /etc/systemd/journald.conf.d
cp -a /etc/systemd/journald.conf.d/60-persistent.conf \
/etc/systemd/journald.conf.d/60-persistent.conf.bak \
2>/dev/null || true
```

只备份存在的管理员文件，不修改 `/usr/lib` 中的 `vendor` 配置。

③ 写入明确的持久与保留策略

```
# /etc/systemd/journald.conf.d/60-persistent.conf
[Journal]
Storage=persistent
SystemMaxUse=512M
MaxRetentionSec=30day
```

`512M` 和 `30day` 来自任务目标。它们表达上限与最长保留，不保证每条日志一定保留满 30 天；磁盘压力和容量上限可能更早触发轮转与清理。

④ 让运行中的 journald 读取配置

```
systemctl restart systemd-journald.service
journalctl --flush
journalctl --disk-usage
```

这一步完成配置应用与当前占用观察。它不能证明跨重启，因为重启尚未发生。

⑤ 建立重启前唯一证据

```
TEST_ID="rhcsa13-persist-$(date +%s)"
logger -p local0.notice -t rhcsa13-persist "$TEST_ID"

journalctl -t rhcsa13-persist --since '-2 min' --no-pager
journalctl --list-boots
```

把 `TEST_ID` 记录到操作草稿。当前查询证明事件已进入当前 journal，不证明它会出现在下一次 `-b -1`。

⑥ 在下次安全重启后完成跨 boot 验收

```
journalctl --list-boots
journalctl -b -1 -t rhcsa13-persist --no-pager
```

只有列表出现上一 boot，且重启前 TEST_ID 能在 `-b -1` 中找到，才能写出“持久 journal 跨重启验证成功”。若尚未实施安全重启，该项必须保持为待验证。

⑦ 解释无法恢复的边界

若历史曾只保存在 `/run/log/journal` 并随重启消失，或已经被容量/vacuum 清理，新配置无法恢复。正确结论是“已为后续 boot 建立持久化和保留预算”，不是“恢复了上一次启动日志”。

典型错误与分支

- 只创建 `/var/log/journal`，不检查有效 `Storage=`；
- 修改配置后不让 `journald` 读取；
- 当前 TEST_ID 可见就声称跨 boot 成功；
- 调查期间执行 `vacuum`，破坏剩余证据；
- `--since` 查询为空便立即调整 `chrony`，而未先放宽范围；
- 把 `SystemMaxUse=512M` 理解为磁盘会立即占满 512M。

验证边界

配置解析、当前事件与磁盘占用属于重启前证据；目录创建、flush 后落盘、跨重启保留和长期容量效果仍需在目标 RHEL 9 主机上分层验证。:::

::: {#RHCSA-13-S01 .topic .wrap data-kind="chapter-wrap"}

[本章收束] 从“看到日志”迁移到“维护证据链”

日志能力的核心不是记住更多路径，而是把每条结论落到明确对象和证据层。稳定工作方法是：先确认事件来源和查询范围，再用结构化字段缩小对象；配置路由前做静态检查，配置后用新 TEST_ID 完成目标层验收；清理和强制轮转前先解释状态与风险。

工作方法：七步闭环

1. 明确事件来源与目标证据位置
2. 先定 boot 和时间，再定 unit/tag/字段
3. 用 verbose 发现字段，用最小条件缩小对象
4. 变更配置前建立基线并执行静态检查
5. 生成新的唯一测试事件
6. 在 journal、文件或接收端逐层验证
7. 对持久、保留和轮转后写入做独立验收

主要判断表

看到的证据	可以证明	不能证明	下一层证据
<code>journalctl -u ...</code> 有记录	当前 journal 有 unit 相关 entry	服务功能正确	监听、访问或业务检查
<code>--list-boots</code> 只有 0	当前可用 journal 只有当前 boot	上一 boot 没事件	有效 Storage、目录、保留与权限
<code>rsyslogd -N1</code> 通过	配置可解析	selector 命中	新 TEST_ID 进入目标文件
<code>rsyslog active</code>	进程当前运行	新规则已加载且有效	unit 日志与目标文件 TEST_ID
TCP 514 已连接	传输层连接建立	接收端已落盘	接收端同一 TEST_ID
<code>logrotate -d</code> 显示策略	配置被读取并解释决策	文件已经轮转	受控真实轮转与文件检查
出现 <code>.1</code> / <code>.gz</code>	产生了归档	写入者切到新文件	新 POST_ID 进入活动文件
当前 TEST_ID 可查	当前 entry 存在	跨重启持久	重启后 <code>-b -1</code> 同一 TEST_ID
<code>--disk-usage</code> 显示占用	当前 journal 体积	保留政策合理	日志速率、预算与审计要求

考试与工作迁移

考试题通常给出明确 facility、文件路径、保留数量或持久化参数；真实工作还要考虑集中平台、TLS、队列、审计、脱敏和容量预测。无论规模如何扩大，底层方法仍是“对象明确、范围明确、证据分层、变更最小、验证不可扩大”。

向下一章交接

本章在查询时间窗口和跨主机排序时，只把“当前时间、时区与同步状态”当作一条独立证据。下一章《系统时间、时区、RTC 与 chrony》将继续回答：日志时间戳为何偏移、主机时钟如何建立可信来源、RTC 与系统时钟如何交互，以及怎样验证时间同步终态。:::

系统时间、时区、RTC 与 chrony

从“现在几点”拆出显示、硬件基准、时间源选择与本机跟踪：用证据证明时间链真正成立。

对象模型操作语义验证诊断经典任务

大字号阅读版

本章阅读导航

先抓住一条主线：外部时间源只有经过解析、NTP 往返、样本积累、来源选择和本机跟踪，才可能真正控制 system clock；时区和 RTC 又分别处在显示层与启动基准层。

专题地图

- 知识专题 一个“时间”为什么要拆成 UTC、本地时间、系统时钟与 RTC
- 操作专题 用 `date` 与 `timedatectl` 建立可比较的时间基线
- 操作专题 查询 RTC，并控制 system clock 与 RTC 的复制方向
- 知识专题 chronyd 从“知道一个服务器”到“控制系统时钟”经历哪些状态
- 操作专题 配置 `server`、`pool`、`iburst` 与持久校时策略
- 操作专题 解读 `chronyc activity`、`sources -v` 与 `tracking`
- 诊断专题 `chronyd active`，但指定来源持续显示 `^?`
- 诊断专题 大偏差时怎样选择 `slew`、`step` 与 `makestep`
- 经典任务 配置指定时区和时间源，并恢复有效跟踪
- 参考解答 从基线、最小修改到分层验收
- 本章收束 把时间管理变成证据矩阵

阅读时持续回答

1. 当前问题发生在显示层、system clock、RTC，还是时间同步链？
2. `active`、`enabled`、来源可用、来源被选中和本机已同步分别由什么证据证明？
3. `sources` 与 `tracking` 各自回答哪个问题？
4. `^?` 的下一条最有区分度的证据是什么？
5. 当前偏差应继续 `slew`，还是确有授权执行 `step`？
6. 这条验证能证明什么，又不能证明什么？

01分清时间对象UTC / local / system clock / RTC

02建立基线date / timedatectl / hwclock

03核对配置server / pool / iburst / makestep

04观察来源activity / sources -v

05判断跟踪tracking / offset / leap status

06诊断与控制风险DNS / UDP 123 / slew / step

本章只轻量引用第 13 章的日志证据、第 15 章的计划任务影响，以及网络、DNS 与防火墙章节的接口；不展开 PTP。

第 14 章 · 正文

一台 Linux 主机上同时存在多种“时间事实”。`date` 展示的是 system clock 经时区规则转换后的结果；RTC 在主机关闭时保留启动基准；`chronyd.service` 只描述守护进程是否在运行；`chronyc sources -v` 描述时间源的测量与选择；`chronyc tracking` 才进一步说明本机时钟正在怎样跟随参考时间。把这些对象混为一谈，就会出现最典型的误判：服务明明是 active，却仍然没有有效参考源；时区差八小时，却误以为必须强制校时；来源名称已经出现，就提前宣布同步成功。

本章沿着“对象 → 状态 → 查询 → 配置 → 来源证据 → 本机跟踪 → 诊断与风险控制”推进。前一章负责日志检索，本章只说明时间错误会怎样削弱日志证据；下一章负责计划任务，本章只建立时区变更和时间跳变对日历调度的接口边界。

概念

系统时钟（system clock）是内核维护、运行中程序通常读取的墙上时钟。文件时间戳、日志记录、证书有效期和许多认证判断依赖它。它可以由管理员手工修改，也可以由 chronyd 通过改变走速或直接跳变来校正；因此“系统时钟当前可读”不等于“它来自可信来源”，更不等于“校正过程对业务无风险”。

概念

UTC、本地时间与时区 描述的是同一时刻的不同解释层。UTC 提供跨主机比较的共同基准；时区规则把 UTC 转换为本地显示和日历语义。修改时区通常不会平移绝对时刻，却会改变用户看到的时间以及按本地日历解释的任务。显示差八小时可能只是时区错误，不能直接推出 system clock 偏差八小时。

概念

RTC（Real-Time Clock）是硬件或虚拟硬件提供的持久时钟，主要在关机期间继续计时，并在启动阶段为 system clock 提供初始基准。运行中应用通常不直接依赖 RTC。`hwclock --systohc` 与 `--hctosys` 的复制方向相反，任一方向都可能把错误值扩散到另一个对象，因此操作前必须先明确谁是真源、谁是目标。

概念

NTP 同步状态 不是单一布尔值，而是一条状态链：服务正在运行、来源名称可解析、NTP 报文可往返、样本足够、来源可选、来源被选中、本机进入有效跟踪。`systemctl is-active chronyd` 只覆盖第一层；即使 `timedatectl` 显示 NTP service active，也不能替代 `sources` 与 `tracking` 的证据。

概念

时间源（time source）是 chronyd 用来测量参考时间的服务器、对等体或参考时钟。配置文件中出现一个名称，只说明“声明过这个对象”；它还要经过名称解析、网络往返、时间质量检查与来源选择。`sources -v` 中的模式字符说明来源类型，选择字符则说明它当前能否参与同步以及是否被选中。

概念

offset 表示本地时钟与参考时间之间估计的差异，但不同输出中的 offset 语义并不完全相同：`sources` 面向某个来源的最近测量，`tracking` 面向本机时钟相对当前参考时间的剩余校正与统计。offset 越小通常越好，却不存在适用于所有系统的固定毫秒阈值；验收必须结合任务、网络延迟和业务容忍度。

概念

stratum 表示时间源距离参考时钟的层级，而不是“数值越小就绝对越准确”的排行榜。一个网络不稳定的低层级来源可能不如稳定的高一层来源。判断来源时应同时观察可达性、测量误差、选择状态和本机 tracking，不能只盯住 Stratum 一列。

概念

slew 与 step 是两种不同的校正方式。slew 通过暂时加快或减慢时钟走速逐步消除偏差，时间连续但大偏差收敛较慢；step 直接把墙上时钟跳到新值，修正迅速却可能让日志顺序、缓存过期、认证、数据库事务和调度行为发生异常。`makestep` 是对这种风险的有限授权，不是看到 `^?` 后的通用修复命令。

操作语义 以下入口分别观察显示层、系统配置、RTC、来源状态和本机跟踪。先理解命令作用对象，再记关键形式。

date

SYNOPSIS

```
date [OPTION]... [+FORMAT]
date [-u] [+FORMAT]
```

读取 system clock，并按当前时区或 UTC 格式化输出。它适合建立本地时间与 UTC 的并列基线，但不能证明时间源有效。

重要参数 / 形式

```
date '+%F %T %Z %Z'
    显示日期、时间、数值偏移和时区缩写。
date -u '+%F %T %Z %Z'
    按 UTC 显示同一 system clock。
date --iso-8601=seconds
    生成便于记录和交换的 ISO 8601 时间。
```

timedatectl

SYNOPSIS

```
timedatectl [OPTIONS...] COMMAND ...
```

统一查询和修改 system clock、时区、RTC 解释模式及网络时间同步开关。综合状态是入口，不是 chrony 来源与跟踪的终局证据。

重要参数 / 形式

```
timedatectl
    查看 Local time、Universal time、RTC、Time zone 和同步状态。
timedatectl list-timezones
    列出可用时区名称。
timedatectl set-timezone Asia/Shanghai
    持久设置时区。
timedatectl set-ntp true
    请求启用系统识别的网络时间同步服务。
```

hwclock

SYNOPSIS

```
hwclock --show [--utc|--localtime]
hwclock --systohc
hwclock --hctosys
```

直接观察或复制 RTC 与 system clock。名称相似但方向相反；除只读查询外，都应先明确真源和目标。

重要参数 / 形式

```
--show
    只读显示 RTC。
--systohc
    把 system clock 写入 RTC。
--hctosys
    用 RTC 设置 system clock，运行中使用可能造成时间跳变。
```

chronyc activity / chronyc sources -v

SYNOPSIS

```
chronyc activity
chronyc sources [-a] [-v]
```

activity 汇总来源在线、离线和未解析状态；**sources -v** 展示每个来源的模式、选择状态、可达性、最近接收和最近样本。它们回答“来源怎样”，不直接等价于“本机时钟怎样”。

重要参数 / 形式

```
^*
    server 来源，且当前是最佳同步来源。
^+
    server 来源，参与组合但不是最佳来源。
^?
    当前不可选，可能是不可达、未同步或样本不足。
Reach / LastRx / Last sample
    分别观察近期响应历史、最近接收时间和最近偏差估计。
```

chronyc tracking

SYNOPSIS

```
chronyc tracking
```

展示 chronyd 当前如何控制本机 system clock。它回答参考来源、剩余校正、频率估计、误差边界和 leap 状态，是同步验收的核心证据之一。

重要字段

Reference ID / Ref time	当前参考对象和最近一次参考更新时间。
System time / Last offset / RMS offset	本机剩余校正、最近偏差和偏差统计。
Frequency / Residual freq / Skew	本机振荡器频率估计及其不确定性。
Leap status	Normal 等状态可参与正常验收；Not synchronised 表示尚未有效同步。

/etc/chrony.conf

SYNOPSIS

```
server hostname [option ...]
pool hostname [option ...]
makestep threshold limit
```

这是 RHEL 9 中 chronyd 的主要持久配置入口。编辑时应保留既有有效指令、避免重复来源，并通过服务重启和运行时证据确认新配置真正生效。

重要参数 / 形式

server time.example.com iburst	配置一个 NTP 服务器，并在初始阶段加快获得首批有效测量。
pool pool.example.com iburst	从一个池名称管理多个可替换来源。
iburst	加快首次更新，不表示长期高频轮询。
makestep 1.0 3	在有限的前几次更新中，允许超过阈值的偏差被 step。

chronyc makestep

SYNOPSIS

```
chronyc makestep
chronyc makestep <threshold> <limit>
```

控制正在运行的 chronyd 立即或临时允许 step。它可能造成墙上时钟跳变，不会修复 DNS、UDP 123 或错误时间源。

重要参数 / 形式

chronyc makestep	取消剩余 slew，并按当前估计立即跳变。
threshold	触发自动 step 的偏差阈值，单位为秒。
limit	该临时规则允许作用的后续更新次数。

[知识专题] 一个“时间”为什么要拆成 UTC、本地时间、系统时钟与 RTC

排查时间问题的第一步不是立即运行 chronyc makestep，而是确认到底哪个对象不符合预期。显示差八小时可能只是时区错误；系统刚启动时 RTC 偏差可能导致初始系统时间错误；chronyd 正常运行却没有参考源，则属于同步链而不是显示层。

① [知识点] UTC 表示时刻，本地时间表示解释结果

UTC 是跨主机比较时间的稳定基准。时区数据库规定某个地区相对 UTC 的偏移、历史变更以及夏令时规则。本地时间由同一 UTC 时刻按当前时区转换而来。


```
date                # 按当前时区显示
date -u             # 按 UTC 显示
date '+%F %T %Z'    # 日期、时间、数值偏移、时区缩写
date -u '+%F %T %Z %Z'
```

若 `date` 与 `date -u` 的差值符合时区偏移，不能仅凭两者不同判定系统时钟错误。修改时区会改变本地显示以及按本地日历解释的任务，但不会把同一时刻的 UTC 值平移。

② [知识点] system clock 是运行中系统的主要墙上时钟

系统时钟由内核维护。应用读取当前日期和时间、文件系统写入时间戳、日志记录事件、TLS 检查证书有效期时，通常依赖这个时钟。它可以被手工设置，也可以由 `chronyd` 通过渐进校正或跳变校正调整。

系统时钟可能向前或向后跳变，因此“经历了多长时间”的测量通常更适合单调时钟。单调时钟不在本章深入，但应记住：墙上时钟适合回答“事件发生在什么时候”，单调时钟适合回答“经过了多久”。

③ [知识点] RTC 为关机期间提供基准，不等于 system clock

RTC 可以在主机关机后继续计时。系统启动时，内核或早期用户空间可从 RTC 初始化系统时钟；运行期间通常由系统时钟服务应用。两者能够互相复制，但复制方向必须明确：

```
RTC → system clock: hwclock --hctosys
system clock → RTC: hwclock --systohc
```

这两个操作会改变目标对象。没有调查当前值、方向和业务影响时，不应把它们当作普通“同步命令”随意执行。

④ [知识点] 服务器 RTC 通常保持 UTC

RTC 本身通常不保存时区信息，系统必须约定其读数按 UTC 还是本地时间解释。服务器一般保持 UTC，以减少夏令时切换和多操作系统解释差异。`timedatectl set-local-rtc 1` 只适合确有兼容需求的环境，并会改变 `/etc/adtime` 中的模式记录。

⑤ [知识点] 时钟链存在多个独立状态

```
时区正确
≠ 系统时钟正确
≠ RTC 正确
≠ chronyd 正在运行
≠ 已取得有效 NTP 测量
≠ 已选中参考源
≠ 系统已进入有效跟踪
```

每个等号右侧都需要独立证据。任何一个局部状态都不能替代整条时间链。

Cheatsheet

UTC 是时刻基准；时区负责显示和日历解释；system clock 是运行中程序使用的墙上时钟；RTC 为关机期间保存基准；改时区不等于校时。

[操作专题] 用 date 与 timedatectl 建立可比较的时间基线

时间变更前要先保存基线。基线至少同时包括本地时间、UTC、时区、RTC、网络同步服务和系统同步判断。只截取一行“Local time”会丢失关键状态。

① [操作] 同时记录本地时间与 UTC

作用对象：系统时钟的两种显示方式。基本语义：`date` 读取系统时钟；`-u` 强制按 UTC 显示。典型形式：

```
date '+local=%F %T %Z %Z'
date -u '+utc=%F %T %Z %Z'
```

验证与边界：两条命令几乎同时执行但仍可能有少量秒差；比较重点是时区偏移和时刻逻辑，不是要求秒字段完全相同。

② [操作] 逐行解读 timedatectl

```
timedatectl
```

常见字段回答不同问题：

字段	回答的问题	不能替代的证据
Local time	当前时区下显示什么时间	不能证明 UTC 正确

字段	回答的问题	不能替代的证据
Universal time	当前 UTC 时刻	不能证明来源可达
RTC time	RTC 当前读数	不能证明运行中应用使用它
Time zone	当前时区和偏移	不能证明系统已同步
System clock synchronized	systemd 视角的同步判断	不替代 <code>sources</code> 与 <code>tracking</code>
NTP service	网络时间同步服务是否启用/活动	不代表已有有效参考源
RTC in local TZ	RTC 是否按本地时间解释	不代表 RTC 数值正确

③ [操作] 查询有效时区并设置目标时区

作用对象：系统的时区链接和解释规则。基本语义：`set-timezone` 选择时区数据库中的一个有效名称。典型形式：

```
timedatectl list-timezones | grep '^Asia/'
timedatectl set-timezone Asia/Shanghai
```

验证：

```
timedatectl
date '+%F %T %Z %Z'
date -u '+%F %T %Z %Z'
readlink -f /etc/localtime
```

边界：`/etc/localtime` 的链接是持久配置证据；`date` 是当前显示证据。修改时区后，要在第 15 章的计划任务范围中重新核对使用本地日历的 `cron` 与 `OnCalendar=`。

④ [操作] 区分手工设置时间与启用网络同步

```
timedatectl set-time '2026-07-12 15:30:00'
timedatectl set-ntp true
```

`set-time` 直接改变系统时钟；系统正在使用网络时间同步时，通常应先明确停用该同步机制，否则手工设时可能被拒绝或很快被重新校正。`set-ntp true` 请求启用系统识别的网络时间同步服务；在 RHEL 9 中通常由 `chronyd` 承担这一职责。启用同步后，不应再把手工设置时间作为常规维护路径。

安全边界：手工改时可能导致日志倒序、认证失败、证书判断异常和计划任务行为变化。考试题目未明确要求时，不应为了让输出“看起来正确”而手工设置时间。

⑤ [操作] 记录基线而不伪造真实输出

推荐把命令和真实输出保存在练习记录中：

```
{
  echo '=== local ==='
  date '+%F %T %Z %Z'
  echo '=== utc ==='
  date -u '+%F %T %Z %Z'
  echo '=== timedatectl ==='
  timedatectl
} | tee /root/time-baseline.txt
```

本讲义不提供虚构的 IP、offset、Reference ID 或 Reach 值。真实环境中必须读取本机证据。

Cheatsheet

先记录 ``date``、``date -u``、``timedatectl``；列时区用 ``list-timezones``，设置用 ``set-timezone``；改时区后验证 ``/etc/localtime`` 和本地/UTC 显示。

[操作专题] 查询 RTC，并控制 system clock 与 RTC 的复制方向

RTC 操作容易因为名称相似而方向相反。操作前先写清楚“以谁为真源、要改变谁”。

① [操作] 只读查询 RTC

```
hwclock --show
hwclock --show --utc
```

`hwclock --show` 读取硬件时钟并按系统记录的 RTC 模式解释。`--utc` 或 `--localtime` 可以显式指定解释方式，但不应在不了解当前 `/etc/adjtime` 时混用并据此修改系统。

② [操作] 查看 RTC 模式记录

```
timedatectl
cat /etc/adjtime
```

`/etc/adjtime` 的第三行通常记录 `UTC` 或 `LOCAL`。该文件还可保存 RTC 漂移相关信息。只需要判断模式时，优先读取，不要先执行会写入它的命令。

③ [操作] 从 system clock 写入 RTC

```
hwclock --systohc
```

作用对象：RTC。前提：已确认 system clock 是可信真源，且需要让 RTC 保存该基准。验证：再次读取 `hwclock --show`，并确认模式仍符合预期。边界：命令成功只证明复制完成，不证明 system clock 原本正确。

④ [操作] 从 RTC 初始化 system clock

```
hwclock --hctosys
```

作用对象：system clock。风险：运行中执行会使系统时间跳变，可能破坏日志、认证和应用顺序。常规启动流程会处理 RTC 初始化，日常排障不应把它作为第一动作。

⑤ [边界] 虚拟机中的 RTC 可能不是独立物理设备

云平台或虚拟机可能提供虚拟 RTC，也可能由宿主机、虚拟化工具或启动机制影响客户机时间。`hwclock` 失败或 RTC 与宿主机相关，并不直接证明 `chronyd` 配置错误。此类平台特性需要真实环境验证。

Cheatsheet

`--show` 只读 RTC；`--systohc` 是系统时钟写 RTC；`--hctosys` 是 RTC 写系统时钟；操作前先明确真源和目标。

[知识专题] chronyd 从“知道一个服务器”到“控制系统时钟”经历哪些状态

`chronyd` 的工作不是收到一个服务器地址后立即把本机时间复制过去。它需要解析名称、交换 NTP 报文、积累测量、排除异常来源、选择参考源，然后根据偏差和策略校正系统时钟。

① [知识点] 配置状态只是声明

`/etc/chrony.conf` 中出现：

```
server time.example.com iburst
```

只说明管理员声明了一个来源。它不能证明：

- 名称能够解析；
- 路由存在；
- UDP 123 请求和响应能够往返；
- 对端提供有效 NTP 响应；
- 已取得足够样本；
- 该源通过选择算法；
- 本机已经跟踪它。

② [知识点] 服务状态与同步状态分离

```
systemctl is-active chronyd.service
systemctl is-enabled chronyd.service
```

`active` 回答守护进程当前是否运行；`enabled` 回答重启后的持久启动配置。两者都不能证明源可用或系统已同步。

③ [知识点] 来源测量、选择和跟踪是三层证据

```
chronyc activity      → 已知来源的在线、离线、未解析数量
chronyc sources -v    → 每个来源的模式、可达性、样本和选择状态
chronyc tracking       → 本机相对选中参考时间的跟踪状态
```

最小闭环要求：服务存在、来源可用、至少一个有效参考源被选择、tracking 不处于未同步状态。

④ [知识点] stratum 是到参考时钟的层级距离

stratum 反映来源与参考时钟之间的层级。直接连接参考时钟的服务器通常是较低 stratum，沿同步链向下增加。它不是“数值越小就一定越准确”的简单排名；网络抖动、源质量、根距离和选择算法同样重要。

⑤ [知识点] offset 不是单一固定验收阈值

chronyc sources -v 和 chronyc tracking 中有多个偏差指标。它们表达的对象和时间窗口不同。不能规定所有主机都必须小于同一个毫秒值。考试最小判断通常是进入有效同步；生产环境还要按 Kerberos、证书、数据库和分布式系统要求设定偏差目标。

Cheatsheet

配置只声明来源；active/enabled 只证明服务层；activity 看来源解析状态；sources 看测量与选择；tracking 看本机时钟。

[操作专题] 配置 server、pool、iburst 与持久校时策略

编辑 chrony 配置时应在现有系统上做最小修改。考试和真实服务器都可能已经包含有效的 driftfile、makestep、rtcsync 或其他来源。整文件覆盖会删除无关但必要的状态。

① [操作] 变更前检查配置和备份

```
grep -Ev '^\s*(#|$\)' /etc/chrony.conf
cp -a /etc/chrony.conf /etc/chrony.conf.pre-rhcsa14
```

备份名要避免被配置加载机制误当成有效片段。RHEL 9 的确定主入口是 /etc/chrony.conf；是否还加载 drop-in，应依据本机文件内容和包版本确认，不跨发行版假设。

② [配置] server 指定一个来源

```
server time.example.com iburst
```

server 指定一个 NTP 来源的名称或地址。一个名称若解析为多个地址，chronyd 的具体地址管理仍由实现处理；在学习模型中，server 表示一个配置来源条目。

③ [配置] pool 从一个池名称管理多个来源

```
pool pool.example.com iburst
```

pool 适合一个 DNS 名称背后提供多个可替换服务器的场景。它与“配置多个独立 server 行”都能增加来源冗余，但来源管理语义不同。

④ [参数] iburst 加快首次有效测量

iburst 在来源初次可用或尚未同步时快速发送一组请求，以更快取得初始测量。它不是让 chronyd 永久高频轮询，也不能绕过 DNS、路由、防火墙和远端故障。

⑤ [配置] makestep threshold limit 是持久策略

常见配置：

```
makestep 1.0 3
```

表示在前若干次时钟更新中，如果修正量超过阈值，可通过 step 立即校正。阈值和次数必须按配置语义读取，不要把它误解为“每次偏差超过 1 秒都永久强制跳变”。

⑥ [配置] rtcsync 与 RTC 的关系

```
rtcsync
```

在 Linux 上，rtcsync 让内核在系统时钟处于同步状态时周期性更新 RTC。它不等同于每次运行 hwclock --systohc，也不能证明当前 RTC 与系统时钟已经完全一致。

⑦ [操作] 应用配置并分别验证服务状态

```
systemctl restart chronyd.service
systemctl is-active chronyd.service
systemctl is-enabled chronyd.service
systemctl status chronyd.service --no-pager -l
```

若题目同时要求当前启动和重启后自动启动：

```
systemctl enable --now chronyd.service
```

仍要分别验证 active 与 enabled，不能用一条 status 的视觉印象代替明确检查。

Cheatsheet

先备份并保留既有配置；`server` 单来源，`pool` 池来源；`iburst` 加快初始采样；`makestep` 是策略；`rtcsync` 让同步状态下的内核更新 RTC。

[操作专题] 解读 `chronyc activity` 与 `chronyc sources -v`

`sources -v` 是最常见的来源证据，但不能只看是否出现目标名称。必须同时看来源模式、选择标记、Reach、最近接收时间和最近样本。

① [操作] 先看来源总体活动状态

```
chronyc activity
```

它会汇总在线、离线、处于 burst、未解析等来源数量。若存在未解析来源，优先检查 DNS；若所有来源都离线，优先检查网络、配置和远端响应。

② [输出] 来源模式字符

在 `sources -v` 的第一列，常见模式包括：

- `^`：NTP server；
- `=`：peer；
- `#`：本地参考时钟。

本章主要训练客户端使用 `^` 来源，不展开 peer 和参考时钟部署。

③ [输出] 选择状态字符

标记	典型含义	下一步判断
<code>*</code>	当前选中的同步来源	再看 tracking 是否正常
<code>+</code>	可接受并参与组合的候选来源	正常候选，不必强制变成 <code>*</code>
<code>-</code>	可用，但未被选择参与组合	结合其他来源质量判断
<code>?</code>	不可选择：可能不可达、未同步或样本不足	查 DNS、网络、远端与样本
<code>x</code>	被判定为错误来源	调查来源冲突和质量，不要强制使用
<code>~</code>	时间变化过大	等待或调查抖动、网络和来源质量

来源模式和选择标记通常组合显示，例如 `^*`、`^+`、`^?`。

④ [输出] Stratum、Poll、Reach 与 LastRx

- `Stratum`：来源层级；数值不能单独决定优劣；
- `Poll`：以 2 的幂表示轮询间隔；
- `Reach`：最近八次传输的八进制可达寄存器；持续为 `0` 表示没有收到有效响应；
- `LastRx`：距最近一次有效样本的时间。

Reach 不是简单百分比，也不能在服务刚启动几秒时要求立即达到稳定值。

⑤ [输出] Last sample 的三组信息

最近样本通常显示：

```
adjusted_offset[measured_offset] +/- error_bound
```

括号外是经过处理的估计偏差，括号内是原始测量偏差，`+/-` 后是误差界。学习时重点是能识别符号、单位和数量级，不把任一值脱离来源状态直接当成最终业务结论。

⑥ [验证] 有目标名称不等于成功

最低可接受证据不是“输出中有 `time.example.com`”，而是：

```
名称已解析
→ Reach 开始积累有效响应
→ 来源不持续为 ^? 或 ^x
→ 至少存在 ^* 或其他有效参考组合
→ tracking 不再 Not synchronised
```

Cheatsheet

activity 看总体；`^^` 是 server；`*`` 选中、``+`` 候选、``-`` 未选、``?`` 不可选、``x`` 错误源；Reach 持续 0 查往返链；最后必须进入 tracking。

[操作专题] 用 `chronyc tracking` 判断本机时钟是否真正受控

`sources` 以来源为中心；`tracking` 以本机系统时钟为中心。一个来源看起来可用时，还要确认本机是否已经选中参考并维持合理跟踪。

① [输出] Reference ID 与 Reference time

- `Reference ID`：当前参考来源的标识；可能显示地址或特定标识；
- `Reference time`：最近一次处理参考测量的时间。

若没有有效参考来源，Reference ID 可能为未同步状态。不要在讲义中背某个固定地址，因为真实环境不同。

② [输出] Stratum 与 Leap status

`Stratum` 表示本机在同步层级中的位置。`Leap status` 是重要的终态判断：正常跟踪时应处于正常状态；`Not synchronised` 明确说明尚未进入有效同步。

③ [输出] System time、Last offset 与 RMS offset

- `System time`：系统时钟相对 NTP 时间的当前估计差异；
- `Last offset`：最后一次时钟更新时的估计偏差；
- `RMS offset`：偏差的长期均方根统计。

它们回答不同时间窗口的问题，不能互相替代。正负号表示系统时钟相对参考时间的方向。

④ [输出] Frequency、Residual freq 与 Skew

`chronyd` 不只修正一次偏差，还估计本机时钟的频率误差：

- `Frequency`：系统时钟快慢的长期估计，单位通常为 ppm；
- `Residual freq`：当前仍未消除的频率差；
- `Skew`：频率估计的不确定性。

这些字段用于理解稳定性，不要求 RHCSA 学员根据单个数值手工调频。

⑤ [输出] Root delay、Root dispersion 与 Update interval

它们反映到参考时钟路径的延迟、累计误差和更新间隔。生产环境评估时间质量时有价值；考试最小验收仍以有效来源和正常 tracking 为核心。

⑥ [操作] 组合最小跟踪检查

```
chronyc sources -v
chronyc tracking
timedatectl
```

判定时避免只 `grep` 一个单词。建议人工确认：

```
存在有效选择来源
Reference time 合理更新
Leap status 不是 Not synchronised
System time/offset 没有持续发散
```

Cheatsheet

`sources` 回答“来源怎样”；`tracking` 回答“本机怎样”；重点看 Reference、Stratum、System/Last/RMS offset、Frequency/Skew、Update interval、Leap status。

[诊断专题] `chronyd` active，但指定来源持续显示 `^?`

`^?` 不是一个需要立即 step 的信号，而是“该来源当前不能参与选择”的症状。诊断要寻找最有区分度的下一条证据。

① [症状] 固定当前证据

```
systemctl is-active chronyd.service
systemctl is-enabled chronyd.service
chronyc activity
chronyc sources -v
chronyc tracking
```

记录目标来源是否未解析、Reach 是否为 0、LastRx 是否为空或过久，以及 tracking 是否 `Not synchronised`。

② [假设一] `chronyd` 没有读取预期配置

```
grep -nEv '^s*(#|$)' /etc/chrony.conf
systemctl status chronyd.service --no-pager -l
journalctl -u chronyd.service -b --no-pager
```

检查名称拼写、重复来源、编辑错文件、服务重启失败和配置解析错误。不要因为服务仍 active 就假设新配置已成功加载；重启失败时旧进程状态和退出信息都要看。

③ [假设二] 名称无法解析

```
getent ahosts time.example.com
chronyc activity
```

`getent` 使用系统名称服务链，比直接指定某个 DNS 工具更贴近应用解析结果。若失败，进入第 19 章的 NSS/DNS 诊断；本章只确认时间源名称是否能够得到地址。

④ [假设三] 网络路径或 UDP 123 往返失败

```
ip route get <resolved-address>
```

随后依据环境检查主机防火墙、上游 ACL/NAT 和远端服务。NTP 客户端通常从临时源端口向服务器 UDP 123 发送请求并等待响应；只开放本机入站 `ntp` 服务不是通用客户端修复。

普通 `telnet` 不能验证 UDP。`nc -u` 也很难证明对端提供了有效 NTP 响应，因此 `chronyd` 自己的 Reach、`ntpdata`（目标版本支持时）和抓包证据更有价值。

⑤ [假设四] 远端响应但样本尚不足

服务刚启动、DNS 刚恢复或来源刚上线时，`^?` 可能短暂存在。先确认 Reach/LastRx 是否开始变化，再等待合理的采样窗口。无限等待不是诊断；若 Reach 持续 0，应回到网络和远端响应。

⑥ [假设五] 来源本身未同步或被判异常

若能交换报文但来源仍不可选，检查远端 leap 状态、来源质量以及本机与其他来源的冲突。`^x` 表示 `chronyd` 已把它判为错误源，不能通过删除其他来源来“强迫”它变成正确源。

⑦ [最小修复与再验证]

修复必须对应已证实的故障层：

```
拼写错误 → 修正配置并重启 chronyd
DNS 错误 → 修复名称服务或使用题目允许的正确地址
路由/ACL 错误 → 修复具体路径或策略
远端不可用 → 使用题目授权的可用来源
样本不足 → 等待并观察 Reach/LastRx
```

再验证：

```
chronyc activity
chronyc sources -v
chronyc tracking
timedatectl
```

^^? 链路：配置加载 → DNS → 路由 → UDP 123 往返 → 远端状态 → 样本 → 选择 → tracking。修复哪一层，就重新验证哪一层及其下游。

[诊断专题] 大偏差时怎样选择 slew、step 与 makestep

时间偏差大并不自动授权跳变。首先判断当前偏差、系统所处阶段和业务对时间连续性的要求。

① [知识点] slew 渐进收敛

slew 通过临时改变系统时钟走速，使偏差逐渐消除。优点是墙上时钟通常不会突然向前或向后跳；缺点是大偏差需要较长时间才能收敛。

② [知识点] step 直接改变墙上时钟

step 立即把系统时钟移到目标附近。向前跳可能让超时提前，向后跳可能产生时间重复。日志排序、数据库事务、缓存过期、证书和调度器都可能受影响。

③ [配置] 启动阶段的 makestep

```
makestep 1.0 3
```

其目的通常是允许启动早期在有限次数内修正大偏差。它比在系统长期运行后无条件 step 更可控，但仍需理解阈值和次数。

④ [运行时操作] chronyc makestep

```
chronyc makestep
```

这是对正在运行的 chronyd 发出的控制命令，可能立即造成时间跳变。它不是持久配置，也不是 ^^? 的修复命令。执行前至少要确认：

```
当前有效参考源是谁
当前偏差是多少
是否允许时间跳变
关键应用是否已评估或停机
执行后怎样验证
```

⑤ [边界] 先修来源，再处理偏差

没有有效参考源时，step 没有可信目标。来源持续 ^^? 时先修复来源链。若系统完全离线且题目要求手工设置时间，应明确真源、授权和后续恢复网络同步的步骤。

⑥ [再验证] 校正后不是只看 date

```
date
date -u
chronyc sources -v
chronyc tracking
timedatectl
```

还要检查关键应用、日志时间和计划任务影响。应用专项验证不在本章完整展开，但必须写入变更记录。

Cheatsheet

slew 保持连续、渐进收敛；step 立即跳变；配置 `makestep` 是有限策略，`chronyc makestep` 是运行时动作；没有有效来源时先修来源。

[知识专题] 时间证据的可信边界与工作迁移

时间同步成功后，系统能够更可靠地记录新事件，但它不会自动修复过去已经写错的时间戳。跨主机调查也不能只看到“都启用了 chronyd”就认为时间轴一致。

① [边界] 错误时区与错误系统时钟的影响不同

错误时区通常不改变事件的 UTC 先后，但会误导人类阅读和本地时间范围查询。错误系统时钟则会真正改变写入的时间戳，破坏跨主机排序。

② [边界] 后续同步不会改写历史日志

某事件在错误系统时间下写入后，chronyd 后续恢复同步不会把旧日志时间自动纠正。调查者要结合启动轮次、消息内容、外部证据和偏差窗口重建时间线。完整 journal 技巧归第 13 章。

③ [边界] active 不代表集群时间一致

多台主机都显示 `chronyd active`，仍可能分别跟踪不同来源、处于未同步状态或有不同偏差。跨主机验收至少要比对来源、Reference time、Leap status 和偏差，而不是只比较服务状态。

④ [工作迁移] 变更前后保存证据矩阵

推荐记录：

本地时间 / UTC / 时区
RTC 模式与读数
chronyd active / enabled
配置的 server / pool
DNS 解析结果
activity / sources / tracking
是否执行过 step
应用影响与观察窗口

⑤ [工作迁移] 自动化应等待条件，不只等待服务启动

在脚本或自动化中，systemctl start chronyd 返回成功仅表示启动请求完成。需要时间正确后再启动关键业务时，应使用 chronyc waitsync 等条件式等待，并根据目标版本定义可接受尝试次数和偏差；本章只作为扩展，不把它替代人工理解。

Cheatsheet

后续同步只改善未来，不改写历史；跨主机比较 sources/tracking，不只看 active；变更记录要包括是否发生 step。

[经典任务] 配置指定时区和时间源，并诊断 active 但持续 ^?

环境

你管理一台 RHEL 9 主机。当前没有可由本讲义控制的 live VM，以下任务用于静态训练；在真实练习机上必须读取实际输出。

已知状态：

- 当前时区不是 Asia/Shanghai；
- chronyd.service 当前为 active，且配置为开机启动；
- /etc/chrony.conf 已有其他有效指令，禁止清空或整文件覆盖；
- 题目要求加入指定来源：

server time.example.com iburst

- 修改后，该来源在 chronyc sources -v 中持续显示 ^?；
- 尚未确定是配置加载、DNS、网络、远端响应还是样本不足。

目标终态

1. 时区持久设置为 Asia/Shanghai；
2. 能解释本地时间、UTC 和 RTC 的关系；
3. RTC 模式被检查，未在无明确需求时改成本地时间；
4. 指定来源配置存在且不重复，既有有效配置被保留；
5. chronyd 当前运行且重启后自动启动；
6. 指定名称能够通过系统名称服务解析；
7. 指定来源取得有效测量，不再持续为 ^? 或 ^x；
8. 至少存在一个有效参考来源；
9. chronyc tracking 不显示 Not synchronised；
10. 未经授权，不通过手工改时或强制 step 掩盖来源故障。

限制条件

- 不关闭 SELinux 或整个防火墙作为默认答案；
- 不删除其他有效时间源来强迫错误源被选中；
- 不把“服务 active”“命令返回 0”“输出中有源名称”当成最终完成；
- 不编造真实 IP、Reach、offset、Reference ID 或标准输出；
- 不把固定毫秒阈值当作所有环境的通用标准。

验收证据

层次	推荐证据	验收意图
时间显示	date、date -u、timedatectl	时区与 UTC 关系正确

层次	推荐证据	验收意图
RTC	<code>timedatectl、hwclock --show</code>	RTC 模式已检查
配置	检查 <code>/etc/chrony.conf</code>	目标源存在且保留既有配置
服务当前/持久	<code>is-active、is-enabled</code>	两个状态分别成立
解析	<code>getent ahosts time.example.com</code>	系统解析链可用
来源	<code>activity、sources -v</code>	来源可达并参与选择
跟踪	<code>tracking</code>	本机进入有效跟踪
安全边界	变更记录	未无调查执行 step

[参考解答] 从基线、最小修改到分层验收

① 调查并保存修改前基线

```
date '+local=%F %T %Z %Z'
date -u '+utc=%F %T %Z %Z'
timedatectl
hwclock --show
systemctl is-active chronyd.service
systemctl is-enabled chronyd.service
grep -nEv '^s*(#|$)' /etc/chrony.conf
chronyc activity
chronyc sources -v
chronyc tracking
```

调查阶段不尝试 step。若 `hwclock` 在虚拟机中不可用，记录错误并继续调查 system clock 与 chrony，不把它误判为 NTP 来源故障。

② 设置时区并验证持久状态

```
timedatectl list-timezones | grep '^Asia/Shanghai$'
timedatectl set-timezone Asia/Shanghai

timedatectl
date '+local=%F %T %Z %Z'
date -u '+utc=%F %T %Z %Z'
readlink -f /etc/localtime
```

这里验证的是显示和持久时区，不是 NTP 同步。

③ 备份并最小编辑 chrony 配置

```
cp -a /etc/chrony.conf /etc/chrony.conf.pre-rhcsa14
grep -nF 'time.example.com' /etc/chrony.conf
```

若不存在目标行，在不删除其他有效指令的前提下加入：

```
server time.example.com iburst
```

再次检查避免重复：

```
grep -nF 'time.example.com' /etc/chrony.conf
```

④ 应用配置并验证服务层

```
systemctl restart chronyd.service
systemctl is-active chronyd.service
systemctl is-enabled chronyd.service
systemctl status chronyd.service --no-pager -l
journalctl -u chronyd.service -b --no-pager
```

若题目发现服务未启用，再执行：

```
systemctl enable --now chronyd.service
```

不要因为 restart 返回成功就跳过日志和状态检查。

⑤ 针对 ^? 选择下一条证据

先查系统解析链：

```
getent ahosts time.example.com
chronyc activity
```

若名称能够解析，选择其中实际地址检查路由：

```
ip route get <resolved-address>
```

然后依据环境检查具体防火墙、ACL、NAT 和远端 NTP 服务。不得用“关闭所有防火墙”替代定位。若有权限和工具，可用抓包确认请求与响应，但不在答案中虚构抓包结果。

⑥ 判断是持续故障还是采样尚未完成

```
chronyc sources -v
```

观察：

- Reach 是否持续为 0；
- LastRx 是否开始更新；
- 目标源是否仍持续 ^?；
- 是否出现 ^x 或 ^~；
- 其他来源是否正常。

若 Reach 开始积累，可等待合理采样窗口后再查；若持续为 0，继续调查往返链和远端响应。

⑦ 完成来源与跟踪验收

```
chronyc activity
chronyc sources -v
chronyc tracking
timedatectl
```

验收逻辑：

```
目标名称已解析
→ 来源取得有效响应和样本
→ 不持续为 ^? 或 ^x
→ 存在有效选中/组合来源
→ tracking 不为 Not synchronised
→ 时区和服务持久状态仍正确
```

题目要求“使用指定来源”时，应确认该来源本身进入有效状态；若还有多个来源，不能武断要求它一定显示 ^*，除非题目明确要求唯一来源或当前选择逻辑应当选中它。

⑧ 何时才考虑 makestep

只有在以下事实明确后才评估：

```
已有可信参考源
当前偏差确实很大
业务允许时间跳变
题目或变更授权允许
```

运行时命令：

```
chronyc makestep
```

执行后重新检查 date、sources -v、tracking，并检查日志、认证、数据库和计划任务影响。经典任务未授权 step，因此默认答案不执行它。

⑨ 典型错误

错误	为什么不成立
看到 active 就结束	只证明守护进程层
看到源名称就结束	可能仍是 ^?、Reach 0 或未选中
ping 不通就认定 NTP 不通	ICMP 与 UDP 123 是不同协议链
用 telnet host 123 验证	telnet 验证 TCP，NTP 通常使用 UDP
直接 chronyc makestep	不修复 DNS、网络或错误来源
删除所有其他来源	可能降低冗余并掩盖错误源
关闭整个防火墙	破坏性大，且未定位具体规则
把 RTC 改成本地时间	通常增加 DST 和多系统风险

[本章收束] 把时间管理变成证据矩阵

本章的核心不是“让命令不报错”，而是把时间链拆成可独立验证的对象：

UTC / 本地显示 / 时区
→ system clock
→ RTC
→ chronyd 当前与持久状态
→ 配置的 server / pool
→ DNS 与 UDP 123 往返
→ sources 测量与选择
→ tracking 跟踪状态
→ step/slew 安全边界

面对时间异常时，先问“哪个对象错误”，再选择命令。面对 ^?，按照配置、解析、网络、远端、样本、选择、跟踪推进。面对大偏差，先建立可信来源和影响评估，再决定是否允许 step。

主要判断表

看到的证据	可以证明	不能证明
chronyd.service 为 active	守护进程当前运行	已有有效来源或已经同步
sources -v 中出现来源名称	chronyd 知道该来源	来源可达、可选或被选中
来源显示 ^* 或 ^+	来源已被选中或参与组合	偏差已经满足具体业务阈值
tracking 的 Leap status 为 Normal	本机正处于有效跟踪状态	其他主机也与本机一致
date 显示看起来正确	当前显示结果合理	来源可信、RTC 正确或历史日志已修复

最终 Cheatsheet

date '+%F %T %Z %Z'; date -u '+%F %T %Z %Z'
timedatectl
timedatectl set-timezone Asia/Shanghai
hwclock --show
systemctl is-active chronyd.service
systemctl is-enabled chronyd.service
chronyc activity
chronyc sources -v
chronyc tracking
getent ahosts time.example.com
ip route get <resolved-address>
journalctl -u chronyd.service -b --no-pager

向下一章交接

本章已经建立“时区改变本地日历解释、step 造成墙上时钟跳变”的边界。第 15 章将在此基础上讨论 at、cron 与 systemd timer 的持久配置、执行环境和实际触发行为。

一次性任务、周期任务与 systemd Timer

从“什么时候运行”走到“以谁、在什么环境、怎样留下可验证结果”。

[对象模型](#) [操作语义](#) [验证](#) [诊断](#) [经典任务](#)

[大字号阅读版](#)

本章阅读导航

先抓住一条主线：计划任务不是一条“到点运行”的孤立命令，而是一条由调度定义、调度器、执行身份、执行环境、工作负载和结果证据共同组成的链路。

专题地图

- 知识专题 先识别任务生命周期，再选择调度器
- 操作专题 使用 `at` 安排未来的一次执行
- 知识专题 正确解释 `cron` 五字段与日期匹配
- 操作专题 用户 `crontab`、系统 `crontab` 与 `/etc/cron.d`
- 诊断专题 手工执行成功，但 `cron` 失败
- 知识专题 `timer` 和 `service` 是两个状态对象
- 操作专题 持久 `timer`、`transient timer` 与安全迁移
- 诊断专题 `timer` 在等待，但任务没有正确完成
- 经典任务 周期健康检查与 `cron`→`timer` 迁移

阅读时持续回答

- 这是一次性、固定日历还是相对时间任务？
- 调度定义保存在哪里，由哪个进程解释？
- 任务最终以哪个用户和工作目录执行？
- `PATH`、`SHELL`、`HOME` 与输出是否明确？
- 当前证据只证明“已配置”，还是已经证明“已触发”？
- `timer` 状态与 `service` 结果是否分别检查？
- 关机错过后是否需要补执行？
- 最终产物是否正确、非空且时间合理？

01识别生命周期一次、日历重复或相对时间

02确认定义`job`、`crontab` 或 `timer`

03确认调度器`atd`、`crond` 或 `systemd`

04重建执行环境用户、目录、`PATH` 与输出

05判断触发与退出`NEXT/LAST`、`Result` 与退出码

06验收真实产物内容、所有者、大小和时间

阅读提示：先用概念块建立对象边界，再用操作语义速查确认接口，最后通过专题、经典任务和参考解答完成分层验证。

第 15 章 · 正文

计划任务表面上是在回答“什么时候运行”，真正决定任务是否可靠的却不止时间。调度定义必须交给某个调度器；调度器到点后以特定用户、环境和工作目录启动命令；命令还要把退出状态、标准输出、标准错误和业务产物留成证据。只看到一行 `crontab`、一个 `at job`，或者一个处于 ``active (waiting)`` 的 `timer`，都不能证明任务已经正确完成。本章从三类需求切入：未来只执行一次、按日历反复执行、由 `systemd` 按日历或相对时间激活。学习重点不是把三套语法混在一起背，而是建立一条统一证据链：

调度需求
→ 调度定义
→ 调度器状态
→ 执行身份与环境
→ 工作负载退出状态
→ 输出、日志与真实产物

最常见的误判有三类：手工执行成功就认为 `cron` 一定成功；把用户 `crontab` 与系统 `cron` 的字段混为一谈；看到 `timer` 正在等待就认为 `service` 已经正确完成。`systemd` 的通用 `unit` 语义属于第 12 章，日志系统属于第 13 章，时区与同步属于第 14 章。本章只引用这些前置证据，不重新展开它们。

概念

一次性任务 是只在未来某个时刻取出并执行一次的调度对象。本章以 ``at` job` 为代表：提交后它进入由 ``atd`` 管理的队列，被执行或删除后不会自动生成下一次任务。队列中存在只能证明定义仍在等待，不能证明届时环境、权限和外部依赖一定满足。

概念

周期任务 是反复接受日历匹配的规则，而不是从创建时刻开始不断累加固定间隔。`cron` 通过分钟、小时、月中日、月份和星期五个字段决定匹配；`systemd timer` 既能表达日历，也能表达开机后或上次激活后的相对时间。选择对象时应先判断任务生命周期，而不是先选自己最熟悉的命令。

概念

执行身份 决定任务拥有哪些 `UID`、`GID`、`HOME`、文件权限和安全上下文。用户 `crontab` 天然以该用户运行；系统 `crontab` 与 ``etc/cron.d`` 通过额外用户字段指定身份；`timer` 通常把身份写在被激活的 `service` 中。身份错误时，时间表达式完全正确也可能只得到权限失败。

概念

执行环境 是调度器启动工作负载时提供的变量、Shell、工作目录、标准输入输出和可用凭据。它通常比交互登录会话精简，不会自动包含 alias、函数、SSH agent 或终端提示。手工成功而 cron 失败时，最有区分度的证据不是反复重跑，而是以目标用户和 ``env -i`` 重建精简环境。

概念

日历表达式 描述哪些墙上时刻符合触发条件。cron 使用五字段，月中日与星期在同时受限时具有特殊的 OR 匹配；``OnCalendar=`` 使用 systemd 的日历语法，可先由 ``systemd-analyze calendar`` 展开。语法可解析只证明表达式成立，不证明调度器已加载或工作负载会成功。

概念

timer/service 对把“何时触发”和“具体执行”拆成两个 unit。``timer`` 管理 NEXT、LAST 和等待状态；``service`` 管理 ``User=``、``WorkingDirectory=``、``ExecStart=``、退出码和 journal。timer 的 ``active (waiting)`` 不能替代 service 的 ``Result=``，service 成功也不能替代业务产物检查。

概念

补执行 是系统在日历触发点处于关闭或 timer 未激活时，恢复后补一次遗漏工作的能力。``Persistent=true`` 只对日历 timer 有意义，它不是逐条回放所有错过的周期，也不能修复工作负载本身的失败。是否真正补跑必须在可控的关机、停用与恢复场景中实测。

操作语义以下入口分别观察或改变一次性队列、cron 规则与 systemd 调度。先明确命令的作用对象，再选择参数和验证层次。

at 命令组

SYNOPSIS

```
at [options] TIMESPEC
atq
at -c JOB_ID
atrm JOB_ID
```

提交、列出、审阅和删除由 ``atd`` 管理的一次性任务。

重要参数 / 形式

```
at now + 20 minutes
    从标准输入读取命令，并安排在 20 分钟后执行一次。
-f FILE
    从文件读取任务正文，适合需要保存和审阅的作业。
at -c JOB_ID
    查看保存的完整脚本、环境、umask、工作目录与最终命令。
atrm JOB_ID
    删除尚未执行的队列项；不能停止已经启动的进程。
```

crontab

SYNOPSIS

```
crontab [-u USER] {-e|-l|-r}
crontab [-u USER] FILE
```

编辑、列出、删除或安装某个用户的 crontab。用户表每条任务只有五个时间字段，执行用户由表的所有者决定。

重要参数 / 形式

```
-e
    通过编辑器修改目标用户的 crontab。
-l
    列出当前安装的任务表，是变更前后最直接的静态证据。
-u USER
    由 root 管理指定用户的表，避免误查当前登录用户。
-r
    删除整张表，不是删除一行；使用前必须明确风险。
```

系统 crontab 与 `/etc/cron.d`

SYNOPSIS

```
minute hour day month weekday user command
```

由系统管理员集中保存任务，并在五个时间字段之后显式指定执行用户。

重要字段 / 形式

user

系统表额外字段；写进用户 crontab 会把用户名当成命令。

`PATH=...`

显式限制命令搜索范围，避免依赖登录环境。

`SHELL=/bin/bash`

指定解释任务行的 Shell；它不自动加载用户的交互启动文件。

`MAILTO=""`

禁用 cron 邮件尝试；重要任务仍应显式保存 stdout/stderr。

`\%`

在命令部分表示字面百分号：未转义 ``%`` 会切分命令与标准输入。

systemd timer

SYNOPSIS

```
[Timer]
OnCalendar=...
OnBootSec=...
OnUnitActiveSec=...
Persistent=true
```

以 `.timer` 表达触发时间，以 `.service` 表达执行身份、命令和退出结果。

重要参数 / 形式

`OnCalendar=`

使用墙上日历时间；可与 `Persistent=true` 组合补一次遗漏。

`OnBootSec=`

从系统启动后经过指定时长触发。

`OnUnitActiveSec=`

从目标 unit 上次激活后计算下一次间隔。

`RandomizedDelaySec=`

在允许范围内随机推迟，帮助多台主机摊开负载。

`AccuracySec=`

允许 systemd 合并唤醒的精度窗口，不是随机延迟。

`systemctl list-timers`

SYNOPSIS

```
systemctl list-timers [--all] [PATTERN...]
```

列出 timer 的下一上和上一次触发时间，以及它将激活的 unit。

重要列 / 形式

`NEXT / LEFT`

预计的下一触发时刻和剩余时间。

`LAST / PASSED`

最近一次触发时刻和距今时间；不表示 service 一定成功。

`--all`

同时显示未运行或没有下一触发时间的 timer，适合排错。

`systemd-analyze calendar`

SYNOPSIS

```
systemd-analyze calendar EXPRESSION
```

解析 ``OnCalendar=` 表达式，显示规范形式和后续触发时间。

重要输出 / 边界

`Normalized form`

展示 systemd 实际理解的规范化表达式。

`Next elapse`

展示下一日历匹配；仍未包含 service 运行结果。

`systemd-analyze verify`

检查 unit 文件语法和部分引用关系，不能替代真实执行。

systemd-run

SYNOPSIS

```
systemd-run --on-active=TIME COMMAND [ARG...]  
systemd-run --on-calendar=EXPR COMMAND [ARG...]
```

创建 transient timer 与 service，用于短期验证 systemd 调度，不在磁盘上建立持久 unit 文件。

重要参数 / 形式

- `--on-active=`
从 transient timer 激活后计算延迟。
- `--on-calendar=`
使用 systemd 日历表达式创建临时日历 timer。
- `--unit=NAME`
指定便于查询的 unit 基名，避免依赖自动生成名称。
- `--property=...`
为 transient service 设置受支持的 unit 属性；使用前应核对本机版本。

[知识专题] 先识别任务生命周期，再选择调度器

一次性、周期日历和 systemd 事件调度解决的不是同一个问题。最短的命令未必是最稳的方案，功能更多的 timer 也不自动优于 cron。考试题如果明确要求 crontab，应按题意使用 cron；已有简单而稳定的 cron 任务，也不应只为了“看起来现代”而迁移。

① [知识点] 三类调度对象的边界

需求	首选对象	定义保存在哪里	最关键的验收
将来执行一次	at job	at spool，由 atd 管理	队列、任务正文、实际产物
简单重复日历	crontab entry	用户 crontab、/etc/crontab 或 /etc/cron.d/*	字段、用户、crond、输出和产物
日历或相对时间，并要求补跑、随机延迟、依赖和统一结果	.timer + .service	systemd unit 文件	NEXT/LAST、service Result、journal 和产物
临时验证一次 systemd 调度思路	transient timer	systemd manager 内存状态	生成的 timer/service、结果和非持久边界

at 适合“今晚执行一次”或“20 分钟后恢复配置”。cron 适合“每周三 15:30”这类稳定日历。systemd timer 适合“系统关机错过后补一次”“开机 10 分钟后执行”“每次任务激活后再过 30 分钟”“多台主机随机摊开”等需求。

② [知识点] 调度定义和工作负载是两个对象

调度配置最好只表达：

何时
→ 以谁
→ 启动哪个稳定入口

业务逻辑应放在可独立运行的脚本或程序中。脚本应有明确 shebang、绝对路径、可执行权限、稳定退出码，以及可检查的输出或产物。把多层管道、条件判断、日期拼接和密钥读取全部塞进一行 crontab，会让引用、%、工作目录和错误处理同时变得脆弱。

对 systemd timer，这个边界更加明确：timer 不直接持有 ExecStart=，它激活 service；service 才定义 User=、WorkingDirectory=、EnvironmentFile= 和 ExecStart= 等执行属性。

③ [知识点] 任务至少有五个状态层次

定义状态：任务是否存在、语法和目标是否正确
调度状态：调度器是否运行，任务是否正在等待
触发状态：是否到点并发生过触发
执行状态：命令以什么身份退出，退出码是什么
功能状态：日志、报告、文件或远端动作是否符合目标

典型误判包括：

- atq 中有任务，于是声称未来一定成功；
- crond.service 为 active，于是声称某一条 cron job 已成功；
- timer 为 active (waiting)，于是声称同名 service 最近执行成功；

- service 显示 `Result=success`，却没有检查脚本是否把错误吞掉或写错目录；
- 报告文件存在，却没有检查所有者、大小和更新时间。

④ [知识点] 执行身份、环境、目录与输出必须显式化

计划任务通常没有当前终端，也不会自动继承登录会话中的 alias、Shell 函数、SSH agent、图形会话和交互口令。稳定任务至少要回答：

- 以哪个用户和组运行；
- `PATH` 中允许找到哪些程序；
- `HOME` 和工作目录是什么；
- 需要哪些环境变量或凭据；
- `stdout/stderr` 到哪里；
- 失败怎样被发现；
- 是否允许上一次尚未结束时再启动一次；
- 关机错过后是否补跑。

[Cheatsheet] 一次用 `at`，简单日历用 `cron`，需要 `service` 结果、补跑或随机延迟时用 `timer`；先让脚本独立成功，再连接调度器；配置存在不等于任务完成。

[操作专题] 使用 `at` 安排未来的一次执行

`at` 从标准输入或文件读取命令，把它们保存为一次性作业，并由 `atd` 在指定时间执行。提交成功时通常会返回 job ID 和计划时间。这个结果只证明任务进入队列；实际执行仍依赖 `atd`、访问控制、保存的环境、目录权限和工作负载本身。

① [操作] 提交交互式或可审阅的一次性任务

作用对象：当前用户提交给 `atd` 的一次性 Shell 作业。基本语义：`at TIMESPEC` 从标准输入读取命令；交互模式用 `Ctrl+D` 结束。`-f FILE` 从文件读取。典型形式：

```
at now + 20 minutes
at 22:30 tomorrow
at -f /root/jobs/collect-report.at 23:00
```

交互输入适合很短的任务；需要审计时，here-document 更容易回看：

```
at now + 20 minutes <<'ATJOB'
/usr/local/sbin/collect-report \
>> /var/log/collect-report.log 2>&1
ATJOB
```

引用结束标记 `'ATJOB'` 可以阻止当前 Shell 在提交时展开 `$变量`、命令替换和反斜杠。这样，变量会在未来执行时由保存的脚本处理。若确实要把当前值固定进任务，应明确取消引用，并在 `at -c JOB` 中确认最终正文。

② [知识点] `at` 保存环境，但不等于重建交互会话

`at` 会生成一段将来交给 Shell 的脚本，其中通常包含提交时的大量环境信息、`umask` 和工作目录切换。它不会保留一个可交互终端，也不能可靠依赖 alias、Shell 函数、口令提示或临时挂载。

因此任务正文仍应：

- 使用绝对命令路径；
- 明确 `stdout/stderr`；
- 避免需要输入；
- 检查目标路径在将来仍存在；
- 对网络、挂载和凭据等外部依赖保留失败证据。

③ [查询] 用 `atq` 和 `at -c` 审核队列

```
atq
at -c 12
at -c 12 | tail -n 35
```

`atq` 通常显示 job ID、计划时间、队列和所有者。`root` 可以看到更多用户的作业；普通用户只管理自己的队列。`at -c JOB` 输出保存的完整脚本，内容可能很长，审核重点是：

```
执行用户
→ 保存的 PATH/HOME
→ 工作目录切换
→ umask
→ 最后的命令正文和重定向
```

`at -c` 是提交后的静态证据，它不能证明未来时刻依赖仍然满足。

④ [操作] 用 `atrm` 删除尚未执行的作业

```
atrm 12
# 等价入口在实现支持时也可见：at -d 12
atq
```

删除后应再次检查队列。job ID 消失只证明队列项被取消；如果作业已经开始执行，删除队列项不能代替停止正在运行的进程。进程控制属于第 11 章《进程、作业、信号与调度优先级》。

⑤ [验证点] 分层验证 `at` 作业

```
systemctl is-active atd.service
systemctl is-enabled atd.service
atq
at -c 12 | tail -n 35
journalctl -u atd.service --since today --no-pager
stat /var/log/collect-report.log
```

验收时至少回答：

1. `atd` 当前是否运行；
2. 任务是否仍在队列，时间和所有者是否正确；
3. `at -c` 中的正文、目录和环境是否符合预期；
4. 到点后队列项是否被取走；
5. `daemon` 日志是否显示执行活动；
6. 目标日志或业务产物是否存在、非空且更新时间合理。

⑥ [边界] 访问控制和邮件不能替代产物验证

`/etc/at.allow` 和 `/etc/at.deny` 控制哪些用户可提交作业。常见逻辑是：存在 `allow` 时只允许其中用户；不存在 `allow` 而存在 `deny` 时允许未被拒绝的用户；两者都不存在时行为应以本机 `at(1)` 和 PAM 配置为准。

`at` 可以发送输出邮件，但这依赖本地邮件传输能力。没有收到邮件，既可能表示没有输出，也可能表示邮件链不可用。重要任务应显式重定向并保留真实产物。

[Cheatsheet] `at` TIME 提交；`atq` 查队列；`at -c JOB` 查保存脚本；`atrm JOB` 取消；最终还要看 `atd`、日志和产物。

[知识专题] 正确解释 `cron` 五字段与日期匹配

`cron` 每分钟检查一次日历字段是否匹配。字段顺序本身不复杂，高频错误来自把“月中日”和“星期”当作必须同时匹配，或者把系统 `crontab` 的用户字段误写进用户 `crontab`。应先确定配置入口，再解释字段。

① [知识点] 用户 `crontab` 的五字段

```
分钟 小时 月中日 月 星期 命令
```

允许值的核心范围：

字段	常用范围
分钟	0-59
小时	0-23
月中日	1-31
月	1-12 或实现支持的英文名称
星期	0-7，0 和 7 通常都表示星期日；也可使用英文名称

例：

```
30 15 * * 3 /usr/local/libexec/weekly-check
```

表示每周三 15:30，而不是“每月 3 日”。

② [知识点] *、列表、范围与步长

形式	语义	示例
*	该字段所有允许值	* * * * * 每分钟匹配
1,15,30	指定列表	分钟字段在 1、15、30 分匹配
1-5	含端点范围	星期一到星期五
*/10	从字段起点按步长匹配	每 10 分钟
8-18/2	范围内按步长匹配	8、10、12、14、16、18 时

*/10 不是“从创建时刻开始每 10 分钟”，而是对日历字段做整除式匹配。需要从上一次激活后计算间隔时，考虑 OnUnitActiveSec=。

③ [知识点] 月中日与星期是 OR，不是普通 AND

cron 要求分钟、小时和月份匹配；对于两个“日”字段，只要月中日或星期至少一个匹配即可。当两个字段都被限制时，这会产生 OR 行为。

```
0 6 1 * 1 /usr/local/libexec/report
```

它不是“仅在每月 1 日恰好为星期一时运行”，而是通常在每月 1 日以及每个星期一 06:00 运行。需要严格 AND 时，应使用脚本内条件、拆分规则，或用 OnCalendar= 表达并通过 systemd-analyze calendar 展开验证。

④ [知识点] 日历异常不能只靠语法解释

夏令时转换可能出现不存在的本地时间或重复的本地时间，导致 cron job 漏跑或重复运行。时区和系统时间配置属于第 14 章《系统时间、时区、RTC 与 chrony》，本章只建立边界：

- 调度表达式按系统采用的时间基准解释；
- 修改时区后，应重新核对 cron 与 timer；
- 对不能漏跑的日历任务，timer 的 Persistent= 可能更合适，但仍需验证实际语义；
- 对不能重复执行的任务，工作负载本身应具备幂等或锁机制。

[Cheatsheet] 用户 crontab 是五个时间字段加命令；* , - / 分别表示全部、列表、范围和步长；分钟、小时、月匹配，并且月中日或星期至少一个匹配。

[操作专题] 管理用户 crontab、系统 crontab 与 /etc/cron.d

配置入口决定字段格式和执行身份。用户 crontab 天然归属于某个用户，不写用户字段；/etc/crontab 与 /etc/cron.d/* 由管理员维护，每一条任务必须显式写执行用户。

① [操作] 管理用户 crontab

```
crontab -l
crontab -e
crontab -l -u monitor
crontab -u monitor -e
```

- crontab -e 在临时文件中编辑并安装新表；
- crontab -l 列出已安装表；
- root 使用 -u USER 管理其他用户；
- crontab -r 删除整张表，不是删除一行。

删除或批量替换前先保存基线：

```
crontab -l -u monitor > /root/monitor.crontab.before
```

不要直接编辑 /var/spool/cron 中的内部文件。它们的路径、标签和管理方式属于实现细节，应通过 crontab 命令维护。

② [知识点] 用户表和系统表的用户字段

用户 crontab：

分钟 小时 月中日 月 星期 命令

`/etc/crontab` 和 `/etc/cron.d/*` :

分钟 小时 月中日 月 星期 执行用户 命令

典型错误:

```
# 错误: 这是 monitor 的用户 crontab, 却额外写了 monitor
30 15 * * 3 monitor /usr/local/libexec/weekly-check
```

cron 会把 `monitor` 当作命令名或命令正文的一部分, 而不是用户字段。

```
# 错误: 这是 /etc/cron.d/weekly-check, 却漏掉执行用户
30 15 * * 3 /usr/local/libexec/weekly-check
```

字段会整体错位, 任务不会以预期形式执行。

③ [配置] 显式设置 `SHELL`、`PATH` 与 `MAILTO`

```
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
MAILTO=""
```

```
30 15 * * 3 /usr/local/libexec/weekly-check \
>> /var/log/weekly-check.log 2>&1
```

- `SHELL=` 决定 cron 用哪个 Shell 解释命令字段; 它不自动改变脚本自己的 shebang;
- `PATH=` 应包含任务真正需要的目录, 但不要无边界复制整个交互环境;
- `MAILTO=` 指定输出邮件接收者; 空值通常关闭邮件; 邮件能否送达仍依赖 MTA;
- `HOME` 和 `LOGNAME` 通常来自目标用户, 但任务不应依赖模糊的当前目录。

需要固定目录时, 优先让脚本自行定位资源, 或在命令中显式进入目录:

```
0 2 * * * cd /srv/report && /usr/local/libexec/build-report \
>> /var/log/build-report.log 2>&1
```

若 `cd` 失败, `&&` 阻止后续命令在错误目录运行。

④ [知识点] 未转义 `%` 会改变命令和标准输入

在 crontab 命令字段中, 第一个未转义 `%` 会被转换为换行; 前半部分作为命令, 后半部分作为标准输入。其后的未转义 `%` 也转换为换行。

```
# 容易失败: %F 会被 cron 解释
0 1 * * * /usr/bin/date +%F >> /var/log/date.log 2>&1

# 必须直接写在 crontab 时, 应转义
0 1 * * * /usr/bin/date +\%F >> /var/log/date.log 2>&1
```

更稳定的设计是把 `date +%F` 放进脚本, 让 crontab 只调用脚本。

⑤ [操作] 在 `/etc/cron.d` 中集中管理系统任务

```
# /etc/cron.d/weekly-check
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
MAILTO=""
```

```
30 15 * * 3 monitor /usr/local/libexec/weekly-check \
>> /var/log/weekly-check.log 2>&1
```

建议:

- 使用简单文件名, 不依赖编辑器备份后缀;
- 文件由 `root` 管理且不得可被普通用户修改;
- 末尾保留换行;

- 脚本有明确 shebang 和执行权限；
- 不通过 `source ~/.bashrc` 把交互环境整体注入计划任务。

⑥ [边界] 周期目录与 anacron 只做必要辨识

`/etc/cron.hourly`、`/etc/cron.daily` 等目录由发行版提供的调度链调用。把脚本放进目录不代表立即执行，也不等于你已经理解具体运行时间。应检查 `/etc/anacrontab`、`/etc/cron.d/0hourly` 或本机对应配置。

本章不完整展开 anacron 的 period、delay 和 job identifier；需要精确控制用户、分钟和日志时，直接使用用户 `crontab`、`/etc/cron.d` 或 `timer` 更容易验收。

⑦ [验证点] 对 cron 做四层验收

```
crontab -l -u monitor
systemctl is-active crond.service
systemctl is-enabled crond.service
journalctl -u crond.service --since today --no-pager
stat /var/log/weekly-check.log
```

对于 `/etc/cron.d`：

```
sed -n '1,120p' /etc/cron.d/weekly-check
stat -c '%U %G %a %n' /etc/cron.d/weekly-check
```

验证层次：

配置：字段、用户、环境和命令正确
调度器：`crond` 当前运行，并满足题目持久要求
触发：日志中出现目标任务活动
执行：目标用户能够运行，错误被记录
功能：业务文件或消息正确更新

[Cheatsheet] 用户表不写用户字段；系统表必须写用户字段；显式设置 PATH 和输出；`crontab -r` 删除整张表；`%` 要转义或移入脚本。

[诊断专题] 手工执行成功，但 cron 失败

“我在终端里运行没问题”只证明当前用户、当前目录和当前环境能运行。cron 使用的用户、环境和输入输出路径可能完全不同。诊断应减少变量，而不是反复等待下一个触发时间。

① [诊断] 先把症状分成三类

没有触发证据
→ 查任务是否装在正确用户、字段时间、`crond` 和时区

有触发证据但命令失败
→ 查身份、环境、路径、权限、shebang、SELinux 和依赖

命令似乎成功但业务产物不对
→ 查退出码设计、工作目录、目标路径、覆盖/追加和业务验证

② [诊断] 确认查的是正确用户和正确入口

```
crontab -l -u monitor
sudo grep -R --line-number --fixed-strings 'weekly-check' \
/etc/crontab /etc/cron.d /var/spool/cron 2>/dev/null
```

同一命令可能同时存在于 `root crontab`、目标用户 `crontab`、`/etc/cron.d` 和 `timer`。重复入口会造成重复执行；只看当前登录用户的 `crontab -l` 可能遗漏真正定义。

直接读取 `spool` 只用于调查，不用于修改。

③ [诊断] 用目标用户和精简环境复现

```
sudo -u monitor env -i \
HOME=/home/monitor \
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin \
SHELL=/bin/bash \
/usr/local/libexec/weekly-check
printf 'exit=%s\n' "$?"
```

这个测试比“切换到 monitor 后在完整登录环境运行”更接近 cron。若失败，下一条最有区分度的证据通常是脚本 stderr、文件权限或缺失命令，而不是继续修改时间字段。

④ [诊断] 按稳定顺序检查常见差异

用户/组

→ 绝对命令路径

→ shebang 与执行权限

→ PATH 和必要变量

→ 工作目录与相对文件

→ 目标目录写权限

→ SELinux AVC 证据

→ 网络、挂载、DNS、凭据等外部依赖

→ stdout/stderr 和退出码

不要把整个 `.bashrc` 或 `.profile` source 进 cron 来掩盖脚本依赖。需要的变量应在调度配置、脚本或受控环境文件中明确声明。

⑤ [诊断] 检查 %、重定向和邮件假设

- 命令中出现 `date +%F`、SQL `%` 或 URL 编码时，先检查未转义 `%`；
- 日志文件由 `root` 预先创建且目标用户不可写时，Shell 在执行命令前就会因重定向失败；
- `MAILTO` 已设置但没有 MTA 时，没有邮件不能证明任务没有运行；
- `>/dev/null 2>&1` 会丢失故障证据，不应作为排错初始状态。

⑥ [诊断] 修复后立即做可验证

不必把正式周期改成每分钟并忘记恢复。优先：

- 用精简环境直接运行脚本；
- 临时创建一个明确标记、很快触发的测试入口；
- 验证后删除测试入口；
- 恢复并核对正式表达式；
- 检查真实产物和时间戳。

[Cheatsheet] 正确用户 → 精简环境 → 绝对路径 → 权限/SELinux → 触发日志 → stdout/stderr → 真实产物；不要用完整登录环境替代复现。

[知识专题] timer 和 service 是两个状态对象

systemd timer 的价值不只是另一种日期语法。它把“何时激活”和“怎样执行”拆成两个 unit，使 service 可以独立验证，并把退出状态纳入 systemd 和 journal。这个拆分也意味着排错时不能只查 timer。

① [知识点] 默认激活同名 service

weekly-check.timer

↓ 默认激活

weekly-check.service

也可用 `Unit=` 指定其他 unit。为了降低认知负担，通常让 timer 和 service 除后缀外同名。只需要 enable timer；被 timer 触发的 service 通常不应单独 enable，否则可能在启动链中额外运行。

若 timer 到点时目标 unit 已经 active，systemd 不会自动启动第二个实例，也不会重启它。重复 timer 不适合搭配长期保持 active 的 `RemainAfterExit=yes` oneshot service，否则可能只触发第一次。

② [知识点] 日历 timer 与单调 timer 的时间基准

属性	起点或时间基准	典型用途
OnCalendar=	实时时钟和日历	每天 02:15、每周三
OnBootSec=	系统启动时刻	开机 10 分钟后
OnActiveSec=	timer 自身被激活时刻	timer 启动后延迟一次
OnStartupSec=	service manager 启动时刻	用户 manager 等场景
OnUnitActiveSec=	目标 unit 上次被激活时刻	每次激活后再等一段时间
OnUnitInactiveSec=	目标 unit 上次变为 inactive 的时刻	任务完成后再等待固定间隔

多个表达式可以共存，任何一个到期都可触发目标 unit。`OnBootSec=` 或 `OnStartupSec=` 已经落在过去时，激活 timer 后可能立即到期；其他单调表达式和日历表达式应按对应规则判断。

③ [知识点] `Persistent=` 只补日历 timer 的遗漏

`Persistent=true` 记录 service 上次由 timer 触发的时间。timer 再次激活时，如果在 inactive 期间本应至少触发一次，就立即补一次。边界：

- 只对带 `OnCalendar=` 的 timer 有效；
- 多次遗漏不会逐次回放，而是形成一次补触发；
- 补触发仍受 `RandomizedDelaySec=` 影响；
- timer 从未启用或状态记录被清理时，不能凭想象推导历史；
- 仍要检查 service 是否成功和业务任务是否幂等。

④ [知识点] `RandomizedDelaySec=` 与 `AccuracySec=` 目的相反

`RandomizedDelaySec=5min` 在计算出的下一次时间上增加 0 到 5 分钟的随机延迟，用于避免多台主机同时产生负载峰值。

`AccuracySec=1min` 允许 systemd 在目标时间之后的一个精度窗口内，把本机多个 timer 的唤醒合并，以减少唤醒开销。它不是“最大允许业务延迟”的完整 SLA，也不是随机摊开主机的属性。

组合顺序可以理解为：

```
先计算基础触发时间
→ 加上 RandomizedDelaySec 的随机值
→ 再在 AccuracySec 窗口内做本机合并
```

需要尽可能精确地观察随机延迟分布时，可把 `AccuracySec=` 设得更小；不要无理由把所有 timer 都设为微秒级精度。

⑤ [知识点] timer 的 waiting 不证明 service 成功

`systemctl list-timers --all` 常见列：

- `NEXT`：预计下一次触发时间；
- `LEFT`：距下一次还有多久；
- `LAST`：最近一次触发时间；
- `PASSED`：距最近一次过去多久；
- `UNIT`：timer 名；
- `ACTIVATES`：目标 unit。

这些列证明调度时间和触发关系。service 的 `Result`、`ExecMainCode`、`ExecMainStatus`、journal 和产物才证明执行结果。

[Cheatsheet] timer 决定触发，service 决定执行；日历用 `OnCalendar`，相对启动或上次激活用单调属性；`Persistent` 只补日历遗漏；随机延迟和精度窗口不是一回事。

[操作专题] 建立、启用和验证持久 timer

稳定流程是 service-first：先确认脚本在目标用户下成功，再建立 oneshot service；service 独立成功后，才让 timer 定期激活它。这样能把脚本问题、service 配置问题和日历问题分层定位。

① [操作] 在目标身份和精简环境中验证工作负载

```
sudo -u reporter env -i \
  HOME=/home/reporter \
  PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin \
  /usr/local/libexec/daily-report
printf 'exit=%s\n' "$?"
stat /var/lib/daily-report/latest.txt
```

先修复脚本、目录权限和外部依赖。不要在脚本尚未成功时同时引入 service 与 timer。

② [操作] 创建并直接验证 oneshot service

```
# /etc/systemd/system/daily-report.service
[Unit]
Description=Generate Daily Operations Report

[Service]
Type=oneshot
User=reporter
Group=reporter
WorkingDirectory=/var/lib/daily-report
UMask=0027
ExecStart=/usr/local/libexec/daily-report
```

静态检查和直接运行：

```
systemd-analyze verify /etc/systemd/system/daily-report.service
systemctl daemon-reload
systemctl start daily-report.service
systemctl show daily-report.service \
    -p Result,ExecMainCode,ExecMainStatus
journalctl -u daily-report.service -b --no-pager
stat /var/lib/daily-report/latest.txt
```

`Type=oneshot` 让 `systemd` 等待命令结束并记录退出状态。对周期触发的一次性脚本，通常不需要 `RemainAfterExit=yes`，否则 `service` 会保持 `active`，后续 `timer` 到期时不会再次启动它。

③ [操作] 创建日历 timer

```
# /etc/systemd/system/daily-report.timer
[Unit]
Description=Run Daily Operations Report

[Timer]
OnCalendar=*-*-* 02:15:00
Persistent=true
RandomizedDelaySec=5min
AccuracySec=30s
Unit=daily-report.service

[Install]
WantedBy=timers.target
```

`AccuracySec=30s` 在本任务中表达允许较小的本机合并窗口；它不是必须值。应根据题目是否要求严格时间和资源摊开做选择。

④ [验证点] 先解析表达式，再加载 unit

```
systemd-analyze calendar '*-*-* 02:15:00'
systemd-analyze verify \
    /etc/systemd/system/daily-report.service \
    /etc/systemd/system/daily-report.timer
systemctl daemon-reload
```

`systemd-analyze calendar` 会规范化表达式并计算后续触发时间。它证明表达式能解析以及日历含义，不证明 `unit` 已加载、`timer` 已启动或 `service` 能运行。

⑤ [操作] 启用并立即启动 timer

```
systemctl enable --now daily-report.timer
systemctl is-enabled daily-report.timer
systemctl is-active daily-report.timer
```

`enable --now` 同时建立持久启用链接并启动当前 `timer`。仍应分别查询 `active` 与 `enabled`，不用一个状态代替另一个。

不要 `enable daily-report.service`。`service` 由 `timer` 触发，并可在排错时手工 `start`。

⑥ [查询] 查询 NEXT/LAST 与 service 结果

```
systemctl list-timers --all | grep -F daily-report
systemctl status daily-report.timer --no-pager -l
systemctl show daily-report.timer \
    -p NextElapseUsecRealtime,LastTriggerUsec,AccuracyUsec,RandomizedDelayUsec

systemctl show daily-report.service \
    -p Result,ExecMainCode,ExecMainStatus
journalctl -u daily-report.timer -u daily-report.service \
    --since today --no-pager
stat /var/lib/daily-report/latest.txt
```

属性名称在 `systemctl show` 中通常使用微秒后缀，与 `unit` 文件中的 `...Sec=` 是同一设置的底层表现。考试核心是读懂状态，不需要记住所有内部 D-Bus 属性。

⑦ [边界] 验收补跑和随机延迟需要真实时间场景

静态检查无法证明：

- 主机关机错过时间后是否补触发；
- 随机延迟落在何处；
- `suspend/resume` 和日历 `timer` 的实际行为；
- 多台主机的负载是否真正摊开。

真实环境测试应记录关机前状态、错过的日历时间、开机时间、`timer LAST`、`service journal` 和产物更新时间。本章当前仅给出推荐验证方法，尚未完成这些实机验证。

[Cheatsheet] 精简环境跑脚本 → 创建 `service` → `verify` → 手工启动 `service` → 创建 `timer` → `calendar` → `enable --now` → `NEXT/LAST` → `service Result` → `journal` 和产物。

[操作专题] 单调 timer、transient timer 与安全迁移

日历 `timer` 回答“钟表到某个时间”；单调 `timer` 回答“从某个事件起过多久”。`systemd-run` 还能临时创建 `timer/service` 对，适合验证命令或做短期任务，但不能替代要求重启后仍存在的持久 `unit`。

① [操作] 使用 `OnBootSec=` 和 `OnUnitActiveSec=`

```
[Timer]
OnBootSec=10min
OnUnitActiveSec=30min
AccuracySec=1min
```

这表示开机约 10 分钟后第一次触发，之后相对于目标 `unit` 每次激活时间每 30 分钟再次触发。若任务运行时间很长，`OnUnitActiveSec=` 的下一次基准从激活开始；需要从任务结束后再等待，应考虑 `OnUnitInactiveSec=`。

单调 `timer` 不受本地时区解释影响。休眠期间是否推进取决于使用的时钟和 `WakeSystem=`；这不是 RHCSA 主操作，本章只保留边界。

② [操作] 创建一次 transient timer

```
systemd-run \
    --unit=health-test \
    --on-active=2min \
    --timer-property=AccuracySec=1s \
    /usr/local/libexec/health-test
```

系统通常会创建：

```
health-test.timer
health-test.service
```

查询：

```
systemctl status health-test.timer --no-pager
systemctl list-timers --all | grep -F health-test
journalctl -u health-test.timer -u health-test.service -b --no-pager
systemctl show health-test.service -p Result,ExecMainStatus
```

`transient unit` 没有写入 `/etc/systemd/system` 的持久文件，`manager` 或系统重启后通常不应被当作仍存在。具体回收时点还与 `unit` 状态和收集策略有关。

③ [操作] 创建临时日历 timer

```
systemd-run \
  --unit=report-preview \
  --on-calendar='*-*-* 23:40:00' \
  /usr/local/libexec/daily-report --preview
```

它适合观察 systemd 生成的 timer/service、journal 和命令运行方式。题目要求持久 timer 时，仍必须编写 `.service` 与 `.timer` 文件，并执行 `daemon-reload`、`enable` 和分层验证。

④ [知识点] transient timer 的边界

- `systemd-run` 创建的 service 默认执行环境与交互 Shell 不同；
- 使用 timer 选项时，立即启动的是 timer，service 到期后才触发；
- `--unit=` 让名称可预测，便于查询和清理；
- 不要把命令参数中的 `$` 展开时机想当然，Shell 展开和 systemd manager 展开是不同层；
- transient 成功不证明对应的持久 unit 文件、用户、权限和 enable 状态正确。

⑤ [操作] 从 cron 迁移到 timer 的安全顺序

```
记录旧 cron 定义与当前产物
→ 确认执行用户、环境、输出和并发语义
→ 目标用户精简环境运行脚本
→ 创建 service 并独立验收
→ 创建 timer 并解析日历
→ 启用 timer，观察至少一次可控触发
→ 检查 service Result、journal 和产物
→ 删除旧 cron 入口
→ 全局搜索重复调度
→ 最终回归
```

不要先删旧任务再开始调试新 service；也不要让旧 cron 和新 timer 长期并行，否则可能重复写入或并发冲突。

⑥ [验证点] 证明只剩一个调度入口

```
crontab -l -u reporter 2>/dev/null || true
grep -R --line-number --fixed-strings 'daily-report' \
  /etc/crontab /etc/cron.d /var/spool/cron 2>/dev/null || true
systemctl list-timers --all | grep -F daily-report
systemctl cat daily-report.timer daily-report.service
```

“旧文件已经删除”不够，还要检查其他用户 `crontab`、`/etc/cron.d` 和同名或不同名 timer。

[Cheatsheet] 相对启动用 `OnBootSec`，从上次激活计时用 `OnUnitActiveSec`；`systemd-run` 只做临时 timer；迁移先证明新链路，再删除旧入口，最后查重。

[诊断专题] timer 在等待，但任务没有正确完成

timer 故障应分成“timer 没有被正确加载”“时间没有按预期到期”“发生触发但 service 失败”“service 成功但业务结果错误”。每一层使用不同证据。

① [诊断] timer 不在 `list-timers` 中

```
systemctl status daily-report.timer --no-pager -l
systemctl cat daily-report.timer
systemd-analyze verify /etc/systemd/system/daily-report.timer
systemctl daemon-reload
systemctl start daily-report.timer
```

重点检查：文件名和 unit 名、[Timer] 段、加载错误、是否忘记 `daemon-reload`、是否启动 timer。`systemctl cat` 显示 systemd 实际合成的定义，不只看编辑器里的文件。

② [诊断] timer 为 waiting，但 NEXT 不符合预期

```
systemd-analyze calendar '*-*-* 02:15:00'
systemctl list-timers --all | grep -F daily-report
systemctl show daily-report.timer \
  -p NextElapseUSecRealtime,AccuracyUSec,RandomizedDelayUSec
```

继续核对系统时区与当前时间，但完整时区和 chrony 诊断归第 14 章。注意 `RandomizedDelaySec=` 和 `AccuracySec=` 都可能让实际触发晚于基础日历时间。

③ [诊断] timer 有 LAST，但 service failed

```
systemctl show daily-report.service \
  -p Result,ExecMainCode,ExecMainStatus
systemctl status daily-report.service --no-pager -l
journalctl -u daily-report.service --since today --no-pager
systemctl cat daily-report.service
```

下一条证据通常来自 service：目标用户、工作目录、环境、ExecStart、文件权限、SELinux 和外部依赖。不要继续只改 timer 日历。

④ [诊断] service success，但没有正确产物

Result=success 只表示 systemd 认可主命令的退出状态。脚本可能捕获错误后仍退出 0，或者把文件写到了错误目录。

```
stat /var/lib/daily-report/latest.txt
head -n 20 /var/lib/daily-report/latest.txt
namei -l /var/lib/daily-report/latest.txt
```

检查脚本是否正传播错误码，目标文件所有者、大小、内容和更新时间是否符合业务要求。

⑤ [诊断] Persistent=true 没有按预期补跑

依次确认：

1. timer 是否使用 OnCalendar=；
2. timer 是否在错过期间 inactive；
3. 重新激活 timer 时是否确实至少错过一次日历事件；
4. RandomizedDelaySec= 是否仍在等待；
5. service 是否已被触发但失败；
6. 状态时间戳是否被清理或 unit 是否换名。

静态阅读配置不能替代关机/开机场景测试。

⑥ [诊断] 任务重复或并发

```
grep -R --line-number --fixed-strings 'daily-report' \
  /etc/crontab /etc/cron.d /var/spool/cron 2>/dev/null || true
systemctl list-timers --all | grep -F report
pgrep -af '/usr/local/libexec/daily-report'
```

可能原因：旧 cron 未删除、存在两个 timer、手工启动与定时触发重叠，或任务运行时间超过周期。是否允许并发属于工作负载设计；可在脚本或 service 中使用明确锁机制，但不在本章展开复杂锁实现。

[Cheatsheet] 不在列表查加载；NEXT 错查日历、时区、随机与精度；有 LAST 查 service；service 成功再查产物；补跑和随机效果必须 live test。

[经典任务] 配置带明确用户和证据链的周期健康检查

环境

系统已有用户 monitor 和脚本 /usr/local/libexec/web-health。脚本读取本机服务状态，并把最终报告写入 /var/lib/web-health/latest.txt。root 手工执行脚本成功，但目前没有可靠周期任务。

现状：

- /var/lib/web-health 已存在，所有者当前为 root:root；
- /var/log/web-health-cron.log 尚不存在；
- monitor 的 crontab 可能为空；
- 不假设本机邮件系统可用；
- 不允许通过 chmod 777 解决权限。

目标终态

1. 每周三 15:30 以 monitor 用户执行；
2. 使用 monitor 的用户 crontab，不使用 root crontab 或 /etc/cron.d；
3. crontab 显式定义 SHELL=/bin/bash、受控 PATH 与 MAILTO=""；
4. stdout/stderr 追加到 /var/log/web-health-cron.log；
5. monitor 对报告目录和日志文件只获得完成任务所需的权限；

6. 任务正文使用绝对路径，不把复杂业务逻辑直接写进 `crontab`；
7. 验收必须证明：目标用户精简环境可运行、`crontab` 字段正确、`crond` 当前运行并满足持久要求、输出和报告文件正确更新。

限制条件

- 不删除其他用户或其他现有 `cron` 任务；
- 不把用户字段写进用户 `crontab`；
- 不把“`crond active`”扩大为任务成功；
- 不用 `>/dev/null 2>&1` 隐藏错误；
- 不编造等待到正式周三 15:30 的执行结果，可设计可撤销的临时验证。

验收证据

用户与目录权限
→ 精简环境退出码
→ `monitor` 的 `crontab` 原文
→ `crond active/enabled`
→ 测试触发或直接脚本证据
→ `cron` 日志
→ 报告文件所有者、大小和更新时间

[经典任务] 将需补执行的 `cron` 任务迁移为 `systemd timer`

环境

`/etc/cron.d/daily-report` 当前每天 02:15 以 `reporter` 用户执行：

```
15 2 * * * reporter /usr/local/libexec/daily-report \  
>> /var/log/daily-report-cron.log 2>&1
```

脚本应生成 `/var/lib/daily-report/latest.txt`。服务器可能在 02:15 关机，开机后必须补执行一次。多台服务器不应在完全相同的时刻开始任务，且管理员希望从 `systemd` 获取明确退出状态和 `journal`。

目标终态

1. 建立 `daily-report.service`，`Type=oneshot`，以 `reporter` 用户和组执行；
2. 明确工作目录、`umask` 和绝对 `ExecStart`；
3. 建立同名 `daily-report.timer`，每天 02:15 触发；
4. 配置 `Persistent=true`；
5. 配置最多 5 分钟随机延迟和可解释的 `AccuracySec=`；
6. `service` 可独立启动，退出状态成功并生成非空报告；
7. `timer` 当前 `active`、持久 `enabled`，并能显示 `NEXT/LAST`；
8. `journal` 能区分 `timer` 触发和 `service` 执行；
9. 新链路验收后删除旧 `/etc/cron.d/daily-report`，并证明没有重复调度入口。

限制条件

- 不 `enable service`，只 `enable timer`；
- 不在 `timer` 中写 `ExecStart=`；
- 不先删除旧 `cron` 再调试新 `service`；
- 不把 `timer active` 解释为 `service` 已成功；
- 不声称已经完成关机错过后的 `live` 补跑测试；
- 不修改其他 `cron` 或 `timer`。

验收证据

```
旧 cron 基线
→ reporter 精简环境
→ service 静态检查与独立执行
→ timer 日历解析与静态检查
→ active/enabled
→ NEXT/LAST
→ service Result/ExecMainStatus
→ journal
→ 报告文件
→ 旧入口删除与全局查重
```

[参考解答] 周期健康检查：先修正身份与文件权限，再安装 crontab

参考解答展示调查顺序和证据链，不代表已在本会话的 RHEL 9 VM 中执行。

① [调查] 保存基线并检查对象

```
id monitor
getent passwd monitor
ls -ld /var/lib/web-health /var/log
ls -l /usr/local/libexec/web-health
head -n 1 /usr/local/libexec/web-health
crontab -l -u monitor > /root/monitor.crontab.before 2>/dev/null || :
systemctl is-active crond.service
systemctl is-enabled crond.service
```

确认脚本确实存在且有 shebang。不要覆盖已有 crontab；编辑时应保留现有条目。

② [操作] 只授予需要的目录与日志权限

一种可审阅的处理方式：

```
install -d -o monitor -g monitor -m 0750 /var/lib/web-health
install -o monitor -g monitor -m 0640 /dev/null \
/var/log/web-health-cron.log
```

若目录内已有其他业务文件，不能直接递归 chown；应先调查具体所有权需求，再做最小修改。

③ [验证] 在精简环境直接运行脚本

```
sudo -u monitor env -i \
HOME=/home/monitor \
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin \
SHELL=/bin/bash \
/usr/local/libexec/web-health \
>> /var/log/web-health-cron.log 2>&1
printf 'exit=%s\n' "$?"

stat -c '%U %G %a %s %y %n' \
/var/log/web-health-cron.log \
/var/lib/web-health/latest.txt
```

失败时先读日志并修复脚本依赖，不要继续安装 cron。

④ [操作] 编辑 monitor 的用户 crontab

```
crontab -u monitor -e
```

保留原有内容并增加：

```
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
MAILTO=""

30 15 * * 3 /usr/local/libexec/web-health >> /var/log/web-health-cron.log 2>&1
```

这里没有用户字段，因为这是 monitor 自己的 crontab。

⑤ [静态核对] 检查字段、环境和调度器

```
crontab -l -u monitor
systemctl is-active crond.service
systemctl is-enabled crond.service
journalctl -u crond.service -b --no-pager | tail -n 50
```

若题目要求重启后调度器仍工作而 `crond` 未 `enabled`，应按第 12 章的 `service` 状态方法处理；本章不重讲完整 `enable` 语义。

⑥ [验证] 设计可撤销的短期测试

不能等待正式周三时，可临时把同一个脚本放入一条最近分钟的测试规则，并加唯一标识或测试参数。测试完成后立即删除测试行，重新核对正式规则。

更安全的最低证据是：

- 精简环境脚本已经成功；
- `crontab` 内容和字段静态正确；
- `crond` `active`；
- `stdout/stderr` 目标可写；
- 报告产物已由 `monitor` 正确生成。

没有真实等待触发时，应在 QA 中标记“正式周期触发未 `live test`”，不能声称已跑通。

⑦ [最终验收]

```
id monitor
crontab -l -u monitor
systemctl is-active crond.service
systemctl is-enabled crond.service
stat -c '%U %G %a %s %y %n' \
    /var/log/web-health-cron.log \
    /var/lib/web-health/latest.txt
```

典型错误：

- 在用户 `crontab` 中写 `monitor` 用户字段；
- `root` 创建日志文件后不给 `monitor` 写权限；
- 使用相对脚本或命令路径；
- 用 `crontab -r` 删除整张表；
- 临时测试后忘记恢复正式规则；
- 仅凭 `crond active` 宣布完成。

[参考解答] cron 迁移 timer：service-first、验证后切换、最后查重

① [调查] 保存旧入口与产物基线

```
cp -a /etc/cron.d/daily-report \
    /root/daily-report.cron.before
sed -n '1,120p' /etc/cron.d/daily-report
id reporter
ls -l /usr/local/libexec/daily-report
stat /var/lib/daily-report/latest.txt 2>/dev/null || true
systemctl list-timers --all | grep -F daily-report || true
```

确认旧任务的时间、用户、命令、输出路径和任何环境变量。若脚本依赖旧 `cron` 里的 `PATH`，应在 `service` 或脚本中显式处理。

② [验证] 用目标用户和精简环境运行脚本

```
install -d -o reporter -g reporter -m 0750 /var/lib/daily-report

sudo -u reporter env -i \
    HOME=/home/reporter \
    PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin \
    /usr/local/libexec/daily-report
printf 'exit=%s\n' "$?"
stat -c '%U %G %a %s %y %n' /var/lib/daily-report/latest.txt
```

脚本失败时先修复其路径、权限、SELinux 或外部依赖。

③ [操作] 创建 oneshot service

```
# /etc/systemd/system/daily-report.service
[Unit]
Description=Generate Daily Operations Report

[Service]
Type=oneshot
User=reporter
Group=reporter
WorkingDirectory=/var/lib/daily-report
UMask=0027
ExecStart=/usr/local/libexec/daily-report
```

静态检查并直接运行：

```
systemd-analyze verify /etc/systemd/system/daily-report.service
systemctl daemon-reload
systemctl start daily-report.service
systemctl show daily-report.service \
    -p Result,ExecMainCode,ExecMainStatus
journalctl -u daily-report.service -b --no-pager
stat /var/lib/daily-report/latest.txt
```

期望 `Result=success` 且 `ExecMainStatus=0`，同时报告文件非空并由正确用户更新。不要只看 `systemctl start` 返回 0。

④ [操作] 创建 timer

```
# /etc/systemd/system/daily-report.timer
[Unit]
Description=Run Daily Operations Report

[Timer]
OnCalendar=*-*-* 02:15:00
Persistent=true
RandomizedDelaySec=5min
AccuracySec=30s
Unit=daily-report.service

[Install]
WantedBy=timers.target
```

⑤ [静态核对] 解析日历并检查两个 unit

```
systemd-analyze calendar '*-*-* 02:15:00'
systemd-analyze verify \
    /etc/systemd/system/daily-report.service \
    /etc/systemd/system/daily-report.timer
systemctl daemon-reload
systemctl cat daily-report.service daily-report.timer
```

`calendar` 输出的下一次时间是静态日历证据；随机延迟和精度窗口还会影响实际调度。

⑥ [操作] 启用 timer，不 enable service

```
systemctl enable --now daily-report.timer
systemctl is-enabled daily-report.timer
systemctl is-active daily-report.timer
systemctl list-timers --all | grep -F daily-report
```

⑦ [验证] 检查触发关系与执行结果

```
systemctl status daily-report.timer --no-pager -l
systemctl show daily-report.service \
    -p Result,ExecMainCode,ExecMainStatus
journalctl -u daily-report.timer -u daily-report.service \
    --since today --no-pager
stat -c '%U %G %a %s %y %n' /var/lib/daily-report/latest.txt
```

若尚未到正式时间，service 的直接成功、timer 的 NEXT、active/enabled 和静态配置只构成阶段性证据；正式日历触发、随机延迟及关机补跑必须后续 live test。

⑧ [切换] 新链路通过后删除旧 cron

```
rm -f /etc/cron.d/daily-report
systemctl try-restart crond.service
```

通常 crond 会自动发现文件变化；try-restart 是否必要应按现场策略决定。不要无理由重启整个系统。

⑨ [最终验收] 搜索重复入口

```
crontab -l -u reporter 2>/dev/null || true
grep -R --line-number --fixed-strings 'daily-report' \
/etc/crontab /etc/cron.d /var/spool/cron 2>/dev/null || true

systemctl is-active daily-report.timer
systemctl is-enabled daily-report.timer
systemctl list-timers --all | grep -F daily-report
systemctl show daily-report.service \
-p Result,ExecMainCode,ExecMainStatus
journalctl -u daily-report.timer -u daily-report.service \
--since today --no-pager
stat /var/lib/daily-report/latest.txt
```

典型错误：

- 在 service 未独立成功前就启用 timer；
- enable service 导致额外启动；
- 配置 Persistent=true 却使用纯单调 timer；
- 只查 timer，不查 service Result；
- 旧 cron 和新 timer 同时保留；
- 将随机延迟理解为固定 5 分钟；
- 将 AccuracySec 理解为随机推开；
- 未做关机补跑 live test 却声称已验证。

[本章收束] 让调度器、执行身份和结果证据对齐

工作方法：先静态、再触发、最后业务

三种调度器共享同一管理逻辑：

```
at: job → atd → 保存的脚本与环境 → 输出/产物
cron: entry → crond → 用户与精简环境 → 输出/产物
timer: timer → service → Result/journal → 输出/产物
```

选择时先问任务生命周期。提交或安装后，先验证静态定义；触发前确认调度器；触发后转向工作负载退出状态；最后以真实产物完成验收。

调度选择矩阵

问题	选择或下一步
只运行一次	at，提交后用 atq 和 at -c 审核
简单固定周期，题目明确要求 crontab	用户 crontab 或 /etc/cron.d
需要明确系统用户字段	/etc/cron.d 或 systemd service 的 User=
需要关机错过后补一次	OnCalendar= + Persistent=true
需要开机后延迟	OnBootSec=
需要从上次激活后计算间隔	OnUnitActiveSec=
需要多台主机摊开负载	RandomizedDelaySec=
需要观察 systemd 临时调度	systemd-run --on-active 或 --on-calendar
手工成功但计划失败	目标用户 + env -i 重建精简环境
timer waiting 但无报告	查 service Result、journal 和产物，不继续只看 timer

帮助入口

```
man at
man atd
man crontab
man 5 crontab
man cron
man systemd.timer
man systemd.time
man systemd-run
systemd-analyze calendar --help
systemctl list-timers --help
```

向下一章交接

本章已经把“任务何时运行”转换成了可验证的调度对象，并反复区分配置、等待、触发、退出和业务产物。下一章《RPM 包、文件归属与软件事务》将转向系统内容的来源与安装状态：当 `at`、`crontab` 或某个辅助命令不存在时，应先确认软件包、文件归属和事务状态，而不是在本章无边界地展开软件安装。

第 16 章 RPM 包、文件归属与软件事务

本章阅读导航

先抓住一条主线：软件包问题不是“装没装”一个判断，而是包文件、RPM 数据库、磁盘文件、配置迁移和最终功能五个层次连续推进。

- 01识别包身份用 NEVRA 区分名称、版本、发行与架构
- 02选择查询对象区分已安装记录、本地 RPM 与磁盘路径
- 03建立文件归属从包查文件，从路径反查已安装包
- 04解释验证差异逐字符读取 rpm -V，不把差异直接判恶
- 05审阅软件事由 DNF 解析依赖并确认完整事务摘要
- 06推进到真实终态检查配置迁移、unit、应用语法与功能

专题地图

知识专题	从包名到 NEVRA：先确认正在谈论哪个包
知识专题	RPM 数据库、磁盘文件与应用状态是三个层次
操作专题	查询已安装包并建立文件归属证据
操作专题	在安装前检查本地 RPM 包文件
操作专题	使用 rpm -V 解释文件差异，而不是机械修复
知识专题	.rpmnew 与 .rpmsave：升级时如何保护本地配置
操作专题	安装、升级、重装与删除：先读事务计划
操作专题	事务完成后，从包状态推进到真实功能
诊断专题	包事务和文件状态异常的证据链
经典任务	解释文件归属与验证差异；预检本地 RPM 并安装验收

阅读时持续回答

- 当前查询对象是包名、本地 RPM 文件还是磁盘路径？
- 这条证据来自 RPM 数据库，还是来自当前磁盘状态？
- NEVRA 的哪个字段决定当前对象的精确身份？
- rpm -V 的字符表示什么差异，文件类别是什么？
- 该差异是合法配置、意外漂移，还是尚不能判断？
- 事务摘要是否包含超出题目目标的安装、升级或删除？
- .rpmnew/.rpmsave 是否需要比较、合并和语法验证？
- 包事务成功后，应用或服务功能是否真正成立？

学习提示：先确定查询对象与证据层，再选择命令；看到差异时先解释字段和文件类别，最后才决定是否修改。完成事务后继续验证配置、unit 与真实功能。

第 16 章 · 正文

在 Linux 系统中，“软件已经装上”不是一个可以只靠一条命令证明的结论。一个 .rpm 文件可能只是保存在磁盘上，尚未写入系统；RPM 数据库可能记录某个包已经安装，但它声明拥有的文件可能被删除、改写或改变权限；DNF 事务可能已经成功结束，但新配置尚未合并、相关 unit 没有进入题目要求的状态，应用也可能仍无法提供真实功能。

本章围绕“包身份 → 查询对象 → 文件归属 → 差异验证 → 软件事务 → 功能终态”推进。最常见的误判有三类：把本地 RPM 文件存在当成已经安装；把 rpm -V 的差异直接判定为恶意或故障；把 DNF 的 Complete! 当成配置、服务和业务功能都已经正确。仓库、模块流与包组留给第 17 章《DNF 仓库、模块流与包组》；服务启动和持久状态留给第 12 章《systemd Unit、服务与依赖关系》。本章只建立完成当前软件事务所必需的接口，并明确不把 --nodeps、--force 或跳过签名校验当作正常解决方案。

概念

NEVRA 是 RPM 包的精确身份，由 Name、Epoch、Version、Release 和 Architecture 组成。包名只描述逻辑对象，本地文件名还可能被任意重命名，命令名和服务名也不等同于包名；因此遇到版本、架构或并行安装问题时，应让 RPM 从包头或数据库读取真实字段，而不是靠文件名或字符串外观猜测。

概念

RPM 数据库 保存系统已经登记的包头、文件清单和文件属性基线，它回答“系统记录安装了什么”。数据库记录不等于磁盘实时镜像：文件可能被删除或修改，挂载也可能遮蔽路径；更不能由此直接推出应用配置、进程和业务功能正确。

概念

包文件归属 是已安装包与路径之间的声明关系。`rpm -ql` 从包走向文件，`rpm -qf` 从当前路径反查拥有者；归属只能说明哪个已安装包声明提供该路径，不说明谁最后修改了文件，也不能查询尚未安装的仓库候选。

概念

配置文件标记 是包作者赋予文件的升级保护语义。被标记为配置文件的路径可能包含合法本地修改；升级时 RPM 会根据包内标记、当前内容和新旧版本关系保留正式文件或旁置另一份，因此`.rpmnew`与`.rpmsave`是需要比较和迁移的证据，不是可以机械删除的垃圾文件。

概念

验证差异 是当前磁盘文件与 RPM 数据库基线之间的可检测变化。`rpm -V` 可以指出大小、模式、摘要、所有者、组、时间戳或 capability 等差异，但它不判断变化是否合理；配置变更可能是预期状态，关键文件无差异也不等于服务或业务功能健康。

概念

软件事务 是一组需要整体审阅和提交的包变更。安装、升级、重装或删除可能牵动多个依赖、运行脚本并改变配置和 unit；DNF 负责选择候选和解析依赖，RPM 执行底层包操作并维护数据库。事务成功只是包层终态，仍需继续验证文件、配置、服务和功能。

操作语义以下命令分别回答“已安装什么、包里有什么、路径归谁、文件哪里不同、包是否可信、事务会改变什么”。先理解作用对象，再记参数。

`rpm -q` 查询已安装包

SYNOPSIS

```
rpm -q [QUERY-OPTIONS] PACKAGE...
```

以本机 RPM 数据库为查询对象，确认安装记录、包头信息和包声明的文件。

重要参数 / 形式

`-q`

进入查询模式；后续选项决定查询哪个对象和显示哪些信息。

`-i`

显示包头的详细信息，例如版本、架构、许可证、安装日期和描述。

`-l`

列出包记录的全部路径。

`-c`

只列出被包标记为配置文件的路径。

`-d`

只列出被包标记为文档的路径。

`rpm -qp` 查询本地 RPM 文件

SYNOPSIS

```
rpm -qp [QUERY-OPTIONS] ./PACKAGE_FILE.rpm
```

`-p` 把查询对象从已安装数据库切换为一个尚未安装的包文件；它适合安装前读取真实包头和内容。

重要参数 / 形式

`-qp`

查看本地 RPM 的详细元数据，不执行安装。

`-qpl`

预览包将声明安装的路径。

`-qpc`

预览被标记为配置文件的路径。

`--requires`

读取包头中的依赖要求；查询本身不解析或安装依赖。

`--scripts`

查看包脚本内容；查询不会运行这些脚本。

`rpm -qf` 从路径反查已安装包

SYNOPSIS

```
rpm -qf /ABSOLUTE/PATH
```

查询哪个已安装包声明拥有当前路径。它只查本机 RPM 数据库，不能发现尚未安装的仓库候选。

重要参数 / 形式

`-f PATH`

把文件路径作为查询键；优先使用已确认的真实绝对路径。

`command -v NAME`

先确定外部命令路径；若结果是 `alias`、函数或 `builtin`，应回到第 03 章继续解析。

`rpm -qf "$(command -v NAME)"`

仅在 `command -v` 返回真实外部文件路径时组合使用。

`rpm -V` 验证已安装文件差异

SYNOPSIS

```
rpm -V PACKAGE
rpm -Vf /ABSOLUTE/PATH
```

比较当前文件与 RPM 数据库中的属性基线。默认无输出表示本次可检查属性未发现差异；有输出时必须逐字符解释，并结合文件类别判断下一条证据。

重要参数 / 形式

`-V PACKAGE`

验证指定已安装包的文件。

`-Vf PATH`

先按路径找到拥有包，再验证该文件范围。

`S M S D L U G T P`

依次表示大小、模式、文件摘要、设备、符号链接、所有者、组、修改时间和 `capability` 等差异位置。

`missing`

包清单记录路径，但当前视图中路径缺失；继续调查版本、挂载、误删或替换。

`rpmkeys --checksig` / `rpm -K` 检查包文件

SYNOPSIS

```
rpmkeys --checksig ./PACKAGE_FILE.rpm
rpm -K ./PACKAGE_FILE.rpm
```

检查包文件的摘要和签名状态。缺少公钥、包未签名和签名或摘要失败是不同结果；该检查也不能替代已安装文件的 `rpm -V`。

重要参数 / 形式

`--checksig FILE`

读取包文件中的摘要和签名并尝试验证。

`-K FILE`

RPM 命令提供的兼容入口；正文以 `rpmkeys` 作为语义更明确的首选写法。

真实输出

按环境记录实际状态，不把“缺少公钥”自动写成“签名失败”。

`dnf` 形成依赖事务

SYNOPSIS

```
dnf install ./PACKAGE_FILE.rpm
dnf upgrade PACKAGE
dnf reinstall PACKAGE
dnf remove PACKAGE
```

DNF 选择候选、解析依赖并显示完整事务计划；确认前应检查将安装、升级和删除哪些包。RPM 的 `-U`、`-F` 和 `-e` 用于理解低层语义，不作为绕开依赖处理的默认路径。

重要参数 / 形式

- `install ./FILE.rpm`
以本地包文件为目标，同时使用已配置仓库满足依赖。
- `upgrade PACKAGE`
推进到当前可用的更新候选；应记录最终 NEVRA。
- `reinstall PACKAGE`
重装相同包内容，适合确认文件意外缺失或损坏后的受控恢复，不用于覆盖未知配置。
- `remove PACKAGE`
删除目标并展示相关依赖变更；摘要范围异常时应取消。
- `rpm -U / -F / -e`
分别表示低层升级或安装、仅 freshen 已安装包、删除；不得用 `--nodeps` 解决依赖。

[知识专题] 从包名到 NEVRA：先确认正在谈论哪个包

软件包调查最容易出现错误，是把“名字看起来一样”当成“对象完全相同”。同一个 Name 可能有不同版本、发行号和架构；某些包允许多个版本并存；一个本地 `.rpm` 文件也可能比系统中已安装的版本更新、相同或更旧。只有把身份维度拆开，后续的安装、升级和删除才不会误伤其他对象。

① [知识点] NEVRA 的五个字段分别承担什么职责

字段	含义	判断边界
Name	包的逻辑名称	不等同于命令名或服务名
Epoch	版本比较的额外优先级	常为未设置；一旦设置会参与比较
Version	上游软件版本	不能只按字符串直觉比较
Release	打包发行号	同一上游版本可有多个发行修订
Architecture	目标架构	<code>noarch</code> 表示内容不绑定特定 CPU 架构

常见文件名看起来像：

name-version-release.arch.rpm

但不要靠从右向左切连字符手工解析所有包名。Name、Version 和 Release 可能包含复杂字符；可靠做法是让 RPM 读取包头。

② [知识点] 包名、NEVRA、RPM 文件和路径是四种不同查询键

- 包名 → 查询一个逻辑包对象
- NEVRA → 精确到版本、发行和架构
- FILE.rpm → 查询一个尚未安装的包文件
- /PATH → 反查哪个已安装包声明拥有该路径

例如，`rpm -q <PACKAGE>` 查询 RPM 数据库；`rpm -qp <FILE.rpm>` 查询包文件；`rpm -qf <PATH>` 查询路径归属。把三者混用，常见结果是“package ... is not installed”或把文件名当成包名导致错误判断。

③ [知识点] Epoch 经常不可见，但不能从版本模型中删除

普通的 `rpm -q` 输出常把注意力集中在 Name、Version、Release 和 Architecture 上。Epoch 未设置时，很多显示形式会省略它；这不代表 Epoch 不参与 RPM 的版本比较模型。需要精确审计时，应使用查询格式分别读取字段，而不是从默认文本输出猜测。

`rpm -q --qf 'Name: %{NAME}\nEpoch: %{EPOCH}\nVersion: %{VERSION}\nRelease: %{RELEASE}\nArch: %{ARCH}\n' <PACKAGE>`

如果 Epoch 显示为未设置值，应记录真实结果，不自行改写成某个虚构数字。

④ [知识点] “已安装”通常不是永久的一对一关系

大多数普通包在某一时刻只保留一个活动版本，但不能把这个经验扩大为规则。内核包是典型的可并行安装对象；多架构环境也可能存在同名的不同架构实例。执行删除、精确验证或脚本化查询前，应先查看所有匹配结果。

`rpm -q <PACKAGE>`
`rpm -q --qf '%{NAME}\t%{EPOCH}\t%{VERSION}\t%{RELEASE}\t%{ARCH}\n' <PACKAGE>`

第一条快速查看所有匹配记录；第二条逐字段保留每个匹配实例的真实身份。脚本不要通过拆分默认输出或模糊 `grep` 来猜测 NEVRA。

⑤ [知识点] RPM 文件名不是可信来源声明

文件名可以被任意重命名。真正的 Name、EVR、Architecture、Packager、Summary 和签名信息来自包头。收到第三方 RPM 时，先查询包头和签名，再决定是否允许进入事务。

[Cheatsheet] `-q` 查已安装数据库，`-qp` 查本地包文件，`-qf` 从路径反查已安装包；精确身份看 NEVRA，不看文件修改时间，也不只看文件名。

[知识专题] RPM 数据库、磁盘文件与应用状态是三个层次

RPM 数据库是包管理的事实入口，但它不是整个系统的实时镜像。数据库记录某包应该包含哪些文件、这些文件安装时具有哪些属性；磁盘上可能发生人工修改、配置管理变更、误删除或权限漂移；应用还可能因为配置语义、依赖服务或外部资源而失败。调查时必须明确自己正在证明哪一层。

① [知识点] RPM 数据库保存的是已安装记录和基线

数据库通常包含：

- 已安装包的包头和 NEVRA；
- 包声明安装的路径；
- 配置文件、文档、许可证等文件分类；
- 安装时用于验证的大小、模式、摘要、所有者、组、时间戳和 capability 等属性；
- 依赖、Provides、脚本等元数据。

本章通过 `rpm` 接口访问数据库，不依赖某个具体后端或内部数据库文件。RHEL 9 的实现细节可能随小版本变化，管理员脚本不应直接解析内部数据库文件。

② [知识点] 文件存在、文件归属和文件一致性是不同问题

```
ls /path 成功
只证明当前路径存在

rpm -qf /path 成功
证明某个已安装包声明拥有该路径

rpm -V <PACKAGE> 无输出
证明本次可检查属性未发现差异
```

它们都不能单独证明应用功能正确。配置文件内容可能与业务目标不符但仍与厂商包完全一致；服务也可能没有启动。

③ [知识点] `rpm -ql`、`-qc` 与 `-qd` 是全集和分类视图

- `rpm -ql <PACKAGE>`：列出包记录的全部路径；
- `rpm -qc <PACKAGE>`：只列出被标记为配置文件的路径；
- `rpm -qd <PACKAGE>`：只列出被标记为文档的路径。

不要用 `rpm -ql | grep /etc` 代替 `rpm -qc`。并非所有配置都一定位于 `/etc`，也不是 `/etc` 下的每个文件都被打包为 RPM 的配置文件。

④ [知识点] RPM 归属不等于“最后修改者”

`rpm -qf` 回答哪个已安装包声明拥有该路径，不回答谁最后修改了它。人工编辑、脚本、应用自身和配置管理系统都可能在安装后改变文件。要理解变化来源，需要结合时间、日志、配置管理记录和 `rpm -V` 差异。

⑤ [知识点] 路径不存在时仍可能在包清单中

一个受包管理的文件被删除后，`rpm -ql <PACKAGE>` 仍会列出它，而 `ls` 会失败，`rpm -V` 通常会报告 `missing`。这正说明数据库基线和磁盘现实是两个对象。

⑥ [知识点] 数据库一致性问题与普通文件漂移要分开

如果大量查询异常、RPM 数据库操作报错或事务中断，应先保留错误信息并调查数据库和事务状态。不要把所有问题都归因于某个文件被修改，也不要没有备份和官方恢复步骤时直接重建数据库。

[Cheatsheet] RPM 数据库回答“系统记录了什么”；磁盘检查回答“文件现在怎样”；应用验证回答“功能是否成立”。任何一层成功都不能替代下一层。

[操作专题] 查询已安装包并建立文件归属证据

查询的目标不是输出越多越好，而是让每条命令回答一个明确问题：包是否安装、精确身份是什么、安装了哪些路径、哪些是配置、某个路径归谁。调查应从包身份和查询方向开始，再进入文件和验证。

① [操作] 使用 `rpm -q` 和 `rpm -qi` 确认安装记录

作用对象： 本机 RPM 数据库中的已安装包。

基本语义：

```
rpm -q <PACKAGE>
rpm -qi <PACKAGE>
```

`rpm -q` 适合快速确认安装状态和默认身份输出；`rpm -qi` 展示更完整的包头信息，如 Summary、Description、License、Install Date 和 Source RPM。

验证与边界： 查询成功只证明数据库中有匹配记录。若需要精确区分架构、Epoch 或多个版本，应使用查询格式并保留所有匹配行。

② [操作] 使用查询格式输出稳定字段

```
rpm -q --qf '%{NAME}\t%{EPOCH}\t%{VERSION}\t%{RELEASE}\t%{ARCH}\n' <PACKAGE>
```

查询格式适合审计和脚本化，但字段内容仍需按实际输出处理。不要用空格切割默认的 `rpm -q` 文本来推断复杂包名。

可用标签可通过以下帮助入口查看：

```
rpm --querytags | less
man rpm
```

③ [操作] 从包查询全部路径、配置和文档

```
rpm -ql <PACKAGE>
rpm -qc <PACKAGE>
rpm -qd <PACKAGE>
```

调查安装后的配置入口时，先用 `-qc` 得到包作者声明的配置文件，再检查文件是否存在、是否产生旁置版本、内容是否满足业务目标。

④ [操作] 从路径反查已安装包

```
rpm -qf /absolute/path
```

先把命令解析为真实路径，再反查：

```
command -v <COMMAND>
rpm -qf "$(command -v <COMMAND>)"
```

如果 `command -v` 返回 Shell builtin、函数或 alias，不能直接把它当成外部文件路径；命令发现的完整方法归第 03 章《本地帮助、命令发现与软件能力查询》。

⑤ [操作] 查询包脚本与依赖只做调查，不等于执行

```
rpm -q --scripts <PACKAGE>
rpm -q --requires <PACKAGE>
rpm -q --provides <PACKAGE>
```

这些命令读取已安装包的元数据。查看 scriptlet 有助于理解安装、升级和删除可能改变哪些系统对象，但查询本身不会运行脚本。

⑥ [操作] 处理“不属于任何包”的结果

`rpm -qf` 找不到所有者时，可能意味着：

- 文件由管理员或应用自行创建；
- 文件来自源码安装、手工解压或容器映射；
- 路径是运行时生成内容；
- 路径拼写、符号链接或挂载视图与预期不同；
- 提供该路径的包尚未安装。

若目标是查找可安装候选包，应转到第 17 章使用 DNF 的仓库查询接口，而不是强行让 `rpm -qf` 查询未安装内容。

[Cheatsheet] 包到文件： `-ql/-qc/-qd`；文件到包： `-qf`；脚本化身份： `--qf`；元数据查询不运行脚本。

[操作专题] 在安装前检查本地 RPM 包文件

本地 RPM 是一个尚未进入系统事务的候选对象。最安全的切入路径是：先确认文件和包头，再检查架构、内容、依赖、脚本和签名，最后让 DNF 形成事务计划。不要因为扩展名是 `.rpm` 就直接以 `root` 身份执行安装。

① [操作] 使用 `-p` 将查询对象切换到包文件

```
rpm -qp ./package.rpm
rpm -qpi ./package.rpm
rpm -qpl ./package.rpm
rpm -qpc ./package.rpm
rpm -qpd ./package.rpm
```

这里的 `-p` 是 package file 选择器。它告诉 RPM 读取指定文件的包头，而不是从已安装数据库中按包名查找。

② [操作] 明确检查 NEVRA 和目标架构

```
rpm -qp --qf '%{NAME}\t%{EPOCH}\t%{VERSION}\t%{RELEASE}\t%{ARCH}\n' ./package.rpm
uname -m
```

`noarch` 包通常不绑定特定 CPU 架构；其他架构必须与系统和运行环境相容。仅仅文件可读，不代表架构可安装。

③ [操作] 检查依赖、Provides 和脚本

```
rpm -qp --requires ./package.rpm
rpm -qp --provides ./package.rpm
rpm -qp --scripts ./package.rpm
```

包脚本可能在高权限下修改用户、目录、缓存、systemd preset 或配置。第三方 RPM 的脚本属于变更评审内容，不应跳过。

④ [操作] 使用 `rpmkeys --checksig` 检查摘要和签名

```
rpmkeys --checksig ./package.rpm
# 兼容入口
rpm -K ./package.rpm
```

该检查面向包文件中包含的摘要和数字签名，帮助判断文件完整性和签名来源。常见情况包括：

- 摘要和签名可成功验证；
- 包有签名，但本机缺少对应公钥；
- 包未签名；
- 摘要或签名验证失败。

必须记录真实输出。缺少公钥不等于包内容一定被篡改，未签名也不等于可信；两者都意味着无法用当前信任链完成来源验证。

⑤ [知识点] 签名检查与 `rpm -V` 不是同一种验证

```
rpmkeys --checksig FILE.rpm
    比较包文件内部的摘要和签名
    目标：包文件的完整性与来源

rpm -V PACKAGE
    比较已安装文件与 RPM 数据库基线
    目标：安装后文件属性是否漂移
```

一个签名有效的包安装后仍可能被人工修改；已安装文件完全匹配基线，也不能证明这个包来自组织允许的来源。

⑥ [操作] 必要时只提取内容，不执行事务

RH124 课程展示了 `rpm2cpio` 与 `cpio` 用于检查或提取包内容。它适合离线审阅单个文件，但不是本章安装事务的主路径。

```
mkdir inspect && cd inspect
rpm2cpio ../package.rpm | cpio -t
rpm2cpio ../package.rpm | cpio -idmv './path/in/package'
```

提取出来的文件不进入 RPM 数据库，也不会执行 scriptlet；因此不能把这种提取视为“安装完成”。

⑦ [操作] 形成安装前决策记录

- 文件路径和权限已确认
- 包头身份已确认
- 架构已确认
- 文件、配置和脚本已审阅
- 签名策略已满足
- 当前安装版本已确认
- DNF 事务摘要可接受

[Cheatsheet] `-p` 查询包文件；`--scripts` 只查看脚本；`rpmkeys --checksig` 检查包文件；提取内容不等于安装。

[操作专题] 使用 `rpm -V` 解释文件差异，而不是机械修复

RPM 验证将当前文件系统状态与数据库记录的基线进行比较。它非常适合回答“哪些受包管理的属性发生了变化”，但输出只是一组差异标记。管理员仍需判断变化是否预期、是否来自配置管理、是否影响功能，以及最小修复是什么。

① [操作] 选择验证范围

- `rpm -V <PACKAGE>` # 验证一个已安装包
- `rpm -Vf /absolute/path` # 通过路径选择所属包并验证
- `rpm -Va` # 验证全部已安装包，范围较重

优先选择最小范围。全系统验证可能产生大量预期配置差异，也会增加分析成本。

② [知识点] 无输出的准确含义

`rpm -V <PACKAGE>` 在本次可比较属性均匹配时通常不输出文件行。它只能证明：

对所选包、本次未禁用且可读取的验证属性，没有发现与 RPM 数据库基线不同的项目。

它不能证明：

- 配置符合业务要求；
- 服务正在运行或开机自启；
- 端口可达；
- 数据内容正确；
- 包来源符合组织信任策略。

③ [输出判断] 读取九个属性位置

典型差异行的前九个位置为：

S M 5 D L U G T P

位置	含义	下一条证据示例
S	文件大小变化	<code>stat</code> 、 <code>diff</code> 、应用生成逻辑
M	模式或文件类型变化	<code>stat -c '%A %a %F'</code>
5	文件摘要变化	<code>diff</code> 、版本控制、配置管理记录
D	设备号变化	<code>stat</code> ，确认特殊文件语义
L	符号链接目标变化	<code>readlink -f</code> 、 <code>ls -l</code>
U	所有者变化	<code>stat -c '%U %u'</code>
G	所属组变化	<code>stat -c '%G %g'</code>
T	修改时间变化	<code>stat</code> ，不能单独判定内容变化
P	capability 变化	<code>getcap</code> 、安全基线

字符 `?` 表示该位置未发现差异；`?` 表示该项无法完成检查。`5` 是历史沿用的显示字符，现代解释宜称为文件摘要差异，不应机械理解为所有版本都只使用 MD5。

④ [输出判断] 读取文件属性标记和路径

九位属性之后通常还有文件类别标记和路径，例如：

- `c`：配置文件；
- `d`：文档文件；
- `g`：ghost 文件；
- `l`：许可证文件；
- `r`：README 文件。

实际可见标记受 RPM 版本和包定义影响。判断时先读差异位置，再读文件类别，最后看路径；不要只看到 `c` 就断言配置错误。

⑤ [输出判断] `missing` 与属性漂移是两种证据

`missing` 表示数据库记录的路径当前不存在。调查链应是：

```
确认包仍安装
→ 确认路径在 rpm -ql 中
→ 检查父目录、挂载和符号链接
→ 查明删除或替换来源
→ 评估是否可用 dnf reinstall 恢复
→ 重新验证并做功能检查
```

不要在不知道本地定制和业务状态时直接重装整个包。

⑥ [诊断] 配置文件差异先判断是否预期

配置文件出现 `S`、`S`、`T` 等差异很常见。下一步优先：

```
rpm -qc <PACKAGE>
stat <CONFIG>
diff -u <KNOWN_BASELINE> <CONFIG>
```

如果没有独立基线，可与 `.rpmnew`、备份、版本控制或同版本包中的文件进行比较。`rpm -V` 只告诉你“不同”，不告诉你哪一方符合业务目标。

⑦ [安全边界] 不把验证差异自动等同于入侵

差异可能来自：

- 合法配置变更；
- 应用首次运行生成内容；
- 管理工具修正权限；
- 升级遗留；
- 人工误操作；
- 未授权修改。

判断必须结合变更记录、日志、所有者、时间线和应用行为。验证输出是调查入口，不是结论。

[Cheatsheet] `rpm -V` 只报告差异：先读九位属性，再读文件类别和路径；无输出不是功能验收，配置差异也不是自动重装理由。

[知识专题] `.rpmnew` 与 `.rpmsave`：升级时如何保护本地配置

升级软件时，包中的配置文件可能与当前系统上的本地修改冲突。RPM 不总是简单覆盖原文件，而会依据包作者的配置标记、当前文件是否被修改以及新旧内容关系，保留当前文件或旧版本。旁置文件不是垃圾，它们是配置迁移证据。

① [知识点] 配置文件保护依赖打包标记和实际状态

包作者可使用配置文件标记控制升级行为。管理员无法只凭扩展名预测所有包的结果。讲义中的 `.rpmnew` 和 `.rpmsave` 描述常见模式，不把它们写成对所有 RPM 都成立的固定公式。

② [知识点] `.rpmnew` 的常见语义

对于通常被标记为“不直接替换”的配置文件，当当前文件已被本地修改且新包提供不同内容时，当前正式路径往往保留，新厂商版本被写为：

```
/path/to/config.rpmnew
```

这表示新版本候选尚未合并，不表示升级事务一定失败。

③ [知识点] `.rpmnew` 的常见语义

在另一类配置替换策略下，新包版本可能取得正式路径，而原有本地文件被保存为：

```
/path/to/config.rpmnew
```

这时需要从旧文件中迁移仍然有效的本地设置，不能只检查正式路径是否存在。

④ [操作] 系统化定位旁置文件

针对事务涉及的目录优先查找，避免无目的扫描整个系统：

```
find /etc -type f \( -name '*.rpmnew' -o -name '*.rpmnew' \) -print
```

还应结合：

```
rpm -qc <PACKAGE>
dnf history info <ID>
```

从事务和包清单确认旁置文件与本次升级的关系。

⑤ [操作] 比较、合并、语法检查，再让应用加载

```
确认当前正式文件和旁置文件
→ diff -u
→ 识别已废弃和新增参数
→ 选择性合并
→ 应用自身的配置语法检查
→ 备份后原子替换
→ 按第 12 章入口 reload 或 restart
→ 功能验证
```

不要直接 `cp config.rpmnew config` 覆盖现有业务配置，也不要未确认新版本语义时把 `.rpmnew` 全量复制回来。

⑥ [边界] 删除旁置文件必须是处理结果，不是清理动作

只有在以下条件都成立后，才考虑删除：

- 已完成差异审阅；
- 需要保留的设置已迁移；
- 新配置语法检查通过；
- 应用和服务验证通过；
- 已有可回退备份或版本记录。

[Cheatsheet] `.rpmnew` 常见为“保留当前，旁置新版本”；`.rpmnew` 常见为“正式路径使用新版本，保存旧文件”。无论哪种都必须 diff、合并、验证。

[操作专题] 安装、升级、重装与删除：先读事务计划

包管理操作会改变 RPM 数据库、文件系统和依赖关系。RHEL 9 中，普通管理任务优先使用 DNF；低层 RPM 选项用于理解语义、查询和特殊调查，而不是绕过依赖保护。任何删除或大范围升级都应先审阅事务摘要。

① [知识点] RPM 与 DNF 的职责边界

```
DNF
├─ 解析包规格和仓库候选
├─ 处理依赖与冲突
├─ 形成事务计划和摘要
├─ 下载包并记录 history
└─ 调用 RPM 完成底层事务

RPM
├─ 读取包头与本地数据库
├─ 安装、升级、删除单个包事务
├─ 执行 scriptlet 和触发器
├─ 更新 RPM 数据库
└─ 查询与验证包及文件
```

因此，本地 RPM 也通常用 DNF 安装：DNF 可以利用已配置仓库补齐依赖。

② [操作] 通过推荐路径安装本地 RPM

```
sudo dnf install ./package.rpm
```

`./` 或绝对路径帮助 DNF 明确这是本地文件，而不是仓库中的包名。执行前应阅读：

- 安装、升级、删除哪些包；
- 依赖和弱依赖；
- 下载和安装大小；
- 是否出现冲突、替换或架构异常。

考试中不应为了省时间盲目使用 `-y` 跳过摘要。

③ [操作] 区分安装、升级和重装

```
sudo dnf install <PACKAGE>
sudo dnf upgrade <PACKAGE>
sudo dnf reinstall <PACKAGE>
```

- `install`：使指定内容进入已安装状态，必要时安装依赖；
- `upgrade`：选择可用的较新版本并处理依赖；
- `reinstall`：重新安装当前可获得的相同包，常用于恢复缺失或损坏的受管理文件；
- 任何操作后都要验证真实 NEVRA 和文件状态。

`reinstall` 不是清除所有本地配置的工具，也不能自动修复应用数据和外部依赖。

④ [知识点] 低层 RPM 安装选项表达不同事务语义

```
rpm -i FILE.rpm      # 安装，不承担一般升级选择
rpm -U FILE.rpm      # 安装或升级
rpm -F FILE.rpm      # 仅当同名包已安装时更新
rpm -e PACKAGE       # 删除已安装包
rpm --test ...        # 进行事务测试，不应用变更
```

课程可能展示 `rpm -ivh` 以说明低层安装。真实 RHEL 9 管理中，依赖可由仓库解决时仍优先 DNF。

⑤ [操作] 删除前检查完整事务摘要

```
sudo dnf remove <PACKAGE>
```

DNF 可能计划删除依赖该包的其他内容，或清理不再需要的依赖。看到删除数量超出预期时应取消，回到：

```
rpm -q --whatrequires <PACKAGE>
dnf history
```

仓库级依赖分析与候选查询在第 17 章深入。

⑥ [操作] 使用 history 建立事务审计线索

```
dnf history
dnf history info <ID>
```

重点查看：

- 命令行和时间；
- 事务成功、失败或中止状态；
- 安装、升级、降级、删除的包；
- 事务前后数据库是否出现外部变化提示。

`dnf history undo <ID>` 依赖旧版本包仍可获得、仓库状态和后续依赖关系，不能当作无条件回滚按钮。

⑦ [安全边界] 这些选项不是默认修复方法

```
rpm --nodeps
rpm --force
rpm --replacefiles
rpm --oldpackage
dnf --allowerase
dnf --nogpgcheck
```

它们会放宽依赖、文件冲突、版本或签名保护。遇到失败时应先读取准确错误，确认是依赖缺失、冲突、签名、架构还是仓库问题，再选择最小修复。仓库配置和 GPG 策略归第 17 章。

⑧ [边界] 包管理事务与服务控制不是同一步

某些包脚本会创建用户、刷新缓存或应用 preset，但不能假设安装后服务必然启动、启用或提供功能。事务完成后进入 O05 的分层验收；完整 systemd 控制归第 12 章。

[Cheatsheet] 普通事务优先 DNF；本地包用路径安装；删除先读摘要；history 是证据，不是保证；不以 `--nodeps`、`--force` 或关闭 GPG 校验换取“成功”。

[操作专题] 事务完成后，从包状态推进到真实功能

DNF 显示事务完成，只能证明包管理层没有报告失败。考试和工作中的终态通常还包含精确版本、关键文件、配置合并、服务状态和实际功能。验收应从最接近事务的证据开始，逐层向外推进。

① [验证] 第一层：确认 RPM 数据库中的最终身份

```
rpm -q <PACKAGE>
rpm -q --qf '%{NAME}\t{%EPOCH}\t{%VERSION}\t{%RELEASE}\t{%ARCH}\n' <PACKAGE>
```

确认实际安装的是目标包、目标版本和目标架构。不要只保留 DNF 最后一行文本。

② [验证] 第二层：确认关键路径和文件归属

```
rpm -ql <PACKAGE>
rpm -qc <PACKAGE>
rpm -qf <CRITICAL_PATH>
```

同时使用 `test -e`、`stat` 或 `readlink` 检查关键路径现实。RPM 清单存在而磁盘路径缺失时，应继续验证。

③ [验证] 第三层：检查文件漂移和配置旁置文件

```
rpm -V <PACKAGE>
find <RELEVANT_DIR> -type f \( -name '*.rpmnew' -o -name '*.rpmsave' \) -print
```

对已知会本地修改的配置，预期差异应有解释；对关键可执行文件或 unit 的意外差异应继续调查。

④ [验证] 第四层：运行应用自身的静态检查

按应用提供的接口检查：

```
<COMMAND> --version
<COMMAND> --help
<APPLICATION> --config-test
```

具体命令取决于包。不要编造不存在的 `--config-test`；应从 man page、`--help` 或包文档确认真实入口。

⑤ [验证] 第五层：检查 unit、进程和功能

若包提供 systemd unit：

```
rpm -ql <PACKAGE> | grep '/systemd/system/'
systemctl status <UNIT>
systemctl is-active <UNIT>
systemctl is-enabled <UNIT>
```

`active`、`enabled` 和真实业务功能是不同的状态。完整启停、依赖和持久化控制归第 12 章《systemd Unit、服务与依赖关系》。

⑥ [验证] 第六层：保留事务和变更证据

```
dnf history info <ID>
```

记录：目标、实际包、时间、事务结果、配置处理、验证命令和仍未确认的风险。真实运维中，这些信息可成为后续 Ansible 自动化的前置基线。

⑦ [工作迁移] 把手工变更转化为幂等状态

后续自动化不应简单复制一条 `dnf install` 命令，而应表达：

```
目标包和允许版本
→ 依赖与仓库前提
→ 配置文件来源和所有权
→ 需要 reload/restart 的条件
→ 包层、文件层和功能层验证
```

Ansible 模块和仓库自动化属于 RHCE 章节，本章只建立可自动化的状态模型。

[Cheatsheet] 事务成功后依次验证：NEVRA → 关键路径 → `rpm -V` 和旁置配置 → 应用静态检查 → unit/进程 → 真实功能 → 事务记录。

[诊断专题] 包事务和文件状态异常的证据链

软件管理故障经常跨越包文件、数据库、依赖、配置和服务。稳定的诊断方法不是反复重装，而是先识别症状属于哪一层，再选择最有区分度的证据。

① [诊断] 本地 RPM 无法安装

```
症状: dnf install ./package.rpm 失败
→ 当前证据：保留完整错误和事务摘要
→ 假设：文件损坏、签名、公钥、架构、依赖或冲突
→ 下一条证据：rpm -qpi、rpmkeys --checksig、rpm -qp --requires、uname -m
→ 最小修复：补齐可信前提或选择正确包，不绕过保护
→ 再验证：重新审阅事务摘要和最终 NEVRA
```

仓库不可用、GPG key 配置或模块流问题转第 17 章。

② [诊断] 包显示已安装，但关键文件不存在

```
rpm -q <PACKAGE>
→ rpm -ql <PACKAGE> | grep <PATH>
→ ls -l / stat / findmnt
→ rpm -V <PACKAGE>
→ 调查删除、挂载覆盖或路径替换来源
→ 确认后再考虑 dnf reinstall
→ 应用和功能再验证
```

先确认该路径确实由当前版本包声明；版本变化可能使旧路径合法消失。

③ [诊断] rpm -V 出现大量配置差异

先按文件类别和目录分组，区分：

- 预期本地配置；
- 版本升级带来的新语义；
- 配置管理系统维护；
- 意外权限或所有权漂移；
- 内容被覆盖或删除。

下一条证据优先是 `rpm -qc`、`stat`、`diff`、变更记录和应用语法检查，而不是 `dnf reinstall`。

④ [诊断] 升级成功，但服务无法读取新配置

```
查找 .rpmnew/.rpmsave
→ 确认当前正式配置由哪个版本产生
→ diff 并识别废弃参数
→ 应用配置语法检查
→ 最小合并
→ reload/restart
→ 日志和功能验证
```


服务控制和日志入口分别引用第 12 章与第 13 章，不在本章重新展开。

⑤ [诊断] 删除事务计划移除大量包

症状: remove 摘要远超目标范围

- 立即取消
- 记录哪些包因依赖被计划删除
- rpm -q --whatrequires 与 dnf history 调查
- 重新确认真正目标和替代依赖
- 不使用 --nodeps 强行删除

摘要本身就是重要证据，取消事务不是失败，而是避免破坏性变更。

⑥ [诊断] DNF 显示成功，但命令仍不可用

rpm -q 确认包

- rpm -ql 查包是否真的提供该命令
- command -v 和 PATH
- rpm -qf 反查实际文件
- stat / rpm -V
- Shell 缓存、权限和解释器
- 应用级验证

如果目标命令由另一个包提供，转第 17 章使用仓库能力查询。

⑦ [诊断] 签名验证提示缺少公钥

缺少公钥表示当前信任库无法验证签名来源。应确认：

- 包来自哪个受控渠道；
- 正确公钥的可信分发位置；
- 指纹是否通过独立渠道核对；
- 组织策略是否允许导入。

不要从与包同一未知来源随手下载一个 key 后立即信任，也不要以 --nogpgcheck 跳过问题。

⑧ [诊断] history 中的 undo 无法完成

可能原因包括旧包不再可用、仓库内容变化、后续事务改变依赖或 RPM 数据库有外部变化。下一步应读取错误中具体缺失的包和版本，评估恢复来源和影响，不应反复执行 undo 或用 --allowerase 扩大变更。

[Cheatsheet] 先保存错误和摘要；按包文件、签名、架构、依赖、数据库、文件、配置、服务分层；选择最小修复，再回到同一层复查。

⑨ [诊断] 用最有区分度的证据切开故障层次

症状	优先取得的下一条证据	此时不要直接做什么
本地 RPM 无法进入事务	rpm -qpi、rpmkeys --checksig、rpm -qp --requires 与完整 DNF 摘要	不使用 --nodeps 或 --nogpgcheck
包已安装但关键路径缺失	rpm -ql、父目录与挂载、rpm -Vf	不在未解释缺失原因时反复重装
rpm -V 出现配置差异	文件类别、stat、diff、变更记录与应用语法检查	不把所有配置差异当成损坏
升级后应用不可用	.rpmnew/.rpmsave、应用静态检查、unit 状态与日志	不把 Complete! 当成功能终态
remove 或 undo 计划异常扩大	完整事务摘要、--whatrequires、history 与缺失版本	不用放宽冲突保护强行提交

这张表只负责选择下一条证据；实际修复仍应回到对应对象层，完成最小变更并重新验证。

[经典任务] 任务一：识别文件所属包并解释验证差异

环境

你在一台 RHEL 9 主机上维护 websvc 应用。题目环境会提供两个真实路径：

<CONFIG_PATH>
<CRITICAL_PATH>

两个路径均应来自同一个已安装 RPM 包，但题目不会直接给出包名。配置文件曾经过合法本地修改；关键路径可能存在内容、权限、所有者或缺失问题。

当前状态

- 不允许假设 `rpm -V` 只输出一行；
- 不允许把所有配置差异当作故障；
- 不知道最近修改来自人工、脚本还是升级；
- 本任务不要求配置仓库，也不要求制作 RPM。

目标终态

1. 识别两个路径所属的已安装包；
2. 记录该包的真实 Name、Epoch、Version、Release 和 Architecture；
3. 列出该包的配置文件、文档和关键文件；
4. 对包和指定路径执行有针对性的验证；
5. 逐条解释验证差异字段、文件类别和下一条证据；
6. 对预期配置变化不做破坏性覆盖；
7. 对确认的意外漂移给出最小修复方案；
8. 完成包层、文件层、配置层和功能层验收。

限制条件

- 不使用 `rpm --force`、`--nodeps` 或无调查的重装；
- 不把 `rpm -V` 无输出扩大为服务健康；
- 不编造路径、包名或验证输出；
- 修改配置前必须保留备份或差异记录。

验收证据

层次	证据
归属	<code>rpm -qf</code>
精确身份	<code>rpm -q --qf</code>
文件分类	<code>rpm -ql/-qc/-qd</code>
差异	<code>rpm -V</code> 、 <code>rpm -Vf</code>
当前属性	<code>stat</code> 、 <code>readlink</code> 、 <code>getcap</code> （按差异选择）
内容	<code>diff</code> 或受控基线
应用	配置语法和功能验证

作答记录模板

对象	已取得的基线	差异解释与下一条证据	再验证结果
<CONFIG_PATH>	归属、文件类别、当前属性	记录预期配置或意外漂移，并选择 <code>diff</code> / 语法检查	保留真实结果
<CRITICAL_PATH>	归属、包清单、当前存在性	按 <code>rpm -Vf</code> 字段选择 <code>stat</code> / <code>readlink</code> / <code>getcap</code>	保留真实结果
<PACKAGE>	NEVRA、 <code>rpm -V</code> 与配置清单	汇总是否需要最小修复，不扩大事务范围	包层与功能层均复查

这个模板用于把“看到差异”转成可审计的判断链；答案不能只贴命令而不解释证据。

[参考解答] 任务一：从归属调查到最小修复

下面使用占位符表示题目提供的真实路径。执行时必须替换为实际对象，并保留真实输出。

① 调查路径归属，不先猜包名

```
rpm -qf <CONFIG_PATH>
rpm -qf <CRITICAL_PATH>
```

如果两条结果不同，应停止套用“同一包”的假设，分别调查；如果某条路径不存在，先用父目录、题目说明和候选包清单确认路径，再通过包清单判断是否应存在。

将共同包名保存为逻辑占位符：

```
pkg=$(rpm -qf <CONFIG_PATH>)
printf '%s\n' "$pkg"
```

注意：rpm -qf 默认可能输出完整包标识。后续命令可直接使用该结果，也可另行查询 Name 字段。

② 记录精确包身份

```
rpm -q --qf 'Name: %{NAME}\nEpoch: %{EPOCH}\nVersion: %{VERSION}\nRelease: %{RELEASE}\nArch: %{ARCH}\n' "$pkg"
```

验收时保留真实字段。Epoch 未设置时不虚构值。

③ 建立包文件视图

```
rpm -ql "$pkg"
rpm -qc "$pkg"
rpm -qd "$pkg"
```

确认 <CONFIG_PATH> 是否被标记为配置文件，确认 <CRITICAL_PATH> 是否仍在当前版本文件清单中。若路径不在清单中，它可能来自旧版本、运行时生成或其他对象。

④ 先验证最小范围，再决定是否扩大

```
rpm -Vf <CONFIG_PATH>
rpm -Vf <CRITICAL_PATH>
rpm -V "$pkg"
```

记录每条输出：

```
九位属性差异
→ 文件类别标记
→ 路径
→ 当前是否存在
```

若无输出，只记录“本次可检查属性未发现差异”，不写“服务正常”。

⑤ 根据差异选择下一条证据

```
stat <CONFIG_PATH>
stat <CRITICAL_PATH>
readlink -f <CRITICAL_PATH> # 仅在 L 差异或符号链接场景
getcap <CRITICAL_PATH>     # 仅在 P 差异场景
```

内容摘要差异：

```
diff -u <KNOWN_BASELINE> <CONFIG_PATH>
```

如果没有独立基线，优先检查 .rpmnew/.rpmserve、版本控制、配置管理记录或从相同 NEVRA 包文件中提取对应文件，不把未知来源文件当作“正确版本”。

⑥ 区分预期配置与意外漂移

- 配置差异与已批准变更一致：记录为预期，不覆盖；
- 权限或所有者与应用要求不符：在确认业务和安全要求后做最小修正；
- 关键文件缺失或内容意外变化：确认没有本地定制后，可评估 `dnf reinstall "$pkg"`；
- 同一包大量文件异常：先调查事务、磁盘和安全事件，不机械逐文件修复。

⑦ 修复后回到同一证据层

```
rpm -V "$pkg"
```

预期配置仍可能继续显示差异，应在记录中解释。然后执行应用真实提供的配置语法检查和功能检查。

⑧ 验收矩阵

层次	合格条件
包身份	NEVRA 与目标一致
文件归属	两个路径归属已明确
文件差异	每条差异有解释或修复记录
配置	合法本地修改未被覆盖，语法检查通过
功能	应用的真实功能检查通过
安全	未使用放宽依赖和冲突保护的快捷选项

典型错误

- 用 `rpm -qa | grep` 的模糊结果直接进入删除或重装；
- 看到配置 5 差异就用厂商文件覆盖；
- 只运行 `rpm -q` 就宣布问题解决；
- 对 `missing` 不检查挂载、符号链接和版本变化；
- 修复后不再次运行同一验证命令。

提交前自检

- 两个路径的归属是否分别核实，而不是先假设同包；
- 是否记录真实 NEVRA，而不是从文件名或默认文本中猜字段；
- 每条 `rpm -V` 输出是否同时解释属性位置、文件类别和路径；
- 配置差异是否先与基线或变更记录比较；
- 最小修复后是否回到同一命令复查，并继续做应用功能验证；
- 报告中是否明确区分“未发现属性差异”和“服务功能健康”。

满足这些条件后，任务一才形成从调查、判断、修复到再验证的闭环。

[经典任务] 任务二：检查本地 RPM 后以推荐路径安装并验收

环境

题目在以下目录提供一个本地 RPM：

```
/home/student/incoming/<PACKAGE_FILE>.rpm
```

系统已经有可用的 RHEL 9 仓库来满足依赖。本任务不得修改仓库配置；仓库和模块流归第 17 章。

当前状态

- 不确定同名包是否已经安装；
- 不确定本地 RPM 的版本、架构、脚本和签名状态；
- 包可能提供配置文件和 `systemd` unit；
- 不能假设安装后服务自动启动。

目标终态

1. 查询本地 RPM 的精确身份、摘要、文件、配置、依赖和脚本；
2. 检查包摘要和签名，并记录真实信任状态；
3. 查询系统当前是否安装同名包及其 NEVRA；
4. 使用 DNF 的本地文件路径执行适当安装或升级；
5. 完整阅读并确认事务摘要；
6. 验证最终包身份、关键文件和配置文件；
7. 检查 `.rpmnew/`、`.rpmsave` 和 `rpm -V`；
8. 若包提供 unit，分别检查 unit 存在、当前状态、持久状态和真实功能。

限制条件

- 不使用 `rpm --nodeps`、`--force`、`dnf --nogpgcheck` 或 `--allowerase`；

- 不把缺少公钥自动改写成“签名失败”；
- 不用 `-y` 跳过第一次事务摘要审阅；
- 不编造服务名、端口或测试输出。

[参考解答] 任务二：本地包预检、事务与分层验收

① 建立文件对象并确认可读

```
pkgfile=/home/student/incoming/<PACKAGE_FILE>.rpm
ls -l "$pkgfile"
file "$pkgfile"
```

`file` 只能作为外形检查，真正包身份仍由 RPM 包头提供。

② 查询本地包头和文件内容

```
rpm -qpi "$pkgfile"
rpm -qp --qf 'Name: %{NAME}\nEpoch: %{EPOCH}\nVersion: %{VERSION}\nRelease: %{RELEASE}\nArch: %{ARCH}\n' "$pkgfile"
rpm -qpl "$pkgfile"
rpm -qpc "$pkgfile"
rpm -qp --requires "$pkgfile"
rpm -qp --scripts "$pkgfile"
```

记录 Name，供下一步查询已安装状态。不要从文件名猜 Name。

```
name=$(rpm -qp --qf '%{NAME}\n' "$pkgfile")
printf '%s\n' "$name"
```

③ 检查摘要、签名和架构

```
rpmkeys --checksig "$pkgfile"
uname -m
```

如果输出显示缺少公钥，应暂停并按组织信任流程取得正确公钥和核对指纹；本任务不允许用 `--nogpgcheck` 跳过。若包确实未签名，应按题目或组织策略决定是否允许，不能把它描述为已验证来源。

④ 查询当前已安装状态

```
rpm -q "$name"
rpm -q --qf '%{NAME}\t%{EPOCH}\t%{VERSION}\t%{RELEASE}\t%{ARCH}\n' "$name"
```

未安装、相同版本、较旧版本或并行架构会影响事务性质。不要只因为 `rpm -q` 非零就直接执行低层 `rpm -i`。

⑤ 通过 DNF 路径形成事务计划

```
sudo dnf install "$pkgfile"
```

在确认前检查：

- 目标包是安装还是升级；
- 需要新增、升级或删除哪些依赖；
- 是否出现意外架构或冲突；
- 总下载和安装大小；
- 是否有超出题目目标的删除。

摘要符合目标后再确认。若失败，保留错误并按 D01 分层调查，不添加破坏性选项。

⑥ 验证 RPM 数据库和关键路径

```
rpm -q "$name"
rpm -q --qf '%{NAME}\t%{EPOCH}\t%{VERSION}\t%{RELEASE}\t%{ARCH}\n' "$name"
rpm -ql "$name"
rpm -qc "$name"
```

从 `rpm -ql` 选择题目要求的关键路径：

```
rpm -qf <CRITICAL_PATH>
stat <CRITICAL_PATH>
```

⑦ 检查文件差异和配置迁移

```
rpm -V "$name"
find <RELEVANT_DIR> -type f \( -name '*.rpmnew' -o -name '*.rpmsave' \) -print
```

若存在旁置文件，执行 diff、选择性合并和应用语法检查，不机械覆盖。

⑧ 进入 unit 和功能层

先确认包是否提供 unit：

```
rpm -ql "$name" | grep -E '/systemd/(system|user)/'
```

只有得到真实 unit 名后才查询：

```
systemctl status <UNIT>
systemctl is-active <UNIT>
systemctl is-enabled <UNIT>
```

是否需要启动、启用、reload 或 restart 由题目终态和第 12 章方法决定。最后使用应用自身的版本、配置和业务接口完成验收。

⑨ 保留事务记录

```
dnf history
dnf history info <ID>
```

记录事务 ID、最终 NEVRA、配置处理和未完成的 live 验证。

验收矩阵

层次	合格条件
包文件	包头、架构、脚本和签名状态已记录
事务	摘要符合目标，无绕过保护选项
数据库	最终 NEVRA 明确
文件	关键路径存在且归属正确
配置	旁置文件已处理或明确记录
unit	提供关系、active、enabled 分别判断
功能	真实应用检查完成
审计	history 与变更记录可追溯

典型错误

- 把 `.rpm` 文件存在当成已安装；
- 只执行 `rpm -qpi`，未检查脚本和签名；
- 用 `rpm -ivh` 遇到依赖失败后添加 `--nodeps`；
- DNF 完成后不检查真实 NEVRA；
- 忽略 `.rpmnew/.rpmsave`；
- 看到 unit 文件就宣布服务可用。

[本章收束] 包管理的终点不是 Complete!

本章建立的主线不是一串 RPM 选项，而是从 包身份与 NEVRA 出发，依次区分本地包文件和已安装数据库、建立文件归属、解释签名与 `rpm -V` 差异、审阅 DNF 事务，并最终推进到配置、unit 和真实功能。

工作方法：每次软件变更都保留六类证据

- 1. 变更前身份：当前已安装 NEVRA、本地 RPM 包头、架构和信任状态；
- 2. 事务范围：DNF 摘要中将安装、升级和删除的完整集合；
- 3. 数据库终态：事务后的真实 NEVRA 和 history 记录；
- 4. 文件终态：关键路径、归属、属性差异和缺失状态；
- 5. 配置终态：`.rpmnew/.rpmsave` 的比较、合并与应用语法检查；
- 6. 功能终态：`unit`、进程、监听或应用真实接口的分层验收。

主要判断表

看到的证据	能证明什么	不能证明什么	下一条高区分度证据
<code>rpm -q PACKAGE</code> 成功	RPM 数据库存在匹配安装记录	文件未被修改、配置正确、服务可用	<code>rpm -ql/-qc</code> 、关键路径和 <code>rpm -V</code>
<code>rpm -qf /path</code> 成功	某已安装包声明拥有该路径	谁最后修改、内容是否正确	<code>stat</code> 、 <code>rpm -Vf</code> 、变更记录
<code>rpm -V</code> 无输出	本次可检查属性未发现差异	业务配置符合要求、服务健康	应用语法、 <code>unit</code> 和功能检查
<code>rpm -V</code> 有输出	当前文件与数据库基线存在指定差异	差异是否恶意或需要恢复	文件类别、 <code>stat</code> 、 <code>diff</code> 、配置记录
<code>rpmkeys --checksig</code> 可验证	包文件摘要和签名可按当前信任库验证	包适合当前系统、事务依赖可满足	NEVRA、架构、Requires 和 DNF 摘要
DNF Complete!	包事务已经提交完成	配置迁移、 <code>unit</code> 状态和业务功能正确	最终 NEVRA、旁置配置、应用与服务验收
存在 <code>.rpmnew/.rpmsave</code>	升级保留了两份需要处理的配置证据	哪一份应直接替换另一份	<code>diff</code> 、发行说明、应用语法检查

向下一章交接

本章假定 DNF 已经能够访问合适的软件源，重点处理包文件、已安装数据库、文件差异和事务终态。第 17 章《DNF 仓库、模块流与包组》将继续回答：候选包从哪里来、仓库元数据如何加载、GPG key 如何配置、模块流和包组如何改变可选范围，以及 DNS、TLS、元数据和签名错误应从哪一层调查。

DNF 仓库、模块流与包组

从 repo 定义到可审计事务：把来源、元数据、候选、信任 and 状态放进同一条证据链。

对象模型操作语义验证诊断经典任务

大字号阅读版

本章阅读导航

先抓住一条主线：DNF 不是看到 repo 文件后就直接安装包，而是依次读取来源定义、取得元数据、形成候选集合、验证信任、求解依赖并提交事务。

01仓库定义确认 repo ID、来源、启用和信任字段

02元数据与缓存确认 `repomd.xml` 和本地视图是否可用

03候选包解析结合架构、版本、过滤、stream 与优先级

04信任验证区分 TLS 传输链与 GPG 包签名链

05事务计划先读 Transaction Summary，再决定是否提交

06历史与交接用 history 审计，再转交 RPM、服务与功能层

专题地图

- *知识专题*从仓库定义到事务：DNF 到底在解析什么
- *知识专题*repo 文件、来源定位与信任字段
- *操作专题*配置并分层验证一个自定义仓库
- *操作专题*从需求定位候选软件并审阅事务
- *知识/操作模块* stream、profile 与模块过滤
- *知识专题*包组、环境组、优先级与排除
- *操作专题*建立本地仓库
- *诊断专题*配置、元数据、TLS、GPG、过滤和依赖
- *经典任务*配置可信仓库；诊断可见但不可用

阅读时持续回答

1. 当前证据是在证明配置、元数据、候选还是事务？
2. 仓库 ID、显示名和文件名分别是什么身份？
3. URL 是否真正指向包含 `repodata` 的仓库根？
4. 目标包为什么进入或离开当前候选集合？
5. TLS 与 GPG 分别验证哪一条信任链？
6. stream 与 profile 改变的是同一个状态吗？
7. Transaction Summary 会增加、升级或删除什么？
8. DNF 成功后还需要向哪一章交接？

阅读建议：每完成一个专题，都用“当前证据能证明什么、还不能证明什么、下一条证据是什么”复述一次；经典任务先独立完成，再对照参考解答检查证据链。

第 17 章 · 正文

一条 `dnf install` 命令背后并不只有“下载一个 RPM”。DNF 必须先读取客户端仓库定义，定位远端或本地内容根，取得并校验仓库元数据，再结合启用范围、架构、版本、模块流、优先级和排除规则形成候选集合。依赖求解器随后生成事务计划；只有事务真正提交后，RPM 已安装数据库才发生变化，DNF history 才留下可审计记录。

这条链上的每一层都可能独立失败。repo 文件存在，不代表仓库已被 DNF 正确识别；仓库出现在 `repolist`，不代表 `repomd.xml` 能够取得；元数据可用，不代表目标包没有被架构、exclude 或 module filtering 隐藏；候选包存在，不代表依赖一定可解；事务成功，也不代表应用配置、服务状态和外部功能已经正确。

- 仓库定义
 - 来源定位
 - 元数据与缓存
 - 候选包、模块和包组
 - TLS / GPG 信任
 - 依赖求解与事务
 - RPM 状态、history 与功能交接

本章围绕这条证据链组织内容。RPM 文件归属与校验留给第 16 章；网络、路由和 DNS 的完整配置留给第 18、19 章；本章只在诊断时识别这些层并给出交接证据。`--allowrasing`、`gpgcheck=0` 和 `sslverify=0` 都不会被当成未经调查的默认答案。

概念

仓库定义（repository definition）是 `/etc/yum.repos.d/*.repo` 中以 `[repo-id]` 开始的客户端 section。它声明仓库身份、内容位置、默认启用状态和信任策略，却不保存软件包清单本身。repo 文件能被读取只证明配置层存在；只有后续元数据请求成功，才能继续讨论候选和事务。

概念

仓库元数据与缓存（metadata and cache）让 DNF 认识包名、版本、架构、依赖、文件、包组和可能存在的模块信息。仓库根通常以 `repodata/repomd.xml` 为入口，本地缓存只是客户端对该元数据的当前视图；清理缓存不能修复错误的 URL、DNS、服务器内容或 GPG key。

概念

GPG 信任（GPG trust）描述安装包是否由受信任签名者签发且内容未被篡改。`gpgcheck=1` 是校验策略，`gpgkey=` 是公钥入口；它们不能互相替代。HTTPS/TLS 保护传输通道和服务身份，GPG 保护包签名，两条信任链必须根据错误层分别调查。

概念

候选包（package candidate）是当前仓库集合、架构、版本、优先级、排除规则和模块状态共同允许求解器选择的软件包构建。服务器目录中存在某个 RPM，并不意味着它一定处于客户端当前候选集合；`repoquery` 观察的正是这个被规则塑造后的视图。

概念

模块流（module stream）表示同一软件栈的一条版本线。启用 stream 会改变模块化包的候选集合，但不等于已经安装其中的全部内容；流名称、默认标记和可用版本必须从当前仓库元数据读取，不能把另一台机器或另一小版本的输出当成固定事实。

概念

profile 是模块针对 server、client、development 等用例定义的软件包集合。`module enable NAME:STREAM` 选择版本线，`module install NAME:STREAM/PROFILE` 才按用例提交安装事务；stream 是版本状态，profile 是安装选择，两者不能混为一谈。

概念

包组与环境组（package group / environment group）是仓库元数据描述的软件集合，不是一个 RPM。成员可能分为 mandatory、default、optional 和 conditional；组安装只说明一组包按元数据进入事务，不保证其中的服务已经配置、启动或能够对外提供功能。

概念

事务历史（transaction history）是 DNF 对已提交软件事务的审计记录，能够回答某次操作安装、升级或删除了哪些包。history 不是无条件回滚系统的快照：`undo` 或 `rollback` 仍依赖旧构建可获得、依赖关系可满足；Red Hat 不支持借此把关键 RHEL 系统包降级到旧版本，因此它们不能被当成核心系统包的盲目恢复按钮。

操作语义

先理解命令作用对象，再记关键形式。速查区只覆盖本章最重要的入口；正文专题仍负责解释状态、证据、验证和边界。

dnf repolist / makecache / repoquery

SYNOPSIS

```
<pre><code>dnf repolist [--all]\ndnf makecache [--refresh]\ndnf repoquery [OPTIONS] [PACKAGE ...]</code></pre>
<p>依次观察仓库是否被识别、真实刷新元数据、查询当前候选内容。三条命令证明的层次不同，不能互相替代。</p>
<div class="forms-title">重要参数 / 形式</div>
<dl><dt>`repolist --all`</dt><dd>列出启用和禁用仓库，验证配置解析与默认启用状态。</dd><dt>`makecache --refresh`</dt><dd>强制重新检查启用仓库元数据，暴露 URL、TLS、HTTP 和元数据错误。</dd><dt>`repoquery --repo ID PKG`</dt><dd>把候选查询限制到指定 repo ID，证明该仓库提供目标构建。</dd><dt>`--installed` / `--available`</dt><dd>分别观察已安装集合或当前可用候选集合。</dd></dl>
```

repo 配置文件

SYNOPSIS

```
<pre><code>/etc/yum.repos.d/NAME.repo\n[repo-id]\nname=...\nbaseurl=... | mirrorlist=... | \nmetalink=...\nenabled=0|1\nngpgcheck=0|1\nngpgkey=...</code></pre>
<p>持久定义仓库身份、来源、默认参与状态和包签名策略。文件名、repo ID 与显示名称是不同身份。</p>
<div class="forms-title">重要参数 / 形式</div>
<dl><dt>`baseurl`</dt><dd>直接指向包含 `repodata` 的仓库根。</dd><dt>`mirrorlist`</dt><dd>指向返回可用镜像 URL 列表的服务。</dd><dt>`metalink`</dt><dd>提供镜像选择及附加元数据。</dd><dt>`enabled`</dt><dd>控制仓库是否默认参与普通查询和求解。</dd><dt>`gpgcheck` / `gpgkey`</dt><dd>分别声明包签名校验策略和公钥入口。</dd></dl>
```

dnf install / upgrade / remove / history

SYNOPSIS

```
<pre><code>dnf install [OPTIONS] PACKAGE ...\ndnf upgrade [PACKAGE ...]\ndnf remove PACKAGE ...\ndnf history [list|info|undo|rollback] [ID]</code></pre>
<p>让求解器生成并提交软件事务，再通过 history 审计已发生的变化。删除和恢复类操作必须先读完整事务摘要。</p>
<div class="forms-title">重要参数 / 形式</div>
<dl><dt>`--assumeno`</dt><dd>生成事务计划但默认拒绝提交，适合预览安装、升级或删除影响。</dd><dt>`--enablerepo=ID`</dt><dd>只在当前命令临时启用指定仓库，不改变 repo 文件。</dd><dt>`--disablerepo=PATTERN`</dt><dd>只在当前命令临时缩小仓库范围；限定来源时必须保留依赖仓库。</dd><dt>`history info ID`</dt><dd>读取指定事务的命令、状态和包变化。</dd></dl>
```

dnf module

SYNOPSIS

```
<pre><code>dnf module list [NAME]\ndnf module info NAME[:STREAM]\ndnf module enable NAME:STREAM\ndnf module install NAME:STREAM[/PROFILE]\ndnf module reset[disable NAME]</code></pre>
<p>观察并改变模块版本线和 profile 安装选择。启用 stream 与安装 profile 改变的是不同状态。</p>
<div class="forms-title">重要参数 / 形式</div>
<dl><dt>`NAME:STREAM`</dt><dd>显式选择模块及版本线，避免依赖模糊默认值。</dd><dt>`NAME:STREAM/PROFILE`</dt><dd>按指定用例安装 profile 中的软件集合。</dd><dt>`reset`</dt><dd>清除显式 stream 选择，不等于删除已安装包。</dd><dt>`disable`</dt><dd>使该模块的流不参与普通模块化候选解析。</dd></dl>
```

dnf group

SYNOPSIS

```
<pre><code>dnf group list [--hidden]\ndnf group info GROUP\ndnf group install GROUP\ndnf group remove GROUP</code></pre>
<p>查询和提交包组或环境组事务。先用 `group info` 读取 mandatory、default 和 optional 成员，再决定安装范围。</p>
<div class="forms-title">重要参数 / 形式</div>
<dl><dt>`list --hidden`</dt><dd>同时查看默认隐藏的组。</dd><dt>`group info`</dt><dd>读取组 ID、描述和成员分类。</dd><dt>`group install`</dt><dd>按组元数据形成安装事务，不等于完成服务配置。</dd></dl>
```

createrepo_c

SYNOPSIS

```
<pre><code>createrepo_c [OPTIONS] DIRECTORY</code></pre>
<p>扫描普通 RPM 目录并生成可供 DNF 读取的 `repodata`。它不会凭空恢复缺失的包组或模块语义。</p>
<div class="forms-title">重要参数 / 形式</div>
<dl><dt> DIRECTORY`</dt><dd>包含目标 RPM 的仓库内容目录。</dd><dt>`--update`</dt><dd>在已有元数据基础上更新普通仓库索引。</dd>
<dt>`file:///PATH`</dt><dd>客户端 repo 定义引用本地绝对路径时使用三条斜线。</dd></dl>
```

[知识专题] 从仓库定义到事务：DNF 到底在解析什么

排查软件安装问题时，最容易犯的错误是把“repo 文件存在”“仓库已启用”“元数据能下载”“包能查询”“依赖能求解”和“事务已完成”混成一个状态。它们分别属于不同对象，也需要不同证据。先把对象分开，才能知道失败停在哪一层。

① [知识点] 仓库定义、仓库内容和本地缓存是三个对象

repo 文件只是客户端配置。它描述仓库 ID、来源、启用和信任策略，但不包含真实的软件包清单。远端或本地仓库内容由 `repodata` 和 RPM 构成：客户端还会在本地保存一份元数据缓存，以便后续查询和求解。

<code>/etc/yum.repos.d/exam.repo</code>	客户端持久定义
<code>http://content/.../BaseOS/</code>	仓库内容根
<code>/var/cache/dnf/...</code>	客户端元数据和下载缓存

修改 repo 文件后，旧缓存可能仍存在。相反，删除缓存不会修复错误的 URL、DNS 或 GPG key。诊断时必须确认自己正在检查哪一层。

② [知识点] `repomd.xml` 是仓库元数据的入口，不是包本身

典型仓库根下存在：

```
repodata/repomd.xml
Packages/ 或其他 RPM 路径
```

`repomd.xml` 记录其他元数据文件的位置和校验值。DNF 获取它之后，才能继续读取包名称、版本、架构、依赖、文件列表、包组或模块等信息。错误 URL 常表现为下载 `repodata/repomd.xml` 时返回 404；这通常说明 `baseurl` 没有指向真正的仓库根。

③ [知识点] 候选集合由多层规则共同缩小

仓库中存在某个包，只说明服务端保存了它。要进入当前 DNF 候选集合，还要满足：

- 仓库被当前命令启用；
- 架构适配本机或属于允许的 `noarch`；
- 版本没有被排除规则过滤；
- `module filtering` 没有隐藏它；
- 优先级和重复候选选择规则允许它参与；
- 必要的依赖也能从当前仓库集合中获得。

因此“包明明在服务器上”不能直接推出客户端一定能安装。

④ [知识点] 查询、求解和提交事务是三个阶段

`dnf repoquery` 主要回答“当前元数据中有哪些候选及其关系”；`dnf install --assumeno` 可让求解器生成事务摘要而不提交；真正执行 `dnf install` 后，才改变 RPM 已安装数据库并产生 DNF history 记录。

```
repoquery 有结果
≠ 依赖一定可解
≠ 事务已执行
≠ 应用已完成配置
```

已安装数据库、文件归属和 RPM 校验属于第 16 章；服务的 active/enabled 和功能属于相应服务章节。本章只建立必要接口。

⑤ [知识点] DNF 成功只覆盖软件事务层

DNF 返回成功可以证明求解和包事务没有报告失败，但不能证明：

- 配置文件符合业务要求；
- systemd 服务正在运行或开机启动；
- 网络端口已监听；
- 防火墙与 SELinux 已允许访问；
- 用户从外部获得了正确功能。

考试题若最终要求“服务可用”，DNF 事务只是中间证据，不是最终终态。

[Cheatsheet] repo 文件是定义；`repodata` 是仓库元数据；缓存是客户端视图；`repolist` 看配置，`makecache` 取元数据，`repoquery` 看候选，事务改变已安装状态；任何一层成功都不能替代下一层验证。

[知识专题] repo 文件、来源定位与信任字段

repo 文件看起来只是几行 INI 配置，但每个字段回答不同问题：仓库叫什么、从哪里取内容、是否参与当前求解、是否校验包签名、使用哪把公钥。最稳妥的切入方式是先分清“身份”“位置”“启用”和“信任”，再讨论临时覆盖和变量展开。

① [知识点] section ID、文件名和 `name` 不是同一身份

```
[exam-baseos]
name=Exam BaseOS for RHEL 9
```

- `exam-baseos` 是仓库 ID，供 `--repo`、`--enablerepo` 等选项引用；
- `name` 是面向人的显示名称；
- `exam.repo` 只是承载配置的文件名。

仓库 ID 应在全部已加载配置中唯一。两个文件重复定义同一 ID 会制造覆盖或重复定义风险，不能仅靠文件名判断实际加载结果。

② [知识点] `baseurl`、`mirrorlist` 和 `metalink` 是三种定位方式

`baseurl` 直接给出一个或多个仓库根；`mirrorlist` 指向返回镜像 URL 列表的服务；`metalink` 除镜像信息外还可带有用于选择和完整性检查的附加元数据。一个仓库通常选择一种主要定位方式，避免同时留下互相矛盾的来源。

```
baseurl=https://repo.example.com/rhel9/BaseOS/
# 或
mirrorlist=https://mirror.example.com/rhel9/baseos.list
# 或
metalink=https://mirrors.example.com/metalink?repo=rhel-9-baseos
```

本章不要求搭建 `mirrorlist` 或 `metalink` 服务，但要求能够识别字段、读取最终错误 URL，并知道 `baseurl` 必须指向包含 `repodata` 的内容根。

③ [知识点] DNF 变量与 Shell 变量属于不同解析器

repo 配置可使用 `$releasever`、`$basearch` 等 DNF 变量。例如：

```
baseurl=https://repo.example.com/rhel/$releasever/$basearch/BaseOS/
```

若用未引用的 here-document 在 Shell 中生成文件，Shell 可能先尝试展开 `$releasever`，导致空字符串或错误路径。可以使用单引号保护的 here-document：

```
cat > /etc/yum.repos.d/exam.repo <<'EOF'
[exam-baseos]
name=Exam BaseOS
baseurl=https://repo.example.com/rhel/$releasever/$basearch/BaseOS/
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release
EOF
```

题目给出固定 RHEL 9 URL 时，直接使用题目参数，不自作主张替换成别的小版本或 RHEL 10 路径。

④ [知识点] `enabled` 只控制是否参与普通命令

`enabled=1` 表示仓库默认参与查询和求解；`enabled=0` 表示默认禁用。命令级选项可以临时覆盖：

```
dnf --enablerepo=exam-appstream repoquery httpd
dnf --disablerepo='*' --enablerepo=exam-baseos,exam-appstream install httpd
```

这种覆盖只影响当前命令，不修改 repo 文件。第二种“只启用指定仓库”只有在题目明确要求限定来源时才使用，并且必须把依赖仓库一起纳入，否则会人为制造依赖缺口。

⑤ [知识点] `gpgcheck` 与 `gpgkey` 分别描述策略和密钥入口

`gpgcheck=1` 要求安装包通过 GPG 签名检查；`gpgkey=` 指向可用于导入或取得公钥的位置。二者不是同义字段：配置了 `key` 但关闭 `gpgcheck`，仍然没有执行目标校验；开启校验但 `key` 错误，则事务会失败。

```
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release
```

GPG 失败的正确方向是核对仓库来源、key 文件、签名者和完整错误，而不是直接改为 `gpgcheck=0`。

⑥ [知识点] TLS 信任与 GPG 信任是两条不同链

HTTPS/TLS 主要保护传输通道和服务器身份；GPG 包签名用于验证软件包是否由受信任签名者签发、内容是否被篡改。可能出现：

- TLS 失败，尚未取得元数据；
- TLS 成功，但 GPG key 不匹配；
- 两者都成功，但包被模块或排除规则过滤。

排错时根据实际错误层推进，不把一种信任机制替代另一种。

⑦ [边界] `sslverify=0` 和 `gpgcheck=0` 都不是默认修复

关闭 TLS 证书验证或包签名校验会移除安全属性，只能掩盖错误，不能证明来源正确。考试与真实工作中应优先修复：

```
错误的 URL / DNS / 时间 / CA 信任
错误或缺失的 GPG key
仓库内容与 key 不匹配
```

[Cheatsheet] `[repo-id]` 是命令行身份，`name` 是显示名；三种来源字段为 `baseurl/mirrorlist/metalink`；`enabled` 控制默认参与；`gpgcheck` 是校验策略，`gpgkey` 是密钥入口；TLS 与 GPG 分层验证，不靠关闭保护通过任务。

[操作专题] 配置并分层验证一个自定义仓库

仓库配置操作的正确顺序不是“写完文件就安装”，而是先建立基线，再完成最小定义，随后依次验证解析、元数据和代表包。这样一旦失败，可以准确知道问题位于配置、传输、元数据还是内容层。

① [操作] 变更前记录现有仓库基线

作用对象：当前 DNF 配置和启用仓库集合。基本语义：在写入新 repo 前保存能解释后续变化的只读证据。典型形式：

```
ls -l /etc/yum.repos.d/
dnf repolist --all
dnf module list --enabled
```

如果系统属于订阅环境，还应避免随意删除现有 Red Hat 仓库定义。本章不展开注册和订阅管理，但要求不以“清空目录”作为配置自定义仓库的默认步骤。

② [操作] 写入最小且可审计的 repo section

```
[exam-baseos]
name=Exam BaseOS
baseurl=http://content.example.com/rhel9.0/x86_64/dvd/BaseOS
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release

[exam-appstream]
name=Exam AppStream
baseurl=http://content.example.com/rhel9.0/x86_64/dvd/AppStream
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release
```

关键参数：题目给出的 ID、URL 和 key 必须逐项保留；`BaseOS` 与 `AppStream` 常互相补充，不能仅凭目标包名称猜只需一个仓库。验证与边界：静态阅读配置可以发现拼写、重复 ID 和明显路径错误，但不能证明服务器可达。

③ [操作] 使用 `dnf repolist --all` 验证解析和启用状态

```
dnf repolist --all
```

这一步能确认：

- repo 文件语法至少能够被 DNF 读取；
- 仓库 ID 出现在加载结果中；

- 当前持久启用状态是 enabled 还是 disabled。

它不能充分确认 URL 可达，也不能证明 `repodata` 完整。看到仓库 ID 后仍要继续刷新元数据。

④ [操作] 使用 `makecache --refresh` 真实请求元数据

```
dnf makecache --refresh
```

`--refresh` 让 DNF 把元数据视为过期并重新检查。该命令能够暴露：

- DNS 或连接错误；
- TLS/CA 错误；
- HTTP 403、404、5xx；
- 错误仓库根；
- `repomd.xml` 或元数据校验失败。

不要在不知道错误类型时反复执行 `dnf clean all`。清缓存只是移除本地状态；如果错误在 URL 或服务器端，清理不会产生修复效果。

⑤ [操作] 用代表包证明仓库内容符合预期

```
dnf repoquery --repo exam-baseos bash
dnf repoquery --repo exam-appstream httpd
```

作用对象：指定仓库的候选包元数据。基本语义：把查询范围限制到某个 repo ID，确认该仓库确实提供预期内容。边界：一个代表包有结果只能证明该查询成功，不能证明仓库全部 RPM、组或模块元数据都完整。

⑥ [验证] 建立四层证据矩阵

层	推荐证据	证明范围
配置	读取 <code>.repo</code> 、 <code>dnf repolist --all</code>	ID、字段和启用状态
元数据	<code>dnf makecache --refresh</code>	仓库元数据可取得并通过校验
内容	<code>dnf repoquery --repo ... <pkg></code>	指定仓库提供代表候选
事务	<code>dnf install</code> 、 <code>dnf history info</code>	求解并提交了软件包事务

四层都通过后，才可以说“仓库能够支撑这个安装任务”；仍不能把它扩大为服务功能已经正确。

[Cheatsheet] 基线 `repolist --all` → 写最小 repo → 再次 `repolist --all` → `makecache --refresh` → `repoquery --repo` → 事务。不要清空所有仓库，不关闭 GPG，不把仓库可见等同于可用。

[操作专题] 用 DNF 和 `repoquery` 从需求定位候选软件

软件题经常只给“需要某个命令”“需要一个 Web 服务器”或“某文件应由哪个包提供”，不直接给包名。此时要先判断已知信息属于描述、包名、命令路径还是依赖关系，再选择最窄的查询接口。查询结果是证据，不应机械安装第一项。

① [操作] 已知描述时使用 `dnf search`

```
dnf search 'web server'
dnf search --all 'network time'
```

`search` 匹配包名称和摘要；`--all` 扩大到描述等字段。结果可能包含工具、库、文档和兼容包，应继续使用 `dnf info` 或 `repoquery` 确认候选身份。

② [操作] 已知包名时区分 installed 和 available

```
dnf list --installed httpd
dnf list --available httpd
dnf info httpd
```

`dnf info` 可能同时出现 installed 与 available 区块。要回答“本机是否已经安装”，最终仍应引用第 16 章的 RPM 查询；要回答“仓库提供什么版本”，使用 available 或 `repoquery`。

③ [操作] 已知命令或路径时使用 `dnf provides`

```
dnf provides '*/semanage'  
dnf provides /usr/bin/rsync
```

通配模式必须引用，避免 Shell 在当前目录先展开。查询结果可能包含旧版本、不同架构或兼容子包，应结合目标路径、RHEL 9 仓库和架构选择。

④ [操作] 使用 `repoquery` 查看来源、版本和架构

```
dnf repoquery --available --qf '%{name}-%{epoch}:%{version}-%{release}.%{arch} %{repoid}' httpd  
dnf repoquery --repo exam-appstream httpd
```

自定义查询格式可以同时显示候选标识和来源仓库。实际可用的格式字段应以当前 `dnf repoquery --querytags` 或帮助为准；讲义中的骨架用于说明查询方向，不虚构固定输出。

⑤ [操作] 查询包文件与依赖关系

```
dnf repoquery -l httpd  
dnf repoquery --requires httpd  
dnf repoquery --whatrequires 'libssl.so.3()(64bit)'
```

- `-l` 读取仓库包的文件列表，不要求包已安装；
- `--requires` 读取包声明的依赖；
- `--whatrequires` 查找声明依赖某能力的候选。

依赖查询用于解释求解错误，但不能取代完整事务摘要，因为弱依赖、冲突、模块和已安装状态也会影响最终计划。

⑥ [诊断] 无结果时按过滤链排查

```
包名或路径是否正确  
→ 当前启用仓库是否包含目标内容  
→ 架构是否匹配  
→ 是否被 excludepkgs/includepkgs 过滤  
→ 是否受到 module filtering  
→ 元数据缓存是否对应当前配置
```

不要在第一步就使用 `--allowerasing`。该选项处理的是冲突事务中允许删除包的问题，不能修复仓库缺失、拼写错误或模块候选不可见。

[Cheatsheet] 描述用 `search`；包名用 `list/info`；命令或文件用 `provides`；来源、版本、文件、依赖用 `repoquery`。无结果依次查 repo、架构、exclude、module 和缓存。

[操作专题] 安装、升级、删除与 DNF 历史

DNF 改变系统前会先生成事务计划。稳定操作的关键不是快速输入 `y`，而是读懂将安装、升级、降级或删除哪些包，以及这些变化来自哪个仓库。事务执行后再用 history 和 RPM 层证据确认结果。

① [操作] 安装与重新安装

```
dnf install httpd  
dnf reinstall httpd
```

`install` 在候选集中选择满足条件的包并解决依赖；`reinstall` 重新安装当前版本或可用等价构建，适用于已确认包文件损坏且配置边界清楚的情况。它不是处理未知应用故障的第一步，因为配置文件、数据和服务状态可能不由重装自动恢复。

② [操作] 检查和执行升级

```
dnf check-upgrade  
dnf upgrade httpd  
dnf upgrade
```

- `check-upgrade` 只检查可用更新；其退出状态和输出应按帮助页解释；
- 指定包升级能限制影响范围；
- 无参数 `upgrade` 可能改变大量系统包，考试题未要求全量更新时不要扩大范围。

升级前应确认仓库集合和候选来源，避免临时仓库意外替换已有包。

③ [操作] 删除前必须阅读 Transaction Summary

```
dnf remove httpd
```

删除事务可能同时移除依赖或不再需要的软件包。提交前至少检查：

- 目标包是否正确；
- 是否出现不期望的服务、库或工具；
- 是否会被破坏其他题目已完成的功能；
- 删除数量和下载/释放空间是否符合预期。

包名不是实际影响集合。若摘要异常，应取消并调查依赖、模块和组状态。

④ [操作] 使用 `--assumeno` 预览事务

```
dnf install --assumeno httpd
dnf remove --assumeno httpd
```

该选项允许 DNF 进行查询和求解，但在确认阶段默认回答否。它适合建立变更前证据。它仍可能刷新元数据，因此不是绝对“零外部访问”；但不会提交包安装或删除事务。

⑤ [操作] 使用 history 审计事务

```
dnf history
dnf history info 12
```

`history` 列出事务 ID、时间、动作和结果摘要；`history info` 展开指定事务涉及的软件包和命令行。它可以帮助回答“刚才改变了什么”，但不是完整业务审计系统，也不能证明包外配置未被修改。

⑥ [边界] `undo` 和 `rollback` 不是无条件恢复按钮

```
dnf history undo 12
dnf history rollback 10
```

这些操作仍需重新求解事务，并依赖旧版本在当前仓库中可取得。若历史事务涉及核心系统包、模块流、后来安装的软件或已删除仓库，恢复可能失败或产生额外影响。执行前先预览摘要并确认回退目标；考试中没有明确要求时，不把 `rollback` 当成首选补救。

⑦ [验证] 从事务交接到已安装状态和功能

```
DNF 事务摘要和 history
→ rpm -q / rpm -qf (第 16 章)
→ 配置文件与语法
→ systemd 当前/持久状态 (第 12 章)
→ 监听、访问或数据结果
```

本章讲清交接点，但不重复相邻章节的完整操作。

[Cheatsheet] 安装 `install`，修复已确认包内容可考虑 `reinstall`，升级先 `check-upgrade`，删除前读完整摘要，预览用 `--assumeno`，历史用 `history/info`；`undo/rollback` 仍受仓库和依赖约束。

[知识专题] 模块、stream、profile 与模块过滤

模块解决的是“同一软件栈存在不同生命周期或版本线时，仓库如何提供可控选择”。它在普通包元数据之上增加 stream 和 profile 状态，因此模块问题不能只用包名解释。最顺的学习路径是先识别四个对象，再理解启用、安装、重置和禁用分别改变什么。

① [知识点] module、stream、profile 和 RPM 是四个层次

```
module: nginx
stream: 1.22
profile: common / server / development
packages: nginx、依赖库、工具等 RPM
```

具体名称、流和 profile 取决于当前 RHEL 9 仓库元数据。讲义不要求背诵某个流一定存在，而要求先查询后操作。

② [知识点] stream 表示版本线，不只是一个包版本

启用一个 stream 会让该 stream 的模块化包参与求解，并可能过滤其他流或非模块候选。一个 stream 可以随仓库更新得到新的构建，但仍属于同一版本线。不能把 stream 当成单个 RPM 的完整 NEVRA。

③ [知识点] profile 是安装用例，不是模块状态本身

profile 描述某个使用场景所需的包集合。例如 common、server、client、development 可能包含不同成员。启用 stream 后不一定安装任何包；安装 profile 才会提交对应软件包事务。

④ [知识点] 模块状态需要从真实输出读取

```
dnf module list
dnf module list --enabled
dnf module list --installed
dnf module info <module>
```

输出标记和可用流会随小版本、仓库和语言环境变化。学习时关注：哪些流可用、哪个流当前 enabled、是否有 installed profile，而不是背固定截图。

⑤ [知识点] enable 与 install 改变不同对象

```
dnf module enable <module>:<stream>
dnf module install <module>:<stream>/<profile>
```

- `enable` 选择活动 stream，改变候选集合；
- `install` 在选择 stream 的同时安装 profile 或默认 profile 中的包。

题目只要求版本流选择时，不额外安装 profile；题目只要求普通包且未提模块时，先查询候选，不主动改变模块状态。

⑥ [知识点] reset、disable 和 remove 不是同义操作

- `module reset` 清除显式 stream 选择，通常不自动删除已安装包；
- `module disable` 阻止该模块各 stream 作为模块候选；
- `module remove` 计划移除模块 profile 的软件内容，实际影响必须读事务摘要。

重置状态后，现有已安装包仍可能存在。要回答“系统还装着什么”，应转到 RPM 层查询。

⑦ [边界] 切换 stream 是有状态迁移

在支持的环境中，`dnf module switch-to` 可用于从一个 stream 迁移到另一个 stream。实际事务取决于当前安装内容和仓库可用构建。切换前应：

```
记录当前 enabled/installed 状态
→ 查看目标 stream 和 profile
→ 预览事务摘要
→ 确认是否降级、升级或删除包
→ 提交后重新验证模块和已安装状态
```

不把 `reset + enable` 的随意组合当成通用切流答案。

⑧ [诊断] module filtering 会制造“仓库有包但查询不到”

当某包存在于仓库但被非活动 stream 隐藏时，普通 `dnf list` 或安装可能显示没有匹配项或被过滤。此时最有区分度的证据是：

```
dnf module list
dnf module provides '<package-or-path>'
dnf repoquery --available <package>
```

结合完整 DNF 错误判断是模块过滤、排除规则还是仓库缺失。

[Cheatsheet] module 是软件栈，stream 是版本线，profile 是用例包集合，RPM 是实际安装单元；enable 选流，install 装 profile，reset 不等于卸载，disable 阻止模块流，切流必须读事务影响。

[操作专题] 查询和改变模块状态

模块操作不应从 `enable` 开始，而应从“当前仓库实际提供什么”开始。下面的顺序把查询、选择、安装和验证分开，避免在不知道当前状态时盲目 `reset` 或切流。

① [操作] 查询可用流和 profile

```
dnf module list <module>
dnf module info <module>:<stream>
dnf module info --profile <module>:<stream>
```

验证：确认题目指定的 stream 和 profile 确实存在。若不存在，先检查 AppStream 元数据、RHEL 小版本和仓库来源，而不是创建一个近似名称。

② [操作] 只启用指定 stream

```
dnf module enable <module>:<stream>
```

提交前阅读事务摘要或模块状态变化。操作后：

```
dnf module list --enabled <module>
```

这只能证明流选择，不证明任何 profile 已安装。

③ [操作] 安装显式 profile

```
dnf module install <module>:<stream>/<profile>
```

省略 profile 时可能安装默认 profile；考试需要精确内容时应使用题目给出的 profile。完成后同时检查：

```
dnf module list --installed <module>
rpm -q <representative-package>
```

第二条属于第 16 章接口，用来证明实际包状态。

④ [操作] 重置或禁用前先建立基线

```
dnf module list --enabled
dnf module list --installed
dnf module reset <module>
# 或按明确目标：
dnf module disable <module>
```

reset 后重新读取 module list，并单独检查已安装包是否仍存在。disable 可能影响后续更新和依赖求解，不能作为解决普通包冲突的随手操作。

⑤ [操作] 切流前使用预览和分层验证

```
dnf --assumeno module switch-to <module>:<target-stream>
```

若当前工具支持该组合，先查看拟升级、降级、替换或删除的包；真实命令语法以 `dnf module switch-to --help` 为准。提交后分别验证：目标 stream、installed profile、代表包版本和应用功能。

[Cheatsheet] 先 `module list/info`，再按题意 enable 或 install；显式 profile 更可控；reset/disable 前后分开查 module 与 RPM；切流必须预览事务，不背固定流。

[知识专题] 包组、环境组与成员分类

包组用于一次表达一类系统能力，例如开发工具或服务器环境。它的价值是让仓库维护者定义“这个用例通常需要哪些包”，但组不是不可分割的整体。理解成员分类和 DNF 的组状态，才能解释为什么某些包已安装、某些 optional 包没有安装，或者组删除后仍有成员保留。

① [知识点] 包组和环境组来自仓库元数据

- package group：一组相关软件包；
- environment group：可以包含多个包组和额外包，表达更大的系统环境。

如果仓库没有相应 comps/group 元数据，`dnf group list` 就无法凭 RPM 文件名自动推断组。

② [知识点] mandatory、default、optional 和 conditional 含义不同

- mandatory：组的必要成员；
- default：正常组安装时默认选择；
- optional：只有显式请求或相关选项时才安装；

- conditional：在满足关联条件时加入。

查看组详情比背“组会安装哪些包”更可靠：

```
dnf group info 'Development Tools'
```

③ [知识点] 显示名和 group ID 的稳定性不同

显示名可能受语言环境影响，脚本和考试操作更适合先通过 `group list --hidden -v` 或 `group info` 找到稳定 ID。不要因中文或英文名称不同误判组不存在。

④ [操作] 查询、安装和删除组

```
dnf group list
dnf group list --hidden
dnf group info '<group-name-or-id>'
dnf group install '<group-name-or-id>'
dnf group remove '<group-name-or-id>'
```

删除组前仍要读 Transaction Summary。用户后来显式安装、其他组共享或其他软件依赖的成员，可能不会简单按“组内全部包”删除。

⑤ [边界] 组安装标记与包实际存在是两个维度

一组成员包可能在安装组之前已经存在，DNF 还会记录组事务和安装原因。反过来，手工安装部分成员不一定让 DNF 认为整个组已安装。验证时既看 group 状态，也按题目要求检查代表包。

⑥ [边界] 包组只完成软件集合，不完成服务配置

安装“Web Server”或类似组后，相关服务仍可能：

- 未启用或未启动；
- 使用默认配置；
- 被 firewalld 或 SELinux 阻止；
- 缺少题目要求的数据。

组事务必须交接到对应服务和安全章节继续验收。

[Cheatsheet] 组来自仓库 comps 元数据；成员分 mandatory/default/optional/conditional；先 `group list/info`，再 `install/remove`；显示名可能本地化，ID 更稳定；组安装不等于服务可用。

[知识专题] 多仓库优先级、成本与排除规则

多个仓库提供同名包时，DNF 不只是“任选一个最高版本”。仓库优先级、成本、包排除、命令级启用范围和模块过滤都会影响候选。这里覆盖 RHCSA 和常见工作所需的必要范围，不扩展为完整企业内容治理策略。

① [知识点] `priority` 数值越小，仓库优先级越高

```
priority=10
```

当不同优先级仓库提供同名候选时，高优先级仓库可阻止低优先级仓库的更高版本被选中。默认值和精确比较细节应以 RHEL 9 的 `dnf.conf(5)` 为准；操作时关注“数值越小越优先”以及它可能导致版本看似被压低。

② [知识点] `cost` 用于同优先级来源的偏好

较低 `cost` 通常更受偏好，可用于倾向本地或较便宜的镜像。它不等同于 `package priority`，也不能替代 GPG、版本或模块规则。考试不要求时不随意增加 `cost`。

③ [知识点] `excludepkgs` 和 `includepkgs` 改变候选集合

```
excludepkgs=kernel* example-old*
includepkgs=httpd* mod_ssl
```

排除可以出现在全局或仓库级配置。一个包“明明能在仓库服务器看到但 DNF 不显示”，应检查 `exclude`。`includepkgs` 则把仓库内容限制到匹配集合。

④ [操作] 使用 `--disableexcludes` 进行有意识的诊断覆盖

```
dnf --disableexcludes=<repo-id> repoquery <package>
```

该选项适合验证“是否确实被 `exclude` 隐藏”。它不是持久修复，也不应无条件用于安装；确认原因后，应修正错误的排除策略或按题目要求保留策略。

⑤ [知识点] 命令级 repo 范围可以比持久配置更窄

`--disablerepo='*'` `--enablerepo=...` 会改变当前命令的仓库集合。若遗漏 BaseOS 或依赖仓库，可能出现“目标包存在但依赖不可解”。因此限定来源时必须把“目标包仓库”和“依赖仓库”一起考虑。

⑥ [诊断] 优先级、排除和模块过滤要分别取证

```
repoquery --repo <id> 能看到包吗？
→ 普通 repoquery 能看到吗？
→ --disableexcludes 后是否出现？
→ module list/provides 是否显示过滤关系？
→ 事务摘要最终选择哪个 repoid？
```

每一步只改变一个变量，避免同时启用所有仓库、禁用排除和 reset 模块，导致无法解释真正原因。

[Cheatsheet] priority 小者优先，cost 用于同层来源偏好；exclude/include 改变候选；临时 `--disableexcludes` 只用于证明原因；限定仓库时别遗漏依赖仓库；与 module filtering 分开调查。

[操作专题] 从 RPM 目录或安装介质建立本地仓库

本地仓库常用于无外网实验、受控内容分发或离线安装。它的本质仍是“客户端能通过仓库协议读取元数据和包”，而不是“文件就在磁盘上”。先判断目录是否已有 `repodata`，再决定直接配置还是使用 `createrepo_c`。

① [知识点] 安装介质与普通 RPM 目录不同

RHEL 安装介质中的 BaseOS/AppStream 通常已经带有 `repodata`，挂载后可直接作为 `file://` 仓库来源。普通收集目录只有 RPM 时，需要生成元数据。

```
find /mnt/rhel9 -maxdepth 3 -name repomd.xml -print
```

看到 `repodata/repomd.xml` 才说明目录具备普通仓库入口。

② [操作] 为普通 RPM 目录生成元数据

```
mkdir -p /srv/repo/custom
cp /path/to/packages/*.rpm /srv/repo/custom/
createrepo_c /srv/repo/custom
```

作用对象：目标目录中的 RPM 集合。基本语义：扫描软件包并生成 `repodata`。验证：

```
test -f /srv/repo/custom/repodata/repomd.xml
```

命令成功和文件存在只能证明普通元数据入口生成，不能证明每个 RPM 签名、依赖或高级元数据都满足任务。

③ [操作] 内容变化后更新仓库元数据

```
cp new-package.rpm /srv/repo/custom/
createrepo_c --update /srv/repo/custom
```

删除或替换 RPM 后也要重新生成或更新元数据。客户端随后使用：

```
dnf clean metadata
dnf makecache --refresh
```

只在确有旧元数据问题时清理相关缓存，不机械执行 `clean all`。

④ [操作] 配置 file:// 本地仓库

```
[local-custom]
name=Local Custom Repository
baseurl=file:///srv/repo/custom
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-example
```

`file:///` 中三个斜杠表示本机绝对路径。DNF 进程需要对路径和文件具有遍历与读取权限。SELinux 或挂载状态问题若影响读取，应转交相应章节，不以 `chmod 777` 作为默认答案。

⑤ [边界] 本地文件不等于可信文件

内容位于本机磁盘并不自动证明来源可信。若题目要求 GPG 校验，应提供与包签名匹配的 key；自己制作的未签名包仓库需要明确的信任与签名流程，本章不以关闭校验来回避。

⑥ [边界] 普通 createrepo_c 不自动重建模块语义

模块仓库需要模块元数据。仅把模块 RPM 拷贝到目录并运行普通 createrepo_c，可能得到可读取的普通包元数据，却失去 stream/profile 和 module filtering 语义。复制或同步模块内容时应保留/注入相应模块元数据；该高级制作流程只建立边界，不作为 RHCSA 必做操作。

⑦ [验证] 从目录推进到客户端候选

```
test -f /srv/repo/custom/repodata/repomd.xml
dnf repolist --all
dnf makecache --refresh
dnf repoquery --repo local-custom <representative-package>
```

若还需要包组或模块，应额外执行 `dnf group list` 或 `dnf module list` 验证对应元数据，不用普通包查询替代。

[Cheatsheet] 先找 `repomd.xml`；普通 RPM 目录用 `createrepo_c`；内容变化后 `--update`；客户端 `file:///` 绝对路径；本地仍需权限和 GPG；普通 `repodata` 不保证 `group/module` 元数据。

[诊断专题] 从症状定位配置、元数据、TLS、GPG、过滤或依赖层

仓库故障通常会在同一条 DNF 命令中连续暴露多个层次。诊断时不要一次修改多个开关，而应遵循：

```
症状
→ 当前证据
→ 假设
→ 下一条最有区分度的证据
→ 最小修复
→ 再验证
```

① [诊断] 仓库没有出现在 `repolist --all`

症状：新建 repo 文件后，目标 ID 完全不存在。当前证据：

```
ls -l /etc/yum.repos.d/
sed -n '1,160p' /etc/yum.repos.d/exam.repo
dnf repolist --all
```

假设：文件扩展名错误、INI 语法错误、section 缺失、ID 重复或文件不可读。下一条证据：读取 DNF 的完整错误与配置文件；必要时使用 `dnf config-manager --dump <id>`，前提是插件存在。最小修复：修正文件名、section 或语法，不删除其他仓库。再验证：`dnf repolist --all`。

② [诊断] `repomd.xml` 404

症状：DNF 报错 URL 以 `/repodata/repomd.xml` 结尾并返回 404。假设：`baseurl` 指向了仓库上层、下层或错误产品路径。最有区分度证据：对照题目 URL 与服务器目录结构；读取错误中的完整最终 URL。最小修复：让 `baseurl` 指向真正包含 `repodata` 的根。再验证：`dnf makecache --refresh`，随后 `repoquery --repo`。边界：`dnf clean all` 不会修复服务器上的 404。

③ [诊断] 名称解析或连接失败

症状：`could not resolve host`、`connection timed out`、`network unreachable`。证据链：

```
getent hosts content.example.com
ip route get <resolved-ip>
curl -I http://content.example.com/path/
```

网络配置和 DNS 主归属第 18、19 章。本章只要求把错误定位到解析、路由、代理或服务端连接层，并保留 repo 配置中已经正确的部分。

④ [诊断] HTTPS 证书验证失败

症状：`certificate verify failed`、`unknown CA`、`certificate not yet valid/expired`。假设：系统时间错误、CA 不受信任、服务器证书链不完整或访问了错误主机名。证据：`timedatectl`、完整证书错误、系统 CA 配置和 URL 主机名。最小修复：修正时间、URL 或 CA 信任。禁止默认方案：`sslverify=0`。

⑤ [诊断] GPG key 或包签名失败

症状：`public key not installed`、`GPG check FAILED`、签名与 key 不匹配。当前证据：`repo` 的 `gpgcheck/gpgkey`、key 文件是否存在、完整事务错误。假设：key 路径错误、仓库包由另一把 key 签名、内容损坏或来源混用。最小修复：使用题目或仓库官方提供的正确 key，修正来源；必要时清除失败下载后重试。再验证：保持 `gpgcheck=1` 重新执行事务。边界：导入了某把 key 不代表它一定匹配当前仓库。

⑥ [诊断] 元数据校验失败或缓存异常

症状：checksum mismatch、repomd 与数据不一致、使用旧候选。假设：镜像正在同步、代理缓存不一致、本地缓存对应旧仓库定义。证据：错误中的元数据文件和 checksum；repo URL 是否近期改变。最小修复：先确认服务器端一致，再按需：

```
dnf clean metadata
dnf makecache --refresh
```

不要把本地清缓存当成服务器元数据损坏的修复。

⑦ [诊断] 仓库可用但包没有匹配项

证据链：

```
dnf repoquery --repo <repoid> <package>
dnf repoquery --available <package>
dnf module provides '<package-or-path>'
dnf module list
grep -R '^[[:space:]]*\\(exclude\\|excludepkgs\\|includepkgs\\)' /etc/dnf /etc/yum.repos.d 2>/dev/null
```

依次区分：包名错误、仓库内容缺失、架构不符、exclude/include、模块过滤和缓存问题。

⑧ [诊断] 依赖冲突或包被替换

症状：conflicting requests、nothing provides、cannot install the best candidate。当前证据：完整 DNF 错误和 Transaction Summary。下一条证据：

```
dnf repoquery --requires <package>
dnf repoquery --whatprovides '<capability>'
dnf module list
dnf repolist --all
```

最小修复：启用缺失的正确仓库、纠正 stream、移除错误 exclude，或选择题目要求的合法候选。边界：`--allowerasing` 允许求解器删除冲突包，可能破坏其他任务；必须先理解它准备删除什么，不作为第一尝试。

⑨ [诊断] 本地仓库有 RPM 但 DNF 看不到

```
目录中是否有 repodata/repomd.xml
→ baseurl 是否使用正确的 file:/// 绝对路径
→ 目录和父目录是否可遍历
→ 元数据是否在添加 RPM 后更新
→ 客户端是否刷新缓存
→ 包架构、排除和模块规则是否允许候选
```

若 `repoquery` 能看到普通包但 `module list` 没有预期流，重点检查模块元数据是否随内容保留。

⑩ [诊断] 包组不存在或内容不完整

先确认启用仓库是否包含 group/comps 元数据：

```
dnf group list --hidden
dnf group info '<group>'
```

普通包查询成功不能证明组元数据存在。自建仓库若只运行 `createrepo_c` 而没有导入 comps，组列表为空可能是预期结果。

[Cheatsheet] repo 不见查文件和 ID；404 查仓库根；解析/连接错交给 DNS/网络层；TLS 查时间与 CA；GPG 查 key/来源；无包查 repo/架构/exclude/module；依赖错误读完整摘要，不默认 `allowerasing`；本地仓库查 repodata、权限和高级元数据。

[操作专题] 工作迁移：最小变更、审计与自动化接口

真实服务器往往已经存在订阅仓库、自定义镜像、历史模块状态和多次软件事务。工作迁移的重点是让每次变更可解释、可回顾，并为后续 Ansible 自动化提供稳定目标，而不是在手工阶段制造隐含状态。

① [操作] 变更前保存软件来源基线

```
dnf repolist --all
dnf module list --enabled
dnf module list --installed
dnf history | head -20
```


必要时备份将修改的单个 repo 文件，而不是打包全部缓存。基线应能回答：新增了哪个 repo、改变了哪个 stream、事务前后差异是什么。

② [操作] 只修改与目标相关的字段

题目要求新增两个仓库时，新建一个清晰的 .repo 文件通常比编辑系统所有仓库更安全。已有仓库若与任务无冲突，不删除、不重命名。临时限定来源用命令选项，持久状态仍由 repo 文件表达。

③ [验证] 保留来源和事务证据

一个可审计的交付至少记录：

repo ID 和来源 URL
元数据刷新结果
目标包来自哪个 repoid
事务 ID 和摘要
跨章的已安装与功能验证入口

不要复制包含凭据的私有 URL、代理密码或订阅 token 到公开报告。

④ [接口] 为 RHCE 自动化准备声明式目标

后续 RHCE 章节可能使用 ansible.builtin.yum_repository 或 ansible.builtin.dnf 表达相同目标。手工阶段应先明确：

仓库 ID、baseurl、enabled、gpgcheck、gpgkey
目标包或包组
模块 stream/profile
验证命令和预期状态

本章不展开 Playbook，但避免只留下“执行过某条命令”的过程描述。

⑤ [边界] 不把内部仓库访问凭据写入普通 repo 模板

若仓库需要用户名、密码、客户端证书或代理凭据，应按组织的凭据管理方案处理。考试通常提供无需秘密的固定 URL；真实工作中不应把密码直接提交到公共 Git 仓库或 Anki 卡片。

[Cheatsheet] 先保存 repo/module/history 基线；最小修改单个配置；记录来源、候选和事务；临时覆盖与持久配置分开；为自动化保留目标字段，不泄露凭据。

[经典任务] 配置带 GPG 校验的 BaseOS/AppStream 仓库并安装软件包

环境

系统为 RHEL 9。当前 /etc/yum.repos.d/ 中没有以下两个仓库。题目提供的内容服务器和 key 在考试环境中应当可用，但本章没有连接真实 VM。

项目	参数
BaseOS repo ID	exam-baseos
BaseOS URL	http://content.example.com/rhel9.0/x86_64/dvd/BaseOS
AppStream repo ID	exam-appstream
AppStream URL	http://content.example.com/rhel9.0/x86_64/dvd/AppStream
GPG key	file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release
目标包	httpd

当前状态

- 其他系统仓库可能存在，不允许无差别删除；
- httpd 是否已安装未知；
- 不假设任何模块流已被修改；
- 本任务只验收到软件包和 DNF 事务层，不要求配置或启动 Web 服务。

目标终态

- 两个仓库持久定义并默认启用；
- gpgcheck=1，使用题目给出的 key；
- 两个仓库元数据能够刷新；
- bash 可从 exam-baseos 查询；
- httpd 可从 exam-appstream 查询；

6. 安装 `httpd`，并能从 `history` 找到对应事务；
7. 不改变无关模块流和包组状态。

限制条件

- 不设置 `gpgcheck=0` 或 `sslverify=0`；
- 不使用 `rpm --nodeps`、`rpm --force`；
- 不默认使用 `--allowerase`；
- 不删除系统中其他有效仓库；
- 不把包安装成功写成 Web 服务已经可用。

验收矩阵

验收层	推荐证据
配置	读取 <code>/etc/yum.repos.d/exam.repo</code>
识别/启用	<code>dnf repolist --all</code>
元数据	<code>dnf makecache --refresh</code>
BaseOS 内容	<code>dnf repoquery --repo exam-baseos bash</code>
AppStream 内容	<code>dnf repoquery --repo exam-appstream httpd</code>
事务预览	<code>dnf install --assumeno httpd</code>
安装状态	<code>rpm -q httpd</code> （第 16 章接口）
审计	<code>dnf history</code> 、 <code>dnf history info <ID></code>

请先独立完成。参考解答从下一页开始。

[参考解答] 从 repo 文件推进到可审计事务

① 调查并保存基线

```
ls -l /etc/yum.repos.d/
dnf repolist --all
dnf module list --enabled
rpm -q httpd
```

`rpm -q` 只是确认初始已安装状态，完整 RPM 语义归第 16 章。不要因为目标包已经安装就跳过仓库验证：题目仍要求两个仓库定义和内容证据。

② 创建最小仓库配置

```
cat > /etc/yum.repos.d/exam.repo <<'EOF'
[exam-baseos]
name=Exam BaseOS
baseurl=http://content.example.com/rhel9.0/x86_64/dvd/BaseOS
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release

[exam-appstream]
name=Exam AppStream
baseurl=http://content.example.com/rhel9.0/x86_64/dvd/AppStream
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release
EOF
```

这里使用单引号 here-document，便于未来包含 DNF 变量时避免 Shell 提前展开。配置中没有凭据或无关字段。

③ 验证解析与元数据

```
dnf repolist --all
dnf makecache --refresh
```

若失败，不继续安装。先根据完整错误定位：

- 仓库 ID 不出现: repo 文件/section 问题;
- `repomd.xml` 404: URL 不是仓库根;
- 无法解析主机: DNS 层;
- 证书失败: 时间/CA/TLS 层;
- checksum 错误: 缓存或服务端同步层。

④ 限定仓库查询代表包

```
dnf repoquery --repo exam-baseos bash
dnf repoquery --repo exam-appstream httpd
```

这两条分别证明仓库提供代表内容。若 `httpd` 无结果, 继续检查 AppStream URL、架构、exclude 和 module filtering, 而不是关闭 GPG。

⑤ 预览并提交事务

```
dnf install --assumeno httpd
dnf install httpd
```

预览时确认没有意外删除、降级或跨来源替换。题目没有要求只使用这两个仓库, 因此不必主动禁用所有其他仓库; 若评分明确要求限定来源, 可使用:

```
dnf --disablerepo='*' \
  --enablerepo=exam-baseos,exam-appstream \
  install httpd
```

必须同时启用 BaseOS 与 AppStream, 以保留依赖解析范围。

⑥ 分层验证

```
rpm -q httpd
dnf history
# 根据实际列表选择本次事务 ID:
dnf history info <TRANSACTION_ID>
```

静态预期:

- `rpm -q` 应确认包在已安装数据库中;
- `history info` 应显示本次安装涉及的软件包和动作;
- 本任务到此只完成软件层, 不声称 `httpd.service` active/enabled 或 HTTP 可访问。

⑦ 典型错误

- 只执行 `repolist`, 没有实际刷新元数据;
- `baseurl` 指向 `.../dvd` 而不是 `.../dvd/BaseOS` 或 `AppStream`;
- 为消除签名报错设置 `gpgcheck=0`;
- 只启用 AppStream, 导致 BaseOS 依赖不可用;
- 未读事务摘要直接接受 `--allowerase`;
- 安装成功后扩大为“Web 服务已完成”。

[经典任务] 诊断 “仓库可见但包不可解析或签名失败”

环境和症状

系统存在仓库 `training-appstream`:

```
[training-appstream]
name=Training AppStream
baseurl=https://repo.example.com/rhel9/AppStream/Packages
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-training
```

已知:

```
dnf repolist --all 能看到 training-appstream 为 enabled
但 dnf makecache --refresh 下载 repodata/repomd.xml 时返回 404
```

修复 URL 后，元数据刷新成功；执行 `dnf install example-server` 又报告 GPG key 不匹配。题目要求保持 TLS 和 GPG 校验。

目标终态

1. 找到真正的仓库根并修正 `baseurl`；
2. 元数据刷新成功；
3. `example-server` 能从该仓库查询；
4. 使用与仓库包签名匹配的 key，保持 `gpgcheck=1`；
5. 安装前读事务摘要；
6. 形成每次判断的证据，不通过关闭保护绕过问题。

限制条件

- 不设置 `sslverify=0`；
- 不设置 `gpgcheck=0`；
- 不把 `dnf clean all` 当成错误 URL 的修复；
- 不默认使用 `--allowerasing`；
- 不修改无关仓库。

验收证据

```
完整失败 URL
修正后的 repo 文件
makecache --refresh
repoquery --repo training-appstream example-server
key 文件和 gpgkey 对应关系
保持 gpgcheck=1 的成功事务或明确的下一步证据
```

请先独立完成。参考解答从下一页开始。

[参考解答] 用最有区分度的证据逐层修复

① 读取完整错误 URL

错误 URL 若为：

```
https://repo.example.com/rhel9/AppStream/Packages/repodata/repomd.xml
```

而服务端的 `repodata` 实际位于 `.../AppStream/repodata/`，则 `Packages` 是错误的下层目录。最小修复是：

```
baseurl=https://repo.example.com/rhel9/AppStream
```

不要先清缓存，因为 404 已经证明请求抵达服务器，只是路径不存在。

② 再验证元数据和候选

```
dnf makecache --refresh
dnf repoquery --repo training-appstream example-server
```

若元数据成功但包无结果，按以下顺序检查：

```
包名与架构
→ 该 repo 是否实际包含包
→ exclude/include
→ module filtering
```

可以补充：

```
dnf module provides 'example-server'
dnf module list
```

③ 调查 GPG 失败

```
grep -E '^(gpgcheck|gpgkey)= ' /etc/yum.repos.d/training.repo
ls -l /etc/pki/rpm-gpg/RPM-GPG-KEY-training
timedatectl
```

读取完整事务错误，确认是“key 尚未导入”“签名者与 key 不匹配”还是包本身损坏。系统时间虽然更常影响 TLS，但明显错误时间也会破坏整个信任调查，因此只作为证据核对，不用来替代签名者检查。

④ 最小修复 key 或仓库来源

使用题目/仓库发布者提供的正确公钥，修正 `gpgkey=`；如果包来自错误镜像，应修正仓库而不是导入未知 key。保持：

```
gpgcheck=1
```

然后预览事务：

```
dnf install --assumeno example-server
```

确认来源仓库、安装和删除集合合理后再提交。

⑤ 再验证并记录证据

```
dnf install example-server
rpm -q example-server
dnf history
dnf history info <TRANSACTION_ID>
```

本章未在真实 RHEL 9 VM 执行，因此不提供虚构输出。实际环境若仍失败，应保留完整错误并继续在当前层调查，而不是打开绕过选项。

⑥ 典型错误分支

错误做法	为什么不成立
<code>dnf clean all</code> 后反复重试 404	删除本地缓存不能创建服务器端不存在的路径
<code>sslverify=0</code>	404 已经不是证书问题；关闭 TLS 验证还移除安全属性
<code>gpgcheck=0</code>	掩盖 key/来源错误，不满足题目终态
导入任意搜索到的 key	无法证明该 key 是仓库发布者的受信任 key
<code>--allowerasing</code>	GPG 错误不是依赖删除问题

[本章收束] 从软件来源推进到可审计的事务终态

DNF 的核心不是一组安装命令，而是一条来源和解析链。`repo` 文件决定默认可见的内容来源和信任策略；元数据决定客户端能够认识哪些包、模块和组；优先级、排除和模块状态进一步塑造候选；求解器把候选组合成事务；`history` 和 `RPM` 层证据再说明实际发生了什么。

以后面对软件问题，应先问：

```
我正在证明配置、元数据、候选、事务，还是最终功能？
```

仓库不见时查定义；`repomd.xml` 错时查仓库根和传输；包不见时查仓库范围、架构、排除和模块；GPG 失败时查 key 与来源；依赖冲突时读完整摘要和依赖，不默认删除现有包。只有每一层都由相应证据支撑，软件来源和事务才真正可解释、可复核、可迁移。

工作方法：先判定失败层，再选择最小动作

面对新的 DNF 题目或真实故障，优先把现象改写成一个可判定问题：是客户端没有加载仓库定义，还是元数据无法取得；是候选包被过滤，还是依赖求解失败；是包签名不可信，还是事务完成后还缺少服务配置。每次只改变能够解释当前证据的最小对象，并在同一层重新验证。

先读完整错误
→ 标记失败层
→ 选择最有区分度的证据
→ 做最小修复
→ 在原层复验
→ 再进入下一层验收

主要判断表

看到的证据	可以确认	仍不能确认	下一步
repo ID 出现在 <code>dnf repolist --all</code>	配置被 DNF 识别，启用状态可见	URL 与元数据可用	<code>dnf makecache --refresh</code>
<code>makecache --refresh</code> 成功	启用仓库的元数据可取得并通过当前校验	目标包一定可见	限定仓库执行 <code>repoquery</code>
<code>repoquery</code> 返回目标候选	当前元数据和过滤规则允许看到该构建	依赖一定可解、事务已经提交	<code>dnf install --assumeno</code>
Transaction Summary 合理	求解器生成了预期事务计划	事务已经发生	明确确认后提交
<code>dnf history info</code> 记录成功事务	DNF 事务层发生了可审计变化	应用配置和服务功能正确	转交对应配置、systemd 和功能验证
模块 stream 已启用	候选版本线发生改变	profile 已安装	查询模块状态与已安装 profile
包组安装完成	一组软件包按组元数据完成事务	组内服务已经配置并运行	转交相应服务章节

本章离场检查

进入下一章前，应能够在不依赖“反复试命令”的情况下完成以下判断：

1. 看到一个 repo 文件时，能够区分文件名、section ID、显示名和内容 URL；
2. 能够说明 `repolist`、`makecache --refresh`、`repoquery` 和事务摘要分别证明哪一层；
3. 遇到“仓库有包但客户端找不到”时，能够继续检查架构、exclude、priority 和 module filtering；
4. 能够区分 TLS 连接问题、GPG key 问题和包签名问题，而不是关闭验证；
5. 能够区分模块 stream、profile、包组和普通包事务；
6. 能够从 DNF history 交接到 RPM、systemd、网络和功能层验证。

配置可见 → 元数据可用 → 候选可解析 → 来源可信 → 事务成功 → 功能交接

向下一章交接

本章结束在“软件来源、候选集合和事务证据”这一层。第 18 章《NetworkManager、IPv4 地址与路由》将完整处理网络连接、地址和路由；第 19 章《主机名、NSS 与 DNS 名称解析》将完整处理名称解析。DNF 诊断中出现 DNS、路由或连接错误时，本章只负责识别错误层并保留证据，不在此重复展开网络配置。RPM 已安装数据库、文件归属和校验继续由第 16 章负责。

NetworkManager、 IPv4 地址与路由

从持久连接配置到内核选路：把 device、profile、地址、路由和验证放进同一条证据链。

[对象模型](#) [操作语义](#) [验证](#) [诊断](#) [经典任务](#)

[大字号阅读版](#)

本章阅读导航

先抓住一条主线：NetworkManager 保存的是连接配置意图，内核承担的是当前地址与选路。配置、激活和当前状态必须分别取证，不能看到一条成功输出就把整个网络终态判为正确。

01识别设备确认 device、接口名、链路与管理状态

02定位 profile区分 NAME、UUID、绑定设备和活动实例

03读取持久属性地址、网关、路由、DNS 与 autoconnect

04激活配置明确何时只是保存，何时真正应用到 device

05核对内核状态使用 `ip address` 与 `ip route` 读取当前事实

06验证具体目的用 `ip route get` 证明 via、dev 与 src

专题地图 - **知识专题** 从设备到内核：NetworkManager 管理的对象 - **知识专题** 从地址前缀到目的路径：IPv4 与路由 - **操作专题** 修改前建立 device/profile/当前状态基线 - **操作专题** 修改现有 profile 为静态 IPv4 - **操作专题** 创建新 profile 与管理 autoconnect - **操作专题** 默认网关、静态路由与 DNS 属性 - **操作专题** keyfile、reload 与重新激活边界 - **操作专题** 从配置到功能的分层验证 - **诊断专题** 链路、地址、路由、DNS 与监听分层定位 - **经典任务** DHCP profile 改静态；修复目的选路

阅读时持续回答 1. 我现在操作的是 device，还是 connection profile？ 2. profile 名称、UUID 与接口名分别是什么？ 3. 看到的是持久配置，还是活动连接的当前状态？ 4. 修改后是否已经激活到目标 device？ 5. 地址的前缀是否产生了正确直连网络？ 6. 默认路由和静态路由的下一跳是否可直连？ 7. 对具体目的，`via`、`dev` 与 `src` 是什么？ 8. 当前证据能证明什么，又不能证明什么？

章节边界

主机名、NSS 与完整 DNS 解析留给第 19 章；SSH 留给第 20 章；firewalld 与完整服务访问链留给第 21 章。

第 18 章 · 正文

一台主机“有网卡”并不等于“已经有可用网络”。一次 IPv4 通信至少经过四个状态层：系统先识别网络设备，NetworkManager 再选择并激活一个 connection profile，随后把地址和路由写入内核，最后内核针对具体目的决定出口、源地址和下一跳。任何一层错位，都可能让“配置看起来正确”与“实际路径正确”同时出现矛盾。

最常见的误判有三类。第一，修改 profile 后没有重新激活，却把保存成功当成当前状态已经变化；第二，接口确实有地址，但前缀、默认网关或静态路由让目标走错出口；第三，ping 一次成功就宣布 DNS、TCP 监听、firewalld 和应用全部正确。解决这些问题的关键不是背更多命令，而是知道每条命令观察或改变哪个对象。

本章沿着下面的主线推进：

- 识别 device 与活动 profile
 - 读取持久配置并保存基线
 - 修改地址、网关、路由或 DNS 属性
 - 激活目标 profile
 - 读取内核地址和路由
 - 对具体目的执行选路验证
 - 进行受限的功能验证

核心概念

概念

网络设备（device）是内核能够发送或接收网络流量的物理或虚拟接口，例如 `enp1s0`。它具有接口名、类型、MAC、链路和管理状态，但它本身不等于一套持久 IP 配置。观察 device 时，重点回答“系统是否看见它、NetworkManager 是否管理它、链路是否可用、当前哪个连接使用它”。

概念

连接配置（connection profile）是 NetworkManager 保存的一组配置意图，包含绑定设备、IPv4 方法、地址、网关、路由、DNS 和自动连接等属性。一个 device 可以保存多个 profile；profile 存在只证明配置对象存在，不证明它正在活动。名称、UUID 和接口名即使文字相同，也仍是不同身份维度。

概念

活动连接与当前内核状态 分属两个相关但不同的证据层。profile 激活后形成 active connection，NetworkManager 再把设置应用到 device；`ip address` 和 `ip route` 显示内核此刻真正采用的地址和路由。修改 profile 后，持久配置与当前状态可以暂时不一致，所以保存成功后仍要激活并复核。

概念

IPv4 地址与前缀 必须作为一个整体读取。地址标识本机，前缀决定本机认为什么是直连网络，并自然产生直连路由。接口上出现正确地址不代表前缀正确；前缀错误可能让网关看似存在，却无法作为正常下一跳被交付。

概念

默认路由 是当没有更具体路由时使用的兜底路径，通常由 `ipv4.gateway` 形成 `default via ...`。默认网关不是“能访问互联网”的保证；它首先必须能够通过当前连接直接到达，并且只在没有更长前缀匹配时才被选择。

概念

静态路由 为某个目的前缀指定下一跳、出口或 metric。它比默认路由更具体时会优先匹配。路由表中存在一条静态路由，只能证明条目被安装；对评分目标仍应使用 `ip route get`，确认该目的实际选中的 `via`、`dev` 和 `src`。

概念

DNS 属性 是 connection profile 向名称解析链提供的服务器和搜索域输入。本章负责设置 `ipv4.dns`、`ipv4.dns-search` 并观察当前连接是否采用；数字 IP 可达而名称失败时，完整的主机名、NSS、解析器和 DNS 记录调查交给第 19 章。

操作语义速查区

下面的命令组只建立接口地图。正文专题会继续解释对象选择、参数组合、验证层次和风险边界，不要求每个小操作重复整份手册页。

nmcli device

SYNOPSIS

```
nmcli device { status | show [IFACE] | connect IFACE | disconnect IFACE }
```

以 device 为中心查看 NetworkManager 状态，或控制设备是否允许连接。

重要参数 / 形式

nmcli device status

列出 device、类型、状态和当前 connection，适合作为第一条基线证据。

nmcli device show enp1s0

显示设备的活动连接信息，包括当前 IP、网关和 DNS 输入。

nmcli device disconnect enp1s0

断开设备并阻止它立即自动重新连接；远程执行前必须有恢复通道。

nmcli connection show

SYNOPSIS

```
nmcli connection show [--active] [ID]
```

以 connection profile 为中心读取全部配置、活动映射或某个 profile 的详细属性。

重要参数 / 形式

nmcli connection show

列出所有 profile 的 NAME、UUID、TYPE 和 DEVICE。

--active

只显示当前活动连接；它不列出所有持久 profile。

-f <FIELDS>

只显示目标字段，便于把配置证据收敛到 ipv4.* 和 connection.*。

nmcli connection add / modify

SYNOPSIS

```
nmcli connection add type ethernet ifname IFACE con-name NAME [PROPERTY VALUE ...]
nmcli connection modify ID PROPERTY VALUE [PROPERTY VALUE ...]
```

创建新 profile，或修改现有 profile 的持久属性。

重要参数 / 形式

ipv4.method manual

使用静态 IPv4 目标；只写地址而不改 method，连接可能仍保留动态行为。

ipv4.addresses 192.0.2.10/24

地址与前缀作为一个值设置。

+ipv4.routes / -ipv4.routes

在多值属性中追加或删除指定路由；不带符号通常替换整个列表。

nmcli connection up / down

SYNOPSIS

```
nmcli connection up ID [ifname IFACE]
nmcli connection down ID
```

激活或停用一个 profile。up 是把持久配置应用到 device 的关键动作。

重要参数 / 形式

nmcli connection up office-lan

激活目标 profile；远程修改当前管理接口时可能立即中断会话。

```
nmcli connection down office-lan
```

停用活动连接，但 autoconnect 候选可能再次被自动激活。

```
connection.autoconnect yes|no
```

控制后续自动激活资格，不代表当前连接已经 up 或 down。

ip address

SYNOPSIS

```
ip [-br] address [show [dev IFACE]]
```

从内核角度读取当前接口地址、前缀、scope 和生命周期。

重要参数 / 形式

-br

使用紧凑输出建立全局基线。

```
show dev enp1s0
```

只查看一个 device，避免把其他接口地址混入判断。

ip address 的边界

它证明当前地址，不证明 profile 持久属性或重启后的结果。

ip route / ip route get

SYNOPSIS

```
ip route [show]
ip route get DESTINATION
```

读取当前路由表，并计算一个具体目的将使用的出口、下一跳和源地址。

重要参数 / 形式

ip route

查看默认路由、直连路由、静态路由与 metric。

```
ip route get 198.51.100.44
```

验证该目的实际选择的 **via**、**dev** 和 **src**，是本章最关键的选路证据。

```
ip route add ...
```

只改变当前内核状态，不能作为持久 NetworkManager 任务的最终答案。

ping / ss

SYNOPSIS

```
ping [-c COUNT] DESTINATION
ss [-lntup]
```

执行有限的 ICMP 连通测试，或查看本机 socket 监听状态。

重要参数 / 形式

```
ping -c 3 192.0.2.1
```

若对端回应，可证明这次 ICMP 往返；失败可能来自对端禁用 ICMP。

```
ss -lntup
```

证明本机是否存在 TCP/UDP 监听，不证明远端流量能穿过 firewalld 到达。

[知识专题] 从设备到内核：NetworkManager 管理的是哪几个对象

最顺的切入方法不是先背命令，而是先回答四个问题：系统看见了哪个 device，保存了哪些 profile，当前激活的是哪个 profile，内核最终收到了什么地址和路由。NetworkManager 把“配置意图”与“当前运行状态”连接起来，但两者并非同一个对象。

① [知识点] device 是流量接口，profile 是可持久配置对象

设备可能是以太网接口、无线接口、桥、bond、VLAN 或其他虚拟接口。本章只以普通以太网 device 为主。device 具有接口名、类型、MAC 地址、carrier 和管理状态；profile 则具有连接名称、UUID、接口绑定、地址、路由、DNS 和 autoconnect 等属性。

```
设备 enp1s0
├─ profile office-dhcp
├─ profile office-static
└─ profile maintenance
```

同一个 device 可以保存多个 profile，但普通场景同一时刻只会会有一个 profile 作为主要活动连接。创建第二个 profile 不会自动删除第一个，也不代表第二个已经被激活。

② [知识点] NAME、UUID 与 DEVICE 是三个不同标识

`nmcli connection show` 的 `NAME` 是 `connection.id`，便于人理解；`UUID` 是稳定的 profile 标识；`DEVICE` 表示该 profile 当前激活到哪个设备，没有激活时通常为空白。接口名属于 device，例如 `enp1s0`。管理员可以把 profile 也命名为 `enp1s0`，但这只是名称相同，不是对象相同。

在脚本或有同名 profile 的环境中，`UUID` 更不易歧义；在考试和手工操作中，先用 `nmcli connection show` 确认唯一名称通常足够。

③ [知识点] active connection 是 profile 的运行实例

持久 profile 被激活后，NetworkManager 把其中的设置应用到 device，并建立 active connection。此时：

- `nmcli connection show --active` 显示活动 profile；
- `nmcli device status` 显示 device 与活动 connection 的映射；
- `nmcli connection show <CON>` 同时可看到小写的 profile 设置和活动连接的 `GENERAL`、`IP4` 等运行信息；
- `ip address` 与 `ip route` 从内核角度验证最终结果。

不要把“profile 存在”扩大解释为“profile 已激活”，也不要把“profile 已修改”扩大解释为“内核已立即采用全部修改”。

④ [知识点] 当前状态、持久状态和功能状态必须分开

维度	回答的问题	典型证据
持久 profile	重连或启动时准备应用什么	<code>nmcli connection show <CON></code>
激活映射	当前哪个 profile 用在哪个 device	<code>nmcli connection show --active</code>
当前内核状态	现在有哪些地址和路由	<code>ip -br address</code> 、 <code>ip route</code>
目的路径	到某个目标实际怎样选路	<code>ip route get <DEST></code>
局部功能	对端是否回应 ICMP、本机是否监听	<code>ping</code> 、 <code>ss</code>

一条证据只能证明它所处的层次。`nmcli` 命令返回成功不能证明远端可达；`ping` 成功不能证明 TCP 服务、DNS 解析或应用层正确。

⑤ [知识点] managed、connected 与 carrier 不是同一个状态

NetworkManager 可能看见一个 device，却不负责管理它；也可能管理 device，但当前没有活动连接；还可能 profile 已激活，而物理链路没有 carrier。诊断时应区分：

```
设备是否存在
→ NetworkManager 是否管理
→ 链路是否有 carrier
→ 是否有活动 profile
→ 是否获得或配置地址
```

`nmcli device status` 适合快速看 `STATE` 与 `CONNECTION`；`ip -br link` 适合从内核角度查看接口是否 `UP` 以及是否有 `LOWER_UP`。

⑥ [知识点] NetworkManager 服务是配置管理者，不是数据平面

NetworkManager 决定何时把 profile 应用到设备，并维护连接状态；真正的数据包转发、地址和路由匹配由内核网络栈完成。因此排错时既要看看 NetworkManager 的对象，也要看看内核状态。只看一侧容易遗漏“配置正确但未应用”或“当前状态被其他来源改变”等差异。

[Cheatsheet] device 是接口，profile 是配置，active connection 是激活实例；`NAME`、`UUID`、接口名分开识别；profile 证持久意图，`ip` 证当前内核状态。

[知识专题] 从地址前缀到目的路径：IPv4 和路由如何共同决定通信

IPv4 配置不是“地址、掩码、网关”三个互不相关的输入框。地址和前缀先形成直连网络，内核据此判断哪些目标可直接交付；不在直连网络中的目标再匹配路由表。排错时最有区分度的问题不是“有没有默认网关”，而是“针对这个具体目的，内核选择了哪条路由”。

① [知识点] 地址必须连同前缀长度读取

`192.0.2.10/24` 中，`192.0.2.10` 是主机地址，`/24` 表示前 24 位属于网络前缀。该配置通常产生到 `192.0.2.0/24` 的直连路由。若把前缀误写成 `/32`，本机可能不再把网关视为同一链路上的普通邻居；若误写成过大的网络范围，本机可能尝试直接解析本应交给路由器的目标。

地址存在只能证明接口上配置了该地址，不能证明前缀正确。

② [知识点] 直连路由由地址和前缀自然产生

接口获得 `192.0.2.10/24` 后，内核通常出现类似路由：

```
192.0.2.0/24 dev enp1s0 proto kernel scope link src 192.0.2.10
```

它表示目的位于该前缀时直接通过 `enp1s0` 发送，默认使用 `192.0.2.10` 作为源地址。普通考试任务不需要手工再添加同一条直连路由。

③ [知识点] 默认网关最终表现为默认路由

`ipv4.gateway 192.0.2.1` 在 profile 激活后通常形成：

```
default via 192.0.2.1 dev enp1s0
```

默认路由由匹配没有更具体路由可用的 IPv4 目的。网关地址原则上应能通过该连接直接到达；如果网关不在直连网络中，普通配置通常无法完成邻居解析和下一跳交付。

④ [知识点] 静态路由为特定目的指定更合适的下一跳

例如目标网络 `198.51.100.0/24` 应经过 `192.0.2.254`，可在 profile 中保存一条静态路由。其逻辑是：

```
目的前缀 198.51.100.0/24
→ 下一跳 192.0.2.254
→ 下一跳可由 enp1s0 的直连网络到达
```

静态路由的目的前缀应描述一组目标，而不是随意填入某个主机地址；若只需要单个主机路由，可使用 `/32`。

⑤ [知识点] 路由选择优先采用更长的前缀

若同时存在：

```
default via 192.0.2.1
198.51.100.0/24 via 192.0.2.254
```

访问 `198.51.100.44` 时，`/24` 比默认路由 `/0` 更具体，因此静态路由优先。只有在相同目的前缀存在多个候选时，`metric` 等属性才参与进一步选择。本章不展开策略路由和多路由表。

⑥ [知识点] metric 是候选路由的成本，不是带宽测速结果

较低 `metric` 通常代表更优先的候选。`ipv4.route-metric` 可影响由连接产生的路由，包括默认路由。多网卡主机若产生多条默认路由，应明确哪条是主路径，并避免无意识地让多条同协议路由使用相同 `metric`。

`metric` 只参与选路，不证明链路质量、延迟或带宽。

⑦ [知识点] ip route get 回答具体目的的真实选择

```
ip route get 198.51.100.44
```

典型字段：

- `via`：下一跳；
- `dev`：出口设备；
- `src`：建议源地址；
- `uid`：执行查询的用户上下文，部分系统会显示；
- `cache`：可能出现的缓存标记，不是持久配置。

`ip route` 适合看全表，`ip route get` 适合验证一个具体目的。路由表中“存在正确条目”与“该目的实际匹配正确条目”是两层证据。

[Cheatsheet] 地址加前缀产生直连路由；网关产生默认路由；更长前缀优先；`metric` 处理同类候选；具体目的用 `ip route get` 看 `via`、`dev`、`src`。

[操作专题] 修改前建立基线：把 device、profile 和当前状态对应起来

网络变更最危险的不是命令不会写，而是改错对象或失去原始状态。开始前先形成一份最小基线：设备是谁，活动 profile 是谁，当前地址和路由是什么，DNS 输入来自哪里。这样才能判断修改后的差异，也能在远程会话中评估回退路径。

① [操作] 查看 device 状态与活动 connection

作用对象：NetworkManager 管理的设备及其当前连接映射。基本形式：

```
nmcli device status
nmcli connection show --active
```

第一条以 device 为中心，第二条以活动 profile 为中心。重点读取 `DEVICE`、`TYPE`、`STATE`、`CONNECTION`、`NAME` 和 `UUID`。

边界：`connected` 表示 NetworkManager 认为连接已经激活，不等于所有业务目标都可达。

② [操作] 列出全部 profile 并确认准确目标

```
nmcli connection show
nmcli -f connection.id,connection.uuid,connection.interface-name,connection.type,connection.autoconnect connection show office-lan
```

先确认活动连接的 `NAME` 或 `UUID`，再修改。不要凭接口名猜 profile 名称，也不要因为看见默认生成的 `Wired connection 1` 就先删除它。

③ [操作] 从内核角度查看 link 和地址

```
ip -br link
ip -br address
ip -br address show dev enp1s0
```

`-br` 适合快速基线；需要地址生命周期、scope 等细节时使用：

```
ip address show dev enp1s0
```

④ [操作] 查看路由表和具体目的路径

```
ip route
ip route get 198.51.100.44
```

先看默认路由、直连路由和静态路由，再用具体目的验证实际匹配。记录 `via`、`dev` 和 `src`。

⑤ [操作] 查看当前连接提供的 DNS 属性

```
nmcli -f IP4.DNS,IP4.DOMAIN device show enp1s0
nmcli -f ipv4.dns,ipv4.dns-search connection show office-lan
```

第一条偏当前连接，第二条偏持久 profile。名称解析完整链路属于下一章，本章只验证 NetworkManager profile 和当前连接贡献的 DNS server/search。

⑥ [操作] 保存可比较的基线

在考试中可直接阅读输出；在工作变更中建议保存：

```
nmcli device status
nmcli connection show --active
nmcli connection show office-lan
ip -br address
ip route
ip route get 198.51.100.44
```

不要把含密码、私钥或企业敏感地址的完整输出随意外发。基线的价值在于比较对象和状态，不是收集越多越好。

[Cheatsheet] 先用 `nmcli device status` 找映射，再用 `nmcli connection show` 找 profile；用 `ip -br address`、`ip route`、`ip route get` 固化当前内核基线。

[操作专题] 把现有连接修改为持久静态 IPv4 配置

如果设备只连接一个固定网络，修改 NetworkManager 自动创建或现有活动 profile 通常比“删除后重建”更安全：可以保留 profile 身份、已有 IPv6 设置、MTU、MAC 绑定和其他未要求改变的属性。核心是确认准确对象、一次写入完整 IPv4 终态，然后重新激活并逐层验证。

① [操作] 确认活动 profile，而不是按接口名猜测

```
nmcli device status
nmcli connection show --active
```

假设确认活动 profile 为 `office-lan`，设备为 `enp1s0`，后续所有 `connection modify` 均应作用于 `office-lan`。

② [操作] 把 IPv4 方法改为 manual 并设置地址

```
nmcli connection modify office-lan \
  ipv4.method manual \
  ipv4.addresses 192.0.2.10/24
```

从 DHCP 转为静态配置时，`ipv4.method manual` 是关键状态。只设置 `ipv4.addresses` 而保留 `auto`，可能继续获得动态属性或造成不符合目标的混合状态。

③ [操作] 设置默认网关

```
nmcli connection modify office-lan ipv4.gateway 192.0.2.1
```

网关应能够从 `192.0.2.10/24` 的直连网络到达。激活后验证默认路由，而不是只读回 profile 字段。

若该连接明确不应提供默认路由，可使用：

```
nmcli connection modify office-lan ipv4.never-default yes
```

这不是普通静态地址任务的默认设置，只在多连接设计明确要求时使用。

④ [操作] 设置 DNS server 和搜索域

```
nmcli connection modify office-lan \
  ipv4.dns "192.0.2.53 192.0.2.54" \
  ipv4.dns-search example.test
```

`ipv4.dns` 和 `ipv4.dns-search` 都是多值属性。命令中的空格列表应整体引用，避免 Shell 把后续值当成新的参数。

⑤ [操作] 理解替换、追加和删除多值属性

```
# 替换整个 DNS 列表
nmcli con mod office-lan ipv4.dns "192.0.2.53 192.0.2.54"

# 追加一个 DNS server
nmcli con mod office-lan +ipv4.dns 192.0.2.55

# 删除指定 DNS server
nmcli con mod office-lan -ipv4.dns 192.0.2.55
```

不带 `+` 或 `-` 通常替换整个属性。先明确目标是“最终只有这些值”还是“在现有列表上增加一项”。

⑥ [操作] 设置 autoconnect，但不把它误当成当前激活

```
nmcli connection modify office-lan connection.autoconnect yes
```

`autoconnect yes` 表示设备满足条件时允许 NetworkManager 自动激活该 profile；它不证明 profile 此刻已经活动，也不证明重启后一定没有其他候选 profile 抢先被选择。

⑦ [操作] 激活修改后的 profile

```
nmcli connection up office-lan
```

这一步可能改变当前地址、默认路由和远程会话路径。通过 SSH 修改远端主机时，优先使用控制台、带外管理或确认过的回退路径。不要在没有调查的情况下先 `connection delete` 当前唯一可用 profile。

⑧ [操作] 使用一条完整命令表达目标终态

```
nmcli connection modify office-lan \
  ipv4.method manual \
  ipv4.addresses 192.0.2.10/24 \
  ipv4.gateway 192.0.2.1 \
  ipv4.dns "192.0.2.53 192.0.2.54" \
  ipv4.dns-search example.test \
  connection.autoconnect yes

nmcli connection up office-lan
```

这不是“执行后就完成”。下一专题必须分别验证 profile、活动映射、当前地址、路由、目的路径和必要功能。

[Cheatsheet] 修改现有 profile: `method manual` + 地址/前缀 + gateway + DNS/search + autoconnect; 多值属性不带前缀为替换; `con up` 后再验证当前状态。

[操作专题] 创建新 profile，并管理多 profile 与 autoconnect

需要保留原配置作为回退，或同一设备会接入不同网络时，应创建独立 profile。多 profile 的关键不在“会创建”，而在明确绑定设备、激活对象和自动选择规则，避免重启后激活了意外的旧配置。

① [操作] 创建绑定指定 device 的以太网 profile

```
nmcli connection add \
  type ethernet \
  ifname enp1s0 \
  con-name office-static
```

`type` 说明连接类型，`ifname` 绑定设备，`con-name` 设置 profile 名称。创建成功只表示 profile 已保存，不代表它已经激活。

② [操作] 创建时直接写入完整静态 IPv4 目标

```
nmcli connection add \
  type ethernet \
  ifname enp1s0 \
  con-name office-static \
  ipv4.method manual \
  ipv4.addresses 192.0.2.10/24 \
  ipv4.gateway 192.0.2.1 \
  ipv4.dns "192.0.2.53 192.0.2.54" \
  ipv4.dns-search example.test \
  connection.autoconnect yes
```

创建后仍要读取 profile，确认命令没有因拼写或引用错误留下缺失属性。

③ [操作] 显式激活目标 profile

```
nmcli connection up office-static
```

一个 device 上已有其他活动 profile 时，激活新 profile 会切换当前连接。应立即核对：

```
nmcli device status
nmcli connection show --active
```

④ [知识点] autoconnect 只参与候选 profile 的自动选择

`connection.autoconnect yes` 使 profile 成为自动连接候选。存在多个候选时，可使用：

```
nmcli connection modify office-static connection.autoconnect-priority 100
nmcli connection modify office-dhcp connection.autoconnect-priority 0
```

数值更高的 profile 优先。该属性只在同一设备有多个可用候选时有意义；它不改变路由 metric，也不等于“当前强制切换到该 profile”。

⑤ [知识点] `connection.autoconnect-priority` 只在多个候选都可自动连接时参与选择

同一 device 上若存在多个 `connection.autoconnect yes` 的 profile，NetworkManager 需要从候选中选择一个。`connection.autoconnect-priority` 的数值越高，通常越优先；它不是链路带宽、路由 metric，也不会强制一个当前不可用的 profile 成功激活。

```
nmcli -f connection.id,connection.autoconnect,connection.autoconnect-priority \
connection show office-static
```

考试环境通常不必主动制造多个自动连接候选。已有多 profile 时，先确认题目要求的目标 profile，再决定是否需要调整优先级；不要用删除其他 profile 代替对象判断。

⑥ [操作] 区分停用 profile 与断开 device

```
nmcli connection down office-static
nmcli device disconnect enp1s0
```

前者停用指定活动 connection，但 device 仍可能自动激活其他候选 profile；后者把 device 置为断开状态，阻止其继续自动连接，直到手工重新连接。远程执行任何一种命令都可能立即断线。

恢复 device：

```
nmcli device connect enp1s0
```

⑦ [操作] 删除 profile 前先证明它不再需要

```
nmcli connection delete office-dhcp
```

删除会移除持久 profile，若它当前活动还会影响连接。考试题没有要求清理旧 profile 时，不要把删除当成默认步骤。先确认：

- 当前活动 profile 已正确；
- 重启后不会选择旧 profile；
- 旧 profile 不承担恢复或其他网络用途；
- 删除不会让远程主机失去唯一可用配置。

[Cheatsheet] 新建用 `type`、`ifname`、`con-name` 明确对象；创建不等于激活；`autoconnect priority` 选择候选，`route metric` 选择路由；删 profile 不是静态配置的必需步骤。

[操作专题] 配置默认网关、静态路由与 DNS 属性

地址解决“我是谁、哪些目标在本地链路”，路由解决“其他目标从哪里走”，DNS 属性解决“当前连接向系统提供哪些名称服务器和搜索域”。三者可以在同一个 profile 中配置，但验证方法不同。

① [操作] 读取 profile 的默认网关与路由属性

```
nmcli -f ipv4.gateway,ipv4.never-default,ipv4.route-metric,ipv4.routes \
connection show office-lan
```

不要只看 `ipv4.gateway`。若 `ipv4.never-default yes`，该连接可能不生成默认路由；若多连接产生默认路由，`metric` 会影响候选优先级。

② [操作] 向 profile 追加一条静态路由

```
nmcli connection modify office-lan \
+ipv4.routes "198.51.100.0/24 192.0.2.254"
```

其中 `198.51.100.0/24` 是目的前缀，`192.0.2.254` 是下一跳。下一跳应能通过该连接的直连网络到达。

③ [操作] 替换整个静态路由列表

```
nmcli connection modify office-lan \
ipv4.routes "198.51.100.0/24 192.0.2.254, 203.0.113.44/32 192.0.2.253"
```

不带 `+` 会把目标属性改为给定列表。已有生产路由较多时，先读取现状，避免无意清空其他条目。

④ [操作] 删除指定静态路由

```
nmcli connection modify office-lan \
-ipv4.routes "198.51.100.0/24 192.0.2.254"
```


删除时应与已存属性的目的、下一跳等内容匹配。删除后重新读取 `ipv4.routes`，再激活并检查当前路由表。

⑤ [操作] 为连接设置 route metric

```
nmcli connection modify office-lan ipv4.route-metric 100
```

多网卡或多默认路由环境中，可让主连接使用更低 metric，备用连接使用更高 metric。不要把多个同协议默认路由由无意识地设置成相同 metric。

⑥ [知识点] `ipv4.never-default` 与静态路由可以同时存在

备用或仅访问专用网络的连接可能设置：

```
nmcli connection modify backup-lan ipv4.never-default yes
nmcli connection modify backup-lan +ipv4.routes "203.0.113.0/24 192.0.2.254"
```

这表示该连接不成为默认出口，但仍可为特定目的提供路由。该模式属于多连接设计，不应在普通单网卡静态地址题中随意套用。

⑦ [知识点] `ignore-auto-dns` 和 `ignore-auto-routes` 主要约束动态来源

当 `ipv4.method auto` 时，DHCP 可能提供 DNS 和路由。若需要保留 DHCP 地址但忽略 DHCP DNS，可设置：

```
nmcli connection modify office-dhcp \
  ipv4.ignore-auto-dns yes \
  ipv4.dns "192.0.2.53 192.0.2.54"
```

类似地，`ipv4.ignore-auto-routes yes` 可忽略自动路由。若 profile 已改为纯 `manual`，这些属性通常不是完成静态地址任务的关键项。

⑧ [操作] 激活后分别验证路由和 DNS 输入

```
nmcli connection up office-lan
ip route
ip route get 198.51.100.44
nmcli -f IP4.DNS,IP4.DOMAIN device show enp1s0
```

看到静态路由条目后仍要验证具体目的。看到 DNS server 后只说明连接输入正确；名称记录、NSS 顺序和实际解析属于第 19 章。

[Cheatsheet] gateway 看默认路由，`routes` 看特定目的，`route-metric` 选候选；`never-default` 禁止该 profile 提供默认路由；DNS 属性只证明连接提供了哪些 server/search。

[操作专题] 理解 keyfile、重新加载与重新激活的边界

RHEL 9 默认把新 NetworkManager profile 保存为 keyfile。日常操作优先使用 `nmcli`，因为它既修改 NetworkManager 的配置对象，也负责持久保存。手工编辑文件只适合迁移、审计或明确需要批量生成配置的场景，并且必须理解“磁盘文件、NetworkManager 已加载对象、活动连接”是三个层次。

① [知识点] RHEL 9 新 profile 默认使用 keyfile

持久 profile 通常位于：

```
/etc/NetworkManager/system-connections/*.nmconnection
```

旧系统遗留的 `ifcfg` profile 仍可能被读取，但本章以 keyfile 为主，不把 `/etc/sysconfig/network-scripts/` 当作新配置的默认位置。

② [知识点] keyfile 权限属于安全边界

连接配置可能包含 Wi-Fi 密钥、VPN 密钥或其他敏感数据。keyfile 应由 root 所有，并限制为 root 可读写。常见要求：

```
chown root:root /etc/NetworkManager/system-connections/office-lan.nmconnection
chmod 600 /etc/NetworkManager/system-connections/office-lan.nmconnection
```

本章不建议为了“让文件生效”放宽权限。

③ [操作] 使用 `nmcli connection reload` 重新读取连接文件

手工新增或修改 keyfile 后：

```
nmcli connection reload
```

也可加载指定文件：

```
nmcli connection load \
/etc/NetworkManager/system-connections/office-lan.nmconnection
```

通过 `nmcli connection modify` 完成的修改已经直接通知 NetworkManager，通常不需要额外 reload。

④ [边界] reload 不等于把新设置应用到活动 device

`connection reload` 只让 NetworkManager 重新读取 profile。活动连接可能仍使用旧地址或旧路由，需要：

```
nmcli connection up office-lan
```

然后用 `ip` 验证当前状态。不要把“文件已加载”扩大为“网络已切换”。

⑤ [边界] general reload 与 connection reload 作用对象不同

- `nmcli connection reload`：重新读取连接 profile；
- `nmcli general reload`：重新加载 NetworkManager 的全局配置和相关状态，具体范围由参数决定。

修改 `.nmconnection` 文件时，首选前者；不要用重启整个 NetworkManager 服务替代精确的 profile 加载和激活。

⑥ [操作] 用 nmcli 反向检查文件编辑结果

```
nmcli connection show
nmcli -f connection.id,connection.uuid,connection.interface-name,ipv4.method,ipv4.addresses,ipv4.gateway,ipv4.routes \
connection show office-lan
```

若 profile 没出现，优先检查文件路径、后缀、所有者、权限和语法，而不是反复重启系统。

[Cheatsheet] 文件在磁盘、profile 已加载、profile 已激活是三层状态；手工改 keyfile 后 `con reload`，随后按需 `con up`；权限保持 `root:root 0600`。

[操作专题] 从配置到功能完成分层验证

网络题的验收不能只靠一条 `ping`，也不能只靠 `nmcli` 返回成功。一个稳定的最小证据链应从持久配置开始，逐步走到当前内核状态、目的路径和必要功能。前一层失败时，不要跳到后面随机修改。

① [验证] 核对持久 profile 属性

```
nmcli -f \
connection.id,connection.uuid,connection.interface-name,connection.autoconnect,connection.autoconnect-priority,\
ipv4.method,ipv4.addresses,ipv4.gateway,ipv4.routes,ipv4.route-metric,ipv4.dns,ipv4.dns-search \
connection show office-lan
```

重点确认目标值和未要求改变的边界，例如是否意外修改了 IPv6。

② [验证] 核对活动 profile 与 device 映射

```
nmcli device status
nmcli connection show --active
```

profile 正确但 DEVICE 为空，说明它没有活动；device 上活动的是另一 profile，则当前状态不会按目标 profile 工作。

③ [验证] 核对当前 IPv4 地址和前缀

```
ip -br address show dev enp1s0
ip address show dev enp1s0
```

检查目标地址、前缀、重复或遗留地址。只看到目标地址不代表路由和 DNS 正确。

④ [验证] 核对当前路由表

```
ip route
```

检查：

- 直连网络是否符合前缀；
- 默认路由是否存在且下一跳正确；
- 静态路由是否存在；
- 多条候选路由的 metric 是否符合设计。

⑤ [验证] 对评分目标执行具体选路查询

```
ip route get 198.51.100.44
```

检查 `via`、`dev`、`src`。这条命令不发送数据包，因此它证明内核的本地选路决定，不证明下一跳或远端一定可达。

⑥ [验证] 核对当前 DNS 输入

```
nmcli -f IP4.DNS,IP4.DOMAIN device show enp1s0
```

若数字 IP 路径正常但名称失败，应停止修改地址和路由，把问题交给第 19 章的主机名、NSS 与 DNS 名称解析链路。

⑦ [验证] 使用 `ping` 做受限的连通测试

```
ping -c 3 192.0.2.1
ping -c 3 198.51.100.44
```

先测直连下一跳，再测远端数字 IP。失败可能由目标不回应 ICMP、防火墙、回程路由或中间网络造成；成功也不能证明 TCP 端口和应用协议正确。

⑧ [验证] 使用 `ss` 判断本机监听，而不是推断远端可达

```
ss -lntup
ss -lnt sport = :22
```

`ss` 能证明本机是否有进程监听指定地址和端口。它不能证明 `firewalld` 已放行、远端路径可达或客户端认证成功；这些内容分别属于第 21 章和第 20 章。

⑨ [验证] 将重连或重启持久性单独列项

本会话没有 live VM，不声称执行了重启验证。在真实环境应在保留恢复入口的前提下验证：

```
连接重新激活后仍正确
→ device 重新连接后仍选择目标 profile
→ 系统重启后地址、路由、DNS 和 autoconnect 仍正确
```

[Cheatsheet] profile → active mapping → address → route table → route get → DNS input → limited function → reboot persistence。每层证据只证明自己的范围。

[诊断专题] 按链路定位：链路、地址、路由、DNS 还是监听

网络故障最容易被“从头重配一遍”掩盖。更稳定的方法是先把症状放到正确层次，再选择一条最能区分假设的证据。每一步只解决当前已经证明的问题，不用后续章节的工具替代本章调查。

症状先写清楚失败的是链路、地址、目的路径、名称还是端口

当前证据读取 device、active profile、address、route 和 route get

假设一次只保留少量可被下一条命令区分的原因

下一条证据优先选择能直接排除一半假设的命令

最小修复只改已证明错误的 profile 属性或活动状态

再验证重新检查持久配置、当前状态、目的路径和必要功能

网络故障最容易出现“反复改同一层”的无效操作。稳定的诊断顺序是：先判断设备和链路，再判断活动 profile 与地址，然后判断具体目的路由，最后进入 DNS、监听、防火墙或应用层。每一步选择最能区分假设的证据。

① [诊断] device 不存在或没有 carrier

症状： `nmcli device status` 没有目标接口，或 device 为 `unavailable/disconnected`。当前证据：

```
nmcli device status
ip -br link
```

假设：设备未被系统识别、接口名判断错误、虚拟网卡未连接、物理链路无 carrier。下一条证据：对照 MAC 地址、虚拟机配置或硬件链路；确认 `LOWER_UP`。

最小修复：连接正确设备或链路，不要先改 IPv4 profile。再验证：device 出现并具备可用链路后，继续检查活动 profile。

② [诊断] profile 正确，但当前仍是旧地址

症状： `nmcli connection show office-lan` 已显示新地址， `ip -br address` 仍是旧地址。假设： profile 尚未重新激活，或另一 profile 仍活动。区分证据：

```
nmcli connection show --active
nmcli device status
```

最小修复： 激活准确 profile：

```
nmcli connection up office-lan
```

再验证： 重新检查当前地址和路由，不要只看命令成功消息。

③ [诊断] 有地址，但直连网关不可达

症状： 目标地址存在， `ping` 网关失败。假设： 前缀错误、网关不在直连网络、链路/VLAN 错误、地址冲突、对端不回应 ICMP。下一条证据：

```
ip address show dev enp1s0
ip route get 192.0.2.1
ip neigh show dev enp1s0
```

本章只轻量引用邻居表。若选路认为网关应直连但邻居解析失败，应调查二层链路和对端，而不是继续添加默认路由。

④ [诊断] 有地址、有默认路由，但访问特定网络走错网关

症状： `ip route get 198.51.100.44` 显示走默认网关，而设计要求走专用下一跳。假设： 缺少更具体静态路由、路由未激活、目的前缀写错。区分证据：

```
nmcli -f ipv4.routes connection show office-lan
ip route
ip route get 198.51.100.44
```

最小修复： 向准确 profile 添加正确目的前缀和可直达下一跳，激活后再次执行 `route get`。

⑤ [诊断] 多条默认路由导致出口不符合预期

症状： 多网卡主机有两条 default，实际流量从备用接口离开。假设： route metric 设计错误，或不应提供默认路由的 profile 未设置 `never-default`。下一条证据：

```
ip route show default
nmcli -f ipv4.gateway,ipv4.never-default,ipv4.route-metric connection show primary-lan
nmcli -f ipv4.gateway,ipv4.never-default,ipv4.route-metric connection show backup-lan
```

最小修复： 调整 metric 或禁止专用连接产生默认路由，不要无调查删除接口配置。

⑥ [诊断] 数字 IP 可达，但名称失败

症状： `ping -c 1 198.51.100.44` 成功，使用主机名失败。当前结论： 地址和数字目的路径至少局部可用。下一条证据： 检查当前 DNS server/search，然后转入第 19 章：

```
nmcli -f IP4.DNS,IP4.DOMAIN device show enp1s0
```

不要继续改 IP 地址、默认网关或静态路由。

⑦ [诊断] 路由正确，但 TCP 连接被拒绝或超时

症状： `ip route get` 正确，TCP 客户端仍失败。下一条证据： 本机服务端先检查监听：

```
ss -lntup
```

无监听转入服务管理；有监听但远端失败，进入第 21 章的 firewall 与完整服务访问链。SSH 认证细节属于第 20 章。

⑧ [诊断] nmcli connection down 后连接又出现

症状： 停用一个 profile 后 device 又连接。假设： device 自动激活了另一个候选 profile，或该 profile 再次符合自动连接条件。区分证据：

```
nmcli connection show --active
nmcli device status
nmcli connection show
```

若目标是让 device 保持断开，使用 `nmcli device disconnect <IFACE>`，但远程执行前必须评估断连。

[Cheatsheet] 链路问题先看 device/link；配置与当前不一致先看 active profile；具体路径错误用 `route get`；数字 IP 通而名称失败转 DNS；有路由但 TCP 失败再看监听、防火墙和应用。

[知识专题] 远程网络变更的最小风险与工作迁移

考试环境常有控制台，真实服务器却经常通过 SSH 管理。网络变更可能在命令执行成功的瞬间切断管理通道，因此工作中的“正确答案”还必须包含风险控制、基线 and 恢复能力。

① [边界] 远程会话不能作为唯一恢复通道

在修改当前管理接口的地址、路由或活动 profile 前，应确认至少一种恢复方式：虚拟机控制台、BMC/iBMC/iDRAC、云控制台、现场人员或预先验证的第二管理链路。

② [操作] 最小化修改范围

题目只要求把现有 profile 改为静态地址时，优先修改准确属性；不要顺便删除其他 profile、禁用 IPv6、重启 NetworkManager 或清理未知路由。每个额外变化都扩大故障面。

③ [操作] 保存对象级基线和目标终态

工作变更记录至少包括：

```
目标 device 与 MAC
活动 profile 名称和 UUID
变更前地址、默认路由和目的路径
目标地址、前缀、网关、DNS、静态路由
激活方法和回退 profile
分层验收命令
```

④ [边界] 命令成功不等于业务终态正确

`nmcli` 返回 0 只能说明请求被接受或执行完成。必须继续证明：profile 正确、当前状态正确、目标路径正确、必要服务可用。反之，远端不回应 `ping` 也不必然说明本机配置错误。

⑤ [工作迁移] 为后续自动化准备稳定状态模型

手工理解应能自然转换为自动化变量：接口、profile 名称、地址/前缀、网关、DNS、路由和 `autoconnect`。自动化不应依赖“删除所有旧配置后重建”，而应声明目标状态、保留未管理边界并提供幂等验证。

[Cheatsheet] 远程改网先有恢复入口；保存 NAME/UUID/device 和路由基线；只改评分对象；命令成功、当前正确、业务正确分别验收。

[经典任务] 把现有 DHCP profile 改为静态地址并验证重连状态

环境

训练环境使用文档保留地址，不代表真实考试环境。

```
主机: node18.example.test
设备: enp1s0
当前活动 profile: office-lan
当前 IPv4 方法: auto
控制台恢复入口: 可用
```

当前状态

`office-lan` 由 DHCP 获得地址和 DNS。设备上还保存一个未激活的 `maintenance` profile。不得删除任何 profile，也不得改变 IPv6 设置。

目标终态

```
profile: office-lan
IPv4 method: manual
地址: 192.0.2.10/24
默认网关: 192.0.2.1
DNS server: 192.0.2.53、192.0.2.54
DNS search: example.test
autoconnect: yes
```

限制条件

- 必须修改现有活动 profile；
- 不得删除 `office-lan` 后重建；
- 不得修改 `/etc/resolv.conf` 作为最终配置；
- 不得禁用 IPv6；
- 不得把 ping 成功作为唯一验收；
- 当前会话不执行真实重启，但必须给出重连和重启后的验收项。

验收证据

1. 持久 profile 的 IPv4 属性；
2. 活动 profile 与 `enp1s0` 的映射；
3. 当前地址和前缀；
4. 默认路由和源地址；
5. 当前 DNS server/search 输入；
6. 到网关的有限 ICMP 验证；
7. `autoconnect`；
8. 真实环境后续重启验证清单。

[参考解答] 经典任务一

1. 调查并保存基线

```
nmcli device status
nmcli connection show --active
nmcli connection show
nmcli -f connection.id,connection.uuid,connection.interface-name,connection.autoconnect,\
ipv4.method,ipv4.addresses,ipv4.gateway,ipv4.dns,ipv4.dns-search \
connection show office-lan

ip -br address show dev enp1s0
ip route
ip route get 192.0.2.1
nmcli -f IP4.DNS,IP4.DOMAIN device show enp1s0
```

调查目的：确认 `office-lan` 确实活动在 `enp1s0`，并记录变更前地址、路由和 DNS。若活动 profile 不是题目所述对象，应先解决对象识别差异，不能继续盲改。

2. 修改准确 profile

```
nmcli connection modify office-lan \
  ipv4.method manual \
  ipv4.addresses 192.0.2.10/24 \
  ipv4.gateway 192.0.2.1 \
  ipv4.dns "192.0.2.53 192.0.2.54" \
  ipv4.dns-search example.test \
  connection.autoconnect yes
```

参数解释：

- `ipv4.method manual`：停止依赖 DHCP 生成 IPv4 终态；
- `ipv4.addresses`：地址和前缀必须同时正确；
- `ipv4.gateway`：为普通外部目的提供默认路由下一跳；
- `ipv4.dns`：不带 +，因此把最终 DNS 列表替换为题目指定值；
- `ipv4.dns-search`：设置搜索域；
- `connection.autoconnect yes`：允许启动或 device 可用时自动激活。

没有修改 IPv6 属性，也没有删除 `maintenance`。

3. 读取持久配置后再激活

```
nmcli -f connection.id,connection.interface-name,connection.autoconnect,\
ipv4.method,ipv4.addresses,ipv4.gateway,ipv4.dns,ipv4.dns-search \
connection show office-lan

nmcli connection up office-lan
```

远程环境中执行 `connection up` 前应确认控制台或其他恢复通道。当前任务已明确控制台可用。

4. 分层验证

```
# 活动映射
nmcli device status
nmcli connection show --active

# 当前地址
ip -br address show dev enp1s0

# 当前路由与具体目的
ip route
ip route get 192.0.2.1
ip route get 198.51.100.44

# 当前 DNS 输入
nmcli -f IP4.DNS,IP4.DOMAIN device show enp1s0

# 有限功能验证
ping -c 3 192.0.2.1
```

预期判断：

- `office-lan` 活动在 `enp1s0`；
- 当前地址包含 `192.0.2.10/24`；
- 默认路由经 `192.0.2.1`；
- 对外目的的 `route get` 使用 `enp1s0`，源地址为 `192.0.2.10`；
- DNS 输入包含两个服务器和 `example.test`；
- 网关若回应 ICMP，可作为直连路径的附加证据。

5. 持久性清单

本章未连接 RHEL 9 VM，不声称已执行以下项目。真实环境应在保留恢复通道时验证：

```
nmcli device disconnect/connect 后目标 profile 再次激活
系统重启后 office-lan 仍活动
重启后地址、默认路由、DNS 输入仍为目标值
maintenance 未被意外激活
```

6. 典型错误

- 按接口名猜 profile 名并改错对象；
- 忘记把 `ipv4.method` 改为 `manual`；
- 使用 `+ipv4.dns` 导致旧 DHCP DNS 被保留；
- 修改完成后未激活；
- 为“清理环境”删除所有旧 profile；
- 直接编辑 `/etc/resolv.conf`，却没有修改连接 profile；
- 只执行一次 `ping` 就宣布全部网络终态正确。

[经典任务] 本机有地址，但到目标网络的选路错误

环境

设备: enp1s0
活动 profile: office-lan
当前地址: 192.0.2.20/24
默认网关: 192.0.2.1
目标网络: 198.51.100.0/24
正确下一跳: 192.0.2.254
测试目的: 198.51.100.44

当前证据

本机已有 192.0.2.20/24
默认路由存在
ip route get 198.51.100.44 显示 via 192.0.2.1

目标终态

- 198.51.100.0/24 经 192.0.2.254；
- 路由持久写入 office-lan；
- 激活后 ip route get 198.51.100.44 选择正确下一跳、device 和源地址。

限制条件

- 不改变 192.0.2.20/24、默认网关和 DNS；
- 不修改 firewalld 或 SSH 配置；
- 不删除或重建 office-lan；
- 不把临时 ip route add 作为最终答案；
- 添加路由前必须确认 192.0.2.254 从本机选路角度属于直连下一跳；
- 已有其他静态路由时不得无调查地替换整个 ipv4.routes 列表。

验收证据

层次	验收命令	目标判断
对象	<code>nmcli connection show --active</code>	office-lan 活动在 enp1s0
持久配置	<code>nmcli -f ipv4.routes connection show office-lan</code>	包含目标前缀和下一跳
当前路由	<code>ip route</code>	内核已安装更具体的 /24 路由
目的选路	<code>ip route get 198.51.100.44</code>	via 192.0.2.254 dev enp1s0 src 192.0.2.20
边界	ping (可选)	只作为对端允许 ICMP 时的附加证据

[参考解答] 经典任务二

1. 证明当前对象和问题层次

```
nmcli device status
nmcli connection show --active
ip -br address show dev enp1s0
ip route
ip route get 198.51.100.44
```

已知地址存在且目的走默认网关，因此问题集中在“缺少更具体路由”或“正确 route 未应用”，而不是先修改地址、DNS 或 firewalld。

2. 确认下一跳可以直连

```
ip route get 192.0.2.254
```

预期应显示 dev enp1s0，通常不需要 via，源地址为 192.0.2.20。这证明从本机选路角度，下一跳属于直连网络；它仍不证明下一跳设备一定在线。

3. 读取现有持久路由

```
nmcli -f ipv4.routes connection show office-lan
```

先看现状，避免替换整个列表时删除其他业务路由。本任务要求追加一条，因此使用 `+ipv4.routes`。

4. 添加持久静态路由并激活

```
nmcli connection modify office-lan \
+ipv4.routes "198.51.100.0/24 192.0.2.254"

nmcli -f ipv4.routes connection show office-lan
nmcli connection up office-lan
```

没有使用 `ip route add` 作为最终方案，因为后者只改变当前内核路由，重连或重启后通常丢失。

5. 验证当前路由和具体目的

```
ip route
ip route get 198.51.100.44
```

预期选路包含：

```
198.51.100.44 via 192.0.2.254 dev enp1s0 src 192.0.2.20
```

输出细节可能含 `uid` 或缓存字段，验收重点是 `via`、`dev`、`src`。

6. 可选连通验证与边界

```
ping -c 3 192.0.2.254
ping -c 3 198.51.100.44
```

下一跳或远端可能禁用 ICMP，因此 `ping` 失败不能推翻已经正确的本地选路证据。若评分要求具体 TCP 服务，应进入监听、`firewalld` 和应用协议的相邻章节验证。

7. 典型错误

- 使用 `ip route add` 完成当前状态，却没有写入 `profile`；
- 把目的写成 `198.51.100.44/24`，没有按网络边界表达目的前缀；
- 下一跳不在本机直连网络；
- 使用不带 `+` 的 `ipv4.routes`，无意替换其他路由；
- 只在 `ip route` 中看见条目，没有执行 `ip route get`；
- 因远端不回应 `ping` 而继续修改已经正确的地址和路由。

[本章收束] 把“网络正确”拆成可以逐项证明的结论

NetworkManager 题目的难点通常不在命令长度，而在对象和证据错位。一个稳定的完成顺序是：先识别 `device` 和活动 `profile`，再读取持久属性；修改后明确触发激活；随后从内核角度检查地址、路由和具体目的路径；最后才做受限的连通或监听验证。

工作方法：每次网络变更都保留四份证据

对象证据：device、profile NAME、UUID、活动映射
配置证据：ipv4.method、addresses、gateway、routes、dns、autoconnect
当前证据：ip address、ip route、ip route get
功能证据：ping 或 ss 所能证明的局部结果

这四份证据不能互相替代。profile 写对但未激活，当前状态仍可能是旧值；当前路径正确但远端不回应 ICMP，也不能直接反推本机配置错误；本机存在监听，同样不能证明 `firewalld` 和远端访问链已经正确。

主要判断表

看到的证据	可以得出的结论	不能据此断言	下一步
<code>nmcli connection show <CON></code> 属性正确	持久 profile 已保存目标值	内核已经采用这些值	核对 active connection，再看 <code>ip</code>

看到的证据	可以得出的结论	不能据此断言	下一步
<code>nmcli connection show --active</code> 显示目标 profile	目标 profile 当前处于活动状态	地址、路由和业务一定正确	查看 address、route 和 route get
<code>ip -br address</code> 地址正确	内核当前已配置该地址和前缀	默认路由、DNS 或远端可达	查看 <code>ip route</code>
<code>ip route</code> 中存在目标路由	路由表包含该条目	具体目的实际选中它	执行 <code>ip route get <DEST></code>
<code>ip route get</code> 的 via/dev/src 正确	本地内核对该目的的选路正确	下一跳或远端一定在线	按要求做受限功能验证
<code>ping</code> 成功	该次 ICMP 往返成功	TCP、DNS、应用和所有路径都正确	根据评分对象继续验证
<code>ss -lntup</code> 显示监听	本机存在对应 socket	远端流量可穿过防火墙到达	转第 21 章检查完整访问链

向下一章交接

本章只把 DNS server 和 search domain 当作 connection profile 的属性，并观察它们是否进入当前连接。若数字 IP 路径正确而名称仍失败，下一步不应继续重写地址和路由，而应进入第 19 章《主机名、NSS 与 DNS 名称解析》，检查主机名、`/etc/hosts`、NSS 顺序、解析器状态和 DNS 记录。

SSH 的客户端、服务端和密钥认证属于第 20 章；端口放行、zone、runtime/permanent 与完整服务访问链属于第 21 章。本章到此只负责把数据包送到正确的本地出口，并提供清楚的网络层证据。

主机名、NSS 与 DNS 名称解析

从“这台主机叫什么”到“应用最终连向哪个地址”：把本机身份、来源顺序、搜索域、DNS 记录与应用证据放进同一条解析链。

对象模型操作语义验证诊断经典任务

大字号阅读版

本章阅读导航

先抓住一条主线：主机名是本机身份；名称解析是应用把名称转换为地址的过程。两者会相互影响，但任何一条命令都不能同时证明本机身份、DNS 记录、NSS 结果和应用功能。

01 本机身份区分 static、transient 与 pretty hostname
02 名称形式判断短名、FQDN 与绝对名称
03 NSS 调度读取 hosts 数据库的来源和顺序
04 本地与 DNS 区分 /etc/hosts 与 A、AAAA、PTR
05 客户端配置核对 DNS server、search 与有效 resolver
06 最终证据用 getent、dig 与应用请求分层验证
专题地图
知识专题三种 hostname 与持久身份
知识专题短名、FQDN、绝对名称与 search
知识专题 NSS hosts: 来源顺序
操作专题 hostnamectl 与 /etc/hosts
知识专题 A、AAAA 与 PTR
操作专题 NetworkManager DNS 与 resolver
操作专题 getent 与 dig 的证据边界
诊断专题短名失败、正反向不一致
诊断专题 dig 成功但应用失败
经典任务配置任务与差异诊断任务
阅读时持续回答

1. 题目改变的是本机身份，还是名称到地址的映射？
2. 当前值与重启后持久值分别在哪里？
3. 应用提交的是短名、FQDN 还是绝对名称？
4. **hosts**：行会先问哪个来源？
5. DNS server 和 search 来自哪个 profile？
6. 失败的是 A、AAAA 还是 PTR？
7. **dig** 与 **getent** 的差异排除了哪一层？
8. 哪条证据真正重复了原始失败路径？

使用方法：先用概念块区分对象，再通过操作语义确认接口，最后沿状态、查询、验证和诊断链完成任务。经典任务与参考解答分别独立阅读。

第 19 章 · 正文

一台 Linux 主机“叫什么名字”，和一个应用“怎样把名字转换为地址”，是两个相互关联但并不相同的问题。主机名属于本机身份；**/etc/hosts** 是本地映射；NSS 决定系统库按什么来源和顺序查找；NetworkManager 保存 DNS server 与 search domain；DNS 服务端分别提供 A、AAAA、PTR 等记录。应用最终得到的结果还可能受自身缓存、长期连接或专用解析库影响。

最常见的误判都来自把这些对象压成一个“DNS 是否正常”的问题：**hostnamectl** 正确不代表 DNS 已发布；**dig** 成功不代表应用通过 NSS 得到同一地址；A 记录存在不代表 PTR 已建立；**/etc/resolv.conf** 当前正确不代表持久 profile 已正确；修改 **/etc/hosts** 也不会把结果传播给其他主机。

本章按照“身份 → 名称形式 → NSS 来源 → 本地或 DNS 数据 → 客户端配置 → 系统查询 → 应用功能”推进。IP 地址、连接 profile 的一般创建与路由属于第 18 章；SSH 主机密钥与身份校验属于第 20 章；本章不部署权威或递归 DNS 服务器。

贯穿本章的主问题

- 系统现在叫什么，重启后又会叫什么？
- 应用查询的名称会被怎样补全、按什么来源查找？
- DNS 服务端返回什么，系统 NSS 最终又返回什么？
- 哪一层证据成功，仍然不能证明下一层成功？

核心概念

概念

主机名的三种状态 static hostname 表达持久机器身份，通常落在 `/etc/hostname`；transient hostname 表达当前运行期可能由网络提供的临时名称；pretty hostname 是面向人的描述文本。三者解决的问题不同，不能因为 `hostnamectl status` 中某一项正确，就推断另外两项、DNS 记录或应用解析已经正确。

概念

本机身份与 DNS 名称 主机名描述本机如何标识自己，DNS 名称描述分布式名称空间中的节点。二者常被配置成相同字符串，但没有自动同步关系：`hostnamectl set-hostname` 不会创建 A、AAAA 或 PTR，DNS 服务端的记录也不会自动改写本机 `/etc/hostname`。

概念

NSS (Name Service Switch) 是应用通过系统库查询主机、用户、组等数据库时使用的来源调度机制。对主机名称而言, `/etc/nsswitch.conf` 的 `hosts` 行决定可能访问 `files`、`dns`、`myhostname`、`resolve` 等来源及其顺序。NSS 是“应用通常怎么查”的主线, 不能用直接 DNS 工具代替。

概念

DNS resolver 是客户端把 DNS 查询送往哪台服务器、怎样重试和怎样处理搜索域的运行入口。NetworkManager connection profile 通常是持久配置源, `/etc/resolv.conf` 是当前有效视图或指向本地 stub 的入口。只检查其中一层, 无法证明持久配置、当前上游和最终查询同时正确。

概念

短名、FQDN 与 search domain `node1` 是需要上下文的短名: `node1.training.example` 已包含所属域; 末尾带点的 `node1.training.example.` 明确表示绝对 DNS 名。search domain 只负责把短名扩展成待查询名称, 不会创建记录, 也不能保证不同客户端对同一短名得到相同目标。

概念

A、AAAA 与 PTR 是独立的 DNS 记录方向: A 把名称映射到 IPv4, AAAA 把名称映射到 IPv6, PTR 在反向命名空间中把地址映射到名称。正向成功并不推导反向成功; A 存在也不推导 AAAA 存在。诊断时必须明确查询类型, 而不是只说“DNS 有结果”。

概念

系统解析结果与应用结果 `getent` 观察系统 NSS 路径, `dig` 观察 DNS 查询, 应用还可能使用缓存、专用运行时或已经建立的连接。因此最末层验收必须重复应用原始失败路径; 工具成功是重要证据, 但不能自动扩大成业务终态正确。

操作语义速查

操作语义先建立六个关键入口的作用对象和证据边界, 再进入正文中的具体操作、验证和诊断。

`hostnamectl`

SYNOPSIS

```
hostnamectl [OPTIONS...] COMMAND [NAME]
```

观察或修改 systemd-hostnamed 管理的主机名状态。它回答本机身份, 不直接查询 DNS。

`hostnamectl status`

同时查看 static、transient、pretty 等状态。

`hostnamectl set-hostname NAME`

设置题目要求的主机名: 默认行为仍要结合当前版本帮助确认。

`--static / --transient / --pretty`

有意只观察或修改某一种状态时使用。

`/etc/hosts`

LINE FORM

```
IP-address primary-name [alias ...]
```

提供本机 `files` 来源的静态名称映射。适合小范围、节点私有或故障隔离, 不会向网络发布 DNS。

`192.0.2.44 repo.training.example repo-local`

地址在前, 主要名称随后, 零个或多个别名位于末尾。

最小修改

修改前先查同名、同地址与 IPv4/IPv6 冲突; 不要删除无关条目。

验证

使用 `getent` 证明 NSS 结果, 不使用 `dig` 证明本地条目。

`/etc/nsswitch.conf` • `hosts:`

POLICY FORM

```
hosts: files dns
hosts: files myhostname dns
```

规定系统主机数据库的来源顺序和可选状态动作。不要背诵一条“永久默认值”, 应读取当前系统真实行。

`files`

通常读取 `/etc/hosts`。

`dns`

通过当前 resolver 查询 DNS。

`[STATUS=ACTION]`

高级状态动作会改变继续或停止策略; 修改前必须先读手册和现状。

getent

SYNOPSIS

```
getent database [key ...]
```

通过系统 NSS 查询数据库，是观察普通应用系统解析路径的主要入口。

```
getent hosts NAME
    查询主机数据库的常用形式。
getent ahostsv4 NAME
    只观察 IPv4 地址族结果。
getent ahostsv6 NAME
    只观察 IPv6 地址族结果。
```

dig

SYNOPSIS

```
dig [@server] name [type] [query-option ...]
```

直接构造 DNS 查询并显示服务端回答。它能隔离 DNS 层，但不会读取 `/etc/hosts`，也不能代表完整 NSS 或应用行为。

```
dig NAME A / AAAA
    分别查询 IPv4 与 IPv6 正向记录。
dig -x ADDRESS
    构造反向 PTR 查询。
dig @SERVER NAME TYPE
    隔离指定 DNS 服务器。
+short
    只显示简短答案；排错时仍应保留完整状态与 ANSWER 证据。
+search
    显式使用 resolver 搜索列表测试短名。
```

nmcli • DNS 属性

SYNOPSIS

```
nmcli connection show [NAME]
nmcli connection modify NAME PROPERTY VALUE
```

修改 NetworkManager connection profile 中的持久 DNS 输入。IP 地址和路由的一般配置留在第 18 章。

```
ipv4.dns / ipv6.dns
    保存对应地址族的 DNS server 列表。
ipv4.dns-search / ipv6.dns-search
    保存搜索域。
ipv4.ignore-auto-dns yes
    只有题意明确要求排除 DHCP 等自动 DNS 时才使用。
nmcli connection up NAME
    重新激活 profile 可能中断远程会话；操作前先评估连接路径。
```

名称解析链：每一层回答不同问题

本机 hostname→名称形式→NSS hosts 策略→/etc/hosts 或 DNS→A / AAAA / PTR→应用结果

层次	首要证据	能证明什么	不能单独证明什么
本机身份	hostnamectl、hostname、/etc/hostname	当前或持久主机名	DNS 已有同名记录
NSS 策略	grep 'hosts:' /etc/nsswitch.conf	来源与顺序	每个应用都没有自有缓存
本地映射	/etc/hosts、getent	本机 files 结果	其他主机也能解析
DNS 客户端	nmcli、/etc/resolv.conf	持久输入与当前入口	指定记录一定存在
DNS 数据	dig	某服务器对某类型查询的回答	NSS 或应用得到同一结果
应用功能	原始应用请求与日志	最终路径是否恢复	所有其他客户端同时恢复

[知识专题] 一台主机为什么同时存在三种 hostname

主机名不是单一字符串。systemd 将它拆成静态、临时和美化三种视图，是为了分别表达持久机器身份、当前运行名称和人类可读描述。实际调查时，最顺的入口是先运行 `hostnamectl status`，再确认 `/etc/hostname`，最后判断当前名称是否经过 NSS 能形成期望的 FQDN。

① [知识点] static hostname 是重启后应恢复的持久身份

静态主机名通常保存在 `/etc/hostname`。在 RHEL 9 中，应优先通过 `hostnamectl set-hostname` 修改，而不是把“编辑文件成功”当作完整操作。静态主机名适合使用可由 DNS 或本地映射解析的名称，例如：

```
node1.training.example
```

静态主机名回答的是：这台机器被持久配置成什么名称？它不直接回答当前 DNS 中是否已有同名 A、AAAA 或 PTR 记录。

② [知识点] transient hostname 表示当前运行期的临时名称

临时主机名存在于当前运行状态中，可能由内核启动参数、DHCP 或其他网络配置提供。当静态主机名未正确设置时，临时名称可能成为当前显示值。它通常不应被当作重启后的唯一保证。

调查时要避免只执行一次 `hostname` 就下结论。`hostname` 主要反映当前内核名称，而 `/etc/hostname` 反映持久静态配置。二者不一致时，应先识别题目要求的是“现在改变”还是“重启后保持”。

③ [知识点] pretty hostname 是描述，不是 DNS 标签

美化主机名用于管理界面或资产描述，可以包含空格和大小写，例如：

```
Training Web Node 1
```

它不应被放入 `/etc/hosts` 或 DNS zone，也不能作为服务端点的稳定标识。考试和日常运维涉及可解析主机身份时，重点通常是静态主机名。

④ [知识点] 本机主机名与 FQDN 不是自动等价关系

FQDN 是完整限定名称。系统当前主机名可能本身就是 FQDN，也可能只是短名。类似 `hostname -f` 的接口会尝试结合当前主机名和名称解析结果得到规范名称，因此其结果依赖 NSS、`/etc/hosts` 和 DNS，而不是单纯读取 `/etc/hostname`。

因此：

```
hostnamectl 显示 node1.training.example
```

只能证明本机身份已经设置；仍需用 `getent hosts node1.training.example` 或应用实际访问确认解析链。

⑤ [知识点] 修改 hostname 不会发布 DNS

`hostnamectl set-hostname` 改变本机状态，但不会：

- 向权威 DNS 创建 A 或 AAAA 记录；
- 创建 PTR 记录；
- 修改其他客户端的 `/etc/hosts`；
- 自动更新 SSH `known_hosts`；
- 证明依赖旧主机名的应用配置已经迁移。

在真实工作中，主机名变更应与 DNS、监控、证书、日志标签和资产平台一起评估。本章只完整处理本机身份和客户端解析链。

Cheatsheet static 看持久身份；transient 看当前临时值；pretty 只作描述；`hostname` 看当前内核名称；`/etc/hostname` 看持久真源；设置主机名不等于发布 DNS。

[知识专题] 短名、FQDN、绝对名称与搜索域

同一个目标可能以 `node1`、`node1.training.example` 或 `node1.training.example.` 出现。这些字符串不是简单的长短差异：它们决定解析器是否使用搜索域、怎样生成查询序列，以及诊断时是否存在歧义。最稳妥的切入路径是分别测试短名、FQDN 和带末尾点的绝对名称。

① [知识点] 短名需要上下文才能成为完整名称

`node1` 没有指出所属 DNS 域。系统解析器可能把它与 search domain 组合：

```
node1 + training.example  
→ node1.training.example
```

短名适合在边界明确的内部环境中使用，但它的含义依赖客户端配置。不同客户端拥有不同 search 列表时，同一个短名可能解析到不同目标。

② [知识点] FQDN 表达从主机标签到 DNS 域的完整路径

node1.training.example 通常被人理解为 FQDN。严格地说，DNS 绝对名称以根标签结束，文本形式可以写为：

```
node1.training.example.
```

末尾点能够明确表示“不再追加搜索域”。日常命令通常省略它；诊断多搜索域或可疑多标签名称时，带末尾点的查询更有区分度。

③ [知识点] search domain 只解决名称补全，不决定 DNS 服务器内容

search training.example 告诉解析器怎样扩展短名，但不会创建任何记录。要让 node1 成功，扩展后的名称仍必须在 /etc/hosts、DNS 或其他 NSS 来源中存在。

search 列表按顺序尝试各搜索域。列表越长，短名失败可能产生越多查询，也更容易因同名资源导致歧义。

/etc/resolv.conf 还支持 domain example.com 表示单一搜索域；search 则可列出按顺序尝试的多个域。如果文件中同时出现 domain 与 search，应以最后出现的有效指令所形成的搜索列表为准。实际运维中仍要回到 NetworkManager profile 确认持久来源，而不是只改当前文件。

④ [知识点] ndots 会影响多标签名称的尝试顺序

resolv.conf 的 options ndots:N 影响包含多少个点的名称会优先按绝对名称尝试。默认行为和具体失败回退由 resolver 实现决定。RHCSA 操作不要求背诵所有算法分支，但必须知道：

- 无点短名通常依赖 search；
- 多标签名称不一定完全绕过 search；
- 带末尾点的绝对名称可消除搜索扩展歧义；
- 诊断时不要只测试一种写法。

⑤ [知识点] dig 默认不等于应用的短名解析

getent hosts node1 走 NSS 和系统 resolver；dig node1 面向 DNS 层，默认行为不应被当作应用查询的完整等价物。需要显式测试 search 时，可使用：

```
dig +search node1 A
```

更可控的做法是先从 /etc/resolv.conf 或 NetworkManager 读取 search domain，再直接查询构造出的 FQDN。

Cheatsheet 短名依赖 search；FQDN 减少歧义；末尾点表示绝对名称；search 不创建记录；应用短名验证优先用 `getent`，DNS 记录验证优先用 `dig`。

[知识专题] NSS 的 hosts 数据库如何选择解析来源

应用通常不会自己逐个读取 /etc/hosts、再访问 DNS。它调用系统解析接口，由 NSS 根据 /etc/nsswitch.conf 中的 hosts: 行选择来源。排错的关键不是背诵某个默认行，而是读取当前系统的真实配置并理解“顺序”和“返回状态”。

① [知识点] hosts: 是数据库策略，不是 DNS 配置

一个常见但不能当作所有系统绝对默认值的示例是：

```
hosts: files dns
```

含义是先通过 files 来源查找主机数据库，再进入 dns。其中：

- files 通常对应 /etc/hosts；
- dns 通过 resolver 配置查询 DNS；
- myhostname 可合成本机名称和本地地址；
- resolve 可把请求交给 systemd-resolved；
- 其他模块是否存在取决于已安装软件和当前配置。

② [知识点] 来源顺序决定冲突时谁先提供答案

如果 /etc/hosts 中有旧地址，而 files 位于 dns 之前，getent hosts 和多数系统应用可能优先取得旧地址，即使 dig 显示 DNS 已经正确。

这不是“DNS 缓存一定没清”，而是两个工具查询了不同路径。正确修复通常是处理过期本地条目，而不是把 dns 移到 files 前面掩盖冲突。

③ [知识点] NSS 还可以根据返回状态决定继续或停止

nsswitch.conf 支持类似以下动作语法：


```
[STATUS=ACTION]
```

状态可能包括 `SUCCESS`、`NOTFOUND`、`UNAVAIL`、`TRYAGAIN`，动作通常是 `return` 或 `continue`。本章不要求穷尽所有模块组合，但要能识别：中括号动作可能让解析链在某种失败状态下提前停止。

修改这类全局策略风险较高。没有明确任务和证据时，不应为了让一次查询成功而改变整个 `hosts:` 行的全局来源顺序。

④ [知识点] `getent` 是观察 NSS 结果的主要接口

```
getent hosts node1.training.example
```

它调用系统数据库接口，因此比 `cat /etc/hosts` 更接近应用最终看到的结果。需要区分地址族时，可使用：

```
getent ahostsv4 node1.training.example
getent ahostsv6 node1.training.example
```

输出可能包含多行、多地址或不同排序。判断重点是是否得到预期地址族和地址，而不是背诵固定行序。

⑤ [知识点] NSS 成功仍不代表应用业务成功

`getent` 成功证明系统解析接口当前能得到结果，但应用仍可能：

- 使用自己的 DNS 客户端库；
- 缓存旧地址；
- 已经保持旧连接；
- 在容器或独立网络命名空间中使用不同 resolver；
- 因端口、路由、TLS 或服务端状态失败。

因此名称解析通过后，仍需执行应用功能验证。

Cheatsheet 先读真实 `'hosts:'` 行；顺序决定冲突优先级；中括号动作可能提前停止；`'getent'` 观察 NSS；NSS 成功不等于业务终态。

[操作专题] 查询和设置主机名并区分当前与持久状态

主机名变更看似简单，但至少包含当前状态、静态持久配置和名称解析三个维度。安全操作从建立基线开始，随后只修改题目要求的类型，再分别验证当前和持久状态。

① [查询] 建立主机名基线

作用对象：systemd-hostnamed 管理的 `hostname` 状态和 `/etc/hostname`。

基本形式：

```
hostnamectl status
hostname
cat /etc/hostname
```

较新 `hostnamectl` 版本可用类型选项配合 `hostname` 子命令隔离查询；RHEL 9 考试中以 `hostnamectl status` 和持久文件交叉验证即可。不要把 `pretty hostname` 当作可解析名称。

② [操作] 设置题目要求的静态主机名

典型形式：

```
hostnamectl set-hostname node1.training.example
```

该操作通常立即更新当前名称并写入静态配置。若只想修改一种类型，应显式使用对应选项，例如：

```
hostnamectl --pretty set-hostname "Training Web Node 1"
```

边界：具体版本对默认同时设置哪些 `hostname` 类型的表现，应以 `hostnamectl status` 验证，不靠记忆代替证据。

③ [操作] 验证当前状态与持久状态

```
hostname
hostnamectl status
cat /etc/hostname
```

验证解释：

- `hostname`：当前内核名称；
- `hostnamectl status`：systemd 管理的各类名称与系统信息；
- `/etc/hostname`：静态持久配置。

若题目明确要求重启后保持，静态配置检查只能作为持久性证据；最终仍需在目标系统上真实重启，再重复检查当前状态、名称解析和应用功能。

④ [操作] 验证主机名是否能沿 NSS 解析

```
getent hosts node1.training.example
```

如果当前主机名只设置了身份但没有本地或 DNS 记录，`getent` 可能失败。此时应进入 `/etc/hosts`、NSS 和 DNS 调查，而不是重复执行 `hostnamectl`。

⑤ [操作] 变更前后记录最小证据

建议保存：

```
hostnamectl status
cat /etc/hostname
grep -E '^[[:space:]]*hosts:' /etc/nsswitch.conf
getent hosts "${hostname}"
```

这些命令分别记录身份、持久真源、解析策略和当前查询结果。生产环境还应检查证书、监控标签和集群配置，但这些扩展不作为本章考试主线。

Cheatsheet `'hostnamectl set-hostname'` 修改；`'hostname'` 看当前；`'/etc/hostname'` 看持久；`'getent'` 看能否解析；身份正确不代表 DNS 正确。

[操作专题] 用 `/etc/hosts` 建立可审计的本地名称映射

`/etc/hosts` 适合少量、明确、本机范围的静态映射。它不是“小型 DNS 服务器”，也不会自动传播。高质量操作不是简单追加一行，而是先查冲突、使用规范字段顺序、再通过 NSS 验证。

① [知识点] 每行由地址、规范名和可选别名组成

```
192.0.2.44 repo.training.example repo-local
```

字段含义：

1. 地址；
2. 规范名称；
3. 零个或多个别名。

空白可由空格或制表符分隔，`#` 后为注释。IPv4 和 IPv6 映射应分别写明，不要把一个地址族的记录当作另一个地址族的替代。

② [查询] 修改前先寻找同名和同地址条目

```
grep -nE '^[[:space:]]*repo-local([[:space:]]|$)' /etc/hosts
grep -nF 'repo.training.example' /etc/hosts
grep -nF '192.0.2.44' /etc/hosts
```

调查目标包括：

- 同名是否指向多个不同地址；
- 旧规范名是否仍保留；
- 别名是否重复；
- 是否存在 IPv4 与 IPv6 的不同预期；
- 是否误改 `localhost` 基础条目。

③ [操作] 使用安全编辑方式完成最小修改

考试中可使用熟悉的文本编辑器。真实工作中应先备份或使用配置管理。不要用无条件 `echo >> /etc/hosts` 反复追加，因为重入后容易产生重复和冲突。

示例目标行：

```
192.0.2.44 repo.training.example repo-local
```

④ [操作] 通过 `getent` 而不是 `dig` 验证本地映射

```
getent hosts repo.training.example
getent hosts repo-local
```

`dig` 和 `host` 面向 DNS，不会证明 `/etc/hosts` 条目被 NSS 使用。`cat /etc/hosts` 只能证明文件内容存在，也不能证明 `hosts`：顺序允许该来源返回答案。

⑤ [边界] 本地映射不会发布到其他节点

`/etc/hosts` 只影响读取该文件并遵循相应 NSS 策略的本机环境。它不提供：

- 集中更新；
- TTL 与动态记录；
- 远端客户端查询；
- 自动 PTR；
- 容器内独立 `hosts` 文件的同步。

当映射数量、节点数量或变更频率上升时，应使用集中 DNS 或配置管理，而不是无限扩展手工 `hosts` 文件。

Cheatsheet 地址在前、规范名其次、别名最后；修改前查冲突；验证用 ``getent``；``dig`` 不读 ``/etc/hosts``；本地映射不会传播。

[知识专题] A、AAAA 与 PTR 是三类独立 DNS 数据

DNS 不是简单的“名称与 IP 对照表”。查询必须说明名称、记录类型和服务器。A、AAAA 与 PTR 分别处理 IPv4 正向、IPv6 正向和地址到名称的反向映射，它们可以独立存在、独立出错。

① [知识点] A 记录把名称映射到 IPv4 地址

```
dig node1.training.example A
```

一项名称可以返回多个 A 记录。多地址可能用于负载分担或容错，并不天然表示冲突。验证时应判断题目是否要求“包含某地址”还是“只能有某地址”。

② [知识点] AAAA 记录把名称映射到 IPv6 地址

```
dig node1.training.example AAAA
```

A 成功而 AAAA 没有答案，不等于 DNS 整体失败。是否需要 AAAA 取决于题目、网络栈和应用。排错时应显式指定记录类型，避免默认查询掩盖地址族差异。

③ [知识点] PTR 记录把地址映射到名称

```
dig -x 192.0.2.44
```

`dig -x` 会构造反向区域名称并查询 PTR。IPv4 使用 `in-addr.arpa`，IPv6 使用 `ip6.arpa`。PTR 所属区域可能由不同团队或网络提供方维护，因此正向记录正确不能保证反向记录存在。

④ [知识点] 正向和反向不要求机械一一对应

一个名称可以有多个地址；多个名称也可能指向同一地址。某些服务依赖反向解析或正反向一致性，但这属于应用策略，不能把它当作 DNS 协议对所有记录的自动约束。

验证矩阵应分别记录：

```
名称 --A----> IPv4
名称 --AAAA-> IPv6
地址 --PTR--> 名称
```

⑤ [知识点] DNS 状态和 ANSWER 数量必须一起解释

常见响应判断：

- `NOERROR` 且有 ANSWER：该类型有数据；
- `NOERROR` 但 ANSWER 为 0：名称可能存在，但该类型没有数据；
- `NXDOMAIN`：被查询名称不存在；
- `SERVFAIL`：服务器无法完成解析；
- `REFUSED`：服务器拒绝该查询；
- 超时：还需区分网络不可达、端口过滤和服务器无响应。

只看到 `NOERROR` 就写“解析正常”是不完整的。

Cheatsheet A 查 IPv4；AAAA 查 IPv6；`dig -x` 查 PTR；正反向独立；状态码要和 ANSWER 一起看。

[操作专题] 通过 NetworkManager 配置 DNS server 与 search domain

在 RHEL 9 默认网络栈中，持久 DNS 配置通常属于 NetworkManager connection profile。`/etc/resolv.conf` 是当前有效视图，可能由 NetworkManager 生成，也可能指向本地 stub。正确路径是先识别 profile 和文件所有权，再修改 profile，最后重新激活并分层验证。

① [查询] 识别活动连接、设备和 resolver 形态

```
nmcli connection show --active
nmcli device status
readlink -f /etc/resolv.conf
cat /etc/resolv.conf
```

不要只看到 `nameserver 127.0.0.1` 或 `127.0.0.53` 就判断上游 DNS 丢失。该地址可能是本地缓存或 stub resolver，需要继续检查当前系统实际使用的组件。

② [查询] 读取 profile 中的持久 DNS 属性

```
nmcli -f connection.id,connection.interface-name,ipv4.method,ipv4.dns,ipv4.dns-search,ipv4.ignore-auto-dns \
connection show exam-net
```

IPv6 环境还应读取：

```
nmcli -f ipv6.method,ipv6.dns,ipv6.dns-search,ipv6.ignore-auto-dns \
connection show exam-net
```

这里查看的是保存配置，不等同于当前设备已经应用。

③ [操作] 替换、追加或删除 DNS server

替换当前 IPv4 DNS 列表：

```
nmcli connection modify exam-net ipv4.dns "192.0.2.53"
```

追加一个服务器：

```
nmcli connection modify exam-net +ipv4.dns "192.0.2.54"
```

删除一个服务器：

```
nmcli connection modify exam-net -ipv4.dns "192.0.2.54"
```

在修改前必须确认题意是替换还是追加。误用默认替换会删除原有备用服务器；无条件追加又可能保留题目明确要求移除的地址。

④ [操作] 配置 search domain

```
nmcli connection modify exam-net ipv4.dns-search "training.example"
```

多个搜索域是多值属性，应在修改后读取 profile 确认顺序。search 只补全短名，不创建记录，也不会修复错误的 `/etc/hosts`。

⑤ [操作] 仅在题意明确时忽略自动 DNS

DHCP 或其他自动配置可能提供 DNS。题目若要求“只使用指定服务器”，可设置：

```
nmcli connection modify exam-net ipv4.ignore-auto-dns yes
```

如果题目只是要求增加服务器，不能机械地启用该选项，因为它会丢弃自动获得的 DNS 和搜索域。IPv6 自动 DNS 需要独立判断 `ipv6.ignore-auto-dns`。

⑥ [操作] 应用配置并控制中断风险

常见考试操作：

```
nmcli connection up exam-net
```

重新激活可能短暂影响远程连接。真实工作中应通过控制台、维护窗口或评估 `nmcli device reapply` 的适用性。不要在未确认 connection 与接口关系时随意 down 全部连接。

⑦ [验证] 分开检查持久 profile 与当前设备状态

```
nmcli -f ipv4.dns,ipv4.dns-search,ipv4.ignore-auto-dns connection show exam-net
nmcli -f GENERAL.CONNECTION,IP4.DNS,IP4.DOMAIN device show
cat /etc/resolv.conf
```

验证解释：

- connection view：重启后应恢复的配置；
- device view：当前活动设备得到的 DNS 信息；
- resolver 文件或 stub：系统解析器当前入口；
- `getent` / `dig`：实际查询行为。

⑧ [边界] 不把直接锁定 `/etc/resolv.conf` 当作标准答案

直接编辑可能被 NetworkManager 在连接重激活时覆盖；使用 `chattr +i` 则可能破坏网络管理器的正常更新。除非系统明确配置为手工管理模式，否则应回到 connection profile 修改。

Cheatsheet profile 是持久真源；device view 是当前状态；`resolv.conf` 是有效入口；`ipv4.dns` 替换，`+` 追加，`-` 删除；`ignore-auto-dns` 只在明确要求时使用。

[操作专题] 用 `getent` 与 `dig` 分别验证系统解析和 DNS

名称解析排错最有价值的对照，是对同一个名称分别运行 `getent` 与 `dig`。两者结果相同可以缩小问题范围；两者结果不同则直接暴露 NSS、本地映射、resolver 配置或指定服务器之间的分层差异。

① [查询] 用 `getent hosts` 模拟系统应用的 NSS 路径

```
getent hosts node1.training.example
```

短名验证：

```
getent hosts node1
```

地址族验证：

```
getent ahostsv4 node1.training.example
getent ahostsv6 node1.training.example
```

`getent` 返回成功只证明当前系统数据库接口得到了结果。输出可能包含多行，地址排序也可能随策略变化。

② [查询] 用 `dig` 显式指定记录类型

```
dig node1.training.example A
dig node1.training.example AAAA
dig -x 192.0.2.44
```

简洁查看答案可使用：

```
dig +short node1.training.example A
```

但 `+short` 隐藏状态、权威信息和失败原因。诊断失败时应回到完整输出。

③ [查询] 用 `@SERVER` 隔离特定 DNS 服务器

```
dig @192.0.2.53 node1.training.example A
```

这条命令回答：指定服务器怎样响应？它会绕开系统默认 `nameserver` 选择，不能证明当前应用已经使用该服务器。

若指定服务器成功而普通 `dig` 失败，下一步应检查 `/etc/resolv.conf`、本地 `stub`、`NetworkManager` 当前 DNS 顺序和网络可达性。

④ [查询] 显式测试 `search domain`

系统短名路径：

```
getent hosts node1
```

DNS 工具显式启用搜索：

```
dig +search node1 A
```

无歧义绝对查询：

```
dig node1.training.example. A
```

三种命令回答不同问题，应在报告中保留实际查询字符串。

⑤ [验证] 建立最小名称解析矩阵

目标	推荐证据	主要证明
本地/NSS FQDN	<code>getent hosts FQDN</code>	系统解析路径
NSS 短名	<code>getent hosts SHORT</code>	<code>search</code> 与 NSS 合成结果
IPv4 DNS	<code>dig FQDN A</code>	默认 DNS 路径的 A 记录
IPv6 DNS	<code>dig FQDN AAAA</code>	默认 DNS 路径的 AAAA 记录
指定服务器	<code>dig @SERVER FQDN A</code>	单台 DNS server 的回答
反向记录	<code>dig -x ADDRESS</code>	PTR 结果
应用终态	应用请求与日志	实际功能

⑥ [边界] 不使用 `ping` 代替名称解析验收

`ping name` 可能调用系统解析器，但后续还依赖 ICMP、路由、防火墙和对端响应。`ping` 失败不能直接证明 DNS 失败；`ping` 成功也不能证明目标服务端口可用。

Cheatsheet ``getent`` 看 NSS；``dig`` 看 DNS；``@SERVER`` 只证明指定服务器；``+short`` 适合取值、不适合解释失败；最终仍需应用验证。

[诊断专题] 短名失败、FQDN 成功与正反向不一致

这类故障的高区分度证据不是“再换一个 DNS”，而是把名称形式和记录类型拆开。诊断按症状、证据、假设、最小修复和再验证推进。

① [诊断] 短名失败而 FQDN 成功

症状：

```
getent hosts node1          失败
getent hosts node1.training.example 成功
```

假设： `search domain` 缺失、顺序错误，或应用没有使用系统 `resolver`。

下一条证据：

```
cat /etc/resolv.conf
nmcli -f ipv4.dns-search,ipv6.dns-search connection show exam-net
nmcli -f IP4.DOMAIN,IP6.DOMAIN device show
```

最小修复：在正确的现有 profile 上配置所需 search domain，重新应用。

再验证：同时重复短名和 FQDN 的 `getent`，不能只看配置文件。

② [诊断] `hostnamectl` 正确而 `hostname -f` 异常

当前证据：静态主机名已经设置，但规范名称派生失败或返回意外值。

假设：当前 `hostname` 无法在 NSS 中映射，或 `/etc/hosts` 的规范名和别名顺序不符合预期。

下一条证据：

```
hostname
getent hosts "$(hostname)"
grep -nF "$(hostname)" /etc/hosts
grep -E '^[[:space:]]*hosts:' /etc/nsswitch.conf
```

最小修复：处理本地映射或 DNS 记录，不重复设置已经正确的 static `hostname`。

③ [诊断] A 成功而 AAAA 没有数据

当前证据：

```
dig node1.training.example A
dig node1.training.example AAAA
```

如果 A 有答案而 AAAA 返回 `NOERROR`、ANSWER 为 0，应先确认题目和应用是否要求 IPv6。没有 AAAA 可能是预期状态，不应直接修改 IPv4 配置或宣布 DNS 整体故障。

④ [诊断] 正向成功而反向 PTR 失败

当前证据：A 或 AAAA 正确，`dig -x` 返回 `NXDOMAIN`、空答案或其他错误。

假设：反向区域未委派、PTR 未创建或由另一管理方维护。

最小修复边界：客户端只能证明问题并提供地址、期望名称和服务器证据；部署或修改权威 DNS 不在本章范围。不要通过本机 `/etc/hosts` 伪造“网络反向解析已经修复”。

⑤ [诊断] 多个 search domain 导致短名命中错误目标

症状：短名能解析，但得到的 FQDN 或地址不是预期目标。

下一条证据：

```
cat /etc/resolv.conf
getent hosts node1
dig node1.first.example. A
dig node1.second.example. A
```

最小修复：调整正确 profile 的 search 顺序，或让关键配置使用无歧义 FQDN。删除搜索域前先确认其他业务依赖。

Cheatsheet 短名失败先查 search；`hostname -f` 异常进入 NSS；A 与 AAAA 分开；PTR 是独立区域；短名命中错误要查 search 顺序。

[诊断专题] `dig` 成功但 NSS 或应用解析失败

“`dig` 成功”只证明某次 DNS 查询得到答案。要解释应用失败，必须继续比较 NSS、默认 resolver、指定服务器和应用自身状态。最有区分度的第一步，是对同一个 FQDN 同时执行 `getent` 和 `dig`。

① [诊断] `dig` 正确而 `getent` 返回旧地址

当前证据：

```
dig @192.0.2.53 app.ops.example A → 新地址
getent hosts app.ops.example      → 旧地址
```

优先假设：`/etc/hosts` 存在过期条目，或 NSS 在 DNS 前使用其他来源。

下一条证据：

```
grep -nF 'app.ops.example' /etc/hosts
grep -E '^[[:space:]]*hosts:' /etc/nsswitch.conf
```

最小修复：删除或修正唯一的过期条目；保持合理 NSS 策略。不要通过改变 `hosts:` 行的全局来源顺序来绕开坏数据。

② [诊断] 指定服务器 `dig` 成功而普通 `dig` 失败

假设：当前默认 `resolver` 没有使用该服务器、DNS 顺序不同、本地 `stub` 异常，或到默认服务器的网络路径失败。

下一条证据：

```
cat /etc/resolv.conf
readlink -f /etc/resolv.conf
nmcli -f ipv4.dns,ipv6.dns connection show exam-net
nmcli -f IP4.DNS,IP6.DNS device show
```

只有在确认持久 `profile` 错误后才修改 `NetworkManager`；不要直接覆盖 `resolv.conf` 形成下一次重连必然复发的临时修复。

③ [诊断] 普通 `dig` 成功而 `getent` 完全失败

可能原因：

- `hosts:` 行没有可用 DNS 来源；
- NSS 动作在 DNS 前提前 `return`；
- 应用/NSS 需要的地址族没有数据；
- 系统解析调用与 `dig` 使用了不同本地 `resolver`；
- 名称形式不同，例如一个使用短名，一个使用 FQDN。

下一条证据：固定同一个 FQDN、同一地址族，再比较 `getent ahostsv4/ahostsv6` 和 `dig A/AAAA`。

④ [诊断] `getent` 已正确而长期应用仍使用旧地址

此时不要继续修改 DNS。调查缓存持有者：

```
应用自身 DNS 缓存
→ 长期运行的语言运行时
→ 本地 caching resolver
→ 已建立的长连接或连接池
→ 容器/命名空间中的独立解析配置
```

最小修复：只 `reload`、`restart` 或刷新已经确认持有旧结果的组件。无调查重启整台主机虽然可能暂时恢复，却破坏证据并扩大影响。

⑤ [诊断] 所有名称工具成功而业务仍失败

名称解析层已经基本通过，下一步转向：

```
地址可达性
→ 路由
→ 端口监听
→ firewalld
→ SELinux
→ TLS/证书
→ 应用协议和认证
```

这些对象分别归属第 18、20、21、28/29 章或应用自身。不要继续在名称解析层随机修改。

⑥ [安全边界] 缓存没有统一“万能清理命令”

`glibc` NSS 查询本身不提供一个适用于所有环境的全局缓存清理按钮。缓存可能由 `nscd`、`SSSD`、`systemd-resolved`、`dnsmasq` 或应用维护，而且当前主机未必启用这些组件。必须先识别实际服务，再使用其官方接口。

Cheatsheet ``dig`` 对、``getent`` 错先查 `hosts/NSS`；指定服务器对、默认路径错先查 `resolver`；``getent`` 对、应用错查应用缓存和连接；名称都对则离开 DNS 层。

[经典任务] 配置静态主机名、本地映射和 DNS 搜索域

环境

系统已经存在并激活 NetworkManager connection profile `exam-net`。IPv4 地址、前缀、网关和路由已经按第 18 章完成，不允许修改。当前主机名来自临时网络配置；DHCP 还提供了一个不符合本题要求的 DNS server。

本任务使用文档保留地址，仅用于练习参数：

```
静态主机名: node1.training.example
DNS server: 192.0.2.53
DNS search: training.example
本地映射: 192.0.2.44 repo.training.example repo-local
```

目标终态

- 当前和持久静态主机名均为 `node1.training.example`；
- `exam-net` 只使用题目指定的 IPv4 DNS server；
- 短名 `node1` 可通过 `training.example` 搜索域解析；
- `repo.training.example` 与别名 `repo-local` 通过 `/etc/hosts` 得到 `192.0.2.44`；
- 不改变现有地址、路由和网关；
- 变更后提供配置、当前状态、查询和功能证据。

限制条件

- 不创建或删除 connection profile；
- 不直接把 `/etc/resolv.conf` 当作持久真源；
- 不使用 `chattr` 锁定 resolver 文件；
- 不部署 DNS 服务器；
- 不随意修改 `hosts` 顺序；
- 重新激活连接前评估远程会话中断风险。

验收矩阵

评分对象	推荐证据	成功边界
静态主机名	<code>hostnamectl status</code> 、 <code>/etc/hostname</code>	当前与持久均正确
profile DNS	<code>nmcli connection show exam-net</code>	<code>server/search/ignore-auto-dns</code> 符合题意
当前 resolver	device view 与 <code>/etc/resolv.conf</code>	当前路径已更新
hosts 条目	<code>getent hosts repo-local</code>	NSS 返回 <code>192.0.2.44</code>
FQDN	<code>getent hosts node1.training.example</code>	系统解析得到预期地址
短名	<code>getent hosts node1</code>	search 生效
DNS 原始记录	<code>dig @192.0.2.53 node1.training.example A</code>	指定服务器给出预期 A
持久性	profile 与 <code>/etc/hostname</code>	live 重启留待真实环境复验

[经典任务] 诊断 `dig` 正确但应用得到错误地址

已知状态

系统当前出现以下证据：

```
dig @192.0.2.53 app.ops.example A 取得题目给定的新地址
getent hosts app.ops.example      取得旧地址 192.0.2.99
getent hosts app                  失败
```

现有 `exam-net` profile 缺少 `ops.example` search domain；`/etc/hosts` 中残留旧的 `app.ops.example app` 映射。应用长期运行并使用短名 `app`。

任务要求

- 建立变更前基线；
- 确认 `hosts`：实际顺序；

3. 删除或修正唯一的过期本地映射；
4. 在现有 `exam-net` 中配置 `ops.example search domain`；
5. 不通过把 `dns` 移到 `files` 前面掩盖冲突；
6. 分别验证短名、FQDN、DNS 原始记录和应用功能；
7. 若命令行解析已恢复而应用仍使用旧地址，只刷新已经确认持有缓存的应用组件。

典型错误

- 看到 `dig` 成功便结束调查；
- 删除整个 `/etc/hosts`；
- 为一次故障改变全局 NSS 来源顺序；
- 无调查地重启 NetworkManager 或整台主机；
- 把应用缓存问题继续解释为 DNS server 内容错误；

[参考解答] 经典任务一：配置主机名、本地映射和搜索域

参考解答不是唯一命令序列，而是一条可审计的状态迁移链。以下命令未在可控的 RHEL 9 VM 中执行；必须在真实环境按接口名、profile 状态和题目给定记录调整。

① 调查：确认现有对象和变更风险

```
hostnamectl status
hostname
cat /etc/hostname

nmcli connection show --active
nmcli -f connection.id,connection.interface-name,ipv4.method,ipv4.addresses,ipv4.gateway,ipv4.dns,ipv4.dns-search,ipv4.ignore-auto-dns \
connection show exam-net

readlink -f /etc/resolv.conf
cat /etc/resolv.conf
grep -E '^[:,space:]*hosts:' /etc/nsswitch.conf

grep -nE 'repo-local|repo\.training\.example|192\.0\.2\.44' /etc/hosts
```

调查目的：

- 确认 `exam-net` 确实是活动 profile；
- 保存地址、网关和路由基线，避免越界修改；
- 判断 DNS 是自动获得还是手工指定；
- 确认 resolver 文件由谁管理；
- 避免重复 `hosts` 条目。

② 操作：设置静态主机名

```
hostnamectl set-hostname node1.training.example
```

立即检查：

```
hostname
hostnamectl status
cat /etc/hostname
```

③ 操作：创建最小 `/etc/hosts` 映射

在保留 `localhost` 和其他有效条目的前提下，加入：

```
192.0.2.44 repo.training.example repo-local
```

静态检查：

```
grep -nE 'repo-local|repo\.training\.example|192\.0\.2\.44' /etc/hosts
```

NSS 验证：

```
getent hosts repo.training.example
getent hosts repo-local
```

如果 `getent` 仍返回意外地址，继续查 `hosts`：顺序和其他重复条目，不能用 `dig` 代替此步。

④ 操作：修改现有 NetworkManager profile

题目要求只使用指定自动配置，因此：

```
nmcli connection modify exam-net ipv4.dns "192.0.2.53"
nmcli connection modify exam-net ipv4.dns-search "training.example"
nmcli connection modify exam-net ipv4.ignore-auto-dns yes
```

操作后先读取保存配置：

```
nmcli -f ipv4.dns,ipv4.dns-search,ipv4.ignore-auto-dns connection show exam-net
```

⑤ 应用：重新激活正确的 profile

确认远程中断风险后：

```
nmcli connection up exam-net
```

若操作发生在远程主机，应准备控制台或回退路径。本题禁止修改 IP 和路由，因此激活后应再次确认这些前置状态未变化。

⑥ 分层验证

持久配置：

```
cat /etc/hostname
nmcli -f ipv4.dns,ipv4.dns-search,ipv4.ignore-auto-dns connection show exam-net
```

当前有效状态：

```
hostname
nmcli -f GENERAL.CONNECTION,IP4.ADDRESS,IP4.GATEWAY,IP4.DNS,IP4.DOMAIN device show
cat /etc/resolv.conf
```

NSS：

```
getent hosts repo-local
getent hosts node1.training.example
getent hosts node1
```

DNS 层：

```
dig @192.0.2.53 node1.training.example A
```

功能层：使用题目指定的真实应用或服务请求，不用 `ping` 替代全部业务验收。

⑦ 持久性声明

静态证据只能确认命令、配置路径和逻辑闭环；真实环境仍需重启系统，重新检查 `hostname` 与 `exam-net` 自动激活状态，并重复 `getent`、`dig` 和应用功能测试。

[参考解答] 经典任务二：从 `dig/getent` 差异定位最小修复

① 复现并固定查询对象

首先对完全相同的 FQDN 和地址类型取证：

```
getent ahostsv4 app.ops.example
dig @192.0.2.53 app.ops.example A
```

不要用 `getent app` 与 `dig app.ops.example` 直接比较，因为名称形式不同会引入 `search` 变量。

② 检查 NSS 策略与本地数据

```
grep -E '^[[:space:]]*hosts:' /etc/nsswitch.conf
grep -nE '^(^[[:space:]]*)app([[:space:]]|$)|app\.ops\.example' /etc/hosts
```

若 `files` 位于 `dns` 前，而 `/etc/hosts` 中存在旧地址，差异已经得到解释。此时不应调整全局顺序，而应修正坏数据。

③ 最小修改 `/etc/hosts`

仅删除或修正过期的 `app.ops.example app` 条目，保留 `localhost` 和其他有效映射。完成后立即重复：

```
getent ahostsv4 app.ops.example
```

期望结果应与题目指定 DNS 记录一致。若仍不同，继续检查其他 NSS 来源，不要假定只有 `files` 和 `dns`。

④ 添加短名所需 `search domain`

建立 profile 基线：

```
nmcli -f ipv4.dns,ipv4.dns-search,ipv4.ignore-auto-dns connection show exam-net
```

配置：

```
nmcli connection modify exam-net ipv4.dns-search "ops.example"
```

本题没有说明要抛弃 DHCP DNS，因此不机械设置 `ipv4.ignore-auto-dns yes`。若 profile 还有其他业务所需 `search` 域，应按题意决定追加还是替换，而不是无条件清空。

应用配置：

```
nmcli connection up exam-net
```

⑤ 重新执行分层验证

```
getent hosts app.ops.example
getent hosts app

dig app.ops.example A
dig @192.0.2.53 app.ops.example A
```

判断边界：

- `getent` FQDN 正确：NSS 冲突已修复；
- `getent` 短名正确：search 已生效；
- 普通 `dig` 与指定服务器结果可解释：默认 `resolver` 路径正确；
- 应用仍旧：进入应用缓存或连接调查。

⑥ 只刷新确认持有旧结果的应用组件

先查看应用文档、服务状态和日志，确认它是否缓存 DNS 或保持连接池。随后选择应用支持的 `reload`、连接池刷新或受控 `restart`。不要：

- 清空未知服务的缓存；
- 重启整台主机代替定位；
- 再次修改已经正确的 DNS 数据；
- 把“进程重启成功”扩大为业务功能已经恢复。

⑦ 最终验收

使用应用本身的请求路径验证目标 FQDN、端口和业务响应，并保留时间、查询名称、返回地址和应用日志证据。若名称解析均正确而业务仍失败，按章节边界转入路由、服务、`firewalld`、`SELinux` 或应用层调查。

[本章收束] 从名称字符串回到应用实际使用的地址

本章的核心不是记住一组零散命令，而是形成以下调用路径：

题目中的名称
→ 判断短名、FQDN 或绝对名称
→ 确认本机 hostname 是否只是身份
→ 读取 NSS hosts 策略
→ 比较 /etc/hosts 与 DNS
→ 检查 NetworkManager 持久 DNS 和当前 resolver
→ 分别查询 A、AAAA、PTR
→ 用 getent 验证系统解析
→ 用应用请求验证最终功能

考试中最常见的得分点是正确设置主机名、hosts 和 DNS；最常见的失分点则是混淆工具边界和持久性。工作中还要进一步记录变更基线、控制远程连接风险、识别缓存持有者，并在名称解析已经正确时及时离开本层。

最终应能快速回答：

- 这是本机身份问题，还是名称到地址的问题？
- 当前值和持久值是否一致？
- 应用走的是 NSS，还是直接 DNS？
- /etc/hosts 和 DNS 谁先返回？
- 短名扩展成了哪个 FQDN？
- A、AAAA、PTR 哪一种记录失败？
- dig 成功为什么仍不能证明应用成功，以及修复后哪条证据真正重复了原始失败路径？

主要判断表

看到的现象	先不要下的结论	下一条最有区分度的证据
hostnamectl 正确	DNS 已正确	getent hosts <FQDN> 与 dig <FQDN> A/AAAA
dig 成功	应用一定成功	对同一名称执行 getent，再重复应用请求
getent 返回旧地址	DNS 一定错误	检查 hosts：顺序与 /etc/hosts 冲突
FQDN 成功、短名失败	网络不通	读取 search domain，并查询扩展后的 FQDN
A 成功、PTR 失败	正向记录也错误	dig -x <ADDRESS>，单独确认反向区域
profile 属性正确	当前 resolver 已应用	nmcli device show、/etc/resolv.conf 与实际查询
工具均成功、业务失败	继续改 NSS/DNS	应用日志、缓存持有者、连接和服务层证据

工作方法

1. 固定查询对象。记录名称形式、地址族、DNS server 和时间，避免比较不同问题。
2. 先读取再修改。保存 hostname、hosts：、/etc/hosts、profile 与 resolver 基线。
3. 让差异定位层次。dig 与 getent 的差异比重复执行同一命令更有价值。
4. 只做最小修复。删除过期条目、补齐 search 或修正 profile，不改变全局来源顺序来掩盖坏数据。
5. 沿原失败路径再验证。配置正确、当前状态正确、查询正确和应用功能正确必须分别成立。
6. 区分静态证据与运行验证。配置文件和 connection profile 只能说明持久输入；真实重连、重启和应用缓存刷新仍需在目标 RHEL 9 环境复验。

向下一章交接

本章把目标名称解析成了地址，但 SSH 连接还要回答另一组问题：客户端究竟连接哪台主机、服务端怎样监听、用户怎样认证、主机密钥怎样证明远端身份。第 20 章《SSH 客户端、服务端与密钥认证》将从这里继续；不要把 SSH 主机密钥警告误判为 DNS 解析失败，也不要篡改 /etc/hosts 绕过尚未调查的主机身份变化。

SSH 客户端、服务端与密钥认证

从“连得上”走到“身份可信、认证可证、配置可回退”：把客户端、服务端、密钥和证据放进同一条链。

对象模型 · 操作语义 · 验证 · 诊断 · 经典任务
大字号阅读版

本章阅读导航

先抓住一条主线：客户端先确认服务器是谁，服务器再确认用户是谁；配置、权限和进程状态只是这条双向信任链的输入，独立新连接才是功能终态。

01解析目标确认别名、真实主机、用户与端口

02建立传输区分名称、路由、过滤与监听

03验证主机用可信指纹约束 known_hosts

04选择身份确认私钥、agent 与客户端匹配

05服务端授权检查有效配置、authorized_keys 与路径可信性

06分层验收证明方法、远端用户、主机和策略终态

专题地图

- 知识专题 SSH 连接阶段与双向身份
- 知识专题 主机密钥与 known_hosts
- 操作专题 用户密钥、authorized_keys 与 ssh-copy-id
- 操作专题 客户端匹配与 ssh-agent
- 知识专题 sshd 配置层与有效值
- 操作专题 安全修改、reload 与第二连接
- 诊断专题 从 ssh -vvv 到服务端日志
- 经典任务 建立密钥认证并安全收紧 sshd

阅读时持续回答

1. 当前失败停在传输、主机身份还是用户认证？
2. 客户端最终连接到谁、使用哪个账号和端口？
3. known_hosts 记录的是哪类身份？
4. 私钥、公钥和 authorized_keys 分别位于哪一端？
5. 客户端是否真的发送了预期密钥？
6. sshd 对目标连接实际采用了哪些配置？
7. 权限、所有权和 SELinux 标签分别证明什么？
8. 当前证据能否证明独立新连接的最终功能？

章节边界：网络与路由留给第 18 章，名称解析留给第 19 章，firewalld 留给第 21 章，持久 SELinux 规则留给第 29 章。本章只建立 SSH 所需的接口和验收位置。

第 20 章 SSH 客户端、服务端与密钥认证

远程登录看起来只是一条 `ssh user@host`，但真正完成的系统工作远不止“连接一个端口”。客户端必须先把别名还原为真实目标并建立传输，再验证服务器主机身份；服务器随后才判断来访者是否可以作为目标用户登录；认证通过后，双方才创建交互 Shell 或远程命令通道。只要把这些阶段混在一起，就会出现高频误判：端口可达便开始重建用户密钥，看到密码提示便忽略主机身份，登录成功却没有发现公钥失败后回退到了密码，或修改 `sshd` 后只看服务仍为 `active` 就宣布完成。

本章以“双向身份链”为中心组织对象。客户端侧保存目标、主机信任、用户私钥和 agent 状态；服务端侧保存主机私钥、用户授权、配置层、路径权限和运行状态。讲义将持续区分三种证据：配置文本说明管理员写了什么，有效配置与进程状态说明软件当前采用什么，独立功能测试才说明指定用户能否按指定方法连接到指定主机。

本章不完整讲解地址、路由、DNS、防火墙或 SELinux 策略。遇到这些对象时，只指出 SSH 链路需要读取什么证据，并把完整修改方法交还给相邻章节。这样可以避免用“关闭防火墙、关闭 SELinux、删除整个 `known_hosts`”之类捷径掩盖真正的失败层。

概念

主机身份回答“正在连接的服务器究竟是谁”。服务端使用 `/etc/ssh/ssh_host_*_key` 中的主机私钥证明身份，客户端将核验后的主机公钥记录在 `known_hosts`。主机身份验证发生在用户认证之前；即使账号密码或用户私钥正确，也不应把凭据交给一个尚未确认身份的对端。可观察证据是首次连接指纹、既有 `known_hosts` 记录和主机密钥变化警告。

概念

用户身份回答“来访者是否有权作为目标账号登录”。公钥认证中，客户端持有私钥并完成签名，服务端目标账号只保存相应公钥；密码认证则由服务端验证口令。用户身份与主机身份是两条独立信任链：`authorized_keys` 不能证明服务器是谁，`known_hosts` 也不能授权某个用户登录。

概念

`known_hosts` 是客户端保存的主机信任记录，而不是“连接缓存”。首次连接时，记录必须来自可信指纹核验；后续密钥变化可能来自重装、轮换、目标错误或中间人攻击。删除条目只能清除旧记录，不能证明新密钥可信，因此安全顺序始终是先调查变化，再核对新指纹，最后只更新相关目标。

概念

`authorized_keys` 是服务端账号的授权入口。一行公钥表示该账号允许对应私钥持有者尝试认证，但文件存在并不等于认证一定成功：`sshd` 还要按有效配置找到该文件，并确认 `home`、`ssh`、授权文件的所有权和可写权限可信，RHEL 上还可能受到 SELinux 标签影响。

概念

客户端配置匹配把命令行目标映射为真实主机、用户、端口和候选身份。`Host` 可以是别名或模式，`HostName` 才是实际目标；命令行、用户配置和系统配置共同参与取值。不能只靠肉眼阅读某一段配置，应该用 `ssh -G` 查看完成 `Host` 与 `Match` 计算后的最终选择。

概念

`sshd` 配置层包含主配置、`Include` 引入的 drop-in 和可能改变特定连接的 `Match` 块。管理员看到的某一行不一定是守护进程采用的值；`sshd -t` 只能证明配置可解析，`sshd -T` 和 `sshd -T -C ...` 才分别展示全局及特定连接上下文的有效配置。

概念

认证阶段位于传输建立和主机验证之后。客户端可能依次尝试多种方法，公钥失败后仍可能回退到 `password` 或 `keyboard-interactive`。因而“进入了 Shell”不能单独证明公钥认证成功；需要限制回退、读取 `ssh -vvv` 的关键阶段，并用远端 `id`、`hostname` 和退出状态完成终态确认。

操作语义速查区

这里先建立接口地图。后续专题解释为什么这样用、操作改变哪个对象，以及每条证据能够证明到哪一层。

ssh

SYNOPSIS

```
ssh [-v] [-F configfile] [-i identity_file] [-J destination]
    [-p port] [-o option] [user@]hostname [command [argument ...]]
```

建立 SSH 客户端会话，完成目标选择、主机验证、用户认证，并请求交互 Shell 或远程命令。

`-i identity_file`

显式指定候选私钥；它不自动限制 agent 中其他身份。

`-p port`

连接非默认服务端端口；端口放行和 SELinux 端口类型分别交给后续章节。

`-F configfile`

使用指定客户端配置文件，适合隔离测试。

`-J destination`

通过跳转主机建立连接；本章只认识接口，不展开复杂跳板拓朴。

`-v / -vv / -vvv`

逐级增加调试信息。诊断时只抓连接目标、主机密钥、密钥 offer、服务端接受和最终方法等关键阶段。

ssh-keygen

SYNOPSIS

```
ssh-keygen -t type [-b bits] [-C comment] [-f output_keyfile]
ssh-keygen -lf public_key
ssh-keygen -F host | ssh-keygen -R host
```

生成和管理用户或主机密钥，并以指纹识别密钥材料；同一工具也可查询或删除 `known_hosts` 中的目标记录。

-t type

显式选择密钥算法。Ed25519 不适用于 FIPS 模式；不要把版本默认值当成稳定考试结论。

-b bits

为支持位数设置的算法指定长度，例如 RSA。

-f path

指定输出或输入文件，避免覆盖既有默认密钥。

-C comment

写入便于审计和轮换的注释，不参与加密验证。

-l

显示公钥或已知主机条目的指纹。

-F / -R

查找或移除指定主机记录；移除前必须完成可信核验。

ssh-copy-id

SYNOPSIS

```
ssh-copy-id [-i identity_file.pub] [user@]hostname
```

使用已有可用认证路径登录远端，并把指定公钥追加到目标账号的授权位置。它不会复制私钥，也不会自动修改 `sshd` 认证策略。

-i identity_file.pub

明确选择要安装的公钥文件。指向私钥是对象方向错误。

user@hostname

决定公钥授权给哪个服务端账号；连接成功后仍要核对远端身份和授权文件状态。

ssh-agent / ssh-add

SYNOPSIS

```
eval "$(ssh-agent -s)"
ssh-add [-t life] private_key
ssh-add -l | ssh-add -L | ssh-add -d private_key | ssh-add -D
```

agent 是客户端本地签名服务；`ssh-add` 管理当前 agent 中已经解锁的身份。agent 中存在身份不代表服务端账号已经授权。

SSH_AUTH_SOCK

当前 Shell 访问 agent 的 Unix socket；变量为空或 socket 不可达时，客户端无法使用该 agent。

-l / -L

分别列出指纹或公钥内容。

-d / -D

移除一个或全部 agent 身份；不删除磁盘私钥，也不撤销服务端授权。

-t life

为缓存身份设置有限生命周期。

客户端匹配：ssh_config 与 ssh -G

SYNOPSIS

```
ssh -G [-F configfile] destination

Host alias
  HostName server.example.com
  User operator
  Port 22
  IdentityFile ~/.ssh/operator_key
  IdentitiesOnly yes
```

客户端配置把别名和匹配条件转换为实际连接参数；`ssh -G` 输出计算后的结果，适合证明客户端最终选择。

Host

定义别名或匹配模式；更具体的规则应优先于通用规则。

Match

按连接条件应用客户端配置；使用时必须通过有效配置而不是肉眼猜测。

HostName / User / Port

分别指定实际目标、远端账号和端口。

IdentityFile

增加候选身份文件。

IdentitiesOnly yes

限制客户端使用显式配置的身份，减少多密钥误选。

服务端配置：sshd_config / drop-in / Match

SYNOPSIS

```
Include /etc/ssh/sshd_config.d/*.conf
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no
AuthorizedKeysFile .ssh/authorized_keys
Match User operator
...
```

服务端配置决定监听、认证方法、账号边界和授权文件位置。主配置、drop-in 顺序与 `Match` 上下文共同决定最终值。

Include

引入其他配置源；必须检查它在主文件中的实际位置。

Match

让一部分指令只作用于指定用户、组或地址。

PermitRootLogin

控制 root 远程登录边界。

PasswordAuthentication

控制 SSH password 方法；不能自动代表 keyboard-interactive 也已关闭。

PubkeyAuthentication

控制用户公钥认证。

AuthorizedKeysFile

指定服务端从何处读取用户授权公钥。

验证链：sshd -t/-T、systemctl、ss、journalctl

SYNOPSIS

```
sshd -t
sshd -T
sshd -T -C user=name,host=host,addr=remote,laddr=local,lport=22
systemctl reload sshd
ss -lntp
journalctl -u sshd --since '-5 min' --no-pager
```

这一组命令分别证明配置可解析、有效值、特定连接上下文、服务应用、监听状态和服务端认证证据；它们必须分层使用。

sshd -t

证明配置可解析及关键文件满足基本检查；不证明已 reload。

sshd -T

输出全局有效配置。

sshd -T -C ...

模拟连接条件并应用 `Match`。

systemctl reload sshd

让新连接采用新配置；执行前必须先通过静态检查并保留旧会话。

ss -lntp

证明本机监听；不证明外部防火墙已放行。

journalctl -u sshd

解释服务端接受或拒绝；应与客户端最小复现按时间关联。

[知识专题] SSH 连接不是一步：先连接、再认主机、再认用户

排错 SSH 时，最有效的切入不是先改配置，而是先判断失败停在哪个阶段。名称解析、TCP 连接、算法协商、主机身份、用户认证和会话创建使用不同对象，也产生不同证据。客户端 `ssh -vvv` 的价值正是在于把一条“连接失败”拆回这些阶段。

① [知识点] TCP 可达只证明传输入口存在

客户端首先把目标名称解析为地址，然后尝试连接指定端口。`Connection timed out` 通常表示请求没有获得及时响应，可能涉及路由、过滤、地址错误或中间网络；`Connection refused` 通常表示目标返回拒绝，常见于该地址端口没有监听。它们都发生在用户认证之前，不能通过重建用户密钥来修复。

网络地址和路由的完整调查属于“NetworkManager、IPv4 地址与路由”；名称解析属于“主机名、NSS 与 DNS 名称解析”；防火墙策略属于“firewalld 与完整服务访问链”。本章只读取这些前置事实，不复制相邻章节。

② [知识点] 主机身份验证发生在用户认证之前

建立加密传输后，服务器出示主机公钥并证明持有对应私钥。客户端将收到的主机密钥与可信记录比较：首次连接时通常没有记录，需要通过独立可信渠道核对指纹；后续连接应与既有记录一致；若密钥变化，客户端必须把它视为身份异常，而不是自动接受。

主机身份验证解决的是中间人问题。即使用户密码或私钥完全正确，也不应把凭据交给一个身份未确认的服务器。

③ [知识点] 用户认证可能有多种方法并按次序尝试

服务器公布当前允许继续尝试的方法，例如 `publickey`、`password` 或 `keyboard-interactive`。客户端按配置和可用凭据尝试。公钥失败后若密码仍允许，客户端可能继续提示账号密码并最终登录成功。因此“没有输入账号密码”不是唯一判断，“成功进入 Shell”也不能证明一定使用了公钥；应读取 `ssh -vvv` 中最终的 `Authenticated ... using "publickey"` 等证据，或在测试时显式禁止回退。

④ [知识点] 认证完成后才创建交互 Shell 或远程命令

以下两条命令都建立 SSH 会话，但请求的远端行为不同：

```
ssh operator@servera
ssh operator@servera 'id && hostname'
```

第一条请求交互 Shell；第二条执行指定命令并把标准输出、标准错误和退出状态传回本地。考试验收更适合使用第二种形式，因为它可以明确证明远端用户和主机身份，而不依赖提示符外观。

[Cheatsheet] `timeout/refused` 属于传输层；`host key` 提示属于服务器身份层；`Permission denied` 属于用户认证层；进入 Shell 后仍要用 `id`、`hostname` 或指定命令确认功能对象。

[知识专题] 主机密钥与 `known_hosts`：客户端如何记住服务器是谁

主机密钥不是登录用户的密钥。它属于 SSH 服务端实例，通常在系统安装或 OpenSSH 服务初始化时生成。客户端保存的是主机公钥或其哈希化主机名记录，用于检测之后连接到的服务器身份是否变化。

① [知识点] 服务端主机密钥与用户密钥是两条独立信任链

典型主机密钥文件位于：

```
/etc/ssh/ssh_host_ed25519_key
/etc/ssh/ssh_host_ed25519_key.pub
/etc/ssh/ssh_host_rsa_key
/etc/ssh/ssh_host_rsa_key.pub
```

没有 `.pub` 后缀的是服务端主机私钥，必须只由特权进程读取；带 `.pub` 的文件可用于通过控制台或其他可信渠道发布指纹。用户自己的 `~/.ssh/id_*` 不应替代主机密钥，主机密钥也不应复制到用户的 `authorized_keys`。

② [查询] 在服务端核对主机公钥指纹

在能够可信访问服务器本地控制台的前提下，可读取某个主机公钥的指纹：

```
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
ssh-keygen -lf /etc/ssh/ssh_host_rsa_key.pub
```

客户端首次连接时显示的算法和 SHA256 指纹应与服务器本地证据一致。电话、受控工单、控制台画面或已经可信的管理渠道都可作为带外核验方式。`ssh-keyscan` 只能从网络抓取当前对端提供的公钥，不能独立证明这个对端就是预期服务器，因此不能把“抓到的值”和“可信身份”混为一谈。

③ [知识点] `known_hosts` 有用户级和系统级两个入口

常见文件为：

```
~/ssh/known_hosts
/etc/ssh/ssh_known_hosts
```

用户级文件只影响当前用户；系统级文件可由管理员预置组织信任。条目通常包含主机标识、密钥算法和公钥材料。启用主机名哈希时，肉眼不再能直接搜索主机名，但 `ssh-keygen -F` 仍可查找。

非默认端口通常以 `[host]:port` 作为记录目标，例如：

```
ssh-keygen -F '[servera.example.com]:2222'
```

④ [操作] 查询和删除指定主机记录，而不是清空整个文件

```
ssh-keygen -F servera.example.com
ssh-keygen -R servera.example.com
ssh-keygen -R '[servera.example.com]:2222'
```

`-F` 用于查询；`-R` 删除与指定目标匹配的条目，并通常保留备份。删除操作不是验证手段。正确顺序是：先确认服务器确实重装、迁移或轮换了主机密钥；再通过可信渠道核对新指纹；最后删除旧条目并重新连接。

⑤ [边界] 不使用“关闭检查”处理主机密钥异常

`StrictHostKeyChecking=no`、把 `UserKnownHostsFile` 指向 `/dev/null`，或直接删除整个 `known_hosts`，都会降低或消除身份连续性。这些配置可能用于明确受控的临时场景，但不应成为考试和日常排错的默认答案。若出现 `REMOTE HOST IDENTIFICATION HAS CHANGED`，最小安全动作是停止连接、核对变更原因和新指纹，再只更新相关条目。

[Cheatsheet] 主机私钥留在服务端；客户端保存主机公钥记录；首次连接先核验指纹；变化时先调查再 `ssh-keygen -R`；`ssh-keyscan` 是采集工具，不是独立信任根。

[操作专题] 用户密钥对：生成、保护、识别和轮换

用户密钥对用于证明客户端持有某项身份。私钥承担签名能力，公钥承担可分发的验证材料。稳定操作应显式指定算法和文件路径，避免覆盖既有默认密钥，也避免依赖不同 RHEL 9 次版本或安全模式下可能变化的默认算法。

① [操作] 生成专用密钥对并明确文件名

非 FIPS 环境可使用 Ed25519：

```
ssh-keygen -t ed25519 -f ~/.ssh/rhcsa20_operator -C 'operator@workstation'
```

需要兼容 FIPS 或明确要求 RSA 时，可使用：

```
ssh-keygen -t rsa -b 3072 -f ~/.ssh/rhcsa20_operator -C 'operator@workstation'
```

命令生成两个文件：

```
~/ssh/rhcsa20_operator      私钥
~/ssh/rhcsa20_operator.pub  公钥
```

`-t` 选择算法，`-b` 对适用算法设置位数，`-f` 指定输出路径，`-C` 写入便于识别的注释。实际使用前先确认目标路径不存在，避免覆盖仍在其他服务器上生效的私钥。

② [知识点] 密语与文件权限解决不同风险

私钥密语加密私钥内容，即使文件副本泄露，攻击者仍需解锁密语；文件权限限制本机其他账号读取。两者应同时考虑。典型权限为：

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/rhcsa20_operator
chmod 644 ~/.ssh/rhcsa20_operator.pub
```

公钥不要求保密，但仍应防止被无意改写。私钥权限过宽时，OpenSSH 客户端可能拒绝使用它。

③ [操作] 用指纹识别密钥而不是只看文件名

```
ssh-keygen -lf ~/.ssh/rhcsa20_operator.pub
ssh-keygen -y -f ~/.ssh/rhcsa20_operator | ssh-keygen -lf -
```

第一条读取公钥指纹；第二组命令从私钥导出公钥后计算指纹，可用于确认一对文件是否对应。执行第二组命令可能要求输入私钥密码。不要把私钥内容输出到终端、聊天、工单或共享目录。

④ [操作] 改变私钥密码而不生成新的身份

```
ssh-keygen -p -f ~/.ssh/rhcsa20_operator
```

这会重新保护同一私钥，公钥身份和指纹不变。若任务要求真正轮换身份，应生成新密钥对、先把新公钥部署并验证，再移除旧授权；不能把“改密码”当成“轮换公钥”。

⑤ [安全边界] 丢失私钥与泄露私钥的处理不同

私钥丢失且没有备份时，无法从公钥恢复私钥，应通过其他可信管理入口部署新公钥。私钥可能泄露时，必须把对应公钥从所有服务器的授权位置撤销，并生成新身份；仅修改本地文件权限或密码不足以撤销已经复制出去的私钥副本。

[Cheatsheet] `-f` 避免覆盖；私钥 600、公钥可读；指纹识别身份；改密码不改指纹；泄露时撤销远端公钥，丢失时通过其他入口部署新公钥。

[操作专题] `authorized_keys` 与 `ssh-copy-id`：把公钥变成账号授权

公钥文件存在于客户端并不等于远端账号已授权。服务端必须在目标账号的授权文件中出现匹配公钥，而且路径、所有权、权限、SELinux 标签和 `sshd` 有效策略都必须允许读取。

① [知识点] `authorized_keys` 的一行是一项授权记录

典型行由可选限制、密钥类型、Base64 公钥主体和注释构成：

```
from="192.0.2.0/24",restrict ssh-ed25519 AAAA... operator@workstation
```

普通考试任务通常只需写入“密钥类型 + 公钥主体 + 注释”。`from=`、`command=`、`restrict` 等属于更细的授权约束，理解其位置即可，不在本章展开复杂堡垒机策略。注释不参与加密验证，可用于标识来源和轮换批次。

② [操作] 使用 `ssh-copy-id` 安全追加指定公钥

```
ssh-copy-id -i ~/.ssh/rhcsa20_operator.pub operator@servera
```

`-i` 应指向公钥文件。`ssh-copy-id` 通常通过现有密码或其他已可用认证方式登录远端，并把尚未存在的公钥追加到目标账号的默认授权位置。它不会把私钥复制到服务器，也不会自动修改 `PasswordAuthentication` 或 `PubkeyAuthentication`。

③ [操作] 手工部署时先创建目录，再追加而不是覆盖

在无法使用 `ssh-copy-id`、但已有可信管理入口时，可在服务端以目标用户身份执行：

```
install -d -m 700 ~/.ssh
cat /tmp/operator.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

这里的 `/tmp/operator.pub` 只是受控传入的公钥示例。不要使用 `>` 覆盖已有 `authorized_keys`，除非任务明确要求重建且已保存现有授权。写入后应删除临时公钥副本，并核对重复行和注释。

④ [验证] 强制只测试公钥认证，防止密码回退造成假成功

```
ssh -i ~/.ssh/rhcsa20_operator \
-o IdentitiesOnly=yes \
-o PreferredAuthentications=publickey \
-o PasswordAuthentication=no \
-o KbdInteractiveAuthentication=no \
operator@servera 'id && hostname'
```

此命令显式选择私钥并禁止两类交互式口令回退。成功后仍应确认远端 `id` 和 `hostname`，并用 `ssh -vvv` 查看最终认证方法。

⑤ [边界] 授权文件中有公钥仍可能被拒绝

高频原因包括：连接的用户名错误；用户 `home`、`.ssh` 或 `authorized_keys` 所有权不正确；路径可被其他用户写入；SELinux 标签异常；`AuthorizedKeysFile` 指向其他位置；`Match` 块对该用户关闭公钥认证；账号被锁定、Shell 不可用或被 `AllowUsers` / `DenyUsers` 拒绝。调查时应读取有效配置和服务端日志，而不是重复复制同一公钥。

[Cheatsheet] `ssh-copy-id -i` 指向 `.pub`；授权是追加不是覆盖；测试时禁用密码回退；“公钥已写入”只证明文件内容，不证明 `sshd` 会读取和接受。

[操作专题] 客户端配置与匹配：让别名、用户、端口和身份可预测

当命令行开始反复出现 `-p`、`-i`、`-o` 和长主机名时，应把稳定连接参数写入客户端配置。但配置文件不是简单的“最后一行覆盖”，OpenSSH 对大多数参数采用最先获得的值，因此规则顺序直接影响结果。

① [知识点] 客户端配置来源按优先级读取

主要来源为：

```
命令行选项
→ ~/.ssh/config
→ /etc/ssh/ssh_config 及其 Include
```

对于大多数参数，一旦取得值，后续来源不再覆盖。因此用户配置可以覆盖系统默认，命令行可以覆盖用户配置；同一配置文件中，应把更具体的 `Host` 块放在通用 `Host *` 之前。

② [操作] 为明确目标建立别名

```
Host servera-operator
    HostName servera.example.com
    User operator
    Port 22
    IdentityFile ~/.ssh/rhcsa20_operator
    IdentitiesOnly yes

Host *
    ServerAliveInterval 60
```

`Host` 是客户端匹配模式或别名；`HostName` 才是实际连接目标；`User` 指定远端账号；`Port` 指定服务端端口；`IdentityFile` 指定候选私钥；`IdentitiesOnly yes` 限制客户端只使用显式配置的身份，而不是把 agent 中所有密钥都依次尝试。

③ [验证] 保护用户配置文件并检查最终值

```
chmod 600 ~/.ssh/config
ssh -G servera-operator | grep -E '^(hostname|user|port|identityfile|identitiesonly) '
```

`ssh -G` 在完成 `Host` 和 `Match` 计算后输出有效客户端配置。它能证明客户端将使用哪些参数，但不能证明服务端在线、主机密钥可信或用户认证会成功。

④ [知识点] 命令行 `-o` 适合一次性验证，不应替代长期配置

```
ssh -o IdentitiesOnly=yes -o PasswordAuthentication=no servera-operator
```

`-o key=value` 可快速覆盖或测试单项配置，尤其适合排错和验收。若相同选项长期重复使用，应回到 `~/.ssh/config`，并用 `ssh -G` 验证，避免多个脚本各自携带不一致参数。

⑤ [诊断] 多把密钥时先确认“客户端选择了什么”

客户端可能从默认文件、配置中的多个 `IdentityFile`、PKCS#11 设备和 agent 获取身份。若服务端在认证尝试上限前始终没看到正确密钥，应检查：

```
ssh -G servera-operator
ssh-add -l
ssh -vvv servera-operator
```

重点寻找实际读取的配置文件、`identityfile` 列表、`Offering public key` 和最终认证方法。不要先在服务器端反复改权限，因为客户端可能根本没有发送目标公钥。

[Cheatsheet] 命令行最高，用户配置早于系统配置；多数参数 first-value-wins；具体 `Host` 在前，`Host *` 在后；`ssh -G` 看选择，`ssh -vvv` 看执行。

[操作专题] `ssh-agent` 与 `ssh-add`：在本地会话中安全使用受密语保护的私钥

`ssh-agent` 不会把私钥上传到服务器。它在本地运行，通过 Unix socket 接收签名请求，使客户端可以使用已经解锁的私钥，而不必每次重新输入密语。agent 的存在、环境变量和已载入身份是三个需要分别确认的状态。

① [操作] 在当前 Shell 启动 agent 并导入环境

```
eval "$(ssh-agent -s)"
printf '%s\n' "$SSH_AUTH_SOCK"
```

`ssh-agent -s` 输出适合 Bourne Shell 的环境变量设置，`eval` 将其导入当前 Shell。`SSH_AUTH_SOCK` 应指向 agent 的本地 socket。桌面会话、登录管理器或其他工具可能已经启动 agent，重复启动会产生多个彼此独立的身份集合。

② [操作] 载入、列出和移除身份

```
ssh-add ~/.ssh/rhcsa20_operator
ssh-add -l
ssh-add -L
ssh-add -d ~/.ssh/rhcsa20_operator
ssh-add -D
```

`-l` 列出指纹，适合确认载入了哪一把密钥；`-L` 输出公钥；`-d` 删除指定身份；`-D` 删除全部身份。删除 agent 中的身份不会删除磁盘上的私钥文件。

③ [操作] 为缓存设置有限生命周期

```
ssh-add -t 1h ~/.ssh/rhcsa20_operator
```

该身份在约一小时后从 agent 中自动移除。生命周期控制适合临时管理会话，但它不是远端授权撤销：即使 agent 中已删除，私钥文件仍可再次载入；真正撤销访问还需要从服务器删除对应公钥。

④ [安全边界] agent forwarding 不是普通密钥认证的必要条件

`ForwardAgent yes` 允许远端会话借助本地 agent 继续向其他主机认证。远端不会直接得到私钥文件，但被入侵的远端环境可能在会话期间滥用转发的 agent。RHCSA 普通直连任务不需要开启 agent forwarding；需要跳转时优先评估 `ProxyJump` 等不暴露 agent 的方案，并把复杂跳板策略留给专门设计。

[Cheatsheet] agent 是本地签名服务；`SSH_AUTH_SOCK` 证明当前 Shell 能否访问它；`ssh-add -l` 看身份；删除 agent 身份不删除文件，也不撤销服务端公钥。

[知识专题] `sshd` 配置源、加载顺序和连接上下文

服务端故障经常不是“某一行写错”，而是管理员看的文件和守护进程实际使用的有效值不同。RHEL 9 通常在主配置中包含 `/etc/ssh/sshd_config.d/*.conf`，但必须检查当前系统的 `Include` 位置和目录内容。OpenSSH 对大多数关键字采用最先取得的值，因此不能套用其他软件“最后一行覆盖”的习惯。

① [知识点] 守护进程、主机密钥和用户授权是三类对象

`sshd` 负责监听、协商和认证；主机私钥证明服务器身份；目标用户的授权文件决定哪些用户公钥可用。`sshd.service` 为 `active` 不表示主机密钥一定有效，也不表示某个用户的 `authorized_keys` 可读。

② [查询] 先读取实际配置入口和 drop-in 列表

```
grep -nE '^[[:space:]]*(Include|Match|Port|PasswordAuthentication|KbdInteractiveAuthentication|PubkeyAuthentication|PermitRootLogin|AuthorizedKeysFile|AllowUsers|
\
/etc/ssh/sshd_config
find /etc/ssh/sshd_config.d -maxdepth 1 -type f -name '*.conf' -printf '%f\n' | sort
```

第一条建立主文件基线并定位 `Include` 与 `Match`；第二条显示候选 drop-in 的字典序。不要在未读取现有文件时创建一个“99-local.conf”并假设数字最大就一定覆盖，因为 `first-value-wins` 可能使早期文件先锁定参数。

③ [知识点] `Match` 让有效值依赖连接上下文

`Match User`、`Match Group`、`Match Address` 等可对特定连接改变部分指令。进入 `Match` 块后，后续允许的指令只作用于匹配连接，直到新的 `Match` 或文件结束。把全局指令误写进条件块，或只运行不带上下文的 `sshd -T`，都可能漏掉目标用户的实际配置。

④ [验证] 使用 `sshd -t`、`-T` 和 `-T -C` 分别回答三个问题

```
sshd -t
sshd -T | grep -E '^(port|pubkeyauthentication|passwordauthentication|kbdinteractiveauthentication|permitrootlogin|authorizedkeysfile) '
sshd -T -C user=operator,host=servera.example.com,addr=192.0.2.50,laddr=192.0.2.10,lport=22 \
| grep -E '^(pubkeyauthentication|passwordauthentication|kbdinteractiveauthentication|permitrootlogin|authorizedkeysfile) '
```

- `sshd -t`：配置能否解析，关键文件是否满足启动检查；
- `sshd -T`：输出全局有效配置；

- `sshd -T -C ...`：按给定连接参数应用 `Match` 后输出有效配置。

静态检查成功不表示守护进程已经重载，也不表示真实网络登录成功。

⑤ [知识点] 认证相关指令必须按目标拆开验证

常见指令包括：

指令	回答的问题
<code>PubkeyAuthentication</code>	是否允许公钥用户认证
<code>PasswordAuthentication</code>	是否允许 SSH password 方法
<code>KbdInteractiveAuthentication</code>	是否允许键盘交互式认证
<code>PermitRootLogin</code>	root 通过 SSH 的允许范围
<code>AuthorizedKeysFile</code>	到哪里读取用户授权公钥
<code>AllowUsers</code> / <code>DenyUsers</code>	哪些用户可进入认证链

“关闭密码登录”不能只凭一行注释判断，也不能把 `PasswordAuthentication no` 自动扩大为所有交互式认证都关闭。应查看有效值和目标连接上下文。

[Cheatsheet] 先找 `Include` 和 `Match`；OpenSSH 多数参数 first-value-wins；`-t` 看可解析，`-T` 看有效值，`-T -C` 看目标连接；三者都不替代真实登录。

[操作专题] 安全修改 `sshd`：先保留入口，再静态检查，再开第二条连接

SSH 是远程管理入口，自身变更比普通服务更需要回退意识。最危险的做法不是某个参数选错，而是在只有一条远程会话时同时关闭现有认证方法、重载服务并退出会话。安全流程必须把基线、最小变更、静态检查、当前状态和独立新连接分开。

① [操作] 变更前记录基线和恢复入口

```
systemctl is-active sshd
ss -lntp | grep -E '(:22\b|sshd)'
```

```
sshd -T > /root/sshd-effective.before
cp -a /etc/ssh/sshd_config.d /root/sshd_config.d.before
```

同时确认至少一个恢复入口：保持当前 SSH 会话不退出；具备 BMC/控制台；或有另一个已验证管理员账号。备份路径只是任务示例，真实环境应遵守本地变更管理要求。

② [操作] 使用最小 drop-in 表达目标状态

例如建立一个内容明确的本地文件：

```
# /etc/ssh/sshd_config.d/10-local-auth.conf
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no
```

文件名前缀必须结合现有目录和 first-value-wins 规则选择。写入后不立即 reload，先读取文件并运行静态检查。不要复制整份发行版主配置到 drop-in，也不要同时修改多个来源而无法判断谁生效。

③ [验证] 先检查语法，再检查有效值和目标用户上下文

```
sshd -t
sshd -T | grep -E '^(permitrootlogin|pubkeyauthentication|passwordauthentication|kbdinteractiveauthentication) '
```

```
sshd -T -C user=operator,host=servera.example.com,addr=192.0.2.50,laddr=192.0.2.10,lport=22 \
| grep -E '^(permitrootlogin|pubkeyauthentication|passwordauthentication|kbdinteractiveauthentication) '
```

若有效值与文件内容不同，应调查更早配置来源和 `Match`，而不是继续增加重复指令。

④ [操作] 使用 reload 应用后检查服务和监听

```
systemctl reload sshd
systemctl is-active sshd
ss -lntp | grep sshd
journalctl -u sshd --since '-5 min' --no-pager
```

`reload` 使新连接使用新配置，通常不会终止既有会话。服务仍为 `active` 和端口仍监听只证明守护进程层，不证明目标用户能按预期认证。

⑤ [验证] 从独立客户端开第二条连接，成功后才关闭旧入口

```
ssh -i ~/.ssh/rhcsa20_operator \  
-o IdentitiesOnly=yes \  
-o PreferredAuthentications=publickey \  
-o PasswordAuthentication=no \  
-o KbdInteractiveAuthentication=no \  
operator@servera 'id && hostname'
```

另行测试不应允许的路径，例如 root 直登或 password-only。只有允许路径成功、禁止路径按预期失败、服务端日志无异常后，才退出旧管理会话。若新连接失败，应使用旧会话恢复 drop-in、重新运行 `sshd -t` 并 reload。

[Cheatsheet] 旧会话不退出；最小 drop-in；`-t` 后才 reload；active+listen 仍不是登录终态；必须用第二连接验证允许和禁止路径。

[知识专题] 权限、所有权、`StrictModes` 与 SELinux：为什么“公钥明明在文件里”仍失败

OpenSSH 不只检查公钥内容，还检查授权路径是否可信。若其他用户可以替换目标用户的 home、`.ssh` 或 `authorized_keys`，公钥认证就失去意义。RHEL 的 SELinux 又增加了对象标签这一独立检查层，因此 Unix mode 正确并不保证标签正确，标签正确也不替代所有权和 mode。

① [知识点] 客户端私钥和配置文件必须防止本机他人读取或改写

保守且常用的设置为：

```
chmod 700 ~/.ssh  
chmod 600 ~/.ssh/config  
chmod 600 ~/.ssh/rhcsa20_operator  
chmod 644 ~/.ssh/rhcsa20_operator.pub
```

私钥过宽时客户端通常拒绝使用。配置文件可改变目标、用户、代理和命令，虽然不含私钥，也不应允许其他用户改写。

② [查询] 在服务端逐层检查路径，而不是只看最终文件

```
namei -l /home/operator/.ssh/authorized_keys  
stat -c '%U:%G %a %n' /home/operator /home/operator/.ssh /home/operator/.ssh/authorized_keys
```

重点是：目标用户和组是否正确；`.ssh` 与授权文件是否归目标用户所有；路径是否可被组或其他用户写入。常用保守设置是 `.ssh` 为 700、`authorized_keys` 为 600。home 不必机械改为 700，但不应被不可信用户写入。

③ [知识点] `StrictModes` 检查的是授权路径的可信性

`StrictModes yes` 时，`sshd` 在接受登录前检查用户文件和 home 的所有权、权限。关闭 `StrictModes` 会掩盖根因并降低安全性，不应作为默认修复。更合理的做法是沿路径修正 owner 和可写位，再重新触发一次登录并查看日志。

④ [验证] 检查并恢复默认 home 路径的 SELinux 标签

```
ls -ldZ /home/operator /home/operator/.ssh /home/operator/.ssh/authorized_keys  
restorecon -Rv /home/operator/.ssh
```

`restorecon` 根据现有策略恢复默认标签，适合文件被复制、解压或手工创建后标签异常的情况。不要用 `chcon` 作为长期默认修复，因为其临时标签可能在重新标记时丢失。

⑤ [边界] 非标准 home、NFS home 和自定义端口需要转入专门章节

若 home 位于非标准路径，可能需要持久 `semanage fcontext` 规则；若通过 NFS 提供 home，可能涉及 SELinux Boolean 和网络文件系统语义；若改变 SSH 监听端口，可能涉及 `ssh_port_t` 和 `firewalld`。当前章只建立“权限与标签必须分别验证”的接口，完整规则归“SELinux 文件规则、端口类型与 Boolean”和“firewalld 与完整服务访问链”。

[Cheatsheet] `namei -l` 看每一级路径；`.ssh` 700、授权文件 600 是保守基线；不关闭 `StrictModes`；`ls -Z` 与 mode 分开；默认 home 标签用 `restorecon`，非标准路径转 SELinux 章节。

[诊断专题] 从客户端到服务端：用最有效区分度的下一条证据推进

SSH 故障适合严格按阶段调查。每次只提出一个假设，再选能区分该假设的证据。不要同时删除 `known_hosts`、重新生成密钥、改 `sshd_config`、放宽权限和关闭 SELinux，因为这样即使恢复也无法知道真正原因。

① [诊断] 症状：名称无法解析或连接目标错误

当前证据：`ssh -vvv` 在连接前显示的目标名称和地址。假设：客户端别名、`HostName` 或名称解析把连接导向错误服务器。下一条证据：`ssh -G alias` 查看最终 `hostname`、`user`、`port`；再引用第 19 章核对 NSS/DNS。最小修复：修正客户端配置或名称记录，不改密钥。再验证：`ssh -vvv` 中目标地址与预期一致。

② [诊断] 症状：timeout 或 connection refused

当前证据：客户端是否已打印 `Connecting to ... port ...`。假设：`timeout` 偏向路径或过滤；`refused` 偏向目标未监听或端口错误。下一条证据：在服务器本地运行 `systemctl is-active sshd` 与 `ss -lnpt`；防火墙转第 21 章。最小修复：启动或修正监听配置，或修正客户端端口。再验证：客户端进入主机密钥或认证阶段。

③ [诊断] 症状：Host key verification failed 或主机身份已变化

当前证据：警告中显示的算法、指纹和 `offending line`。假设：合法重装/轮换、连接到错误地址，或中间人攻击。下一条证据：通过控制台读取 `/etc/ssh/ssh_host_*.key.pub` 指纹，并核对当前 DNS/IP。最小修复：只有在确认合法变化后，使用 `ssh-keygen -R` 删除特定记录。再验证：新连接显示的新指纹与可信证据一致。

④ [诊断] 症状：客户端没有发送预期公钥

当前证据：`ssh -vvv` 没有目标密钥的 `Offering public key`。假设：`IdentityFile` 未匹配、文件权限错误、agent 无身份、别名匹配了其他 `Host` 块。下一条证据：`ssh -G`、`stat`、`ssh-add -l`。最小修复：显式 `-i` 与 `IdentitiesOnly=yes`，修正客户端配置或载入 agent。再验证：调试输出出现目标指纹的 `offer`。

⑤ [诊断] 症状：客户端发送了公钥，但服务端拒绝

当前证据：`Offering public key` 后仍出现 `Authentications that can continue`，没有接受信息。假设：连接用户名错误；授权文件无匹配公钥；有效配置关闭公钥；路径权限/标签不可信；账号策略拒绝。下一条证据：同时读取：

```
sshd -T -C user=operator,host=servera.example.com,addr=192.0.2.50,laddr=192.0.2.10,lport=22
namei -l /home/operator/.ssh/authorized_keys
ls -ldZ /home/operator/.ssh /home/operator/.ssh/authorized_keys
journalctl -u sshd --since '-5 min' --no-pager
```

最小修复：只修正被证据指向的配置、授权内容、owner/mode 或标签。再验证：`key-only` 连接成功，日志显示目标用户公钥认证完成。

⑥ [诊断] 症状：登录成功但方法、用户或主机不符合任务

当前证据：客户端最终认证方法和远端命令结果。假设：公钥失败后回退密码；客户端 `User` 错误；别名指向错误主机；远端命令在跳转节点执行。下一条证据：`ssh -vvv` 最终 `Authenticated ... using ...`，远端 `id`、`hostname`，客户端 `ssh -G`。最小修复：禁止回退、修正 `User/HostName/IdentityFile`。再验证：指定用户、指定主机、指定认证方法和命令退出状态全部符合要求。

[Cheatsheet] 先定位阶段：`ssh -G` 查选择，`ssh -vvv` 查执行；客户端没 `offer` 就先查客户端；已 `offer` 未接受再查服务端；每次修复后重新触发同一个最小测试。

[知识专题] 工作迁移与安全边界：把考试中的 SSH 变成可维护入口

考试通常只要求在少量主机之间建立连接，真实工作还要考虑轮换、离职撤权、资产重装、批量预置信任和自动化。扩展内容仍围绕本章对象，不展开大型身份平台或证书体系。

① [知识点] 远程入口变更必须设计回退

任何关闭密码、禁止 `root` 或改变端口的操作，都应先证明替代路径可用。保持旧会话、准备控制台、记录有效配置和只修改一个来源，能够把“配置错误”从远程失联事故降级为可回退变更。

② [知识点] 密钥轮换是“先加新、验证、再删旧”

```
生成新密钥对
→ 在服务器追加新公钥
→ 使用新私钥完成 key-only 验证
→ 从所有服务器删除旧公钥
→ 记录旧指纹已撤销
```

直接覆盖本地私钥或先删除旧公钥，会在任一步失败时失去入口。注释和指纹应作为轮换清单的稳定标识。

③ [知识点] 批量主机信任需要可信分发，不是盲目扫描

组织可通过配置管理分发 `/etc/ssh/ssh_known_hosts`，但输入的主机公钥必须来自可信资产流程。`ssh-keyscan` 可帮助收集格式化密钥，却不能在不可信网络上自行解决身份认证。首次信任的来源决定了之后检测变化是否有意义。

④ [知识点] 为 Ansible 准备的是稳定状态，不是把私钥散发到受管主机

后续 RHCE 自动化通常由控制节点发起 SSH。应明确控制节点上的用户、私钥、known_hosts 和 agent 状态，以及受管节点上的账号、authorized_keys、sudo 和 sshd 有效策略。私钥留在控制节点；受管节点只接收公钥。Inventory、ansible_user、私钥字段和批量幂等配置属于 RHCE 章节，本章提供可验证的手工状态模型。

[Cheatsheet] 变更先有回退；轮换先加后删；批量信任需要可信来源；控制节点持私钥，受管节点持公钥；不要把“能连”扩大为“身份和权限都正确”。

[经典任务] 任务一：为指定用户建立可审计的密钥认证

环境与当前状态

- 客户端：workstation.example.com，当前本地用户 student；
- 服务端：servera.example.com；
- 目标远端账号：operator1；
- operator1 当前可以使用账号密码登录；
- 客户端尚无本任务专用密钥和别名；
- 服务端主机密钥指纹可通过服务器控制台取得；
- 本任务不要求改变 sshd 全局认证策略。

目标终态

1. 在客户端生成专用密钥：

```
~/ .ssh/rhcsa20_operator
~/ .ssh/rhcsa20_operator.pub
```

2. 私钥必须使用密语保护；不覆盖其他密钥；
3. 首次信任 servera 前，通过控制台核对主机指纹；
4. 将公钥追加到 operator1 的授权文件；
5. 在 ~/.ssh/config 建立别名 servera-operator，显式设置 HostName、User、IdentityFile 和 IdentitiesOnly yes；
6. 在当前登录会话中使用 ssh-agent 载入专用私钥；
7. 执行 ssh servera-operator 'id && hostname' 时，必须使用公钥认证且不得回退到密码；
8. 不得把私钥复制到服务端，不得关闭主机密钥检查。

验收证据

- 专用公钥指纹；
- ssh-add -l 中的专用密钥指纹；
- ssh -G servera-operator 的有效目标、用户和身份文件；
- ssh -vvv 中主机身份、密钥 offer、服务端接受和最终认证方法；
- 远端 id 与 hostname；
- 服务端 .ssh、authorized_keys 的 owner、mode 和 SELinux 标签。

[经典任务] 任务二：安全修复 sshd 配置并避免锁死

环境与当前状态

- 管理员已通过一条现有 SSH 会话连接到 serverb.example.com；
- operator1 的公钥已经写入服务器，但登录仍失败；
- /etc/ssh/sshd_config.d/ 中存在本地 drop-in，至少一项认证配置无效或未按预期生效；
- .ssh 路径的所有权、权限或 SELinux 标签可能异常；
- sshd 当前仍在运行并监听默认端口；
- 管理员可以另开一个客户端终端，但不能依赖服务器重启。

目标策略

```
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no
```

`operator1` 必须能够使用指定私钥登录；`root` 直登和口令式登录必须按策略失败。

限制条件

- 不关闭当前管理会话；
- 不停止 `firewalld`，不关闭 `SELinux`；
- 不删除全部现有 SSH 配置；
- 不在 `sshd -t` 通过前 `reload`；
- 不用 `chmod 777`、关闭 `StrictModes` 或 `StrictHostKeyChecking=no` 作为修复；
- 新连接未验证成功前不得退出旧会话。

验收证据

- 配置源与 `Include`、`Match` 基线；
- `sshd -t`；
- `sshd -T` 和目标用户的 `sshd -T -C ...`；
- `systemctl is-active sshd` 与 `ss -lntp`；
- `namei -l`、`stat`、`ls -Z`；
- 独立客户端的 `key-only` 成功验证；
- `root` 和 `password-only` 的预期失败；
- `journalctl -u sshd` 的服务端证据。

[参考解答] 任务一：为指定用户建立可审计的密钥认证

1. 建立客户端基线

先检查现有目录和目标文件，避免覆盖：

```
install -d -m 700 ~/.ssh
ls -l ~/.ssh/rhcsa20_operator ~/.ssh/rhcsa20_operator.pub 2>/dev/null
```

若目标文件已存在，应先确认是否属于旧任务或正在使用。除非题目明确要求重建，不直接覆盖。

2. 生成受密语保护的专用密钥

非 FIPS 环境可选择 Ed25519：

```
ssh-keygen -t ed25519 \
-f ~/.ssh/rhcsa20_operator \
-C 'student@workstation-rhcsa20'
```

若环境处于 FIPS 模式或题目要求 RSA：

```
ssh-keygen -t rsa -b 3072 \
-f ~/.ssh/rhcsa20_operator \
-C 'student@workstation-rhcsa20'
```

按提示设置非空密语。随后验证：

```
stat -c '%U:%G %a %n' ~/.ssh/rhcsa20_operator ~/.ssh/rhcsa20_operator.pub
ssh-keygen -lf ~/.ssh/rhcsa20_operator.pub
```

预期私钥只有当前用户可读写，公钥指纹可作为后续 `agent` 和远端授权核对依据。

3. 首次连接前核对服务端主机指纹

在服务端可信控制台读取实际启用的主机公钥指纹，例如：

```
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

客户端发起一次连接并停在首次信任提示：

```
ssh operator1@servera.example.com
```

只有显示的算法和 SHA256 指纹与控制台证据一致时才接受。若不一致，停止任务并调查名称、地址或主机重装情况。

4. 部署公钥

使用已有密码路径：

```
ssh-copy-id -i ~/.ssh/rhcsa20_operator.pub operator1@servera.example.com
```

注意 `-i` 指向 `.pub` 文件。完成后在服务端检查：

```
ssh operator1@servera.example.com \  
'namei -l ~/.ssh/authorized_keys; stat -c "%U:%G %a %n" ~/.ssh ~/.ssh/authorized_keys; ls -ldZ ~/.ssh ~/.ssh/authorized_keys'
```

此时使用密码进入服务器只用于检查文件状态，不能作为公钥成功证据。

5. 建立客户端别名

编辑 `~/.ssh/config`，添加：

```
Host servera-operator  
  HostName servera.example.com  
  User operator1  
  IdentityFile ~/.ssh/rhcsa20_operator  
  IdentitiesOnly yes
```

然后：

```
chmod 600 ~/.ssh/config  
ssh -G servera-operator | grep -E '^(hostname|user|port|identityfile|identitiesonly) '
```

应确认 `hostname` 为服务端 FQDN，`user` 为 `operator1`，`identityfile` 为专用私钥。

6. 启动或复用 agent 并载入身份

先看当前 Shell 是否已有 agent：

```
printf '%s\n' "$SSH_AUTH_SOCK"  
ssh-add -l
```

若不可达，再启动：

```
eval "$(ssh-agent -s)"
```

载入专用密钥并核对指纹：

```
ssh-add -t 1h ~/.ssh/rhcsa20_operator  
ssh-add -l
```

`ssh-add -l` 中应出现与 `ssh-keygen -lf ~/.ssh/rhcsa20_operator.pub` 一致的指纹。

7. 执行不允许回退的功能验证

```
ssh -vvv \  
-o PreferredAuthentications=publickey \  
-o PasswordAuthentication=no \  
-o KbdInteractiveAuthentication=no \  
servera-operator 'id && hostname'
```

检查要点：

1. 连接目标和端口正确；
2. 主机密钥与已知记录匹配；
3. 出现专用密钥的 `Offering public key`；
4. 服务端接受该密钥；
5. 最终认证方法为 `publickey`；
6. `id` 显示 `operator1`，`hostname` 显示 `servera` 的预期名称；

7. 命令退出状态为 0。

8. 典型错误与最小修复

症状	先查证据	最小修复
ssh-copy-id 提示文件无效	file、文件名是否以 .pub 结尾	改为公钥文件，不复制私钥
能登录但提示了账号密码	ssh -vvv 最终方法	禁止回退后重新验证，调查公钥拒绝原因
没有 offer 专用密钥	ssh -G、ssh-add -l	修正别名、IdentityFile 或载入 agent
host key changed	控制台指纹、DNS/IP	确认合法变更后只删除目标记录
服务端拒绝公钥	journal、owner/mode/label	只修正被证据指出的路径或策略

任务完成后，不需要从服务端删除公钥；它正是目标持久状态。可按要求从 agent 移除临时身份：

```
ssh-add -d ~/.ssh/rhcsa20_operator
```

[参考解答] 任务二：安全修复 sshd 配置并避免锁死

1. 保留现有会话并建立恢复能力

当前 SSH 会话保持打开。确认另一个客户端终端可用，并记录当前服务与监听：

```
systemctl is-active sshd
systemctl status sshd --no-pager
ss -lntp | grep sshd
```

若已有 BMC 或控制台，确认其可用。不要在只有一条不可恢复会话时继续关闭认证方法。

2. 读取配置源而不是先写新文件

```
grep -nE '^[[:space:]]*(Include|Match|Port|PermitRootLogin|PubkeyAuthentication|PasswordAuthentication|KbdInteractiveAuthentication|AuthorizedKeysFile|AllowUsers|
\
/etc/ssh/sshd_config

find /etc/ssh/sshd_config.d -maxdepth 1 -type f -name '*.conf' -print | sort

for f in /etc/ssh/sshd_config.d/*.conf; do
[ -e "$f" ] || continue
printf '\n### %s\n' "$f"
grep -nE '^[[:space:]]*(Match|Port|PermitRootLogin|PubkeyAuthentication|PasswordAuthentication|KbdInteractiveAuthentication|AuthorizedKeysFile|AllowUsers|DenyUser
"$f"
done
```

建立备份和有效配置基线：

```
cp -a /etc/ssh/sshd_config /root/sshd_config.before-rhcsa20
cp -a /etc/ssh/sshd_config.d /root/sshd_config.d.before-rhcsa20
sshd -T > /root/sshd-effective.before-rhcsa20
```

3. 先运行静态检查定位无效内容

```
sshd -t
```

若返回错误，按提示定位文件和行号。只修改错误参数或语法，不删除其他管理员配置。修复后重复 sshd -t，直到无错误。

4. 检查目标用户的授权路径

```
getent passwd operator1
namei -l /home/operator1/.ssh/authorized_keys
stat -c '%U:%G %a %n' \
/home/operator1 \
/home/operator1/.ssh \
/home/operator1/.ssh/authorized_keys
ls -ldZ \
/home/operator1 \
/home/operator1/.ssh \
/home/operator1/.ssh/authorized_keys
```

在默认本地 home 场景，可进行保守修复：

```
chown -R operator1:operator1 /home/operator1/.ssh
chmod 700 /home/operator1/.ssh
chmod 600 /home/operator1/.ssh/authorized_keys
restorecon -Rv /home/operator1/.ssh
```

不机械改变整个 home 为 700；只确保路径不被不可信用户写入。若 home 为非标准路径或 NFS，停止套用默认修复并转入 SELinux/NFS 专题调查。

5. 建立最小认证策略 drop-in

先根据目录顺序和 first-value-wins 判断合适文件名。假设检查后确认 10-local-auth.conf 会最先为这些参数提供本地值，可写入：

```
cat > /etc/ssh/sshd_config.d/10-local-auth.conf <<'EOF'
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no
EOF
```

再次静态检查：

```
sshd -t
```

6. 验证全局和目标连接的有效配置

```
sshd -T | grep -E '^(permitrootlogin|pubkeyauthentication|passwordauthentication|kbdinteractiveauthentication|authorizedkeysfile) '

sshd -T -C user=operator1,host=serverb.example.com,addr=192.0.2.50,laddr=192.0.2.20,lport=22 \
| grep -E '^(permitrootlogin|pubkeyauthentication|passwordauthentication|kbdinteractiveauthentication|authorizedkeysfile) '
```

这里的地址应替换为题目真实客户端和服务端地址。若输出仍不是目标值，说明更早配置或 Match 块正在生效，应回到第 2 步调查，而不是添加更多重复行。

7. reload 并验证守护进程层

```
systemctl reload sshd
systemctl is-active sshd
ss -lnpt | grep sshd
journalctl -u sshd --since '-5 min' --no-pager
```

若 reload 失败，保持旧会话，读取日志，恢复备份或修正 drop-in；不要重启服务器来掩盖配置错误。

8. 从第二个终端验证允许路径

```
ssh -vvv \
-i ~/.ssh/rhcsa20_operator \
-o IdentitiesOnly=yes \
-o PreferredAuthentications=publickey \
-o PasswordAuthentication=no \
-o KbdInteractiveAuthentication=no \
operator1@serverb.example.com 'id && hostname'
```

必须确认：

- 主机身份记录匹配；
- 客户端发送目标公钥；

- 服务端接受；
- 最终方法为 `publickey`；
- 远端用户为 `operator1`；
- 主机为 `serverb`。

9. 验证禁止路径

验证 `root` 直登不被允许：

```
ssh -o PreferredAuthentications=publickey,password,keyboard-interactive \
root@serverb.example.com
```

验证 `operator1` 不能只用口令方法：

```
ssh \
-o PubkeyAuthentication=no \
-o PreferredAuthentications=password,keyboard-interactive \
operator1@serverb.example.com
```

预期结果是认证失败。该负向测试只能在客户端方法限制、`sshd -T -C` 有效配置和服务端日志三者一致时证明策略终态；不能只看到 `Permission denied` 就结束。

10. 关联服务端日志并收束

在服务器旧会话中：

```
journalctl -u sshd --since '-10 min' --no-pager
```

将允许路径和禁止路径的时间、用户、来源地址和结果对应起来。全部验收完成后，才关闭原管理会话。保留的备份按组织变更流程处理，不在未确认回退窗口结束前删除。

典型错误

错误	为什么危险	正确处理
修改后直接 <code>restart sshd</code> 并退出会话	失去唯一入口	先 <code>-t</code> ，用 <code>reload</code> ，保持旧会话并开第二连接
只看 drop-in 文本	<code>first-value-wins</code> 和 <code>Match</code> 可能改变有效值	用 <code>sshd -T/-T -C</code>
只设置 <code>PasswordAuthentication no</code>	<code>keyboard-interactive</code> 可能仍允许口令式认证	分别检查两个有效值
<code>chmod 777 ~/.ssh</code>	授权路径不再可信	正确 owner，目录 700、文件 600
关闭 SELinux 或 StrictModes	掩盖根因并降低安全	查日志、mode、owner、label 后最小修复
服务 active 就宣布完成	active 不证明新登录	独立客户端验证允许和禁止路径

[本章收束] 把 SSH 还原成一条可证明、可回退的身份链

处理 SSH 时，先不要问“该改哪个配置”，而要把当前事实放回正确层次：客户端最终选中了哪个目标；服务器用哪把主机密钥证明身份；客户端实际发送了哪个用户身份；服务端对这个用户和来源应用了什么有效策略；授权路径是否同时满足内容、所有权、权限和标签；最后，独立新连接能否以指定方法到达指定用户和主机。按照这条链推进，才能让每次修复只改变一个被证据指向的对象。

可执行工作方法

1. 用 `ssh -G` 建立客户端选择基线。
2. 用 `ssh -vvv` 定位失败阶段，只摘取关键状态。
3. 主机密钥异常先带外核验，不先删除信任记录。
4. 客户端没有 offer 目标密钥时，只查客户端身份选择。
5. 已 offer 但未接受时，转查 `sshd -T -C`、授权路径和日志。
6. 修改 `sshd` 前保留旧会话，`sshd -t` 通过后才 `reload`。
7. 从第二终端分别验证允许路径和禁止路径。
8. 用远端 `id`、`hostname` 和退出状态确认终态。

章末检查清单

- 主机身份与用户身份没有混写；

- 私钥只保留在客户端或控制节点；
- `known_hosts` 更新前已经核验合法变化；
- `ssh-copy-id -i` 指向公钥文件；
- 客户端有效配置已经用 `ssh -G` 核对；
- 公钥测试禁止了 `password` 和 `keyboard-interactive` 回退；
- `sshd -t` 先于 `reload`；
- `sshd -T -C` 覆盖了目标连接的 `Match` 上下文；
- `owner`、`mode` 与 SELinux `label` 分别检查；
- 旧管理会话在第二连接验收前保持可用；
- `active`、`listen` 与真实登录没有被扩大为同一个结论。

主要判断表

证据或症状	能证明什么	不能证明什么	下一条高区分度证据
<code>ssh -G alias</code>	客户端最终目标、用户、端口和身份选择	服务器在线或会接受认证	<code>ssh -vvv alias</code>
<code>Connection refused</code>	目标返回了端口拒绝	一定是 <code>firewalld</code> 或 <code>sshd</code> 唯一问题	服务端 <code>ss -lntp</code>
主机密钥变化警告	当前对端密钥与记录不一致	新密钥一定恶意或一定合法	带外主机指纹与目标地址
<code>Offering public key</code>	客户端发送了某把候选公钥	服务端已授权或已接受	服务端 <code>journal</code> 与授权路径
<code>sshd -t</code> 无输出	配置可解析并通过基本检查	已 <code>reload</code> 、目标 <code>Match</code> 正确或能登录	<code>sshd -T/-T -C</code>
<code>systemctl is-active sshd</code>	守护进程当前处于 <code>active</code>	端口对外可达或用户认证成功	<code>ss -lntp</code> 与独立客户端测试
<code>authorized_keys</code> 有目标公钥	授权文本存在	路径可信、标签正确或 <code>sshd</code> 会读取	<code>namei -l</code> 、 <code>ls -Z</code> 、 <code>sshd -T -C</code>
<code>key-only</code> 新连接成功	指定用户公钥链可用	禁止路径也符合目标策略	<code>root/password-only</code> 负向验证

向下一章交接

本章已经证明本机 `sshd` 的有效配置、运行状态和监听入口，也给出了从独立客户端发起连接的功能测试。下一章“`firewalld` 与完整服务访问链”将在此基础上继续回答：监听端口怎样关联 `zone`、`service`、`runtime/permanent` 规则，以及为什么“本机正在监听”仍不等于外部流量一定能到达。遇到外部 `timeout` 时，不关闭防火墙作为捷径，而是把 SSH 已建立的监听证据交给下一章继续调查。

firewalld 与完整服务访问链

从服务进程到远端请求：把监听、SELinux、zone、双态规则和协议验收放进同一条证据链。
对象模型操作语义验证诊断经典任务
大字号阅读版

本章阅读导航

先抓住一条主线：允许远端访问不是一个命令的结果，而是多个独立对象同时满足条件。服务进程、真实监听、SELinux 端口许可、流量进入的 zone、runtime 规则、permanent 规则和远端协议请求，必须按层建立证据。

01定义目标终态明确服务、地址、端口、协议和允许来源

02证明真实监听服务 active 之后继续检查 socket

03识别实际 zone从接口或来源绑定确认规则作用范围

04核对两份状态runtime 回答现在，permanent 回答下次加载

05完成本机验证区分回环地址、服务器地址与应用协议

06完成远端验收从独立客户端验证真实访问路径

专题地图 | 类型 | 专题 | |---|---| | 知识专题 | firewalld 在完整访问链中的职责边界 | | 知识专题 | zone、default zone 与流量分类 | | 操作专题 | 查询和维护 interface/source 绑定 | | 知识专题 | runtime、permanent 与 reload | | 操作专题 | service、port/protocol 与查询闭环 | | 知识专题 | 配置来源、覆盖关系与静态检查 | | 操作专题 | rich rule 的必要表达范围 | | 操作专题 | 从监听到远端请求的分层验收 | | 诊断专题 | 从症状推进到下一条区分性证据 | | 诊断专题 | 最小放行、远端变更与清理边界 | | 经典任务 | 发布非标准端口 Web 服务 | | 经典任务 | 修复“permanent 已有但仍然超时” |

阅读时持续回答 1. 当前请求最终进入哪个 zone？ 2. 规则存在于 runtime、permanent，还是两者？ 3. service 定义实际包含哪些端口和协议？ 4. 应用真正监听的哪个地址和端口？ 5. 本机请求证明了哪一段路径？ 6. 远端失败时，下一条最有区分度的证据是什么？ 7. 非标准端口是否触发 SELinux 独立边界？ 8. 哪些修改会在 reload 后丢失或覆盖当前状态？

章节边界：NetworkManager 连接配置归第 18 章；systemd 服务生命周期归第 12 章；SELinux 端口类型和 AVC 证据归第 29 章。本章只引用这些对象来完成访问链验收。

第 21 章 · 正文

一项网络服务可以“看起来已经完成”，却仍然无法从远端使用。systemctl is-active httpd 可能返回 active，但进程只监听在 127.0.0.1；8088/tcp 可能已经加入 firewalld，却加在不处理实际流量的 zone；permanent 配置可能正确，而当前 runtime 尚未加载；本机 curl 可能成功，但该请求没有经过远端客户端所使用的接口和防火墙路径。

真正稳定的处理方法，是把“开放端口”改写为一组可以逐层判定的终态：

```
服务配置正确
→ 服务进程已运行
→ SELinux 允许进程绑定目标端口
→ socket 在正确地址/端口/协议上监听
→ 本机应用协议能够响应
→ 请求被分类到预期 zone
→ runtime 当前允许
→ permanent 能在下次加载后恢复
→ 独立远端客户端完成协议访问
```

本章围绕 firewalld 的核心对象推进：zone 决定规则集合，interface/source 绑定决定流量分类，service 或 port/protocol 表达允许内容，runtime 与 permanent 分别保存“现在”和“下次加载”的状态。诊断时不把局部成功扩大为整体结论，而是用下一条有区分度的证据缩小故障层。

概念

firewalld 是动态防火墙管理服务，它通过 zone、service、port 和 rich rule 等高层对象维护主机的包过滤状态。它能允许或拒绝进入主机的流量，却不安装应用、不启动服务，也不会因为“开放端口”而生成监听 socket。观察它时主要使用 firewall-cmd；判断应用是否真正存在，还要转向 systemctl、ss 和协议客户端。

概念

Zone 是一组信任边界和允许规则，也是一次入站请求被分类后的处理上下文。接口绑定描述“从哪个接口进入”，来源绑定描述“来自哪个地址范围”；没有显式绑定时，流量可能回落到 default zone。规则写入某个 zone 并不等于实际请求会经过它，所以每次变更都应先确认实际分类。

概念

接口绑定与来源绑定 都把流量映射到 zone，但它们表达不同维度。interface 适合描述接入路径，source 适合描述可信网段或单个来源。绑定本身也有 runtime/permanent 两层状态；远程移动管理接口或来源时，可能立刻改变 SSH 会话所经过的规则，因此必须先保留管理通道和回滚路径。

概念

Runtime 与 permanent 是 firewalld 同时维护的两份配置。runtime 是当前实际生效的规则；permanent 是 reload、firewalld 重启或系统重启时用于重建 runtime 的持久状态。修改 permanent 不会自动改变现在，修改 runtime 也不会自动保存以后；两者可以暂时不同，但最终必须按题目要求分别验证。

概念

Service 定义与 port/protocol 是两种允许流量的表达方式。service 是带名称的通信需求集合，可能包含一个或多个端口、协议或辅助信息；port 规则直接指定端口号或范围及传输协议。标准服务优先使用 service，非标准端口或没有合适定义时使用 PORT/PROTO，并确保它与真实 listener 一致。

概念

完整服务访问链 是由多个独立证据组成的业务终态。服务 active 只证明 systemd 的当前管理状态；ss 证明真实 socket；本机 curl 证明一段应用路径；firewalld 查询证明指定 zone 的规则；只有独立远端客户端成功完成协议请求，才证明该客户端到服务的整条访问链可用。任何一层通过，都不能替代其余层。

操作语义 以下入口分别观察 zone 分类、允许规则、双态转换和访问证据。先理解命令作用对象，再记关键参数。

firewall-cmd 查询 zone 与规则

SYNOPSIS

```
firewall-cmd [--permanent] [--zone=ZONE] OPTION
```

读取 firewalld 当前或持久配置。查询时显式写出 zone，可以避免把默认上下文误当成真实流量所在位置。

重要参数 / 形式

```
--get-active-zones
    列出当前绑定了 interface 或 source 的活动 zone；不表示这些 zone 一定放行目标服务。
--get-zone-of-interface=IFACE
    查询指定接口当前属于哪个 zone。
--get-zone-of-source=CIDR
    查询指定来源当前属于哪个 zone；source 不使用主机名。
--zone=ZONE --list-all
    查看一个 zone 的当前完整摘要，包括 interfaces、sources、services、ports 和 rich rules。
--permanent --zone=ZONE --list-all
    查看同一 zone 的持久摘要，用于与 runtime 对照。
```

firewall-cmd 添加、删除和查询允许项

SYNOPSIS

```
firewall-cmd [--permanent] --zone=ZONE \
    --add-service=SERVICE | --add-port=PORT/PROTO
```

对明确的 zone 添加或删除 service、port/protocol，并用同类 `--query-*` 或 `--list-*` 完成闭环。

重要参数 / 形式

```
--add-service=http
    按 service 定义允许通信需求；不会让 --list-ports 出现同一端口。
--add-port=8088/tcp
    直接允许目标端口和协议；TCP 与 UDP 是不同对象。
--remove-service=SERVICE / --remove-port=PORT/PROTO
    从目标状态中删除对应允许项；删除前分别核对 runtime 和 permanent。
--query-service=SERVICE / --query-port=PORT/PROTO
    返回 yes/no，常见退出码为存在时 0、不存在时 1，适合脚本条件判断。
--get-services / --info-service=SERVICE
    查看可用 service 名称及其实际定义，避免只凭名称猜端口。
```

--permanent、--reload 与 --runtime-to-permanent

SYNOPSIS

```
firewall-cmd --permanent ...
firewall-cmd --reload
firewall-cmd --runtime-to-permanent
```

控制两份状态之间的关系。`--permanent` 选择持久配置；`--reload` 用 permanent 重建 runtime；`--runtime-to-permanent` 把整个当前 runtime 保存为 permanent。

重要参数 / 形式

```
--permanent
    让随后支持该选项的操作作用于持久配置；它本身不让修改立即生效。
--reload
    重新加载持久配置并重建 runtime；未持久化的 runtime-only 修改会丢失。
--runtime-to-permanent
    整体保存当前 runtime，不是只提交上一条命令。使用前必须审查是否存在不应持久化的临时规则。
--check-config
```

检查 permanent XML 和语义是否可解析；不证明当前规则、监听或远端功能正确。

interface/source 绑定

SYNOPSIS

```
firewall-cmd [--permanent] --zone=ZONE \  
--change-interface=IFACE | --add-source=CIDR
```

改变流量被分类到哪个 zone。绑定操作比单纯开放端口更容易影响现有远程会话，因此必须先查基线。

重要参数 / 形式

`--change-interface=IFACE`

将接口放入目标 zone；若接口尚未绑定，通常表现为添加。NetworkManager 管理的 connection profile 细节留给第 18 章。

`--query-interface=IFACE`

判断指定接口是否属于当前给出的 zone。

`--add-source=CIDR`

将单个地址或网段作为来源绑定到目标 zone。

`--change-source=CIDR`

将既有来源移动到另一个 zone。

rich rule

SYNOPSIS

```
firewall-cmd [--permanent] --zone=ZONE \  
--add-rich-rule='rule family="ipv4" source address="CIDR" \  
port port="PORT" protocol="tcp" accept'
```

在简单 service/port 无法表达“仅允许指定来源”等条件时使用。查询和删除应复用完整规则文本。

重要参数 / 形式

`family="ipv4" / family="ipv6"`

当规则包含地址时显式限定地址族，并与 source 地址一致。

`source address="CIDR"`

限制匹配来源；主机名不是稳定的 source 表达。

`service name="http" 或 port port="8088" protocol="tcp"`

选择通信目标。能用简单 service/port 完成时，不应为了“高级”而改用 rich rule。

`accept / reject / drop`

分别允许、显式拒绝或静默丢弃。诊断时三者表现可能不同。

ss、curl 与 nc

SYNOPSIS

```
ss -lntp  
curl -I http://HOST:PORT/  
nc -vz HOST PORT
```

分别验证 socket、应用协议和有限的传输层连通。最终验收优先使用真实协议客户端，而不是只看 TCP 建连。

重要参数 / 形式

`ss -lntp`

查看 TCP 监听、数字地址、端口和关联进程；UDP 使用 `ss -lnup`。

`curl -I URL`

对 HTTP 服务请求响应头；它能证明请求路径上的 HTTP 交互，不证明其他客户端或重启后状态。

`nc -vz HOST PORT`

尝试 TCP 建连并输出详细信息；成功不代表 HTTP、TLS 或业务内容正确。

知识专题

firewalld 在完整服务访问链中的职责边界

排错最常见的问题不是“不会加规则”，而是把一个局部证据扩大成整体结论。先明确每个工具观察的对象，再决定下一步，能避免在错误层反复修改。

① 知识点 防火墙允许与应用监听是两个对象

```
firewall-cmd --zone=public --query-port=8088/tcp
```

回答“public 的当前规则是否允许 8088/tcp”。它不能证明应用已经启动、监听地址正确或请求实际进入 public。

```
ss -lntp '( sport = :8088 )'
```

回答“主机是否存在 TCP 8088 listener，以及监听地址和进程”。它不能证明 firewalld 允许远端访问，也不能证明 HTTP 内容正确。

② 判断 active、监听、协议响应和远端可达是四个终态

证据	能证明	不能证明
systemctl is-active httpd	systemd 当前认为服务 active	端口、地址、协议和远端可达正确
ss -lntp	TCP listener 的地址、端口和进程	HTTP 内容与远端路径正常
本机 curl	本机到目标地址的 HTTP 路径	指定远端客户端可达、permanent 正确
远端 curl	该客户端到目标的 HTTP 请求成功	所有来源、所有协议和重启后状态正确

③ 边界 非标准端口存在独立的 SELinux 分支

服务配置要求监听非标准端口时，SELinux 可能阻止进程绑定，即使 firewalld 已经放行。此时先用服务状态、日志和 ss 确认“进程没有建立 listener”，再把 SELinux port type 作为独立假设交给第 29 章的方法验证。不要用关闭 SELinux 或改为 Permissive 作为默认答案。

④ 验证 每条证据都写清证明边界

稳定的验收记录应写成：

```
证据: firewall-cmd --zone=internal --query-port=8088/tcp → yes
证明: internal 的 runtime 当前包含 8088/tcp
不能证明: 请求实际进入 internal、应用正在监听、permanent 已保存
下一证据: 查询接口/来源绑定，并检查 permanent 与远端请求
```

Cheatsheet 防火墙允许不等于有监听；服务 active 不等于端口正确；本机成功不等于远端成功；远端成功也不能自动证明 permanent 正确。

知识专题

zone、default zone 与流量分类

zone 的本质是规则集合与信任边界。考试中最危险的误判，是没有确认请求实际进入哪个 zone，就直接在默认 zone 中增加规则。

① 知识点 active zone 只说明当前存在绑定

```
firewall-cmd --get-active-zones
```

输出中的 zone 当前至少绑定了 interface 或 source。它不表示该 zone 一定允许目标服务，也不表示系统只存在这些 zone。查看所有定义使用 --get-zones，查看一个 zone 的规则使用 --zone=ZONE --list-all。

② 知识点 default zone 是未显式选择时的回落对象

```
firewall-cmd --get-default-zone
```

default zone 不是“最安全”或“最常用”的同义词。改变它可能影响所有依赖默认分类的连接，因此 --set-default-zone=... 不是修复单个服务的首选捷径。

③ 比较 interface 与 source 描述不同分类维度

```
firewall-cmd --get-zone-of-interface=enp1s0
firewall-cmd --get-zone-of-source=192.0.2.0/24
```

interface 适合回答请求从哪条接入路径进入；source 适合表达某个地址或网段的信任范围。source 支持 IP/CIDR、MAC 或 ipset，不支持主机名。名称到地址的稳定映射属于第 19 章。

④ 查询 不靠模糊优先级记忆，读取实际分类

现代 firewalld 还可能存在 zone priority 等版本相关能力。RHCSA 主路径不要求背完内部匹配算法，而是建立可复查顺序：

```
查询来源绑定
→ 查询接口绑定
→ 查看 active zones
→ 读取目标 zone 的完整规则
→ 明确此次修改写入哪个 zone
```

Cheatsheet `--get-default-zone` 看回落；`--get-active-zones` 看当前绑定；`--get-zone-of-interface/source` 看具体对象；`--list-all` 看目标 zone 的规则全貌。
操作专题

查询和维护 interface/source 绑定

绑定改变的是流量分类，而不是单一端口。远程管理服务器时，错误移动接口或来源可能立即让当前 SSH 会话失去允许规则，因此先建立当前/持久基线，再做最小修改。

① 查询 建立绑定基线

```
firewall-cmd --state
firewall-cmd --get-default-zone
firewall-cmd --get-active-zones
firewall-cmd --get-zone-of-interface=enp1s0
firewall-cmd --permanent --get-zone-of-interface=enp1s0
firewall-cmd --get-zone-of-source=192.0.2.0/24
firewall-cmd --permanent --get-zone-of-source=192.0.2.0/24
```

先记录当前管理接口、当前 SSH 所需的允许项和预备回滚命令。若 permanent 与 runtime 已经不同，先理解差异来源，不要立即 reload 覆盖现场。

② 操作 添加、移动、查询和移除 interface

```
firewall-cmd --zone=public --add-interface=enp1s0
firewall-cmd --zone=internal --change-interface=enp1s0
firewall-cmd --zone=internal --query-interface=enp1s0
firewall-cmd --remove-interface=enp1s0
```

`--change-interface` 在接口尚未绑定时通常相当于添加；已经绑定时把它移到目标 zone。NetworkManager 管理的接口可能由 connection profile 持久关联，完整配置留给第 18 章。

③ 操作 添加、移动、查询和移除 source

```
firewall-cmd --zone=internal --add-source=192.0.2.0/24
firewall-cmd --zone=dmz --change-source=192.0.2.0/24
firewall-cmd --zone=dmz --query-source=192.0.2.0/24
firewall-cmd --zone=dmz --remove-source=192.0.2.0/24
```

source 绑定适合把特定网段映射到专用 zone。不要把来源范围写得比题意更宽，也不要把临时客户端地址无调查地持久化。

④ 验证 绑定也要核对两份状态

```
firewall-cmd --get-zone-of-source=192.0.2.0/24
firewall-cmd --permanent --get-zone-of-source=192.0.2.0/24
firewall-cmd --zone=internal --list-all
firewall-cmd --permanent --zone=internal --list-all
```

查询结果证明分类和规则所在位置，但不能证明远端客户端已完成协议访问。绑定修改后仍要继续验证 listener、本机请求和远端请求。

⑤ 边界 远程变更先保住管理通道

推荐顺序：确认当前 SSH 来源和接口 → 在目标 zone 先允许管理服务 → 记录回滚命令 → 修改绑定 → 立即开第二个会话验证 → 再处理 permanent。不要关闭防火墙或把接口移入 `trusted` 作为“保险”。

Cheatsheet 绑定决定规则作用范围；先保存管理通道，再移动接口或来源；变更后同时查具体绑定和目标 zone 的完整规则。

知识专题

runtime、permanent 与 reload：两份状态怎样转换

firewalld 的双态模型允许先临时试验、再决定是否持久化，也允许先准备持久配置、再统一应用。便利的代价是：只看到一份状态时，很容易误判“已经完成”。

① 知识点 不带 `--permanent` 修改当前 runtime

```
firewall-cmd --zone=public --add-port=8088/tcp
```

该规则立即进入当前 runtime，但 reload、firewalld 重启或系统重启后会丢失。它适合临时验证和低风险试验，不满足“重启后仍生效”的持久终态。

② 知识点 带 `--permanent` 只修改持久配置

```
firewall-cmd --permanent --zone=public --add-port=8088/tcp
```

命令成功表示 permanent 已更新，不表示当前 runtime 已放行。下一步要么补充等价 runtime 修改，要么在审查当前临时状态后执行 reload。

③ 比较 四种双态组合表达不同终态

Runtime	Permanent	解释	常见下一步
yes	yes	当前和下次加载都允许	继续 listener 与远端验收
yes	no	当前允许，reload 后丢失	若要求持久，补 permanent
no	yes	持久配置存在，当前尚未应用或后来被改变	补 runtime，或审查后 reload
no	no	两层都不允许	确认实际 zone 后最小添加

④ 操作 同时满足“现在”和“以后”的两条路径

逐层修改：

```
firewall-cmd --zone=public --add-port=8088/tcp
firewall-cmd --permanent --zone=public --add-port=8088/tcp
```

优点是不会为了单条变更覆盖其他 runtime-only 状态，适合远程或已有临时规则的系统。

先准备持久配置再统一应用：

```
firewall-cmd --permanent --zone=public --add-port=8088/tcp
firewall-cmd --check-config
firewall-cmd --reload
```

适合维护窗口中有计划地重建 runtime。reload 前必须确认现有 runtime-only 项是否可以丢失。

⑤ 边界 `--runtime-to-permanent` 保存整个当前配置

```
firewall-cmd --runtime-to-permanent
```

它不是“保存上一条操作”，而是把整个 runtime 复制为 permanent。若当前含有超宽来源、临时测试端口或应到期的规则，这些状态也可能被保存，因此使用前要比较完整摘要并清理不应持久化的内容。

⑥ 验证 每个目标规则至少查询两次

```
firewall-cmd --zone=public --query-port=8088/tcp
firewall-cmd --permanent --zone=public --query-port=8088/tcp
```

yes/yes 只证明该 zone 的双态规则，不能证明实际请求进入该 zone，也不能证明应用监听或远端成功。

Cheatsheet runtime 回答“现在”；permanent 回答“下次加载”；reload 用 permanent 重建 runtime；`runtime-to-permanent` 是整体复制。
操作专题

service、port/protocol 与增删查列闭环

规则表达应尽量贴近业务对象。标准服务使用有语义的 service 定义；非标准端口或没有合适定义时直接使用 port/protocol。无论选择哪一种，都要完成查询、修改和验证闭环。

① 查询 先看 service 真实定义

```
firewall-cmd --get-services
firewall-cmd --info-service=http
firewall-cmd --path-service=http
```

service 名称只是入口，实际允许内容来自定义。不要根据“http”这个词猜所有 Web 端口都包含在内；非标准 8088 通常需要单独 port 规则或自定义 service。

② 操作 标准服务优先使用 service

```
firewall-cmd --zone=public --add-service=http
firewall-cmd --permanent --zone=public --add-service=http
firewall-cmd --zone=public --query-service=http
firewall-cmd --permanent --zone=public --query-service=http
firewall-cmd --zone=public --list-services
```

删除时使用对称的 `--remove-service=http`。`--list-services` 只列 service 名称；service 打开的端口不会自动出现在 `--list-ports`，需要 `--list-all` 或 `--info-service` 组合理解。

③ 操作 非标准端口使用 PORT/PROTO

```
firewall-cmd --zone=public --add-port=8088/tcp
firewall-cmd --permanent --zone=public --add-port=8088/tcp
firewall-cmd --zone=public --query-port=8088/tcp
firewall-cmd --permanent --zone=public --query-port=8088/tcp
firewall-cmd --zone=public --list-ports
```

端口可写为单个端口或范围，例如 `8000-8010/tcp`。TCP 与 UDP 同号端口是两个不同对象，必须按真实 listener 和题意选择。

④ 参数 --add-protocol 不是开放应用端口

`--add-protocol=gre` 等操作允许的是 IP protocol，不接受 `8088/tcp` 这样的端口表达。普通 Web、SSH、DNS 等服务访问通常使用 service 或 port/protocol，不要把“protocol”这个词混为一谈。

⑤ 边界 timeout 只适用于 runtime 临时规则

```
firewall-cmd --zone=public --add-port=8088/tcp --timeout=20m
```

规则到期后自动移除，且 `--timeout` 不能与 `--permanent` 组合。它适合受控测试，不应被误写成题目要求的持久答案。

⑥ 验证 规则必须和 listener 对齐

```
ss -lntp '( sport = :8088 )'
firewall-cmd --zone=public --query-port=8088/tcp
```

若应用监听 `8088/tcp`，只允许 `http` service（通常指标准端口）并不能自动覆盖它；反过来，开放 8088 也不会让应用改为监听该端口。

Cheatsheet 标准通信需求用 service；非标准端口用 ``PORT/PROTO``；TCP/UDP 分开；add/remove/query/list 形成闭环。

知识专题

配置来源、覆盖关系与 `--check-config`

大多数 RHCSA 任务可以用 `firewall-cmd` 完成，但理解配置来源能避免直接修改 vendor 文件，也能解释自定义 service 和静态检查的边界。

① 知识点 vendor 与管理员目录职责分离

<code>/usr/lib/firewalld/zones/</code>	发行版或软件包提供的 zone
<code>/usr/lib/firewalld/services/</code>	发行版或软件包提供的 service
<code>/etc/firewalld/zones/</code>	管理员自定义或覆盖的 zone
<code>/etc/firewalld/services/</code>	管理员自定义或覆盖的 service

不要直接编辑 `/usr/lib/firewalld/` 中的 vendor 文件，因为软件包更新可能覆盖它。需要自定义时，通过 `firewall-cmd --permanent --new-service=...` 等接口或在 `/etc/firewalld/` 中建立管理员对象。

② 查询 读取真实定义与路径

```
firewall-cmd --info-zone=public
firewall-cmd --path-zone=public
firewall-cmd --info-service=http
firewall-cmd --path-service=http
```

`--path-*` 能帮助确认当前对象来自 vendor 还是管理员目录。路径存在不代表 runtime 已经加载其内容，仍需查询当前 zone。

③ 配置 自定义 service 的必要结构

下面只说明最小对象，不在本章扩展所有 service XML 元素：

```
<?xml version="1.0" encoding="utf-8"?>
<service>
  <short>web-alt</short>
  <description>Alternative HTTP listener</description>
  <port protocol="tcp" port="8088"/>
</service>
```

自定义 service 的价值是复用和表达业务语义；只需一次开放的非标准端口，直接 port 规则通常更简单。

④ 验证 `--check-config` 只检查 permanent

```
firewall-cmd --check-config
```

通过表示持久 XML 和语义可解析。它不能证明：

- 当前 runtime 已经加载；
- 实际请求进入目标 zone；
- 应用正在监听；
- SELinux 允许端口；
- 远端客户端能够访问。

⑤ 边界 手工编辑后仍要按双态应用和验收

修改 `/etc/firewalld/` 后先 `--check-config`，再在维护窗口中 reload，随后重新查询 runtime/permanent 和业务访问。不要把“文件保存成功”当作 firewalld 已接受配置。

Cheatsheet `/usr/lib` 属于 vendor，`/etc` 属于管理员；`--path-*` 查真实来源；`--check-config` 只证明 permanent 可解析。
操作专题

rich rule 的必要表达范围

rich rule 用于把来源、服务或端口与动作组合起来。本章只覆盖主机入站服务常用的最小模型，不延伸到复杂转发、标记、策略对象或底层 nftables 设计。

① 语法 最小来源限制规则

```
RULE='rule family="ipv4" source address="192.0.2.0/24" \
port port="8088" protocol="tcp" accept'

firewall-cmd --zone=public --add-rich-rule="$RULE"
firewall-cmd --permanent --zone=public --add-rich-rule="$RULE"
```

规则表达：在 public 中，仅允许 192.0.2.0/24 访问 8088/tcp。它不会自动拒绝 zone 中其他已经存在的 service/port 规则，所以要先读取完整 zone 状态。

② 参数 地址与 family 保持一致

包含 source/destination 地址时显式写 `family="ipv4"` 或 `family="ipv6"`。IPv4 family 不能配 IPv6 地址，反之亦然。无需地址条件的简单规则通常直接使用 service/port。

③ 比较 service 或 port 表达通信目标

```
service name="http"
```

适合复用已定义 service；

```
port port="8088" protocol="tcp"
```

适合非标准端口。不要在同一最小规则中堆叠多个独立目标，使查询和删除变得难以复现。

④ 知识点 accept、reject 与 drop 的观察差异

- **accept**：允许匹配的新连接；
- **reject**：拒绝并向对端返回错误，客户端通常较快得到失败；
- **drop**：静默丢弃，客户端常表现为等待或超时。

这些表现只是诊断线索，网络路径和客户端重试也会影响最终症状。

⑤ 操作 查询和删除复用同一规则文本

```
firewall-cmd --zone=public --query-rich-rule="$RULE"
firewall-cmd --permanent --zone=public --query-rich-rule="$RULE"
firewall-cmd --zone=public --remove-rich-rule="$RULE"
firewall-cmd --permanent --zone=public --remove-rich-rule="$RULE"
```

使用变量或从 `--list-rich-rules` 复制规范化文本，避免删除时因空格、引号或字段差异匹配失败。

⑥ 边界 简单需求不使用 rich rule

“允许所有来源访问 http”直接添加 `http` service；“允许所有来源访问 8088/tcp”直接添加 port。只有来源限制、动作差异或必要日志/限速时才进入 rich rule。

Cheatsheet 简单需求用 service/port；来源限制再用 rich rule；地址配 family；查询和删除复用完整规则。

操作专题

从监听到远端请求的分层验收

完整验收从业务终态反推证据。每层失败时停在该层调查，不要在 listener 不存在时继续反复加防火墙规则。

① 目标 先把题意改写成五元组

```
服务: httpd
地址: 服务器外部地址 (不是仅 127.0.0.1)
端口: 8088
协议: TCP / HTTP
允许来源: 题目指定客户端或网段
```

若这些信息不清楚，`firewall-cmd` 的 zone、service、port 和 source 选择都无法唯一确定。

② 验证 服务管理状态

```
systemctl is-active httpd
systemctl status httpd --no-pager
```

这一步只证明第 12 章中的服务当前状态。若失败，读取服务日志和配置语法；若 active，继续检查 socket，而不是直接宣布完成。

③ 验证 真实监听

```
ss -lntp '( sport = :8088 )'
```

重点读取：

- **LISTEN** 是否存在；
- local address 是 `127.0.0.1`、某个具体服务器地址，还是 `0.0.0.0/[:::]`；
- 端口和协议是否符合题意；
- 进程是否属于目标服务。

④ 验证 本机协议路径

```
curl -I http://127.0.0.1:8088/
curl -I http://192.0.2.20:8088/
```

回环成功但服务器地址失败，优先检查监听地址、应用 virtual host 或本机策略；两者都成功后，才把重点转向实际 zone 和远端路径。

⑤ 验证 实际 zone 与双态规则

```
firewall-cmd --get-active-zones
firewall-cmd --get-zone-of-interface=enp1s0
firewall-cmd --zone=internal --query-port=8088/tcp
firewall-cmd --permanent --zone=internal --query-port=8088/tcp
```

查询必须使用真实 zone。规则在 public 返回 yes，而请求实际进入 internal，并不能允许该请求。

⑥ 验证 独立远端协议请求

在题目指定客户端执行：

```
curl -I http://192.0.2.20:8088/
nc -vz 192.0.2.20 8088
```

curl 是最终 HTTP 功能证据；nc 仅辅助判断 TCP 是否建立。无法从服务器本机替代真正的远端验证，因为本机请求经过的路由和防火墙路径可能不同。

⑦ 判断 纵向验收矩阵

层	推荐证据	通过后能证明	仍不能证明
服务	systemctl is-active	当前服务状态	listener 与协议
Socket	ss -lntp	地址/端口/进程	HTTP 功能与防火墙
本机协议	curl 本机地址	应用在本机路径响应	远端 zone 与持久性
分类	zone/interface/source 查询	请求应使用的规则集合	规则是否允许
Runtime	query-*	当前规则允许	reload 后仍允许
Permanent	--permanent query-*	下次加载配置包含规则	当前已经应用
远端协议	客户端 curl	该客户端整条路径可用	其他来源和重启后状态

Cheatsheet 服务 → listener → 本机协议 → 实际 zone → runtime → permanent → 远端协议；每层只证明自己的对象。

诊断专题

从症状推进到下一条有区分度的证据

诊断不从“多运行几条命令”开始，而是遵循：症状 → 当前证据 → 假设 → 下一条最有区分度的证据 → 最小修复 → 再验证。

① 诊断 Connection refused

当前含义：对端快速拒绝，常见于目标地址没有 listener，或路径中有显式 reject。

下一证据：

```
ss -lntp '( sport = :8088 )'
```

listener 不存在时优先调查应用配置、服务日志、监听地址和 SELinux；listener 存在时再查 rich rule 是否 reject、客户端是否访问了错误地址。

② 诊断 连接超时

当前含义：客户端长期得不到响应，可能是 drop、路由、中间设备、错误地址或服务器规则。

下一证据：先确认服务器本机地址请求成功，再查询实际 zone 和 runtime；不要仅凭“超时”直接关闭防火墙。

```
curl -I http://192.0.2.20:8088/
firewall-cmd --get-zone-of-interface=enp1s0
firewall-cmd --zone=internal --query-port=8088/tcp
```

③ 诊断 服务 active，但 ss 没有目标 listener

假设：服务配置未加载、绑定失败、监听在其他端口、进程已快速退出或 SELinux 阻止非标准端口。

下一证据：服务状态详情、配置语法和日志。此时继续添加 firewallld 规则没有区分度。

④ 诊断 回环成功，服务器地址失败

假设：应用只监听 127.0.0.1，或应用配置不接受目标地址。

下一证据：ss 的 local address。修复应用监听后，再重新做本机地址请求；firewalld 不会改变 listener 地址。

⑤ 诊断 本机服务器地址成功，远端失败

假设：实际 zone 错误、runtime 未放行、来源 rich rule 不匹配、上游路由或中间设备问题。

下一证据：--get-zone-of-interface/source 与目标 zone 的 --list-all。若服务器规则正确，再向路由或中间设备层推进。

⑥ 诊断 permanent 为 yes，runtime 为 no

说明持久配置存在，但当前尚未应用或后来被修改。最小修复可以是补等价 runtime；在确认没有重要 runtime-only 状态后，也可以 reload。不要把再次写 permanent 当作当前修复。

⑦ 诊断 runtime 为 yes，permanent 为 no

说明当前可用但 reload/重启后丢失。若题目要求持久化，补 permanent 后再次查询两层；若只是临时测试，不要无条件保存整个 runtime。

⑧ 诊断 规则存在于错误 zone

命令成功只能证明“某个 zone 被修改”。下一证据是实际接口/来源绑定。最小修复是在正确 zone 建立目标规则；是否删除错误 zone 的遗留项，要先调查是否有其他业务依赖。

⑨ 诊断 service 与非标准 listener 不匹配

若 http service 已允许，而应用监听 8088，读取 --info-service=http 和 ss 即可区分。修复是允许实际 8088/tcp 或建立合适自定义 service，不是继续重复添加 http。

⑩ 诊断 TCP/UDP 写错

8088/tcp 与 8088/udp 是不同规则。使用 ss -lntp/ss -lnup 和应用协议确定真实传输层，不能仅按端口号判断。

标准模板

症状：远端请求超时

当前证据：本机 curl 成功，listener 正确

假设：请求进入的 zone 没有 runtime 规则

下一证据：查询接口/来源绑定和该 zone 的 query-port

最小修复：只在实际 zone 添加目标规则

再验证：双态查询 + 远端协议请求

Cheatsheet refused 先查 listener；timeout 先分本机/远端；本机成功再查实际 zone；双态漂移分别修复“现在”和“以后”。

诊断专题

最小放行、远程变更与清理边界

防火墙排错的目标不是“让所有请求都能过”，而是在不破坏已有业务和安全边界的前提下，建立题目要求的最小终态。

① 安全 不关闭 firewalld 验证服务

关闭防火墙只能暂时绕过这一层，还会放大暴露面，并不能解释原规则为什么失败。正确做法是先证明 listener 和本机协议，再查询实际 zone 和目标规则。

② 安全 不把接口移入 trusted 作为默认修复

trusted 通常接受所有流量，可能掩盖错误 zone、错误协议和来源范围问题。题目只要求一个服务时，最小答案是目标 zone + 目标 service/port + 必要来源。

③ 判断 最小规则至少包含四个维度

哪个 zone
允许哪个来源（若有要求）
哪个目标端口或 service
哪个协议

少一个维度都可能把规则写宽或写错。rich rule 只在简单规则不能表达来源限制时使用。

④ 边界 reload 前审查 runtime-only 状态

```
firewall-cmd --zone=public --list-all
firewall-cmd --permanent --zone=public --list-all
```

重点比较 services、ports、sources、interfaces 和 rich rules。reload 会用 permanent 重建 runtime，因此不能把它当作无副作用的“刷新按钮”。

⑤ 清理 删除遗留规则前确认依赖

规则位于错误 zone 不代表一定可以立刻删除；它可能服务于另一接口或来源。先查绑定、服务清单和业务 owner，再对 runtime/permanent 做对称清理，最后验证现有业务。

⑥ 恢复 远程操作保留第二条管理路径

在有控制台/BMC 时记录恢复入口；没有控制台时，先在目标 zone 保留 SSH，再改变绑定。操作后不要只依赖当前会话，打开第二个连接确认新请求也可建立。

Cheatsheet 不关防火墙、不进 trusted；只改目标 zone/来源/协议/端口；reload 前比对双态；远程操作先保住管理通道。
经典任务

发布非标准端口 Web 服务并完成全链验收

环境与当前状态

- 服务器：servera.lab.example.com，业务地址 192.0.2.20；
- 远端测试客户端：clienta.lab.example.com，来源地址位于 192.0.2.0/24；
- 接口：enp1s0，实际 zone 必须通过查询确定；
- 服务：httpd 已安装，配置目标端口为 8088/tcp；
- SELinux：保持 Enforcing。题目说明端口类型已由前置任务正确配置，但仍须把它视为独立核对边界；
- firewallld 正在运行，当前规则未知。

目标终态

1. httpd 当前运行，并在服务器外部地址可用的 8088/tcp 上真实监听；
2. 本机对回环地址和服务器业务地址的 HTTP 请求都能得到响应；
3. 只在处理 enp1s0 入站流量的实际 zone 中允许 8088/tcp；
4. runtime 与 permanent 均包含目标规则；
5. clienta 能取得 HTTP 响应；
6. 不开放无关端口或来源，不降低 SELinux，不关闭 firewallld，不使用 trusted 绕过调查。

验收矩阵

对象	必须提交的证据
服务	systemctl is-active httpd
监听	ss 显示地址、8088/tcp 和目标进程
本机应用	两次 curl -I：回环地址、服务器业务地址
Zone	interface/active zone 查询能说明实际分类
Runtime	实际 zone 的 --query-port=8088/tcp 返回 yes
Permanent	同一查询带 --permanent 返回 yes
远端应用	clienta 上的 curl -I 成功

请先独立完成。参考解答从下一页开始。
参考解答

先证明 listener，再在实际 zone 建立双态规则

① 调查 读取服务状态与真实监听

```
systemctl is-active httpd
systemctl status httpd --no-pager
ss -lnpt '( sport = :8088 )'
```

服务 active 后仍必须看到 8088 listener。若没有 listener，本题停在应用/SELinux 层，继续添加防火墙规则没有意义。若只监听 `127.0.0.1:8088`，先修复应用监听地址。

② 验证 完成本机协议证据

```
curl -I http://127.0.0.1:8088/
curl -I http://192.0.2.20:8088/
```

第一条证明回环路径；第二条证明服务在业务地址路径响应。两条都成功仍不能替代远端请求。

③ 调查 识别实际 zone

```
firewall-cmd --state
firewall-cmd --get-active-zones
firewall-cmd --get-zone-of-interface=enp1s0
```

假设查询显示 `enp1s0` 位于 `internal`。后续所有规则都显式使用 `--zone=internal`，不依赖 default zone。

④ 操作 建立 runtime 和 permanent

```
firewall-cmd --zone=internal --add-port=8088/tcp
firewall-cmd --permanent --zone=internal --add-port=8088/tcp
```

选择直接 port 规则，是因为服务使用非标准端口，内置 `http` service 不一定包含 8088。分别修改两层，避免为单条规则 reload 并覆盖未知 runtime-only 状态。

⑤ 验证 核对双态与 zone 全貌

```
firewall-cmd --zone=internal --query-port=8088/tcp
firewall-cmd --permanent --zone=internal --query-port=8088/tcp
firewall-cmd --zone=internal --list-all
firewall-cmd --permanent --zone=internal --list-all
```

期望得到 yes/yes，并确认没有无关来源、宽端口范围或 rich rule 冲突。此时只完成防火墙状态，仍需要远端应用验收。

⑥ 验证 从独立客户端完成协议访问

在 `clienta` 执行：

```
curl -I http://192.0.2.20:8088/
nc -vz 192.0.2.20 8088
```

`curl` 是最终 HTTP 证据；`nc` 只作为 TCP 辅助。成功后把证据记录为“`clienta` → `192.0.2.20:8088` HTTP 可用”，不要扩大为所有来源或所有协议均正确。

⑦ 诊断 典型错误分支

- `ss` 无 listener：回到服务配置、日志和 SELinux 端口边界；
- 回环成功、业务地址失败：检查监听地址；
- 本机业务地址成功、远端超时：检查实际 zone、runtime、rich rule 和上游路径；
- runtime yes、permanent no：补持久规则；
- permanent yes、runtime no：补 runtime，或审查后 reload；
- 只添加 `http` service：读取 service 定义，确认是否覆盖真实 8088。

答案主线 服务 → listener → 本机协议 → 实际 zone → runtime/permanent → 远端协议。参数选择由真实对象决定，而不是从“Web 服务”三个字直接猜命令。
经典任务

修复“permanent 已配置但远端仍然超时”

预置场景

- `httpd` 正在 `0.0.0.0:8088` 监听；
- 在服务器本机访问 `http://192.0.2.20:8088/` 成功；
- `firewall-cmd --permanent --zone=public --query-port=8088/tcp` 返回 yes；
- `firewall-cmd --zone=public --query-port=8088/tcp` 返回 no；

- `clienta` 访问超时;
- 当前接口/来源绑定尚未调查。

要求

1. 解释为什么 `permanent` 中存在规则仍不足以证明当前放行;
2. 找出实际处理 `clienta` 请求的 `zone`;
3. 在不关闭 `firewalld`、不移动接口到 `trusted` 的条件下完成最小修复;
4. 使正确 `zone` 的 `runtime` 与 `permanent` 都满足目标;
5. 对 `public` 中的遗留规则先调查依赖, 再决定是否删除;
6. 用远端 HTTP 请求完成最终验收。

请先独立完成。参考解答从下一页开始。

参考解答

区分“持久配置存在”和“实际 zone 当前允许”

① 判断 已知证据已经排除什么

服务 `listener` 和本机业务地址请求都正常, 说明应用层至少在服务器本机成立。`public permanent` 为 `yes` 只证明下次加载时 `public` 包含规则; `public runtime` 为 `no` 说明当前 `public` 尚未允许。更关键的是, 尚不知道请求是否进入 `public`。

② 调查 查询实际分类

```
firewall-cmd --get-active-zones
firewall-cmd --get-zone-of-interface=enp1s0
firewall-cmd --get-zone-of-source=192.0.2.0/24
```

假设结果显示 `clienta` 来源或 `enp1s0` 实际使用 `internal`。那么 `public` 中的规则与当前请求没有直接关系。

③ 调查 核对正确 zone 的两份状态

```
firewall-cmd --zone=internal --query-port=8088/tcp
firewall-cmd --permanent --zone=internal --query-port=8088/tcp
firewall-cmd --zone=internal --list-all
firewall-cmd --permanent --zone=internal --list-all
```

若得到 `no/no`, 根因是目标规则写入了错误 `zone`, 同时当前 `runtime` 未允许。

④ 操作 在正确 zone 建立最小终态

```
firewall-cmd --zone=internal --add-port=8088/tcp
firewall-cmd --permanent --zone=internal --add-port=8088/tcp
```

这里不需要 `reload`, 因为分别修改两层即可完成目标, 也不会覆盖其他 `runtime-only` 状态。

⑤ 验证 双态与远端协议

```
firewall-cmd --zone=internal --query-port=8088/tcp
firewall-cmd --permanent --zone=internal --query-port=8088/tcp
```

然后在 `clienta`:

```
curl -I http://192.0.2.20:8088/
```

只有远端 HTTP 成功, 才完成本题业务终态。

⑥ 清理 处理 `public` 遗留规则

先确认 `public` 当前绑定的接口、来源和其他业务:

```
firewall-cmd --zone=public --list-all
firewall-cmd --permanent --zone=public --list-all
```


若确认 8088 不再被任何请求或业务需要，再删除持久遗留项：

```
firewall-cmd --permanent --zone=public --remove-port=8088/tcp
```

是否同时处理 runtime 取决于其当前状态；已知 runtime 为 no，无需为“对称”而执行无意义删除。最后再次查询两个 zone，并确认远端业务不受影响。

答案主线 permanent 存在 ≠ 当前生效；目标规则存在 ≠ 请求经过该 zone。先找实际分类，再修正 zone，最后才清理错误位置。
本章收束

把“开放端口”改写为可验证的服务访问链

firewalld 的核心不是参数数量，而是三个判断：请求进入哪里、当前允许什么、下次加载恢复什么。zone 与绑定回答规则作用范围；service 和 port/protocol 回答允许对象；runtime 与 permanent 回答时间维度；rich rule 只在来源限制等必要条件扩展表达。

真正的业务终态还在 firewalld 之外。服务 active、listener、本机协议、SELinux 端口边界、远端协议访问分别属于不同层。稳定操作始终遵循：先调查会改变决策的证据，做最小修改，立即验证局部状态，再推进到下一层。

主要判断表

看到的证据	正确解释	下一步
服务 active	systemd 当前管理状态成立	用 ss 查 listener
ss 监听 127.0.0.1	仅回环可用	修复应用监听地址
runtime yes、permanent no	当前允许，reload 后丢失	需要持久时补 permanent
runtime no、permanent yes	持久已有，当前未应用	补 runtime 或审查后 reload
目标规则在 public	只证明 public 被配置	查询实际接口/来源 zone
curl 127.0.0.1 成功	回环 HTTP 路径成立	测服务器地址，再测远端
远端超时	某处无响应，根因未定	本机协议 → zone → runtime → 上游路径
--check-config 通过	permanent 可解析	reload/查询/业务验收仍需执行

工作方法

1. 把任务写成服务、地址、端口、协议和来源
2. 证明服务与真实 listener
3. 用本机请求隔离应用问题
4. 查询实际 interface/source 绑定
5. 在实际 zone 中维护最小规则
6. 分别验证 runtime 和 permanent
7. 从独立客户端完成真实协议请求
8. 清理前调查依赖，reload 前审查双态差异

向下一章交接

本章结束后，读者已经能把网络服务访问拆成对象、状态和证据链。下一章进入块设备、分区表与设备签名：对象从“入站流量与规则”切换为“设备、分区和磁盘上的持久标识”，但方法保持一致——先确认对象身份，再修改状态，最后用独立证据验证终态。

静态可信声明：本章依据 RHEL 9 课程、正式项目规范和官方 firewalld 语义完成静态核对。当前会话未连接 RHEL 9 虚拟机，因此不声称命令已在真实环境跑通；需要实机确认的版本行为已记录在修订说明中。

块设备、分区表与设备签名

从“这是不是目标盘”到“磁盘、内核与 udev 是否一致”：把高风险存储操作放进一条可证明的证据链。

对象模型操作语义验证诊断经典任务
大字号阅读版

本章阅读导航

先抓住一条主线：任何分区写操作都必须先证明设备身份、已有布局、内容签名和当前使用状态；写入后再证明磁盘表、内核设备树与 udev 已经收敛。

01 识别设备从路径、父子关系和身份字段确认目标对象

02识别布局区分 GPT/MBR、分区条目和真实扇区边界</div><div class="model-card">03识别内容读取文件系统、Swap、LVM、RAID 等签名</div><div class="model-card">04决定操作只在证据门通过后选择 parted 或 fdisk</div><div class="model-card">05同步视图让磁盘表、内核视图和 udev 节点重新一致</div><div class="model-card">06分层验收分别证明布局、节点、持久标识和内容边界</div>

专题地图

- 知识专题 从磁盘到分区：块设备对象怎样关联
- 知识专题 GPT、MBR 与分区条目分别记录什么
- 知识专题 起点、终点、大小、对齐与分区类型
- 知识专题 设备签名与持久标识
- 操作专题 写入前基线、parted、fdisk 与状态同步
- 诊断专题 新分区未出现、遗留签名与身份冲突
- 经典任务 创建 GPT 分区并完成四层验收
- 经典任务 诊断内核节点缺失和旧签名

<div><h2>阅读时持续回答</h2><ol class="questions">当前路径到底对应哪一个块设备？分区表、分区类型和内容签名分别属于哪一层？设备是否正在被挂载、Swap、LVM、RAID 或映射层使用？命令修改的是磁盘表、内核视图还是 udev 状态？当前证据能证明什么，又不能证明什么？出现冲突时，下一条最有区分度的证据是什么？<div class="nav-note">章节边界：本章只把设备、分区表、分区条目、内核视图、持久标识和签名讲清。文件系统进入第 23 章；挂载与 Swap 持久性进入第 24 章；LVM 生命周期进入第 25 章。</div></div>

第 22 章 • 正文

存储操作最危险的地方，不是命令难记，而是“看起来很像正确设备”的对象太多。`/dev/vdb` 可能是练习盘，也可能已经承载文件系统、Swap、LVM、RAID 或业务数据；`parted` 已经显示新分区，不代表内核立即生成了 `/dev/vdb1`；`FSTYPE` 为空，也不代表设备可以覆盖。一次错误的 `mklabel`、`rm` 或签名清除，可能让原有数据失去正常访问入口。

本章解释块设备、磁盘、分区、分区表、内容签名、内核设备视图和持久标识之间的关系，并沿“身份 → 布局 → 内容 → 占用 → 修改 → 同步 → 验收”推进。最常见的误判，是用一个局部字段替代完整证据：只凭设备名选盘、只凭 `FSTYPE` 判空、只凭分区类型判断内容、只凭命令成功宣布完成。

本章的写操作只改变分区表及分区条目；它不会提前创建文件系统、配置挂载、启用 Swap 或建立 LVM。所有需要真实输出才能确认的版本与设备行为都保留为推荐验证步骤，不声称已经在 RHEL 9 虚拟机实测。

概念

块设备 是内核以固定大小数据块提供随机访问的设备对象，磁盘、分区以及某些映射设备都可能表现为块设备。块设备节点只是访问入口，不自动说明设备的业务用途、是否空闲或是否安全覆盖；观察它应先从 `'lsblk'` 的父子关系、类型和身份字段开始。

概念

分区 是分区表把整块磁盘的一段地址范围描述成的子对象。分区本身仍是块设备，但它和文件系统、Swap、LVM PV 不是同一对象：分区条目只定义边界和用途元数据，上层内容需要另外创建和验证。

概念

分区表 是写在整块磁盘上的布局元数据，GPT 或 MBR 记录分区条目的起止、类型和标识。重建分区表不是无害初始化，而是替换原布局入口；即使数据区没有全部覆盖，原分区也可能立刻失去正常访问路径。

概念

设备签名 是文件系统、Swap、LVM、RAID 或分区表留在特定偏移的识别元数据。签名只是一段识别入口，却足以影响自动探测和上层工具；删除少量签名字节也可能让完整数据失去正常识别，因此 `'wipefs -a'` 绝不能成为默认清理步骤。

概念

内核设备视图 是内核当前已经读取并注册的块设备树。磁盘上的分区表已经改变时，内核仍可能暂时保留旧布局；`'partprobe'` 请求重读分区表，`'udevadm settle'` 只等待设备事件处理完成，两者解决的是不同层次。

概念

持久标识 把设备名称稳定在某个对象维度，例如硬件或后端身份、控制器路径、分区条目或文件系统内容。`'/dev/disk/by-id/'`、`'by-path/'`、`'by-partuuid/'` 和 `'by-uuid/'` 的稳定依据不同；它们不是“永不变化”，而是比短设备名更明确地说明自己在跟踪什么。

概念

对齐 是让分区边界与设备逻辑扇区、物理扇区和最优 I/O 单元合理匹配。1 MiB 起点适合许多常见设备，但不是脱离设备拓扑的绝对规则；考试和工作中应优先接受工具的合理默认值，并用扇区、大小和 I/O 字段验证结果。

操作语义速查

以下七个入口分别观察拓扑、签名、分区表、内核重读和 `udev` 状态。先明确命令作用对象，再记关键形式。

`lsblk`

SYNOPSIS

```
lsblk [options] [device ...]
```

从 `sysfs` 与 `udev` 数据建立当前块设备树，适合确认父子关系、类型、身份字段、分区标识和挂载摘要。

重要参数 / 形式

`-o LIST`

显式选择输出列；讲义、脚本和验收记录不要依赖默认列。

`-f`

显示文件系统和 Swap 的常见属性摘要，但不能代替完整签名与占用调查。

`-p`

使用完整设备路径显示名称。

`-J`

输出 JSON，便于结构化处理；仍应显式指定列，并在需要父子树时保留 `NAME` 或使用 `--tree`。

blkid

SYNOPSIS

```
blkid [options] [device ...]
blkid -p [options] device
```

读取 libblkid 可识别的内容标签、类型、UUID 和分区属性；低层探测更适合调查单个可疑设备。

重要参数 / 形式

`blkid device`

读取常见属性；无普通输出不能直接证明设备为空。

`-p`

对指定设备进行低层探测，适合冲突或不确定签名。

`-o export`

以键值形式输出，便于逐项判断。

wipefs

SYNOPSIS

```
wipefs [options] device
```

默认列出可识别签名和偏移；带删除选项时会破坏这些识别入口。

重要参数 / 形式

`wipefs device`

只读列出签名，是调查流程的默认形式。

`-n`

与 `-a` 或 `-o` 组合时，以 `no-act` 方式预览将处理的签名，不执行写入。

`-a`

删除指定设备上所有可见签名；作用于整盘时通常只清除整盘可见的分区表签名，不会递归清除各分区内容。高风险，禁止作为空盘检查或默认清理步骤。

`-o OFFSET`

只处理已确认偏移的签名，仍需先完成身份和占用调查。

parted

SYNOPSIS

```
parted [options] [device [command [parameters ...]]]
```

查看或直接修改 GPT/MBR 分区表；许多修改会立即写盘，没有统一的最终提交阶段。

重要参数 / 形式

`print`

显示当前分区表、分区条目和设备信息。

`unit MiB|s`

指定输入与显示单位，避免把大小误当终点。

`mklabel gpt|msdos`

重建整盘分区表，属于极高风险写操作。

`mkpart NAME [FS-TYPE] START END`

GPT 场景：`NAME` 是分区名称；`FS-TYPE` 只用于设置合适的分区类型提示，不会创建文件系统。

`mkpart PART-TYPE [FS-TYPE] START END`

MBR 场景：`PART-TYPE` 通常为 `primary`、`extended` 或 `logical`；起止必须使用明确单位。

`rm NUMBER`

删除分区条目，不迁移数据，也不安全擦除数据区。

fdisk

SYNOPSIS

```
fdisk [options] device
fdisk -l [device ...]
```

查看或交互式编辑分区表；交互修改先保留在内存中的未提交布局里，直到写入命令提交。

重要参数 / 形式

`-l`

只读列出磁盘、扇区、I/O 大小、分区边界和类型。

`p`

打印当前未提交布局。

`n / t / d`

分别创建分区、修改类型和删除条目。

`w`

把未提交布局写入磁盘并退出。

`q`

放弃尚未写入的交互修改并退出。

partprobe

SYNOPSIS

```
partprobe [options] [device ...]
```

向内核报告分区表变化，请求重新读取指定磁盘的布局。

重要参数 / 形式

`partprobe device`

请求重读单个目标磁盘，错误信息是后续占用调查的重要证据。

`-s`

显示设备及其分区摘要；它仍不能替代 `parted`、`lsblk` 的分层验证。

`-d`

只测试而不通知内核；用于区分探测与实际重读。

udevadm info / settle

SYNOPSIS

```
udevadm info [options] device
udevadm settle [options]
```

`info` 查询设备属性和 `udev` 数据库；`settle` 等待当前设备事件队列处理完成。

重要参数 / 形式

`info --query=property --name=device`

读取指定设备的属性，适合验证 PARTUUID、分区名称和路径信息。

`settle`

等待事件收敛；它不修改分区表，也不保证内核重读成功。

`monitor`

观察后续内核与 `udev` 事件，适合诊断设备节点没有按预期生成。

安全边界：任何写操作前必须证明设备身份、挂载与 Swap 状态、LVM/RAID/device-mapper 归属以及已有签名。`wipefs -a`、`mklabel` 和删除分区都不能用来“试试看能否清干净”。证据不足时，正确动作是停止写入并继续调查。

[知识专题] 从磁盘到分区：块设备对象怎样关联

刚接触存储时，容易把“设备文件”“磁盘”“分区”“文件系统”混成一个对象。最顺的切入方式是从内核设备树开始：整块磁盘是父对象，分区是它的子对象；文件系统、Swap、LVM 或 RAID 签名可以写在整盘或分区上，但它们属于更高一层的内容。本章只负责识别这些内容，不完整展开文件系统、挂载和 LVM 生命周期。

① [知识点] 磁盘与分区都是块设备，但承担不同角色

lsblk 的 TYPE 字段可以帮助区分常见对象：

TYPE	常见含义	本章判断重点
disk	整块磁盘或虚拟磁盘	可承载分区表，也可能直接承载签名
part	分区表描述的磁盘范围	必须确认所属父磁盘和边界
rom	光驱类设备	通常不作为分区目标
loop	回环设备	多由镜像或容器运行时建立
lvm	device-mapper 上的 LVM 逻辑卷	属于第 25 章主对象
crypt	加密映射设备	不是普通分区条目

TYPE=part 只能证明内核把它识别为分区设备，不能证明其内容类型、是否空闲或是否安全删除。

② [知识点] 设备命名反映驱动和总线，不保证长期不变

常见命名形式包括：

- SCSI、SATA、USB 及许多虚拟控制器： /dev/sda、 /dev/sdb ；
- virtio-blk： /dev/vda、 /dev/vdb ；
- NVMe： 整盘 /dev/nvme0n1，分区 /dev/nvme0n1p1 ；
- MMC： 整盘 /dev/mmcblk0，分区 /dev/mmcblk0p1。

NVMe 与 MMC 名称中的 p 用于把磁盘编号和分区编号分开。设备枚举顺序可能因控制器探测、虚拟机配置或硬件变化而改变，因此不能把“上次是 /dev/sdb”当作本次写入依据。

③ [知识点] 父子关系比短设备名更能解释对象

以下字段共同建立拓扑：

- NAME：简短内核名称；
- PATH：完整设备路径；
- TYPE：设备类型；
- PKNAME：父内核设备名；
- MAJ:MIN：内核主次设备号；
- 树形缩进：显示分区、映射层和父设备关系。

对 /dev/vdb2 进行操作前，应确认它确实位于目标 /dev/vdb 下，而不是同名脚本变量误指向另一块盘。

④ [知识点] 身份确认需要多条一致证据

破坏性写入前，至少交叉核对以下几类证据：

```
设备路径和父子关系
→ 容量
→ MODEL / SERIAL / WWN（可用时）
→ 现有分区表与签名
→ 挂载点和上层归属
→ 题目或变更单指定的目标
```

虚拟磁盘可能没有 SERIAL 或 WWN；这不表示无法识别，而是需要依靠虚拟机配置、容量、控制器槽位、父子关系和现有状态组合判断。同容量磁盘较多时，单凭大小仍不足以放行写操作。

⑤ [知识点] lsblk 默认输出不是稳定接口

不同 util-linux 版本可能调整默认列，因此讲义、考试记录和脚本应显式指定字段。调查命令可以分为两个层次：

```
# 快速拓扑
lsblk -o NAME,PATH,SIZE,TYPE,PKNAME,MOUNTPOINTS

# 破坏性操作前的扩展视图
lsblk -o NAME,PATH,MAJ:MIN,SIZE,TYPE,PKNAME,FSTYPE,FSVER,LABEL,UUID,PARTLABEL,PARTUUID,PTTYPE,PTUUID,MOUNTPOINTS,MODEL,SERIAL,WWN
```

若某列在目标 RHEL 9 小版本不可用，应通过 lsblk --help 查看本机支持的列并移除该列，而不是改用默认输出后猜字段位置。

[Cheatsheet] 先看树，再看身份；PATH 是访问入口，SERIAL/WWN 是身份线索，PKNAME 说明父子关系；任何单个字段都不足以证明设备可以覆盖。

[知识专题] GPT、MBR 与分区条目分别记录什么

分区表是整块磁盘上的元数据结构，它描述分区边界、类型和标识。选择 GPT 或 MBR，不是在现有数据盘上切换显示风格，而是在决定磁盘布局格式。理解这一点后，`mklabel` 的风险、分区类型的边界和持久标识的归属会更清楚。

① [知识点] 分区表属于整盘，分区条目描述地址范围

一个分区条目至少需要表达：

- 分区编号；
- 起始位置；
- 结束位置或长度；
- 类型信息；
- 在 GPT 中可用的分区名称和唯一标识。

分区表写入磁盘后，内核仍需重新读取这些条目，才会生成或更新分区设备节点。

② [知识点] MBR 是受历史结构限制的旧布局

传统 MBR 在磁盘开头保存引导信息和四个分区表项，因此常见“最多四个主分区”的限制。若需要更多分区，可使用一个扩展分区作为容器，再在其中建立逻辑分区。

在 512 字节逻辑扇区的常见情形下，MBR 使用 32 位扇区地址，通常面临约 2 TiB 的寻址限制。该限制与扇区大小有关，因此不要把“2 TiB”脱离上下文解释为所有设备上的绝对数学上限。

③ [知识点] GPT 提供主、备份元数据和 GUID 标识

GPT 通常在磁盘前部保存主头和分区条目数组，并在磁盘末尾保存备份。它还包含保护性 MBR，以降低旧工具把 GPT 磁盘误认为空盘的风险。

GPT 使用 GUID 表达磁盘和分区身份及类型，通常比 MBR 更适合现代大容量磁盘和较多分区。工具常默认建立 128 个分区条目，但这属于常见实现选择，不应误记为 GPT 格式只能有 128 个分区。

④ [知识点] PTYPE、PTUUID、PARTUUID 和 PARTLABEL 不在同一层

标识	所属对象	典型变化时机
PTTYPE	整盘分区表格式	GPT 与 MBR 之间重建布局时改变
PTUUID	整盘分区表标识	重建分区表时通常改变
PARTUUID	某个分区条目	删除并重建分区时通常改变
PARTLABEL	GPT 分区名称	管理员可修改，且不保证全局唯一
文件系统 UUID	分区或设备上的文件系统签名	重新格式化时通常改变，归第 23 章深入

不能用文件系统 UUID 代替分区表 UUID，也不能因为 `PARTLABEL=data` 就认定它必然承载某个文件系统。

⑤ [知识点] mklabel 是重建布局，不是无害初始化

```
parted -s <DISK> mklabel gpt
```

这条命令会在目标磁盘上建立新的 GPT 元数据，使原分区布局失去正常访问入口。即使数据区未被逐字节覆盖，原内容也可能无法再通过原分区节点访问。因此只有在设备身份、占用和数据保留要求已经明确时才能执行。

[Cheatsheet] GPT/MBR 属于整盘；分区条目属于表；PTUUID 标识表，PARTUUID 标识条目；mklabel 会替换布局，绝不是查看命令。

[知识专题] 起点、终点、大小、对齐与分区类型

分区题经常给出“创建 512 MiB”或“从 1 MiB 开始”的要求。工具实际写入的是扇区边界，因此显示值可能因单位换算和对齐略有差异。稳定做法是明确单位、理解起止语义，并在写入后验证实际结果。

① [知识点] 绝对终点与相对大小是两种表达

parted 常用绝对起止位置，例如：

```
parted -s /dev/vdb mkpart data 1MiB 2049MiB
```

该分区从 1 MiB 到 2049 MiB，名义跨度约 2048 MiB。`fdisk` 则常在起始扇区处接受相对大小：


```
Last sector ...: +512M
```

考试时应先判断题目给的是最终大小还是终点位置，不能把 `2048MiB` 直接当作绝对终点。

② [知识点] MiB/GiB 与 MB/GB 不是同一种单位

- `MiB`、`GiB` 使用 1024 的幂；
- `MB`、`GB` 使用 1000 的幂；
- `s` 表示扇区。

对大小评分敏感的任务，优先使用题目要求的单位或明确的二进制单位，并在输出中检查实际扇区数和显示大小。

③ [知识点] 对齐是为了匹配设备 I/O 边界

现代分区工具会依据逻辑扇区、物理扇区和最优 I/O 大小选择合适的默认起点。使用 1 MiB 起点通常能与常见 4 KiB 物理扇区和更大擦除块较好对齐，但它不是“所有设备唯一正确”的神奇数字。

可用以下证据检查几何信息：

```
lsblk -o NAME,PHY-SEC,LOG-SEC,MIN-IO,OPT-IO,ALIGNMENT
fdisk -l <DISK>
parted <DISK> unit s print
```

在考试新盘上，接受工具合理默认值通常比手工输入未经计算的扇区更安全。

④ [知识点] 分区类型表达预期用途，不创建上层对象

将分区设为 LVM、Swap 或 Linux filesystem 类型，只改变分区表中的用途元数据。它不会：

- 创建文件系统；
- 写入 Swap 签名；
- 创建 LVM PV；
- 自动挂载或启用设备。

后续章节会分别处理这些上层对象。本章只负责正确设置和验证分区条目。

⑤ [知识点] MBR 类型代码和 GPT 类型不能机械混用

模拟题可能在 MBR 场景中要求把类型代码设为 `82` 或 `8e`。GPT 使用 GUID 类型，现代 `fdisk` 也可能显示类型别名。稳定做法是：

```
先确认当前分区表类型
→ 在工具中列出可选类型
→ 选择与题意匹配的名称或代码
→ 写入后重新打印验证
```

不要把某个 MBR 十六进制代码背成所有环境下的唯一输入。

⑥ [知识点] `parted` 的文件系统类型提示不等于 `mkfs`

某些 `parted mkpart` 形式允许提供 `FS-TYPE`，该信息主要帮助选择分区类型或兼容标志。它不会在分区上建立文件系统。创建完成后，如果 `FSTYPE` 仍为空，这是符合本章任务边界的状态。

[Cheatsheet] `parted` 常用绝对起止，`fdisk` 常用 `+SIZE`；单位写清楚；类型是元数据，不是格式化；边界以写入后的扇区和大小为准。

[知识专题] 设备签名与持久标识：设备上已经有什么

分区表只描述布局，内容签名告诉工具“这个地址范围看起来是什么”。文件系统超级块、Swap 头、LVM 标签和 RAID 元数据都可能被 libblkid 识别。旧实验留下的签名可能与新分区表同时存在，导致自动探测、安装程序或管理工具给出冲突判断。

① [知识点] 签名是识别入口，不是完整数据本身

签名通常是位于特定偏移的魔数和元数据片段。工具通过它识别类型、版本、标签和 UUID。删除签名可能只改动少量字节，却足以使正常工具无法识别原内容，因此仍然属于破坏性操作。

② [知识点] 一个设备可以残留多层或多种签名

例如，一个曾经作为 LVM PV 的分区后来被直接格式化，设备上可能同时残留旧 LVM 元数据和新文件系统签名。又如，整盘曾使用 GPT，后来创建 MBR 时，磁盘末尾仍可能残留 GPT 备份头。

看到多个签名时，不能立即把它们全部视为垃圾。需要判断：

- 哪个签名对应当前有效对象
- 哪个签名是历史残留
- 上层是否仍在使⽤
- 题目是否明确允许丢弃

③ [知识点] blkid 普通模式与低层探测用途不同

常见查询：

```
blkid <DEVICE>
blkid -p <DEVICE>
blkid -p -o export <DEVICE>
```

普通模式适合读取常见标签、UUID 和类型；低层 -p 直接探测指定设备，更适合调查冲突或不确定签名。若普通 blkid 没有输出，不能直接得出“设备为空”的结论。

④ [知识点] wipefs 默认列出签名和偏移

```
wipefs <DEVICE>
```

该命令用于列出工具可识别的签名及其偏移，不会因为命令名包含“wipe”就默认清除内容。真正危险的是带删除选项的形式，例如 -a。调查流程中应保持只读。

⑤ [知识点] wipefs -a 不是空盘检查命令

wipefs -a <DEVICE> 会删除指定设备上 libblkid 可见的签名。对整盘设备执行时，通常只清除整盘上的分区表签名，不会递归清除各分区内部的文件系统或 LVM 签名；对分区设备执行时，则可能清除该分区上的文件系统、Swap、LVM 或 RAID 签名。它不等于安全擦除数据，但足以破坏正常识别入口，因此默认答案中禁止无调查使用。

若题目明确授权丢弃设备内容，仍应先：

1. 保存 wipefs 和 blkid -p 输出；
2. 确认设备未被挂载、启用或作为上层成员；
3. 使用 wipefs -n -a <DEVICE> 进行模拟检查；
4. 优先定向清除已确认的签名，而不是盲目全清；
5. 清除后重新验证分区表、签名和设备视图。

⑥ [知识点] 文件系统、Swap、LVM、RAID 签名需要不同的上层证据

FSTYPE=trfs、TYPE=swap、LVM2_member 或 linux_raid_member 只是入口。真正决定是否可覆盖，还要查看对应上层对象是否存在和是否活动。例如：

- 文件系统：检查挂载关系；
- Swap：检查当前是否启用；
- LVM：检查是否属于 PV/VG/LV；
- RAID：检查是否属于阵列；
- device-mapper：检查是否有映射和 holders。

本章只建立调查接口，不完整展开这些上层生命周期。

⑦ [知识点] 持久链接稳定在不同维度

链接目录	主要稳定维度	可能变化的场景
/dev/disk/by-id/	设备序列、WWN 或后端标识	后端克隆、序列缺失或虚拟化配置变化
/dev/disk/by-path/	控制器、端口和拓扑路径	设备换端口或控制器路径改变
/dev/disk/by-partuuid/	GPT/MBR 分区条目标识	删除重建分区或重建分区表
/dev/disk/by-partlabel/	GPT 分区名称	改名或出现同名分区
/dev/disk/by-uuid/	文件系统或 Swap UUID	重新格式化或重写签名，归后续章节深入

持久链接不是“永不变化”，而是把名称稳定在某个对象维度。选择前先明确自己要稳定的是硬件身份、物理路径、分区条目还是文件系统内容。

[Cheatsheet] blkid 读常见属性，blkid -p 做低层探测，wipefs 看签名和偏移；无输出不等于空盘，多签名不等于都可删除。

[操作专题] 破坏性操作前建立设备基线

稳定的存储操作从调查开始，而不是从 `fdisk` 或 `parted` 开始。最顺的顺序是：先确认目标身份，再确认布局和签名，随后确认当前占用和上层归属，最后决定是否允许写入。

① [查询] 先建立拓扑和身份视图

```
lsblk -o NAME,PATH,MAJ:MIN,SIZE,TYPE,PKNAME,MOUNTPOINTS,MODEL,SERIAL,WWN
```

作用对象：内核当前维护的块设备树。验证重点：目标路径、父子关系、容量和身份字段是否与题意一致。边界：该命令不读取所有历史签名，也不能证明设备没有被其他上层对象使用。

② [查询] 查看分区表和分区边界

```
parted <DISK> unit MiB print
fdisk -l <DISK>
```

`parted` 适合明确显示分区表类型、起止和名称；`fdisk -l` 提供扇区、I/O 大小和分区类型的独立视角。两条输出应指向同一设备，并与 `lsblk` 的父子关系一致。

③ [查询] 探测内容签名

```
blkid -p <DISK>
wipefs <DISK>

# 对已有分区逐一检查
blkid -p <PARTITION>
wipefs <PARTITION>
```

对整盘和分区分别检查，因为分区表签名位于整盘，而文件系统、LVM 或 RAID 签名可能位于分区中。

④ [查询] 检查挂载、Swap 和上层归属

```
lsblk -o NAME,PATH,TYPE,FSTYPE,MOUNTPOINTS
findmnt --source <DEVICE>
swapon --show
pvs
cat /proc/mdstat
```

这些命令分别排除常见占用。若设备存在 device-mapper 映射、加密层或 holders，还应检查：

```
ls -l /sys/class/block/<KERNEL_NAME>/holders/
```

不存在某一种占用，不等于不存在其他占用。

⑤ [验证点] 明确写入放行条件

只有以下条件同时满足时，才进入创建或删除分区：

- 设备身份与目标一致
- 现有布局和签名已记录
- 没有需要保留的数据
- 没有当前挂载、Swap 或上层成员关系
- 题目或变更单明确授权改变该设备

若任何条件不明确，正确动作是停止并继续调查，而不是使用 `--force`、`wipefs -a` 或重新格式化来“清理异常”。

⑥ [工作迁移] 保存变更前基线

真实工作中可把输出保存到变更记录：

```
stamp=$(date +%Y%m%d-%H%M%S)
lsblk -O > "lsblk-before-${stamp}.txt"
parted <DISK> unit s print > "parted-before-${stamp}.txt"
fdisk -l <DISK> > "fdisk-before-${stamp}.txt"
wipefs <DISK> > "wipefs-before-${stamp}.txt"
```

考试中不必机械创建文件，但应保留“先观察、后写入、再比较”的思路。

[Cheatsheet] 身份 → 布局 → 签名 → 占用 → 授权；任一证据冲突都停止。FSTYPE 为空只是一个字段，不是写入许可。

[操作专题] 使用 parted 查看和修改分区表

parted 同时支持 GPT 和 MBR，并适合使用明确单位进行非交互操作。它的危险点是许多修改会立即写入磁盘，不存在统一的“最后按 w 才提交”阶段。因此每条命令前都应重新确认设备路径。

① [查询] 查看当前布局和帮助

```
parted <DISK> print
parted <DISK> unit MiB print
parted <DISK> help
```

在交互模式中可以先执行 print，确认设备型号、容量、分区表类型和现有条目。unit 只改变显示和输入单位，不改变已有分区内容。

② [操作] 在确认可丢弃的新盘上建立 GPT

```
parted -s <DISK> mklabel gpt
```

作用对象：整块磁盘的分区表。风险：原布局失去正常访问入口。验证：立即运行 parted <DISK> print 和 lsblk，确认没有写错设备。

③ [操作] 使用明确起止创建分区

```
parted -s <DISK> mkpart data 1MiB 2049MiB
parted -s <DISK> mkpart future-lvm 2049MiB 3073MiB
```

GPT 的第一个参数常作为分区名称。这里没有创建文件系统或 LVM PV，只建立了两个分区条目。

④ [参数] 设置分区用途标志

```
parted -s <DISK> set 2 lvm on
```

该操作把第 2 分区标记为 LVM 用途。它不验证分区中是否已有 LVM 签名，也不会执行 pvcreate。

⑤ [操作] 删除分区条目

```
parted -s <DISK> rm <NUMBER>
```

rm 删除分区表中的条目，不会安全迁移或删除原数据区。删除后，上层对象可能立即失去设备节点。执行前必须确认编号、父磁盘、挂载和上层归属。

⑥ [验证点] 同时检查磁盘表和内核树

```
parted <DISK> unit MiB print
fdisk -l <DISK>
lsblk -o NAME,PATH,START,SIZE,TYPE,PARTLABEL,PARTUUID,PARTTYPE <DISK>
```

parted / fdisk 读取磁盘布局，lsblk 主要反映内核与 udev 视图。若两者不一致，进入同步诊断，不要重复执行 mklabel 或 mkpart。

⑦ [帮助入口] 不确定子命令时先查看本机帮助

```
parted --help
parted <DISK> help mkpart
man parted
```

版本差异最容易出现在类型、flag 和脚本化输出上。考试环境以本机帮助为准。

[Cheatsheet] print 先看：mklabel 改整盘；mkpart 改条目；set ... lvm on 只改用途；parted 修改常立即生效。

[操作专题] 使用 fdisk 交互式管理分区

fdisk 的特点是交互式未提交布局：多数修改先保存在内存中，执行 **w** 才写入磁盘，执行 **q** 可放弃未写入变化。这并不降低设备选错的风险，但提供了提交前再次打印检查的机会。

① [查询] 只读列出布局

```
fdisk -l <DISK>
```

输出可包含逻辑/物理扇区大小、I/O 大小、磁盘标签类型、磁盘标识和分区表。人类可读输出可能随版本变化，不应依赖固定列宽进行脆弱解析。

② [操作] 进入交互并先打印

```
fdisk <DISK>
```

进入后首先使用：

p	打印当前未提交布局
m	查看帮助

当前未提交布局可能已包含本轮尚未写入的变化，因此 **p** 显示的是当前交互会话状态，不一定等于磁盘原始状态。

③ [操作] 创建分区

常见按键：

n	新建分区
<Enter>	接受合理默认起始扇区
+512M	指定相对大小

默认起点通常已经考虑对齐。若题目没有强制扇区，不要为了“看起来整齐”输入未经计算的起点。

④ [操作] 修改分区类型

t	修改类型
L	列出类型（提示中支持时）

先确认当前是 GPT 还是 MBR，再选择类型名称、别名或代码。写入前用 **p** 检查最终类型。

⑤ [操作] 删除未提交的分区条目

```
d      删除分区条目
```

此时通常还未写盘，但删除后继续执行 **w** 就会提交。若发现选错设备或编号，使用 **q** 退出，不要通过继续创建来“补回来”。

⑥ [提交边界] **w** 与 **q** 的意义

w	写入分区表并退出
q	不保存本次未写入变化并退出

q 只能放弃当前交互会话中尚未写盘的修改，不能撤销之前已执行并写入的其他工具命令。

⑦ [验证点] 写盘后重新从外部读取

退出后执行：

```
fdisk -l <DISK>
parted <DISK> unit MiB print
lsblk -o NAME,PATH,START,SIZE,TYPE,PARTTYPE,PARTUUID <DISK>
```

不要只相信交互会话最后一次 **p** 的屏幕内容。

[Cheatsheet] **p** 看未提交布局，**n** 新建，**t** 改类型，**d** 删除，**w** 写盘，**q** 放弃未提交；写盘后必须从外部重新查询。

[操作专题] 让磁盘布局、内核视图和 udev 重新一致

分区工具已经显示新条目，但 `/dev/vdb2` 没有出现，说明故障不一定在分区表。需要分开判断：磁盘上是否已写入、内核是否重读、udev 是否完成节点和链接处理。

① [知识点] 三个状态源不能互相替代

磁盘上的分区表
→ 内核块设备和分区对象
→ udev 设备节点、属性与持久链接

`parted print` 能看到新分区，只证明磁盘表可被工具读取；`lsblk` 能看到分区，说明内核已建立对象；`/dev/disk/by-partuuid/` 出现链接，说明相应 udev 规则已完成。

② [操作] 请求内核重读分区表

```
partprobe <DISK>
```

`partprobe` 通知操作系统分区表已改变。它不会创建新的分区表，也不会清除签名。如果设备正在使用，内核可能拒绝安全更新。

③ [操作] 等待 udev 事件处理

```
udevadm settle
```

该命令等待当前设备事件队列处理完成。它不会主动重读分区表，也不会修复错误的分区条目。

④ [查询] 查询具体设备的 udev 属性

```
udevadm info --query=property --name=<PARTITION>
```

可核对 `ID_PART_ENTRY_UUID`、`ID_PART_ENTRY_NAME`、父设备和其他属性。实际字段取决于设备和规则。

⑤ [验证点] 检查设备节点与持久链接

```
lsblk -o NAME,PATH,TYPE,PKNAME,PARTLABEL,PARTUUID <DISK>
ls -l /dev/disk/by-partuuid/
ls -l /dev/disk/by-partlabel/ 2>/dev/null
```

链接目录可能不存在或没有匹配条目，尤其在无标签、非 GPT 或设备属性不足时。不能把某一链接缺失直接解释为分区表损坏。

⑥ [边界] 设备繁忙时不要重复写分区表

若 `partprobe` 报告设备忙或无法更新，下一步应调查挂载、Swap、LVM、RAID、device-mapper 和打开使用者。重复执行 `mklabel`、`mkpart` 或重启分区工具，只会增加数据风险。

⑦ [边界] 重启是运行决策，不是默认修复

在生产环境中，若内核无法安全在线重读而变更又必须生效，可能需要维护窗口和重启。但在考试新盘场景，优先确认设备未使用并完成 `partprobe`/udev 验证。不能把重启作为跳过调查的通用答案。

[Cheatsheet] `parted` 看磁盘表，`partprobe` 请求内核重读，`udevadm settle` 等事件，`lsblk` 和 `/dev/disk/` 验收；三层顺序不能倒置。

[诊断专题] 新分区未出现或旧布局仍可见

最常见症状是：分区工具显示已经创建，但 `lsblk` 没有对应节点；或者删除分区后，旧节点仍短时间存在。诊断必须先确定“哪一层不同步”，而不是立即重新写盘。

① [诊断点] 先证明分区表是否真的写入目标磁盘

```
parted <DISK> unit s print
fdisk -l <DISK>
```

若两者都没有新条目，问题可能是命令未执行、选错设备、交互修改未 `w` 提交，或命令报错。此时不应先运行 `partprobe`，因为磁盘上没有可同步的新布局。

② [诊断点] 比较磁盘表与内核树

```
lsblk -o NAME,PATH,START,SIZE,TYPE,PKNAME <DISK>
```

磁盘表有新条目而 `lsblk` 没有，是典型的内核视图滞后。下一条最有区分度的证据是 `partprobe` 的返回和内核日志，而不是再次创建同一分区。

③ [诊断点] 执行最小同步

```
partprobe <DISK>
udevadm settle
```

随后重新运行 `lsblk`。如果节点出现，继续验证 `PARTUUID` 和持久链接；如果仍未出现，保留错误信息并进入占用调查。

④ [诊断点] 调查设备是否正在使用

```
findmnt
swapon --show
pvs
cat /proc/mdstat
ls -l /sys/class/block/<KERNEL_NAME>/holders/
```

设备使用状态可能使内核无法在线接受新的布局。此时最小修复不是强制删除，而是让上层对象安全释放，或将变更安排到维护窗口。

⑤ [诊断点] 查看内核设备事件证据

```
journalctl -k --since "-10 min"
udevadm monitor --kernel --udev --property
```

`udevadm monitor` 适合在再次触发设备事件时观察内核与 `udev` 流程，但不应为了产生事件而重复破坏性写入。

⑥ [再验证] 四层证据必须收敛

```
parted/fdisk: 磁盘表正确
lsblk: 内核节点正确
udevadm / /dev/disk: 属性和链接正确
blkid/wipefs: 没有非预期内容变化
```

[Cheatsheet] 有表无节点：先 `partprobe`；仍失败：查使用者和内核日志；绝不通过重复 `mklabel` 解决同步问题。

[诊断专题] 未知、冲突和遗留设备签名

另一个高频症状是：安装程序、LVM 或文件系统工具提示设备已有内容，普通 `blkid` 却没有清晰输出；或者同一分区同时显示多个历史类型。诊断目标不是尽快“擦干净”，而是确认哪些签名仍代表有效对象。

① [诊断点] 先使用低层探测和签名列表

```
blkid -p -o export <DEVICE>
wipefs <DEVICE>
```

记录识别类型、UUID、标签和偏移。不同工具显示的信息可能互补，不应只取一条输出下结论。

② [诊断点] 判断签名所属层

- GPT/MBR 签名通常属于整盘分区表；
- 文件系统、Swap、LVM PV 或 RAID 成员签名常位于整盘或分区；
- device-mapper 或 LVM 逻辑卷是内核映射对象，不能仅靠底层签名解释其当前状态。

③ [诊断点] 关联上层当前状态

```
findmnt --source <DEVICE>
swapon --show
pvs -o pv_name,vg_name,pv_uuid
cat /proc/mdstat
lsblk -o NAME,PATH,TYPE,FSTYPE,MOUNTPOINTS
```

若签名对应当前活动对象，停止清除。若题目声明设备为可丢弃练习盘，也必须确保没有误选父盘或相邻分区。

④ [诊断点] 普通 `blkid` 无输出的解释边界

可能原因包括：

- 没有已知签名；
- 权限或缓存行为不同；
- 多个冲突签名造成歧义；
- 签名损坏或版本不识别；
- 探测对象选择错误。

因此下一步应是 `blkid -p` 与 `wipefs`，不是直接格式化。

⑤ [最小修复] 只有授权后才设计清除

先模拟：

```
wipefs -n -a <DEVICE>
```

如果确认仅某个历史签名需要移除，可依据 `wipefs` 显示的偏移进行定向处理，并在执行前保存输出。实际清除命令必须由任务明确授权，不应作为本章默认答案。

⑥ [再验证] 清除后重新建立完整证据

```
wipefs <DEVICE>
blkid -p <DEVICE>
parted <DISK> print
lsblk -o NAME,PATH,TYPE,FSTYPE,PARTUUID,MOUNTPOINTS
```

清除签名后，必须确认没有误删当前分区表或有效内容入口。

[Cheatsheet] 多签名先归属，后处理；`blkid` 无输出不等于空；`wipefs -a` 不是诊断动作，任何清除都要有授权、基线和再验证。

[诊断专题] 身份、类型和证据冲突时怎样停止

破坏性操作真正需要训练的是“何时不执行”。以下冲突都应触发停止条件。

① [边界] 同容量不等于同一设备

两块 20 GiB 虚拟磁盘可能只有控制器槽位、序列或现有布局不同。若题目只说“新增磁盘”，先用虚拟机或硬件清单确认新增对象，再将其与 `lsblk` 属性匹配。

② [边界] 分区类型和内容签名不一致并不自动表示故障

`PARTTYPE` 表示分区表中的用途类型，`FSTYPE` 表示探测到的内容签名。以下状态都可能出现：

```
LVM 分区类型 + 无 LVM 签名：只完成了分区类型设置
Linux filesystem 类型 + XFS 签名：常见有效组合
Swap 类型 + 旧 XFS 签名：可能是复用不完整，需要调查
```

不能通过修改类型把 XFS “转换”为 Swap，也不能通过清除 `FSTYPE` 解决所有不一致。

③ [边界] 工具输出冲突时先确认读取层

- `parted` / `fdisk` 主要看磁盘分区表；
- `lsblk` 主要看内核设备树并融合 `udev`/签名属性；
- `blkid` / `wipefs` 看内容签名；
- `/dev/disk` 看 `udev` 生成的链接。

先解释每个工具在读什么，再判断是否真的矛盾。

④ [边界] 命令成功不等于目标终态正确

`parted` 返回成功只能说明命令被接受；仍需证明：

- 写到了正确设备；
- 分区边界和类型符合要求；

- 内核节点已更新；
- 没有破坏非目标签名；
- 没有越过文件系统、挂载或 LVM 的章节边界。

⑤ [工作迁移] 变更单应记录身份和回退限制

存储布局变化通常不能像文本配置一样简单回滚。真实工作中至少记录：

变更前设备身份与布局
变更目标和授权范围
预期分区表、边界和类型
内核重读是否要求维护窗口
验证命令和结果
失败时的停止条件

如果没有可靠备份或恢复方案，不应把“重新创建原分区”描述成保证可用的回滚。

[Cheatsheet] 证据冲突时停止；类型不是内容；命令成功只证明局部；存储变更的回退能力必须事先评估。

[经典任务] 在确认空闲的练习盘上创建 GPT 分区

服务器新增一块题目指定为 `/dev/vdb` 的 8 GiB 练习盘。该设备过去可能被其他实验使用，因此不能假设它为空。要求在确认没有需要保留的数据和当前使用者后完成以下终态：

1. 使用 GPT 分区表；
2. 创建名为 `data` 的第 1 分区，从 1 MiB 开始，容量约 2 GiB；
3. 创建名为 `future-lvm` 的第 2 分区，容量约 1 GiB，并设置为 LVM 用途类型；
4. 让内核和 `udev` 识别两个分区；
5. 验证两个分区的 `PARTUUID`；
6. 不创建文件系统，不执行 `pvccreate`，不挂载；
7. 不使用无调查的 `wipefs -a`、`--force` 或重启代替验证。

验收矩阵

层	必须提供的证据
身份	<code>/dev/vdb</code> 的路径、容量、父子关系和可用身份字段
现状	写入前分区表、签名和占用调查
磁盘表	GPT、两个分区的起止、名称和类型
内核	<code>lsblk</code> 中出现 <code>/dev/vdb1</code> 与 <code>/dev/vdb2</code>
udev	<code>PARTUUID</code> 和可用持久链接
内容边界	没有非预期文件系统或 LVM 签名

请先独立完成。参考解答从下一页开始。

[参考解答] 从设备身份到四层验证

① [调查] 建立写入前基线

```
lsblk -o NAME,PATH,MAJ:MIN,SIZE,TYPE,PKNAME,FSTYPE,PARTLABEL,PARTUUID,PTTYPE,PTUUID,MOUNTPOINTS,MODEL,SERIAL,wwn
parted /dev/vdb unit MiB print
fdisk -l /dev/vdb
blkid -p /dev/vdb
wipefs /dev/vdb
findmnt
swapon --show
pvs
cat /proc/mdstat
```

根据真实输出确认：

- `/dev/vdb` 与题目指定的新盘一致；

- 不含需要保留的分区或签名；
- 没有挂载、Swap、LVM 或 RAID 归属；
- 若存在历史签名，题目确实授权丢弃，且已记录具体类型和偏移。

若任何证据不明确，应停止，不执行后续命令。

② [操作] 创建 GPT 和两个分区

```
parted -s /dev/vdb mklabel gpt
parted -s /dev/vdb mkpart data 1MiB 2049MiB
parted -s /dev/vdb mkpart future-lvm 2049MiB 3073MiB
parted -s /dev/vdb set 2 lvm on
```

`mklabel` 只在调查确认后执行。`mkpart` 建立分区条目；`set 2 lvm on` 设置用途类型，不创建 PV。

③ [同步] 请求内核重读并等待 udev

```
partprobe /dev/vdb
udevadm settle
```

若 `partprobe` 报告设备忙或无法重读，不重复写盘；返回占用调查，保存错误信息，并说明需要释放使用者或安排维护窗口。

④ [验证] 分层检查目标终态

```
parted /dev/vdb unit MiB print
fdisk -l /dev/vdb
lsblk -o NAME,PATH,START,SIZE,TYPE,PKNAME,FSTYPE,PARTLABEL,PARTUUID,PARTTYPE,PTTYPE,PTUUID /dev/vdb
udevadm info --query=property --name=/dev/vdb1
udevadm info --query=property --name=/dev/vdb2
blkid -p /dev/vdb1
blkid -p /dev/vdb2
wipefs /dev/vdb1
wipefs /dev/vdb2
```

验收时允许工具因扇区对齐显示略有差异，但应确认：

- 分区表为 GPT；
- 第 1 分区名为 `data`，大小约 2 GiB；
- 第 2 分区名为 `future-lvm`，大小约 1 GiB，类型为 LVM 用途；
- 两个内核节点都存在并具有 `PARTUUID`；
- 没有执行 `mkfs` 或 `pvccreate`，因此内容签名应符合这一边界。

⑤ [典型错误]

- 仅凭 `/dev/vdb` 名称直接执行 `mklabel`；
- 把 `2048MiB` 当作从 1 MiB 开始的绝对终点，得到约 2047 MiB；
- 认为 `set 2 lvm on` 已经创建 PV；
- `parted print` 有分区而 `lsblk` 无节点时重复创建；
- 为清理历史状态默认执行 `wipefs -a`；
- 编造示例 `PARTUUID` 作为验收结果。

[Cheatsheet] 基线调查 → `mklabel gpt` → 两次 `mkpart` → 设置第 2 分区用途 → `partprobe` → `udevadm settle` → 分区表、内核、udev、签名四层验收。

[经典任务] 诊断新分区未出现和遗留签名

管理员报告：已经在 `/dev/vdc` 上创建新分区，`parted /dev/vdc print` 能看到条目，但 `lsblk` 没有显示新的分区节点。该磁盘以前用于实验，普通 `blkid /dev/vdc1` 没有明确输出。当前不允许重新创建分区表，不允许格式化，也不允许清除未经确认的签名。

要求：

1. 证明磁盘上的分区表是否正确；
2. 判断内核是否仍使用旧布局；
3. 请求安全重读并等待 udev；
4. 若设备正在使用，停止并报告具体阻断层；
5. 调查可能的旧文件系统、LVM 或 RAID 签名；

6. 给出最小修复和分层再验证；
7. 不把重启或 `wipefs -a` 作为默认答案。

[参考解答] 用最有区分度的证据推进

① [当前证据] 比较磁盘表与内核视图

```
parted /dev/vdc unit s print
fdisk -l /dev/vdc
lsblk -o NAME,PATH,START,SIZE,TYPE,PKNAME,FSTYPE,MOUNTPOINTS /dev/vdc
```

- 若 `parted` 与 `fdisk` 都没有新条目，先检查交互式 `fdisk` 是否忘记 `w`、命令是否写错设备或执行失败；
- 若磁盘表有条目而 `lsblk` 无节点，假设转为“内核尚未采用新布局”。

② [最小修复] 重读并等待事件

```
partprobe /dev/vdc
udevadm settle
lsblk -o NAME,PATH,START,SIZE,TYPE,PKNAME,PARTUUID /dev/vdc
```

若节点出现，继续签名调查。若仍失败，保存 `partprobe` 的错误并调查占用。

③ [占用调查] 找到阻断层

```
findmnt
swapon --show
pvs
cat /proc/mdstat
ls -l /sys/class/block/vdc/holders/
journalctl -k --since "-10 min"
```

发现活动使用者时，不强制重读。报告是挂载、Swap、LVM、RAID、映射层还是其他内核使用者阻止更新，并提出安全释放或维护窗口方案。

④ [签名调查] 解释普通 `blkid` 无输出

```
blkid -p -o export /dev/vdc1
wipefs /dev/vdc1
```

根据实际输出判断是否存在单一签名、冲突签名或无法识别的内容。再用对应上层查询判断它是否仍有效。未获得授权时，到此为止，不执行清除。

⑤ [再验证] 证明各层收敛

```
parted /dev/vdc unit s print
lsblk -o NAME,PATH,START,SIZE,TYPE,PKNAME,FSTYPE,PARTUUID /dev/vdc
udevadm info --query=property --name=/dev/vdc1
blkid -p /dev/vdc1
wipefs /dev/vdc1
```

最终报告应区分：磁盘表正确、内核节点已更新、udev 属性已生成、签名仍保留或已按授权最小处理。任何一层未完成，都不能扩大为“问题已解决”。

[Cheatsheet] 表有节点无 → `partprobe`；等待事件 → `udevadm settle`；仍失败 → 查使用者和内核日志；签名不明 → `blkid -p` + `wipefs`；未授权不清除。

[本章收束] 每次写入只改变一个存储层

块设备名称、分区表、内核设备树、udev 链接和内容签名各自回答不同问题。稳定的工作方法不是“先清空再重做”，而是始终沿同一条证据链推进：

```
确认设备身份
→ 记录现有布局、签名和占用
→ 只修改分区表或指定条目
→ 请求内核重读并等待 udev
→ 用磁盘表、内核、持久标识和签名分层验收
```

① [操作决策] 写操作前先通过五道证据门

- 目标设备的路径、容量、父子关系和可用身份字段相互一致；

- 现有分区表、分区边界和签名已经记录；
- 挂载、Swap、LVM、RAID、device-mapper 与 holders 已经调查；
- 题目或变更单明确允许改变该设备；
- 失败时的停止条件、维护窗口和恢复限制已经清楚。

任何一道证据门未通过，都不进入 `mklabel`、删除分区或签名清除。

② [检查清单] 完成分区操作后逐层收束

- `parted` 或 `fdisk` 看到预期分区表和边界；
- `lsblk` 看到正确父子关系和内核分区节点；
- `PARTUUID`、`PARTLABEL` 或所需持久链接已经生成；
- `blkid -p` 与 `wipefs` 没有暴露非预期内容；
- 没有越过题目要求创建文件系统、启用 Swap、创建 PV 或挂载；
- 所有结论都说明“能证明什么”和“不能证明什么”。

③ [主要判断表] 证据入口与证明边界

证据入口	能证明什么	不能单独证明什么
<code>lsblk</code>	当前内核块设备树、父子关系、常见属性和部分签名摘要	设备绝对空闲、所有历史签名都不存在
<code>parted ... print</code>	磁盘上的分区表类型、条目、起止和名称	内核已经采用最新布局
<code>fdisk -l</code>	扇区、I/O 大小、分区边界和类型的独立视角	<code>udev</code> 节点已经生成
<code>blkid -p</code>	指定设备当前可探测的低层内容签名	该内容是否仍被上层活动对象使用
<code>wipefs</code>	可识别签名及其偏移	所有数据都已清除，或所有签名都可以删除
<code>partprobe</code>	已向内核请求重读分区表	重读一定成功、 <code>udev</code> 一定已经处理完成
<code>udevadm settle</code>	已等待当前 <code>udev</code> 事件队列收敛	磁盘分区表正确，或设备未被占用

④ [安全边界] 三个最常见的错误扩大

不要把 `FSTYPE` 为空扩大为“设备是空盘”；不要把分区类型扩大为“文件系统、Swap 或 LVM 已经创建”；不要把命令返回成功扩大为“磁盘表、内核、`udev` 与内容层已经全部正确”。存储章节的可靠结论必须来自多层证据收敛。

⑤ [向下一章交接] 从分区设备进入文件系统对象

本章停在“块设备和分区条目已经正确存在，并且身份与签名状态清楚”。第 23 章《文件系统、标签、UUID 与容量管理》将在这个可靠块设备之上创建和识别文件系统，讨论文件系统标签、UUID、容量证据和增长边界；挂载与 Swap 持久性留给第 24 章，LVM 对象生命周期留给第 25 章。

本章最终安全边界只有一句：设备身份和已有内容没有被证据证明之前，不执行破坏性写入。

文件系统、标签、UUID 与容量管理

从块设备签名到文件系统几何：把创建、身份、容量、增长与检查放进同一条证据链。

对象模型操作语义验证诊断经典任务

大字号阅读版

本章阅读导航

先抓住一条主线：块设备提供可寻址空间，文件系统管理其中的结构和身份；设备已经变大，不代表文件系统已经增长，已有签名也绝不代表可以直接格式化。

- 01识别真实对象确认设备路径、父子层级与当前用途
- 02读取签名与身份取得 TYPE、LABEL、UUID 和挂载证据
- 03建立类型模型区分 XFS 与 ext4 的工具和能力边界
- 04比较容量层次分别观察设备、文件系统、数据块和 inode
- 05选择最小操作创建、增长、缩小或检查只改变目标层
- 06完成分层验收分别验证容量、身份、结构和已有数据

专题地图

知识专题	从块空间到文件系统实例
知识专题	XFS 与 ext4 的工具和能力矩阵
操作专题	建立文件系统证据基线
操作专题	在确认空设备上创建文件系统
知识专题	LABEL、UUID 与身份生命周期
知识专题	块设备、文件系统、数据块与 inode 容量
操作专题	用 df 与 du 解释空间差异
操作专题	扩大 XFS 或 ext4
诊断专题	检查、修复与典型容量故障
经典任务	创建带标签 XFS；保留数据在线增长

阅读时持续回答

1. 当前命令作用于块设备、文件系统还是目录树？
2. 目标上是否已有需要保留的签名和数据？
3. TYPE、LABEL、UUID 分别证明什么？
4. 底层容量和文件系统容量是否已经同步？
5. XFS 与 ext4 的增长、缩小和检查工具怎样不同？
6. 操作后怎样独立证明身份、容量和数据都正确？

章节边界：分区表与设备签名归第 22 章；挂载、fstab 与 Swap 归第 24 章；LVM 容量池和 LV 生命周期归第 25 章。本章只在理解文件系统动作所必需时轻量引用它们。

先判断命令改变哪一层，再讨论怎样执行

磁盘、分区或逻辑卷只提供一段可以寻址的块空间。真正让系统能够创建目录、记录文件名、分配数据块并追踪空闲空间的，是建立在块设备之上的文件系统。存储操作中最常见的误判，是把底层设备、文件系统实例、挂载关系和目录内容当成同一个对象：看到设备变大就以为文件系统已经扩容，看到 `blkid` 有签名就以为文件系统健康，或者看到容量异常就重新执行 `mkfs`。

本章沿着“对象身份 → 类型与元数据 → 容量层次 → 类型专用操作 → 分层验证”推进。每次创建、增长、缩小或检查都必须回答：目标是谁，前置条件是否满足，操作会改变什么，哪些已有状态必须保留，以及哪一条证据才能证明最终结果。

目标块设备是谁？

上面是否已有签名？

文件系统是什么类型？

底层和文件系统各有多大？

操作后身份、容量和数据分别怎样验收？

概念

文件系统对象 是建立在一个块设备上的独立数据组织实例，它拥有自己的类型、元数据、分配结构、标签和 UUID。块设备只是容器，挂载只是把这个实例接入目录树；因此 `mkfs` 创建的是新文件系统，而不是创建分区、扩大容量或修复挂载配置。

概念

标签（LABEL） 是便于人阅读的文件系统属性，可表达用途，但不保证唯一。标签属于文件系统实例而不是挂载点；多个文件系统可以同名，所以写操作和持久引用不能只凭一个看起来正确的 LABEL 就忽略设备容量、父子层级和 UUID。

概念

UUID 用来区分当前文件系统实例，通常在创建时生成。它比 `/dev/vdX` 这类枚举路径更适合被外部配置引用，但并非物理磁盘永恒身份：重新格式化会产生新 UUID，块级克隆又可能复制 UUID，因此必须从当前目标读取真实值。

概念

块设备容量 是分区、LV 或其他块设备当前向上提供的地址范围，是文件系统能够增长到哪里的外部上限。`lsblk` 看到设备已经变大，只证明容器层发生了变化；文件系统是否接管新增空间，还要继续查看类型专用几何和 `df`。

概念

文件系统容量 是文件系统元数据当前能够管理的块和 inode 范围。`df` 观察已挂载文件系统的总体分配状态，`du` 遍历目录树并汇总可见文件占用；二者差异不是谁“不准”，而是观察对象不同。

概念

XFS 与 ext4 的调整边界 决定了工具选择和风险。XFS 在 RHEL 9 的稳定路径中支持在线增长但不能原地缩小；ext4 可以增长，并可在卸载、检查和正确顺序下缩小。增长不是重新格式化，缩小也绝不是增长命令的反向调用。

操作语义先建立关键命令的接口地图

这里集中列出本章最重要的入口。正式专题会继续解释操作前提、输出、验证和安全边界。

lsblk -f / blkid

SYNOPSIS

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,LABEL,UUID,MOUNTPOINTS
blkid [-s TAG ...] DEVICE
```

先从全局层级识别候选设备，再对目标设备精确读取内容签名。二者能证明类型和身份线索，不能单独证明文件系统健康或设备可以安全格式化。

lsblk -f

快速查看 FSTYPE、LABEL、UUID 与挂载点。

-o ...

显式选择字段，避免把设备容量与文件系统身份混为一列。

blkid -s TYPE -s LABEL -s UUID DEVICE

只读取本章最关心的签名属性。

mkfs.xfs / mkfs.ext4

SYNOPSIS

```
mkfs.xfs [-L LABEL] DEVICE
mkfs.ext4 [-L LABEL] DEVICE
```

在命令最后指定的块设备上创建一个新的文件系统实例。该动作会写入新的元数据，必须建立在“无需保留原有内容”的明确判断上。

-L LABEL

在创建时设置文件系统标签。

DEVICE

真正被写入的块设备；路径必须在执行前重新核对。

已有签名

不是自动使用强制参数的理由，应先查清数据和归属。

xfs_info / tune2fs -l

SYNOPSIS

```
xfs_info MOUNT_POINT_OR_DEVICE
tune2fs -l DEVICE
```

读取文件系统类型专用的几何和元数据。它们回答文件系统当前怎样组织空间，不替代底层设备容量、挂载关系或业务数据检查。

xfs_info

查看 XFS 的数据区、块大小、allocation group 和格式特性。

tune2fs -l

只列出 ext 文件系统 superblock 信息，不执行修复。

xfs_growfs

SYNOPSIS

```
xfs_growfs [OPTIONS] MOUNT_POINT
```

让一个已经挂载的 XFS 接管底层设备新增的空间。默认不指定目标块数时，增长到当前底层可提供的最大范围。

MOUNT_POINT

作用对象的访问入口，不是任意父目录，也不是重新格式化用的设备路径。

-D SIZE

按文件系统块数设置目标数据区大小；普通扩容通常无需手工指定。

前提

底层设备已经变大，并且目标确实是已挂载 XFS。

resize2fs

SYNOPSIS

```
resize2fs DEVICE [NEW_SIZE]
```

调整 ext2/ext3/ext4 文件系统大小。省略目标大小时通常增长到容器可用边界；缩小与增长具有完全不同的前提和风险。

省略 **NEW_SIZE**

在底层已扩大后增长到可用最大范围。

NEW_SIZE

显式目标大小，必须确认单位和底层边界。

缩小

必须卸载并先执行 **e2fsck -f**，且文件系统必须先于底层容器缩小。

fsck / xfs_repair / e2fsck

SYNOPSIS

```
fsck -N DEVICE
xfs_repair -n DEVICE
e2fsck -n DEVICE
e2fsck -f DEVICE
```

检查和修复必须按文件系统类型选择工具，并以未挂载状态作为实际修复的基本边界。`fsck.xfs` 的成功退出不代表 XFS 已经完成检查。

fsck -N

只显示将调用的类型助手，不修改文件系统。

-n

只读收集结构问题证据。

e2fsck -f

强制完整检查；ext4 缩小前必须执行。

xfs_repair -L

可能清除日志并造成数据丢失，不是普通答案。

df / du

SYNOPSIS

```
df [-hT|-i|-B1] PATH
du [-shx|--apparent-size] PATH
```

`df` 读取包含目标路径的已挂载文件系统整体状态；`du` 遍历目录树并汇总它能看到的文件。两者只能回答各自层次的问题。

df -hT

用可读单位显示类型、总量、已用和可用。

df -i

查看 inode 容量，解释“还有字节却不能新建文件”。

du -shx

汇总目录并限制在同一文件系统。

--apparent-size

观察逻辑长度，而不是实际分配块。

[知识专题] 从块空间到文件系统实例：先分清命令作用层

面对一个存储目标时，最顺的切入点不是立即运行 `mkfs`，而是先画出对象链：底层块设备是谁、上面是否已有签名、文件系统是什么类型、当前管理多大空间、是否已经挂载。只要其中一层没有确认，写操作就可能落在错误对象上。

① [知识点] 块设备只提供地址范围，文件系统负责组织数据

块设备向上提供固定大小的数据块和读写接口。文件系统在这些块之上建立自己的元数据结构，用来回答：

- 哪些块空闲，哪些块已分配；
- 哪些目录项指向哪些 inode；
- 文件大小、权限、所有者和时间戳是什么；
- 崩溃后怎样通过日志或检查工具恢复一致状态；
- 标签、UUID 和类型等身份信息是什么。

因此，`lsblk` 显示设备存在，只能证明内核看到了块设备；它不能自动证明设备已经有文件系统，更不能证明文件系统健康。

② [知识点] `mkfs` 的作用对象是命令最后指定的块设备

`mkfs.xfs /dev/vdb1` 与 `mkfs.ext4 /dev/vdb1` 都直接对 `/dev/vdb1` 写入新的文件系统结构。目标可以是分区、整盘、LV 或其他块设备，命令并不会替你判断这个对象是否安全，也不会创建分区。

`mkfs` 不是：

- 分区工具；
- 文件系统检查工具；
- 扩容工具；
- 修复 UUID 引用错误的工具；
- 清理未知状态时的默认动作。

只在已经确认无需保留数据的新对象上运行 `mkfs`。如果 `blkid`、`lsblk -f`、挂载关系或上层归属显示目标正在使用，应停止写入并回到设备调查。

③ [知识点] 文件系统、挂载关系和目录内容是三个不同对象

文件系统可以存在但未挂载；目录可以存在但没有文件系统挂载在其上；同一个文件系统挂载后，目录树才成为访问入口。完整链路是：

```
块设备
→ 文件系统实例
→ 当前挂载关系
→ 目录树中的文件与目录
```

本章主要处理前两层，并在容量和 XFS 增长时轻量引用“已挂载”前提。挂载、`/etc/fstab` 和启动持久性归第 24 章《挂载、fstab、Swap 与启动持久性》。

④ [知识点] 同一个路径可以同时涉及多层容量

假设 `/srv/archive` 是一个挂载点：

- `lsblk` 看到提供存储的块设备大小；
- `xfs_info` 或 `tune2fs -l` 看到文件系统内部几何；
- `df -hT /srv/archive` 看到文件系统总量、已用和可用；

- `du -sh /srv/archive` 汇总目录树中可见文件占用。

当这些数字不一致时，不应先问“哪个命令错了”，而要先问“每条命令观察的是哪一层”。

⑤ [边界] 设备枚举名不是长期身份

`/dev/vdb1`、`/dev/sdb1` 等路径是当前内核枚举结果。它们适合描述当前操作对象，但不能凭练习环境记忆直接套用到另一台机器。写操作前至少重新核对：

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,LABEL,UUID,MOUNTPOINTS
blkid /dev/vdb1
```

发现设备名称、容量、父设备或已有签名与预期不一致时，停止操作。

[Cheatsheet] 块设备提供空间；文件系统组织空间；挂载提供目录入口；`mkfs` 创建新文件系统而不是扩容；路径相似不等于对象相同。

[知识专题] XFS 与 ext4：工具、增长、缩小和检查边界

XFS 与 ext4 都是 RHEL 9 常见的本地文件系统，但不能只把它们理解成 `mkfs` 后缀不同。类型决定了专用查询工具、增长入口、缩小能力、检查流程以及故障恢复方式。考试题要求哪一种类型，就按该类型建立和验证；已有系统则必须先查明类型再选择工具。

① [知识点] RHEL 9 默认本地文件系统语境偏向 XFS

RHEL 9 默认安装常使用 XFS。XFS 面向大容量和并发 I/O，支持在线增长；ext4 具有成熟的通用能力，既能增长，也能在严格离线条件下缩小。默认选择并不意味着可以忽略题目指定类型，也不意味着现有文件系统必然是 XFS。

判断类型应读取当前证据：

```
lsblk -f
blkid <DEVICE>
df -T <PATH>
```

② [知识点] 两类文件系统有不同的工具矩阵

目的	XFS	ext4
创建	<code>mkfs.xfs</code>	<code>mkfs.ext4</code> / <code>mke2fs -t ext4</code>
查询基本身份	<code>blkid</code> 、 <code>lsblk -f</code>	<code>blkid</code> 、 <code>lsblk -f</code>
查询专用信息	<code>xfs_info</code>	<code>tune2fs -l</code> 、 <code>dumpe2fs -h</code>
修改 LABEL/UUID	<code>xfs_admin</code>	<code>tune2fs</code>
增长	<code>xfs_growfs</code>	<code>resize2fs</code>
缩小	RHEL 9 支持边界内不提供原地缩小	<code>resize2fs</code> ，必须离线
检查/修复	<code>xfs_repair</code>	<code>e2fsck</code>

`fsck` 是统一入口或分派器，不代表所有文件系统都采用同一检查算法。

③ [知识点] XFS 的稳定操作模型是在线增长、不可原地缩小

XFS 增长需要：

1. 底层块设备已经提供新增空间；
2. XFS 已挂载；
3. 对挂载点执行 `xfs_growfs`；
4. 使用 `df` 和 `xfs_info` 验证。

RHEL 9 的受支持管理路径中，XFS 不提供原地缩小。需要更小的 XFS 时，通常采用备份或迁移数据、重建较小文件系统、再恢复数据的方案。不要把上游工具可能出现的实验性选项当作 RHCSA 稳定答案。

④ [知识点] ext4 可以在线增长，但缩小必须离线

`resize2fs` 可以扩大未挂载 ext4，也可以在内核和文件系统支持时在线扩大已挂载 ext4。缩小 ext4 时必须先卸载并完成强制一致性检查：

```
卸载
→ e2fsck -f
→ resize2fs <DEVICE> <TARGET_SIZE>
→ 再缩小底层容器
```

如果还需要缩小分区或 LV，必须先让文件系统变得更小，再处理底层对象。反过来会截断文件系统仍可能使用的块。

⑤ [知识点] XFS 与 ext4 的崩溃恢复路径不同

XFS 在挂载时回放日志。fsck.xfs 是兼容占位程序，会成功退出但不执行传统意义的检查。真正的 XFS 元数据检查使用 xfs_repair -n，并应在文件系统未挂载时进行。

ext4 使用日志恢复和 e2fsck。执行实际检查或修复时同样以未挂载状态为基本边界。任何修复工具都只能尽力恢复文件系统结构一致性，不能保证业务文件内容完整。

[Cheatsheet] XFS: xfs_info、xfs_growfs、xfs_repair，只增长；ext4: tune2fs -l、resize2fs、e2fsck，可离线缩小。

[操作专题] 建立可信的文件系统证据基线

对任何创建、扩容、修改标签或检查操作，先保存变更前基线。最有效的调查不是堆积所有命令，而是让每条证据回答一个明确问题：对象是谁、是否已有签名、类型是什么、身份是什么、当前文件系统多大、是否已挂载。

① [操作] 使用 lsblk -f 建立全局拓扑与文件系统视图

作用对象：内核识别的块设备及其层级。

基本形式：

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,LABEL,UUID,MOUNTPOINTS
lsblk -f
```

重点字段：

- PATH：当前完整设备路径；
- SIZE：块设备层容量；
- TYPE：disk、part、lvm 等块设备类型；
- FSTYPE：识别到的内容类型；
- LABEL、UUID：文件系统身份；
- MOUNTPOINTS：当前挂载入口。

边界：FSTYPE 为空不能单独证明设备安全空闲。设备仍可能含有未识别数据、分区表、加密或其他上层元数据。

② [操作] 使用 blkid 精确读取目标设备签名

作用对象：指定块设备上的可识别内容签名。

```
blkid /dev/vdb1
blkid -s TYPE -s LABEL -s UUID /dev/vdb1
```

典型结果字段包括 TYPE、LABEL、UUID。这些值必须从当前目标读取，不能复制讲义或练习中的示例。

验证边界：blkid 识别到 XFS 或 ext4，只证明签名存在；它不证明文件系统已挂载、容量充足或元数据健康。

③ [操作] 使用 xfs_info 查询 XFS 几何

作用对象：XFS 文件系统实例。

```
xfs_info /srv/archive
# 对未挂载目标也可按本机手册支持形式使用设备路径
xfs_info /dev/vdb1
```

常见字段：

- bsize：文件系统块大小；
- blocks：数据区块数量；
- agcount：allocation group 数量；
- sectsz、sunit、swidth：扇区和条带相关几何；
- reflink、crc 等格式特性。

输出的具体默认值与设备大小、创建参数和 xfsprogs 版本有关，不应背诵某一份示例输出。

④ [操作] 使用 `tune2fs -l` 查询 ext4 superblock

作用对象：ext2/ext3/ext4 文件系统。

```
tune2fs -l /dev/vdb1
```

重点字段通常包括：

- Filesystem volume name；
- Filesystem UUID；
- Filesystem state；
- Block count、Free blocks；
- Block size；
- Inode count、Free inodes；
- Last mounted on、Last mount time。

`tune2fs -l` 是读取信息，不等同于 `e2fsck`。看到 `clean` 也不能替代需要时的离线检查。

⑤ [验证] 用两类入口交叉确认身份

创建或修改属性后，至少使用两个独立视图：

```
blkid /dev/vdb1
lsblk -f /dev/vdb1
```

对 XFS 可继续使用：

```
xfs_info /dev/vdb1
```

对 ext4 可继续使用：

```
tune2fs -l /dev/vdb1 | grep -E 'Filesystem volume name|Filesystem UUID|Block size|Block count'
```

如果两条命令仍显示旧 LABEL 或 UUID，先等待 udev 事件完成，再重新读取；不要立即重复修改。

[Cheatsheet] `lsblk -f` 看全局层级，`blkid` 看目标签名，`xfs_info` 看 XFS 几何，`tune2fs -l` 看 ext4 superblock。

[操作专题] 在确认空设备上创建带标签的文件系统

创建文件系统是一条清晰但高风险的写路径：确认目标、排除已有使用、执行一次创建、立即读取真实身份。危险不在命令复杂，而在目标设备选错或把已有文件系统当成空设备。

① [操作] 创建前重新确认目标对象

假设题目指定候选设备 `/dev/vdb1`，创建前至少执行：

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,LABEL,UUID,MOUNTPOINTS /dev/vdb
blkid /dev/vdb1
findmnt -S /dev/vdb1
```

必要时继续检查它是否属于 LVM、RAID、加密或其他存储层。发现已有文件系统、挂载、业务数据或上层归属时，停止执行 `mkfs`，回到题意和相邻章节处理。

② [操作] 创建带 LABEL 的 XFS

```
mkfs.xfs -L ARCHIVE /dev/vdb1
```

- `-L ARCHIVE`：创建时设置文件系统 LABEL；
- `/dev/vdb1`：实际被写入的块设备；
- 默认格式特性由当前 RHEL 9 的 `xfsprogs` 和设备条件决定。

不要在已有 XFS 上重复运行该命令来“修复”容量或 UUID。创建动作会建立新的元数据结构。

③ [操作] 创建带 LABEL 的 ext4

```
mkfs.ext4 -L ARCHIVE /dev/vdb1
```

`mkfs.ext4` 通常是 `mke2fs` 的 `ext4` 前端。LABEL 有长度限制；`ext4` 标签最多 16 字节。对于包含多字节字符的标签，按字节限制理解，并在创建后读取真实结果。

④ [参数] 不把强制参数当作默认答案

当工具发现已有签名时可能拒绝继续或要求确认。正确反应不是立即添加 `-f` 或其他强制选项，而是重新调查：

```
已有签名是什么
→ 是否需要保留
→ 目标设备是否选错
→ 是否属于其他存储层
→ 题目是否明确要求重建
```

只有证据明确说明旧内容无需保留且题目要求重建时，才评估强制行为。本章经典任务不需要强制参数。

⑤ [验证] 创建后读取类型、标签和 UUID

```
blkid /dev/vdb1
lsblk -f /dev/vdb1
```

验证目标：

- `TYPE` 与题目指定类型一致；
- `LABEL` 为实际设置值；
- `UUID` 已由文件系统生成；
- 未影响相邻设备；
- 没有把挂载或 `fstab` 作为本章创建任务的隐含要求。

[Cheatsheet] 调查目标 → 确认无需保留 → `mkfs.<TYPE> -L <LABEL> <DEVICE>` → `blkid` + `lsblk -f`；强制格式化不是默认修复。

[知识专题] LABEL 与 UUID：文件系统身份怎样生成、修改和失效

文件系统身份位于文件系统元数据中。LABEL 适合人阅读和表达用途，UUID 适合稳定区分实例。两者都不是挂载点名称，也不是底层物理设备永远不变的属性。理解身份生命周期，可以避免为了修复错误引用而误格式化设备。

① [知识点] LABEL 可读，但不能假设唯一

管理员可以给多个文件系统设置相同 LABEL，因此 `LABEL=DATA` 只有在环境中保持唯一时才可靠。标签适合表达用途，例如 `ARCHIVE`、`BACKUP`，但不能因为标签符合预期就忽略设备容量、父设备和 UUID。

查询：

```
lsblk -f
blkid
```

② [知识点] UUID 标识的是当前文件系统实例

文件系统创建时通常生成 UUID。它比 `/dev/vdX` 枚举路径更适合被外部配置引用，但仍有两个边界：

- 重新格式化会创建新的文件系统和新的 UUID；
- 块级克隆可能复制 UUID，导致两个实例拥有相同身份，需要显式处理。

因此，UUID 必须来自当前目标，而不是来自旧截图、命令历史或另一台机器。

③ [操作] 修改 XFS 的 LABEL 或 UUID

XFS 属性修改以未挂载文件系统为稳定边界：

```
xfs_admin -L ARCHIVE2 /dev/vdb1
NEW_UUID=$(uuidgen)
xfs_admin -U "$NEW_UUID" /dev/vdb1
udevadm settle
```

只修改需要改变的属性。不要为了“刷新 UUID”无理由更改生产文件系统身份，因为外部配置可能引用旧值。

④ [操作] 修改 ext4 的 LABEL 或 UUID

```
tune2fs -L ARCHIVE2 /dev/vdb1
NEW_UUID=$(uuidgen)
tune2fs -U "$NEW_UUID" /dev/vdb1
udevadm settle
```

`tune2fs` 是 ext 系列文件系统属性管理工具。修改后用 `blkid` 和 `lsblk -f` 重新读取，不能只相信命令退出状态。

⑤ [诊断] 身份变化后先修正引用，不要再次格式化

症状示例：外部配置仍引用旧 UUID，而 `blkid` 显示新 UUID。调查链是：

```
读取当前 UUID
→ 确认文件系统 and 数据仍然存在
→ 找出引用旧 UUID 的配置
→ 最小修改引用
→ 再验证
```

完整的 `/etc/fstab` 修改和启动验收归第 24 章。本章只建立接口：属性变化会影响所有依赖 LABEL 或 UUID 的外部配置。

[Cheatsheet] LABEL 便于阅读但可能重复；UUID 属于文件系统实例；重建会换 UUID；修改属性后 `udevadm settle`，再用 `blkid` / `lsblk -f` 验证。

[知识专题] 容量的四层模型：块设备、文件系统、数据块和 inode

“空间不足”不是一个单一状态。底层设备可能还有空闲范围，但文件系统尚未增长；数据块可能还有余量，但 inode 已经耗尽；目录树汇总可能很小，但文件系统仍显示大量已用空间。只有先确定容量层，后续命令才有意义。

① [知识点] 块设备容量是文件系统能够增长的外部上限

`lsblk` 的 `SIZE` 描述当前块设备大小。分区或 LV 已扩大后，`lsblk` 可能立即显示新容量，但文件系统仍按旧几何管理原来的块范围。

```
lsblk -b -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS
```

看到底层变大只是“扩容前置条件完成”，不是文件系统扩容终态。

② [知识点] 文件系统几何决定当前管理范围

XFS 的数据区块数与块大小可以从 `xfs_info` 读取；ext4 的块数和块大小可从 `tune2fs -l` 读取。文件系统容量通常包含元数据、日志、保留块或其他内部开销，因此不应简单期待 `df` 的总量与设备字节数完全相等。

③ [知识点] `df` 报告已挂载文件系统的总体空间

```
df -hT /srv/archive
df -B1 /srv/archive
```

常见字段：

- `Filesystem`：当前挂载源；
- `Type`：文件系统类型；
- `Size`、`Used`、`Avail`；
- `Use%`；
- `Mounted on`。

`df` 观察的是文件系统分配器视角，通常只能报告已挂载文件系统。它不告诉你某一个目录具体用了多少。

④ [知识点] inode 是独立于数据块的容量维度

大量小文件可能先耗尽 inode，而数据块仍有余量：

```
df -i /srv/archive
```

如果 `IUse%` 达到 100%，新文件创建会失败，即使 `df -h` 仍显示可用字节。删除不再需要的小文件、调整应用文件布局或迁移到适合的文件系统才是正确方向，不是简单扩大单个文件大小。

⑤ [知识点] 可用空间不等于业务可以写入

`df` 显示 `Avail` 大于零，也不能保证目标用户能写：目录权限、ACL、只读挂载、配额、SELinux、应用自身限制都可能阻止写入。这些问题分别归权限、挂载和安全章节。本章只说明容量证据不能扩大为完整业务可用性结论。

[Cheatsheet] `lsblk` 看容器，专用信息工具看文件系统几何，`df -h` 看数据块容量，`df -i` 看 inode；任一层有余量都不能替代其他层。

[操作专题] 用 `df` 与 `du` 回答不同的空间问题

`df` 和 `du` 经常被放在一起，却不是同一类统计。`df` 读取文件系统整体分配状态；`du` 遍历路径，累加它能够看到的文件所占块数。两者差异本身就是诊断证据，不应被当作“其中一个不准”。

① [操作] 用 `df -hT` 查看文件系统类型和整体容量

```
df -hT /srv/archive
```

将路径作为参数时，`df` 报告包含该路径的文件系统。使用 `-T` 显示类型，`-h` 采用可读单位；需要精确比较扩容前后字节数时可用 `-B1`。

边界：`df /srv/archive/subdir` 与 `df /srv/archive` 通常指向同一个文件系统，不代表两个目录各自拥有独立容量。

② [操作] 用 `du -sh` 汇总目录树实际分配空间

```
du -sh /srv/archive
du -shx /srv/archive/* 2>/dev/null
```

- `-s`：只输出汇总；
- `-h`：可读单位；
- `-x`：限制在同一文件系统，避免跨入子挂载。

`du` 需要遍历权限。无法读取的目录、被排除的挂载或竞态变化都会影响结果。

③ [参数] 逻辑大小与实际分配块不同

```
du -h sparse-file
du -h --apparent-size sparse-file
ls -lh sparse-file
```

稀疏文件的逻辑长度可能很大，但未实际分配同等数据块。默认 `du` 更接近已分配空间，`--apparent-size` 查看逻辑长度。判断存储压力时不要只看 `ls -l` 的文件大小。

④ [诊断] `df` 明显大于 `du` 时的高频原因

按区分度调查：

1. 已删除但仍被进程打开的文件：目录项消失，`du` 看不到，文件系统仍占块；使用 `lssof +L1` 或针对性进程证据检查。
2. 权限或遍历遗漏：当前用户无法读取部分目录；以合适权限重新统计。
3. 文件系统元数据、日志和 ext4 保留块：不会全部映射为普通文件。
4. 被挂载遮蔽的旧目录内容：旧数据仍占下层文件系统，但当前路径显示的是另一个挂载；完整处理归第 24 章。
5. 统计时数据持续变化：日志、数据库或缓存正在写入。

⑤ [诊断] `du` 结果大于预期时先定位目录，不立即删除

```
du -x -h --max-depth=1 /srv/archive | sort -h
du -x -a /srv/archive | sort -n | tail
```

先找出高占用路径，再识别文件所有者、用途、更新时间 and 是否仍被进程使用。不要把“删除最大的文件”当作通用答案；数据库、日志和应用文件可能需要应用级轮转或停写流程。

[Cheatsheet] `df` 看文件系统整体，`du` 看可遍历文件树；`-x` 不跨文件系统；`--apparent-size` 看逻辑长度；`df > du` 先查已删除但仍打开的文件。

[操作专题] 扩大已有文件系统：先扩容器，再让文件系统接管空间

文件系统增长不是创建新文件系统。正确的扩容链路必须保留现有身份和数据：先确认底层块设备已变大，再按文件系统类型执行增长，最后比较变更前后的容量和数据证据。

① [操作] 记录扩容前基线

假设 `/srv/archive` 是目标路径：

```
findmnt -no SOURCE,FSTYPE,TARGET /srv/archive
lsblk -b -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS
df -B1 -T /srv/archive
blkid <SOURCE_DEVICE>
sha256sum /srv/archive/marker.txt
```

本章只要求轻量使用 `findmnt` 确认当前源和类型；挂载配置本身归第 24 章。标记文件用于证明扩容没有通过重建文件系统完成。

② [判断] 底层设备必须先提供新增空间

比较：

```
lsblk 的设备 SIZE
与
df/xfs_info/tune2fs 的文件系统容量
```

若块设备大小没有变化，文件系统工具无法凭空获得空间，应回到第 22 章分区或第 25 章 LVM 处理。若块设备已变大而 `df` 仍是旧值，才进入文件系统增长。

③ [操作] 扩大已挂载 XFS

```
xfs_growfs /srv/archive
```

作用对象：挂载在 `/srv/archive` 的 XFS 文件系统。

默认不指定目标大小时，增长到当前底层设备能够提供的最大可用范围。目标参数是挂载点，不是随意选择的父目录。

增长后：

```
df -B1 -T /srv/archive
xfs_info /srv/archive
```

④ [操作] 扩大 ext4

底层设备已经增大后，`ext4` 可按状态选择：

```
# 已挂载，在线增长到容器可用最大范围
resize2fs /dev/vdb1

# 未挂载，也可执行增长
resize2fs /dev/vdb1
```

不指定大小时，`resize2fs` 通常增长到当前容器可用边界。若指定目标大小，必须明确单位并确认底层能够容纳。

⑤ [验证] 容量、身份和数据必须分别验收

```
lsblk -b -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS
df -B1 -T /srv/archive
blkid <SOURCE_DEVICE>
sha256sum /srv/archive/marker.txt
```

验收含义：

- `lsblk`：底层容量；
- `df`：文件系统已接管新增容量；
- `blkid`：类型、LABEL、UUID 未被无意重建；
- 校验值：已有文件内容未改变。

⑥ [边界] 命令成功不能替代目标容量判断

`xfs_growfs` 或 `resize2fs` 无报错，只能证明工具没有报告失败。还需确认：

- 增加的容量是否符合题意；
- 操作对象是否正确；
- `df` 是否变化；
- 数据是否保留；

- 文件系统是否仍为指定类型。

[Cheatsheet] 基线 → 底层已扩展 → 按类型增长 → `lsblk + df` + 专用信息工具 → UUID/LABEL 和标记文件不变。

[知识专题] 缩小不是增长的反向命令

缩小文件系统需要把仍在使用的数据和元数据移动到新的边界以内，并保证底层容器不会截断有效块。它的风险远高于增长。RHCSA 中最重要的不是记住复杂缩容技巧，而是准确判断哪些文件系统根本不能缩小、`ext4` 缩小时顺序为何不可颠倒。

① [边界] XFS 在 RHEL 9 支持路径中不能原地缩小

不存在与 `xfs_growfs` 对称的稳定 RHCSA 缩小命令。需要更小 XFS 时，采用：

```
备份或迁移数据
→ 创建较小的新 XFS
→ 恢复数据
→ 分层验证
```

直接缩小其分区或 LV 会截断文件系统，造成损坏。

② [边界] `ext4` 缩小必须卸载

在线 `ext4` 可以增长，但不能在线缩小。标准顺序：

```
umount <MOUNT_POINT>
e2fsck -f <DEVICE>
resize2fs <DEVICE> <TARGET_SIZE>
```

卸载是否安全、哪个进程占用挂载点、如何修改底层分区或 LV，分别需要相邻章节的状态证据。

③ [操作语义] 文件系统必须先于底层容器缩小

正确顺序：

```
确认备份和停机窗口
→ 卸载
→ 检查文件系统
→ 缩小文件系统
→ 缩小分区或 LV
→ 重新接入并验证
```

错误顺序 先 `lvreduce/缩分区` → 再 `resize2fs` 会让底层边界先变小，文件系统仍可能引用被截断的块。

④ [判断] 目标大小必须容纳现有数据和元数据

`du` 看到的数据量不是安全最小值。文件系统还需要 inode 表、日志、块组、目录结构和空闲空间。`resize2fs -P <DEVICE>` 可以估算 `ext` 文件系统最小块数，但估算值仍不应被理解为无需备份的保证。

⑤ [安全] 缩小前必须具备可恢复路径

因为缩小涉及离线、数据移动和多层边界，生产变更至少应具备：

- 已验证备份；
- 明确回滚或重建方案；
- 停机窗口；
- 文件系统和底层设备的变更前记录；
- 缩小后重新挂载、容量、数据和应用验证。

本章只覆盖文件系统层必要范围，不把复杂缩容作为普通考试创建题的默认答案。

[Cheatsheet] XFS 不原地缩小；`ext4` 必须离线，先 `e2fsck -f`，先缩文件系统再缩底层；缩小前必须有恢复路径。

[诊断专题] 文件系统检查与修复：一致性、日志回放和数据恢复不是一回事

检查工具处理的是文件系统结构。它们可能重新连接孤立 inode、修复分配记录或丢弃无法恢复的元数据，但不能保证每个业务文件内容正确。诊断时先判断类型、挂载状态和故障性质，再选择只读检查或实际修复。

① [知识点] `fsck` 是前端，实际行为由文件系统类型决定

```
fsck -N /dev/vdb1
```

`-N` 只显示计划执行的命令，可用于理解分派。不要在未知类型或仍挂载的文件系统上直接运行实际 `fsck`。不同类型可能调用不同助手，XFS 更不能按 ext4 流程理解。

② [边界] `fsck.xfs` 成功退出不代表 XFS 已检查

RHEL 9 中 `fsck.xfs` 主要为兼容启动脚本存在，会立即返回成功。以下结论是错误的：

```
fsck.xfs 返回 0  
→ XFS 元数据已扫描且健康
```

XFS 的正常崩溃恢复首先依靠挂载时日志回放；需要离线检查时使用 `xfs_repair -n`。

③ [操作] 对 XFS 执行只读检查

在确认文件系统未挂载后：

```
xfs_repair -n /dev/vdb1
```

`-n` 表示不修改文件系统，用于收集结构问题证据。若文件系统带有需要回放的脏日志，正常路径通常是先在安全条件下挂载并卸载以回放日志，再进行检查。

④ [操作] 实际修复 XFS 的风险边界

```
xfs_repair /dev/vdb1
```

只在未挂载、已有备份或恢复计划、并确认需要修复时执行。`xfs_repair -L` 会清除日志，只在日志无法回放且已接受潜在数据丢失时作为最后手段，不是普通答案。

⑤ [操作] 对 ext4 执行检查

只读检查：

```
e2fsck -n /dev/vdb1
```

强制完整检查并允许交互修复：

```
e2fsck -f /dev/vdb1
```

自动回答“是”的选项可能造成不可逆修改，不应在没有备份和输出审阅时作为默认方式。执行实际检查和修复时以未挂载状态为基本边界。

⑥ [验证] 修复后验证结构、身份和数据

修复命令退出后继续：

```
blkid /dev/vdb1  
lsblk -f /dev/vdb1  
# 按类型读取几何  
xfs_info /dev/vdb1  
# 或  
tune2fs -l /dev/vdb1
```

随后在第 24 章的挂载流程中重新接入，检查日志、关键目录和业务文件。修复工具没有报错不能证明业务数据完整。

[Cheatsheet] 先识别类型和挂载状态；`fsck.xfs` 不做真实检查；XFS 用 `xfs_repair -n`，ext4 用 `e2fsck -n/-f`；实际修复前必须有恢复路径。

[诊断专题] 根据症状定位容量、身份或一致性层

稳定诊断链是：症状 → 当前证据 → 假设 → 下一条最有区分度的证据 → 最小修复 → 再验证。文件系统问题尤其不能以重新格式化作为通用起点，因为这会把可诊断问题变成数据丢失。

① [诊断] `lsblk` 已变大，但 `df` 仍是旧容量

当前证据：底层块设备容量已经增加，文件系统视图没有变化。

假设：文件系统尚未增长，或使用了错误类型的增长工具。

下一条证据：

```
findmnt -no SOURCE,FSTYPE,TARGET <PATH>
lsblk -b -o NAME,PATH,SIZE,FSTYPE,MOUNTPOINTS
df -B1 -T <PATH>
```

最小修复：XFS 用 `xfs_growfs <MOUNT_POINT>`；ext4 用 `resize2fs <DEVICE>`。

再验证：`df`、专用信息工具、UUID 和标记文件。

② [诊断] `blkid` 有签名，但文件系统无法正常接入

`blkid` 只证明存在可识别签名。继续检查：

```
类型是否正确
→ 设备是否为预期对象
→ 是否已经挂载在其他位置
→ 内核或系统日志有什么错误
→ 是否需要离线只读检查
```

不要因为挂载失败而执行 `mkfs`。那会覆盖原文件系统，使后续修复和取证更困难。

③ [诊断] `df` 使用率很高，但 `du` 汇总较小

优先调查：

```
du -shx <MOUNT_POINT>
ls -l
```

若发现已删除但仍打开的大文件，应确认所属进程和应用的安全释放方式。直接重启整机或杀进程不是第一答案：先判断服务管理关系和业务影响。

④ [诊断] 数据块仍有空间，但无法创建新文件

检查 inode：

```
df -h <PATH>
df -i <PATH>
```

若 `IUse%` 为 100%，问题是 inode 耗尽。下一步定位产生大量小文件的目录和应用，而不是只扩大单个文件或把 `df -h` 的可用字节当作反证。

⑤ [诊断] 重新格式化后旧 UUID 引用失效

事实：`mkfs` 创建了新文件系统实例，旧 UUID 不再对应当前签名。

正确路径：

```
blkid <DEVICE>
lsblk -f <DEVICE>
```

确认当前 UUID，再修正外部引用。不要为了让 UUID“匹配旧配置”继续反复格式化。完整持久挂载修复归第 24 章。

[Cheatsheet] 设备大而 `df` 小：文件系统未增长；签名存在但不可用：查类型/日志/检查；`df > du`：查打开的已删除文件；能用块但不能建文件：查 inode；UUID 错：修引用，不重建。

[经典任务] 在确认空设备上创建带标签的 XFS

服务器提供候选块设备 `/dev/vdb1`。该名称只用于本任务，真实环境必须重新识别设备。要求在确认该设备没有需要保留的数据、没有当前挂载且不属于其他存储用途后，在其上创建 XFS，LABEL 设置为 `ARCHIVE`。

完成后必须从当前文件系统读取真实 UUID，并用至少两条独立命令证明：

- 文件系统类型为 XFS；
- LABEL 为 `ARCHIVE`；
- UUID 已实际生成；
- 没有把相邻设备作为操作对象。

限制条件：

- 不使用 `wipefs -a`；
- 不使用无调查的强制格式化；
- 不复制示例 UUID；
- 发现已有文件系统、挂载、LVM/RAID/加密归属或数据时停止写操作；
- 本任务不要求挂载或配置 `/etc/fstab`。

验收证据应包含调查命令、实际创建命令和创建后查询。不能只提交 `mkfs.xfs` 一条命令。

请先独立完成。参考解答从下一页开始。

[参考解答] 从设备调查到文件系统身份验证

① [调查] 重新识别候选对象

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,LABEL,UUID,MOUNTPOINTS /dev/vdb
blkid /dev/vdb1
findmnt -S /dev/vdb1
```

若命令显示目标已有文件系统、挂载点或与题意不符的容量和父设备，停止继续。必要时由相邻章节的方法检查 LVM、RAID 或加密归属。本解答不假设 `blkid` 无输出就一定安全。

② [判断] 明确创建条件

只有在以下条件同时成立时继续：

```
设备路径与容量符合题意
且
没有需要保留的文件系统或数据
且
没有当前挂载
且
没有其他上层存储归属
```

任何一项不满足，都不应通过强制参数掩盖。

③ [操作] 创建带标签的 XFS

```
mkfs.xfs -L ARCHIVE /dev/vdb1
```

该命令直接在 `/dev/vdb1` 上建立新 XFS。`ARCHIVE` 是文件系统 LABEL，不是挂载点或设备名。

④ [验证] 交叉读取真实身份

```
blkid /dev/vdb1
lsblk -f /dev/vdb1
xfs_info /dev/vdb1
```

判断标准：

- `TYPE="xfs"`；
- `LABEL="ARCHIVE"`；
- UUID 为当前命令实际读取到的值；
- `xfs_info` 能识别 XFS 几何；
- `/dev/vdb` 上其他对象未被改变。

⑤ [典型错误]

- 直接对 `/dev/vdb` 而不是题目指定的 `/dev/vdb1` 格式化；
- 因 `blkid` 无输出就省略其他归属调查；
- 复制示例 UUID 写进答案；
- 发现旧签名后直接加 `-f`；
- 为了验证而提前进入挂载和 `fstab`，导致章节层次混乱。

[Cheatsheet] 身份/签名/挂载调查 → 明确无需保留 → `mkfs.xfs -L ARCHIVE` → `blkid` + `lsblk -f` + `xfs_info`。

[经典任务] 扩展已有 XFS，并证明数据与身份未被重建

`/srv/archive` 当前是一个已挂载的 XFS 文件系统。基础设施操作已经扩大其底层块设备，但文件系统仍显示旧容量。目录中存在标记文件 `/srv/archive/marker.txt`，该文件必须保留。

要求：

1. 确认当前挂载源和文件系统类型；
2. 证明底层块设备容量已经大于文件系统当前容量；
3. 使 XFS 接管全部新增空间；
4. 证明扩容后 UUID、LABEL 和标记文件内容没有被无意改变；
5. 给出分层验证证据。

限制条件：

- 不重新执行 `mkfs.xfs`；
- 不修改分区或 LVM；
- 不删除标记文件；
- 不把命令退出状态或“显示 active”之类局部证据扩大为终态正确；
- 不编造设备名、旧容量、新容量或 UUID。

请先独立完成。参考解答从下一页开始。

[参考解答] 用容量层次和数据校验完成 XFS 在线增长

① [调查] 保存扩容前基线

```
findmnt -no SOURCE,FSTYPE,TARGET /srv/archive
SOURCE=$(findmnt -no SOURCE /srv/archive)
lsblk -b -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS
df -B1 -T /srv/archive
blkid "$SOURCE"
xfs_info /srv/archive
sha256sum /srv/archive/marker.txt
```

记录真实 `SOURCE`、类型、设备容量、文件系统容量、LABEL、UUID 和校验值。若 `FSTYPE` 不是 XFS，停止并选择对应文件系统流程。

② [判断] 区分底层容量与文件系统容量

`lsblk` 应显示目标源所在块设备已经扩大，而 `df -B1` 和 `xfs_info` 仍反映旧文件系统范围。若底层容量没有增加，本章没有可执行的文件系统增长动作，应回到分区或 LVM 层。

③ [操作] 对挂载点增长 XFS

```
xfs_growfs /srv/archive
```

不指定 `-D` 目标块数时，XFS 使用当前底层可提供的最大数据区范围。命令作用于挂载在 `/srv/archive` 的 XFS，而不是创建新文件系统。

④ [验证] 比较变更后容量

```
lsblk -b -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS
df -B1 -T /srv/archive
xfs_info /srv/archive
```

`df` 的文件系统总容量应增加，且仍为 XFS。若没有变化，重新核对挂载源、底层设备是否真的扩大以及命令是否作用于正确挂载点。

⑤ [验证] 身份与数据保留

```
SOURCE=$(findmnt -no SOURCE /srv/archive)
blkid "$SOURCE"
sha256sum /srv/archive/marker.txt
```

与基线比较：

- UUID 未因重建而变化；

- LABEL 未被无意修改；
- 标记文件校验值一致；
- 文件系统容量已经增加。

⑥ [典型错误]

- 对底层设备执行 `mkfs.xfs`，把扩容变成重建；
- 把 `xfs_growfs` 的目标写成错误目录或未挂载设备；
- 只看 `lsblk` 变大就宣布完成；
- 只看 `df` 变大，不验证身份和数据；
- 底层尚未扩展时反复执行增长命令。

[Cheatsheet] 保存源/类型/容量/UUID/校验值 → 确认底层大于文件系统 → `xfs_growfs <MOUNT_POINT>` → `df / xfs_info` → UUID、LABEL 和数据校验不变。

[知识专题] 工作迁移：把考试命令变成可审计的存储变更

真实工作中的文件系统通常承载已有业务，而不是空白练习设备。安全变更应把“能执行命令”升级为“能说明对象、风险、证据和回滚”。

① [工作方法] 变更前保存对象快照

至少记录：

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,LABEL,UUID,MOUNTPOINTS
blkid
findmnt
# 针对目标文件系统
df -B1 -T <PATH>
df -i <PATH>
```

还应记录应用停写状态、备份时间和关键文件校验值。

② [工作方法] 一次只推进一个存储层

把变更拆成：

```
底层容量变更
→ 文件系统增长
→ 挂载/持久配置
→ 应用功能验证
```

每层完成后立即验证，不把多个不可逆操作串成一条无法定位失败点的脚本。

③ [工作方法] 自动化前先建立幂等状态模型

后续使用 Ansible 自动化时，必须明确：

- 目标设备怎样稳定识别；
- 文件系统已存在时是否允许重建；
- 类型、LABEL、UUID 和大小的期望状态；
- 增长是否允许、缩小是否禁止；
- 操作后的验证和失败保护。

自动化模块不是绕过调查和破坏性边界的理由。本章只提供手工对象模型，RHCE 存储自动化另章展开。

[Cheatsheet] 变更前基线、一次推进一层、保留恢复路径、用独立证据验收、自动化也不得默认重建已有文件系统。

[本章收束] 把每次文件系统变更还原成对象、条件和证据

文件系统管理最危险的误判，是只看见一个设备路径或一条成功退出的命令，就把它扩大成“存储已经正确”。稳定方法始终从对象层次开始：先确认真正的块设备，再读取文件系统类型和身份，然后比较底层容量与文件系统容量，最后才选择创建、增长、缩小或检查工具。

一条可执行的工作方法

识别真实对象
→ 保存 TYPE、LABEL、UUID 与容量基线
→ 判断已有签名是否需要保留
→ 按 XFS/ext4 选择专用工具
→ 一次只改变一个存储层
→ 分别验证容量、身份与数据

任何写操作前都应能回答：命令最后作用于哪个设备；该设备是否已有需要保留的文件系统；文件系统当前是否挂载；底层容量是否已经满足前提；失败时是否有恢复路径。答不清这些问题，就不进入 `mkfs`、修复或缩小操作。

章末检查清单

- 能把块设备容量、文件系统容量、`df` 使用量和 `du` 汇总量分开展释；
- 能用 `lsblk -f` 与 `blkid` 交叉识别 TYPE、LABEL 和 UUID；
- 知道 `mkfs` 创建新实例，不能用于扩容或修复引用；
- 能按类型选择 `xfs_info`、`tune2fs -l`、`xfs_growfs`、`resize2fs`、`xfs_repair` 或 `e2fsck`；
- 能说明 XFS 不可原地缩小，ext4 缩小必须离线并先检查；
- 能解释每条验证命令证明什么，以及它不能证明什么；
- 面对已有签名、挂载失败或容量不一致时，先取得下一条有区分度的证据，而不是重新格式化。

主要判断表

看到的现象	首先说明什么	下一条证据	不应直接做什么
<code>lsblk</code> 容量已增大， <code>df</code> 仍是旧值	底层与文件系统容量不同步	读取 FSTYPE、挂载点和专用几何	重新运行 <code>mkfs</code>
<code>blkid</code> 能看到 TYPE/UUID	设备上存在可识别签名	查挂载状态、日志与只读检查结果	把签名存在等同于健康
<code>df</code> 高、 <code>du</code> 低	整体分配与可遍历目录树不一致	<code>lsdf +L1</code> 、权限和子挂载证据	盲目删除最大目录
<code>df -h</code> 有空间但不能新建文件	可能不是数据块容量问题	<code>df -i</code>	只扩大单个文件
题目要求缩小 XFS	当前类型不支持原地缩小	规划迁移、重建和恢复	缩小分区或 LV 截断 XFS
已有签名但用途不明确	破坏性前提未满足	查设备归属、挂载与数据责任人	使用 <code>-f</code> 或 <code>wipefs -a</code> 掩盖风险

向下一章交接

本章已经建立了文件系统自身的类型、身份、几何和容量状态。第 24 章《挂载、fstab、Swap 与启动持久性》将在此基础上继续回答：怎样把文件系统接入目录树；怎样用 UUID 或 LABEL 建立持久引用；怎样区分当前挂载、配置状态与重启后的真实终态。本章不提前展开完整的 `mount`、`findmnt`、`/etc/fstab` 和 Swap 配置流程。

挂载、fstab、Swap 与启动持久性

把“当前可用”和“下次启动仍成立”拆成两层状态，再用挂载关系、fstab、systemd unit 与 Swap 证据逐层验收。

对象模型操作语义验证诊断经典任务

[大字号阅读版](#)

本章阅读导航

先抓住一条主线：目录存在、当前挂载、fstab 声明、systemd unit 与启动后状态不是同一件事；Swap 也要把签名、当前启用和持久配置分开证明。

- 01识别源与目标确认文件系统或 Swap 对象是谁
- 02读取当前状态用 findmnt 或 swapon --show 建立事实
- 03修改当前关系mount、umount、swapon、swapoff
- 04写入持久策略fstab 六字段、UUID、选项与优先级
- 05理解启动 unit生成 .mount / .swap 及失败证据
- 06分层验收静态检查、重入测试、功能与重启边界

专题地图

- 知识专题挂载对象、当前关系与遮蔽
- 操作专题findmnt 证据与当前挂载
- 诊断专题busy 引用与安全卸载
- 知识专题fstab 六字段、选项与启动风险
- 知识专题systemd mount unit 与网络边界
- 操作专题Swap 三层状态与优先级
- 诊断专题从症状定位到正确状态层
- 经典任务持久挂载与新增 Swap

阅读时持续回答

- 对象是文件系统、挂载关系还是 Swap 区域？
- 当前 source 与 target 分别是什么？
- 这一条命令改变当前状态还是持久配置？
- 当前选项与 fstab 文本是否一致？
- 哪个证据能证明 unit 生成与启动结果？
- mount -a 或 swapon -a 实际测试了什么？
- 失败时下一条最有区分度的证据是什么？
- 哪些结论仍需要真实重启或业务验证？

学习提示：每次验证都先说明证据属于当前状态、持久配置、unit 状态还是功能结果；不能用其中一层替代其他层。

第 24 章 · 正文

目录存在，不等于文件系统已经挂载；当前已经挂载，也不等于重启后仍会出现；Swap 已经带有签名，也不等于内核正在使用它。存储题最常见的失误不是某个选项记错，而是拿一层证据替另一层下结论：看到 `/srv/project` 能写文件便认为挂载完成，看到 `/etc/fstab` 有一行便认为启动安全，看到 `free -h` 有 Swap 总量便认为新增区域已经启用。

本章围绕两条状态链推进。文件系统侧是“源对象 → 当前挂载 → 挂载点可见内容 → fstab → systemd mount unit → 启动重建”；Swap 侧是“块对象 → Swap 签名 → 当前启用 → 优先级 → fstab → systemd swap unit”。前一章已经负责文件系统创建、UUID 与容量管理；本章只消费已确认的文件系统和块对象。NFS 的协议与权限语义留给第 26 章，autofs 的按需触发留给第 27 章。

概念

当前挂载（active mount） 是某个 mount namespace 中已经建立的“文件系统源 - 目录目标”关系。设备存在和目录存在都不能证明这条关系成立；必须从内核当前挂载表读取 source、target、类型和选项。手工 `mount` 通常只改变这一运行时层。

概念

持久挂载（persistent mount） 是系统能够在下一次启动或明确重入时重新建立目标关系的配置状态。RHEL 9 中常见真源是 `/etc/fstab`；当前已挂载并不会反向生成可靠的 fstab 条目，所以“现在可用”和“下次启动仍成立”必须分别验证。

概念

挂载点遮蔽（mount-point shadowing） 指新文件系统挂到非空目录后，目录原有内容暂时不可见，但并未被删除。卸载后原内容会重新出现。它既可能造成“数据消失”的误判，也可能掩盖挂载失败期间写入父文件系统的数据。

概念

`fstab` 条目 是由六个字段组成的持久声明，分别描述源、目标、类型、选项、`dump` 字段和启动检查序列。它表达管理员意图，不是当前挂载事实；文本存在也不等于条目可解析、源可用或启动一定成功。

概念

`systemd mount unit` 是 `systemd` 对一个挂载点的运行对象。`fstab` 条目通常经生成器转换为临时 `mount` 或 `swap` unit；unit 名由路径转义得到，状态和 `Journal` 能解释启动阶段发生了什么，但生成 unit 不是应直接编辑的配置真源。

概念

`Swap` 当前状态与持久配置 至少分三层：`mkswap` 写入 `Swap` 签名，`swapon` 让内核当前启用该区域，`fstab` 决定后续启动或 `swapon -a` 是否重新启用。`USED=0` 仍可能是已启用状态；具体设备与优先级应从 `swapon --show` 读取。

操作语义先理解命令作用对象，再记关键形式。以下入口覆盖当前挂载、证据查询、`busy` 调查、`fstab` 验证与 `Swap` 生命周期。

mount / umount

SYNOPSIS

```
mount [-t TYPE] [-o OPTIONS] SOURCE TARGET
mount TARGET
umount TARGET
```

建立或解除当前 `mount namespace` 中的挂载关系；它们本身不会自动创建或删除 `fstab` 持久条目。

重要参数 / 形式

```
mount SOURCE TARGET
    显式建立当前关系。

mount TARGET
    从 fstab 补齐源、类型与选项，适合单条重入验证。

-t TYPE
    显式指定文件系统类型。

-o ro,rw,nodev,nosuid,noexec
    为当前挂载选择行为或限制。

umount TARGET
    解除当前关系；busy 时先调查引用，不默认使用 lazy 或 force。
```

findmnt

SYNOPSIS

```
findmnt [--mountpoint DIR] [--source SOURCE]
findmnt --target PATH
findmnt --fstab [--mountpoint DIR]
findmnt --verify
```

从当前挂载表或配置表读取 `source`、`target`、类型、选项和层次关系，是本章的主证据入口。

重要参数 / 形式

```
--mountpoint DIR / -M DIR
    只匹配这个目录本身作为挂载点的记录。

--target PATH / -T PATH
    查询任意路径实际位于哪个已挂载文件系统；路径不是独立挂载点时会返回其父文件系统。

--source SOURCE
    查找某个源当前挂到哪里。

-R TARGET
    查看目标下的嵌套挂载。

--fstab
    读取 fstab 视图，而不是当前关系。
```

`--verify`

静态核对 `fstab`；不能替代实际挂载或真实重启。

`fuser` / `lsof`

SYNOPSIS

```
fuser -vm TARGET
lsof TARGET
lsof +D DIRECTORY
```

在 ``umount`` 报 `busy` 时识别持有工作目录、打开文件或其他引用的进程，再选择最小释放方式。

重要参数 / 形式

`fuser -m`

把目标按已挂载文件系统解释。

`-v`

显示用户、PID 和访问类型。

`lsof TARGET`

查询明确路径的打开对象。

`lsof +D DIRECTORY`

递归扫描目录树；范围大时成本较高，不应无条件使用。

fstab 编辑与重入验证

SYNOPSIS

```
systemctl daemon-reload
findmnt --verify
mount -a
```

`fstab` 修改后，先让 `systemd` 重新加载生成器结果，再做静态核对和当前环境中的实际重入测试。

重要参数 / 形式

`daemon-reload`

重新读取 `unit` 与 `generator` 结果；不等于挂载成功。

`findmnt --verify`

发现字段、类型和部分引用问题。

`mount -a`

尝试 `fstab` 中未挂载且非 `noauto` 的文件系统；已挂载目标可能不会被重新测试。

真实重启

仍是启动链最终验证之一，但必须在静态检查、重入测试和恢复条件充分后执行。

`mkswap` / `swapon` / `swapoff`

SYNOPSIS

```
mkswap [-L LABEL] DEVICE
swapon [-p PRIORITY] DEVICE
swapoff DEVICE
swapon -a
```

分别写入 `Swap` 签名、当前启用、当前停用，以及按 `fstab` 重新启用自动条目；每一步改变的状态层不同。

重要参数 / 形式

`mkswap -L LABEL`

写入签名和可读标签；这是覆盖性操作，必须先确认设备身份。

`swapon -p N`

以显式优先级启用，数值越高越优先。

`swapoff DEVICE`

只停用目标区域；先检查 `USED`、内存余量和其他 `Swap`。

`swapon -a`

依据 fstab 启用自动 Swap 条目；不应与危险的 `swapoff -a` 混淆。

swapon --show / systemd unit

SYNOPSIS

```
swapon --show --output NAME,TYPE,SIZE,USED,PRI0
systemctl status srv-project.mount
systemctl status dev-vdb2.swap
```

读取具体 Swap 的当前状态与优先级，并从 `.mount` / `.swap` unit` 观察 systemd 当前结果与失败入口。

重要参数 / 形式

`NAME,TYPE,SIZE,USED,PRI0`

设备级验收需要的核心列。

`systemd-escape --path --suffix=mount PATH`

把挂载路径转换为对应 unit 名。

`systemctl status UNIT`

查看 active/result 与近期日志；active 仍不能证明业务数据正确。

知识专题挂载不是复制：源、目标、当前关系与可见内容

第一次接触挂载时，人们容易把挂载点理解为“存储设备对应的文件夹”。这个比喻只能帮助入门，不能支撑排错。准确模型是：文件系统已经存在于某个源对象上，`mount` 把它的根目录接入当前目录树的某个目标目录；进程随后通过路径访问该文件系统。判断时必须分别识别源、目标、当前关系和目录原有内容。

①知识点source、target 和 active mount 是三个不同对象

- source：承载文件系统的对象，可以写成设备路径、`UUID=`、`LABEL=`，也可能是网络源；
- target：目录树中的挂载点，必须是目录；
- active mount：当前 mount namespace 中已经建立的源-目标关系。

设备存在只能证明源对象存在；目录存在只能证明目标目录存在。只有 `findmnt`、`mount` 的当前表或内核挂载信息才能证明 active mount。

```
findmnt --mountpoint /srv/project
findmnt -T /srv/project/subdir/file
```

第一条只匹配 `/srv/project` 本身作为挂载点的记录；第二条回答“给定路径实际位于哪个已挂载文件系统上”。如果 `/srv/project` 只是根文件系统中的普通目录，`findmnt -T` 仍会返回根文件系统，因此验收时还要比较期望的 source 和 target，不能只看命令有输出。

②知识点当前挂载关系属于运行时状态

手工执行：

```
mount UUID=<REAL_UUID> /srv/project
```

只改变当前运行状态。系统重启后，内核重新建立根文件系统和启动过程要求的挂载，手工关系不会凭空恢复。要形成启动持久性，必须有可被启动系统消费的配置，RHEL 9 常见入口是 `/etc/fstab`。

在容器、隔离服务或使用不同 mount namespace 的进程中，某个挂载关系还可能只对特定 namespace 可见。本章不展开 namespace 管理命令，但保留一个判断边界：不要假设“一个终端能看到”就代表所有隔离进程都能看到。

③知识点挂载到非空目录会遮蔽，不会删除

假设 `/srv/project` 原本包含 `local.txt`，挂载另一个文件系统后，路径 `/srv/project` 显示的是新文件系统的内容，`local.txt` 暂时不可见。正常卸载后，原内容重新出现。

这会制造两类高频事故：

1. 误判数据丢失：挂载后原目录项“消失”，其实被遮蔽；
2. 写入错误层：本应挂载的文件系统未挂载，应用继续向普通目录写数据；稍后挂载成功，这些数据又被遮蔽。

变更前应检查挂载点是否已有需要保留的内容：

```
findmnt --mountpoint /srv/project
ls -la /srv/project
```

若目录非空，先确定这些内容属于预期文件系统还是父文件系统，不能直接覆盖现场。

④验证点“路径可写”不是挂载成功的证据

`touch /srv/project/test` 成功只证明当前路径可写；它可能仍位于根文件系统。更可靠的证据链是：

```
findmnt --no SOURCE,TARGET,FSTYPE,OPTIONS --mountpoint /srv/project
df -hT /srv/project
```

`findmnt` 核对 `source`、`target`、类型和当前选项，`df` 提供该路径所在文件系统的容量视角。功能写入测试应放在身份核对之后。

Cheatsheet

`source` 是文件系统来源，`target` 是目录，`active mount` 是当前关系；目录存在 ≠ 已挂载；挂载会遮蔽原内容；先用 `findmnt` 证明身份，再做读写测试。

操作专题用 findmnt 建立可信的挂载证据

`mount` 不带参数可以列出当前挂载，但输出通常很长，容易让目标关系淹没在伪文件系统和系统挂载中。`findmnt` 更适合围绕“目标路径、源对象、配置表或层次关系”定向查询。本专题把它作为本章主证据入口，而不是只在操作结束时随手运行一次。

①操作点按挂载点查询当前关系

```
findmnt --mountpoint /srv/project
findmnt --no SOURCE,TARGET,FSTYPE,OPTIONS --mountpoint /srv/project
```

第一条保留表头，便于阅读；第二条适合任务验收或脚本化采集。检查时至少回答：

- `SOURCE` 是否为目标文件系统；
- `TARGET` 是否正好是题目要求的目录；
- `FSTYPE` 是否符合要求；
- `OPTIONS` 中是否存在目标选项和意外限制。

②操作点查询任意路径实际落在哪个文件系统

```
findmnt -T /srv/project/reports/summary.txt
```

`-T / --target` 的路径解析适合调查“应用实际写入了哪个文件系统”。当挂载缺失时，它可能返回 `/` 对应的文件系统，这正是区别“目录存在”和“目标挂载存在”的证据。

③操作点按源查询并识别同源多挂载

```
findmnt --source UUID=<REAL_UUID>
findmnt --source /dev/mapper/projectvg-projectlv
```

一个源可能通过 `bind mount` 或以其他方式出现在多个目标。按源查询可以发现“文件系统确实挂载了，但不在题目要求的位置”。设备路径可能存在多种别名，必要时结合 `lsblk -f`、`blkid` 和 `findmnt` 输出交叉核对。

④操作点查看子挂载和层次关系

```
findmnt -R /srv/project
```

递归视图对卸载诊断很重要。即使没有进程打开普通文件，目标下方的子挂载仍可能使整体卸载流程失败或留下意外状态。先从层次关系判断，再决定是否逐个处理子挂载。

⑤比较点`findmnt`、`df`、`lsblk`和`mountpoint`的证据边界

命令	主要回答	不能单独证明
<code>findmnt</code>	当前挂载源、目标、类型、选项、层次	<code>fstab</code> 一定正确；业务数据一定正确
<code>df -hT PATH</code>	路径所在文件系统的容量和使用率	期望 <code>source</code> ；持久配置
<code>lsblk -f</code>	块设备、文件系统签名、UUID、部分挂载点	当前挂载的全部层次与选项
<code>mountpoint -q DIR</code>	目录是否是一个挂载点	挂载源、类型和选项是否正确

Cheatsheet

精确挂载点用 `findmnt --mountpoint`；任意路径归属用 `findmnt -T`；同源挂载用 `--source`；`busy` 前先 `findmnt -R`；`df` 是容量证据，不是持久证据。

操作专题建立和调整当前挂载

当前挂载操作的重点不是记忆一种固定命令，而是明确 `source` 的表达式、`target` 是否安全、选项改变了哪些行为，以及本次操作是否只存在于当前运行状态。

①操作点使用设备路径、UUID 或 LABEL 挂载

基本形式：

```
mount <SOURCE> <TARGET>
```

常见示例：

```
mount /dev/vdb1 /srv/project
mount UUID=<REAL_UUID> /srv/project
mount LABEL=PROJECT /srv/project
```

设备路径直观，但枚举名或映射别名可能变化；UUID 通常最适合唯一标识文件系统；LABEL 可读性更强，但管理员需要保证不会出现歧义。UUID 和 LABEL 必须从当前对象查询，不能复制讲义示例。

```
blkid /dev/vdb1
lsblk -f /dev/vdb1
```

文件系统创建、重新格式化和部分签名操作归第 23 章，本章只消费已经确认的身份。

②参数点 `-t`、`-o`` 与只读挂载

```
mount -t xfs -o nodev,nosuid UUID=<REAL_UUID> /srv/project
mount -o ro UUID=<REAL_UUID> /srv/archive
```

- `-t` 显式指定文件系统类型；通常内核和工具可根据签名识别，但题目或异常场景可能要求明确类型；
- `-o` 传入逗号分隔选项；
- `ro` 建立只读挂载，`rw` 表示读写；
- `nodev`、`nosuid`、`noexec` 分别限制设备文件解释、`setuid/setgid` 语义和直接执行，不能互相替代。

`noexec` 不是完整安全沙箱，解释器仍可能读取脚本；这些选项应按目标使用，不应机械添加。

③操作点利用 `fstab` 简写建立当前挂载

当 `/etc/fstab` 已有唯一条目时，可只给 `target` 或 `source`：

```
mount /srv/project
mount UUID=<REAL_UUID>
```

此时 `mount` 从 `fstab` 补齐另一端、类型和选项。这种操作适合验证单条配置，但仍要用 `findmnt` 检查实际结果。

④操作点 `remount` 只改变当前状态

```
mount -o remount,ro /srv/project
mount -o remount,rw /srv/project
```

`remount` 用于调整当前已挂载对象的部分选项。它不会自动修改 `/etc/fstab`。若题目要求重启后也保持只读，必须同步修改持久配置并重新验证。

⑤验证点挂载后按身份、策略和功能三层验收

```
findmnt -no SOURCE,TARGET,FSTYPE,OPTIONS --mountpoint /srv/project
df -hT /srv/project
touch /srv/project/.rh24-write-test
rm -f /srv/project/.rh24-write-test
```

只读目标不应执行写入测试来“证明失败”；应改为读取已知文件或创建符合任务要求的验收方式。功能测试失败时，先根据错误区分只读、权限、SELinux、容量或文件系统问题，不要立即重新格式化。

Cheatsheet

`mount SOURCE TARGET` 建立当前关系；UUID 唯一性通常优于路径，LABEL 要防重复；`-o` 改变行为；remount 不改 fstab；验收依次看 source/target/type/options、容量、功能。

诊断专题安全卸载：target is busy 是保护证据

卸载失败不是命令“不给面子”，而是内核在阻止仍被引用的文件系统突然消失。正确流程是识别引用者、释放最小范围的引用、再重试；lazy 或 force 只能作为理解边界，不能成为考试和工作中的默认答案。

①操作点`umount`解除当前关系

```
umount /srv/project
```

通常优先使用明确的挂载点。卸载不会删除文件系统内容，也不会删除挂载点目录，更不会自动删除 fstab 条目。若持久条目仍存在，下一次 mount -a 或启动可能重新挂载。

②诊断点当前工作目录会造成 busy

Shell 位于挂载点内部时：

```
cd /srv/project
umount /srv/project
```

可能失败。先离开该文件系统：

```
cd /
umount /srv/project
```

服务进程也可能把该文件系统作为工作目录或根目录；不能只检查当前终端。

③诊断点打开文件、执行映像和映射对象会保持引用

```
fuser -vm /srv/project
```

fuser -vm 按文件系统显示相关进程、用户和访问类型，是 busy 的首选证据之一。如果系统安装了 lsof，可先查询挂载点本身；只有在目录树规模可控时才使用递归扫描：

```
lsof +f -- /srv/project
# 或在范围较小且可接受递归成本时
lsof +D /srv/project
```

不要看到 PID 就直接 kill -9。先识别进程属于哪个服务、是否可以正常停止、是否正在写数据，再通过应用或 systemctl 释放引用。

④诊断点子挂载需要先处理

```
findmnt -R /srv/project
```

若 /srv/project/cache 还有独立挂载，应先理解依赖并从内到外卸载。不要使用递归卸载隐藏自己尚未识别的业务挂载。

⑤边界lazy 和 force 为什么不是第一选择

- umount -l 会立即从目录树分离，等引用释放后再清理；进程可能继续访问旧对象，现场更难理解；
- umount -f 主要面向特定不可达网络文件系统等情况。本地正常挂载不应靠 force 跳过调查。

考试中看到 busy，推荐链路是：

```
症状 → findmnt -R → fuser/lsof → 识别管理者 → 最小停止或离开目录 → umount → findmnt 再验证
```

⑥验证点卸载后证明关系消失而目录仍在

```
findmnt --mountpoint /srv/project
mountpoint /srv/project
ls -ld /srv/project
```

预期是 current mount 消失，挂载点目录仍存在。若目录原有内容重新出现，需要按遮蔽模型解释，不要把它当成卸载生成的新文件。

busy 先查子挂载，再查进程引用；当前 Shell 要离开目录；优先正常停止服务；`l`/`-f` 不是通用修复；卸载后验证关系消失、目录保留。

知识专题fstab 六字段：把对象、接入点和启动策略写清楚

/etc/fstab 的价值不是“记住一行模板”，而是用六个字段声明一个可重复建立的存储状态。每个字段回答不同问题：挂载谁、挂到哪里、按什么类型解释、使用什么策略、是否参与 dump、启动检查顺序如何。字段写对只是起点，还需要 systemd 重新加载、静态核对和实际挂载。

①知识点六字段的顺序与责任

```
fs_spec fs_file fs_vfstype fs_mntops fs_freq fs_passno
```

fs_spec

指定源对象。可写设备路径、UUID=...、LABEL=... 等；普通本地文件系统通常优先使用从当前对象查询到的真实 UUID。

fs_file

指定接入目录树的位置，例如 /srv/project。Swap 不形成目录挂载，第二字段通常写 none。

fs_vfstype

指定解释类型，例如 xfs、ext4，或用于交换区域的 swap。

fs_mntops

指定逗号分隔策略，例如 defaults,nodev,nosuid 或 defaults,pri=30。

fs_freq

传统 dump 字段。RHCSA 常见本地挂载和 Swap 条目通常写 0。

fs_passno

指定启动检查序列；0 表示不参与该序列。不要把某个文件系统的示例值机械套到所有类型。

普通挂载骨架：

```
UUID=<REAL_UUID> /srv/project xfs defaults,nodev,nosuid 0 0
```

Swap 骨架：

```
UUID=<REAL_UUID> none swap defaults,pri=30 0 0
```

②比较点UUID、LABEL 与设备路径

- UUID=：通常唯一，设备枚举顺序变化时仍能匹配同一文件系统；重新格式化后 UUID 会改变；
- LABEL=：可读性好，适合明确命名；若多个对象使用相同 LABEL，解析会产生歧义；
- /dev/vdb1、/dev/sdb1：简单直观，但某些环境中枚举顺序可能变化；
- /dev/mapper/... 或 /dev/<VG>/<LV>：LVM 对象常用稳定映射，但 LVM 创建和扩容归第 25 章。

写入前必须查询当前对象：

```
blkid /dev/vdb1
lsblk -f /dev/vdb1
```

③知识点`defaults` 不是“没有选项”

defaults 表示一组默认挂载行为，由 mount 和文件系统实现共同解释。它不会自动添加 nodev、nosuid、noexec，也不会表达“失败无所谓”。安全或启动策略必须显式写出。

常用通用选项：

选项	作用
rw / ro	读写或只读
nodev	不解释该文件系统上的块/字符设备文件
nosuid	忽略 setuid/setgid 位的提权语义
noexec	不允许从该挂载直接执行二进制；不是完整隔离
noauto	mount -a 和正常自动挂载流程跳过，需手动触发

选项	作用
<code>nofail</code>	不把该挂载作为达到相关启动 target 的强制要求
<code>_netdev</code>	强制把挂载按依赖网络的挂载处理

文件系统专用选项数量很多，本章只覆盖通用且高价值的判断。遇到陌生选项应查 `mount(8)` 和对应文件系统手册页。

④边界字段中的空格不能直接出现

`fstab` 以空白分隔字段。路径或标签中包含空格时，需要使用转义形式，例如 `\040`。引号不能自动消除 `fstab` 字段分隔语义。考试环境通常避免复杂路径，但工作中看到带空格条目时，应先确认转义而不是反复增加引号。

⑤边界`nofail`不能修复错误配置

启动风险：`nofail` 改变 `systemd` 依赖和启动完成条件，使相关挂载失败时系统仍可能继续到达目标状态。它不会修复错误 UUID、错误类型、损坏文件系统或权限问题。必需业务盘若错误地使用 `nofail`，系统可能“正常启动”但应用把数据写进未挂载目录，风险反而更大。

Cheatsheet

六字段依次是 source、target、type、options、dump、fsck pass；UUID 要从当前对象查询；`defaults` 不包含所有安全选项；`noauto` 是不自动挂，`nofail` 是失败不阻断 target；空格用 `\040`。

操作专题从修改 fstab 到无重启完整验证

编辑 `fstab` 后立即重启，是把一个可控的配置错误升级为启动故障。更稳妥的流程是在当前会话中完成静态检查、实际建立状态、身份核对和功能验证。无重启验证不能百分之百替代真实启动，但可以提前发现绝大多数语法、身份和挂载错误。

①操作点变更前建立基线并保留可回退副本

```
cp -a /etc/fstab /etc/fstab.rh24-before
findmnt --mountpoint /srv/project
findmnt --fstab --mountpoint /srv/project
```

备份不是让系统自动回滚，而是保留变更前配置。还应记录目标当前是否已挂载，因为 `mount -a` 通常跳过已经挂载的文件系统；若目标原本已由手工命令挂载，单纯运行 `mount -a` 不能证明新 `fstab` 条目能独立建立关系。

②操作点写入真实身份和明确策略

```
UUID=<REAL_UUID> /srv/project xfs defaults,nodev,nosuid 0 0
```

编辑时避免重复条目。一个目标出现多行或一个 source 被意外声明到多个目标，会使后续行为和验收复杂化。写完后先读取对应行，确认没有把占位符写入真配置。

③操作点让 systemd 重新运行生成器

```
systemctl daemon-reload
```

`daemon-reload` 让 `systemd` 重新读取 unit 配置并重新运行生成器，包括由 `fstab` 产生的 `mount/swap` unit。它不会自动把所有新条目挂载，也不会证明配置正确。

④验证点`findmnt --verify`做静态核对

```
findmnt --verify
```

它用于检查 `fstab` 的可解析性和部分明显问题。静态检查通过不能证明源设备当前可用、文件系统可挂载或业务权限正确；失败时应按具体报告修正，不要直接重启尝试。

⑤操作点用`mount -a`实际尝试尚未挂载的条目

```
mount -a
```

`mount -a` 处理 `fstab` 中适用且未被 `noauto` 排除的挂载，并通常忽略已经挂载的对象。命令无输出只表示没有直接报告错误，不能证明目标 source、类型和选项满足题意。

为了验证单条新配置能从“未挂载”状态建立，推荐在确认安全后：

```
umount /srv/project
mount /srv/project
```

或在目标本来未挂载时直接 `mount -a`。生产系统不能为了测试随意卸载业务盘，应选择维护窗口或真实重启验证。

⑥验证点当前、持久、unit 和功能四层收口

```
findmnt --no SOURCE,TARGET,FSTYPE,OPTIONS --mountpoint /srv/project
findmnt --fstab --mountpoint /srv/project
df -hT /srv/project
systemctl status srv-project.mount
```

然后按任务执行最小功能验证。四层分别回答：当前关系、持久声明、容量/类型视角、systemd unit 状态。任何一层都不能独自扩大为“启动后业务一定正常”。

Cheatsheet

备份和基线 → 写真实 UUID → `daemon-reload` → `findmnt --verify` → 从未挂载状态实际挂载 → `findmnt`/`df`/unit/功能分层验收；无重启验证不等于真实重启。

知识专题systemd mount unit、启动依赖与网络边界

RHEL 9 的启动不是按 fstab 从上到下机械执行命令。systemd 读取由生成器创建的 unit，结合依赖和排序启动挂载。理解 unit 名和失败入口，可以把“启动卡住”从猜测变成可查询对象。

①知识点挂载点路径决定`.mount` unit 名

`/srv/project` 对应：

```
systemd-escape --path --suffix=mount /srv/project
# srv-project.mount
```

根挂载对应特殊的 `-.mount`。unit 名必须与路径转义规则一致，不能随意起名。由 fstab 生成的 unit 可查询：

```
systemctl status srv-project.mount
systemctl show srv-project.mount -p ActiveState,SubState,Result,Where,What
systemctl cat srv-project.mount
```

生成结果通常位于 `/run/systemd/generator*` 等运行时目录，重载或重启会重建；管理员应修改 fstab，而不是编辑生成文件。

②知识点unit active 只证明当前挂载成功

`active (mounted)` 表明 systemd 当前认为挂载已建立。它不能证明：

- source 是题目要求的那个对象；
- 选项完全正确；
- 应用目录权限和 SELinux 正确；
- 下一次设备缺失时启动仍符合业务要求。

因此 unit 状态必须与 `findmnt` 和功能证据组合。

③诊断点mount unit 失败时的证据链

```
systemctl status srv-project.mount
journalctl -b -u srv-project.mount
findmnt --verify
blkid
```

常见假设包括：source 不存在、UUID 过期、挂载点缺失、类型错误、选项不支持、文件系统本身异常。下一条证据应区分这些假设，而不是立即 `mkfs`。

④边界`\_netdev` 解决分类和排序，不解决连通性

systemd 通常根据文件系统类型识别网络挂载。若底层依赖网络但类型本身看起来像本地文件系统，例如网络块设备上承载普通文件系统，可在 fstab 选项中使用 `_netdev` 强制按网络挂载处理。它影响与网络上线及 `remote-fs.target` 等相关的依赖和排序。

`_netdev` 不会配置 IP、DNS、认证或远端服务，也不会让不可达存储变得可达。NFS 语法、版本和身份映射归第 26 章。

⑤边界`nofail` 改变启动要求，应由业务重要性决定

非关键、可选设备有时适合 `nofail`；关键数据盘通常不应为了“让机器能启动”而静默跳过。决策前要回答：

- 缺失时应用是否会误写父文件系统；
- 是否有监控能发现挂载失败；
- 是否允许系统继续提供降级服务；
- 恢复入口归第 30 章还是本章当前可修复。

Cheatsheet

路径用 `systemd-escape` 转 `mount` 名；generated unit 不直接编辑；失败查 status + 本次启动 Journal；`\_netdev` 是网络分类，`nofail` 是启动依赖策略，都不是错误配置修复器。

操作专题Swap 的签名、当前启用和持久配置

Swap 不是普通文件系统挂载。它没有目录挂载点，内核把它作为换页后备区域使用。考试任务经常要求“新增 Swap、立即启用、重启后仍有效、不得影响现有 Swap”，这实际上要求分别处理签名、当前表、fstab 和设备级验收。

①操作点变更前记录现有 Swap

```
swapon --show --output NAME,TYPE,SIZE,USED,PRI0
cat /proc/swaps
free -h
```

swapon --show 是设备级证据；free -h 只提供汇总。记录基线是为了证明新增操作没有停掉或覆盖已有 Swap。

②操作点`mkswap`写入 Swap 签名

对已经由前置章节认可可写、无需保留数据的对象：

```
mkswap -L EXAMSWAP /dev/vdb2
blkid /dev/vdb2
```

mkswap 会写入新的 Swap 元数据和 UUID。它不是无损“标记命令”，不得对未知数据对象试运行。只存在 TYPE="swap" 仍不能证明当前启用。

③操作点`swapon`当前启用并设置优先级

```
swapon -p 30 /dev/vdb2
```

或先写入 fstab 再执行：

```
swapon -a
```

swapon -a 根据 fstab 启用适用的 Swap 条目，通常跳过 noauto。当前命令成功不会自动写入 fstab。

④配置点按真实 UUID 持久启用

```
UUID=<REAL_SWAP_UUID> none swap defaults,pri=30 0 0
```

Swap 的第二字段通常写 none、第三字段为 swap。最后两个字段通常为 0 0。修改后：

```
systemctl daemon-reload
findmnt --verify
```

Swap unit 名通常由设备身份和生成器决定，实际验收以 swapon --show、fstab 和必要的 systemctl 查询组合为主。

⑤验证点`USED=0`仍可能是正常活动状态

```
swapon --show --output NAME,TYPE,SIZE,USED,PRI0
```

只要设备出现在活动表中，它就是已启用 Swap；USED=0 只是当前没有页面被换出到该区域。不要为了制造使用量而人为施加内存压力。

⑥验证点从停用状态重新启用验证持久条目

在确认目标无关键 Swap 使用、系统内存充足且只操作新增区域后：

```
swapoff /dev/vdb2
swapon -a
swapon --show --output NAME,TYPE,SIZE,USED,PRI0
```

这一过程验证 fstab 能重新启用该区域，但仍不是完整真实重启。禁止使用 `swapoff -a`，因为题目通常要求保留现有 Swap，且大范围换入可能导致内存压力或失败。

Cheatsheet

基线 → `mkswap` 写签名 → `blkid` 取 UUID → `swapon -p` 当前启用 → fstab `none swap pri=` → `daemon-reload`/verify → `swapon --show`; `USED=0` 不等于未启用。

知识专题Swap 优先级和安全停用

多个 Swap 区域可以同时活动。优先级让内核在多个后备区域之间做选择；停用则要求把该区域上的页面迁回 RAM 或其他 Swap。两者都需要设备级证据，不能只看总量。

①知识点数值更高的优先级更先使用

fstab 中使用：

```
UUID=<UUID_A> none swap defaults,pri=10 0 0
UUID=<UUID_B> none swap defaults,pri=30 0 0
```

当前启用也可用：

```
swapon -p 30 /dev/vdb2
```

实际优先级以 `swapon --show ... PRI0` 为准。不要只凭 fstab 文本判断，因为当前区域可能是在修改前以其他优先级启用的。

②知识点显式优先级比依赖默认分配更可验证

未显式设置时，优先级由系统分配，可能不符合题目要求。考试要求具体优先级时，应写入 `pri=<N>` 并重新启用目标 Swap，确认当前 PRI0。

③边界`swapoff` 前检查目标、使用量和可用内存

```
swapon --show --output NAME,SIZE,USED,PRI0
free -h
```

若目标 Swap 有大量使用且 RAM/其他 Swap 不足，`swapoff` 可能失败或造成严重内存压力。生产系统应安排窗口；考试环境也应只停用明确的新区域，不做全局清理。

④操作点删除或替换 Swap 的正确顺序

真正删除一个 Swap 时，通常按：

```
确认目标和内存余量
→ swapoff 指定区域
→ 从 fstab 移除或修改条目
→ daemon-reload
→ 再处理签名、分区或 LV
```

擦除签名和分区属于第 22 章边界；本章只强调不能在仍活动时破坏底层对象。

⑤验证点新增任务必须证明旧 Swap 保持不变

操作前后保存 `swapon --show`，比较已有 NAME、SIZE 和 PRI0。只证明总量增加，不足以证明原有条目没有被意外停用或改变。

Cheatsheet

`pri` 数值越高越优先；修改 fstab 后要让目标重新启用才会反映当前 PRI0；`swapoff` 先查 USED 和内存；永远优先指定单个目标，不用 `swapoff -a` 作为默认步骤。

诊断专题从症状推进到挂载、fstab、unit 或 Swap 层

存储诊断最危险的动作是“看到失败就重建”。错误 UUID、挂载缺失、unit 失败和 Swap 未启用都不要求重新格式化。诊断要沿状态层逐步缩小，而不是破坏上游对象来制造一个新状态。

①诊断点目录存在，但数据写进了根文件系统

症状： `/srv/project` 可写，但容量和预期数据不对。

```
findmnt --mountpoint /srv/project
findmnt -T /srv/project
```

若 `--mountpoint` 没有记录，或 `-T` 显示它属于 `/`，说明目标文件系统当前未接入。下一步查看 `fstab`、`unit` 和 `source`，不要删除目录或重新 `mkfs`。

②诊断点`mount -a`报 special device/UUID 不存在

```
blkid
lsblk -f
findmnt --fstab
```

区分：抄错 UUID、对象被重新格式化后 UUID 变化、设备尚未出现、配置使用了错误路径。修正 `fstab` 后 `daemon-reload`、`verify`、再挂载。重新格式化只会再次改变 UUID 并可能毁掉数据。

③诊断点wrong fs type、bad option 或 bad superblock

这类错误只给出了可能原因的集合。继续核对：

```
blkid <DEVICE>
findmnt --fstab --mountpoint /srv/project
journalctl -b -u srv-project.mount
```

可能是 `fstab` 类型错误、选项不支持、文件系统工具缺失或文件系统异常。文件系统检查与修复归第 23 章，不在未确认前执行破坏性命令。

④诊断点fstab 正确但 unit 仍 failed

```
systemctl status srv-project.mount
systemctl reset-failed srv-project.mount
systemctl start srv-project.mount
journalctl -b -u srv-project.mount
```

`reset-failed` 只清理失败状态计数，不修复根因。修正配置后再启动并核对 `findmnt`。

⑤诊断点挂载当前成功，但重启后缺失

检查：

```
findmnt --fstab --mountpoint /srv/project
systemctl cat srv-project.mount
systemctl is-enabled srv-project.mount
```

由 `fstab` 生成的 `mount unit` 不应机械套用 `service` 的 `enable` 逻辑；重点是条目是否存在、是否被 `noauto` 排除、`generator` 是否加载、启动依赖是否正确。真实重启仍需后续环境验证。

⑥诊断点Swap 有签名但总量没有增加

```
blkid /dev/vdb2
swapon --show
swapon /dev/vdb2
```

`blkid` 只证明签名。若 `swapon` 失败，读取具体错误并检查对象是否已被占用、是否重复启用或底层不合适。不要再次 `mkswap` 作为第一修复。

⑦诊断点fstab 中有 `pri=30`，当前 PRIO 却不同

配置只影响下一次启用。只针对该区域安全执行：

```
swapoff /dev/vdb2
swapon -a
swapon --show --output NAME,PRI0
```

若无法停用，先处理内存余量，不要扩大到 `swapoff -a`。

⑧诊断点使用 `nofail` 后系统启动但业务异常

检查目标是否真正挂载：

```
findmnt --mountpoint /srv/project
systemctl status srv-project.mount
journalctl -b -u srv-project.mount
```

`nofail` 可能让启动继续，却不会让应用自动停止向父文件系统写入。最小修复是恢复目标挂载并评估错误层数据，而不是继续忽略失败。

Cheatsheet

路径异常先 `findmnt`；UUID 错先查当前签名；unit failed 看本次启动 Journal；Swap 签名和活动表分开；配置改了但当前没变，考虑重新建立当前状态；任何诊断都不以重新格式化开场。

操作专题把考试步骤迁移成可审计的变更

真实系统往往已有 `fstab`、已有 Swap、已有应用数据和自动启动依赖。把本章迁移到工作中，需要在命令之外保留基线、变更范围、回退条件和分层证据。

①工作迁移变更前保存“对象与关系”而非只备份文件

除了备份 `fstab`，还应记录：

```
findmnt --mountpoint /srv/project
findmnt --fstab --mountpoint /srv/project
swapon --show --output NAME,TYPE,SIZE,USED,PRI0
blkid /dev/vdb1 /dev/vdb2
```

这样才能回答变更前发生了什么，而不是只有两个文本文件可比。

②工作迁移区分即时验证、重入验证和重启验证

- 即时验证：操作后立即查看当前状态；
- 重入验证：从未挂载/未启用状态用 `fstab` 再建立；
- 重启验证：真实启动完成后核对 unit、当前关系和业务功能。

本会话只能提供前两层的推荐命令，不能声称已完成第三层。

③工作迁移为监控选择可判定证据

对关键挂载，监控不应只检查目录存在。至少应检查期望 `target` 的 `source`、文件系统类型或标识，并监控容量。对 Swap，应记录活动区域和优先级，而不是只看总量。

④工作迁移为自动化准备稳定终态

手工终态应明确为：

```
真实 source 身份
+ 明确 target
+ 明确 fstype/options
+ 当前关系
+ 持久条目
+ unit/启动预期
+ 功能证据
```

RHCE 自动化章节再把该终态映射到 `ansible.posix.mount` 等模块。本章不展开 YAML，但要求状态模型足够稳定，避免自动化只“写一行 `fstab`”却不验证当前功能。

Cheatsheet

备份配置也记录当前关系；即时/重入/重启三种验证分开；监控检查身份而非目录；自动化消费终态，不替代本章对象模型。

经典任务为已有 XFS 建立安全的持久挂载

服务器中 `/dev/vdb1` 已由前置任务创建为 XFS 文件系统，但当前未挂载。`/srv/project` 已存在且为空，`/etc/fstab` 没有该对象的条目。要求：

- 使用当前系统查询到的真实 UUID；
- 持久挂载到 `/srv/project`；
- 挂载选项为 `defaults,nodev,nosuid`；
- 不重启完成当前和持久验证；
- 不重新格式化，不使用示例 UUID，不默认添加 `nofail`；
- 验收必须证明 source、target、类型、选项、容量视角和写入功能。

请先独立完成。参考解答从下一页开始。

参考解答从未挂载状态完成 fstab 与当前功能闭环

①调查确认源、挂载点和当前基线

```
lsblk -f /dev/vdb1
blkid /dev/vdb1
findmnt --mountpoint /srv/project
ls -la /srv/project
findmnt --fstab --mountpoint /srv/project
```

预期：`/dev/vdb1` 的 `TYPE` 为 `xfs`，可以读到真实 `UUID`；目标没有精确的当前挂载记录；目录没有需要保留的内容；`fstab` 没有冲突条目。任一条件不符合都先停止写配置。

②操作备份并写入真实条目

```
cp -a /etc/fstab /etc/fstab.rh24-c01-before
vi /etc/fstab
```

加入：

```
UUID=<从 blkid 获取的真实 UUID> /srv/project xfs defaults,nodev,nosuid 0 0
```

占位文本不能保留在真实文件中。

③验证重载、静态核对和实际挂载

```
systemctl daemon-reload
findmnt --verify
mount /srv/project
```

因为目标在操作前未挂载，`mount /srv/project` 会按 `fstab` 补齐 `source`、类型和选项，直接测试该条配置。

④分层验收当前关系、容量、unit 和功能

```
findmnt -no SOURCE,TARGET,FSTYPE,OPTIONS --mountpoint /srv/project
df -hT /srv/project
systemctl status srv-project.mount
touch /srv/project/.rh24-test
rm -f /srv/project/.rh24-test
```

检查 `OPTIONS` 中至少存在目标限制；输出中还会包含内核和文件系统添加的其他当前选项，不要求与 `fstab` 文本逐字相同。

⑤重入验证证明能从未挂载状态重建

在确认没有进程使用测试挂载后：

```
cd /
umount /srv/project
findmnt --mountpoint /srv/project
mount -a
findmnt -no SOURCE,TARGET,FSTYPE,OPTIONS --mountpoint /srv/project
```

这验证 `fstab` 在当前系统中可以重新建立状态。真实重启仍列入后续 `live test`。

⑥典型错误常见错误与修正方向

- 只执行 `touch`，实际写入根文件系统；
- 复制讲义 `UUID`；
- 目标已经手工挂载时直接运行 `mount -a`，误以为新条目被测试；
- 使用 `nofail` 掩盖错误 `UUID`；
- 忘记检查非空目录导致数据遮蔽。

经典任务新增 Swap、设置优先级并保持现有区域不变

系统已经存在至少一个活动 Swap。`/dev/vdb2` 是由前置任务确认可写的空闲分区，当前没有 Swap 签名。要求：

- 在 `/dev/vdb2` 上创建 Swap，LABEL 为 `EXAMSWAP`；
- 当前启用，优先级为 `30`；
- 使用真实 UUID 写入 `/etc/fstab`，启动时自动启用；
- 不得停用、删除或修改已有 Swap；
- 不得使用 `swapoff -a`；
- 使用设备级输出证明新区域、优先级和原有区域都正确；
- 在安全前提下，从停用状态用持久配置重新启用新区域。

请先独立完成。参考解答从下一页开始。

参考解答保留基线并完成 Swap 三层状态

①调查保存已有 Swap 和设备状态

```
swapon --show --output NAME,TYPE,SIZE,USED,PRI0 | tee /root/rh24-swap-before.txt
free -h
lsblk -f /dev/vdb2
blkid /dev/vdb2
```

若 `/dev/vdb2` 已有未知签名或归属，不执行 `mkswap`。空闲性调查的完整方法归第 22 章。

②操作写入 Swap 签名并获取真实 UUID

```
mkswap -L EXAMSWAP /dev/vdb2
blkid /dev/vdb2
```

记录输出中的 Swap UUID。

③操作当前按优先级启用

```
swapon -p 30 /dev/vdb2
swapon --show --output NAME,TYPE,SIZE,USED,PRI0
```

确认新区域出现且 `PRI0` 为 30；原有 Swap 仍存在。

④配置写入持久条目

```
cp -a /etc/fstab /etc/fstab.rh24-c02-before
vi /etc/fstab
```

加入：

```
UUID=<真实 Swap UUID> none swap defaults,pri=30 0 0
```

然后：

```
systemctl daemon-reload
findmnt --verify
```

⑤重入验证只停用新增区域，再从 fstab 恢复

先检查新区域 `USED`、系统可用内存和其他 Swap；满足安全条件后：

```
swapoff /dev/vdb2
swapon --show --output NAME,TYPE,SIZE,USED,PRI0
swapon -a
swapon --show --output NAME,TYPE,SIZE,USED,PRI0 | tee /root/rh24-swap-after.txt
```

禁止 `swapoff -a`。比较 before/after，确认原有区域没有消失或改变，新区域重新出现且优先级为 30。

⑥典型错误常见错误与修正方向

- `mkswap` 后只看 `blkid`，没有 `swapon`；
- 只运行 `free -h`，无法证明具体设备和优先级；
- `fstab` 写入 `pri=30` 后不重新启用，当前 `PRI0` 仍是旧值；
- 为了验证执行 `swapoff -a`，破坏现有 Swap；
- `USED=0` 被误判为未启用。

本章收束任何持久存储任务都要同时回答“现在”和“下次启动”

挂载与 Swap 的共同方法是把配置和运行状态拆开：

- 对象身份
 - 当前关系或活动状态
 - 当前策略
 - `fstab` 持久声明
 - `systemd` 生成 `unit`
 - 实际重入或启动
 - 功能与数据证据

文件系统侧，`findmnt` 是当前关系的主证据；`fstab` 和生成 `unit` 说明持久意图；`df` 和读写验证补充容量与功能。Swap 侧，`blkid` 证明签名，`swapon --show` 证明当前活动和优先级，`fstab` 证明启动策略。`mount -a`、`swapon -a` 和 `findmnt --verify` 都是验证工具，但都不能独立替代真实重启和业务验收。

遇到故障时，不重新创建对象来“试一试”。先确定失败位于身份、当前关系、配置、生成 `unit`、启动依赖还是功能层：选择下一条最有区分度的证据，完成最小修复，再从对应层向下重新验收。

最终 Cheatsheet

```
# 当前挂载
findmnt -no SOURCE,TARGET,FSTYPE,OPTIONS --mountpoint /srv/project
findmnt -T /srv/project/path
findmnt -R /srv/project

# 当前操作
mount UUID=<UUID> /srv/project
umount /srv/project
fuser -vm /srv/project

# 持久配置验证
systemctl daemon-reload
findmnt --verify
mount -a
systemctl status srv-project.mount

# Swap
mkswap -L EXAMSWAP /dev/vdb2
swapon -p 30 /dev/vdb2
swapoff /dev/vdb2
swapon -a
swapon --show --output NAME,TYPE,SIZE,USED,PRI0
```

主要判断表

看到的证据	可以证明	仍不能证明
<code>blkid</code> 显示 <code>UUID</code> / <code>TYPE=swap</code>	对象具有相应签名	已挂载、已启用或会在启动时恢复
<code>findmnt</code> 命中目标	当前 <code>mount namespace</code> 存在挂载关系	<code>/etc/fstab</code> 正确、重启后仍成立
<code>/etc/fstab</code> 存在条目	管理员声明了持久策略	条目可解析、源可用、当前已经生效
<code>findmnt --verify</code> 无致命问题	<code>fstab</code> 结构和部分引用通过静态核对	<code>mount -a</code> 一定成功、真实重启一定安全
<code>mount -a</code> 返回成功	当前环境能处理尚未挂载的自动条目	已经挂载的目标条目被重新测试、业务数据正确
<code>swapon --show</code> 出现设备	Swap 当前已启用并可读取优先级	<code>fstab</code> 持久配置正确、重启后必然恢复
<code>.mount</code> / <code>.swap</code> <code>unit</code> 为 <code>active</code>	<code>systemd</code> 当前认为该 <code>unit</code> 成功	应用正在访问正确数据、启动链没有被 <code>nofail</code> 掩盖

向下一章交接

本章把“已有文件系统或 Swap 如何进入当前状态并形成启动持久性”讲完整。下一章《LVM 逻辑存储》将向下进入 PV、VG、LV 和容量变更；那里创建出的 LV 仍需回到本章模型，分别完成文件系统、当前挂载、`fstab` 与重入验证。不要把 `lvcreate` 成功扩大成“存储已经可用”，也不要再在 LVM 章节重复本章全部持久挂载机制。

LVM 逻辑存储

把设备容量沿 PV、VG、LV、文件系统和挂载点逐层交付，并用证据证明创建、扩展与数据保护。

对象模型 操作语义 验证 诊断 经典任务

大字号阅读版讲义

本章阅读导航 先抓住一条主线：应用看到的目录容量，不是直接从“磁盘”得到，而是沿着块设备、PV、VG、LV、文件系统和挂载点逐层交付。任何一层没有建立或没有同步增长，最终目录都可能得不到预期空间。

01锁定业务路径先从挂载点确认当前源设备与文件系统

02确认设备身份写入前证明目标设备没有需保留数据或冲突归属

03建立容量池PV 提供容量，VG 汇聚并按 PE 管理

04表达分配目标用 -L、-l、加号和百分比精确表达容量

05同步上层容量LV 增长后让 XFS 或 ext4 使用新增块空间
06完成分层验收分别证明 LVM、文件系统、挂载和已有数据
专题地图

知识专题从设备到业务目录：六层对象与容量链
知识专题PE、LE、绝对值、增量与百分比
操作专题用定制报告建立 LVM 现状图
操作专题从确认空闲的设备创建 PV、VG 和 LV
操作专题VG 空间不足时扩展容量池
操作专题保留数据扩展 LV 与文件系统
诊断专题容量没有出现时定位断裂层
诊断专题缩容、删除和已有签名的安全边界
经典任务创建指定 extent 的项目卷并扩展
经典任务保留 ext4 数据扩展现有报告卷

本章收束把容量链还原成证据链

阅读时持续回答 1. 当前业务目录究竟由哪个文件系统和设备提供？ 2. 目标设备是否已有文件系统、挂载、PV 或 VG 归属？ 3. 目标 VG 的 PE 多大，还剩多少可分配 extent？ 4. 题目要求的是最终总量，还是在当前基础上增加？ 5. `L` 与 `l` 哪个更直接表达题意？ 6. LV 已经增长后，文件系统是否也增长？ 7. 当前挂载关系是否仍指向原来的 LV？ 8. 哪条证据证明已有数据没有被重新格式化或替换？

本章边界：分区表归第 22 章，文件系统通用机制归第 23 章，`fstab` 持久挂载归第 24 章。本章只调用这些前置能力，不把相邻章节重新复制一遍。

第 25 章 · 正文

一块设备显示“还有空间”，并不等于应用已经获得可用容量。LVM 处理的是块设备层的容量组织：设备先被初始化为 PV，PV 汇入 VG，VG 再把固定大小的 extent 分配给 LV。LV 仍然只是一个块设备；上层文件系统必须增长，挂载点也必须继续指向正确源，业务目录才真正获得新增空间。

最常见的误判有四类：看到 `/dev/vdb1` 就假定它空闲；看到 `lvextend` 成功就假定目录容量已经增长；把 `-L 512M` 误当成“增加 512 MiB”；在遇到签名、空间不足或错误归属时，用删除、重建或强制参数绕过证据。本章不从命令清单开始，而是按“对象 → 状态 → 查询 → 修改 → 验证 → 诊断 → 安全边界”推进。

创建任务沿正向链工作：设备 → PV → VG → LV。扩容任务则应从业务路径反向定位：挂载点 → 文件系统 → LV → VG → PV。两条链最终都必须回到同一套验收：容量来源正确、对象关系正确、文件系统可见容量正确、挂载源没有变化、原数据仍然可复核。

概念

物理卷（Physical Volume，PV）是获得 LVM 管理身份的块设备或分区。`pvcreate` 会写入 LVM 标签和元数据，使设备可以加入 VG；它不是只读检测，也不是“看到新盘就先执行”的准备动作。判断一个设备能否成为 PV，必须先结合 `lsblk`、`findmnt`、现有签名和 `pvs` 结果确认没有需要保留的数据或冲突归属。

概念

卷组（Volume Group，VG）把一个或多个 PV 汇聚为逻辑容量池。LV 能否继续增长，取决于其所属 VG 是否还有空闲 extent，而不是系统中是否存在其他磁盘或其他 VG 的剩余空间。`vgs` 中的 `vg\_free` 和 `vg\_free\_count` 分别从容量单位和 extent 数量回答“还能分配多少”。

概念

逻辑卷（Logical Volume，LV）是从 VG 中分配出来、向上层呈现为块设备的对象。它可以承载文件系统、swap 或其他上层用途；“LV 已存在”只证明块设备层成立，不自动证明文件系统、当前挂载或持久挂载。应从 `lvs` 读取真实路径、大小和底层映射，不靠设备名猜测。

概念

物理区域与逻辑区域（PE/LE）是 LVM 的固定分配单位。PE 大小属于 VG；在线性 LV 的基本模型中，一个 LE 对应一个 PE，因此“PE 大小 × LE 数量”得到 LV 的标称容量。题目给出精确 extent 数量时，直接使用 `l` 比先换算成近似 MiB 更能表达评分目标。

概念

容量池与空闲 extent 描述的是 VG 当前还能分配给 LV 的容量。底层磁盘未分区空间、新增但尚未 `pvcreate` 的设备、未加入目标 VG 的 PV，都不是当前 LV 可直接使用的空间。扩容失败时，下一条有区分度的证据通常是目标 VG 的 `vg\_free`、`vg\_extent\_size` 和 `vg\_free\_count`。

概念

块设备容量与文件系统容量 属于两个状态层。`lvs` 读取 LV 的块设备大小，`df` 读取已挂载文件系统向用户呈现的容量；两者因文件系统元数据和显示舍入而略有差异很正常。真正需要诊断的是 LV 明显增长而 `df` 完全不变，这通常说明文件系统增长尚未完成。

概念

挂载点到 LV 的映射 决定业务目录实际上使用哪个存储对象。目录存在或可写都不能证明挂载成立；`findmnt /srv/project` 才能确认当前源设备和文件系统类型。面对扩容任务，应从挂载点反向找到 LV 和 VG；若源是 NFS 或普通分区，就不能强行套用本地 LVM 流程。

概念

绝对容量与增量容量 是 `lvcreate`、`lvextend` 中必须分清的操作语义。`-L 2G` 或 `-l 70` 表示目标总量；`-L +512M` 或 `-l +20` 表示在当前基础上增加。加号不是可省略的装饰字符；漏掉它会“把增加多少”改写成“最终达到多少”。

操作语义速查 进入详细专题前，先把最键入口放进同一张接口地图。速查区只说明命令作用对象、基本形式和最重要参数；正文再解释调查顺序、验证边界和错误分支。

`pvs` / `vgs` / `lvs` **SYNOPSIS**

```
pvs [OPTIONS] [PV ...]
vgs [OPTIONS] [VG ...]
lvs [OPTIONS] [LV ...]
```

分别读取 PV、VG、LV 的 LVM 元数据。它们是调查与验证接口，不改变目标对象；重点不是背默认列，而是用 `o` 只显示影响当前决策的字段。 **重要参数 / 形式** `pvs -o pv\_name,vg\_name,pv\_size,pv\_free,pv\_attr`：判断哪些设备是 PV、属于哪个 VG，以及 PV 层还有多少未分配容量。 `vgs -o vg\_name,vg\_size,vg\_free,vg\_extent\_size,vg\_extent\_count,vg\_free\_count`：判断目标 VG 的总量、PE 大小和空闲 extent；这是创建与扩展 LV 前的核心容量证据。 `lvs -o lv\_name,vg\_name,lv\_size,lv\_path,lv\_attr,segtype,devices`：判断 LV 的名称、路径、大小、属性、类型和底层映射。 `--segments`：进入 segment 视图；精确 extent 题可结合 `seg\_start\_pe`、`seg\_size\_pe` 观察线性段。

```
## `pvcreate` **SYNOPSIS**
```

```
pvcreate [OPTIONS] DEVICE [DEVICE ...]
```

把已确认可安全使用的块设备初始化为 PV。该操作会写入 LVM 元数据，必须位于设备身份、已有签名、挂载和数据要求调查之后。 **重要参数 / 形式** `pvcreate /dev/vdb1`：把指定设备初始化为 PV；设备名必须来自当前系统证据。 `pvs /dev/vdb1 -o pv\_name,vg\_name,pv\_size,pv\_free,pv\_attr`：操作后证明设备已成为 PV，并确认它是否已加入 VG。 `--force` / `--ff`：能绕过部分保护，不是本章默认答案。遇到已有签名或归属时先停下调查，不能用强制参数消除不确定性。

```
## `vgcreate` / `vgextend` **SYNOPSIS**
```

```
vgcreate [OPTIONS] VG PV [PV ...]
vgextend [OPTIONS] VG PV [PV ...]
```

`vgcreate` 用 PV 新建容量池；`vgextend` 把新的 PV 加入已有容量池。两者改变的是 VG 的成员与可分配容量，不直接扩大任何文件系统。 **重要参数 / 形式** `vgcreate -s 16M projectvg /dev/vdb1`：创建 `projectvg`，并把 PE 大小设为 16 MiB；`-s` 是 VG 属性。 `vgextend projectvg /dev/vdc1`：把已初始化为 PV 的设备加入目标 VG；参数顺序是“目标 VG + 新 PV”。 `vgs projectvg -o vg\_name,pv\_count,vg\_size,vg\_free,vg\_extent\_size,vg\_free\_count`：创建或扩展后重新证明成员数量、PE 和空闲容量。

```
## `lvcreate` **SYNOPSIS**
```

```
lvcreate [OPTIONS] -n LV VG
```

从 VG 的空闲 extent 中创建 LV。命令必须直接表达题目给出的容量方式，并在创建后验证真实路径和大小。 **重要参数 / 形式** `-n NAME`：指定 LV 名称，例如 `-n projectlv`。 `-L SIZE`：按容量创建，例如 `-L 800M` 或 `-L 2G`。 `-l EXTENTS`：按 extent 数量创建，例如 `-l 50`。 `-l 100%FREE`：使用 VG 当前全部空闲 extent；只有题目明确要求占满余量时才使用。

```
## `lvextend` **SYNOPSIS**
```

```
lvextend [OPTIONS] LV
```

从目标 VG 的空闲 extent 中扩大已有 LV。它不会自动发现新磁盘，也不会借用其他 VG 的容量；空间不足时应先调查并扩展 VG。 **重要参数 / 形式** `-L 3G`：把 LV 的目标总大小设为 3 GiB。 `-L +512M`：在当前基础上增加 512 MiB。 `-l 70`：把目标总 extent 数设为 70。 `-l +20`：在当前基础上增加 20 个 extent。 `-l +100%FREE`：把 VG 当前全部空闲 extent 追加给已有 LV。 `-r` / `--resizefs`：在 LV 扩展后调用文件系统调整工具。它跨越 LV 和文件系统两个状态层，操作后仍要分别查询。

```
## `xfs_growfs` / `resize2fs` **SYNOPSIS**
```

```
xfs_growfs [OPTIONS] MOUNT_POINT
resize2fs [OPTIONS] DEVICE [SIZE]
```

在底层 LV 已经扩大后，让文件系统使用新增块空间。必须先确认文件系统类型；不能用重新格式化代替增长。 **重要参数 / 形式** `xfs\_growfs /srv/project`：扩大当前挂载在 `/srv/project` 的 XFS。XFS 在 RHEL 9 主线中支持增长，不支持原地缩小。 `resize2fs /dev/datavg/reportslv`：扩大 ext4 文件系统；不指定大小时通常增长到当前块设备可用上限。 `lvextend -r ...`：可把 LV 与受支持文件系统的增长串联，但任何失败后都要重新查询实际中间状态。

```
## `findmnt` / `lsblk` / `df` **SYNOPSIS**
```

```
findmnt [OPTIONS] [DEVICE|MOUNT_POINT]
lsblk [OPTIONS] [DEVICE ...]
df [OPTIONS] [FILE ...]
```

分别从挂载表、块设备树和已挂载文件系统视角观察状态。三者共同完成“目录由谁提供、设备层级是什么、用户可见容量是多少”的验证。 **重要参数 / 形式** `findmnt /srv/project`：证明当前挂载源、目标和文件系统类型；不证明 `fstab` 持久性。 `lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS`：观察设备父子关系、块设备大小、文件系统和挂载点；不能替代 `pvs` 的 LVM 归属证据。 `df -hT /srv/project`：读取目标目录所在文件系统的类型、总量和使用率；不证明 VG 空闲 extent。

[知识专题] 从设备到业务目录：容量链必须逐层成立

LVM 题目的对象不是抽象的“磁盘空间”，而是一条有明确身份和状态边界的容量链。正向创建时，容量从底层设备进入 LVM，再交给文件系统和挂载点；扩容时，应从业务目录反向找到对应 LV 和 VG。无论沿哪个方向工作，都不能让一层的成功代替下一层。

① [知识点] 设备存在不等于设备可写

`/dev/vdb1` 只是当前内核提供的块设备节点。它可能已包含文件系统、属于某个 VG、作为 swap 使用，或正在被挂载。写操作前至少要回答：

设备是否与题目指定对象一致？
是否已有 `FSTYPE`、挂载点或 `holder`？
是否已经出现在 `pvs` 中？
题目是否明确允许覆盖其内容？

一个稳定的调查入口是：

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS /dev/vdb
findmnt --source /dev/vdb1
pvs /dev/vdb1 -o pv_name,vg_name,pv_size,pv_free,pv_attr
```

设备信息存在冲突时应停止，不使用 `wipefs -a`、`pvcreate --force` 或删除现有 VG 作为默认“清理”步骤。分区创建和签名识别的完整机制留给第 22 章。

② [知识点] PV、VG、LV 分别承担身份、汇聚和分配

容量关系可以压缩为：

```
块设备 /dev/vdb1
└─ pvcreate
PV: 可被 LVM 使用的容量身份
└─ vgcreate / vgextend
VG: 由一个或多个 PV 汇成的容量池
└─ lvcreate / lvextend
LV: 从 VG 获得容量的逻辑块设备
```

PV 不会自动进入任意 VG；新磁盘也不会自动成为 LV 的可用空间。若目标 VG 余量不足，新设备必须先成为 PV，再通过 `vgextend` 加入目标 VG，最后才能由 `lvextend` 消耗新增空闲 extent。

③ [知识点] 文件系统和挂载点属于 LVM 上层

LV 可以没有文件系统，也可以承载非文件系统用途。经典任务在新 LV 上创建 XFS 或 ext4，只是某一种上层用途。文件系统创建、标签、UUID 与增长机制的主归属是第 23 章；当前挂载与 `fstab` 持久性的主归属是第 24 章。

本章仍必须保留两个接口判断：

- `lvs` 变大只证明 LV 块设备增长；
- `findmnt` 与 `df` 才能证明业务目录仍指向目标文件系统，并看到新增容量。

④ [知识点] 从挂载点反向定位比从设备名正向猜更安全

题目若写“扩展 `/srv/reports`”，评分对象是业务目录，不是某个预设设备名。反向链如下：

```
findmnt /srv/reports
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS
lvs -o lv_name,vg_name,lv_size,lv_path,devices
vgs -o vg_name,vg_free,vg_extent_size,vg_free_count
pvs -o pv_name,vg_name,pv_size,pv_free
```

每条命令缩小一次对象集合。若 `findmnt` 显示 NFS，应转入第 26 章；若源是普通分区，也不能假设其背后存在 VG。

⑤ [验证] 六层验收回答六个不同问题

设备层`lsblk`：目标设备是否存在，大小、类型和当前上层关系是什么。

PV 层`pvs`：设备是否获得 LVM 身份，属于哪个 VG。

VG 层`vgs`：容量池总量、PE 和空闲 extent 是否符合预期。

LV 层`lvs`：LV 路径、大小、类型和底层映射是否正确。

文件系统层`df -hT`：文件系统是否识别新增空间。

挂载与数据层`findmnt`、文件读取或校验和：业务路径和原数据是否仍正确。

[Cheatsheet] 创建从设备向上走：扩容从挂载点向下查。设备、PV、VG、LV、文件系统和挂载点是六个对象层，任何查询只证明其中一部分。

[知识专题] PE、LE 与容量表达：先判断题目在说什么

LVM 同时支持容量单位、extent 数量和百分比表达。真正的难点不是记住四个参数，而是确定参照对象与时间语义：题目要求最终总量，还是在当前状态上增加；要求精确 extent，还是给出 MiB/GiB 容量；百分比相对哪个容量池。

① [知识点] PE 大小属于 VG，LE 是 LV 的分配视图

创建 VG 时确定 PE 大小。在线性 LV 的基础模型中，一个 LE 对应一个 PE，因此：

```
LV 标称容量 = LE 数量 × PE 大小
```

例如 PE 为 16 MiB：

```
50 LE × 16 MiB = 800 MiB
70 LE × 16 MiB = 1120 MiB
```

计算的价值是核对参数语义和目标终态，不是要求所有 extent 题都换算成容量后再使用 `-L`。

② [参数] 大写 `-L` 按容量，小写 `-l` 按 extent 或百分比

```
lvcreate -L 800M -n projectlv projectvg
lvcreate -l 50 -n projectlv projectvg
```

两条命令在 PE 为 16 MiB 时可能得到相同标称容量，但表达的评分对象不同。题目明确要求“50 个 extent”时使用 `-l 50`；题目只给“800 MiB”时使用 `-L 800M`。

③ [参数] 加号把目标总量改成相对增量

```
-L 3G      最终目标总量为 3 GiB
-L +512M   在当前基础上增加 512 MiB
-l 70      最终目标总 extent 数为 70
-l +20     在当前基础上增加 20 个 extent
```

若当前 LV 已为 2 GiB，`lvextend -L 512M` 不是“增加 512 MiB”，而是给出更小的目标总量；`lvextend` 不执行缩小，命令会失败或明显不符合题意。此时正确形式是 `-L +512M`。

④ [参数] `%FREE` 的参照对象是 VG 当前空闲 extent

新建 LV：

```
lvcreate -l 100%FREE -n data1v datavg
```

扩展已有 LV：

```
lvextend -l +100%FREE datavg/data1v
```

第一条把当前全部空闲 extent 分配给新 LV；第二条把全部空闲 extent 追加给已有 LV。它们都会耗尽目标 VG 的余量，因此只有题目明确要求占满时才使用。生产环境通常需要为其他 LV、未来扩展或恢复策略保留空间。

⑤ [查询] 精确 extent 题不要用人类似可读容量反推

```
vgs projectvg \
-o vg_name,vg_extent_size,vg_extent_count,vg_free_count

lvs --segments projectvg/projectlv \
-o lv_name,seg_start_pe,seg_size_pe,segtype,devices
```

`df -h` 的容量经过文件系统元数据和单位舍入，不能用来证明精确 LE 数量。普通单段线性 LV 可从 segment 视图核对 extent；复杂多段 LV 需要综合所有段，而不是只看第一行。

[Cheatsheet] `-L` 看容量，`-l` 看 extent；无加号是目标总量，有加号是相对增量；`%FREE` 以 VG 当前空闲 extent 为参照；精确 extent 用 `vgs` 与 segment 报告验收。

[操作专题] 用定制报告建立现状图，而不是无目的地跑命令

调查的目标是决定下一步。先从业务路径和设备树确认对象，再分别读取 PV、VG、LV 元数据。`-o` 不是美化输出的选项，而是把证据压缩到本次判断所需字段，降低看错对象和忽略余量的风险。

① [查询] `findmnt` 与 `lsblk` 先回答“谁在使用谁”

```
findmnt /srv/project
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS
```

`findmnt` 以挂载关系为中心，适合从目录找到源；`lsblk` 以块设备树为中心，适合观察磁盘、分区、LVM 映射和文件系统。它们能提示设备占用，但不完整描述 LVM 元数据，因此下一步仍要使用 `pvs`、`vgs`、`lvs`。

② [查询] `pvs` 证明 PV 身份与 VG 归属

```
pvs -o pv_name,vg_name,pv_size,pv_free,pv_attr
```

``pv_name`` 哪个设备已经是 PV。

``vg_name`` 该 PV 属于哪个 VG；空白通常表示尚未加入 VG。

``pv_size`` 该 PV 参与 LVM 管理的总容量。

``pv_free`` 从 PV 视角尚未分配给 LV 的容量。

``pv_attr`` PV 属性摘要，用于发现状态差异。

`pv_free` 不是扩展任意 LV 的直接许可；还必须确认该 PV 所属 VG 与目标 LV 的 VG 相同。

③ [查询] `vgs` 证明容量池是否有足够余量

```
vgs projectvg \
-o vg_name,pv_count,lv_count,vg_size,vg_free,vg_extent_size,vg_extent_count,vg_free_count
```

``vg_free`` 适合判断“+512M”、“+2G”等容量增量能否满足。

``vg_extent_size`` 每个 PE 的大小，是 extent 计算基础。

``vg_free_count`` 适合判断“增加 20 个 extent”能否满足。

``pv_count`` / ``lv_count`` 帮助确认容量池成员与对象数量是否发生预期变化。

④ [查询] `lvs` 证明 LV 的身份、路径、大小和映射

```
lvs projectvg/projectlv \
-o lv_name,vg_name,lv_size,lv_path,lv_attr,segtype,devices
```

`lv_path` 给出上层工具应使用的真实设备路径；`devices` 展示底层映射；`lv_size` 只证明 LV 层大小。扩展后必须继续查看文件系统和挂载层。

⑤ [操作方法] 把查询串成决策树

```
目标是业务目录
→ findmnt: 源是什么、FSTYPE 是什么
→ lvs: 它是否为 LV、属于哪个 VG
→ vgs: 目标 VG 余量是否满足
→ pvs: VG 的容量来自哪些设备
→ 只有缺失证据得到回答后，才执行写操作
```

[Cheatsheet] `findmnt` 从目录找源；`lsblk` 看设备树；`pvs` 看设备身份与归属；`vgs` 看容量池和空闲 extent；`lvs` 看 LV 路径与大小。查询顺序应缩小对象，而不是堆积输出。

[操作专题] 从确认空闲的设备创建 PV、VG 和 LV

创建链必须把每一次写入放在证据门之后。下面使用 `/dev/vdb1`、`projectvg`、`projectlv` 作为明确教学场景；真实环境必须以题目和当前查询结果替换。命令成功不是终点，每一层都要立即用对应报告验证。

① [操作] 写入前建设设备安全门

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS /dev/vdb
findmnt --source /dev/vdb1
pvs /dev/vdb1 -o pv_name,vg_name,pv_size,pv_free,pv_attr
```


停止条件包括：设备名与题意不一致、已有需要保留的文件系统、当前正在挂载、已属于现有 VG、或题目没有授权覆盖。看到这些证据时，不继续执行 `pvccreate`，更不能默认清除签名。

② [操作] `pvccreate` 只改变 PV 层

```
pvccreate /dev/vdb1
pvs /dev/vdb1 -o pv_name,vg_name,pv_size,pv_free,pv_attr
```

第二条应证明 `/dev/vdb1` 已是 PV。此时 `vg_name` 仍可能为空，因为 PV 尚未加入 VG；这不是失败，而是对象链的中间状态。

③ [操作] `vgcreate -s` 建立容量池并确定 PE

```
vgcreate -s 16M projectvg /dev/vdb1
vgs projectvg \
-o vg_name,pv_count,vg_size,vg_free,vg_extent_size,vg_extent_count,vg_free_count
pvs /dev/vdb1 -o pv_name,vg_name,pv_size,pv_free
```

`-s 16M` 把 PE 大小设为 16 MiB。验证必须同时看到：VG 名称正确、PV 已属于 `projectvg`、`vg_extent_size` 符合要求、VG 有可分配余量。

④ [操作] `lvcreate` 按题意分配容量

题目要求 50 个 extent：

```
lvcreate -l 50 -n projectlv projectvg
```

题目要求 800 MiB：

```
lvcreate -L 800M -n projectlv projectvg
```

创建后：

```
lvs projectvg/projectlv \
-o lv_name,vg_name,lv_size,lv_path,lv_attr,segtype,devices
lvs --segments projectvg/projectlv \
-o lv_name,seg_start_pe,seg_size_pe,segtype,devices
```

若评分目标是 extent，应重点核对 segment 的 extent 数和 VG 的 PE 大小，而不是只接受舍入后的 `lv_size`。

⑤ [边界] 文件系统与持久挂载只保留接口骨架

新 LV 仍是块设备。题目若要求 XFS 和持久挂载，需要调用第 23、24 章已经建立的能力：

```
mkfs.xfs /dev/projectvg/projectlv
mkdir -p /srv/project
blkid /dev/projectvg/projectlv
# 使用当前机器读取的真实 UUID 建立持久挂载记录
findmnt --verify
mount -a
findmnt /srv/project
```

本章不重新解释 `mkfs.xfs` 参数、UUID 生命周期或 `fstab` 六字段。这里强调的是：只有文件系统、当前挂载和持久配置均完成，业务目录才成立；不得把 LV 创建成功当作完整任务完成。

[Cheatsheet] 调查设备 → `pvccreate` → `pvs` → `vgcreate -s` → `vgs` → `lvcreate -L/-l -n` → `lvs`。每个写操作后立刻证明对应对象层。

[操作专题] VG 空间不足：先把新容量加入正确容量池

`lvextend` 只能消耗目标 VG 的空闲 PE。它不会搜索其他磁盘，不会自动初始化新设备，也不会借用其他 VG 的余量。空间不足时，稳定分支是“证明不足 → 调查候选设备 → 新建 PV → `vgextend` → 重新验证 VG → 再扩展 LV”。

① [诊断] 先证明不足发生在 VG 层

```
vgs projectvg \
-o vg_name,vg_size,vg_free,vg_extent_size,vg_free_count
```

若任务要求增加 20 个 extent，而 `vg_free_count` 小于 20，当前 VG 的确无法满足。若 `vg_free_count` 足够，则空间不足错误可能来自选错 LV/VG、参数语义或其他限制，不应直接加入新设备。

② [操作] 对候选设备重新执行安全调查

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS /dev/vdc
findmnt --source /dev/vdc1
pvs /dev/vdc1 -o pv_name,vg_name,pv_size,pv_free,pv_attr
```

新设备名不等于新设备状态。若 `/dev/vdc1` 已属于其他 VG，应停止并确认题意；不能为了当前任务强行夺取它。

③ [操作] 让设备先成为 PV，再加入目标 VG

```
pvccreate /dev/vdc1
vgextend projectvg /dev/vdc1
```

`vgextend` 参数顺序是目标 VG 在前、新 PV 在后。新磁盘不会直接作为 `lvextend` 参数，也不会因为 `pvccreate` 成功就自动属于 `projectvg`。

④ [验证] 重新证明成员关系和空闲 extent

```
pvs /dev/vdc1 -o pv_name,vg_name,pv_size,pv_free
vgs projectvg \
-o vg_name,pv_count,vg_size,vg_free,vg_extent_size,vg_free_count
```

只有当 `/dev/vdc1` 已属于 `projectvg` 且 `vg_free_count` 或 `vg_free` 满足目标增量，才继续执行 `lvextend`。重复运行同一条空间不足命令不会自动修复容量池。

[Cheatsheet] `lvextend` 空间不足时先查 `vgs`；新设备先调查，再 `pvccreate` 和 `vgextend`；重新验证 VG 余量后才扩展 LV。

[操作专题] 保留数据扩展 LV 与文件系统

安全扩容不是把命令缩成一行，而是把现有对象、容量来源、中间状态和原数据都纳入证据。扩容前先从挂载点确认文件系统类型、LV 和 VG；扩容后分别证明 LV、文件系统、挂载和数据。若使用 `-r`，仍不能省略分层验收。

① [操作] 建立扩容前基线

```
findmnt /srv/project
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS
lvs projectvg/projectlv -o lv_name,vg_name,lv_size,lv_path,devices
vgs projectvg -o vg_name,vg_free,vg_extent_size,vg_free_count
df -hT /srv/project
sha256sum /srv/project/marker.txt
```

基线至少回答：挂载点的源和 FSTYPE、当前 LV 大小、VG 余量、文件系统可见容量、需要保留的数据证据。若文件不存在或业务需要一致性快照，应使用符合场景的备份与一致性方法；单个校验和只证明被检查对象。

② [参数] 明确总量还是增量，再执行 `lvextend`

增加 20 个 extent：

```
lvextend -l +20 projectvg/projectlv
```

增加 512 MiB：

```
lvextend -L +512M datavg/reportslv
```

扩展到总大小 3 GiB：

```
lvextend -L 3G datavg/reportslv
```

执行前再次确认 VG 余量；执行后立即运行 `lvs`，记录 LV 的实际中间状态。不要在不清楚结果时重复追加相同增量，否则可能得到双倍扩容。

③ [操作] `-r` 联动两个状态层，但不是原子承诺

```
lvextend -l +20 -r projectvg/projectlv
```

`-r / --resizefs` 在扩大 LV 后调用文件系统调整工具。它能减少手工步骤，但不意味着任意文件系统都支持、也不保证失败时自动恢复原大小。命令报错后必须重新查询：

```
lvs projectvg/projectlv -o lv_name,lv_size,lv_path
findmnt /srv/project
df -hT /srv/project
```

可能出现的部分终态是“LV 已增长、文件系统未增长”。此时最小修复是确认 FSTYPE 和正确增长入口，而不是再次执行相同的 `lvextend +增量`。

④ [操作] 手工增长已挂载 XFS

底层 LV 已扩大且 `findmnt` 证明 `/srv/project` 是目标 XFS：

```
xfs_growfs /srv/project
```

增长入口使用已挂载文件系统的挂载点。操作后：

```
df -hT /srv/project
findmnt /srv/project
```

XFS 在 RHEL 9 主线中不可原地缩小。若目标是减少容量，应重新设计迁移方案，不能先 `lvreduce` 再尝试修复文件系统。

⑤ [操作] 手工增长 ext4

底层 LV 已扩大且确认文件系统为 ext4：

```
resize2fs /dev/datavg/reportslv
```

不指定大小时，`resize2fs` 通常增长到当前块设备可用上限。操作后比较：

```
lvs datavg/reportslv -o lv_name,lv_size,lv_path
df -hT /srv/reports
findmnt /srv/reports
```

ext4 支持缩小不等于本章提供通用缩容答案；缩小需要离线条件、文件系统先行和更严格的容量计算，主归属在第 23 章。

⑥ [验证] 扩容后的证据必须跨越六层

VG 余量`vgs`：新增容量是否来自正确 VG，剩余 extent 是否合理。

LV 大小`lvs`：LV 是否按目标总量或增量增长。

文件系统容量`df -hT`：上层是否真正使用新增块空间。

当前挂载`findmnt`：业务目录是否仍由原目标 LV 提供。

设备映射`lsblk`：设备树、FSTYPE 和挂载关系是否一致。

已有数据读取文件或比较校验和：扩容是否保留指定数据证据。

[Cheatsheet] 基线 → 确认 VG 余量 → `lvextend` 表达正确增量 → `-r` 或匹配的文件系统增长工具 → `lvs`、`df`、`findmnt`、数据证据分层验收。

[诊断专题] 容量没有按预期出现：让下一条证据定位断裂层

诊断不从“再试一个命令”开始，而从症状所在层开始。每个路径都按“症状 → 当前证据 → 假设 → 下一条有区分度的证据 → 最小修复 → 再验证”推进。

① [诊断] `lvs` 已增大，`df` 完全没变化

当前证据：LV 层增长，文件系统层未体现。优先假设：文件系统增长未执行、执行失败，或目标挂载点/设备选错。下一条证据：

```
findmnt /srv/project
lsblk -f
lvs projectvg/projectlv -o lv_name,lv_size,lv_path
df -hT /srv/project
```

确认 FSTYPE 后，对正确对象执行 `xfs_growfs <MOUNT_POINT>` 或 `resize2fs <LV_PATH>`。不要再次追加相同 LV 增量。

② [诊断] `lvextend` 报可用空间不足

当前证据：扩展请求无法从目标 VG 分配。下一条证据：

```
vgs <VG> -o vg_name,vg_free,vg_extent_size,vg_free_count
```

若余量不足，调查候选设备并执行 `pvccreate` → `vgextend` → 重新验证；若余量足够，检查是否选错 VG/LV、增量是否超过预期或对象状态是否异常。

③ [诊断] 目录存在且可写，但 `findmnt` 没有结果

当前证据：目录只是当前目录树中的普通目录，目标文件系统没有挂载。风险：继续写入会占用承载该目录的上层文件系统，常见是根文件系统。下一条证据：

```
findmnt -T /srv/project  
df -hT /srv/project
```

进入第 24 章检查当前挂载和持久配置。不要用“目录里已经有文件”证明挂载成功。

④ [诊断] `pvccreate` 发现已有文件系统签名或 PV

当前证据：设备不是无状态空白对象。下一条证据：

```
lsblk -f /dev/vdb1  
findmnt --source /dev/vdb1  
pvs /dev/vdb1 -o pv_name,vg_name,pv_size,pv_free,pv_attr
```

重新核对题目与设备身份。除非题目明确要求清除且数据无需保留，否则不使用强制选项继续。

⑤ [诊断] `vgextend` 提示设备属于其他 VG

设备已有容量归属。下一条证据是：

```
pvs /dev/vdc1 -o pv_name,vg_name,pv_size,pv_free  
vgs -o vg_name,pv_count,lv_count,vg_size,vg_free
```

确认该 VG 是否承载其他 LV 和数据。默认停止，不执行 `vgreduce`、`pvremove` 或强制迁移来“抢”容量。

⑥ [诊断] `lvs` 与 `df` 只差少量容量

少量差异通常来自文件系统元数据、保留空间、单位换算和舍入。判断是否异常，应比较扩容前后趋势，并确认目标是否达到允许范围；精确 extent 验收使用 `vgs` 和 segment 字段，不使用 `df -h` 反推。

⑦ [诊断] `lvextend -r` 返回错误，终态不清楚

不要假设整个操作回滚。先查询：

```
lvs <VG>/<LV> -o lv_name,lv_size,lv_path  
findmnt <MOUNT_POINT>  
df -hT <MOUNT_POINT>
```

若 LV 已增长而文件系统未增长，按 FSTYPE 执行最小文件系统修复；若 LV 没增长，则回到 VG 余量和命令目标。任何情况下都不应盲目再次执行同一增量。

[Cheatsheet] 先判定断裂层：VG、LV、文件系统、挂载或数据。下一条证据必须能区分假设：最小修复只改变出错层，随后重新完成跨层验收。

[诊断专题] 删除、缩容和强制覆盖：不能把高风险动作压成快捷表

扩容通常只需要增加可用容量，而缩容和删除会改变数据可达边界。它们不是对称操作。面对空间规划错误、选错设备或已有签名时，最重要的能力是知道何时停止，而不是能背出最多的破坏性命令。

危险边界：先缩小 LV 会直接截断块设备尾部。

若文件系统仍在被截断区域，数据和元数据可能立即损坏。任何缩容都必须由文件系统能力和实际占用先决定，不能从 `lvreduce` 开始。

① [边界] XFS 不可原地缩小

RHEL 9 主线中的 XFS 只支持增长。题目若要求减少 XFS 所在存储，应将需求识别为迁移设计：建立新的较小文件系统、复制并验证数据、切换挂载，再在确认退役后处理旧对象。本章不把这条高风险迁移简化为考试命令串。

② [边界] ext4 可缩小，不等于可以机械照抄

ext4 的缩小通常需要卸载、文件系统检查、先缩文件系统再缩底层 LV，并准确保留安全余量。完整流程属于第 23 章的文件系统容量管理；本章只保留原则：文件系统必须先安全缩小，LV 才能随后缩小。

③ [边界] 删除命令只服务于明确退役

lvremove、vgremove、pvremove 会逐层解除对象和元数据。只有在以下条件都明确时才讨论：目标对象身份无歧义、没有挂载或活动依赖、数据已经按要求保留、题目明确要求退役。删除不是修复创建错误的默认方法。

④ [边界] 已有签名不是要求使用 --force

工具检测到已有文件系统、PV 或 VG 归属，是阻止数据破坏的证据。稳定处理是查明来源和题意，必要时选择正确设备；只有明确的清除任务才进入签名移除流程，而该流程主归属第 22 章。

⑤ [边界] 校验和是局部证据，不是完整备份

经典任务通过 marker.txt 或 sha256sum 证明指定文件在扩容前后保持一致。它能排除“重新格式化后重新写同名空文件”等错误路径，但不能证明整个数据库或应用的一致性。生产变更仍需要业务级备份、停写、快照或一致性控制。

[Cheatsheet] XFS 不缩；ext4 缩容先文件系统、后 LV，且不在本章给出机械答案；删除和 force 只在对象、数据和退役要求全部明确时使用。

[知识专题] 从考试容量题迁移到生产变更

考试环境用具体设备名和容量训练对象模型；生产环境还要面对业务一致性、容量增长速度、共享 VG、回退与变更窗口。迁移的重点不是增加更多命令，而是把调查、停止条件和验收写成可复核流程。

① [工作迁移] 保存扩容前基线和变更对象

建议至少记录：挂载源、FSTYPE、LV 路径和大小、VG 总量与余量、PV 成员、业务数据验证方法。基线让“操作后变了什么”可比较，也能防止多次执行相同增量。

② [工作迁移] 不默认吃满 VG

100%FREE 在考试中可能是明确要求，在生产中却会消耗快照、其他 LV 和未来增长的空间。容量设计应说明保留量、增长阈值和下一次扩容来源，而不是只追求当前最大化。

③ [工作迁移] 把“LV 增长”和“业务可用”写成两个验收项

变更单中至少分别列出：

LVM 终态：目标 LV 大小、VG 剩余容量、PV 成员正确
业务终态：文件系统容量增长、挂载源不变、应用数据可读、必要功能正常

这能避免底层成功被误写成业务完成。

④ [工作迁移] 明确回退边界

扩容本身通常不可通过简单缩容无风险撤销；新 PV 加入 VG 后也可能开始承载数据。生产变更的回退应更多依赖备份、快照、应用切换和预先设计，而不是在失败后临时执行 lvreduce 或删除对象。

[Cheatsheet] 生产扩容：保存基线、说明容量来源、保留 VG 余量、把 LVM 与业务终态分开验收、提前声明不可逆边界。

[经典任务] 创建指定 extent 的项目卷并保留数据扩展

环境与当前状态

- `/dev/vdb1` 和 `/dev/vdc1` 是题目提供的候选分区；题目说明其内容无需保留，但仍须先查询确认身份与占用。
- `projectvg`、`projectlv` 和 `/srv/project` 尚不存在。
- 文件系统与持久挂载能力已经在第 23、24 章学习。

目标终态

1. 使用 `/dev/vdb1` 创建 PV。
2. 创建 `projectvg`，PE 大小为 16 MiB。
3. 创建 `projectlv`，初始大小为 50 个 extent。
4. 在 LV 上创建 XFS，并通过真实 UUID 持久挂载到 `/srv/project`。
5. 写入 `/srv/project/marker.txt`，内容为 `keep-this-data`。
6. 在保留该文件的前提下，再为 LV 增加 20 个 extent，并让 XFS 使用新增空间。
7. 若 `projectvg` 的空闲 extent 不足，先把 `/dev/vdc1` 安全加入容量池。

限制条件

- 不重新格式化已有 XFS。
- 不删除并重建 LV。
- 不缩小其他 LV。
- 不使用 `--force`、`-ff`、无调查的签名清除或删除命令。
- 不把目录存在、命令返回成功或 `lvs` 单层增长当作完整验收。

验收矩阵

层次	必须证明的终态	推荐证据
设备/PV	候选设备身份正确，PV 归属正确	<code>lsblk</code> 、 <code>pvs</code>
VG	PE 为 16 MiB，容量来源正确	<code>vgs</code>
LV	初始 50 extent，最终 70 extent	<code>lvs --segments</code> 、 <code>lvs</code>
文件系统	XFS 使用新增块空间	<code>df -hT</code>
挂载	<code>/srv/project</code> 当前源为目标 LV	<code>findmnt</code>
持久性	持久挂载条目使用真实 UUID 且可验证	第 24 章验证链
数据	<code>marker.txt</code> 内容或校验和保持一致	<code>cat</code> 、 <code>sha256sum</code>

请先独立完成。参考解答从新页开始。

[参考解答] 从设备安全门到分层验收

① 调查两个候选设备

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS /dev/vdb /dev/vdc
findmnt --source /dev/vdb1
findmnt --source /dev/vdc1
pvs -o pv_name,vg_name,pv_size,pv_free,pv_attr
```

只在证据与题意一致、设备没有需保留数据或冲突归属时继续。题目说明“内容无需保留”不能替代设备名核对。

② 创建 PV 和指定 PE 的 VG

```
pvccreate /dev/vdb1
pvs /dev/vdb1 -o pv_name,vg_name,pv_size,pv_free,pv_attr

vgcreate -s 16M projectvg /dev/vdb1
pvs /dev/vdb1 -o pv_name,vg_name,pv_size,pv_free
vgs projectvg \
-o vg_name,pv_count,vg_size,vg_free,vg_extent_size,vg_extent_count,vg_free_count
```

参数选择：-s 16M 直接表达 PE 目标；验证应看到 /dev/vdb1 已属于 projectvg，vg_extent_size 为 16 MiB。

③ 按 extent 创建 LV

```
lvcreate -l 50 -n projectlv projectvg
lvs projectvg/projectlv \
-o lv_name,vg_name,lv_size,lv_path,lv_attr,segtype,devices
lvs --segments projectvg/projectlv \
-o lv_name,seg_start_pe,seg_size_pe,segtype,devices
```

题目按 extent 评分，因此使用 -l 50，不先换算成 -L 800M。在线性单段场景中，segment 报告应体现 50 个 extent。

④ 调用文件系统与持久挂载能力

```
mkfs.xfs /dev/projectvg/projectlv
mkdir -p /srv/project
blkid /dev/projectvg/projectlv
```

使用 blkid 读取的真实 UUID 按第 24 章方法建立持久挂载记录，然后在重启前验证：

```
findmnt --verify
mount -a
findmnt /srv/project
df -hT /srv/project
```

这里不复述 fstab 六字段：关键边界是不能抄写示例 UUID，也不能仅凭目录存在宣布完成。

⑤ 建立数据保留证据

```
printf '%s\n' 'keep-this-data' > /srv/project/marker.txt
cat /srv/project/marker.txt
sha256sum /srv/project/marker.txt
```

记录扩容前校验值。该证据只覆盖指定文件，不替代完整备份。

⑥ 判断 VG 是否有 20 个空闲 extent

```
vgs projectvg -o vg_name,vg_free,vg_extent_size,vg_free_count
```

若 vg_free_count 至少为 20，直接进入扩展。若不足，先重新调查 /dev/vdc1，再执行：

```
pvccreate /dev/vdc1
vgextend projectvg /dev/vdc1
pvs /dev/vdc1 -o pv_name,vg_name,pv_size,pv_free
vgs projectvg \
-o vg_name,pv_count,vg_size,vg_free,vg_extent_size,vg_free_count
```

只有新 PV 已属于 `projectvg` 且余量满足，才继续。

⑦ 增加 20 个 extent 并扩大 XFS

推荐联动形式：

```
lvextend -l +20 -r projectvg/projectlv
```

参数选择：小写 `-l` 按 extent；`+20` 表示增加量；`-r` 继续调整受支持文件系统。若命令报错，不重复执行，先查询 LV 和文件系统实际状态。

手工分离形式也成立：

```
lvextend -l +20 projectvg/projectlv
xfs_growfs /srv/project
```

⑧ 最终分层验证

```
pvs -o pv_name,vg_name,pv_size,pv_free
vgs projectvg \
-o vg_name,pv_count,vg_size,vg_free,vg_extent_size,vg_extent_count,vg_free_count
lvs projectvg/projectlv \
-o lv_name,vg_name,lv_size,lv_path,lv_attr,segtype,devices
lvs --segments projectvg/projectlv \
-o lv_name,seg_start_pe,seg_size_pe,segtype,devices
findmnt /srv/project
df -hT /srv/project
cat /srv/project/marker.txt
sha256sum /srv/project/marker.txt
```

最终判断：LV 应由 50 extent 增加到 70 extent；PE 为 16 MiB 时标称容量为 1120 MiB。`df` 可能因文件系统元数据和舍入略小，但应相对扩容前明显增长；挂载源与校验值应保持一致。

典型错误分支

- `lvextend` 报空间不足：先查 `vgs`，不要反复执行。
- `lvs` 已增长而 `df` 未变：确认 FSTYPE，执行 `xfs_growfs`，不要再次追加 20 extent。
- `findmnt` 无结果：进入第 24 章修复挂载，不把目录中的文件当作目标文件系统数据。
- `pvccreate` 检测到已有签名：停止并重新确认设备，禁止默认 `force`。

[经典任务] 保留 ext4 数据扩展现有报告卷

环境与当前状态

- `/srv/reports` 当前已挂载，文件系统类型为 ext4。
- 挂载源应通过查询确认，预期为 `/dev/datavg/reportslv`。
- LV 当前约为 2 GiB。
- `/srv/reports/report.db` 已存在，内容必须保持不变。
- `datavg` 是否有足够余量未知：`/dev/vdd1` 是题目提供的候选扩容设备。

目标终态

- 把 LV 增加 512 MiB，而不是设置为总大小 512 MiB。
- 本任务不使用 `-r`，要求分别扩展 LV 和 ext4。
- 若 VG 余量不足，安全地把 `/dev/vdd1` 加入 `datavg`。
- 最终挂载源仍为原 LV，文件系统容量增加，`report.db` 校验和不变。

限制条件

- 不重新格式化 ext4。
- 不删除或重建 LV。
- 不使用 `force` 或无调查的签名清除。
- 不以 `lvs` 增长代替文件系统和数据验收。

验收矩阵

层次	目标	证据
映射	<code>/srv/reports</code> 指向预期 ext4 LV	<code>findmnt</code> 、 <code>lsblk -f</code>
VG	有至少 512 MiB 可分配余量	<code>vgs</code>
LV	当前基础上增加 512 MiB	<code>lvs</code>
文件系统	ext4 使用新增空间	<code>df -hT</code>
数据	<code>report.db</code> 校验和不变	<code>sha256sum</code>

请先独立完成。参考解答从新页开始。

[参考解答] 分开观察 LV 与 ext4 的增长

① 建立映射、容量和数据基线

```
findmnt /srv/reports
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS
lvs datavg/reportslv -o lv_name,vg_name,lv_size,lv_path,devices
vgs datavg -o vg_name,vg_free,vg_extent_size,vg_free_count
df -hT /srv/reports
sha256sum /srv/reports/report.db
```

先确认实际挂载源和 ext4 类型。若源不是目标 LV，停止并调查；不能按题面预期盲目操作设备。

② 确认或补充 VG 余量

若 `vg_free` 足够 512 MiB，直接继续。若不足，调查候选设备：

```
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS /dev/vdd
findmnt --source /dev/vdd1
pvs /dev/vdd1 -o pv_name,vg_name,pv_size,pv_free,pv_attr
```

证据允许后：

```
pvcreeate /dev/vdd1
vgextend datavg /dev/vdd1
pvs /dev/vdd1 -o pv_name,vg_name,pv_size,pv_free
vgs datavg -o vg_name,pv_count,vg_size,vg_free,vg_extent_size,vg_free_count
```

③ 只扩展 LV，观察预期中间状态

```
lvextend -L +512M datavg/reportslv
lvs datavg/reportslv -o lv_name,vg_name,lv_size,lv_path
findmnt /srv/reports
df -hT /srv/reports
```

参数选择：`+512M` 表示在当前基础上增加。此时 `lvs` 应增长，而 `df` 可能尚未变化；这是本任务刻意保留的中间状态，用于证明 LV 和文件系统是两个对象层。

④ 扩展 ext4

```
resize2fs /dev/datavg/reportslv
```

设备路径应来自 `lvs -o lv_path` 或当前映射证据，而不是未经确认的手工拼接。

⑤ 最终验证

```
lvs datavg/reportslv -o lv_name,vg_name,lv_size,lv_path
df -hT /srv/reports
findmnt /srv/reports
sha256sum /srv/reports/report.db
```

LV 应比基线增加 512 MiB，ext4 可见容量应增长，挂载源不变，校验和一致。

典型错误分支

- 写成 `lvextend -L 512M`：这是目标总量语义，不是增量；应停止并改为 `+512M`。
- `lvs` 增长、`df` 不变：执行正确的 `resize2fs`，不重复扩展 LV。
- `resize2fs` 报设备或类型错误：重新核对 `findmnt`、`lsblk -f` 和 `lv_path`。
- 校验和变化：不能把容量增长视为完成，应保留现场并调查数据或挂载源变化。

[本章收束] 容量链也是证据链

LVM 的核心不是背三个缩写，而是知道容量在每一层如何获得身份、被汇聚、被分配并最终交付给业务目录。创建题从确认设备开始，逐层建立 PV、VG、LV；扩容题从挂载点反向确认文件系统、LV 和 VG，先证明容量来源，再改变对象；诊断题则比较各层证据，找出哪一层没有成立或没有同步。

最稳定的工作路径是：

- 目标目录由谁提供
- 对应哪个文件系统和 LV
- LV 属于哪个 VG
- VG 是否有足够空闲 extent
- 参数表达的是总量还是增量
- LV 与文件系统是否都增长
- 挂载和原数据是否仍正确

主要判断表

现象或目标	先看什么	下一步原则
创建指定 extent 的 LV	<code>vg_extent_size</code> 、 <code>vg_free_count</code>	用 <code>-l</code> 直接表达 extent
增加 512 MiB	当前 LV、 <code>vg_free</code>	使用 <code>-L +512M</code>
<code>lvextend</code> 空间不足	<code>vgs</code> 余量	新设备先 <code>pvccreate</code> 、再 <code>vgextend</code>
<code>lvs</code> 增长、 <code>df</code> 不变	FSTYPE、挂载源	只修复文件系统层，不重复增量
目录存在但未挂载	<code>findmnt -T</code>	转入第 24 章修复挂载
设备已有签名或 VG 归属	<code>lsblk -f</code> 、 <code>pvs</code>	停止并重新确认，默认不 force
要求缩小 XFS	文件系统类型	识别为不支持的原地缩小需求，设计迁移

本章工作方法

写操作前重新确认设备身份和数据保留要求，查询只保留会改变下一步决策的字段；对 `-L`、`-l`、加号和 `%FREE` 明确参照对象，每改变一层就立即使用该层报告验证。最终验收必须跨越 LV、文件系统、挂载和数据层，删除、缩容和强制覆盖不作为空间问题的默认修复。

向下一章交接

第 26 章《NFS 客户端与远程文件系统》会处理远端源、NFSv3/v4 必要差异、客户端挂载与身份映射。若 `findmnt` 显示业务目录来自 NFS，本章的 PV/VG/LV 容量链不再适用；应保留当前证据，转入远程文件系统的服务端容量、导出与客户端访问链。

NFS 客户端与远程文件系统

从 `server:/path` 到业务用户读写：把网络、export、协议、挂载、身份和权限放进同一条证据链。

对象模型操作语义验证诊断经典任务

大号号阅读版

本章阅读导航 先抓住一条主线：NFS 不是“把远端目录写进一条 mount 命令”，而是让一个远端 export 经过协议和授权进入本机目录树。任何一步都需要独立证据，尤其不能把“服务器可达”“已经挂载”和“业务用户能写”视为同一状态。

01识别远端源分清服务器、export、`server:/path` 和本地挂载点

02选择协议证据根据 NFSv3/v4 选择 `showmount`、`rpcinfo` 或 2049 证据

03建立当前挂载用 `mount -t nfs` 改变当前名称空间

04读回实际状态用 `findmnt` 与 `nfsstat -m` 读取源、版本和选项

05建立持久声明用 NFS `fstab` 条目和 `_netdev` 描述可重建状态

06以目标身份验收比较 UID/GID、远端权限、`root_squash` 与真实读写

专题地图 - **知识专题** 从远端 export 到本地目录：先分清对象 - **知识专题** NFSv3 与 NFSv4：只掌握会改变调查路径的差异 - **知识专题**

UID/GID、远端权限与 `root_squash` - **操作专题** 建立临时挂载、读取当前证据、写入持久声明 - **诊断专题** 挂载失败、身份权限异常与远端失联 - **经典任务** 持久挂载并验证业务身份；按层定位拒绝与权限问题

阅读时持续回答 1. 当前正在判断网络、export、协议、挂载还是权限？ 2. `showmount` 在这个版本场景中能证明什么？ 3. 当前内核实际挂载的源、版本和选项是什么？ 4. `fstab` 条目能否在当前挂载被移除后重新建立？ 5. 访问进程的数值 UID/GID 和附加组是什么？ 6. `root_squash` 是故障，还是预期的安全边界？ 7. 远端失联时，等待、报错和强制卸载各有什么风险？ 8. 现有证据能证明到哪一层，下一条最有区分度的证据是什么？

章节边界：本地挂载通用机制留给第 24 章；按需触发和空闲卸载留给第 27 章；本章不完整展开 NFS 服务端配置。

第 26 章 · 正文

一台服务器上的目录可以通过 NFS 暴露给网络中的客户端。客户端挂载以后，用户看到的仍是一棵普通目录树，但目录中的文件对象、权限判断和可用性已经跨越网络边界。正因为界面看起来与本地文件系统相似，NFS 故障也很容易被误判：服务器能解析不代表 export 允许当前客户端，目录已经挂载不代表业务用户能写，root 可以列目录不代表应用身份正确，当前挂载成功也不代表系统重新启动后仍会建立同一关系。

本章从客户端视角建立一条完整证据链：先确定服务器和 export，再确认协议版本与访问路径，建立当前挂载，写入持久声明，最后以目标用户的数值 UID/GID 和实际读写行为验收。服务端 `/etc/exports`、`exportfs`、服务启停和防火墙配置只作为诊断时需要向服务端管理员索取的证据，不在本章完整展开；按需挂载留给下一章《autofs 自动挂载》。

概念

NFS **export** 是服务端向特定客户端开放的远端目录及其访问策略。它不是客户端本地目录，也不等同于文件本身；客户端必须通过服务端提供的 NFS 名称空间引用它。判断 export 时既要核对路径，也要核对允许的客户端范围、只读或读写策略和安全方式。

概念

客户端源 **server:/path** 是客户端对远端 export 的引用。冒号左侧确定服务器，右侧是服务器向 NFS 客户端呈现的路径；它与本机是否存在同名路径无关。源格式正确只是对象正确，仍不能证明网络、协议或授权已经成立。

概念

NFSv3 与 NFSv4 的区别在客户端最直接地改变发现和端口证据。NFSv3 常通过 `rpcbind` 查找 MOUNT 与其他 RPC 服务；NFSv4 的主路径更集中，通常以 TCP 2049 和已知 NFSv4 路径为核心。版本不是装饰性选项，而是决定下一条证据的分流条件。

概念

身份映射 是把客户端访问进程的身份用于服务端权限判断的机制。常见 `sec=sys` 场景主要依赖数值 UID、主 GID 和附加组，而不是用户名的外观：两端同名用户若数值不同，仍可能被当作不同身份。

概念

root_squash 是服务端常见的安全默认：来自客户端 UID/GID 0 的请求会被映射成匿名身份，避免客户端 root 自动取得服务端 root 权限。客户端 root 不能写并不自动表示 NFS 故障，关键是题目要求的业务身份能否完成目标操作。

概念

远端文件系统状态 至少分为远端信息、当前挂载、持久声明和业务功能四层。showmount、findmnt、fstab 与目标用户读写分别回答不同问题；上一层成功不能自动证明下一层正确。

操作语义以下入口分别改变当前挂载、读取挂载证据、发现传统 export、读取 RPC 注册、描述持久状态和验证访问身份。先明确命令作用对象，再选择参数。

mount -t nfs

SYNOPSIS

```
mount -t nfs [-o option[,option...]] server:/path mountpoint
```

建立或重建当前 NFS 挂载关系，把远端源接入本地目录树。

重要参数 / 形式

server:/path

远端服务器与 export 路径的组合，冒号不可省略。

-t nfs

明确使用 NFS 文件系统助手。

-o vers=3 / vers=4.2

题目明确版本、兼容性已知或需要诊断版本分支时固定版本。

rw / ro

请求读写或只读挂载；显示为 rw 仍不证明业务身份有写权限。

findmnt

SYNOPSIS

```
findmnt [-T path] [-t type] [-o columns]
findmnt --verify --verbose
```

从当前挂载表或 fstab 读取源、目标、类型和选项，是本章最主要的结构化验证入口。

重要参数 / 形式

-T /srv/project

按路径定位其所属挂载，适合精确确认目标对象。

-o SOURCE,TARGET,FSTYPE,OPTIONS

只输出最能区分假设的字段。

--verify --verbose

静态检查 fstab 语法与可解析性：不等同于真实远端挂载成功。

showmount / rpcinfo

SYNOPSIS

```
showmount -e server
rpcinfo -p server
```

前者查询传统 MOUNT 协议提供的 export 列表，后者读取 rpcbind 注册的 RPC 程序，主要服务于 NFSv3 调查。

重要参数 / 形式

showmount -e

列出传统 MOUNT 服务愿意报告的 export；v4-only 服务器可能不提供该证据。

rpcinfo -p

查看 RPC 程序与端口注册；端口存在不证明 export 允许当前客户端。

版本边界

showmount 失败只能证明传统查询失败，不能单独否定 NFSv4。

nfsstat -m

SYNOPSIS

```
nfsstat -m
```

读取当前 NFS 挂载的实际版本、传输方式、安全方式和协商选项。

重要参数 / 形式

`-m`
显示已挂载 NFS 文件系统及其挂载参数。

`vers=` / `proto=` / `sec=`
确认实际协议版本、传输和认证方式。

`hard` / `soft`
读取远端失联时的重试语义：不能只为避免等待就默认改成 `soft`。

NFS fstab 条目

SYNOPSIS

```
server:/path mountpoint nfs defaults,_netdev 0 0
```

描述可由系统重新建立的持久 NFS 挂载，不是当前挂载本身。

重要参数 / 形式

`nfs`
文件系统类型。

`_netdev`
显式标记网络依赖；不保证 DNS、服务器或业务权限正确。

`defaults`
采用常规默认选项；仅在条目或业务语义明确时增加改变生命周期的选项。

`id` / `getent` / `ls -ln`

SYNOPSIS

```
id user
getent passwd user
getent group group
ls -ldn path
```

建立目标访问身份、NSS 解析结果和远端对象数值属主的证据。

重要参数 / 形式

`id user`
读取 UID、主 GID 和附加组。

`getent`
通过当前 NSS 配置解析用户和组，而不是只读取本地文件。

`ls -ln`
以数值 UID/GID 显示属主，避免用户名映射掩盖数值不一致。

`umount` / `fuser`

SYNOPSIS

```
fuser -vm mountpoint
umount mountpoint
```

调查占用并正常解除当前挂载，适合持久条目重建验证和受控维护。

重要参数 / 形式

`fuser -vm`
列出引用挂载点的进程、用户和访问类型。

`umount`
正常解除挂载；先让 Shell 和应用离开目标路径。

强制或 `lazy` 卸载
只属于已经评估未完成 I/O 和数据风险的应急分支，不是默认答案。

[知识专题] 从远端 export 到本地目录：先分清对象

面对一个 NFS 任务，最顺的切入方式不是先试几种 `mount` 参数，而是把服务端、远端源、当前挂载和本地路径分开。只有对象明确，后续报错才能定位到网络、协议、export、挂载或权限中的某一层。

① [知识点] Export、远端源和挂载点是三个不同对象

服务端 export 是服务器公布并授权的目录。客户端引用它时使用以下形式：

```
<server>:/<export-path>
```

例如：

```
storage.example.com:/exports/project
```

本地挂载点则是客户端目录，例如 `/srv/project`。三者关系是：

```
服务端 export /exports/project
    ↓ 客户端写成远端源
storage.example.com:/exports/project
    ↓ 挂载到本地目录
/srv/project
```

远端源中的路径必须按服务器提供的 NFS 名称空间填写。客户端本地是否存在 `/exports/project` 与远端源无关。路径拼写错误、把本地路径误写进源字段，或漏掉主机名后的冒号，都会使任务从一开始就指向错误对象。

② [知识点] 挂载建立名称空间接入，不复制远端数据

挂载成功后，对 `/srv/project` 的文件操作被交给 NFS 客户端和远端服务器。卸载不会删除服务端数据，只是解除本地目录与远端文件系统的关联。

如果本地挂载点原来非空，挂载期间原内容会被远端文件系统遮蔽，卸载后才重新出现。这个通用机制主归属第 24 章《挂载、fstab、Swap 与启动持久性》；本章只强调考试和排错中的直接后果：挂载前先确认目标目录是否已有需要保留的内容，不要把“看不到”误判为“被删除”。

③ [知识点] 发现、当前挂载、持久声明和功能是四层状态

状态层	问题	典型证据	证据边界
远端信息	服务器和 export 是否合理	<code>getent hosts</code> 、 <code>showmount -e</code> 、服务端证据	不证明一定能挂载
当前状态	当前内核是否已有正确 NFS 挂载	<code>findmnt</code> 、 <code>nfsstat -m</code>	不证明重启后存在
持久状态	是否存在正确持久声明	<code>/etc/fstab</code> 、 <code>findmnt --verify</code> 、重建测试	不证明用户权限正确
功能状态	指定身份能否执行目标操作	<code>sudo -u</code> 读写测试、 <code>ls -ln</code>	只证明已测试的身份和操作

稳定的答案总是按层验收，而不是看到 `mount` 返回零就结束。

[Cheatsheet] `server:/path` 是远端源；本地目录只是接入点；`showmount` 看传统 export 信息，`findmnt` 看当前关系，`fstab` 看持久声明，目标用户读写才是功能终态。

[知识专题] NFSv3 与 NFSv4：只掌握会改变调查路径的差异

NFS 版本差异很多，但本章只保留会直接影响客户端发现、端口判断、路径和验证的部分。不要把协议版本当作背诵题；它真正决定的是下一条最有区分度的证据。

① [知识点] 默认可以协商，明确要求或排错时再固定 vers=

典型挂载可以不写版本：

```
mount -t nfs storage.example.com:/exports/project /srv/project
```

Linux NFS 客户端会与服务器协商可用版本。题目明确要求某版本、服务器兼容性已知，或需要验证某个版本分支时，才显式指定：

```
mount -t nfs -o vers=3 storage.example.com:/exports/project /srv/project
mount -t nfs -o vers=4.2 storage.example.com:/exports/project /srv/project
```

命令中写了 `vers=` 仍要在操作后读取实际挂载证据。未经验证，不把“命令没有报错”扩大为“实际使用了预期的所有参数”。

② [知识点] NFSv3 依赖更多 RPC 辅助服务，NFSv4 的主路径更集中

NFSv3 通常通过服务端 rpcbind 查找 MOUNT、NFS 和其他辅助 RPC 程序；除 NFS 服务外，还可能涉及 mountd、锁和状态相关服务。服务端的部分端口可能动态分配，因此只测试 TCP 2049 不能覆盖完整的 NFSv3 链路。

NFSv4 把更多功能整合进协议，常见主数据路径使用 TCP 2049，不需要客户端通过传统 MOUNT 协议获得同样的信息。由此形成诊断分流：

```
明确或疑似 NFSv3
→ rpcbind/MOUNT/NFS 辅助 RPC 证据

明确或疑似 NFSv4
→ 名称/路由/TCP 2049/已知 NFSv4 路径/实际挂载证据
```

不要为了“让 NFS 工作”在客户端盲目启用所有 RPC 服务。先确定使用的版本和失败层，再处理真正缺失的组件。

③ [知识点] showmount 有版本边界，空结果不是“服务器没有 NFS”的定论

showmount -e <server> 查询的是服务器 MOUNT 协议提供的 export 列表。它对 NFSv3 环境很有价值：

```
showmount -e storage.example.com
```

但仅提供 NFSv4 的服务器可能不运行 rpcbind/MOUNT 服务，showmount 因而可能超时、被拒绝或无法返回列表。此时正确判断是：

```
showmount 无结果
→ 传统 MOUNT 查询不可用
✗ 已证明服务器没有 NFSv4 export
```

如果题目已经给出准确的 NFSv4 源，应继续检查名称解析、TCP 2049、路径语义和实际挂载报错。若需要从 NFSv4 服务器浏览导出树，是否能挂载 server:/ 取决于服务端如何设置 NFSv4 根和可浏览路径，不能作为所有服务器都支持的固定技巧。

[Cheatsheet] 默认先协商，必要时用 vers= 固定；v3 看 rpcbind/MOUNT 及辅助 RPC，v4 主路径更集中；showmount 失败只证明传统查询失败，不能否定 v4。

[知识专题] UID/GID、远端权限与 root_squash

NFS 最常见的“已经挂载但不能用”并不是挂载问题，而是身份和权限问题。最有效的调查方法不是反复 chmod 本地挂载点，而是把访问进程的数值身份、远端文件的数值属主和服务端 export 策略放在同一条链上。

① [知识点] sec=sys 常按客户端数值 UID/GID 传递身份

对常见 AUTH_SYS（挂载选项常显示为 sec=sys）访问，客户端请求携带进程的 UID、主 GID 和附加组。服务端用这些数值与服务端文件系统上的 owner、group 和权限位进行判断。

客户端侧先建立访问身份证据：

```
id project1
getent passwd project1
getent group project
```

文件显示可以同时观察名称和数值：

```
ls -l /srv/project
ls -ln /srv/project
stat -c 'uid=%u gid=%g mode=%A name=%n' /srv/project/path
```

用户名相同但 UID 不同，会被服务端视为不同身份；用户名不同但 UID 相同，数值权限判断可能仍把它们视为同一身份。RHCSA 任务中优先用数值证据解释权限，不依赖“看起来名字一样”。

② [知识点] 挂载成功不会绕过服务端文件权限

挂载建立了访问路径，但不会自动授予写权限。最终结果可能同时受以下因素影响：

```
export 是否只读或可写
→ 服务端文件 owner/group/mode/ACL
→ 客户端进程 UID/GID/附加组
→ 安全方式 sec=
→ root 或其他身份映射
```


本地挂载点在挂载前的 owner 和 mode，不会替代远端根目录的权限。它们主要影响挂载前的本地目录，以及客户端是否能沿父目录路径到达挂载点。挂载后看到的根目录属主和权限来自远端文件系统。

③ [知识点] `root_squash` 让客户端 root 不自动拥有服务端 root 权限

服务端 `export` 的常见默认策略 `root_squash` 会把来自客户端 UID/GID 0 的请求映射为匿名身份。这样做是为了防止任意客户端 root 直接成为共享存储上的服务端 root。

典型表现包括：

- root 能列目录但不能在受限目录中创建文件；
- root 创建的文件在服务端显示为匿名 UID/GID；
- 业务用户能够写，但 root 的测试结果不同；
- `chown` 等特权操作被拒绝。

不要把 `no_root_squash` 当作“写入失败”的默认修复。正确做法是明确题目要求由哪个业务身份访问，再校对两端 UID/GID、附加组、`export` 读写模式和远端文件权限。匿名身份的具体用户名可能因系统配置而异，应以数值 UID/GID 和服务端配置为准。

[Cheatsheet] 先 `id`，再 `ls -ln`；挂载成功不增加权限；客户端 root 受 `root_squash` 约束是安全默认，不用 `no_root_squash` 掩盖身份设计错误。

[操作专题] 准备客户端并建立临时 NFS 挂载

临时挂载适合先验证远端源和协议，也适合在写入持久配置前建立基线。操作顺序应从“已有状态”开始：确认目标目录是否已经挂载、确认软件能力和名称解析，然后才创建目录并挂载。

① [操作] 确认 `nfs-utils` 和帮助入口

作用对象：客户端 NFS 辅助程序和诊断工具。基本语义：RHEL9 使用 `nfs-utils` 提供 `mount.nfs`、`showmount`、`nfsstat` 等能力。典型操作：

```
rpm -q nfs-utils
dnf install nfs-utils
```

安装后可使用本地手册确认语义：

```
man 5 nfs
man 8 mount.nfs
man 8 showmount
man 8 nfsstat
```

边界：安装包成功不证明服务器可达；通常也不需要为了 NFSv4 挂载手工启动一个“客户端 NFS 服务”。具体依赖由挂载流程和 `systemd` 按需处理。

② [操作] 调查名称、已知 `export` 和已有挂载

作用对象：服务器名称、目标路径和客户端当前名称空间。基本形式：

```
getent hosts storage.example.com
findmnt -T /srv/project
showmount -e storage.example.com
```

在执行 `showmount` 前先明确其版本边界。若服务器是 `v4-only`，`showmount` 失败不是停止调查的理由。若任务使用 NFSv3，可补充：

```
rpcinfo -p storage.example.com
```

`rpcinfo` 输出用于确认服务端 `rpcbind` 注册的 RPC 程序；它不是权限测试，也不证明 `export` 允许当前客户端。

③ [操作] 创建挂载点并执行 `mount -t nfs`

作用对象：当前内核挂载关系。基本形式：

```
mkdir -p /srv/project
mount -t nfs storage.example.com:/exports/project /srv/project
```

显式版本示例：

```
mount -t nfs -o vers=4.2 \
storage.example.com:/exports/project /srv/project
```

关键检查：

```
findmnt -T /srv/project -o SOURCE,TARGET,FSTYPE,OPTIONS
nfsstat -m
```

边界：

- 如果目标已经挂载，先读取现状，不重复叠加挂载；
- 不用 `-o soft` 作为“防止卡住”的默认选项；
- 不把 `rw` 显示为挂载选项扩大解释为业务用户一定能写；
- 临时挂载在重启后不会自动恢复。

[Cheatsheet] 包：`nfs-utils`；源：`server:/path`；当前挂载：`mount -t nfs`；操作后用 `findmnt` 和 `nfsstat -m` 读回实际状态。

[操作专题] 从 `findmnt`、`mount` 和 `nfsstat` 读取当前证据

同一个挂载可以从多个工具观察，但工具回答的问题不同。优先选择能够直接区分假设的字段，而不是在冗长输出中寻找看起来熟悉的一行。

① [操作] 用 `findmnt` 建立源、目标、类型和选项映射

作用对象：当前挂载表与配置文件。典型形式：

```
findmnt -T /srv/project
findmnt -T /srv/project -o SOURCE,TARGET,FSTYPE,OPTIONS
findmnt -t nfs,nfs4
```

`-T` 按任意路径查找其所属挂载，即使给的是挂载点下的文件路径也能定位。排错时重点读：

- `SOURCE` 是否是预期服务器和路径；
- `TARGET` 是否是预期本地目录；
- `FSTYPE` 是否是 NFS；
- `OPTIONS` 中是否包含预期的版本、安全方式和读写模式。

`findmnt` 无结果时，目录仍可能存在，只是它不属于单独的当前挂载。

② [操作] 把 `mount` 作为兼容的快速视图

以下命令可以浏览当前 NFS 挂载：

```
mount -t nfs,nfs4
```

它适合人工快速查看，但字段筛选和按路径定位通常不如 `findmnt` 清晰。不要使用不精确的 `mount | grep nfs` 结果作为唯一证据：服务器名、路径或目录中可能恰好包含匹配字符串。

③ [操作] 用 `nfsstat -m` 查看 NFS 专属挂载信息

```
nfsstat -m
```

该视图可帮助确认：

- 实际 NFS major/minor 版本；
- 传输协议；
- `sec=` 安全方式；
- `hard / soft`、超时和重传相关选项；
- 读写、缓存和其他协商参数。

`nfsstat -m` 只能证明当前 NFS 客户端挂载参数，不能证明服务端数据内容正确、应用没有锁冲突或业务用户具备权限。验证应继续进入目标身份和真实文件操作。

[Cheatsheet] 精确路径用 `findmnt -T`；快速浏览可用 `mount -t nfs,nfs4`；协议和 NFS 专属选项用 `nfsstat -m`；三者都不是业务功能测试。

[操作专题] 用 `/etc/fstab` 建立持久 NFS 挂载

持久挂载不是“把成功命令抄进文件”这么简单。它需要声明可解析的源和挂载点，明确网络依赖，并通过卸载后重建来证明条目确实能够独立建立当前状态。

① [操作] 编写 NFS fstab 条目并解释 `_netdev`

示例条目：

```
storage.example.com:/exports/project /srv/project nfs defaults,_netdev 0 0
```

六个字段分别是：远端源、本地挂载点、文件系统类型、挂载选项、dump 字段和 fsck 顺序。此处只说明 NFS 特有部分；通用六字段主归属第 24 章。

`_netdev` 明确该文件系统依赖网络，使 `systemd` 按远程文件系统处理相关顺序。它不表示：

- DNS 一定正确；
- 网络在线目标一定代表服务器可访问；
- 服务器宕机时系统绝不会等待；
- 挂载会在失败后按业务期望自动恢复；
- 用户权限正确。

除非题目明确要求，不随意添加 `nofail`、`bg`、`soft` 或 `x-systemd.automount`。这些选项会改变启动、错误返回或生命周期语义；按需挂载的完整设计留给第 27 章。

② [操作] 先静态核对，再由 fstab 重建当前状态

变更前保留基线：

```
cp -a /etc/fstab /etc/fstab.before-rhcsa26
```

编辑后执行：

```
findmnt --verify --verbose  
systemctl daemon-reload
```

静态核对通过后，使用条目建立挂载：

```
mount /srv/project  
findmnt -T /srv/project -o SOURCE,TARGET,FSTYPE,OPTIONS
```

如果之前已经手工挂载，为证明 `fstab` 自身可工作，应在确认没有进程占用后卸载，再重建：

```
umount /srv/project  
mount -a  
findmnt -T /srv/project -o SOURCE,TARGET,FSTYPE,OPTIONS
```

`mount -a` 可能处理系统中的其他 `fstab` 条目。在真实生产系统中应先评估全局影响；考试实验环境中仍要先运行 `findmnt --verify`，避免一个错误条目影响整体测试。

③ [验证] 当前、持久和功能分层验收

建议验收矩阵：

```
# 当前关系  
findmnt -T /srv/project -o SOURCE,TARGET,FSTYPE,OPTIONS  
  
# NFS 协议和选项  
nfsstat -m  
  
# 指定身份  
id project1  
  
# 最小功能测试  
sudo -u project1 sh -c '  
umask 022  
printf "%s\n" "nfs-client-check" > /srv/project/.client-check  
cat /srv/project/.client-check  
printf "%s\n" "append" >> /srv/project/.client-check  
rm -f /srv/project/.client-check  
'
```

不要伪造输出，也不要把 `root` 测试替代业务用户测试。若题目只要求只读访问，则使用 `test -r`、`cat` 或读取指定文件，不额外创建文件改变远端数据。

[Cheatsheet] `fstab`: `server:/path target nfs defaults,_netdev 0 0`；先 `findmnt --verify`，再 `daemon-reload`；卸载后由 `mount -a` 重建；最后按目标身份做真实功能测试。

[诊断专题] 服务器可达，但 NFS 挂载失败

“服务器可达”只排除了很小一部分问题。诊断必须先记录原始报错，再选择能最大程度区分假设的下一条证据。不要因为 `ping` 成功就跳过协议和 `export`，也不要因为 `showmount` 失败就直接修改路径。

① [诊断] 名称、路由、端口和协议层分开取证

症状：`mount` 报名称解析失败、无路由、连接拒绝或超时。当前证据：原始命令和完整错误文本。下一条证据：

```
getent hosts storage.example.com
ip route get <resolved-address>
```

根据版本继续：

```
NFSv4 路径
→ TCP 2049 和服务端 NFS 服务证据

NFSv3 路径
→ 服务端 rpcbind、MOUNT、NFS 及辅助 RPC 注册/防火墙证据
```

客户端可以使用允许的连接检查工具，但不要把“端口通”扩大为“export 允许”。连接被拒绝与超时也不同：前者说明目标主动拒绝或无监听，后者更可能涉及过滤、路由、服务无响应或丢包。

② [诊断] 按 v3/v4 边界解释 `showmount` 和 `rpcinfo`

症状：`showmount -e` 没有返回预期列表。假设 A：服务器或网络确实不可用。假设 B：服务器是 v4-only，不提供传统 MOUNT 查询。假设 C：rpcbind/MOUNT 被防火墙过滤。最有区分度的证据：题目指定版本、服务端版本配置、TCP 2049、`rpcinfo -p` (v3) 和已知源的实际挂载报错。

不要为了让 `showmount` 成功去改变 v4-only 服务器设计。`showmount` 是发现工具，不是 NFSv4 服务健康的唯一验收。

③ [诊断] `access denied by server` 指向 `export` 或安全策略层

典型报错中的“server denied”意味着请求已经到达服务端，服务端拒绝了挂载或访问。调查顺序：

```
客户端请求的 server:/path 是否准确
→ 该路径在目标 NFS 版本的名称空间中是否有效
→ export 是否允许当前客户端地址/网段
→ export 是否要求不同 sec= 安全方式
→ 服务端是否已加载最新 export 配置
```

客户端对本地挂载点执行 `chmod 777` 不能修改服务端 `export` 许可。最小修复应发生在错误的那一层：纠正客户端源路径，或由服务端管理员修正 `export` 范围和安全策略，然后重新挂载并分层验证。

[Cheatsheet] 先留原始报错：解析→路由→版本对应端口/RPC→路径→export 客户端范围→sec=；`access denied by server` 不是本地目录 mode 问题。

[诊断专题] 已经挂载，但属主、读写或 root 行为不符合预期

此类故障要保持同一测试身份。用 root 复现、再用普通用户修复，或先后切换多个用户，会混淆 UID/GID、附加组和 `root_squash` 三个变量。

① [诊断] 先用目标用户重现，并记录数值身份

症状：`Permission denied`、只能读不能写、创建文件属主异常。当前证据：指定用户执行的最小操作及错误。下一条证据：

```
id project1
sudo -u project1 test -r /srv/project && echo readable
sudo -u project1 test -w /srv/project && echo writable
ls -ldn /srv/project
```

`test -w` 是权限预判，不等同于创建一定成功；最终仍需按题目执行最小真实操作。

② [诊断] 比较数值 owner/group 和附加组

若目录由某组写入，除了主 GID，还要查看用户的附加组：

```
id project1
getent group project
ls -ldn /srv/project
```

需要服务端证据时，应让服务端管理员用数值形式检查远端目录和文件。若两端用户同名但 UID 不同，修复应是统一身份来源或调整正确的文件属主/组，而不是扩大权限位。

③ [诊断] 区分普通权限问题与 `root_squash`

如果只有客户端 `root` 行为异常，或 `root` 创建内容显示匿名属主，应检查 `export` 的 `root` 映射。判断链：

```
业务用户能否完成要求
→ root 是否本来就不是业务访问身份
→ 服务端是否使用 root_squash
→ 匿名 UID/GID 是否具备目标权限
```

题目要求普通用户读写时，`root` 被 `squash` 不构成故障。只有业务需求明确要求某种特权操作时，才需要重新设计服务端授权；也不应默认启用 `no_root_squash`。

[Cheatsheet] 固定测试身份：`id` 看 UID/GID/附加组；`ls -ln` 看数值属主；业务用户失败查权限链，只有 `root` 异常再查 `root_squash`。

[诊断专题] 远端不可用、请求等待与安全卸载边界

网络文件系统把本地文件操作变成远端请求。服务器失联时，应用可能不是立即报错，而是等待服务器恢复。排错人员若不了解这一点，容易反复终止进程、强制卸载或把 `soft` 写进所有配置，进而增加数据风险。

① [知识点] 默认 `hard` 语义更偏向等待远端恢复

常见 NFS 挂载使用 `hard` 语义：请求超时后继续重试，直至服务器响应。对正在访问挂载点的进程，这可能表现为系统调用长时间不返回，甚至处于不可中断 I/O 等待。

此时：

- `kill -9` 不一定能立即终止处于不可中断等待的任务；
- 新的 `ls`、`df`、`stat` 或 Shell 自动补全也可能触发远端访问并一起等待；
- 应优先恢复名称解析、网络路径或 NFS 服务，而不是制造更多访问请求。

② [安全边界] `soft` 不是通用的“避免卡死”方案

`soft` 类挂载在重试耗尽后向应用返回错误。对某些只读或可容忍失败的工作负载，经过应用和数据设计评估后可能使用；但对一般读写文件系统，它可能使应用收到部分失败、I/O 错误或产生数据完整性问题。

因此本章默认：

```
不因担心等待而自动添加 soft
不把调小 timeo/retrans 当作普适修复
先依据业务语义、应用容错和数据重要性评估
```

考试未明确要求时，保留安全默认并解释远端不可用行为。

③ [诊断] 先恢复远端链路，再处理占用和卸载

症状：挂载点命令等待，或 `umount` 报 `busy`。当前证据：当前 NFS 源、网络和占用情况。调查：

```
findmnt -T /srv/project -o SOURCE,TARGET,FSTYPE,OPTIONS
fuser -vm /srv/project
```

先让 Shell 离开挂载目录，正常停止正在使用该路径的应用，再尝试普通卸载：

```
cd /
umount /srv/project
```

`umount -f` 或 `lazy unmount` 会改变失败处理方式，可能隐藏仍在运行的引用或未完成 I/O，只能在已经评估数据风险、正常恢复不可行的应急场景使用，不能作为考试和日常操作的首选答案。

恢复后重新验证：

```
网络和服务器恢复
→ 当前挂载是否恢复或需要重建
→ 文件系统读写是否正常
→ 应用是否有未完成事务或损坏
→ 持久配置是否仍符合目标
```

[Cheatsheet] hard 失联可能长期等待；D 状态不能靠反复 `kill -9`；`soft` 有数据风险；先恢复远端和正常停止使用者，再考虑卸载，强制手段只作受控应急分支。

[操作专题] 把 NFS 调查写成可重复的证据清单

考试中最容易丢分的不是不知道 `mount`，而是修改过多、验证不足。下面的清单把每一步限制为一个问题，既便于手工操作，也为后续 Ansible 自动化准备稳定的状态模型。

① [操作] 变更前建立基线

```
rpm -q nfs-utils
getent hosts storage.example.com
findmnt -T /srv/project
```

记录：目标目录是否存在、是否已挂载、当前源和选项、是否已有 `fstab` 条目。不要先删除旧条目或覆盖目录。

② [操作] 变更只改变必要对象

```
缺包      → 安装 nfs-utils
缺挂载点  → mkdir -p
源或选项错 → 纠正 mount/fstab
身份错    → 统一 UID/GID 或正确属主/组
export 拒绝 → 服务端修正授权
```

避免把一个层的故障用另一层的高风险配置绕过。例如：`export` 拒绝不靠 `chmod 777`，`root squash` 不靠 `no_root_squash`，远端超时不靠默认 `soft`。

③ [验证] 建立证据矩阵而不是一句“成功”

验收对象	推荐证据
软件能力	<code>rpm -q nfs-utils</code>
名称解析	<code>getent hosts</code>
当前源和目标	<code>findmnt -T</code>
协议和选项	<code>nfsstat -m</code>
持久配置	<code>findmnt --verify</code> 、卸载后重建
访问身份	<code>id</code> 、 <code>getent</code>
数据功能	目标用户按题意读写
远端故障边界	日志、网络、应用等待和恢复复验

[Cheatsheet] 基线→最小变更→读回状态→重建持久状态→目标身份功能；任何工具只证明自己的那一层。

[经典任务] 配置持久 NFS 客户端挂载并验证读写身份

任务环境

你管理一台 RHEL 9 客户端。存储服务器提供以下已知远端源：

```
storage.example.com:/exports/project
```

要求在客户端将其挂载到：

```
/srv/project
```

业务用户 `project1` 已经存在。服务端管理员确认该 `export` 允许本客户端访问，并且预期 `project1` 具有读写权限。本题不允许修改服务端、不使用 `autofs`，也不把 `soft`、`nofail`、`no_root_squash` 或 `chmod 777` 作为捷径。

当前状态

- 不确定客户端是否已安装 NFS 工具；
- `/srv/project` 可能不存在；
- 当前没有已确认的正确挂载；

- `/etc/fstab` 中可能存在其他系统条目，不能破坏；
- 本会话没有真实服务器输出，所有命令都需要在实验环境执行后按实际结果判断。

目标终态

1. 客户端具备 NFS 挂载工具；
2. 当前挂载源、目标和类型准确；
3. `/etc/fstab` 中存在带 `_netdev` 的持久条目；
4. 卸载当前挂载后，通过持久配置可以重新建立；
5. 可以读取实际 NFS 版本和挂载选项；
6. `project1` 可以创建、读取、追加并删除测试文件；
7. 验证过程不把 `root` 的成功替代为业务用户成功。

验收证据

验收项	必须提供的证据
软件	<code>rpm -q nfs-utils</code>
解析	<code>getent hosts storage.example.com</code>
当前挂载	<code>findmnt -T /srv/project</code>
NFS 参数	<code>nfsstat -m</code>
fstab 静态核对	<code>findmnt --verify --verbose</code>
持久重建	卸载后 <code>mount -a</code> ，再查 <code>findmnt</code>
身份	<code>id project1</code>
功能	<code>sudo -u project1</code> 的最小读写测试

[参考解答] 按“调查—当前—持久—身份—功能”完成任务

① [调查] 保留已有状态并确认客户端能力

```
rpm -q nfs-utils || dnf install -y nfs-utils
getent hosts storage.example.com
findmnt -T /srv/project
```

如果 `findmnt` 已显示一个挂载，先比较其 `SOURCE`、`TARGET`、`FSTYPE` 和 `OPTIONS`，不要直接叠加。检查现有配置：

```
grep -nE '^(^|[:space:]+)/srv/project([[:space:]]|$)' /etc/fstab
```

此处的 `grep` 只用于定位相关行，最终仍要解析和重建验证。

② [操作] 建立受控的当前挂载

确认挂载点不存在或可安全使用：

```
mkdir -p /srv/project
```

建立临时挂载：

```
mount -t nfs \
storage.example.com:/exports/project \
/srv/project
```

读回实际状态：

```
findmnt -T /srv/project -o SOURCE,TARGET,FSTYPE,OPTIONS
nfsstat -m
```

若挂载失败，保留原始错误，按名称/路由、版本、`export` 和安全方式分层调查；不要随机追加多个选项。

③ [配置] 写入并核对持久声明

先备份：

```
cp -a /etc/fstab /etc/fstab.before-rhcsa26
```

确保最终只保留一个目标条目：

```
storage.example.com:/exports/project /srv/project nfs defaults,_netdev 0 0
```

静态核对并让 systemd 重新运行生成器：

```
findmnt --verify --verbose
systemctl daemon-reload
```

如果静态检查指出其他既有条目问题，不应无授权修改与本题无关的系统配置；记录问题并只纠正本题引入的错误。

④ [验证] 证明 fstab 能独立重建挂载

先确认没有进程在使用测试目录：

```
fuser -vm /srv/project
```

在安全条件下：

```
cd /
umount /srv/project
mount -a
findmnt -T /srv/project -o SOURCE,TARGET,FSTYPE,OPTIONS
nfsstat -m
```

若不希望 `mount -a` 影响其他条目，也可以先使用 `mount /srv/project` 直接按目标匹配 `fstab`；但完整考试验收仍应保证 `fstab` 整体没有由本题引入的语法错误。

⑤ [验证] 使用业务身份完成真实读写

```
id project1
sudo -u project1 sh -c '
  set -eu
  file=/srv/project/.rhcsa26-client-check
  printf "%s\n" "created-by-project1" > "$file"
  cat "$file"
  printf "%s\n" "append-ok" >> "$file"
  tail -n 1 "$file"
  rm -f "$file"
'
```

如果失败：

```
id project1
ls -ldn /srv/project
findmnt -T /srv/project -o SOURCE,TARGET,FSTYPE,OPTIONS
```

然后请服务端管理员按数值 UID/GID 检查远端目录、export 读写模式和 root/匿名映射。不要在客户端对挂载点执行 `chmod 777` 试图改变远端 export 策略。

⑥ [验收] 汇总而不扩大结论

最终可以声明的证据是：

- 软件包存在；
- 名称按 NSS 解析；
- 当前源、目标和类型符合任务；
- `nfsstat -m` 记录实际协议和选项；
- `fstab` 通过静态核对并能重建挂载；
- `project1` 完成了指定的创建、读取、追加和删除。

本章没有连接 live VM，因此不能声称这些命令已在真实 RHEL 9 环境跑通。

[经典任务] 诊断服务器可达但挂载被拒绝或写入权限不符

场景

客户端要使用：

```
files.example.com:/departments/ops
```

挂载点为 `/mnt/ops`。用户报告“服务器能通，但 NFS 不能用”。当前可能出现以下任一症状：

- `showmount -e` 超时；
- `mount` 报 `access denied by server`；
- 挂载成功，但用户 `opsuser` 创建文件时报 `Permission denied`；
- `root` 与 `opsuser` 的测试结果不同。

限制

- 不关闭客户端或服务端防火墙；
- 不关闭 SELinux；
- 不默认使用 `no_root_squash`；
- 不执行 `chmod 777`；
- 不反复尝试所有 NFS 版本；
- 必须说明每条证据正在区分哪两个假设。

验收

提交一条从症状到最小修复的调查链，并在修复后分别证明当前挂载、持久状态（如题目要求）和 `opsuser` 功能。

[参考解答] 先确定失败层，再做最小修复

① [调查] 固定原始命令、症状和目标版本

```
getent hosts files.example.com
findmnt -T /mnt/ops
```

记录题目是否明确版本。若未明确，先使用默认协商，不把随机固定版本作为第一步。

② [分流] `showmount` 超时

`showmount` 超时至少存在三种假设：服务器/网络不可用、传统 MOUNT/RPC 被过滤、服务器是 v4-only。下一条证据应围绕实际版本：

```
若要求 NFSv3
→ rpcinfo -p files.example.com
→ 服务端 rpcbind/MOUNT/NFS 和防火墙证据

若要求或已知 NFSv4
→ TCP 2049、已知路径和 mount 原始错误
→ 不要求 showmount 必须成功
```

最小修复是恢复对应协议链，不是为了 `showmount` 改变服务器版本设计。

③ [分流] `access denied by server`

核对客户端请求：

```
mount -t nfs files.example.com:/departments/ops /mnt/ops
```

向服务端管理员索取：

- 目标路径在该 NFS 版本名称空间中的准确形式；
- `export` 允许的客户端主机或网段；
- `export` 的 `ro/rw` 和 `sec=`；
- 最新 `export` 配置是否已经加载。

最小修复只更正错误源路径或服务端授权。客户端挂载点 `mode` 不能解决服务端挂载拒绝。

④ [分流] 已挂载但 opsuser 不能写

```
findmnt -T /mnt/ops -o SOURCE,TARGET,FSTYPE,OPTIONS
nfsstat -m
id opsuser
ls -ldn /mnt/ops
sudo -u opsuser sh -c 'touch /mnt/ops/.ops-check'
```

比较：客户端 opsuser UID/GID/附加组、服务端远端目录数值属主、export 是否只读。如果 root 与 opsuser 不同，再检查 root_squash，但题目要求 opsuser 写入时应优先修正 opsuser 的身份和远端权限。

⑤ [修复后验证]

```
同一条源重新挂载
→ findmnt 确认当前源和目标
→ nfsstat -m 确认版本/选项
→ opsuser 执行题目要求的读写
→ 如要求持久化，再做 fstab 静态核对与重建
```

调查结论必须精确到层，例如“服务端 export 未允许本客户端地址”，而不是笼统写“网络问题”或“NFS 权限问题”。

[本章收束] 把“远端目录能用”还原成一条可验证链

NFS 客户端任务的核心不是记住一行 mount，而是始终知道当前证据属于哪一层。远端 export 通过 server:/path 被引用；协议版本决定发现和端口证据；当前挂载由 findmnt 与 nfsstat -m 读回；持久状态由 fstab 静态核对和卸载后重建证明；最终授权则由数值 UID/GID、远端权限、export 策略和目标用户实际操作共同决定。

可执行的工作方法

```
先保留原始症状和当前状态
→ 明确 server:/path 与本地挂载点
→ 按 NFS 版本选择网络、端口和 RPC 证据
→ 建立或修正当前挂载
→ 读回实际源、版本和选项
→ 由 fstab 独立重建持久状态
→ 固定目标用户，比较 UID/GID 与远端数值权限
→ 做最小功能测试
→ 只修复已被证据定位的那一层
```

章末检查清单

- ☐ 我能说清 export、客户端源和本地挂载点的区别。
- ☐ 我不会把 showmount 当作所有 NFSv4 场景的唯一证据。
- ☐ 我能用 findmnt 和 nfsstat -m 读取当前挂载的实际状态。
- ☐ 我能解释 _netdev 能做什么，以及不能保证什么。
- ☐ 我会在修改权限前先固定测试身份并查看数值 UID/GID。
- ☐ 我知道 root_squash 往往是预期安全边界，而不是必须取消的故障。
- ☐ 我不会把 soft、chmod 777、no_root_squash 或强制卸载作为无调查默认答案。
- ☐ 我能分别验收当前状态、持久状态和业务功能。

主要判断表

现象或证据	它能支持的判断	它不能单独证明什么	下一条更有区分度的证据
getent hosts 返回地址	NSS 能解析服务器名称	NFS 端口、export 或权限正确	路由与版本对应的端口/RPC
showmount -e 返回 export	传统 MOUNT 查询成功	v4 实际挂载与业务权限正确	已知源的挂载与 nfsstat -m
showmount 超时或失败	传统 MOUNT 查询不可用	服务器没有 NFSv4	版本信息、TCP 2049、实际挂载报错
findmnt 显示 NFS	当前内核存在挂载关系	fstab 能重建、用户能读写	nfsstat -m、fstab 重建、用户测试
fstab 有正确条目	存在持久声明	当前已挂载、远端可达	findmnt --verify 后卸载重建
rw 出现在挂载选项	客户端请求了读写语义	export 与文件权限允许该用户写	固定业务用户的最小创建测试
root 不能写	root 请求未获得目标权限	业务用户也失败	业务用户测试与 root_squash 证据
访问挂载点长期等待	远端请求可能在重试	一定是 CPU 或本地磁盘故障	当前 NFS 源、网络、服务器与进程状态

向下一章交接

本章处理的是“声明后稳定存在”的 NFS 客户端挂载。需要在访问路径时才触发挂载、空闲后自动卸载，或用通配符映射多个远端目录时，进入第 27 章《autofs 自动挂载》。两章共享远端源和身份模型，但生命周期、配置对象和验证入口不同，不能把 `x-systemd.automount` 或 `autofs map` 机械混入本章答案。

RHEL 9 · RHCSA 实操讲义

autofs 自动挂载

从 map 到触发载荷：把路径模型、按需挂载、空闲卸载和证据链放进同一套判断框架。

对象模型操作语义验证诊断经典任务
大字号阅读版

本章阅读导航 先抓住一条主线：**autofs** 先建立可触发的路径入口，真正访问 key 时才建立远端载荷；载荷空闲卸载后，入口仍然可以再次触发。**因此**，“配置文件存在”“服务 active”“路径存在”“当前 NFS 已挂载”和“目标身份可读写”必须分别证明。

01识别路径模型判断 direct 还是 indirect

02建立 **map** 关系master → key → location

03静态展开确认有效配置视图

04加载触发层启动或 reload autofs

05访问 key触发远端载荷

06分层验收源、功能、超时与再触发

专题地图 - **知识专题** 从 map 到载荷：autofs 管理的对象链 - **知识专题** indirect 与 direct 的路径模型 - **操作专题** 配置可验证的 indirect NFS 自动挂载 - **知识专题** wildcard `\*` 与替换 `&` - **操作专题** 用 direct map 管理任意绝对路径 - **操作专题** reload、restart、timeout 与 busy - **操作专题** 配置、触发、载荷与功能证据矩阵 - **诊断专题** 路径存在却没有挂载时如何定位 - **经典任务** 用户家目录自动挂载与错误 key 修复

阅读时持续回答 1. master map 与子 map 各自决定什么？ 2. 当前 key 是相对目录名还是完整绝对路径？ 3. `automount -m` 证明了哪一层，尚未证明哪一层？ 4. 当前看到的是 `autofs` 触发层还是 NFS 载荷？ 5. 哪一次精确路径访问会触发挂载？ 6. 超时后载荷消失是故障还是预期生命周期？ 7. 当前挂载源正确后，是否已用目标身份验证功能？ 8. 失败属于 entry、key、trigger、remote、permission 还是 busy？

章节边界：NFS 远端导出与身份语义留给第 26 章；固定挂载与 `/etc/fstab` 留给第 24 章；systemd automount 不作为本章主线替代。

第 27 章 · 正文 网络文件系统并不一定要从开机到关机始终保持挂载。大量用户家目录、偶尔访问的项目共享和低频归档路径，都更适合在真正访问时才连接远端；无人使用一段时间后，载荷可以自然卸载。`autofs` 正是把这种生命周期表达为配置：master map 建立入口，子 map 描述 key 与远端位置，路径访问触发真实挂载，timeout 管理空闲后的卸载。本章最常见的误判，是把不同状态压成一句“挂载好了”。服务 `active` 只说明 daemon 在运行；`automount -m` 只说明 map 能被读取；父目录或触发路径存在，不代表远端 NFS 载荷此刻已经出现；当前载荷正确，也还没有证明题目指定的用户能够读写。后续所有操作都沿着“配置 → 加载 → 触发 → 当前载荷 → 功能 → 超时 → 再触发”推进，并明确每条证据能证明什么、不能证明什么。

概念

Master Map 是 autofs 的入口索引。它把一个 indirect map 的父挂载点，或者 direct map 使用的 `/-` 标记，关联到具体子 map；它决定“哪些本地路径交给 automounter 管理”，却不直接描述每个远端载荷。看到 master entry 时应继续追踪子 map，而不能把入口配置误认成完整挂载定义。

概念

Indirect Map 在一个共同父目录下按相对 key 形成最终路径。若 master mount point 是 `/rhome`，key 是 `remoteuser0`，触发路径就是 `/rhome/remoteuser0`。子 map 中若误写完整路径，请求 key 将无法匹配；因此 indirect 的核心边界是“父路由 master 提供，子 map 只写相对 key”。

概念

Direct Map 让子 map 的 key 自身成为完整绝对路径。master map 使用 `/-` 表示 direct 模式，实际路径写在子 map，例如 `/srv/docs`。`/-` 不是要创建或访问的目录；它只是告诉 automounter：不要再把父挂载点与 key 拼接。

概念

Key 是路径访问与 map 条目之间的匹配对象。它在 indirect map 中通常是一个目录组件，在 direct map 中则是完整绝对路径；wildcard `*` 可以接住没有显式条目的请求，而 `&` 把实际请求 key 代入远端位置。判断 key 的含义，是定位“不触发”问题时最有区分度的一步。

概念

触发式挂载 包含两层文件系统：**autofs** 类型的触发层负责拦截路径访问，**nfs / nfs4** 载荷层才是真正的远端文件系统。服务启动后可以已经存在触发层，而在访问 key 前没有任何对应 NFS 载荷；所以“配置存在但尚未挂载”往往是正常初态，而不是错误。

概念

空闲超时 管理的是已触发载荷的生命周期，而不是配置文件的生命周期。载荷在无活动引用并超过 timeout 后可以卸载；若 Shell 的当前目录仍位于其中、文件仍被打开或其他进程仍在使用，载荷可能保持 busy。超时卸载后再次访问能够恢复，才说明按需生命周期完整。

操作语义速查 这一页先建立最关键的接口地图。详细操作仍在后续专题中展开；这里不把所有参数塞进横向表格，而是强调每个入口改变或观察哪一层状态。

`/etc/auto.master` 与 `/etc/auto.master.d/*.autofs`

SYNOPSIS

```
mount-point  map-name  [master-options]
/-           map-name  [master-options]
```

建立 master entry: indirect 模式写父挂载点, direct 模式写 `/-`。它只建立入口和子 map 的关联。

```
/rhome /etc/auto.rhome --timeout=60
```

让 `/rhome` 下的请求交给 indirect map 解析。

```
/- /etc/auto.direct --timeout=60
```

声明 direct map, 实际绝对路径写在子 map。

```
*.autofs
```

RHEL 课程环境中用于 `/etc/auto.master.d/` 的本地 master 片段后缀。

direct / indirect 子 map

SYNOPSIS

```
key  -mount-options  location
```

把请求 key 映射为挂载选项和远端位置。map 类型决定 key 是相对名称还是完整路径。

```
alpha -fstype=nfs,rw host:/exports/alpha
```

indirect 条目; 最终路径由 master point 与 `alpha` 拼接。

```
/srv/docs -fstype=nfs,ro host:/exports/docs
```

direct 条目; key 本身就是本地绝对路径。

```
* ... /rhome/&
```

* 匹配请求 key, `&` 将该 key 代入 location。

```
automount -m
```

SYNOPSIS

```
automount -m
```

展开当前可读取的 master map 与子 map, 用于确认 entry、map 来源和 key 是否进入有效配置视图。

```
-m
```

打印 maps 后退出; 属于静态配置证据。

证明边界

不证明服务已应用新配置, 不证明远端可访问, 也不证明 key 已经触发。

```
systemctl reload/restart autofs
```

SYNOPSIS

```
systemctl enable --now autofs
systemctl reload autofs
systemctl restart autofs
```

控制 daemon 当前状态、启动持久性和配置重新读取。修改 map 后优先走“静态展开 → reload → 重新触发”。

```
enable --now
```

首次部署时同时建立当前运行和开机启用状态。

```
reload
```

重新读取配置, 尽量保留现有 daemon 生命周期。

```
restart
```

完整重启服务, 影响更强; 不能修复错误 key 或远端路径。

```
stat / ls -ld
```

SYNOPSIS

```
stat /base/key
ls -ld /base/key
```

对精确 key 发起路径解析, 从而有意识地触发按需挂载。只列父目录不一定枚举所有尚未触发的 key。

```
stat /rhome/remotouser0
```

直接访问目标 key，适合触发与错误复现。

```
ls -ld PATH
```

查询目标路径本身，避免普通 `ls PATH` 继续遍历大量内容。

```
findmnt
```

SYNOPSIS

```
findmnt -t autofs
findmnt -t nfs,nfs4
findmnt -T /base/key
```

读取当前挂载关系，用于区分触发层与远端载荷，并核对某个路径当前实际落在哪个文件系统上。

```
-t autofs
```

观察受管理的触发层。

```
-t nfs,nfs4
```

观察当前已经触发的 NFS 载荷。

```
-T PATH
```

从具体路径反查覆盖它的挂载源、目标与类型。

[知识专题] 从 map 到载荷：autofs 管理的到底是什么

理解 autofs 的最短路径不是先背三行配置，而是先画出对象链。master map 决定哪些路由 automounter 管理；子 map 决定某个 key 对应什么文件系统；内核 autofs 文件系统负责在路径访问时通知用户态 daemon；daemon 再执行真实挂载。排错时必须知道证据位于哪一层，否则容易把“目录已经出现”误判为“NFS 已经挂上”，或把“服务 active”误判为“远端读写已经成功”。

① [知识点] master map 是入口索引，子 map 才描述具体载荷

master map 的基本形式是：

```
mount-point map-name options
```

例如：

```
/rhome /etc/auto.rhome --timeout=60
```

这一行只说明 `/rhome` 由 `/etc/auto.rhome` 管理，并为该 map 设置 60 秒空闲超时。它没有直接给出某个用户目录来自哪台服务器。具体 key、挂载选项和远端源在子 map 中定义：

```
remotouser0 -fstype=nfs,rw serverb.lab.example.com:/rhome/remotouser0
```

因此排错时应分开问：master entry 是否被加载，子 map 是否可读，目标 key 是否存在或能被 wildcard 匹配，远端 location 是否正确。

② [知识点] key 是访问路径与 map 条目之间的匹配对象

对 indirect map，key 通常是一个目录组件。master map 中的父路径为 `/rhome`，key 为 `remotouser0`，最终触发路径就是：

```
/rhome/remotouser0
```

访问 `/rhome/remotouser0/project/file` 时，autofs 首先用 `remotouser0` 查找 map，然后把剩余子路径交给已经挂载的远端文件系统。把完整路径 `/rhome/remotouser0` 错写成 indirect key，会使请求的 key 与 map 不匹配。

对 direct map，key 则是完整绝对路径。本章后面会单独建立 direct/indirect 的比较模型。

③ [知识点] 触发层与载荷层必须分开观察

服务加载 indirect map 后，系统会建立一个 `autofs` 类型的触发文件系统。它负责拦截对 key 的访问，但不等于远端 NFS 已经挂载。触发前可能出现以下状态：

```
findmnt -t autofs      # 能看到受管理的触发层
findmnt -t nfs,nfs4    # 看不到目标远端载荷
```

对精确 key 执行 `stat /rhome/remotouser0` 后，才应在当前挂载关系中看到对应 NFS 源。空闲超时后，NFS 载荷可以消失；只要触发层与 map 仍有效，再次访问就应重新挂载。

④ [知识点] 状态链比一条“成功命令”更重要

本章采用以下状态链：

```
配置文件存在
→ master 与子 map 可解析
→ autofs.service 当前运行
→ 触发层已建立
→ 精确 key 能触发
→ 当前载荷源正确
→ 目标身份完成所需读写
→ 空闲后可卸载
→ 再次访问可恢复
```

`automount -m` 只能推进到“map 可解析”；`systemctl is-active` 只能证明 daemon 当前运行；`findmnt -T` 只能证明当前路径对应哪个已挂载文件系统；只有按题目身份完成访问，才能证明业务功能层。

⑤ [知识点] idle、busy 与 expire 构成自动卸载模型

超时并不是从“配置完成”开始计算，而是与载荷的实际使用和空闲状态有关。如果进程的当前工作目录位于挂载中、文件仍被打开、或存在其他活动引用，载荷可能保持 busy，不能在预期时间卸载。此时“超过 60 秒仍存在”不应立即解释为 timeout 配置无效，应先确认是否真正空闲。

超时卸载的价值不是追求每次都立即消失，而是让不再使用的网络挂载能够自然释放。考试和日常验证时，先关闭使用该目录的 Shell 或程序，再等待超时并查询当前 NFS 载荷；不要默认使用强制卸载破坏仍在使用的进程。

[Cheatsheet] master map 定义入口，子 map 定义 key 与载荷；autofs 触发层不等于 nfs 载荷；active、可解析、已触发和可读写要分层验证；超时前确保没有活动引用。

[知识专题] indirect 与 direct：路径是怎样由 key 形成的

两类 map 使用同样的“key -options location”语法，但 key 的含义不同。indirect map 适合在同一个父目录下管理一组同类路径，例如用户家目录或项目目录；direct map 适合把若干完整路径直接映射到远端源。最容易出现的错误是把 direct 的绝对路径写法放进 indirect map，或在 master map 中忘记 direct map 必须使用 `/-`。

① [知识点] indirect map 使用“父挂载点 + 相对 key”

示例：

```
# /etc/auto.master.d/projects.autofs
/projects /etc/auto.projects --timeout=60

# /etc/auto.projects
alpha -fstype=nfs,rw files.example.com:/exports/projects/alpha
beta -fstype=nfs,rw files.example.com:/exports/projects/beta
```

最终路径分别是：

```
/projects/alpha
/projects/beta
```

子 map 中只写 `alpha` 和 `beta`，不写 `/projects/alpha`。master mount point 与 key 的拼接关系就是 indirect map 的核心。

② [知识点] direct map 使用 `/-` 与完整绝对路径 key

示例：

```
# /etc/auto.master.d/direct.autofs
/- /etc/auto.direct --timeout=60

# /etc/auto.direct
/srv/docs -fstype=nfs,rw files.example.com:/exports/docs
/opt/catalog -fstype=nfs,ro files.example.com:/exports/catalog
```

`/-` 不是实际目录，也不是要访问的路径；它是 master map 中的 direct map 标记。真正的本地挂载点写在子 map 的 key 中。direct map 可以在文件系统层次的不同位置管理多个完整路径。

③ [知识点] map 一行包含 key、选项和 location

子 map 基本形式是：

```
key -options location
```

对于 NFS：

```
key -fstype=nfs,rw host:/export/path
```

选项字段以 `-` 开头，多个选项以逗号分隔。`location` 使用 NFS 的 `host:/path` 表示法。NFS 是 `autofs` 常见默认类型，但在考试答案和教学示例中显式写 `-fstype=nfs,rw` 能减少对象歧义；题目若指定只读或其他挂载选项，应按要求替换，而不是机械复制 `rw`。

④ [知识点] master map 选项与子 map 选项作用层次不同

master entry 的选项通常影响整个 map，例如：

```
/projects /etc/auto.projects --timeout=60
```

子 map 选项描述具体载荷，例如：

```
alpha -fstype=nfs,rw files.example.com:/exports/projects/alpha
```

不要把 `--timeout=60` 写进 NFS mount options，也不要把 `-fstype=nfs,rw` 写成 master map 的文件路径。阅读配置时先按字段位置判断作用对象。

⑤ [知识点] /etc/auto.master.d/*.autofs 适合放本地扩展入口

RHEL 9 的课程环境常在 `/etc/auto.master.d/` 中创建以 `.autofs` 结尾的 master map 片段，例如：

```
/etc/auto.master.d/rhome.autofs
```

这样能把本地任务与主文件中的 `vendor` 或站点配置分开。文件扩展名错误、路径拼错或子 map 文件不存在，都会使预期 entry 不进入有效配置。无论写入 `/etc/auto.master` 还是 drop-in，最终都要用 `automount -m` 确认实际加载结果，而不是只相信编辑器中的内容。

⑥ [比较点] 选择 direct 还是 indirect 的判断方法

优先问本地路径是否共享一个自然父目录：

- `/rhome/alice`、`/rhome/bob`、`/rhome/carol`：适合 indirect map；
- `/srv/docs` 与 `/opt/catalog`：没有共同业务父目录，适合 direct map；
- 题目明确给出“在 `/base/key` 下按 key 挂载”：使用 indirect；
- 题目要求几个任意绝对路径分别按需挂载：使用 direct。

选择 map 类型是在表达本地路径模型，而不是在选择 NFS 版本。

[Cheatsheet] indirect：master point + relative key；direct：master 使用 `/-`，子 map key 为完整路径；子 map 行是 `key -options location`；`.autofs` 片段必须由有效 master 配置加载。

[操作专题] 配置一个可验证的 indirect NFS 自动挂载

本专题建立考试中最常见的完整流程：远端 NFS 已由第 26 章准备，本机需要把用户或项目目录按 key 自动挂载。操作不能从直接写配置开始，应先确认不会与已有 entry 冲突，再建立 master 和子 map，静态展开、加载、精确触发，并用目标身份完成功能验证。

① [操作] 调查现状并确认前置对象

作用对象：本机已有 `autofs` 配置、服务状态和目标路径。

基本语义：先建立基线，避免重复 master entry、覆盖站点配置或把远端问题误当成本地 map 问题。

```
rpm -q autofs nfs-utils
systemctl is-active autofs
systemctl is-enabled autofs
automount -m
findmnt -t autofs
findmnt -t nfs,nfs4
ls -ld /rhome
```

需要 NFS 服务器名称时，可使用第 26 章的名称解析和远端源验证方法。这里不要求为了配置 `autofs` 先做一个永久手工挂载；若为了诊断临时挂载，完成后必须卸载，最终终态仍由 `autofs` 提供。

② [操作] 安装客户端组件并创建 master 片段

作用对象：`autofs` 和 NFS 客户端软件包、master map 入口。


```
dnf install autofs nfs-utils

cat > /etc/auto.master.d/rhome.autofs <<'EOF'
/rhome /etc/auto.rhome --timeout=60
EOF
```

这里显式设置 60 秒仅用于任务与练习，使超时验证可控；不要把它推断为所有系统的默认值。若目标目录已经存在，应先检查其中是否有本地文件。挂载后这些内容可能被覆盖在载荷下面，不能用“目录原来是空的”作为无条件假设。

③ [操作] 创建具体 key 或 wildcard 子 map

只管理一个用户时：

```
cat > /etc/auto.rhome <<'EOF'
remoteuser0 -fstype=nfs,rw serverb.lab.example.com:/rhome/remoteuser0
EOF
```

管理同名用户目录族时：

```
cat > /etc/auto.rhome <<'EOF'
* -fstype=nfs,rw serverb.lab.example.com:/rhome/&
EOF
```

map 文件应保持普通文本格式。不要在字段之间混入全角空格，不要把 shell 引号写进最终 map，不要把整条配置当成命令执行。

④ [操作] 用 `automount -m` 做静态展开

作用对象：当前可读取的 master map 与子 map。

```
automount -m
```

应在展开结果中确认：

```
master mount point: /rhome
map source: /etc/auto.rhome
key: remoteuser0 或 wildcard
location: serverb.lab.example.com:/rhome/...
timeout: 与任务要求一致
```

此时还不能声明远端已挂载。`automount -m` 不测试远端服务器是否可达，也不证明用户具有写权限；它的主要价值是尽早发现 master 未加载、子 map 不存在、字段结构错误和 key 写错。

⑤ [操作] 首次启用或重新加载服务

首次部署：

```
systemctl enable --now autofs.service
```

服务已经运行且修改了配置：

```
systemctl reload autofs.service
```

随后分别检查：

```
systemctl is-active autofs.service
systemctl is-enabled autofs.service
findmnt -t autofs
```

`active` 与 `enabled` 是两个维度。reload 成功也只说明 daemon 接受了重新读取请求，仍需触发目标 key。

⑥ [操作] 精确触发并核对当前源

不要只执行 `ls /rhome`。父目录是否显示潜在 key 受 browse 行为和 map 类型影响；直接访问精确路径更可靠：

```
stat /rhome/remoteuser0
# 或
ls -ld /rhome/remoteuser0
```

触发后查询：

```
findmnt -T /rhome/remotouser0
findmnt -t nfs,nfs4
```

核对的不是“有一条 NFS”即可，而是本地目标、远端主机和导出路径都与题目一致。

⑦ [验证] 用题目身份验证读写，而不是用 root 代替

```
sudo -u remotouser0 test -r /rhome/remotouser0
sudo -u remotouser0 touch /rhome/remotouser0/.autofs-client-test
sudo -u remotouser0 rm /rhome/remotouser0/.autofs-client-test
```

如果题目只要求只读，则不要创建文件；使用对应身份读取一个已知对象。root 写成功不能证明普通用户可写，普通用户失败也不能靠本地 `chmod 777` 自动修复远端导出和 UID/GID 问题。

⑧ [验证] 检查空闲卸载与重新触发

退出位于目标目录中的 Shell，关闭引用该目录的程序，然后等待超过显式 timeout：

```
cd /
# 等待超过任务设定的空闲超时
findmnt -t nfs,nfs4
```

对应载荷消失后，再次执行：

```
stat /rhome/remotouser0
findmnt -T /rhome/remotouser0
```

再次出现正确源，才证明“按需挂载—空闲卸载—再触发”的生命周期闭环。静态环境中只能给出推荐验证步骤，不能声称已经观察到实际时间行为。

[Cheatsheet] 基线 → master drop-in → 子 map → `automount -m` → enable/reload → `stat` 精确触发 → `findmnt -T` → 目标身份读写 → 空闲后再触发。

[知识专题] wildcard 与 &：让请求 key 进入远端路径

逐个列出所有用户或项目 key 并不总是可维护。当本地 key 与远端末级目录同名，并且所有条目使用相同选项时，可以用 wildcard map 表达一个规则。* 与 & 的关系很简单，但要记住它只是字符串匹配与替换，不会预先检查远端目录是否存在。

① [知识点] * 匹配未被具体条目定义的请求 key

```
* -fstype=nfs,rw serverb.lab.example.com:/rhome/&
```

访问 `/rhome/remotouser0` 时，请求 key 是 `remotouser0`，wildcard 条目被选中。访问 `/rhome/alice` 时，请求 key 变成 `alice`。wildcard 让一个 map 覆盖同构目录族，但也扩大了可尝试的 key 范围。

② [知识点] & 在 location 中代入实际 key

对请求 key `remotouser0`，上面的 location 解析为：

```
serverb.lab.example.com:/rhome/remotouser0
```

& 不是 shell 变量，不需要 \$，也不是引用 master mount point。它表示当前匹配的 key。把远端位置写成固定 `/rhome/remotouser0` 会使所有请求都指向同一目录；漏写 & 则失去同名映射能力。

③ [边界] wildcard 不证明远端同名目录真实存在

`automount -m` 能显示 wildcard 规则，却不会为每个可能的 key 访问远端服务器。访问一个不存在的 key 时，触发可能失败，并且失败查找可能被短暂缓存。诊断时应分别检查：请求 key 是否正确、替换后的远端路径是否存在、NFS 服务器是否允许客户端访问。

④ [比较点] 具体 key 与 wildcard 的使用边界

只有一个考试用户时，具体 key 更直观：

```
remotouser0 -fstype=nfs,rw serverb:/rhome/remotouser0
```

题目明确需要“所有同名用户目录”或希望表达可扩展规则时，wildcard 更合适：

```
* -fstype=nfs,rw server:/rhome/&
```

不要为展示技巧而把一个固定、异名映射强行改为 wildcard；map 应忠实表达题目对象关系。

[Cheatsheet] * 捕获请求 key，& 代入该 key；规则可解析不代表远端目录存在；具体对象用具体 key，同名目录族再用 wildcard。

[操作专题] 用 direct map 管理任意绝对路径

当需要按需挂载的本地路径分散在不同父目录下时，建立多个只有一个 key 的 indirect map 会增加结构复杂度。direct map 允许在一个子 map 中直接列出完整本地路径。它的核心不是“更高级”，而是 key 的路径语义发生了变化。

① [操作] 建立 direct master entry

```
cat > /etc/auto.master.d/direct.autofs <<'EOF'
/- /etc/auto.direct --timeout=60
EOF
```

/- 必须出现在 master map 的 mount-point 字段。不要把实际路径 /srv/docs 写在这一字段并同时在 direct map 中再次写完整路径；那会混合两种模型。

② [操作] 在子 map 中使用完整绝对路径 key

```
cat > /etc/auto.direct <<'EOF'
/srv/docs -fstype=nfs,rw files.example.com:/exports/docs
/opt/catalog -fstype=nfs,ro files.example.com:/exports/catalog
EOF
```

确保父目录层次不会与已有关键本地数据冲突。automounter 可以创建需要的挂载目录，但在生产变更前仍应调查目标路径是否已经被其他文件系统、服务或应用使用。

③ [操作] 展开、reload 与逐路径触发

```
automount -m
systemctl reload autofs.service

stat /srv/docs
findmnt -T /srv/docs

stat /opt/catalog
findmnt -T /opt/catalog
```

一个 direct map 中的两个 key 是两个独立触发对象；第一个成功不能证明第二个 location 也正确。

④ [诊断点] direct map 修改后优先显式 reload

上游 autofs 语义要求 direct map 和 master map 的变化刷新 daemon。为保证考试和工作流程确定性，本章对 direct map 采用：

```
修改 → automount -m → systemctl reload autofs → 精确访问 → findmnt
```

不要依赖“下次操作也许自动识别”的模糊预期。若 entry 当前正在使用，某些变更可能要等旧载荷过期后才完全体现；先退出引用路径的进程，再进行最小变更和复验。

⑤ [比较点] direct 与 indirect 的错误输出判断

若 master 使用 /projects /etc/auto.projects，子 map 的 key 应是 alpha；若 master 使用 /- /etc/auto.direct，子 map 的 key 应是 /projects/alpha。看到“map 能展开但访问 /projects/alpha 无载荷”时，首先比较 master 类型与 key 形式，而不是直接修改远端服务器。

[Cheatsheet] direct master entry 固定用 /-；子 map key 写完整绝对路径；每个路径独立触发；direct/master 变化采用显式 reload。

[操作专题] reload、restart、timeout 与 busy：控制配置和生命周期

服务控制不是“改完配置一律 restart”。reload 的目标是让运行中的 automounter 重新比较 master map、刷新相应 map，并尽量保留仍在使用的状态；restart 会停止并重新启动服务，影响更大。超时则决定空闲载荷何时尝试卸载。把三者分开，才能在不扩大影响的情况下完成变更。

① [操作] 首次部署建立当前与持久状态

```
systemctl enable --now autofs.service
systemctl is-active autofs.service
systemctl is-enabled autofs.service
```

`enable --now` 同时请求当前启动和开机启用，但验证仍要分开。服务 `active` 只说明 `automount daemon` 正在运行，不说明每个 `map`、`key` 和远端 `location` 都正确。

② [操作] 修改配置后优先使用 `reload`

```
automount -m
systemctl reload autofs.service
systemctl status autofs.service --no-pager
```

普通 `indirect map` 的变更通常可以在后续查找中被识别，但 `master map`、`direct map` 以及影响浏览行为的变更需要显式 `reload`。为了形成统一、可复现的考试流程，本章在修改任何本地 `map` 后都建议先静态展开再 `reload`。

③ [边界] `restart` 是更强操作，不是解析错误的替代品

```
systemctl restart autofs.service
```

`restart` 可用于 `daemon` 状态异常、课程步骤明确要求，或 `reload` 无法建立预期管理关系时。执行前应确认当前载荷和使用者，因为停止服务可能与 `busy mount` 发生冲突。若 `automount -m` 已经显示语法错误，`restart` 只会让错误再次加载，不会修复配置。

④ [参数] 在 `master entry` 中显式设置 `timeout`

```
/rhome /etc/auto.rhome --timeout=60
```

`timeout` 以秒为单位描述空闲载荷尝试卸载前的时间。上游程序默认值与发行版安装配置可能不同，因此任务需要可观察超时时应显式设置，不制作“所有 RHEL 9 必定为某个默认秒数”的背诵结论。

将 `timeout` 设为 0 会禁用自动卸载。这是配置边界，不是普通考试答案：若题目要求“空闲后自动卸载”，0 显然不满足终态。

⑤ [诊断点] `busy` 载荷不会因计时到达而安全消失

超时未卸载时，先检查最常见的活动引用：

```
pwd
findmnt -T /rhome/remoteuser0
fuser -vm /rhome/remoteuser0
```

另一个 Shell 的当前目录、打开文件或后台进程都可能保持 `busy`。先让使用者退出或关闭文件，再重新等待；不要在没有调查时使用强制卸载。强制解除仍被进程使用的挂载可能造成 I/O 错误或业务异常。

⑥ [验证] `reload/restart` 后仍要回到功能证据

无论控制命令是否成功，最终都要执行：

```
automount -m
→ systemctl is-active/is-enabled
→ findmnt -t autofs
→ stat 精确 key
→ findmnt -T 目标路径
→ 目标身份读写
```

服务控制结果只是中间证据。把 `systemctl reload` 返回成功直接写成“自动挂载完成”，属于典型的终态扩大。

[Cheatsheet] 首次部署 `enable --now`；修改后“`automount -m` + `reload`”；`restart` 影响更强；`timeout` 管空闲载荷；未卸载先查 `busy`，不默认强制。

[操作专题] 建立配置、触发、载荷与功能的证据矩阵

验证的目标不是堆命令，而是让每条命令回答一个不同问题。`autofs` 尤其容易出现“触发前无 NFS 结果是正常的”“触发层存在但载荷失败”“载荷成功但目标用户不能写”等看似矛盾的状态。下面的矩阵给出一条从静态到动态、从本机到远端功能的验收路径。

① [验证点] automount -m 证明有效配置视图

```
automount -m
```

检查 master mount point、map 文件、key、选项、location 和 timeout。它不能证明：

- autofs.service 当前已经应用该配置；
- 服务器名称可解析；
- NFS 导出允许本机访问；
- 目标用户具有读写权限。

因此它是第一层，而不是最终层。

② [验证点] systemctl 分开回答 current 与 persistent

```
systemctl is-active autofs.service
systemctl is-enabled autofs.service
systemctl status autofs.service --no-pager
```

`is-active` 回答 daemon 现在是否运行，`is-enabled` 回答系统启动时是否配置为启用。`status` 适合查看最近状态和错误摘要，但不能替代 map 展开与功能访问。

③ [验证点] findmnt -t autofs 观察触发层

```
findmnt -t autofs
```

它用于确认 automounter 已为 master entry 建立管理层。看到 `/rhome` 的 `autofs` 触发层不等于 `/rhome/remotouser0` 的 NFS 已经挂载；反过来，目标载荷超时消失后，触发层仍可能保留。

④ [验证点] 精确访问触发 key

```
stat /rhome/remotouser0
# 或
ls -ld /rhome/remotouser0
```

精确访问比只列父目录更有区分度。`ls /rhome` 是否展示未触发 key 可能受 `browse` 行为影响，不能把父目录为空直接判为 map 错误。

⑤ [验证点] findmnt -T 核对目标路径的真实来源

```
findmnt -T /rhome/remotouser0
```

触发后，应对 TARGET、SOURCE、FSTYPE 和 OPTIONS。`findmnt -T` 会解析给定路径，可能本身引发路径查找，因此不要把它作为“绝对不会触发”的前置观察命令；当需要明确触发时，先使用 `stat`，再把 `findmnt -T` 用于源关系验收。

⑥ [验证点] 当前 NFS 列表用于比较触发前后和超时后

```
findmnt -t nfs,nfs4 -o TARGET,SOURCE,FSTYPE,OPTIONS
```

同一任务可以记录三个时刻：

```
触发前：目标载荷尚不存在
触发后：目标载荷与远端源出现
空闲后：目标载荷消失
再次访问：目标载荷重新出现
```

不要要求文件系统类型一定显示为 `nfs` 而不是 `nfs4`，除非题目明确指定版本；关键是源、目标和功能正确。

⑦ [验证点] ls /stat 只能证明路径访问，目标身份才证明权限

`stat` 成功说明路径可解析并能读取元数据，但不一定证明题目身份能创建文件。完整功能测试应按要求使用实际用户：

```
sudo -u remotouser0 test -r /rhome/remotouser0
sudo -u remotouser0 test -w /rhome/remotouser0
```

若需要创建测试文件，使用唯一、可清理的名称，并在验证后删除。不要假设 `root` 的结果代表普通用户。

⑧ [验证点] journal 是运行证据，不是孤立的“答案生成器”

```
journalctl -u autofs.service -b --no-pager
```

日志适合确认 reload、key lookup、mount、expire 和错误消息的时间顺序。先用 map、服务和挂载证据缩小层次，再读相应时间段；不要从一条模糊日志直接跳到关闭 SELinux、修改服务端权限或重装软件。

[Cheatsheet] automount -m 看配置；is-active/is-enabled 看服务两维；findmnt -t autofs 看触发层；stat 触发；findmnt -T 核对源；目标身份验证功能；journal 解释运行过程。

[诊断专题] 路径存在却没有挂载：按证据定位失败层

“访问失败”并不是一个根因。autofs 的故障至少可落在配置加载、key 匹配、触发、远端 NFS、身份权限和过期回收六层。稳定诊断遵循：症状 → 当前证据 → 假设 → 下一条最有区分度的证据 → 最小修复 → 再验证。

① [诊断点] automount -m 中没有目标 master entry

症状：配置文件已经创建，但有效 map 中看不到 /rhome 或 /- entry。

当前假设：.autofs 后缀错误、文件放错目录、master 片段语法错误、map 路径拼错或 entry 被其他配置冲突。

下一条证据：核对文件名、内容和 automount -m 的错误信息：

```
ls -l /etc/auto.master /etc/auto.master.d
sed -n '1,120p' /etc/auto.master.d/rhome.autofs
automount -m
```

最小修复：只修正入口文件名或对应字段，重新展开并 reload；不要删除所有已有 autofs 配置。

② [诊断点] master 可见，但具体 key 不出现或不触发

症状：/projects 已由 autofs 管理，访问 /projects/alpha 没有对应 NFS 载荷。

当前假设：indirect key 错写成完整路径、key 拼写错误、wildcard 规则不正确或子 map 未被读取。

下一条证据：比较 master 类型、请求 key 与子 map 第一字段：

```
automount -m
sed -n '1,120p' /etc/auto.projects
stat /projects/alpha
journalctl -u autofs.service -b --no-pager
```

若 master 是 /projects /etc/auto.projects，正确 key 应为 alpha，不是 /projects/alpha。

③ [诊断点] 目录存在，但 findmnt 没有远端载荷

症状：ls -ld /rhome 成功，用户据此认为已经挂载。

判断：目录存在只证明路径节点存在。它可能是普通本地目录，也可能是 autofs 触发层，或只是载荷超时后留下的可触发入口。

下一条证据：

```
findmnt -t autofs
findmnt -t nfs,nfs4
stat /rhome/remotouser0
findmnt -T /rhome/remotouser0
```

不要在没有确认载荷时向该目录写大量数据，否则可能写入本地根文件系统或触发错误位置。

④ [诊断点] 服务 active，但触发时报远端或权限错误

症状：autofs.service active，map 可展开，精确访问 key 报错。

判断：本地 daemon 已运行不代表 NFS location 可用。根据错误转入第 26 章的前置对象：名称解析、路由、远端导出、NFS 服务、来源限制和 UID/GID 权限。

下一条证据：核对 map 展开的实际 host:/path，再使用第 26 章的 NFS 证据。不要通过客户端挂载点 chmod 777 修复服务端拒绝；客户端目录权限不能改变服务器导出策略。

⑤ [诊断点] wildcard 中某个用户失败，其他用户成功

症状：/rhome/alice 可触发，/rhome/bob 失败。

当前假设：master、服务和 wildcard 总体机制基本成立，差异更可能位于替换后的远端目录或用户身份。

下一条证据：明确 `&` 对每个请求形成的远端路径，检查失败 key 的 journal 与 NFS 源。不要重写整个 master map；先修正单个远端对象或题目身份。

⑥ [诊断点] 修改配置后仍表现为旧关系

症状：文件内容已经更新，但触发仍指向旧源。

当前假设：修改的是错误文件、daemon 尚未 reload、direct/master 变更未刷新、旧载荷仍在使用，或失败 key 存在短暂负缓存。

下一条证据：

```
automount -m
systemctl reload autofs.service
findmnt -T /目标路径
journalctl -u autofs.service -b --no-pager
```

先退出使用旧载荷的进程，让其自然过期，再重新触发。不要在不知道引用者的情况下强制卸载。

⑦ [诊断点] 超过 timeout 仍未卸载

症状：显式设置 60 秒，等待更久后 NFS 载荷仍存在。

当前假设：目录并未空闲、Shell 当前目录在载荷中、文件仍打开、后台进程在扫描，或 timeout 未进入有效 master entry。

下一条证据：

```
automount -m
findmnt -T /目标路径
fuser -vm /目标路径
```

最小修复是释放活动引用或修正有效 timeout，然后重新计时；不是修改成更短时间并反复 restart。

⑧ [诊断点] SELinux 或其他安全证据出现时不关闭保护机制

若 journal 或审计日志出现访问控制证据，保留当前 map、源、身份和错误时间，转入第 28 章的 AVC 分析。关闭 SELinux 只能改变症状，不能证明策略层根因，也会破坏考试和真实工作的安全边界。

[Cheatsheet] 先问 entry 是否有效、key 是否匹配、是否真的触发、源是否正确、身份是否有权、载荷是否 busy；每次只修最小层，再从 `automount -m` 开始复验。

[知识专题] 从考试配置迁移到日常运维

考试环境通常只有少量主机和明确路径，真实系统则可能已有站点 master、LDAP/SSSD map、数百个用户目录和长时间运行的进程。工作迁移的关键不是引入更多命令，而是保持对象和证据可审计：变更前保存有效 map，使用独立 drop-in，控制 wildcard 范围，明确 timeout，并把配置正确与远端业务可用分开监控。

① [工作迁移] 变更前保存有效配置视图

在生产变更单中同时记录：

```
automount -m
systemctl status autofs.service --no-pager
findmnt -t autofs
findmnt -t nfs,nfs4
```

这样可区分“文件被改了”与“daemon 实际加载了什么”。对已有站点配置，不应直接清空 `/etc/auto.master`；使用独立 `.autofs` 片段，并检查同一 indirect mount point 是否已经定义。

② [工作迁移] wildcard 是扩展能力，也是故障范围

wildcard 可减少条目数量，但任何拼错 key 都会尝试形成一个远端路径。在用户规模较大时，应配合服务端目录生命周期、身份来源和监控设计，而不是把所有未知路径都当作正常。对少量异构共享，显式 key 通常更清楚。

③ [工作迁移] 监控应观察“能否触发”和“是否长期卡住”

只监控 autofs.service active 容易漏掉远端失败。更有价值的检查包括：选定代表 key 的只读访问、当前源是否正确、触发耗时、mount 失败日志，以及载荷是否长期保持 busy。监控脚本必须避免高频访问导致挂载永远不空闲。

④ [帮助入口] 从机制到参数逐层查找

```
man 5 auto.master
man 5 autofs
man 8 automount
man 8 autofs
systemctl help autofs.service
```

`auto.master(5)` 解释 master entry 和 direct/indirect 路径；`autofs(5)` 解释子 map 格式、wildcard 与 location；`automount(8)` 解释 daemon 和参数；`autofs(8)` 解释服务控制及 reload 行为。

[Cheatsheet] 生产变更保存有效视图；drop-in 优于清空主文件；wildcard 控制范围；监控真实触发而非只看 active；机制疑问回到四个手册入口。

[经典任务] 任务一：按用户 key 自动挂载远端家目录

环境与当前状态

你在示例客户端 `servera` 上工作。第 26 章已经确认远端 NFS 服务可用，并提供：

```
serverb.lab.example.com:/rhome/remoteuser0
```

本机已经存在用户 `remoteuser0`，其家目录应为：

```
/rhome/remoteuser0
```

当前没有为 `/rhome` 配置本题 `autofs` map。目标路径不应写入 `/etc/fstab`，也不得通过普通手工 NFS 挂载冒充终态。

目标终态

1. 安装并启用需要的客户端组件；
2. 使用 `/etc/auto.master.d/rhome.autofs` 注册 indirect map；
3. 使用 `/etc/auto.rhome` 建立同名用户目录 wildcard 规则；
4. 挂载选项包含 `rw`，显式 timeout 为 60 秒；
5. 访问 `/rhome/remoteuser0` 时挂载 `serverb.lab.example.com:/rhome/remoteuser0`；
6. `autofs.service` 当前运行且开机启用；
7. `remoteuser0` 能按题目要求读写；
8. 目录空闲后载荷可以卸载，再次访问能够重新触发。

限制条件

- 不修改远端 `/etc/exports`；
- 不使用 `chmod 777`；
- 不关闭 SELinux 或防火墙；
- 不删除现有站点 `autofs` 配置；
- 不把 `systemctl` 成功作为最终验收；
- 不编造命令输出，按当前系统实际证据判断。

验收证据

```
配置: automount -m
服务当前: systemctl is-active autofs
服务持久: systemctl is-enabled autofs
触发层: findmnt -t autofs
触发: stat /rhome/remoteuser0
当前源: findmnt -T /rhome/remoteuser0
功能: 以 remoteuser0 验证读写
生命周期: 空闲后载荷消失，再次访问恢复
```

请先独立完成。参考解答从下一页开始。

[参考解答] 任务一：调查、配置与分层验收

① [调查] 建立变更前基线

```
rpm -q autofs nfs-utils
automount -m
systemctl is-active autofs.service
systemctl is-enabled autofs.service
findmnt -t autofs
findmnt -t nfs,nfs4
getent hosts serverb.lab.example.com
ls -ld /rhome
```

若 `/rhome` 已被其他 master entry 管理，不应再创建重复 indirect entry；先调查站点配置并在现有 map 中做最小修改。NFS 导出与身份问题归第 26 章，本题不修改服务器端。

② [操作] 安装组件并创建 master 片段

```
dnf install autofs nfs-utils

cat > /etc/auto.master.d/rhome.autofs <<'EOF'
/rhome /etc/auto.rhome --timeout=60
EOF
```

③ [操作] 创建 wildcard 子 map

```
cat > /etc/auto.rhome <<'EOF'
* -fstype=nfs,rw serverb.lab.example.com:/rhome/&
EOF
```

访问 key `remoteuser0` 时，`&` 被替换为 `remoteuser0`，得到题目指定的远端源。

④ [静态验证] 先确认 map 关系

```
automount -m
```

确认 `/rhome`、`/etc/auto.rhome`、wildcard、location 和 timeout 均位于有效视图。若这一层错误，不进入服务 restart 猜测。

⑤ [加载] 建立服务当前与持久状态

```
systemctl enable --now autofs.service
systemctl reload autofs.service
systemctl is-active autofs.service
systemctl is-enabled autofs.service
findmnt -t autofs
```

首次 `enable --now` 后再 reload 并非绝对必要，但在参考解答中用于明确“配置已经按最终内容重新读取”。实际环境可在确认首次启动发生于配置写入之后时省略重复 reload。

⑥ [触发与当前源] 精确访问 key

```
stat /rhome/remoteuser0
findmnt -T /rhome/remoteuser0
findmnt -t nfs,nfs4 -o TARGET,SOURCE,FSTYPE,OPTIONS
```

核对远端源必须是 `serverb.lab.example.com:/rhome/remoteuser0`，而不是仅看到任意 NFS 条目。

⑦ [功能] 使用目标身份

```
sudo -u remoteuser0 test -r /rhome/remoteuser0
sudo -u remoteuser0 test -w /rhome/remoteuser0
sudo -u remoteuser0 touch /rhome/remoteuser0/.autofs-client-test
sudo -u remoteuser0 rm /rhome/remoteuser0/.autofs-client-test
```

若失败，保存错误和源关系，调查远端所有者、UID/GID 与导出策略；不在客户端挂载点使用 `chmod 777`。

⑧ [生命周期] 空闲、卸载与再触发

```
cd /
# 确认没有 Shell 或程序继续使用目标路径, 并等待超过 60 秒
findmnt -t nfs,nfs4 -o TARGET,SOURCE

stat /rhome/remotouser0
findmnt -T /rhome/remotouser0
journalctl -u autofs.service -b --no-pager
```

静态核对环境无法承诺真实环境一定在精确第 60 秒完成卸载; 若仍存在, 先使用 `fuser -vm` 查活动引用。

典型错误

- `master` 写成 `/rhome/remotouser0/etc/auto.rhome`, 导致路径模型错误;
- `indirect` 子 `map key` 写成完整 `/rhome/remotouser0`;

典型错误 (续)

- `.autofs` 文件扩展名或子 `map` 路径拼错;
- 只执行 `ls /rhome`, 没有精确访问 `key`;
- `automount -m` 成功后直接宣布任务完成;
- `root` 能写就认为 `remotouser0` 一定能写;
- 为解决远端权限失败执行 `chmod 777` 或关闭 SELinux;
- Shell 停留在目标目录中却期待 `timeout` 卸载。

[经典任务] 任务二: `map` 可见但 `/projects/alpha` 不触发

已知证据

```
autofs.service 为 active 且 enabled
automount -m 能看到: /projects → /etc/auto.projects
findmnt -t autofs 能看到 /projects
访问 /projects/alpha 后没有对应 NFS 载荷
远端源 files.example.com:/exports/projects/alpha 已由第 26 章确认可用
```

当前 `/etc/auto.projects` 为:

```
/projects/alpha -fstype=nfs,rw files.example.com:/exports/projects/alpha
```

目标

在不修改远端服务器、不删除其他 `autofs` 配置、不使用普通手工挂载替代的前提下, 定位并修复最小错误, 使 `/projects/alpha` 能按需挂载, 随后用配置、触发、当前源和超时再触发完成验收。

请先独立完成。参考解答从下一页开始。

[参考解答] 任务二: 比较 `master` 类型与 `key` 形式

① [调查] 利用已有证据缩小范围

服务 `active`、`master entry` 可见且 `/projects` 触发层存在, 说明软件包、`daemon` 与 `master point` 大体成立。远端源也已独立确认, 下一条最有区分度的证据不是 `restart`, 而是比较 `indirect` 路径模型:

```
master mount point: /projects
请求路径: /projects/alpha
请求 key: alpha
当前 map key: /projects/alpha
```

当前子 `map` 把完整路径写成了 `indirect key`, 因此 `key` 不匹配。

② [最小修复] 只改第一字段

```
cat > /etc/auto.projects <<'EOF'
alpha -fstype=nfs,rw files.example.com:/exports/projects/alpha
EOF
```

③ [加载与再验证]

```
automount -m
systemctl reload autofs.service
stat /projects/alpha
findmnt -T /projects/alpha
journalctl -u autofs.service -b --no-pager
```

确认当前源与目标正确，再按题目身份执行读写测试。若需要验证 timeout，先退出目标目录并清除活动引用，等待后观察载荷，再次 stat 确认能够恢复。

④ [为什么其他修复不合适]

- restart 不会自动把错误 key 改成 alpha；
- 修改远端 /exports 与已知证据矛盾；
- 在 /etc/fstab 中写永久挂载改变了题目生命周期；
- mount -t nfs 只能证明远端可挂载，不能修复 autofs key；
- 删除 /projects 目录不改变 map 匹配规则；
- 关闭安全机制或 chmod 777 与本层错误无关。

[本章收束] 自动挂载的终态是一条可重复触发的证据链

完成 autofs 任务时，不要把目标写成“目录存在”或“服务已经启动”。真正的终态是：路径模型正确，master 与子 map 能进入有效配置视图，daemon 当前运行且按题意持久启用，精确 key 能建立正确载荷，目标身份能够完成所需访问，载荷空闲后可以卸载，并能在下一次访问时重新出现。

工作方法

```
识别 direct / indirect
→ 建立 master 与子 map
→ automount -m 静态展开
→ enable 或 reload
→ stat 精确 key
→ findmnt 核对触发层与载荷源
→ 目标身份功能验证
→ 空闲卸载与再次触发
```

任何一步失败，都先给它定位到 entry、key、trigger、remote、permission 或 busy 层，再选择一条最有区分度的证据。反复 restart、永久手工挂载、全局放宽权限和关闭安全机制，都不能替代对象判断。

主要判断表

看到的证据	可以证明	仍然不能证明
配置文件存在	管理员写入了目标配置	文件已被 master 加载、字段正确
automount -m 能展开 entry	master 与子 map 当前可解析	daemon 已 reload、远端能挂载
autofs.service 为 active	automount daemon 当前运行	指定 key 存在、载荷源正确
findmnt -t autofs 有结果	触发层已经建立	对应 NFS 载荷当前存在
精确 stat /base/key 成功	路径解析与触发至少可继续	当前源一定符合题目、目标用户可写
findmnt -T PATH 源正确	当前路径落在目标载荷上	timeout 正常、重启后仍可用
空闲后 NFS 载荷消失	自动卸载可能正常发生	触发能力仍然有效
再次访问重新出现	按需生命周期可以恢复	所有远端权限与业务语义均正确

向下一章交接

本章只负责 autofs 的路径、map、触发和生命周期。如果当前源已经正确，但访问仍因权限或安全策略失败，应保留 findmnt、目标身份和错误日志证据：NFS 导出、UID/GID 与远端身份问题回到第 26 章；出现 SELinux AVC 时进入第 28 章，用上下文与 AVC 证据继续定位，而不是关闭 SELinux。

静态验证边界：本章依据 RHEL 9 课程、细分课件和手册页完成静态核对；当前会话没有可控制的 RHEL 9 VM，未执行真实 map reload、NFS 触发、busy 引用、timeout 与冷启动测试。

本章阅读导航

先抓住一条主线：DAC 允许只表示传统权限这一层通过；SELinux 还会用进程 domain、目标 type、对象类别和请求权限重新判断。调查时先取得模式和上下文，再用 AVC 把拒绝还原成可验证的访问请求。

- 01区分访问控制层确认 DAC 与 MAC 各自回答什么
- 02识别安全上下文从四段字段找到 domain 与 type
- 03分开当前与持久模式Enforcing、Permissive、Disabled 不混写
- 04建立主体与目标证据用 ps -Z 与 ls -Z 对应真实对象
- 05逐字段读取 AVC提炼 scontext、tcontext、tclass 与权限
- 06先分类再修复把证据交给文件、端口或 Boolean 入口

专题地图

*知识专题*DAC 允许之后，SELinux 怎样判断一次访问

*知识专题*从四段安全上下文识别 domain 与 type

*操作专题*识别当前模式、持久模式与策略状态

*操作专题*用 ps -Z 与 ls -Z 建立主体和目标证据

*知识专题*把一条 AVC 还原成一次访问请求

*操作专题*重新触发、检索并解释近期拒绝

*诊断专题*先证据后修复，并正确使用 Permissive

*经典任务*完成自定义 Web 目录 AVC 取证

*参考解答*从基线、复现到 AVC 分类

*本章收束*把拒绝写成可交接的判断句

阅读时持续回答

1. 普通权限允许后，SELinux 还会检查什么？
2. 进程 domain 和文件 type 从哪里观察？
3. 当前模式与持久模式是否一致？
4. Permissive 成功到底证明了什么？
5. AVC 中谁是主体、谁是目标、目标是什么类别？
6. 没查到 AVC 时，下一条证据是什么？
7. audit2why 能解释什么，不能替你决定什么？
8. 哪些修复必须留给第 29 章？

阅读方法：先区分访问控制层，再核对模式与上下文；只有取得 AVC 后，才把拒绝分类到下一步配置入口。

第 28 章 SELinux 模型、模式、上下文与 AVC 证据

一个服务报告 `Permission denied` 时，最危险的处理方式不是命令写错，而是把所有“拒绝”都看成同一件事。Linux 传统权限可能拒绝，服务自身配置可能拒绝，网络链路可能拒绝，SELinux 也可能在普通权限允许之后继续拒绝。若不先区分证据层，管理员很容易通过放宽权限、切换到 `Permissive`，甚至关闭 SELinux，让症状暂时消失，却没有真正解释系统为什么拒绝访问。

本章解释 SELinux 调查中的第一组核心对象：策略在判断谁访问谁，系统当前以什么模式执行策略，进程和对象当前带有什么上下文，以及一条 AVC 记录能证明什么。主线是“访问模型 → 模式状态 → 上下文证据 → AVC 解读 → 诊断分类”。文件路径的持久类型规则、端口类型和 Boolean 的完整修复操作留到下一章《SELinux 文件规则、端口类型与 Boolean》；`firewalld` 仍属于第 21 章。

概念

DAC 与 MAC 自主访问控制 (DAC) 根据 Linux UID、GID、权限位和 ACL 判断访问；SELinux 提供额外的强制访问控制 (MAC)。DAC 拒绝时，SELinux 不会替它放行；DAC 允许也只能证明传统权限这一层通过，不能证明策略会允许。``ls -l``、``namei -l`` 与 ``getfacl`` 观察 DAC，``ps -Z``、``ls -Z`` 和 AVC 才进入 SELinux 证据层。

概念

Policy SELinux policy 是 domain、type、对象类别、权限、转换和约束的规则集合。它不是 Apache 配置，也不是 `firewalld` 规则；应用把目录或端口改到新位置，不会自动修改 SELinux policy。``sestatus`` 能观察已加载策略名，但具体访问结论仍要由上下文与 AVC 证明。

概念

Domain 进程安全上下文第三段的 type 通常称为 domain，例如 ``httpd_t``。domain 是策略眼中的访问主体身份，不等同于命令名或 Linux 用户；两个都叫 ``httpd`` 的进程也应 ``ps -Z`` 显示的当前上下文为准。

概念

Type 文件、目录、端口和其他对象的安全上下文第三段描述策略用途，例如 ``httpd_sys_content_t`` 或 ``var_t``。文件名和路径字符串不能替代 type；当前 type 可由 ``ls -Z`` 观察，但当前标签正确也不能自动证明持久文件规则存在。

概念

安全上下文常见形式是 ``user:role:type:level``。四段都属于上下文，但 RHCSA 服务故障最常从第三段开始：进程的 type 作为 source domain，目标对象的 type 作为 target type。第四段可能包含 MCS 类别，不能因为其中出现额外冒号就误拆字段。

概念

Enforcing / Permissive / Disabled Enforcing 加载策略并执行拒绝；Permissive 仍加载策略、维护标签并记录本应发生的拒绝，只是不执行拒绝；Disabled 不进入正常策略判断链。Permissive 是受控诊断变量，不是最终修复；当前模式与 ``/etc/selinux/config`` 中的持久计划必须分开验证。

概念

AVC Access Vector Cache 相关审计记录把一次拒绝还原为“主体 domain 对目标 type 的某类对象请求某项权限”。初步阅读优先找 ``{ permission }``、``scontext``、``tcontext`` 和 ``tclass``，再用 PID、命令名、路径和事件序号关联业务动作。AVC 是证据，不是自动修复指令。

操作语义以下六组入口分别回答模式、临时切换、上下文、Audit 检索、辅助解释和 Journal 旁证。先理解命令作用对象，再记关键形式。

getenforce / sestatus

SYNOPSIS

```
getenforce
sestatus [-v] [-b]
```

读取当前 SELinux 模式和状态总览。`getenforce` 只回答当前模式；`sestatus` 还能显示启用状态、加载策略、当前模式与配置文件模式。

重要参数 / 形式

getenforce

快速返回 Enforcing、Permissive 或 Disabled。

sestatus

比较 Current mode 与 Mode from config file。

-v

在系统支持的输出中附加部分文件和进程上下文信息；不是本章默认认证入口。

setenforce

SYNOPSIS

```
setenforce {0|1|Permissive|Enforcing}
```

临时改变已启用 SELinux 的当前执行模式，不修改持久配置。需要足够权限；Disabled 不能靠该命令直接恢复为 Enforcing。

重要参数 / 形式

setenforce 0

当前切到 Permissive，用于受控诊断窗口。

setenforce 1

当前恢复 Enforcing；调查结束必须重新验证。

ps -Z / ls -Z

SYNOPSIS

```
ps -eZ
ps -Z -p PID
ls -Zd PATH
ls -lZ PATH
```

把业务进程和文件系统对象映射到当前 SELinux 上下文。**ps -Z** 观察主体 domain，**ls -Z** 观察目标 type。

重要参数 / 形式

ps -eZ

浏览系统进程及其上下文。

ps -Z -p PID

精确核对目标进程，不用命名替代 domain。

ls -Zd DIR

查看目录对象本身，而不是目录内容。

ls -lZ FILE

同时显示 DAC 属性和文件当前上下文。

ausearch

SYNOPSIS

```
ausearch -m MESSAGE_TYPES -ts START_TIME [OPTIONS]
```

按 Audit 消息类型和时间范围检索拒绝事件。稳定顺序是记录时间、重新触发、查询近期事件，再逐步缩小。

重要参数 / 形式

-m AVC,USER_AVC

选择最常见的 SELinux 拒绝消息类型。

-m AVC,USER_AVC,SELINUX_ERR,USER_SELINUX_ERR

扩展到 SELinux 错误类消息。

-ts recent

从近期时间窗口开始查询；复现时间明确时可使用更精确起点。

-i

解释部分数字字段，适合人工阅读。

--raw

保留原始记录，适合交给后续解析工具。

audit2why

SYNOPSIS

```
audit2why [OPTIONS] < audit-records
```

解释原始 AVC 可能因何被策略拒绝。它提供调查线索，不负责选择最小、安全、持久的修复。

重要参数 / 形式

```
ausearch ... --raw | audit2why
    保留原始 Audit 字段后再解释。
audit2why < saved-avc.log
    对已保存的原始事件离线分析。
```

journalctl

SYNOPSIS

```
journalctl [OPTIONS...] [MATCHES...]
```

读取服务症状、内核消息和可选的 setroubleshoot 摘要。Journal 是旁证入口；Audit 事件仍是 AVC 的主要原始证据。

重要参数 / 形式

```
journalctl -u UNIT --since "-10 min"
    按服务与时间查看症状。
journalctl -t setroubleshoot
    在相应组件存在时读取 SELinux 摘要。
journalctl -k --since "-10 min"
    查看近期内核消息，适合 Audit 链异常时继续取证。
```

[知识专题] DAC 允许之后，SELinux 怎样判断一次访问

SELinux 排错不应从命令清单开始，而应先建立一个足够小、能够解释日志的决策模型。面对任何拒绝，先问四个问题：谁发起访问，目标当前是什么类型，请求了什么行为，目标属于哪一种对象类别。只要能把日志还原成这四个对象，就可以继续判断它属于文件类型、端口类型、Boolean，还是根本不属于 SELinux。

主体进程 domain

+

目标对象 type

+

类别 tclass

+

行为 permission

→

Policy allow / deny

① [知识点] DAC 与 MAC 是两个独立的判断层

DAC 主要依赖 Linux 身份和对象权限，例如：

- 进程以哪个 UID、GID 运行；
- 文件 owner、group 与 mode；
- 路径各级目录是否具备搜索权限；
- 是否存在 POSIX ACL；
- 进程是否具备相关 capability。

SELinux 不替代这些机制，而是在系统中增加一层策略判断。因此：

```
DAC 拒绝 → SELinux 不会把这次访问变成允许
DAC 允许 → 仍需通过 SELinux 策略判断
```

这也是为什么 `chmod 777` 既不是安全答案，也不一定能解决服务访问问题。它最多改变 DAC；若 AVC 显示 `httpd_t` 不允许读取目标类型，SELinux 结论不会因为 mode 变为 `777` 而自动改变。

证据边界

`namei -l`、`getfacl` 或 `ls -l` 能帮助确认 DAC，但不能证明 SELinux 允许；`ls -Z` 能显示对象类型，但不能证明 DAC 允许。排错时应清楚自己正在证明哪一层。

② [知识点] 主体、目标、行为和对象类别构成最小访问请求

SELinux 的一次访问判断可压缩为：

```
source domain
+ target type
+ object class
+ requested permission
→ policy decision
```

以 Web 服务读取文件为例：

```
httpd_t
+ var_t
+ file
+ read
→ allow 或 deny
```

四个部分分别回答：

- `httpd_t`：谁发起访问；
- `var_t`：目标对象当前被策略视为什么用途；
- `file`：目标是普通文件，而不是目录、TCP socket 或其他对象；
- `read`：进程请求读取，而不是写入、执行或绑定端口。

同一个 domain 对不同 type、不同 class 或不同 permission 的结论可能完全不同，所以只说“SELinux 不允许 httpd”过于粗糙。准确表述应接近：“`httpd_t` 对 `var_t:file` 请求 `read` 时被拒绝。”

③ [知识点] domain 与 type 是 Type Enforcement 的工作接口

安全上下文的第三段统一称为 `type`。当它附着在进程上时，通常称为 `domain`；附着在文件、目录、端口等对象上时，通常称为 `type`。这不是两套独立字段，而是从主体与目标两个角度描述 Type Enforcement 对象。

例如：

上下文片段	所属对象	直观含义
<code>httpd_t</code>	httpd 进程	Web 服务进程域
<code>sshd_t</code>	sshd 进程	SSH 服务进程域
<code>httpd_sys_content_t</code>	文件/目录	适合由 httpd 读取的 Web 内容类型
<code>var_t</code>	文件/目录	<code>/var</code> 下较通用的对象类型，通常不是自定义 Web 内容的最终类型

本章只建立 domain/type 与 AVC 的阅读能力。怎样为路径建立持久文件类型规则、怎样应用规则，属于下一章。

④ [知识点] policy 决定允许集合，不等于“应用配置”

SELinux policy 是一组安全规则、类型定义、转换和约束的集合。RHEL 常见安装加载 `targeted` 策略，它重点限制被纳入策略的服务域；某些用户进程可能运行在较宽松的 `unconfined_t` 等域中，但仍然带有 SELinux 上下文。

下面三个“策略”不要混为一谈：

- 应用配置：例如 Apache 的 `DocumentRoot`；
- 网络或防火墙策略：例如 `firewalld zone`；
- SELinux policy：决定 domain 对 type/class/permission 的访问集合。

应用配置把目录改到 `/srv/exam-site`，不会自动向 SELinux policy 声明这个目录应作为 Web 内容。端口改为 82，也不会自动改变 SELinux 端口类型。下一章将处理这些显式配置入口。

⑤ [知识点] 策略允许只是局部结论，业务成功仍需其他证据

即使 SELinux 允许访问，也不能直接推出服务已达到业务终态。完整链路仍可能受到以下因素影响：

```
应用配置
→ 服务进程状态
→ DAC
→ SELinux
→ 监听与路由
→ firewalld
→ 客户端请求与业务响应
```


反过来，某次 `curl` 成功也不能证明 SELinux 持久配置正确：系统可能仍处于 Permissive，或者对象当前标签仅通过一次临时修改得到。验证时必须把当前模式、持久模式、当前上下文和最终功能分开。

[Cheatsheet] DAC 与 SELinux 都必须允许：把拒绝翻译为 `domain + type + class + permission`；policy 允许只证明 SELinux 这一层，不等于服务终态正确。

[知识专题] 从四段安全上下文识别 domain 与 type

看到 `system_u:system_r:httpd_t:s0` 时，初学者常把整串字符当作必须完整背诵的标签。更有效的方法是先识别四段结构，再根据对象类型决定哪些字段最有区分度。RHCSA 服务故障最常从第三段开始，但这并不意味着其他字段不存在或可以随意改写。

① [知识点] `user:role:type:level` 是四段结构

代表性上下文：

```
system_u:system_r:httpd_t:s0
```

可拆为：

字段	示例	本章需要掌握的判断
SELinux user	<code>system_u</code>	SELinux 身份映射，不等同于 <code>/etc/passwd</code> 中的同名 Linux 用户
role	<code>system_r</code>	角色字段：服务进程常见 <code>system_r</code>
type/domain	<code>httpd_t</code>	RHCSA 服务故障最常用的策略身份
level/range	<code>s0</code>	MLS/MCS 安全级别或范围；本章只要求识别位置与结构

SELinux user 与 Linux 用户不是同一个命名空间。Linux 用户决定 DAC 凭据；SELinux user 参与 MAC 身份映射。两者可能关联，但不能仅凭用户名字符串推断上下文。

② [知识点] 进程上下文的第三段是 domain

进程运行后，策略通常让它进入某个 domain。例如 httpd 主进程可能显示：

```
system_u:system_r:httpd_t:s0
```

其中 `httpd_t` 是后续 AVC 的 source domain。进程 domain 比可执行文件名更适合描述策略身份：多个进程都叫 `httpd`，策略判断仍以当前进程上下文为准；一个自定义程序若没有发生预期 domain transition，也可能留在其他 domain 中。

本章不展开 domain transition 规则，但应记住：命令名不等于 domain，必须用 `ps -Z` 取得当前证据。

③ [知识点] 文件和目录上下文的第三段描述对象用途

文件上下文例如：

```
system_u:object_r:httpd_sys_content_t:s0
```

这里常见 `object_r` 角色，而真正决定服务访问边界的通常仍是第三段 `httpd_sys_content_t`。比较下面两种当前类型：

```
httpd_sys_content_t → 适合作为 Web 内容的对象类型
var_t                → /var 范围中的通用类型，未必允许 httpd_t 按目标行为访问
```

文件名、扩展名和路径字符串本身不是策略判断的直接替代品。两个都叫 `index.html` 的文件，如果 type 不同，`httpd_t` 的访问结论也可能不同。

④ [知识点] level 常显示为 `s0`，但不能把它当作 type

RHEL 的 targeted 策略中，经常看到 `s0`，容器或 MCS 场景可能还带类别范围，例如 `s0:c123,c456`。本章不教授 MLS/MCS 配置，只训练两个边界：

- level 是第四段，不要把其中的冒号类别误拆成新的上下文字段；
- RHCSA 常见 Web 服务拒绝通常优先比较第三段 type/domain，而不是先修改 level。

⑤ [知识点] `unconfined_t` 仍然是 domain，不代表 SELinux 已关闭

用户 Shell 常可能运行在 `unconfined_t` 等较宽松域中。看到 `unconfined_t` 不能得出“SELinux Disabled”的结论；是否启用和当前模式仍应由 `getenforce`、`sestatus` 判断。domain 描述策略身份，模式描述策略是否执行拒绝，它们是两个维度。

⑥ [知识点] 上下文是当前对象属性，持久期望来自其他配置

`ps -Z` 和 `ls -Z` 显示对象当前携带的上下文。对于进程，这个状态随进程生命周期存在；对于文件，当前标签可能受创建位置、移动方式、临时修改或重新标记影响。

因此：

当前标签正确 $\neq$ 持久路径规则已经正确
当前进程域正确 $\neq$ 应用功能已经成功

第 29 章将把“当前标签”与“策略期望标签、持久 `fcontext` 规则”组成闭环。本章只负责准确观察和解读。

[Cheatsheet] 四段为 `user:role:type:level`：进程 `type` 称为 `domain`；文件 `type` 描述策略用途；命令名、路径名和当前标签都不能替代实际上下文证据。

[操作专题] 识别当前模式、持久模式与策略状态

模式调查的本质是分清三件事：SELinux 是否启用、当前是否执行拒绝、系统下次按配置启动时计划进入什么模式。把它们压缩成一个“SELinux 开着”会遗漏最关键的持久性差异。

① [操作] 使用 `getenforce` 快速读取当前模式

作用对象：当前运行内核中的 SELinux 状态。基本语义：输出 `Enforcing`、`Permissive` 或 `Disabled`。典型形式：

`getenforce`

代表性输出：

`Enforcing`

它适合快速回答“此刻是否执行策略拒绝”，但不显示加载的策略名，也不直接显示 `/etc/selinux/config` 中的计划模式。

② [操作] 使用 `sestatus` 同时读取状态总览

`sestatus`

重点字段可按下面方式解读：

SELinux status: enabled
Loaded policy name: targeted
Current mode: enforcing
Mode from config file: enforcing

- `SELinux status`：SELinux 基础设施是否启用；
- `Loaded policy name`：当前加载的策略，例如 `targeted`；
- `Current mode`：当前是否执行拒绝；
- `Mode from config file`：配置文件记录的启动模式。

`Current mode` 与 `Mode from config file` 不一致并不矛盾。例如管理员执行 `setenforce 0` 后，当前可能是 `permissive`，而配置文件仍是 `enforcing`。

边界

启动时还可能存在内核参数等覆盖因素。RHCSA 常规任务优先核对 `sestatus` 与 `/etc/selinux/config`；若出现异常启动状态，再转到第 30 章《启动链、GRUB、Target 与系统恢复》。

③ [操作] 使用 `setenforce` 进行临时模式切换

`setenforce 0` # 当前切到 `Permissive`
`setenforce 1` # 当前切回 `Enforcing`

作用对象：已启用 SELinux 的当前运行状态。持久性：不修改 `/etc/selinux/config`，重启后按持久配置决定。限制：`Disabled` 状态不能通过 `setenforce 1` 直接变为 `Enforcing`。

切换前后应立即取证：

```
getenforce
setenforce 0
getenforce
# 复现并收集证据
setenforce 1
getenforce
```

不要在终端中执行 `setenforce 0` 后忘记恢复。经典任务要求把“诊断窗口”与“最终安全状态”分开记录。

④ [操作] 读取 `/etc/selinux/config` 中的持久计划

```
grep -E '^[[:space:]]*SELINUX(=|TYPE=)' /etc/selinux/config
```

代表性配置：

```
SELINUX=enforcing
SELINUXTYPE=targeted
```

`SELINUX=` 记录启动模式，`SELINUXTYPE=` 选择策略类型。修改该文件不会立刻改变当前模式；当前模式仍用 `getenforce` 或 `sestatus` 验证。

本章不把 `SELINUX=disabled` 作为解决服务故障的方法。`Disabled` 会停止加载策略，并造成持久对象不被正常标记；以后重新启用还可能需重启和文件系统重新标记。这类启动级恢复边界属于第 30 章。

⑤ [知识点] Enforcing、Permissive 与 Disabled 的证据含义

状态	策略加载	对象标记	AVC	不允许的操作
Enforcing	是	正常维护	记录可见拒绝	真正拒绝
Permissive	是	继续维护	记录本应发生的拒绝	不执行拒绝
Disabled	否	不正常维护持久对象标签	无正常策略判断链	SELinux 不参与

Permissive 适合有限诊断，不适合作为服务题最终状态。`Disabled` 比 `Permissive` 更激进，也更难安全恢复。

⑥ [诊断边界] Permissive 测试的正反结论

若切换到 `Permissive` 后目标操作成功，可以得出：

SELinux enforcement 参与了原失败路径，应收集 AVC 并判断具体策略条件。

不能得出：

已经修好，保持 `Permissive` 即可。

若切换到 `Permissive` 后同一症状仍然存在，可以得出：

当前观察到的症状不是只靠取消 SELinux 拒绝就能消失，应优先检查其他层。

也不要因此删除已有 AVC；系统可能同时存在 SELinux 偏差和另一个故障，只是后者仍然阻塞最终功能。

⑦ [边界] 单域 Permissive 不属于本章核心操作

RHEL 支持在系统保持 `Enforcing` 时把个别 domain 设为 `permissive`。这是策略开发和精细调试工具，会降低该 domain 的保护。本章只认识这项能力，不把它作为 RHCSA 默认诊断答案，也不制作要求直接修改单域状态的核心操作题。

[验证]

```
getenforce
sestatus
grep -E '^[[:space:]]*SELINUX(=|TYPE=)' /etc/selinux/config
```

[Cheatsheet] `getenforce` 看当前；`sestatus` 比较当前、策略和配置模式；`setenforce` 只临时切换；持久计划在 `/etc/selinux/config`；最终应恢复题目要求的 `Enforcing`。

[操作专题] 用 `ps -Z` 与 `ls -Z` 建立主体和目标证据

模式只说明策略是否执行，不能解释哪个 domain 正在访问哪个 type。下一步要把业务对象与安全上下文对应起来。稳定做法是先精确识别目标进程或路径，再显示上下文，而不是把 `grep` 结果直接当作策略身份。

① [操作] 使用 `ps -eZ` 浏览进程域

```
ps -eZ
```

它在进程列表前显示 SELinux 上下文，适合建立全局视图。为了便于判断，还可以定制普通进程字段：

```
ps -e -o label,pid,ppid,user,comm,args
```

不同 `ps` 版本的默认列顺序可能不同，因此不要依赖“第三列一定是 domain”之类的位置记忆，应识别完整上下文的第三段。

② [操作] 精确查询目标 PID 的上下文

先用可靠条件得到 PID，再查询：

```
pidof httpd
ps -Z -p <PID>
```

或在已知服务主 PID 时：

```
systemctl show httpd.service -p MainPID --value
ps -Z -p "$(systemctl show httpd.service -p MainPID --value)"
```

第二种形式引用了 `systemd` 的前置能力，本章不重新解释 `unit` 状态。重点是：PID 选择必须可靠，`ps -Z` 才能成为目标进程的证据。

代表性输出可能包含：

LABEL	PID	TTY	TIME	CMD
system_u:system_r:httpd_t:s0	1842	?	00:00:00	httpd

此时能证明 PID 1842 当前运行于 `httpd_t`，不能证明它一定能读取目标文件，也不能证明所有 worker 都处于同一上下文。多进程服务可按 PID 集合继续检查。

③ [操作] 使用 `ls -Zd` 查看目录本身

```
ls -Zd /srv/exam-site
```

`-Z` 显示上下文，`-d` 让 `ls` 显示目录对象本身，而不是列出目录内容。若要查看具体文件：

```
ls -lZ /srv/exam-site/index.html
```

代表性输出：

```
-rw-r--r--. root root unconfined_u:object_r:var_t:s0 /srv/exam-site/index.html
```

第一段 `-rw-r--r--.`，属于 DAC/扩展属性表现；中间的 `unconfined_u:object_r:var_t:s0` 是 SELinux 上下文。不要因为文件 owner 是 `root` 就忽略 `var_t`，也不要因为 type 看起来可疑就跳过普通路径权限。

④ [操作] 区分目录对象与目录内容

下面两条命令回答不同问题：

<code>ls -Zd /srv/exam-site</code>	# 目录本身的上下文
<code>ls -lZ /srv/exam-site</code>	# 目录内条目的上下文

服务访问路径时，目录搜索、文件读取和创建新对象可能分别涉及不同 class 与 permission。只检查首页文件而忽略父目录，仍可能遗漏 AVC 中对 `dir` 类的 `search` 或 `getattr` 拒绝。

⑤ [操作] 使用 `id -Z` 观察当前进程的 SELinux 上下文

```
id -Z
```

该命令常用于查看当前 Shell 所在上下文。它能帮助理解 Linux 用户与 SELinux user/domain 的区别，但不能代替对目标服务进程执行 `ps -Z`。管理员 Shell 处于 `unconfined_t`，不代表 httpd 也处于该 domain。

⑥ [验证边界] 当前上下文不等于持久配置

`ls -Z` 只能证明对象现在携带什么标签。对象可能通过临时方式被改过，也可能在重新标记、复制或重新创建后发生变化。因此本章的结论应写成：

当前文件 type 为 `var_t`，与 AVC 中的 target type 一致；下一章应检查路径的持久文件上下文规则。

而不是直接写：

文件配置错误，执行某条修复命令即可。

同样，`ps -Z` 证明当前 domain，但不证明服务配置、当前监听、开机启动或业务响应。

⑦ [帮助入口] 查询本地命令说明

```
man ps
man ls
man selinux
```

系统若提供 SELinux 专用服务手册页，也可以在第 29 章结合文件类型、端口和 Boolean 使用。这里先掌握通用观察接口。

[Cheatsheet] 全局进程域：`ps -eZ`；单 PID：`ps -Z -p`；目录本身：`ls -Zd`；具体文件：`ls -lZ`；当前上下文是证据，不是持久规则证明。

[知识专题] 把一条 AVC 还原成一次访问请求

AVC 是 Access Vector Cache 的缩写。SELinux 访问决定会被缓存，相关拒绝事件通常以 `AVC` 或 `USER_AVC` 类型进入 Audit 记录。日志看起来很长，但初步分析不需要解释每个数字：先把 source、target、class 与 permission 找出来，就能形成可验证假设。

① [知识点] 代表性 AVC 的稳定骨架

下面是为了讲解字段而整理的代表性裁剪记录，不是本会话真实执行输出：

```
type=AVC msg=audit(1770000000.123:421): avc: denied { open read } for \
pid=1842 comm="httpd" path="/srv/exam-site/index.html" \
scontext=system_u:system_r:httpd_t:s0 \
tcontext=unconfined_u:object_r:var_t:s0 \
tclass=file permissive=0
```

将它压缩成一句话：

`httpd_t` domain 对 `var_t` 类型的普通文件请求 `open/read` 时，被策略拒绝；事件发生在 `enforcing` 路径中。

这句话已经足以把下一步收缩到文件对象类型，而不是非标准端口或 Boolean。

② [知识点] `{ open read }` 是被拒绝的 permission

`avc: denied { ... }` 中的集合描述被拒绝的具体行为。常见示例：

permission	常见直观含义	仍需结合的字段
<code>read</code>	读取内容	<code>tclass=file</code> 、目标 path/type
<code>open</code>	打开对象	文件类型和访问模式
<code>getattr</code>	读取元数据	目标 class/type
<code>search</code>	遍历目录	<code>tclass=dir</code> 、父目录上下文
<code>write</code>	修改内容	目标 type 与 DAC 写权限
<code>name_bind</code>	绑定网络端口	<code>tclass=tcp_socket / udp_socket</code> 与端口类型

同一业务动作可能产生多个 permission。不要只看到 `read` 就忽略同一事件中目录 `search` 或文件 `open` 的拒绝。

③ [知识点] `scontext` 指向发起访问的 `source context`

```
scontext=system_u:system_r:httpd_t:s0
```

第三段 `httpd_t` 是 source domain。将它与 `ps -Z` 对照，可以验证日志中的主体是否就是当前目标服务。若 `comm="httpd"`，但 `scontext` 不是预期 domain，应继续调查进程启动方式和 domain transition，而不是仅凭命令名下结论。

④ [知识点] `tcontext` 指向目标对象的当前 `context`

```
tcontext=unconfined_u:object_r:var_t:s0
```

第三段 `var_t` 是 target type。将它与 `ls -Z` 对照，可以确认当前对象是否仍带有日志记录中的类型。目标可能在调查期间被改动，所以日志证据与当前状态都要保留时间关系。

⑤ [知识点] `tclass` 决定“目标是什么对象”

`tclass=file` 表示普通文件；还可能看到：

- `dir`：目录；
- `tcp_socket`、`udp_socket`：网络 socket；
- `lnk_file`：符号链接；
- `process`：进程对象；
- `sock_file`：Unix socket 文件。

同一个 permission 名称在不同 class 中可能含义不同。诊断时应把 `tclass` 与 permission 成对阅读。

⑥ [知识点] `comm`、`exe`、PID 和路径用于关联业务动作

- `comm`：进程短名称，可能被长度限制；
- `pid`：事件发生时的进程实例，之后可能退出或被复用；
- `exe`：完整可执行路径常位于同一 Audit 事件的其他记录中；
- `path / name`：目标路径可能位于 AVC 或配套 PATH 记录中。

因此，一条完整 Audit 事件可能由多条记录组成。它们通过相同的 `msg=audit(时间:序号)` 关联。`ausearch` 会按事件组织输出，比直接 `grep audit.log` 更适合初步调查。

⑦ [知识点] `permissive=0` 与 `permissive=1` 说明执行边界

- `permissive=0`：该拒绝发生在执行策略拒绝的路径；
- `permissive=1`：策略会拒绝，但当前处于允许通过的 `permissive` 路径。

在系统级 Permissive 或单域 Permissive 中，业务动作可能继续执行并触发更多后续 AVC。因此，Permissive 中看到的拒绝序列可能比 Enforcing 更长；不能把每条后续 AVC 都机械视为原始失败的第一根因。

⑧ [知识点] 没有 AVC 不等于 SELinux 一定未参与

常见原因包括：

- 没有重新触发失败；
- 查询时间范围不对；
- `auditd` 没有运行；
- 记录进入 Journal 或内核消息；
- `dontaudit` 规则抑制了某些拒绝；
- 当前症状由其他层造成。

因此“无结果”也是一个需要继续验证的状态，而不是立即关闭 SELinux 的理由。

[Cheatsheet] 先读 `{ permission }`、`scontext`、`tcontext`、`tclass`；再用 `comm`、PID、路径和事件序号关联业务；把长日志翻译成一句 domain 对 type:class 请求 permission。

[操作专题] 重新触发、检索并解释近期拒绝

日志调查最常见的失败不是不会写 `ausearch`，而是没有建立时间边界：管理员查询几小时的旧记录，却不知道哪条属于当前复现。推荐顺序是记录当前模式与时间，清楚地重新触发一次目标动作，然后立即查询 recent 或明确时间范围。

① [操作] 先建立复现时间与当前模式

```
date --iso-8601=seconds
getenforce
```

随后执行一次最小、可重复的目标操作，例如：

```
curl -sS -o /dev/null -w '%{http_code}\n' http://127.0.0.1/
```

这里的 `curl` 只是代表业务触发入口。真实服务可能通过文件访问、启动服务或绑定端口触发拒绝。复现前不要一次改动多个配置，否则新日志无法对应单一假设。

② [操作] 使用完整消息类型集合搜索近期拒绝

官方排错入口：

```
ausearch \
-m AVC,USER_AVC,SELINUX_ERR,USER_SELINUX_ERR \
-ts recent
```

为了人工阅读，可增加解释选项：

```
ausearch \
-m AVC,USER_AVC,SELINUX_ERR,USER_SELINUX_ERR \
-ts recent \
-i
```

`-m` 选择消息类型，`-ts` 限制开始时间，`-i` 将部分数字字段解释为更易读的名称。若只训练最常见的核心入口，也可以先使用：

```
ausearch -m AVC,USER_AVC -ts recent -i
```

③ [参数] `-i` 面向人工阅读，`--raw` 面向后续工具

`-i` 会解释字段，适合直接阅读；`--raw` 保留原始 Audit 记录，适合通过管道交给解析工具。不要把两者混成同一个目的。

人工阅读：

```
ausearch -m AVC,USER_AVC -ts recent -i
```

交给 `audit2why`：

```
ausearch -m AVC,USER_AVC -ts recent --raw | audit2why
```

④ [操作] 根据已知对象继续缩小结果

`ausearch` 的多个筛选条件通常按 AND 组合。已知命令名、PID 或可执行路径时，可以在本地 `man ausearch` 中选择相应参数继续缩小。考试和工作中应先保留一份未过度过滤的近期事件，再增加条件，避免因命令名截断或 PID 已变化而错过记录。

稳定的最小路径仍是：

```
近期消息类型
→ 识别事件序号
→ 读取完整事件
→ 再按业务对象缩小
```

⑤ [操作] 用 `journalctl` 查服务症状与 `setroubleshoot` 摘要

服务侧症状：

```
journalctl -u httpd.service --since "-10 min"
```

若系统安装并运行了相应 `setroubleshoot` 组件，可查询：

```
journalctl -t setroubleshoot
```

Journal 摘要便于阅读，但 Audit 事件仍是 SELinux 拒绝的主要原始证据。没有 `setroubleshoot` 输出不能证明不存在 AVC。

⑥ [操作] auditd 未运行时的后备入口

先确认：

```
systemctl is-active auditd
```

若 Audit daemon 未运行，应按环境要求恢复服务并重新触发。SELinux 活跃但 Audit daemon 不工作时，官方资料还建议从内核消息中搜索特定类型：

```
dmesg | grep -i -e 'type=1300' -e 'type=1400'
```

本章不展开 Audit 规则配置；目标只是避免把“Audit 文件无记录”误写成“SELinux 未参与”。

⑦ [操作] 使用 audit2why 辅助解释

```
ausearch -m AVC,USER_AVC -ts recent --raw | audit2why
```

audit2why 可能帮助指出约束、Boolean 或缺少允许规则等方向。它的输出应被视为线索，仍要与以下事实交叉核对：

- 当前进程 domain；
- 当前对象 type；
- 业务目标是否合理；
- 标准策略是否已有文件类型、端口类型或 Boolean；
- 建议是否扩大了不必要的权限。

⑧ [安全边界] 不把 audit2allow 当作第一修复手段

自动从拒绝生成允许规则，可能把误配置包装成新的权限。常见拒绝首先应检查：

```
错误或非预期标签
非标准目录未声明
非标准端口未映射
业务行为需要已有 Boolean
应用或 DAC 本身配置错误
```

只有确认标准策略无法表达合法需求时，才进入自定义策略模块；该能力不属于本章，也不是 RHCSA 常规服务题的默认答案。

⑨ [边界] dontaudit 调试必须成对恢复

极少数拒绝可能被 dontaudit 抑制。官方排错资料提供临时禁用和恢复方式：

```
semodule -DB # 临时禁用 dontaudit 规则后重新复现
semodule -B # 调查结束后恢复
```

这是会改变策略日志行为的高级诊断步骤，不是每次无 AVC 都应执行的默认命令。必须先检查复现、时间范围、Audit daemon 和 Journal，并确保调查后恢复。

[Cheatsheet] 记录时间并复现：ausearch -m AVC,USER_AVC,SELINUX_ERR,USER_SELINUX_ERR -ts recent；人工读用 -i，管道解析用 --raw；Journal 是辅助；audit2why 是线索，不是修复判决。

[诊断专题] 先证据后修复，并正确使用 Permissive

SELinux 排错的目标不是尽快让错误消失，而是找到最小、可解释、可持久验证的配置入口。下面的诊断链把每一步都写成一个可证伪的假设。已有可靠证据时可以从中间切入，但不能跳过结论所依赖的状态。

① [诊断] 从症状建立多层假设，而不是直接归因

症状：“httpd 访问自定义目录返回 403。”

初步假设至少包括：

- Apache 配置或目录授权规则拒绝；
- 服务读取的并不是预期文件；
- DAC 路径权限不允许；
- SELinux domain/type 组合被拒绝；
- 请求经过代理、虚拟主机或其他网络路径。

先用服务日志、配置检查、路径权限和请求结果建立基线，再把 SELinux 作为其中一层调查。

② [诊断] 下一条最有区分度的证据通常是模式和近期 AVC

若系统当前是 Permissive，业务成功不能证明策略允许；若当前是 Enforcing，业务失败也不能证明 SELinux 是根因。最有区分度的组合是：

```
getenforce
# 重新触发一次目标操作
ausearch -m AVC,USER_AVC -ts recent -i
```

- Enforcing + 有匹配 AVC：SELinux 明确拒绝了某个请求；
- Enforcing + 无匹配 AVC：检查 Audit/Journal、时间和其他层；
- Permissive + 有 AVC + 业务成功：策略原本会拒绝，但当前未执行；
- Permissive + 无 AVC：不能凭空编造 SELinux 根因。

③ [诊断] 用上下文证据验证 AVC 中的主体和目标

```
ps -Z -p <PID>
ls -Zd <DIRECTORY>
ls -lZ <FILE>
```

若当前状态与日志一致，可以继续分类；若不一致，应先回答对象是否在记录后被修改、进程是否重启、PID 是否复用、请求是否命中了另一个路径。

④ [诊断] 把 AVC 分类到标准策略入口

AVC 特征	典型分类	下一章的自然入口
<code>tclass=file/dir</code> ，目标 type 不符合服务用途	文件上下文	持久 fcontext 规则与 <code>restorecon</code>
<code>denied { name_bind }, tclass=tcp_socket</code>	端口类型	<code>semanage port</code>
domain/type 看似正常，行为属于策略预置可选能力	Boolean	<code>getsebool</code> / <code>setsebool -P</code>
建议缺少规则，但标准入口均不适用	可能需要自定义策略	超出 RHCSA 本章，先复核需求
Permissive 后仍是相同业务错误且无相关 AVC	非 SELinux 主阻塞	回到应用、DAC、网络或数据层

本章只做到“有证据地分类”，不执行前三类的完整修复。

⑤ [诊断] Permissive 是实验变量，不是最终答案

一个可审计的 Permissive 诊断窗口应记录：

1. 切换前模式；
2. 目标操作和时间；
3. 切换后是否成功；
4. 新增 AVC；
5. 调查结束后恢复 Enforcing；
6. 当前和持久模式的最终证据。

仅执行 `setenforce 0` 并说“现在能用了”，既没有修复，也没有保留原因。

⑥ [诊断] 最小修复必须留给正确对象

AVC 指向 `var_t:file` 读取拒绝时，最小修复对象通常是路径的 SELinux 文件类型，而不是：

- 把整个目录改成 `777`；
- 开启所有 httpd Boolean；
- 新增任意自定义 allow 规则；
- 永久保持 Permissive；
- 关闭 SELinux。

本章应输出交接结论：

当前证据指向自定义路径文件类型；下一章检查并建立持久文件上下文规则，然后重新验证当前标签和业务。

⑦ [验证] 诊断完成不等于业务修复完成

本章结束时可以形成一个完整的诊断终态：

- 当前与持久模式已明确；

- 进程 domain 与对象 type 已明确；
- AVC 中的 class 与 permission 已明确；
- 拒绝被分类到正确配置入口；
- 系统恢复到题目要求的安全模式；
- 未声称尚未执行的第 29 章修复已经完成。

这比“服务 active”或“命令返回 0”更可信，因为每个结论都有独立证据。

⑧ [工作迁移] 为变更单保留最小证据包

真实工作中，可保留：

故障时间与复现步骤
当前/持久 SELinux 模式
目标进程 PID、命令行和上下文
目标路径及当前上下文
相关 Audit 事件原文
一句话的 AVC 翻译
分类与建议的标准配置入口
变更后需要重新验证的层次

不要把整份 `/var/log/audit/audit.log` 无差别复制进报告，也不要只保留工具生成的结论而丢失原始 AVC。

[Cheatsheet] 症状 → 模式与复现 → AVC → `ps -Z / ls -Z` → 四元翻译 → 分类标准入口 → 最小修复 → 恢复 Enforcing → 分层再验证。

[经典任务] 在不关闭 SELinux 的前提下完成自定义 Web 目录 AVC 取证

一台 RHEL 9 服务器已经安装并启动 `httpd`。管理员把站点目录改为 `/srv/exam-site`，目录中已有 `/srv/exam-site/index.html`，不得删除、覆盖或改写该文件。

已知环境：

- Apache 仍监听标准 TCP 80，本任务不处理非标准端口；
- 普通目录搜索和文件读取权限已经满足；
- `firewalld` 不是本机回环请求的阻塞点；
- SELinux 当前为 `Permissive`；
- `/etc/selinux/config` 计划下次启动进入 `Enforcing`；
- 当前通过 `curl http://127.0.0.1/` 可以读取页面；
- Audit 日志中应能找到本次访问对应的 AVC；
- 当前没有可由本会话控制的真实实验机，因此题目中的命令输出均需在你的 RHEL 9 环境中取得，本参考解答只给出推荐调查路径。

请完成以下目标：

1. 证明当前模式和持久配置原本不一致；
2. 确认 `httpd` 主进程当前运行在哪个 SELinux domain；
3. 确认 `/srv/exam-site` 目录和 `index.html` 当前是什么 type；
4. 记录时间并重新触发一次 HTTP 请求；
5. 使用 `ausearch` 提取近期 AVC，并保留一份适合人工阅读的结果；
6. 从事件中提炼 source domain、target type、object class 和 denied permission；
7. 使用原始事件通过 `audit2why` 做辅助解释；
8. 判断下一步应进入“持久文件上下文规则”调查，而不是端口类型或 Boolean；
9. 本章内不得执行 `semanage fcontext`、`restorecon`、`chcon`、`semanage port` 或 `setsebool`；
10. 调查结束后将当前模式恢复为 `Enforcing`，并证明当前模式与持久配置均为 `Enforcing`；
11. 明确记录：在第 29 章修复文件规则之前，`Enforcing` 下页面仍可能访问失败，这不影响本任务的诊断终态成立。

限制条件：

不得关闭 SELinux
不得把 `Permissive` 作为最终状态
不得 `chmod 777`
不得删除或替换现有页面
不得使用 `audit2allow` 生成策略模块
不得把“页面在 `Permissive` 下可访问”写成最终修复

验收矩阵

目标	推荐证据	能证明什么
当前模式	<code>getenforce</code>	当前是否执行拒绝
当前与配置模式	<code>sestatus</code>	两个模式维度与策略名
持久配置	<code>grep ... /etc/selinux/config</code>	配置文件记录
进程域	<code>ps -Z -p <PID></code>	当前 httpd domain
目录/文件类型	<code>ls -Zd</code> 、 <code>ls -lZ</code>	当前对象上下文
近期拒绝	<code>ausearch ... -ts recent -i</code>	人工可读的 SELinux 事件
辅助解释	<code>ausearch ... --raw audit2why</code>	原因线索
最终安全模式	再次执行 <code>getenforce</code> 、 <code>sestatus</code>	已恢复 <code>Enforcing</code>
诊断交接	一句四元翻译与分类	下一章应处理的对象

卡住时再想一想

页面在 `Permissive` 下成功，证明的是策略允许，还是只证明拒绝没有执行？

`comm="httpd"` 能否替代 `scontext` 判断 domain？

`ls -Z` 显示当前 type 后，是否已经证明持久路径规则存在？

请先独立完成调查。参考解答从下一页开始。

[参考解答] 从基线、复现到 AVC 分类

① 建立模式基线

```
getenforce
sestatus
grep -E '^[[:space:]]*SELINUX(=|TYPE=)' /etc/selinux/config
```

预期要识别的关系是：

```
Current mode: permissive
Mode from config file: enforcing
```

或 `getenforce` 返回 `Permissive`，配置文件记录 `SELINUX=enforcing`。这说明当前诊断状态不会跨重启自动保持，并且当前业务成功不能证明策略允许。

不要立即执行 `setenforce 1`。题目要求先在可重复的诊断窗口中收集 AVC，完成取证后再恢复。

② 精确取得 httpd 主进程和 domain

```
main_pid=$(systemctl show httpd.service -p MainPID --value)
printf 'MainPID=%s\n' "$main_pid"
ps -Z -p "$main_pid"
```

若 `MainPID` 为 `0`，应先回到服务状态调查，而不是继续假定进程存在：

```
systemctl status httpd.service
journalctl -u httpd.service --since "-10 min"
```

目标是得到类似 `httpd_t` 的当前 domain。不要只运行 `ps aux | grep httpd`，因为它没有显示 SELinux 上下文，还可能包含 `grep` 本身或多个 worker。

③ 取得目录和文件当前 type

```
ls -Zd /srv/exam-site
ls -LZ /srv/exam-site/index.html
```

记录完整四段上下文，并单独提取第三段 type。若看到 `var_t` 或其他不符合 Web 内容用途的类型，只形成假设，不在本章直接修复。

同时保留 DAC 基线，证明题目给定条件仍成立：

```
namei -l /srv/exam-site/index.html
```

④ 记录时间并重新触发一次请求

```
date --iso-8601=seconds
curl -sS -D - http://127.0.0.1/ -o /dev/null
```

在 `Permissive` 下请求可能成功，但此时策略仍会记录本应发生的拒绝。只触发一次有助于把近期事件与操作关联。

⑤ 提取人工可读 AVC

```
ausearch \
-m AVC,USER_AVC,SELINUX_ERR,USER_SELINUX_ERR \
-ts recent \
-i
```

若结果较多，先用事件时间和 `comm` 找到与刚才请求相符的事件，再保留同一事件序号的完整记录。不要只复制单独一行而丢失配套 `PATH` 或 `SYSCALL` 记录。

若没有结果，按顺序调查：

```
systemctl is-active auditd
journalctl -t setroubleshoot --since "-10 min"
dmesg | grep -i -e 'type=1300' -e 'type=1400'
```

先确认复现和日志链，不要直接关闭 SELinux 或生成策略。

⑥ 用四元模型翻译事件

假设代表性 AVC 包含：

```
avc: denied { open read }
scontext=...:httpd_t:s0
tcontext=...:var_t:s0
tclass=file
permissive=1
```

应写成：

```
httpd_t domain 对 var_t:file 请求 open/read；策略会拒绝，但当前 permissive 路径未执行拒绝。
```

再与当前状态交叉核对：

```
ps -Z -p "$main_pid"
ls -lZ /srv/exam-site/index.html
```

若日志和当前 type 不一致，先调查对象是否已被修改或请求是否访问了其他文件。

⑦ 通过原始事件调用 audit2why

```
ausearch \
-m AVC,USER_AVC \
-ts recent \
--raw | audit2why
```

将输出作为辅助线索。即使工具提到可能生成允许规则，也要先检查标准策略入口。当前 `tclass=file`、目标 type 为 `var_t`，而业务是自定义 Web 内容，最合理的分类是文件上下文问题。

⑧ 形成交接结论，不越界执行修复

推荐结论：

```
证据显示 httpd_t 对 /srv/exam-site/index.html 的当前 var_t:file
请求 open/read 时产生 AVC。目标为文件对象，不是 tcp_socket；当前任务也没有
数据库外连等 Boolean 需求。下一步应在第 29 章检查该路径的持久文件上下文规则，
再按规则恢复当前标签并重新验证。
```

这里不得使用 `chcon` 制造临时正确标签，也不得提前执行第 29 章操作。

⑨ 恢复 Enforcing 并验证模式终态

```
setenforce 1
getenforce
sestatus
grep -E '^[[:space:]]*SELINUX(=|TYPE=)' /etc/selinux/config
```

目标是：当前为 Enforcing，配置文件也计划 Enforcing。

可以再次请求页面以观察 Enforcing 下的真实表现：

```
curl -sS -D - http://127.0.0.1/ -o /dev/null
```

若访问失败，这与诊断结论一致：本章已经完成证据收集和分类，但尚未执行文件规则修复。不要把“页面仍失败”误判为本任务失败，也不要把它隐藏在报告中。

⑩ 分层验收与典型错误

模式层：当前与持久均为 Enforcing。主体层：目标 httpd 进程 domain 已记录。目标层：目录和文件当前 type 已记录。事件层：近期 AVC 原文、事件序号和四元翻译已保留。诊断层：已分类到文件上下文，不误归因到端口或 Boolean。边界层：未关闭 SELinux、未放宽 DAC、未生成本地策略、未抢先执行第 29 章修复。

典型错误：

1. 看到 `curl` 在 Permissive 下成功，写成“SELinux 已配置完成”；
2. 只保存 `audit2why` 文本，丢失原始 AVC；
3. 只看 `comm="httpd"`，不核对 `scontext`；

- 4. 看到 `ls -Z` 后直接宣称持久规则存在；
- 5. 调查结束忘记 `setenforce 1`；
- 6. 用 `audit2allow` 为错误标签生成额外权限。

[本章收束] 从拒绝表象回到可验证的策略对象

工作方法：把每一次拒绝写成一条可核对的判断句

本章最终训练的不是“记住几条 SELinux 命令”，而是把故障还原为三张状态图。

访问决策: `source domain + target type + object class + permission → policy decision`
模式关系: 当前模式 $\neq$ 持久计划
证据推进: 症状 → 复现 → AVC → 上下文核对 → 分类标准入口 → 再验证

真正可交接的诊断结论应包含：当前模式、复现时间、主体 domain、目标 type、对象类别、被拒绝权限、原始事件位置，以及为什么下一步应进入某个标准配置入口。只写“SELinux 拒绝了”仍然不够精确。

主要判断表

已取得的证据	可以得出的结论	不能扩大的结论	下一条有区分度的证据
<code>getenforce</code> 为 <code>Enforcing</code>	当前执行策略拒绝	当前故障一定由 SELinux 引起	重新触发并查询近期 AVC
当前为 <code>Permissive</code> 且存在 AVC	策略本应拒绝，但当前未执行拒绝	当前配置已经正确	解读四元组并恢复 <code>Enforcing</code> 验证
<code>ps -Z</code> 显示 <code>httpd_t</code>	目标进程当前处于该 domain	它一定能访问目标对象	<code>ls -Z</code> 与 AVC 的 <code>tcontext</code>
<code>ls -Z</code> 显示 <code>var_t</code>	对象当前 type 为 <code>var_t</code>	持久 <code>fcontext</code> 规则一定错误或不存在	第 29 章的期望类型与持久规则查询
AVC 为 <code>tclass=file</code> 、 <code>read</code>	文件读取路径被策略拒绝	端口或 Boolean 一定无问题	核对 path、当前 type 和业务目标
<code>audit2why</code> 给出建议	获得一个解释方向	建议就是最小、安全修复	回到原始 AVC 和标准策略入口
<code>Permissive</code> 后仍失败	取消 SELinux 拒绝不足以消除当前症状	SELinux 完全没有任何偏差	服务日志、DAC、配置、网络等其他层

向下一章交接

本章完成到“分类标准配置入口”为止。第 29 章《SELinux 文件规则、端口类型与 Boolean》将继续回答三类问题：怎样为路径建立持久文件上下文规则，怎样让服务使用策略认可的端口类型，以及怎样判断并持久设置已有 Boolean。进入下一章前，应已经能够把一条 AVC 翻译成 `domain 对 type:class 请求 permission`，并知道当前标签、持久规则与业务终态不是同一件事。

最终安全边界：不通过 `chmod 777`、`Permissive`、`Disabled` 或未经分析的本地 `allow` 规则绕过问题；不把“有 AVC”扩大成“所有故障都由 SELinux 引起”；不把工具建议当作最终判决；每一次修改都要能回到原始事件和目标业务解释。

第 29 章 SELinux 文件规则、端口类型与 Boolean

本章阅读导航

先抓住一条主线：确认 SELinux 参与拒绝之后，不是立刻“放宽策略”，而是先判断问题属于路径对象、协议端口还是策略预留的可选行为，再建立持久声明并用分层证据验收。

- 01 识别对象路径、协议端口或可选行为
- 02 建立基线规则、预期、当前与功能分开
- 03 选择入口 fcontext、port type 或 Boolean
- 04 最小修改新增、修改或删除已有声明
- 05 应用当前状态让持久声明作用于现有对象
- 06 分层验收配置、运行、网络、SELinux 与功能

专题地图

- 知识专题文件、端口和行为：先选对策略入口
- 知识专题当前标签、预期标签与持久规则
- 操作专题建立文件上下文基线并安全恢复标签
- 操作专题管理 fcontext 规则生命周期
- 知识专题文件类型与最小写能力
- 操作专题管理协议相关的非标准端口
- 操作专题查询并持久设置 Boolean
- 诊断专题规则覆盖、端口冲突与跨层证据
- 经典任务自定义目录和 TCP 82 Web 服务
- 参考解答调查、修改、应用和分层验收

阅读时持续回答

1. 现在看到的是持久规则、预期标签还是当前标签？
2. 目标表达式是否同时覆盖目录本身和后代？
3. 已有记录应使用新增、修改还是删除？
4. 端口号对应的是 TCP 还是 UDP？
5. Boolean 是否精确描述真实业务行为？
6. 这条验证命令能证明什么，不能证明什么？
7. 最终成功是否仍依赖临时标签或 runtime-only 状态？

判断提示：每次变更都先区分持久声明、当前状态与最终功能；阅读经典任务时先独立列出验收矩阵，再对照参考解答。

第 29 章 · 正文

上一章已经建立 SELinux 模式、进程域、对象类型和 AVC 证据的最小判断框架。本章从“已经确认 SELinux 参与拒绝”这一节点继续：面对自定义目录、非标准端口或策略默认关闭的业务行为，怎样选择正确的标准策略入口，怎样区分当前状态与持久声明，并用最小授权完成可重复的验收。

最常见的误判，是把一次可见变化当作长期正确状态：`chcon` 后 `ls -Z` 看起来正确，却没有路径规则；`semanage port` 成功，却没有进程监听；Boolean 为 `on`，却没有重新触发真实应用。为避免这些局部成功，本章始终沿着“对象 - 声明 - 当前状态 - 功能证据”推进。

SELinux 模型与 AVC 解读归第 28 章；firewalld 规则归第 21 章；服务 unit 与启动状态归第 12 章。本章只在综合验收时引用这些证据入口，不重复展开相邻章节。

概念

当前标签 是文件或目录此刻携带的 SELinux 安全上下文，可由 `ls -Z` 观察。它可能来自创建继承、`restorecon`、`chcon` 或文件移动等过程，因此“当前看起来正确”只证明这一刻的对象状态，不能证明系统已经建立路径级持久声明。

概念

持久 fcontext 规则 是管理员登记的“路径表达式到 SELinux 类型”的映射。它描述该路径未来应获得什么默认类型，却不会在创建或修改规则时自动遍历磁盘对象；已有文件仍需由 `restorecon` 按规则应用。规则存在、预期正确和当前标签正确必须分别验证。

概念

路径正则 是 fcontext 规则识别对象的表达方式。经典形式 `'/srv/site(/.*)?'` 同时覆盖目录本身和斜杠后的所有后代；外层单引号防止 Shell 把括号、星号和问号当作自己的语法。表达式过宽会扩大影响面，过窄则可能漏掉目录或内容。

概念

端口类型 把“协议 + 端口或端口范围”映射为 SELinux 策略对象，决定某类受限进程是否能使用该网络端口。TCP 82 与 UDP 82 是不同对象；端口类型正确只证明 SELinux 的一个条件，不证明应用配置、真实监听或 firewalld 已经正确。

概念

Boolean 是策略作者预留的条件开关，用于整体启用或关闭一组已定义规则，例如允许 Web 应用主动连接数据库。它不负责修复错误文件标签、错误端口映射或 DAC 权限；选择 Boolean 时必须从真实业务行为出发，并区分当前值与使用 `-P` 保存的持久值。

概念

最小授权 是只改变能够解释当前证据的最窄状态：静态内容使用只读类型，写入能力只授予必要子目录；端口映射同时限定协议和端口；Boolean 只启用语义匹配的开关。关闭 SELinux、`chmod 777`、批量可写类型或盲目生成策略模块都不是默认修复。

操作语义以下命令分别维护路径规则、计算预期标签、应用当前标签、管理协议端口和切换策略 Boolean。先理解作用对象，再选择参数。

semanage fcontext

SYNOPSIS

```
semanage fcontext {-l|-a|-m|-d} [-t TYPE] FILE_SPEC
```

查询或维护持久的路径表达式到文件类型映射；它修改规则，不直接重标记现有文件。

重要参数 / 形式

`-l` / `-C -l`

列出全部规则，或只列出本地定制。

`-a -t TYPE FILE_SPEC`

在表达式不存在时新增规则。

`-m -t TYPE FILE_SPEC`

修改已经存在且确认需要调整的规则。

`-d FILE_SPEC`

删除本地规则；删除后不会自动改变磁盘上的当前标签。

matchpathcon / restorecon

SYNOPSIS

```
matchpathcon [-V] PATH
restorecon [-n] [-R] [-v] PATH ...
```

前者根据当前规则计算路径的预期上下文；后者把该预期应用到现有对象。

重要参数 / 形式

`matchpathcon PATH`

显示路径按规则应获得的默认上下文。

`matchpathcon -V PATH`

比较预期上下文与磁盘当前上下文。

`restorecon -nRv PATH`

递归预览将发生的变化，不实际修改。

`restorecon -Rv PATH`

递归应用当前文件上下文规则。

chcon

SYNOPSIS

```
chcon -t TYPE PATH ...
```

直接修改对象当前标签，适合受控实验或临时定位；它不建立路径未来应获得什么类型的持久规则。

重要参数 / 形式

`-t TYPE`

只替换上下文中的类型字段。

`chcon -t TYPE PATH`

当前标签可跨普通重启保留，但可能被 `restorecon`、重标记或对象重建覆盖。

semanage port

SYNOPSIS

```
semanage port {-l|-a|-m|-d} [-t TYPE] -p {tcp|udp} PORT
```

查询或维护 SELinux 的协议相关端口映射；端口号不能脱离 TCP 或 UDP 单独判断。

重要参数 / 形式

- `-l / -C -l`
列出全部端口映射，或只列出本地定制。
- `-p tcp|udp`
明确协议；同一数字的 TCP 和 UDP 是不同对象。
- `-a/-m/-d`
分别用于新增、修改和删除本地端口映射。
- `-t TYPE`
指定目标 SELinux 端口类型。

getsebool / setsebool

SYNOPSIS

```
getsebool {NAME|-a}
setsebool [-P] NAME {on|off}
```

查询当前 Boolean，并切换策略预先定义的可选行为；选择开关前先阅读名称和描述是否精确匹配业务。

重要参数 / 形式

- `getsebool NAME`
查询指定 Boolean 当前值。
- `getsebool -a`
列出全部当前值，通常配合筛选发现相关项。
- `setsebool NAME on|off`
只改变当前加载策略中的值。
- `setsebool -P NAME on|off`
同时建立持久值；等待命令完成后再验证。

[知识专题] 文件、端口和行为：先选对策略入口

SELinux 拒绝本身不是修复答案。相同的“服务不可用”现象可能来自目标文件类型不合适、服务绑定了策略未授权的端口，或者服务确实需要策略默认关闭的一项可选行为。最快的切入方法不是依次尝试命令，而是先问：失败动作主要落在路径对象、网络端口，还是策略预留的业务行为上。

① [知识点] 文件上下文规则管理路径对象的用途

受限进程不是按传统文件名猜测对象用途，而是根据目标对象类型和请求行为执行策略规则。将站点从 `/var/www/html` 移到 `/srv/site` 后，即使普通权限允许读取，文件也可能仍带有与 Web 内容不相符的类型。此时应建立路径的持久 `fcontext` 规则，并把规则应用到当前对象。

典型闭环是：

```
持久规则: semanage fcontext
          ↓
预期标签: matchpathcon
          ↓
当前标签: restorecon 应用, ls -Z 查看
          ↓
真实访问: 重新请求服务功能
```

② [知识点] 端口类型管理受限服务可使用的网络端口

服务配置文件决定应用计划监听什么端口；SELinux 端口类型决定相应进程域是否被策略允许绑定该协议和端口。两者不一致时，服务可能在启动或重载时因 `name_bind` 被拒绝。

端口映射不能脱离协议：

```
(http_port_t, tcp, 82)
```

与 UDP 82 不是同一条记录。端口可能已经属于其他类型或位于一个已有范围中，所以操作前应先查，不应把 `-a` 当作固定答案。

③ [知识点] Boolean 管理策略已经预留的可选行为

Boolean 不直接给某个具体文件贴标签，也不把某个端口改成服务端口。它一次启用或关闭策略中围绕某项业务行为准备好的一组条件规则。例如 Web 应用确实需要主动连接数据库时，可以调查 `httpd_can_network_connect_db`；如果只是站点目录标签错误，开启该 Boolean 没有帮助。

选择标准入口的快速判断：

读取、写入或搜索某个路径失败
→ 检查当前标签、预期标签和 `fcontext` 规则

服务绑定非标准端口失败
→ 检查协议、端口及其 SELinux 端口类型

合法业务行为被默认策略关闭
→ 搜索并理解对应 Boolean

④ [边界] 标准入口优先于扩大权限和自定义策略

遇到拒绝时，先排除错误配置，再检查 SELinux 已经提供的文件类型、端口类型和 Boolean。以下做法不能作为默认答案：

- 关闭 SELinux 或长期保持 Permissive；
- 使用 `chmod 777` 试图绕过 SELinux；
- 将整个业务目录都标记成可写类型；
- 看见 AVC 后立即用 `audit2allow` 生成本地模块；
- 不理解含义就批量开启名称相近的 Boolean。

只有标准策略确实无法表达经过确认的合法需求时，才进入自定义策略设计；这属于更高级的 SELinux 工作，不在本章完整展开。

[Cheatsheet] 路径对象看 `fcontext`，协议端口看 `port type`，可选行为看 Boolean；先修误配置，再考虑扩展策略。

[知识专题] 当前标签、预期标签与持久规则是三个状态

文件标签问题最容易出现“看起来已经修好”的误判。`ls -Z` 显示正确，只证明此刻对象标签正确；它不能证明系统存在持久路径规则。反过来，`semanage fcontext` 已经添加规则，也不能证明磁盘上已有文件已经被重新标记。稳定判断必须把规则、预期和当前三个状态分开。

① [知识点] `ls -Z` 读取当前对象标签

常用形式：

```
ls -Zd /srv/site
ls -lZ /srv/site
```

目录本身和目录中的文件是不同对象。只执行 `ls -lZ /srv/site` 时，容易只看到内容而忽略目录自身，因此调查路径穿越或目录类型时应显式查看目录本身。

当前标签可能来自：

- 创建文件时根据父目录和策略获得；
- `restorecon` 按默认规则恢复；
- `chcon` 直接修改；
- 复制、移动、解压或应用创建文件时的具体行为；
- 文件系统重标记。

所以当前值本身不能回答它是否会在下一次恢复标签后继续存在。

② [知识点] `matchpathcon` 读取路径的预期标签

```
matchpathcon /srv/site/index.html
matchpathcon -V /srv/site/index.html
```

第一条命令根据当前加载的文件上下文规则计算该路径的默认上下文。`-V` 用于比较磁盘上的当前标签与默认值。该证据回答的是“策略认为此路径应该是什么”，而不是“服务是否一定有权执行某个具体操作”。

如果 `matchpathcon` 给出的预期类型本身就不符合目标，继续重复 `restorecon` 没有意义，因为 `restorecon` 只会忠实应用当前规则。下一步应检查本地 `fcontext` 定制和规则重叠。

③ [知识点] `semanage fcontext` 读取和维护持久规则

```
semanage fcontext -l
semanage fcontext -C -l
```

`-l` 列出策略提供的规则和本地定制；`-C -l` 只列出本地自定义记录，适合定位管理员覆盖。持久规则维护的是路径表达式，不直接遍历并修改现有文件。

稳定闭环应分别证明：

状态	推荐证据	证明范围
持久路径规则	<code>semanage fcontext -C -l</code> 或精确筛选 <code>-l</code>	管理员定义了怎样的路径映射
预期标签	<code>matchpathcon</code>	当前规则为该路径计算出的默认上下文
当前标签	<code>ls -Z</code>	inode 此刻携带的标签
应用功能	服务请求或读写测试	目标业务行为是否真正完成

④ [边界] `chcon` 不是“重启即失效”，但仍不是持久规则

`chcon -t TYPE PATH` 直接改变对象当前标签。这个标签通常可以跨普通重启保留，因此把 `chcon` 简化成“重启后必定失效”并不准确。真正的边界是：它没有建立路径级默认规则，后续 `restorecon`、文件系统重新标记、对象重建或其他标签恢复过程可以覆盖它。

因此 `chcon` 可以用于受控实验或临时定位，但考试和稳定运维的最终配置应回到：

```
semanage fcontext 建规则
→ restorecon 应用规则
→ matchpathcon 与 ls -Z 比较
→ 功能验证
```

[Cheatsheet] 当前看 `ls -Z`、预期看 `matchpathcon`、持久规则看 `semanage fcontext`；`chcon` 改当前对象，不定义路径未来应该是什么。

[操作专题] 建立文件上下文基线并安全执行 `restorecon`

在修改路径规则前，先记录规则、预期和当前状态。这样可以判断故障来自规则缺失、当前标签漂移，还是服务需要的类型选择本身错误，也能避免在包含现有业务数据的目录上盲目递归重标记。

① [操作] 对一个路径建立三层基线

作用对象：目标目录本身、关键文件以及它们匹配的 `fcontext` 规则。

```
ls -Zd /srv/site
ls -lZ /srv/site
matchpathcon /srv/site
matchpathcon /srv/site/index.html
semanage fcontext -C -l | grep -F '/srv/site'
```

若本地列表没有结果，不代表策略完全没有默认规则；还应按需检查完整列表或直接以 `matchpathcon` 的结果作为“当前规则最终推导出的预期值”。使用 `grep` 只是缩小输出，不要因为正则符号被 `grep` 解释而误判记录不存在；需要精确查看时应保留完整行。

② [操作] 先预览再递归应用

```
restorecon -nRv /srv/site
restorecon -Rv /srv/site
```

关键参数：

- `-n`：只显示计划变更，不真正修改；
- `-R`：递归处理目录树；
- `-v`：显示处理信息。

在真实服务器上，递归操作前先用 `-n` 预览更稳妥，特别是目录中可能混有上传区、缓存、密钥或其他用途不同的子目录时。预览发现大量意外变化，应先调整规则边界，而不是直接执行。

③ [操作] 应用后比较规则、预期与当前

```
matchpathcon -V /srv/site
matchpathcon -V /srv/site/index.html
ls -Zd /srv/site
ls -lZ /srv/site
```

标签一致仍只是 SELinux 对象状态的一部分。随后必须重新触发服务读取、写入或执行操作，并查看最新失败证据。旧 AVC 不能证明当前修复仍失败。

④ [边界] `restorecon` 没有输出不等于“命令没有工作”

目标本来就符合预期时，普通或非详细模式可能没有需要报告的变化。判断应基于：

1. `matchpathcon` 给出的预期；

2. `ls -Z` 给出的当前;
3. `restorecon -nRv` 是否计划变化;
4. 重新触发后的实际功能。

不要为了获得“有输出”而反复执行 `chcon` 制造差异。

[Cheatsheet] 基线先分规则、预期、当前; 递归恢复先 `-nRv` 预览, 再 `-Rv` 应用; 最后重新触发原始业务。

[操作专题] 用 `semanage fcontext -a/-m/-d` 管理规则生命周期

`semanage fcontext` 维护的是本地策略存储中的文件上下文定制。新增、修改和删除对应不同的现状, 不能把 `-a` 重复执行当作幂等操作。考试环境常见的错误是规则已经存在, 但类型错误; 真实工作中还常见旧规则过宽、多个表达式重叠或删除规则后没有重新恢复当前标签。

① [操作] `-a` 只用于新增不存在的规则

```
semanage fcontext -a -t httpd_sys_content_t '/srv/site(/.*)?'\nrestorecon -Rv /srv/site
```

作用对象: 路径表达式 `'/srv/site(/.*)?'` 的持久默认类型。

`-t` 指定目标类型。表达式使用单引号, 避免 Shell 把括号、问号或星号解释为自己的语法。`(/.*)?` 的直观作用是: 既匹配 `/srv/site` 目录本身, 也匹配斜杠之后的所有后代路径。

只给 `/srv/site` 建一条不含后代的规则, 通常不能覆盖目录中的文件; 只给 `/srv/site/.*` 建规则又可能漏掉目录本身。经典形式同时覆盖二者。

② [操作] 已有规则类型不符时使用 `-m`

```
semanage fcontext -m -t httpd_sys_content_t '/srv/site(/.*)?'\nrestorecon -Rv /srv/site
```

判断过程:

```
semanage fcontext -a 报记录已存在\n→ 查询本地和完整规则\n→ 确认表达式就是目标记录\n→ 判断原类型是否确实错误\n→ 使用 -m 修改\n→ restorecon 应用
```

不要因为 `-a` 报错就直接改写一个看不懂的现有规则。表达式可能属于已有业务, 或者目标路径实际被另一条更高优先级本地规则命中。

③ [操作] 不再需要本地定制时使用 `-d`

```
semanage fcontext -d '/srv/site(/.*)?'\nmatchpathcon /srv/site/index.html\nrestorecon -nRv /srv/site\nrestorecon -Rv /srv/site
```

删除操作只移除本地规则, 不会自动把磁盘上已经存在的标签改回策略默认值。应先重新查询删除后的预期标签, 再预览和应用 `restorecon`。如果删除后路径不再有适合业务用途的默认类型, 服务可能重新失败, 因此删除也必须经过功能验证。

④ [配置] 本地规则优先于策略包规则

管理员通过 `semanage fcontext` 添加的本地定制用于覆盖或补充策略包的默认文件上下文。它们应被视为本机配置资产, 不能直接手工编辑底层数据库文件代替 `semanage`。

本地表达式发生重叠时, 不能只凭“看起来更具体”猜谁生效。当前工具语义下, 本地规则按较新的记录优先匹配, 第一条匹配项决定预期上下文。稳定方法是直接对具体路径执行 `matchpathcon`, 并通过 `semanage fcontext -C -l` 检查本地记录和新增顺序造成的覆盖。

⑤ [配置] 规则应尽量窄并按用途拆分

假设站点大部分内容只读, 但 `/srv/site/uploads` 确实需要 Web 进程写入, 应将范围拆开:

```
semanage fcontext -a -t httpd_sys_content_t '/srv/site(/.*)?'\nsemanage fcontext -a -t httpd_sys_rw_content_t '/srv/site/uploads(/.*)?'\nrestorecon -Rv /srv/site
```

然后用 `matchpathcon` 分别检查普通页面和上传目录。真实环境中还应留意本地规则的覆盖顺序，确保窄规则真正成为最终匹配；发现预期不符时，调整规则而不是给整个站点扩大写类型。

⑥ [帮助入口] 工具缺失和语法不确定时从本地帮助恢复

```
semanage fcontext -h
man semanage-fcontext
man restorecon
man matchpathcon
dnf provides '*/semanage'
```

`semanage` 不存在时，先检查拼写和已启用仓库，再用 `dnf provides` 查询当前系统中由哪个包提供命令，不必依赖脱离环境的包名记忆。

[Cheatsheet] 不存在才 `-a`，已存在且确需改才 `-m`，移除本地覆盖用 `-d`；每次规则变化后重新计算预期，再决定 `restorecon`。

[知识专题] 文件类型代表能力边界，可写类型必须最小化

给文件选择 SELinux 类型，不是给路径添加一个“允许访问”的通用标志，而是在声明对象用途。不同类型让策略允许不同域执行不同操作。类型选择过窄会阻止合法业务，选择过宽则会扩大被入侵进程可以影响的数据范围。

① [知识点] `httpd_sys_content_t` 适合 Web 只读内容

静态页面、样式文件和只需要被 Web 服务读取的资源，典型目标类型是 `httpd_sys_content_t`。它表达“这是 Web 内容”，不表示所有进程都能访问，也不改变普通文件权限。

② [知识点] `httpd_sys_rw_content_t` 只用于确实需要写入的对象

上传目录、应用缓存或会话目录可能需要 Web 进程写入。只有经过业务确认的最小子目录才应使用可写类型。把整个 `/srv/site` 标成可写类型虽然可能让错误暂时消失，却扩大了 Web 进程在被攻陷后可修改的内容。

③ [知识点] SELinux 类型不会绕过 DAC

目录已经标记为可写类型，不代表 `httpd` 一定能写入。路径上每一级目录的搜索权限、目标目录的所有者和模式、ACL 等 DAC 条件仍然必须允许。反过来，`chmod 777` 也不会修复不匹配的 SELinux 类型。

建议的分层证据：

```
namei -l /srv/site/uploads/file.dat
ls -Zd /srv/site/uploads
matchpathcon /srv/site/uploads
```

④ [边界] 不要从服务名称机械推导类型

策略包可能为同一服务提供多个用途不同的类型，名称也可能随策略更新。选择前应查看服务对应的 SELinux 手册页、现有默认目录标签或官方文档，例如：

```
man httpd_selinux
matchpathcon /var/www/html
```

考试题的典型类型可以记忆，但真实工作中仍应以目标系统已安装策略为准。

[Cheatsheet] 类型声明用途：静态内容只读，写入目录单独划分；DAC 和 SELinux 必须同时允许。

[操作专题] 为非标准端口建立协议相关的 SELinux 类型

服务监听非标准端口时，应同时管理四个独立状态：应用配置中的端口、SELinux 端口类型、进程真实监听以及 `firewalld` 的网络放行。只修改其中一项不会自动同步其他层。

① [操作] 先查询类型、协议、端口和本地定制

```
semanage port -l | grep -w http_port_t
semanage port -C -l
```

第一条按类型查看策略允许的端口集合，第二条只列本地定制。还应检查目标端口是否已属于其他类型或端口范围。筛选输出时同时关注协议列，不能因为数字相同就把 TCP 和 UDP 记录混为一谈。

② [操作] 目标端口没有记录时使用 `-a`

```
semanage port -a -t http_port_t -p tcp 82
```

参数语义：

- `-a`：添加一条本地端口映射；
- `-t http_port_t`：指定目标 SELinux 端口类型；
- `-p tcp`：指定协议；
- `82`：端口号，也可以是命令支持的端口范围形式。

命令成功只说明本地策略存储接受了映射，不说明 `httpd` 配置正确或已经监听。

③ [操作] 已有记录且确需改变类型时使用 `-m`

```
semanage port -m -t http_port_t -p tcp 82
```

使用前必须确认：

1. TCP 82 当前确实已有记录；
2. 该记录不是另一个合法服务所需；
3. 题目或业务目标明确要求由 `httpd` 使用；
4. 修改不会破坏同机其他服务。

如果端口位于一个范围记录中，应先调查精确记录和影响面，不要假设修改单个数字一定是正确操作。

④ [操作] 删除不再需要的本地端口映射

```
semanage port -d -p tcp 82
```

删除后应重新查询端口列表，并确认没有仍在使用该映射的服务。删除 SELinux 映射不会自动修改应用配置、停止监听或移除防火墙规则，清理需要逐层处理。

⑤ [验证] 把五层证据分开

```
# 应用配置（示例为 Apache）
grep -R '^[[:space:]]*Listen' /etc/httpd/conf /etc/httpd/conf.d

# SELinux 端口类型
semanage port -l | grep -w http_port_t

# 服务当前与持久状态
systemctl is-active httpd.service
systemctl is-enabled httpd.service

# 真实监听
ss -lntp | grep ':82'

# firewallld 与最终功能
firewall-cmd --list-ports
curl http://localhost:82/
```

这些命令来自不同章节，本章只使用它们建立验收矩阵，不重新展开 `Apache`、`systemd` 或 `firewalld` 的完整机制。

[Cheatsheet] 端口身份是协议加数字；先查再 `-a/-m`；端口类型正确后仍要验证配置、服务、监听、防火墙和实际请求。

[操作专题] 从业务行为找到并持久设置 Boolean

Boolean 的关键不是记住尽可能多的名称，而是从受限进程正在尝试的合法业务行为出发，找到策略已经预留的精确开关，理解它会扩大哪类能力，然后建立当前和持久终态。

① [操作] 查询当前值并搜索相关开关

```
getsebool httpd_can_network_connect_db
getsebool -a | grep '^httpd_'
semanage boolean -l | grep -i httpd
```

`getsebool` 直接读取当前已加载策略中的值；`semanage boolean -l` 提供更完整的列表和描述，有助于区分名称相近但行为范围不同的开关。搜索只用于发现相关项，最终选择要回到业务语义。

② [操作] 不带 `-P` 只改变当前值

```
setsebool httpd_can_network_connect_db on
```

这种形式适合明确知道需要临时验证当前行为的场景。它不能证明重启或策略重新加载后仍保持。测试后应根据目标决定持久化或恢复原值，并重新触发原始业务。

③ [操作] 使用 `-P` 建立持久值

```
setsebool -P httpd_can_network_connect_db on
getsebool httpd_can_network_connect_db
```

`-P` 将设置写入持久策略配置，处理可能比普通当前值修改更慢。命令尚未返回时不要重复执行；完成后再查询当前值，并在后续真实环境验证重启或策略重新加载后的状态。

④ [判断] Boolean 必须与真实业务行为匹配

以下判断不是同一件事：

- Web 服务读取本地静态目录：优先检查文件类型；
- Web 服务绑定 TCP 82：优先检查端口类型；
- Web 应用主动连接 MariaDB：调查数据库网络连接 Boolean；
- Web 服务读取 NFS 上的内容：调查与网络文件系统相关的 Boolean 和挂载状态。

不要为了“让它能连出去”启用范围更宽的开关，除非业务确实需要更宽能力并经过风险评估。

⑤ [验证] 当前值正确不等于业务必然成功

Boolean 只控制策略条件。数据库地址、DNS、路由、防火墙、数据库账号、应用配置和服务状态仍然可能失败。Boolean 修改后应重新触发原始应用请求，并以最新日志和功能结果判断。

[Cheatsheet] 从业务行为搜索 Boolean；先读说明再设置；持久终态用 `-P`；当前值正确后仍需重做真实功能。

[诊断专题] `restorecon` 后标签仍不符合预期

文件上下文诊断的核心是判断“恢复工具没有执行”“当前标签没有变化”还是“规则本身给出了不正确的预期”。重复运行命令只能验证同一事实，下一条证据应能区分这些假设。

① [诊断] 症状：`restorecon` 没有改变标签

证据链：

```
症状：执行后 ls -Z 看起来不变
→ 当前证据：matchpathcon 与 ls -Z 是否本来就一致
→ 假设 A：对象已经符合预期
→ 假设 B：检查了目录内容，却没有检查目录本身
→ 假设 C：没有递归处理需要改变的后代对象
→ 下一证据：restorecon -nRv + matchpathcon -V
→ 最小修复：只对确认的目标范围应用 -Rv
→ 再验证：当前标签和原始业务
```

② [诊断] 症状：恢复后仍得到“错误”的类型

如果 `restorecon` 把对象改成了某个类型，但该类型不符合业务目标，说明工具正在按规则工作。下一步不是 `chcon`，而是：

```
matchpathcon /srv/site/index.html
semanage fcontext -C -l
```

检查是否存在旧本地规则、过宽表达式或后添加的重叠规则。修正规则后再应用 `restorecon`。

③ [诊断] 症状：新增窄规则后仍被宽规则覆盖

本地规则重叠时，以具体路径的 `matchpathcon` 结果为准。先确认哪条本地记录最后匹配，再通过修改、删除并重新添加必要规则调整优先关系。不要只根据正则字符串长度判断。

修复后至少检查两个代表性对象：


```
matchpathcon /srv/site/index.html
matchpathcon /srv/site/uploads/test.dat
```

确保只读区域和可写区域分别得到目标类型。

④ [诊断] 症状：删除规则后磁盘标签没有自动回退

`semanage fcontext -d` 只删除规则。随后执行：

```
matchpathcon /srv/site/index.html
restorecon -nRv /srv/site
```

如果预期回到了策略默认值，再执行最小范围的 `restorecon`。删除前后都应记录功能影响，避免误删现有业务所需的本地规则。

⑤ [诊断] 症状：文件移动后保留来源标签

不同文件操作对标签的影响不同。一个文件从用户家目录移动到 Web 目录后仍携带 `user_home_t` 时，优先按目标路径恢复：

```
restorecon -v /var/www/html/<FILE>
```

若目标目录本身是标准路径，通常不需要为单个文件创建新的 `fcontext` 规则。先使用已有默认规则，避免无意义的本地定制。

⑥ [安全边界] 变更前控制递归范围

在 `/srv` 或应用数据根目录上直接执行大范围 `restorecon -R` 可能改变多个业务对象。先用 `findmnt` 确认挂载边界、用 `restorecon -nRv` 预览，并把操作收缩到题目要求的目录。不要为了省步骤对整个系统执行无调查重标记。

[Cheatsheet] `restorecon` 服从规则；结果不对先查预期和本地覆盖；删除规则不会自动修改磁盘标签；递归前先预览范围。

[诊断专题] 端口已有类型、Boolean 无效与跨层证据推进

SELinux 状态正确但业务仍失败并不矛盾。文件规则、端口类型和 Boolean 只是完整访问链中的几个条件。高质量诊断应让每条命令排除一个假设，而不是看到局部成功就宣布完成。

① [诊断] `semanage port -a` 报端口已定义

```
症状：新增失败
→ 当前证据：协议、端口、现有类型或范围
→ 假设：端口已属于其他类型，或已位于范围记录
→ 下一证据：semanage port -l / -C -l
→ 最小修复：确认无业务冲突后使用 -m，或保留原映射并改服务端口
→ 再验证：端口类型、服务启动、真实监听
```

不要无条件删除已有记录再重建；这可能破坏另一个服务的合法映射。

② [诊断] 端口类型正确但服务仍无法监听

继续检查：

- 应用配置语法是否正确；
- 端口是否已被其他进程占用；
- 服务日志中的当前错误是否仍是 SELinux 拒绝；
- 配置是否被其他文件覆盖；
- 服务是否实际重启或重载了新配置。

`semanage port` 成功不能证明任何进程已经监听。

③ [诊断] Boolean 已为 on，但业务仍失败

重新触发失败并取得当前证据。常见分支包括：

```
目标文件类型错误
目标端口类型错误
DAC 不允许
应用地址或凭据错误
网络或远端服务不可达
选择的 Boolean 与行为不匹配
```

不要继续开启更多 Boolean 试错。Boolean 的名字相似不代表作用相同。

④ [诊断] 防火墙放行、服务 active，但外部访问失败

建立独立证据矩阵：

层	代表证据	只能证明
服务配置	语法检查、配置文件	应用计划采用的参数
服务当前状态	<code>systemctl is-active</code>	unit 当前运行状态
持久启动	<code>systemctl is-enabled</code>	重启时的启动配置
监听	<code>ss -lntp</code>	本机 socket 当前存在
firewalld	zone/端口查询	网络过滤层允许
SELinux	fcontext、port、Boolean	强制策略相关条件
功能	本机与远端协议请求	业务路径实际完成

一层成功不能替代其他层。

⑤ [诊断] 当前访问成功但担心重启后失效

至少检查：

- fcontext 本地规则存在；
- `matchpathcon` 与当前标签一致；
- 端口映射是本地持久记录；
- Boolean 使用了持久设置；
- 服务为 enabled；
- firewalld 修改使用了持久配置；
- 再次执行 `restorecon` 后功能仍正常。

本会话没有可控 RHEL 9 VM，重启级验收只作为推荐验证写入质量报告，不声明已完成。

⑥ [安全边界] 只修改能够解释当前拒绝的最小状态

修复前写出假设：“因为目标路径预期类型错误，所以修改 fcontext”“因为 TCP 82 已有不匹配类型，所以修改该端口映射”“因为业务确实需要 httpd 连接数据库，所以启用精确 Boolean”。如果不能把修改与证据连接起来，应继续调查。

[Cheatsheet] 新增失败先查现有记录；SELinux 状态正确后继续沿服务、监听、网络和应用层推进；每次只做能解释证据的最小修改。

[经典任务] 在自定义目录和 TCP 82 上发布 Web 服务

环境

一台 RHEL 9 主机已经安装 `httpd`。管理员准备将现有静态站点放在 `/srv/exam-site`，并让 Apache 监听 TCP 82。SELinux 处于 Enforcing；firewalld 正在运行。`/srv/exam-site/index.html` 已经存在，禁止删除、重建或改写其内容。

当前状态并不完全已知：

- Apache 配置可能已经改为 TCP 82，也可能存在语法错误；
- 目录和文件当前标签可能由 `chcon` 临时修改过；
- 系统可能存在旧 fcontext 或端口本地定制；
- TCP 82 可能尚未映射，也可能已属于其他类型；
- 服务当前和开机状态未知。

目标终态

1. `/srv/exam-site` 及其后代拥有持久、最小的 Web 只读文件上下文规则；
2. 当前目录和文件标签与预期规则一致；
3. TCP 82 按 TCP 协议属于适合 httpd 的端口类型；
4. `httpd.service` 当前运行，并配置为系统启动时自动启动；
5. 进程实际监听 TCP 82；
6. firewalld 允许必要的 TCP 82 访问；
7. 本机及题目指定客户端能够读取现有首页；

8. 再次执行 `restorecon -Rv /srv/exam-site` 后，标签和访问仍正确。

限制

- 不关闭或长期放宽 SELinux；
- 不使用 `chcon` 作为最终持久配置；
- 不给整个站点使用可写类型；
- 不生成自定义 SELinux 策略模块；
- 不删除现有端口映射后盲目重建；
- 不把 `active`、监听存在或单次 `curl` 成功单独当作完整终态。

验收证据

目标	推荐证据
规则存在	<code>semanage fcontext -C -l</code>
预期标签	<code>matchpathcon</code>
当前标签	<code>ls -Z</code>
端口类型	<code>semanage port -l</code>
当前/持久服务	<code>systemctl is-active</code> 、 <code>is-enabled</code>
真实监听	<code>ss -lntp</code>
防火墙	永久和当前规则查询
最终功能	本机与客户端 <code>curl</code>
抗临时标签	再次 <code>restorecon</code> 后重复访问

[参考解答] 自定义目录和 TCP 82 Web 服务

参考解答展示的是证据顺序，不是假设所有机器都处于同一初始状态。看到已有规则时必须根据实际查询选择 `-m`，不能机械复制 `-a`。

① 调查服务配置、当前状态和普通权限

```
httpd -t
grep -R '^[:space:]]*Listen' /etc/httpd/conf /etc/httpd/conf.d
systemctl status httpd.service --no-pager
namei -l /srv/exam-site/index.html
ls -Zd /srv/exam-site
ls -lZ /srv/exam-site
```

先修正题目范围内的 Apache 语法错误和普通路径权限。不要用扩大到 `777` 的方式“排除权限”；只确保服务能够穿越路径并读取现有文件。

② 调查文件规则、预期和当前标签

```
semanage fcontext -C -l | grep -F '/srv/exam-site'
matchpathcon /srv/exam-site
matchpathcon /srv/exam-site/index.html
restorecon -nRv /srv/exam-site
```

若目标表达式不存在，新增：

```
semanage fcontext -a -t httpd_sys_content_t '/srv/exam-site(/.*)?'
```

若同一表达式已经存在但类型错误，确认后修改：

```
semanage fcontext -m -t httpd_sys_content_t '/srv/exam-site(/.*)?'
```

应用并检查：

```
restorecon -Rv /srv/exam-site
matchpathcon -V /srv/exam-site
matchpathcon -V /srv/exam-site/index.html
ls -Zd /srv/exam-site
ls -LZ /srv/exam-site
```

③ 调查并配置 TCP 82 的端口类型

```
semanage port -l | grep -E '(^|[:space:])82([,[:space:]]|$)'
```

```
semanage port -l | grep -w http_port_t
```

```
semanage port -C -l
```

若 TCP 82 没有任何映射：

```
semanage port -a -t http_port_t -p tcp 82
```

若 TCP 82 已有记录且确认应改由 httpd 使用、不会破坏其他服务：

```
semanage port -m -t http_port_t -p tcp 82
```

再次查询，确认协议和类型。

④ 启动服务并验证真实监听

```
systemctl enable --now httpd.service
systemctl is-active httpd.service
systemctl is-enabled httpd.service
ss -lntp | grep ':82'
```

若服务仍失败，查看当前服务日志和近期 AVC，判断错误是否仍然与端口或文件访问有关。不要根据旧日志继续修改。

⑤ 处理 firewalld 并验证功能

firewalld 的完整配置归第 21 章。此处只按题目所用 zone 做最小持久放行，并核对当前和永久状态。例如目标 zone 已确认后：

```
firewall-cmd --permanent --add-port=82/tcp
firewall-cmd --reload
firewall-cmd --list-ports
firewall-cmd --permanent --list-ports
```

本机功能验证：

```
curl http://localhost:82/
```

再从题目指定客户端验证，避免只证明回环路径。

⑥ 证明配置不依赖临时标签

```
restorecon -Rv /srv/exam-site
matchpathcon -V /srv/exam-site/index.html
curl http://localhost:82/
```

最终记录：fcontext 本地规则、预期与当前标签、TCP 端口类型、服务 active/enabled、真实监听、firewalld 当前/永久规则和最终页面请求。只有这些证据共同满足，才能认为配置形成完整闭环。

典型错误

- 把临时标签或机械新增当作持久配置：chcon 后只看 ls -Z，或 TCP 82 已有记录时仍重复 -a；
- 把局部状态或扩大授权当作完整功能：只看 systemctl status 或本机 curl，给静态站点整个目录可写类型，且页面成功后不再执行 restorecon 复验。

[经典任务] 最小放行 Web 应用连接 MariaDB

环境

`httpd.service` 当前运行，站点页面能够正常加载。文件上下文、监听端口、`firewalld` 和数据库网络连通性已经由题面确认。应用需要连接题面指定的 MariaDB 服务；数据库地址、端口和凭据配置已核对。SELinux 为 Enforcing，重新触发请求后，近期拒绝证据指向 `httpd_t` 主动建立数据库网络连接的行为。

目标终态

- 从目标系统策略中查找并理解与 `httpd` 连接数据库相匹配的 Boolean；
- 只启用必要的精确开关；
- 修改在策略重新加载或系统重启后保持；
- 重新触发应用数据库操作并成功；
- 不启用范围更宽的无关网络连接开关，不生成本地策略模块。

[参考解答] 最小放行 Web 应用连接 MariaDB

① 记录当前 Boolean 和业务证据

```
getsebool httpd_can_network_connect_db
semanage boolean -l | grep -i 'httpd.*database\|httpd_can_network_connect_db'
```

确认开关说明与“Web 应用主动连接数据库”一致。不要只因为名称包含 `httpd` 就启用。

② 建立持久终态

```
setsebool -P httpd_can_network_connect_db on
getsebool httpd_can_network_connect_db
```

等待命令完成后再查询，不要并发或重复运行多个 `-P` 操作。

③ 重新触发并分层验证

重新执行应用中需要数据库的具体操作。若仍失败，取得当前应用日志、服务日志和最新 AVC，再检查：

- 应用实际连接的主机和端口；
- 数据库服务是否可达；
- 防火墙和路由；
- 数据库账号与权限；
- 是否选择了错误的 Boolean；
- 是否还存在文件或端口类型问题。

Boolean 为 on 只是策略状态证据，最终验收是目标应用操作成功。

④ 典型错误

- 直接启用范围更宽的 `httpd_can_network_connect`，但题目只要求数据库连接；
- 不读 Boolean 描述，仅凭网上示例复制；
- 不带 `-P`，只修改当前值；
- `setsebool -P` 尚未完成就重复执行；
- Boolean 已开启后不重新触发应用，只看命令返回码。

[知识专题] 从考试操作迁移到真实变更和自动化

手工命令的价值不仅是完成一道题，还要形成可以被审计和自动化的状态模型。文件规则、端口映射和 Boolean 都具有清晰的“查询现状—声明终态—应用—验证”结构，适合后续由 Ansible 模块或 RHEL System Role 管理。但自动化不能替代前面的对象判断。

① [工作迁移] 变更前保存本地定制基线

```
semanage fcontext -C -l
semanage port -C -l
semanage boolean -l
```

真实系统可能已经存在历史定制。先保存基线，确认目标规则是否被其他服务使用，再设计最小变更和回滚条件。

② [工作迁移] 把“命令成功”改写成状态验收

自动化任务返回 changed 或命令退出码为 0，只证明工具执行成功。验收仍应包含：

声明状态存在

→ 当前内核或文件状态已应用

→ 服务重新加载了相关配置

→ 业务功能成功

→ 重启级持久性待验证

③ [工作迁移] 保留可逆性和影响范围

删除或修改本地策略记录前记录旧值；对 fcontext 先预览递归变化；对端口确认同机服务冲突；对 Boolean 记录原值和业务理由。回滚不是简单执行相反命令，而是恢复原状态后重新验证受影响业务。

④ [边界] 本章不展开自动化模块

Ansible 中管理 SELinux 文件规则、端口和 Boolean 的模块字段、collection 版本和幂等性属于 RHCE 自动化章节。本章只提供稳定手工对象模型，作为后续自动化的输入。

[Cheatsheet] 变更前保存本地定制；命令成功不是终态；回滚要恢复状态并重做业务验证。

[本章收束] 用最小持久修改完成证据闭环

本章的核心不是背三组命令，而是始终先判断正在改变哪个对象。文件路径、协议端口和策略可选行为属于三个不同的策略入口；当前标签、持久规则和最终功能也属于三个不同的状态层。把它们混在一起，就会出现“标签看起来正确但没有规则”“端口类型正确但没有监听”“Boolean 已开启但业务仍失败”这类误判。

稳定的工作方法是：

确认失败层次

→ 建立当前证据

→ 选择文件规则、端口类型或 Boolean

→ 做最窄的持久修改

→ 将声明应用到当前对象

→ 重新触发原始业务

→ 分层验证当前、持久与功能状态

主要判断表

现象或目标	首要对象	下一条有区分度的证据	典型持久入口	不能单独证明
自定义目录无法被服务读取	路径与文件类型	matchpathcon 对比 ls -Z	semanage fcontext + restorecon	服务已可用
restorecon 后仍是错误类型	fcontext 规则	semanage fcontext -C -l	修改或删除冲突规则	业务根因已消失
服务不能绑定非标准端口	协议、端口与 port type	semanage port -l 并核对协议	semanage port -a/-m	进程已经监听
业务需要策略预留的可选行为	Boolean	semanage boolean -l 阅读语义	setsebool -P	网络、账号或应用配置正确
页面当前能够访问	完整访问链	再查持久规则并重做 restorecon	各层分别持久化	重启后仍成立
命令返回成功	工具所管理的局部状态	查询真实状态并触发功能	依对象而定	整体终态正确

工作方法

- 调查只收集会改变决策的证据，不用重复执行失败命令代替分析。
- 新增前先查询；已有记录时区分修改、保留和删除，不无条件重建。
- 递归重标记前先预览范围；可写类型只授予真正需要写入的最小子目录。
- 每条验证命令都注明证明边界：规则存在不等于当前已应用，监听存在不等于远端可访问，Boolean 为 on 不等于应用功能成功。
- 本章没有连接 RHEL 9 虚拟机；命令和流程为课程、官方文档与手册页基础上的静态核对，重启级与端到端功能验证仍需目标环境执行。

向下一章交接

下一章进入启动链、GRUB、Target 与系统恢复。本章只留下一个必要接口：chcon 之类的当前标签修改可能在恢复标签或系统重新标记时被覆盖，而持久 fcontext 规则描述路径未来应获得的类型。系统重新标记的启动与恢复流程归下一章；本章不提前展开 GRUB、救援目标或全系统 relabel 操作。

启动链、GRUB、Target 与系统恢复

从固件到 PID 1：先判断启动阶段与真实根，再完成最小修复和重启验收。

对象模型操作语义验证诊断经典任务

大字号阅读版

本章阅读导航

先抓住一条主线：启动不是一个瞬间，而是 firmware、GRUB、kernel、initramfs、真实根和 systemd 逐层移交控制权。恢复时先判断当前阶段和路径语境，再区分一次性状态、持久配置与最终功能。

01定位启动阶段先判断控制权已经到达哪里

02识别当前真实根区分 /、/sysroot 与 chroot 后的 /

03区分状态寿命临时参数、持久条目、default target

04选择最小操作只改变有证据支持的对象

05完成分层验证配置、当前状态与功能分别证明

06受控重启复验保证控制台、回退与同对象复查

专题地图 | 类型 | 专题 | |---| | 知识专题 | 从按下电源到 PID 1：启动链如何逐层交接 | | 操作专题 | 查询当前内核、启动条目与 GRUB environment | | 操作专题 | 临时启动参数与持久启动参数不能互相代替 | | 知识专题 | Target 的当前状态、默认状态与一次性目标 | | 知识专题 | rescue、emergency 与 `rd.break` 的环境边界 | | 操作专题 | 使用 `rd.break` 恢复 root 口令并保持 SELinux 正确 | | 诊断专题 | `fstab` 错误、启动证据与重启风险 | | 经典任务 | root 口令恢复与 `fstab` emergency 诊断 |

阅读时持续回答 1. 当前故障发生在 firmware、GRUB、initramfs 还是 systemd？ 2. 当前 `` 是临时根还是真实根？ 3. 本次修改只影响一次启动，还是写入持久状态？ 4. `active target` 与 `default target` 分别回答什么？ 5. 当前证据能证明什么，又不能证明什么？ 6. 下一条最有区分度的证据是什么？ 7. 重启前是否有控制台、回退条目和明确验收项？

学习提示：遇到启动与恢复任务时，始终先确认启动阶段、真实根和状态寿命；任何修改都应由证据驱动，并为重启后的同对象复验预留控制台与回退路径。

第 30 章 • 正文

一台 Linux 主机从按下电源到出现登录提示符，并不是“系统启动了”这样一个不可再分的动作。控制权会在固件、引导加载程序、内核、initramfs、真实根文件系统和 systemd 之间逐层移交。每一层读取的配置不同，能产生的证据不同，能够安全执行的修复也不同。

恢复章节最危险的误判，通常不是不会敲命令，而是没有确认自己究竟处在哪一层：在 initramfs 的 `/` 中修改口令、把 GRUB 菜单中的一次性参数误认为持久配置、把 `set-default` 误认为当前已经切换，或在 `fstab` 尚未闭环时用重启反复试错。它们都属于“操作对象或状态寿命判断错误”。

本章按“启动对象 → 状态寿命 → 查询接口 → 恢复操作 → 证据链 → 重启验收”推进。systemd unit 的基础控制留给第 12 章；`fstab` 完整语法留给第 24 章；SELinux 模式和上下文模型留给第 28 章。本章只引入完成启动恢复所需的接口与边界。

概念

Firmware 是 BIOS 或 UEFI 等固件环境，负责基本硬件初始化、识别启动设备并进入下一阶段。它不理解 `/etc/fstab`、systemd unit 或 root 口令；如果尚未进入 kernel，systemd Journal 也不能为这一层提供证据。

概念

Boot Loader / GRUB 负责选择启动条目，加载 kernel 与 initramfs，并把 kernel command line 传给下一阶段。菜单中按 ``e`` 的编辑通常只影响一次启动；``grubby`` 管理的启动条目才承担持久参数，因此两者不能互相替代。

概念

Kernel 与 **initramfs** 分别承担内核初始化和早期用户空间。initramfs 包含发现真实根所需的驱动、工具与逻辑，把真实根挂载到 `/sysroot`` 后再完成根切换；因此 initramfs 不是磁盘系统的普通 `/``。

概念

rootfs 与 `/sysroot`` 是恢复操作中必须区分的路径语境。在 ``rd.break`` 中，当前 `/`` 是临时根，真实系统通常位于 `/sysroot``；只有重挂载正确对象并进入 ``chroot /sysroot``、`/etc/shadow``、`/etc/fstab`` 和 `/.autorelabel`` 才指向目标系统。

概念

systemd target 是 unit 依赖集合与同步点，不是一个持续运行的“运行级别进程”。active target 描述当前已经到达的状态，default target 描述正常启动入口，``systemd.unit=`` 则可以只覆盖一次启动。

概念

rescue、**emergency** 与 ``rd.break`` 都能提供维护入口，但不在同一阶段。rescue 和 emergency 属于主系统的 systemd 目标；``rd.break`` 在 initramfs 中暂停。环境不同，真实根位置、可用 unit、挂载状态和证据入口也不同。

概念

``chroot`` 改变当前进程及其子进程的路径解析根，使 `/etc``、`/var`` 和 `/`` 指向指定目录。它不会启动新 kernel、创建完整 namespace 或自动生成新的 systemd，因此“进入 chroot”只解决作用路径，不等于启动了一个独立系统。

概念

SELinux relabel 是在口令恢复后重新建立文件安全上下文的持久恢复步骤。``touch /.autorelabel`` 只是向下一次启动发出完整重标记请求；它不能证明重标记已经完成，也不能代替启动后对模式、失败 unit 和登录功能的检查。

操作语义速查

操作语义以下接口分别观察启动条目、GRUB 环境、target 状态、真实根、账号数据库、SELinux 重标记请求和启动证据。先确认作用对象，再决定参数。

grubby

SYNOPSIS

```
grubby --info=ALL
grubby --default-kernel
grubby --update-kernel=KERNEL --args="PARAMETERS"
grubby --update-kernel=KERNEL --remove-args="PARAMETERS"
```

查询或修改 RHEL 管理的 kernel 启动条目。它回答未来条目状态，不直接证明本次运行参数或重启成功。

`--info=ALL`

列出全部启动条目及关键字段。

`--default-kernel`

显示默认 kernel 路径。

`--update-kernel=ALL`

把后续参数操作作用于全部现有条目；使用前评估是否需要保留未修改回退条目。

`--args=...`

向目标条目持久添加参数。

`--remove-args=...`

从目标条目持久删除指定参数。

grub2-editenv

SYNOPSIS

```
grub2-editenv - list
grub2-editenv - unset menu_auto_hide
```

读取或修改默认 GRUB environment block。它不等同于 BLS 启动条目，也不等同于本次 kernel 的 `/proc/cmdline`。

`- list`

读取默认环境块中的变量。

`- unset menu_auto_hide`

取消自动隐藏菜单；这属于持久显示策略，不是一次 root 口令恢复的必要步骤。

systemctl target 组

SYNOPSIS

```
systemctl get-default
systemctl set-default TARGET
systemctl isolate TARGET
systemctl list-units --type=target --state=active
```

分开观察和修改 default target 与当前 active target。`set-default` 不立即切换，`isolate` 可能停止当前管理通道。

`get-default`

查询正常启动入口。

`set-default TARGET`

修改持久默认目标。

`isolate TARGET`

把当前系统切换到目标依赖集合，可能停止不再需要的 unit。

`systemd.unit=TARGET`

作为 kernel 参数，只覆盖一次启动目标。

findmnt / mount

SYNOPSIS

```
findmnt /
findmnt /sysroot
mount -o remount,rw /sysroot
```

先确认当前路径对应哪个挂载对象及其选项，再把真实根重新挂载为可写。不要凭提示符猜测 `/` 的语境。

findmnt PATH

显示目标、源、文件系统类型和挂载选项。

-o remount,rw

在不重新建立挂载的前提下，把指定挂载改为读写。

/sysroot

只在 initramfs 恢复语境中作为真实根的可能位置，仍应先查询确认。

chroot / passwd

SYNOPSIS

```
chroot /sysroot
passwd root
```

把路径解析根切换到真实系统，再修改真实账号数据库。命令成功不等于作用对象正确，顺序不能颠倒。

chroot /sysroot

让后续绝对路径相对于真实根解析。

passwd root

交互式修改目标系统 root 口令；不要把明文口令写入命令历史。

touch /.autorelabel

SYNOPSIS

```
touch /.autorelabel
```

在真实根中创建重标记请求，让下一次正常启动重新建立 SELinux 文件上下文。它不是即时执行命令。

/.autorelabel

必须在 `chroot /sysroot` 之后创建，确保落在目标系统根目录。

启动边界

重标记可能耗时，不能强制中断；完成后仍检查 SELinux 模式、failed unit 和登录功能。

journalctl boot 组

SYNOPSIS

```
journalctl -b
journalctl -k -b
journalctl --list-boots
journalctl -b -1
```

按启动轮次读取 systemd Journal。它适合 kernel、initramfs 后段和 systemd 阶段，但不能还原 firmware 尚未进入 kernel 的失败。

-b

限定当前启动。

-k -b

限定当前启动中的 kernel 消息。

--list-boots

列出当前可查询的启动轮次。

-b -1

读取上一次可用启动；无结果不等于上次没有事件。

[知识专题] 从按下电源到 PID 1：启动链如何逐层交接

排查启动故障时，最先要回答的不是“用哪个修复命令”，而是“控制权已经走到哪里”。如果屏幕还停留在固件界面，`journalctl` 无能为力；如果 `rd.break` 已经中断在 `initramfs`，`/etc/shadow` 也不在当前 `/` 下；如果 `systemd` 已经进入 `emergency`，则 `failed unit` 和本次启动 `Journal` 才是高价值证据。启动阶段决定了证据入口和修复边界。

启动链纵向主模型

01

Firmware 初始化硬件并选择可启动入口

↓

02

GRUB / Boot Loader 选择条目，加载 kernel 与 `initramfs`，传递 `command line`

↓

03

Kernel 初始化内核能力并启动早期用户空间

↓

04

`initramfs` 发现存储，把真实根挂载到 `/sysroot`

↓

05

`switch_root` / `rootfs` 从临时根切换到磁盘上的真实系统

↓

06

`systemd` PID 1 按 `default target` 及依赖启动 `unit`

↓

07

Target 与功能终态到达同步点后仍需验证挂载、服务和登录功能

① [知识点] firmware 选择启动设备，但不理解 Linux 的服务状态

主机接通电源后，BIOS 或 UEFI 完成基本初始化并按照固件配置选择可启动设备。传统 BIOS 与 UEFI 的磁盘布局和启动文件位置不同，但本章只需要掌握共同边界：firmware 的任务是找到并启动下一阶段，而不是读取 `/etc/fstab`、启动 `systemd` 服务或验证 `root` 密码。

固件阶段常见证据包括：

- 固件或 BMC 控制台上的硬件自检信息；
- 启动设备顺序；
- 是否能看到目标磁盘或 EFI 启动项；
- 是否已经出现 GRUB 菜单或 GRUB 错误。

如果目标磁盘根本没有被固件识别，继续修改 `grub.cfg` 不会解决问题。反过来，只要已经看到 GRUB 菜单，就说明控制权至少已离开 `firmware` 层。

② [知识点] GRUB 把启动条目解析为 kernel、initramfs 与 command line

GRUB 的核心输出不是“一个菜单”，而是一组可启动条目。一个条目至少关联：

- `kernel image`；
- `initramfs image`；
- `kernel command line`；
- 标题或条目标识；
- 可能的默认选择与回退信息。

RHEL 9 使用 Boot Loader Specification (BLS) 风格的启动条目，常见条目位于 `/boot/loader/entries/`。管理员通常通过 `grubby` 查询和修改条目，而不是直接手工改生成后的 GRUB 主配置。GRUB `environment` 则保存类似菜单隐藏、已保存条目等引导环境变量，由 `grub2-editenv` 读取和修改。

GRUB 菜单中的按 `e` 编辑属于一次性启动编辑。它能临时追加 `rd.break`、`systemd.unit=` 或其他 `kernel` 参数，但正常情况下不会回写 BLS 条目。一次性编辑适合恢复和测试；需要长期生效时，必须改持久条目并再次查询确认。

③ [知识点] kernel 与 initramfs 不是同一个对象

boot loader 把 `kernel` 与 `initramfs` 放入内存并移交控制权。`kernel` 初始化内存管理、调度、设备模型等内核能力；`initramfs` 则提供一个临时的早期用户空间，包含启动所需的工具、脚本、`systemd` 组件和驱动模块。

`initramfs` 的价值在于解决“要挂载真实根，先得有访问真实根所需能力”的循环依赖。例如真实根位于 LVM、RAID、特定存储控制器或加密设备上，早期用户空间需要先装载相应模块、发现设备并建立映射，才能找到真正的 `/`。

`dracut` 用于生成 `initramfs`。它是重要帮助入口，但不能把“重建 `initramfs`”当作任何启动故障的默认答案。只有证据指向 `initramfs` 缺少模块、内容损坏或启动环境与硬件不匹配时，重建才是合理假设。

④ [知识点] `initramfs` 中的 `/` 与真实系统的 `/sysroot` 必须分开

在 `initramfs` 阶段，当前 `/` 是内存中的临时根。真实磁盘根通常被挂载到 `/sysroot`。这解释了 `rd.break` 口令恢复的两条关键命令：

```
mount -o remount,rw /sysroot
chroot /sysroot
```

第一条改变真实根挂载的读写属性；第二条改变当前进程的路径解析根。执行 `chroot /sysroot` 后，`/etc/shadow`、`/etc/fstab` 和 `/.autorelabel` 才指向目标系统中的文件。

如果在 `chroot` 前直接运行 `passwd root`，即使命令能够执行，它也可能操作 `initramfs` 环境中的文件，而不是磁盘系统的账号数据库。因此“命令成功”不能代替“作用对象正确”。

⑤ [知识点] 根切换后，磁盘系统中的 `systemd` 成为 PID 1

`initramfs` 完成真实根挂载后，启动流程通过根切换进入磁盘系统。之后 `systemd` 读取系统中的 `unit`、生成器输出和 `default target`，逐步启动 `mount`、`swap`、`service`、`socket` 等对象，最终达到文本登录、图形登录或指定维护目标。

这一阶段的高价值证据包括：

```
systemctl --failed
systemctl list-units --type=target --state=active
journalctl -b
journalctl -k -b
```

`systemctl --failed` 只列出 `systemd` 已知的失败 `unit`；`journalctl -b` 查看本次启动的记录；`journalctl -k -b` 聚焦本次启动的 `kernel` 消息。它们能帮助定位 `systemd`、`mount` 或 `kernel` 阶段的失败，却不能追溯尚未进入 `kernel` 的 `firmware` 故障。

Cheatsheet

```
firmware
→ GRUB 选择条目并传递 command line
→ kernel + initramfs
→ 真实根挂载到 /sysroot
→ switch_root
→ systemd PID 1
→ default target
```

[操作专题] 查询当前内核、启动条目与 GRUB environment

启动变更前必须先建立基线。查询时至少区分三种事实：当前正在运行什么、下次默认会启动什么、全部可用条目分别配置了什么。只看其中一个维度，很容易把当前运行状态误认为持久启动状态。

① [操作] 查询当前 `kernel` 与当前真正生效的 `command line`

作用对象：当前运行中的 `kernel`。

基本语义：`uname -r` 显示当前 `kernel release`；`/proc/cmdline` 显示本次启动时 `kernel` 实际收到的参数。

```
uname -r
cat /proc/cmdline
```

典型判断：

- `uname -r` 不能证明下一次默认仍会启动同一 `kernel`；
- `/proc/cmdline` 能证明本次参数，但不能证明持久条目已经更新；
- 临时在 GRUB 菜单追加的参数会出现在 `/proc/cmdline`，即使持久条目没有变化。

② [操作] 使用 `grubby` 查询默认 `kernel` 与全部启动条目

作用对象：RHEL 管理的 `kernel` 启动条目。

```
grubby --default-kernel
grubby --info=ALL
```

常见输出字段可能包括 `index`、`kernel`、`args`、`root`、`initrd`、`title` 和 `id`。审核时重点理解字段语义，而不要把某个小版本的字段顺序背成固定模板。

`index` 可能随着 `kernel` 安装、删除或条目变化而改变。需要长期引用某个条目时，`kernel` 路径、条目 ID 或明确标题通常比记住裸索引更稳妥。

③ [操作] 使用 `grub2-editenv` 读取 GRUB environment

作用对象：GRUB environment block 中的变量。

```
grub2-editenv - list
```

这个命令可用于查看保存的条目、菜单隐藏等环境变量。它不等同于读取全部 BLS 启动条目，也不等同于查看当前 `kernel command line`。三者分别回答：

```
GRUB environment: 引导环境变量是什么
BLS/grubby: 启动条目是什么
/proc/cmdline: 本次 kernel 实际收到什么
```

④ [操作] 在菜单可能隐藏时进入 GRUB

RHEL 9 的某些版本和安装状态可能默认隐藏 GRUB 菜单。启动时可尝试按 `Esc`、`F8` 或按住 `Shift` 进入菜单，具体按键时机受固件、控制台和虚拟化环境影响。

如果业务需求是让菜单长期显示，可以调查 `menu_auto_hide` 环境变量；但为了执行一次 `rd.break` 恢复，没有必要先永久改变菜单显示策略。恢复操作应尽量减少持久变更。

⑤ [操作] 建立变更前基线

在修改 `kernel` 参数或默认条目前，推荐保存以下证据：

```
uname -r
cat /proc/cmdline
grubby --default-kernel
grubby --info=ALL
grub2-editenv - list
```

工作环境中还应把输出保存到变更记录。考试环境中至少要在头脑中明确：当前 `kernel`、默认 `kernel`、目标条目和需要修改的参数分别是什么。

验证与边界：查询命令本身不会证明目标 `kernel` 可以成功启动。持久条目修改后的最终验证仍需要受控重启，但应先完成静态核对和风险评估。

Cheatsheet

```
当前 kernel: uname -r
当前参数: cat /proc/cmdline
默认 kernel: grubby --default-kernel
全部条目: grubby --info=ALL
GRUB 环境: grub2-editenv - list
```

[操作专题] 临时启动参数与持久启动参数不能互相代替

同一个 `kernel` 参数可以通过 GRUB 菜单临时追加，也可以通过 `grubby` 写入启动条目。两种方式看起来都能让参数“生效”，但它们的生命周期完全不同。恢复时偏向一次性参数；长期配置则必须修改持久条目，并在变更前后分别查询。

① [操作] 在 GRUB 菜单中临时编辑一次启动

作用对象：当前选中的一次启动过程。

典型流程：

1. 进入 GRUB 菜单；
2. 选择要启动的条目；
3. 按 `e` 编辑；
4. 定位包含 `kernel` 路径和现有参数的行；
5. 在行末追加所需参数；
6. 按界面提示继续启动，常见为 `Ctrl+x`。

例如进入 `initramfs` 断点：

```
rd.break
```

或一次性进入 rescue:

```
systemd.unit=rescue.target
```

只追加需要的参数，不把“删除其他参数直到只剩 `ro`”写成通用步骤。无证据删除 `root=`、LVM、console、crashkernel 或其他参数，可能制造新的启动故障。

② [操作] 用 grubby 持久添加 kernel 参数

作用对象：指定 kernel 条目或全部 kernel 条目。

```
grubby --update-kernel=ALL --args="<PARAMETERS>"
```

对单一 kernel 可用明确的 kernel 路径替换 ALL。选择范围前先回答：

- 参数是否应用于所有已安装 kernel；
- 是否只为当前默认 kernel 添加；
- 是否需要保留一个未修改的已知可启动条目作为回退。

③ [操作] 用 grubby 删除持久 kernel 参数

```
grubby --update-kernel=ALL --remove-args="<PARAMETERS>"
```

删除时要传入需要移除的参数，而不是通过手工编辑大段启动行来“清理”。参数可能包含值，应以查询到的实际形式为准。

④ [操作] 分别验证未来条目与当前运行状态

修改后先查询条目：

```
grubby --info=ALL
```

在早期 RHEL 9.0 环境或完成 kernel 安装、删除之后，更应重新检查所有条目，不要假定新条目一定继承了预期参数。这里的安全原则不是记住某个版本特例，而是把“安装结果”重新还原成可查询的启动条目。

重启前，这只能证明条目文本发生变化。完成受控重启后，再检查：

```
cat /proc/cmdline
uname -r
```

二者形成两个验证层：

```
grubby 输出：未来启动条目
/proc/cmdline：本次实际生效参数
```

⑤ [边界] 不直接把生成后的 grub.cfg 当作首选配置真源

RHEL 9 的启动条目、BLS 和 GRUB 主配置存在生成关系。日常 kernel 参数管理优先使用 grubby。直接编辑生成后的 grub.cfg 容易被后续更新覆盖，也更容易误改复杂逻辑。

本章不完整展开 GRUB 重装、手工恢复 core image、Secure Boot 密钥或复杂串口菜单配置。只有启动条目和参数管理属于本章主线。

Cheatsheet

```
一次启动：GRUB 菜单按 e，追加参数
持久添加：grubby --update-kernel=... --args="..."
持久删除：grubby --update-kernel=... --remove-args="..."
条目验证：grubby --info=ALL
生效验证：cat /proc/cmdline
```

[知识专题] Target 的当前状态、默认状态与一次性启动目标

target 常被简化成传统“运行级别”的替代物，但真正有用的理解是：target 是一组 unit 依赖和同步关系。系统可以同时有多个 active target；default target 只是正常启动时的入口；`isolate` 改变当前状态；`systemd.unit=` 则只覆盖一次启动。把这些状态混在一起，会出现“已经 set-default，为什么现在还没切换”或“已经 isolate，为什么重启又回去了”的误判。

① [知识点] 常见 target 代表不同启动终态

Target	主要含义	典型用途
multi-user.target	多用户、文本环境及其依赖	常见服务器默认目标
graphical.target	在多用户基础上加入图形登录能力	桌面或图形管理环境
rescue.target	单用户维护环境，加载较多基本系统能力	维护真实系统
emergency.target	极小维护环境，只拉起非常少的依赖	处理严重启动或挂载故障

graphical.target 通常依赖 multi-user.target，因此系统可能同时显示多个 active target。不能把 list-units --type=target 的多行结果解释为“系统同时处于多个传统运行级别”。

② [操作] 查询和设置 default target

```
systemctl get-default
systemctl set-default multi-user.target
```

get-default 回答“正常启动默认从哪个 target 开始”；set-default 改变持久默认值，但不会自动停止当前图形会话或立即切换当前系统。

修改后再次执行：

```
systemctl get-default
```

这只能验证持久配置，不证明系统已经在新 target 的运行状态中。

③ [操作] 查询当前 active target

```
systemctl list-units --type=target --state=active
```

当前 active target 是运行状态证据。若要聚焦某个目标，可查询：

```
systemctl is-active multi-user.target
systemctl status rescue.target
```

active 只说明该 target 已达到，不自动证明该目标下的每个业务功能都健康。例如到达 multi-user.target 不等于自定义 Web 应用一定可用。

④ [操作] 使用 isolate 切换当前目标

```
systemctl isolate rescue.target
```

isolate 会启动目标所需 unit，并停止不属于目标依赖的其他 unit。它可能中断：

- 图形会话；
- SSH 或网络连接；
- 业务服务；
- 依赖被停止服务的监控或管理通道。

因此远程主机上执行前必须确认控制台或回退通道。考试环境也应理解该操作的当前影响，不把它当作无风险的“查看模式”。

⑤ [操作] 用 kernel 参数指定一次性启动 target

在 GRUB kernel command line 追加：

```
systemd.unit=rescue.target
```

或：

```
systemd.unit=emergency.target
```

该参数只影响本次启动，不改变 systemctl get-default 的持久默认目标。维护完成后，若当前 systemd 环境允许，可以尝试：

```
systemctl default
```


让系统回到 default target；也可以在风险评估后正常重启。

⑥ [比较] 当前、默认与一次性的三维状态

```
当前 active: systemctl list-units / is-active
持久 default: systemctl get-default / set-default
一次启动: systemd.unit=<target>
```

这三个维度必须分别验证。任何一个维度都不能替代另外两个。

Cheatsheet

```
默认: get-default / set-default
当前: list-units --type=target / isolate
一次启动: systemd.unit=
```

[知识专题] rescue、emergency 与 rd.break 的环境边界

三种入口都可能出现 root shell，因此最容易被混为同一模式。区分它们的关键不是提示符长什么样，而是 systemd 已经执行到哪里、真实根挂载在哪里、哪些依赖已经启动、修改哪个路径才会影响磁盘系统。

① [知识点] rescue.target 已经进入真实系统

rescue target 通常在真实根上运行，提供较完整的基本系统与单用户维护环境。它适合：

- 在真实系统中修改配置；
- 停止大部分业务后维护；
- 处理不需要早期用户空间介入的问题；
- 从当前系统切换到较小运行集合。

进入后仍应确认根挂载状态和必要文件系统，不要仅凭“rescue”名称假设所有挂载都可写。

② [知识点] emergency.target 只拉起极少依赖

emergency target 比 rescue 更小，常用于 mount、local-fs 或关键依赖失败后的维护。很多教材把它简写为“根只读”，但实际挂载状态与进入路径、生成的 mount unit 和系统版本有关。正确做法是查询：

```
findmnt /
findmnt -no OPTIONS /
```

需要写入时再根据证据执行：

```
mount -o remount,rw /
```

③ [知识点] rd.break 在 initramfs 阶段中断

rd.break 是 dracut 早期用户空间的断点参数。它在根切换前提供 shell，常用于 root 口令恢复。此时：

- 当前 / 属于 initramfs；
- 真实系统通常位于 /sysroot；
- /sysroot 可能以只读方式挂载；
- 正常系统的 systemd 尚未完整启动。

因此 systemctl get-default、普通服务状态和磁盘系统路径不能按正常启动环境理解。

④ [操作] 根据环境选择正确的重挂载对象

在正常系统、rescue 或 systemd emergency 中，目标通常是当前根：

```
findmnt /
mount -o remount,rw /
```

在 rd.break 的 initramfs shell 中，目标通常是真实根：

```
findmnt /sysroot
mount -o remount,rw /sysroot
```

操作前先查询，不把命令顺序背成脱离环境的口诀。

⑤ [知识点] `chroot` 只改变路径解析根，不创建完整新系统

```
chroot /sysroot
```

执行后，当前进程将 `/sysroot` 视作 `/`。它适合让 `passwd`、编辑器和路径操作作用于真实系统。但 `chroot`：

- 不启动新的 kernel；
- 不自动创建 PID、网络或 mount namespace；
- 不保证 `/proc`、`/sys`、`/dev` 都具备完整绑定；
- 不等于容器或虚拟机；
- 不会自动让磁盘系统的 `systemd` 成为当前 PID 1。

`root` 口令恢复只需要有限路径和命令，因此课程流程通常可以直接 `chroot /sysroot`。更复杂的外部救援环境可能需要额外挂载伪文件系统，但不属于本章经典任务主线。

⑥ [比较] 三种环境的判断表

入口	阶段	真实根常见位置	主要证据	典型用途
rescue	正常 systemd	<code>/</code>	<code>systemctl</code> 、 <code>Journal</code> 、 <code>findmnt /</code>	单用户维护
emergency	极小 systemd	<code>/</code>	failed unit、 <code>Journal</code> 、 <code>findmnt /</code>	严重依赖或挂载故障
<code>rd.break</code>	initramfs	<code>/sysroot</code>	<code>/proc/cmdline</code> 、 <code>findmnt /sysroot</code>	口令恢复、早期启动调试

Cheatsheet

rescue/emergency: 真实根通常是 `/`
`rd.break`: 真实根通常是 `/sysroot`
任何环境: 先 `findmnt`、再 `remount`

[操作专题] 使用 `rd.break` 恢复 root 口令并保持 SELinux 正确

`root` 口令恢复是本章最经典的考试任务，但真正的评分对象不只是 `passwd` 成功。完整终态包括：修改发生在真实根、口令数据库可持久使用、SELinux 标签在下次启动后正确、临时参数没有写入持久条目、系统能够正常进入目标状态。

① [操作] 在 GRUB 中只为本次启动追加 `rd.break`

1. 重启并进入 GRUB 菜单；
2. 选择目标 kernel 条目；
3. 按 `e` 编辑；
4. 定位 kernel command line；
5. 在现有参数末尾追加：

```
rd.break
```

6. 按界面提示继续启动，常见为 `Ctrl+x`。

不要无调查删除现有 `root=`、LVM、`console` 或其他参数。只添加断点参数即可达到恢复目的。

② [操作] 证明 `/sysroot` 是目标真实根并重挂载为读写

进入 shell 后先查询：

```
cat /proc/cmdline
findmnt /sysroot
```

确认 command line 包含 `rd.break`，并识别 `/sysroot` 的来源和挂载选项。然后执行：

```
mount -o remount,rw /sysroot
findmnt -no TARGET,SOURCE,FSTYPE,OPTIONS /sysroot
```

第二次 `findmnt` 用于验证 `rw` 已生效。不能把 `mount` 无报错扩大为“真实根一定可写”。

③ [操作] `chroot` 到真实系统后修改口令

```
chroot /sysroot
passwd root
```

推荐交互输入题目给出的新口令。这样避免把明文口令保存在命令行、Shell 历史或进程参数中。考试模拟答案中常见的管道或 `--stdin` 写法虽然可能可用，但不是本讲义默认安全答案。

完成后可检查命令退出状态，但不要尝试读取或展示 `/etc/shadow` 中的口令哈希作为“验证”。正确验证应在正常启动后进行认证。

④ [操作] 在真实根中创建 SELinux 重标记请求

仍处于 `chroot` 时执行：

```
touch /.autorelabel
ls -l /.autorelabel
```

修改口令会改写安全相关文件。在 SELinux Enforcing 系统上，从 `initramfs chroot` 修改文件后，应让下一次启动执行重标记。`touch /.autorelabel` 创建的是下一次启动请求，不是立即完成重标记。

如果在 `chroot` 前执行 `touch /.autorelabel`，文件会创建在 `initramfs` 的 `/` 中，退出后不会留在真实系统。因此路径语境是该步骤的核心。

⑤ [操作] 依次退出 `chroot` 与 `initramfs shell`

```
exit
exit
```

第一次退出 `chroot`，回到 `initramfs shell`；第二次退出断点 `shell`，让启动继续。也可以使用对应快捷键发送 EOF，但教学和答题时写出两次 `exit` 更能明确环境层次。

不要在重标记过程中强制断电。文件较多或存储较慢时，首次启动可能明显变长。

⑥ [验证] 正常启动后的分层验收

启动完成后至少检查：

```
cat /proc/cmdline
getenforce
systemctl --failed
journalctl -b -p err
```

验收含义：

- `/proc/cmdline` 不应因为本次临时编辑而把 `rd.break` 变成持久参数；
- `getenforce` 应符合原系统预期，不能以关闭 SELinux 代替恢复；
- `systemctl --failed` 用于发现本次启动的 `unit` 失败；
- `Journal` 用于确认重标记或启动过程中的错误；
- 最终还要用新 `root` 口令完成实际认证。

⑦ [典型错误] 命令顺序看似相似，作用对象却不同

错误	为什么错误	正确修正
在 <code>chroot</code> 前执行 <code>passwd root</code>	可能修改 <code>initramfs</code> 环境	先 <code>chroot /sysroot</code>
未重挂载 <code>/sysroot</code> 为 <code>rw</code>	口令或标记文件无法持久写入	<code>findmnt</code> 后 <code>remount</code>
在 <code>chroot</code> 前 <code>touch /.autorelabel</code>	标记文件创建在临时根	在 <code>chroot</code> 内创建
通过关闭 SELinux 避免重标记	改变安全终态并掩盖标签问题	保持原模式并重标记
把 <code>rd.break</code> 写入全部条目	恢复参数永久生效	只在 GRUB 菜单临时追加
重标记时强制重启	可能留下不完整标签状态	等待流程完成

```
GRUB 追加 rd.break
→ findmnt /sysroot
→ mount -o remount,rw /sysroot
→ chroot /sysroot
→ passwd root
→ touch /.autorelabel
→ exit
→ exit
→ 正常启动后分层验证
```

[诊断专题] `/etc/fstab` 错误为什么会把系统带入 emergency

systemd 会根据 `/etc/fstab` 生成 mount unit，并把关键本地文件系统纳入启动依赖。如果设备标识、文件系统类型、选项或目标不正确，mount unit 可能超时或失败，进而阻止 `local-fs.target` 达成，系统可能进入 emergency。正确修复不是“看到 emergency 就注释 fstab”，而是把 failed unit、日志、设备身份和具体配置行对应起来。

① [症状] emergency 是结果，不是根因

常见表征包括：

- 控制台提示进入 emergency mode；
- 某个 `.mount` unit 失败；
- 启动长时间等待设备；
- `local-fs.target` 或相关依赖未能完成；
- 提示检查 Journal 或输入 root 口令维护。

同样的表征可能来自不同原因：UUID 不存在、设备尚未出现、文件系统类型错误、选项拼写错误、挂载点冲突或远程文件系统依赖不满足。不要用单一修复猜全部故障。

② [当前证据] 先读取 failed unit 与本次启动日志

```
systemctl --failed
journalctl -b -p err
journalctl -b
```

如果已知失败 unit，可聚焦：

```
systemctl status <NAME.mount>
journalctl -b -u <NAME.mount>
```

目标是得到：

- 失败的 unit 名称；
- 对应挂载点；
- systemd 或 mount 报告的具体错误；
- 是否为设备超时、类型不符或选项错误。

③ [假设] 将 `.mount` unit 映射回挂载点和 `fstab` 行

systemd 的 mount unit 名称由挂载路径转义得到。例如 `/data` 常对应 `data.mount`，更深路径会被编码。可以使用：

```
systemd-escape --path --suffix=mount /data
```

然后检查：

```
grep -nEv '^\s*(#|$)' /etc/fstab
findmnt --fstab
```

不要一次删除或注释多行。先锁定失败 unit 对应的唯一记录。

④ [下一条证据] 比较配置声明与真实设备身份

```
lsblk -f
blkid
findmnt --verify
```

重点比较：

- `UUID=` 或 `LABEL=` 是否真实存在；
- `FSTYPE` 与 `fstab` 第三字段是否一致；
- 目标目录是否合理；
- 选项是否被当前文件系统支持；
- 设备是否本应在启动时存在；
- 远程文件系统是否需要网络依赖或自动挂载策略。

`findmnt --verify` 是静态检查入口，但它不能代替真实挂载。静态通过后仍需执行受控挂载验证。

⑤ [最小修复] 只修正有证据支持的字段

典型最小修复包括：

- 用 `blkid` 看到的真实 UUID 替换错误 UUID；
- 修正文件系统类型；
- 删除确认无效的选项；
- 修正目标目录；
- 对确属非关键、允许缺失的设备评估 `nofail`，但不能为了通过启动而掩盖必需挂载失败。

本章不教授格式化、重建文件系统或高风险修复工具。已有数据设备出现错误时，不执行 `mkfs`、`wipefs -a` 或未经调查的强制修复。

⑥ [再验证] 重新加载、模拟挂载并检查终态

修改 `/etc/fstab` 后：

```
systemctl daemon-reload
findmnt --verify
mount -a
findmnt <TARGET>
systemctl --failed
```

这些证据分别回答：

- `systemd` 是否重新读取生成关系；
- 静态语法和引用是否合理；
- 所有可尝试的 `fstab` 挂载是否执行；
- 目标路径实际挂载到什么源；
- 是否仍有失败 `unit`。

`mount -a` 无输出或退出成功仍不能证明目标挂载正确。必须用 `findmnt <TARGET>` 检查 `source`、`fstype` 和 `options`。

⑦ [重启验证] 当前闭环后再进行受控重启

只有在当前环境中完成以下条件后，才进入重启验证：

```
失败 unit 已定位
→ 配置只做最小修复
→ findmnt --verify 通过目标检查
→ mount -a 没有目标错误
→ findmnt 显示正确 source
→ systemctl --failed 无相关失败
```

重启后再次执行：

```
systemctl --failed
findmnt <TARGET>
journalctl -b -p err
```

这才证明持久启动路径正确。

```
emergency 症状
→ systemctl --failed
→ journalctl -b
→ 映射 mount unit 与 fstab
→ lsblk -f / blkid
→ findmnt --verify
→ 最小修复
→ daemon-reload + mount -a
→ findmnt 目标
→ 受控重启复验
```

[诊断专题] 按启动阶段选择证据：控制台、kernel log 与 boot Journal

“查日志”不是一条万能建议。日志系统本身也要经过启动过程才能工作。证据选择应跟随阶段：firmware 与 GRUB 主要依赖控制台；kernel 与 initramfs 依赖控制台和 kernel 消息；systemd 阶段才适合通过 boot Journal、failed unit 和 target 状态推进。

① [诊断] firmware 与 GRUB 前后的证据边界

如果屏幕停留在：

- BMC 或固件自检；
- 无可启动设备；
- EFI 启动项错误；
- GRUB 自身错误或菜单无法加载；

则 systemd Journal 尚未形成。优先保存控制台信息、固件启动顺序和 GRUB 表征。不要因为 `journalctl -b -1` 没有相关记录，就断言 firmware 没有失败。

② [诊断] kernel 与 initramfs 阶段

若已经看到 kernel 或 initramfs 错误，可关注：

- 当前 kernel command line；
- 根设备、LVM 或驱动发现信息；
- initramfs emergency shell；
- 控制台上的 kernel 报错。

如果系统后来成功进入用户空间，可以查询本次 kernel 记录：

```
journalctl -k -b
```

也可按关键字检索，但不要只靠 grep 一条错误就停止调查。需要结合前后文和时间顺序。

③ [诊断] systemd 阶段的三条主证据

```
systemctl --failed
journalctl -b
journalctl -b -p err
```

它们分别提供：

- systemd 当前标记为失败的 unit；
- 本次启动完整 Journal；
- 本次启动较高优先级的错误记录。

优先级筛选能减少噪声，但某些关键因果记录可能是 notice、warning 或普通信息，因此不能只保留 error 级别。

④ [诊断] 区分本次与前次启动

```
journalctl --list-boots
journalctl -b 0
journalctl -b -1
```

`-b 0` 是本次启动，`-b -1` 是上一次可用启动记录。前次记录能否查看，取决于 Journal 是否持久保存以及记录是否被轮转。没有前次记录不自动意味着系统从未失败。

⑤ [验证边界] 日志、target 与功能终态不是同一个证明

- Journal 中出现“Started”通常只说明 unit 启动动作成功；
- target active 只说明依赖同步点达到；
- `systemctl --failed` 为空不代表应用功能和数据都正确；
- 某条 error 也不一定是当前故障的根因。

启动诊断最终仍要回到目标：能否登录、目标文件系统是否正确挂载、服务是否提供功能、系统是否能在无需人工干预时启动。

Cheatsheet

```
firmware/GRUB: 控制台证据
kernel/initramfs: 控制台 + /proc/cmdline + kernel log
systemd: failed unit + journalctl -b
上次启动: --list-boots + -b -1
```

[诊断专题] 重启风险、回退路径与最小变更

启动相关变更的特殊风险在于：当前系统可能运行正常，而错误只有在下一次引导时才暴露。如果唯一的远程通道依赖当前网络和服务，错误重启可能直接失去管理能力。因此“配置完成，重启看看”不是稳健方法。

① [操作] 变更前记录可回退基线

```
uname -r
cat /proc/cmdline
grubby --default-kernel
grubby --info=ALL
systemctl get-default
systemctl --failed
findmnt --verify
```

对 `fstab` 或启动条目做工作变更时，还应备份原文件或保存差异。考试环境可以简化记录形式，但不能省略对象识别。

② [边界] 一次只改变一个启动维度

高风险组合包括同时：

- 更换默认 kernel；
- 修改全部 kernel 参数；
- 改 default target；
- 改多条 `fstab`；
- 重建 initramfs；
- 删除旧启动条目。

多个变量同时改变后，即使启动失败，也难以定位是哪一步造成。优先最小变更、逐层验证，并保留已知可启动条目。

③ [边界] `systemctl isolate` 可能切断当前管理通道

`isolate rescue.target` 或 `isolate emergency.target` 会停止目标不需要的 unit。远程执行前要确认：

- 是否有控制台或 BMC；
- 网络和 SSH 是否会被停止；
- 业务能否接受服务中断；
- 当前 Shell 是否属于即将被停止的会话。

考试任务若只要求设置下次默认目标，不应为了“验证”无必要地 isolate 当前系统。

④ [边界] SELinux 重标记不是普通快速重启

创建 `/.autorelabel` 后，下一次启动会遍历文件系统并恢复标签。系统规模越大，耗时越长。期间强制断电可能留下不完整状态。正确操作是：

- 保持控制台可见；
- 等待重标记和后续重启完成；
- 启动后检查 SELinux 模式、失败 unit 和 Journal；
- 不用永久关闭 SELinux 规避等待。

⑤ [边界] 不使用破坏性捷径掩盖启动故障

本章禁止把以下动作作为默认答案：

- 删除大段现有 kernel 参数；
- 关闭 SELinux；
- 对必需挂载随意加入 `nofail`；
- 未调查就重建 GRUB 或 initramfs；
- 对已有数据设备执行 `mkfs`、`wipefs -a`；
- 在重标记或文件系统写入中强制断电；
- 把“命令没有报错”当作重启后终态。

⑥ [验证] 重启前后使用同一组对象重新查询

重启前：

```
启动条目、default target、fstab、failed unit、目标挂载
```

重启后：

```
uname -r、/proc/cmdline、active target、目标挂载、failed unit、boot Journal
```

使用同一对象的前后证据，才能证明持久变更真正生效，而不是只在当前会话中暂时可用。

Cheatsheet

```
先基线
→ 最小变更
→ 静态与当前验证
→ 确认控制台/回退
→ 受控重启
→ 用同一对象复验
```

[经典任务] 使用 `rd.break` 恢复 root 口令

任务环境

一台启用了 SELinux Enforcing 的 RHEL 9 主机可以通过本地控制台访问。系统能够到达 GRUB，但 root 口令未知。题目要求把 root 口令修改为考场给定的新值。现有 default target、kernel 选择和其他 kernel 参数均不得永久改变。

当前状态

- 可以进入 GRUB 菜单并编辑目标启动条目；
- root 口令未知，无法以 root 正常认证；
- 系统使用 SELinux Enforcing；
- 当前没有理由关闭 SELinux 或重建 GRUB；
- 本会话没有可执行 live test 的 RHEL 9 VM。

目标终态

1. 真实磁盘系统的 root 口令已更新；
2. SELinux 保持原预期模式；
3. 下一次启动完成必要的文件标签重标记；
4. `rd.break` 只用于本次启动，不出现在持久条目；
5. default target 不变；
6. 系统能够正常启动并接受新 root 口令认证；
7. 无与本次恢复相关的失败 unit。

限制条件

- 不使用 `selinux=0` 或 `enforcing=0`；
- 不删除原 kernel command line 中的其他参数；
- 不把新口令明文写入命令历史或答案日志；
- 不把 `rd.break` 写入持久启动条目；

- 不在 SELinux 重标记过程中强制断电；
- 不以读取 `/etc/shadow` 哈希作为口令验证。

验收矩阵

层次	验收对象	推荐证据	能证明什么
阶段	initramfs 断点	<code>cat /proc/cmdline</code>	本次包含 <code>rd.break</code>
根	真实根	<code>findmnt /sysroot</code>	<code>/sysroot</code> 的来源与选项
写入	根可写	<code>findmnt -no OPTIONS /sysroot</code>	已包含 <code>rw</code>
作用域	chroot	当前路径语境	<code>passwd</code> 作用于真实系统
SELinux	重标记请求	<code>ls -l /.autorelabel</code>	文件创建在真实根
持久性	临时参数	正常启动后 <code>/proc/cmdline</code>	<code>rd.break</code> 未持久化
安全模式	SELinux	<code>getenforce</code>	模式保持预期
系统状态	unit 与日志	<code>systemctl --failed</code> 、 <code>journalctl -b</code>	启动未留下相关失败
功能	root 认证	使用新口令登录	目标口令真正可用

[参考解答] 使用 `rd.break` 恢复 root 口令

一、调查与进入正确阶段

1. 通过本地控制台重启主机并进入 GRUB 菜单。
2. 选择需要启动的 kernel 条目，按 `e` 编辑。
3. 定位 kernel command line，在现有参数末尾仅追加：

```
rd.break
```

4. 按界面提示继续启动，常见为 `Ctrl+x`。

判断边界：本步骤只需要增加断点，不删除其他参数，也不调用 `grubby`。使用 `grubby` 会把恢复参数写入持久条目，改变任务边界。

二、确认真实根并改为读写

进入 `initramfs shell` 后：

```
cat /proc/cmdline
findmnt /sysroot
```

确认本次参数包含 `rd.break`，并确认 `/sysroot` 是真实系统根。然后：

```
mount -o remount,rw /sysroot
findmnt -no TARGET,SOURCE,FSTYPE,OPTIONS /sysroot
```

只有看到 `rw` 后，才进入下一步。

三、进入真实系统并修改口令

```
chroot /sysroot
passwd root
```

根据交互提示输入题目给定的新口令。不要在答案中写出真实口令值，也不要使用包含明文口令的管道作为默认流程。

四、请求 SELinux 重标记

仍在 `chroot` 中执行：

```
touch /.autorelabel
ls -l /.autorelabel
```

这里的 `/` 已经是真实根。若先退出 `chroot` 再创建文件，标记请求会落在 `initramfs` 临时根中，无法影响正常系统。

五、退出两层环境并等待启动

```
exit
exit
```

第一次退出 chroot，第二次退出 initramfs 断点。系统继续启动并执行 SELinux 重标记。等待流程完成，不强制断电。

六、正常启动后的分层验证

```
cat /proc/cmdline
getenforce
systemctl get-default
systemctl --failed
journalctl -b -p err
```

然后使用新 root 口令完成实际认证。

验证解释：

- `/proc/cmdline` 不含持久 `rd.break`；
- `getenforce` 保持原安全终态；
- default target 未被改变；
- failed unit 与 Journal 没有与恢复相关的未解决错误；
- 实际认证证明口令终态，而不是只证明 `passwd` 命令返回成功。

七、典型错误与修复

- 错误：在 chroot 前运行 `passwd`。修复：重新确认 `/sysroot`，进入 chroot 后再改真实账号数据库。
- 错误：未把 `/sysroot` remount 为 `rw`。修复：用 `findmnt` 验证选项，不凭命令无输出猜测。
- 错误：在 initramfs 根创建 `/.autorelabel`。修复：在 chroot 内创建。
- 错误：用关闭 SELinux 规避标签问题。修复：保持原模式并完成重标记。
- 错误：重标记期间强制重启。修复：等待完成，启动后再检查 Journal。

[经典任务] 诊断错误 `fstab` 导致的 emergency

任务环境

一台 RHEL 9 主机在修改 `/etc/fstab` 后进入 emergency。目标数据文件系统仍存在且包含数据，不允许格式化或重新创建。emergency shell 可用，题目要求恢复无人干预启动，并确保目标文件系统仍按原计划持久挂载。

当前状态

- 至少一个 `.mount` unit 失败；
- `/etc/fstab` 中有一条设备标识、文件系统类型或选项与真实设备不一致；
- 真实 UUID、LABEL 和文件系统类型必须从本机证据获得；
- 目标挂载属于必需挂载，不能通过注释或随意增加 `nofail` 隐藏问题。

目标终态

1. 找到实际失败的 mount unit；
2. 将失败 unit 映射到唯一的 `fstab` 行；
3. 只修正有证据支持的字段；
4. `findmnt --verify` 不再报告目标错误；
5. `mount -a` 能处理目标条目；
6. `findmnt <TARGET>` 显示正确 source、fstype 和 options；
7. 相关 failed unit 消失；
8. 系统能进入原 default target；
9. 受控重启后仍能无人干预启动并挂载目标。

限制条件

- 不执行 `mkfs`、`wipefs -a` 或重新创建已有文件系统；
- 不注释必需挂载；
- 不用 `nofail` 掩盖故障；

- 不在定位前反复 reboot；
- 不把 `mount -a` 无输出当作唯一验收；
- 不编造 UUID、设备名或真实输出。

验收矩阵

层次	推荐证据	目标判断
症状	<code>systemctl --failed</code>	找到失败 mount unit
原因	<code>journalctl -b -u <unit></code>	得到具体错误类型
配置	<code>grep -nEv '^s*(# \$)' /etc/fstab</code>	定位对应行
设备	<code>lsblk -f、blkid</code>	获取真实身份和类型
静态	<code>findmnt --verify</code>	配置引用和语法合理
当前	<code>mount -a</code>	可执行挂载动作
终态	<code>findmnt <TARGET></code>	目标实际挂载正确
systemd	<code>systemctl --failed</code>	相关失败已消失
持久	重启后复验	启动链和挂载持久正确

[参考解答] 诊断错误 `fstab` 导致的 emergency

一、从症状取得当前证据

进入 emergency shell 后，先确认根是否可写：

```
findmnt /
findmnt -no OPTIONS /
```

如果需要修改而根为只读：

```
mount -o remount,rw /
findmnt -no OPTIONS /
```

随后查询失败对象：

```
systemctl --failed
journalctl -b -p err
```

若已经看到失败的 mount unit，例如 `<NAME.mount>`：

```
systemctl status <NAME.mount>
journalctl -b -u <NAME.mount>
```

二、把 unit 映射到具体配置行

根据挂载点推导 unit 名称，或反向查看 unit 的 `Where=` 状态信息。需要时可使用：

```
systemd-escape --path --suffix=mount <TARGET>
```

检查有效 `fstab` 行：

```
grep -nEv '^s*(#|$)' /etc/fstab
findmnt --fstab
```

只锁定目标行，不批量注释其他记录。

三、用设备证据验证假设

```
lsblk -f
blkid
findmnt --verify
```

比较：

- `fstab` 的 UUID/LABEL 是否存在；
- 文件系统类型是否一致；
- 挂载点是否正确；
- 选项是否有效；
- 设备是否应在本地启动阶段出现。

根据实际证据修改 `/etc/fstab`。答案中不填写虚构 UUID；实际考试应复制本机 `blkid` 或 `lsblk -f` 给出的值。

四、最小修复后重新加载与当前验证

```
systemctl daemon-reload
findmnt --verify
mount -a
findmnt <TARGET>
systemctl --failed
```

若 `mount -a` 报错，返回 Journal 和目标 unit，不要继续重启。若 `mount -a` 成功，也要确认 `findmnt <TARGET>` 的 source、fstype 和 options 与要求一致。

五、返回默认目标或进行受控重启

当前验证闭环后，可根据环境选择：

```
systemctl default
```

或在确认控制台与回退路径后正常重启。重启后：

```
systemctl get-default
systemctl --failed
findmnt <TARGET>
journalctl -b -p err
```

最终验收不是“系统离开 emergency”这一单点，而是系统能无人干预到达原 default target，目标文件系统按正确 source 持久挂载，且相关 mount unit 不再失败。

六、典型错误与修复

- 错误：直接注释失败行。修复：必需挂载必须修正真实原因，而不是隐藏。
- 错误：给必需挂载加 `nofail`。修复：只有业务明确允许设备缺失时才评估该选项。
- 错误：看到设备不挂载就执行 `mkfs`。修复：先通过 `lsblk -f`、`blkid` 和日志识别已有文件系统。
- 错误：`mount -a` 无输出后立即 reboot。修复：用 `findmnt <TARGET>` 和 `systemctl --failed` 分层验证，并始终保持最小变更，使故障原因和修复一一对应。

[本章收束] 先判阶段、再判根、最后决定是否重启

启动与恢复题目表面上涉及 GRUB、target、`rd.break`、`fstab` 和 Journal，底层却共享同一条决策链：

```
症状出现在哪里
→ 当前处于哪个启动阶段
→ 当前 / 与真实根分别是什么
→ 目标状态是一次性还是持久
→ 这一阶段最有区分度的证据是什么
→ 做哪一个最小修复
→ 当前环境如何验证
→ 是否具备控制台和回退条件
→ 受控重启后如何用同一对象复验
```

工作方法：把恢复过程写成可回放的证据链

真正稳定的操作不是背下一串命令，而是让每一步都能回答三个问题：它正在改变哪个对象，下一条证据怎样证明它改变正确，以及这条证据还不能证明什么。
`grubby --info=ALL` 可以证明启动条目文本，不能证明目标 kernel 已经成功启动；`systemctl get-default` 可以证明默认 target，不能证明当前系统已切换；

`mount -a` 没有报错，也不能代替对目标挂载点和数据可见性的验证。

主要判断表

场景	先查什么	能证明什么	仍不能证明什么
当前 kernel 与参数	<code>uname -r</code> 、 <code>cat /proc/cmdline</code>	本次实际运行和收到的参数	下次默认条目是否相同
持久 kernel 参数	<code>grubby --info=ALL</code>	启动条目中的未来配置	条目能否成功完成启动
default target	<code>systemctl get-default</code>	正常启动入口	当前 active target 和业务功能
<code>rd.break</code> 环境	<code>cat /proc/cmdline</code> 、 <code>findmnt /sysroot</code>	当前阶段与真实根位置	口令、标签和最终登录是否正确
<code>fstab</code> emergency	<code>systemctl --failed</code> 、 <code>journalctl -b</code>	失败 unit 和时间序列	设备身份与配置字段谁错误
SELinux 重标记	<code>touch /.autorelabel</code> 后正常启动	已请求下一次完整重标记	重标记是否完成、模式和服务是否正常
重启后验收	重查同一组对象并验证功能	持久状态真正进入运行状态	所有外部业务都必然正确

向下一章交接

本章结束于“系统能够安全启动、进入预期 target，并能用证据确认恢复终态”。下一章《Podman 镜像、容器与 Rootless 运行》将在这个可用主机上引入容器镜像、容器实例和普通用户运行边界。容器启动故障仍可能需要 systemd 与 Journal 证据，但不会重新展开本章的 GRUB、initramfs 和系统恢复流程。

验证范围：本章命令、参数和流程依据 RHEL 9 课程、官方文档与 man page 完成静态核对。固件按键时机、GRUB 菜单行为、`rd.break` 环境、SELinux 重标记耗时和重启结果仍需在目标 RHEL 9 主机上验证。

Podman 镜像、容器与 Rootless 运行

从镜像引用到当前实例：把用户上下文、端口、挂载、数据与运行证据放进同一条验证链。

对象模型操作语义验证诊断经典任务

大字号阅读版

本章阅读导航 先抓住一条主线：Podman 命令不是在修改一个笼统的“容器”，而是在不同用户上下文中依次处理镜像引用、本地镜像、容器配置对象、运行进程、宿主端口和外部数据。

01确认用户上下文rootless 状态属于目标宿主用户

02证明镜像身份完整引用、tag、digest 与本地 image ID

03建立容器配置名称、命令、环境、端口和挂载

04判断运行状态主进程、退出码、日志与 exec

05验收宿主链路port、ss、curl 与应用结果

06证明数据边界可写层、bind mount、volume 与删除结果

专题地图

知识专题	Registry、镜像引用、Tag 与 Digest
操作专题	拉取、列出、标记并检查镜像
知识专题	镜像、容器配置、可写层与运行进程
操作专题	创建、运行、启停与删除容器
知识专题	Rootless、用户命名空间与权限边界
操作专题	环境、端口、命令与用户身份
知识专题	可写层、Bind Mount 与 Named Volume
操作专题	SELinux <code>:Z/:z</code> 与挂载验证
操作专题	<code>ps</code> 、 <code>inspect</code> 、 <code>logs</code> 、 <code>exec</code> 与 <code>port</code>
诊断专题	按对象层定位退出、不可访问和权限拒绝
经典任务	部署并重建 Rootless Web 容器
经典任务	修复端口与 SELinux 挂载双故障

阅读时持续回答

1. 当前命令以哪个宿主用户执行？
2. 操作对象是镜像、容器配置还是运行进程？
3. `running` 只证明了哪一层？
4. 宿主端口和容器端口的方向是什么？
5. 数据位于可写层、宿主目录还是 named volume？
6. DAC、UID/GID 映射和 SELinux 哪一层拒绝了访问？
7. 删除容器时哪些数据会保留？
8. 当前证据是否越界声称了注销或重启后的持久运行？

第 31 章 · 正文

把一个应用“放进容器”以后，系统中并没有只出现一个新对象。Registry 中有镜像引用，目标用户的本地存储中有镜像；`podman create` 建立容器配置对象，`podman start` 才产生运行进程。端口发布把宿主 socket 与容器网络连接起来，挂载把宿主目录或 Podman volume 暴露给容器，而 rootless 运行又把这些状态限定在某个普通用户的命名空间、存储和权限范围内。最常见的误判都来自把这些对象合并成一句“容器已经好了”：镜像已拉取不等于实例已创建；容器 `running` 不等于应用监听正确；`podman port` 有映射不等于宿主请求成功；容器内 UID 0 不等于宿主 root；删除容器后，bind mount 中的数据通常仍在，而只写入可写层的数据会失去唯一入口。本章沿着“用户上下文 → 镜像 → 容器配置 → 运行进程 → 端口与挂载 → 功能与数据”推进。用户级 systemd、linger 与 Quadlet 只在章末建立交接，完整内容归下一章《容器持久化、用户 systemd 与 Quadlet》；firewalld 和 SELinux 通用规则也不在本章重复展开。

概念

镜像引用（image reference）是定位 Registry 内容的名称体系，通常由 Registry、namespace/repository 和 tag 组成，也可用 digest 固定到具体内容。Tag 便于发布和阅读，但可以被重新指向；digest 更适合作为内容身份。操作时应按题目保留完整引用，并用 image ID、RepoDigests 和 inspect 字段证明实际拉取内容。

概念

镜像（image）是当前 Podman 存储中的只读模板和层集合，它可以拥有多个 tag，也能被多个容器共同引用。镜像存在只证明创建实例所需的模板已在当前用户存储中，并不能证明容器已经创建、进程正在运行或应用能够访问。

概念

容器实例（container instance）是从镜像创建的独立对象，保存名称、命令、环境、端口、挂载和可写层；启动后才出现容器主进程。停止只改变运行状态，删除则移除配置对象和可写层。理解这一层次，是区分 `create`、`run`、`start`、`stop` 和 `rm` 的基础。

概念

Rootless 与 Rootful 描述 Podman 由普通宿主用户还是宿主 root 运行。两者拥有不同的存储、容器、卷和认证状态；root 查询不到普通用户的容器，并不能证明对象不存在。考试要求某用户运行 rootless 容器时，login、pull、run、inspect 和删除都必须保持在该用户上下文。

概念

用户命名空间（user namespace）把容器内 UID/GID 映射到宿主身份范围。Rootless 容器内 UID 0 可以在命名空间内拥有管理语义，但不会因此变成宿主 UID 0，也不能绕过宿主目录的 DAC 权限或 SELinux。遇到挂载拒绝时，要同时调查调用用户、映射后的进程身份和文件标签。

概念

端口发布（port publishing）是创建配置中的宿主端口到容器端口映射，例如 `-p 8088:8080`。它只建立转发入口，不保证容器进程正在监听，也不保证 HTTP 请求成功。可靠验收必须继续检查容器状态、`podman port`、宿主 socket 和协议响应。

概念

Bind mount 与 Named volume 都能把数据放到容器可写层之外，但所有权不同。Bind mount 明确引用管理员选择的宿主路径；named volume 由 Podman 管理名称和实际目录。删除容器通常不会删除这两类外部数据，而容器可写层会随容器对象消失，因此删除前必须先用 inspect 证明数据来源。

概念

SELinux `Z` 与 `z` 是 Podman 挂载语法中的重标记请求。`Z` 适合当前容器私有使用，`z` 适合多个容器共享同一内容；两者都会修改宿主对象的标签，所以不能机械附加到系统共享目录。它们也不能替代正确的宿主路径、所有权、模式和 UID/GID 映射。

操作语义速查 在进入详细专题前，先把本章最关键的接口分成六组。速查区只建立命令的作用对象和参数方向；正文再解释证据边界、诊断分支和安全限制。
`podman pull` / `podman images` / `podman image inspect` **SYNOPSIS**

```
podman pull [OPTIONS] IMAGE
podman images [OPTIONS]
podman image inspect [OPTIONS] IMAGE [IMAGE...]
```

在当前用户的 Podman 存储中获取、列出并检查镜像。三条命令分别回答“拉取什么”“当前有哪些引用”“实际内容和默认配置是什么”。**重要参数 / 形式**
`podman pull REGISTRY/NS/IMAGE:TAG`：优先使用题目给定的完整引用，避免短名称歧义。`podman images --format FORMAT`：选择 repository、tag、ID 等稳定字段建立清单。`podman image inspect IMAGE --format FORMAT`：精确提取 ID、架构、默认用户、命令或 digest；字段需按目标版本核对。
`podman run` / `create` / `start` / `stop` / `rm` **SYNOPSIS**

```
podman run [OPTIONS] IMAGE [COMMAND [ARG...]]
podman create [OPTIONS] IMAGE [COMMAND [ARG...]]
podman start [OPTIONS] CONTAINER
podman stop [OPTIONS] CONTAINER
podman rm [OPTIONS] CONTAINER
```

创建参数位于镜像名前，容器内命令位于镜像名后。`run` 是创建并启动；`start` 只能启动已有对象，不能补改创建时的端口、环境和挂载。**重要参数 / 形式**
`--name NAME`：为实例建立稳定管理名称；名称冲突时先检查旧对象是否可复用。`-d`：后台运行，不保证主进程不会立即退出。`--rm`：容器退出后自动删除对

象；会减少排错现场，不适合需要保留退出证据的任务。`--user UID[:GID]`：指定容器内进程身份；不等于切换宿主用户，也不能绕过 DAC 或 SELinux。
`podman ps` / `inspect` / `logs` / `exec` / `port` **SYNOPSIS**

```
podman ps [OPTIONS]
podman container inspect [OPTIONS] CONTAINER
podman logs [OPTIONS] CONTAINER
podman exec [OPTIONS] CONTAINER COMMAND [ARG...]
podman port CONTAINER [PRIVATE_PORT[/PROTO]]
```

这一组负责保存运行证据。`ps -a` 选择对象，`inspect` 读取创建配置和状态，`logs` 读取 stdout/stderr，`exec` 做定向容器内取证，`port` 显示发布映射。****重要参数 / 形式**** `podman ps -a`：同时显示 created、running、exited 等对象。`--format FORMAT`：提取明确字段；不要把整段 JSON 当作唯一可读证据。`podman logs --tail N NAME`：查看最近日志；日志为空不自动等于应用无错误。`podman exec NAME COMMAND`：只适用于 running 容器，并要求镜像中存在目标命令。
`podman volume` **SYNOPSIS**

```
podman volume create [OPTIONS] [NAME]
podman volume ls [OPTIONS]
podman volume inspect [OPTIONS] VOLUME [VOLUME...]
podman volume rm [OPTIONS] VOLUME [VOLUME...]
```

管理由 Podman 命名和定位的外部数据对象。Named volume 与容器对象生命周期分离，删除容器通常不会删除它。****重要参数 / 形式**** `podman volume create NAME`：创建稳定命名卷，便于重建容器时复用。`podman volume inspect NAME`：读取实际 Mountpoint 和元数据，证明容器配置引用的对象。`podman rm --volumes NAME`：可能同时处理关联的匿名卷；不要把它当作普通删除默认形式。
Rootless 身份与用户状态 **SYNOPSIS**

```
id
podman info
podman unshare COMMAND [ARG...]
loginctl show-user USER [OPTIONS]
```

确认当前宿主用户、rootless 存储上下文和 UID/GID 映射。`loginctl` 在本章只用于观察用户状态，不执行 linger 配置。****重要参数 / 形式**** `id`：证明当前宿主 UID、主组和附加组。`podman info`：确认 rootless、存储和运行时信息；输出字段随版本变化。`podman unshare cat /proc/self/uid_map`：在 rootless 用户命名空间中观察映射，不编造固定数值。`loginctl show-user USER -p State -p Linger`：只读取用户 manager 状态；持久运行配置归下一章。
端口与挂载验收 **SYNOPSIS**

```
ss -lnt
curl -fsS URL
ls -ldZ HOST_PATH
podman exec CONTAINER COMMAND [ARG...]
```

把 Podman 内部配置继续推进到宿主监听、协议功能和宿主数据。最终验收不能停在容器状态或命令返回码。****重要参数 / 形式**** `-p HOST_PORT:CONTAINER_PORT`：宿主端口在前，容器端口在后。`-e KEY=VALUE`：把环境变量固化到容器创建配置。`-v HOST:CONTAINER:Z`：私有 bind mount 重标记；使用前确认路径和共享关系。`-v VOLUME:CONTAINER`：使用 named volume；通过 volume inspect 和容器 inspect 联合验证。

[知识专题] 从 Registry 引用到本地镜像：名称、Tag 与 Digest

处理镜像的第一步不是直接输入 `podman pull nginx`，而是识别题目真正给出的镜像身份。短名称依赖本机 Registry 配置和别名；tag 可能被重新指向；同一个镜像 ID 也可以拥有多个名称。因此，稳定的路径是：先保存题目给出的完整引用，再通过本地镜像列表和 inspect 证明实际内容。

① [知识点] 完整镜像引用由多个可独立变化的部分组成

典型引用：

```
registry.lab.example.com/training/webapp:1.0
```

可拆成：

部分	示例	作用
Registry	registry.lab.example.com	定位提供镜像的服务
Namespace/Repository	training/webapp	组织镜像名称和访问范围
Tag	1.0	指向某个版本或发布通道的可读引用
Digest	sha256:...	对镜像内容的不可变标识

题目给出完整引用时，应优先原样使用。省略 Registry 的短名称需要由 registries.conf、短名称别名和交互选择来补全，批量或考试环境中容易产生歧义。

② [知识点] Tag 是名称，Digest 才是内容身份

Tag 可以移动。Registry 管理员可以让 `webapp:stable` 今天指向一个 digest，明天指向另一个 digest；本地系统已经拉取的旧镜像不会因为远端 tag 移动而自动变化。Digest 由镜像内容决定，更适合回答“实际拿到的是哪一份内容”。

这并不表示考试中必须把所有 tag 改成 digest。正确做法是：按题目指定的 tag 操作，同时用 `podman image inspect` 记录 RepoDigests、ID、架构和默认命令等证据。

③ [知识点] 一个镜像可以有多个名称，名称变化不等于复制内容

`podman tag SOURCE TARGET` 为本地镜像增加另一个引用。它通常不会复制所有镜像层。以下两个名称可以指向同一个 IMAGE ID：

```
localhost/webapp:latest
registry.lab.example.com/training/webapp:1.0
```

验证 tag 操作不能只看命令是否返回成功，应在 `podman images` 中核对新引用，并用 image ID 或 inspect 证明新旧名称指向预期内容。

④ [知识点] 本地镜像属于当前 Podman 存储上下文

Rootful Podman 默认使用系统级存储；rootless Podman 默认使用当前用户的数据目录。普通用户 `websvc` 拉取的镜像，不应假定 root 或另一个普通用户可以直接在自己的 `podman images` 中看到。

因此，题目要求“面向某用户运行 rootless 容器”时，镜像拉取、容器创建、状态查询和删除都应保持在同一目标用户上下文。

⑤ [知识点] Registry、运行默认值与存储配置来自不同文件

常见配置入口：

配置	系统级入口	用户级入口	本章用途
Registry 与短名称	<code>/etc/containers/registries.conf</code>	<code>~/.config/containers/registries.conf</code>	决定未限定名称如何补全、Registry 是否被标记为 insecure 或 blocked
Podman 默认值	<code>/usr/share/containers/containers.conf</code> 、 <code>/etc/containers/containers.conf</code>	<code>~/.config/containers/containers.conf</code>	影响部分命令默认为
容器存储	<code>/etc/containers/storage.conf</code>	<code>~/.config/containers/storage.conf</code>	决定 graphroot、驱动等存储状态

命令行参数通常用于覆盖一次操作的默认值，但不能把所有配置文件都理解为同一种“整体覆盖”。遇到版本和加载顺序问题，应查当前系统的 `podman(1)`、`containers.conf(5)`、`containers-registries.conf(5)` 和 `containers-storage.conf(5)`。

[Cheatsheet] 完整引用优先；tag 可移动，digest 标识内容：`podman tag` 增加名称而不是复制层；镜像属于当前用户的 Podman 存储；Registry、运行默认值和存储分别查对应配置。

[操作专题] 拉取、列出、标记并检查镜像

镜像操作应形成一个小闭环：明确引用，拉取到目标用户存储，列出本地镜像，inspect 关键字段，需要时增加 tag，最后再次核对。`podman search` 不能作为“镜像一定存在”的可靠证明；真正的存在性和可用性由 Registry 请求、pull 结果和本地 inspect 共同决定。

① [操作] 以目标用户拉取完整镜像引用

作用对象：当前用户的本地镜像存储。

基本形式：

```
podman pull registry.lab.example.com/training/webapp:1.0
```

私有 Registry 需要认证时，先在同一用户上下文中执行：

```
podman login registry.lab.example.com
```

认证信息属于该用户。不要为了“让普通用户看到镜像”而改用 root 登录并拉取，因为 root 与普通用户的 Podman 状态是分离的。

验证：

```
podman images
podman image inspect registry.lab.example.com/training/webapp:1.0
```

② [操作] 用明确字段核对镜像身份

`podman images` 适合快速查看 repository、tag、image ID、创建时间和大小；`podman image inspect` 提供完整结构化配置。需要稳定提取时，可在目标版本支持的前提下使用 `--format`：

```
podman image inspect registry.lab.example.com/training/webapp:1.0 \
  --format '{{.Id}} {{.Architecture}} {{.Os}}'
```

还应检查：

- RepoTags 与 RepoDigests；
- 镜像默认 `Cmd` / `Entrypoint`；
- 镜像默认 `User`；
- 暴露端口仅是镜像元数据，不等于宿主已发布端口。

③ [操作] 为导入或本地镜像增加目标 tag

```
podman tag localhost/webapp:latest \
  registry.lab.example.com/training/webapp:1.0
```

验证：

```
podman images --format '{{.Repository}}:{{.Tag}} {{.Id}}'
```

如果题目还要求 push，那属于 Registry 发布操作；本章只建立 tag 身份和本地验证，不把构建与发布扩展为主线。

④ [操作] 从归档加载镜像时区分 `load` 与容器文件系统导入

考试材料可能提供由 `podman save` 生成的镜像归档。对应入口是：

```
podman load -i image.tar
```

或：

```
podman load < image.tar
```

`podman load` 恢复镜像归档及其名称信息；`podman import` 则把文件系统 tar 作为新镜像导入，语义不同。拿到文件前先判断它来自 `image save` 还是 `container export`，不要机械试命令。

⑤ [验证] 镜像闭环

```
目标用户身份
→ 完整引用
→ pull/load 成功
→ podman images 可见
→ image inspect 的 ID、digest、架构和默认配置符合目标
→ 需要的 tag 指向正确 image ID
```

[Cheatsheet] 同一用户执行 `login/pull/images/inspect`；短名不可靠时用完整引用；`tag` 后核对 image ID；镜像归档用 `load`，容器文件系统 tar 才考虑 `import`。

[知识专题] 镜像不是容器：配置对象、可写层与运行进程

容器生命周期至少包含三个层次：镜像提供只读初始内容；容器对象保存名称、命令、环境、端口和挂载配置，并拥有可写层；容器启动后才出现运行进程。`podman ps` 只显示其中一部分状态，因此排错时必须知道自己正在寻找哪一层。

① [知识点] Image 是模板，Container 是实例

同一镜像可以创建多个容器。它们共享镜像的只读层，但各自拥有：

- 容器 ID 和名称；
- 创建时的命令与参数；
- 环境变量；
- 端口发布；
- 挂载；

- 独立可写层；
- 启停与退出状态。

删除某个容器通常不会删除镜像，也不会影响从同一镜像创建的其他容器。

② [知识点] create、run 与 start 改变不同阶段

```
podman create
→ 建立容器对象，状态通常为 created

podman start
→ 启动已存在容器中的配置命令

podman run
→ create + start 的组合，并根据参数决定是否附着、后台运行或自动删除
```

podman start 不接受所有 podman run 参数，因为端口、挂载和环境变量已经在创建时固化。若这些配置错误，通常需要保留证据后删除并重新创建，而不是反复 stop/start。

③ [知识点] 容器的主进程决定生命周期

容器不是轻量虚拟机。容器的主进程退出，容器通常就进入 exited 状态。镜像默认命令如果是一次性命令，podman run -d 也不会让它永久运行。

因此，容器“立即消失”时第一条证据不是再次运行，而是：

```
podman ps -a
podman logs <CONTAINER>
podman container inspect <CONTAINER>
```

重点查看状态、退出码、实际命令和错误消息。

④ [知识点] Created、Running、Exited 与 Removed 是不同状态

状态	含义	可执行的典型下一步
created	对象存在，主进程尚未启动	podman start
running	主进程仍在运行	logs、exec、port、协议验证
exited	主进程已退出，对象仍在	logs、inspect、必要时 start
removed	容器对象已删除	只能重新 create/run

podman ps 默认只列出 running 容器；podman ps -a 才能看到 created、exited 等对象。

⑤ [知识点] 容器删除与数据删除必须分开判断

删除容器通常会移除容器配置和可写层，但以下内容不应混在一起：

数据位置	删除容器后的典型结果
容器可写层	随容器对象删除
bind mount 的宿主文件	仍由宿主管理，通常保留
显式命名的 named volume	通常保留，需单独 podman volume rm
匿名 volume	使用 --rm 或 podman rm --volumes 时可能被一并移除
本地镜像	不因普通 podman rm 被删除

删除前应先 inspect 挂载来源，避免把“容器可写层中的唯一数据”误认为持久数据。

[Cheatsheet] create 只建对象，start 启动旧对象，run 创建并启动；主进程退出即容器退出；ps -a 查非运行对象；删容器不等于删镜像、bind mount 或命名卷。

[操作专题] 创建、运行、停止、再次启动与删除容器

生命周期操作的关键不是把命令背成一串，而是每次操作前后都保存对象证据。名称冲突、错误端口、错误挂载和错误命令都可能迫使你重建容器；若先 rm -f，最重要的退出码、日志和配置就可能丢失。

① [操作] 只创建、不启动

```
podman create --name rh31-web \
registry.lab.example.com/training/webapp:1.0
```

验证：

```
podman ps -a --filter name=rh31-web
podman container inspect rh31-web
```

适用场景：需要先核对创建配置，或后续由其他生命周期入口启动。一般考试任务更常直接使用 `podman run`。

② [操作] 创建并后台运行

```
podman run -d --name rh31-web \
registry.lab.example.com/training/webapp:1.0
```

`-d` 只表示 Podman 不持续附着在前台，不保证应用不会退出。命令返回容器 ID 后，立即检查：

```
podman ps -a --filter name=rh31-web
podman logs rh31-web
```

③ [操作] 停止与再次启动

```
podman stop rh31-web
podman start rh31-web
```

`stop` 默认先请求主进程有序终止，并在超时后采取更强制措施。不要把 `podman kill` 当作常规停止入口。

验证：

```
podman ps -a --filter name=rh31-web
podman container inspect rh31-web \
--format '{{.State.Status}} {{.State.ExitCode}}'
```

④ [操作] 删除前建立证据

推荐顺序：

```
podman ps -a --filter name=rh31-web
podman container inspect rh31-web > rh31-web.inspect.json
podman logs rh31-web > rh31-web.log 2>&1
podman stop rh31-web
podman rm rh31-web
```

在考试环境中不一定需要把证据写入文件，但思路应保持一致：先确认名称、状态、挂载和端口，再删除错误对象。

⑤ [诊断边界] 同名容器阻止 `run`

症状：

```
Error: the container name "rh31-web" is already in use
```

证据链：

```
podman ps -a --filter name=rh31-web
podman container inspect rh31-web
```

若旧对象配置正确，只是 exited，可直接 `start`；若端口、挂载或环境错误，保存证据后删除并按正确参数重建。不要看到名称冲突就直接 `podman rm -f`。

[Cheatsheet] `run -d` 后立刻 `ps -a`；`stop/start` 不改变创建参数；重建前保存 `inspect` 与 `logs`；名称冲突先判断旧对象是否可复用。

[知识专题] Rootless：用户命名空间、每用户状态与权限边界

Rootless 并不是“容器里不能有 root 用户”，而是容器进程由普通宿主用户发起，并通过用户命名空间映射 UID/GID。容器内 UID 0 可以在自己的命名空间中拥有一部分管理能力，但在宿主看来仍受普通用户权限限制。这是判断目录权限、端口和状态可见性的基础。

① [知识点] 容器内 UID 0 不等于宿主 UID 0

默认 rootless 映射中，调用 Podman 的宿主用户被映射为 rootless 用户命名空间的 UID 0。容器进程即使显示为 root，也不会自动获得读取宿主 `/root`、修改系统文件或绕过 SELinux 的能力。

确认宿主身份：

```
id
whoami
podman info
```

需要查看映射时，可使用：

```
podman unshare cat /proc/self/uid_map
podman unshare cat /proc/self/gid_map
```

这些输出依赖环境，不应编造固定数值。

② [知识点] `/etc/subuid` 与 `/etc/subgid` 提供附加映射范围

RHEL 创建普通用户时通常会为 rootless 容器配置 subordinate UID/GID 范围。静态检查入口：

```
grep '^websvc:' /etc/subuid /etc/subgid
```

如果映射缺失或被修改，实际处理可能需要管理员修复并执行与 Podman 版本匹配的迁移操作。本章只训练识别和证据，不展开所有高级 `--uidmap`、`--gidmap` 组合。

③ [知识点] 不同宿主用户拥有不同 Podman 状态

普通用户创建的容器不会自动出现在 root 或其他用户的 `podman ps -a` 中。以下两条命令查询的是不同状态集合：

```
sudo -iu websvc podman ps -a
sudo podman ps -a
```

因此，root 看到“没有容器”不能证明 `websvc` 没有 rootless 容器；root 拉取了镜像也不能证明 `websvc` 的本地镜像存储中已有该镜像。

④ [知识点] Rootless 对宿主目录权限提出双重要求

容器访问 bind mount 时，至少要同时满足：

1. 调用 Podman 的宿主用户能够遍历和访问源路径；
2. 映射后的容器进程 UID/GID 对宿主对象具有合适 DAC 权限；
3. SELinux 策略允许容器域访问该标签。

即使宿主用户可以 `ls` 目录，也不表示镜像中的应用 UID 一定能写。反过来，把目录改为 `777` 既不能解决所有 UID 映射问题，也会破坏最小权限。

⑤ [知识点] Rootless 端口限制是宿主策略，不应死记单一数字

非特权用户能否绑定某个低端口，取决于宿主机内核参数和 Podman 网络实现等状态。调查入口：

```
sysctl net.ipv4.ip_unprivileged_port_start
```

在考试任务未要求低端口时，优先选择题目给出的高位宿主端口。端口权限问题不能靠切换为 rootful 容器掩盖，因为那会改变题目的安全和状态边界。

⑥ [边界] `loginctl` 在本章只用于观察，不用于配置持久运行

可观察目标用户的会话和 linger 状态：

```
loginctl show-user websvc -p State -p Linger
```

但 `loginctl enable-linger`、`systemctl --user` 与 Quadlet 属于下一章。当前容器 running 只证明当前实例，不证明注销或重启后的持续运行。

[Cheatsheet] rootless 的“root”仍是宿主普通用户；所有 Podman 命令保持目标用户一致；目录访问按宿主 DAC、UID/GID 映射和 SELinux 三层判断；低端口阈值查系统，不死记；持久运行留给下一章。

[操作专题] 组合名称、环境变量、端口发布与启动命令

podman run 的参数分成两部分：镜像名前是 Podman 的创建参数，镜像名后是交给容器执行的命令和参数。端口、环境和挂载在创建时固化；命令位置错误会导致 Podman 把参数解释给错误对象。

① [操作] 识别基本命令骨架

```
podman run [PODMAN OPTIONS] IMAGE [COMMAND [ARG...]]
```

示例：

```
podman run -d --name rh31-web \
registry.lab.example.com/training/webapp:1.0
```

若在镜像后追加命令，通常会覆盖或参与镜像的默认命令组合：

```
podman run --rm registry.lab.example.com/training/webapp:1.0 \
/bin/sh -c 'id && env | sort'
```

② [操作] 设置容器名称与环境变量

```
podman run -d --name rh31-web \
-e APP_MODE=exam \
-e LOG_LEVEL=info \
registry.lab.example.com/training/webapp:1.0
```

验证环境配置：

```
podman container inspect rh31-web
podman exec rh31-web /usr/bin/env
```

exec 只适用于 running 容器，而且容器中必须存在指定命令。inspect 是创建配置证据，exec env 是容器内部运行时证据，两者回答的问题不同。

--user UID[:GID] 改变容器内主进程使用的身份，不是把当前操作切换为另一个宿主用户。它必须结合镜像内账号、rootless UID/GID 映射和挂载目录权限判断。题目要求容器进程以特定身份运行时，应同时检查创建配置与容器内 id；不要把 --user 当作绕过宿主 DAC 或 SELinux 的办法。

③ [操作] 正确理解端口发布方向

```
-p 8088:8080
```

含义是：宿主端口 8088 转发到容器端口 8080。宿主端口在前，容器端口在后。

完整示例：

```
podman run -d --name rh31-web \
-p 8088:8080 \
registry.lab.example.com/training/webapp:1.0
```

不要根据镜像的 EXPOSE 元数据猜应用一定监听某端口。需要结合镜像说明、日志、容器内 socket 或任务参数确认容器端口。

④ [验证] 端口必须按三层证明

第一层：容器配置声明了什么？

```
podman port rh31-web
podman container inspect rh31-web
```

第二层：宿主是否出现相应监听？

```
ss -lnt
```

第三层：协议是否真的成功？

```
curl -v http://127.0.0.1:8088/
```

`podman port` 不能证明应用健康；`ss` 不能证明 HTTP 内容正确；`curl` 失败也不能直接证明端口发布错误，因为应用可能未监听、启动未完成或请求路径错误。

⑤ [诊断] Running 但不可访问

推荐链路：

```
podman ps
→ inspect 实际命令、环境和端口
→ logs 查看应用是否启动
→ podman port 查看映射
→ 必要时 exec/容器内 socket 检查
→ ss 查看宿主监听
→ curl 做协议验证
```

若发现把 `-p 8088:80` 写成了 `-p 80:8088`，端口配置不能通过 `stop/start` 修改；保存证据后重建容器。

[Cheatsheet] 镜像名前是 Podman 参数，镜像后是容器命令：`-p HOST:CONTAINER`；环境变量用 `inspect` 与容器内 `env` 双证据；端口按配置、监听、协议三层验收。

[知识专题] 容器数据到底放在哪里：可写层、Bind Mount 与 Named Volume

“容器里能看到文件”并不能回答数据是否持久。文件可能位于镜像只读层、容器可写层、宿主 bind mount 或 Podman-managed volume。删除和重建前必须先识别来源，否则最容易在容器可写层里丢失唯一数据。

① [知识点] 容器可写层与容器对象同生命周期

容器对镜像文件系统的普通修改通常进入该容器的可写层。停止并再次启动同一个容器时，这些数据仍可见；删除容器后，可写层随对象消失。

适合可写层的数据：临时缓存、无需跨重建保留的运行时文件。业务数据、配置输入和题目要求持久保存的文件不应只放在可写层。

② [知识点] Bind mount 把明确宿主路径暴露给容器

语法：

```
-v /HOST/PATH:/CONTAINER/PATH[:OPTIONS]
```

源路径由宿主管理，便于直接检查和备份。删除容器不会自动删除宿主源目录，但容器进程能否访问取决于宿主权限、UID/GID 映射和 SELinux 标签。

③ [知识点] Named volume 由 Podman 管理身份和路径

```
podman volume create webdata
podman volume inspect webdata
```

挂载：

```
-v webdata:/var/lib/webapp
```

显式命名 volume 独立于某个容器，便于多个重建实例复用。其实际挂载点由 Podman 管理，不应把内部路径当作稳定公共接口；检查和删除优先使用 `podman volume` 子命令。

④ [知识点] 匿名 Volume 与 Named Volume 的删除边界不同

省略 source：

```
-v /var/lib/webapp
```

Podman 会创建匿名 volume。匿名 volume 可以随 `podman run --rm` 或 `podman rm --volumes` 删除；显式命名 volume 不会被这些入口自动删除，需要单独管理。

⑤ [验证] 用 inspect 识别挂载来源，而不是只在容器内 ls

```
podman container inspect rh31-web
podman volume ls
podman volume inspect webdata
```

需要确认：

- Type 是 bind 还是 volume；
- Source 是哪个宿主路径或 volume；
- Destination 是容器内哪个路径；
- Options 是否包含只读、`z` 或 `Z`；
- 重建后是否仍指向同一数据来源。

[Cheatsheet] 可写层随容器删除；bind mount 明确指向宿主路径；named volume 独立管理；匿名卷可能随 `--rm/--volumes` 删除；持久性要通过 source 和重建验证。

[操作专题] 配置 Bind Mount、Named Volume 与 SELinux `:Z/:z`

挂载失败不是一个单一“权限问题”。稳定顺序是：先确认宿主路径和目标用户，再确认容器进程身份和 DAC，最后确认 SELinux。`chmod 777`、关闭 SELinux 或无条件 `--privileged` 都会掩盖真正的边界，并引入更大的风险。

① [操作] 准备 rootless 用户可管理的宿主目录

训练示例：

```
install -d -m 0750 -o websvc -g websvc /home/websvc/site
printf '%s\n' 'RHCSA-31 rootless web' > /home/websvc/site/index.html
chown websvc:websvc /home/websvc/site/index.html
```

随后切换到目标用户上下文：

```
sudo -iu websvc
id
```

实际考试中可能没有 `sudo` 或 `install`，可使用等价命令，但原则不变：源目录必须真实存在，目标用户必须可遍历和访问。

② [操作] 为单个容器使用私有重标记 `:Z`

```
podman run -d --name rh31-web \
-v /home/websvc/site:/opt/app/site:Z \
registry.lab.example.com/training/webapp:1.0
```

`:Z` 请求 Podman 为当前容器应用私有、非共享的 SELinux 标签。只有当前容器或具有匹配私有标签的对象应使用该内容。

③ [操作] 多个容器共享同一内容时使用 `:z`

```
-v /srv/shared-content:/opt/app/shared:z
```

`:z` 使用共享容器内容标签，适用于多个容器需要同时读写相同源目录的场景。不要仅因为记忆方便把所有挂载都写成 `:z`；共享边界应由任务决定。

④ [安全边界] 重标记会修改宿主文件标签

对大量 inode 递归重标记会延长容器启动时间；对系统目录、整个 home 或同时供其他受限服务使用的路径重标记，可能破坏其他服务。应优先为容器准备专用子目录，而不是把 `/home`、`/etc`、`/var` 等大范围路径直接加 `:Z`。

本章不把 `--security-opt label=disable` 作为常规答案。只有官方文档明确适用且任务安全边界允许时才考虑这种高级例外。

⑤ [操作] 创建并使用 Named Volume

```
podman volume create webdata
podman run -d --name rh31-web \
-v webdata:/var/lib/webapp \
registry.lab.example.com/training/webapp:1.0
```

验证：

```
podman volume ls
podman volume inspect webdata
podman container inspect rh31-web
```


⑥ [验证] 挂载要从配置、标签、容器和宿主四层证明

```
podman container inspect rh31-web
ls -ldZ /home/websvc/site
podman exec rh31-web ls -ldZ /opt/app/site
podman exec rh31-web sh -c 'test -r /opt/app/site/index.html'
cat /home/websvc/site/index.html
```

容器内创建测试文件后，还应在宿主源目录中确认同一文件，而不是只在容器内看见它。

[Cheatsheet] 源目录先存在且目标用户可访问；单容器私有用 `:Z`，多容器共享用 `:z`；重标记会修改宿主标签；不对系统大目录随意重标记；最终从 `inspect`、`ls -Z`、容器内访问和宿主文件四层验证。

[操作专题] 用 `ps`、`inspect`、`logs`、`exec` 与 `port` 建立运行证据

Podman 的多个查询命令并非同义词。`ps` 适合选择对象；`inspect` 读取结构化配置和状态；`logs` 读取日志驱动保存的 `stdout/stderr`；`exec` 在运行容器中执行定向命令；`port` 只列端口映射。排错时应让每条命令回答一个明确问题。

① [操作] 用 `podman ps` 和 `podman ps -a` 选择对象

```
podman ps
podman ps -a
podman ps -a --filter name=rh31-web
podman ps -a --filter status=exited
```

默认 `ps` 只显示 `running` 容器。容器立即退出、`created` 未启动或停止后，都必须用 `-a` 才能看到。

② [操作] 用显式对象类型的 `inspect` 避免歧义

```
podman image inspect IMAGE
podman container inspect CONTAINER
podman volume inspect VOLUME
```

虽然 `podman inspect` 能检查多种对象，讲义和考试答案中优先写清对象类型，便于确定自己正在读取镜像默认配置，还是容器实际创建配置。

容器 `inspect` 常见关注点：

- `.State.Status`；
- `.State.ExitCode`；
- `.Config.Cmd` 与镜像后的实际命令；
- 环境变量；
- 端口绑定；
- `Mounts`；
- 容器进程和创建时间。

③ [操作] 用 `logs` 读取 `stdout/stderr` 证据

```
podman logs rh31-web
podman logs --tail 50 rh31-web
podman logs --since 10m rh31-web
```

`podman logs` 只能显示容器日志驱动已经收集到的记录。应用只写内部文件、尚未刷新缓冲区或刚启动尚未输出时，空日志不能证明“没有错误”。

④ [操作] 用 `exec` 做最小、定向的容器内取证

```
podman exec rh31-web /usr/bin/env
podman exec rh31-web /bin/sh -c 'id; ls -ld /opt/app/site'
```

边界：

- 只能对 `running` 容器执行；
- 容器中必须存在相应命令；
- `exec` 成功只证明该命令在容器内成功，不证明宿主端口或外部业务成功；
- 不要把进入交互 `Shell` 当作唯一排错方法。

⑤ [操作] 用 `podman port`、`ss`、`curl` 完成外部链路

```
podman port rh31-web
ss -lnt
curl -v http://127.0.0.1:8088/
```

远程访问还会受到路由、`firewalld` 和上游网络设备影响，完整防火墙规则归第 21 章。本章只要求证明本机发布、监听和协议响应。

⑥ [帮助入口] 命令不确定时查本机版本

```
podman run --help
podman container inspect --help
man podman-run
man podman-inspect
man podman-volume
```

RHEL 9 不同小版本的 Podman 行为和字段可能不同，考试环境应以本机 man page 为最终命令边界。

[Cheatsheet] `ps` 选对象，`inspect` 看配置与状态，`logs` 看 stdout/stderr，`exec` 做容器内取证，`port/ss/curl` 看宿主链路；每条证据都有限定范围。

[诊断专题] 容器失败时按对象层推进，而不是先删除重建

一个稳定的容器诊断应从症状开始，把“镜像、容器对象、主进程、端口、挂载和宿主功能”逐层分开。只有下一条证据能区分假设时，才执行它；修复应尽量只改变已经证明错误的参数。

① [诊断] 容器立即退出

症状：`podman run -d` 返回 ID，但 `podman ps` 看不到。

当前证据：

```
podman ps -a --filter name=rh31-web
podman logs rh31-web
podman container inspect rh31-web
```

主要假设：

- 镜像默认命令本来就是一次性任务；
- 覆盖命令不存在或不可执行；
- 环境变量或配置缺失；
- 应用启动后主动报错退出；
- `bind mount` 覆盖了应用所需文件。

最小修复：根据退出码、实际命令和日志修正对应参数，不要用无限循环命令伪装“容器存活”。

再验证：`ps -a`、exit code、logs 和目标功能。

② [诊断] 容器 Running，但 `curl` 失败

当前证据：

```
podman ps --filter name=rh31-web
podman container inspect rh31-web
podman logs rh31-web
podman port rh31-web
ss -lnt
curl -v http://127.0.0.1:8088/
```

区分问题：

- 应用是否真正监听容器端口；
- 发布是否写成正确的 `HOST:CONTAINER`；
- 宿主是否监听预期地址；
- 应用是否尚未就绪；
- URL、路径或协议是否错误。

最小修复：端口配置错误时保留证据后重建；应用配置错误时修正环境或挂载；不要直接把问题归咎于 `firewalld`。

③ [诊断] Bind mount 报 `permission denied`

证据链：

```
目标用户能否遍历源目录
→ 源对象 owner/group/mode
→ 容器实际运行 UID/GID 与映射
→ inspect 的 Mounts 和 options
→ ls -Z / AVC 证据
→ 只修复错误层
```

推荐命令：

```
namei -l /home/websvc/site
ls -ldZ /home/websvc/site
podman container inspect rh31-web
podman unshare id
```

需要 SELinux AVC 时，引用第 28、29 章的方法，不在本章完整展开。默认修复不是 `chmod 777`、`setenforce 0` 或 `--privileged`。

④ [诊断] Root 看不到普通用户的容器

症状：管理员执行 `podman ps -a` 为空，但用户声称容器存在。

最有区分度的证据：

```
sudo -iu websvc podman ps -a
sudo podman ps -a
```

若前者存在而后者为空，这是正常的每用户状态隔离，不是容器丢失。所有后续 `inspect`、`logs`、`stop` 和 `rm` 必须回到 `websvc` 上下文。

⑤ [诊断] 删除容器后数据消失

先回答数据原来在哪里：

```
podman container inspect OLD_CONTAINER
```

若没有 `bind mount` 或 `named volume`，文件很可能只在可写层。已经删除且没有备份时，本章不能承诺恢复。后续应把业务数据迁移到明确的宿主目录或 `named volume`，并通过删除重建测试证明持久性。

⑥ [安全边界] 何时拒绝“快捷修复”

以下做法不能作为默认答案：

- `podman rm -f`：会破坏现场和有序停止；
- `--privileged`：扩大容器对宿主的访问能力；
- `chmod 777`：破坏最小权限且不能解决 SELinux；
- 关闭 SELinux：绕过策略而非修复标签和访问模型；
- 改用 `rootful` 容器：改变题目要求和状态所有者；
- 只看 `running`：不能证明端口、协议和数据。

[Cheatsheet] 立即退出查 `ps -a/logs/inspect`；不可访问查应用、发布、监听、协议；挂载拒绝按 DAC、映射、SELinux 分层；`root` 与普通用户状态分开；删除前先确认数据来源。

[经典任务] 部署带端口、环境变量和持久目录的 Rootless Web 容器

以下参数用于训练，不代表真实考试基础设施。

环境

- 主机已安装 Podman；
- 普通用户 `websvc` 已存在，并具备 rootless Podman 所需 subordinate UID/GID；
- 镜像为 `registry.lab.example.com/training/webapp:1.0`；
- 容器内应用监听 TCP `8080`；
- 宿主目录 `/home/websvc/site` 已包含必须保留的 `index.html`；
- SELinux 保持 Enforcing。

当前状态

- 可能存在名为 `rh31-web` 的 stopped 容器；
- 不确定目标用户是否已经拉取镜像；
- 不允许删除宿主目录中的现有文件。

目标终态

1. 全部 Podman 操作以 `websvc` 身份执行；
2. 当前用户本地存储中存在指定完整镜像；
3. 容器名为 `rh31-web`；
4. 设置环境变量 `APP_MODE=exam`；
5. 发布宿主 `8088` 到容器 `8080`；
6. bind mount `/home/websvc/site` 到 `/opt/app/site`，并为单容器选择正确 SELinux 重标记；
7. 容器当前为 running；
8. 宿主 `curl http://127.0.0.1:8088/` 能读取现有页面；
9. 容器删除并按同样配置重建后，宿主目录数据仍存在。

限制条件

- 不使用 `--privileged`；
- 不关闭 SELinux；
- 不执行 `chmod 777`；
- 不使用 `root` 创建同名容器；
- 不无调查执行 `podman rm -f`；
- 不配置 `user systemd`、`linger` 或 `Quadlet`。

验收证据

```
id / podman info
→ image inspect
→ ps -a / container inspect
→ logs
→ podman port
→ ss
→ curl
→ 宿主文件
→ 删除重建后的数据
```

请先独立完成。参考解答从下一页开始。

[参考解答] 任务一：从用户身份到数据重建的分层实施

本解答只提供推荐命令和判断逻辑，未在本会话控制的 RHEL 9 虚拟机中执行。实际考试应按题目给出的主机名、镜像、端口和路径替换训练参数。

① [调查] 进入目标用户上下文并建立基线

管理员先确认账号和映射：

```
id websvc
grep '^websvc:' /etc/subuid /etc/subgid
```

进入目标用户：

```
sudo -iu websvc
id
podman info
podman ps -a --filter name=rh31-web
podman images
```

若同名容器存在，先检查：

```
podman container inspect rh31-web
podman logs rh31-web
```

- 若对象配置完全正确，只是 stopped，可考虑 `podman start rh31-web`；
- 若端口、环境或挂载错误，保存证据后 stop/rm 重建；
- 不直接 `rm -f`。

② [调查] 核对宿主目录

```
ls -ldZ /home/websvc/site
ls -lZ /home/websvc/site
cat /home/websvc/site/index.html
```

确认 websvc 可遍历和读取。若任务要求容器写入，还要确保映射后的应用 UID/GID 具备所需写权限；不能仅用 777 解决。

③ [操作] 拉取并检查镜像

```
podman pull registry.lab.example.com/training/webapp:1.0
podman images
podman image inspect registry.lab.example.com/training/webapp:1.0
```

检查镜像实际存在于 websvc 的本地存储，并确认默认命令、架构和镜像身份。

④ [操作] 删除已证明配置错误的旧对象

仅当基线证明必须重建：

```
podman stop rh31-web 2>/dev/null || true
podman rm rh31-web
```

若容器已经 stopped，stop 可能不需要。命令中的 `|| true` 只是说明流程容忍“对象本来未运行”，考试中也先判断状态后执行更精确命令。

⑤ [操作] 创建正确容器

```
podman run -d --name rh31-web \
-e APP_MODE=exam \
-p 8088:8080 \
-v /home/websvc/site:/opt/app/site:Z \
registry.lab.example.com/training/webapp:1.0
```

参数解释：

- `-d`：后台运行，不保证应用一定持续运行；
- `--name rh31-web`：建立稳定管理名称；
- `-e APP_MODE=exam`：写入创建配置；
- `-p 8088:8080`：宿主 8088 到容器 8080；
- `:Z`：单容器私有重标记；
- 最后是完整镜像引用。

⑥ [验证] 容器对象和主进程

```
podman ps -a --filter name=rh31-web
podman container inspect rh31-web
podman logs --tail 50 rh31-web
```

必须确认：

- 状态是 running；
- 实际环境包含 `APP_MODE=exam`；
- Mounts 指向正确 source/destination；
- 端口绑定方向正确；
- 日志中没有立即退出或配置错误。

⑦ [验证] 宿主监听和 HTTP

```
podman port rh31-web
ss -lnt
curl -v http://127.0.0.1:8088/
```

`curl` 返回目标页面才证明本机 HTTP 功能；只有 `podman ps` 为 running 不够。

⑧ [验证] 数据确实来自宿主目录

```
cat /home/websvc/site/index.html
podman exec rh31-web cat /opt/app/site/index.html
```

两处内容应一致。若容器写入测试文件：

```
podman exec rh31-web sh -c \
  'printf "%s\n" container-write > /opt/app/site/container-write.txt'
cat /home/websvc/site/container-write.txt
```

只有宿主源目录能看到同一文件，才证明写入进入 bind mount，而不是其他路径。

⑨ [验证] 删除并重建后数据仍在

先保存创建参数或 inspect 证据，再：

```
podman stop rh31-web
podman rm rh31-web
cat /home/websvc/site/index.html
```

数据仍在宿主目录。按相同 `podman run` 命令重建后：

```
podman run -d --name rh31-web \
  -e APP_MODE=exam \
  -p 8088:8080 \
  -v /home/websvc/site:/opt/app/site:Z \
  registry.lab.example.com/training/webapp:1.0

curl http://127.0.0.1:8088/
```

这一步证明数据持久性来自 bind mount，而不是旧容器可写层。

⑩ [边界] 本任务未证明注销或重启后自动运行

可观察：

```
loginctl show-user websvc -p State -p Linger
```

但本章不执行 `enable-linger`，也不创建 user systemd 或 Quadlet。当前验收仅到“当前 rootless 容器、端口和数据正确”。持久运行归下一章。

[Cheatsheet] 目标用户基线 → 目录 → 镜像 → 必要时证据重建 → `run(name/env/port/volume)` → `ps/logs/inspect` → `port/ss/curl` → 宿主数据 → 删除重建后再验。

[经典任务] 修复 Running 但不可访问且挂载被 SELinux 拒绝的容器

环境与症状

普通用户 `websvc` 已运行名为 `rh31-broken` 的 rootless 容器。镜像内应用监听 `8080`，但现有容器错误地发布为：

```
8088:80
```

同时把 `/home/websvc/site` 挂载到 `/opt/app/site`，创建时没有使用 `:Z` 或 `:z`。当前证据：

- `podman ps` 显示 running；
- `curl http://127.0.0.1:8088/` 失败；
- 应用日志包含访问站点目录失败的信息；
- SELinux 保持 Enforcing。

目标

1. 保留并读取现有容器证据；
2. 证明端口发布方向和容器应用端口不匹配；
3. 按 DAC、UID/GID 映射与 SELinux 标签区分挂载失败层；
4. 不关闭 SELinux、不使用 `--privileged`、不执行 `chmod 777`；

5. 以最小参数修正后重建；
6. 重新证明 running、正确端口、宿主监听、HTTP 和宿主数据。

[参考解答] 任务二：把两个故障拆成端口层与挂载层

① [调查] 保存当前容器配置、状态与日志

```
podman ps -a --filter name=rh31-broken
podman container inspect rh31-broken > rh31-broken.inspect.json
podman logs rh31-broken > rh31-broken.log 2>&1
podman port rh31-broken
```

从 inspect 和 port 证明宿主 8088 实际映射到容器 80，而应用要求容器 8080。此时已经证明必须重建，单纯 restart 无法修改发布配置。

② [调查] 检查挂载和宿主 DAC

```
namei -l /home/websvc/site
ls -ldZ /home/websvc/site
podman container inspect rh31-broken
```

确认 source/destination 正确，websvc 能遍历源路径，且挂载 options 中缺少正确 SELinux 重标记。若 DAC 本身错误，先按最小权限修正 owner/group/mode；若 DAC 已满足，再进入 SELinux 证据。

③ [调查] 读取 SELinux 证据

本章只给出入口，完整 AVC 解释归 SELinux 章节：

```
ls -ldZ /home/websvc/site
sudo ausearch -m AVC -ts recent
```

如果证据指向容器域被宿主目录标签拒绝，使用专用目录和 :Z 是比关闭 SELinux 更小的修复。

④ [操作] 有序停止并删除错误对象

```
podman stop rh31-broken
podman rm rh31-broken
```

旧 inspect 和 logs 已保存，可用于对比。

⑤ [操作] 按正确端口和私有挂载重建

```
podman run -d --name rh31-broken \
  -p 8088:8080 \
  -v /home/websvc/site:/opt/app/site:Z \
  registry.lab.example.com/training/webapp:1.0
```

若多个容器确实需要共享该目录，应根据共享边界改用 :z；不能仅因 :Z 更宽松就默认使用。

⑥ [分层验证]

```
podman ps -a --filter name=rh31-broken
podman logs --tail 50 rh31-broken
podman container inspect rh31-broken
podman port rh31-broken
ss -lnt
curl -v http://127.0.0.1:8088/
ls -ldZ /home/websvc/site
podman exec rh31-broken test -r /opt/app/site/index.html
cat /home/websvc/site/index.html
```

验收结论必须逐层表达：

- ps -a：当前主进程是否运行；
- inspect：创建配置是否已修正；
- logs：应用是否仍报错；
- port/ss：发布和宿主监听；

- curl: HTTP 功能；
- `ls -Z` 与容器读取：挂载和 SELinux；
- 宿主文件：数据来源。

[Cheatsheet] 先保存旧 inspect/logs → 证明 `8088:80` 错误 → 检查 DAC/映射/标签 → stop/rm → `8088:8080` + `:Z` 重建 → 对象、日志、端口、监听、HTTP、数据逐层再验。

[本章收束] 把“容器能跑”还原成一条可证明的状态链

本章最重要的工作方法，是始终先问“当前证据属于哪一个对象层”。镜像、容器对象、容器主进程、宿主监听、协议功能和外部数据不是同一个状态；rootless 又要求所有操作和查询保持在同一宿主用户上下文。只有把这些层次分开，才不会把局部成功扩大为最终完成。

工作方法：从身份到功能逐层推进

- 确认目标宿主用户
- 证明镜像引用与本地镜像身份
- 记录容器创建配置
- 判断主进程状态与退出码
- 检查端口发布、挂载和环境
- 证明宿主监听与协议功能
- 证明数据位于可持续来源
- 必要时保存证据后最小重建

每次验证都要同时写清“能证明什么”和“不能证明什么”。`podman ps` 中的 `running` 只证明主进程仍存在；`podman port` 只证明 Podman 记录了映射；`ss` 只证明宿主 socket；`curl` 才能继续证明 HTTP 请求；宿主源目录中的文件则证明数据真正进入外部存储。

主要判断表

看到的证据	可以证明	仍不能证明
<code>podman images</code> 中存在目标引用	当前用户本地存储存在该引用	容器已经创建或正在使用它
<code>podman ps</code> 显示 <code>running</code>	容器主进程当前仍运行	应用已就绪、端口可达、数据正确
<code>podman port NAME</code> 有映射	创建配置中存在端口发布	宿主 socket 和协议功能成功
<code>ss -lnt</code> 出现宿主端口	本机存在对应 TCP 监听	HTTP 内容或远端访问正确
<code>podman inspect</code> 显示挂载	容器配置引用了某个 source	进程拥有 DAC/映射/SELinux 访问权
<code>curl</code> 返回目标内容	当前请求链在本机成功	注销或重启后仍会自动运行
删除容器后宿主文件仍在	数据不只位于旧容器可写层	下一次创建参数一定正确

向下一章交接

本章的终点是“当前 rootless 容器实例、端口、环境、挂载和数据可验证”。用户退出后容器是否继续、主机启动后如何自动创建或启动、如何把 `podman run` 参数转化为可维护的 Quadlet 声明，以及怎样验证冷启动，全部进入下一章《容器持久化、用户 systemd 与 Quadlet》。本章不执行 `loginctl enable-linger`，也不把当前 `running` 当作持久运行证据。

章末速查

```
# 用户与存储上下文
id
podman info
podman images

# 容器对象与运行证据
podman ps -a
podman container inspect NAME
podman logs --tail 50 NAME
podman port NAME

# 宿主功能与数据
ss -lnt
curl -fsS http://127.0.0.1:HOST_PORT/
ls -ldZ HOST_PATH
podman exec NAME id
podman exec NAME test -r CONTAINER_PATH

# 删除前的数据边界
podman inspect NAME --format '{{json .Mounts}}'
podman volume ls
podman volume inspect VOLUME
```

静态验证声明：本章依据 RHEL 9 课程、Red Hat 容器文档、Podman 手册、细分课件和模拟题进行静态核对。当前未执行命令级 live test；涉及版本默认值、用户映射、网络后端和 SELinux 标签变化的结论，应在目标 RHEL 9 环境中再次验证。

RHEL 9 · RHCSA 实操讲义

容器持久化、用户 systemd 与 Quadlet

把一次性运行实例转化为可重建声明，并证明它在用户未登录时仍能启动、保存数据和提供功能。

对象模型 操作语义 验证 诊断 经典任务

大字号阅读版

本章阅读导航

先抓住一条主线：容器实例只是“现在发生了什么”，Quadlet 才描述“下一次应该怎样重建”；用户 systemd manager、**[Install]** 与 linger 共同决定它何时被启动。

01识别作用域确认目标用户、rootless Podman 与 user manager

02记录当前实例保留镜像、名称、端口、环境、挂载和数据证据

03写入声明把期望状态翻译为 `.container` 字段

04生成与启动 `daemon-reload` 生成 unit, start/restart 应用配置

05分层验证unit、容器、数据和协议功能分别取证

06验证生命周期注销和冷启动后，先在目标用户未登录时测功能

专题地图 - **知识专题** 当前实例与声明配置 - **知识专题** 用户 systemd manager 与 linger - **知识专题** Quadlet 搜索路径、名称映射与生成物 - **操作专题** 把 `podman run` 参数翻译为 `.container` - **操作专题** 从声明到运行：reload、start 与证据链 - **知识专题** 依赖、顺序与 Restart - **操作专题** 数据持久化、声明变更与镜像更新 - **诊断专题** unit 未生成、启动失败、循环重启与冷启动失败 - **经典任务** 迁移现有 rootless 容器并完成未登录验收

阅读时持续回答 1. 当前看到的是运行实例、源声明，还是 generated unit？ 2. 命令连接的是系统 manager，还是目标用户的 manager？ 3. `Linger=yes` 证明了什么，又没有证明什么？ 4. 修改 `.container` 后，何时需要 reload，何时需要 restart？ 5. `active`、容器运行、端口映射和业务功能分别属于哪一层？ 6. 数据是否真正脱离容器可写层？ 7. 冷启动验证是否在目标用户首次登录之前完成？

章节边界：容器基本运行归第 31 章；systemd 通用模型归第 12 章；SELinux 持久规则归第 29 章。本章只讲把这些前置对象组织为可重建的用户级容器服务。

第 32 章 · 正文

容器能够运行，不等于它已经成为一项可维护的系统服务。手工执行一次 `podman run`，只建立了一个当前实例；用户退出、主机重启、镜像更新或配置变化以后，这个实例是否还能以相同参数恢复，取决于另一个层次的声明和生命周期管理。本章把容器从“当前正在运行的对象”提升为“可以由用户 systemd manager 重建和监督的声明对象”。最常见的误判是把 `Linger=yes`、`active` 或 `podman ps` 中的一行运行状态直接扩大解释为“重启后必然可用”。正确路径必须把源声明、生成 unit、运行实例、外部数据、功能和登录外生命周期分开取证。主线不是把长串 `podman run` 塞进手写 service，也不是把旧的 `podman generate systemd` 当成唯一答案，而是使用 Quadlet `.container` 文件表达期望状态，再由 generator 生成普通 `.service`，最后通过 `systemctl --user`、`loginctl`、`journalctl` 和 Podman 证据完成分层验收。

概念

声明式容器配置描述的是“下一次应当怎样创建容器”，而不是当前实例已经发生的全部历史。当前容器可以通过 `inspect` 观察，Quadlet 则是后续重建的配置真源；两者发生漂移时，必须先保存当前事实，再决定哪些参数进入长期声明。

概念

用户 systemd manager 属于一个具体 UID，并通过 `systemctl --user` 管理该用户的 unit。它与 PID 1 的 system scope 是两个管理域，所以 root 的普通 `systemctl`、root 的 Podman 存储和目标普通用户的 rootless 容器不能互相替代。

概念

linger 是用户登录外生命周期策略。启用后，系统可在开机时创建该用户的 manager，并在最后一次注销后保留它；linger 只回答 manager 是否能脱离登录会话存在，不能证明 Quadlet 路径、字段、数据权限或应用功能正确。

概念

Quadlet 使用 `.container` 等声明文件描述 Podman 对象。generator 在 user manager 启动或 `daemon-reload` 时读取源文件并生成普通 service，使管理员维护简短字段，而不是长期维护一份由旧版本 Podman 展开的复杂 `ExecStart`。

概念

生成 unit 是 Quadlet 的派生结果。`report.container` 通常生成 `report.service`；它可以被 `status`、`show` 和 `journal` 观察，但不应直接编辑。配置变化应回到源声明，再通过 reload 重新生成。

概念

容器重建意味着旧实例可以被停止、删除并按声明重新创建，因此容器 ID 变化是正常现象。真正需要保持的是题目指定的名称、端口、环境、挂载和功能终态，而不是让某个实例永远不变。

概念

持久数据必须位于宿主目录或 Podman volume 等独立数据层。只在容器可写层中的文件会随实例删除而失去；最有区分度的验证不是简单 restart，而是删除旧实例、由 Quadlet 重建后，新实例仍能读取同一份外部数据。

版本敏感：先确认 Podman，再选择主线

Quadlet 从 Podman 4.6 开始提供，而 RHEL 9 的不同小版本可能携带不同 Podman 版本。进入操作前先执行 `podman version` 并查看本机 `man podman-systemd.unit`。本章以 Quadlet 为主线；较早环境中的 `podman generate systemd` 只作为兼容背景，不作为唯一答案。

操作语义速查

systemctl --user

SYNOPSIS

```
systemctl --user COMMAND [UNIT ...]
```

连接当前用户的 systemd manager，观察或改变 user unit。对 rootless Quadlet，必须在目标用户的完整会话中执行，不能把 system scope 的结果混进来。

status NAME.service

查看当前加载和运行摘要；适合快速定位，但不能替代结构化字段与功能测试。

show NAME.service -p ...

读取 LoadState、ActiveState、SubState、Result、NRestarts 等最小状态向量。

start / stop / restart

改变当前运行状态；restart 才会让运行实例采用已重新生成的配置。

loginctl enable-linger

SYNOPSIS

```
loginctl enable-linger USER
loginctl show-user USER -p Linger -p State
```

让指定用户的 manager 可以在开机时创建，并在最后一次注销后继续存在。该操作通常需要管理员授权。

enable-linger USER

写入持久的 linger 状态；它不创建或修复容器声明。

show-user USER -p Linger

验证策略状态；Linger=yes 仍不能证明 unit 或应用已经正常。

user-status USER

观察用户 manager、会话和相关进程；适合冷启动后由其他管理员账号取证。

Quadlet .container

SYNOPSIS

```
~/.config/containers/systemd/NAME.container

[Container]
Image=IMAGE
ContainerName=NAME
PublishPort=HOST:CONTAINER
Volume=SOURCE:TARGET[:OPTIONS]
Environment=KEY=VALUE
```

把容器的期望状态写成字段。文件名决定生成的 service 名，字段决定下一次创建的实例参数。

Image=

必需字段；优先使用完整镜像引用，并区分可变 tag 与实际 image ID。

ContainerName=

覆盖默认的 systemd-NAME 容器名；显式命名时要先处理同名旧实例。

PublishPort=

左侧宿主端口，右侧容器端口；端口映射存在不等于协议功能正确。

Volume=

左侧外部数据源，右侧容器内目标；需要同时满足传统权限、UID 映射和 SELinux 边界。

Environment=

可重复使用；这里遵循 unit/Quadlet 语法，不能机械套用 shell 引号规则。

daemon-reload 与声明应用

SYNOPSIS

```
systemctl --user daemon-reload
systemctl --user restart NAME.service
```

reload 让 manager 重新运行 generator 并读取源声明，restart 才让当前运行实例采用新生成的 service。两步改变的对象不同。

daemon-reload

证明 manager 已重新读取配置入口；它不会自动重启业务。

restart NAME.service

停止旧实例并按新配置重建；执行前应确认持久数据和可接受的停机影响。

[Install] WantedBy=default.target

表达 user manager 内的启动关系；generated service 不能通过普通 `enable` 流程获得持久启用关系。

journalctl --user-unit

SYNOPSIS

```
journalctl --user-unit=NAME.service [-b] [-n N] [--no-pager]
```

读取 user unit 的生成、启动、退出和重启证据。查询必须处于正确用户作用域，日志是进入失败分支的入口，而不是事后装饰。

-b

限制为当前 boot，便于区分历史故障和本次冷启动。

-n N

先读取最近 N 条，快速找到第一条高价值错误。

--since

按变更时间缩小范围，避免在大量历史记录中猜测。

重启后未登录验证

SYNOPSIS

```
# 主机重启后，先不要登录目标用户
curl -fsS http://SERVER:PORT/
loginctl user-status USER

# 随后进入目标用户完整会话复核
systemctl --user status NAME.service
journalctl --user-unit=NAME.service -b
```

冷启动验收必须避免登录动作污染证据。先从其他账号或远端验证业务功能，再进入目标用户会话查看 unit、容器和日志。

外部协议测试

证明真实入口在目标用户尚未登录时已经可用；比单看 active 更接近终态。

loginctl user-status USER

观察 user manager 是否已经存在；它仍不能替代协议和数据验证。

登录后回查

用于解释启动链和日志，不得把登录后才恢复的服务误判为开机成功。

[知识专题] 当前实例与声明配置：先分清“已经发生”与“下一次应发生”

把现有容器迁移为 systemd 服务时，最危险的做法是直接抄一条记忆中的模板。现有实例可能包含题目要求保留的数据、端口、环境变量和用户映射，也可能包含临时试验参数。正确切入点是先建立当前事实，再决定哪些事实应进入声明，哪些应该被纠正。

① [查询] 用当前实例建立迁移基线

podman ps -a 能显示名称、状态、端口等摘要，podman inspect 能读取创建时的详细配置。它们回答的是：当前这个实例怎样被创建、现在处于什么状态。容器 ID 不是长期配置标识；实例被删除和重建后，ID 通常变化。

建议在迁移前至少记录：

```
podman ps -a --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'
podman inspect report > ~/report-before.json
podman port report
podman logs --tail 50 report
```

`inspect` 是基线证据，不是自动生成声明的权威模板。它可能保留历史临时选项，仍需根据任务目标筛选。

② [知识点] 声明配置描述下一次生成的期望状态

`.container` 文件表达下一次服务启动时应使用的镜像、名称、端口、挂载和环境。修改声明不会在同一瞬间改写当前容器；`systemd` 先要重新加载生成结果，运行实例再通过 `restart` 或下一次启动采用新配置。

因此需要区分三次变化：

```
编辑 .container
→ systemctl --user daemon-reload 重新生成 service
→ systemctl --user restart NAME.service 重建运行实例
```

只完成第一步时，当前业务可能仍在使用旧参数；只执行 `restart` 而没有 `reload`，也可能继续使用旧的 generated unit。

③ [操作决策] 从现有事实筛选可维护终态

迁移时将现有事实分成三类：

类别	例子	处理
必须保留	业务数据目录、规定端口、必要环境变量	进入声明并逐项验证
应纠正	临时随机端口、错误名称、数据只在可写层	按任务终态重构
暂不进入主线	调试用 shell、临时 capability、实验标签	有明确需求才保留

迁移前不要先删除旧实例。先确认数据在何处、是否有外部副本、端口是否被依赖，再安排最小停机切换。

Cheatsheet

``inspect`` 建立当前事实；``container`` 表达下一次期望；编辑、`reload`、`restart` 是三个不同动作；容器 ID 变化不等于数据丢失，数据位置才是关键。

[知识专题] 用户 systemd manager：作用域比命令名字更重要

容器由哪个用户运行，会决定 Podman 存储、容器可见性、用户 bus、unit 搜索范围和日志作用域。很多“unit not found”并不是 unit 文件不存在，而是当前命令连到了错误的 manager。

① [知识点] system scope 与 user scope 是两个管理域

普通命令：

```
systemctl status sshd.service
```

连接系统 manager，管理系统级 unit。用户命令：

```
systemctl --user status report.service
```

连接当前用户的 manager，管理该用户的 unit。对于 rootless 容器，Quadlet、Podman 存储和 user unit 必须归属于同一目标用户。

以下证据应在目标用户上下文中取得：

```
id
podman ps -a
systemctl --user list-unit-files
systemctl --user status report.service
```

不要把 root 的 `podman ps` 为空解释为目标用户没有容器，也不要将系统 manager 的 `Unit report.service could not be found` 直接解释为 Quadlet 未生成。

② [知识点] 登录会话与用户 manager 相关，但不是同一对象

用户通过 SSH、控制台或其他 PAM 会话登录时，通常会得到用户 runtime 目录和 user manager 访问环境。注销最后一个会话后，如果没有 linger，用户 manager 可以被停止；由它监督的长期 user service 也会失去管理者。

查询用户生命周期：

```
loginctl show-user appsvc -p Linger -p State -p Sessions
loginctl user-status appsvc
```

`State=active` 只表示当前存在活跃会话或 manager 状态，不等同于重启后会自动创建；`Linger=yes` 才是脱离登录会话的持久策略之一。

③ [边界] linger 只解决 manager 的登录外生命周期

启用：

```
sudo loginctl enable-linger appsvc
```

验证：

```
loginctl show-user appsvc -p Linger
```

Linger=yes 可以支持开机时创建 user manager，并使最后一次注销后 manager 继续存在。但它不能证明：

- .container 在正确目录；
- [Install] 已表达启动关系；
- generated service 能加载；
- 容器镜像可用；
- 端口和数据目录无冲突；
- 应用功能正确。

Cheatsheet

rootless 容器、Quadlet、user unit 和日志都要在同一 UID 作用域内取证；`--user` 不是装饰选项；linger 解决登录外生命周期，不解决配置正确性。

[知识专题] Quadlet 的输入、搜索路径和生成物

Quadlet 的价值在于让管理员维护简短的容器声明，由 Podman generator 根据当前版本生成具体 ExecStart= 等实现细节。这样升级 Podman 后不需要继续维护一份历史生成脚本。

① [查询] 先按运行身份确定 Quadlet 搜索路径

本章主线使用当前用户目录：

```
$XDG_CONFIG_HOME/containers/systemd/  
~/.config/containers/systemd/
```

管理员也可以为特定 UID 或全部用户提供系统级用户声明：

```
/etc/containers/systemd/users/<UID>/  
/etc/containers/systemd/users/
```

rootful 系统声明使用：

```
/etc/containers/systemd/  
/usr/share/containers/systemd/
```

考试任务应先明确 rootless 还是 rootful。把 rootless Quadlet 放入 ~/.config/systemd/user/ 是常见路径错误；该目录适合普通 user unit，不是本章 .container 主路径。

② [知识点] 文件名映射为 service 和默认容器名

report.container 生成 report.service。若未写 ContainerName=，Podman 默认采用带 systemd- 前缀的名称，例如 systemd-report，以减少与手工容器重名。

```
report.container  
└ generator  
report.service  
└ podman run  
systemd-report    (未显式 ContainerName 时)
```

任务明确要求容器名时应写 ContainerName=report，并在迁移前处理已有同名实例，避免启动时发生 name already in use。

③ [边界] generated service 是派生状态，不是配置真源

generator 在开机和 manager reload 时读取声明。管理员应修改 .container，然后执行：

```
systemctl --user daemon-reload
```

可以使用以下命令观察生成结果：

```
systemctl --user status report.service
systemctl --user cat report.service
systemctl --user show report.service -p FragmentPath -p SourcePath -p LoadState
```

实际字段显示会随 systemd/Podman 版本变化，不能把某条固定输出当成唯一评分证据。核心判断是 service 能否被正确加载，并且来源可追溯到目标 `.container`。

Cheatsheet

rootless 路径是 `~/config/containers/systemd/`；后缀必须是 `.container`；`NAME.container → NAME.service`；默认容器名可能是 `systemd-NAME`；只编辑源声明，不编辑生成物。

[操作专题] 把 `podman run` 参数翻译为 `.container`

Quadlet 不是另一种 shell 命令拼接，而是把容器参数映射为可读字段。转换时应逐项回答：该字段控制哪个对象，是否能从题目终态验证，是否需要重复出现。

① [操作] 镜像与容器名称

作用对象：下一次创建的容器实例。

基本形式：

```
[Container]
Image=registry.example.com/rhcsa/report:9
ContainerName=report
```

`Image=` 是 `.container` 的必需字段。优先使用完整镜像名，避免短名称解析不确定。tag 表示可变名称；若任务要求精确不可漂移版本，可使用 digest，但考试题通常会给定 tag 或完整引用。

`ContainerName=` 是可选字段。省略时要接受默认 `systemd-` 前缀名称；题目指定名字时再显式设置。

验证：

```
podman inspect report --format '{{.Name}} {{.ImageName}} {{.Image}}'
```

不要把 `podman images` 中目标 tag 当成运行实例已经采用该 image ID 的证据。

② [操作] 端口与环境变量

基本形式：

```
[Container]
PublishPort=18080:8080
Environment=REPORT_MODE=exam
Environment=LOG_LEVEL=info
```

`PublishPort=` 对应 `podman run --publish`，可重复使用。`18080:8080` 表示宿主端口 18080 映射到容器端口 8080。只有容器端口而没有固定宿主端口时可能产生动态宿主端口，不适合要求固定入口的题目。

`Environment=` 对应 `--env`，可重复使用。包含空格或特殊字符时要遵循 systemd 单元语法，避免随意套用 shell 引号规则。

验证：

```
podman port report
podman inspect report --format '{{json .Config.Env}}'
curl -fsS http://127.0.0.1:18080/
```

端口存在只能证明转发表达，`curl` 或实际协议测试才进入功能层。

③ [操作] 数据卷、服务重启与开机关系

基本形式：

```
[Container]
Volume=/home/appsvc/report-data:/var/lib/report:Z

[Service]
Restart=on-failure
RestartSec=5

[Install]
WantedBy=default.target
```

`Volume=` 对应 `--volume`，可重复使用。宿主路径在左，容器路径在右。`:Z` 为该容器准备私有 SELinux 标签；共享语义和完整标签诊断归第 29、31 章，本章只要求保留最小安全边界。

`Restart=on-failure` 属于生成 service 的 `[Service]` 配置，适合进程异常退出后重试；它不会修复不可拉取镜像、重名容器、端口占用和持续权限错误，只会让失败反复出现。

Quadlet 生成的 service 属于 transient/generated unit，不能通过 `systemctl --user enable report.service` 获得持久启用关系。自动启动关系必须写进源文件 `[Install]`，由 generator 在生成阶段应用；用户级服务通常使用 `default.target`。

完整示例：

```
[Unit]
Description=Persistent rootless report container

[Container]
Image=registry.example.com/rhcsa/report:9
ContainerName=report
PublishPort=18080:8080
Volume=/home/appsvc/report-data:/var/lib/report:Z
Environment=REPORT_MODE=exam

[Service]
Restart=on-failure
RestartSec=5
TimeoutStartSec=900

[Install]
WantedBy=default.target
```

`TimeoutStartSec=900` 不是所有题目都需要。镜像首次拉取可能超过 systemd 默认启动超时，只有证据表明启动卡在拉取且网络正常时，才考虑预拉镜像或延长超时。

Cheatsheet

``Image`` 必需；``ContainerName`` 控制实例名；``PublishPort`` 左宿主右容器；``Volume`` 左源右目标；``Environment`` 可重复；``Restart`` 不修复根因；Quadlet 开机关系写 ``[Install]``。

[操作专题] 从声明到运行：reload、start 与证据链

最小流程不是一句 `start`。每一步产生不同证据，失败时应停在当前层调查，而不是继续堆命令。

① [操作] 建立目录和声明

```
install -d -m 0755 ~/.config/containers/systemd
vi ~/.config/containers/systemd/report.container
```

检查：

```
ls -l ~/.config/containers/systemd/report.container
sed -n '1,200p' ~/.config/containers/systemd/report.container
```

文件应属于目标用户，宿主数据目录也要在该用户和容器映射可访问的权限范围内。不要为了跳过调查直接 `chmod 777`。

② [操作] 重新生成并启动 unit

```
systemctl --user daemon-reload
systemctl --user start report.service
```

若需要立即应用声明变更：


```
systemctl --user restart report.service
```

`daemon-reload` 让 manager 重新读取 source unit 并运行 generator，不会自行重启已运行服务。`restart` 负责让新生成的 service 进入运行实例。

③ [验证] 沿 manager、实例和功能三层取证

```
systemctl --user status report.service --no-pager
systemctl --user show report.service \
  -p LoadState -p ActiveState -p SubState -p Result -p NRestarts
podman ps -a --filter name=report
podman port report
curl -fsS http://127.0.0.1:18080/
```

`ActiveState=active` 证明 systemd 当前认为主服务活动；它不能独立证明应用内容正确。若镜像内进程保持运行但返回错误页面，unit 仍可能 active。

帮助入口：

```
man podman-systemd.unit
man systemctl
man systemd.service
```

Cheatsheet

创建声明后 reload；需要当前采用新配置再 restart；`status/show` 查 manager，`podman` 查实例，协议请求查功能。

[知识专题] 依赖、顺序与 Restart：三种不同关系

容器服务可能依赖另一个服务、挂载点或网络准备。依赖关系不能只靠排列文件名，也不能只写 `After=` 就假设依赖对象一定会启动。

① [知识点] `Requires=` 表达启动和失败关系

```
[Unit]
Requires=database.service
```

要求 database unit 一起参与事务；所需 unit 无法启动时，当前 unit 通常不能成功进入目标状态。它表达“需要谁”，不保证先后顺序。

② [知识点] `After=` 只表达顺序

```
[Unit]
After=database.service
```

表示两者都进入同一事务时，本 unit 在 database 之后启动。它不会主动把 database 拉入事务。常见组合是：

```
[Unit]
Requires=database.service
After=database.service
```

依赖其他 Quadlet 时使用生成后的 `.service` 名称，例如 `After=database.service`，不是 `After=database.container`。

③ [边界] Restart 处理退出结果，不处理声明错误

```
[Service]
Restart=on-failure
RestartSec=5
```

该策略处理非零退出、信号终止等失败。以下问题通常会形成稳定失败或循环：

- `ContainerName=` 与旧实例重名；
- 宿主端口已占用；
- 数据目录不可访问；
- 应用参数错误，启动即退出；
- 镜像引用无效。

出现循环时先取证：

```
systemctl --user show report.service -p Result -p NRestarts
journalctl --user-unit=report.service -b -n 100 --no-pager
podman ps -a --filter name=report
```

不要把 `Restart=always` 当成“提高可靠性”的无条件答案。若应用正常完成后本应退出，`always` 会制造无意义循环。

Cheatsheet

`Requires` 解决“需要谁”，`After` 解决“谁先谁后”，`Restart` 解决“失败退出后怎么办”；三者不可互换。

[操作专题] 让数据独立于容器实例

持久化的目标不是让容器 ID 永远不变，而是让业务数据不依赖一次实例的可写层。最强的验证不是“容器重启后文件还在”，而是删除并由声明重建实例后，外部数据仍能被新实例读取。

① [查询] 识别数据究竟位于哪一层

调查现有挂载：

```
podman inspect report --format '{{.json .Mounts}}'
podman volume ls
findmnt -T /home/appsvc/report-data
```

如果关键文件只存在容器内且没有对应挂载，应先导出或复制到规划的持久目录，再删除旧实例。不要先 `podman rm` 再确认数据。

② [操作] 准备宿主目录和最小权限

```
install -d -m 0750 ~/report-data
printf 'persistent-marker\n' > ~/report-data/marker.txt
ls -ldZ ~/report-data
```

宿主传统权限、容器内 UID、rootless 用户映射和 SELinux 标签共同决定访问。默认故障顺序：

```
宿主路径是否正确
→ 所有者和模式是否允许
→ 容器内进程使用哪个 UID
→ 当前 SELinux 上下文和 AVC
```

禁止以 `chmod 777` 或关闭 SELinux 代替调查。

③ [验证] 用删除和重建证明数据独立

最低层：

```
podman exec report cat /var/lib/report/marker.txt
```

更强验证：

```
systemctl --user stop report.service
podman ps -a --filter name=report
systemctl --user start report.service
podman exec report cat /var/lib/report/marker.txt
```

若维护窗口和任务允许，可在确认数据外置后删除实例，再由 `unit` 重建；不要删除宿主数据目录或 `named volume`。

```
systemctl --user stop report.service
podman rm report
systemctl --user start report.service
podman exec report cat /var/lib/report/marker.txt
```

该流程证明数据跨实例，而不是只跨进程 `restart`。实际考试中是否手工 `podman rm` 要依据题目和现有状态决定，不把破坏性步骤当成无条件模板。

Cheatsheet

可写层随实例；外部目录/`volume` 才是持久层；先定位数据再删除；最强验证是新实例读取旧数据。

[操作专题] 声明变更、镜像更新与重建边界

容器维护常见误判是“镜像已经 pull，所以服务已经更新”。本地 tag 指向新 image ID，不会自动把已运行容器替换成新实例；Quadlet 源文件变化也不会自动让当前容器采用新值。

① [查询] 变更前保存实例和功能基线

```
podman inspect report > ~/report-before.json
systemctl --user show report.service \
  -p ActiveState -p SubState -p Result -p NRestarts
curl -fsS http://127.0.0.1:18080/ > ~/report-before.out
```

基线至少覆盖运行状态、镜像、端口、挂载、环境和业务输出。

② [操作] 修改声明后 reload + restart

```
vi ~/.config/containers/systemd/report.container
systemctl --user daemon-reload
systemctl --user restart report.service
```

若只改了宿主数据内容或应用自身可热加载配置，是否需要容器 restart 取决于应用；不要把 systemd daemon-reload 误认为应用 reload。

③ [验证] 区分镜像引用、image ID 与当前实例

```
podman image inspect registry.example.com/rhcsa/report:9 \
  --format '{{.Id}}'
podman inspect report --format '{{.Image}} {{.ImageName}}'
```

只有当前实例的 .Image 与期望 image ID 一致，才能证明它采用了当前本地镜像。更新后重新验证端口、环境、挂载、数据和功能。自动更新机制属于扩展，不在 RHCSA 主线完整展开。

Cheatsheet

pull 改变本地镜像，不直接替换容器；源变更要 reload；实例采用新配置要 restart/recreate；更新后回归数据和业务功能。

[诊断专题] linger 已启用，但 unit 没有生成

症状可能是：loginsctl show-user 显示 Linger=yes，但 systemctl --user status report.service 返回 unit not found。此时 linger 已经排除一个生命周期问题，但不能证明 Quadlet 已被目标 manager 读取。

① [诊断] 先确认作用域和版本

```
id
podman version
systemctl --user --version
loginsctl show-user "$USER" -p Linger
```

假设 A：命令在错误用户下执行。下一条最有区分度的证据是 id 与 echo \$HOME，再比较声明的所有者和路径。

假设 B：Podman 版本低于 Quadlet 支持范围。下一条证据是 podman version 和本机 man podman-systemd.unit 是否存在相关 .container 说明。

② [诊断] 检查路径、后缀和内容

```
find ~/.config/containers/systemd -maxdepth 1 -type f -printf '%f\n'
sed -n '1,200p' ~/.config/containers/systemd/report.container
```

常见错误：

- 放到 ~/.config/systemd/user/；
- 文件名是 report.service 或 report.container.txt；
- [Container] 拼写错误；
- 缺少必需的 Image=；
- 文件属于其他用户；
- 编辑器写入不可见字符或错误换行。

③ [诊断] reload 后从 manager 日志取证

```
systemctl --user daemon-reload
systemctl --user list-unit-files | grep -F report
journalctl --user -b --since '-10 min' --no-pager
```

最小修复：只修正错误路径、字段或用户归属，再 reload。不要为了让 unit 出现而另写一份手工 `.service`，否则会绕过本章声明主线并制造两个配置真源。

再验证：`report.service` 可加载、来源指向 `.container`，然后才进入启动层。

Cheatsheet

`Linger=yes + unit not found` 优先查用户、版本、路径、后缀、必需字段和 generator 日志；不要先改成 rootful，也不要另建同名手工 service。

[诊断专题] unit 已生成，但容器启动失败或循环重启

这里的目标是从 systemd 结果推进到 Podman 和宿主资源，而不是反复执行 restart。

① [诊断] 读取最小状态向量

```
systemctl --user show report.service \
  -p LoadState -p ActiveState -p SubState -p Result -p NRestarts
journalctl --user-unit=report.service -b -n 100 --no-pager
```

`Result=exit-code`、`NRestarts` 增长和 journal 中首个失败原因，比最后一行“Scheduled restart job”更有价值。

② [诊断] 按错误类型进入下一层

日志线索	假设	下一条证据
name already in use	同名手工实例残留	<code>podman ps -a --filter name=report</code>
address already in use	宿主端口冲突	<code>ss -ltnp grep :18080</code> 、 <code>podman ps --format</code>
permission denied	宿主 DAC、UID 映射或 SELinux	<code>namei -l</code> 、 <code>ls -ldZ</code> 、AVC 日志
image not known/pull failed	镜像引用或网络/认证	<code>podman image exists</code> 、手工 <code>podman pull</code> 的错误
容器很快 Exited	应用入口或参数失败	<code>podman ps -a</code> 、 <code>podman logs</code>

手工运行诊断命令时不要輕易创建另一个同名容器；优先读取 generated service、journal 和失败实例证据。

③ [诊断] 最小修复后清除失败并重验

修正根因后：

```
systemctl --user daemon-reload # 仅当声明有修改
systemctl --user reset-failed report.service
systemctl --user restart report.service
```

再按 unit、容器、功能和数据四层验证。`reset-failed` 只清理失败计数和状态，不修复配置。

Cheatsheet

先看 `Result/NRestarts` 和第一失败原因；重名查容器，端口查监听，权限查 DAC/映射/SELinux，退出查应用日志；修根因后再 restart。

[诊断专题] 当前运行正常，但注销或重启后失败

此类故障说明当前运行层已经通过，问题集中在 user manager 生命周期、安装关系或“未登录验证方法”上。

① [诊断] 分开检查 [Install] 与 linger

源声明：

```
sed -n '/^\[Install\]/,$p' ~/.config/containers/systemd/report.container
```

linger：

```
logindctl show-user appsvnc -p Linger
```

两者缺一不可： `WantedBy=default.target` 表达 manager 启动时拉起 service；linger 支持 manager 在开机和注销后存在。

② [诊断] 注销测试不能依赖同一个会话

在目标用户会话中确认当前功能后，退出该用户全部会话。从另一个管理员账号或远端客户端验证：

```
curl -fsS http://HOST:18080/  
loginctl show-user appsvc -p Linger -p State
```

若重新登录目标用户后服务才恢复，说明“登录触发启动”，不能算作未登录持久成功。

③ [诊断] 冷启动证据应先于目标用户登录

重启后不要先登录 `appsvc`。先从其他账号或外部主机测试业务入口，再查看：

```
loginctl user-status appsvc
```

随后进入目标用户完整会话，核对：

```
systemctl --user status report.service --no-pager  
journalctl --user-unit=report.service -b --no-pager  
podman ps -a
```

本会话没有 RHEL 9 VM，以上属于推荐 live-test 流程，不能声明已执行。

Cheatsheet

当前 active 只证明现在；`[Install]` 解决 manager 内的启动关系，linger 解决 manager 的登录外生命周期；冷启动验收要在目标用户首次登录前先测功能。

[经典任务] 把现有 rootless 容器迁移为 Quadlet，并验证未登录持久运行

环境

服务器上存在普通用户 `appsvc`。该用户当前手工运行一个名为 `report` 的 rootless 容器：

```
镜像: registry.example.com/rhcsa/report:9  
宿主端口: 18080  
容器端口: 8080  
宿主数据目录: /home/appsvc/report-data  
容器数据目录: /var/lib/report  
环境变量: REPORT_MODE=exam
```

`/home/appsvc/report-data/marker.txt` 是必须保留的数据。当前容器可以访问，但 `appsvc` 注销后不再可靠运行，系统重启后也未建立未登录启动证据。

目标终态

1. 以 `appsvc` 的 rootless Podman 运行，不改成 rootful。
2. 使用 `~/.config/containers/systemd/report.container` 作为维护真源。
3. 生成并运行 `report.service`，容器名称保持 `report`。
4. 端口、环境变量和数据挂载符合题目。
5. 异常退出时采用 `Restart=on-failure`，间隔 5 秒。
6. 用户注销后服务继续运行。
7. 主机重启后，在 `appsvc` 尚未登录时，外部仍能访问宿主端口。
8. 标记文件在迁移、注销和重启后仍存在。

限制条件

- 不得 `chmod 777`；
- 不得关闭 SELinux；
- 不得删除 `/home/appsvc/report-data`；
- 不得把数据只复制进容器可写层；
- 不得直接编辑 generated service；
- 不得以“重新登录后服务恢复”代替未登录启动验证；
- 删除旧容器前必须确认持久数据位置。

验收证据

层	必须给出的证据
身份	id、目标用户的 <code>podman ps -a</code>
声明	<code>.container</code> 路径、关键字段
生成	<code>report.service</code> 可加载且来源正确
unit	<code>ActiveState</code> 、 <code>SubState</code> 、 <code>Result</code> 、 <code>NRestarts</code>
实例	名称、镜像、端口、环境和挂载
数据	宿主和容器内都能读到标记文件
功能	从宿主或远端请求 <code>18080</code> 成功
生命周期	<code>Linger=yes</code> 、注销后功能、冷启动未登录功能
日志	当前 boot 的 user unit journal 无持续失败

[参考解答] 经典任务一

以下命令中的主机名、镜像可访问性和应用返回内容由实际实验环境决定。本章只提供静态核对后的参考流程，不声称已在 RHEL 9 VM 执行。

一、调查当前实例和数据

以 `appsvc` 的完整登录环境操作：

```
id
podman version
podman ps -a --filter name=report
podman inspect report > ~/report-before.json
podman port report
podman inspect report --format '{{json .Mounts}}'
ls -lZ ~/report-data/marker.txt
```

确认 `marker.txt` 位于宿主目录，而不是只存在容器可写层。若当前实例没有正确挂载，应先安全导出数据，不能直接删除实例。

记录当前功能：

```
curl -fsS http://127.0.0.1:18080/
```

二、建立 Quadlet 声明

```
install -d -m 0755 ~/.config/containers/systemd
cat > ~/.config/containers/systemd/report.container <<'EOF'
[Unit]
Description=Persistent rootless report container

[Container]
Image=registry.example.com/rhcsa/report:9
ContainerName=report
PublishPort=18080:8080
Volume=/home/appsvc/report-data:/var/lib/report:Z
Environment=REPORT_MODE=exam

[Service]
Restart=on-failure
RestartSec=5
TimeoutStartSec=900

[Install]
WantedBy=default.target
EOF
```

静态核对：

```
sed -n '1,200p' ~/.config/containers/systemd/report.container
ls -l ~/.config/containers/systemd/report.container
```

`TimeoutStartSec=900` 用于降低首次拉取镜像超过默认超时的风险；若镜像已预拉且环境不需要，可省略。不要通过 `PodmanArgs=` 绕过有明确字段的标准配置。

三、处理同名旧实例

由于声明显式使用 `ContainerName=report`，旧实例若仍存在会发生重名。先停止当前业务并再次确认数据：

```
podman stop report
ls -lZ ~/report-data/marker.txt
```

只有确认数据已经外置后，才删除旧实例：

```
podman rm report
```

若题目不允许删除现有容器，可暂时不指定 `ContainerName=`，接受默认名称，并在维护窗口另行迁移；但这不满足本任务“名称保持 report”的终态。

四、重新生成并启动

```
systemctl --user daemon-reload
systemctl --user start report.service
```

检查 generated unit：

```
systemctl --user status report.service --no-pager
systemctl --user show report.service \
  -p LoadState -p ActiveState -p SubState -p Result -p NRestarts
systemctl --user cat report.service
```

不要对 generated service 机械执行：

```
systemctl --user enable report.service
```

开机关系已经写在 `.container` 的 `[Install]` 中，由 generator 应用。

五、验证容器参数、功能与数据

```
podman ps -a --filter name=report
podman inspect report --format '{{.Name}} {{.ImageName}} {{.Image}}'
podman port report
podman inspect report --format '{{json .Config.Env}}'
podman inspect report --format '{{json .Mounts}}'
podman exec report cat /var/lib/report/marker.txt
cat ~/report-data/marker.txt
curl -fsS http://127.0.0.1:18080/
```

如果 unit active 但 `curl` 失败，继续检查应用监听地址、容器内端口、应用日志和主机防火墙；不要把 active 扩大解释为功能正确。

六、启用并验证 linger

由有权限的管理员执行：

```
sudo loginctl enable-linger appsvc
loginctl show-user appsvc -p Linger
```

预期证据是 `Linger=yes`。这一步不替代 `.container` 和 service 验证。

七、验证注销后运行

退出 `appsvc` 的全部登录会话。使用另一个账号或远端主机验证：

```
curl -fsS http://SERVER:18080/
loginctl show-user appsvc -p Linger -p State
```

若只有重新登录 `appsvc` 后才恢复，继续调查 `[Install]`、generator 生成关系和 user manager，而不是宣告完成。

八、验证重启后未登录运行

在得到授权和维护窗口后重启。主机起来后，先不要登录 `appsvc`。从另一个账号或远端客户端执行功能测试：

```
curl -fsS http://SERVER:18080/  
loginctl user-status appsvc
```

随后进入 `appsvc` 的完整登录环境，回查：

```
systemctl --user status report.service --no-pager  
systemctl --user show report.service \  
-p ActiveState -p SubState -p Result -p NRestarts  
journalctl --user-unit=report.service -b --no-pager  
podman ps -a --filter name=report  
podman exec report cat /var/lib/report/marker.txt
```

九、典型错误

1. Quadlet 放错目录： `~/.config/systemd/user/report.container` 不属于本章 rootless Quadlet 搜索路径。
2. 只开 `linger`： `manager` 可以存在，但没有正确 generated unit，容器仍不会启动。
3. 只看 `active`： 应用可能返回错误、端口不可达或数据挂载错误。
4. 直接 `enable` generated service： 忽略 Quadlet transient unit 的 `[Install]` 机制。
5. 先删容器再找数据： 若数据在可写层，会造成不可恢复损失。
6. 用 `chmod 777` 修权限： 破坏最小权限，且可能仍绕不过 SELinux 或 UID 映射。
7. 登录后验证冷启动： 登录动作本身可能创建 `manager` 或触发 unit，污染证据。

[经典任务] 诊断 `linger` 已启用但 unit 未生成，并处理重启循环

场景

用户 `websvc` 的 `Linger=yes`。管理员声称已经创建容器服务，但出现两组症状：

1. `systemctl --user status portal.service` 最初返回 `unit not found`；
2. 修改后 unit 出现，却快速进入 `failed`，并且 `NRestarts` 持续增长。

已知系统中存在：

- 文件 `~/.config/systemd/user/portal.container`；
- 一个停止状态的手工容器 `portal`；
- 宿主端口 `18090` 可能被其他进程使用；
- 数据目录 `~/portal-data` 不得删除；
- 禁止改成 `rootful`、禁止关闭 SELinux、禁止用 `Restart=always` 掩盖错误。

要求

按“症状 -> 当前证据 -> 假设 -> 下一条区分度最高的证据 -> 最小修复 -> 再验证”提交处理过程，并最终满足：

- `portal.container` 位于正确路径；
- generated `portal.service` 可加载；
- 容器名称和端口无冲突；
- 数据目录可由应用访问；
- unit 不再循环；
- 协议功能和持久数据通过验证。

[参考解答] 经典任务二

第一阶段：unit not found

症状： `portal.service` 不存在。

当前证据：

```
id  
loginctl show-user websvc -p Linger  
find ~/.config -maxdepth 4 -type f -name 'portal*' -print
```

假设： `.container` 放入普通 user unit 目录，而不是 Quadlet 目录。

下一条最有区分度的证据：路径显示为：

```
/home/websvc/.config/systemd/user/portal.container
```

最小修复：

```
install -d -m 0755 ~/.config/containers/systemd
mv ~/.config/systemd/user/portal.container \
    ~/.config/containers/systemd/portal.container
systemctl --user daemon-reload
```

再验证：

```
systemctl --user status portal.service --no-pager
systemctl --user show portal.service -p LoadState -p SourcePath -p FragmentPath
```

若仍不存在，继续查 Podman 版本、`Image=`、文件后缀和 user journal，而不是创建同名手工 service。

第二阶段：restart loop

症状：service 出现但快速失败。

当前证据：

```
systemctl --user show portal.service \
    -p ActiveState -p SubState -p Result -p NRestarts
journalctl --user-unit=portal.service -b -n 100 --no-pager
```

假设按日志分支：

分支 A：容器重名

```
podman ps -a --filter name=portal
```

若旧手工实例存在，先确认其挂载和数据：

```
podman inspect portal --format '{{json .Mounts}}'
ls -lZ ~/portal-data
```

确认数据外置后，停止并删除旧实例：

```
podman stop portal 2>/dev/null || true
podman rm portal
```

分支 B：宿主端口冲突

```
ss -ltnp | grep ':18090 '
podman ps --format 'table {{.Names}}\t{{.Ports}}'
```

如果题目指定端口，停止或调整真正冲突对象前先确认业务归属：不能随意改成随机端口并宣告完成。

分支 C：数据目录访问失败

```
namei -l ~/portal-data
ls -ldZ ~/portal-data
podman inspect portal --format '{{json .Config.User}}' 2>/dev/null
```

先修正最小传统权限和所有者，再检查 AVC。不得 `chmod 777`，不得 `setenforce 0`。

第三阶段：再生成、再启动和分层验证

声明有修改时：

```
systemctl --user daemon-reload
systemctl --user reset-failed portal.service
systemctl --user restart portal.service
```

验证：

```
systemctl --user show portal.service \
  -p ActiveState -p SubState -p Result -p NRestarts
podman ps -a --filter name=portal
podman port portal
podman exec portal test -r /var/lib/portal/marker.txt
curl -fsS http://127.0.0.1:18090/
journalctl --user-unit=portal.service -b -n 50 --no-pager
```

`NRestarts` 在一次历史故障后不一定立即变成零；关键是修复后不再持续增长，当前 `Result` 和业务功能正确。

[本章收束] 把容器持久性拆成一条可执行的工作方法

面对“让 rootless 容器重启后自动运行”这类要求，不要直接从模板或启动命令开始。先把任务翻译为七个可证明的对象：目标 UID、现有实例、源声明、generated unit、运行实例、外部数据和未登录功能。任何一层没有证据，都不能用下一层的成功代替。

- 1 确认身份与版本目标用户、rootless 存储、Podman 版本、本机 man page
- 2 保存当前基线实例参数、端口、挂载、数据位置和当前功能
- 3 建立源声明把题目终态翻译为 Quadlet 字段和 `[Install]`
- 4 重新生成与切换 reload 生成 unit，安全处理旧实例，再 start/restart
- 5 分层验收 manager、实例、数据、协议和 journal 分别取证
- 6 验证登录外生命周期 linger、注销测试、冷启动未登录测试

章末检查清单

- 操作命令连接的是目标用户的 user manager，而不是 system scope。
- `podman version` 和本机 `podman-systemd.unit(5)` 支持所用 Quadlet 语义。
- 现有实例的镜像、端口、环境、挂载和数据位置已保存为基线。
- `.container` 位于正确搜索路径，文件名、后缀和所有者正确。
- `Image`、`ContainerName`、`PublishPort`、`Volume`、`Environment` 与目标终态一致。
- `[Install] WantedBy=default.target` 与 `Linger=yes` 分别完成自己的职责。
- reload、restart 的先后和作用对象清楚，未直接编辑 generated unit。
- unit、容器、数据、协议功能和日志都已分别验证。
- 持久数据经过实例删除和重建验证，而不仅是简单 restart。
- 冷启动功能测试发生在目标用户首次登录之前。

主要判断表

看到的证据	能证明什么	仍不能证明什么	下一条高分度证据
<code>.container</code> 文件存在	源声明文件已放置	generator 已识别、字段正确	<code>daemon-reload</code> 后检查 <code>LoadState</code> 和 user journal
<code>report.service</code> 可加载	generated unit 已出现	unit 能启动、实例参数正确	<code>systemctl --user show</code> 与 journal
<code>ActiveState=active</code>	manager 当前认为服务活动	应用响应、数据挂载、冷启动成功	<code>podman inspect</code> 、协议请求、数据读取
<code>podman ps</code> 显示运行	容器进程存在	对外功能正确、数据独立	<code>podman port</code> 、 <code>curl</code> 、宿主与容器内文件对照
<code>Linger=yes</code>	user manager 可脱离登录会话存在	Quadlet 会启动、业务可用	注销后和冷启动未登录的外部功能测试
restart 后文件仍在	当前重启未丢失文件	删除和重建后仍持久	删除实例并由声明重建，再读取外部数据
journal 无新错误	当前查询范围内未见失败记录	业务终态完全正确	协议、内容和持久数据验收

向下一章交接

本章已经把单个容器转换为可维护、可重建、可诊断的用户级服务。进入下一章“RHCSA 综合任务与证据矩阵”时，不再重复 Quadlet 字段，而是把这里的证据链与用户、网络、存储、SELinux、防火墙和重启验收组合成整机终态。

静态可信边界：本章依据 RH134、RHEL 9 容器文档、Podman 4.6.1 Quadlet 手册和 systemd 手册进行静态核对。当前会话没有可控 RHEL 9 VM，未执行真实注销、冷启动、端口请求、实例删除与重建测试；这些项目保留在 `revision-notes.md` 的 live-test 清单中。

RHEL 9 · RHCSA 实操讲义

RHCSA 综合任务与证据矩阵

把跨用户、网络、安全、存储、调度与容器的要求，组织成可恢复、可验证、能够通过重启门的完整终态。

对象模型风险排序分层验收诊断恢复综合任务

大字号阅读版

本章阅读导航

先抓住一条主线：综合任务不是命令清单，而是把初始状态经过受控变更推进到一组互相一致、能够被证据逐项证明的终态。

- 01拆解终态把题目改写为对象、参数、限制和证据
- 02建立依赖识别硬依赖、共享对象和冲突域
- 03调查基线确认环境事实，不猜接口、设备和已有状态
- 04风险分批低风险先闭环，高风险操作通过证据门
- 05分层验收分别证明静态、当前、功能和持久状态
- 06重启与恢复共享对象回归，重启后按矩阵重新验收

专题地图

- *知识专题* 从题目原文提取评分对象与可验证终态
- *知识专题* 构造依赖图、共享对象和冲突域
- *知识专题* 风险排序、证据门、恢复点与执行批次
- *操作专题* 建立整机调查基线和任务账本
- *操作专题* 建立四层证据矩阵
- *操作专题* 用局部闭环和回归集合推进任务
- *诊断专题* 卡题、误操作和交叉影响的证据式恢复
- *经典任务* 五类领域任务与一次模拟考试级终检

阅读时持续回答

1. 题目真正评分的是哪个对象和状态？
2. 哪些参数来自题目，哪些环境事实必须查询？
3. 哪些任务存在硬依赖或共享对象？
4. 下一步操作的风险等级和停止条件是什么？
5. 当前证据能证明哪一层，又不能证明哪一层？
6. 修改后最小功能验证是什么？
7. 重启前还缺哪些静态检查与恢复入口？
8. 重启后应从哪张矩阵逐项复查？

方法提醒：先从原题建立评分行，再按依赖和风险推进；每次变更都要留下能够说明证明范围的证据。

第 33 章 · 正文

RHCSA 综合任务最容易制造一种错觉：题目很多、命令很多，于是只要按记忆把命令逐条敲完，整套系统就会自然到达正确终态。真实情况恰好相反。用户、网络、仓库、服务、端口、SELinux、存储、挂载、计划任务和容器分别属于不同对象层；它们又通过用户身份、路径、设备、端口、名称解析和启动生命周期互相连接。一个局部命令成功，可能只说明当前动作没有报错，并不能证明最终功能正确，更不能证明系统重启后仍然成立。

本章不重讲前 32 章的子系统基础，而是把已经建立的对象组合成一套稳定的综合执行方法。核心不是背一份固定考试顺序，而是把每道题转换成可验证终态，识别依赖与共享对象，控制高风险操作，并用证据矩阵逐项关闭缺口。各技术机制在需要时回链对应章节；RHCE 自动化实现不在本章展开。

概念

综合任务 是多个系统对象和限制条件共同构成的终态问题。它的评分对象不是使用过哪些命令，而是用户、路径、连接、服务、设备、挂载、调度或容器最终是否满足题目。命令只是查询或改变对象的接口，因此“命令执行成功”不能替代对终态的验收。

概念

依赖关系与共享对象 决定执行顺序和回归范围。硬依赖不成立时，后续功能无法建立或验证；共享对象则会被多个任务共同使用，例如一个用户同时关联 ACL、cron 和 rootless 容器。修改共享对象后，只回归相关任务，但不能假设先前通过的状态仍然成立。

概念

风险排序与证据门 用于控制不可逆或高影响操作。网络切换、分区、文件系统签名、fstab 和重启都可能扩大故障面；进入这些步骤前，必须确认对象身份、保持条件、停止条件和恢复入口。证据不足时应标记为 blocked，而不是用 force、删除重建或全局放宽安全策略换取短暂成功。

概念

调查基线 是变更前对关键对象身份和状态的最小快照。它不是把所有查询命令运行一遍，而是只收集会改变决策的字段：实际 profile、设备签名、已有挂载、用户关系、活动 zone、配置来源和消费者身份。没有基线，就无法判断变化是否由本次操作引起，也难以进行最小恢复。

概念

四层证据 分别回答不同问题：静态证据证明声明或语法可解析；当前证据证明对象此刻已经加载或生效；功能证据从真实消费者角度证明可用；持久或重启证据证明生命周期跨越后仍成立。题目要求哪一层，就必须给出能够证明那一层的证据，不能用 active、文件存在或退出码互相替代。

概念

验收矩阵 把原始要求、目标对象、依赖、风险、四层证据和剩余缺口放在同一记录中。它的价值不是表格本身，而是让每个 PASS 都能追溯到具体证据，并让 pending、blocked 与 deferred 明确说明仍缺什么。

概念

恢复路径 是失败前预先保留的返回入口和判断顺序。文本配置可以依靠保留属性与上下文的副本恢复；网络需要控制台或第二会话；启动故障需要明确进入救援路径；存储写入则不能靠复制配置文件撤销。诊断应从最后一条可信证据开始，选择最有区分度的下一步，而不是反复执行失败命令。

操作语义

本章不以某一个命令为中心。下面的入口把“任务拆解、对象调查、分层验收、高风险证据门、最终重启和错误恢复”组织成可重复的操作接口；详细命令仍在对应专题与经典任务中按对象展开。

任务拆解

SYNOPSIS

原始要求 → 对象 → 精确参数 → 限制 → 必需证据

把自然语言题面转换成可逐项判定的评分行。

对象

用户、路径、连接、unit、端口、设备、卷、挂载、调度或容器。

参数

题目明确给出的 UID、地址、端口、容量、路径和时间。

限制

不得删除数据、不得降低 SELinux、必须保留已有对象等。

证据

写明需要静态、当前、功能、持久或重启中的哪些层。

基线查询组

SYNOPSIS

getent | id | stat | getfacl | nmcli | lsblk | findmnt | systemctl

确认对象身份、配置来源和初始状态；只查询会改变下一步决策的字段。

身份与权限

getent、id、namei、getfacl、sudo -l。

网络与服务

nmcli、ip、getent hosts、systemctl、ss。

存储

lsblk、blkid、pvs/vgs/lvs、findmnt、swapon。

容器与调度

podman、systemctl --user、loginctl、journalctl。

执行顺序规划

SYNOPSIS

硬依赖 → 低风险前置 → 局部闭环 → 共享对象回归

依赖决定能否继续，风险决定需要多少前置证据，共享对象决定修改后的回归集合。

硬依赖

网络和名称解析不成立时，仓库、NFS 与镜像访问无法可靠验收。

低风险前置

先完成只读调查和不会破坏现状的对象创建。

局部闭环

一次只推进一个状态层，修改后立即做最小验证。

回归集合

用户、路径、端口、设备或 profile 变化后复查所有消费者。

分层验收

SYNOPSIS

VERIFY {static | current | functional | persistent}

每条证据只承担一个证明目标，并明确不能证明的相邻状态。

static	配置、语法、声明和稳定标识。
current	此刻加载、运行、监听、挂载或激活。
functional	目标用户、客户端或数据消费者实际完成动作。
persistent	重新登录、reload、退出会话或重启后仍成立。

高风险证据门

SYNOPSIS

IDENTITY + KEEP + STOP + RECOVERY → WRITE

在网络切换、分区、mkfs、fstab 和重启前，先证明对象身份与保持条件，并准备停止点和恢复入口。

IDENTITY

确认真实接口、profile、设备、签名、挂载和使用者。

KEEP

明确不得删除、覆盖或降低保护的状态。

STOP

证据不完整或输出矛盾时停止写入。

RECOVERY

准备第二会话、控制台、配置副本或启动恢复路径。

重启与恢复决策

SYNOPSIS

静态检查通过 + 当前功能闭环 + 恢复入口可用 → reboot

重启用于验证持久链，不用于猜故障；失败后从最后可信层开始诊断。

重启前

检查网络自动连接、fstab、关键服务、用户 unit 和剩余 blocked 项。

重启后

按原始题目和验收矩阵逐项复查，不凭命令历史判断。

失败恢复

症状 → 当前证据 → 假设 → 高区分度证据 → 最小修复 → 再验证。

[知识专题] 从题目原文提取评分对象与可验证终态

综合题的第一步不是打开终端，而是把自然语言要求转换成一组可以逐项判定的状态。题目常使用“配置”“确保”“允许”“创建”“运行”等动词；这些动词没有直接告诉你应该用哪条命令，也没有说明一条命令成功后还需要哪些证据。稳定切入点是先找对象，再补齐参数、限制和生命周期。

① [知识点] 模糊动词必须改写为“对象 + 参数 + 终态”

例如“配置 Web 服务”至少可能包含：软件存在、配置语法正确、服务当前运行、开机自动启动、监听指定端口、SELinux 允许、firewalld 放行、内容可读取、远端客户端能访问。题面没有要求的层不要擅自扩展，但题面明确要求的层不能被一句“服务已启动”替代。

建议把每项要求改写为：

原始要求
→ 目标对象
→ 精确参数
→ 当前终态
→ 持久终态
→ 功能验收
→ 禁止事项或保持条件

② [知识点] 一道题可能包含多个独立评分行

“创建用户 deploy，使其属于 opsadmins，并能在 /srv/project 中协作”至少涉及三个对象：账号、附加组关系、目录访问。若再要求新文件自动继承组和审计用户只读，则还涉及 setgid 与默认 ACL。把所有内容写在一行会导致某个子状态失败时无法定位缺口。

任务账本应把它们拆成独立行，同时保留原题编号，避免最后检查时只看到“用户题已做”。

③ [知识点] 正向要求、限制条件和保持现状同样重要

下面三类内容都必须进入终态模型：

- 正向状态：服务监听 `8082/tcp`；
- 限制条件：SELinux 保持 Enforcing；
- 保持条件：不得删除 `/srv/project` 中现有文件。

只记录正向状态会诱导出高风险捷径，例如清空目录重建、删除现有网络连接、关闭 SELinux 或重新格式化不明设备。保持条件应被视为验收项，而不是旁注。

④ [知识点] 命令不是评分对象，也不能代替证据

执行过 `systemctl enable --now httpd` 只能证明命令被调用；它不能单独证明配置语法正确、进程没有立即退出、监听端口正确、远端访问成功。命令历史最多是变更记录，真正的评分证据来自对象状态。

⑤ [知识点] 将题目参数与环境事实分开

题目给出的 IP、端口、大小和路径属于目标参数；接口名、现有连接 profile、设备签名、UUID 和当前 zone 属于环境事实，必须查询，不能猜测。即使训练题给出 `/dev/vdb`，正式操作前仍应通过 `lsblk`、`blkid` 和挂载关系确认它没有被其他对象使用。

[Cheatsheet] 先写对象，再写参数；把一道大题拆成独立评分行；限制条件进入矩阵；命令历史不是终态证据；题目参数可以照抄，环境事实必须查询。

[知识专题] 构造依赖图、共享对象和冲突域

综合任务不是一条固定线性流水线。有些要求存在硬依赖，有些只是为了降低返工而适合提前完成，还有一些彼此独立，可以按风险和耗时自由安排。依赖图的目标不是画得复杂，而是避免在前置条件尚未成立时误判后续故障。

① [知识点] 区分硬依赖、顺序优化和独立任务

硬依赖表示前置对象不成立，后续对象无法正确完成。例如仓库不可用时不能可靠安装缺失软件；NFS 名称无法解析时 autofs 功能验收无法完成。

顺序优化表示后续任务并非绝对不能做，但先完成前置工作可以减少回归。例如先完成用户和组，再设置目录 ACL，可以避免后续修改所有权时重新检查访问。

独立任务不应被人为串成一条长链。若本地日志查询与未使用该日志的 LVM 创建没有共享对象，两者可以独立闭环。

② [知识点] 共享对象决定回归范围

典型共享对象包括：

- 用户与组：ACL、sudo、计划任务、rootless 容器；
- 路径：挂载、SELinux 标签、服务 DocumentRoot、容器卷；
- 端口：应用配置、监听、SELinux 端口类型、firewalld；
- 设备：分区、PV、VG、文件系统、Swap、fstab；
- NetworkManager profile：地址、路由、DNS、仓库、NFS、远程访问；
- 时间：日志时间线、cron 触发、证据关联。

修改共享对象后不需要把整台主机从头测试，但必须重新执行与它相关的回归集合。

③ [知识点] 跨层服务访问链必须逐层证明

一个非标准端口的 HTTP 服务通常经过：

```
配置文件
→ 配置语法
→ 服务进程
→ 监听套接字
→ SELinux 文件与端口策略
→ firewalld
→ 本机请求
→ 远端请求
```

任何一层失败都可能表现为“访问不了”。直接反复重启服务无法区分这些原因。

④ [知识点] 存储路径会覆盖已有目录视图

把文件系统挂载到一个已有内容的目录后，原目录内容不会被删除，但会被挂载对象遮蔽。若服务或容器已经使用该路径，挂载顺序和数据迁移必须先明确。综合任务中创建挂载点前，应检查 `findmnt`、目录内容和使用者；不能只确认目录是否存在。

⑤ [知识点] 用户级 systemd 依赖登录生命周期

rootless 容器、用户 Quadlet 和 `systemctl --user` 不只依赖容器定义，还依赖目标用户、用户 manager、运行时目录和 linger。用 root 上下文运行 `podman` 成功不能证明目标用户的容器会在退出登录后保持。

[Cheatsheet] 硬依赖决定能不能做；顺序优化决定是否容易返工；共享对象决定回归范围；服务访问按配置、进程、端口、安全和客户端逐层证明。

[知识专题] 风险排序、证据门、恢复点与执行批次

综合任务的稳定顺序通常不是按题号，也不是把最熟悉的题先做完。更可靠的方法是先完成低风险、强前置和能改善调查能力的对象；进入网络切换、分区、格式化、fstab 和重启等高影响动作前设置证据门。

① [知识点] 风险按影响面和可恢复性分级

可使用以下简化等级：

等级	典型操作	主要策略
R0 只读	<code>getent</code> 、 <code>lsblk</code> 、 <code>nmcli show</code>	尽早执行并保存基线
R1 低风险	创建用户、写新 <code>repo</code> 文件	操作后立即局部验证
R2 可中断服务	修改服务配置、重启 <code>daemon</code>	先做语法检查与恢复副本
R3 连接或共享影响	切换网络 <code>profile</code> 、修改防火墙	保留控制台或第二会话
R4 数据/启动关键	分区、 <code>mkfs</code> 、 <code>fstab</code> 、重启	必须通过证据门和恢复入口

风险等级不是禁止操作，而是决定需要多少前置证据和回退准备。

② [知识点] 高风险写操作需要证据门

分区前的最低证据门包括：目标设备身份、容量、当前挂载、文件系统签名、是否属于 LVM/Swap。fstab 写入后的最低证据门包括：语法验证、目标存在、手工挂载成功、重复项检查。未通过证据门时应标记为 `blocked`，而不是继续尝试破坏性参数。

③ [知识点] 网络修改必须保留可达性恢复路径

通过 SSH 修改当前连接时，应优先保留控制台或第二会话，并避免先删除正在工作的 `profile`。可以创建或修改候选 `profile`，核对全部属性后再激活；新连接验证成功后，再处理旧 `profile` 的自动连接优先级。

④ [知识点] 配置副本不是万能回滚

对文本配置，保留带权限和上下文的副本有助于恢复；对 LVM、文件系统和分区，复制文本配置不能恢复数据结构。高风险存储操作的真正安全边界是确认对象身份、避免误写，以及在写入前停止操作。

⑤ [知识点] 重启是最终验收门，不是首轮排错工具

重启会同时改变服务、挂载、网络、用户 `manager` 和容器状态，使故障面扩大。只有启动关键配置已经通过静态检查，当前功能已闭环，恢复入口可用时，才进入重启。重启后按既有矩阵复查，不随机重做全部任务。

[Cheatsheet] 先只读，再低风险；网络保留第二入口；分区和 `fstab` 有证据门；文本副本不能撤销存储写入；重启用于证明持久性，不用于猜故障。

[操作专题] 建立整机调查基线和任务账本

调查基线的目的不是收集所有可能信息，而是为当前题目提供对象身份、初始状态和冲突证据。命令应围绕题目涉及的对象选择，输出应保存到可回看的工作记录中。下面的宽基线是候选集合，不要求每场考试机械执行全部命令。

① [查询] 建立主机、时间与启动基线

作用对象：主机身份、当前启动、时间可信度和默认 `target`。

```
hostnamectl
uname -r
uptime
who -b
timedatectl
systemctl get-default
systemctl --failed
journalctl -b -p warning..alert --no-pager
```

`systemctl --failed` 只能提示失败 `unit`，不能证明所有业务正常。日志时间线若依赖错误时钟，也不能直接用于判断先后顺序。

② [查询] 建立身份、权限和路径基线

```
getent passwd deploy auditor svcweb
getent group opsadmins
id deploy
sudo -l -U deploy
stat -c '%n %U:%G %a %A' /srv/project
namei -om /srv/project
getfacl -p /srv/project
findmnt -T /srv/project
```

`namei -om` 用于逐级观察路径权限；`findmnt -T` 用于确认路径实际位于哪个挂载对象。只看末级目录权限可能遗漏父目录不可遍历或被挂载覆盖。

③ [查询] 建立网络、仓库、服务与安全基线

```
nmcli -f NAME,UUID,TYPE,DEVICE,AUTOCONNECT connection show
nmcli -f GENERAL,IP4,IP6 device show
ip -brief address
ip route
getent hosts repo.lab.example.com nfs.lab.example.com

dnf repolist -v
systemctl status httpd --no-pager
ss -ltnup
firewall-cmd --get-active-zones
firewall-cmd --list-all
firewall-cmd --permanent --list-all
getenforce
semanage port -l | grep -E '^http_port_t'
```

firewalld 的 runtime 与 permanent 是两个状态面；`ss` 证明本机监听，不证明防火墙和远端路径。

④ [查询] 建立存储与挂载基线

```
lsblk -e7 -o NAME,PATH,SIZE,TYPE,FSTYPE,FSVER,LABEL,UUID,MOUNTPOINTS
blkid
pvs
vgs
lvs -a -o lv_name,vg_name,lv_size,lv_attr,devices
findmnt --real
swapon --show
cat /etc/fstab
```

目标设备必须同时结合 `lsblk`、`blkid`、LVM 归属和挂载关系判断。设备名存在不代表它空闲。

⑤ [查询] 建立调度、远程文件系统和容器基线

```
systemctl status autofs crond chronyd --no-pager
automount -m
chronyc sources -v
chronyc tracking
ls -l /etc/cron.d

logintctl show-user svcweb -p Linger -p State
# 以下命令必须在 svcweb 的真实用户会话中执行
podman info
podman ps -a
systemctl --user --failed
```

root 运行的 `podman ps` 与 `svcweb` 的容器存储完全不同。调查 rootless 对象时必须明确用户上下文。

⑥ [操作] 建立任务账本

建议维护如下字段：

U-01 · 创建 deploy

对象：用户；初始状态：不存在；依赖：无；风险：R1。

下一动作：创建账号；缺失证据：身份记录与新登录会话；状态：`planned`。

W-03 · 远端访问 8082

对象：完整服务访问链；初始状态：未知；依赖：网络、httpd、SELinux 与 firewalld；风险：R2。

下一动作：先确认监听；缺失证据：远端功能；状态：`blocked`。

状态可以使用 `planned`、`in_progress`、`verified`、`blocked`、`deferred`。不要把“执行过”当成 `verified`。

[Cheatsheet] 基线只围绕题目对象；身份、路径、网络、存储和用户上下文分开记录；任务账本记录缺失证据而不是命令次数。

[操作专题] 建立静态、当前、功能和持久证据矩阵

证据矩阵把“我做过什么”改写为“系统现在能够证明什么”。每项要求不必机械填满四层，但必须覆盖题目要求的生命周期。证据命令应尽量短、可重复、结果具有唯一判断边界。

① [验证] 静态配置证据

静态证据回答“声明是否正确”。典型入口包括：

```
visudo -cf /etc/sudoers
httpd -t
findmnt --verify --verbose
systemd-analyze verify /path/to/unit.service
podman quadlet --dryrun # 仅在当前版本提供该入口时使用
```

还包括读取文件的精确字段，例如 `nmcli connection show exam-static`、`grep -R` 检查 `repo` 或 `cron` 配置。静态通过不表示对象已经加载。

② [验证] 当前状态证据

当前证据回答“此刻是否生效”：

```
systemctl is-active httpd
systemctl is-enabled httpd
ss -lnt '( sport = :8082 )'
findmnt /srv/data
swapon --show
firewall-cmd --query-port=8082/tcp
getenforce
```

`is-active` 与 `is-enabled` 分属当前和持久配置，不得互相替代。

③ [验证] 功能证据

功能证据必须从实际消费者视角执行：

- 目标用户创建和读取协作文件；
- 从另一台主机请求 HTTP；
- 对新文件验证 ACL 继承；
- 访问 `autofs` 键触发远程挂载；
- 修改容器持久数据并重建容器后读取；
- 在精简环境下执行计划脚本并核对产物和日志。

功能验证应避免只由 `root` 执行，因为 `root` 可能绕过题目真正要求的访问控制。

④ [验证] 持久与重启证据

持久证据包括配置层和生命周期层：

```
nmcli -g connection.autoconnect connection show exam-static
systemctl is-enabled httpd
firewall-cmd --permanent --query-port=8082/tcp
findmnt --verify --verbose
loginctl show-user svcweb -p Linger
```

最终持久性通常仍需重启或重新登录验证。当前会话没有 live VM，因此本章只能给出推荐验证命令，不声称已执行。

⑤ [边界] 为证据写明证明边界

示例：

证据	能证明	不能证明
<code>httpd -t</code>	Apache 配置语法可解析	服务已运行、端口可访问
<code>ss -lnt</code>	本机进程监听端口	firewalld、SELinux、远端路由
<code>findmnt /srv/data</code>	当前已挂载	<code>/etc/fstab</code> 正确、重启后仍挂载
<code>podman ps</code>	当前用户容器运行	退出登录或重启后保持

证据	能证明	不能证明
<code>crontab -l</code>	调度条目存在	到点后成功执行

⑥ [验证] 定义完成状态

一项任务只有在所有必需证据层都关闭后才标记 `verified`。由于环境不可达而无法验证远端功能时，应标记 `blocked` 或 `deferred`，并记录缺失证据；不能把静态检查通过升级为完整成功。

[Cheatsheet] 静态看声明，当前看此刻，功能看消费者，持久看生命周期；每条证据都写清不能证明什么；缺证据就不写 PASS。

[操作专题] 用局部闭环、检查点和回归集合推进任务

综合任务的执行单位应是一个可验证的小闭环，而不是一口气修改多个子系统后再统一排错。局部闭环能把故障限制在最近的一组变更中，也便于记录恢复点。

① [操作] 每次只推进一个明确状态层

以 HTTP 服务为例：先让配置语法正确，再让服务运行，再确认监听，再处理 SELinux 和 firewalld，最后做客户端请求。若在配置语法尚未通过时同时修改防火墙，后续出现失败就会增加无关假设。

② [操作] 修改后立即运行最小验收

最小验收应贴近变更对象：

- 修改 `sudoers` 后立即 `visudo -cf`；
- 修改 `httpd` 后立即 `httpd -t`；
- 写入 `fstab` 后立即 `findmnt --verify`；
- 写入 `autofs` map 后先 `automount -m`；
- 写入 `Quadlet` 后重新加载用户 `manager` 并查询生成 `unit`。

③ [操作] 为共享对象建立回归集合

示例：修改 `/srv/web` 的挂载或标签后，回归集合至少包括：

```
findmnt -T /srv/web
→ stat/ls -Z
→ httpd 配置和进程
→ 本机请求
→ 远端请求
```

修改 `svcweb` 用户或 `home` 后，回归用户 `systemd`、容器存储和数据路径。

④ [操作] 保存检查点和差异

文本配置可保存权限和上下文：

```
cp -a /etc/httpd/conf.d/rhcsa33.conf \
/root/rhcsa33-backup/httpd-rhcsa33.conf.before
```

更重要的是记录变更前后的关键查询，例如 `nmcli connection show`、`semanage port -l`、`findmnt`。检查点不是为了堆积副本，而是为了明确最后一个可信状态。

⑤ [操作] 临时验证与最终持久配置分开

临时挂载、runtime 防火墙规则、手工启动容器可以用于快速验证，但必须在账本中标记为当前层证据；随后还要完成 `fstab`、permanent `firewalld` 或 `Quadlet` 等持久状态。临时成功若没有迁移到持久层，不能关账。

[Cheatsheet] 一次改一个层；修改后立即最小验收；共享对象变化触发回归；临时成功和持久配置分开记录。

[诊断专题] 卡题、误操作和交叉影响的证据式恢复

卡题时最危险的反应是扩大修改范围：服务不通就同时改配置、防火墙和 SELinux；挂载失败就重新格式化；网络不通就删除全部 `profile`。正确做法是固定症状和最后证据，明确哪一层已经证实，再选一条能区分假设的证据。

① [诊断] 保存症状和最后一条可信证据

记录具体对象、命令、错误信息和时间。不要只写“网络坏了”或“容器起不来”。例如：

症状: 远端 `curl 192.0.2.20:8082` 超时
已证实: `httpd -t` 通过; `ss` 显示 `0.0.0.0:8082` 监听
未证实: `active zone`、`runtime/permanent` 端口、远端路由

② [诊断] 把假设限制在失败层

若本机 `curl` 成功而远端失败, 应用内容和本机监听的优先级下降, 下一步应看 `active zone`、`firewalld`、网络路径。若本机也失败, 则先看服务、监听和 SELinux, 不应直接调整远端防火墙。

③ [诊断] 选择最有区分度的下一条证据

高区分度证据应能一次排除多个假设。例如 `findmnt -T /srv/web` 能确认路径是否被其他文件系统覆盖; `ausearch -m AVC -ts recent` 能确认最近是否存在 SELinux 拒绝; `journalctl -u web33.service` 能把用户 `unit` 和容器运行错误关联起来。

④ [诊断] 只修复被证据支持的最小对象

看到 `AVC` 不等于关闭 SELinux; 应确定目标文件类型、端口类型或 Boolean。看到 `fstab` 失败不等于注释全部条目; 应定位具体行、设备标识和挂载点。

⑤ [诊断] 恢复后从失败层重新验证

修复 `firewalld` 后不必重新创建用户和 LVM, 但应重新执行监听、`runtime/permanent` 查询和远端请求。恢复路径仍然遵守证据矩阵。

⑥ [诊断] 六个高频误判分支

症状	常见误判	下一条证据
新组关系不生效	<code>usermod</code> 失败	新登录会话中的 <code>id</code>
服务 <code>active</code> 但访问失败	重启次数不够	<code>ss</code> 、SELinux、 <code>firewalld</code> 、客户端
<code>lvs</code> 已增大但容量未变	LV 扩容失败	<code>findmnt</code> 、文件系统类型和 <code>df</code>
<code>autofs</code> 路径未出现在 <code>findmnt</code>	<code>autofs</code> 失败	先访问具体键, 再查 <code>findmnt</code>
<code>cron</code> 脚本手工成功	调度已经正确	精简环境、身份、绝对路径和日志
<code>root</code> 的容器正常	<code>rootless</code> 任务完成	目标用户的 <code>podman</code> 与用户 <code>manager</code>

[Cheatsheet] 固定症状; 区分已证实和未证实层; 选高区分度证据; 只修最小对象; 从失败层重新验收。

[经典任务] 任务一：项目身份、协作目录与最小 `sudo`

环境与初始状态

- `opsadmins` 组和 `deploy` 用户尚不存在;
- `auditor` 用户已经存在;
- `/srv/project` 已存在, 包含不可删除的现有文件和子目录;
- `httpd.service` 由另一任务维护;
- 本任务给出推荐操作与验收入口; 实际结果需在 RHEL 9 环境填写。

目标终态

- 创建组 `opsadmins`, GID 为 `3100`;
- 创建用户 `deploy`, UID 为 `2201`, 主组保持默认私有组, 附加组为 `opsadmins`;
- `/srv/project` 归属 `root:opsadmins`, 目录模式为 `2770`;
- 组成员能够在目录中协作, 新建对象继承组;
- `auditor` 对现有和未来对象具有读取文件、遍历目录的能力, 但不能写入;
- `deploy` 只能通过 `sudo` 执行 `/usr/bin/systemctl restart httpd.service`;
- 不得删除或改写已有文件内容, 不得授予通用 `root` Shell。

限制与验收

- 必须使用新登录会话验证附加组;
- 必须验证现有对象和新对象的 ACL;
- 必须使用 `visudo` 语法检查;
- 不能使用 `chmod 777`。

[参考解答] 任务一：身份、ACL 与 sudo 的证据闭环

① 调查与拆解

```
getent group opsadmins
getent passwd deploy auditor
stat -c '%n %U:%G %a %A' /srv/project
namei -om /srv/project
getfacl -R -p /srv/project
sudo -l -U deploy 2>/dev/null || true
```

若 `deploy` 或 `GID/UID` 已被占用，应停止并根据题目确认是否修改现有对象，不能重复创建或强制覆盖。

② 创建身份对象

```
groupadd -g 3100 opsadmins
useradd -u 2201 -G opsadmins deploy
getent group opsadmins
id deploy
```

`id deploy` 读取账号数据库，但当前已经打开的 `deploy` 会话不会自动刷新附加组。最终功能验证必须使用新的登录会话。

③ 设置协作目录的传统权限

```
chown root:opsadmins /srv/project
chmod 2770 /srv/project
stat -c '%n %U:%G %a %A' /srv/project
```

`setgid` 位使新建对象继承目录组；它不自动给既有子目录添加组写权限，也不能代替 ACL。

④ 为 auditor 设置现有与未来访问

对现有目录授予 `r-x`，对现有普通文件授予 `r--`：

```
find /srv/project -type d -exec setfacl -m u:auditor:r-x {} +
find /srv/project -type f -exec setfacl -m u:auditor:r-- {} +
```

对所有现有目录设置默认 ACL，使未来子对象继续继承：

```
find /srv/project -type d -exec setfacl -m d:u:auditor:r-x {} +
getfacl -p /srv/project
```

默认 ACL 仍受到创建程序请求模式和 ACL mask 影响。验证时应实际创建文件，而不是只读取默认条目。

⑤ 配置精确 sudo 规则

使用独立文件：

```
visudo -f /etc/sudoers.d/deploy-httpd
```

内容：

```
deploy ALL=(root) /usr/bin/systemctl restart httpd.service
```

校验：

```
chmod 0440 /etc/sudoers.d/deploy-httpd
visudo -cf /etc/sudoers
sudo -l -U deploy
```

不要加入通配符或 `/bin/sh`。精确命令行只允许目标 `service` 的 `restart`。

⑥ 以真实消费者身份验证

打开新的 `deploy` 登录会话后：

```
id
umask
touch /srv/project/deploy-test
stat -c '%n %U:%G %a %A' /srv/project/deploy-test
getfacl -p /srv/project/deploy-test
sudo /usr/bin/systemctl restart httpd.service
sudo -l
```

以 auditor 验证读取和拒绝写入：

```
runuser -u auditor -- cat /srv/project/deploy-test
runuser -u auditor -- sh -c 'echo denied >> /srv/project/deploy-test'
```

第二条预期被拒绝：不要在文档中伪造具体错误输出。

⑦ 证据矩阵

要求	静态/身份	当前	功能	持久
deploy 身份	getent、id deploy	新会话 id	能以组身份创建文件	账号数据库持久
协作目录	stat、getfacl	当前权限有效	deploy 创建对象并继承组	文件系统权限持久
auditor 只读	ACL 条目与 mask	当前访问	读取成功、写入被拒绝	默认 ACL 作用于新对象
最小 sudo	visudo -cf、sudo -l -U	规则已加载	指定命令成功、其他命令被拒绝	sudoers 文件持久

典型错误

- 只执行 usermod -aG 后在旧会话中判断失败；
- 只给顶层目录 ACL，忽略已有子目录；
- 设置默认 ACL 后不创建新对象验证；
- sudoers 使用过宽通配符；
- 通过 root 读取文件代替 auditor 功能验证。

[经典任务] 任务二：静态网络、仓库与非标准 HTTP 服务

环境与初始状态

- 目标接口为 enp1s0；当前 DHCP profile 为 Wired connection 1；
- 通过控制台操作，允许保留旧 profile 作为临时恢复入口；
- repo.lab.example.com 和 client.lab.example.com 位于训练网络；
- /srv/web/index.html 已存在且内容不得改写；
- SELinux 必须保持 Enforcing。

目标终态

- 主机名：servera.lab.example.com；
- profile：exam-static；
- IPv4：192.0.2.20/24；网关 192.0.2.1；DNS 192.0.2.53；
- BaseOS：http://repo.lab.example.com/rhel9/BaseOS；
- AppStream：http://repo.lab.example.com/rhel9/AppStream；
- httpd 使用 /srv/web，监听 8082/tcp；
- 服务当前运行且开机启动；
- SELinux 文件与端口策略正确；
- firewalld runtime 与 permanent 均允许 8082/tcp；
- client.lab.example.com 能读取首页。

[参考解答] 任务二：从网络 profile 到远端 HTTP 的完整访问链

① 调查并保留恢复入口

```
hostnamectl
nmcli -f NAME,UUID,TYPE,DEVICE,AUTOCONNECT connection show
nmcli device status
nmcli connection show 'Wired connection 1'
ip route
```

不要先删除当前 profile。确认控制台可用，并记录旧连接属性。

② 创建并核对静态 profile

```
nmcli connection add type ethernet ifname enp1s0 con-name exam-static \
  ipv4.method manual \
  ipv4.addresses 192.0.2.20/24 \
  ipv4.gateway 192.0.2.1 \
  ipv4.dns 192.0.2.53 \
  connection.autoconnect yes

nmcli connection show exam-static
hostnamectl set-hostname servera.lab.example.com
```

确认参数后激活：

```
nmcli connection up exam-static
ip -brief address show enp1s0
ip route
getent hosts repo.lab.example.com
```

新连接确认可用后，可避免旧 profile 抢占自动连接：

```
nmcli connection modify 'Wired connection 1' connection.autoconnect no
```

不需要为了完成任务删除旧 profile。

③ 配置并验证仓库

创建 /etc/yum.repos.d/rhcsa33.repo：

```
[rhel9-baseos]
name=RHEL 9 BaseOS training repository
baseurl=http://repo.lab.example.com/rhel9/BaseOS
enabled=1
gpgcheck=0

[rhel9-appstream]
name=RHEL 9 AppStream training repository
baseurl=http://repo.lab.example.com/rhel9/AppStream
enabled=1
gpgcheck=0
```

`gpgcheck=0` 只因为本训练任务明确提供无签名内网源；真实环境应遵循仓库签名策略。

```
dnf clean all
dnf repolist -v
dnf makecache --disablerepo='*' \
  --enablerepo=rhel9-baseos,rhel9-appstream
```

④ 安装工具并写入 httpd 配置

```
dnf install -y httpd policycoreutils-python-utils firewallld
```

创建 /etc/httpd/conf.d/rhcsa33.conf：

```
Listen 8082
<VirtualHost *:8082>
    DocumentRoot "/srv/web"
    <Directory "/srv/web">
        Require all granted
    </Directory>
</VirtualHost>
```

先验证静态配置：

```
httpd -t
```

⑤ 建立 SELinux 持久规则

先查询是否已有针对该路径的持久规则：

```
semanage fcontext -l | grep '/srv/web' || true
```

若没有匹配规则，再添加；若已有规则但类型错误，应在确认现有用途后使用 `-m` 修改：

```
semanage fcontext -a -t httpd_sys_content_t '/srv/web(/.*)?'
restorecon -RFv /srv/web
matchpathcon /srv/web /srv/web/index.html
ls -Zd /srv/web /srv/web/index.html
```

检查端口现状：

```
semanage port -l | grep '^http_port_t'
```

若 `8082/tcp` 尚未属于任何类型：

```
semanage port -a -t http_port_t -p tcp 8082
```

若端口已存在但类型错误，应在确认现有用途后使用 `-m` 修改；不要不经调查重复 `-a`。

⑥ 启动服务并检查监听

```
systemctl enable --now httpd.service
systemctl is-active httpd.service
systemctl is-enabled httpd.service
ss -lnt '( sport = :8082 )'
curl --fail http://127.0.0.1:8082/
```

本机请求通过后再处理外部访问层。

⑦ 配置 firewalld 双态

```
systemctl enable --now firewalld.service
firewall-cmd --get-active-zones
ZONE=$(firewall-cmd --get-zone-of-interface=enp1s0)
printf 'active zone for enp1s0: %s\n' "$ZONE"
```

若输出为空或为 `no zone`，应先调查接口归属，不要把规则写入猜测的默认 `zone`。确认后在同一 `zone` 写入并查询 `runtime/permanent`：

```
firewall-cmd --zone="$ZONE" --add-port=8082/tcp
firewall-cmd --permanent --zone="$ZONE" --add-port=8082/tcp
firewall-cmd --zone="$ZONE" --query-port=8082/tcp
firewall-cmd --permanent --zone="$ZONE" --query-port=8082/tcp
```

⑧ 从远端消费者验证

在 `client.lab.example.com`：

```
curl --fail http://192.0.2.20:8082/
```

若本机成功而远端失败，优先调查 `active zone`、网络路径与远端路由，不要继续修改 `DocumentRoot`。

⑨ 证据矩阵

对象	静态	当前	功能	持久
网络	<code>nmcli con show exam-static</code>	地址、路由、DNS	仓库/NFS 名称可解析	autoconnect、重启后连接
仓库	repo 文件、 <code>repolist -v</code>	元数据可刷新	可查询或安装题目软件	repo 文件持久
httpd	<code>httpd -t</code>	active、监听 8082	本机与远端 curl	enabled
SELinux	fcontext/port 规则	Enforcing、标签生效	无策略拒绝且请求成功	<code>semanage</code> 规则持久
firewalld	permanent 查询	runtime 查询	远端访问	重载/重启后仍存在

典型错误

- 删除当前网络 profile 后失联；
- 只验证 ping，未验证 DNS 和仓库；
- 使用 `chcon` 代替持久 fcontext 规则；
- 只添加 runtime 防火墙规则；
- `httpd` active 但实际仍监听其他端口；
- 本机 curl 成功就跳过远端功能验收。

[经典任务] 任务三：从空闲设备建立 LVM、文件系统、Swap 与持久挂载

环境与初始状态

- 题目提供一块 2 GiB 训练磁盘 `/dev/vdb`；
- 设备可能含有旧签名，必须先调查；
- 系统盘及其他卷不得改动；
- 当前 `/srv/data` 不在其他挂载下。

目标终态

- GPT 分区表，使用 `/dev/vdb1` 作为 PV；
- VG: `vgexam`；
- LV: `lvdata`，大小 1 GiB，XFS，挂载 `/srv/data`；
- LV: `lvswap`，大小 256 MiB，启用为 Swap；
- 使用 UUID 写入 `/etc/fstab`；
- 当前挂载、写入和 Swap 正常；
- 配置通过启动前静态检查，重启后自动恢复。

[参考解答] 任务三：设备证据门与存储分层验收

① 通过设备证据门

```
lsblk -e7 -o NAME,PATH,SIZE,TYPE,FSTYPE,LABEL,UUID,MOUNTPOINTS
blkid /dev/vdb /dev/vdb1 2>/dev/null || true
findmnt --source /dev/vdb 2>/dev/null || true
pvs --segments -o pv_name,vg_name,pv_size,pv_free
swapon --show
```

只有确认 `/dev/vdb` 是题目指定设备、没有被挂载、不是现有 PV/Swap，且旧签名不含需要保留的数据后，才能继续。若证据不清楚，应停止而不是运行 `wipefs -a`。

② 创建分区并等待内核识别

```
parted -s /dev/vdb mklabel gpt
parted -s /dev/vdb mkpart primary 1MiB 100%
partprobe /dev/vdb
udevadm settle
lsblk /dev/vdb
```

③ 创建 LVM 对象

```
pvccreate /dev/vdb1
vgcreate vgexam /dev/vdb1
lvcreate -L 1G -n lvdata vgexam
lvcreate -L 256M -n lvswap vgexam

pvs
vgs
lvs -o lv_name,vg_name,lv_size,lv_attr,devices
```

若 VG 空闲空间不足，应回到容量规划，不要使用强制选项或缩减其他未知 LV。

④ 创建文件系统和 Swap 签名

```
mkfs.xfs /dev/vgexam/lvdata
mkswap /dev/vgexam/lvswap
blkid /dev/vgexam/lvdata /dev/vgexam/lvswap
```

这些命令会写入数据结构，因此只能在对象身份已经确认后执行。

⑤ 建立当前挂载并验证写入

```
mkdir -p /srv/data
mount /dev/vgexam/lvdata /srv/data
findmnt /srv/data
touch /srv/data/.rhcsa33-write-test
stat /srv/data/.rhcsa33-write-test
```

确认 `/srv/data` 原来没有需要保留且被挂载遮蔽的内容。

⑥ 使用 UUID 写入 fstab

查询真实 UUID，不在讲义中编造，并在写入前检查空值与冲突：

```
DATA_UUID=$(blkid -s UUID -o value /dev/vgexam/lvdata)
SWAP_UUID=$(blkid -s UUID -o value /dev/vgexam/lvswap)
test -n "$DATA_UUID" && test -n "$SWAP_UUID"
grep -nE '(/srv/data|vgexam|lvswap)' /etc/fstab || true
grep -nF "UUID=$DATA_UUID" /etc/fstab || true
grep -nF "UUID=$SWAP_UUID" /etc/fstab || true
```

确认没有重复 UUID、设备或挂载点后再追加：

```
printf 'UUID=%s /srv/data xfs defaults 0 0\n' "$DATA_UUID" >> /etc/fstab
printf 'UUID=%s none swap defaults 0 0\n' "$SWAP_UUID" >> /etc/fstab
```

⑦ 验证 fstab 与 Swap

```
findmnt --verify --verbose
umount /srv/data
mount -a
findmnt /srv/data
swapon -a
swapon --show
```

可再次确认写入测试文件仍存在，证明挂载的是预期文件系统。

⑧ 重启门

重启前必须同时满足：

- `findmnt --verify` 没有阻断错误；
- UUID 与 `blkid` 一致；
- 挂载点存在；
- 当前 `mount -a` 和 `swapon -a` 成功；
- 系统其他 `fstab` 条目没有被误改；
- 具有控制台或恢复入口。

重启后推荐验证：

```
findmnt /srv/data
swapon --show
systemctl --failed
journalctl -b -p warning..alert --no-pager
```

证据矩阵

层	数据 LV	Swap LV
对象	pvs/vgs/lvs	lvs
签名	blkid 显示 XFS	blkid 显示 swap
当前	findmnt /srv/data	swapon --show
功能	实际写入与读取	内核已激活 Swap
持久	fstab UUID、findmnt --verify	fstab UUID、重启后 swapon

典型错误

- 仅凭 /dev/vdb 名称判断空闲；
- 创建 LV 后忘记创建文件系统；
- fstab 使用错误 UUID 或重复挂载点；
- mount -a 未验证就重启；
- 把 lvs 容量当成 df 文件系统容量；
- 使用无调查的 wipefs -a 或 --force。

[经典任务] 任务四：autofs、时间同步、日志与计划任务

环境与目标

- 本地用户 remote1 已存在，home 为 /rhome/remote1；
- NFS 导出：nfs.lab.example.com:/exports/home；
- 使用 autofs 间接映射挂载到 /rhome/<user>；
- 唯一允许的上游时间源：time.lab.example.com；
- 每周三 15:30 由 root 执行 /usr/local/sbin/rhcsa33-health；
- 脚本写入 /var/tmp/rhcsa33-health.txt，并以 tag rhcsa33-health 写入 journal；
- autofs、chronyd、crond 在重启后自动启动。

限制

- 未访问具体键时不能仅因 findmnt 没有 NFS 挂载就判定 autofs 失败；
- 计划任务必须使用绝对路径和确定环境；
- chronyd active 不能替代指定时间源的选择与跟踪证据。

[参考解答] 任务四：远程依赖、时间可信链和可审计调度

① 检查前置依赖

```
getent hosts nfs.lab.example.com time.lab.example.com
id remote1
systemctl status autofs chronyd crond --no-pager
```

若名称解析失败，先回到“NetworkManager、IPv4 地址与路由”和“主机名、NSS 与 DNS 名称解析”章节的证据入口。

② 配置 autofs 间接映射

创建 /etc/auto.master.d/rhome.autofs：

```
/rhome /etc/auto.rhome
```

创建 /etc/auto.rhome：

```
* -fstype=nfs4,rw nfs.lab.example.com:/exports/home/&
```

设置合理权限后检查解析：

```
chmod 0644 /etc/auto.master.d/rhome.autofs /etc/auto.rhome
automount -m
systemctl enable --now autofs.service
systemctl restart autofs.service
```

③ 触发并验证具体键

```
ls -ld /rhome/remotel
findmnt -T /rhome/remotel
runuser -u remotel -- test -r /rhome/remotel
```

autofs 是按访问触发：未访问前没有远程挂载可以是正常状态。还应检查远端 UID/GID 与本地身份是否匹配。

④ 配置唯一 chrony 源

先保留配置副本并查看现有活动源：

```
cp -a /etc/chrony.conf /etc/chrony.conf.rhcsa33.before
grep -nE '^[:space:]]*(server|pool)[[:space:]]' /etc/chrony.conf
```

本任务明确要求唯一源，因此只注释活动的 `server / pool` 行，再添加：

```
server time.lab.example.com iburst
```

重启并验证：

```
systemctl enable --now chronyd.service
systemctl restart chronyd.service
chronyc sources -v
chronyc tracking
timedatectl
```

必须关注源状态、当前选中源和 tracking，而不是只看 service active。

⑤ 创建可审计脚本

`/usr/local/sbin/rhcsa33-health`：

```
#!/bin/bash
set -eu
PATH=/usr/sbin:/usr/bin:/sbin:/bin
stamp=$(date --iso-8601=seconds)
printf '%s host=%s status=ok\n' "$stamp" "$(hostname -f)" \
  >> /var/tmp/rhcsa33-health.txt
logger -t rhcsa33-health -- "health check completed at $stamp"
```

```
chmod 0755 /usr/local/sbin/rhcsa33-health
```

在接近 cron 的精简环境中手工验证：

```
env -i HOME=/root SHELL=/bin/bash PATH=/usr/sbin:/usr/bin:/sbin:/bin \
  /usr/local/sbin/rhcsa33-health
tail -n 2 /var/tmp/rhcsa33-health.txt
journalctl -t rhcsa33-health --since '-5 min'
```

⑥ 创建 cron 声明

`/etc/cron.d/rhcsa33-health`：

```
SHELL=/bin/bash
PATH=/usr/sbin:/usr/bin:/sbin:/bin
30 15 * * * root /usr/local/sbin/rhcsa33-health
```

```
chmod 0644 /etc/cron.d/rhcsa33-health
systemctl enable --now crond.service
systemctl is-active crond.service
systemctl is-enabled crond.service
```

当前会话无法等待真实触发时间，因此实际到点执行必须列入 live test；手工运行只证明脚本和环境，不证明 cron 已触发。

⑦ 证据矩阵

对象	静态	当前	功能	持久
autofs	automount -m	service active	访问键后 findmnt、目标用户读取	enabled、重启后再次触发
chrony	配置中的唯一源	service active	sources 选中、tracking 有效	enabled、重启后重新同步
cron	/etc/cron.d 字段	crond active	精简环境脚本、真实到点产物	enabled、文件持久
日志	logger tag	journal 可查询	文件与日志时间关联	journal 持久策略取决于第 13 章配置

典型错误

- autofs 未触发就用 findmnt 判定失败；
- map 中通配符和 & 对应错误；
- chronyd active 但指定源为 ^?；
- cron 文件漏写用户字段；
- 使用相对路径或依赖交互 Shell 环境；
- 手工运行成功就宣称计划任务已执行。

[经典任务] 任务五：rootless Podman、用户 systemd、Quadlet 与持久数据

环境与目标

- 用户 svcweb 已存在；
- 镜像：registry.lab.example.com/rhel9/httpd-24:latest，容器内监听 8080；
- 容器名：web33；主机端口：8088；
- 数据目录：/home/svcweb/site，包含 index.html；
- 使用 Quadlet 文件 web33.container；
- SELinux 保持 Enforcing；
- svcweb 退出登录后用户服务仍运行；
- 宿主机重启后容器自动恢复，数据保持。

限制

- 所有 Podman 与 systemctl --user 操作必须在 svcweb 用户上下文执行；
- 不得用 root 容器代替 rootless 容器；
- 不得把手工容器成功扩大为 Quadlet 持久状态正确。

[参考解答] 任务五：从手工容器到用户级持久生命周期

① 确认用户上下文和路径

以 svcweb 的真实登录会话执行：

```
id
printf '%s\n' "$HOME" "$XDG_RUNTIME_DIR"
podman info
mkdir -p "$HOME/site" "$HOME/.config/containers/systemd"
stat -c '%n %U:%G %a' "$HOME/site"
```

若 podman info 显示 rootful 存储路径或 HOME 不属于 svcweb，应先修正会话上下文。

② 先用手工容器验证镜像、端口和数据

```
podman pull registry.lab.example.com/rhel9/httpd-24:latest
podman run --rm --name web33-test \
  -p 8088:8080 \
  -v "$HOME/site:/var/www/html:Z" \
  registry.lab.example.com/rhel9/httpd-24:latest
```

在另一个会话验证：

```
curl --fail http://127.0.0.1:8088/
ls -Zd /home/svcweb/site
```

手工验证通过后停止测试容器。若失败，先解决镜像、端口、数据或 SELinux，不要直接进入 Quadlet。

③ 写入 Quadlet 声明

/home/svcweb/.config/containers/systemd/web33.container:

```
[Unit]
Description=RHCSA 33 rootless web container
After=network-online.target
Wants=network-online.target

[Container]
Image=registry.lab.example.com/rhel9/httpd-24:latest
ContainerName=web33
PublishPort=8088:8080
Volume=/home/svcweb/site:/var/www/html:Z

[Service]
Restart=always

[Install]
WantedBy=default.target
```

④ 重新生成并检查用户 unit

仍在 svcweb 会话：

```
systemctl --user daemon-reload
systemctl --user list-unit-files | grep web33
systemctl --user start web33.service
systemctl --user status web33.service --no-pager
podman ps
curl --fail http://127.0.0.1:8088/
```

Quadlet 生成的 unit 在不同版本中可能显示为 `generated`，不应机械要求 `systemctl --user enable`。`[Install] WantedBy=default.target` 承担启动关联，验证应结合 `list-unit-files`、default target 依赖和重启结果。

⑤ 启用 linger

由 root 执行：

```
loginctl enable-linger svcweb
loginctl show-user svcweb -p Linger -p State
```

退出 svcweb 登录后，从 root 或其他用户验证端口和进程仍存在：

```
curl --fail http://127.0.0.1:8088/
loginctl user-status svcweb
```

⑥ 验证持久数据

修改 host 数据文件后确认容器读取变化；重启容器或 unit 后再次确认。不要把容器可重建与数据持久化混为一谈。

⑦ 重启后验证计划

```
loginctl show-user svcweb -p Linger
curl --fail http://127.0.0.1:8088/
# 登录为 svcweb 后
systemctl --user status web33.service --no-pager
podman ps
```

当前会话没有 live VM，因此这些命令是推荐验收入口，不是已执行结果。

证据矩阵

层	证据
用户上下文	<code>id</code> 、HOME、rootless <code>podman info</code>
容器功能	手工容器、端口、curl、数据目录标签
Quadlet 静态	<code>.container</code> 文件、生成 unit 可见
当前	用户 unit active、 <code>podman ps</code> 、curl
退出登录	linger、退出后 curl 与 user-status
重启	开机后 curl、用户 unit、数据仍在

典型错误

- root 执行 `podman`，验证了错误的容器存储；
- 直接写 Quadlet，未先隔离镜像或数据问题；
- 对生成 unit 机械执行 `enable` 并把失败当成 Quadlet 故障；
- 漏写 `:Z` 导致 SELinux 拒绝；
- 只验证容器运行，未验证数据和退出登录生命周期；
- 启用 `linger` 后不做退出登录测试。

[经典任务] 任务六：模拟考试级最终检查、重启决策与恢复演练

一台训练主机已经执行了多项操作，但存在以下不完整状态：

1. `deploy` 已加入 `opsadmins`，当前打开的旧会话仍看不到该组；
2. `exam-static` 当前激活，但 `connection.autoconnect=no`；
3. 两个 `repo` 文件均存在，其中 `AppStream` 元数据刷新失败；
4. `httpd` active，配置端口为 `8082`，但只有 `runtime` `firewalld` 规则，SELinux 端口类型未知；
5. `vgexam/lvdata` 已创建并挂载，`fstab` 的挂载点误写为 `/srv/date`；
6. `autofs` `master map` 正确，`indirect map` 中远端路径拼写错误；
7. 健康检查脚本手工运行成功，`/etc/cron.d` 条目漏写用户字段；
8. `svcweb` 手工容器成功，Quadlet 的数据路径写成 `/home/svcweb/sites`，`linger` 未启用。

要求：

- 不删除现有对象重做；
- 建立调查基线、依赖图和风险队列；
- 按最小修改修复；
- 逐项关闭静态、当前、功能和持久证据；
- 对共享对象执行回归；
- 判断是否允许重启；
- 若启动失败，指出应进入的恢复路径；
- 重启后形成最终证据矩阵。

[参考解答] 任务六：从缺陷清单到最终可交卷状态

① 先建立缺陷账本，而不是立即修复

ID	对象	已知症状	风险	下一条证据
U1	deploy 会话	旧会话无附加组	R1	新登录 <code>id</code>
N1	exam-static	当前 active、autoconnect no	R3	<code>nmcli con show</code>
R1	AppStream	元数据失败	R1	DNS、HTTP、baseurl
W1	HTTP 访问链	active 但安全层不完整	R2	<code>ss</code> 、 <code>semanage</code> 、双态 <code>firewalld</code>
S1	fstab	挂载点拼写错误	R4	<code>findmnt --verify</code> 、当前挂载
A1	autofs map	远端路径错误	R2	<code>automount -m</code> 、访问键、journal
C1	cron	漏用户字段	R1	文件结构、crond 日志
P1	Quadlet	路径和 linger 错误	R2	用户 unit journal、路径、loginctl

② 建立依赖与执行批次

推荐批次：

批次 1：只读基线与身份新会话验证
批次 2：网络 `autoconnect`、DNS、仓库
批次 3：`httpd` 配置/监听、SELinux、`firewalld`、远端请求
批次 4：修复 `fstab` 并通过重启证据门
批次 5：`autofs`、`chrony`、`cron` 的功能与日志
批次 6：`svcweb` Quadlet、数据、linger
批次 7：共享对象回归与最终矩阵
批次 8：满足重启后重启验收

网络先于仓库、NFS 和镜像；`fstab` 在最终重启前必须优先关闭；用户和路径修复后需要回归容器和权限。

③ 分项最小修复

- U1：不重复 `usermod`，打开新的 deploy 登录会话并执行 `id`。若新会话仍缺组，再查询 `/etc/group` 或 NSS。
- N1：`nmcli connection modify exam-static connection.autoconnect yes`，核对 profile，不需要重新创建。
- R1：对失败 repo 使用 `dnf repoinfo`、`curl` 或元数据刷新区分 DNS、路径和 HTTP；只修正错误的 baseurl。
- W1：保留现有服务，按 `httpd -t` → `ss` → `semanage port` → `firewalld runtime/permanent` → 本机/远端 `curl` 闭环。
- S1：把 `/srv/date` 修正为 `/srv/data`，执行 `findmnt --verify`、卸载/`mount -a` 回归；禁止通过注释整份 `fstab` 绕过。
- A1：修正 indirect map 的导出路径，`automount -m` 后重启 `autofs`，访问具体键并查 journal。
- C1：在 `/etc/cron.d` 时间字段后补 `root`，保持绝对路径和显式 PATH；手工精简环境验证与真实到点验证分开记录。
- P1：在 `svcweb` 会话修正 Volume 路径，重新加载用户 manager，检查 journal，启用 linger 后退出登录验证。

④ 共享对象回归

- 网络变化后：仓库、NFS 名称解析、HTTP 远端访问、镜像仓库；
- `/srv/data` 与 `fstab` 修复后：依赖该路径的服务和文件所有权；
- `svcweb` 路径修复后：SELinux 标签、容器数据和 Quadlet；
- 时间或时区变化后：日志事件与计划任务时间解释。

⑤ 重启门判定

只有以下条件全部满足才允许重启：

NetworkManager profile 持久且可达
→ `findmnt --verify` 无阻断错误
→ 当前 `mount -a` / `swapon -a` 正常
→ 关键服务 `enabled` 且当前功能已验证
→ `firewalld permanent` 与 SELinux 持久规则正确
→ `autofs` / `cron` / `chrony` 声明可解析
→ `linger` 与 `Quadlet` 启动关系已建立
→ 控制台或恢复入口可用

若 `fstab` 仍有错误、网络仅当前可用或用户容器尚未建立持久关系，应推迟重启并把任务标记 `blocked`。

⑥ 重启失败的恢复入口

- fstab 或挂载导致紧急模式：进入“启动链、GRUB、Target 与系统恢复”，在救援环境定位并修复具体 fstab 行；
- 网络失联：使用虚拟机控制台，检查 profile 和设备绑定；
- 用户容器未启动但系统正常：先保持系统在线，登录目标用户检查用户 unit journal；
- 服务访问失败：不要进入系统恢复，回到服务访问链逐层诊断。

⑦ 重启后最终矩阵

deploy 组关系静态：NSS 记录当前：新会话 id 功能：协作目录操作重启/新会话：关系仍正确结果： `verified / pending`
网络与仓库静态：profile 与 repo 定义当前：地址、DNS、元数据功能：访问依赖资源重启后：自动连接且元数据可用结果： `verified / pending`
HTTP 完整访问链静态：应用、SELinux 与永久防火墙当前：服务 active 且监听功能：远端 curl 重启后：自动启动并可访问结果： `verified / pending`
存储静态：fstab 中稳定标识与路径当前：文件系统挂载、Swap 激活功能：写入与读取重启后：自动挂载与激活结果： `verified / pending`
autofs、cron 与时间静态：map、cron 与 chrony 声明当前：相关服务运行功能：触发、产物与日志重启后：能够再次触发结果： `verified / pending`
rootless 容器静态：Quadlet 与 linger当前：用户 unit 与容器功能：HTTP 与持久数据重启后：无登录自动恢复结果： `verified / pending`
当前材料只能提供静态设计，因此实际结果栏必须由 RHEL 9 live test 填写，不能预先写成全部 verified。

⑧ 最终交卷检查

从原始题目逐条读回，而不是从命令历史读回：

1. 每个原题都有矩阵行；
2. 每行的参数与题目完全一致；
3. 限制条件没有被捷径破坏；
4. 当前、功能和持久证据没有互相代替；
5. blocked 项明确列出缺失证据；
6. 重启后只复查矩阵，不凭记忆判断。

[本章收束] 一致终态比命令数量更重要

RHCSA 综合任务的稳定能力不是更快地背出命令，而是能够把模糊要求转换为对象状态，知道每条证据能证明什么，在共享对象变化后回归相关功能，并在重启前主动关闭启动风险。

最终工作流可以压缩为：

- 读原题并拆评分行
- 调查对象身份和初始状态
- 建立依赖图、共享对象和风险队列
- 按局部闭环完成最小变更
- 记录静态、当前、功能和持久证据
- 修改共享对象后回归
- 通过重启门
- 重启后按原题与证据矩阵逐项验收

主要判断表

看到的证据	可以得出的结论	仍不能得出的结论
命令退出码为 0	本次调用按命令自身约定成功	目标系统已经达到完整终态
配置语法检查通过	声明可被对应程序解析	服务已运行、端口可达、功能正确
<code>active</code>	当前 unit 正在运行	已 enabled、客户端可用、重启后保持
当前挂载存在	当前路径已经挂载	fstab 正确、重启后仍挂载
runtime 防火墙已放行	当前规则集允许该服务或端口	permanent 状态已保存
本机请求成功	应用、监听和本机路径大体成立	远端路由、zone 与外部访问成立
root 容器运行	root 的容器实例可用	目标用户的 rootless 生命周期正确
重启后仍成立	持久链在本次重启中生效	其他尚未验收的任务也正确

工作方法

在考试中，用任务账本限制遗漏，用依赖图减少返工，用证据矩阵避免把局部成功扩大为完成。在真实维护窗口中，同样的方法可以转化为变更前基线、风险门、回退条件、消费者验证和变更后证据。

向后续学习与工作交接

本章是 RHCSA 手工系统管理的收束章，没有下一章需要继续展开某个 RHCSA 子系统。后续进入 RHCE 或实际自动化时，应保留本章的对象模型、依赖关系和验收矩阵，再把“手工修改接口”替换为模块、变量、模板与幂等任务。自动化只改变实现方式，不降低终态证据标准。